

Python Mastery Hub

AI / ML / DL / Data + Backend Track

Production Python Reference

Sources

- Fluent Python, 1st ed. · Ramalho (O'Reilly 2015)
- High Performance Python, 2nd ed. · Gorelick & Ozsvald (O'Reilly 2020)
- Python Notes for Professionals · GoalKicker

~16 weeks · 30 core chapters · 6 phases + Phase T

4 checkpoint projects · AI/ML production focus

FP Edition Note

PDF is 1st edition (2015). Three 2nd-ed chapters are absent from the PDF:

Ch 5 Data Class Builders -> use docs.python.org/3/library/dataclasses.html

Ch 8 Type Hints -> PNP Ch 87 (p.437) included as substitute

Ch 15 More About Type Hints -> PNP Ch 87 (p.437) included as substitute

Python Mastery Hub

AI / ML / DL / Data + Backend Track — Production Python Reference

Goal: Write efficient, blazing-fast production Python code and think idiomatically in Python.

Profile: ML engineer with solid Python skills. Applying to AI/ML/DL/Data daily + some backend work.

Timeline: ~16 weeks · 6 phases + Phase T + Tier 3 · 30 core chapters · 4 checkpoint projects

Books

ID	Book	File
FP	Fluent Python, 1st ed. — Ramalho (O'Reilly 2015)	Fluent Python.pdf
HPP	High Performance Python, 2nd ed. — Gorelick & Ozsvald (O'Reilly 2020)	high-performance-python.pdf
PNP	Python Notes for Professionals — GoalKicker	PythonNotesForProfessionals.pdf

FP Edition Note: The PDF is the **1st edition (2015)**. This plan references 2nd-edition chapter names (the curriculum content is 2nd-ed aligned), but PDF page numbers below point to your actual file. Chapters that were added in the 2nd edition and are **absent from the PDF** are marked △ with an alternative source.

FP 2nd-ed → 1st-ed chapter number shifts (where they differ):

Ch 6→1st Ch 8 | Ch 7→1st Ch 5 | Ch 9→1st Ch 7 | Ch 11→1st Ch 9 | Ch 12→1st Ch 10 | Ch 13→1st Ch 11 | Ch 16→1st Ch 13 | Ch 17→1st Ch 14 | Ch 18→1st Ch 15 | Ch 19+20→1st Ch 17 | Ch 21→1st Ch 18 | Ch 22→1st Ch 19

How to use PNP: drill companion only — open it when FP/HPP introduces something, code through the examples, move on. Never read PNP front-to-back.

The 6 Phases

PHASE 01 — The Foundation

~2 weeks · 3 chapters

Goal: Understand how Python and the machine actually work before writing a single optimization.

#	Book	Chapter	PDF	Why it matters
1	FP	Ch 1 — The Python Data Model	p.3	Dunder methods — the grammar underlying every Python framework

2	HPP	Ch 1 — Understanding Performant Python	p.1	CPU/memory/VM mental model — the foundation for every optimization decision
3	HPP	Ch 2 — Profiling to Find Bottlenecks	p.21	<code>cProfile</code> , <code>line_profiler</code> , <code>memory_profiler</code> — measure first, always

PNP companions:

- Ch 57 — `dis` module (*p.315 — optional: read once to see bytecode; use when debugging slow code in Phase T*)

PHASE 02 — Core Data Structures

~3 weeks · 7 chapters

Goal: Know what your ML data actually lives in, and the performance cost of each choice.

#	Book	Chapter	PDF	Why it matters
1	FP	Ch 2 — An Array of Sequences	p.19	Lists, tuples, arrays — pick the right container for your data pipeline
2	FP	Ch 3 — Dictionaries and Sets	p.63	Backbone of model configs, vocab maps, feature stores, label encoders
3	FP	Ch 4 — Unicode Text vs Bytes	p.97	Practical sections only: str vs bytes, encoding/decoding, UTF-8 gotchas. Skip normalization and codecs unless doing NLP.
4	FP	Ch 6 — Object References & Mutability	p.219 (<i>1st ed Ch 8</i>)	Stop silent bugs when sharing tensors — copies vs views (<i>moved up: tensor aliasing bugs poison everything</i>)
5	HPP	Ch 3 — Lists and Tuples (performance)	p.65	Memory layout — why tuples are faster for fixed data, how CPython allocates
6	HPP	Ch 4 — Dictionaries and Sets (performance)	p.79	Hash table internals, O(1) lookups, memory cost
7	FP	Ch 5 — Data Class Builders	⚠ not in PDF	Clean dataclasses for experiment configs, dataset records, hyperparameter bundles

FP Ch 5 substitute: Not in the PDF. Read the official Python docs: docs.python.org/3/library/dataclasses.html + PEP 557. Takes ~1 hour. Covers `@dataclass`, `field()`, `__post_init__`, frozen vs mutable — everything you need for the checkpoint project.

PNP companions:

- Ch 19 — Dictionary (*p.108 — `defaultdict` patterns, merging, unpacking*)
- Ch 7 — `enum` module (*p.51 — `Enum`, `IntEnum` — type-safe label sets, dataset splits, model states*)
- Ch 47 — `collections` module (*p.275 — `Counter`, `defaultdict`, `namedtuple`, `deque`, `OrderedDict`*)
- Ch 49 — `json` module (*p.284 — loading configs, API payloads, experiment metadata*)
- `pathlib` — no chapter in PNP; use docs.python.org/3/library/pathlib.html (*20-min read, essential for checkpoint projects*)

Why these cuts: Ch 20+21 (list comprehensions), Ch 22 (slicing), Ch 28 (tuple basics), Ch 60 (deque) are all below the level of "solid Python skills" — you know these. Ch 76 (copying) is fully covered by FP Ch 6. Ch 79 (`os`) is superseded by `pathlib` for path work; `os.environ` is the one part worth knowing and it takes 5 minutes.

Checkpoint Project — after Phase 02

Build a Dataset and DataLoader from scratch

Using only the standard library, implement a class that loads CSV records, supports indexing and length, and yields shuffled minibatches via a generator. No NumPy, no PyTorch.

Deliverable: working module + 5 unit tests + README explaining why you chose list vs tuple vs dict for each internal structure.

PHASE 03 — Functions, Types & Decorators

~2 weeks · 3 chapters

Goal: Understand the core pattern every ML framework (PyTorch, JAX, TF) is built on. Write type-safe production code.

#	Book	Chapter	PDF	Why it matters
1	FP	Ch 7 — Functions as First-Class Objects	p.139 (1st ed Ch 5)	Higher-order functions, closures — foundation of functional ML APIs
2	FP	Ch 8 — Type Hints in Functions	△ not in PDF	Production ML code needs type safety — <code>Callable</code> , <code>Optional</code> , <code>Union</code> , <code>mypy</code>
3	FP	Ch 9 — Decorators and Closures	p.183 (1st ed Ch 7)	<code>@torch.no_grad</code> , <code>@tf.function</code> , <code>@lru_cache</code> — now you fully understand why they work

FP Ch 8 substitute: Use PNP Ch 87 — Type Hints (p.437) for syntax, then `mypy` docs for practical usage. The 1st ed has a brief "Function Annotations" section at p.154 (inside Ch 5) — worth reading as context, but it predates the full `typing` system.

Note on FP Ch 15 (More About Type Hints): intentionally in Phase 05.

`TypeVar`, `Generic[T]`, `Protocol[T]` require knowing Protocols first (FP Ch 13). Reading Ch 15 here would hit forward references.

PNP companions:

- Ch 37 — Decorators (p.203 — *decorator factories, class decorators — patterns not in FP Ch 9*)
- Ch 43 — Importing modules & `importlib` (p.256 — *dynamic imports, §43.10 covers `__import__()` / `importlib`-style loading*)
- Ch 44 — Modules vs Packages (p.262 — *`__init__.py`, relative imports, namespace packages*)
- Ch 56 — `functools` module (p.312 — *`lru_cache`, `partial`, `total_ordering`, `reduce`*)

Why these cuts: Ch 33 (functions) and Ch 35 (functional programming) are redundant with FP

Ch 7 — FP covers this better. Ch 36 (partial functions) is in functools Ch 56. Ch 13 (variable scope) — the `nonlocal/global` section is two pages in FP Ch 7; read it there. Ch 87 (type hints) partially duplicates FP Ch 8; skip what FP Ch 8 already covers.

PHASE 04 — Iterators, Numerical Computing & Compilation

~4 weeks · 5 chapters

Goal: Build memory-efficient data pipelines, master NumPy/Pandas, escape Python's speed ceiling.

#	Book	Chapter	PDF	Why it matters
1	FP	Ch 17 — Iterators, Generators & Classic Coroutines	p.401 <i>(1st ed Ch 14)</i>	Lazy evaluation for large datasets — architectural basis of PyTorch DataLoader
2	HPP	Ch 5 — Iterators and Generators (performance)	p.97	Generators vs lists: memory profiling on real ML data volumes
3	HPP	Ch 6 — Matrix and Vector Computation	p.109	NumPy vectorization, numexpr, Pandas performance — the beating heart of all ML code
4	HPP	Ch 11 — Using Less RAM	p.341	Train on bigger datasets without OOM — <code>array.array</code> , <code>np.float32</code> , object reuse
5	HPP	Ch 7 — Compiling to C	p.161	<i>(focus: Numba @jit/@vectorize fully; GPU section read; Cython concepts only; everything else reference)</i>

PNP companions:

- Ch 53 — `itertools` module (p.294 — `chain`, `product`, `groupby`, `takewhile`, `islice` — FP Ch 17 doesn't cover these)
- Ch 67 — Generators (p.347 — `yield from`, `coroutine patterns`, `recursion` — read the sections FP Ch 17 doesn't cover)

Why these cuts: Ch 21 §6 (generator expressions) and Ch 32 (iterables/iterators) are redundant with FP Ch 17. Ch 68+69 (reduce/map) are basic Python covered in Phase 03.

Checkpoint Project — after Phase 04

NumPy-only minibatch trainer + Numba speedup

Write a linear regression trainer using only NumPy. Profile with `cProfile` and `line_profiler`. Identify the slowest pure-Python loop. Beat it twice: once with vectorization, once with `@numba.jit`.

Deliverable: notebook with three timings (naive / vectorized / compiled) + paragraph explaining the speedup ratio of each.

PHASE 05 — Classes & Protocols

~3 weeks · 5 chapters

Goal: Build proper ML components — models, datasets, trainers, evaluation loops. Then type them

correctly.

#	Book	Chapter	PDF	Why it matters
1	FP	Ch 11 — A Pythonic Object	p.247 (1st ed Ch 9)	<code>__repr__</code> , <code>__len__</code> , <code>__eq__</code> — the public interface every ML component should implement
2	FP	Ch 12 — Special Methods for Sequences	p.275 (1st ed Ch 10)	<code>__setitem__</code> , <code>__missing__</code> (merge into Ch 11 study session — treat as extension, not a separate chapter)
3	FP	Ch 13 — Interfaces, Protocols & ABCs	p.307 (1st ed Ch 11)	Duck typing vs strict contracts — how <code>nn.Module</code> and sklearn estimators are designed
4	FP	Ch 15 — More About Type Hints	△ not in PDF	<code>TypeVar</code> , <code>Generic[T]</code> , <code>Protocol[T]</code> , <code>overload</code> — you understand Protocols now, so these land correctly (moved from Phase 03: depends on Ch 13)
5	FP	Ch 18 — with, match, and else Blocks	p.447 (1st ed Ch 15)	Context managers for GPU memory, file handles, experiment tracking

FP Ch 15 substitute: Not in the PDF. Use PNP Ch 87 — Type Hints (p.437) for `Generic/TypeVar` syntax, then the official `typing` module docs for `Protocol[T]` and `overload`. 30 min of targeted reading fills the gap.

Read order: Ch 11 → Ch 12 (same session) → Ch 13 → Ch 15 → Ch 18.
Ch 15 immediately after Ch 13: you just learned what a Protocol is, now you type it with `Protocol[T]`.

Also read during this phase (from skip/skim list):

FP Ch 16 (Operator Overloading) — skim after Ch 11. Custom metric/loss classes use `+/-`. (1st ed Ch 13, PDF p.371)

FP Ch 22 (Dynamic Attributes & Properties) — skim after Ch 13. `@property` is everyday code. (1st ed Ch 19, PDF p.585)

PNP companions:

- Ch 38 — Classes (p.209 — `__slots__`, *properties, static/class methods, descriptors*)
- Ch 77 — Context Managers (p.398 — `@contextmanager` from `contextlib` — *cleaner than `__enter__`/`__exit__` classes*)

Why these cuts: Ch 39 (metaclasses) — you're skipping FP Ch 24 (metaprogramming); reading PNP metaclasses is inconsistent with that decision. Ch 50 (sqlite3) — post-plan when you need it. Ch 64 (indexing/slicing) and Ch 72 (sorting) are fully covered by FP Ch 11+12.

Note on `__slots__`: implement on any class with many instances — 30–50% memory reduction for large datasets of sample objects.

Checkpoint Project — after Phase 05

Tiny nn.Module-style API

Implement a minimal neural-network base class with `__call__`, `parameters()`, `state_dict()`, `load_state_dict()`, and `__repr__`. Build a 2-layer MLP on top and train on a toy regression task with manual backprop.

Deliverable: ~150 LOC framework + working training script.

PHASE T — Testing & Debugging Interlude

~1 week · off-book module

Goal: The biggest gap between "read *Fluent Python*" and "ship ML systems." Do not skip.

Topic	Content
1 — pytest	Fixtures, parametrize, monkeypatch. Write 10 tests against your Phase 05 checkpoint code
2 — pdb/ipdb	Breakpoints, post-mortem, navigating frames. Drop into a deliberately buggy training loop
3 — logging	<code>logging</code> module: levels, handlers, formatters. Add structured logging to your checkpoint project
4 — PyTorch debugging	<code>torch.autograd.set_detect_anomaly</code> , gradient checking, NaN traps, shape mismatch diagnostics
5 — Reproducibility	Seeding, deterministic flags, snapshotting environments with <code>uv</code>

PNP companions:

- Ch 137 — `logging` module (p.597 — *levels, handlers, formatters, logger hierarchy*)
- Ch 88 — Exceptions (p.440 — *catching, re-raising, hierarchy, finally*)

Why these cuts: Ch 89 (custom exceptions) — subclassing `Exception` takes 2 minutes, not a companion chapter. Ch 90 (common exceptions) — you already know `NameError` and `TypeError`. Ch 84 (subprocess) — post-plan when you need to launch training jobs. Ch 57 (dis) — covered in Phase 01 as optional.

PHASE 06 — Concurrency & Async

~3 weeks · 5 chapters

Goal: Parallelize data loading, training pipelines, and ML model serving APIs.

Block A — Parallelism (processes & threads)

#	Book	Chapter	PDF	Why it matters
1	FP	Ch 19 — Concurrency	p.505 (1st ed Ch 17)	GIL, threads, processes — the full truth behind <code>DataLoader`'s` <code>num_workers</code></code>
2	FP	Ch 20 — Concurrent Executors	p.505 (1st ed Ch 17)	<code>ThreadPoolExecutor</code> , <code>ProcessPoolExecutor</code> — batch inference, parallel preprocessing

3	HPP	Ch 9 — The multiprocessing Module	p.245	Shared memory, worker pools — tune <code>num_workers</code> , avoid pickling traps
---	-----	-----------------------------------	-------	--

FP Ch 19+20 PDF note: The 2nd ed split the concurrency chapter into Ch 19 (GIL/models overview) + Ch 20 (Executors). Your PDF has only the original Ch 17 at p.505, which covers both. Read it as one chapter — all the content is there, just not separated. The GIL/thread model intro is woven through pp.505–530.

Block B — Async I/O

#	Book	Chapter	PDF	Why it matters
4	FP	Ch 21 — Asynchronous Programming	p.537 (1st ed Ch 18)	<code>async/await</code> — serve ML models via FastAPI without blocking
5	HPP	Ch 8 — Asynchronous I/O	p.213	Event loops, async file reads — maximize disk throughput during data loading

PNP companions:

- Ch 54 — `asyncio` module (p.301 — *coroutines, executors, UVLoop, common misconceptions*)
- Ch 59 — `queue` module (p.322 — *thread-safe queues for producer/consumer pipelines*)

Before the checkpoint: read Pydantic v2 + FastAPI docs (see Tier 3 below).

Checkpoint Project — after Phase 06

FastAPI inference endpoint with warm process pool

Wrap a trained model behind a FastAPI async endpoint. Use a `ProcessPoolExecutor` with persistent workers for CPU-bound inference. Load-test with `httpx` or `locust`.

Deliverable: working API + load test results vs naive synchronous baseline + paragraph on why processes (not threads) for inference.

Tier 3 — Short Official Docs (not in any book)

Readable in 1–2 days each. Do them **before the Phase 06 checkpoint**.

Resource	Time	Why
Pydantic v2 — <code>docs.pydantic.dev</code> Concepts section	1 day	FastAPI is built on it. ML config validation, API schemas, structured LLM outputs.
FastAPI — <code>fastapi.tiangolo.com/tutorial</code> Tutorial section	1 day	The checkpoint project needs this. Dependency injection, lifespan events, path params.

structlog removed: `stdlib logging` from Phase T is sufficient for Python mastery. Structlog is production polish — pick it up when you're running real services.

From FP

Chapter	PDF	Action	When	Reason
Ch 10 — Design Patterns	p.167 (1sted Ch 6)	SKIP	—	Academic value, low practical ML return
Ch 14 — Inheritance Pitfalls	p.347 (1sted Ch 12)	SKIP	—	Good theory, rarely the bottleneck in ML codebases
Ch 16 — Operator Overloading	p.371 (1sted Ch 13)	SKIM	Phase 05	Custom metric/loss classes with <code>+</code> / <code>-</code> are common in ML — read the patterns, skip edge cases
Ch 22 — Dynamic Attributes & Properties	p.585 (1sted Ch 19)	SKIM	Phase 05	<code>@property</code> for lazy-loaded attributes is everyday code — read after Ch 13
Ch 23 — Attribute Descriptors	p.625 (1sted Ch 20)	SKIP	—	Skip unless writing metaclass-heavy code
Ch 24 — Class Metaprogramming	p.655 (1sted Ch 21)	SKIP	—	Fascinating but far from typical ML workflows

From HPP

Chapter	PDF	Action	Reason
Ch 10 — Clusters & Job Queues	p.311	DEFER	Read when moving to distributed training (Ray, Dask, Celery)
Ch 12 — Lessons from the Field	p.393	DEFER	Good stories — save for after you finish the plan

From PNP

Chapters	Action	Reason
Ch 61 (webbrowser), Ch 62 (tkinter), Ch 63 (pyautogui)	SKIP	No ML/backend relevance
Ch 52 (locale), Ch 58 (base64)	SKIP	Edge-case utility only
Ch 85 (setup.py)	SKIM	Know it exists; use <code>pyproject.toml</code> + <code>uv</code> in practice

Post-Plan Resources — Learn When a Project Demands It

Do not read these upfront. Pick them up the first time a real project requires them.

Tool	Resource	When
Polars	<code>docs.pola.rs</code> — User Guide	When Pandas becomes your bottleneck

PyArrow / Parquet	arrow.apache.org/docs/python	First time you work with large ML datasets
DuckDB	duckdb.org/docs/api/python	SQL analytics on local files without a database server
structlog	structlog.org	Running production services that need JSON-structured logs
SQLAlchemy async	SQLAlchemy 2.0 asyncio docs	First backend project with a real relational database
Ray / Dask	Ray docs — Data + Train	Moving to distributed training or large-scale pipelines
subprocess	PNP Ch 84 (p.424)	First time you need to launch training jobs or shell commands
sqlite3	PNP Ch 50 (p.289)	First time you need local experiment metadata storage
HPP Ch 10	Already in your PDF at p.311	Distributed compute — Celery, job queues, cluster design

Progress Tracker

Phase	Status	Started	Completed	Notes
Phase 01 — Foundation	☐ Not started	—	—	
Phase 02 — Data Structures	☐ Not started	—	—	
★ Checkpoint 1	☐ Not started	—	—	Dataset + DataLoader
Phase 03 — Functions/Types/Decorators	☐ Not started	—	—	
Phase 04 — Iterators/NumPy/Compilation	☐ Not started	—	—	
★ Checkpoint 2	☐ Not started	—	—	NumPy trainer + Numba
Phase 05 — Classes & Protocols	☐ Not started	—	—	
★ Checkpoint 3	☐ Not started	—	—	Tiny nn.Module API
Phase T — Testing & Debugging	☐ Not started	—	—	
Tier 3 — Short Reads	☐ Not started	—	—	Pydantic v2, FastAPI
Phase 06 — Concurrency & Async	☐ Not started	—	—	
★ Checkpoint 4	☐ Not started	—	—	FastAPI inference endpoint

Key Rules

1. Read sequentially within each phase. Phases build on each other.
2. Follow the listed order within a phase — *what* before *how fast*.
3. Code every chapter. Passive reading yields ~20% of the value.
4. Hit every checkpoint project — they are the difference between *read a book* and *can ship*.
5. PNP is a drill companion, not a read-through. If a PNP chapter duplicates FP/HPP, skip it.

6. Skipped chapters are reference — return when a real problem demands them.

7. Post plan resources are not procrastination fuel — pick them up only when a project needs them.

PHASE 01 · The Foundation

~2 weeks · 3 chapters

Understand how Python and the machine work before writing a single optimization.

Chapters in this phase:

- FP Ch 1 | The Python Data Model
- HPP Ch 1 | Understanding Performant Python
- HPP Ch 2 | Profiling to Find Bottlenecks
- PNP Ch 57 | The dis Module [optional companion]

FP Ch 1 | The Python Data Model

The Python Data Model

Guido's sense of the aesthetics of language design is amazing. I've met many fine language designers who could build theoretically beautiful languages that no one would ever use, but Guido is one of those rare people who can build a language that is just slightly less theoretically beautiful but thereby is a joy to write programs in.¹

— Jim Hugunin

Creator of Jython, cocreator of AspectJ, architect of the .Net DLR

One of the best qualities of Python is its consistency. After working with Python for a while, you are able to start making informed, correct guesses about features that are new to you.

However, if you learned another object-oriented language before Python, you may have found it strange to use `len(collection)` instead of `collection.len()`. This apparent oddity is the tip of an iceberg that, when properly understood, is the key to everything we call *Pythonic*. The iceberg is called the Python data model, and it describes the API that you can use to make your own objects play well with the most idiomatic language features.

You can think of the data model as a description of Python as a framework. It formalizes the interfaces of the building blocks of the language itself, such as sequences, iterators, functions, classes, context managers, and so on.

While coding with any framework, you spend a lot of time implementing methods that are called by the framework. The same happens when you leverage the Python data model. The Python interpreter invokes special methods to perform basic object operations, often triggered by special syntax. The special method names are always written with leading and trailing double underscores (i.e., `__getitem__`). For example, the syn-

1. *Story of Jython*, written as a Foreword to *Jython Essentials* (O'Reilly, 2002), by Samuele Pedroni and Noel Rappin.

`tax obj[key]` is supported by the `__getitem__` special method. In order to evaluate `my_collection[key]`, the interpreter calls `my_collection.__getitem__(key)`.

The special method names allow your objects to implement, support, and interact with basic language constructs such as:

- Iteration
- Collections
- Attribute access
- Operator overloading
- Function and method invocation
- Object creation and destruction
- String representation and formatting
- Managed contexts (i.e., with blocks)



Magic and Dunder

The term *magic method* is slang for special method, but when talking about a specific method like `__getitem__`, some Python developers take the shortcut of saying “under-under-getitem” which is ambiguous, because the syntax `__x` has another special meaning.² Being precise and pronouncing “under-under-getitem-under-under” is tiresome, so I follow the lead of author and teacher Steve Holden and say “dunder-getitem.” All experienced Pythonistas understand that shortcut. As a result, the special methods are also known as *dunder methods*.³

A Pythonic Card Deck

The following is a very simple example, but it demonstrates the power of implementing just two special methods, `__getitem__` and `__len__`.

Example 1-1 is a class to represent a deck of playing cards.

Example 1-1. A deck as a sequence of cards

```
import collections
```

2. See “Private and “Protected” Attributes in Python” on page 262.

3. I personally first heard “dunder” from Steve Holden. Wikipedia [credits](#) Mark Johnson and Tim Hochberg for the first written records of “dunder” in responses to the question “How do you pronounce __ (double underscore)?” in the python-list on September 26, 2002: [Johnson’s message](#); [Hochberg’s \(11 minutes later\)](#).

```

Card = collections.namedtuple('Card', ['rank', 'suit'])

class FrenchDeck:
    ranks = [str(n) for n in range(2, 11)] + list('JQKA')
    suits = 'spades diamonds clubs hearts'.split()

    def __init__(self):
        self._cards = [Card(rank, suit) for suit in self.suits
                        for rank in self.ranks]

    def __len__(self):
        return len(self._cards)

    def __getitem__(self, position):
        return self._cards[position]

```

The first thing to note is the use of `collections.namedtuple` to construct a simple class to represent individual cards. Since Python 2.6, `namedtuple` can be used to build classes of objects that are just bundles of attributes with no custom methods, like a database record. In the example, we use it to provide a nice representation for the cards in the deck, as shown in the console session:

```

>>> beer_card = Card('7', 'diamonds')
>>> beer_card
Card(rank='7', suit='diamonds')

```

But the point of this example is the `FrenchDeck` class. It's short, but it packs a punch. First, like any standard Python collection, a deck responds to the `len()` function by returning the number of cards in it:

```

>>> deck = FrenchDeck()
>>> len(deck)
52

```

Reading specific cards from the deck—say, the first or the last—should be as easy as `deck[0]` or `deck[-1]`, and this is what the `__getitem__` method provides:

```

>>> deck[0]
Card(rank='2', suit='spades')
>>> deck[-1]
Card(rank='A', suit='hearts')

```

Should we create a method to pick a random card? No need. Python already has a function to get a random item from a sequence: `random.choice`. We can just use it on a deck instance:

```

>>> from random import choice
>>> choice(deck)
Card(rank='3', suit='hearts')
>>> choice(deck)
Card(rank='K', suit='spades')

```

```
>>> choice(deck)
Card(rank='2', suit='clubs')
```

We’ve just seen two advantages of using special methods to leverage the Python data model:

- The users of your classes don’t have to memorize arbitrary method names for standard operations (“How to get the number of items? Is it `.size()`, `.length()`, or what?”).
- It’s easier to benefit from the rich Python standard library and avoid reinventing the wheel, like the `random.choice` function.

But it gets better.

Because our `__getitem__` delegates to the `[]` operator of `self._cards`, our deck automatically supports slicing. Here’s how we look at the top three cards from a brand new deck, and then pick just the aces by starting on index 12 and skipping 13 cards at a time:

```
>>> deck[:3]
[Card(rank='2', suit='spades'), Card(rank='3', suit='spades'),
Card(rank='4', suit='spades')]
>>> deck[12::13]
[Card(rank='A', suit='spades'), Card(rank='A', suit='diamonds'),
Card(rank='A', suit='clubs'), Card(rank='A', suit='hearts')]
```

Just by implementing the `__getitem__` special method, our deck is also iterable:

```
>>> for card in deck: # doctest: +ELLIPSIS
...     print(card)
Card(rank='2', suit='spades')
Card(rank='3', suit='spades')
Card(rank='4', suit='spades')
...
```

The deck can also be iterated in reverse:

```
>>> for card in reversed(deck): # doctest: +ELLIPSIS
...     print(card)
Card(rank='A', suit='hearts')
Card(rank='K', suit='hearts')
Card(rank='Q', suit='hearts')
...
```



Ellipsis in doctests

Whenever possible, the Python console listings in this book were extracted from doctests to ensure accuracy. When the output was too long, the elided part is marked by an ellipsis (...) like in the last line in the preceding code. In such cases, we used the `# doctest: +ELLIPSIS` directive to make the doctest pass. If you are trying these examples in the interactive console, you may omit the doctest directives altogether.

Iteration is often implicit. If a collection has no `__contains__` method, the `in` operator does a sequential scan. Case in point: `in` works with our `FrenchDeck` class because it is iterable. Check it out:

```
>>> Card('Q', 'hearts') in deck
True
>>> Card('7', 'beasts') in deck
False
```

How about sorting? A common system of ranking cards is by rank (with aces being highest), then by suit in the order of spades (highest), then hearts, diamonds, and clubs (lowest). Here is a function that ranks cards by that rule, returning 0 for the 2 of clubs and 51 for the ace of spades:

```
suit_values = dict(spades=3, hearts=2, diamonds=1, clubs=0)

def spades_high(card):
    rank_value = FrenchDeck.ranks.index(card.rank)
    return rank_value * len(suit_values) + suit_values[card.suit]
```

Given `spades_high`, we can now list our deck in order of increasing rank:

```
>>> for card in sorted(deck, key=spades_high): # doctest: +ELLIPSIS
...     print(card)
Card(rank='2', suit='clubs')
Card(rank='2', suit='diamonds')
Card(rank='2', suit='hearts')
... (46 cards omitted)
Card(rank='A', suit='diamonds')
Card(rank='A', suit='hearts')
Card(rank='A', suit='spades')
```

Although `FrenchDeck` implicitly inherits from `object`,⁴ its functionality is not inherited, but comes from leveraging the data model and composition. By implementing the special methods `__len__` and `__getitem__`, our `FrenchDeck` behaves like a standard Python sequence, allowing it to benefit from core language features (e.g., iteration and slicing)

4. In Python 2, you'd have to be explicit and write `FrenchDeck(object)`, but that's the default in Python 3.

and from the standard library, as shown by the examples using `random.choice`, `reversed`, and `sorted`. Thanks to composition, the `__len__` and `__getitem__` implementations can hand off all the work to a `list` object, `self._cards`.



How About Shuffling?

As implemented so far, a `FrenchDeck` cannot be shuffled, because it is *immutable*: the cards and their positions cannot be changed, except by violating encapsulation and handling the `_cards` attribute directly. In [Chapter 11](#), that will be fixed by adding a one-line `__setitem__` method.

How Special Methods Are Used

The first thing to know about special methods is that they are meant to be called by the Python interpreter, and not by you. You don't write `my_object.__len__()`. You write `len(my_object)` and, if `my_object` is an instance of a user-defined class, then Python calls the `__len__` instance method you implemented.

But for built-in types like `list`, `str`, `bytearray`, and so on, the interpreter takes a shortcut: the CPython implementation of `len()` actually returns the value of the `ob_size` field in the `PyVarObject` C struct that represents any variable-sized built-in object in memory. This is much faster than calling a method.

More often than not, the special method call is implicit. For example, the statement `for i in x:` actually causes the invocation of `iter(x)`, which in turn may call `x.__iter__()` if that is available.

Normally, your code should not have many direct calls to special methods. Unless you are doing a lot of metaprogramming, you should be implementing special methods more often than invoking them explicitly. The only special method that is frequently called by user code directly is `__init__`, to invoke the initializer of the superclass in your own `__init__` implementation.

If you need to invoke a special method, it is usually better to call the related built-in function (e.g., `len`, `iter`, `str`, etc). These built-ins call the corresponding special method, but often provide other services and—for built-in types—are faster than method calls. See, for example, [“A Closer Look at the `iter` Function”](#) on page 436 in [Chapter 14](#).

Avoid creating arbitrary, custom attributes with the `__foo__` syntax because such names may acquire special meanings in the future, even if they are unused today.

Emulating Numeric Types

Several special methods allow user objects to respond to operators such as `+`. We will cover that in more detail in [Chapter 13](#), but here our goal is to further illustrate the use of special methods through another simple example.

We will implement a class to represent two-dimensional vectors—that is Euclidean vectors like those used in math and physics (see [Figure 1-1](#)).

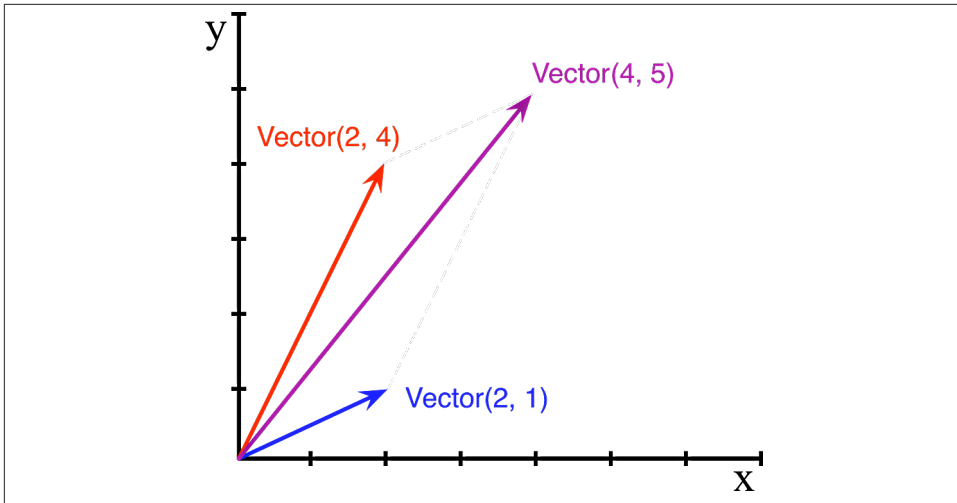


Figure 1-1. Example of two-dimensional vector addition; $\text{Vector}(2, 4) + \text{Vector}(2, 1)$ results in $\text{Vector}(4, 5)$.



The built-in `complex` type can be used to represent two-dimensional vectors, but our class can be extended to represent n -dimensional vectors. We will do that in [Chapter 14](#).

We will start by designing the API for such a class by writing a simulated console session that we can use later as a doctest. The following snippet tests the vector addition pictured in [Figure 1-1](#):

```
>>> v1 = Vector(2, 4)
>>> v2 = Vector(2, 1)
>>> v1 + v2
Vector(4, 5)
```

Note how the `+` operator produces a `Vector` result, which is displayed in a friendly manner in the console.

The `abs` built-in function returns the absolute value of integers and floats, and the magnitude of complex numbers, so to be consistent, our API also uses `abs` to calculate the magnitude of a vector:

```
>>> v = Vector(3, 4)
>>> abs(v)
5.0
```

We can also implement the `*` operator to perform scalar multiplication (i.e., multiplying a vector by a number to produce a new vector with the same direction and a multiplied magnitude):

```
>>> v * 3
Vector(9, 12)
>>> abs(v * 3)
15.0
```

Example 1-2 is a `Vector` class implementing the operations just described, through the use of the special methods `__repr__`, `__abs__`, `__add__` and `__mul__`.

Example 1-2. A simple two-dimensional vector class

```
from math import hypot
```

```
class Vector:
```

```
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

    def __repr__(self):
        return 'Vector(%r, %r)' % (self.x, self.y)

    def __abs__(self):
        return hypot(self.x, self.y)

    def __bool__(self):
        return bool(abs(self))

    def __add__(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Vector(x, y)

    def __mul__(self, scalar):
        return Vector(self.x * scalar, self.y * scalar)
```

Note that although we implemented four special methods (apart from `__init__`), none of them is directly called within the class or in the typical usage of the class illustrated by the console listings. As mentioned before, the Python interpreter is the only frequent

caller of most special methods. In the following sections, we discuss the code for each special method.

String Representation

The `__repr__` special method is called by the `repr` built-in to get the string representation of the object for inspection. If we did not implement `__repr__`, vector instances would be shown in the console like `<Vector object at 0x10e100070>`.

The interactive console and debugger call `repr` on the results of the expressions evaluated, as does the `%r` placeholder in classic formatting with the `%` operator, and the `!r` conversion field in the new **Format String Syntax** used in the `str.format` method.



Speaking of the `%` operator and the `str.format` method, you will notice I use both in this book, as does the Python community at large. I am increasingly favoring the more powerful `str.format`, but I am aware many Pythonistas prefer the simpler `%`, so we'll probably see both in Python source code for the foreseeable future.

Note that in our `__repr__` implementation, we used `%r` to obtain the standard representation of the attributes to be displayed. This is good practice, because it shows the crucial difference between `Vector(1, 2)` and `Vector('1', '2')`—the latter would not work in the context of this example, because the constructor's arguments must be numbers, not `str`.

The string returned by `__repr__` should be unambiguous and, if possible, match the source code necessary to re-create the object being represented. That is why our chosen representation looks like calling the constructor of the class (e.g., `Vector(3, 4)`).

Contrast `__repr__` with `__str__`, which is called by the `str()` constructor and implicitly used by the `print` function. `__str__` should return a string suitable for display to end users.

If you only implement one of these special methods, choose `__repr__`, because when no custom `__str__` is available, Python will call `__repr__` as a fallback.



“Difference between `__str__` and `__repr__` in Python” is a Stack Overflow question with excellent contributions from Pythonistas Alex Martelli and Martijn Pieters.

Arithmetic Operators

Example 1-2 implements two operators: `+` and `*`, to show basic usage of `__add__` and `__mul__`. Note that in both cases, the methods create and return a new instance of `Vector`, and do not modify either operand—`self` or `other` are merely read. This is the expected behavior of infix operators: to create new objects and not touch their operands. I will have a lot more to say about that in [Chapter 13](#).



As implemented, **Example 1-2** allows multiplying a `Vector` by a number, but not a number by a `Vector`, which violates the commutative property of multiplication. We will fix that with the special method `__rmul__` in [Chapter 13](#).

Boolean Value of a Custom Type

Although Python has a `bool` type, it accepts any object in a boolean context, such as the expression controlling an `if` or `while` statement, or as operands to `and`, `or`, and `not`. To determine whether a value `x` is *truthy* or *falsey*, Python applies `bool(x)`, which always returns `True` or `False`.

By default, instances of user-defined classes are considered *truthy*, unless either `__bool__` or `__len__` is implemented. Basically, `bool(x)` calls `x.__bool__()` and uses the result. If `__bool__` is not implemented, Python tries to invoke `x.__len__()`, and if that returns zero, `bool` returns `False`. Otherwise `bool` returns `True`.

Our implementation of `__bool__` is conceptually simple: it returns `False` if the magnitude of the vector is zero, `True` otherwise. We convert the magnitude to a Boolean using `bool(abs(self))` because `__bool__` is expected to return a boolean.

Note how the special method `__bool__` allows your objects to be consistent with the truth value testing rules defined in the “[Built-in Types](#)” chapter of *The Python Standard Library* documentation.



A faster implementation of `Vector.__bool__` is this:

```
def __bool__(self):  
    return bool(self.x or self.y)
```

This is harder to read, but avoids the trip through `abs`, `__abs__`, the squares, and square root. The explicit conversion to `bool` is needed because `__bool__` must return a boolean and `or` returns either operand as is: `x or y` evaluates to `x` if that is *truthy*, otherwise the result is `y`, whatever that is.

Overview of Special Methods

The “Data Model” chapter of *The Python Language Reference* lists 83 special method names, 47 of which are used to implement arithmetic, bitwise, and comparison operators.

As an overview of what is available, see Tables 1-1 and 1-2.



The grouping shown in the following tables is not exactly the same as in the official documentation.

Table 1-1. Special method names (operators excluded)

Category	Method names
String/bytes representation	<code>__repr__</code> , <code>__str__</code> , <code>__format__</code> , <code>__bytes__</code>
Conversion to number	<code>__abs__</code> , <code>__bool__</code> , <code>__complex__</code> , <code>__int__</code> , <code>__float__</code> , <code>__hash__</code> , <code>__index__</code>
Emulating collections	<code>__len__</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>__delitem__</code> , <code>__contains__</code>
Iteration	<code>__iter__</code> , <code>__reversed__</code> , <code>__next__</code>
Emulating callables	<code>__call__</code>
Context management	<code>__enter__</code> , <code>__exit__</code>
Instance creation and destruction	<code>__new__</code> , <code>__init__</code> , <code>__del__</code>
Attribute management	<code>__getattr__</code> , <code>__getattribute__</code> , <code>__setattr__</code> , <code>__delattr__</code> , <code>__dir__</code>
Attribute descriptors	<code>__get__</code> , <code>__set__</code> , <code>__delete__</code>
Class services	<code>__prepare__</code> , <code>__instancecheck__</code> , <code>__subclasscheck__</code>

Table 1-2. Special method names for operators

Category	Method names and related operators
Unary numeric operators	<code>__neg__</code> <code>-</code> , <code>__pos__</code> <code>+</code> , <code>__abs__</code> <code>abs()</code>
Rich comparison operators	<code>__lt__</code> <code>></code> , <code>__le__</code> <code><=</code> , <code>__eq__</code> <code>==</code> , <code>__ne__</code> <code>!=</code> , <code>__gt__</code> <code>></code> , <code>__ge__</code> <code>>=</code>
Arithmetic operators	<code>__add__</code> <code>+</code> , <code>__sub__</code> <code>-</code> , <code>__mul__</code> <code>*</code> , <code>__truediv__</code> <code>/</code> , <code>__floordiv__</code> <code>//</code> , <code>__mod__</code> <code>%</code> , <code>__divmod__</code> <code>divmod()</code> , <code>__pow__</code> <code>**</code> or <code>pow()</code> , <code>__round__</code> <code>round()</code>
Reversed arithmetic operators	<code>__radd__</code> , <code>__rsub__</code> , <code>__rmul__</code> , <code>__rtruediv__</code> , <code>__rfloordiv__</code> , <code>__rmod__</code> , <code>__rdivmod__</code> , <code>__rpow__</code>
Augmented assignment arithmetic operators	<code>__iadd__</code> , <code>__isub__</code> , <code>__imul__</code> , <code>__itruediv__</code> , <code>__ifloordiv__</code> , <code>__imod__</code> , <code>__ipow__</code>
Bitwise operators	<code>__invert__</code> <code>~</code> , <code>__lshift__</code> <code><<</code> , <code>__rshift__</code> <code>>></code> , <code>__and__</code> <code>&</code> , <code>__or__</code> <code> </code> , <code>__xor__</code> <code>^</code>

Category	Method names and related operators
Reversed bitwise operators	<code>__rlshift__</code> , <code>__rrshift__</code> , <code>__rand__</code> , <code>__rxor__</code> , <code>__ror__</code>
Augmented assignment bitwise operators	<code>__ilshift__</code> , <code>__irshift__</code> , <code>__iand__</code> , <code>__ixor__</code> , <code>__ior__</code>



The reversed operators are fallbacks used when operands are swapped (`b * a` instead of `a * b`), while augmented assignments are shortcuts combining an infix operator with variable assignment (`a = a * b` becomes `a *= b`). [Chapter 13](#) explains both reversed operators and augmented assignment in detail.

Why len Is Not a Method

I asked this question to core developer Raymond Hettinger in 2013 and the key to his answer was a quote from [The Zen of Python](#): “practicality beats purity.” In [“How Special Methods Are Used” on page 8](#), I described how `len(x)` runs very fast when `x` is an instance of a built-in type. No method is called for the built-in objects of CPython: the length is simply read from a field in a C struct. Getting the number of items in a collection is a common operation and must work efficiently for such basic and diverse types as `str`, `list`, `memoryview`, and so on.

In other words, `len` is not called as a method because it gets special treatment as part of the Python data model, just like `abs`. But thanks to the special method `__len__`, you can also make `len` work with your own custom objects. This is a fair compromise between the need for efficient built-in objects and the consistency of the language. Also from [The Zen of Python](#): “Special cases aren’t special enough to break the rules.”



If you think of `abs` and `len` as unary operators, you may be more inclined to forgive their functional look-and-feel, as opposed to the method call syntax one might expect in an OO language. In fact, the ABC language—a direct ancestor of Python that pioneered many of its features—had an `#` operator that was the equivalent of `len` (you’d write `#s`). When used as an infix operator, written `x#s`, it counted the occurrences of `x` in `s`, which in Python you get as `s.count(x)`, for any sequence `s`.

Chapter Summary

By implementing special methods, your objects can behave like the built-in types, enabling the expressive coding style the community considers Pythonic.

A basic requirement for a Python object is to provide usable string representations of itself, one used for debugging and logging, another for presentation to end users. That is why the special methods `__repr__` and `__str__` exist in the data model.

Emulating sequences, as shown with the `FrenchDeck` example, is one of the most widely used applications of the special methods. Making the most of sequence types is the subject of [Chapter 2](#), and implementing your own sequence will be covered in [Chapter 10](#) when we create a multidimensional extension of the `Vector` class.

Thanks to operator overloading, Python offers a rich selection of numeric types, from the built-ins to `decimal.Decimal` and `fractions.Fraction`, all supporting infix arithmetic operators. Implementing operators, including reversed operators and augmented assignment, will be shown in [Chapter 13](#) via enhancements of the `Vector` example.

The use and implementation of the majority of the remaining special methods of the Python data model is covered throughout this book.

Further Reading

The “[Data Model](#)” chapter of *The Python Language Reference* is the canonical source for the subject of this chapter and much of this book.

Python in a Nutshell, 2nd Edition (O’Reilly) by Alex Martelli has excellent coverage of the data model. As I write this, the most recent edition of the *Nutshell* book is from 2006 and focuses on Python 2.5, but there have been very few changes in the data model since then, and Martelli’s description of the mechanics of attribute access is the most authoritative I’ve seen apart from the actual C source code of CPython. Martelli is also a prolific contributor to Stack Overflow, with more than 5,000 answers posted. See his user profile at [Stack Overflow](#).

David Beazley has two books covering the data model in detail in the context of Python 3: *Python Essential Reference, 4th Edition* (Addison-Wesley Professional), and *Python Cookbook, 3rd Edition* (O’Reilly), coauthored with Brian K. Jones.

The Art of the Metaobject Protocol (AMOP, MIT Press) by Gregor Kiczales, Jim des Rivieres, and Daniel G. Bobrow explains the concept of a metaobject protocol (MOP), of which the Python data model is one example.

Soapbox

Data Model or Object Model?

What the Python documentation calls the “Python data model,” most authors would say is the “Python object model.” Alex Martelli’s *Python in a Nutshell 2E*, and David Beazley’s *Python Essential Reference 4E* are the best books covering the “Python data model,” but

they always refer to it as the “object model.” On Wikipedia, the first definition of **object model** is “The properties of objects in general in a specific computer programming language.” This is what the “Python data model” is about. In this book, I will use “data model” because the documentation favors that term when referring to the Python object model, and because it is the title of the **chapter of *The Python Language Reference*** most relevant to our discussions.

Magic Methods

The Ruby community calls their equivalent of the special methods *magic methods*. Many in the Python community adopt that term as well. I believe the special methods are actually the opposite of magic. Python and Ruby are the same in this regard: both empower their users with a rich metaobject protocol that is not magic, but enables users to leverage the same tools available to core developers.

In contrast, consider JavaScript. Objects in that language have features that are magic, in the sense that you cannot emulate them in your own user-defined objects. For example, before JavaScript 1.8.5, you could not define read-only attributes in your JavaScript objects, but some built-in objects always had read-only attributes. In JavaScript, read-only attributes were “magic,” requiring supernatural powers that a user of the language did not have until ECMAScript 5.1 came out in 2009. The metaobject protocol of JavaScript is evolving, but historically it has been more limited than those of Python and Ruby.

Metaobjects

The Art of the Metaobject Protocol (AMOP) is my favorite computer book title. Less subjectively, the term *metaobject protocol* is useful to think about the Python data model and similar features in other languages. The *metaobject* part refers to the objects that are the building blocks of the language itself. In this context, *protocol* is a synonym of *interface*. So a *metaobject protocol* is a fancy synonym for object model: an API for core language constructs.

A rich metaobject protocol enables extending a language to support new programming paradigms. Gregor Kiczales, the first author of the *AMOP* book, later became a pioneer in aspect-oriented programming and the initial author of AspectJ, an extension of Java implementing that paradigm. Aspect-oriented programming is much easier to implement in a dynamic language like Python, and several frameworks do it, but the most important is **`zope.interface`**, which is briefly discussed in “**Further Reading**” on page 342 of **Chapter 11**.

PART II

Data Structures

HPP Ch 1 | Understanding Performant Python

Understanding Performant Python

Questions You'll Be Able to Answer After This Chapter

- What are the elements of a computer's architecture?
- What are some common alternate computer architectures?
- How does Python abstract the underlying computer architecture?
- What are some of the hurdles to making performant Python code?
- What strategies can help you become a highly performant programmer?

Programming computers can be thought of as moving bits of data and transforming them in special ways to achieve a particular result. However, these actions have a time cost. Consequently, *high performance programming* can be thought of as the act of minimizing these operations either by reducing the overhead (i.e., writing more efficient code) or by changing the way that we do these operations to make each one more meaningful (i.e., finding a more suitable algorithm).

Let's focus on reducing the overhead in code in order to gain more insight into the actual hardware on which we are moving these bits. This may seem like a futile exercise, since Python works quite hard to abstract away direct interactions with the hardware. However, by understanding both the best way that bits can be moved in the real hardware and the ways that Python's abstractions force your bits to move, you can make progress toward writing high performance programs in Python.

The Fundamental Computer System

The underlying components that make up a computer can be simplified into three basic parts: the computing units, the memory units, and the connections between them. In addition, each of these units has different properties that we can use to understand them. The computational unit has the property of how many computations it can do per second, the memory unit has the properties of how much data it can hold and how fast we can read from and write to it, and finally, the connections have the property of how fast they can move data from one place to another.

Using these building blocks, we can talk about a standard workstation at multiple levels of sophistication. For example, the standard workstation can be thought of as having a central processing unit (CPU) as the computational unit, connected to both the random access memory (RAM) and the hard drive as two separate memory units (each having different capacities and read/write speeds), and finally a bus that provides the connections between all of these parts. However, we can also go into more detail and see that the CPU itself has several memory units in it: the L1, L2, and sometimes even the L3 and L4 cache, which have small capacities but very fast speeds (from several kilobytes to a dozen megabytes). Furthermore, new computer architectures generally come with new configurations (for example, Intel’s SkyLake CPUs replaced the frontside bus with the Intel Ultra Path Interconnect and restructured many connections). Finally, in both of these approximations of a workstation we have neglected the network connection, which is effectively a very slow connection to potentially many other computing and memory units!

To help untangle these various intricacies, let’s go over a brief description of these fundamental blocks.

Computing Units

The *computing unit* of a computer is the centerpiece of its usefulness—it provides the ability to transform any bits it receives into other bits or to change the state of the current process. CPUs are the most commonly used computing unit; however, graphics processing units (GPUs) are gaining popularity as auxiliary computing units. They were originally used to speed up computer graphics but are becoming more applicable for numerical applications and are useful thanks to their intrinsically parallel nature, which allows many calculations to happen simultaneously. Regardless of its type, a computing unit takes in a series of bits (for example, bits representing numbers) and outputs another set of bits (for example, bits representing the sum of those numbers). In addition to the basic arithmetic operations on integers and real numbers and bitwise operations on binary numbers, some computing units also provide very specialized operations, such as the “fused multiply add” operation, which takes in three numbers, A , B , and C , and returns the value $A * B + C$.

The main properties of interest in a computing unit are the number of operations it can do in one cycle and the number of cycles it can do in one second. The first value is measured by its instructions per cycle (IPC),¹ while the latter value is measured by its clock speed. These two measures are always competing with each other when new computing units are being made. For example, the Intel Core series has a very high IPC but a lower clock speed, while the Pentium 4 chip has the reverse. GPUs, on the other hand, have a very high IPC and clock speed, but they suffer from other problems like the slow communications that we discuss in “Communications Layers” on page 8.

Furthermore, although increasing clock speed almost immediately speeds up all programs running on that computational unit (because they are able to do more calculations per second), having a higher IPC can also drastically affect computing by changing the level of *vectorization* that is possible. Vectorization occurs when a CPU is provided with multiple pieces of data at a time and is able to operate on all of them at once. This sort of CPU instruction is known as single instruction, multiple data (SIMD).

In general, computing units have advanced quite slowly over the past decade (see Figure 1-1). Clock speeds and IPC have both been stagnant because of the physical limitations of making transistors smaller and smaller. As a result, chip manufacturers have been relying on other methods to gain more speed, including simultaneous multithreading (where multiple threads can run at once), more clever out-of-order execution, and multicore architectures.

Hyperthreading presents a virtual second CPU to the host operating system (OS), and clever hardware logic tries to interleave two threads of instructions into the execution units on a single CPU. When successful, gains of up to 30% over a single thread can be achieved. Typically, this works well when the units of work across both threads use different types of execution units—for example, one performs floating-point operations and the other performs integer operations.

Out-of-order execution enables a compiler to spot that some parts of a linear program sequence do not depend on the results of a previous piece of work, and therefore that both pieces of work could occur in any order or at the same time. As long as sequential results are presented at the right time, the program continues to execute correctly, even though pieces of work are computed out of their programmed order. This enables some instructions to execute when others might be blocked (e.g., waiting for a memory access), allowing greater overall utilization of the available resources.

¹ Not to be confused with interprocess communication, which shares the same acronym—we’ll look at that topic in Chapter 9.

Finally, and most important for the higher-level programmer, there is the prevalence of multicore architectures. These architectures include multiple CPUs within the same unit, which increases the total capability without running into barriers to making each individual unit faster. This is why it is currently hard to find any machine with fewer than two cores—in this case, the computer has two physical computing units that are connected to each other. While this increases the total number of operations that *can* be done per second, it can make writing code more difficult!

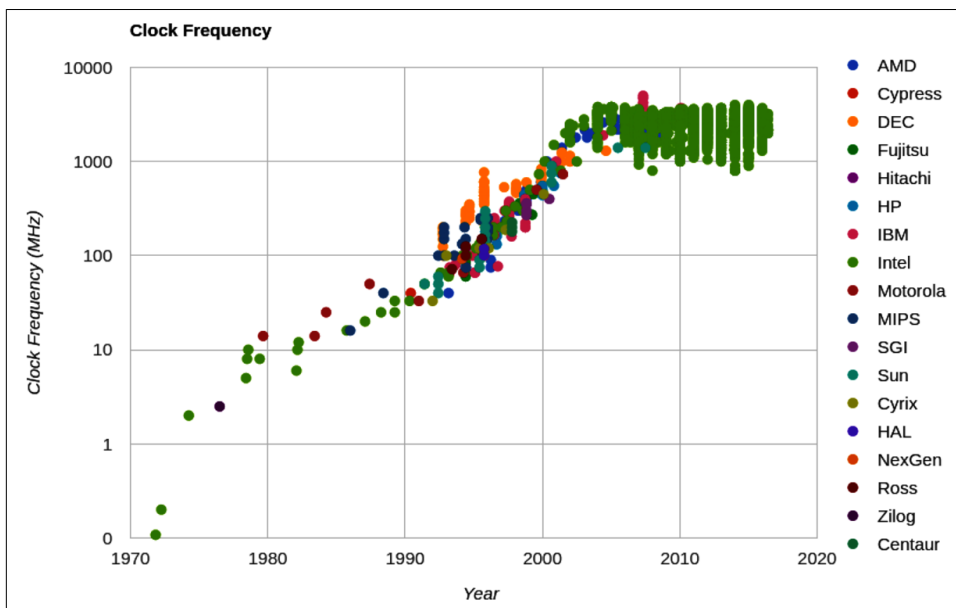


Figure 1-1. Clock speed of CPUs over time (from *CPU DB*)

Simply adding more cores to a CPU does not always speed up a program’s execution time. This is because of something known as *Amdahl’s law*. Simply stated, Amdahl’s law is this: if a program designed to run on multiple cores has some subroutines that must run on one core, this will be the limitation for the maximum speedup that can be achieved by allocating more cores.

For example, if we had a survey we wanted one hundred people to fill out, and that survey took 1 minute to complete, we could complete this task in 100 minutes if we had one person asking the questions (i.e., this person goes to participant 1, asks the questions, waits for the responses, and then moves to participant 2). This method of having one person asking the questions and waiting for responses is similar to a serial process. In serial processes, we have operations being satisfied one at a time, each one waiting for the previous operation to complete.

However, we could perform the survey in parallel if we had two people asking the questions, which would let us finish the process in only 50 minutes. This can be done

because each individual person asking the questions does not need to know anything about the other person asking questions. As a result, the task can easily be split up without having any dependency between the question askers.

Adding more people asking the questions will give us more speedups, until we have one hundred people asking questions. At this point, the process would take 1 minute and would be limited simply by the time it takes a participant to answer questions. Adding more people asking questions will not result in any further speedups, because these extra people will have no tasks to perform—all the participants are already being asked questions! At this point, the only way to reduce the overall time to run the survey is to reduce the amount of time it takes for an individual survey, the serial portion of the problem, to complete. Similarly, with CPUs, we can add more cores that can perform various chunks of the computation as necessary until we reach a point where the bottleneck is the time it takes for a specific core to finish its task. In other words, the bottleneck in any parallel calculation is always the smaller serial tasks that are being spread out.

Furthermore, a major hurdle with utilizing multiple cores in Python is Python's use of a *global interpreter lock* (GIL). The GIL makes sure that a Python process can run only one instruction at a time, regardless of the number of cores it is currently using. This means that even though some Python code has access to multiple cores at a time, only one core is running a Python instruction at any given time. Using the previous example of a survey, this would mean that even if we had 100 question askers, only one person could ask a question and listen to a response at a time. This effectively removes any sort of benefit from having multiple question askers! While this may seem like quite a hurdle, especially if the current trend in computing is to have multiple computing units rather than having faster ones, this problem can be avoided by using other standard library tools, like `multiprocessing` (Chapter 9), technologies like `numpy` or `numexpr` (Chapter 6), `Cython` (Chapter 7), or distributed models of computing (Chapter 10).



Python 3.2 also saw a major rewrite of the GIL, which made the system much more nimble, alleviating many of the concerns around the system for single-thread performance. Although it still locks Python into running only one instruction at a time, the GIL now does better at switching between those instructions and doing so with less overhead.

Memory Units

Memory units in computers are used to store bits. These could be bits representing variables in your program or bits representing the pixels of an image. Thus, the abstraction of a memory unit applies to the registers in your motherboard as well as your RAM and hard drive. The one major difference between all of these types of

memory units is the speed at which they can read/write data. To make things more complicated, the read/write speed is heavily dependent on the way that data is being read.

For example, most memory units perform much better when they read one large chunk of data as opposed to many small chunks (this is referred to as *sequential read* versus *random data*). If the data in these memory units is thought of as pages in a large book, this means that most memory units have better read/write speeds when going through the book page by page rather than constantly flipping from one random page to another. While this fact is generally true across all memory units, the amount that this affects each type is drastically different.

In addition to the read/write speeds, memory units also have *latency*, which can be characterized as the time it takes the device to find the data that is being used. For a spinning hard drive, this latency can be high because the disk needs to physically spin up to speed and the read head must move to the right position. On the other hand, for RAM, this latency can be quite small because everything is solid state. Here is a short description of the various memory units that are commonly found inside a standard workstation, in order of read/write speeds:²

Spinning hard drive

Long-term storage that persists even when the computer is shut down. Generally has slow read/write speeds because the disk must be physically spun and moved. Degraded performance with random access patterns but very large capacity (10 terabyte range).

Solid-state hard drive

Similar to a spinning hard drive, with faster read/write speeds but smaller capacity (1 terabyte range).

RAM

Used to store application code and data (such as any variables being used). Has fast read/write characteristics and performs well with random access patterns, but is generally limited in capacity (64 gigabyte range).

L1/L2 cache

Extremely fast read/write speeds. Data going to the CPU *must* go through here. Very small capacity (megabytes range).

Figure 1-2 gives a graphic representation of the differences between these types of memory units by looking at the characteristics of currently available consumer hardware.

² Speeds in this section are from <https://oreil.ly/pToi7>.

A clearly visible trend is that read/write speeds and capacity are inversely proportional—as we try to increase speed, capacity gets reduced. Because of this, many systems implement a tiered approach to memory: data starts in its full state in the hard drive, part of it moves to RAM, and then a much smaller subset moves to the L1/L2 cache. This method of tiering enables programs to keep memory in different places depending on access speed requirements. When trying to optimize the memory patterns of a program, we are simply optimizing which data is placed where, how it is laid out (in order to increase the number of sequential reads), and how many times it is moved among the various locations. In addition, methods such as asynchronous I/O and preemptive caching provide ways to make sure that data is always where it needs to be without having to waste computing time—most of these processes can happen independently, while other calculations are being performed!

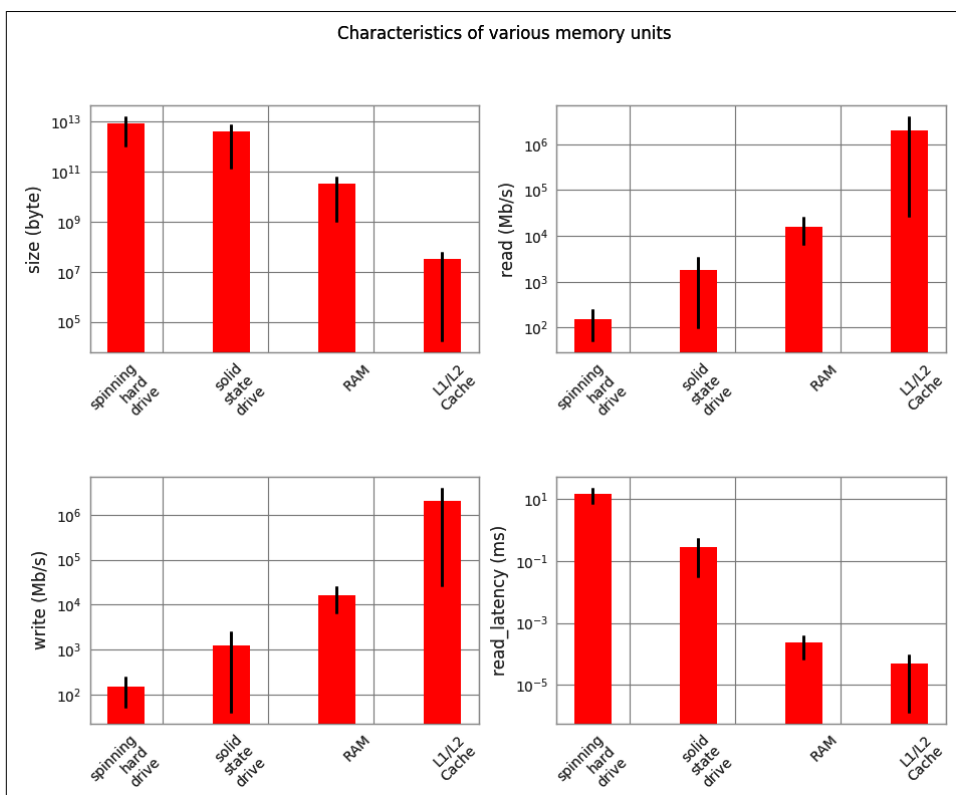


Figure 1-2. Characteristic values for different types of memory units (values from February 2014)

Communications Layers

Finally, let's look at how all of these fundamental blocks communicate with each other. Many modes of communication exist, but all are variants on a thing called a *bus*.

The *frontside bus*, for example, is the connection between the RAM and the L1/L2 cache. It moves data that is ready to be transformed by the processor into the staging ground to get ready for calculation, and it moves finished calculations out. There are other buses, too, such as the external bus that acts as the main route from hardware devices (such as hard drives and networking cards) to the CPU and system memory. This external bus is generally slower than the frontside bus.

In fact, many of the benefits of the L1/L2 cache are attributable to the faster bus. Being able to queue up data necessary for computation in large chunks on a slow bus (from RAM to cache) and then having it available at very fast speeds from the cache lines (from cache to CPU) enables the CPU to do more calculations without waiting such a long time.

Similarly, many of the drawbacks of using a GPU come from the bus it is connected on: since the GPU is generally a peripheral device, it communicates through the PCI bus, which is much slower than the frontside bus. As a result, getting data into and out of the GPU can be quite a taxing operation. The advent of heterogeneous computing, or computing blocks that have both a CPU and a GPU on the frontside bus, aims at reducing the data transfer cost and making GPU computing more of an available option, even when a lot of data must be transferred.

In addition to the communication blocks within the computer, the network can be thought of as yet another communication block. This block, though, is much more pliable than the ones discussed previously; a network device can be connected to a memory device, such as a network attached storage (NAS) device or another computing block, as in a computing node in a cluster. However, network communications are generally much slower than the other types of communications mentioned previously. While the frontside bus can transfer dozens of gigabits per second, the network is limited to the order of several dozen megabits.

It is clear, then, that the main property of a bus is its speed: how much data it can move in a given amount of time. This property is given by combining two quantities: how much data can be moved in one transfer (bus width) and how many transfers the bus can do per second (bus frequency). It is important to note that the data moved in one transfer is always sequential: a chunk of data is read off of the memory and moved to a different place. Thus, the speed of a bus is broken into these two quantities because individually they can affect different aspects of computation: a large bus width can help vectorized code (or any code that sequentially reads through memory) by making it possible to move all the relevant data in one transfer, while, on

the other hand, having a small bus width but a very high frequency of transfers can help code that must do many reads from random parts of memory. Interestingly, one of the ways that these properties are changed by computer designers is by the physical layout of the motherboard: when chips are placed close to one another, the length of the physical wires joining them is smaller, which can allow for faster transfer speeds. In addition, the number of wires itself dictates the width of the bus (giving real physical meaning to the term!).

Since interfaces can be tuned to give the right performance for a specific application, it is no surprise that there are hundreds of types. **Figure 1-3** shows the bitrates for a sampling of common interfaces. Note that this doesn't speak at all about the latency of the connections, which dictates how long it takes for a data request to be responded to (although latency is very computer-dependent, some basic limitations are inherent to the interfaces being used).

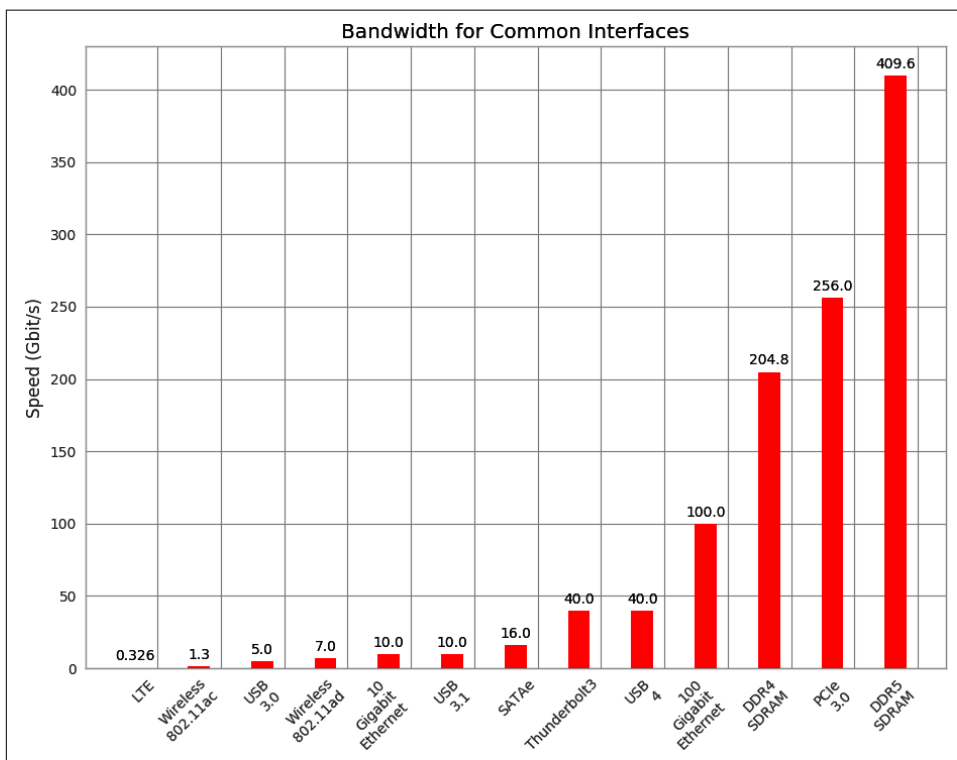


Figure 1-3. Connection speeds of various common interfaces³

³ Data is from <https://oreil.ly/7SC8d>.

Putting the Fundamental Elements Together

Understanding the basic components of a computer is not enough to fully understand the problems of high performance programming. The interplay of all of these components and how they work together to solve a problem introduces extra levels of complexity. In this section we will explore some toy problems, illustrating how the ideal solutions would work and how Python approaches them.

A warning: this section may seem bleak—most of the remarks in this section seem to say that Python is natively incapable of dealing with the problems of performance. This is untrue, for two reasons. First, among all of the “components of performant computing,” we have neglected one very important component: the developer. What native Python may lack in performance, it gets back right away with speed of development. Furthermore, throughout the book we will introduce modules and philosophies that can help mitigate many of the problems described here with relative ease. With both of these aspects combined, we will keep the fast development mindset of Python while removing many of the performance constraints.

Idealized Computing Versus the Python Virtual Machine

To better understand the components of high performance programming, let’s look at a simple code sample that checks whether a number is prime:

```
import math

def check_prime(number):
    sqrt_number = math.sqrt(number)
    for i in range(2, int(sqrt_number) + 1):
        if (number / i).is_integer():
            return False
    return True

print(f"check_prime(10,000,000) = {check_prime(10_000_000)}")
# check_prime(10,000,000) = False
print(f"check_prime(10,000,019) = {check_prime(10_000_019)}")
# check_prime(10,000,019) = True
```

Let’s analyze this code using our abstract model of computation and then draw comparisons to what happens when Python runs this code. As with any abstraction, we will neglect many of the subtleties in both the idealized computer and the way that Python runs the code. However, this is generally a good exercise to perform before solving a problem: think about the general components of the algorithm and what would be the best way for the computing components to come together to find a solution. By understanding this ideal situation and having knowledge of what is actually happening under the hood in Python, we can iteratively bring our Python code closer to the optimal code.

Idealized computing

When the code starts, we have the value of `number` stored in RAM. To calculate `sqrt_number`, we need to send the value of `number` to the CPU. Ideally, we could send the value once; it would get stored inside the CPU's L1/L2 cache, and the CPU would do the calculations and then send the values back to RAM to get stored. This scenario is ideal because we have minimized the number of reads of the value of `number` from RAM, instead opting for reads from the L1/L2 cache, which are much faster. Furthermore, we have minimized the number of data transfers through the frontside bus, by using the L1/L2 cache which is connected directly to the CPU.



This theme of keeping data where it is needed and moving it as little as possible is very important when it comes to optimization. The concept of “heavy data” refers to the time and effort required to move data around, which is something we would like to avoid.

For the loop in the code, rather than sending one value of `i` at a time to the CPU, we would like to send both `number` and *several* values of `i` to the CPU to check at the same time. This is possible because the CPU vectorizes operations with no additional time cost, meaning it can do multiple independent computations at the same time. So we want to send `number` to the CPU cache, in addition to as many values of `i` as the cache can hold. For each of the `number/i` pairs, we will divide them and check if the result is a whole number; then we will send a signal back indicating whether any of the values was indeed an integer. If so, the function ends. If not, we repeat. In this way, we need to communicate back only one result for many values of `i`, rather than depending on the slow bus for every value. This takes advantage of a CPU's ability to *vectorize* a calculation, or run one instruction on multiple data in one clock cycle.

This concept of vectorization is illustrated by the following code:

```
import math

def check_prime(number):
    sqrt_number = math.sqrt(number)
    numbers = range(2, int(sqrt_number)+1)
    for i in range(0, len(numbers), 5):
        # the following line is not valid Python code
        result = (number / numbers[i:(i + 5)]).is_integer()
        if any(result):
            return False
    return True
```

Here, we set up the processing such that the division and the checking for integers are done on a set of five values of `i` at a time. If properly vectorized, the CPU can do this line in one step as opposed to doing a separate calculation for every `i`. Ideally, the `any(result)` operation would also happen in the CPU without having to transfer the

results back to RAM. We will talk more about vectorization, how it works, and when it benefits your code in [Chapter 6](#).

Python's virtual machine

The Python interpreter does a lot of work to try to abstract away the underlying computing elements that are being used. At no point does a programmer need to worry about allocating memory for arrays, how to arrange that memory, or in what sequence it is being sent to the CPU. This is a benefit of Python, since it lets you focus on the algorithms that are being implemented. However, it comes at a huge performance cost.

It is important to realize that at its core, Python is indeed running a set of very optimized instructions. The trick, however, is getting Python to perform them in the correct sequence to achieve better performance. For example, it is quite easy to see that, in the following example, `search_fast` will run faster than `search_slow` simply because it skips the unnecessary computations that result from not terminating the loop early, even though both solutions have runtime $O(n)$. However, things can get complicated when dealing with derived types, special Python methods, or third-party modules. For example, can you immediately tell which function will be faster: `search_unknown1` or `search_unknown2`?

```
def search_fast(haystack, needle):
    for item in haystack:
        if item == needle:
            return True
    return False

def search_slow(haystack, needle):
    return_value = False
    for item in haystack:
        if item == needle:
            return_value = True
    return return_value

def search_unknown1(haystack, needle):
    return any((item == needle for item in haystack))

def search_unknown2(haystack, needle):
    return any([item == needle for item in haystack])
```

Identifying slow regions of code through profiling and finding more efficient ways of doing the same calculations is similar to finding these useless operations and removing them; the end result is the same, but the number of computations and data transfers is reduced drastically.

One of the impacts of this abstraction layer is that vectorization is not immediately achievable. Our initial prime number routine will run one iteration of the loop per

value of `i` instead of combining several iterations. However, looking at the abstracted vectorization example, we see that it is not valid Python code, since we cannot divide a float by a list. External libraries such as `numpy` will help with this situation by adding the ability to do vectorized mathematical operations.

Furthermore, Python’s abstraction hurts any optimizations that rely on keeping the L1/L2 cache filled with the relevant data for the next computation. This comes from many factors, the first being that Python objects are not laid out in the most optimal way in memory. This is a consequence of Python being a garbage-collected language—memory is automatically allocated and freed when needed. This creates memory fragmentation that can hurt the transfers to the CPU caches. In addition, at no point is there an opportunity to change the layout of a data structure directly in memory, which means that one transfer on the bus may not contain all the relevant information for a computation, even though it might have all fit within the bus width.⁴

A second, more fundamental problem comes from Python’s dynamic types and the language not being compiled. As many C programmers have learned throughout the years, the compiler is often smarter than you are. When compiling code that is static, the compiler can do many tricks to change the way things are laid out and how the CPU will run certain instructions in order to optimize them. Python, however, is not compiled: to make matters worse, it has dynamic types, which means that inferring any possible opportunities for optimizations algorithmically is drastically harder since code functionality can be changed during runtime. There are many ways to mitigate this problem, foremost being the use of Cython, which allows Python code to be compiled and allows the user to create “hints” to the compiler as to how dynamic the code actually is.

Finally, the previously mentioned GIL can hurt performance if trying to parallelize this code. For example, let’s assume we change the code to use multiple CPU cores such that each core gets a chunk of the numbers from 2 to `sqr tN`. Each core can do its calculation for its chunk of numbers, and then, when the calculations are all done, the cores can compare their calculations. Although we lose the early termination of the loop since each core doesn’t know if a solution has been found, we can reduce the number of checks each core has to do (if we had `M` cores, each core would have to do `sqr tN / M` checks). However, because of the GIL, only one core can be used at a time. This means that we would effectively be running the same code as the unparallelized version, but we no longer have early termination. We can avoid this problem by using multiple processes (with the `multiprocessing` module) instead of multiple threads, or by using Cython or foreign functions.

4 In [Chapter 6](#), we’ll see how we can regain this control and tune our code all the way down to the memory utilization patterns.

So Why Use Python?

Python is highly expressive and easy to learn—new programmers quickly discover that they can do quite a lot in a short space of time. Many Python libraries wrap tools written in other languages to make it easy to call other systems; for example, the scikit-learn machine learning system wraps LIBLINEAR and LIBSVM (both of which are written in C), and the numpy library includes BLAS and other C and Fortran libraries. As a result, Python code that properly utilizes these modules can indeed be as fast as comparable C code.

Python is described as “batteries included,” as many important tools and stable libraries are built in. These include the following:

`unicode` *and* `bytes`

Baked into the core language

`array`

Memory-efficient arrays for primitive types

`math`

Basic mathematical operations, including some simple statistics

`sqlite3`

A wrapper around the prevalent SQL file-based storage engine SQLite3

`collections`

A wide variety of objects, including a deque, counter, and dictionary variants

`asyncio`

Concurrent support for I/O-bound tasks using `async` and `await` syntax

A huge variety of libraries can be found outside the core language, including these:

`numpy`

A numerical Python library (a bedrock library for anything to do with matrices)

`scipy`

A very large collection of trusted scientific libraries, often wrapping highly respected C and Fortran libraries

`pandas`

A library for data analysis, similar to R’s data frames or an Excel spreadsheet, built on `scipy` and `numpy`

`scikit-learn`

Rapidly turning into the default machine learning library, built on `scipy`

tornado

A library that provides easy bindings for concurrency

PyTorch and TensorFlow

Deep learning frameworks from Facebook and Google with strong Python and GPU support

NLTK, SpaCy, and Gensim

Natural language-processing libraries with deep Python support

Database bindings

For communicating with virtually all databases, including Redis, MongoDB, HDF5, and SQL

Web development frameworks

Performant systems for creating websites, such as aiohttp, django, pyramid, flask, and tornado

OpenCV

Bindings for computer vision

API bindings

For easy access to popular web APIs such as Google, Twitter, and LinkedIn

A large selection of managed environments and shells is available to fit various deployment scenarios, including the following:

- The standard distribution, available at <http://python.org>
- pipenv, pyenv, and virtualenv for simple, lightweight, and portable Python environments
- Docker for simple-to-start-and-reproduce environments for development or production
- Anaconda Inc.'s Anaconda, a scientifically focused environment
- Sage, a Matlab-like environment that includes an integrated development environment (IDE)
- IPython, an interactive Python shell heavily used by scientists and developers
- Jupyter Notebook, a browser-based extension to IPython, heavily used for teaching and demonstrations

One of Python's main strengths is that it enables fast prototyping of an idea. Because of the wide variety of supporting libraries, it is easy to test whether an idea is feasible, even if the first implementation might be rather flaky.

If you want to make your mathematical routines faster, look to `numpy`. If you want to experiment with machine learning, try `scikit-learn`. If you are cleaning and manipulating data, then `pandas` is a good choice.

In general, it is sensible to raise the question, “If our system runs faster, will we as a team run slower in the long run?” It is always possible to squeeze more performance out of a system if enough work-hours are invested, but this might lead to brittle and poorly understood optimizations that ultimately trip up the team.

One example might be the introduction of Cython (see “[Cython](#)” on page 167), a compiler-based approach to annotating Python code with C-like types so the transformed code can be compiled using a C compiler. While the speed gains can be impressive (often achieving C-like speeds with relatively little effort), the cost of supporting this code will increase. In particular, it might be harder to support this new module, as team members will need a certain maturity in their programming ability to understand some of the trade-offs that have occurred when leaving the Python virtual machine that introduced the performance increase.

How to Be a Highly Performant Programmer

Writing high performance code is only one part of being highly performant with successful projects over the longer term. Overall team velocity is far more important than speedups and complicated solutions. Several factors are key to this—good structure, documentation, debuggability, and shared standards.

Let’s say you create a prototype. You didn’t test it thoroughly, and it didn’t get reviewed by your team. It does seem to be “good enough,” and it gets pushed to production. Since it was never written in a structured way, it lacks tests and is undocumented. All of a sudden there’s an inertia-causing piece of code for someone else to support, and often management can’t quantify the cost to the team.

As this solution is hard to maintain, it tends to stay unloved—it never gets restructured, it doesn’t get the tests that’d help the team refactor it, and nobody else likes to touch it, so it falls to one developer to keep it running. This can cause an awful bottleneck at times of stress and raises a significant risk: what would happen if that developer left the project?

Typically, this development style occurs when the management team doesn’t understand the ongoing inertia that’s caused by hard-to-maintain code. Demonstrating that in the longer-term tests and documentation can help a team stay highly productive and can help convince managers to allocate time to “cleaning up” this prototype code.

In a research environment, it is common to create many Jupyter Notebooks using poor coding practices while iterating through ideas and different datasets. The

intention is always to “write it up properly” at a later stage, but that later stage never occurs. In the end, a working result is obtained, but the infrastructure to reproduce it, test it, and trust the result is missing. Once again the risk factors are high, and the trust in the result will be low.

There’s a general approach that will serve you well:

Make it work

First you build a good-enough solution. It is very sensible to “build one to throw away” that acts as a prototype solution, enabling a better structure to be used for the second version. It is always sensible to do some up-front planning before coding; otherwise, you’ll come to reflect that “We saved an hour’s thinking by coding all afternoon.” In some fields this is better known as “Measure twice, cut once.”

Make it right

Next, you add a strong test suite backed by documentation and clear reproducibility instructions so that another team member can take it on.

Make it fast

Finally, we can focus on profiling and compiling or parallelization and using the existing test suite to confirm that the new, faster solution still works as expected.

Good Working Practices

There are a few “must haves”—documentation, good structure, and testing are key.

Some project-level documentation will help you stick to a clean structure. It’ll also help you and your colleagues in the future. Nobody will thank you (yourself included) if you skip this part. Writing this up in a *README* file at the top level is a sensible starting point; it can always be expanded into a *docs/* folder later if required.

Explain the purpose of the project, what’s in the folders, where the data comes from, which files are critical, and how to run it all, including how to run the tests.

Micha recommends also using Docker. A top-level Dockerfile will explain to your future-self exactly which libraries you need from the operating system to make this project run successfully. It also removes the difficulty of running this code on other machines or deploying it to a cloud environment.

Add a *tests/* folder and add some unit tests. We prefer *pytest* as a modern test runner, as it builds on Python’s built-in *unittest* module. Start with just a couple of tests and then build them up. Progress to using the *coverage* tool, which will report how many lines of your code are actually covered by the tests—it’ll help avoid nasty surprises.

If you're inheriting legacy code and it lacks tests, a high-value activity is to add some tests up front. Some “integration tests” that check the overall flow of the project and confirm that with certain input data you get specific output results will help your sanity as you subsequently make modifications.

Every time something in the code bites you, add a test. There's no value to being bitten twice by the same problem.

Docstrings in your code for each function, class, and module will always help you. Aim to provide a useful description of what's *achieved* by the function, and where possible include a short example to demonstrate the expected output. Look at the docstrings inside `numpy` and `scikit-learn` if you'd like inspiration.

Whenever your code becomes too long—such as functions longer than one screen—be comfortable with refactoring the code to make it shorter. Shorter code is easier to test and easier to support.



When you're developing your tests, think about following a test-driven development methodology. When you know exactly what you need to develop and you have testable examples at hand—this method becomes very efficient.

You write your tests, run them, watch them fail, and *then* add the functions and the necessary minimum logic to support the tests that you've written. When your tests all work, you're done. By figuring out the expected input and output of a function ahead of time, you'll find implementing the logic of the function relatively straightforward.

If you can't define your tests ahead of time, it naturally raises the question, do you really understand what your function needs to do? If not, can you write it correctly in an efficient manner? This method doesn't work so well if you're in a creative process and researching data that you don't yet understand well.

Always use source control—you'll only thank yourself when you overwrite something critical at an inconvenient moment. Get used to committing frequently (daily, or even every 10 minutes) and pushing to your repository every day.

Keep to the standard PEP8 coding standard. Even better, adopt `black` (the opinionated code formatter) on a pre-commit source control hook so it just rewrites your code to the standard for you. Use `flake8` to lint your code to avoid other mistakes.

Creating environments that are isolated from the operating system will make your life easier. Ian prefers `Anaconda`, while Micha prefers `pipenv` coupled with `Docker`. Both are sensible solutions and are significantly better using the operating system's global Python environment!

Remember that automation is your friend. Doing less manual work means there's less chance of errors creeping in. Automated build systems, continuous integration with automated test suite runners, and automated deployment systems turn tedious and error-prone tasks into standard processes that anyone can run and support.

Finally, remember that readability is far more important than being clever. Short snippets of complex and hard-to-read code will be hard for you and your colleagues to maintain, so people will be scared of touching this code. Instead, write a longer, easier-to-read function and back it with useful documentation showing what it'll return, and complement this with tests to confirm that it *does* work as you expect.

Some Thoughts on Good Notebook Practice

If you're using Jupyter Notebooks, they're great for visual communication, but they facilitate laziness. If you find yourself leaving long functions inside your Notebooks, be comfortable extracting them out to a Python module and then adding tests.

Consider prototyping your code in IPython or the QtConsole; turn lines of code into functions in a Notebook and then promote them out of the Notebook and into a module complemented by tests. Finally, consider wrapping the code in a class if encapsulation and data hiding are useful.

Liberally spread `assert` statements throughout a Notebook to check that your functions are behaving as expected. You can't easily test code inside a Notebook, and until you've refactored your functions into separate modules, `assert` checks are a simple way to add some level of validation. You shouldn't trust this code until you've extracted it to a module and written sensible unit tests.

Using `assert` statements to check data in your code should be frowned upon. It is an easy way to assert that certain conditions are being met, but it isn't idiomatic Python. To make your code easier to read by other developers, check your expected data state and then raise an appropriate exception if the check fails. A common exception would be `ValueError` if a function encounters an unexpected value. The **Bulwark library** is an example of a testing framework focused on Pandas to check that your data meets the specified constraints.

You may also want to add some sanity checks at the end of your Notebook—a mixture of logic checks and `raise` and `print` statements that demonstrate that you've just generated exactly what you needed. When you return to this code in six months, you'll thank yourself for making it easy to see that it worked correctly all the way through!

One difficulty with Notebooks is sharing code with source control systems. **nbdime** is one of a growing set of new tools that let you diff your Notebooks. It is a lifesaver and enables collaboration with colleagues.

Getting the Joy Back into Your Work

Life can be complicated. In the five years since your authors wrote the first edition of this book, we've jointly experienced through friends and family a number of life situations, including depression, cancer, home relocations, successful business exits and failures, and career direction shifts. Inevitably, these external events will have an impact on anyone's work and outlook on life.

Remember to keep looking for the joy in new activities. There are always interesting details or requirements once you start poking around. You might ask, "why did they make that decision?" and "how would I do it differently?" and all of a sudden you're ready to start a conversation about how things might be changed or improved.

Keep a log of things that are worth celebrating. It is so easy to forget about accomplishments and to get caught up in the day-to-day. People get burned out because they're always running to keep up, and they forget how much progress they've made.

We suggest that you build a list of items worth celebrating and note how you celebrate them. Ian keeps such a list—he's happily surprised when he goes to update the list and sees just how many cool things have happened (and might otherwise have been forgotten!) in the last year. These shouldn't just be work milestones; include hobbies and sports, and celebrate the milestones you've achieved. Micha makes sure to prioritize his personal life and spend days away from the computer to work on nontechnical projects. It is critical to keep developing your skill set, but it is not necessary to burn out!

Programming, particularly when performance focused, thrives on a sense of curiosity and a willingness to always delve deeper into the technical details. Unfortunately, this curiosity is the first thing to go when you burn out; so take your time and make sure you enjoy the journey, and keep the joy and the curiosity.

HPP Ch 2 | Profiling to Find Bottlenecks

Profiling to Find Bottlenecks

Questions You'll Be Able to Answer After This Chapter

- How can I identify speed and RAM bottlenecks in my code?
- How do I profile CPU and memory usage?
- What depth of profiling should I use?
- How can I profile a long-running application?
- What's happening under the hood with CPython?
- How do I keep my code correct while tuning performance?

Profiling lets us find bottlenecks so we can do the least amount of work to get the biggest practical performance gain. While we'd like to get huge gains in speed and reductions in resource usage with little work, practically you'll aim for your code to run "fast enough" and "lean enough" to fit your needs. Profiling will let you make the most pragmatic decisions for the least overall effort.

Any measurable resource can be profiled (not just the CPU!). In this chapter we look at both CPU time and memory usage. You could apply similar techniques to measure network bandwidth and disk I/O too.

If a program is running too slowly or using too much RAM, you'll want to fix whichever parts of your code are responsible. You could, of course, skip profiling and fix what you *believe* might be the problem—but be wary, as you'll often end up "fixing" the wrong thing. Rather than using your intuition, it is far more sensible to first profile, having defined a hypothesis, before making changes to the structure of your code.

Sometimes it's good to be lazy. By profiling first, you can quickly identify the bottlenecks that need to be solved, and then you can solve just enough of these to achieve the performance you need. If you avoid profiling and jump to optimization, you'll quite likely do more work in the long run. Always be driven by the results of profiling.

Profiling Efficiently

The first aim of profiling is to test a representative system to identify what's slow (or using too much RAM, or causing too much disk I/O or network I/O). Profiling typically adds an overhead (10× to 100× slowdowns can be typical), and you still want your code to be used in as similar to a real-world situation as possible. Extract a test case and isolate the piece of the system that you need to test. Preferably, it'll have been written to be in its own set of modules already.

The basic techniques that are introduced first in this chapter include the `%timeit` magic in IPython, `time.time()`, and a timing decorator. You can use these techniques to understand the behavior of statements and functions.

Then we will cover `cProfile` (“Using the `cProfile` Module” on page 35), showing you how to use this built-in tool to understand which functions in your code take the longest to run. This will give you a high-level view of the problem so you can direct your attention to the critical functions.

Next, we'll look at `line_profiler` (“Using `line_profiler` for Line-by-Line Measurements” on page 40), which will profile your chosen functions on a line-by-line basis. The result will include a count of the number of times each line is called and the percentage of time spent on each line. This is exactly the information you need to understand what's running slowly and why.

Armed with the results of `line_profiler`, you'll have the information you need to move on to using a compiler (Chapter 7).

In Chapter 6, you'll learn how to use `perf stat` to understand the number of instructions that are ultimately executed on a CPU and how efficiently the CPU's caches are utilized. This allows for advanced-level tuning of matrix operations. You should take a look at Example 6-8 when you're done with this chapter.

After `line_profiler`, if you're working with long-running systems, then you'll be interested in `py-spy` to peek into already-running Python processes.

To help you understand why your RAM usage is high, we'll show you `memory_profiler` (“Using `memory_profiler` to Diagnose Memory Usage” on page 46). It is particularly useful for tracking RAM usage over time on a labeled chart, so you can explain to colleagues why certain functions use more RAM than expected.



Whatever approach you take to profiling your code, you must remember to have adequate unit test coverage in your code. Unit tests help you to avoid silly mistakes and to keep your results reproducible. Avoid them at your peril.

Always profile your code before compiling or rewriting your algorithms. You need evidence to determine the most efficient ways to make your code run faster.

Next, we'll give you an introduction to the Python bytecode inside CPython ([“Using the `dis` Module to Examine CPython Bytecode](#)” on page 55), so you can understand what's happening “under the hood.” In particular, having an understanding of how Python's stack-based virtual machine operates will help you understand why certain coding styles run more slowly than others.

Before the end of the chapter, we'll review how to integrate unit tests while profiling ([“Unit Testing During Optimization to Maintain Correctness](#)” on page 59) to preserve the correctness of your code while you make it run more efficiently.

We'll finish with a discussion of profiling strategies ([“Strategies to Profile Your Code Successfully](#)” on page 62) so you can reliably profile your code and gather the correct data to test your hypotheses. Here you'll learn how dynamic CPU frequency scaling and features like Turbo Boost can skew your profiling results, and you'll learn how they can be disabled.

To walk through all of these steps, we need an easy-to-analyze function. The next section introduces the Julia set. It is a CPU-bound function that's a little hungry for RAM; it also exhibits nonlinear behavior (so we can't easily predict the outcomes), which means we need to profile it at runtime rather than analyzing it offline.

Introducing the Julia Set

The **Julia set** is an interesting CPU-bound problem for us to begin with. It is a fractal sequence that generates a complex output image, named after Gaston Julia.

The code that follows is a little longer than a version you might write yourself. It has a CPU-bound component and a very explicit set of inputs. This configuration allows us to profile both the CPU usage and the RAM usage so we can understand which parts of our code are consuming two of our scarce computing resources. This implementation is *deliberately* suboptimal, so we can identify memory-consuming operations and slow statements. Later in this chapter we'll fix a slow logic statement and a memory-consuming statement, and in [Chapter 7](#) we'll significantly speed up the overall execution time of this function.

We will analyze a block of code that produces both a false grayscale plot ([Figure 2-1](#)) and a pure grayscale variant of the Julia set ([Figure 2-3](#)), at the complex point

$c = -0.62772 - 0.42193j$. A Julia set is produced by calculating each pixel in isolation; this is an “embarrassingly parallel problem,” as no data is shared between points.

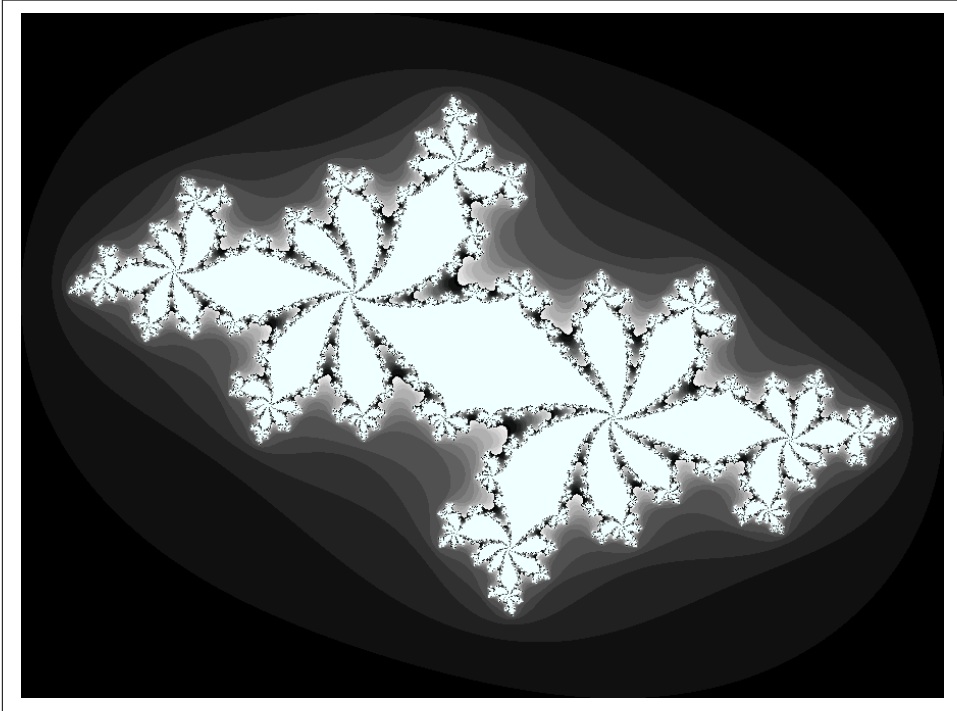


Figure 2-1. Julia set plot with a false gray scale to highlight detail

If we chose a different c , we’d get a different image. The location we have chosen has regions that are quick to calculate and others that are slow to calculate; this is useful for our analysis.

The problem is interesting because we calculate each pixel by applying a loop that could be applied an indeterminate number of times. On each iteration we test to see if this coordinate’s value escapes toward infinity, or if it seems to be held by an attractor. Coordinates that cause few iterations are colored darkly in [Figure 2-1](#), and those that cause a high number of iterations are colored white. White regions are more complex to calculate and so take longer to generate.

We define a set of z coordinates that we’ll test. The function that we calculate squares the complex number z and adds c :

$$f(z) = z^2 + c$$

We iterate on this function while testing to see if the escape condition holds using `abs`. If the escape function is `False`, we break out of the loop and record the number of iterations we performed at this coordinate. If the escape function is never `False`, we stop after `maxiter` iterations. We will later turn this `z`'s result into a colored pixel representing this complex location.

In pseudocode, it might look like this:

```
for z in coordinates:
    for iteration in range(maxiter): # limited iterations per point
        if abs(z) < 2.0: # has the escape condition been broken?
            z = z*z + c
        else:
            break
    # store the iteration count for each z and draw later
```

To explain this function, let's try two coordinates.

We'll use the coordinate that we draw in the top-left corner of the plot at $-1.8-1.8j$. We must test `abs(z) < 2` before we can try the update rule:

```
z = -1.8-1.8j
print(abs(z))

2.54558441227
```

We can see that for the top-left coordinate, the `abs(z)` test will be `False` on the zeroth iteration as $2.54 \geq 2.0$, so we do not perform the update rule. The output value for this coordinate is 0.

Now let's jump to the center of the plot at $z = 0 + 0j$ and try a few iterations:

```
c = -0.62772-0.42193j
z = 0+0j
for n in range(9):
    z = z*z + c
    print(f"{n}: z={z: .5f}, abs(z)={abs(z):0.3f}, c={c: .5f}")

0: z=-0.62772-0.42193j, abs(z)=0.756, c=-0.62772-0.42193j
1: z=-0.41171+0.10778j, abs(z)=0.426, c=-0.62772-0.42193j
2: z=-0.46983-0.51068j, abs(z)=0.694, c=-0.62772-0.42193j
3: z=-0.66777+0.05793j, abs(z)=0.670, c=-0.62772-0.42193j
4: z=-0.18516-0.49930j, abs(z)=0.533, c=-0.62772-0.42193j
5: z=-0.84274-0.23703j, abs(z)=0.875, c=-0.62772-0.42193j
6: z= 0.02630-0.02242j, abs(z)=0.035, c=-0.62772-0.42193j
7: z=-0.62753-0.42311j, abs(z)=0.757, c=-0.62772-0.42193j
8: z=-0.41295+0.10910j, abs(z)=0.427, c=-0.62772-0.42193j
```

We can see that each update to `z` for these first iterations leaves it with a value where `abs(z) < 2` is `True`. For this coordinate we can iterate 300 times, and still the test will be `True`. We cannot tell how many iterations we must perform before the condition

becomes False, and this may be an infinite sequence. The maximum iteration (`max_iter`) break clause will stop us from iterating potentially forever.

In [Figure 2-2](#), we see the first 50 iterations of the preceding sequence. For $0+0j$ (the solid line with circle markers), the sequence appears to repeat every eighth iteration, but each sequence of seven calculations has a minor deviation from the previous sequence—we can't tell if this point will iterate forever within the boundary condition, or for a long time, or maybe for just a few more iterations. The dashed cutoff line shows the boundary at +2.

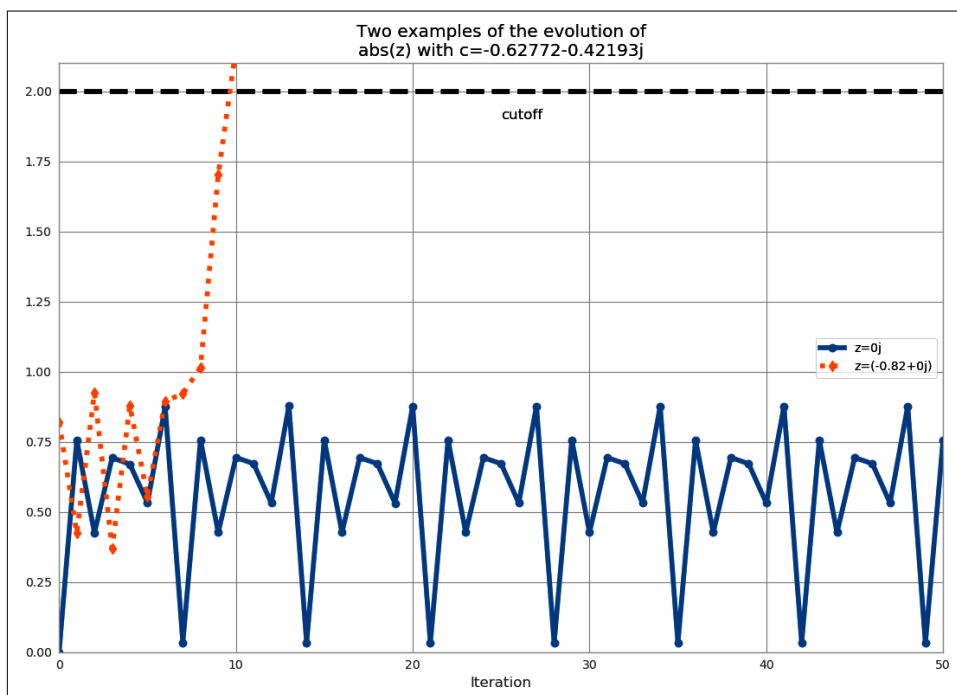


Figure 2-2. Two coordinate examples evolving for the Julia set

For $-0.82+0j$ (the dashed line with diamond markers), we can see that after the ninth update, the absolute result has exceeded the +2 cutoff, so we stop updating this value.

Calculating the Full Julia Set

In this section we break down the code that generates the Julia set. We'll analyze it in various ways throughout this chapter. As shown in [Example 2-1](#), at the start of our module we import the `time` module for our first profiling approach and define some coordinate constants.

Example 2-1. Defining global constants for the coordinate space

```
"""Julia set generator without optional PIL-based image drawing"""
import time

# area of complex space to investigate
x1, x2, y1, y2 = -1.8, 1.8, -1.8, 1.8
c_real, c_imag = -0.62772, -.42193
```

To generate the plot, we create two lists of input data. The first is `zs` (complex z coordinates), and the second is `cs` (a complex initial condition). Neither list varies, and we could optimize `cs` to a single `c` value as a constant. The rationale for building two input lists is so that we have some reasonable-looking data to profile when we profile RAM usage later in this chapter.

To build the `zs` and `cs` lists, we need to know the coordinates for each z . In [Example 2-2](#), we build up these coordinates using `xcoord` and `ycoord` and a specified `x_step` and `y_step`. The somewhat verbose nature of this setup is useful when porting the code to other tools (such as `numpy`) and to other Python environments, as it helps to have everything *very* clearly defined for debugging.

Example 2-2. Establishing the coordinate lists as inputs to our calculation function

```
def calc_pure_python(desired_width, max_iterations):
    """Create a list of complex coordinates (zs) and complex parameters (cs),
    build Julia set"""
    x_step = (x2 - x1) / desired_width
    y_step = (y1 - y2) / desired_width
    x = []
    y = []
    ycoord = y2
    while ycoord > y1:
        y.append(ycoord)
        ycoord += y_step
    xcoord = x1
    while xcoord < x2:
        x.append(xcoord)
        xcoord += x_step
    # build a list of coordinates and the initial condition for each cell.
    # Note that our initial condition is a constant and could easily be removed,
    # we use it to simulate a real-world scenario with several inputs to our
    # function
    zs = []
    cs = []
    for ycoord in y:
        for xcoord in x:
            zs.append(complex(xcoord, ycoord))
            cs.append(complex(c_real, c_imag))
```

```

print("Length of x:", len(x))
print("Total elements:", len(zs))
start_time = time.time()
output = calculate_z_serial_purepython(max_iterations, zs, cs)
end_time = time.time()
secs = end_time - start_time
print(calculate_z_serial_purepython.__name__ + " took", secs, "seconds")

# This sum is expected for a 1000^2 grid with 300 iterations
# It ensures that our code evolves exactly as we'd intended
assert sum(output) == 33219980

```

Having built the `zs` and `cs` lists, we output some information about the size of the lists and calculate the output list via `calculate_z_serial_purepython`. Finally, we sum the contents of `output` and `assert` that it matches the expected output value. Ian uses it here to confirm that no errors creep into the book.

As the code is deterministic, we can verify that the function works as we expect by summing all the calculated values. This is useful as a sanity check—when we make changes to numerical code, it is *very* sensible to check that we haven't broken the algorithm. Ideally, we would use unit tests and test more than one configuration of the problem.

Next, in [Example 2-3](#), we define the `calculate_z_serial_purepython` function, which expands on the algorithm we discussed earlier. Notably, we also define an output list at the start that has the same length as the input `zs` and `cs` lists.

Example 2-3. Our CPU-bound calculation function

```

def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while abs(z) < 2 and n < maxiter:
            z = z * z + c
            n += 1
        output[i] = n
    return output

```

Now we call the calculation routine in [Example 2-4](#). By wrapping it in a `__main__` check, we can safely import the module without starting the calculations for some of the profiling methods. Here, we're not showing the method used to plot the output.

Example 2-4. `__main__` for our code

```
if __name__ == "__main__":  
    # Calculate the Julia set using a pure Python solution with  
    # reasonable defaults for a laptop  
    calc_pure_python(desired_width=1000, max_iterations=300)
```

Once we run the code, we see some output about the complexity of the problem:

```
# running the above produces:  
Length of x: 1000  
Total elements: 1000000  
calculate_z_serial_purepython took 8.087012767791748 seconds
```

In the false-grayscale plot (Figure 2-1), the high-contrast color changes gave us an idea of where the cost of the function was slow changing or fast changing. Here, in Figure 2-3, we have a linear color map: black is quick to calculate, and white is expensive to calculate.

By showing two representations of the same data, we can see that lots of detail is lost in the linear mapping. Sometimes it can be useful to have various representations in mind when investigating the cost of a function.

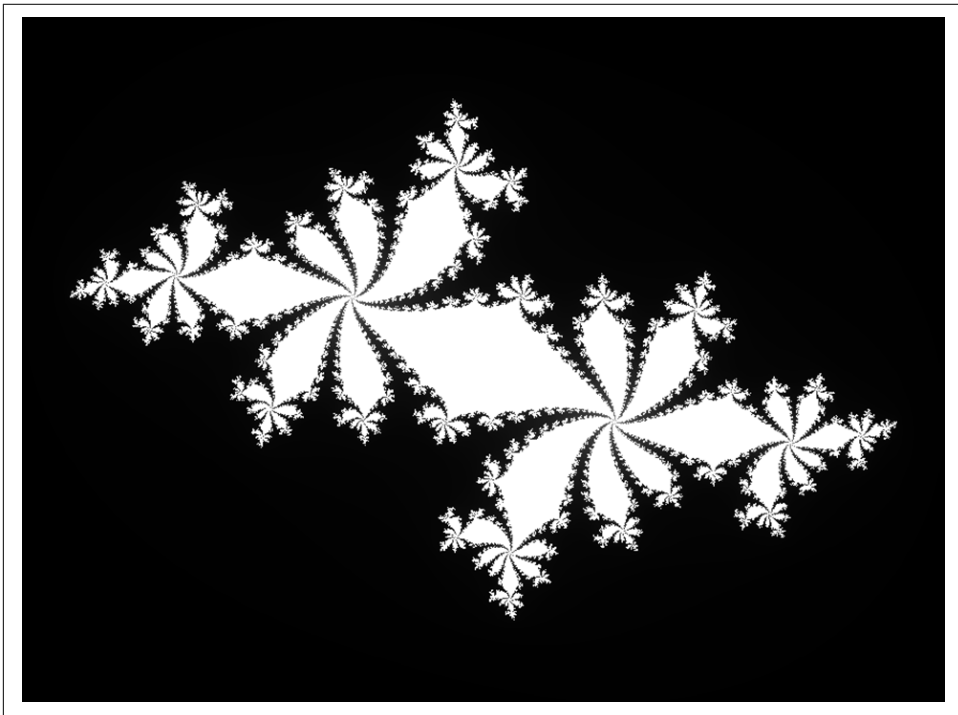


Figure 2-3. Julia plot example using a pure gray scale

Simple Approaches to Timing—print and a Decorator

After [Example 2-4](#), we saw the output generated by several `print` statements in our code. On Ian’s laptop, this code takes approximately 8 seconds to run using CPython 3.7. It is useful to note that execution time always varies. You must observe the normal variation when you’re timing your code, or you might incorrectly attribute an improvement in your code to what is simply a random variation in execution time.

Your computer will be performing other tasks while running your code, such as accessing the network, disk, or RAM, and these factors can cause variations in the execution time of your program.

Ian’s laptop is a Dell 9550 with an Intel Core I7 6700HQ (2.6 GHz, 6 MB cache, Quad Core with Hyperthreading) and 32 GB of RAM running Linux Mint 19.1 (Ubuntu 18.04).

In `calc_pure_python` ([Example 2-2](#)), we can see several `print` statements. This is the simplest way to measure the execution time of a piece of code *inside* a function. It is a basic approach, but despite being quick and dirty, it can be very useful when you’re first looking at a piece of code.

Using `print` statements is commonplace when debugging and profiling code. It quickly becomes unmanageable but is useful for short investigations. Try to tidy up the `print` statements when you’re done with them, or they will clutter your `stdout`.

A slightly cleaner approach is to use a *decorator*—here, we add one line of code above the function that we care about. Our decorator can be very simple and just replicate the effect of the `print` statements. Later, we can make it more advanced.

In [Example 2-5](#), we define a new function, `timefn`, which takes a function as an argument: the inner function, `measure_time`, takes `*args` (a variable number of positional arguments) and `**kwargs` (a variable number of key/value arguments) and passes them through to `fn` for execution. Around the execution of `fn`, we capture `time.time()` and then print the result along with `fn.__name__`. The overhead of using this decorator is small, but if you’re calling `fn` millions of times, the overhead might become noticeable. We use `@wraps(fn)` to expose the function name and docstring to the caller of the decorated function (otherwise, we would see the function name and docstring for the decorator, not the function it decorates).

Example 2-5. Defining a decorator to automate timing measurements

```
from functools import wraps

def timefn(fn):
    @wraps(fn)
    def measure_time(*args, **kwargs):
```

```

t1 = time.time()
result = fn(*args, **kwargs)
t2 = time.time()
print(f"@timefn: {fn.__name__} took {t2 - t1} seconds")
return result
return measure_time

```

```

@timefn
def calculate_z_serial_purepython(maxiter, zs, cs):
    ...

```

When we run this version (we keep the print statements from before), we can see that the execution time in the decorated version is ever-so-slightly quicker than the call from `calc_pure_python`. This is due to the overhead of calling a function (the difference is very tiny):

```

Length of x: 1000
Total elements: 1000000
@timefn:calculate_z_serial_purepython took 8.00485110282898 seconds
calculate_z_serial_purepython took 8.004898071289062 seconds

```



The addition of profiling information will inevitably slow down your code—some profiling options are very informative and induce a heavy speed penalty. The trade-off between profiling detail and speed will be something you have to consider.

We can use the `timeit` module as another way to get a coarse measurement of the execution speed of our CPU-bound function. More typically, you would use this when timing different types of simple expressions as you experiment with ways to solve a problem.



The `timeit` module temporarily disables the garbage collector. This might impact the speed you'll see with real-world operations if the garbage collector would normally be invoked by your operations. See the [Python documentation](#) for help on this.

From the command line, you can run `timeit` as follows:

```

python -m timeit -n 5 -r 1 -s "import julia1" \
    "julia1.calc_pure_python(desired_width=1000, max_iterations=300)"

```

Note that you have to import the module as a setup step using `-s`, as `calc_pure_python` is inside that module. `timeit` has some sensible defaults for short sections of code, but for longer-running functions it can be sensible to specify the number of loops (`-n 5`) and the number of repetitions (`-r 5`) to repeat the experiments. The best result of all the repetitions is given as the answer. Adding the verbose

flag (-v) shows the cumulative time of all the loops by each repetition, which can help your variability in the results.

By default, if we run `timeit` on this function without specifying -n and -r, it runs 10 loops with 5 repetitions, and this takes six minutes to complete. Overriding the defaults can make sense if you want to get your results a little faster.

We're interested only in the best-case results, as other results will probably have been impacted by other processes:

```
5 loops, best of 1: 8.45 sec per loop
```

Try running the benchmark several times to check if you get varying results—you may need more repetitions to settle on a stable fastest-result time. There is no “correct” configuration, so if you see a wide variation in your timing results, do more repetitions until your final result is stable.

Our results show that the overall cost of calling `calc_pure_python` is 8.45 seconds (as the best case), while single calls to `calculate_z_serial_purepython` take 8.0 seconds as measured by the `@timefn` decorator. The difference is mainly the time taken to create the `zs` and `cs` lists.

Inside IPython, we can use the magic `%timeit` in the same way. If you are developing your code interactively in IPython or in a Jupyter Notebook, you can use this:

```
In [1]: import julia1
In [2]: %timeit julia1.calc_pure_python(desired_width=1000, max_iterations=300)
```



Be aware that “best” is calculated differently by the `timeit.py` approach and the `%timeit` approach in Jupyter and IPython. `timeit.py` uses the minimum value seen. IPython in 2016 switched to using the mean and standard deviation. Both methods have their flaws, but generally they're both “reasonably good”; you can't compare between them, though. Use one method or the other; don't mix them.

It is worth considering the variation in load that you get on a normal computer. Many background tasks are running (e.g., Dropbox, backups) that could impact the CPU and disk resources at random. Scripts in web pages can also cause unpredictable resource usage. [Figure 2-4](#) shows the single CPU being used at 100% for some of the timing steps we just performed; the other cores on this machine are each lightly working on other tasks.

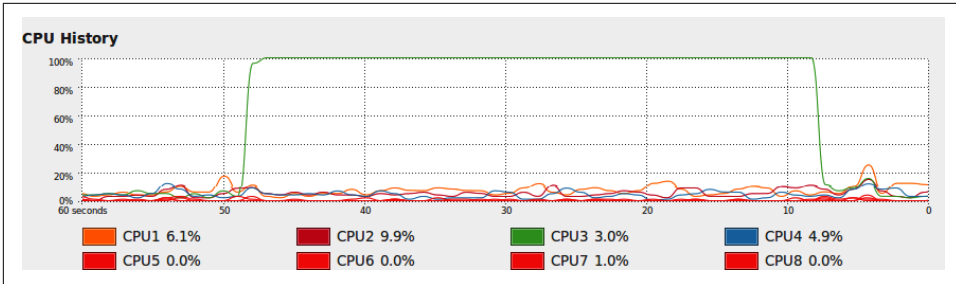


Figure 2-4. System Monitor on Ubuntu showing variation in background CPU usage while we time our function

Occasionally, the System Monitor shows spikes of activity on this machine. It is sensible to watch your System Monitor to check that nothing else is interfering with your critical resources (CPU, disk, network).

Simple Timing Using the Unix time Command

We can step outside of Python for a moment to use a standard system utility on Unix-like systems. The following will record various views on the execution time of your program, and it won't care about the internal structure of your code:

```
$ /usr/bin/time -p python julia1_nopil.py
Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 8.279886722564697 seconds
real 8.84
user 8.73
sys 0.10
```

Note that we specifically use `/usr/bin/time` rather than `time` so we get the system's `time` and not the simpler (and less useful) version built into our shell. If you try `time --verbose quick-and-dirty` get an error, you're probably looking at the shell's built-in `time` command and not the system command.

Using the `-p` portability flag, we get three results:

- `real` records the wall clock or elapsed time.
- `user` records the amount of time the CPU spent on your task outside of kernel functions.
- `sys` records the time spent in kernel-level functions.

By adding `user` and `sys`, you get a sense of how much time was spent in the CPU. The difference between this and `real` might tell you about the amount of time spent

waiting for I/O; it might also suggest that your system is busy running other tasks that are distorting your measurements.

`time` is useful because it isn't specific to Python. It includes the time taken to start the python executable, which might be significant if you start lots of fresh processes (rather than having a long-running single process). If you often have short-running scripts where the startup time is a significant part of the overall runtime, then `time` can be a more useful measure.

We can add the `--verbose` flag to get even more output:

```
$ /usr/bin/time --verbose python julia1_nopil.py
Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 8.477287530899048 seconds
Command being timed: "python julia1_nopil.py"
User time (seconds): 8.97
System time (seconds): 0.05
Percent of CPU this job got: 99%
Elapsed (wall clock) time (h:mm:ss or m:ss): 0:09.03
Average shared text size (kbytes): 0
Average unshared data size (kbytes): 0
Average stack size (kbytes): 0
Average total size (kbytes): 0
Maximum resident set size (kbytes): 98620
Average resident set size (kbytes): 0
Major (requiring I/O) page faults: 0
Minor (reclaiming a frame) page faults: 26645
Voluntary context switches: 1
Involuntary context switches: 27
Swaps: 0
File system inputs: 0
File system outputs: 0
Socket messages sent: 0
Socket messages received: 0
Signals delivered: 0
Page size (bytes): 4096
Exit status: 0
```

Probably the most useful indicator here is Major (requiring I/O) page faults, as this indicates whether the operating system is having to load pages of data from the disk because the data no longer resides in RAM. This will cause a speed penalty.

In our example, the code and data requirements are small, so no page faults occur. If you have a memory-bound process, or several programs that use variable and large amounts of RAM, you might find that this gives you a clue as to which program is being slowed down by disk accesses at the operating system level because parts of it have been swapped out of RAM to disk.

Using the cProfile Module

cProfile is a built-in profiling tool in the standard library. It hooks into the virtual machine in CPython to measure the time taken to run every function that it sees. This introduces a greater overhead, but you get correspondingly more information. Sometimes the additional information can lead to surprising insights into your code.

cProfile is one of two profilers in the standard library, alongside `profile`. `profile` is the original and slower pure Python profiler; cProfile has the same interface as `profile` and is written in C for a lower overhead. If you're curious about the history of these libraries, see [Armin Rigo's 2005 request](#) to include cProfile in the standard library.

A good practice when profiling is to generate a *hypothesis* about the speed of parts of your code before you profile it. Ian likes to print out the code snippet in question and annotate it. Forming a hypothesis ahead of time means you can measure how wrong you are (and you will be!) and improve your intuition about certain coding styles.



You should never avoid profiling in favor of a gut instinct (we warn you—you *will* get it wrong!). It is definitely worth forming a hypothesis ahead of profiling to help you learn to spot possible slow choices in your code, and you should always back up your choices with evidence.

Always be driven by results that you have measured, and always start with some quick-and-dirty profiling to make sure you're addressing the right area. There's nothing more humbling than cleverly optimizing a section of code only to realize (hours or days later) that you missed the slowest part of the process and haven't really addressed the underlying problem at all.

Let's hypothesize that `calculate_z_serial_purepython` is the slowest part of the code. In that function, we do a lot of dereferencing and make many calls to basic arithmetic operators and the `abs` function. These will probably show up as consumers of CPU resources.

Here, we'll use the cProfile module to run a variant of the code. The output is spartan but helps us figure out where to analyze further.

The `-s cumulative` flag tells cProfile to sort by cumulative time spent inside each function; this gives us a view into the slowest parts of a section of code. The cProfile output is written to screen directly after our usual `print` results:

```
$ python -m cProfile -s cumulative julia1_nopil.py
...
Length of x: 1000
Total elements: 1000000
```

```
calculate_z_serial_purepython took 11.498265266418457 seconds
36221995 function calls in 12.234 seconds
```

Ordered by: cumulative time

ncalls	totttime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	12.234	12.234	{built-in method builtins.exec}
1	0.038	0.038	12.234	12.234	julia1_nopil.py:1(<module>)
1	0.571	0.571	12.197	12.197	julia1_nopil.py:23 (calc_pure_python)
1	8.369	8.369	11.498	11.498	julia1_nopil.py:9 (calculate_z_serial_purepython)
34219980	3.129	0.000	3.129	0.000	{built-in method builtins.abs}
2002000	0.121	0.000	0.121	0.000	{method 'append' of 'list' objects}
1	0.006	0.006	0.006	0.006	{built-in method builtins.sum}
3	0.000	0.000	0.000	0.000	{built-in method builtins.print}
2	0.000	0.000	0.000	0.000	{built-in method time.time}
4	0.000	0.000	0.000	0.000	{built-in method builtins.len}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

Sorting by cumulative time gives us an idea about where the majority of execution time is spent. This result shows us that 36,221,995 function calls occurred in just over 12 seconds (this time includes the overhead of using cProfile). Previously, our code took around 8 seconds to execute—we’ve just added a 4-second penalty by measuring how long each function takes to execute.

We can see that the entry point to the code `julia1_nopil.py` on line 1 takes a total of 12 seconds. This is just the `__main__` call to `calc_pure_python`. `ncalls` is 1, indicating that this line is executed only once.

Inside `calc_pure_python`, the call to `calculate_z_serial_purepython` consumes 11 seconds. Both functions are called only once. We can derive that approximately 1 second is spent on lines of code inside `calc_pure_python`, separate to calling the CPU-intensive `calculate_z_serial_purepython` function. However, we can’t derive *which* lines take the time inside the function using cProfile.

Inside `calculate_z_serial_purepython`, the time spent on lines of code (without calling other functions) is 8 seconds. This function makes 34,219,980 calls to `abs`, which take a total of 3 seconds, along with other calls that do not cost much time.

What about the `{abs}` call? This line is measuring the individual calls to the `abs` function inside `calculate_z_serial_purepython`. While the per-call cost is negligible (it is recorded as 0.000 seconds), the total time for 34,219,980 calls is 3 seconds. We couldn’t predict in advance exactly how many calls would be made to `abs`, as the Julia function has unpredictable dynamics (that’s why it is so interesting to look at).

At best we could have said that it will be called a minimum of 1 million times, as we’re calculating 1000×1000 pixels. At most it will be called 300 million times, as we

calculate 1,000,000 pixels with a maximum of 300 iterations. So 34 million calls is roughly 10% of the worst case.

If we look at the original grayscale image (Figure 2-3) and, in our mind's eye, squash the white parts together and into a corner, we can estimate that the expensive white region accounts for roughly 10% of the rest of the image.

The next line in the profiled output, {method 'append' of 'list' objects}, details the creation of 2,002,000 list items.



Why 2,002,000 items? Before you read on, think about how many list items are being constructed.

This creation of 2,002,000 items is occurring in `calc_pure_python` during the setup phase.

The `zs` and `cs` lists will be 1000×1000 items each (generating $1,000,000 \times 2$ calls), and these are built from a list of 1,000 x and 1,000 y coordinates. In total, this is 2,002,000 calls to `append`.

It is important to note that this `cProfile` output is not ordered by parent functions; it is summarizing the expense of all functions in the executed block of code. Figuring out what is happening on a line-by-line basis is very hard with `cProfile`, as we get profile information only for the function calls themselves, not for each line within the functions.

Inside `calculate_z_serial_purepython`, we can account for `{abs}`, and in total this function costs approximately 3.1 seconds. We know that `calculate_z_serial_purepython` costs 11.4 seconds in total.

The final line of the profiling output refers to `lsprof`; this is the original name of the tool that evolved into `cProfile` and can be ignored.

To get more control over the results of `cProfile`, we can write a statistics file and then analyze it in Python:

```
$ python -m cProfile -o profile.stats julia1.py
```

We can load this into Python as follows, and it will give us the same cumulative time report as before:

```
In [1]: import pstats
In [2]: p = pstats.Stats("profile.stats")
In [3]: p.sort_stats("cumulative")
Out[3]: <pstats.Stats at 0x7f77088edf28>
```



```
In [4]: p.print_stats()
Fri Jun 14 17:59:28 2019    profile.stats
```

36221995 function calls in 12.169 seconds

Ordered by: cumulative time

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.000	0.000	12.169	12.169	{built-in method builtins.exec}
1	0.033	0.033	12.169	12.169	julia1_nopil.py:1(<module>)
1	0.576	0.576	12.135	12.135	julia1_nopil.py:23 (calc_pure_python)
1	8.266	8.266	11.429	11.429	julia1_nopil.py:9 (calculate_z_serial_purepython)
34219980	3.163	0.000	3.163	0.000	{built-in method builtins.abs}
2002000	0.123	0.000	0.123	0.000	{method 'append' of 'list' objects}
1	0.006	0.006	0.006	0.006	{built-in method builtins.sum}
3	0.000	0.000	0.000	0.000	{built-in method builtins.print}
4	0.000	0.000	0.000	0.000	{built-in method builtins.len}
2	0.000	0.000	0.000	0.000	{built-in method time.time}
1	0.000	0.000	0.000	0.000	{method 'disable' of '_lsprof.Profiler' objects}

To trace which functions we're profiling, we can print the caller information. In the following two listings we can see that `calculate_z_serial_purepython` is the most expensive function, and it is called from one place. If it were called from many places, these listings might help us narrow down the locations of the most expensive parents:

```
In [5]: p.print_callers()
Ordered by: cumulative time
```

Function		was called by...
		ncalls tottime cumtime
{built-in method builtins.exec}	<-	
julia1_nopil.py:1(<module>)	<-	1 0.033 12.169 {built-in method builtins.exec}
julia1_nopil.py:23(calc_pure_python)	<-	1 0.576 12.135 :1(<module>)
julia1_nopil.py:9(...)	<-	1 8.266 11.429 :23(calc_pure_python)
{built-in method builtins.abs}	<-	34219980 3.163 3.163 :9(calculate_z_serial_purepython)
{method 'append' of 'list' objects}	<-	2002000 0.123 0.123 :23(calc_pure_python)
{built-in method builtins.sum}	<-	1 0.006 0.006 :23(calc_pure_python)
{built-in method builtins.print}	<-	3 0.000 0.000 :23(calc_pure_python)
{built-in method builtins.len}	<-	2 0.000 0.000 :9(calculate_z_serial_purepython)
		2 0.000 0.000 :23(calc_pure_python)

```
{built-in method time.time}          <-      2      0.000      0.000
                                         :23(calc_pure_python)
```

We can flip this around the other way to show which functions call other functions:

```
In [6]: p.print_callees()
        Ordered by: cumulative time

Function                                called...
{built-in method builtins.exec}        ->      1      0.033      12.169
                                         julia1_nopil.py:1(<module>)
julia1_nopil.py:1(<module>)             ->      1      0.576      12.135
                                         julia1_nopil.py:23
                                         (calc_pure_python)
julia1_nopil.py:23(calc_pure_python)    ->      1      8.266      11.429
                                         julia1_nopil.py:9
                                         (calculate_z_serial_purepython)
                                         2      0.000      0.000
                                         {built-in method builtins.len}
                                         3      0.000      0.000
                                         {built-in method builtins.print}
                                         1      0.006      0.006
                                         {built-in method builtins.sum}
                                         2      0.000      0.000
                                         {built-in method time.time}
                                         2002000      0.123      0.123
                                         {method 'append' of 'list' objects}
julia1_nopil.py:9(...)                  -> 34219980      3.163      3.163
                                         {built-in method builtins.abs}
                                         {built-in method builtins.len}
```

cProfile is rather verbose, and you need a side screen to see it without lots of word wrapping. Since it is built in, though, it is a convenient tool for quickly identifying bottlenecks. Tools like `line_profiler` and `memory_profiler`, which we discuss later in this chapter, will then help you to drill down to the specific lines that you should pay attention to.

Visualizing cProfile Output with SnakeViz

`snakeviz` is a visualizer that draws the output of cProfile as a diagram in which larger boxes are areas of code that take longer to run. It replaces the older `runsake` tool.

Use `snakeviz` to get a high-level understanding of a cProfile statistics file, particularly if you're investigating a new project for which you have little intuition. The diagram will help you visualize the CPU-usage behavior of the system, and it may highlight areas that you hadn't expected to be expensive.

To install SnakeViz, use `$ pip install snakeviz`.

In [Figure 2-5](#) we have the visual output of the *profile.stats* file we’ve just generated. The entry point for the program is shown at the top of the diagram. Each layer down is a function called from the function above.

The width of the diagram represents the entire time taken by the program’s execution. The fourth layer shows that the majority of the time is spent in `calculate_z_serial_purepython`. The fifth layer breaks this down some more—the unannotated block to the right occupying approximately 25% of that layer represents the time spent in the `abs` function. Seeing these larger blocks quickly brings home how the time is spent inside your program.

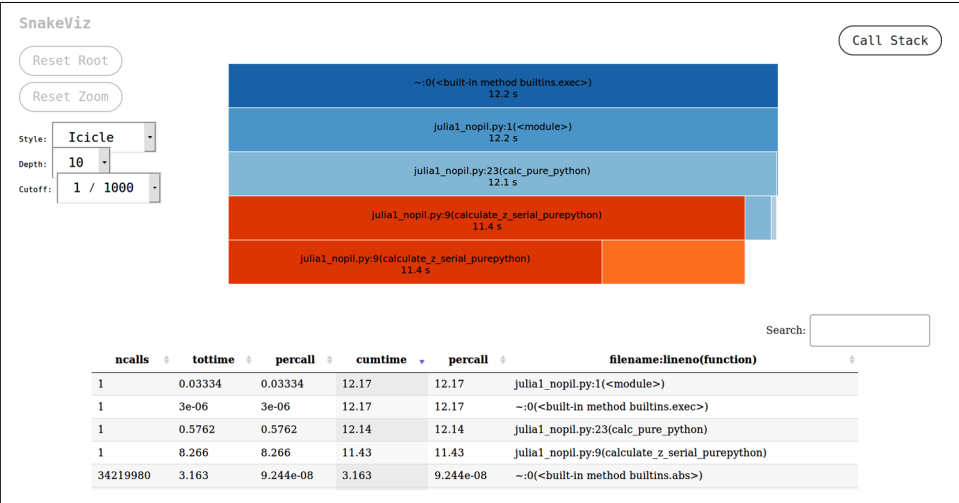


Figure 2-5. *snakeviz* visualizing *profile.stats*

The next section down shows a table that is a pretty-printed version of the statistics we’ve just been looking at, which you can sort by `cumtime` (cumulative time), `percall` (cost per call), or `ncalls` (number of calls altogether), among other categories. Starting with `cumtime` will tell you which functions cost the most overall. They’re a pretty good place to start your investigations.

If you’re comfortable looking at tables, the console output for `cProfile` may be adequate for you. To communicate to others, we strongly suggest you use diagrams—such as this output from *snakeviz*—to help others quickly understand the point you’re making.

Using `line_profiler` for Line-by-Line Measurements

In Ian’s opinion, Robert Kern’s `line_profiler` is the strongest tool for identifying the cause of CPU-bound problems in Python code. It works by profiling individual

functions on a line-by-line basis, so you should start with `cProfile` and use the high-level view to guide which functions to profile with `line_profiler`.

It is worthwhile printing and annotating versions of the output from this tool as you modify your code, so you have a record of changes (successful or not) that you can quickly refer to. Don't rely on your memory when you're working on line-by-line changes.

To install `line_profiler`, issue the command `pip install line_profiler`.

A decorator (`@profile`) is used to mark the chosen function. The `kernprof` script is used to execute your code, and the CPU time and other statistics for each line of the chosen function are recorded.

The arguments are `-l` for line-by-line (rather than function-level) profiling and `-v` for verbose output. Without `-v`, you receive an `.lprof` output that you can later analyze with the `line_profiler` module. In [Example 2-6](#), we'll do a full run on our CPU-bound function.

Example 2-6. Running `kernprof` with line-by-line output on a decorated function to record the CPU cost of each line's execution

```
$ kernprof -l -v julia1_lineprofiler.py
...
Wrote profile results to julia1_lineprofiler.py.lprof
Timer unit: 1e-06 s

Total time: 49.2011 s
File: julia1_lineprofiler.py
Function: calculate_z_serial_purepython at line 9

Line #      Hits   Per Hit   % Time  Line Contents
=====
      9                                @profile
     10                                def calculate_z_serial_purepython(maxiter,
                                     zs, cs):
     11                                """Calculate output list using Julia update rule"""
     12                                output = [0] * len(zs)
     13                                for i in range(len(zs)):
     14                                    n = 0
     15                                    z = zs[i]
     16                                    c = cs[i]
     17                                    while abs(z) < 2 and n < maxiter:
     18                                        z = z * z + c
     19                                        n += 1
     20                                    output[i] = n
     21                                return output
```

Introducing `kernprof.py` adds a substantial amount to the runtime. In this example, `calculate_z_serial_purepython` takes 49 seconds; this is up from 8 seconds using simple `print` statements and 12 seconds using `cProfile`. The gain is that we get a line-by-line breakdown of where the time is spent inside the function.

The % Time column is the most helpful—we can see that 38% of the time is spent on the `while` testing. We don't know whether the first statement (`abs(z) < 2`) is more expensive than the second (`n < maxiter`), though. Inside the loop, we see that the update to `z` is also fairly expensive. Even `n += 1` is expensive! Python's dynamic lookup machinery is at work for every loop, even though we're using the same types for each variable in each loop—this is where compiling and type specialization (Chapter 7) give us a massive win. The creation of the output list and the updates on line 20 are relatively cheap compared to the cost of the `while` loop.

If you haven't thought about the complexity of Python's dynamic machinery before, do think about what happens in that `n += 1` operation. Python has to check that the `n` object has an `__add__` function (and if it didn't, it'd walk up any inherited classes to see if they provided this functionality), and then the other object (1 in this case) is passed in so that the `__add__` function can decide how to handle the operation. Remember that the second argument might be a `float` or other object that may or may not be compatible. This all happens dynamically.

The obvious way to further analyze the `while` statement is to break it up. While there has been some discussion in the Python community around the idea of rewriting the `.pyc` files with more detailed information for multipart, single-line statements, we are unaware of any production tools that offer a more fine-grained analysis than `line_profiler`.

In Example 2-7, we break the `while` logic into several statements. This additional complexity will increase the runtime of the function, as we have more lines of code to execute, but it *might* also help us understand the costs incurred in this part of the code.



Before you look at the code, do you think we'll learn about the costs of the fundamental operations this way? Might other factors complicate the analysis?

Example 2-7. Breaking the compound `while` statement into individual statements to record the cost of each part of the original statement

```
$ kernprof -l -v julia1_lineprofiler2.py
...
Wrote profile results to julia1_lineprofiler2.py.lprof
```

Timer unit: 1e-06 s

Total time: 82.88 s

File: julia1_lineprofiler2.py

Function: calculate_z_serial_purepython at line 9

Line #	Hits	Per Hit	% Time	Line Contents
=====				
9				@profile
10				def calculate_z_serial_purepython(maxiter,
				zs, cs):
11				"""Calculate output list using Julia update rule"""
12	1	3309.0	0.0	output = [0] * len(zs)
13	1000001	0.4	0.5	for i in range(len(zs)):
14	1000000	0.4	0.5	n = 0
15	1000000	0.5	0.5	z = zs[i]
16	1000000	0.4	0.5	c = cs[i]
17	1000000	0.4	0.5	while True:
18	34219980	0.6	23.1	not_yet_escaped = abs(z) < 2
19	34219980	0.4	18.3	iterations_left = n < maxiter
20	34219980	0.4	17.3	if not_yet_escaped and iterations_left:
21	33219980	0.5	20.5	z = z * z + c
22	33219980	0.4	17.3	n += 1
23				else:
24	1000000	0.4	0.5	break
25	1000000	0.4	0.5	output[i] = n
26	1	0.0	0.0	return output

This version takes 82 seconds to execute, while the previous version took 49 seconds. Other factors *did* complicate the analysis. In this case, having extra statements that have to be executed 34,219,980 times each slows down the code. If we hadn't used kernprof.py to investigate the line-by-line effect of this change, we might have drawn other conclusions about the reason for the slowdown, as we'd have lacked the necessary evidence.

At this point it makes sense to step back to the earlier `timeit` technique to test the cost of individual expressions:

```
Python 3.7.3 (default, Mar 27 2019, 22:11:17)
Type 'copyright', 'credits', or 'license' for more information
IPython 7.5.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: z = 0+0j
In [2]: %timeit abs(z) < 2
97.6 ns ± 0.138 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
In [3]: n = 1
In [4]: maxiter = 300
In [5]: %timeit n < maxiter
42.1 ns ± 0.0355 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
```

From this simple analysis, it looks as though the logic test on `n` is more than two times faster than the call to `abs`. Since the order of evaluation for Python statements is both left to right and opportunistic, it makes sense to put the cheapest test on the left side of the equation. On 1 in every 301 tests for each coordinate, the `n < maxiter` test will be `False`, so Python wouldn't need to evaluate the other side of the `and` operator.

We never know whether `abs(z) < 2` will be `False` until we evaluate it, and our earlier observations for this region of the complex plane suggest it is `True` around 10% of the time for all 300 iterations. If we wanted to have a strong understanding of the time complexity of this part of the code, it would make sense to continue the numerical analysis. In this situation, however, we want an easy check to see if we can get a quick win.

We can form a new hypothesis stating, “By swapping the order of the operators in the `while` statement, we will achieve a reliable speedup.” We *can* test this hypothesis using `kernprof`, but the additional overheads of profiling this way might add too much noise. Instead, we can use an earlier version of the code, running a test comparing `while abs(z) < 2 and n < maxiter:` against `while n < maxiter and abs(z) < 2:`, which we see in [Example 2-8](#).

Running the two variants *outside* of `line_profiler` means they run at similar speeds. The overheads of `line_profiler` also confuse the result, and the results on line 17 for both versions are similar. We should reject the hypothesis that in Python 3.7 changing the order of the logic results in a consistent speedup—there's no clear evidence for this. Ian notes that with Python 2.7 we *could* accept this hypothesis, but with Python 3.7 that's no longer the case.

Using a more suitable approach to solve this problem (e.g., swapping to using Cython or PyPy, as described in [Chapter 7](#)) would yield greater gains.

We can be confident in our result because of the following:

- We stated a hypothesis that was easy to test.
- We changed our code so that only the hypothesis would be tested (never test two things at once!).
- We gathered enough evidence to support our conclusion.

For completeness, we can run a final `kernprof` on the two main functions including our optimization to confirm that we have a full picture of the overall complexity of our code.

Example 2-8. Swapping the order of the compound while statement makes the function fractionally faster

```
$ kernprof -l -v julia1_lineprofiler3.py
...
Wrote profile results to julia1_lineprofiler3.py.lprof
Timer unit: 1e-06 s

Total time: 48.9154 s
File: julia1_lineprofiler3.py
Function: calculate_z_serial_purepython at line 9
```

Line #	Hits	Per Hit	% Time	Line Contents
9				@profile
10				def calculate_z_serial_purepython(maxiter,
				zs, cs):
11				"""Calculate output list using Julia update rule"""
12	1	3312.0	0.0	output = [0] * len(zs)
13	1000001	0.4	0.8	for i in range(len(zs)):
14	1000000	0.4	0.7	n = 0
15	1000000	0.4	0.8	z = zs[i]
16	1000000	0.4	0.8	c = cs[i]
17	34219980	0.5	38.2	while n < maxiter and abs(z) < 2:
18	33219980	0.5	30.7	z = z * z + c
19	33219980	0.4	27.1	n += 1
20	1000000	0.4	0.8	output[i] = n
21	1	1.0	0.0	return output

As expected, we can see from the output in [Example 2-9](#) that `calculate_z_serial_purepython` takes most (97%) of the time of its parent function. The list-creation steps are minor in comparison.

Example 2-9. Testing the line-by-line costs of the setup routine

```
Total time: 88.334 s
File: julia1_lineprofiler3.py
Function: calc_pure_python at line 24
```

Line #	Hits	Per Hit	% Time	Line Contents
24				@profile
25				def calc_pure_python(draw_output,
				desired_width,
				max_iterations):
26				"""Create a list of complex...
...				
44	1	1.0	0.0	zs = []
45	1	0.0	0.0	cs = []
46	1001	0.7	0.0	for ycoord in y:
47	1001000	0.6	0.7	for xcoord in x:


```

48 1000000      0.9      1.0      zs.append(complex(xcoord, ycoord))
49 1000000      0.9      1.0      cs.append(complex(c_real, c_imag))
50
51      1      40.0      0.0      print("Length of x:", len(x))
52      1      7.0      0.0      print("Total elements:", len(zs))
53      1      4.0      0.0      start_time = time.time()
54      1 85969310.0 97.3      output = calculate_z_serial_purepython \
                                (max_iterations, zs, cs)
55      1      4.0      0.0      end_time = time.time()
56      1      1.0      0.0      secs = end_time - start_time
57      1      36.0      0.0      print(calculate_z_serial...
58
59      1      6345.0      0.0      assert sum(output) == 33219980

```

`line_profiler` gives us a great insight into the cost of lines inside loops and expensive functions; even though profiling adds a speed penalty, it is a great boon to scientific developers. Remember to use representative data to make sure you're focusing on the lines of code that'll give you the biggest win.

Using `memory_profiler` to Diagnose Memory Usage

Just as Robert Kern's `line_profiler` package measures CPU usage, the `memory_profiler` module by Fabian Pedregosa and Philippe Gervais measures memory usage on a line-by-line basis. Understanding the memory usage characteristics of your code allows you to ask yourself two questions:

- Could we use *less* RAM by rewriting this function to work more efficiently?
- Could we use *more* RAM and save CPU cycles by caching?

`memory_profiler` operates in a very similar way to `line_profiler` but runs far more slowly. If you install the `psutil` package (optional but recommended), `memory_profiler` will run faster. Memory profiling may easily make your code run 10 to 100 times slower. In practice, you will probably use `memory_profiler` occasionally and `line_profiler` (for CPU profiling) more frequently.

Install `memory_profiler` with the command `pip install memory_profiler` (and optionally with `pip install psutil`).

As mentioned, the implementation of `memory_profiler` is not as performant as the implementation of `line_profiler`. It may therefore make sense to run your tests on a smaller problem that completes in a useful amount of time. Overnight runs might be sensible for validation, but you need quick and reasonable iterations to diagnose problems and hypothesize solutions. The code in [Example 2-10](#) uses the full $1,000 \times 1,000$ grid, and the statistics took about two hours to collect on Ian's laptop.



The requirement to modify the source code is a minor annoyance. As with `line_profiler`, a decorator (`@profile`) is used to mark the chosen function. This will break your unit tests unless you make a dummy decorator—see “[No-op @profile Decorator](#)” on [page 60](#).

When dealing with memory allocation, you must be aware that the situation is not as clear-cut as it is with CPU usage. Generally, it is more efficient to overallocate memory in a process that can be used at leisure, as memory allocation operations are relatively expensive. Furthermore, garbage collection is not instantaneous, so objects may be unavailable but still in the garbage collection pool for some time.

The outcome of this is that it is hard to really understand what is happening with memory usage and release inside a Python program, as a line of code may not allocate a deterministic amount of memory *as observed from outside the process*. Observing the gross trend over a set of lines is likely to lead to better insight than would be gained by observing the behavior of just one line.

Let’s take a look at the output from `memory_profiler` in [Example 2-10](#). Inside `calculate_z_serial_purepython` on line 12, we see that the allocation of 1,000,000 items causes approximately 7 MB of RAM to be added to this process.¹ This does not mean that the output list is definitely 7 MB in size, just that the process grew by approximately 7 MB during the internal allocation of the list.

In the parent function on line 46, we see that the allocation of the `zs` and `cs` lists changes the Mem usage column from 48 MB to 125 MB (a change of +77 MB). Again, it is worth noting that this is not necessarily the true size of the arrays, just the size that the process grew by after these lists had been created.

At the time of writing, the `memory_usage` module exhibits a bug—the Increment column does not always match the change in the Mem usage column. During the first edition of this book, these columns were correctly tracked; you might want to check the status of this bug on [GitHub](#). We recommend you use the Mem usage column, as this correctly tracks the change in process size per line of code.

¹ `memory_profiler` measures memory usage according to the International Electrotechnical Commission’s MiB (mebibyte) of 2^{20} bytes. This is slightly different from the more common but also more ambiguous MB (megabyte has two commonly accepted definitions!). 1 MiB is equal to 1.048576 (or approximately 1.05) MB. For our discussion, unless we’re dealing with very specific amounts, we’ll consider the two equivalent.

Example 2-10. memory_profiler's result on both of our main functions, showing an unexpected memory use in calculate_z_serial_purepython

```
$ python -m memory_profiler julia1_memoryprofiler.py
...
```

Line #	Mem usage	Increment	Line Contents
9	126.363 MiB	126.363 MiB	@profile
10			def calculate_z_serial_purepython(maxiter,
			zs, cs):
11			"""Calculate output list using...
12	133.973 MiB	7.609 MiB	output = [0] * len(zs)
13	136.988 MiB	0.000 MiB	for i in range(len(zs)):
14	136.988 MiB	0.000 MiB	n = 0
15	136.988 MiB	0.000 MiB	z = zs[i]
16	136.988 MiB	0.000 MiB	c = cs[i]
17	136.988 MiB	0.258 MiB	while n < maxiter and abs(z) < 2:
18	136.988 MiB	0.000 MiB	z = z * z + c
19	136.988 MiB	0.000 MiB	n += 1
20	136.988 MiB	0.000 MiB	output[i] = n
21	136.988 MiB	0.000 MiB	return output

...

Line #	Mem usage	Increment	Line Contents
24	48.113 MiB	48.113 MiB	@profile
25			def calc_pure_python(draw_output,
			desired_width,
			max_iterations):
26			"""Create a list of complex...
27	48.113 MiB	0.000 MiB	x_step = (x2 - x1) / desired_width
28	48.113 MiB	0.000 MiB	y_step = (y1 - y2) / desired_width
29	48.113 MiB	0.000 MiB	x = []
30	48.113 MiB	0.000 MiB	y = []
31	48.113 MiB	0.000 MiB	ycoord = y2
32	48.113 MiB	0.000 MiB	while ycoord > y1:
33	48.113 MiB	0.000 MiB	y.append(ycoord)
34	48.113 MiB	0.000 MiB	ycoord += y_step
35	48.113 MiB	0.000 MiB	xcoord = x1
36	48.113 MiB	0.000 MiB	while xcoord < x2:
37	48.113 MiB	0.000 MiB	x.append(xcoord)
38	48.113 MiB	0.000 MiB	xcoord += x_step
44	48.113 MiB	0.000 MiB	zs = []
45	48.113 MiB	0.000 MiB	cs = []
46	125.961 MiB	0.000 MiB	for ycoord in y:
47	125.961 MiB	0.258 MiB	for xcoord in x:
48	125.961 MiB	0.512 MiB	zs.append(complex(xcoord, ycoord))
49	125.961 MiB	0.512 MiB	cs.append(complex(c_real, c_imag))
50			
51	125.961 MiB	0.000 MiB	print("Length of x:", len(x))

```

52 125.961 MiB    0.000 MiB    print("Total elements:", len(zs))
53 125.961 MiB    0.000 MiB    start_time = time.time()
54 136.609 MiB   10.648 MiB    output = calculate_z_serial...
55 136.609 MiB    0.000 MiB    end_time = time.time()
56 136.609 MiB    0.000 MiB    secs = end_time - start_time
57 136.609 MiB    0.000 MiB    print(calculate_z_serial_purepython...
58
59 136.609 MiB    0.000 MiB    assert sum(output) == 33219980

```

Another way to visualize the change in memory use is to sample over time and plot the result. `memory_profiler` has a utility called `mprof`, used once to sample the memory usage and a second time to visualize the samples. It samples by time and not by line, so it barely impacts the runtime of the code.

Figure 2-6 is created using `mprof run julia1_memoryprofiler.py`. This writes a statistics file that is then visualized using `mprof plot`. Our two functions are bracketed: this shows where in time they are entered, and we can see the growth in RAM as they run. Inside `calculate_z_serial_purepython`, we can see the steady increase in RAM usage throughout the execution of the function; this is caused by all the small objects (`int` and `float` types) that are created.

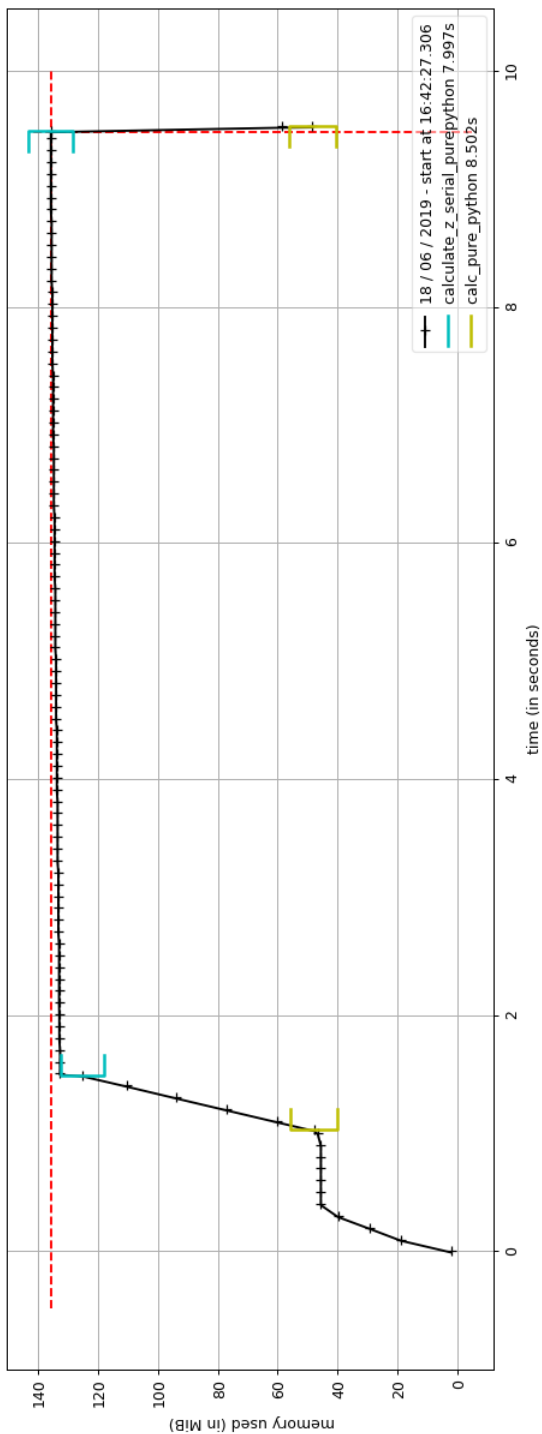


Figure 2-6. memory_profiler report using mprof

In addition to observing the behavior at the function level, we can add labels using a context manager. The snippet in [Example 2-11](#) is used to generate the graph in [Figure 2-7](#). We can see the `create_output_list` label: it appears momentarily at around 1.5 seconds after `calculate_z_serial_purepython` and results in the process being allocated more RAM. We then pause for a second; `time.sleep(1)` is an artificial addition to make the graph easier to understand.

Example 2-11. Using a context manager to add labels to the `mprof` graph

```
@profile
def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    with profile.timestamp("create_output_list"):
        output = [0] * len(zs)
        time.sleep(1)
    with profile.timestamp("calculate_output"):
        for i in range(len(zs)):
            n = 0
            z = zs[i]
            c = cs[i]
            while n < maxiter and abs(z) < 2:
                z = z * z + c
                n += 1
            output[i] = n
    return output
```

In the `calculate_output` block that runs for most of the graph, we see a very slow, linear increase in RAM usage. This will be from all of the temporary numbers used in the inner loops. Using the labels really helps us to understand at a fine-grained level where memory is being consumed. Interestingly, we see the “peak RAM usage” line—a dashed vertical line just before the 10-second mark—occurring before the termination of the program. Potentially this is due to the garbage collector recovering some RAM from the temporary objects used during `calculate_output`.

What happens if we simplify our code and remove the creation of the `zs` and `cs` lists? We then have to calculate these coordinates inside `calculate_z_serial_purepython` (so the same work is performed), but we’ll save RAM by not storing them in lists. You can see the code in [Example 2-12](#).

In [Figure 2-8](#), we see a major change in behavior—the overall envelope of RAM usage drops from 140 MB to 60 MB, reducing our RAM usage by half!

/home/ian/anaconda3/envs/high_performance_python_book_2e/bin/python julia1_memoryprofiler_with_labels.py

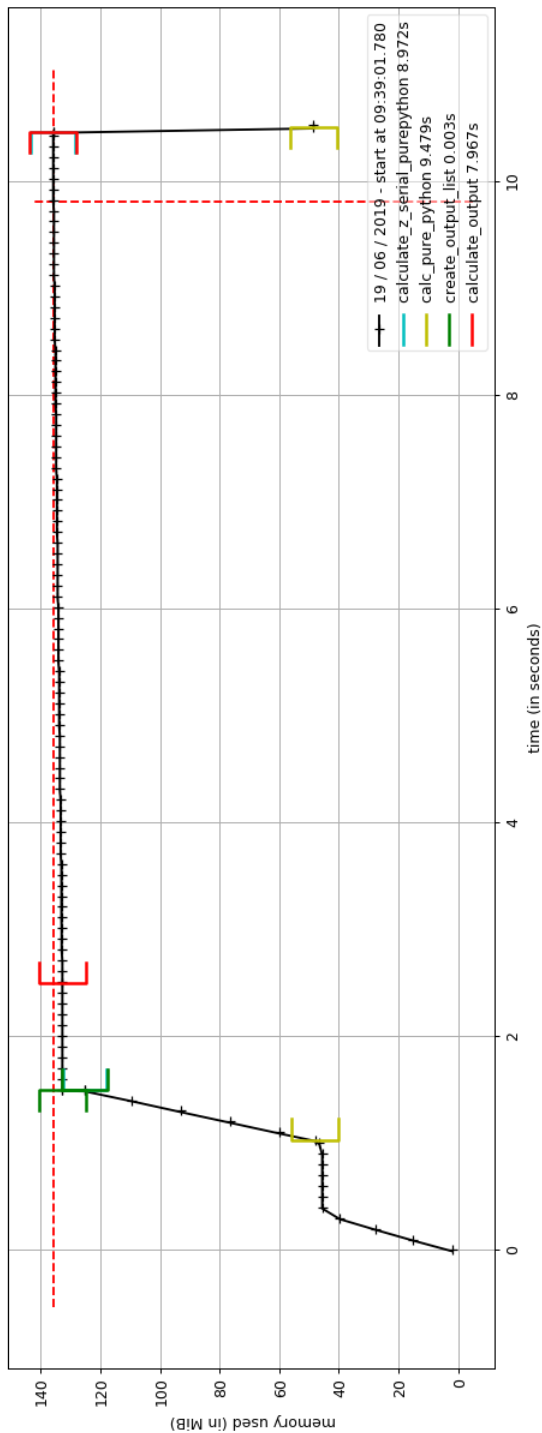


Figure 2-7. memory_profiler report using mprof with labels

/home/ian/anaconda3/envs/high_performance_python_book_2e/bin/python julia1_memoryprofiler2.py

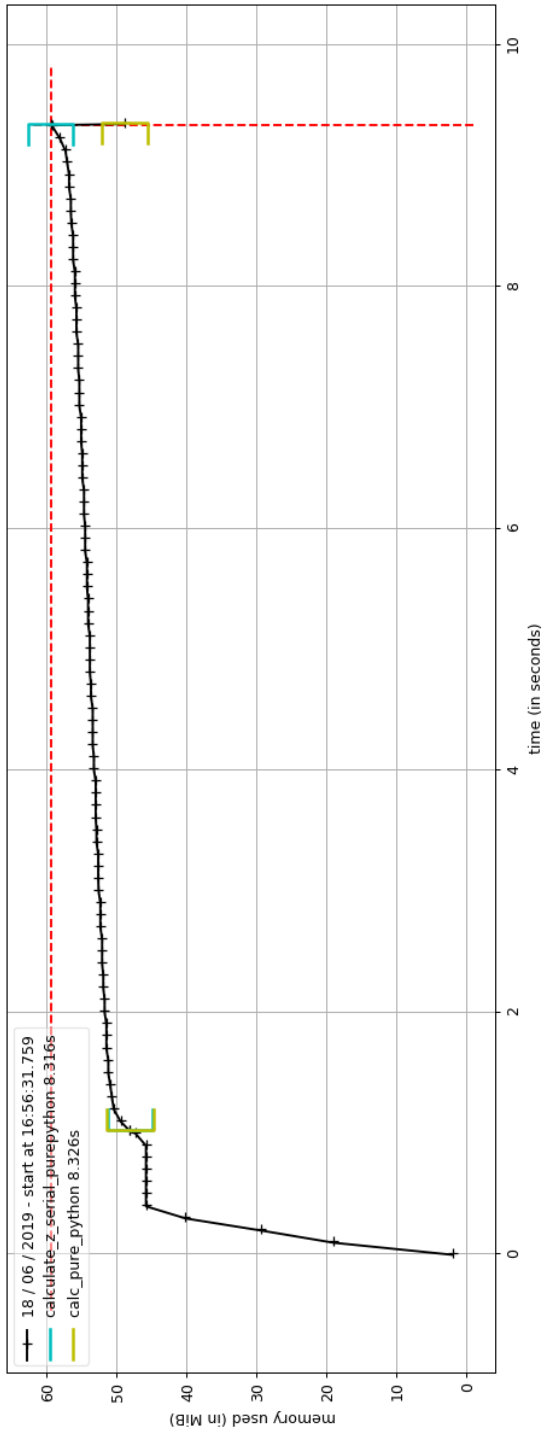


Figure 2-8. memory_profiler after removing two large lists

Example 2-12. Creating complex coordinates on the fly to save RAM

```
@profile
def calculate_z_serial_purepython(maxiter, x, y):
    """Calculate output list using Julia update rule"""
    output = []
    for ycoord in y:
        for xcoord in x:
            z = complex(xcoord, ycoord)
            c = complex(c_real, c_imag)
            n = 0
            while n < maxiter and abs(z) < 2:
                z = z * z + c
                n += 1
            output.append(n)
    return output
```

If we want to measure the RAM used by several statements, we can use the IPython magic `%memit`, which works just like `%timeit`. In [Chapter 11](#), we will look at using `%memit` to measure the memory cost of lists and discuss various ways of using RAM more efficiently.

`memory_profiler` offers an interesting aid to debugging a large process via the `--pdb-mmem=XXX` flag. The `pdb` debugger will be activated after the process exceeds `XXX` MB. This will drop you in directly at the point in your code where too many allocations are occurring, if you're in a space-constrained environment.

Introspecting an Existing Process with PySpy

`py-spy` is an intriguing new sampling profiler—rather than requiring any code changes, it introspects an already-running Python process and reports in the console with a top-like display. Being a sampling profiler, it has almost no runtime impact on your code. It is written in Rust and requires elevated privileges to introspect another process.

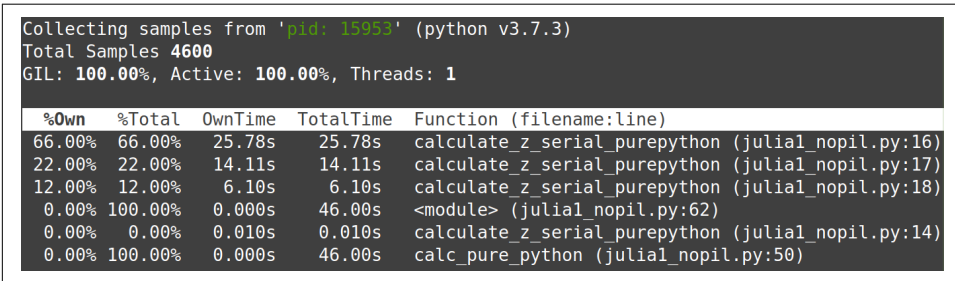
This tool could be very useful in a production environment with long-running processes or complicated installation requirements. It supports Windows, Mac, and Linux. Install it using `pip install py-spy` (note the dash in the name—there's a separate `pyspy` project that isn't related). If your process is already running, you'll want to use `ps` to get its process identifier (the PID); then this can be passed into `py-spy` as shown in [Example 2-13](#).

Example 2-13. Running PySpy at the command line

```
$ ps -A -o pid,rss,cmd | ack python
...
15953 96156 python julia1_nopil.py
```

```
...
$ sudo env "PATH=$PATH" py-spy --pid 15953
```

In [Figure 2-9](#), you'll see a static picture of a top-like display in the console; this updates every second to show which functions are currently taking most of the time.



```
Collecting samples from 'pid: 15953' (python v3.7.3)
Total Samples 4600
GIL: 100.00%, Active: 100.00%, Threads: 1
```

%Own	%Total	OwnTime	TotalTime	Function (filename:line)
66.00%	66.00%	25.78s	25.78s	calculate_z_serial_purepython (julia1_nopil.py:16)
22.00%	22.00%	14.11s	14.11s	calculate_z_serial_purepython (julia1_nopil.py:17)
12.00%	12.00%	6.10s	6.10s	calculate_z_serial_purepython (julia1_nopil.py:18)
0.00%	100.00%	0.000s	46.00s	<module> (julia1_nopil.py:62)
0.00%	0.00%	0.010s	0.010s	calculate_z_serial_purepython (julia1_nopil.py:14)
0.00%	100.00%	0.000s	46.00s	calc_pure_python (julia1_nopil.py:50)

Figure 2-9. Introspecting a Python process using PySpy

PySpy can also export a flame chart. Here, we'll run that option while asking PySpy to run our code directly without requiring a PID using `$ py-spy --flame profile.svg -- python julia1_nopil.py`. You'll see in [Figure 2-10](#) that the width of the display represents the entire program's runtime, and each layer moving down the image represents functions called from above.

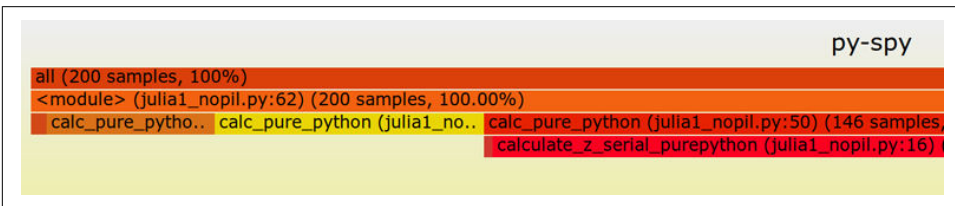


Figure 2-10. Part of a flame chart for PySpy

Bytecode: Under the Hood

So far we've reviewed various ways to measure the cost of Python code (for both CPU and RAM usage). We haven't yet looked at the underlying bytecode used by the virtual machine, though. Understanding what's going on "under the hood" helps to build a mental model of what's happening in slow functions, and it'll help when you come to compile your code. So let's introduce some bytecode.

Using the dis Module to Examine CPython Bytecode

The `dis` module lets us inspect the underlying bytecode that we run inside the stack-based CPython virtual machine. Having an understanding of what's happening in the virtual machine that runs your higher-level Python code will help you to understand

why some styles of coding are faster than others. It will also help when you come to use a tool like Cython, which steps outside of Python and generates C code.

The `dis` module is built in. You can pass it code or a module, and it will print out a disassembly. In [Example 2-14](#), we disassemble the outer loop of our CPU-bound function.



You should try to disassemble one of your own functions and to follow *exactly* how the disassembled code matches to the disassembled output. Can you match the following `dis` output to the original function?

Example 2-14. Using the built-in `dis` to understand the underlying stack-based virtual machine that runs our Python code

```
In [1]: import dis
In [2]: import julia1_nopil
In [3]: dis.dis(julia1_nopil.calculate_z_serial_purepython)
11      0 LOAD_CONST                1 (0)
        2 BUILD_LIST                  1
        4 LOAD_GLOBAL                0 (len)
        6 LOAD_FAST                  1 (zs)
        8 CALL_FUNCTION              1
       10 BINARY_MULTIPLY
       12 STORE_FAST                3 (output)

12      14 SETUP_LOOP              94 (to 110)
       16 LOAD_GLOBAL                1 (range)
       18 LOAD_GLOBAL                0 (len)
       20 LOAD_FAST                  1 (zs)
       22 CALL_FUNCTION              1
       24 CALL_FUNCTION              1
       26 GET_ITER
    >>  28 FOR_ITER                  78 (to 108)
       30 STORE_FAST                4 (i)

13      32 LOAD_CONST                1 (0)
       34 STORE_FAST                5 (n)
...
19    >>  98 LOAD_FAST              5 (n)
       100 LOAD_FAST              3 (output)
       102 LOAD_FAST              4 (i)
       104 STORE_SUBSCR
       106 JUMP_ABSOLUTE          28
    >>  108 POP_BLOCK

20    >>  110 LOAD_FAST              3 (output)
       112 RETURN_VALUE
```

The output is fairly straightforward, if terse. The first column contains line numbers that relate to our original file. The second column contains several `>>` symbols; these are the destinations for jump points elsewhere in the code. The third column is the operation address; the fourth has the operation name. The fifth column contains the parameters for the operation. The sixth column contains annotations to help line up the bytecode with the original Python parameters.

Refer back to [Example 2-3](#) to match the bytecode to the corresponding Python code. The bytecode starts on Python line 11 by putting the constant value 0 onto the stack, and then it builds a single-element list. Next, it searches the namespaces to find the `len` function, puts it on the stack, searches the namespaces again to find `zs`, and then puts that onto the stack. Inside Python line 12, it calls the `len` function from the stack, which consumes the `zs` reference in the stack; then it applies a binary multiply to the last two arguments (the length of `zs` and the single-element list) and stores the result in `output`. That's the first line of our Python function now dealt with. Follow the next block of bytecode to understand the behavior of the second line of Python code (the outer `for` loop).



The jump points (`>>`) match to instructions like `JUMP_ABSOLUTE` and `POP_JUMP_IF_FALSE`. Go through your own disassembled function and match the jump points to the jump instructions.

Having introduced bytecode, we can now ask: what's the bytecode and time cost of writing a function out explicitly versus using built-ins to perform the same task?

Different Approaches, Different Complexity

There should be one—and preferably only one—obvious way to do it. Although that way may not be obvious at first unless you're Dutch.²

—Tim Peters, *The Zen of Python*

There will be various ways to express your ideas using Python. Generally, the most sensible option should be clear, but if your experience is primarily with an older version of Python or another programming language, you may have other methods in mind. Some of these ways of expressing an idea may be slower than others.

You probably care more about readability than speed for most of your code, so your team can code efficiently without being puzzled by performant but opaque code.

² The language creator Guido van Rossum is Dutch, and not everyone has agreed with his “obvious” choices, but on the whole we like the choices that Guido makes!

Sometimes you will want performance, though (without sacrificing readability). Some speed testing might be what you need.

Take a look at the two code snippets in [Example 2-15](#). Both do the same job, but the first generates a lot of additional Python bytecode, which will cause more overhead.

Example 2-15. A naive and a more efficient way to solve the same summation problem

```
def fn_expressive(upper=1_000_000):
    total = 0
    for n in range(upper):
        total += n
    return total

def fn_terse(upper=1_000_000):
    return sum(range(upper))

assert fn_expressive() == fn_terse(), "Expect identical results from both functions"
```

Both functions calculate the sum of a range of integers. A simple rule of thumb (but one you *must* back up using profiling!) is that more lines of bytecode will execute more slowly than fewer equivalent lines of bytecode that use built-in functions. In [Example 2-16](#), we use IPython's `%timeit` magic function to measure the best execution time from a set of runs. `fn_terse` runs over twice as fast as `fn_expressive`!

Example 2-16. Using `%timeit` to test our hypothesis that using built-in functions should be faster than writing our own functions

```
In [2]: %timeit fn_expressive()
52.4 ms ± 86.4 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

In [3]: %timeit fn_terse()
18.1 ms ± 1.38 ms per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

If we use the `dis` module to investigate the code for each function, as shown in [Example 2-17](#), we can see that the virtual machine has 17 lines to execute with the more expressive function and only 6 to execute with the very readable but terser second function.

Example 2-17. Using `dis` to view the number of bytecode instructions involved in our two functions

```
In [4]: import dis
In [5]: dis.dis(fn_expressive)
 2          0 LOAD_CONST          1 (0)
          2 STORE_FAST          1 (total)
```

```

3          4 SETUP_LOOP                24 (to 30)
          6 LOAD_GLOBAL                0 (range)
          8 LOAD_FAST                   0 (upper)
         10 CALL_FUNCTION                1
         12 GET_ITER
>>      14 FOR_ITER                   12 (to 28)
         16 STORE_FAST                  2 (n)

4          18 LOAD_FAST                 1 (total)
         20 LOAD_FAST                 2 (n)
         22 INPLACE_ADD
         24 STORE_FAST                 1 (total)
         26 JUMP_ABSOLUTE              14
>>      28 POP_BLOCK

5    >>  30 LOAD_FAST                 1 (total)
         32 RETURN_VALUE

In [6]: dis.dis(fn_terse)
8          0 LOAD_GLOBAL                0 (sum)
         2 LOAD_GLOBAL                1 (range)
         4 LOAD_FAST                   0 (upper)
         6 CALL_FUNCTION                1
         8 CALL_FUNCTION                1
        10 RETURN_VALUE

```

The difference between the two code blocks is striking. Inside `fn_expressive()`, we maintain two local variables and iterate over a list using a `for` statement. The `for` loop will be checking to see if a `StopIteration` exception has been raised on each loop. Each iteration applies the `total.__add__` function, which will check the type of the second variable (`n`) on each iteration. These checks all add a little expense.

Inside `fn_terse()`, we call out to an optimized C list comprehension function that knows how to generate the final result without creating intermediate Python objects. This is much faster, although each iteration must still check for the types of the objects that are being added together (in [Chapter 4](#), we look at ways of fixing the type so we don't need to check it on each iteration).

As noted previously, you *must* profile your code—if you just rely on this heuristic, you will inevitably write slower code at some point. It is definitely worth learning whether a shorter and still readable way to solve your problem is built into Python. If so, it is more likely to be easily readable by another programmer, and it will *probably* run faster.

Unit Testing During Optimization to Maintain Correctness

If you aren't already unit testing your code, you are probably hurting your longer-term productivity. Ian (blushing) is embarrassed to note that he once spent a day

optimizing his code, having disabled unit tests because they were inconvenient, only to discover that his significant speedup result was due to breaking a part of the algorithm he was improving. You do not need to make this mistake even once.



Add unit tests to your code for a saner life. You'll be giving your current self and your colleagues faith that your code works, and you'll be giving a present to your future-self who has to maintain this code later. You really will save a lot of time in the long term by adding tests to your code.

In addition to unit testing, you should also strongly consider using `coverage.py`. It checks to see which lines of code are exercised by your tests and identifies the sections that have no coverage. This quickly lets you figure out whether you're testing the code that you're about to optimize, such that any mistakes that might creep in during the optimization process are quickly caught.

No-op @profile Decorator

Your unit tests will fail with a `NameError` exception if your code uses `@profile` from `line_profiler` or `memory_profiler`. The reason is that the unit test framework will not be injecting the `@profile` decorator into the local namespace. The no-op decorator shown here solves this problem. It is easiest to add it to the block of code that you're testing and remove it when you're done.

With the no-op decorator, you can run your tests without modifying the code that you're testing. This means you can run your tests after every profile-led optimization you make so you'll never be caught out by a bad optimization step.

Let's say we have the trivial `ex.py` module shown in [Example 2-18](#). It has a test (for `pytest`) and a function that we've been profiling with either `line_profiler` or `memory_profiler`.

Example 2-18. Simple function and test case where we wish to use @profile

```
import time

def test_some_fn():
    """Check basic behaviors for our function"""
    assert some_fn(2) == 4
    assert some_fn(1) == 1
    assert some_fn(-1) == 1

@profile
def some_fn(useful_input):
    """An expensive function that we wish to both test and profile"""
```

```

# artificial "we're doing something clever and expensive" delay
time.sleep(1)
return useful_input ** 2

if __name__ == "__main__":
    print(f"Example call `some_fn(2)` == {some_fn(2)}")

```

If we run `pytest` on our code, we'll get a `NameError`, as shown in [Example 2-19](#).

Example 2-19. A missing decorator during testing breaks out tests in an unhelpful way!

```

$ pytest utility.py
===== test session starts =====
platform linux -- Python 3.7.3, pytest-4.6.2, py-1.8.0, pluggy-0.12.0
rootdir: noop_profile_decorator
plugins: cov-2.7.1
collected 0 items / 1 errors

===== ERRORS =====
_____ ERROR collecting utility.py _____
utility.py:20: in <module>
    @profile
E   NameError: name 'profile' is not defined

```

The solution is to add a no-op decorator at the start of our module (you can remove it after you're done with profiling). If the `@profile` decorator is not found in one of the namespaces (because `line_profiler` or `memory_profiler` is not being used), the no-op version we've written is added. If `line_profiler` or `memory_profiler` has injected the new function into the namespace, our no-op version is ignored.

For both `line_profiler` and `memory_profiler`, we can add the code in [Example 2-20](#).

Example 2-20. Add a no-op `@profile` decorator to the namespace while unit testing

```

# check for line_profiler or memory_profiler in the local scope, both
# are injected by their respective tools or they're absent
# if these tools aren't being used (in which case we need to substitute
# a dummy @profile decorator)
if 'line_profiler' not in dir() and 'profile' not in dir():
    def profile(func):
        return func

```

Having added the no-op decorator, we can now run our `pytest` successfully, as shown in [Example 2-21](#), along with our profilers—with no additional code changes.

Example 2-21. With the no-op decorator, we have working tests, and both of our profilers work correctly

```
$ pytest utility.py
===== test session starts =====
platform linux -- Python 3.7.3, pytest-4.6.2, py-1.8.0, pluggy-0.12.0
rootdir: /home/ian/workspace/personal_projects/high_performance_python_book_2e/
high-performance-python-2e/examples_ian/ian/ch02/noop_profile_decorator
plugins: cov-2.7.1
collected 1 item

utility.py .

===== 1 passed in 3.04 seconds =====

$ kernprof -l -v utility.py
Example call `some_fn(2)` == 4
...
Line #      Hits          Time  Per Hit   % Time  Line Contents
=====
    20                                  @profile
    21                                  def some_fn(useful_input):
    22                                  """An expensive function that...
    23                                  # artificial 'we're doing...
    24              1      1001345.0  1001345.0    100.0      time.sleep(1)
    25              1           7.0        7.0      0.0      return useful_input ** 2

$ python -m memory_profiler utility.py
Example call `some_fn(2)` == 4
Filename: utility.py

Line #      Mem usage      Increment   Line Contents
=====
    20      48.277 MiB      48.277 MiB   @profile
    21                                  def some_fn(useful_input):
    22                                  """An expensive function that we wish to...
    23                                  # artificial 'we're doing something clever...
    24      48.277 MiB       0.000 MiB   time.sleep(1)
    25      48.277 MiB       0.000 MiB   return useful_input ** 2
```

You can save yourself a few minutes by avoiding the use of these decorators, but once you've lost hours making a false optimization that breaks your code, you'll want to integrate this into your workflow.

Strategies to Profile Your Code Successfully

Profiling requires some time and concentration. You will stand a better chance of understanding your code if you separate the section you want to test from the main body of your code. You can then unit test the code to preserve correctness, and you can pass in realistic fabricated data to exercise the inefficiencies you want to address.

Do remember to disable any BIOS-based accelerators, as they will only confuse your results. On Ian's laptop, the Intel Turbo Boost feature can temporarily accelerate a CPU above its normal maximum speed if it is cool enough. This means that a cool CPU may run the same block of code faster than a hot CPU. Your operating system may also control the clock speed—a laptop on battery power is likely to more aggressively control CPU speed than a laptop on AC power. To create a more stable benchmarking configuration, we do the following:

- Disable Turbo Boost in the BIOS.
- Disable the operating system's ability to override the SpeedStep (you will find this in your BIOS if you're allowed to control it).
- Use only AC power (never battery power).
- Disable background tools like backups and Dropbox while running experiments.
- Run the experiments many times to obtain a stable measurement.
- Possibly drop to run level 1 (Unix) so that no other tasks are running.
- Reboot and rerun the experiments to double-confirm the results.

Try to hypothesize the expected behavior of your code and then validate (or disprove!) the hypothesis with the result of a profiling step. Your choices will not change (you should only drive your decisions by using the profiled results), but your intuitive understanding of the code will improve, and this will pay off in future projects as you will be more likely to make performant decisions. Of course, you will verify these performant decisions by profiling as you go.

Do not skimp on the preparation. If you try to performance test code deep inside a larger project without separating it from the larger project, you are likely to witness side effects that will sidetrack your efforts. It is likely to be harder to unit test a larger project when you're making fine-grained changes, and this may further hamper your efforts. Side effects could include other threads and processes impacting CPU and memory usage and network and disk activity, which will skew your results.

Naturally, you're already using source code control (e.g., Git or Mercurial), so you'll be able to run multiple experiments in different branches without ever losing "the versions that work well." If you're *not* using source code control, do yourself a huge favor and start to do so!

For web servers, investigate dower and dozer; you can use these to visualize in real time the behavior of objects in the namespace. Definitely consider separating the code you want to test out of the main web application if possible, as this will make profiling significantly easier.

Make sure your unit tests exercise all the code paths in the code that you're analyzing. Anything you don't test that is used in your benchmarking may cause subtle errors

that will slow down your progress. Use `coverage.py` to confirm that your tests are covering all the code paths.

Unit testing a complicated section of code that generates a large numerical output may be difficult. Do not be afraid to output a text file of results to run through `diff` or to use a pickled object. For numeric optimization problems, Ian likes to create long text files of floating-point numbers and use `diff`—minor rounding errors show up immediately, even if they’re rare in the output.

If your code might be subject to numerical rounding issues due to subtle changes, you are better off with a large output that can be used for a before-and-after comparison. One cause of rounding errors is the difference in floating-point precision between CPU registers and main memory. Running your code through a different code path can cause subtle rounding errors that might later confound you—it is better to be aware of this as soon as they occur.

Obviously, it makes sense to use a source code control tool while you are profiling and optimizing. Branching is cheap, and it will preserve your sanity.

Wrap-Up

Having looked at profiling techniques, you should have all the tools you need to identify bottlenecks around CPU and RAM usage in your code. Next, we’ll look at how Python implements the most common containers, so you can make sensible decisions about representing larger collections of data.

PNP Ch 57 | The dis Module [optional companion]

Chapter 57: The dis module

Section 57.1: What is Python bytecode?

Python is a hybrid interpreter. When running a program, it first assembles it into *bytecode* which can then be run in the Python interpreter (also called a *Python virtual machine*). The `dis` module in the standard library can be used to make the Python bytecode human-readable by disassembling classes, methods, functions, and code objects.

```
>>> def hello():
...     print "Hello, World"
...
>>> dis.dis(hello)
2          0 LOAD_CONST           1 ('Hello, World')
          3 PRINT_ITEM
          4 PRINT_NEWLINE
          5 LOAD_CONST           0 (None)
          8 RETURN_VALUE
```

The Python interpreter is stack-based and uses a first-in last-out system.

Each operation code (opcode) in the Python assembly language (the bytecode) takes a fixed number of items from the stack and returns a fixed number of items to the stack. If there aren't enough items on the stack for an opcode, the Python interpreter will crash, possibly without an error message.

Section 57.2: Constants in the dis module

```
EXTENDED_ARG = 145 # All opcodes greater than this have 2 operands
HAVE_ARGUMENT = 90 # All opcodes greater than this have at least 1 operands

cmp_op = ('<', '<=', '==', '!=', '>', '>=', 'in', 'not in', 'is', 'is ...
          # A list of comparator id's. The indices are used as operands in some opcodes

# All opcodes in these lists have the respective types as there operands
hascompare = [107]
hasconst = [100]
hasfree = [135, 136, 137]
hasjabs = [111, 112, 113, 114, 115, 119]
hasjrel = [93, 110, 120, 121, 122, 143]
haslocal = [124, 125, 126]
hasname = [90, 91, 95, 96, 97, 98, 101, 106, 108, 109, 116]

# A map of opcodes to ids
opmap = {'BINARY_ADD': 23, 'BINARY_AND': 64, 'BINARY_DIVIDE': 21, 'BIN...
# A map of ids to opcodes
opname = ['STOP_CODE', 'POP_TOP', 'ROT_TWO', 'ROT_THREE', 'DUP_TOP', '...
```

Section 57.3: Disassembling modules

To disassemble a Python module, first this has to be turned into a `.pyc` file (Python compiled). To do this, run

```
python -m compileall <file>.py
```

Then in an interpreter, run

```
import dis
import marshal
```

```
with open("<file>.pyc", "rb") as code_f:
    code_f.read(8) # Magic number and modification time
    code = marshal.load(code_f) # Returns a code object which can be disassembled
    dis.dis(code) # Output the disassembly
```

This will compile a Python module and output the bytecode instructions with `dis`. The module is never imported so it is safe to use with untrusted code.

PHASE 02 · Core Data Structures

~3 weeks · 7 chapters

Know what ML data lives in, and the performance cost of each choice.

Chapters in this phase:

- FP Ch 2 | An Array of Sequences
- FP Ch 3 | Dictionaries and Sets
- FP Ch 4 | Unicode Text vs Bytes
- FP Ch 8 | Object References & Mutability [= plan Ch 6]
- HPP Ch 3 | Lists and Tuples (performance)
- HPP Ch 4 | Dictionaries and Sets (performance)
- PNP Ch 19 | Dictionary [companion]
- PNP Ch 7 | Enum [companion]
- PNP Ch 47 | Collections Module [companion]
- PNP Ch 49 | JSON Module [companion]

FP Ch 2 | An Array of Sequences

An Array of Sequences

As you may have noticed, several of the operations mentioned work equally for texts, lists and tables. Texts, lists and tables together are called *trains*. [...] The FOR command also works generically on trains.¹

— Geurts, Meertens, and Pemberton
ABC Programmer's Handbook

Before creating Python, Guido was a contributor to the ABC language—a 10-year research project to design a programming environment for beginners. ABC introduced many ideas we now consider “Pythonic”: generic operations on sequences, built-in tuple and mapping types, structure by indentation, strong typing without variable declarations, and more. It’s no accident that Python is so user-friendly.

Python inherited from ABC the uniform handling of sequences. Strings, lists, byte sequences, arrays, XML elements, and database results share a rich set of common operations including iteration, slicing, sorting, and concatenation.

Understanding the variety of sequences available in Python saves us from reinventing the wheel, and their common interface inspires us to create APIs that properly support and leverage existing and future sequence types.

Most of the discussion in this chapter applies to sequences in general, from the familiar `list` to the `str` and `bytes` types that are new in Python 3. Specific topics on lists, tuples, arrays, and queues are also covered here, but the focus on Unicode strings and byte sequences is deferred to [Chapter 4](#). Also, the idea here is to cover sequence types that are ready to use. Creating your own sequence types is the subject of [Chapter 10](#).

1. Leo Geurts, Lambert Meertens, and Steven Pemberton, *ABC Programmer's Handbook*, p. 8.

Overview of Built-In Sequences

The standard library offers a rich selection of sequence types implemented in C:

Container sequences

`list`, `tuple`, and `collections.deque` can hold items of different types.

Flat sequences

`str`, `bytes`, `bytearray`, `memoryview`, and `array.array` hold items of one type.

Container sequences hold references to the objects they contain, which may be of any type, while *flat sequences* physically store the value of each item within its own memory space, and not as distinct objects. Thus, flat sequences are more compact, but they are limited to holding primitive values like characters, bytes, and numbers.

Another way of grouping sequence types is by mutability:

Mutable sequences

`list`, `bytearray`, `array.array`, `collections.deque`, and `memoryview`

Immutable sequences

`tuple`, `str`, and `bytes`

Figure 2-1 helps visualize how mutable sequences differ from immutable ones, while also inheriting several methods from them. Note that the built-in concrete sequence types do not actually subclass the `Sequence` and `MutableSequence` abstract base classes (ABCs) depicted, but the ABCs are still useful as a formalization of what functionality to expect from a full-featured sequence type.

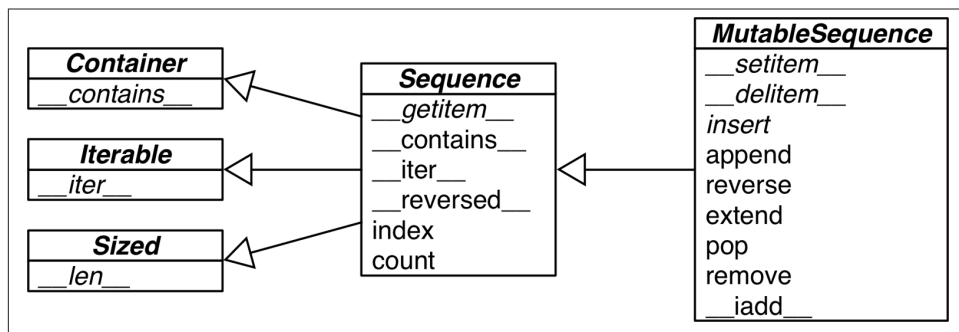


Figure 2-1. UML class diagram for some classes from `collections.abc` (superclasses are on the left; inheritance arrows point from subclasses to superclasses; names in italic are abstract classes and abstract methods)

Keeping in mind these common traits—mutable versus immutable; container versus flat—is helpful to extrapolate what you know about one sequence type to others.

The most fundamental sequence type is the `list`—mutable and mixed-type. I am sure you are comfortable handling them, so we'll jump right into list comprehensions, a powerful way of building lists that is somewhat underused because the syntax may be unfamiliar. Mastering list comprehensions opens the door to generator expressions, which—among other uses—can produce elements to fill up sequences of any type. Both are the subject of the next section.

List Comprehensions and Generator Expressions

A quick way to build a sequence is using a list comprehension (if the target is a `list`) or a generator expression (for all other kinds of sequences). If you are not using these syntactic forms on a daily basis, I bet you are missing opportunities to write code that is more readable and often faster at the same time.

If you doubt my claim that these constructs are “more readable,” read on. I'll try to convince you.



For brevity, many Python programmers refer to list comprehensions as *listcomps*, and generator expressions as *genexprs*. I will use these words as well.

List Comprehensions and Readability

Here is a test: which do you find easier to read, [Example 2-1](#) or [Example 2-2](#)?

Example 2-1. Build a list of Unicode codepoints from a string

```
>>> symbols = '§ç£¥€¤'
>>> codes = []
>>> for symbol in symbols:
...     codes.append(ord(symbol))
...
>>> codes
[36, 162, 163, 165, 8364, 164]
```

Example 2-2. Build a list of Unicode codepoints from a string, take two

```
>>> symbols = '§ç£¥€¤'
>>> codes = [ord(symbol) for symbol in symbols]
>>> codes
[36, 162, 163, 165, 8364, 164]
```

Anybody who knows a little bit of Python can read [Example 2-1](#). However, after learning about listcomps, I find [Example 2-2](#) more readable because its intent is explicit.

A for loop may be used to do lots of different things: scanning a sequence to count or pick items, computing aggregates (sums, averages), or any number of other processing tasks. The code in [Example 2-1](#) is building up a list. In contrast, a listcomp is meant to do one thing only: to build a new list.

Of course, it is possible to abuse list comprehensions to write truly incomprehensible code. I've seen Python code with listcomps used just to repeat a block of code for its side effects. If you are not doing something with the produced list, you should not use that syntax. Also, try to keep it short. If the list comprehension spans more than two lines, it is probably best to break it apart or rewrite as a plain old for loop. Use your best judgment: for Python as for English, there are no hard-and-fast rules for clear writing.



Syntax Tip

In Python code, line breaks are ignored inside pairs of `[]`, `{}`, or `()`. So you can build multiline lists, listcomps, genexps, dictionaries and the like without using the ugly `\` line continuation escape.

Listcomps No Longer Leak Their Variables

In Python 2.x, variables assigned in the for clauses in list comprehensions were set in the surrounding scope, sometimes with tragic consequences. See the following Python 2.7 console session:

```
Python 2.7.6 (default, Mar 22 2014, 22:59:38)
[GCC 4.8.2] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> x = 'my precious'
>>> dummy = [x for x in 'ABC']
>>> x
'C'
```

As you can see, the initial value of `x` was clobbered. This no longer happens in Python 3.

List comprehensions, generator expressions, and their siblings `set` and `dict` comprehensions now have their own local scope, like functions. Variables assigned within the expression are local, but variables in the surrounding scope can still be referenced. Even better, the local variables do not mask the variables from the surrounding scope.

This is Python 3:

```
>>> x = 'ABC'
>>> dummy = [ord(x) for x in x]
>>> x ❶
'ABC'
>>> dummy ❷
```

```
[65, 66, 67]
>>>
```

- ❶ The value of `x` is preserved.
- ❷ The list comprehension produces the expected list.

List comprehensions build lists from sequences or any other iterable type by filtering and transforming items. The `filter` and `map` built-ins can be composed to do the same, but readability suffers, as we will see next.

Listcomps Versus `map` and `filter`

Listcomps do everything the `map` and `filter` functions do, without the contortions of the functionally challenged Python `lambda`. Consider [Example 2-3](#).

Example 2-3. The same list built by a listcomp and a `map/filter` composition

```
>>> symbols = '$ç£¥€¤'
>>> beyond_ascii = [ord(s) for s in symbols if ord(s) > 127]
>>> beyond_ascii
[162, 163, 165, 8364, 164]
>>> beyond_ascii = list(filter(lambda c: c > 127, map(ord, symbols)))
>>> beyond_ascii
[162, 163, 165, 8364, 164]
```

I used to believe that `map` and `filter` were faster than the equivalent listcomps, but Alex Martelli pointed out that's not the case—at least not in the preceding examples. The [02-array-seq/listcomp_speed.py](#) script in [the *Fluent Python* code repository](#) is a simple speed test comparing listcomp with `filter/map`.

I'll have more to say about `map` and `filter` in [Chapter 5](#). Now we turn to the use of listcomps to compute Cartesian products: a list containing tuples built from all items from two or more lists.

Cartesian Products

Listcomps can generate lists from the Cartesian product of two or more iterables. The items that make up the cartesian product are tuples made from items from every input iterable. The resulting list has a length equal to the lengths of the input iterables multiplied. See [Figure 2-2](#).

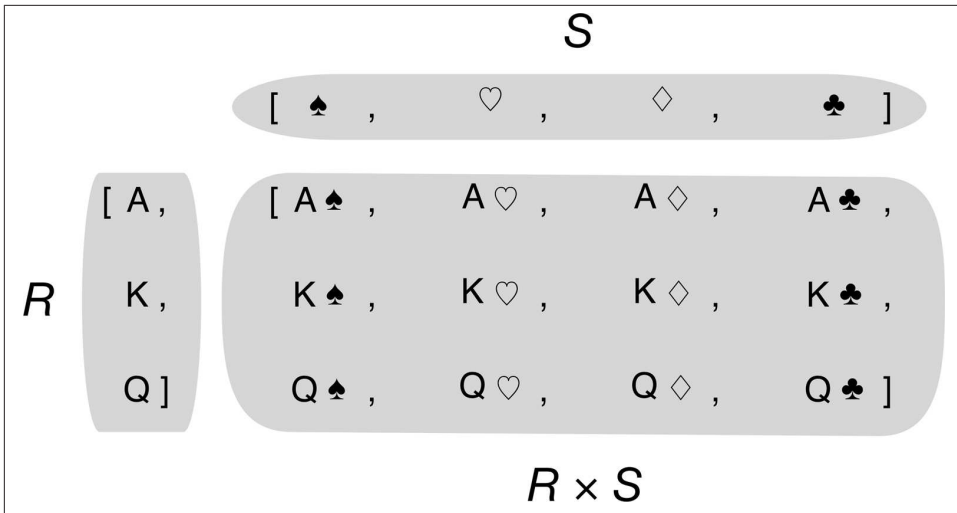


Figure 2-2. The Cartesian product of a sequence of three card ranks and a sequence of four suits results in a sequence of twelve pairings

For example, imagine you need to produce a list of T-shirts available in two colors and three sizes. **Example 2-4** shows how to produce that list using a listcomp. The result has six items.

Example 2-4. Cartesian product using a list comprehension

```
>>> colors = ['black', 'white']
>>> sizes = ['S', 'M', 'L']
>>> tshirts = [(color, size) for color in colors for size in sizes] ❶
>>> tshirts
[('black', 'S'), ('black', 'M'), ('black', 'L'), ('white', 'S'),
 ('white', 'M'), ('white', 'L')]
>>> for color in colors: ❷
...     for size in sizes:
...         print((color, size))
...
('black', 'S')
('black', 'M')
('black', 'L')
('white', 'S')
('white', 'M')
('white', 'L')
>>> tshirts = [(color, size) for size in sizes ❸
...             for color in colors]
>>> tshirts
[('black', 'S'), ('white', 'S'), ('black', 'M'), ('white', 'M'),
 ('black', 'L'), ('white', 'L')]
```

- ❶ This generates a list of tuples arranged by color, then size.
- ❷ Note how the resulting list is arranged as if the for loops were nested in the same order as they appear in the listcomp.
- ❸ To get items arranged by size, then color, just rearrange the for clauses; adding a line break to the listcomp makes it easy to see how the result will be ordered.

In **Example 1-1** (**Chapter 1**), the following expression was used to initialize a card deck with a list made of 52 cards from all 13 ranks of each of the 4 suits, grouped by suit:

```
self._cards = [Card(rank, suit) for suit in self.suits
               for rank in self.ranks]
```

Listcomps are a one-trick pony: they build lists. To fill up other sequence types, a genexp is the way to go. The next section is a brief look at genexps in the context of building nonlist sequences.

Generator Expressions

To initialize tuples, arrays, and other types of sequences, you could also start from a listcomp, but a genexp saves memory because it yields items one by one using the iterator protocol instead of building a whole list just to feed another constructor.

Genexps use the same syntax as listcomps, but are enclosed in parentheses rather than brackets.

Example 2-5 shows basic usage of genexps to build a tuple and an array.

Example 2-5. Initializing a tuple and an array from a generator expression

```
>>> symbols = '$¢£¥€¤'
>>> tuple(ord(symbol) for symbol in symbols) ❶
(36, 162, 163, 165, 8364, 164)
>>> import array
>>> array.array('I', (ord(symbol) for symbol in symbols)) ❷
array('I', [36, 162, 163, 165, 8364, 164])
```

- ❶ If the generator expression is the single argument in a function call, there is no need to duplicate the enclosing parentheses.
- ❷ The array constructor takes two arguments, so the parentheses around the generator expression are mandatory. The first argument of the array constructor defines the storage type used for the numbers in the array, as we'll see in **"Arrays"** on page 48.

Example 2-6 uses a genexp with a Cartesian product to print out a roster of T-shirts of two colors in three sizes. In contrast with **Example 2-4**, here the six-item list of T-shirts is never built in memory: the generator expression feeds the for loop producing one

item at a time. If the two lists used in the Cartesian product had 1,000 items each, using a generator expression would save the expense of building a list with a million items just to feed the for loop.

Example 2-6. Cartesian product in a generator expression

```
>>> colors = ['black', 'white']
>>> sizes = ['S', 'M', 'L']
>>> for tshirt in ('%s %s' % (c, s) for c in colors for s in sizes): ❶
...     print(tshirt)
...
black S
black M
black L
white S
white M
white L
```

- ❶ The generator expression yields items one by one; a list with all six T-shirt variations is never produced in this example.

Chapter 14 is devoted to explaining how generators work in detail. Here the idea was just to show the use of generator expressions to initialize sequences other than lists, or to produce output that you don't need to keep in memory.

Now we move on to the other fundamental sequence type in Python: the tuple.

Tuples Are Not Just Immutable Lists

Some introductory texts about Python present tuples as “immutable lists,” but that is short selling them. Tuples do double duty: they can be used as immutable lists and also as records with no field names. This use is sometimes overlooked, so we will start with that.

Tuples as Records

Tuples hold records: each item in the tuple holds the data for one field and the position of the item gives its meaning.

If you think of a tuple just as an immutable list, the quantity and the order of the items may or may not be important, depending on the context. But when using a tuple as a collection of fields, the number of items is often fixed and their order is always vital.

Example 2-7 shows tuples being used as records. Note that in every expression, sorting the tuple would destroy the information because the meaning of each data item is given by its position in the tuple.

Example 2-7. Tuples used as records

```
>>> lax_coordinates = (33.9425, -118.408056) ❶
>>> city, year, pop, chg, area = ('Tokyo', 2003, 32450, 0.66, 8014) ❷
>>> traveler_ids = [('USA', '31195855'), ('BRA', 'CE342567'), ❸
...                 ('ESP', 'XDA205856')]
>>> for passport in sorted(traveler_ids): ❹
...     print('%s/%s' % passport) ❺
...
BRA/CE342567
ESP/XDA205856
USA/31195855
>>> for country, _ in traveler_ids: ❻
...     print(country)
...
USA
BRA
ESP
```

- ❶ Latitude and longitude of the Los Angeles International Airport.
- ❷ Data about Tokyo: name, year, population (millions), population change (%), area (km²).
- ❸ A list of tuples of the form (country_code, passport_number).
- ❹ As we iterate over the list, `passport` is bound to each tuple.
- ❺ The % formatting operator understands tuples and treats each item as a separate field.
- ❻ The for loop knows how to retrieve the items of a tuple separately—this is called “unpacking.” Here we are not interested in the second item, so it’s assigned to `_`, a dummy variable.

Tuples work well as records because of the tuple unpacking mechanism—our next subject.

Tuple Unpacking

In [Example 2-7](#), we assigned ('Tokyo', 2003, 32450, 0.66, 8014) to `city`, `year`, `pop`, `chg`, `area` in a single statement. Then, in the last line, the % operator assigned each item in the `passport` tuple to one slot in the format string in the `print` argument. Those are two examples of *tuple unpacking*.



Tuple unpacking works with any iterable object. The only requirement is that the iterable yields exactly one item per variable in the receiving tuple, unless you use a star (*) to capture excess items as explained in “Using * to grab excess items” on page 29. The term *tuple unpacking* is widely used by Pythonistas, but *iterable unpacking* is gaining traction, as in the title of [PEP 3132 — Extended Iterable Unpacking](#).

The most visible form of tuple unpacking is *parallel assignment*; that is, assigning items from an iterable to a tuple of variables, as you can see in this example:

```
>>> lax_coordinates = (33.9425, -118.408056)
>>> latitude, longitude = lax_coordinates # tuple unpacking
>>> latitude
33.9425
>>> longitude
-118.408056
```

An elegant application of tuple unpacking is swapping the values of variables without using a temporary variable:

```
>>> b, a = a, b
```

Another example of tuple unpacking is prefixing an argument with a star when calling a function:

```
>>> divmod(20, 8)
(2, 4)
>>> t = (20, 8)
>>> divmod(*t)
(2, 4)
>>> quotient, remainder = divmod(*t)
>>> quotient, remainder
(2, 4)
```

The preceding code also shows a further use of tuple unpacking: enabling functions to return multiple values in a way that is convenient to the caller. For example, the `os.path.split()` function builds a tuple (`path`, `last_part`) from a filesystem path:

```
>>> import os
>>> _, filename = os.path.split('/home/luciano/.ssh/idrsa.pub')
>>> filename
'idrsa.pub'
```

Sometimes when we only care about certain parts of a tuple when unpacking, a dummy variable like `_` is used as placeholder, as in the preceding example.



If you write internationalized software, `_` is not a good dummy variable because it is traditionally used as an alias to the `gettext.gettext` function, as recommended in the [gettext module documentation](#). Otherwise, it's a nice name for placeholder variable.

Another way of focusing on just some of the items when unpacking a tuple is to use the `*`, as we'll see right away.

Using `*` to grab excess items

Defining function parameters with `*args` to grab arbitrary excess arguments is a classic Python feature.

In Python 3, this idea was extended to apply to parallel assignment as well:

```
>>> a, b, *rest = range(5)
>>> a, b, rest
(0, 1, [2, 3, 4])
>>> a, b, *rest = range(3)
>>> a, b, rest
(0, 1, [2])
>>> a, b, *rest = range(2)
>>> a, b, rest
(0, 1, [])
```

In the context of parallel assignment, the `*` prefix can be applied to exactly one variable, but it can appear in any position:

```
>>> a, *body, c, d = range(5)
>>> a, body, c, d
(0, [1, 2], 3, 4)
>>> *head, b, c, d = range(5)
>>> head, b, c, d
([0, 1], 2, 3, 4)
```

Finally, a powerful feature of tuple unpacking is that it works with nested structures.

Nested Tuple Unpacking

The tuple to receive an expression to unpack can have nested tuples, like `(a, b, (c, d))`, and Python will do the right thing if the expression matches the nesting structure.

[Example 2-8](#) shows nested tuple unpacking in action.

Example 2-8. Unpacking nested tuples to access the longitude

```
metro_areas = [
    ('Tokyo', 'JP', 36.933, (35.689722, 139.691667)), # ❶
    ('Delhi NCR', 'IN', 21.935, (28.613889, 77.208889)),
    ('Mexico City', 'MX', 20.142, (19.433333, -99.133333)),
```

```

('New York-Newark', 'US', 20.104, (40.808611, -74.020386)),
('Sao Paulo', 'BR', 19.649, (-23.547778, -46.635833)),
]

print('{:15} | {:^9} | {:^9}'.format('', 'lat.', 'long.'))
fmt = '{:15} | {:9.4f} | {:9.4f}'
for name, cc, pop, (latitude, longitude) in metro_areas: # ❷
    if longitude <= 0: # ❸
        print(fmt.format(name, latitude, longitude))

```

- ❶ Each tuple holds a record with four fields, the last of which is a coordinate pair.
- ❷ By assigning the last field to a tuple, we unpack the coordinates.
- ❸ `if longitude <= 0:` limits the output to metropolitan areas in the Western hemisphere.

The output of **Example 2-8** is:

	lat.	long.
Mexico City	19.4333	-99.1333
New York-Newark	40.8086	-74.0204
Sao Paulo	-23.5478	-46.6358



Before Python 3, it was possible to define functions with nested tuples in the formal parameters (e.g., `def fn(a, (b, c), d):`). This is no longer supported in Python 3 function definitions, for practical reasons explained in [PEP 3113 — Removal of Tuple Parameter Unpacking](#). To be clear: nothing changed from the perspective of users calling a function. The restriction applies only to the definition of functions.

As designed, tuples are very handy. But there is a missing feature when using them as records: sometimes it is desirable to name the fields. That is why the `namedtuple` function was invented. Read on.

Named Tuples

The `collections.namedtuple` function is a factory that produces subclasses of `tuple` enhanced with field names and a class name—which helps debugging.



Instances of a class that you build with `namedtuple` take exactly the same amount of memory as tuples because the field names are stored in the class. They use less memory than a regular object because they don't store attributes in a per-instance `__dict__`.

Recall how we built the Card class in [Example 1-1](#) in [Chapter 1](#):

```
Card = collections.namedtuple('Card', ['rank', 'suit'])
```

[Example 2-9](#) shows how we could define a named tuple to hold information about a city.

Example 2-9. Defining and using a named tuple type

```
>>> from collections import namedtuple
>>> City = namedtuple('City', 'name country population coordinates') ❶
>>> tokyo = City('Tokyo', 'JP', 36.933, (35.689722, 139.691667)) ❷
>>> tokyo
City(name='Tokyo', country='JP', population=36.933, coordinates=(35.689722,
139.691667))
>>> tokyo.population ❸
36.933
>>> tokyo.coordinates
(35.689722, 139.691667)
>>> tokyo[1]
'JP'
```

- ❶ Two parameters are required to create a named tuple: a class name and a list of field names, which can be given as an iterable of strings or as a single space-delimited string.
- ❷ Data must be passed as positional arguments to the constructor (in contrast, the tuple constructor takes a single iterable).
- ❸ You can access the fields by name or position.

A named tuple type has a few attributes in addition to those inherited from tuple.

[Example 2-10](#) shows the most useful: the `_fields` class attribute, the class method `_make(iterable)`, and the `_asdict()` instance method.

Example 2-10. Named tuple attributes and methods (continued from the previous example)

```
>>> City._fields ❶
('name', 'country', 'population', 'coordinates')
>>> LatLong = namedtuple('LatLong', 'lat long')
>>> delhi_data = ('Delhi NCR', 'IN', 21.935, LatLong(28.613889, 77.208889))
>>> delhi = City._make(delhi_data) ❷
>>> delhi._asdict() ❸
OrderedDict([('name', 'Delhi NCR'), ('country', 'IN'), ('population',
21.935), ('coordinates', LatLong(lat=28.613889, long=77.208889))])
>>> for key, value in delhi._asdict().items():
    print(key + ': ', value)
```

```
name: Delhi NCR
country: IN
population: 21.935
```

```
coordinates: LatLong(lat=28.613889, long=77.208889)
>>>
```

- ❶ `_fields` is a tuple with the field names of the class.
- ❷ `_make()` allow you to instantiate a named tuple from an iterable; `City(*delhi_data)` would do the same.
- ❸ `_asdict()` returns a `collections.OrderedDict` built from the named tuple instance. That can be used to produce a nice display of city data.

Now that we’ve explored the power of tuples as records, we can consider their second role as an immutable variant of the `list` type.

Tuples as Immutable Lists

When using a tuple as an immutable variation of `list`, it helps to know how similar they actually are. As you can see in [Table 2-1](#), tuple supports all list methods that do not involve adding or removing items, with one exception—tuple lacks the `__reversed__` method. However, that is just for optimization; `reversed(my_tuple)` works without it.

Table 2-1. Methods and attributes found in list or tuple (methods implemented by object are omitted for brevity)

	list	tuple	
<code>s.__add__(s2)</code>	•	•	<code>s + s2</code> —concatenation
<code>s.__iadd__(s2)</code>	•		<code>s += s2</code> —in-place concatenation
<code>s.append(e)</code>	•		Append one element after last
<code>s.clear()</code>	•		Delete all items
<code>s.__contains__(e)</code>	•	•	<code>e in s</code>
<code>s.copy()</code>	•		Shallow copy of the list
<code>s.count(e)</code>	•	•	Count occurrences of an element
<code>s.__delitem__(p)</code>	•		Remove item at position <code>p</code>
<code>s.extend(it)</code>	•		Append items from iterable <code>it</code>
<code>s.__getitem__(p)</code>	•	•	<code>s[p]</code> —get item at position
<code>s.__getnewargs__()</code>		•	Support for optimized serialization with <code>pickle</code>
<code>s.index(e)</code>	•	•	Find position of first occurrence of <code>e</code>
<code>s.insert(p, e)</code>	•		Insert element <code>e</code> before the item at position <code>p</code>
<code>s.__iter__()</code>	•	•	Get iterator
<code>s.__len__()</code>	•	•	<code>len(s)</code> —number of items
<code>s.__mul__(n)</code>	•	•	<code>s * n</code> —repeated concatenation

	list	tuple
<code>s.__imul__(n)</code>	•	<code>s *= n</code> —in-place repeated concatenation
<code>s.__rmul__(n)</code>	• •	<code>n * s</code> —reversed repeated concatenation ^a
<code>s.pop([p])</code>	•	Remove and return last item or item at optional position <code>p</code>
<code>s.remove(e)</code>	•	Remove first occurrence of element <code>e</code> by value
<code>s.reverse()</code>	•	Reverse the order of the items in place
<code>s.__reversed__()</code>	•	Get iterator to scan items from last to first
<code>s.__setitem__(p, e)</code>	•	<code>s[p] = e</code> —put <code>e</code> in position <code>p</code> , overwriting existing item
<code>s.sort([key], [reverse])</code>	•	Sort items in place with optional keyword arguments <code>key</code> and <code>reverse</code>

^a Reversed operators are explained in [Chapter 13](#).

Every Python programmer knows that sequences can be sliced using the `s[a:b]` syntax. We now turn to some less well-known facts about slicing.

Slicing

A common feature of `list`, `tuple`, `str`, and all sequence types in Python is the support of slicing operations, which are more powerful than most people realize.

In this section, we describe the *use* of these advanced forms of slicing. Their implementation in a user-defined class will be covered in [Chapter 10](#), in keeping with our philosophy of covering ready-to-use classes in this part of the book, and creating new classes in [Part IV](#).

Why Slices and Range Exclude the Last Item

The Pythonic convention of excluding the last item in slices and ranges works well with the zero-based indexing used in Python, C, and many other languages. Some convenient features of the convention are:

- It's easy to see the length of a slice or range when only the stop position is given: `range(3)` and `my_list[:3]` both produce three items.
- It's easy to compute the length of a slice or range when start and stop are given: just subtract `stop - start`.
- It's easy to split a sequence in two parts at any index `x`, without overlapping: simply get `my_list[:x]` and `my_list[x:]`. For example:

```
>>> l = [10, 20, 30, 40, 50, 60]
>>> l[:2] # split at 2
[10, 20]
>>> l[2:]
[30, 40, 50, 60]
```

```
>>> l[:3] # split at 3
[10, 20, 30]
>>> l[3:]
[40, 50, 60]
```

But the best arguments for this convention were written by the Dutch computer scientist Edsger W. Dijkstra (see the last reference in “[Further Reading](#)” on page 59).

Now let’s take a close look at how Python interprets slice notation.

Slice Objects

This is no secret, but worth repeating just in case: `s[a:b:c]` can be used to specify a stride or step `c`, causing the resulting slice to skip items. The stride can also be negative, returning items in reverse. Three examples make this clear:

```
>>> s = 'bicycle'
>>> s[:3]
'bye'
>>> s[::-1]
'elcycib'
>>> s[::-2]
'eccb'
```

Another example was shown in [Chapter 1](#) when we used `deck[12::13]` to get all the aces in the unshuffled deck:

```
>>> deck[12::13]
[Card(rank='A', suit='spades'), Card(rank='A', suit='diamonds'),
 Card(rank='A', suit='clubs'), Card(rank='A', suit='hearts')]
```

The notation `a:b:c` is only valid within `[]` when used as the indexing or subscript operator, and it produces a slice object: `slice(a, b, c)`. As we will see in “[How Slicing Works](#)” on page 281, to evaluate the expression `seq[start:stop:step]`, Python calls `seq.__getitem__(slice(start, stop, step))`. Even if you are not implementing your own sequence types, knowing about slice objects is useful because it lets you assign names to slices, just like spreadsheets allow naming of cell ranges.

Suppose you need to parse flat-file data like the invoice shown in [Example 2-11](#). Instead of filling your code with hardcoded slices, you can name them. See how readable this makes the for loop at the end of the example.

Example 2-11. Line items from a flat-file invoice

```
>>> invoice = """
... 0.....6.....40.....52...55.....
... 1909 Pimoroni PiBrella                $17.50  3    $52.50
... 1489 6mm Tactile Switch x20           $4.95   2    $9.90
... 1510 Panavise Jr. - PV-201            $28.00  1   $28.00
... 1601 PiTFT Mini Kit 320x240           $34.95  1   $34.95
... """
```



```
>>> SKU = slice(0, 6)
>>> DESCRIPTION = slice(6, 40)
>>> UNIT_PRICE = slice(40, 52)
>>> QUANTITY = slice(52, 55)
>>> ITEM_TOTAL = slice(55, None)
>>> line_items = invoice.split('\n')[2:]
>>> for item in line_items:
...     print(item[UNIT_PRICE], item[DESCRIPTION])
...
$17.50    Pimoroni PiBrella
$4.95     6mm Tactile Switch x20
$28.00    Panavise Jr. - PV-201
$34.95    PiTFT Mini Kit 320x240
```

We'll come back to slice objects when we discuss creating your own collections in [“Vector Take #2: A Sliceable Sequence” on page 280](#). Meanwhile, from a user perspective, slicing includes additional features such as multidimensional slices and ellipsis (...) notation. Read on.

Multidimensional Slicing and Ellipsis

The `[]` operator can also take multiple indexes or slices separated by commas. This is used, for instance, in the external NumPy package, where items of a two-dimensional `numpy.ndarray` can be fetched using the syntax `a[i, j]` and a two-dimensional slice obtained with an expression like `a[m:n, k:l]`. [Example 2-22](#) later in this chapter shows the use of this notation. The `__getitem__` and `__setitem__` special methods that handle the `[]` operator simply receive the indices in `a[i, j]` as a tuple. In other words, to evaluate `a[i, j]`, Python calls `a.__getitem__((i, j))`.

The built-in sequence types in Python are one-dimensional, so they support only one index or slice, and not a tuple of them.

The ellipsis—written with three full stops (...) and not ... (Unicode U+2026)—is recognized as a token by the Python parser. It is an alias to the `Ellipsis` object, the single instance of the `ellipsis` class.² As such, it can be passed as an argument to functions and as part of a slice specification, as in `f(a, ..., z)` or `a[i:...]`. NumPy uses ... as a shortcut when slicing arrays of many dimensions; for example, if `x` is a four-dimensional array, `x[i, ...]` is a shortcut for `x[i, :, :, :]`. See the [Tentative NumPy Tutorial](#) to learn more about this.

At the time of this writing, I am unaware of uses of `Ellipsis` or multidimensional indexes and slices in the Python standard library. If you spot one, let me know. These syntactic features exist to support user-defined types and extensions such as NumPy.

2. No, I did not get this backwards: the `ellipsis` class name is really all lowercase and the instance is a built-in named `Ellipsis`, just like `bool` is lowercase but its instances are `True` and `False`.

Slices are not just useful to extract information from sequences; they can also be used to change mutable sequences in place—that is, without rebuilding them from scratch.

Assigning to Slices

Mutable sequences can be grafted, excised, and otherwise modified in place using slice notation on the left side of an assignment statement or as the target of a `del` statement. The next few examples give an idea of the power of this notation:

```
>>> l = list(range(10))
>>> l
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> l[2:5] = [20, 30]
>>> l
[0, 1, 20, 30, 5, 6, 7, 8, 9]
>>> del l[5:7]
>>> l
[0, 1, 20, 30, 5, 8, 9]
>>> l[3::2] = [11, 22]
>>> l
[0, 1, 20, 11, 5, 22, 9]
>>> l[2:5] = 100 ❶
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only assign an iterable
>>> l[2:5] = [100]
>>> l
[0, 1, 100, 22, 9]
```

- ❶ When the target of the assignment is a slice, the right side must be an iterable object, even if it has just one item.

Everybody knows that concatenation is a common operation with sequences of any type. Any introductory Python text explains the use of `+` and `*` for that purpose, but there are some subtle details on how they work, which we cover next.

Using `+` and `*` with Sequences

Python programmers expect that sequences support `+` and `*`. Usually both operands of `+` must be of the same sequence type, and neither of them is modified but a new sequence of the same type is created as result of the concatenation.

To concatenate multiple copies of the same sequence, multiply it by an integer. Again, a new sequence is created:

```
>>> l = [1, 2, 3]
>>> l * 5
[1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
>>> 5 * 'abcd'
'abcdabcdabcdabcd'
```

Both + and * always create a new object, and never change their operands.



Beware of expressions like `a * n` when `a` is a sequence containing mutable items because the result may surprise you. For example, trying to initialize a list of lists as `my_list = [[]] * 3` will result in a list with three references to the same inner list, which is probably not what you want.

The next section covers the pitfalls of trying to use * to initialize a list of lists.

Building Lists of Lists

Sometimes we need to initialize a list with a certain number of nested lists—for example, to distribute students in a list of teams or to represent squares on a game board. The best way of doing so is with a list comprehension, as in [Example 2-12](#).

Example 2-12. A list with three lists of length 3 can represent a tic-tac-toe board

```
>>> board = [['_'] * 3 for i in range(3)] ❶
>>> board
[['_', '_', '_'], ['_', '_', '_'], ['_', '_', '_']]
>>> board[1][2] = 'X' ❷
>>> board
[['_', '_', '_'], ['_', '_', 'X'], ['_', '_', '_']]
```

- ❶ Create a list of three lists of three items each. Inspect the structure.
- ❷ Place a mark in row 1, column 2, and check the result.

A tempting but wrong shortcut is doing it like [Example 2-13](#).

Example 2-13. A list with three references to the same list is useless

```
>>> weird_board = [['_'] * 3] * 3 ❶
>>> weird_board
[['_', '_', '_'], ['_', '_', '_'], ['_', '_', '_']]
>>> weird_board[1][2] = 'O' ❷
>>> weird_board
[['_', '_', 'O'], ['_', '_', 'O'], ['_', '_', 'O']]
```

- ❶ The outer list is made of three references to the same inner list. While it is unchanged, all seems right.
- ❷ Placing a mark in row 1, column 2, reveals that all rows are aliases referring to the same object.

The problem with **Example 2-13** is that, in essence, it behaves like this code:

```
row = ['_'] * 3
board = []
for i in range(3):
    board.append(row) ❶
```

- ❶ The same row is appended three times to board.

On the other hand, the list comprehension from **Example 2-12** is equivalent to this code:

```
>>> board = []
>>> for i in range(3):
...     row = ['_'] * 3 # ❶
...     board.append(row)
...
>>> board
[['_', '_', '_'], ['_', '_', '_'], ['_', '_', '_']]
>>> board[2][0] = 'x'
>>> board # ❷
[['_', '_', '_'], ['_', '_', '_'], ['x', '_', '_']]
```

- ❶ Each iteration builds a new row and appends it to board.
❷ Only row 2 is changed, as expected.



If either the problem or the solution in this section are not clear to you, relax. **Chapter 8** was written to clarify the mechanics and pitfalls of references and mutable objects.

So far we have discussed the use of the plain + and * operators with sequences, but there are also the += and *= operators, which produce very different results depending on the mutability of the target sequence. The following section explains how that works.

Augmented Assignment with Sequences

The augmented assignment operators += and *= behave very differently depending on the first operand. To simplify the discussion, we will focus on augmented addition first (+=), but the concepts also apply to *= and to other augmented assignment operators.

The special method that makes += work is `__iadd__` (for “in-place addition”). However, if `__iadd__` is not implemented, Python falls back to calling `__add__`. Consider this simple expression:

```
>>> a += b
```

If `a` implements `__iadd__`, that will be called. In the case of mutable sequences (e.g., `list`, `bytearray`, `array.array`), `a` will be changed in place (i.e., the effect will be similar to `a.extend(b)`). However, when `a` does not implement `__iadd__`, the expression `a += b` has the same effect as `a = a + b`: the expression `a + b` is evaluated first, producing a new object, which is then bound to `a`. In other words, the identity of the object bound to `a` may or may not change, depending on the availability of `__iadd__`.

In general, for mutable sequences, it is a good bet that `__iadd__` is implemented and that `+=` happens in place. For immutable sequences, clearly there is no way for that to happen.

What I just wrote about `+=` also applies to `*=`, which is implemented via `__imul__`. The `__iadd__` and `__imul__` special methods are discussed in [Chapter 13](#).

Here is a demonstration of `*=` with a mutable sequence and then an immutable one:

```
>>> l = [1, 2, 3]
>>> id(l)
4311953800 ❶
>>> l *= 2
>>> l
[1, 2, 3, 1, 2, 3]
>>> id(l)
4311953800 ❷
>>> t = (1, 2, 3)
>>> id(t)
4312681568 ❸
>>> t *= 2
>>> id(t)
4301348296 ❹
```

- ❶ ID of the initial list
- ❷ After multiplication, the list is the same object, with new items appended
- ❸ ID of the initial tuple
- ❹ After multiplication, a new tuple was created

Repeated concatenation of immutable sequences is inefficient, because instead of just appending new items, the interpreter has to copy the whole target sequence to create a new one with the new items concatenated.³

We've seen common use cases for `+=`. The next section shows an intriguing corner case that highlights what “immutable” really means in the context of tuples.

3. `str` is an exception to this description. Because string building with `+=` in loops is so common in the wild, CPython is optimized for this use case. `str` instances are allocated in memory with room to spare, so that concatenation does not require copying the whole string every time.

A += Assignment Puzzler

Try to answer without using the console: what is the result of evaluating the two expressions in [Example 2-14](#)?⁴

Example 2-14. A riddle

```
>>> t = (1, 2, [30, 40])
>>> t[2] += [50, 60]
```

What happens next? Choose the best answer:

- a. `t` becomes `(1, 2, [30, 40, 50, 60])`.
- b. `TypeError` is raised with the message `'tuple' object does not support item assignment`.
- c. Neither.
- d. Both **a** and **b**.

When I saw this, I was pretty sure the answer was **b**, but it's actually **d**, “Both **a** and **b**.”! [Example 2-15](#) is the actual output from a Python 3.4 console (actually the result is the same in a Python 2.7 console).⁵

Example 2-15. The unexpected result: item `t2` is changed and an exception is raised

```
>>> t = (1, 2, [30, 40])
>>> t[2] += [50, 60]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> t
(1, 2, [30, 40, 50, 60])
```

[Online Python Tutor](#) is an awesome online tool to visualize how Python works in detail. [Figure 2-3](#) is a composite of two screenshots showing the initial and final states of the tuple `t` from [Example 2-15](#).

- 4. Thanks to Leonardo Rochaël and Cesar Kawakami for sharing this riddle at the 2013 PythonBrasil Conference.
- 5. A reader suggested that the operation in the example can be performed with `t[2].extend([50,60])`, without errors. We're aware of that, but the intent of the example is to discuss the odd behavior of the `+=` operator.

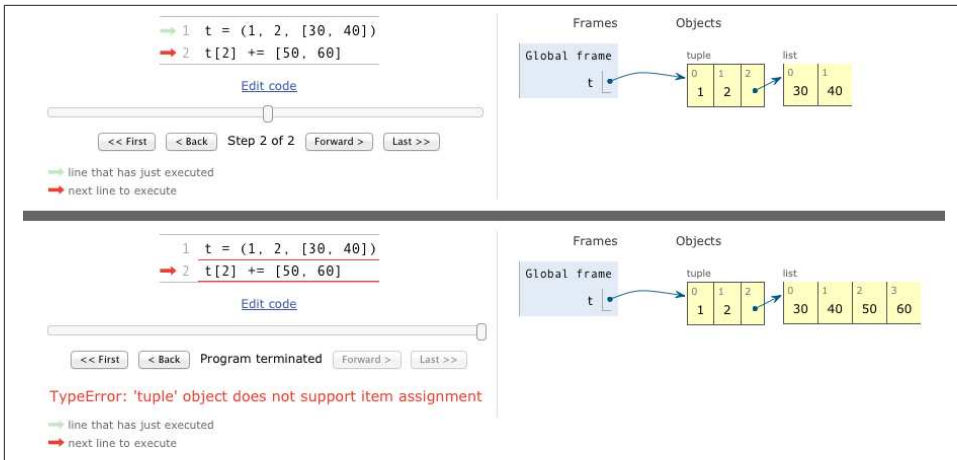


Figure 2-3. Initial and final state of the tuple assignment puzzler (diagram generated by Online Python Tutor)

If you look at the bytecode Python generates for the expression `s[a] += b` (Example 2-16), it becomes clear how that happens.

Example 2-16. Bytecode for the expression `s[a] += b`

```
>>> dis.dis('s[a] += b')
1      0 LOAD_NAME                0 (s)
      3 LOAD_NAME                1 (a)
      6 DUP_TOP_TWO
      7 BINARY_SUBSCR
      8 LOAD_NAME                2 (b)
     11 INPLACE_ADD
     12 ROT_THREE
     13 STORE_SUBSCR
     14 LOAD_CONST               0 (None)
     17 RETURN_VALUE
```

- ❶ Put the value of `s[a]` on TOS (Top Of Stack).
- ❷ Perform `TOS += b`. This succeeds if TOS refers to a mutable object (it's a list, in Example 2-15).
- ❸ Assign `s[a] = TOS`. This fails if `s` is immutable (the `t` tuple in Example 2-15).

This example is quite a corner case—in 15 years of using Python, I have never seen this strange behavior actually bite somebody.

I take three lessons from this:

- Putting mutable items in tuples is not a good idea.

- Augmented assignment is not an atomic operation—we just saw it throwing an exception after doing part of its job.
- Inspecting Python bytecode is not too difficult, and is often helpful to see what is going on under the hood.

After witnessing the subtleties of using `+` and `*` for concatenation, we can change the subject to another essential operation with sequences: sorting.

list.sort and the sorted Built-In Function

The `list.sort` method sorts a list in place—that is, without making a copy. It returns `None` to remind us that it changes the target object, and does not create a new list. This is an important Python API convention: functions or methods that change an object in place should return `None` to make it clear to the caller that the object itself was changed, and no new object was created. The same behavior can be seen, for example, in the `random.shuffle` function.



The convention of returning `None` to signal in-place changes has a drawback: you cannot cascade calls to those methods. In contrast, methods that return new objects (e.g., all `str` methods) can be cascaded in the fluent interface style. See Wikipedia’s [Wikipedia’s “Fluent interface” entry](#) for further description of this topic.

In contrast, the built-in function `sorted` creates a new list and returns it. In fact, it accepts any iterable object as an argument, including immutable sequences and generators (see [Chapter 14](#)). Regardless of the type of iterable given to `sorted`, it always returns a newly created list.

Both `list.sort` and `sorted` take two optional, keyword-only arguments:

`reverse`

If `True`, the items are returned in descending order (i.e., by reversing the comparison of the items). The default is `False`.

`key`

A one-argument function that will be applied to each item to produce its sorting key. For example, when sorting a list of strings, `key=str.lower` can be used to perform a case-insensitive sort, and `key=len` will sort the strings by character length. The default is the identity function (i.e., the items themselves are compared).



The key optional keyword parameter can also be used with the `min()` and `max()` built-ins and with other functions from the standard library (e.g., `itertools.groupby()` and `heapq.nlargest()`).

Here are a few examples to clarify the use of these functions and keyword arguments⁶:

```
>>> fruits = ['grape', 'raspberry', 'apple', 'banana']
>>> sorted(fruits)
['apple', 'banana', 'grape', 'raspberry'] ❶
>>> fruits
['grape', 'raspberry', 'apple', 'banana'] ❷
>>> sorted(fruits, reverse=True)
['raspberry', 'grape', 'banana', 'apple'] ❸
>>> sorted(fruits, key=len)
['grape', 'apple', 'banana', 'raspberry'] ❹
>>> sorted(fruits, key=len, reverse=True)
['raspberry', 'banana', 'grape', 'apple'] ❺
>>> fruits
['grape', 'raspberry', 'apple', 'banana'] ❻
>>> fruits.sort() ❼
>>> fruits
['apple', 'banana', 'grape', 'raspberry'] ❽
```

- ❶ This produces a new list of strings sorted alphabetically.
- ❷ Inspecting the original list, we see it is unchanged.
- ❸ This is simply reverse alphabetical ordering.
- ❹ A new list of strings, now sorted by length. Because the sorting algorithm is stable, “grape” and “apple,” both of length 5, are in the original order.
- ❺ These are the strings sorted in descending order of length. It is not the reverse of the previous result because the sorting is stable, so again “grape” appears before “apple.”
- ❻ So far, the ordering of the original `fruits` list has not changed.
- ❼ This sorts the list in place, and returns `None` (which the console omits).
- ❽ Now `fruits` is sorted.

Once your sequences are sorted, they can be very efficiently searched. Fortunately, the standard binary search algorithm is already provided in the `bisect` module of the Python standard library. We discuss its essential features next, including the convenient

6. The examples also demonstrate that Timsort—the sorting algorithm used in Python—is stable (i.e., it preserves the relative ordering of items that compare equal). Timsort is discussed further in the “Soapbox” sidebar at the end of this chapter.

`bisect.insert` function, which you can use to make sure that your sorted sequences stay sorted.

Managing Ordered Sequences with `bisect`

The `bisect` module offers two main functions—`bisect` and `insert`—that use the binary search algorithm to quickly find and insert items in any sorted sequence.

Searching with `bisect`

`bisect(haystack, needle)` does a binary search for `needle` in `haystack`—which must be a sorted sequence—to locate the position where `needle` can be inserted while maintaining `haystack` in ascending order. In other words, all items appearing up to that position are less than or equal to `needle`. You could use the result of `bisect(haystack, needle)` as the `index` argument to `haystack.insert(index, needle)`—however, using `insert` does both steps, and is faster.



Raymond Hettinger—a prolific Python contributor—has a [Sorted Collection recipe](#) that leverages the `bisect` module but is easier to use than these standalone functions.

Example 2-17 uses a carefully chosen set of “needles” to demonstrate the insert positions returned by `bisect`. Its output is in [Figure 2-4](#).

Example 2-17. `bisect` finds insertion points for items in a sorted sequence

```
import bisect
import sys

HAYSTACK = [1, 4, 5, 6, 8, 12, 15, 20, 21, 23, 23, 26, 29, 30]
NEEDLES = [0, 1, 2, 5, 8, 10, 22, 23, 29, 30, 31]

ROW_FMT = '{0:2d} @ {1:2d}    {2}{0:<2d}'

def demo(bisect_fn):
    for needle in reversed(NEEDLES):
        position = bisect_fn(HAYSTACK, needle)  ❶
        offset = position * ' | '               ❷
        print(ROW_FMT.format(needle, position, offset))  ❸

if __name__ == '__main__':
    if sys.argv[-1] == 'left': ❹
        bisect_fn = bisect.bisect_left
    else:
```

```

bisect_fn = bisect.bisect

print('DEMO:', bisect_fn.__name__) ❸
print('haystack ->', ' '.join('%2d' % n for n in HAYSTACK))
demo(bisect_fn)

```

- ❶ Use the chosen bisect function to get the insertion point.
- ❷ Build a pattern of vertical bars proportional to the offset.
- ❸ Print formatted row showing needle and insertion point.
- ❹ Choose the bisect function to use according to the last command-line argument.
- ❺ Print header with name of function selected.

```

02-array-seq/ $ python3 bisect_demo.py
DEMO: bisect
haystack -> 1 4 5 6 8 12 15 20 21 23 23 26 29 30
31 @ 14      | | | | | | | | | | | | | | 31
30 @ 14      | | | | | | | | | | | | | | 30
29 @ 13      | | | | | | | | | | | | | | 29
23 @ 11      | | | | | | | | | | | | | | 23
22 @ 9       | | | | | | | | | | | | | | 22
10 @ 5       | | | | | 10
 8 @ 5       | | | | | 8
 5 @ 3       | | | 5
 2 @ 1       | 2
 1 @ 1       | 1
 0 @ 0       0

```

Figure 2-4. Output of *Example 2-17* with `bisect` in use—each row starts with the notation `needle @ position` and the needle value appears again below its insertion point in the haystack

The behavior of `bisect` can be fine-tuned in two ways.

First, a pair of optional arguments, `lo` and `hi`, allow narrowing the region in the sequence to be searched when inserting. `lo` defaults to 0 and `hi` to the `len()` of the sequence.

Second, `bisect` is actually an alias for `bisect_right`, and there is a sister function called `bisect_left`. Their difference is apparent only when the needle compares equal to an item in the list: `bisect_right` returns an insertion point after the existing item, and `bisect_left` returns the position of the existing item, so insertion would occur before

it. With simple types like `int` this makes no difference, but if the sequence contains objects that are distinct yet compare equal, then it may be relevant. For example, `1` and `1.0` are distinct, but `1 == 1.0` is `True`. [Figure 2-5](#) shows the result of using `bisect_left`.

```
02-array-seq/ $ python3 bisect_demo.py left
DEMO: bisect_left
haystack -> 1 4 5 6 8 12 15 20 21 23 23 26 29 30
31 @ 14      | | | | | | | | | | | | | 31
30 @ 13      | | | | | | | | | | | | | 30
29 @ 12      | | | | | | | | | | | | | 29
23 @ 9       | | | | | | | | | | | | | 23
22 @ 9       | | | | | | | | | | | | | 22
10 @ 5       | | | | | 10
8 @ 4        | | | 8
5 @ 2        | 5
2 @ 1        | 2
1 @ 0        1
0 @ 0        0
```

Figure 2-5. Output of [Example 2-17](#) with `bisect_left` in use (compare with [Figure 2-4](#) and note the insertion points for the values 1, 8, 23, 29, and 30 to the left of the same numbers in the haystack).

An interesting application of `bisect` is to perform table lookups by numeric values—for example, to convert test scores to letter grades, as in [Example 2-18](#).

Example 2-18. Given a test score, `grade` returns the corresponding letter grade

```
>>> def grade(score, breakpoints=[60, 70, 80, 90], grades='FDCBA'):
...     i = bisect.bisect(breakpoints, score)
...     return grades[i]
...
>>> [grade(score) for score in [33, 99, 77, 70, 89, 90, 100]]
['F', 'A', 'C', 'C', 'B', 'A', 'A']
```

The code in [Example 2-18](#) is from the [bisect module documentation](#), which also lists functions to use `bisect` as a faster replacement for the `index` method when searching through long ordered sequences of numbers.

These functions are not only used for searching, but also for inserting items in sorted sequences, as the following section shows.

Inserting with `bisect.insort`

Sorting is expensive, so once you have a sorted sequence, it's good to keep it that way. That is why `bisect.insort` was created.

`insort(seq, item)` inserts `item` into `seq` so as to keep `seq` in ascending order. See [Example 2-19](#) and its output in [Figure 2-6](#).

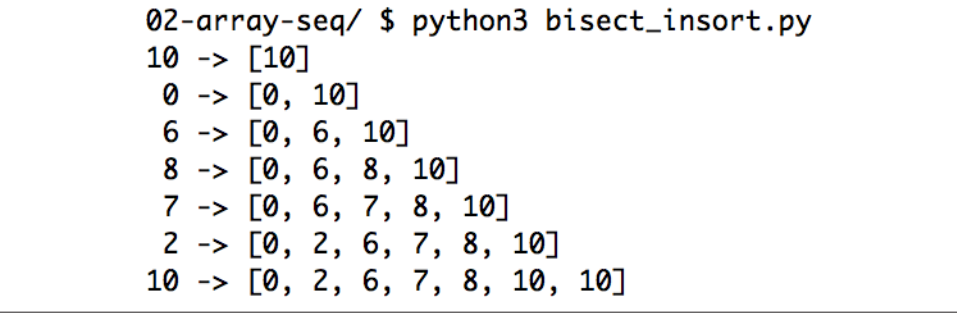
Example 2-19. Insert keeps a sorted sequence always sorted

```
import bisect
import random

SIZE = 7

random.seed(1729)

my_list = []
for i in range(SIZE):
    new_item = random.randrange(SIZE*2)
    bisect.insort(my_list, new_item)
    print('%2d ->' % new_item, my_list)
```



```
02-array-seq/ $ python3 bisect_insort.py
10 -> [10]
0 -> [0, 10]
6 -> [0, 6, 10]
8 -> [0, 6, 8, 10]
7 -> [0, 6, 7, 8, 10]
2 -> [0, 2, 6, 7, 8, 10]
10 -> [0, 2, 6, 7, 8, 10, 10]
```

Figure 2-6. Output of [Example 2-19](#)

Like `bisect`, `insort` takes optional `lo`, `hi` arguments to limit the search to a subsequence. There is also an `insort_left` variation that uses `bisect_left` to find insertion points.

Much of what we have seen so far in this chapter applies to sequences in general, not just lists or tuples. Python programmers sometimes overuse the `list` type because it is so handy—I know I've done it. If you are handling lists of numbers, arrays are the way to go. The remainder of the chapter is devoted to them.

When a List Is Not the Answer

The `list` type is flexible and easy to use, but depending on specific requirements, there are better options. For example, if you need to store 10 million floating-point values, an array is much more efficient, because an array does not actually hold full-fledged `float` objects, but only the packed bytes representing their machine values—just like an array in the C language. On the other hand, if you are constantly adding and removing items from the ends of a list as a FIFO or LIFO data structure, a deque (double-ended queue) works faster.



If your code does a lot of containment checks (e.g., `item in my_collection`), consider using a set for `my_collection`, especially if it holds a large number of items. Sets are optimized for fast membership checking. But they are not sequences (their content is unordered). We cover them in [Chapter 3](#).

For the remainder of this chapter, we discuss mutable sequence types that can replace lists in many cases, starting with arrays.

Arrays

If the list will only contain numbers, an `array.array` is more efficient than a `list`: it supports all mutable sequence operations (including `.pop`, `.insert`, and `.extend`), and additional methods for fast loading and saving such as `.frombytes` and `.tofile`.

A Python array is as lean as a C array. When creating an array, you provide a typecode, a letter to determine the underlying C type used to store each item in the array. For example, `b` is the typecode for signed `char`. If you create an `array('b')`, then each item will be stored in a single byte and interpreted as an integer from `-128` to `127`. For large sequences of numbers, this saves a lot of memory. And Python will not let you put any number that does not match the type for the array.

Example 2-20 shows creating, saving, and loading an array of 10 million floating-point random numbers.

Example 2-20. Creating, saving, and loading a large array of floats

```
>>> from array import array ❶
>>> from random import random
>>> floats = array('d', (random() for i in range(10**7))) ❷
>>> floats[-1] ❸
0.07802343889111107
>>> fp = open('floats.bin', 'wb')
>>> floats.tofile(fp) ❹
>>> fp.close()
>>> floats2 = array('d') ❺
```

```

>>> fp = open('floats.bin', 'rb')
>>> floats2.fromfile(fp, 10**7) ❸
>>> fp.close()
>>> floats2[-1] ❹
0.07802343889111107
>>> floats2 == floats ❺
True

```

- ❶ Import the array type.
- ❷ Create an array of double-precision floats (typecode 'd') from any iterable object—in this case, a generator expression.
- ❸ Inspect the last number in the array.
- ❹ Save the array to a binary file.
- ❺ Create an empty array of doubles.
- ❻ Read 10 million numbers from the binary file.
- ❼ Inspect the last number in the array.
- ❽ Verify that the contents of the arrays match.

As you can see, `array.tofile` and `array.fromfile` are easy to use. If you try the example, you'll notice they are also very fast. A quick experiment shows that it takes about 0.1s for `array.fromfile` to load 10 million double-precision floats from a binary file created with `array.tofile`. That is nearly 60 times faster than reading the numbers from a text file, which also involves parsing each line with the `float` built-in. Saving with `array.tofile` is about 7 times faster than writing one float per line in a text file. In addition, the size of the binary file with 10 million doubles is 80,000,000 bytes (8 bytes per double, zero overhead), while the text file has 181,515,739 bytes, for the same data.



Another fast and more flexible way of saving numeric data is the **pickle module** for object serialization. Saving an array of floats with `pickle.dump` is almost as fast as with `array.tofile`—however, `pickle` handles almost all built-in types, including complex numbers, nested collections, and even instances of user-defined classes automatically (if they are not too tricky in their implementation).

For the specific case of numeric arrays representing binary data, such as raster images, Python has the `bytes` and `bytearray` types discussed in [Chapter 4](#).

We wrap up this section on arrays with [Table 2-2](#), comparing the features of `list` and `array.array`.

Table 2-2. Methods and attributes found in list or array (deprecated array methods and those also implemented by object were omitted for brevity)

	list	array
<code>s.__add__(s2)</code>	•	• <code>s + s2</code> —concatenation
<code>s.__iadd__(s2)</code>	•	• <code>s += s2</code> —in-place concatenation
<code>s.append(e)</code>	•	• Append one element after last
<code>s.byteswap()</code>		• Swap bytes of all items in array for endianness conversion
<code>s.clear()</code>	•	• Delete all items
<code>s.__contains__(e)</code>	•	• <code>e in s</code>
<code>s.copy()</code>	•	• Shallow copy of the list
<code>s.__copy__()</code>		• Support for <code>copy.copy</code>
<code>s.count(e)</code>	•	• Count occurrences of an element
<code>s.__deepcopy__()</code>		• Optimized support for <code>copy.deepcopy</code>
<code>s.__delitem__(p)</code>	•	• Remove item at position <code>p</code>
<code>s.extend(it)</code>	•	• Append items from iterable <code>it</code>
<code>s.frombytes(b)</code>		• Append items from byte sequence interpreted as packed machine values
<code>s.fromfile(f, n)</code>		• Append <code>n</code> items from binary file <code>f</code> interpreted as packed machine values
<code>s.fromlist(l)</code>		• Append items from list; if one causes <code>TypeError</code> , none are appended
<code>s.__getitem__(p)</code>	•	• <code>s[p]</code> —get item at position
<code>s.index(e)</code>	•	• Find position of first occurrence of <code>e</code>
<code>s.insert(p, e)</code>	•	• Insert element <code>e</code> before the item at position <code>p</code>
<code>s.itemsize</code>		• Length in bytes of each array item
<code>s.__iter__()</code>	•	• Get iterator
<code>s.__len__()</code>	•	• <code>len(s)</code> —number of items
<code>s.__mul__(n)</code>	•	• <code>s * n</code> —repeated concatenation
<code>s.__imul__(n)</code>	•	• <code>s *= n</code> —in-place repeated concatenation
<code>s.__rmul__(n)</code>	•	• <code>n * s</code> —reversed repeated concatenation ^a
<code>s.pop([p])</code>	•	• Remove and return item at position <code>p</code> (default: last)
<code>s.remove(e)</code>	•	• Remove first occurrence of element <code>e</code> by value
<code>s.reverse()</code>	•	• Reverse the order of the items in place
<code>s.__reversed__()</code>	•	• Get iterator to scan items from last to first
<code>s.__setitem__(p, e)</code>	•	• <code>s[p] = e</code> —put <code>e</code> in position <code>p</code> , overwriting existing item
<code>s.sort([key], [reverse])</code>	•	• Sort items in place with optional keyword arguments <code>key</code> and <code>reverse</code>
<code>s.tobytes()</code>		• Return items as packed machine values in a bytes object
<code>s.tofile(f)</code>		• Save items as packed machine values to binary file <code>f</code>
<code>s.tolist()</code>		• Return items as numeric objects in a list

^a Reversed operators are explained in [Chapter 13](#).



As of Python 3.4, the array type does not have an in-place sort method like `list.sort()`. If you need to sort an array, use the sorted function to rebuild it sorted:

```
a = array.array(a.typecode, sorted(a))
```

To keep a sorted array sorted while adding items to it, use the `bisect.insort` function (as seen in [“Inserting with bisect.insort” on page 47](#)).

If you do a lot of work with arrays and don’t know about `memoryview`, you’re missing out. See the next topic.

Memory Views

The built-in `memoryview` class is a shared-memory sequence type that lets you handle slices of arrays without copying bytes. It was inspired by the NumPy library (which we’ll discuss shortly in [“NumPy and SciPy” on page 52](#)). Travis Oliphant, lead author of NumPy, answers [When should a memoryview be used?](#) like this:

A `memoryview` is essentially a generalized NumPy array structure in Python itself (without the math). It allows you to share memory between data-structures (things like PIL images, SQLite databases, NumPy arrays, etc.) without first copying. This is very important for large data sets.

Using notation similar to the array module, the `memoryview.cast` method lets you change the way multiple bytes are read or written as units without moving bits around (just like the C cast operator). `memoryview.cast` returns yet another `memoryview` object, always sharing the same memory.

See [Example 2-21](#) for an example of changing a single byte of an array of 16-bit integers.

Example 2-21. Changing the value of an array item by poking one of its bytes

```
>>> numbers = array.array('h', [-2, -1, 0, 1, 2])
>>> memv = memoryview(numbers) ❶
>>> len(memv)
5
>>> memv[0] ❷
-2
>>> memv_oct = memv.cast('B') ❸
>>> memv_oct.tolist() ❹
[254, 255, 255, 255, 0, 0, 1, 0, 2, 0]
>>> memv_oct[5] = 4 ❺
```

```
>>> numbers
array('h', [-2, -1, 1024, 1, 2]) ❹
```

- ❶ Build `memoryview` from array of 5 short signed integers (typecode 'h').
- ❷ `memv` sees the same 5 items in the array.
- ❸ Create `memv_oct` by casting the elements of `memv` to typecode 'B' (unsigned char).
- ❹ Export elements of `memv_oct` as a list, for inspection.
- ❺ Assign value 4 to byte offset 5.
- ❻ Note change to `numbers`: a 4 in the most significant byte of a 2-byte unsigned integer is 1024.

We'll see another short example with `memoryview` in the context of binary sequence manipulations with `struct` ([Chapter 4, Example 4-4](#)).

Meanwhile, if you are doing advanced numeric processing in arrays, you should be using the NumPy and SciPy libraries. We'll take a brief look at them right away.

NumPy and SciPy

Throughout this book, I make a point of highlighting what is already in the Python standard library so you can make the most of it. But NumPy and SciPy are so awesome that a detour is warranted.

For advanced array and matrix operations, NumPy and SciPy are the reason why Python became mainstream in scientific computing applications. NumPy implements multi-dimensional, homogeneous arrays and matrix types that hold not only numbers but also user-defined records, and provides efficient elementwise operations.

SciPy is a library, written on top of NumPy, offering many scientific computing algorithms from linear algebra, numerical calculus, and statistics. SciPy is fast and reliable because it leverages the widely used C and Fortran code base from the [Netlib Repository](#). In other words, SciPy gives scientists the best of both worlds: an interactive prompt and high-level Python APIs, together with industrial-strength number-crunching functions optimized in C and Fortran.

As a very brief demo, [Example 2-22](#) shows some basic operations with two-dimensional arrays in NumPy.

Example 2-22. Basic operations with rows and columns in a `numpy.ndarray`

```
>>> import numpy ❶
>>> a = numpy.arange(12) ❷
>>> a
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```

>>> type(a)
<class 'numpy.ndarray'>
>>> a.shape ❸
(12,)
>>> a.shape = 3, 4 ❹
>>> a
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
>>> a[2] ❺
array([ 8,  9, 10, 11])
>>> a[2, 1] ❻
9
>>> a[:, 1] ❼
array([1, 5, 9]) ❽
>>> a.transpose()
array([[ 0,  4,  8],
       [ 1,  5,  9],
       [ 2,  6, 10],
       [ 3,  7, 11]])

```

- ❶ Import Numpy, after installing (it's not in the Python standard library).
- ❷ Build and inspect a `numpy.ndarray` with integers 0 to 11.
- ❸ Inspect the dimensions of the array: this is a one-dimensional, 12-element array.
- ❹ Change the shape of the array, adding one dimension, then inspecting the result.
- ❺ Get row at index 2.
- ❻ Get element at index 2, 1.
- ❼ Get column at index 1.
- ❽ Create a new array by transposing (swapping columns with rows).

NumPy also supports high-level operations for loading, saving, and operating on all elements of a `numpy.ndarray`:

```

>>> import numpy
>>> floats = numpy.loadtxt('floats-10M-lines.txt') ❶
>>> floats[-3:] ❷
array([ 3016362.69195522,  535281.10514262,  4566560.44373946])
>>> floats *= .5 ❸
>>> floats[-3:]
array([ 1508181.34597761,  267640.55257131,  2283280.22186973])
>>> from time import perf_counter as pc ❹
>>> t0 = pc(); floats /= 3; pc() - t0 ❺
0.03690556302899495
>>> numpy.save('floats-10M', floats) ❻
>>> floats2 = numpy.load('floats-10M.npy', 'r+') ❼
>>> floats2 *= 6

```

```
>>> floats2[-3:] ⑧  
memmap([ 3016362.69195522, 535281.10514262, 4566560.44373946])
```

- ❶ Load 10 million floating-point numbers from a text file.
- ❷ Use sequence slicing notation to inspect the last three numbers.
- ❸ Multiply every element in the `floats` array by `.5` and inspect the last three elements again.
- ❹ Import the high-resolution performance measurement timer (available since Python 3.3).
- ❺ Divide every element by 3; the elapsed time for 10 million floats is less than 40 milliseconds.
- ❻ Save the array in a `.npy` binary file.
- ❼ Load the data as a memory-mapped file into another array; this allows efficient processing of slices of the array even if it does not fit entirely in memory.
- ❽ Inspect the last three elements after multiplying every element by 6.



Installing NumPy and SciPy from source is not a breeze. The **Installing the SciPy Stack** page on SciPy.org recommends using special scientific Python distributions such as Anaconda, Enthought Canopy, and WinPython, among others. These are large downloads, but come ready to use. Users of popular GNU/Linux distributions can usually find NumPy and SciPy in the standard package repositories. For example, installing them on Debian or Ubuntu is as easy as:

```
$ sudo apt-get install python-numpy python-sciipy
```

This was just an appetizer. NumPy and SciPy are formidable libraries, and are the foundation of other awesome tools such as the **Pandas** and **Blaze** data analysis libraries, which provide efficient array types that can hold nonnumeric data as well as import/export functions compatible with many different formats (e.g., `.csv`, `.xls`, SQL dumps, HDF5, etc.). These packages deserve entire books about them. This is not one of those books. But no overview of Python sequences would be complete without at least a quick look at NumPy arrays.

Having looked at flat sequences—standard arrays and NumPy arrays—we now turn to a completely different set of replacements for the plain old `list`: queues.

Dequeues and Other Queues

The `.append` and `.pop` methods make a list usable as a stack or a queue (if you use `.append` and `.pop(0)`, you get LIFO behavior). But inserting and removing from the left of a list (the 0-index end) is costly because the entire list must be shifted.

The class `collections.deque` is a thread-safe double-ended queue designed for fast inserting and removing from both ends. It is also the way to go if you need to keep a list of “last seen items” or something like that, because a deque can be bounded—i.e., created with a maximum length—and then, when it is full, it discards items from the opposite end when you append new ones. **Example 2-23** shows some typical operations performed on a deque.

Example 2-23. Working with a deque

```
>>> from collections import deque
>>> dq = deque(range(10), maxlen=10) ❶
>>> dq
deque([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], maxlen=10)
>>> dq.rotate(3) ❷
>>> dq
deque([7, 8, 9, 0, 1, 2, 3, 4, 5, 6], maxlen=10)
>>> dq.rotate(-4)
>>> dq
deque([1, 2, 3, 4, 5, 6, 7, 8, 9, 0], maxlen=10)
>>> dq.appendleft(-1) ❸
>>> dq
deque([-1, 1, 2, 3, 4, 5, 6, 7, 8, 9], maxlen=10)
>>> dq.extend([11, 22, 33]) ❹
>>> dq
deque([3, 4, 5, 6, 7, 8, 9, 11, 22, 33], maxlen=10)
>>> dq.extendleft([10, 20, 30, 40]) ❺
>>> dq
deque([40, 30, 20, 10, 3, 4, 5, 6, 7, 8], maxlen=10)
```

- ❶ The optional `maxlen` argument sets the maximum number of items allowed in this instance of deque; this sets a read-only `maxlen` instance attribute.
- ❷ Rotating with `n > 0` takes items from the right end and prepends them to the left; when `n < 0` items are taken from left and appended to the right.
- ❸ Appending to a deque that is full (`len(d) == d.maxlen`) discards items from the other end; note in the next line that the 0 is dropped.
- ❹ Adding three items to the right pushes out the leftmost -1, 1, and 2.
- ❺ Note that `extendleft(iter)` works by appending each successive item of the `iter` argument to the left of the deque, therefore the final position of the items is reversed.

Table 2-3 compares the methods that are specific to `list` and `deque` (removing those that also appear in `object`).

Note that `deque` implements most of the `list` methods, and adds a few specific to its design, like `popleft` and `rotate`. But there is a hidden cost: removing items from the middle of a `deque` is not as fast. It is really optimized for appending and popping from the ends.

The `append` and `popleft` operations are atomic, so `deque` is safe to use as a LIFO queue in multithreaded applications without the need for using locks.

Table 2-3. Methods implemented in `list` or `deque` (those that are also implemented by `object` were omitted for brevity)

	list	deque
<code>s.__add__(s2)</code>	•	<code>s + s2</code> —concatenation
<code>s.__iadd__(s2)</code>	• •	<code>s += s2</code> —in-place concatenation
<code>s.append(e)</code>	• •	Append one element to the right (after last)
<code>s.appendleft(e)</code>		• Append one element to the left (before first)
<code>s.clear()</code>	• •	Delete all items
<code>s.__contains__(e)</code>	•	<code>e in s</code>
<code>s.copy()</code>	•	Shallow copy of the list
<code>s.__copy__()</code>		• Support for <code>copy.copy</code> (shallow copy)
<code>s.count(e)</code>	• •	Count occurrences of an element
<code>s.__delitem__(p)</code>	• •	Remove item at position <code>p</code>
<code>s.extend(i)</code>	• •	Append items from iterable <code>i</code> to the right
<code>s.extendleft(i)</code>		• Append items from iterable <code>i</code> to the left
<code>s.__getitem__(p)</code>	• •	<code>s[p]</code> —get item at position
<code>s.index(e)</code>	•	Find position of first occurrence of <code>e</code>
<code>s.insert(p, e)</code>	•	Insert element <code>e</code> before the item at position <code>p</code>
<code>s.__iter__()</code>	• •	Get iterator
<code>s.__len__()</code>	• •	<code>len(s)</code> —number of items
<code>s.__mul__(n)</code>	•	<code>s * n</code> —repeated concatenation
<code>s.__imul__(n)</code>	•	<code>s *= n</code> —in-place repeated concatenation
<code>s.__rmul__(n)</code>	•	<code>n * s</code> —reversed repeated concatenation ^a
<code>s.pop()</code>	• •	Remove and return last item ^b
<code>s.popleft()</code>		• Remove and return first item
<code>s.remove(e)</code>	• •	Remove first occurrence of element <code>e</code> by value
<code>s.reverse()</code>	• •	Reverse the order of the items in place
<code>s.__reversed__()</code>	• •	Get iterator to scan items from last to first

	list	deque
<code>s.rotate(n)</code>	•	Move <i>n</i> items from one end to the other
<code>s.__setitem__(p, e)</code>	• •	<code>s[p] = e</code> —put <i>e</i> in position <i>p</i> , overwriting existing item
<code>s.sort([key], [reverse])</code>	•	Sort items in place with optional keyword arguments <i>key</i> and <i>reverse</i>

^a Reversed operators are explained in [Chapter 13](#).

^b `a_list.pop(p)` allows removing from position *p* but `deque` does not support that option.

Besides `deque`, other Python standard library packages implement queues:

queue

This provides the synchronized (i.e., thread-safe) classes `Queue`, `LifoQueue`, and `PriorityQueue`. These are used for safe communication between threads. All three classes can be bounded by providing a `maxsize` argument greater than 0 to the constructor. However, they don't discard items to make room as `deque` does. Instead, when the queue is full the insertion of a new item blocks—i.e., it waits until some other thread makes room by taking an item from the queue, which is useful to throttle the number of live threads.

multiprocessing

Implements its own bounded `Queue`, very similar to `queue.Queue` but designed for interprocess communication. A specialized `multiprocessing.JoinableQueue` is also available for easier task management.

asyncio

Newly added to Python 3.4, `asyncio` provides `Queue`, `LifoQueue`, `PriorityQueue`, and `JoinableQueue` with APIs inspired by the classes contained in the `queue` and `multiprocessing` modules, but adapted for managing tasks in asynchronous programming.

heapq

In contrast to the previous three modules, `heapq` does not implement a queue class, but provides functions like `heappush` and `heappop` that let you use a mutable sequence as a heap queue or priority queue.

This ends our overview of alternatives to the `list` type, and also our exploration of sequence types in general—except for the particulars of `str` and binary sequences, which have their own chapter ([Chapter 4](#)).

Chapter Summary

Mastering the standard library sequence types is a prerequisite for writing concise, effective, and idiomatic Python code.

Python sequences are often categorized as mutable or immutable, but it is also useful to consider a different axis: flat sequences and container sequences. The former are more compact, faster, and easier to use, but are limited to storing atomic data such as numbers, characters, and bytes. Container sequences are more flexible, but may surprise you when they hold mutable objects, so you need to be careful to use them correctly with nested data structures.

List comprehensions and generator expressions are powerful notations to build and initialize sequences. If you are not yet comfortable with them, take the time to master their basic usage. It is not hard, and soon you will be hooked.

Tuples in Python play two roles: as records with unnamed fields and as immutable lists. When a tuple is used as a record, tuple unpacking is the safest, most readable way of getting at the fields. The new `*` syntax makes tuple unpacking even better by making it easier to ignore some fields and to deal with optional fields. Named tuples are not so new, but deserve more attention: like tuples, they have very little overhead per instance, yet provide convenient access to the fields by name and a handy `._asdict()` to export the record as an `OrderedDict`.

Sequence slicing is a favorite Python syntax feature, and it is even more powerful than many realize. Multidimensional slicing and ellipsis (`...`) notation, as used in NumPy, may also be supported by user-defined sequences. Assigning to slices is a very expressive way of editing mutable sequences.

Repeated concatenation as in `seq * n` is convenient and, with care, can be used to initialize lists of lists containing immutable items. Augmented assignment with `+=` and `*=` behaves differently for mutable and immutable sequences. In the latter case, these operators necessarily build new sequences. But if the target sequence is mutable, it is usually changed in place—but not always, depending on how the sequence is implemented.

The `sort` method and the `sorted` built-in function are easy to use and flexible, thanks to the key optional argument they accept, with a function to calculate the ordering criterion. By the way, `key` can also be used with the `min` and `max` built-in functions. To keep a sorted sequence in order, always insert items into it using `bisect.insort`; to search it efficiently, use `bisect.bisect`.

Beyond lists and tuples, the Python standard library provides `array.array`. Although NumPy and SciPy are not part of the standard library, if you do any kind of numerical processing on large sets of data, studying even a small part of these libraries can take you a long way.

We closed by visiting the versatile and thread-safe `collections.deque`, comparing its API with that of `list` in Table 2-3 and mentioning other queue implementations in the standard library.

Further Reading

Chapter 1, “Data Structures” of *Python Cookbook, 3rd Edition* (O’Reilly) by David Beazley and Brian K. Jones has many recipes focusing on sequences, including “Recipe 1.11. Naming a Slice,” from which I learned the trick of assigning slices to variables to improve readability, illustrated in our [Example 2-11](#).

The second edition of *Python Cookbook* was written for Python 2.4, but much of its code works with Python 3, and a lot of the recipes in Chapters 5 and 6 deal with sequences. The book was edited by Alex Martelli, Anna Martelli Ravenscroft, and David Ascher, and it includes contributions by dozens of Pythonistas. The third edition was rewritten from scratch, and focuses more on the semantics of the language—particularly what has changed in Python 3—while the older volume emphasizes pragmatics (i.e., how to apply the language to real-world problems). Even though some of the second edition solutions are no longer the best approach, I honestly think it is worthwhile to have both editions of *Python Cookbook* on hand.

The official Python [Sorting HOW TO](#) has several examples of advanced tricks for using sorted and `list.sort`.

[PEP 3132 — Extended Iterable Unpacking](#) is the canonical source to read about the new use of `*extra` as a target in parallel assignments. If you’d like a glimpse of Python evolving, [Missing *-unpacking generalizations](#) is a bug tracker issue proposing even wider use of iterable unpacking notation. [PEP 448 — Additional Unpacking Generalizations](#) resulted from the discussions in that issue. At the time of this writing, it seems likely the proposed changes will be merged to Python, perhaps in version 3.5.

Eli Bendersky’s blog post [“Less Copies in Python with the Buffer Protocol and memoryviews](#) includes a short tutorial on `memoryview`.

There are numerous books covering NumPy in the market, even some that don’t mention “NumPy” in the title. Wes McKinney’s *Python for Data Analysis* (O’Reilly) is one such title.

Scientists love the combination of an interactive prompt with the power of NumPy and SciPy so much that they developed IPython, an incredibly powerful replacement for the Python console that also provides a GUI, integrated inline graph plotting, literate programming support (interleaving text with code), and rendering to PDF. Interactive, multimedia IPython sessions can even be shared over HTTP as IPython notebooks. See screenshots and video at [The IPython Notebook](#). IPython is so hot that in 2012 its core developers, most of whom are researchers at UC Berkeley, received a \$1.15 million grant from the Sloan Foundation for enhancements to be implemented over the 2013–2014 period.

In The Python Standard Library, [8.3. collections — Container datatypes](#) includes short examples and practical recipes using deque (and other collections).

The best defense of the Python convention of excluding the last item in ranges and slices was written by Edsger W. Dijkstra himself, in a short memo titled “**Why Numbering Should Start at Zero**”. The subject of the memo is mathematical notation, but it’s relevant to Python because Prof. Dijkstra explains with rigor and humor why the sequence 2, 3, ..., 12 should always be expressed as $2 \leq i < 13$. All other reasonable conventions are refuted, as is the idea of letting each user choose a convention. The title refers to zero-based indexing, but the memo is really about why it is desirable that `'ABCDE'[1:3]` means `'BC'` and not `'BCD'` and why it makes perfect sense to write `2, 3, ..., 12` as `range(2, 13)`. (By the way, the memo is a handwritten note, but it’s beautiful and totally readable. Somebody should create a Dijkstra font—I’d buy it.)

Soapbox

The Nature of Tuples

In 2012, I presented a poster about the ABC language at PyCon US. Before creating Python, Guido had worked on the ABC interpreter, so he came to see my poster. Among other things, we talked about the ABC *compounds*, which are clearly the predecessors of Python tuples. Compounds also support parallel assignment and are used as composite keys in dictionaries (or *tables*, in ABC parlance). However, compounds are not sequences. They are not iterable and you cannot retrieve a field by index, much less slice them. You either handle the compound as whole or extract the individual fields using parallel assignment, that’s all.

I told Guido that these limitations make the main purpose of compounds very clear: they are just records without field names. His response: “Making tuples behave as sequences was a hack.”

This illustrates the pragmatic approach that makes Python so much better and more successful than ABC. From a language implementer perspective, making tuples behave as sequences costs little. As a result, tuples may not be as “conceptually pure” as compounds, but we have many more ways of using them. They can even be used as immutable lists, of all things!

It is really useful to have immutable lists in the language, even if their type is not called `frozenset` but is really tuple behaving as a sequence.

“Elegance Begets Simplicity”

The use of the syntax `*extra` to assign multiple items to a parameter started with function definitions a long time ago (I have a book about Python 1.4 from 1996 that covers that). Starting with Python 1.6, the form `*extra` can be used in the context of function calls to unpack an iterable into multiple arguments, a complementary operation. This is elegant, makes intuitive sense, and made the `apply` function redundant (it’s now gone). Now, with Python 3, the `*extra` notation also works on the left of parallel assignments to grab excess items, enhancing what was already a handy language feature.

With each of these changes, the language became more flexible, more consistent, and simpler at the same time. “Elegance begets simplicity” is the motto on my favorite PyCon T-shirt from Chicago, 2009. It is decorated with a painting by Bruce Eckel depicting hexagram 22 from the I Ching, 賁 (bi), “Adorning,” sometimes translated as “Grace” or “Beauty.”

Flat Versus Container Sequences

To highlight the different memory models of the sequence types, I used the terms *container sequence* and *flat sequence*. The “container” word is from [the Data Model documentation](#):

Some objects contain references to other objects; these are called containers.

I used the term “container sequence” to be specific, because there are containers in Python that are not sequences, like `dict` and `set`. Container sequences can be nested because they may contain objects of any type, including their own type.

On the other hand, *flat sequences* are sequence types that cannot be nested because they only hold simple atomic types like integers, floats, or characters.

I adopted the term *flat sequence* because I needed something to contrast with “container sequence.” I can’t cite a reference to support the use of *flat sequence* in this specific context: as the category of Python sequence types that are not containers. On Wikipedia, this usage would be tagged “original research.” I prefer to call it “our term,” hoping you’ll find it useful and adopt it too.

Mixed Bag Lists

Introductory Python texts emphasize that lists can contain objects of mixed types, but in practice that feature is not very useful: we put items in a list to process them later, which implies that all items should support at least some operation in common (i.e., they should all “quack” whether or not they are genetically 100% ducks). For example, you can’t sort a list in Python 3 unless the items in it are comparable:

```
>>> l = [28, 14, '28', 5, '9', '1', 0, 6, '23', 19]
>>> sorted(l)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unorderable types: str() < int()
```

Unlike lists, tuples often hold items of different types. That is natural, considering that each item in a tuple is really a field, and each field type is independent of the others.

Key Is Brilliant

The key optional argument of `list.sort`, `sorted`, `max`, and `min` is a great idea. Other languages force you to provide a two-argument comparison function like the deprecated `cmp(a, b)` function in Python 2. Using `key` is both simpler and more efficient. It’s simpler because you just define a one-argument function that retrieves or calculates whatever criterion you want to use to sort your objects; this is easier than writing a two-argument

function to return -1, 0, 1. It is also more efficient because the key function is invoked only once per item, while the two-argument comparison is called every time the sorting algorithm needs to compare two items. Of course, Python also has to compare the keys while sorting, but that comparison is done in optimized C code and not in a Python function that you wrote.

By the way, using `key` actually lets us sort a mixed bag of numbers and number-like strings. You just need to decide whether you want to treat all items as integers or strings:

```
>>> l = [28, 14, '28', 5, '9', '1', 0, 6, '23', 19]
>>> sorted(l, key=int)
[0, '1', 5, 6, '9', 14, 19, '23', 28, '28']
>>> sorted(l, key=str)
[0, '1', 14, 19, '23', 28, '28', 5, 6, '9']
```

Oracle, Google, and the Timbot Conspiracy

The sorting algorithm used in `sorted` and `list.sort` is Timsort, an adaptive algorithm that switches from insertion sort to merge sort strategies, depending on how ordered the data is. This is efficient because real-world data tends to have runs of sorted items. There is a [Wikipedia article](#) about it.

Timsort was first deployed in CPython, in 2002. Since 2009, Timsort is also used to sort arrays in both standard Java and Android, a fact that became widely known when Oracle used some of the code related to Timsort as evidence of Google infringement of Sun's intellectual property. See [Oracle v. Google - Day 14 Filings](#).

Timsort was invented by Tim Peters, a Python core developer so prolific that he is believed to be an AI, the Timbot. You can read about that conspiracy theory in [Python Humor](#). Tim also wrote `The Zen of Python: import this`.

FP Ch 3 | Dictionaries and Sets

Dictionaries and Sets

Any running Python program has many dictionaries active at the same time, even if the user's program code doesn't explicitly use a dictionary.

— A. M. Kuchling

Chapter 18, "Python's Dictionary Implementation"

The `dict` type is not only widely used in our programs but also a fundamental part of the Python implementation. Module namespaces, class and instance attributes, and function keyword arguments are some of the fundamental constructs where dictionaries are deployed. The built-in functions live in `__builtins__.__dict__`.

Because of their crucial role, Python dicts are highly optimized. *Hash tables* are the engines behind Python's high-performance dicts.

We also cover sets in this chapter because they are implemented with hash tables as well. Knowing how a hash table works is key to making the most of dictionaries and sets.

Here is a brief outline of this chapter:

- Common dictionary methods
- Special handling for missing keys
- Variations of `dict` in the standard library
- The `set` and `frozenset` types
- How hash tables work
- Implications of hash tables (key type limitations, unpredictable ordering, etc.)

Generic Mapping Types

The `collections.abc` module provides the `Mapping` and `MutableMapping` ABCs to formalize the interfaces of `dict` and similar types (in Python 2.6 to 3.2, these classes are imported from the `collections` module, and not from `collections.abc`). See [Figure 3-1](#).

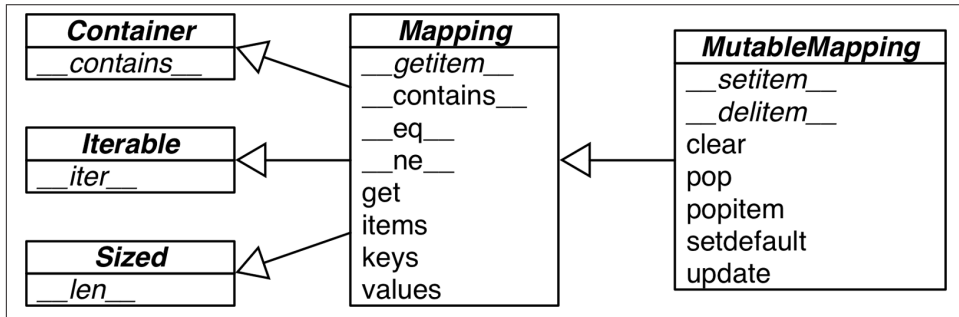


Figure 3-1. UML class diagram for the `MutableMapping` and its superclasses from `collections.abc` (inheritance arrows point from subclasses to superclasses; names in *italic* are abstract classes and abstract methods)

Implementations of specialized mappings often extend `dict` or `collections.UserDict`, instead of these ABCs. The main value of the ABCs is documenting and formalizing the minimal interfaces for mappings, and serving as criteria for `isinstance` tests in code that needs to support mappings in a broad sense:

```
>>> my_dict = {}
>>> isinstance(my_dict, abc.Mapping)
True
```

Using `isinstance` is better than checking whether a function argument is of `dict` type, because then alternative mapping types can be used.

All mapping types in the standard library use the basic `dict` in their implementation, so they share the limitation that the keys must be *hashable* (the values need not be hashable, only the keys).

What Is Hashable?

Here is part of the definition of hashable from the [Python Glossary](#):

An object is hashable if it has a hash value which never changes during its lifetime (it needs a `__hash__()` method), and can be compared to other objects (it needs an `__eq__()` method). Hashable objects which compare equal must have the same hash value. [...]

The atomic immutable types (str, bytes, numeric types) are all hashable. A frozen set is always hashable, because its elements must be hashable by definition. A tuple is hashable only if all its items are hashable. See tuples `tt`, `tl`, and `tf`:

```
>>> tt = (1, 2, (30, 40))
>>> hash(tt)
8027212646858338501
>>> tl = (1, 2, [30, 40])
>>> hash(tl)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
>>> tf = (1, 2, frozenset([30, 40]))
>>> hash(tf)
-4118419923444501110
```



At the time of this writing, the [Python Glossary](#) states: “All of Python’s immutable built-in objects are hashable” but that is inaccurate because a tuple is immutable, yet it may contain references to unhashable objects.

User-defined types are hashable by default because their hash value is their `id()` and they all compare not equal. If an object implements a custom `__eq__` that takes into account its internal state, it may be hashable only if all its attributes are immutable.

Given these ground rules, you can build dictionaries in several ways. The [Built-in Types](#) page in the Library Reference has this example to show the various means of building a dict:

```
>>> a = dict(one=1, two=2, three=3)
>>> b = {'one': 1, 'two': 2, 'three': 3}
>>> c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
>>> d = dict([('two', 2), ('one', 1), ('three', 3)])
>>> e = dict({'three': 3, 'one': 1, 'two': 2})
>>> a == b == c == d == e
True
```


In addition to the literal syntax and the flexible `dict` constructor, we can use *dict comprehensions* to build dictionaries. See the next section.

dict Comprehensions

Since Python 2.7, the syntax of listcomps and genexps was applied to `dict` comprehensions (and set comprehensions as well, which we'll soon visit). A *dictcomp* builds a `dict` instance by producing key:value pair from any iterable. **Example 3-1** shows the use of `dict` comprehensions to build two dictionaries from the same list of tuples.

Example 3-1. Examples of dict comprehensions

```
>>> DIAL_CODES = [ ❶
...     (86, 'China'),
...     (91, 'India'),
...     (1, 'United States'),
...     (62, 'Indonesia'),
...     (55, 'Brazil'),
...     (92, 'Pakistan'),
...     (880, 'Bangladesh'),
...     (234, 'Nigeria'),
...     (7, 'Russia'),
...     (81, 'Japan'),
... ]
>>> country_code = {country: code for code, country in DIAL_CODES} ❷
>>> country_code
{'China': 86, 'India': 91, 'Bangladesh': 880, 'United States': 1,
 'Pakistan': 92, 'Japan': 81, 'Russia': 7, 'Brazil': 55, 'Nigeria':
 234, 'Indonesia': 62}
>>> {code: country.upper() for country, code in country_code.items()} ❸
... if code < 66}
{1: 'UNITED STATES', 55: 'BRAZIL', 62: 'INDONESIA', 7: 'RUSSIA'}
```

- ❶ A list of pairs can be used directly with the `dict` constructor.
- ❷ Here the pairs are reversed: `country` is the key, and `code` is the value.
- ❸ Reversing the pairs again, values uppercased and items filtered by `code < 66`.

If you're used to liscomps, dictcomps are a natural next step. If you aren't, the spread of the listcomp syntax means it's now more profitable than ever to become fluent in it.

We now move to a panoramic view of the API for mappings.

Overview of Common Mapping Methods

The basic API for mappings is quite rich. **Table 3-1** shows the methods implemented by `dict` and two of its most useful variations: `defaultdict` and `OrderedDict`, both defined in the `collections` module.

Table 3-1. Methods of the mapping types `dict`, `collections.defaultdict`, and `collections.OrderedDict` (common object methods omitted for brevity); optional arguments are enclosed in [...]

	<code>dict</code>	<code>defaultdict</code>	<code>OrderedDict</code>	
<code>d.clear()</code>	•	•	•	Remove all items
<code>d.__contains__(k)</code>	•	•	•	<code>k in d</code>
<code>d.copy()</code>	•	•	•	Shallow copy
<code>d.__copy__()</code>		•		Support for <code>copy.copy</code>
<code>d.default_factory</code>		•		Callable invoked by <code>__missing__</code> to set missing values ^a
<code>d.__delitem__(k)</code>	•	•	•	<code>del d[k]</code> —remove item with key <code>k</code>
<code>d.fromkeys(it, [initial])</code>	•	•	•	New mapping from keys in iterable, with optional initial value (defaults to <code>None</code>)
<code>d.get(k, [default])</code>	•	•	•	Get item with key <code>k</code> , return <code>default</code> or <code>None</code> if missing
<code>d.__getitem__(k)</code>	•	•	•	<code>d[k]</code> —get item with key <code>k</code>
<code>d.items()</code>	•	•	•	Get <i>view</i> over items—(key, value) pairs
<code>d.__iter__()</code>	•	•	•	Get iterator over keys
<code>d.keys()</code>	•	•	•	Get <i>view</i> over keys
<code>d.__len__()</code>	•	•	•	<code>len(d)</code> —number of items
<code>d.__missing__(k)</code>		•		Called when <code>__getitem__</code> cannot find the key
<code>d.move_to_end(k, [last])</code>			•	Move <code>k</code> first or last position (<code>last</code> is <code>True</code> by default)
<code>d.pop(k, [default])</code>	•	•	•	Remove and return value at <code>k</code> , or <code>default</code> or <code>None</code> if missing
<code>d.popitem()</code>	•	•	•	Remove and return an arbitrary (key, value) item ^b
<code>d.__reversed__()</code>			•	Get iterator for keys from last to first inserted
<code>d.setdefault(k, [default])</code>	•	•	•	If <code>k in d</code> , return <code>d[k]</code> ; else set <code>d[k] = default</code> and return it
<code>d.__setitem__(k, v)</code>	•	•	•	<code>d[k] = v</code> —put <code>v</code> at <code>k</code>
<code>d.update(m, /**kargs)</code>	•	•	•	Update <code>d</code> with items from mapping or iterable of (key, value) pairs
<code>d.values()</code>	•	•	•	Get <i>view</i> over values

^a `default_factory` is not a method, but a callable instance attribute set by the end user when `defaultdict` is instantiated.

^b `OrderedDict.popitem()` removes the first item inserted (FIFO); an optional `last` argument, if set to `True`, pops the last item (LIFO).

The way `update` handles its first argument `m` is a prime example of *duck typing*: it first checks whether `m` has a `keys` method and, if it does, assumes it is a mapping. Otherwise, `update` falls back to iterating over `m`, assuming its items are `(key, value)` pairs. The constructor for most Python mappings uses the logic of `update` internally, which means they can be initialized from other mappings or from any iterable object producing `(key, value)` pairs.

A subtle mapping method is `setdefault`. We don't always need it, but when we do, it provides a significant speedup by avoiding redundant key lookups. If you are not comfortable using it, the following section explains how, through a practical example.

Handling Missing Keys with `setdefault`

In line with the *fail-fast* philosophy, `dict` access with `d[k]` raises an error when `k` is not an existing key. Every Pythonista knows that `d.get(k, default)` is an alternative to `d[k]` whenever a default value is more convenient than handling `KeyError`. However, when updating the value found (if it is mutable), using either `__getitem__` or `get` is awkward and inefficient. Consider [Example 3-2](#), a suboptimal script written just to show one case where `dict.get` is not the best way to handle a missing key.

[Example 3-2](#) is adapted from an example by Alex Martelli,¹ which generates an index like that in [Example 3-3](#).

Example 3-2. `index0.py` uses `dict.get` to fetch and update a list of word occurrences from the index (a better solution is in [Example 3-4](#))

```
"""Build an index mapping word -> list of occurrences"""
```

```
import sys
import re

WORD_RE = re.compile('\w+')

index = {}
with open(sys.argv[1], encoding='utf-8') as fp:
    for line_no, line in enumerate(fp, 1):
        for match in WORD_RE.finditer(line):
            word = match.group()
            column_no = match.start()+1
            location = (line_no, column_no)
            # this is ugly; coded like this to make a point
            occurrences = index.get(word, []) ❶
            occurrences.append(location)      ❷
            index[word] = occurrences        ❸
```

1. The original script appears in slide 41 of Martelli's “[Re-learning Python](#)” presentation. His script is actually a demonstration of `dict.setdefault`, as shown in our [Example 3-4](#).

```
# print in alphabetical order
for word in sorted(index, key=str.upper): ④
    print(word, index[word])
```

- ① Get the list of occurrences for word, or [] if not found.
- ② Append new location to occurrences.
- ③ Put changed occurrences into index dict; this entails a second search through the index.
- ④ In the key= argument of sorted I am not calling str.upper, just passing a reference to that method so the sorted function can use it to normalize the words for sorting.²

Example 3-3. Partial output from [Example 3-2](#) processing the Zen of Python; each line shows a word and a list of occurrences coded as pairs: (line-number, column-number)

```
$ python3 index0.py ../../data/zen.txt
a [(19, 48), (20, 53)]
Although [(11, 1), (16, 1), (18, 1)]
ambiguity [(14, 16)]
and [(15, 23)]
are [(21, 12)]
aren [(10, 15)]
at [(16, 38)]
bad [(19, 50)]
be [(15, 14), (16, 27), (20, 50)]
beats [(11, 23)]
Beautiful [(3, 1)]
better [(3, 14), (4, 13), (5, 11), (6, 12), (7, 9), (8, 11),
(17, 8), (18, 25)]
...
```

The three lines dealing with occurrences in [Example 3-2](#) can be replaced by a single line using dict.setdefault. [Example 3-4](#) is closer to Alex Martelli's original example.

Example 3-4. index.py uses dict.setdefault to fetch and update a list of word occurrences from the index in a single line; contrast with [Example 3-2](#)

"""Build an index mapping word -> list of occurrences"""

```
import sys
import re

WORD_RE = re.compile('\w+')

index = {}
```

2. This is an example of using a method as a first-class function, the subject of [Chapter 5](#).

```

with open(sys.argv[1], encoding='utf-8') as fp:
    for line_no, line in enumerate(fp, 1):
        for match in WORD_RE.finditer(line):
            word = match.group()
            column_no = match.start()+1
            location = (line_no, column_no)
            index.setdefault(word, []).append(location) ❶

# print in alphabetical order
for word in sorted(index, key=str.upper):
    print(word, index[word])

```

- ❶ Get the list of occurrences for word, or set it to [] if not found; `setdefault` returns the value, so it can be updated without requiring a second search.

In other words, the end result of this line...

```
my_dict.setdefault(key, []).append(new_value)
```

...is the same as running...

```

if key not in my_dict:
    my_dict[key] = []
my_dict[key].append(new_value)

```

...except that the latter code performs at least two searches for key—three if it's not found—while `setdefault` does it all with a single lookup.

A related issue, handling missing keys on any lookup (and not only when inserting), is the subject of the next section.

Mappings with Flexible Key Lookup

Sometimes it is convenient to have mappings that return some made-up value when a missing key is searched. There are two main approaches to this: one is to use a default dict instead of a plain dict. The other is to subclass dict or any other mapping type and add a `__missing__` method. Both solutions are covered next.

defaultdict: Another Take on Missing Keys

Example 3-5 uses `collections.defaultdict` to provide another elegant solution to the problem in **Example 3-4**. A `defaultdict` is configured to create items on demand whenever a missing key is searched.

Here is how it works: when instantiating a `defaultdict`, you provide a callable that is used to produce a default value whenever `__getitem__` is passed a nonexistent key argument.

For example, given an empty defaultdict created as `dd = defaultdict(list)`, if 'new-key' is not in `dd`, the expression `dd['new-key']` does the following steps:

1. Calls `list()` to create a new list.
2. Inserts the list into `dd` using 'new-key' as key.
3. Returns a reference to that list.

The callable that produces the default values is held in an instance attribute called `default_factory`.

Example 3-5. index_default.py: using an instance of defaultdict instead of the setdefault method

```
"""Build an index mapping word -> list of occurrences"""
```

```
import sys
import re
import collections

WORD_RE = re.compile('\w+')

index = collections.defaultdict(list) ❶
with open(sys.argv[1], encoding='utf-8') as fp:
    for line_no, line in enumerate(fp, 1):
        for match in WORD_RE.finditer(line):
            word = match.group()
            column_no = match.start()+1
            location = (line_no, column_no)
            index[word].append(location) ❷

# print in alphabetical order
for word in sorted(index, key=str.upper):
    print(word, index[word])
```

- ❶ Create a defaultdict with the list constructor as `default_factory`.
- ❷ If `word` is not initially in the `index`, the `default_factory` is called to produce the missing value, which in this case is an empty list that is then assigned to `index[word]` and returned, so the `.append(location)` operation always succeeds.

If no `default_factory` is provided, the usual `KeyError` is raised for missing keys.



The `default_factory` of a `defaultdict` is only invoked to provide default values for `__getitem__` calls, and not for the other methods. For example, if `dd` is a `defaultdict`, and `k` is a missing key, `dd[k]` will call the `default_factory` to create a default value, but `dd.get(k)` still returns `None`.

The mechanism that makes `defaultdict` work by calling `default_factory` is actually the `__missing__` special method, a feature supported by all standard mapping types that we discuss next.

The `__missing__` Method

Underlying the way mappings deal with missing keys is the aptly named `__missing__` method. This method is not defined in the base `dict` class, but `dict` is aware of it: if you subclass `dict` and provide a `__missing__` method, the standard `dict.__getitem__` will call it whenever a key is not found, instead of raising `KeyError`.



The `__missing__` method is just called by `__getitem__` (i.e., for the `d[k]` operator). The presence of a `__missing__` method has no effect on the behavior of other methods that look up keys, such as `get` or `__contains__` (which implements the `in` operator). This is why the `default_factory` of `defaultdict` works only with `__getitem__`, as noted in the warning at the end of the previous section.

Suppose you'd like a mapping where keys are converted to `str` when looked up. A concrete use case is the [Pingo.io](#) project, where a programmable board with GPIO pins (e.g., the Raspberry Pi or the Arduino) is represented by a board object with a `board.pins` attribute, which is a mapping of physical pin locations to pin objects, and the physical location may be just a number or a string like "A0" or "P9_12". For consistency, it is desirable that all keys in `board.pins` are strings, but it is also convenient that looking up `my_arduino.pin[13]` works as well, so beginners are not tripped when they want to blink the LED on pin 13 of their Arduinos. [Example 3-6](#) shows how such a mapping would work.

Example 3-6. When searching for a nonstring key, `StrKeyDict0` converts it to `str` when it is not found

Tests for item retrieval using `d[key]` notation::

```
>>> d = StrKeyDict0([( '2', 'two'), ( '4', 'four')])
>>> d[ '2' ]
'two'
```

```
>>> d[4]
'four'
>>> d[1]
Traceback (most recent call last):
...
KeyError: '1'
```

Tests for item retrieval using `d.get(key)` notation::

```
>>> d.get('2')
'two'
>>> d.get(4)
'four'
>>> d.get(1, 'N/A')
'N/A'
```

Tests for the `in` operator::

```
>>> 2 in d
True
>>> 1 in d
False
```

Example 3-7 implements a class `StrKeyDict0` that passes the preceding tests.



A better way to create a user-defined mapping type is to subclass `collections.UserDict` instead of `dict` (as we'll do in [Example 3-8](#)). Here we subclass `dict` just to show that `__missing__` is supported by the built-in `dict.__getitem__` method.

Example 3-7. `StrKeyDict0` converts nonstring keys to str on lookup (see tests in [Example 3-6](#))

```
class StrKeyDict0(dict): ❶

    def __missing__(self, key):
        if isinstance(key, str): ❷
            raise KeyError(key)
        return self[str(key)] ❸

    def get(self, key, default=None):
        try:
            return self[key] ❹
        except KeyError:
            return default ❺

    def __contains__(self, key):
        return key in self.keys() or str(key) in self.keys() ❻
```


- ❶ `StrKeyDict0` inherits from `dict`.
- ❷ Check whether `key` is already a `str`. If it is, and it's missing, raise `KeyError`.
- ❸ Build `str` from `key` and look it up.
- ❹ The `get` method delegates to `__getitem__` by using the `self[key]` notation; that gives the opportunity for our `__missing__` to act.
- ❺ If a `KeyError` was raised, `__missing__` already failed, so we return the default.
- ❻ Search for unmodified `key` (the instance may contain non-`str` keys), then for a `str` built from the `key`.

Take a moment to consider why the test `isinstance(key, str)` is necessary in the `__missing__` implementation.

Without that test, our `__missing__` method would work OK for any `key k`—`str` or not `str`—whenever `str(k)` produced an existing key. But if `str(k)` is not an existing key, we'd have an infinite recursion. The last line, `self[str(key)]` would call `__getitem__` passing that `str` `key`, which in turn would call `__missing__` again.

The `__contains__` method is also needed for consistent behavior in this example, because the operation `k in d` calls it, but the method inherited from `dict` does not fall back to invoking `__missing__`. There is a subtle detail in our implementation of `__contains__`: we do not check for the `key` in the usual Pythonic way—`k in my_dict`—because `str(key) in self` would recursively call `__contains__`. We avoid this by explicitly looking up the `key` in `self.keys()`.



A search like `k in my_dict.keys()` is efficient in Python 3 even for very large mappings because `dict.keys()` returns a view, which is similar to a set, and containment checks in sets are as fast as in dictionaries. Details are documented in the “[Dictionary](#)” [view objects](#) section of the documentation. In Python 2, `dict.keys()` returns a list, so our solution also works there, but it is not efficient for large dictionaries, because `k in my_list` must scan the list.

The check for the unmodified `key`—`key in self.keys()`—is necessary for correctness because `StrKeyDict0` does not enforce that all keys in the dictionary must be of type `str`. Our only goal with this simple example is to make searching “friendlier” and not enforce types.

So far we have covered the `dict` and `defaultdict` mapping types, but the standard library comes with other mapping implementations, which we discuss next.

Variations of dict

In this section, we summarize the various mapping types included in the `collections` module of the standard library, besides `defaultdict`:

`collections.OrderedDict`

Maintains keys in insertion order, allowing iteration over items in a predictable order. The `popitem` method of an `OrderedDict` pops the first item by default, but if called as `my_odict.popitem(last=True)`, it pops the last item added.

`collections.ChainMap`

Holds a list of mappings that can be searched as one. The lookup is performed on each mapping in order, and succeeds if the key is found in any of them. This is useful to interpreters for languages with nested scopes, where each mapping represents a scope context. The “[ChainMap objects](#)” section of the [collections docs](#) has several examples of `ChainMap` usage, including this snippet inspired by the basic rules of variable lookup in Python:

```
import builtins
pylookup = ChainMap(locals(), globals(), vars(builtins))
```

`collections.Counter`

A mapping that holds an integer count for each key. Updating an existing key adds to its count. This can be used to count instances of hashable objects (the keys) or as a multiset—a set that can hold several occurrences of each element. `Counter` implements the `+` and `-` operators to combine tallies, and other useful methods such as `most_common([n])`, which returns an ordered list of tuples with the *n* most common items and their counts; see the [documentation](#). Here is `Counter` used to count letters in words:

```
>>> ct = collections.Counter('abracadabra')
>>> ct
Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
>>> ct.update('aaaaazzz')
>>> ct
Counter({'a': 10, 'z': 3, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
>>> ct.most_common(2)
[('a', 10), ('z', 3)]
```

`collections.UserDict`

A pure Python implementation of a mapping that works like a standard dict.

While `OrderedDict`, `ChainMap`, and `Counter` come ready to use, `UserDict` is designed to be subclassed, as we’ll do next.

Subclassing UserDict

It's almost always easier to create a new mapping type by extending `UserDict` rather than `dict`. Its value can be appreciated as we extend our `StrKeyDict0` from [Example 3-7](#) to make sure that any keys added to the mapping are stored as `str`.

The main reason why it's preferable to subclass from `UserDict` rather than from `dict` is that the built-in has some implementation shortcuts that end up forcing us to override methods that we can just inherit from `UserDict` with no problems.³

Note that `UserDict` does not inherit from `dict`, but has an internal `dict` instance, called `data`, which holds the actual items. This avoids undesired recursion when coding special methods like `__setitem__`, and simplifies the coding of `__contains__`, compared to [Example 3-7](#).

Thanks to `UserDict`, `StrKeyDict` ([Example 3-8](#)) is actually shorter than `StrKeyDict0` ([Example 3-7](#)), but it does more: it stores all keys as `str`, avoiding unpleasant surprises if the instance is built or updated with data containing nonstring keys.

Example 3-8. `StrKeyDict` always converts non-string keys to `str`—on insertion, update, and lookup

```
import collections
```

```
class StrKeyDict(collections.UserDict): ❶

    def __missing__(self, key): ❷
        if isinstance(key, str):
            raise KeyError(key)
        return self[str(key)]

    def __contains__(self, key):
        return str(key) in self.data ❸

    def __setitem__(self, key, item):
        self.data[str(key)] = item ❹
```

- ❶ `StrKeyDict` extends `UserDict`.
- ❷ `__missing__` is exactly as in [Example 3-7](#).
- ❸ `__contains__` is simpler: we can assume all stored keys are `str` and we can check on `self.data` instead of invoking `self.keys()` as we did in `StrKeyDict0`.

3. The exact problem with subclassing `dict` and other built-ins is covered in “[Subclassing Built-In Types Is Tricky](#)” on page 348.

- ④ `__setitem__` converts any key to a `str`. This method is easier to overwrite when we can delegate to the `self.data` attribute.

Because `UserDict` subclasses `MutableMapping`, the remaining methods that make `StrKeyDict` a full-fledged mapping are inherited from `UserDict`, `MutableMapping`, or `Mapping`. The latter have several useful concrete methods, in spite of being abstract base classes (ABCs). The following methods are worth noting:

`MutableMapping.update`

This powerful method can be called directly but is also used by `__init__` to load the instance from other mappings, from iterables of (`key`, `value`) pairs, and key-word arguments. Because it uses `self[key] = value` to add items, it ends up calling our implementation of `__setitem__`.

`Mapping.get`

In `StrKeyDict0` (Example 3-7), we had to code our own `get` to obtain results consistent with `__getitem__`, but in Example 3-8 we inherited `Mapping.get`, which is implemented exactly like `StrKeyDict0.get` (see [Python source code](#)).



After I wrote `StrKeyDict`, I discovered that Antoine Pitrou authored [PEP 455 — Adding a key-transforming dictionary to collections](#) and a patch to enhance the `collections` module with a `TransformDict`. The patch is attached to [issue18986](#) and may land in Python 3.5. To experiment with `TransformDict`, I extracted it into a standalone module ([03-dict-set/transformdict.py](#) in the *Fluent Python* code repository). `TransformDict` is more general than `StrKeyDict`, and is complicated by the requirement to preserve the keys as they were originally inserted.

We know there are several immutable sequence types, but how about an immutable dictionary? Well, there isn't a real one in the standard library, but a stand-in is available. Read on.

Immutable Mappings

The mapping types provided by the standard library are all mutable, but you may need to guarantee that a user cannot change a mapping by mistake. A concrete use case can be found, again, in the Pingo.io project I described in “[The `\_\_missing\_\_` Method](#)” on [page 72](#): the `board.pins` mapping represents the physical GPIO pins on the device. As such, it's nice to prevent inadvertent updates to `board.pins` because the hardware can't possibly be changed via software, so any change in the mapping would make it inconsistent with the physical reality of the device.

Since Python 3.3, the `types` module provides a wrapper class called `MappingProxyType`, which, given a mapping, returns a `mappingproxy` instance that is a read-only but dynamic view of the original mapping. This means that updates to the original mapping can be seen in the `mappingproxy`, but changes cannot be made through it. See [Example 3-9](#) for a brief demonstration.

Example 3-9. `MappingProxyType` builds a read-only `mappingproxy` instance from a dict

```
>>> from types import MappingProxyType
>>> d = {1: 'A'}
>>> d_proxy = MappingProxyType(d)
>>> d_proxy
mappingproxy({1: 'A'})
>>> d_proxy[1] ❶
'A'
>>> d_proxy[2] = 'x' ❷
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'mappingproxy' object does not support item assignment
>>> d[2] = 'B'
>>> d_proxy ❸
mappingproxy({1: 'A', 2: 'B'})
>>> d_proxy[2]
'B'
>>>
```

- ❶ Items in `d` can be seen through `d_proxy`.
- ❷ Changes cannot be made through `d_proxy`.
- ❸ `d_proxy` is dynamic: any change in `d` is reflected.

Here is how this could be used in practice in the Pingo.io scenario: the constructor in a concrete `Board` subclass would fill a private mapping with the pin objects, and expose it to clients of the API via a public `.pins` attribute implemented as a `mappingproxy`. That way the clients would not be able to add, remove, or change pins by accident.⁴

Now that we've covered most mapping types in the standard library and when to use them, we will move to the set types.

4. We are not actually using `MappingProxyType` in Pingo.io because it is new in Python 3.3 and we need to support Python 2.7 at this time.

Set Theory

Sets are a relatively new addition in the history of Python, and somewhat underused. The set type and its immutable sibling `frozenset` first appeared in a module in Python 2.3 and were promoted to built-ins in Python 2.6.



In this book, the word “set” is used to refer both to `set` and `frozenset`. When talking specifically about the `set` class, its name appears in the constant width font used for source code: `set`.

A set is a collection of unique objects. A basic use case is removing duplication:

```
>>> l = ['spam', 'spam', 'eggs', 'spam']
>>> set(l)
{'eggs', 'spam'}
>>> list(set(l))
['eggs', 'spam']
```

Set elements must be hashable. The `set` type is not hashable, but `frozenset` is, so you can have `frozenset` elements inside a `set`.

In addition to guaranteeing uniqueness, the set types implement the essential set operations as infix operators, so, given two sets `a` and `b`, `a | b` returns their union, `a & b` computes the intersection, and `a - b` the difference. Smart use of set operations can reduce both the line count and the runtime of Python programs, at the same time making code easier to read and reason about—by removing loops and lots of conditional logic.

For example, imagine you have a large set of email addresses (the haystack) and a smaller set of addresses (the needles) and you need to count how many needles occur in the haystack. Thanks to set intersection (the `&` operator) you can code that in a simple line (see [Example 3-10](#)).

Example 3-10. Count occurrences of needles in a haystack, both of type `set`

```
found = len(needles & haystack)
```

Without the intersection operator, you’d have write [Example 3-11](#) to accomplish the same task as [Example 3-10](#).

Example 3-11. Count occurrences of needles in a haystack (same end result as [Example 3-10](#))

```
found = 0
for n in needles:
    if n in haystack:
        found += 1
```

Example 3-10 runs slightly faster than **Example 3-11**. On the other hand, **Example 3-11** works for any iterable objects `needles` and `haystack`, while **Example 3-10** requires that both be sets. But, if you don't have sets on hand, you can always build them on the fly, as shown in **Example 3-12**.

Example 3-12. Count occurrences of needles in a haystack; these lines work for any iterable types

```
found = len(set(needles) & set(haystack))

# another way:
found = len(set(needles).intersection(haystack))
```

Of course, there is an extra cost involved in building the sets in **Example 3-12**, but if either the `needles` or the `haystack` is already a set, the alternatives in **Example 3-12** may be cheaper than **Example 3-11**.

Any one of the preceding examples are capable of searching 1,000 values in a haystack of 10,000,000 items in a little over 3 milliseconds—that's about 3 microseconds per needle.

Besides the extremely fast membership test (thanks to the underlying hash table), the `set` and `frozenset` built-in types provide a rich selection of operations to create new sets or, in the case of `set`, to change existing ones. We will discuss the operations shortly, but first a note about syntax.

set Literals

The syntax of `set` literals—`{1}`, `{1, 2}`, etc.—looks exactly like the math notation, with one important exception: there's no literal notation for the empty set, so we must remember to write `set()`.



Syntax Quirk

Don't forget: to create an empty set, you should use the constructor without an argument: `set()`. If you write `{}`, you're creating an empty dict—this hasn't changed.

In Python 3, the standard string representation of sets always uses the `{...}` notation, except for the empty set:

```
>>> s = {1}
>>> type(s)
<class 'set'>
>>> s
{1}
>>> s.pop()
1
```

```
>>> s
set()
```

Literal set syntax like `{1, 2, 3}` is both faster and more readable than calling the constructor (e.g., `set([1, 2, 3])`). The latter form is slower because, to evaluate it, Python has to look up the `set` name to fetch the constructor, then build a list, and finally pass it to the constructor. In contrast, to process a literal like `{1, 2, 3}`, Python runs a specialized `BUILD_SET` bytecode.

Take a look at the bytecode for the two operations, as output by `dis.dis` (the disassembler function):

```
>>> from dis import dis
>>> dis('{1}')
1          0 LOAD_CONST          0 (1)
           3 BUILD_SET           1
           6 RETURN_VALUE

>>> dis('set([1])')
1          0 LOAD_NAME           0 (set)
           3 LOAD_CONST          0 (1)
           6 BUILD_LIST         1
           9 CALL_FUNCTION       1 (1 positional, 0 keyword pair)
          12 RETURN_VALUE
```

- ❶ Disassemble bytecode for literal expression `{1}`.
- ❷ Special `BUILD_SET` bytecode does almost all the work.
- ❸ Bytecode for `set([1])`.
- ❹ Three operations instead of `BUILD_SET`: `LOAD_NAME`, `BUILD_LIST`, and `CALL_FUNCTION`.

There is no special syntax to represent `frozenset` literals—they must be created by calling the constructor. The standard string representation in Python 3 looks like a `frozenset` constructor call. Note the output in the console session:

```
>>> frozenset(range(10))
frozenset({0, 1, 2, 3, 4, 5, 6, 7, 8, 9})
```

Speaking of syntax, the familiar shape of listcomps was adapted to build sets as well.

Set Comprehensions

Set comprehensions (*setcomps*) were added in Python 2.7, together with the dictcomps that we saw in “dict Comprehensions” on page 66. Example 3-13 is a simple example.

Example 3-13. Build a set of Latin-1 characters that have the word “SIGN” in their Unicode names

```
>>> from unicodedata import name ❶
>>> {chr(i) for i in range(32, 256) if 'SIGN' in name(chr(i), '')} ❷
{'§', '=', 'ç', '#', '¤', '<', '¥', 'µ', '×', '$', '¶', '£', '©',
'ª', '+', '÷', '±', '>', '¬', '®', '%'}
```

- ❶ Import name function from unicodedata to obtain character names.
- ❷ Build set of characters with codes from 32 to 255 that have the word 'SIGN' in their names.

Syntax matters aside, let’s now review the rich assortment of operations provided by sets.

Set Operations

Figure 3-2 gives an overview of the methods you can expect from mutable and immutable sets. Many of them are special methods for operator overloading. Table 3-2 shows the math set operators that have corresponding operators or methods in Python. Note that some operators and methods perform in-place changes on the target set (e.g., `&=`, `difference_update`, etc.). Such operations make no sense in the ideal world of mathematical sets, and are not implemented in `frozenset`.

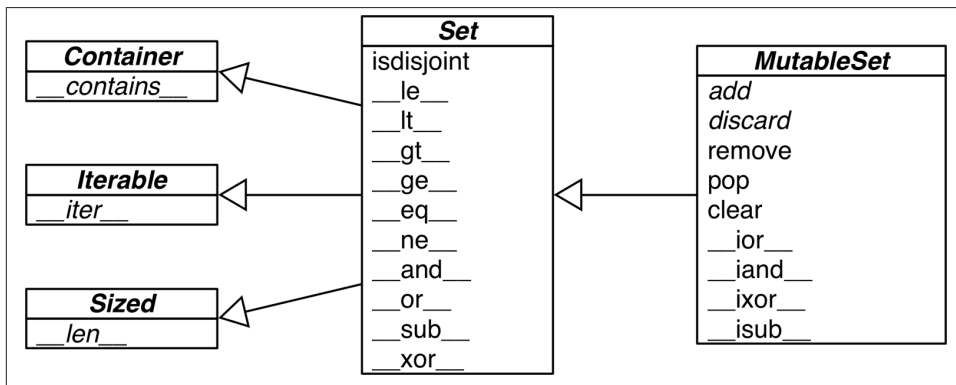


Figure 3-2. UML class diagram for `MutableSet` and its superclasses from `collections.abc` (names in *italics* are abstract classes and abstract methods; reverse operator methods omitted for brevity)



The infix operators in [Table 3-2](#) require that both operands be sets, but all other methods take one or more iterable arguments. For example, to produce the union of four collections, *a*, *b*, *c*, and *d*, you can call `a.union(b, c, d)`, where *a* must be a set, but *b*, *c*, and *d* can be iterables of any type.

Table 3-2. Mathematical set operations: these methods either produce a new set or update the target set in place, if it's mutable

Math symbol	Python operator	Method	Description
$S \cap Z$	<code>s & z</code>	<code>s.__and__(z)</code>	Intersection of <i>s</i> and <i>z</i>
	<code>z & s</code>	<code>s.__rand__(z)</code>	Reversed <code>&</code> operator
		<code>s.intersection(it, ...)</code>	Intersection of <i>s</i> and all sets built from iterables <i>it</i> , etc.
	<code>s &= z</code>	<code>s.__iand__(z)</code>	<i>s</i> updated with intersection of <i>s</i> and <i>z</i>
		<code>s.intersection_update(it, ...)</code>	<i>s</i> updated with intersection of <i>s</i> and all sets built from iterables <i>it</i> , etc.
$S \cup Z$	<code>s z</code>	<code>s.__or__(z)</code>	Union of <i>s</i> and <i>z</i>
	<code>z s</code>	<code>s.__ror__(z)</code>	Reversed <code> </code>
		<code>s.union(it, ...)</code>	Union of <i>s</i> and all sets built from iterables <i>it</i> , etc.
	<code>s = z</code>	<code>s.__ior__(z)</code>	<i>s</i> updated with union of <i>s</i> and <i>z</i>
		<code>s.update(it, ...)</code>	<i>s</i> updated with union of <i>s</i> and all sets built from iterables <i>it</i> , etc.
$S \setminus Z$	<code>s - z</code>	<code>s.__sub__(z)</code>	Relative complement or difference between <i>s</i> and <i>z</i>
	<code>z - s</code>	<code>s.__rsub__(z)</code>	Reversed <code>-</code> operator
		<code>s.difference(it, ...)</code>	Difference between <i>s</i> and all sets built from iterables <i>it</i> , etc.
	<code>s -= z</code>	<code>s.__isub__(z)</code>	<i>s</i> updated with difference between <i>s</i> and <i>z</i>
		<code>s.difference_update(it, ...)</code>	<i>s</i> updated with difference between <i>s</i> and all sets built from iterables <i>it</i> , etc.
		<code>s.symmetric_difference(it)</code>	Complement of <i>s & set(it)</i>
$S \Delta Z$	<code>s ^ z</code>	<code>s.__xor__(z)</code>	Symmetric difference (the complement of the intersection <i>s & z</i>)
	<code>z ^ s</code>	<code>s.__rxor__(z)</code>	Reversed <code>^</code> operator
		<code>s.symmetric_difference_update(it, ...)</code>	<i>s</i> updated with symmetric difference of <i>s</i> and all sets built from iterables <i>it</i> , etc.
	<code>s ^= z</code>	<code>s.__ixor__(z)</code>	<i>s</i> updated with symmetric difference of <i>s</i> and <i>z</i>



As I write this, there is a Python bug report—(issue 8743)—that says: “The `set()` operators (*or*, *and*, *sub*, *xor*, and their in-place counterparts) require that the parameter also be an instance of `set()`,” with the undesired side effect that these operators don’t work with `collections.abc.Set` subclasses. The bug is already fixed in trunk for Python 2.7 and 3.4, and should be history by the time you read this.

Table 3-3 lists set predicates: operators and methods that return True or False.

Table 3-3. Set comparison operators and methods that return a bool

Math symbol	Python operator	Method	Description
		<code>s.isdisjoint(z)</code>	<code>s</code> and <code>z</code> are disjoint (have no elements in common)
$e \in S$	<code>e in s</code>	<code>s.__contains__(e)</code>	Element <code>e</code> is a member of <code>s</code>
$S \subseteq Z$	<code>s <= z</code>	<code>s.__le__(z)</code>	<code>s</code> is a subset of the <code>z</code> set
		<code>s.issubset(it)</code>	<code>s</code> is a subset of the set built from the iterable <code>it</code>
$S \subset Z$	<code>s < z</code>	<code>s.__lt__(z)</code>	<code>s</code> is a proper subset of the <code>z</code> set
$S \supseteq Z$	<code>s >= z</code>	<code>s.__ge__(z)</code>	<code>s</code> is a superset of the <code>z</code> set
		<code>s.issuperset(it)</code>	<code>s</code> is a superset of the set built from the iterable <code>it</code>
$S \supset Z$	<code>s > z</code>	<code>s.__gt__(z)</code>	<code>s</code> is a proper superset of the <code>z</code> set

In addition to the operators and methods derived from math set theory, the set types implement other methods of practical use, summarized in Table 3-4.

Table 3-4. Additional set methods

	set	frozenset	
<code>s.add(e)</code>	•		Add element <code>e</code> to <code>s</code>
<code>s.clear()</code>	•		Remove all elements of <code>s</code>
<code>s.copy()</code>	•	•	Shallow copy of <code>s</code>
<code>s.discard(e)</code>	•		Remove element <code>e</code> from <code>s</code> if it is present
<code>s.__iter__()</code>	•	•	Get iterator over <code>s</code>
<code>s.__len__()</code>	•	•	<code>len(s)</code>
<code>s.pop()</code>	•		Remove and return an element from <code>s</code> , raising <code>KeyError</code> if <code>s</code> is empty
<code>s.remove(e)</code>	•		Remove element <code>e</code> from <code>s</code> , raising <code>KeyError</code> if <code>e not in s</code>

This completes our overview of the features of sets.

We now change gears to discuss how dictionaries and sets are implemented with hash tables. After reading the rest of this chapter, you will no longer be surprised by the

apparently unpredictable behavior sometimes exhibited by `dict`, `set`, and their brethren.

dict and set Under the Hood

Understanding how Python dictionaries and sets are implemented using hash tables is helpful to make sense of their strengths and limitations.

Here are some questions this section will answer:

- How efficient are Python `dict` and `set`?
- Why are they unordered?
- Why can't we use any Python object as a `dict` key or `set` element?
- Why does the order of the `dict` keys or `set` elements depend on insertion order, and may change during the lifetime of the structure?
- Why is it bad to add items to a `dict` or `set` while iterating through it?

To motivate the study of hash tables, we start by showcasing the amazing performance of `dict` and `set` with a simple test involving millions of items.

A Performance Experiment

From experience, all Pythonistas know that dicts and sets are fast. We'll confirm that with a controlled experiment.

To see how the size of a `dict`, `set`, or `list` affects the performance of search using the `in` operator, I generated an array of 10 million distinct double-precision floats, the “haystack.” I then generated an array of needles: 1,000 floats, with 500 picked from the haystack and 500 verified not to be in it.

For the `dict` benchmark, I used `dict.fromkeys()` to create a `dict` named `haystack` with 1,000 floats. This was the setup for the `dict` test. The actual code I clocked with the `timeit` module is [Example 3-14](#) (like [Example 3-11](#)).

Example 3-14. Search for needles in haystack and count those found

```
found = 0
for n in needles:
    if n in haystack:
        found += 1
```

The benchmark was repeated another four times, each time increasing tenfold the size of `haystack`, to reach a size of 10,000,000 in the last test. The result of the `dict` performance test is in [Table 3-5](#).

Table 3-5. Total time for using `in` operator to search for 1,000 needles in haystack dicts of five sizes on a Core i7 laptop running Python 3.4.0 (tests timed the loop in Example 3-14)

len of haystack	Factor	dict time	Factor
1,000	1x	0.000202s	1.00x
10,000	10x	0.000140s	0.69x
100,000	100x	0.000228s	1.13x
1,000,000	1,000x	0.000290s	1.44x
10,000,000	10,000x	0.000337s	1.67x

In concrete terms, to check for the presence of 1,000 floating-point keys in a dictionary with 1,000 items, the processing time on my laptop was 0.000202s, and the same search in a dict with 10,000,000 items took 0.000337s. In other words, the time per search in the haystack with 10 million items was 0.337μs for each needle—yes, that is about one third of a microsecond per needle.

To compare, I repeated the benchmark, with the same haystacks of increasing size, but storing the haystack as a `set` or as `list`. For the `set` tests, in addition to timing the `for` loop in Example 3-14, I also timed the one-liner in Example 3-15, which produces the same result: count the number of elements from `needles` that are also in `haystack`.

Example 3-15. Use `set` intersection to count the needles that occur in haystack

```
found = len(needles & haystack)
```

Table 3-6 shows the tests side by side. The best times are in the “`set & time`” column, which displays results for the `set &` operator using the code from Example 3-15. The worst times are—as expected—in the “`list time`” column, because there is no hash table to support searches with the `in` operator on a `list`, so a full scan must be made, resulting in times that grow linearly with the size of the haystack.

Table 3-6. Total time for using `in` operator to search for 1,000 keys in haystacks of 5 sizes, stored as `dicts`, `sets`, and `lists` on a Core i7 laptop running Python 3.4.0 (tests timed the loop in Example 3-14 except the `set &`, which uses Example 3-15)

len of haystack	Factor	dict time	Factor	set time	Factor	set & time	Factor	list time	Factor
1,000	1x	0.000202s	1.00x	0.000143s	1.00x	0.000087s	1.00x	0.010556s	1.00x
10,000	10x	0.000140s	0.69x	0.000147s	1.03x	0.000092s	1.06x	0.086586s	8.20x
100,000	100x	0.000228s	1.13x	0.000241s	1.69x	0.000163s	1.87x	0.871560s	82.57x
1,000,000	1,000x	0.000290s	1.44x	0.000332s	2.32x	0.000250s	2.87x	9.189616s	870.56x
10,000,000	10,000x	0.000337s	1.67x	0.000387s	2.71x	0.000314s	3.61x	97.948056s	9,278.90x

If your program does any kind of I/O, the lookup time for keys in dicts or sets is negligible, regardless of the `dict` or `set` size (as long as it does fit in RAM). See the code used to generate [Table 3-6](#) and accompanying discussion in [Appendix A, Example A-1](#).

Now that we have concrete evidence of the speed of dicts and sets, let's explore how that is achieved. The discussion of the hash table internals explains, for example, why the key ordering is apparently random and unstable.

Hash Tables in Dictionaries

This is a high-level view of how Python uses a hash table to implement a `dict`. Many details are omitted—the CPython code has some optimization tricks⁵—but the overall description is accurate.



To simplify the ensuing presentation, we will focus on the internals of `dict` first, and later transfer the concepts to sets.

A hash table is a sparse array (i.e., an array that always has empty cells). In standard data structure texts, the cells in a hash table are often called “buckets.” In a `dict` hash table, there is a bucket for each item, and it contains two fields: a reference to the key and a reference to the value of the item. Because all buckets have the same size, access to an individual bucket is done by offset.

Python tries to keep at least 1/3 of the buckets empty; if the hash table becomes too crowded, it is copied to a new location with room for more buckets.

To put an item in a hash table, the first step is to calculate the *hash value* of the item key, which is done with the `hash()` built-in function, explained next.

Hashes and equality

The `hash()` built-in function works directly with built-in types and falls back to calling `__hash__` for user-defined types. If two objects compare equal, their hash values must also be equal, otherwise the hash table algorithm does not work. For example, because `1 == 1.0` is true, `hash(1) == hash(1.0)` must also be true, even though the internal representation of an `int` and a `float` are very different.⁶

5. The source code for the CPython `dictobject.c` module is rich in comments. See also the reference for the *Beautiful Code* book in “[Further Reading](#)” on page 94.

6. Because we just mentioned `int`, here is a CPython implementation detail: the hash value of an `int` that fits in a machine word is the value of the `int` itself.

Also, to be effective as hash table indexes, hash values should scatter around the index space as much as possible. This means that, ideally, objects that are similar but not equal should have hash values that differ widely. [Example 3-16](#) is the output of a script to compare the bit patterns of hash values. Note how the hashes of 1 and 1.0 are the same, but those of 1.0001, 1.0002, and 1.0003 are very different.

Example 3-16. Comparing hash bit patterns of 1, 1.0001, 1.0002, and 1.0003 on a 32-bit build of Python (bits that are different in the hashes above and below are highlighted with ! and the right column shows the number of bits that differ)

```
32-bit Python build
1          00000000000000000000000000000001
1.0        00000000000000000000000000000001      != 0
-----
1.0        00000000000000000000000000000001
          ! !!! ! !! ! !      ! ! !! !!!      != 16
1.0001     00101110101101010000101011011101
-----
1.0001     0010111010110101010000101011011101
          !!!  !!!!  !!!!!  !!!!!  !! !      != 20
1.0002     01011101011010100001010110111001
-----
1.0002     01011101011010100001010110111001
          ! !  ! !!! ! !  !! ! !  ! !!!!      != 17
1.0003     00001100000111110010000010010110
-----
```

The code to produce [Example 3-16](#) is in [Appendix A](#). Most of it deals with formatting the output, but it is listed as [Example A-3](#) for completeness.



Starting with Python 3.3, a random salt value is added to the hashes of `str`, `bytes`, and `datetime` objects. The salt value is constant within a Python process but varies between interpreter runs. The random salt is a security measure to prevent a DOS attack. Details are in a note in the documentation for [the `\_\_hash\_\_` special method](#).

With this basic understanding of object hashes, we are ready to dive into the algorithm that makes hash tables operate.

The hash table algorithm

To fetch the value at `my_dict[search_key]`, Python calls `hash(search_key)` to obtain the *hash value* of `search_key` and uses the least significant bits of that number as an offset to look up a bucket in the hash table (the number of bits used depends on the current size of the table). If the found bucket is empty, `KeyError` is raised. Otherwise,

the found bucket has an item—a `found_key:found_value` pair—and then Python checks whether `search_key == found_key`. If they match, that was the item sought: `found_value` is returned.

However, if `search_key` and `found_key` do not match, this is a *hash collision*. This happens because a hash function maps arbitrary objects to a small number of bits, and—in addition—the hash table is indexed with a subset of those bits. In order to resolve the collision, the algorithm then takes different bits in the hash, massages them in a particular way, and uses the result as an offset to look up a different bucket.⁷ If that is empty, `KeyError` is raised; if not, either the keys match and the item value is returned, or the collision resolution process is repeated. See [Figure 3-3](#) for a diagram of this algorithm.

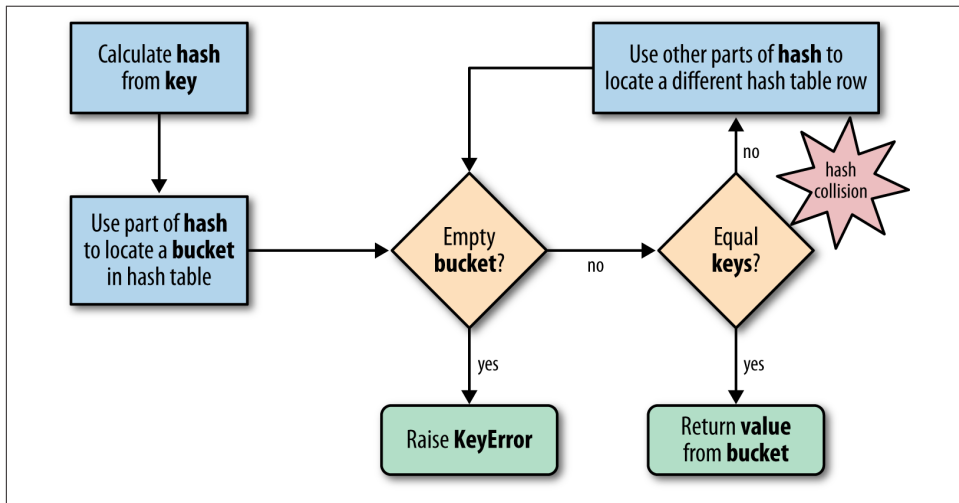


Figure 3-3. Flowchart for retrieving an item from a dict; given a key, this procedure either returns a value or raises `KeyError`

The process to insert or update an item is the same, except that when an empty bucket is located, the new item is put there, and when a bucket with a matching key is found, the value in that bucket is overwritten with the new value.

Additionally, when inserting items, Python may determine that the hash table is too crowded and rebuild it to a new location with more room. As the hash table grows, so does the number of hash bits used as bucket offsets, and this keeps the rate of collisions low.

7. The C function that shuffles the hash bits in case of collision has a curious name: `per turb`. For all the details, see [dictobject.c](#) in the CPython source code.

This implementation may seem like a lot of work, but even with millions of items in a `dict`, many searches happen with no collisions, and the average number of collisions per search is between one and two. Under normal usage, even the unluckiest keys can be found after a handful of collisions are resolved.

Knowing the internals of the `dict` implementation we can explain the strengths and limitations of this data structure and all the others derived from it in Python. We are now ready to consider why Python `dicts` behave as they do.

Practical Consequences of How dict Works

In the following subsections, we'll discuss the limitations and benefits that the underlying hash table implementation brings to `dict` usage.

Keys must be hashable objects

An object is hashable if all of these requirements are met:

1. It supports the `hash()` function via a `hash()` method that always returns the same value over the lifetime of the object.
2. It supports equality via an `eq()` method.
3. If `a == b` is `True` then `hash(a) == hash(b)` must also be `True`.

User-defined types are hashable by default because their hash value is their `id()` and they all compare not equal.



If you implement a class with a custom `__eq__` method, you must also implement a suitable `__hash__`, because you must always make sure that if `a == b` is `True` then `hash(a) == hash(b)` is also `True`. Otherwise you are breaking an invariant of the hash table algorithm, with the grave consequence that `dicts` and `sets` will not deal reliably with your objects. If a custom `__eq__` depends on mutable state, then `__hash__` must raise `TypeError` with a message like `unhashable type: 'MyClass'`.

dicts have significant memory overhead

Because a `dict` uses a hash table internally, and hash tables must be sparse to work, they are not space efficient. For example, if you are handling a large quantity of records, it makes sense to store them in a list of tuples or named tuples instead of using a list of dictionaries in JSON style, with one `dict` per record. Replacing `dicts` with tuples reduces the memory usage in two ways: by removing the overhead of one hash table per record and by not storing the field names again with each record.

For user-defined types, the `__slots__` class attribute changes the storage of instance attributes from a dict to a tuple in each instance. This will be discussed in “Saving Space with the `__slots__` Class Attribute” on page 264 (Chapter 9).

Keep in mind we are talking about space optimizations. If you are dealing with a few million objects and your machine has gigabytes of RAM, you should postpone such optimizations until they are actually warranted. Optimization is the altar where maintainability is sacrificed.

Key search is very fast

The dict implementation is an example of trading space for time: dictionaries have significant memory overhead, but they provide fast access regardless of the size of the dictionary—as long as it fits in memory. As Table 3-5 shows, when we increased the size of a dict from 1,000 to 10,000,000 elements, the time to search grew by a factor of 2.8, from 0.000163s to 0.000456s. The latter figure means we could search more than 2 million keys per second in a dict with 10 million items.

Key ordering depends on insertion order

When a hash collision happens, the second key ends up in a position that it would not normally occupy if it had been inserted first. So, a dict built as `dict([(key1, value1), (key2, value2)])` compares equal to `dict([(key2, value2), (key1, value1)])`, but their key ordering may not be the same if the hashes of key1 and key2 collide.

Example 3-17 demonstrates the effect of loading three dicts with the same data, just in different order. The resulting dictionaries all compare equal, even if their order is not the same.

Example 3-17. dialcodes.py fills three dictionaries with the same data sorted in different ways

```
# dial codes of the top 10 most populous countries
```

```
DIAL_CODES = [  
    (86, 'China'),  
    (91, 'India'),  
    (1, 'United States'),  
    (62, 'Indonesia'),  
    (55, 'Brazil'),  
    (92, 'Pakistan'),  
    (880, 'Bangladesh'),  
    (234, 'Nigeria'),  
    (7, 'Russia'),  
    (81, 'Japan'),  
]
```

```
d1 = dict(DIAL_CODES) ❶  
print('d1:', d1.keys())  
d2 = dict(sorted(DIAL_CODES)) ❷
```

```
print('d2:', d2.keys())
d3 = dict(sorted(DIAL_CODES, key=lambda x:x[1])) ❸
print('d3:', d3.keys())
assert d1 == d2 and d2 == d3 ❹
```

- ❶ d1: built from the tuples in descending order of country population.
- ❷ d2: filled with tuples sorted by dial code.
- ❸ d3: loaded with tuples sorted by country name.
- ❹ The dictionaries compare equal, because they hold the same key:value pairs.

Example 3-18 shows the output.

Example 3-18. Output from dialcodes.py shows three distinct key orderings

```
d1: dict_keys([880, 1, 86, 55, 7, 234, 91, 92, 62, 81])
d2: dict_keys([880, 1, 91, 86, 81, 55, 234, 7, 92, 62])
d3: dict_keys([880, 81, 1, 86, 55, 7, 234, 91, 92, 62])
```

Adding items to a dict may change the order of existing keys

Whenever you add a new item to a `dict`, the Python interpreter may decide that the hash table of that dictionary needs to grow. This entails building a new, bigger hash table, and adding all current items to the new table. During this process, new (but different) hash collisions may happen, with the result that the keys are likely to be ordered differently in the new hash table. All of this is implementation-dependent, so you cannot reliably predict when it will happen. If you are iterating over the dictionary keys and changing them at the same time, your loop may not scan all the items as expected—not even the items that were already in the dictionary before you added to it.

This is why modifying the contents of a `dict` while iterating through it is a bad idea. If you need to scan and add items to a dictionary, do it in two steps: read the `dict` from start to finish and collect the needed additions in a second `dict`. Then update the first one with it.



In Python 3, the `.keys()`, `.items()`, and `.values()` methods return dictionary views, which behave more like sets than the lists returned by these methods in Python 2. Such views are also dynamic: they do not replicate the contents of the `dict`, and they immediately reflect any changes to the `dict`.

We can now apply what we know about hash tables to sets.

How Sets Work—Practical Consequences

The `set` and `frozenset` types are also implemented with a hash table, except that each bucket holds only a reference to the element (as if it were a key in a `dict`, but without a value to go with it). In fact, before `set` was added to the language, we often used dictionaries with dummy values just to perform fast membership tests on the keys.

Everything said in “[Practical Consequences of How dict Works](#)” on page 90 about how the underlying hash table determines the behavior of a `dict` applies to a `set`. Without repeating the previous section, we can summarize it for sets with just a few words:

- Set elements must be hashable objects.
- Sets have a significant memory overhead.
- Membership testing is very efficient.
- Element ordering depends on insertion order.
- Adding elements to a set may change the order of other elements.

Chapter Summary

Dictionaries are a keystone of Python. Beyond the basic `dict`, the standard library offers handy, ready-to-use specialized mappings like `defaultdict`, `OrderedDict`, `ChainMap`, and `Counter`, all defined in the `collections` module. The same module also provides the easy-to-extend `UserDict` class.

Two powerful methods available in most mappings are `setdefault` and `update`. The `setdefault` method is used to update items holding mutable values, for example, in a `dict` of `list` values, to avoid redundant searches for the same key. The `update` method allows bulk insertion or overwriting of items from any other mapping, from iterables providing `(key, value)` pairs and from keyword arguments. Mapping constructors also use `update` internally, allowing instances to be initialized from mappings, iterables, or keyword arguments.

A clever hook in the mapping API is the `__missing__` method, which lets you customize what happens when a key is not found.

The `collections.abc` module provides the `Mapping` and `MutableMapping` abstract base classes for reference and type checking. The little-known `MappingProxyType` from the `types` module creates immutable mappings. There are also ABCs for `Set` and `MutableSet`.

The hash table implementation underlying `dict` and `set` is extremely fast. Understanding its logic explains why items are apparently unordered and may even be reordered behind our backs. There is a price to pay for all this speed, and the price is in memory.

Further Reading

In *The Python Standard Library*, [8.3. collections — Container datatypes](#) includes examples and practical recipes with several mapping types. The Python source code for the module `Lib/collections/init.py` is a great reference for anyone who wants to create a new mapping type or grok the logic of the existing ones.

Chapter 1 of *Python Cookbook, Third edition* (O'Reilly) by David Beazley and Brian K. Jones has 20 handy and insightful recipes with data structures—the majority using `dict` in clever ways.

Written by A.M. Kuchling—a Python core contributor and author of many pages of the official Python docs and how-tos—Chapter 18, “Python’s Dictionary Implementation: Being All Things to All People,” in the book *Beautiful Code* (O'Reilly) includes a detailed explanation of the inner workings of the Python `dict`. Also, there are lots of comments in the source code of the `dictobject.c` Python module. Brandon Craig Rhodes’ presentation *The Mighty Dictionary* is excellent and shows how hash tables work by using lots of slides with... tables!

The rationale for adding sets to the language is documented in [PEP 218 — Adding a Built-In Set Object Type](#). When PEP 218 was approved, no special literal syntax was adopted for sets. The set literals were created for Python 3 and backported to Python 2.7, along with `dict` and `set` comprehensions. [PEP 274 — Dict Comprehensions](#) is the birth certificate of dictcomps. I could not find a PEP for setcomps; apparently they were adopted because they get along well with their siblings—a jolly good reason.

Soapbox

My friend Geraldo Cohen once remarked that Python is “simple and correct.”

The `dict` type is an example of simplicity and correctness. It’s highly optimized to do one thing well: retrieve arbitrary keys. It’s fast and robust enough to be used all over the Python interpreter itself. If you need predictable ordering, use `OrderedDict`. That is not a requirement in most uses of mappings, so it makes sense to keep the core implementation simple and offer variations in the standard library.

Contrast with PHP, where arrays are described like this in the official [PHP Manual](#):

An array in PHP is actually an ordered map. A map is a type that associates values to keys. This type is optimized for several different uses; it can be treated as an array, list

(vector), hash table (an implementation of a map), dictionary, collection, stack, queue, and probably more.

From that description, I don't know what is the real cost of using PHP's `list/Ordered Dict` hybrid.

The goal of this and the previous chapter in this book was to showcase the Python collection types optimized for particular uses. I made the point that beyond the trusty `list` and `dict` there are specialized alternatives for different use cases.

Before finding Python, I had done web programming using Perl, PHP, and JavaScript. I really enjoyed having a literal syntax for mappings in these languages, and I badly miss it whenever I have to use Java or C. A good literal syntax for mappings makes it easy to do configuration, table-driven implementations, and to hold data for prototyping and testing. The lack of it pushed the Java community to adopt the verbose and overly complex XML as a data format.

JSON was proposed as “**The Fat-Free Alternative to XML**” and became a huge success, replacing XML in many contexts. A concise syntax for lists and dictionaries makes an excellent data interchange format.

PHP and Ruby imitated the hash syntax from Perl, using `=>` to link keys to values. JavaScript followed the lead of Python and uses `:`. Of course, JSON came from JavaScript, but it also happens to be an almost exact subset of Python syntax. JSON is compatible with Python except for the spelling of the values `true`, `false`, and `null`. The syntax everybody now uses for exchanging data is the Python `dict` and `list` syntax.

Simple and correct.

FP Ch 4 | Unicode Text vs Bytes

CHAPTER 4

Text versus Bytes

Humans use text. Computers speak bytes.¹

— Esther Nam and Travis Fischer

Character Encoding and Unicode in Python

Python 3 introduced a sharp distinction between strings of human text and sequences of raw bytes. Implicit conversion of byte sequences to Unicode text is a thing of the past. This chapter deals with Unicode strings, binary sequences, and the encodings used to convert between them.

Depending on your Python programming context, a deeper understanding of Unicode may or may not be of vital importance to you. In the end, most of the issues covered in this chapter do not affect programmers who deal only with ASCII text. But even if that is your case, there is no escaping the `str` versus `byte` divide. As a bonus, you'll find that the specialized binary sequence types provide features that the “all-purpose” Python 2 `str` type does not have.

In this chapter, we will visit the following topics:

- Characters, code points, and byte representations
- Unique features of binary sequences: `bytes`, `bytearray`, and `memoryview`
- Codecs for full Unicode and legacy character sets
- Avoiding and dealing with encoding errors
- Best practices when handling text files
- The default encoding trap and standard I/O issues
- Safe Unicode text comparisons with normalization

1. Slide 12 of PyCon 2014 talk “Character Encoding and Unicode in Python” ([slides](#), [video](#)).

- Utility functions for normalization, case folding, and brute-force diacritic removal
- Proper sorting of Unicode text with `locale` and the PyUCA library
- Character metadata in the Unicode database
- Dual-mode APIs that handle `str` and `bytes`

Let's start with the characters, code points, and bytes.

Character Issues

The concept of “string” is simple enough: a string is a sequence of characters. The problem lies in the definition of “character.”

In 2015, the best definition of “character” we have is a Unicode character. Accordingly, the items you get out of a Python 3 `str` are Unicode characters, just like the items of a `unicode` object in Python 2—and not the raw bytes you get from a Python 2 `str`.

The Unicode standard explicitly separates the identity of characters from specific byte representations:

- The identity of a character—its *code point*—is a number from 0 to 1,114,111 (base 10), shown in the Unicode standard as 4 to 6 hexadecimal digits with a “U+” prefix. For example, the code point for the letter A is U+0041, the Euro sign is U+20AC, and the musical symbol G clef is assigned to code point U+1D11E. About 10% of the valid code points have characters assigned to them in Unicode 6.3, the standard used in Python 3.4.
- The actual bytes that represent a character depend on the *encoding* in use. An encoding is an algorithm that converts code points to byte sequences and vice versa. The code point for A (U+0041) is encoded as the single byte `\x41` in the UTF-8 encoding, or as the bytes `\x41\x00` in UTF-16LE encoding. As another example, the Euro sign (U+20AC) becomes three bytes in UTF-8—`\xe2\x82\xac`—but in UTF-16LE it is encoded as two bytes: `\xac\x20`.

Converting from code points to bytes is *encoding*; converting from bytes to code points is *decoding*. See [Example 4-1](#).

Example 4-1. Encoding and decoding

```
>>> s = 'café'
>>> len(s) # ①
4
>>> b = s.encode('utf8') # ②
>>> b
b'caf\x33\xa9' # ③
>>> len(b) # ④
5
```

```
>>> b.decode('utf8') # ❸
'café'
```

- ❶ The str 'café' has four Unicode characters.
- ❷ Encode str to bytes using UTF-8 encoding.
- ❸ bytes literals start with a b prefix.
- ❹ bytes b has five bytes (the code point for “é” is encoded as two bytes in UTF-8).
- ❺ Decode bytes to str using UTF-8 encoding.



If you need a memory aid to help distinguish `.decode()` from `.encode()`, convince yourself that byte sequences can be cryptic machine core dumps while Unicode str objects are “human” text. Therefore, it makes sense that we *decode* bytes to str to get human-readable text, and we *encode* str to bytes for storage or transmission.

Although the Python 3 str is pretty much the Python 2 unicode type with a new name, the Python 3 bytes is not simply the old str renamed, and there is also the closely related bytearray type. So it is worthwhile to take a look at the binary sequence types before advancing to encoding/decoding issues.

Byte Essentials

The new binary sequence types are unlike the Python 2 str in many regards. The first thing to know is that there are two basic built-in types for binary sequences: the immutable bytes type introduced in Python 3 and the mutable bytearray, added in Python 2.6. (Python 2.6 also introduced bytes, but it’s just an alias to the str type, and does not behave like the Python 3 bytes type.)

Each item in bytes or bytearray is an integer from 0 to 255, and not a one-character string like in the Python 2 str. However, a slice of a binary sequence always produces a binary sequence of the same type—including slices of length 1. See [Example 4-2](#).

Example 4-2. A five-byte sequence as bytes and as bytearray

```
>>> cafe = bytes('café', encoding='utf_8') ❶
>>> cafe
b'caf\xc3\xa9'
>>> cafe[0] ❷
99
>>> cafe[:1] ❸
b'c'
>>> cafe_arr = bytearray(cafe)
>>> cafe_arr ❹
```

```
bytearray(b'caf\xc3\xa9')
>>> cafe_arr[-1:] ❸
bytearray(b'\xa9')
```

- ❶ bytes can be built from a `str`, given an encoding.
- ❷ Each item is an integer in `range(256)`.
- ❸ Slices of bytes are also bytes—even slices of a single byte.
- ❹ There is no literal syntax for `bytearray`: they are shown as `bytearray()` with a bytes literal as argument.
- ❺ A slice of `bytearray` is also a `bytearray`.



The fact that `my_bytes[0]` retrieves an `int` but `my_bytes[:1]` returns a bytes object of length 1 should not be surprising. The only sequence type where `s[0] == s[:1]` is the `str` type. Although practical, this behavior of `str` is exceptional. For every other sequence, `s[i]` returns one item, and `s[i:i+1]` returns a sequence of the same type with the `s[i]` item inside it.

Although binary sequences are really sequences of integers, their literal notation reflects the fact that ASCII text is often embedded in them. Therefore, three different displays are used, depending on each byte value:

- For bytes in the printable ASCII range—from space to `~`—the ASCII character itself is used.
- For bytes corresponding to tab, newline, carriage return, and `\`, the escape sequences `\t`, `\n`, `\r`, and `\\` are used.
- For every other byte value, a hexadecimal escape sequence is used (e.g., `\x00` is the null byte).

That is why in **Example 4-2** you see `b'caf\xc3\xa9'`: the first three bytes `b'caf'` are in the printable ASCII range, the last two are not.

Both bytes and `bytearray` support every `str` method except those that do formatting (`format`, `format_map`) and a few others that depend on Unicode data, including `case fold`, `isdecimal`, `isidentifier`, `isnumeric`, `isprintable`, and `encode`. This means that you can use familiar string methods like `endswith`, `replace`, `strip`, `translate`, `upper`, and dozens of others with binary sequences—only using bytes and not `str` arguments. In addition, the regular expression functions in the `re` module also work on binary sequences, if the regex is compiled from a binary sequence instead of a `str`. The `%` operator does not work with binary sequences in Python 3.0 to 3.4, but should be sup-

ported in version 3.5 according to [PEP 461 — Adding % formatting to bytes and bytearray](#).

Binary sequences have a class method that `str` doesn't have, called `fromhex`, which builds a binary sequence by parsing pairs of hex digits optionally separated by spaces:

```
>>> bytes.fromhex('31 4B CE A9')
b'1K\xce\xa9'
```

The other ways of building bytes or bytearray instances are calling their constructors with:

- A `str` and an encoding keyword argument.
- An iterable providing items with values from 0 to 255.
- A single integer, to create a binary sequence of that size initialized with null bytes. (This signature will be deprecated in Python 3.5 and removed in Python 3.6. See [PEP 467 — Minor API improvements for binary sequences](#).)
- An object that implements the buffer protocol (e.g., `bytes`, `bytearray`, `memoryview`, `array.array`); this copies the bytes from the source object to the newly created binary sequence.

Building a binary sequence from a buffer-like object is a low-level operation that may involve type casting. See a demonstration in [Example 4-3](#).

Example 4-3. Initializing bytes from the raw data of an array

```
>>> import array
>>> numbers = array.array('h', [-2, -1, 0, 1, 2]) ❶
>>> octets = bytes(numbers) ❷
>>> octets
b'\xfe\xff\xff\xff\x00\x00\x01\x00\x02\x00' ❸
```

- ❶ Typecode 'h' creates an array of short integers (16 bits).
- ❷ `octets` holds a copy of the bytes that make up `numbers`.
- ❸ These are the 10 bytes that represent the five short integers.

Creating a bytes or bytearray object from any buffer-like source will always copy the bytes. In contrast, `memoryview` objects let you share memory between binary data structures. To extract structured information from binary sequences, the `struct` module is invaluable. We'll see it working along with bytes and `memoryview` in the next section.

Structs and Memory Views

The struct module provides functions to parse packed bytes into a tuple of fields of different types and to perform the opposite conversion, from a tuple into packed bytes. struct is used with bytes, bytearray, and memoryview objects.

As we've seen in “Memory Views” on page 51, the memoryview class does not let you create or store byte sequences, but provides shared memory access to slices of data from other binary sequences, packed arrays, and buffers such as Python Imaging Library (PIL) images,² without copying the bytes.

Example 4-4 shows the use of memoryview and struct together to extract the width and height of a GIF image.

Example 4-4. Using memoryview and struct to inspect a GIF image header

```
>>> import struct
>>> fmt = '<3s3sHH' # ❶
>>> with open('filter.gif', 'rb') as fp:
...     img = memoryview(fp.read()) # ❷
...
>>> header = img[:10] # ❸
>>> bytes(header) # ❹
b'GIF89a+\x02\xe6\x00'
>>> struct.unpack(fmt, header) # ❺
(b'GIF', b'89a', 555, 230)
>>> del header # ❻
>>> del img
```

- ❶ struct format: < little-endian; 3s3s two sequences of 3 bytes; HH two 16-bit integers.
- ❷ Create memoryview from file contents in memory...
- ❸ ...then another memoryview by slicing the first one; no bytes are copied here.
- ❹ Convert to bytes for display only; 10 bytes are copied here.
- ❺ Unpack memoryview into tuple of: type, version, width, and height.
- ❻ Delete references to release the memory associated with the memoryview instances.

Note that slicing a memoryview returns a new memoryview, without copying bytes (Leonardo Rochaël—one of the technical reviewers—pointed out that even less byte copying would happen if I used the mmap module to open the image as a memory-mapped file.

2. Pillow is PIL's most active fork.

I will not cover `mmap` in this book, but if you read and change binary files frequently, learning more about `mmap` — [Memory-mapped file support](#) will be very fruitful).

We will not go deeper into `memoryview` or the `struct` module in this book, but if you work with binary data, you'll find it worthwhile to study their docs: [Built-in Types » Memory Views](#) and [struct — Interpret bytes as packed binary data](#).

After this brief exploration of binary sequence types in Python, let's see how they are converted to/from strings.

Basic Encoders/Decoders

The Python distribution bundles more than 100 *codecs* (encoder/decoder) for text to byte conversion and vice versa. Each codec has a name, like `'utf_8'`, and often aliases, such as `'utf8'`, `'utf-8'`, and `'U8'`, which you can use as the encoding argument in functions like `open()`, `str.encode()`, `bytes.decode()`, and so on. [Example 4-5](#) shows the same text encoded as three different byte sequences.

Example 4-5. The string “El Niño” encoded with three codecs producing very different byte sequences

```
>>> for codec in ['latin_1', 'utf_8', 'utf_16']:
...     print(codec, 'El Niño'.encode(codec), sep='\t')
...
latin_1 b'El Ni\xf1o'
utf_8   b'El Ni\xc3\xb1o'
utf_16  b'\xff\xfe\x00l\x00 \x00N\x00i\x00\xfa\x00o\x00'
```

[Figure 4-1](#) demonstrates a variety of codecs generating bytes from characters like the letter “A” through the G-clef musical symbol. Note that the last three encodings are variable-length, multibyte encodings.

char.	code point	ascii	latin1	cp1252	cp437	gb2312	utf-8	utf-16le
A	U+0041	41	41	41	41	41	41	41 00
¿	U+00BF	*	BF	BF	A8	*	C2 BF	BF 00
Ã	U+00C3	*	C3	C3	*	*	C3 83	C3 00
á	U+00E1	*	E1	E1	A0	A8 A2	C3 A1	E1 00
Ω	U+03A9	*	*	*	EA	A6 B8	CE A9	A9 03
€	U+06BF	*	*	*	*	*	DA BF	BF 06
“	U+201C	*	*	93	*	A1 B0	E2 80 9C	1C 20
€	U+20AC	*	*	80	*	*	E2 82 AC	AC 20
Г	U+250C	*	*	*	DA	A9 B0	E2 94 8C	0C 25
气	U+6C14	*	*	*	*	C6 F8	E6 B0 94	14 6C
氣	U+6C23	*	*	*	*	*	E6 B0 A3	23 6C
👉	U+1D11E	*	*	*	*	*	F0 9D 84 9E	34 D8 1E DD

Figure 4-1. Twelve characters, their code points, and their byte representation (in hex) in seven different encodings (asterisks indicate that the character cannot be represented in that encoding)

All those asterisks in [Figure 4-1](#) make clear that some encodings, like ASCII and even the multibyte GB2312, cannot represent every Unicode character. The UTF encodings, however, are designed to handle every Unicode code point.

The encodings shown in [Figure 4-1](#) were chosen as a representative sample:

latin1 *a.k.a.* iso8859_1

Important because it is the basis for other encodings, such as cp1252 and Unicode itself (note how the latin1 byte values appear in the cp1252 bytes and even in the code points).

cp1252

A latin1 superset by Microsoft, adding useful symbols like curly quotes and the € (euro); some Windows apps call it “ANSI,” but it was never a real ANSI standard.

cp437

The original character set of the IBM PC, with box drawing characters. Incompatible with latin1, which appeared later.

gb2312

Legacy standard to encode the simplified Chinese ideographs used in mainland China; one of several widely deployed multibyte encodings for Asian languages.

utf-8

The most common 8-bit encoding on the Web, by far;³ backward-compatible with ASCII (pure ASCII text is valid UTF-8).

utf-16le

One form of the UTF-16 16-bit encoding scheme; all UTF-16 encodings support code points beyond U+FFFF through escape sequences called “surrogate pairs.”



UTF-16 superseded the original 16-bit Unicode 1.0 encoding—UCS-2—way back in 1996. UCS-2 is still deployed in many systems, but it only supports code points up to U+FFFF. As of Unicode 6.3, more than 50% of the allocated code points are above U+10000, including the increasingly popular emoji pictographs.

With this overview of common encodings now complete, we move to handling issues in encoding and decoding operations.

Understanding Encode/Decode Problems

Although there is a generic `UnicodeError` exception, the error reported is almost always more specific: either a `UnicodeEncodeError` (when converting `str` to binary sequences) or a `UnicodeDecodeError` (when reading binary sequences into `str`). Loading Python modules may also generate a `SyntaxError` when the source encoding is unexpected. We’ll show how to handle all of these errors in the next sections.



The first thing to note when you get a Unicode error is the exact type of the exception. Is it a `UnicodeEncodeError`, a `UnicodeDecodeError`, or some other error (e.g., `SyntaxError`) that mentions an encoding problem? To solve the problem, you have to understand it first.

Coping with UnicodeEncodeError

Most non-UTF codecs handle only a small subset of the Unicode characters. When converting text to bytes, if a character is not defined in the target encoding, `UnicodeEncodeError` will be raised, unless special handling is provided by passing an `errors` argument to the encoding method or function. The behavior of the error handlers is shown in [Example 4-6](#).

3. As of September, 2014, [W3Techs: Usage of Character Encodings for Websites](#) claims that 81.4% of sites use UTF-8, while [Built With: Encoding Usage Statistics](#) estimates 79.4%.

Example 4-6. Encoding to bytes: success and error handling

```
>>> city = 'São Paulo'
>>> city.encode('utf_8') ❶
b'S\xc3\xa3o Paulo'
>>> city.encode('utf_16')
b'\xff\xfeS\x00\xe3\x00o\x00 \x00P\x00a\x00u\x00l\x00o\x00'
>>> city.encode('iso8859_1') ❷
b'S\xe3o Paulo'
>>> city.encode('cp437') ❸
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File ".../lib/python3.4/encodings/cp437.py", line 12, in encode
    return codecs.charmap_encode(input,errors,encoding_map)
UnicodeEncodeError: 'charmap' codec can't encode character '\xe3' in
position 1: character maps to <undefined>
>>> city.encode('cp437', errors='ignore') ❹
b'So Paulo'
>>> city.encode('cp437', errors='replace') ❺
b'S?o Paulo'
>>> city.encode('cp437', errors='xmlcharrefreplace') ❻
b'S&#227;o Paulo'
```

- ❶ The 'utf_?' encodings handle any str.
- ❷ 'iso8859_1' also works for the 'São Paulo' str.
- ❸ 'cp437' can't encode the 'ã' ("a" with tilde). The default error handler —'strict'—raises UnicodeEncodeError.
- ❹ The error='ignore' handler silently skips characters that cannot be encoded; this is usually a very bad idea.
- ❺ When encoding, error='replace' substitutes unencodable characters with '?'; data is lost, but users will know something is amiss.
- ❻ 'xmlcharrefreplace' replaces unencodable characters with an XML entity.



The codecs error handling is extensible. You may register extra strings for the errors argument by passing a name and an error handling function to the `codecs.register_error` function. See [the `codecs.register\_error` documentation](#).

Coping with UnicodeDecodeError

Not every byte holds a valid ASCII character, and not every byte sequence is valid UTF-8 or UTF-16; therefore, when you assume one of these encodings while converting a binary sequence to text, you will get a `UnicodeDecodeError` if unexpected bytes are found.

On the other hand, many legacy 8-bit encodings like 'cp1252', 'iso8859_1', and 'koi8_r' are able to decode any stream of bytes, including random noise, without generating errors. Therefore, if your program assumes the wrong 8-bit encoding, it will silently decode garbage.



Garbled characters are known as gremlins or mojibake (文字化け —Japanese for “transformed text”).

Example 4-7 illustrates how using the wrong codec may produce gremlins or a `UnicodeDecodeError`.

Example 4-7. Decoding from str to bytes: success and error handling

```
>>> octets = b'Montr\xe9al' ❶
>>> octets.decode('cp1252') ❷
'Montréal'
>>> octets.decode('iso8859_7') ❸
'Montrial'
>>> octets.decode('koi8_r') ❹
'MontrMal'
>>> octets.decode('utf_8') ❺
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 5:
invalid continuation byte
>>> octets.decode('utf_8', errors='replace') ❻
'Montr✠al'
```

- ❶ These bytes are the characters for “Montréal” encoded as `latin1`; `'\xe9'` is the byte for “é”.
- ❷ Decoding with `'cp1252'` (Windows 1252) works because it is a proper superset of `latin1`.
- ❸ ISO-8859-7 is intended for Greek, so the `'\xe9'` byte is misinterpreted, and no error is issued.
- ❹ KOI8-R is for Russian. Now `'\xe9'` stands for the Cyrillic letter “И”.
- ❺ The `'utf_8'` codec detects that `octets` is not valid UTF-8, and raises `UnicodeDecodeError`.
- ❻ Using `'replace'` error handling, the `\xe9` is replaced by “✠” (code point U+FFFD), the official Unicode REPLACEMENT CHARACTER intended to represent unknown characters.

SyntaxError When Loading Modules with Unexpected Encoding

UTF-8 is the default source encoding for Python 3, just as ASCII was the default for Python 2 (starting with 2.5). If you load a `.py` module containing non-UTF-8 data and no encoding declaration, you get a message like this:

```
SyntaxError: Non-UTF-8 code starting with '\xe1' in file ola.py on line
  1, but no encoding declared; see http://python.org/dev/peps/pep-0263/
for details
```

Because UTF-8 is widely deployed in GNU/Linux and OSX systems, a likely scenario is opening a `.py` file created on Windows with `cp1252`. Note that this error happens even in Python for Windows, because the default encoding for Python 3 is UTF-8 across all platforms.

To fix this problem, add a magic coding comment at the top of the file, as shown in [Example 4-8](#).

Example 4-8. ola.py: “Hello, World!” in Portuguese

```
# coding: cp1252
```

```
print('Olá, Mundo!')
```



Now that Python 3 source code is no longer limited to ASCII and defaults to the excellent UTF-8 encoding, the best “fix” for source code in legacy encodings like `'cp1252'` is to convert them to UTF-8 already, and not bother with the coding comments. If your editor does not support UTF-8, it's time to switch.

Non-ASCII Names in Source Code: Should You Use Them?

Python 3 allows non-ASCII identifiers in source code:

```
>>> ação = 'PBR' # ação = stock
>>> ε = 10**.-6 # ε = epsilon
```

Some people dislike the idea. The most common argument to stick with ASCII identifiers is to make it easy for everyone to read and edit code. That argument misses the point: you want your source code to be readable and editable by its intended audience, and that may not be “everyone.” If the code belongs to a multinational corporation or is open source and you want contributors from around the world, the identifiers should be in English, and then all you need is ASCII.

But if you are a teacher in Brazil, your students will find it easier to read code that uses Portuguese variable and function names, correctly spelled. And they will have no difficulty typing the cedillas and accented vowels on their localized keyboards.

Now that Python can parse Unicode names and UTF-8 is the default source encoding, I see no point in coding identifiers in Portuguese without accents, as we used to do in Python 2 out of necessity—unless you need the code to run on Python 2 also. If the names are in Portuguese, leaving out the accents won't make the code more readable to anyone.

This is my point of view as a Portuguese-speaking Brazilian, but I believe it applies across borders and cultures: choose the human language that makes the code easier to read by the team, then use the characters needed for correct spelling.

Suppose you have a text file, be it source code or poetry, but you don't know its encoding. How do you detect the actual encoding? The next section answers that with a library recommendation.

How to Discover the Encoding of a Byte Sequence

How do you find the encoding of a byte sequence? Short answer: you can't. You must be told.

Some communication protocols and file formats, like HTTP and XML, contain headers that explicitly tell us how the content is encoded. You can be sure that some byte streams are not ASCII because they contain byte values over 127, and the way UTF-8 and UTF-16 are built also limits the possible byte sequences. But even then, you can never be 100% positive that a binary file is ASCII or UTF-8 just because certain bit patterns are not there.

However, considering that human languages also have their rules and restrictions, once you assume that a stream of bytes is human *plain text* it may be possible to sniff out its encoding using heuristics and statistics. For example, if b'\x00' bytes are common, it is probably a 16- or 32-bit encoding, and not an 8-bit scheme, because null characters in plain text are bugs; when the byte sequence b'\x20\x00' appears often, it is likely to be the space character (U+0020) in a UTF-16LE encoding, rather than the obscure U+2000 EN QUAD character—whatever that is.

That is how the package **Chardet — The Universal Character Encoding Detector** works to identify one of 30 supported encodings. Chardet is a Python library that you can use in your programs, but also includes a command-line utility, `chardetect`. Here is what it reports on the source file for this chapter:

```
$ chardetect 04-text-byte.asciidoc
04-text-byte.asciidoc: utf-8 with confidence 0.99
```

Although binary sequences of encoded text usually don't carry explicit hints of their encoding, the UTF formats may prepend a byte order mark to the textual content. That is explained next.

BOM: A Useful Gremlin

In [Example 4-5](#), you may have noticed a couple of extra bytes at the beginning of a UTF-16 encoded sequence. Here they are again:

```
>>> u16 = 'El Niño'.encode('utf_16')
>>> u16
b'\xff\xfeE\x00l\x00 \x00N\x00i\x00\xfa\x00o\x00'
```

The bytes are `b'\xff\xfe'`. That is a *BOM*—byte-order mark—denoting the “little-endian” byte ordering of the Intel CPU where the encoding was performed.

On a little-endian machine, for each code point the least significant byte comes first: the letter 'E', code point U+0045 (decimal 69), is encoded in byte offsets 2 and 3 as 69 and 0:

```
>>> list(u16)
[255, 254, 69, 0, 108, 0, 32, 0, 78, 0, 105, 0, 241, 0, 111, 0]
```

On a big-endian CPU, the encoding would be reversed; 'E' would be encoded as 0 and 69.

To avoid confusion, the UTF-16 encoding prepends the text to be encoded with the special character ZERO WIDTH NO-BREAK SPACE (U+FEFF), which is invisible. On a little-endian system, that is encoded as `b'\xff\xfe'` (decimal 255, 254). Because, by design, there is no U+FFFE character, the byte sequence `b'\xff\xfe'` must mean the ZERO WIDTH NO-BREAK SPACE on a little-endian encoding, so the codec knows which byte ordering to use.

There is a variant of UTF-16—UTF-16LE—that is explicitly little-endian, and another one explicitly big-endian, UTF-16BE. If you use them, a BOM is not generated:

```
>>> u16le = 'El Niño'.encode('utf_16le')
>>> list(u16le)
[69, 0, 108, 0, 32, 0, 78, 0, 105, 0, 241, 0, 111, 0]
>>> u16be = 'El Niño'.encode('utf_16be')
>>> list(u16be)
[0, 69, 0, 108, 0, 32, 0, 78, 0, 105, 0, 241, 0, 111]
```

If present, the BOM is supposed to be filtered by the UTF-16 codec, so that you only get the actual text contents of the file without the leading ZERO WIDTH NO-BREAK SPACE. The standard says that if a file is UTF-16 and has no BOM, it should be assumed to be UTF-16BE (big-endian). However, the Intel x86 architecture is little-endian, so there is plenty of little-endian UTF-16 with no BOM in the wild.

This whole issue of endianness only affects encodings that use words of more than one byte, like UTF-16 and UTF-32. One big advantage of UTF-8 is that it produces the same byte sequence regardless of machine endianness, so no BOM is needed. Nevertheless, some Windows applications (notably Notepad) add the BOM to UTF-8 files anyway—and Excel depends on the BOM to detect a UTF-8 file, otherwise it assumes the content

is encoded with a Windows codepage. The character U+FEFF encoded in UTF-8 is the three-byte sequence `b'\xef\xbb\xbf'`. So if a file starts with those three bytes, it is likely to be a UTF-8 file with a BOM. However, Python does not automatically assume a file is UTF-8 just because it starts with `b'\xef\xbb\xbf'`.

We now move on to handling text files in Python 3.

Handling Text Files

The best practice for handling text is the “Unicode sandwich” (Figure 4-2).⁴ This means that bytes should be decoded to `str` as early as possible on input (e.g., when opening a file for reading). The “meat” of the sandwich is the business logic of your program, where text handling is done exclusively on `str` objects. You should never be encoding or decoding in the middle of other processing. On output, the `str` are encoded to bytes as late as possible. Most web frameworks work like that, and we rarely touch bytes when using them. In Django, for example, your views should output Unicode `str`; Django itself takes care of encoding the response to bytes, using UTF-8 by default.

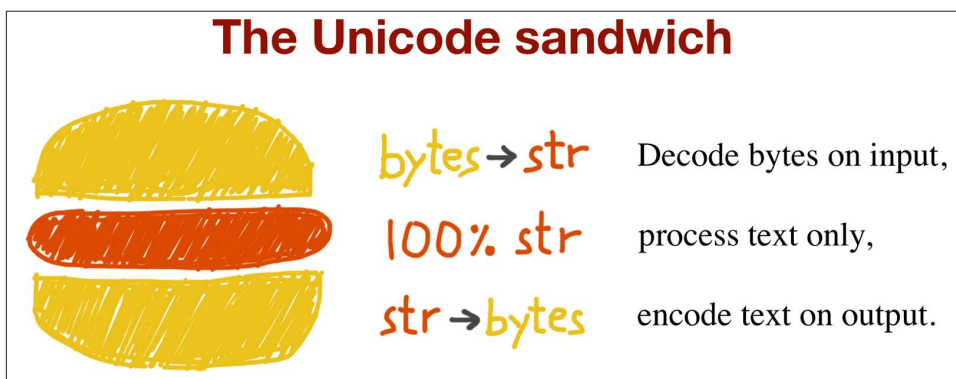


Figure 4-2. Unicode sandwich: current best practice for text processing

Python 3 makes it easier to follow the advice of the Unicode sandwich, because the open built-in does the necessary decoding when reading and encoding when writing files in text mode, so all you get from `my_file.read()` and pass to `my_file.write(text)` are `str` objects.⁵

4. I first saw the term “Unicode sandwich” in Ned Batchelder’s excellent “[Pragmatic Unicode](#)” talk at US PyCon 2012.

5. Python 2.6 or 2.7 users have to use `io.open()` to get automatic decoding/encoding when reading/writing.

Therefore, using text files is simple. But if you rely on default encodings you will get bitten.

Consider the console session in [Example 4-9](#). Can you spot the bug?

Example 4-9. A platform encoding issue (if you try this on your machine, you may or may not see the problem)

```
>>> open('cafe.txt', 'w', encoding='utf_8').write('café')
4
>>> open('cafe.txt').read()
'cafÃ©'
```

The bug: I specified UTF-8 encoding when writing the file but failed to do so when reading it, so Python assumed the system default encoding—Windows 1252—and the trailing bytes in the file were decoded as characters 'Ã©' instead of 'é'.

I ran [Example 4-9](#) on a Windows 7 machine. The same statements running on recent GNU/Linux or Mac OSX work perfectly well because their default encoding is UTF-8, giving the false impression that everything is fine. If the encoding argument was omitted when opening the file to write, the locale default encoding would be used, and we'd read the file correctly using the same encoding. But then this script would generate files with different byte contents depending on the platform or even depending on locale settings in the same platform, creating compatibility problems.



Code that has to run on multiple machines or on multiple occasions should never depend on encoding defaults. Always pass an explicit `encoding=` argument when opening text files, because the default may change from one machine to the next, or from one day to the next.

A curious detail in [Example 4-9](#) is that the `write` function in the first statement reports that four characters were written, but in the next line five characters are read. [Example 4-10](#) is an extended version of [Example 4-9](#), explaining that and other details.

Example 4-10. Closer inspection of [Example 4-9](#) running on Windows reveals the bug and how to fix it

```
>>> fp = open('cafe.txt', 'w', encoding='utf_8')
>>> fp ❶
<_io.TextIOWrapper name='cafe.txt' mode='w' encoding='utf_8'>
>>> fp.write('café')
4 ❷
>>> fp.close()
>>> import os
>>> os.stat('cafe.txt').st_size
5 ❸
>>> fp2 = open('cafe.txt')
```



```

>>> fp2 ④
<_io.TextIOWrapper name='cafe.txt' mode='r' encoding='cp1252'>
>>> fp2.encoding ⑤
'cp1252'
>>> fp2.read()
'cafÃ©' ⑥
>>> fp3 = open('cafe.txt', encoding='utf_8') ⑦
>>> fp3
<_io.TextIOWrapper name='cafe.txt' mode='r' encoding='utf_8'>
>>> fp3.read()
'café' ⑧
>>> fp4 = open('cafe.txt', 'rb') ⑨
>>> fp4
<_io.BufferedReader name='cafe.txt'> ⑩
>>> fp4.read() ⑪
b'caf\xc3\xa9'

```

- ① By default, open operates in text mode and returns a TextIOWrapper object.
- ② The write method on a TextIOWrapper returns the number of Unicode characters written.
- ③ os.stat reports that the file holds 5 bytes; UTF-8 encodes 'é' as 2 bytes, 0xc3 and 0xa9.
- ④ Opening a text file with no explicit encoding returns a TextIOWrapper with the encoding set to a default from the locale.
- ⑤ A TextIOWrapper object has an encoding attribute that you can inspect: cp1252 in this case.
- ⑥ In the Windows cp1252 encoding, the byte 0xc3 is an “Ã” (A with tilde) and 0xa9 is the copyright sign.
- ⑦ Opening the same file with the correct encoding.
- ⑧ The expected result: the same four Unicode characters for 'café'.
- ⑨ The 'rb' flag opens a file for reading in binary mode.
- ⑩ The returned object is a BufferedReader and not a TextIOWrapper.
- ⑪ Reading that returns bytes, as expected.



Do not open text files in binary mode unless you need to analyze the file contents to determine the encoding—even then, you should be using Chardet instead of reinventing the wheel (see “[How to Discover the Encoding of a Byte Sequence](#)” on page 109). Ordinary code should only use binary mode to open binary files, like raster images.

The problem in [Example 4-10](#) has to do with relying on a default setting while opening a text file. There are several sources for such defaults, as the next section shows.

Encoding Defaults: A Madhouse

Several settings affect the encoding defaults for I/O in Python. See the *default_encodings.py* script in [Example 4-11](#).

Example 4-11. Exploring encoding defaults

```
import sys, locale

expressions = """
    locale.getpreferredencoding()
    type(my_file)
    my_file.encoding
    sys.stdout.isatty()
    sys.stdout.encoding
    sys.stdin.isatty()
    sys.stdin.encoding
    sys.stderr.isatty()
    sys.stderr.encoding
    sys.getdefaultencoding()
    sys.getfilesystemencoding()
"""

my_file = open('dummy', 'w')

for expression in expressions.split():
    value = eval(expression)
    print(expression.rjust(30), '->', repr(value))
```

The output of [Example 4-11](#) on GNU/Linux (Ubuntu 14.04) and OSX (Mavericks 10.9) is identical, showing that UTF-8 is used everywhere in these systems:

```
$ python3 default_encodings.py
locale.getpreferredencoding() -> 'UTF-8'
type(my_file) -> <class '_io.TextIOWrapper'>
my_file.encoding -> 'UTF-8'
sys.stdout.isatty() -> True
sys.stdout.encoding -> 'UTF-8'
sys.stdin.isatty() -> True
sys.stdin.encoding -> 'UTF-8'
sys.stderr.isatty() -> True
sys.stderr.encoding -> 'UTF-8'
sys.getdefaultencoding() -> 'utf-8'
sys.getfilesystemencoding() -> 'utf-8'
```

On Windows, however, the output is [Example 4-12](#).

Example 4-12. Default encodings on Windows 7 (SP 1) cmd.exe localized for Brazil; PowerShell gives same result

```
Z:\>chcp ❶
Página de código ativa: 850
Z:\>python default_encodings.py ❷
  locale.getpreferredencoding() -> 'cp1252' ❸
      type(my_file) -> <class '_io.TextIOWrapper'>
      my_file.encoding -> 'cp1252' ❹
  sys.stdout.isatty() -> True ❺
  sys.stdout.encoding -> 'cp850' ❻
  sys.stdin.isatty() -> True
  sys.stdin.encoding -> 'cp850'
  sys.stderr.isatty() -> True
  sys.stderr.encoding -> 'cp850'
  sys.getdefaultencoding() -> 'utf-8'
  sys.getfilesystemencoding() -> 'mbcs'
```

- ❶ chcp shows the active codepage for the console: 850.
- ❷ Running *default_encodings.py* with output to console.
- ❸ `locale.getpreferredencoding()` is the most important setting.
- ❹ Text files use `locale.getpreferredencoding()` by default.
- ❺ The output is going to the console, so `sys.stdout.isatty()` is True.
- ❻ Therefore, `sys.stdout.encoding` is the same as the console encoding.

If the output is redirected to a file, like this:

```
Z:\>python default_encodings.py > encodings.log
```

The value of `sys.stdout.isatty()` becomes False, and `sys.stdout.encoding` is set by `locale.getpreferredencoding()`, 'cp1252' in that machine.

Note that there are four different encodings in [Example 4-12](#):

- If you omit the encoding argument when opening a file, the default is given by `locale.getpreferredencoding()` ('cp1252' in [Example 4-12](#)).
- The encoding of `sys.stdout/stdin/stderr` is given by the **PYTHONIOENCODING environment variable**, if present, otherwise it is either inherited from the console or defined by `locale.getpreferredencoding()` if the output/input is redirected to/from a file.

- `sys.getdefaultencoding()` is used internally by Python to convert binary data to/from `str`; this happens less often in Python 3, but still happens.⁶ Changing this setting is not supported.⁷
- `sys.getfilesystemencoding()` is used to encode/decode filenames (not file contents). It is used when `open()` gets a `str` argument for the filename; if the filename is given as a bytes argument, it is passed unchanged to the OS API. The Python [Unicode HOWTO](#) says: “on Windows, Python uses the name `mbscs` to refer to whatever the currently configured encoding is.” The acronym MBCS stands for Multi Byte Character Set, which for Microsoft are the legacy variable-width encodings like `gb2312` or `Shift_JIS`, but not UTF-8. (On this topic, a useful answer on Stack-Overflow is “[Difference between MBCS and UTF-8 on Windows](#)”.)



On GNU/Linux and OSX all of these encodings are set to UTF-8 by default, and have been for several years, so I/O handles all Unicode characters. On Windows, not only are different encodings used in the same system, but they are usually codepages like 'cp850' or 'cp1252' that support only ASCII with 127 additional characters that are not the same from one encoding to the other. Therefore, Windows users are far more likely to face encoding errors unless they are extra careful.

To summarize, the most important encoding setting is that returned by `locale.getpreferredencoding()`: it is the default for opening text files and for `sys.stdout/stdin/stderr` when they are redirected to files. However, the [documentation](#) reads (in part):

```
locale.getpreferredencoding(do_setlocale=True)
```

Return the encoding used for text data, according to user preferences. User preferences are expressed differently on different systems, and might not be available programmatically on some systems, so this function only returns a guess. [...]

Therefore, the best advice about encoding defaults is: do not rely on them.

If you follow the advice of the Unicode sandwich and always are explicit about the encodings in your programs, you will avoid a lot of pain. Unfortunately, Unicode is

6. While researching this subject, I did not find a list of situations when Python 3 internally converts `bytes` to `str`. Python core developer Antoine Pitrou says on the [comp.python.devel](#) list that CPython internal functions that depend on such conversions “don’t get a lot of use in py3k.”
7. The Python 2 `sys.setdefaultencoding` function was misused and is no longer documented in Python 3. It was intended for use by the core developers when the internal default encoding of Python was still undecided. In the same [comp.python.devel](#) thread, Marc-André Lemburg states that the `sys.setdefaultencoding` must never be called by user code and the only values supported by CPython are 'ascii' in Python 2 and 'utf-8' in Python 3.

painful even if you get your bytes correctly converted to `str`. The next two sections cover subjects that are simple in ASCII-land, but get quite complex on planet Unicode: text normalization (i.e., converting text to a uniform representation for comparisons) and sorting.

Normalizing Unicode for Saner Comparisons

String comparisons are complicated by the fact that Unicode has combining characters: diacritics and other marks that attach to the preceding character, appearing as one when printed.

For example, the word “café” may be composed in two ways, using four or five code points, but the result looks exactly the same:

```
>>> s1 = 'café'
>>> s2 = 'cafe\u0301'
>>> s1, s2
('café', 'café')
>>> len(s1), len(s2)
(4, 5)
>>> s1 == s2
False
```

The code point U+0301 is the COMBINING ACUTE ACCENT. Using it after “e” renders “é”. In the Unicode standard, sequences like ‘é’ and ‘e\u0301’ are called “canonical equivalents,” and applications are supposed to treat them as the same. But Python sees two different sequences of code points, and considers them not equal.

The solution is to use Unicode normalization, provided by the `unicodedata.normalize` function. The first argument to that function is one of four strings: ‘NFC’, ‘NFD’, ‘NFKC’, and ‘NFKD’. Let’s start with the first two.

Normalization Form C (NFC) composes the code points to produce the shortest equivalent string, while NFD decomposes, expanding composed characters into base characters and separate combining characters. Both of these normalizations make comparisons work as expected:

```
>>> from unicodedata import normalize
>>> s1 = 'café' # composed "e" with acute accent
>>> s2 = 'cafe\u0301' # decomposed "e" and acute accent
>>> len(s1), len(s2)
(4, 5)
>>> len(normalize('NFC', s1)), len(normalize('NFC', s2))
(4, 4)
>>> len(normalize('NFD', s1)), len(normalize('NFD', s2))
(5, 5)
>>> normalize('NFC', s1) == normalize('NFC', s2)
True
```

```
>>> normalize('NFD', s1) == normalize('NFD', s2)
True
```

Western keyboards usually generate composed characters, so text typed by users will be in NFC by default. However, to be safe, it may be good to sanitize strings with `normalize('NFC', user_text)` before saving. NFC is also the normalization form recommended by the W3C in [Character Model for the World Wide Web: String Matching and Searching](#).

Some single characters are normalized by NFC into another single character. The symbol for the ohm (Ω) unit of electrical resistance is normalized to the Greek uppercase omega. They are visually identical, but they compare unequal so it is essential to normalize to avoid surprises:

```
>>> from unicodedata import normalize, name
>>> ohm = '\u2126'
>>> name(ohm)
'OHM SIGN'
>>> ohm_c = normalize('NFC', ohm)
>>> name(ohm_c)
'GREEK CAPITAL LETTER OMEGA'
>>> ohm == ohm_c
False
>>> normalize('NFC', ohm) == normalize('NFC', ohm_c)
True
```

In the acronyms for the other two normalization forms—NFKC and NFKD—the letter K stands for “compatibility.” These are stronger forms of normalization, affecting the so-called “compatibility characters.” Although one goal of Unicode is to have a single “canonical” code point for each character, some characters appear more than once for compatibility with preexisting standards. For example, the micro sign, ‘μ’ (U+00B5), was added to Unicode to support round-trip conversion to latin1, even though the same character is part of the Greek alphabet with code point U+03BC (GREEK SMALL LETTER MU). So, the micro sign is considered a “compatibility character.”

In the NFKC and NFKD forms, each compatibility character is replaced by a “compatibility decomposition” of one or more characters that are considered a “preferred” representation, even if there is some formatting loss—ideally, the formatting should be the responsibility of external markup, not part of Unicode. To exemplify, the compatibility decomposition of the one half fraction ‘½’ (U+00BD) is the sequence of three characters ‘1/2’, and the compatibility decomposition of the micro sign ‘μ’ (U+00B5) is the lowercase mu ‘μ’ (U+03BC).⁸

8. Curiously, the micro sign is considered a “compatibility character” but the ohm symbol is not. The end result is that NFC doesn’t touch the micro sign but changes the ohm symbol to capital omega, while NFKC and NFKD change both the ohm and the micro into other characters.

Here is how the NFKC works in practice:

```
>>> from unicodedata import normalize, name
>>> half = '½'
>>> normalize('NFKC', half)
'1/2'
>>> four_squared = '4²'
>>> normalize('NFKC', four_squared)
'42'
>>> micro = 'μ'
>>> micro_kc = normalize('NFKC', micro)
>>> micro, micro_kc
('μ', 'μ')
>>> ord(micro), ord(micro_kc)
(181, 956)
>>> name(micro), name(micro_kc)
('MICRO SIGN', 'GREEK SMALL LETTER MU')
```

Although '1/2' is a reasonable substitute for '½', and the micro sign is really a lowercase Greek mu, converting '4²' to '42' changes the meaning. An application could store '4²' as '4²', but the `normalize` function knows nothing about formatting. Therefore, NFKC or NFKD may lose or distort information, but they can produce convenient intermediate representations for searching and indexing: users may be pleased that a search for '1/2 inch' also finds documents containing '½ inch'.



NFKC and NFKD normalization should be applied with care and only in special cases—e.g., search and indexing—and not for permanent storage, because these transformations cause data loss.

When preparing text for searching or indexing, another operation is useful: case folding, our next subject.

Case Folding

Case folding is essentially converting all text to lowercase, with some additional transformations. It is supported by the `str.casefold()` method (new in Python 3.3).

For any string `s` containing only latin1 characters, `s.casefold()` produces the same result as `s.lower()`, with only two exceptions—the micro sign 'μ' is changed to the Greek lowercase mu (which looks the same in most fonts) and the German Eszett or “sharp s” (ß) becomes “ss”:

```
>>> micro = 'μ'
>>> name(micro)
'MICRO SIGN'
>>> micro_cf = micro.casefold()
>>> name(micro_cf)
```

```

'GREEK SMALL LETTER MU'
>>> micro, micro_cf
('μ', 'μ')
>>> eszett = 'ß'
>>> name(eszett)
'LATIN SMALL LETTER SHARP S'
>>> eszett_cf = eszett.casefold()
>>> eszett, eszett_cf
('ß', 'ss')

```

As of Python 3.4, there are 116 code points for which `str.casefold()` and `str.lower()` return different results. That's 0.11% of a total of 110,122 named characters in Unicode 6.3.

As usual with anything related to Unicode, case folding is a complicated issue with plenty of linguistic special cases, but the Python core team made an effort to provide a solution that hopefully works for most users.

In the next couple of sections, we'll put our normalization knowledge to use developing utility functions.

Utility Functions for Normalized Text Matching

As we've seen, NFC and NFD are safe to use and allow sensible comparisons between Unicode strings. NFC is the best normalized form for most applications. `str.casefold()` is the way to go for case-insensitive comparisons.

If you work with text in many languages, a pair of functions like `nfc_equal` and `fold_equal` in [Example 4-13](#) are useful additions to your toolbox.

Example 4-13. `normeq.py`: normalized Unicode string comparison

```

"""
Utility functions for normalized Unicode string comparison.

```

Using Normal Form C, case sensitive:

```

>>> s1 = 'café'
>>> s2 = 'cafe\u0301'
>>> s1 == s2
False
>>> nfc_equal(s1, s2)
True
>>> nfc_equal('A', 'a')
False

```

Using Normal Form C with case folding:

```

>>> s3 = 'Straße'
>>> s4 = 'strasse'
>>> s3 == s4

```



```

False
>>> nfc_equal(s3, s4)
False
>>> fold_equal(s3, s4)
True
>>> fold_equal(s1, s2)
True
>>> fold_equal('A', 'a')
True
"""

from unicodedata import normalize

def nfc_equal(str1, str2):
    return normalize('NFC', str1) == normalize('NFC', str2)

def fold_equal(str1, str2):
    return (normalize('NFC', str1).casefold() ==
            normalize('NFC', str2).casefold())

```

Beyond Unicode normalization and case folding—which are both part of the Unicode standard—sometimes it makes sense to apply deeper transformations, like changing 'café' into 'cafe'. We'll see when and how in the next section.

Extreme “Normalization”: Taking Out Diacritics

The Google Search secret sauce involves many tricks, but one of them apparently is ignoring diacritics (e.g., accents, cedillas, etc.), at least in some contexts. Removing diacritics is not a proper form of normalization because it often changes the meaning of words and may produce false positives when searching. But it helps coping with some facts of life: people sometimes are lazy or ignorant about the correct use of diacritics, and spelling rules change over time, meaning that accents come and go in living languages.

Outside of searching, getting rid of diacritics also makes for more readable URLs, at least in Latin-based languages. Take a look at the URL for the Wikipedia article about the city of São Paulo:

http://en.wikipedia.org/wiki/S%C3%A3o_Paulo

The %C3%A3 part is the URL-escaped, UTF-8 rendering of the single letter “ã” (“a” with tilde). The following is much friendlier, even if it is not the right spelling:

http://en.wikipedia.org/wiki/Sao_Paulo

To remove all diacritics from a `str`, you can use a function like [Example 4-14](#).

Example 4-14. Function to remove all combining marks (module *sanitize.py*)

```
import unicodedata
import string

def shave_marks(txt):
    """Remove all diacritic marks"""
    norm_txt = unicodedata.normalize('NFD', txt) ❶
    shaved = ''.join(c for c in norm_txt
                     if not unicodedata.combining(c)) ❷
    return unicodedata.normalize('NFC', shaved) ❸
```

- ❶ Decompose all characters into base characters and combining marks.
- ❷ Filter out all combining marks.
- ❸ Recompose all characters.

Example 4-15 shows a couple of uses of `shave_marks`.

Example 4-15. Two examples using `shave_marks` from Example 4-14

```
>>> order = "Herr Voß: • ½ cup of Etker™ caffè latte • bowl of açai."
>>> shave_marks(order)
'Herr Voß: • ½ cup of Etker™ caffè latte • bowl of acai.' ❶
>>> Greek = 'Ζέφυρος, Ζέφиро'
>>> shave_marks(Greek)
'Ζεφυρος, Zefiro' ❷
```

- ❶ Only the letters “è”, “ç”, and “ï” were replaced.
- ❷ Both “ξ” and “é” were replaced.

The function `shave_marks` from Example 4-14 works all right, but maybe it goes too far. Often the reason to remove diacritics is to change Latin text to pure ASCII, but `shave_marks` also changes non-Latin characters—like Greek letters—which will never become ASCII just by losing their accents. So it makes sense to analyze each base character and to remove attached marks only if the base character is a letter from the Latin alphabet. This is what Example 4-16 does.

Example 4-16. Function to remove combining marks from Latin characters (import statements are omitted as this is part of the *sanitize.py* module from Example 4-14)

```
def shave_marks_latin(txt):
    """Remove all diacritic marks from Latin base characters"""
    norm_txt = unicodedata.normalize('NFD', txt) ❶
    latin_base = False
    keepers = []
    for c in norm_txt:
        if unicodedata.combining(c) and latin_base: ❷
            continue # ignore diacritic on Latin base char
```

```

        keepers.append(c) ❸
        # if it isn't combining char, it's a new base char
        if not unicodedata.combining(c): ❹
            latin_base = c in string.ascii_letters
    shaved = ''.join(keepers)
    return unicodedata.normalize('NFC', shaved) ❺

```

- ❶ Decompose all characters into base characters and combining marks.
- ❷ Skip over combining marks when base character is Latin.
- ❸ Otherwise, keep current character.
- ❹ Detect new base character and determine if it's Latin.
- ❺ Recompose all characters.

An even more radical step would be to replace common symbols in Western texts (e.g., curly quotes, em dashes, bullets, etc.) into ASCII equivalents. This is what the function `asciize` does in [Example 4-17](#).

Example 4-17. Transform some Western typographical symbols into ASCII (this snippet is also part of `sanitize.py` from [Example 4-14](#))

```

single_map = str.maketrans( ❶
    """f„†^‘ ’“”•—~"""
    """f"*^<' ' " _ - - ~>""")

multi_map = str.maketrans({ ❷
    '€': '<euro>',
    '…': '...',
    'Œ': 'OE',
    '™': '(TM)',
    'œ': 'oe',
    '‰': '<per mille>',
    '‡': '**',
})

multi_map.update(single_map) ❸

def dewinize(txt):
    """Replace Win1252 symbols with ASCII chars or sequences"""
    return txt.translate(multi_map) ❹

def asciize(txt):
    no_marks = shave_marks_latin(dewinize(txt)) ❺
    no_marks = no_marks.replace('ß', 'ss') ❻
    return unicodedata.normalize('NFKC', no_marks) ❼

```

- ❶ Build mapping table for char-to-char replacement.
- ❷ Build mapping table for char-to-string replacement.

- ③ Merge mapping tables.
- ④ `dewinize` does not affect ASCII or `latin1` text, only the Microsoft additions in `latin1` in `cp1252`.
- ⑤ Apply `dewinize` and remove diacritical marks.
- ⑥ Replace the Eszett with “ss” (we are not using case fold here because we want to preserve the case).
- ⑦ Apply NFKC normalization to compose characters with their compatibility code points.

Example 4-18 shows `asciize` in use.

Example 4-18. Two examples using `asciize` from Example 4-17

```
>>> order = "Herr Voß: • ½ cup of OEtker™ caffè latte • bowl of açai."
>>> dewinize(order)
'Herr Voß: - ½ cup of OEtker(TM) caffè latte - bowl of açai.' ①
>>> asciize(order)
'Herr Voss: - 1/2 cup of OEtker(TM) caffè latte - bowl of açai.' ②
```

- ① `dewinize` replaces curly quotes, bullets, and [™] (trademark symbol).
- ② `asciize` applies `dewinize`, drops diacritics, and replaces the 'ß'.



Different languages have their own rules for removing diacritics. For example, Germans change the 'ü' into 'ue'. Our `asciize` function is not as refined, so it may or not be suitable for your language. It works acceptably for Portuguese, though.

To summarize, the functions in `sanitize.py` go way beyond standard normalization and perform deep surgery on the text, with a good chance of changing its meaning. Only you can decide whether to go so far, knowing the target language, your users, and how the transformed text will be used.

This wraps up our discussion of normalizing Unicode text.

The next Unicode matter to sort out is... sorting.

Sorting Unicode Text

Python sorts sequences of any type by comparing the items in each sequence one by one. For strings, this means comparing the code points. Unfortunately, this produces unacceptable results for anyone who uses non-ASCII characters.

Consider sorting a list of fruits grown in Brazil:

```
>>> fruits = ['caju', 'atemoia', 'cajá', 'açai', 'acerola']
>>> sorted(fruits)
['acerola', 'atemoia', 'açai', 'caju', 'cajá']
```

Sorting rules vary for different locales, but in Portuguese and many languages that use the Latin alphabet, accents and cedillas rarely make a difference when sorting.⁹ So “cajá” is sorted as “caja,” and must come before “caju.”

The sorted fruits list should be:

```
['açai', 'acerola', 'atemoia', 'cajá', 'caju']
```

The standard way to sort non-ASCII text in Python is to use the `locale.strxfrm` function which, according to the [locale module docs](#), “transforms a string to one that can be used in locale-aware comparisons.”

To enable `locale.strxfrm`, you must first set a suitable locale for your application, and pray that the OS supports it. On GNU/Linux (Ubuntu 14.04) with the `pt_BR` locale, the sequence of commands in [Example 4-19](#) works.

Example 4-19. Using the `locale.strxfrm` function as sort key

```
>>> import locale
>>> locale.setlocale(locale.LC_COLLATE, 'pt_BR.UTF-8')
'pt_BR.UTF-8'
>>> fruits = ['caju', 'atemoia', 'cajá', 'açai', 'acerola']
>>> sorted_fruits = sorted(fruits, key=locale.strxfrm)
>>> sorted_fruits
['açai', 'acerola', 'atemoia', 'cajá', 'caju']
```

So you need to call `setlocale(LC_COLLATE, «your_locale»)` before using `locale.strxfrm` as the key when sorting.

There are a few caveats, though:

- Because locale settings are global, calling `setlocale` in a library is not recommended. Your application or framework should set the locale when the process starts, and should not change it afterwards.
- The locale must be installed on the OS, otherwise `setlocale` raises a `locale.Error: unsupported locale setting` exception.
- You must know how to spell the locale name. They are pretty much standardized in the Unix derivatives as `'language_code.encoding'`, but on Windows the syntax is more complicated: `Language Name-Language Variant_Region Name.code page>`. Note that the Language Name, Language Variant, and Region Name parts can have spaces inside them, but the parts after the first are prefixed with special

9. Diacritics affect sorting only in the rare case when they are the only difference between two words—in that case, the word with a diacritic is sorted after the plain word.

different characters: a hyphen, an underline character, and a dot. All parts seem to be optional except the language name. For example: `English_United_States.850` means Language Name “English”, region “United States”, and codepage “850”. The language and region names Windows understands are listed in the MSDN article [Language Identifier Constants and Strings](#), while [Code Page Identifiers](#) lists the numbers for the last part.¹⁰

- The locale must be correctly implemented by the makers of the OS. I was successful on Ubuntu 14.04, but not on OSX (Mavericks 10.9). On two different Macs, the call `setlocale(LC_COLLATE, 'pt_BR.UTF-8')` returns the string `'pt_BR.UTF-8'` with no complaints. But `sorted(fruits, key=locale.strxfrm)` produced the same incorrect result as `sorted(fruits)` did. I also tried the `fr_FR`, `es_ES`, and `de_DE` locales on OSX, but `locale.strxfrm` never did its job.¹¹

So the standard library solution to internationalized sorting works, but seems to be well supported only on GNU/Linux (perhaps also on Windows, if you are an expert). Even then, it depends on locale settings, creating deployment headaches.

Fortunately, there is a simpler solution: the PyUCA library, available on *PyPI*.

Sorting with the Unicode Collation Algorithm

James Tauber, prolific Django contributor, must have felt the pain and created [PyUCA](#), a pure-Python implementation of the Unicode Collation Algorithm (UCA). [Example 4-20](#) shows how easy it is to use.

Example 4-20. Using the `pyuca.Collator.sort_key` method

```
>>> import pyuca
>>> coll = pyuca.Collator()
>>> fruits = ['caju', 'atemoia', 'cajá', 'açai', 'acerola']
>>> sorted_fruits = sorted(fruits, key=coll.sort_key)
>>> sorted_fruits
['açai', 'acerola', 'atemoia', 'cajá', 'caju']
```

This is friendly and just works. I tested it on GNU/Linux, OSX, and Windows. Only Python 3.X is supported at this time.

PyUCA does not take the locale into account. If you need to customize the sorting, you can provide the path to a custom collation table to the `Collator()` constructor. Out of

10. Thanks to Leonardo Rachael who went beyond his duties as tech reviewer and researched these Windows details, even though he is a GNU/Linux user himself.

11. Again, I could not find a solution, but did find other people reporting the same problem. Alex Martelli, one of the tech reviewers, had no problem using `setlocale` and `locale.strxfrm` on his Mac with OSX 10.9. In summary: your mileage may vary.

the box, it uses `allkeys.txt`, which is bundled with the project. That's just a copy of the **Default Unicode Collation Element Table** from Unicode 6.3.0.

By the way, that table is one of the many that comprise the Unicode database, our next subject.

The Unicode Database

The Unicode standard provides an entire database—in the form of numerous structured text files—that includes not only the table mapping code points to character names, but also metadata about the individual characters and how they are related. For example, the Unicode database records whether a character is printable, is a letter, is a decimal digit, or is some other numeric symbol. That's how the `str` methods `isidentifier`, `isprintable`, `isdecimal`, and `isnumeric` work. `str.casefold` also uses information from a Unicode table.

The `unicodedata` module has functions that return character metadata; for instance, its official name in the standard, whether it is a combining character (e.g., diacritic like a combining tilde), and the numeric value of the symbol for humans (not its code point).

Example 4-21 shows the use of `unicodedata.name()` and `unicodedata.numeric()` along with the `.isdecimal()` and `.isnumeric()` methods of `str`.

Example 4-21. Demo of Unicode database numerical character metadata (callouts describe each column in the output)

```
import unicodedata
import re

re_digit = re.compile(r'\d')

sample = '1\xbc\x2\xu0969\u136b\u216b\u2466\u2480\u3285'

for char in sample:
    print('U+%04x' % ord(char),          ❶
          char.center(6),                ❷
          're_dig' if re_digit.match(char) else '-', ❸
          'isdig' if char.isdigit() else '-', ❹
          'isnum' if char.isnumeric() else '-', ❺
          format(unicodedata.numeric(char), '5.2f'), ❻
          unicodedata.name(char),        ❼
          sep='\t')
```

- ❶ Code point in U+0000 format.
- ❷ Character centralized in a `str` of length 6.
- ❸ Show `re_dig` if character matches the `r'\d'` regex.
- ❹ Show `isdig` if `char.isdigit()` is `True`.

- 5 Show `isnum` if `char.isnumeric()` is `True`.
- 6 Numeric value formatted with width 5 and 2 decimal places.
- 7 Unicode character name.

Running [Example 4-21](#) gives you the result in [Figure 4-3](#).

```

$ python3 numerics_demo.py
U+0031  1      re_dig isdig isnum  1.00  DIGIT ONE
U+00bc  ¼      -      -      isnum  0.25  VULGAR FRACTION ONE QUARTER
U+00b2  ²      -      isdig isnum  2.00  SUPERScript TWO
U+0969  ३      re_dig isdig isnum  3.00  DEVANAGARI DIGIT THREE
U+136b  ፫      -      isdig isnum  3.00  ETHIOPIC DIGIT THREE
U+216b  XII     -      -      isnum 12.00  ROMAN NUMERAL TWELVE
U+2466  ⑦      -      isdig isnum  7.00  CIRCLED DIGIT SEVEN
U+2480  ⑬      -      -      isnum 13.00  PARENTHEZIZED NUMBER THIRTEEN
U+3285  ⑆      -      -      isnum  6.00  CIRCLED IDEOGRAPH SIX
$

```

Figure 4-3. Nine numeric characters and metadata about them; `re_dig` means the character matches the regular expression `r'\d'`;

The sixth column of [Figure 4-3](#) is the result of calling `unicodedata.numeric(char)` on the character. It shows that Unicode knows the numeric value of symbols that represent numbers. So if you want to create a spreadsheet application that supports Tamil digits or Roman numerals, go for it!

[Figure 4-3](#) shows that the regular expression `r'\d'` matches the digit “1” and the Devanagari digit 3, but not some other characters that are considered digits by the `isdig` function. The `re` module is not as savvy about Unicode as it could be. The new `regex` module available in PyPI was designed to eventually replace `re` and provides better Unicode support.¹² We’ll come back to the `re` module in the next section.

Throughout this chapter we’ve used several `unicodedata` functions, but there are many more we did not cover. See the standard library documentation for the [unicodedata module](#).

We will wrap up our tour of `str` versus `bytes` with a quick look at a new trend: dual-mode APIs offering functions that accept `str` or `bytes` arguments with special handling depending on the type.

12. Although it was not better than `re` at identifying digits in this particular sample.

Dual-Mode str and bytes APIs

The standard library has functions that accept str or bytes arguments and behave differently depending on the type. Some examples are in the re and os modules.

str Versus bytes in Regular Expressions

If you build a regular expression with bytes, patterns such as `\d` and `\w` only match ASCII characters; in contrast, if these patterns are given as str, they match Unicode digits or letters beyond ASCII. [Example 4-22](#) and [Figure 4-4](#) compare how letters, ASCII digits, superscripts, and Tamil digits are matched by str and bytes patterns.

Example 4-22. ramanujan.py: compare behavior of simple str and bytes regular expressions

```
import re

re_numbers_str = re.compile(r'\d+') ❶
re_words_str = re.compile(r'\w+')
re_numbers_bytes = re.compile(rb'\d+') ❷
re_words_bytes = re.compile(rb'\w+')

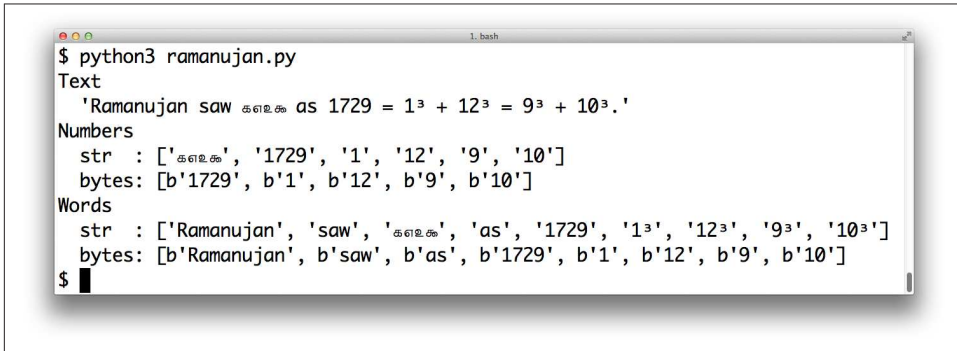
text_str = ("Ramanujan saw \u0be7\u0bed\u0be8\u0bef" ❸
            " as 1729 = 13 + 123 = 93 + 103." ) ❹

text_bytes = text_str.encode('utf_8') ❺

print('Text', repr(text_str), sep='\n ')
print('Numbers')
print(' str :', re_numbers_str.findall(text_str)) ❻
print(' bytes:', re_numbers_bytes.findall(text_bytes)) ❼
print('Words')
print(' str :', re_words_str.findall(text_str)) ❽
print(' bytes:', re_words_bytes.findall(text_bytes)) ❾
```

- ❶ The first two regular expressions are of the str type.
- ❷ The last two are of the bytes type.
- ❸ Unicode text to search, containing the Tamil digits for 1729 (the logical line continues until the right parenthesis token).
- ❹ This string is joined to the previous one at compile time (see “2.4.2. String literal concatenation” in *The Python Language Reference*).
- ❺ A bytes string is needed to search with the bytes regular expressions.
- ❻ The str pattern `r'\d+'` matches the Tamil and ASCII digits.
- ❼ The bytes pattern `rb'\d+'` matches only the ASCII bytes for digits.

- ❶ The `str` pattern `r'\w+'` matches the letters, superscripts, Tamil, and ASCII digits.
- ❷ The bytes pattern `rb'\w+'` matches only the ASCII bytes for letters and digits.



```
$ python3 ramanujan.py
Text
'Ramanujan saw க௭௭௭ as 1729 = 1³ + 12³ = 9³ + 10³.'
Numbers
str : ['க௭௭௭', '1729', '1³', '12³', '9³', '10³']
bytes: [b'1729', b'1³', b'12³', b'9³', b'10³']
Words
str : ['Ramanujan', 'saw', 'க௭௭௭', 'as', '1729', '1³', '12³', '9³', '10³']
bytes: [b'Ramanujan', b'saw', b'as', b'1729', b'1³', b'12³', b'9³', b'10³']
$
```

Figure 4-4. Screenshot of running `ramanujan.py` from [Example 4-22](#)

[Example 4-22](#) is a trivial example to make one point: you can use regular expressions on `str` and bytes, but in the second case bytes outside of the ASCII range are treated as nondigits and nonword characters.

For `str` regular expressions, there is a `re.ASCII` flag that makes `\w`, `\W`, `\b`, `\B`, `\d`, `\D`, `\s`, and `\S` perform ASCII-only matching. See the [documentation of the `re` module](#) for full details.

Another important dual-mode module is `os`.

str Versus bytes on os Functions

The GNU/Linux kernel is not Unicode savvy, so in the real world you may find filenames made of byte sequences that are not valid in any sensible encoding scheme, and cannot be decoded to `str`. File servers with clients using a variety of OSes are particularly prone to this problem.

In order to work around this issue, all `os` module functions that accept filenames or pathnames take arguments as `str` or bytes. If one such function is called with a `str` argument, the argument will be automatically converted using the codec named by `sys.getfilesystemencoding()`, and the OS response will be decoded with the same codec. This is almost always what you want, in keeping with the Unicode sandwich best practice.

But if you must deal with (and perhaps fix) filenames that cannot be handled in that way, you can pass bytes arguments to the `os` functions to get bytes return values. This

feature lets you deal with any file or pathname, no matter how many gremlins you may find. See [Example 4-23](#).

Example 4-23. `listdir` with `str` and `bytes` arguments and results

```
>>> os.listdir('.') # ❶
['abc.txt', 'digits-of-π.txt']
>>> os.listdir(b'.') # ❷
[b'abc.txt', b'digits-of-\xc0\x80.txt']
```

- ❶ The second filename is “digits-of- π .txt” (with the Greek letter pi).
- ❷ Given a byte argument, `listdir` returns filenames as bytes: `b'\xc0\x80'` is the UTF-8 encoding of the Greek letter pi).

To help with manual handling of `str` or `bytes` sequences that are file or pathnames, the `os` module provides special encoding and decoding functions:

`fsencode(filename)`

Encodes `filename` (can be `str` or `bytes`) to `bytes` using the codec named by `sys.getfilesystemencoding()` if `filename` is of type `str`, otherwise returns the `filename` `bytes` unchanged.

`fsdecode(filename)`

Decodes `filename` (can be `str` or `bytes`) to `str` using the codec named by `sys.getfilesystemencoding()` if `filename` is of type `bytes`, otherwise returns the file name `str` unchanged.

On Unix-derived platforms, these functions use the `surrogateescape` error handler (see the sidebar that follows) to avoid choking on unexpected bytes. On Windows, the `strict` error handler is used.

Using `surrogateescape` to Deal with Gremlins

A trick to deal with unexpected bytes or unknown encodings is the `surrogateescape` codec error handler described in [PEP 383 — Non-decodable Bytes in System Character Interfaces](#) introduced in Python 3.1.

The idea of this error handler is to replace each nondecodable byte with a code point in the Unicode range from `U+DC00` to `U+DCFF` that lies in the so-called “Low Surrogate Area” of the standard—a code space with no characters assigned, reserved for internal use in applications. On encoding, such code points are converted back to the byte values they replaced. See [Example 4-24](#).

Example 4-24. Using `surrogateescape` error handling

```
>>> os.listdir('.') ❶
['abc.txt', 'digits-of-π.txt']
```

```

>>> os.listdir(b'.') ❷
[b'abc.txt', b'digits-of-\xcf\x80.txt']
>>> pi_name_bytes = os.listdir(b'.')[1] ❸
>>> pi_name_str = pi_name_bytes.decode('ascii', 'surrogateescape') ❹
>>> pi_name_str ❺
'digits-of-\udccf\udc80.txt'
>>> pi_name_str.encode('ascii', 'surrogateescape') ❻
b'digits-of-\xcf\x80.txt'

```

- ❶ List directory with a non-ASCII filename.
- ❷ Let's pretend we don't know the encoding and get filenames as bytes.
- ❸ `pi_name_bytes` is the filename with the pi character.
- ❹ Decode it to `str` using the 'ascii' codec with 'surrogateescape'.
- ❺ Each non-ASCII byte is replaced by a surrogate code point: `'\xcf\x80'` becomes `'\udccf\udc80'`.
- ❻ Encode back to ASCII bytes: each surrogate code point is replaced by the byte it replaced.

This ends our exploration of `str` and bytes. If you are still with me, congratulations!

Chapter Summary

We started the chapter by dismissing the notion that `1 character == 1 byte`. As the world adopts Unicode (80% of websites already use UTF-8), we need to keep the concept of text strings separated from the binary sequences that represent them in files, and Python 3 enforces this separation.

After a brief overview of the binary sequence data types—bytes, bytearray, and memoryview—we jumped into encoding and decoding, with a sampling of important codecs, followed by approaches to prevent or deal with the infamous `UnicodeEncodeError`, `UnicodeDecodeError`, and the `SyntaxError` caused by wrong encoding in Python source files.

While on the subject of source code, I presented my position on the debate about non-ASCII identifiers: if the maintainers of the code base want to use a human language that has non-ASCII characters, the identifiers should follow suit—unless the code needs to run on Python 2 as well. But if the project aims to attract an international contributor base, identifiers should be made from English words, and then ASCII suffices.

We then considered the theory and practice of encoding detection in the absence of metadata: in theory, it can't be done, but in practice the Chardet package pulls it off pretty well for a number of popular encodings. Byte order marks were then presented

as the only encoding hint commonly found in UTF-16 and UTF-32 files—sometimes in UTF-8 files as well.

In the next section, we demonstrated opening text files, an easy task except for one pitfall: the `encoding=` keyword argument is not mandatory when you open a text file, but it should be. If you fail to specify the encoding, you end up with a program that manages to generate “plain text” that is incompatible across platforms, due to conflicting default encodings. We then exposed the different encoding settings that Python uses as defaults and how to detect them: `locale.getpreferredencoding()`, `sys.getfilesystemencoding()`, `sys.getdefaultencoding()`, and the encodings for the standard I/O files (e.g., `sys.stdout.encoding`). A sad realization for Windows users is that these settings often have distinct values within the same machine, and the values are mutually incompatible; GNU/Linux and OSX users, in contrast, live in a happier place where UTF-8 is the default pretty much everywhere.

Text comparisons are surprisingly complicated because Unicode provides multiple ways of representing some characters, so normalizing is a prerequisite to text matching. In addition to explaining normalization and case folding, we presented some utility functions that you may adapt to your needs, including drastic transformations like removing all accents. We then saw how to sort Unicode text correctly by leveraging the standard `locale` module—with some caveats—and an alternative that does not depend on tricky locale configurations: the external PyUCA package.

Finally, we glanced at the Unicode database (a source of metadata about every character), and wrapped up with brief discussion of dual-mode APIs (e.g., the `re` and `os` modules, where some functions can be called with `str` or `bytes` arguments, prompting different yet fitting results).

Further Reading

Ned Batchelder’s 2012 PyCon US talk “[Pragmatic Unicode — or — How Do I Stop the Pain?](#)” was outstanding. Ned is so professional that he provides a full transcript of the talk along with the slides and video. Esther Nam and Travis Fischer gave an excellent PyCon 2014 talk “Character encoding and Unicode in Python: How to (ノ ◕□◕)ノ (ノ ◕□◕)ノ with dignity” ([slides](#), [video](#)), from which I quoted this chapter’s short and sweet epigraph: “Humans use text. Computers speak bytes.” Lennart Regebro—one of this book’s technical reviewers—presents his “Useful Mental Model of Unicode (UMMU)” in the short post “[Unconfusing Unicode: What Is Unicode?](#)”. Unicode is a complex standard, so Lennart’s UMMU is a really useful starting point.

The official [Unicode HOWTO](#) in the Python docs approaches the subject from several different angles, from a good historic intro to syntax details, codecs, regular expressions, filenames, and best practices for Unicode-aware I/O (i.e., the Unicode sandwich), with plenty of additional reference links from each section. [Chapter 4](#), “[Strings](#)”, of Mark

Pilgrim’s awesome book *Dive into Python 3* also provides a very good intro to Unicode support in Python 3. In the same book, [Chapter 15](#) describes how the Chardet library was ported from Python 2 to Python 3, a valuable case study given that the switch from the old `str` to the new `bytes` is the cause of most migration pains, and that is a central concern in a library designed to detect encodings.

If you know Python 2 but are new to Python 3, Guido van Rossum’s [What’s New in Python 3.0](#) has 15 bullet points that summarize what changed, with lots of links. Guido starts with the blunt statement: “Everything you thought you knew about binary data and Unicode has changed.” Armin Ronacher’s blog post [“The Updated Guide to Unicode on Python”](#) is deep and highlights some of the pitfalls of Unicode in Python 3 (Armin is not a big fan of Python 3).

Chapter 2, “Strings and Text,” of the *Python Cookbook, Third Edition* (O’Reilly), by David Beazley and Brian K. Jones, has several recipes dealing with Unicode normalization, sanitizing text, and performing text-oriented operations on byte sequences. Chapter 5 covers files and I/O, and it includes “Recipe 5.17. Writing Bytes to a Text File,” showing that underlying any text file there is always a binary stream that may be accessed directly when needed. Later in the cookbook, the `struct` module is put to use in “Recipe 6.11. Reading and Writing Binary Arrays of Structures.”

Nick Coghlan’s Python Notes blog has two posts very relevant to this chapter: [“Python 3 and ASCII Compatible Binary Protocols”](#) and [“Processing Text Files in Python 3”](#). Highly recommended.

Binary sequences are about to gain new constructors and methods in Python 3.5, with one of the current constructor signatures being deprecated (see [PEP 467 — Minor API improvements for binary sequences](#)). Python 3.5 should also see the implementation of [PEP 461 — Adding % formatting to bytes and bytearray](#).

A list of encodings supported by Python is available at [Standard Encodings](#) in the `codecs` module documentation. If you need to get that list programmatically, see how it’s done in the `/Tools/unicode/listcodecs.py` script that comes with the CPython source code.

Martijn Faassen’s [“Changing the Python Default Encoding Considered Harmful”](#) and Tarek Ziadé’s [“sys.setdefaultencoding Is Evil”](#) explain why the default encoding you get from `sys.getdefaultencoding()` should never be changed, even if you discover how.

The books *Unicode Explained* by Jukka K. Korpela (O’Reilly) and *Unicode Demystified* by Richard Gillam (Addison-Wesley) are not Python-specific but were very helpful as I studied Unicode concepts. *Programming with Unicode* by Victor Stinner is a free, self-published book (Creative Commons BY-SA) covering Unicode in general as well as tools and APIs in the context of the main operating systems and a few programming languages, including Python.

The W3C pages [Case Folding: An Introduction](#) and [Character Model for the World Wide Web: String Matching and Searching](#) cover normalization concepts, with the former being a gentle introduction and the latter a working draft written in dry standard-speak—the same tone of the [Unicode Standard Annex #15 — Unicode Normalization Forms](#). The [Frequently Asked Questions / Normalization](#) from [Unicode.org](#) is more readable, as is the [NFC FAQ](#) by Mark Davis—author of several Unicode algorithms and president of the Unicode Consortium at the time of this writing.

Soapbox

What Is “Plain Text”?

For anyone who deals with non-English text on a daily basis, “plain text” does not imply “ASCII.” The [Unicode Glossary](#) defines *plain text* like this:

Computer-encoded text that consists only of a sequence of code points from a given standard, with no other formatting or structural information.

That definition starts very well, but I don’t agree with the part after the comma. HTML is a great example of a plain-text format that carries formatting and structural information. But it’s still plain text because every byte in such a file is there to represent a text character, usually using UTF-8. There are no bytes with nontext meaning, as you can find in a *.png* or *.xls* document where most bytes represent packed binary values like RGB values and floating-point numbers. In plain text, numbers are represented as sequences of digit characters.

I am writing this book in a plain-text format called—ironically—[AsciiDoc](#), which is part of the toolchain of O’Reilly’s excellent [Atlas book publishing platform](#). AsciiDoc source files are plain text, but they are UTF-8, not ASCII. Otherwise, writing this chapter would have been really painful. Despite the name, AsciiDoc is just great.

The world of Unicode is constantly expanding and, at the edges, tool support is not always there. That’s why I had to use images for Figures [4-1](#), [4-3](#), and [4-4](#): not all characters I wanted to show were available in the fonts used to render the book. On the other hand, the Ubuntu 14.04 and OSX 10.9 terminals display them perfectly well—including the Japanese characters for the word “mojibake”: 文字化け.

Unicode Riddles

Imprecise qualifiers such as “often,” “most,” and “usually” seem to pop up whenever I write about Unicode normalization. I regret the lack of more definitive advice, but there are so many exceptions to the rules in Unicode that it is hard to be absolutely positive.

For example, the μ (micro sign) is considered a “compatibility character” but the Ω (ohm) and Å (Ångström) symbols are not. The difference has practical consequences: NFC normalization—recommended for text matching—replaces the Ω (ohm) by Ω (uppercase Greek omega) and the Å (Ångström) by Å (uppercase A with ring above). But as a “compatibility character” the μ (micro sign) is not replaced by the visually

identical μ (lowercase Greek mu), except when the stronger NFKC or NFKD normalizations are applied, and these transformations are lossy.

I understand the μ (micro sign) is in Unicode because it appears in the `latin1` encoding and replacing it with the Greek mu would break round-trip conversion. After all, that's why the micro sign is a “compatibility character.” But if the ohm and Ångström symbols are not in Unicode for compatibility reasons, then why have them at all? There are already code points for the GREEK CAPITAL LETTER OMEGA and the LATIN CAPITAL LETTER A WITH RING ABOVE, which look the same and replace them on NFC normalization. Go figure.

My take after many hours studying Unicode: it is hugely complex and full of special cases, reflecting the wonderful variety of human languages and the politics of industry standards.

How Are `str` Represented in RAM?

The official Python docs avoid the issue of how the code points of a `str` are stored in memory. This is, after all, an implementation detail. In theory, it doesn't matter: whatever the internal representation, every `str` must be encoded to bytes on output.

In memory, Python 3 stores each `str` as a sequence of code points using a fixed number of bytes per code point, to allow efficient direct access to any character or slice.

Before Python 3.3, CPython could be compiled to use either 16 or 32 bits per code point in RAM; the former was a “narrow build,” and the latter a “wide build.” To know which you have, check the value of `sys.maxunicode`: 65535 implies a “narrow build” that can't handle code points above U+FFFF transparently. A “wide build” doesn't have this limitation, but consumes a lot of memory: 4 bytes per character, even while the vast majority of code points for Chinese ideographs fit in 2 bytes. Neither option was great, so you had to choose depending on your needs.

Since Python 3.3, when creating a new `str` object, the interpreter checks the characters in it and chooses the most economic memory layout that is suitable for that particular `str`: if there are only characters in the `latin1` range, that `str` will use just one byte per code point. Otherwise, 2 or 4 bytes per code point may be used, depending on the `str`. This is a simplification; for the full details, look up [PEP 393 — Flexible String Representation](#).

The flexible string representation is similar to the way the `int` type works in Python 3: if the integer fits in a machine word, it is stored in one machine word. Otherwise, the interpreter switches to a variable-length representation like that of the Python 2 `long` type. It is nice to see the spread of good ideas.

Functions as Objects

FP Ch 8 | Object References & Mutability [= plan Ch 6]

Object References, Mutability, and Recycling

‘You are sad,’ the Knight said in an anxious tone: ‘let me sing you a song to comfort you. [...] The name of the song is called “HADDOCKS’ EYES”’.

‘Oh, that’s the name of the song, is it?’ Alice said, trying to feel interested.

‘No, you don’t understand,’ the Knight said, looking a little vexed. ‘That’s what the name is CALLED. The name really IS “THE AGED AGED MAN.”’ (adapted from Chapter VIII. ‘It’s my own Invention’).

— Lewis Carroll

Through the Looking-Glass, and What Alice Found There

Alice and the Knight set the tone of what we will see in this chapter. The theme is the distinction between objects and their names. A name is not the object; a name is a separate thing.

We start the chapter by presenting a metaphor for variables in Python: variables are labels, not boxes. If reference variables are old news to you, the analogy may still be handy if you need to explain aliasing issues to others.

We then discuss the concepts of object identity, value, and aliasing. A surprising trait of tuples is revealed: they are immutable but their values may change. This leads to a discussion of shallow and deep copies. References and function parameters are our next theme: the problem with mutable parameter defaults and the safe handling of mutable arguments passed by clients of our functions.

The last sections of the chapter cover garbage collection, the `del` command, and how to use weak references to “remember” objects without keeping them alive.

This is a rather dry chapter, but its topics lie at the heart of many subtle bugs in real Python programs.

Let's start by unlearning that a variable is like a box where you store data.

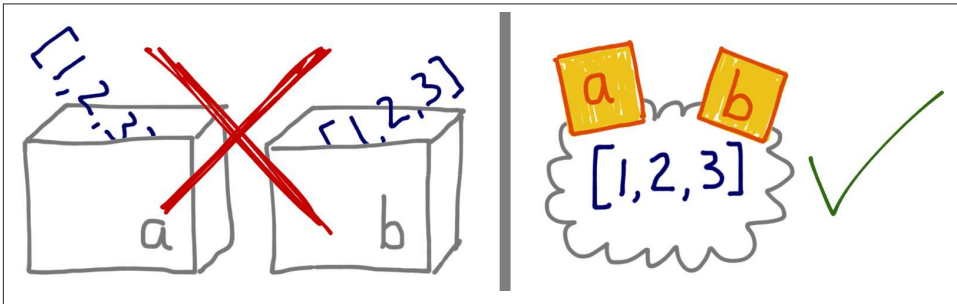
Variables Are Not Boxes

In 1997, I took a summer course on Java at MIT. The professor, Lynn Andrea Stein—an award-winning computer science educator who currently teaches at Olin College of Engineering—made the point that the usual “variables as boxes” metaphor actually hinders the understanding of reference variables in OO languages. Python variables are like reference variables in Java, so it's better to think of them as labels attached to objects.

Example 8-1 is a simple interaction that the “variables as boxes” idea cannot explain. **Figure 8-1** illustrates why the box metaphor is wrong for Python, while sticky notes provide a helpful picture of how variables actually work.

Example 8-1. Variables `a` and `b` hold references to the same list, not copies of the list

```
>>> a = [1, 2, 3]
>>> b = a
>>> a.append(4)
>>> b
[1, 2, 3, 4]
```



*Figure 8-1. If you imagine variables are like boxes, you can't make sense of assignment in Python; instead, think of variables as sticky notes—**Example 8-1** then becomes easy to explain*

Prof. Stein also spoke about assignment in a very deliberate way. For example, when talking about a seesaw object in a simulation, she would say: “Variable `s` is assigned to the seesaw,” but never “The seesaw is assigned to variable `s`.” With reference variables, it makes much more sense to say that the variable is assigned to an object, and not the other way around. After all, the object is created before the assignment. **Example 8-2** proves that the righthand side of an assignment happens first.

Example 8-2. Variables are assigned to objects only after the objects are created

```
>>> class Gizmo:
...     def __init__(self):
...         print('Gizmo id: %d' % id(self))
...
>>> x = Gizmo()
Gizmo id: 4301489152 ❶
>>> y = Gizmo() * 10 ❷
Gizmo id: 4301489432 ❸
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for *: 'Gizmo' and 'int'
>>>
>>> dir() ❹
['Gizmo', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'x']
```

- ❶ The output `Gizmo id: ...` is a side effect of creating a `Gizmo` instance.
- ❷ Multiplying a `Gizmo` instance will raise an exception.
- ❸ Here is proof that a second `Gizmo` was actually instantiated before the multiplication was attempted.
- ❹ But variable `y` was never created, because the exception happened while the right-hand side of the assignment was being evaluated.



To understand an assignment in Python, always read the right-hand side first: that's where the object is created or retrieved. After that, the variable on the left is bound to the object, like a label stuck to it. Just forget about the boxes.

Because variables are mere labels, nothing prevents an object from having several labels assigned to it. When that happens, you have *aliasing*, our next topic.

Identity, Equality, and Aliases

Lewis Carroll is the pen name of Prof. Charles Lutwidge Dodgson. Mr. Carroll is not only equal to Prof. Dodgson: they are one and the same. [Example 8-3](#) expresses this idea in Python.

Example 8-3. *charles* and *lewis* refer to the same object

```
>>> charles = {'name': 'Charles L. Dodgson', 'born': 1832}
>>> lewis = charles ❶
>>> lewis is charles
True
>>> id(charles), id(lewis) ❷
```

```
(4300473992, 4300473992)
>>> lewis['balance'] = 950 ❸
>>> charles
{'name': 'Charles L. Dodgson', 'balance': 950, 'born': 1832}
```

- ❶ lewis is an alias for charles.
- ❷ The is operator and the id function confirm it.
- ❸ Adding an item to lewis is the same as adding an item to charles.

However, suppose an impostor—let’s call him Dr. Alexander Pedachenko—claims he is Charles L. Dodgson, born in 1832. His credentials may be the same, but Dr. Pedachenko is not Prof. Dodgson. **Figure 8-2** illustrates this scenario.

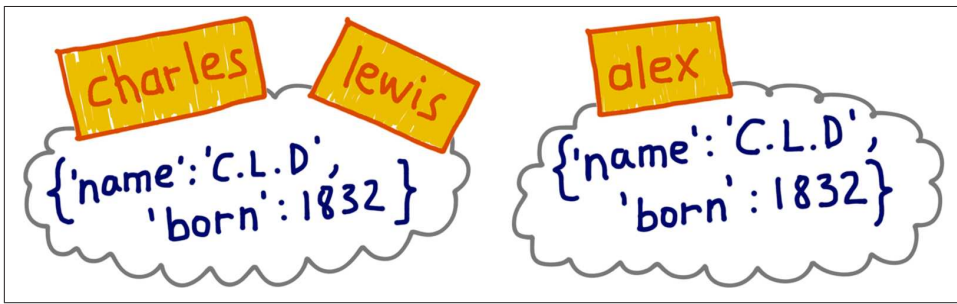


Figure 8-2. *charles* and *lewis* are bound to the same object; *alex* is bound to a separate object of equal contents

Example 8-4 implements and tests the *alex* object depicted in **Figure 8-2**.

Example 8-4. alex and charles compare equal, but alex is not charles

```
>>> alex = {'name': 'Charles L. Dodgson', 'born': 1832, 'balance': 950} ❶
>>> alex == charles ❷
True
>>> alex is not charles ❸
True
```

- ❶ alex refers to an object that is a replica of the object assigned to charles.
- ❷ The objects compare equal, because of the `__eq__` implementation in the dict class.
- ❸ But they are distinct objects. This is the Pythonic way of writing the negative identity comparison: `a is not b`.

Example 8-3 is an example of *aliasing*. In that code, *lewis* and *charles* are aliases: two variables bound to the same object. On the other hand, *alex* is not an alias for

`charles`: these variables are bound to distinct objects. The objects bound to `alex` and `charles` have the same *value*—that’s what `==` compares—but they have different identities.

In *The Python Language Reference*, “3.1. Objects, values and types” states:

Every object has an identity, a type and a value. An object’s identity never changes once it has been created; you may think of it as the object’s address in memory. The `is` operator compares the identity of two objects; the `id()` function returns an integer representing its identity.

The real meaning of an object’s ID is implementation-dependent. In CPython, `id()` returns the memory address of the object, but it may be something else in another Python interpreter. The key point is that the ID is guaranteed to be a unique numeric label, and it will never change during the life of the object.

In practice, we rarely use the `id()` function while programming. Identity checks are most often done with the `is` operator, and not by comparing IDs. Next, we’ll talk about `is` versus `==`.

Choosing Between `==` and `is`

The `==` operator compares the values of objects (the data they hold), while `is` compares their identities.

We often care about values and not identities, so `==` appears more frequently than `is` in Python code.

However, if you are comparing a variable to a singleton, then it makes sense to use `is`. By far, the most common case is checking whether a variable is bound to `None`. This is the recommended way to do it:

```
x is None
```

And the proper way to write its negation is:

```
x is not None
```

The `is` operator is faster than `==`, because it cannot be overloaded, so Python does not have to find and invoke special methods to evaluate it, and computing is as simple as comparing two integer IDs. In contrast, `a == b` is syntactic sugar for `a.__eq__(b)`. The `__eq__` method inherited from object compares object IDs, so it produces the same result as `is`. But most built-in types override `__eq__` with more meaningful implementations that actually take into account the values of the object attributes. Equality may involve a lot of processing—for example, when comparing large collections or deeply nested structures.

To wrap up this discussion of identity versus equality, we'll see that the famously immutable tuple is not as rigid as you may expect.

The Relative Immutability of Tuples

Tuples, like most Python collections—lists, dicts, sets, etc.—hold references to objects.

¹ If the referenced items are mutable, they may change even if the tuple itself does not. In other words, the immutability of tuples really refers to the physical contents of the tuple data structure (i.e., the references it holds), and does not extend to the referenced objects.

Example 8-5 illustrates the situation in which the value of a tuple changes as result of changes to a mutable object referenced in it. What can never change in a tuple is the identity of the items it contains.

Example 8-5. t1 and t2 initially compare equal, but changing a mutable item inside tuple t1 makes it different

```
>>> t1 = (1, 2, [30, 40]) ❶
>>> t2 = (1, 2, [30, 40]) ❷
>>> t1 == t2 ❸
True
>>> id(t1[-1]) ❹
4302515784
>>> t1[-1].append(99) ❺
>>> t1
(1, 2, [30, 40, 99])
>>> id(t1[-1]) ❻
4302515784
>>> t1 == t2 ❼
False
```

- ❶ t1 is immutable, but t1[-1] is mutable.
- ❷ Build a tuple t2 whose items are equal to those of t1.
- ❸ Although distinct objects, t1 and t2 compare equal, as expected.
- ❹ Inspect the identity of the list at t1[-1].
- ❺ Modify the t1[-1] list in place.
- ❻ The identity of t1[-1] has not changed, only its value.
- ❼ t1 and t2 are now different.

1. On the other hand, single-type sequences like `str`, `bytes`, and `array.array` are flat: they don't contain references but physically hold their data—characters, bytes, and numbers—in contiguous memory.

This relative immutability of tuples is behind the riddle “A += Assignment Puzzler” on page 40. It’s also the reason why some tuples are unhashable, as we’ve seen in “What Is Hashable?” on page 65.

The distinction between equality and identity has further implications when you need to copy an object. A copy is an equal object with a different ID. But if an object contains other objects, should the copy also duplicate the inner objects, or is it OK to share them? There’s no single answer. Read on for a discussion.

Copies Are Shallow by Default

The easiest way to copy a list (or most built-in mutable collections) is to use the built-in constructor for the type itself. For example:

```
>>> l1 = [3, [55, 44], (7, 8, 9)]
>>> l2 = list(l1) ❶
>>> l2
[3, [55, 44], (7, 8, 9)]
>>> l2 == l1 ❷
True
>>> l2 is l1 ❸
False
```

- ❶ `list(l1)` creates a copy of `l1`.
- ❷ The copies are equal.
- ❸ But refer to two different objects.

For lists and other mutable sequences, the shortcut `l2 = l1[:]` also makes a copy.

However, using the constructor or `[:]` produces a *shallow copy* (i.e., the outermost container is duplicated, but the copy is filled with references to the same items held by the original container). This saves memory and causes no problems if all the items are immutable. But if there are mutable items, this may lead to unpleasant surprises.

In [Example 8-6](#), we create a shallow copy of a list containing another list and a tuple, and then make changes to see how they affect the referenced objects.



If you have a connected computer on hand, I highly recommend watching the interactive animation for [Example 8-6](#) at the [Online Python Tutor](#). As I write this, direct linking to a prepared example at [pythontutor.com](#) is not working reliably, but the tool is awesome, so taking the time to copy and paste the code is worthwhile.

Example 8-6. Making a shallow copy of a list containing another list; copy and paste this code to see it animated at the Online Python Tutor

```
l1 = [3, [66, 55, 44], (7, 8, 9)]
l2 = list(l1)          # ❶
l1.append(100)          # ❷
l1[1].remove(55)       # ❸
print('l1:', l1)
print('l2:', l2)
l2[1] += [33, 22]       # ❹
l2[2] += (10, 11)       # ❺
print('l1:', l1)
print('l2:', l2)
```

- ❶ l2 is a shallow copy of l1. This state is depicted in [Figure 8-3](#).
- ❷ Appending 100 to l1 has no effect on l2.
- ❸ Here we remove 55 from the inner list l1[1]. This affects l2 because l2[1] is bound to the same list as l1[1].
- ❹ For a mutable object like the list referred by l2[1], the operator += changes the list in place. This change is visible at l1[1], which is an alias for l2[1].
- ❺ += on a tuple creates a new tuple and rebinds the variable l2[2] here. This is the same as doing l2[2] = l2[2] + (10, 11). Now the tuples in the last position of l1 and l2 are no longer the same object. See [Figure 8-4](#).

The output of [Example 8-6](#) is [Example 8-7](#), and the final state of the objects is depicted in [Figure 8-4](#).

Example 8-7. Output of [Example 8-6](#)

```
l1: [3, [66, 44], (7, 8, 9), 100]
l2: [3, [66, 44], (7, 8, 9)]
l1: [3, [66, 44, 33, 22], (7, 8, 9), 100]
l2: [3, [66, 44, 33, 22], (7, 8, 9, 10, 11)]
```

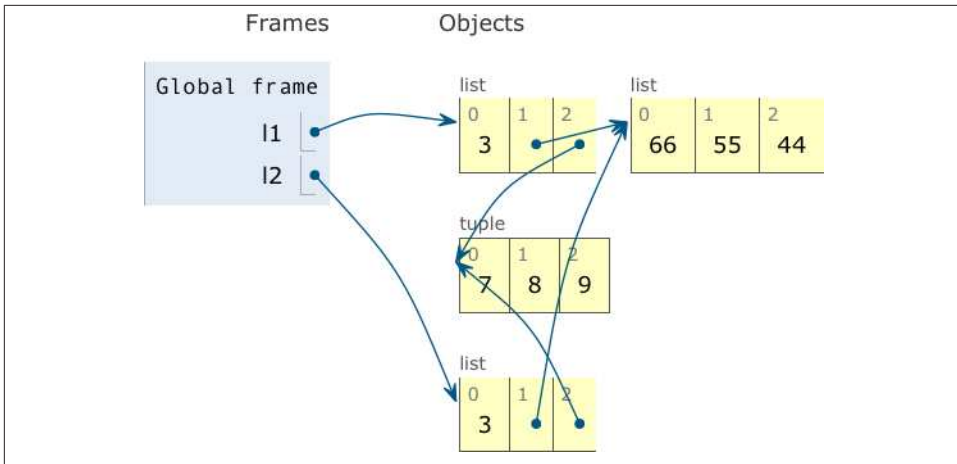


Figure 8-3. Program state immediately after the assignment `l2 = list(l1)` in **Example 8-6**. `l1` and `l2` refer to distinct lists, but the lists share references to the same inner list object `[66, 55, 44]` and tuple `(7, 8, 9)`. (Diagram generated by the Online Python Tutor.)

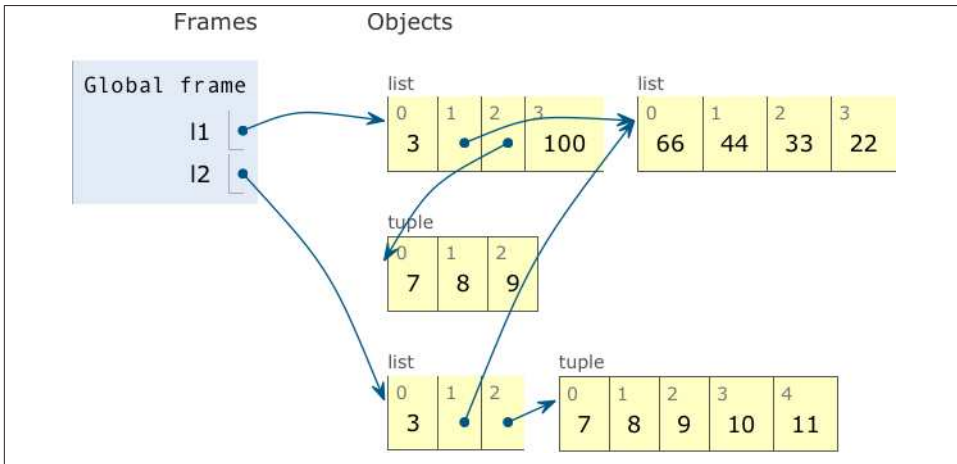


Figure 8-4. Final state of `l1` and `l2`: they still share references to the same list object, now containing `[66, 44, 33, 22]`, but the operation `l2[2] += (10, 11)` created a new tuple with content `(7, 8, 9, 10, 11)`, unrelated to the tuple `(7, 8, 9)` referenced by `l1[2]`. (Diagram generated by the Online Python Tutor.)

It should be clear now that shallow copies are easy to make, but they may or may not be what you want. How to make deep copies is our next topic.

Deep and Shallow Copies of Arbitrary Objects

Working with shallow copies is not always a problem, but sometimes you need to make deep copies (i.e., duplicates that do not share references of embedded objects). The `copy` module provides the `deepcopy` and `copy` functions that return deep and shallow copies of arbitrary objects.

To illustrate the use of `copy()` and `deepcopy()`, [Example 8-8](#) defines a simple class, `Bus`, representing a school bus that is loaded with passengers and then picks up or drops off passengers on its route.

Example 8-8. Bus picks up and drops off passengers

class Bus:

```
def __init__(self, passengers=None):
    if passengers is None:
        self.passengers = []
    else:
        self.passengers = list(passengers)

def pick(self, name):
    self.passengers.append(name)

def drop(self, name):
    self.passengers.remove(name)
```

Now in the interactive [Example 8-9](#) we will create we will create a bus object (`bus1`) and two clones—a shallow copy (`bus2`) and a deep copy (`bus3`)—to observe what happens as `bus1` drops off a student.

Example 8-9. Effects of using copy versus deepcopy

```
>>> import copy
>>> bus1 = Bus(['Alice', 'Bill', 'Claire', 'David'])
>>> bus2 = copy.copy(bus1)
>>> bus3 = copy.deepcopy(bus1)
>>> id(bus1), id(bus2), id(bus3)
(4301498296, 4301499416, 4301499752) ❶
>>> bus1.drop('Bill')
>>> bus2.passengers
['Alice', 'Claire', 'David'] ❷
>>> id(bus1.passengers), id(bus2.passengers), id(bus3.passengers)
(4302658568, 4302658568, 4302657800) ❸
>>> bus3.passengers
['Alice', 'Bill', 'Claire', 'David'] ❹
```

- ❶ Using `copy` and `deepcopy`, we create three distinct `Bus` instances.
- ❷ After `bus1` drops 'Bill', he is also missing from `bus2`.

- ❸ Inspection of the `passengers` attribute shows that `bus1` and `bus2` share the same list object, because `bus2` is a shallow copy of `bus1`.
- ❹ `bus3` is a deep copy of `bus1`, so its `passengers` attribute refers to another list.

Note that making deep copies is not a simple matter in the general case. Objects may have cyclic references that would cause a naïve algorithm to enter an infinite loop. The `deepcopy` function remembers the objects already copied to handle cyclic references gracefully. This is demonstrated in [Example 8-10](#).

Example 8-10. Cyclic references: `b` refers to `a`, and then is appended to `a`; `deepcopy` still manages to copy `a`

```
>>> a = [10, 20]
>>> b = [a, 30]
>>> a.append(b)
>>> a
[10, 20, [[...], 30]]
>>> from copy import deepcopy
>>> c = deepcopy(a)
>>> c
[10, 20, [[...], 30]]
```

Also, a deep copy may be too deep in some cases. For example, objects may refer to external resources or singletons that should not be copied. You can control the behavior of both `copy` and `deepcopy` by implementing the `__copy__()` and `__deepcopy__()` special methods as described in the [copy module documentation](#).

The sharing of objects through aliases also explains how parameter passing works in Python, and the problem of using mutable types as parameter defaults. These issues will be covered next.

Function Parameters as References

The only mode of parameter passing in Python is *call by sharing*. That is the same mode used in most OO languages, including Ruby, SmallTalk, and Java (this applies to Java reference types; primitive types use call by value). Call by sharing means that each formal parameter of the function gets a copy of each reference in the arguments. In other words, the parameters inside the function become aliases of the actual arguments.

The result of this scheme is that a function may change any mutable object passed as a parameter, but it cannot change the identity of those objects (i.e., it cannot altogether replace an object with another). [Example 8-11](#) shows a simple function using `+=` on one of its parameters. As we pass numbers, lists, and tuples to the function, the actual arguments passed are affected in different ways.

Example 8-11. A function may change any mutable object it receives

```
>>> def f(a, b):
...     a += b
...     return a
...
>>> x = 1
>>> y = 2
>>> f(x, y)
3
>>> x, y ❶
(1, 2)
>>> a = [1, 2]
>>> b = [3, 4]
>>> f(a, b)
[1, 2, 3, 4]
>>> a, b ❷
([1, 2, 3, 4], [3, 4])
>>> t = (10, 20)
>>> u = (30, 40)
>>> f(t, u) ❸
(10, 20, 30, 40)
>>> t, u
((10, 20), (30, 40))
```

- ❶ The number `x` is unchanged.
- ❷ The list `a` is changed.
- ❸ The tuple `t` is unchanged.

Another issue related to function parameters is the use of mutable values for defaults, as discussed next.

Mutable Types as Parameter Defaults: Bad Idea

Optional parameters with default values are a great feature of Python function definitions, allowing our APIs to evolve while remaining backward-compatible. However, you should avoid mutable objects as default values for parameters.

To illustrate this point, in [Example 8-12](#), we take the `Bus` class from [Example 8-8](#) and change its `__init__` method to create `HauntedBus`. Here we tried to be clever and instead of having a default value of `passengers=None`, we have `passengers=[]`, thus avoiding the `if` in the previous `__init__`. This “cleverness” gets us into trouble.

Example 8-12. A simple class to illustrate the danger of a mutable default

```
class HauntedBus:
    """A bus model haunted by ghost passengers"""

    def __init__(self, passengers=[]): ❶
```

```

        self.passengers = passengers ❷

    def pick(self, name):
        self.passengers.append(name) ❸

    def drop(self, name):
        self.passengers.remove(name)

```

- ❶ When the `passengers` argument is not passed, this parameter is bound to the default list object, which is initially empty.
- ❷ This assignment makes `self.passengers` an alias for `passengers`, which is itself an alias for the default list, when no `passengers` argument is given.
- ❸ When the methods `.remove()` and `.append()` are used with `self.passengers` we are actually mutating the default list, which is an attribute of the function object.

Example 8-13 shows the eerie behavior of the `HauntedBus`.

Example 8-13. Buses haunted by ghost passengers

```

>>> bus1 = HauntedBus(['Alice', 'Bill'])
>>> bus1.passengers
['Alice', 'Bill']
>>> bus1.pick('Charlie')
>>> bus1.drop('Alice')
>>> bus1.passengers ❶
['Bill', 'Charlie']
>>> bus2 = HauntedBus() ❷
>>> bus2.pick('Carrie')
>>> bus2.passengers
['Carrie']
>>> bus3 = HauntedBus() ❸
>>> bus3.passengers ❹
['Carrie']
>>> bus3.pick('Dave')
>>> bus2.passengers ❺
['Carrie', 'Dave']
>>> bus2.passengers is bus3.passengers ❻
True
>>> bus1.passengers ❼
['Bill', 'Charlie']

```

- ❶ So far, so good: no surprises with `bus1`.
- ❷ `bus2` starts empty, so the default empty list is assigned to `self.passengers`.
- ❸ `bus3` also starts empty, again the default list is assigned.
- ❹ The default is no longer empty!
- ❺ Now Dave, picked by `bus3`, appears in `bus2`.

- ❹ The problem: `bus2.passengers` and `bus3.passengers` refer to the same list.
- ❺ But `bus1.passengers` is a distinct list.

The problem is that `Bus` instances that don't get an initial passenger list end up sharing the same passenger list among themselves.

Such bugs may be subtle. As [Example 8-13](#) demonstrates, when a `HauntedBus` is instantiated with passengers, it works as expected. Strange things happen only when a `HauntedBus` starts empty, because then `self.passengers` becomes an alias for the default value of the `passengers` parameter. The problem is that each default value is evaluated when the function is defined—i.e., usually when the module is loaded—and the default values become attributes of the function object. So if a default value is a mutable object, and you change it, the change will affect every future call of the function.

After running the lines in [Example 8-13](#), you can inspect the `HauntedBus.__init__` object and see the ghost students haunting its `__defaults__` attribute:

```
>>> dir(HauntedBus.__init__) # doctest: +ELLIPSIS
['__annotations__', '__call__', ..., '__defaults__', ...]
>>> HauntedBus.__init__.__defaults__
(['Carrie', 'Dave'],)
```

Finally, we can verify that `bus2.passengers` is an alias bound to the first element of the `HauntedBus.__init__.__defaults__` attribute:

```
>>> HauntedBus.__init__.__defaults__[0] is bus2.passengers
True
```

The issue with mutable defaults explains why `None` is often used as the default value for parameters that may receive mutable values. In [Example 8-8](#), `__init__` checks whether the `passengers` argument is `None`, and assigns a new empty list to `self.passengers`. As explained in the following section, if `passengers` is not `None`, the correct implementation assigns a copy of it to `self.passengers`. Let's now take a closer look.

Defensive Programming with Mutable Parameters

When you are coding a function that receives a mutable parameter, you should carefully consider whether the caller expects the argument passed to be changed.

For example, if your function receives a `dict` and needs to modify it while processing it, should this side effect be visible outside of the function or not? Actually it depends on the context. It's really a matter of aligning the expectation of the coder of the function and that of the caller.

The last bus example in this chapter shows how a `TwilightBus` breaks expectations by sharing its passenger list with its clients. Before studying the implementation, see in

Example 8-14 how the `TwilightBus` class works from the perspective of a client of the class.

Example 8-14. Passengers disappear when dropped by a `TwilightBus`

```
>>> basketball_team = ['Sue', 'Tina', 'Maya', 'Diana', 'Pat'] ❶
>>> bus = TwilightBus(basketball_team) ❷
>>> bus.drop('Tina') ❸
>>> bus.drop('Pat')
>>> basketball_team ❹
['Sue', 'Maya', 'Diana']
```

- ❶ `basketball_team` holds five student names.
- ❷ A `TwilightBus` is loaded with the team.
- ❸ The bus drops one student, then another.
- ❹ The dropped passengers vanished from the basketball team!

`TwilightBus` violates the “Principle of least astonishment,” a best practice of interface design. It surely is astonishing that when the bus drops a student, her name is removed from the basketball team roster.

Example 8-15 is the implementation `TwilightBus` and an explanation of the problem.

Example 8-15. A simple class to show the perils of mutating received arguments

```
class TwilightBus:
    """A bus model that makes passengers vanish"""

    def __init__(self, passengers=None):
        if passengers is None:
            self.passengers = [] ❶
        else:
            self.passengers = passengers ❷

    def pick(self, name):
        self.passengers.append(name)

    def drop(self, name):
        self.passengers.remove(name) ❸
```

- ❶ Here we are careful to create a new empty list when `passengers` is `None`.
- ❷ However, this assignment makes `self.passengers` an alias for `passengers`, which is itself an alias for the actual argument passed to `__init__` (i.e., `basketball_team` in **Example 8-14**).
- ❸ When the methods `.remove()` and `.append()` are used with `self.passengers`, we are actually mutating the original list received as argument to the constructor.

The problem here is that the bus is aliasing the list that is passed to the constructor. Instead, it should keep its own passenger list. The fix is simple: in `__init__`, when the `passengers` parameter is provided, `self.passengers` should be initialized with a copy of it, as we did correctly in [Example 8-8](#) (“Deep and Shallow Copies of Arbitrary Objects” on page 228):

```
def __init__(self, passengers=None):
    if passengers is None:
        self.passengers = []
    else:
        self.passengers = list(passengers) ❶
```

- ❶ Make a copy of the `passengers` list, or convert it to a list if it’s not one.

Now our internal handling of the passenger list will not affect the argument used to initialize the bus. As a bonus, this solution is more flexible: now the argument passed to the `passengers` parameter may be a tuple or any other iterable, like a set or even database results, because the `list` constructor accepts any iterable. As we create our own list to manage, we ensure that it supports the necessary `.remove()` and `.append()` operations we use in the `.pick()` and `.drop()` methods.



Unless a method is explicitly intended to mutate an object received as argument, you should think twice before aliasing the argument object by simply assigning it to an instance variable in your class. If in doubt, make a copy. Your clients will often be happier.

del and Garbage Collection

Objects are never explicitly destroyed; however, when they become unreachable they may be garbage-collected.

— “Data Model” chapter of *The Python Language Reference*

The `del` statement deletes names, not objects. An object may be garbage collected as result of a `del` command, but only if the variable deleted holds the last reference to the object, or if the object becomes unreachable.² Rebinding a variable may also cause the number of references to an object to reach zero, causing its destruction.

2. If two objects refer to each other, as in [Example 8-10](#), they may be destroyed if the garbage collector determines that they are otherwise unreachable because their only references are their mutual references.



There is a `__del__` special method, but it does not cause the disposal of the instance, and should not be called by your code. `__del__` is invoked by the Python interpreter when the instance is about to be destroyed to give it a chance to release external resources. You will seldom need to implement `__del__` in your own code, yet some Python beginners spend time coding it for no good reason. The proper use of `__del__` is rather tricky. See the [__del__ special method documentation](#) in the “Data Model” chapter of *The Python Language Reference*.

In CPython, the primary algorithm for garbage collection is reference counting. Essentially, each object keeps count of how many references point to it. As soon as that *refcount* reaches zero, the object is immediately destroyed: CPython calls the `__del__` method on the object (if defined) and then frees the memory allocated to the object. In CPython 2.0, a generational garbage collection algorithm was added to detect groups of objects involved in reference cycles—which may be unreachable even with outstanding references to them, when all the mutual references are contained within the group. Other implementations of Python have more sophisticated garbage collectors that do not rely on reference counting, which means the `__del__` method may not be called immediately when there are no more references to the object. See “[PyPy, Garbage Collection, and a Deadlock](#)” by A. Jesse Jiryu Davis for discussion of improper and proper use of `__del__`.

To demonstrate the end of an object’s life, [Example 8-16](#) uses `weakref.finalize` to register a callback function to be called when an object is destroyed.

Example 8-16. Watching the end of an object when no more references point to it

```
>>> import weakref
>>> s1 = {1, 2, 3}
>>> s2 = s1
>>> def bye():
...     print('Gone with the wind...')
...
>>> ender = weakref.finalize(s1, bye)
>>> ender.alive
True
>>> del s1
>>> ender.alive
True
>>> s2 = 'spam'
Gone with the wind...
>>> ender.alive
False
```

❶ `s1` and `s2` are aliases referring to the same set, `{1, 2, 3}`.

- ❷ This function must not be a bound method of the object about to be destroyed or otherwise hold a reference to it.
- ❸ Register the `bye` callback on the object referred by `s1`.
- ❹ The `.alive` attribute is `True` before the `finalize` object is called.
- ❺ As discussed, `del` does not delete an object, just a reference to it.
- ❻ Rebinding the last reference, `s2`, makes `{1, 2, 3}` unreachable. It is destroyed, the `bye` callback is invoked, and `ender.alive` becomes `False`.

The point of [Example 8-16](#) is to make explicit that `del` does not delete objects, but objects may be deleted as a consequence of being unreachable after `del` is used.

You may be wondering why the `{1, 2, 3}` object was destroyed in [Example 8-16](#). After all, the `s1` reference was passed to the `finalize` function, which must have held on to it in order to monitor the object and invoke the callback. This works because `finalize` holds a *weak reference* to `{1, 2, 3}`, as explained in the next section.

Weak References

The presence of references is what keeps an object alive in memory. When the reference count of an object reaches zero, the garbage collector disposes of it. But sometimes it is useful to have a reference to an object that does not keep it around longer than necessary. A common use case is a cache.

Weak references to an object do not increase its reference count. The object that is the target of a reference is called the *referent*. Therefore, we say that a weak reference does not prevent the referent from being garbage collected.

Weak references are useful in caching applications because you don't want the cached objects to be kept alive just because they are referenced by the cache.

[Example 8-17](#) shows how a `weakref.ref` instance can be called to reach its referent. If the object is alive, calling the weak reference returns it, otherwise `None` is returned.



[Example 8-17](#) is a console session, and the Python console automatically binds the `_` variable to the result of expressions that are not `None`. This interfered with my intended demonstration but also highlights a practical matter: when trying to micro-manage memory we are often surprised by hidden, implicit assignments that create new references to our objects. The `_` console variable is one example. Traceback objects are another common source of unexpected references.

Example 8-17. A weak reference is a callable that returns the referenced object or None if the referent is no more

```
>>> import weakref
>>> a_set = {0, 1}
>>> wref = weakref.ref(a_set) ❶
>>> wref
<weakref at 0x100637598; to 'set' at 0x100636748>
>>> wref() ❷
{0, 1}
>>> a_set = {2, 3, 4} ❸
>>> wref() ❹
{0, 1}
>>> wref() is None ❺
False
>>> wref() is None ❻
True
```

- ❶ The `wref` weak reference object is created and inspected in the next line.
- ❷ Invoking `wref()` returns the referenced object, `{0, 1}`. Because this is a console session, the result `{0, 1}` is bound to the `_` variable.
- ❸ `a_set` no longer refers to the `{0, 1}` set, so its reference count is decreased. But the `_` variable still refers to it.
- ❹ Calling `wref()` still returns `{0, 1}`.
- ❺ When this expression is evaluated, `{0, 1}` lives, therefore `wref()` is not `None`. But `_` is then bound to the resulting value, `False`. Now there are no more strong references to `{0, 1}`.
- ❻ Because the `{0, 1}` object is now gone, this last call to `wref()` returns `None`.

The [weakref module documentation](#) makes the point that the `weakref.ref` class is actually a low-level interface intended for advanced uses, and that most programs are better served by the use of the `weakref` collections and `finalize`. In other words, consider using `WeakKeyDictionary`, `WeakValueDictionary`, `WeakSet`, and `finalize` (which use weak references internally) instead of creating and handling your own `weakref.ref` instances by hand. We just did that in [Example 8-17](#) in the hope that showing a single `weakref.ref` in action could take away some of the mystery around them. But in practice, most of the time Python programs use the `weakref` collections.

The next subsection briefly discusses the `weakref` collections.

The WeakValueDictionary Skit

The class `WeakValueDictionary` implements a mutable mapping where the values are weak references to objects. When a referred object is garbage collected elsewhere in the

program, the corresponding key is automatically removed from `WeakValueDictionary`. This is commonly used for caching.

Our demonstration of a `WeakValueDictionary` is inspired by the classic *Cheese Shop* skit by Monty Python, in which a customer asks for more than 40 kinds of cheese, including cheddar and mozzarella, but none are in stock.³

Example 8-18 implements a trivial class to represent each kind of cheese.

Example 8-18. Cheese has a kind attribute and a standard representation

class Cheese:

```
def __init__(self, kind):
    self.kind = kind

def __repr__(self):
    return 'Cheese(%r)' % self.kind
```

In **Example 8-19**, each cheese is loaded from a catalog to a stock implemented as a `WeakValueDictionary`. However, all but one disappear from the stock as soon as the catalog is deleted. Can you explain why the Parmesan cheese lasts longer than the others?⁴ The tip after the code has the answer.

Example 8-19. Customer: “Have you in fact got any cheese here at all?”

```
>>> import weakref
>>> stock = weakref.WeakValueDictionary() ❶
>>> catalog = [Cheese('Red Leicester'), Cheese('Tilsit'),
...             Cheese('Brie'), Cheese('Parmesan')]
...
>>> for cheese in catalog:
...     stock[cheese.kind] = cheese ❷
...
>>> sorted(stock.keys())
['Brie', 'Parmesan', 'Red Leicester', 'Tilsit'] ❸
>>> del catalog
>>> sorted(stock.keys())
['Parmesan'] ❹
>>> del cheese
>>> sorted(stock.keys())
[]
```

3. `cheeseshop.python.org` is also an alias for PyPI—the Python Package Index software repository—which started its life quite empty. At the time of this writing, the Python Cheese Shop has 41,426 packages. Not bad, but still far from the more than 131,000 modules available in CPAN—the Comprehensive Perl Archive Network—the envy of all dynamic language communities.

4. Parmesan cheese is aged at least a year at the factory, so it is more durable than fresh cheese, but this is not the answer we are looking for.

- ❶ `stock` is a `WeakValueDictionary`.
- ❷ The `stock` maps the name of the cheese to a weak reference to the cheese instance in the `catalog`.
- ❸ The `stock` is complete.
- ❹ After the `catalog` is deleted, most cheeses are gone from the `stock`, as expected in `WeakValueDictionary`. Why not all, in this case?



A temporary variable may cause an object to last longer than expected by holding a reference to it. This is usually not a problem with local variables: they are destroyed when the function returns. But in [Example 8-19](#), the for loop variable `cheese` is a global variable and will never go away unless explicitly deleted.

A counterpart to the `WeakValueDictionary` is the `WeakKeyDictionary` in which the keys are weak references. The [`weakref.WeakKeyDictionary` documentation](#) hints on possible uses:

[A `WeakKeyDictionary`] can be used to associate additional data with an object owned by other parts of an application without adding attributes to those objects. This can be especially useful with objects that override attribute accesses.

The `weakref` module also provides a `WeakSet`, simply described in the docs as “Set class that keeps weak references to its elements. An element will be discarded when no strong reference to it exists any more.” If you need to build a class that is aware of every one of its instances, a good solution is to create a class attribute with a `WeakSet` to hold the references to the instances. Otherwise, if a regular set was used, the instances would never be garbage collected, because the class itself would have strong references to them, and classes live as long as the Python process unless you deliberately delete them.

These collections, and weak references in general, are limited in the kinds of objects they can handle. The next section explains.

Limitations of Weak References

Not every Python object may be the target, or referent, of a weak reference. Basic `list` and `dict` instances may not be referents, but a plain subclass of either can solve this problem easily:

```
class MyList(list):
    """list subclass whose instances may be weakly referenced"""

a_list = MyList(range(10))
```



```
# a_list can be the target of a weak reference
wref_to_a_list = weakref.ref(a_list)
```

A set instance can be a referent, and that's why a set was used in [Example 8-17](#). User-defined types also pose no problem, which explains why the silly Cheese class was needed in [Example 8-19](#). But int and tuple instances cannot be targets of weak references, even if subclasses of those types are created.

Most of these limitations are implementation details of CPython that may not apply to other Python interpreters. They are the result of internal optimizations, some of which are discussed in the following (highly optional) section.

Tricks Python Plays with Immutables



You may safely skip this section. It discusses some Python implementation details that are not really important for *users* of Python. They are shortcuts and optimizations done by the CPython core developers, which should not bother you when using the language, and that may not apply to other Python implementations or even future versions of CPython. Nevertheless, while experimenting with aliases and copies you may stumble upon these tricks, so I felt they were worth mentioning.

I was surprised to learn that, for a tuple `t`, `t[:]` does not make a copy, but returns a reference to the same object. You also get a reference to the same tuple if you write `tuple(t)`.⁵ [Example 8-20](#) proves it.

Example 8-20. A tuple built from another is actually the same exact tuple

```
>>> t1 = (1, 2, 3)
>>> t2 = tuple(t1)
>>> t2 is t1 ❶
True
>>> t3 = t1[:]
>>> t3 is t1 ❷
True
```

- ❶ t1 and t2 are bound to the same object.
- ❷ And so is t3.

The same behavior can be observed with instances of `str`, `bytes`, and `frozenset`. Note that a `frozenset` is not a sequence, so `fs[:]` does not work if `fs` is a `frozenset`. But

5. This is clearly documented. Type `help(tuple)` in the Python console to read: “If the argument is a tuple, the return value is the same object.” I thought I knew everything about tuples before writing this book.

`fs.copy()` has the same effect: it cheats and returns a reference to the same object, and not a copy at all, as [Example 8-21](#) shows.⁶

Example 8-21. String literals may create shared objects

```
>>> t1 = (1, 2, 3)
>>> t3 = (1, 2, 3) # ❶
>>> t3 is t1 # ❷
False
>>> s1 = 'ABC'
>>> s2 = 'ABC' # ❸
>>> s2 is s1 # ❹
True
```

- ❶ Creating a new tuple from scratch.
- ❷ `t1` and `t3` are equal, but not the same object.
- ❸ Creating a second `str` from scratch.
- ❹ Surprise: `a` and `b` refer to the same `str`!

The sharing of string literals is an optimization technique called *interning*. CPython uses the same technique with small integers to avoid unnecessary duplication of “popular” numbers like 0, -1, and 42. Note that CPython does not intern all strings or integers, and the criteria it uses to do so is an undocumented implementation detail.



Never depend on `str` or `int` interning! Always use `==` and not `is` to compare them for equality. Interning is a feature for internal use of the Python interpreter.

The tricks discussed in this section, including the behavior of `frozenset.copy()`, are “white lies”; they save memory and make the interpreter faster. Do not worry about them, they should not give you any trouble because they only apply to immutable types. Probably the best use of these bits of trivia is to win bets with fellow Pythonistas.

6. The white lie of having the `copy` method not copying anything can be explained by interface compatibility: it makes `frozenset` more compatible with `set`. Anyway, it makes no difference to the end user whether two identical immutable objects are the same or are copies.

Chapter Summary

Every Python object has an identity, a type, and a value. Only the value of an object changes over time.⁷

If two variables refer to immutable objects that have equal values (`a == b` is `True`), in practice it rarely matters if they refer to copies or are aliases referring to the same object because the value of an immutable object does not change, with one exception. The exception is immutable collections such as tuples and frozensets: if an immutable collection holds references to mutable items, then its value may actually change when the value of a mutable item changes. In practice, this scenario is not so common. What never changes in an immutable collection are the identities of the objects within.

The fact that variables hold references has many practical consequences in Python programming:

- Simple assignment does not create copies.
- Augmented assignment with `+=` or `*=` creates new objects if the lefthand variable is bound to an immutable object, but may modify a mutable object in place.
- Assigning a new value to an existing variable does not change the object previously bound to it. This is called a rebinding: the variable is now bound to a different object. If that variable was the last reference to the previous object, that object will be garbage collected.
- Function parameters are passed as aliases, which means the function may change any mutable object received as an argument. There is no way to prevent this, except making local copies or using immutable objects (e.g., passing a tuple instead of a list).
- Using mutable objects as default values for function parameters is dangerous because if the parameters are changed in place, then the default is changed, affecting every future call that relies on the default.

In CPython, objects are discarded as soon as the number of references to them reaches zero. They may also be discarded if they form groups with cyclic references but no outside references. In some situations, it may be useful to hold a reference to an object that will not—by itself—keep an object alive. One example is a class that wants to keep track of all its current instances. This can be done with weak references, a low-level mechanism underlying the more useful collections `WeakValueDictionary`, `WeakKeyDictionary`, `WeakSet`, and the `finalize` function from the `weakref` module.

7. Actually the type of an object may be changed by merely assigning a different class to its `__class__` attribute, but that is pure evil and I regret writing this footnote.

Further Reading

The “[Data Model](#)” chapter of *The Python Language Reference* starts with a clear explanation of object identities and values.

Wesley Chun, author of the *Core Python* series of books, made a great presentation about many of the topics covered in this chapter during OSCON 2013. You can download the slides from the “[Python 103: Memory Model & Best Practices](#)” talk page. There is also a [YouTube video](#) of a longer presentation Wesley gave at EuroPython 2011, covering not only the theme of this chapter but also the use of special methods.

Doug Hellmann wrote a long series of excellent blog posts titled [Python Module of the Week](#), which became a book, *The Python Standard Library by Example*. His posts “[copy – Duplicate Objects](#)” and “[weakref – Garbage-Collectable References to Objects](#)” cover some of the topics we just discussed.

More information on the CPython generational garbage collector can be found in the [gc module documentation](#), which starts with the sentence “This module provides an interface to the optional garbage collector.” The “optional” qualifier here may be surprising, but the “[Data Model](#)” chapter also states:

An implementation is allowed to postpone garbage collection or omit it altogether—it is a matter of implementation quality how garbage collection is implemented, as long as no objects are collected that are still reachable.

Fredrik Lundh—creator of key libraries like ElementTree, Tkinter, and the PIL image library—has a short post about the Python garbage collector titled “[How Does Python Manage Memory?](#)” He emphasizes that the garbage collector is an implementation feature that behaves differently across Python interpreters. For example, Jython uses the Java garbage collector.

The CPython 3.4 garbage collector improved handling of objects with a `__del__` method, as described in [PEP 442 — Safe object finalization](#).

Wikipedia has an article about [string interning](#), mentioning the use of this technique in several languages, including Python.

Soapbox

Equal Treatment to All Objects

I learned Java before I discovered Python. The `==` operator in Java never felt right for me. It is much more common for programmers to care about equality than identity, but for objects (not primitive types) the Java `==` compares references, and not object values. Even for something as basic as comparing strings, Java forces you to use the `.equals` method. Even then, there is another catch: if you write `a.equals(b)` and `a` is `null`, you get a null pointer exception. The Java designers felt the need to overload `+` for strings, so why not go ahead and overload `==` as well?

Python gets this right. The `==` operator compares object values and `is` compares references. And because Python has operator overloading, `==` works sensibly with all objects in the standard library, including `None`, which is a proper object, unlike Java's `null`.

And of course, you can define `__eq__` in your own classes to decide what `==` means for your instances. If you don't override `__eq__`, the method inherited from `object` compares object IDs, so the fallback is that every instance of a user-defined class is considered different.

These are some of the things that made me switch from Java to Python as soon as I finished reading the Python Tutorial one afternoon in September 1998.

Mutability

This chapter would be redundant if all Python objects were immutable. When you are dealing with unchanging objects, it makes no difference whether variables hold the actual objects or references to shared objects. If `a == b` is true, and neither object can change, they might as well be the same. That's why string interning is safe. Object identity becomes important only when objects are mutable.

In “pure” functional programming, all data is immutable: appending to a collection actually creates a new collection. Python, however, is not a functional language, much less a pure one. Instances of user-defined classes are mutable by default in Python—as in most object-oriented languages. When creating your own objects, you have to be extra careful to make them immutable, if that is a requirement. Every attribute of the object must also be immutable, otherwise you end up with something like the tuple: immutable as far as object IDs go, but the value of a tuple may change if it holds a mutable object.

Mutable objects are also the main reason why programming with threads is so hard to get right: threads mutating objects without proper synchronization produce corrupted data. Excessive synchronization, on the other hand, causes deadlocks.

Object Destruction and Garbage Collection

There is no mechanism in Python to directly destroy an object, and this omission is actually a great feature: if you could destroy an object at any time, what would happen to existing strong references pointing to it?

Garbage collection in CPython is done primarily by reference counting, which is easy to implement, but is prone to memory leaking when there are reference cycles, so with version 2.0 (October 2000) a generational garbage collector was implemented, and it is able to dispose of unreachable objects kept alive by reference cycles.

But the reference counting is still there as a baseline, and it causes the immediate disposal of objects with zero references. This means that, in CPython—at least for now—it's safe to write this:

```
open('test.txt', 'wt', encoding='utf-8').write('1, 2, 3')
```

That code is safe because the reference count of the file object will be zero after the `write` method returns, and Python will immediately close the file before destroying the object representing it in memory. However, the same line is not safe in Jython or IronPython that use the garbage collector of their host runtimes (the Java VM and the .NET CLR), which are more sophisticated but do not rely on reference counting and may take longer to destroy the object and close the file. In all cases, including CPython, the best practice is to explicitly close the file, and the most reliable way of doing it is using the `with` statement, which guarantees that the file will be closed even if exceptions are raised while it is open. Using `with`, the previous snippet becomes:

```
with open('test.txt', 'wt', encoding='utf-8') as fp:  
    fp.write('1, 2, 3')
```

If you are into the subject of garbage collectors, you may want to read Thomas Perl's paper "[Python Garbage Collector Implementations: CPython, PyPy and GaS](#)", from which I learned the bit about the safety of the `open().write()` in CPython.

Parameter Passing: Call by Sharing

A popular way of explaining how parameter passing works in Python is the phrase: "Parameters are passed by value, but the values are references." This not wrong, but causes confusion because the most common parameter passing modes in older languages are *call by value* (the function gets a copy of the argument) and *call by reference* (the function gets a pointer to the argument). In Python, the function gets a copy of the arguments, but the arguments are always references. So the value of the referenced objects may be changed, if they are mutable, but their identity cannot. Also, because the function gets a copy of the reference in an argument, rebinding it has no effect outside of the function. I adopted the term *call by sharing* after reading up on the subject in *Programming Language Pragmatics, Third Edition* by Michael L. Scott (Morgan Kaufmann), particularly "8.3.1: Parameter Modes."

The Full Quote of Alice and the Knights's Song

I love this passage, but it was too long as a chapter opener. So here is the complete dialog about the Knight's song, its name, and how the song and its name are called:

'You are sad,' the Knight said in an anxious tone: 'let me sing you a song to comfort you.'

'Is it very long?' Alice asked, for she had heard a good deal of poetry that day.

'It's long,' said the Knight, 'but very, VERY beautiful. Everybody that hears me sing it—either it brings the TEARS into their eyes, or else—'

'Or else what?' said Alice, for the Knight had made a sudden pause.

'Or else it doesn't, you know. The name of the song is called "HADDOCKS' EYES".'

'Oh, that's the name of the song, is it?' Alice said, trying to feel interested.

'No, you don't understand,' the Knight said, looking a little vexed. 'That's what the name is CALLED. The name really IS "THE AGED AGED MAN".'

'Then I ought to have said "That's what the SONG is called"?' Alice corrected herself.

'No, you oughtn't: that's quite another thing! The SONG is called "WAYS AND MEANS": but that's only what it's CALLED, you know!'

'Well, what IS the song, then?' said Alice, who was by this time completely bewildered.

'I was coming to that,' the Knight said. 'The song really IS "A-SITTING ON A GATE": and the tune's my own invention.'

— Lewis Carroll

Chapter VIII, "It's My Own Invention," Through the Looking-Glass

HPP Ch 3 | Lists and Tuples (performance)

Lists and Tuples

Questions You'll Be Able to Answer After This Chapter

- What are lists and tuples good for?
- What is the complexity of a lookup in a list/tuple?
- How is that complexity achieved?
- What are the differences between lists and tuples?
- How does appending to a list work?
- When should I use lists and tuples?

One of the most important things in writing efficient programs is understanding the guarantees of the data structures you use. In fact, a large part of performant programming is knowing what questions you are trying to ask of your data and picking a data structure that can answer these questions quickly. In this chapter we will talk about the kinds of questions that lists and tuples can answer quickly, and how they do it.

Lists and tuples are a class of data structures called *arrays*. An array is a flat list of data with some intrinsic ordering. Usually in these sorts of data structures, the relative ordering of the elements is as important as the elements themselves! In addition, this *a priori* knowledge of the ordering is incredibly valuable: by knowing that data in our array is at a specific position, we can retrieve it in $O(1)$!¹ There are also many

¹ $O(1)$ uses *Big-Oh Notation* to denote how efficient an algorithm is. A good introduction to the topic can be found in [this *dev.to* post by Sarah Chima](#) or in the introductory chapters of *Introduction to Algorithms* by Thomas H. Cormen et al. (MIT Press).

ways to implement arrays, and each solution has its own useful features and guarantees. This is why in Python we have two types of arrays: lists and tuples. *Lists* are dynamic arrays that let us modify and resize the data we are storing, while *tuples* are static arrays whose contents are fixed and immutable.

Let's unpack these previous statements a bit. System memory on a computer can be thought of as a series of numbered buckets, each capable of holding a number. Python stores data in these buckets *by reference*, which means the number itself simply points to, or refers to, the data we actually care about. As a result, these buckets can store any type of data we want (as opposed to numpy arrays, which have a static type and can store only that type of data).²

When we want to create an array (and thus a list or tuple), we first have to allocate a block of system memory (where every section of this block will be used as an integer-sized pointer to actual data). This involves going to the system kernel and requesting the use of *N consecutive* buckets. **Figure 3-1** shows an example of the system memory layout for an array (in this case, a list) of size 6.



In Python, lists also store how large they are, so of the six allocated blocks, only five are usable—the zeroth element is the length.

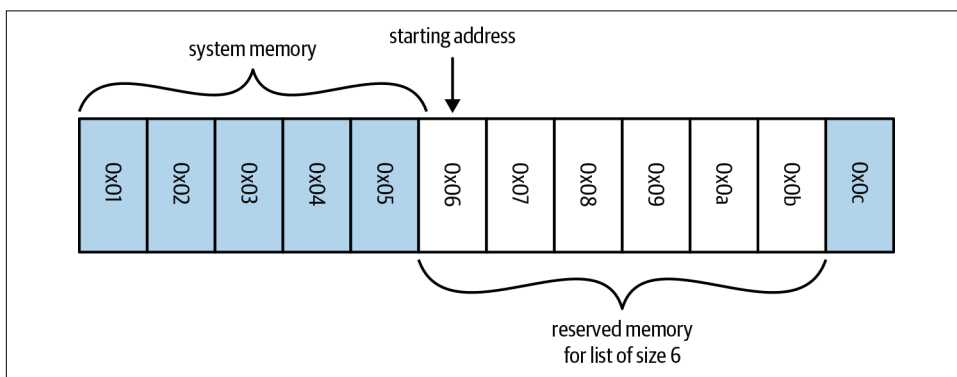


Figure 3-1. Example of system memory layout for an array of size 6

In order to look up any specific element in our list, we simply need to know which element we want and remember which bucket our data started in. Since all of the data will occupy the same amount of space (one “bucket,” or, more specifically, one

² In 64-bit computers, having 12 KB of memory gives you 725 buckets, and having 52 GB of memory gives you 3,250,000,000 buckets!

integer-sized pointer to the actual data), we don't need to know anything about the type of data that is being stored to do this calculation.



If you knew where in memory your list of N elements started, how would you find an arbitrary element in the list?

If, for example, we needed to retrieve the zeroth element in our array, we would simply go to the first bucket in our sequence, M , and read out the value inside it. If, on the other hand, we needed the fifth element in our array, we would go to the bucket at position $M + 5$ and read its content. In general, if we want to retrieve element i from our array, we go to bucket $M + i$. So, by having our data stored in consecutive buckets, and having knowledge of the ordering of our data, we can locate our data by knowing which bucket to look at in one step (or $O(1)$), regardless of how big our array is ([Example 3-1](#)).

Example 3-1. Timings for lookups in lists of different sizes

```
>>> %%timeit l = list(range(10))
...: l[5]
...:
30.1 ns ± 0.996 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)

>>> %%timeit l = list(range(10_000_000))
...: l[100_000]
...:
28.9 ns ± 0.894 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)
```

What if we were given an array with an unknown order and wanted to retrieve a particular element? If the ordering were known, we could simply look up that particular value. However, in this case, we must do a search operation. The most basic approach to this problem is called a *linear search*, where we iterate over every element in the array and check if it is the value we want, as seen in [Example 3-2](#).

Example 3-2. A linear search through a list

```
def linear_search(needle, array):
    for i, item in enumerate(array):
        if item == needle:
            return i
    return -1
```

This algorithm has a worst-case performance of $O(n)$. This case occurs when we search for something that isn't in the array. In order to know that the element we are

searching for isn't in the array, we must first check it against every other element. Eventually, we will reach the final `return -1` statement. In fact, this algorithm is exactly the algorithm that `list.index()` uses.

The only way to increase the speed is by having some other understanding of how the data is placed in memory, or of the arrangement of the buckets of data we are holding. For example, hash tables (“How Do Dictionaries and Sets Work?” on page 83), which are a fundamental data structure powering Chapter 4, solve this problem in $O(1)$ by adding extra overhead to insertions/retrievals and enforcing a strict and peculiar sorting of the item. Alternatively, if your data is sorted so that every item is larger (or smaller) than its neighbor to the left (or right), then specialized search algorithms can be used that can bring your lookup time down to $O(\log n)$. This may seem like an impossible step to take from the constant-time lookups we saw before, but sometimes it is the best option (especially since search algorithms are more flexible and allow you to define searches in creative ways).

Exercise

Given the following data, write an algorithm to find the index of the value 61:

[9, 18, 18, 19, 29, 42, 56, 61, 88, 95]

Since you know the data is ordered, how can you do this faster?

Hint: If you split the array in half, you know all the values on the left are smaller than the smallest element in the right set. You can use this!

A More Efficient Search

As alluded to previously, we can achieve better search performance if we first sort our data so that all elements to the left of a particular item are smaller (or larger) than that item. The comparison is done through the `__eq__` and `__lt__` magic functions of the object and can be user-defined if using custom objects.



Without the `__eq__` and `__lt__` methods, a custom object will compare only to objects of the same type, and the comparison will be done using the instance's placement in memory. With those two magic functions defined, you can use the `functools.total_ordering` decorator from the standard library to automatically define all the other ordering functions, albeit at a small performance penalty.

The two ingredients necessary are the sorting algorithm and the searching algorithm. Python lists have a built-in sorting algorithm that uses Tim sort. Tim sort can sort

through a list in $O(n)$ in the best case (and in $O(n \log n)$ in the worst case). It achieves this performance by utilizing multiple types of sorting algorithms and using heuristics to guess which algorithm will perform the best, given the data (more specifically, it hybridizes insertion and merge sort algorithms).

Once a list has been sorted, we can find our desired element using a binary search (Example 3-3), which has an average case complexity of $O(\log n)$. It achieves this by first looking at the middle of the list and comparing this value with the desired value. If this midpoint's value is less than our desired value, we consider the right half of the list, and we continue halving the list like this until the value is found, or until the value is known not to occur in the sorted list. As a result, we do not need to read all values in the list, as was necessary for the [linear search](#); instead, we read only a small subset of them.

Example 3-3. Efficient searching through a sorted list—binary search

```
def binary_search(needle, haystack):
    imin, imax = 0, len(haystack)
    while True:
        if imin > imax:
            return -1
        midpoint = (imin + imax) // 2
        if haystack[midpoint] > needle:
            imax = midpoint
        elif haystack[midpoint] < needle:
            imin = midpoint+1
        else:
            return midpoint
```

This method allows us to find elements in a list without resorting to the potentially heavyweight solution of a dictionary. This is especially true when the list of data that is being operated on is intrinsically sorted. It is more efficient to do a binary search on the list to find an object rather than converting your data to a dictionary and then doing a lookup on it. Although a dictionary lookup takes only $O(1)$, converting the data to a dictionary takes $O(n)$ (and a dictionary's restriction of no repeating keys may be undesirable). On the other hand, the binary search will take $O(\log n)$.

In addition, the `bisect` module from Python's standard library simplifies much of this process by giving easy methods to add elements into a list while maintaining its sorting, in addition to finding elements using a heavily optimized binary search. It does this by providing alternative functions that add the element into the correct sorted placement. With the list always being sorted, we can easily find the elements we are looking for (examples of this can be found in the [documentation for the bisect module](#)). In addition, we can use `bisect` to find the closest element to what we are looking for very quickly (Example 3-4). This can be extremely useful for comparing two datasets that are similar but not identical.

Example 3-4. Finding close values in a list with the `bisect` module

```
import bisect
import random

def find_closest(haystack, needle):
    # bisect.bisect_left will return the first value in the haystack
    # that is greater than the needle
    i = bisect.bisect_left(haystack, needle)
    if i == len(haystack):
        return i - 1
    elif haystack[i] == needle:
        return i
    elif i > 0:
        j = i - 1
        # since we know the value is larger than needle (and vice versa for the
        # value at j), we don't need to use absolute values here
        if haystack[i] - needle > needle - haystack[j]:
            return j
    return i

important_numbers = []
for i in range(10):
    new_number = random.randint(0, 1000)
    bisect.insort(important_numbers, new_number)

# important_numbers will already be in order because we inserted new elements
# with bisect.insort
print(important_numbers)
# > [14, 265, 496, 661, 683, 734, 881, 892, 973, 992]

closest_index = find_closest(important_numbers, -250)
print(f"Closest value to -250: {important_numbers[closest_index]}")
# > Closest value to -250: 14

closest_index = find_closest(important_numbers, 500)
print(f"Closest value to 500: {important_numbers[closest_index]}")
# > Closest value to 500: 496

closest_index = find_closest(important_numbers, 1100)
print(f"Closest value to 1100: {important_numbers[closest_index]}")
# > Closest value to 1100: 992
```

In general, this touches on a fundamental rule of writing efficient code: pick the right data structure and stick with it! Although there may be more efficient data structures for particular operations, the cost of converting to those data structures may negate any efficiency boost.

Lists Versus Tuples

If lists and tuples both use the same underlying data structure, what are the differences between the two? Summarized, the main differences are as follows:

- Lists are *dynamic* arrays; they are mutable and allow for resizing (changing the number of elements that are held).
- Tuples are *static* arrays; they are immutable, and the data within them cannot be changed after they have been created.
- Tuples are cached by the Python runtime, which means that we don't need to talk to the kernel to reserve memory every time we want to use one.

These differences outline the philosophical difference between the two: tuples are for describing multiple properties of one unchanging thing, and lists can be used to store collections of data about completely disparate objects. For example, the parts of a telephone number are perfect for a tuple: they won't change, and if they do, they represent a new object or a different phone number. Similarly, the coefficients of a polynomial fit a tuple, since different coefficients represent a different polynomial. On the other hand, the names of the people currently reading this book are better suited for a list: the data is constantly changing both in content and in size but is still always representing the same idea.

It is important to note that both lists and tuples can take mixed types. This can, as you will see, introduce quite a bit of overhead and reduce some potential optimizations. This overhead can be removed if we force all our data to be of the same type. In [Chapter 6](#), we will talk about reducing both the memory used and the computational overhead by using `numpy`. In addition, tools like the standard library module `array` can reduce these overheads for other, nonnumerical situations. This alludes to a major point in performant programming that we will touch on in later chapters: generic code will be much slower than code specifically designed to solve a particular problem.

In addition, the immutability of a tuple as opposed to a list, which can be resized and changed, makes it a lightweight data structure. This means that there isn't much overhead in memory when storing tuples, and operations with them are quite straightforward. With lists, as you will learn, their mutability comes at the price of extra memory needed to store them and extra computations needed when using them.

Exercise

For the following example datasets, would you use a tuple or a list? Why?

1. First 20 prime numbers
2. Names of programming languages
3. A person's age, weight, and height
4. A person's birthday and birthplace
5. The result of a particular game of pool
6. The results of a continuing series of pool games

Solution:

1. Tuple, since the data is static and will not change.
2. List, since this dataset is constantly growing.
3. List, since the values will need to be updated.
4. Tuple, since that information is static and will not change.
5. Tuple, since the data is static.
6. List, since more games will be played. (In fact, we could use a list of tuples since each individual game's results will not change, but we will need to add more results as more games are played.)

Lists as Dynamic Arrays

Once we create a list, we are free to change its contents as needed:

```
>>> numbers = [5, 8, 1, 3, 2, 6]
>>> numbers[2] = 2 * numbers[0] ❶
>>> numbers
[5, 8, 10, 3, 2, 6]
```

- ❶ As described previously, this operation is $O(1)$ because we can find the data stored within the zeroth and second elements immediately.

In addition, we can append new data to a list and grow its size:

```
>>> len(numbers)
6
>>> numbers.append(42)
>>> numbers
[5, 8, 10, 3, 2, 6, 42]
>>> len(numbers)
7
```

This is possible because dynamic arrays support a `resize` operation that increases the capacity of the array. When a list of size N is first appended to, Python must create a new list that is big enough to hold the original N items in addition to the extra one that is being appended. However, instead of allocating $N + 1$ items, M items are actually allocated, where $M > N$, in order to provide extra headroom for future appends. Then the data from the old list is copied to the new list, and the old list is destroyed.

The philosophy is that one append is probably the beginning of many appends, and by requesting extra space, we can reduce the number of times this allocation must happen and thus the total number of memory copies that are necessary. This is important since memory copies can be quite expensive, especially when list sizes start growing. [Figure 3-2](#) shows what this overallocation looks like in Python 3.7. The formula dictating this growth is given in [Example 3-5](#).³

³ The code responsible for this overallocation can be seen in the Python source code in [Objects/listobject.c:list_resize](#).

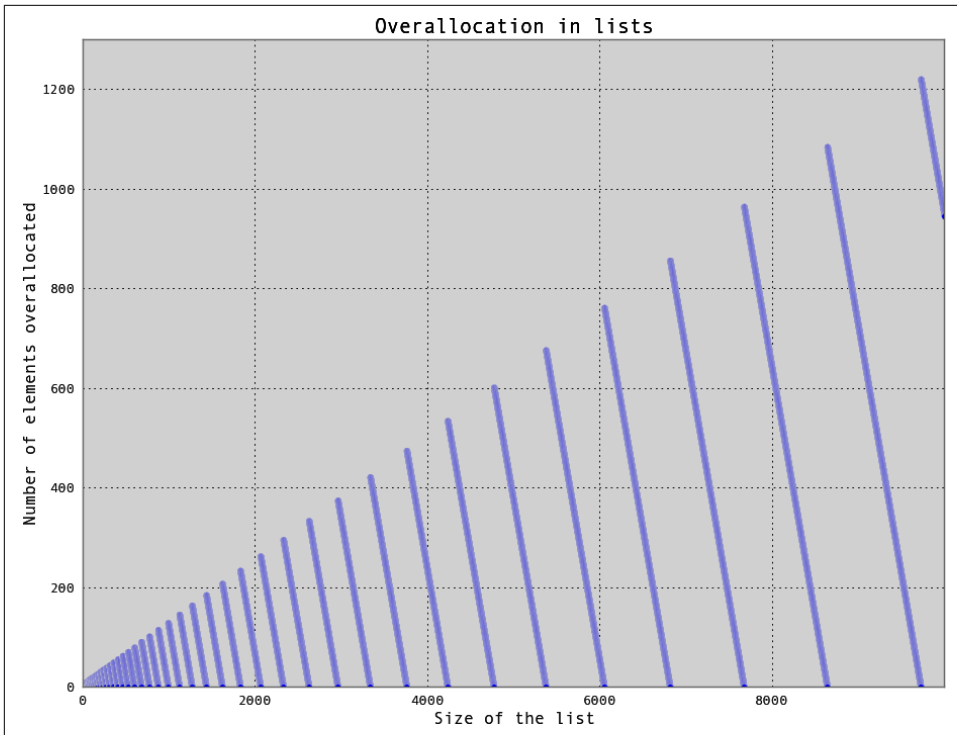


Figure 3-2. Graph showing how many extra elements are being allocated to a list of a particular size. For example, if you create a list with 8,000 elements using `appends`, Python will allocate space for about 8,600 elements, overallocating 600 elements!

Example 3-5. List allocation equation in Python 3.7

```
M = (N >> 3) + (3 if N < 9 else 6)
```

N	0	1-4	5-8	9-16	17-25	26-35	36-46	...	991-1120
---	---	-----	-----	------	-------	-------	-------	-----	----------

M	0	4	8	16	25	35	46	...	1120
---	---	---	---	----	----	----	----	-----	------

As we append data, we utilize the extra space and increase the effective size of the list, `N`. As a result, `N` grows as we append new data, until `N == M`. At this point, there is no extra space to insert new data into, and we must create a *new* list with more extra space. This new list has extra headroom as given by the equation in [Example 3-5](#), and we copy the old data into the new space.

This sequence of events is shown visually in [Figure 3-3](#). The figure follows the various operations being performed on list `l` in [Example 3-6](#).

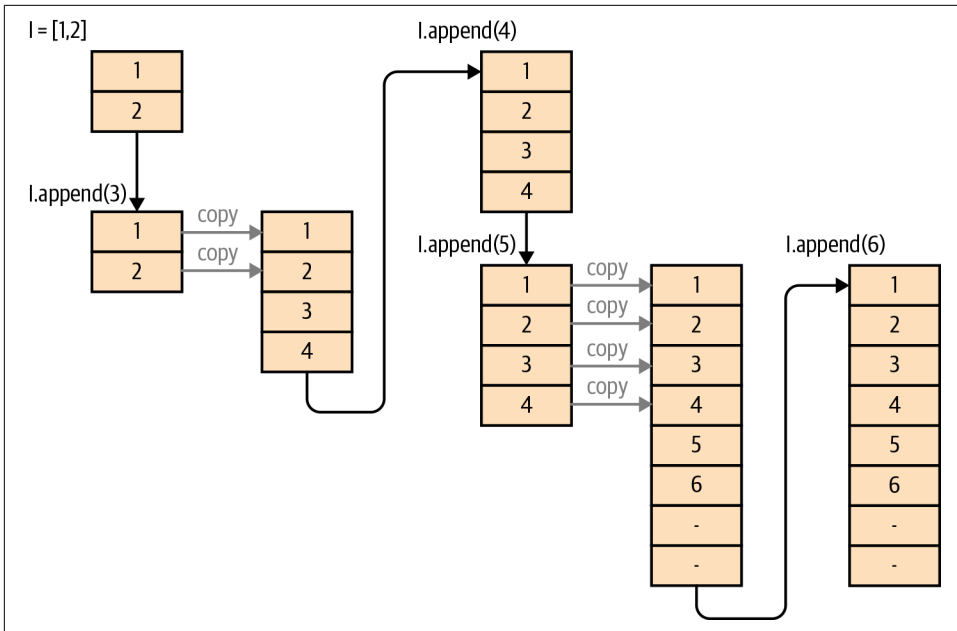


Figure 3-3. Example of how a list is mutated on multiple appends

Example 3-6. Resizing a list

```
l = [1, 2]
for i in range(3, 7):
    l.append(i)
```



This extra allocation happens on the first append. When a list is directly created, as in the preceding example, only the number of elements needed is allocated.

While the amount of extra headroom allocated is generally quite small, it can add up. In [Example 3-7](#), we can see that even for 100,000 elements, we use $2.7\times$ the memory by building the list with appends versus a list comprehension:

Example 3-7. Memory and time consequences of appends versus list comprehensions

```
>>> %memit [i*i for i in range(100_000)]
peak memory: 70.50 MiB, increment: 3.02 MiB

>>> %%memit l = []
... for i in range(100_000):
...     l.append(i * 2)
```

```

...
peak memory: 67.47 MiB, increment: 8.17 MiB

>>> %timeit [i*i for i in range(100_000)]
7.99 ms ± 219 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

>>> %%timeit l = []
... for i in range(100_000):
...     l.append(i * 2)
...
12.2 ms ± 184 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

```

The overall runtime is also slower, because of the extra Python statements that must be run as well as the cost of reallocating memory. This effect becomes especially pronounced when you are maintaining many small lists or when keeping a particularly large list. If we are storing 1,000,000 lists, each containing 10 elements, we would suppose that 10,000,000 elements' worth of memory is being used. In actuality, however, up to 16,000,000 elements could have been allocated if the `append` operator was used to construct the list. Similarly, for a large list of 100,000,000 elements, we actually have 112,500,007 elements allocated!

Tuples as Static Arrays

Tuples are fixed and immutable. This means that once a tuple is created, unlike a list, it cannot be modified or resized:

```

>>> t = (1, 2, 3, 4)
>>> t[0] = 5
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment

```

However, although they don't support resizing, we can concatenate two tuples together and form a new tuple. The operation is similar to the `resize` operation on lists, but we do not allocate any extra space for the resulting tuple:

```

>>> t1 = (1, 2, 3, 4)
>>> t2 = (5, 6, 7, 8)
>>> t1 + t2
(1, 2, 3, 4, 5, 6, 7, 8)

```

If we consider this to be comparable to the `append` operation on lists, we see that it performs in $O(n)$ as opposed to the $O(1)$ speed of lists. This is because we must allocate and copy the tuple every time something is added to it, as opposed to only when our extra headroom ran out for lists. As a result of this, there is no in-place `append`-like operation; adding two tuples always returns a new tuple that is in a new location in memory.

Not storing the extra headroom for resizing has the advantage of using fewer resources. A list of size 100,000,000 created with any `append` operation actually uses 112,500,007 elements' worth of memory, while a tuple holding the same data will only ever use exactly 100,000,000 elements' worth of memory. This makes tuples lightweight and preferable when data becomes static.

Furthermore, even if we create a list *without* `append` (and thus we don't have the extra headroom introduced by an `append` operation), it will *still* be larger in memory than a tuple with the same data. This is because lists have to keep track of more information about their current state in order to efficiently `resize`. While this extra information is quite small (the equivalent of one extra element), it can add up if several million lists are in use.

Another benefit of the static nature of tuples is something Python does in the background: resource caching. Python is garbage collected, which means that when a variable isn't used anymore, Python frees the memory used by that variable, giving it back to the operating system for use in other applications (or for other variables). For tuples of sizes 1–20, however, when they are no longer in use, the space isn't immediately given back to the system: up to 20,000 of each size are saved for future use. This means that when a new tuple of that size is needed in the future, we don't need to communicate with the operating system to find a region in memory to put the data into, since we have a reserve of free memory already. However, this also means that the Python process will have some extra memory overhead.

While this may seem like a small benefit, it is one of the fantastic things about tuples: they can be created easily and quickly since they can avoid communications with the operating system, which can cost your program quite a bit of time. [Example 3-8](#) shows that instantiating a list can be 5.1× slower than instantiating a tuple—which can add up quickly if this is done in a fast loop!

Example 3-8. Instantiation timings for lists versus tuples

```
>>> %timeit l = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
95 ns ± 1.87 ns per loop (mean ± std. dev. of 7 runs, 10000000 loops each)

>>> %timeit t = (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
12.5 ns ± 0.199 ns per loop (mean ± std. dev. of 7 runs, 100000000 loops each)
```

Wrap-Up

Lists and tuples are fast and low-overhead objects to use when your data already has an intrinsic ordering to it. This intrinsic ordering allows you to sidestep the search problem in these structures: if the ordering is known beforehand, lookups are $O(1)$, avoiding an expensive $O(n)$ linear search. While lists can be resized, you must take care to properly understand how much overallocation is happening to ensure that the

dataset can still fit in memory. On the other hand, tuples can be created quickly and without the added overhead of lists, at the cost of not being modifiable. In “[Aren’t Python Lists Good Enough?](#)” on page 115, we discuss how to preallocate lists to alleviate some of the burden regarding frequent appends to Python lists, and we look at other optimizations that can help manage these problems.

In the next chapter, we go over the computational properties of dictionaries, which solve the search/lookup problems with unordered data at the cost of overhead.

HPP Ch 4 | Dictionaries and Sets (performance)

Dictionaries and Sets

Questions You'll Be Able to Answer After This Chapter

- What are dictionaries and sets good for?
- How are dictionaries and sets the same?
- What is the overhead when using a dictionary?
- How can I optimize the performance of a dictionary?
- How does Python use dictionaries to keep track of namespaces?

Sets and dictionaries are ideal data structures to be used when your data has no intrinsic order (except for insertion order) but does have a unique object that can be used to reference it (the reference object is normally a string, but it can be any hashable type). This reference object is called the *key*, while the data is the *value*. Dictionaries and sets are almost identical, except that sets do not actually contain values: a set is simply a collection of unique keys. As the name implies, sets are very useful for doing set operations.



A *hashable* type is one that implements both the `__hash__` magic function and either `__eq__` or `__cmp__`. All native types in Python already implement these, and any user classes have default values. See “[Hash Functions and Entropy](#)” on page 88 for more details.

While we saw in the previous chapter that we are restricted to, at best, $O(\log n)$ lookup time on lists/tuples with no intrinsic order (through a search operation), dictionaries and sets give us $O(1)$ lookups based on the arbitrary index. In addition, like

lists/tuples, dictionaries and sets have $O(1)$ insertion time.¹ As we will see in “[How Do Dictionaries and Sets Work?](#)” on page 83, this speed is accomplished through the use of an open address hash table as the underlying data structure.

However, there is a cost to using dictionaries and sets. First, they generally take up a larger footprint in memory. Also, although the complexity for insertions/lookups is $O(1)$, the actual speed depends greatly on the hashing function that is in use. If the hash function is slow to evaluate, any operations on dictionaries or sets will be similarly slow.

Let’s look at an example. Say we want to store contact information for everyone in the phone book. We would like to store this in a form that will make it simple to answer the question “What is John Doe’s phone number?” in the future. With lists, we would store the phone numbers and names sequentially and scan through the entire list to find the phone number we required, as shown in [Example 4-1](#).

Example 4-1. Phone book lookup with a list

```
def find_phonenumber(phonebook, name):
    for n, p in phonebook:
        if n == name:
            return p
    return None

phonebook = [
    ("John Doe", "555-555-5555"),
    ("Albert Einstein", "212-555-5555"),
]
print(f"John Doe's phone number is {find_phonenumber(phonebook, 'John Doe')}")
```



We could also do this by sorting the list and using the `bisect` module (from [Example 3-4](#)) in order to get $O(\log n)$ performance.

With a dictionary, however, we can simply have the “index” be the names and the “values” be the phone numbers, as shown in [Example 4-2](#). This allows us to simply look up the value we need and get a direct reference to it, instead of having to read every value in our dataset.

¹ As we will discuss in “[Hash Functions and Entropy](#)” on page 88, dictionaries and sets are very dependent on their hash functions. If the hash function for a particular datatype is not $O(1)$, any dictionary or set containing that type will no longer have its $O(1)$ guarantee.

Example 4-2. Phone book lookup with a dictionary

```
phonebook = {  
    "John Doe": "555-555-5555",  
    "Albert Einstein": "212-555-5555",  
}  
print(f"John Doe's phone number is {phonebook['John Doe']}")
```

For large phone books, the difference between the $O(1)$ lookup of the dictionary and the $O(n)$ time for linear search over the list (or, at best, the $O(\log n)$ complexity with the `bisect` module) is quite substantial.



Create a script that times the performance of the list-bisect method versus a dictionary for finding a number in a phone book. How does the timing scale as the size of the phone book grows?

If, on the other hand, we wanted to answer the question “How many unique first names are there in my phone book?” we could use the power of sets. Recall that a set is simply a collection of *unique* keys—this is the exact property we would like to enforce in our data. This is in stark contrast to a list-based approach, where that property needs to be enforced separately from the data structure by comparing all names with all other names. [Example 4-3](#) illustrates.

Example 4-3. Finding unique names with lists and sets

```
def list_unique_names(phonebook):  
    unique_names = []  
    for name, phonenumber in phonebook: ❶  
        first_name, last_name = name.split(" ", 1)  
        for unique in unique_names: ❷  
            if unique == first_name:  
                break  
        else:  
            unique_names.append(first_name)  
    return len(unique_names)  
  
def set_unique_names(phonebook):  
    unique_names = set()  
    for name, phonenumber in phonebook: ❸  
        first_name, last_name = name.split(" ", 1)  
        unique_names.add(first_name) ❹  
    return len(unique_names)  
  
phonebook = [  
    ("John Doe", "555-555-5555"),  
    ("Albert Einstein", "212-555-5555"),
```

```

    ("John Murphey", "202-555-5555"),
    ("Albert Rutherford", "647-555-5555"),
    ("Guido van Rossum", "301-555-5555"),
]

print("Number of unique names from set method:", set_unique_names(phonebook))
print("Number of unique names from list method:", list_unique_names(phonebook))

```

- ❶, ❸ We must go over all the items in our phone book, and thus this loop costs $O(n)$.
- ❷ Here, we must check the current name against all the unique names we have already seen. If it is a new unique name, we add it to our list of unique names. We then continue through the list, performing this step for every item in the phone book.
- ❹ For the set method, instead of iterating over all unique names we have already seen, we can simply add the current name to our set of unique names. Because sets guarantee the uniqueness of the keys they contain, if you try to add an item that is already in the set, that item simply won't be added. Furthermore, this operation costs $O(1)$.

The list algorithm's inner loop iterates over `unique_names`, which starts out as empty and then grows, in the worst case, when all names are unique, to be the size of phone book. This can be seen as performing a **linear search** for each name in the phone book over a list that is constantly growing. Thus, the complete algorithm performs as $O(n^2)$.

On the other hand, the set algorithm has no inner loop; the `set.add` operation is an $O(1)$ process that completes in a fixed number of operations regardless of how large the phone book is (there are some minor caveats to this, which we will cover while discussing the implementation of dictionaries and sets). Thus, the only nonconstant contribution to the complexity of this algorithm is the loop over the phone book, making this algorithm perform in $O(n)$.

When timing these two algorithms using a phonebook with 10,000 entries and 7,412 unique first names, we see how drastic the difference between $O(n)$ and $O(n^2)$ can be:

```

>>> %timeit list_unique_names(large_phonebook)
1.13 s ± 26.8 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

>>> %timeit set_unique_names(large_phonebook)
4.48 ms ± 177 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

```

In other words, the set algorithm gave us a $252\times$ speedup! In addition, as the size of the phonebook grows, the speed gains increase (we get a $557\times$ speedup with a phone book with 100,000 entries and 15,574 unique first names).

How Do Dictionaries and Sets Work?

Dictionaries and sets use *hash tables* to achieve their $O(1)$ lookups and insertions. This efficiency is the result of a very clever usage of a **hash function** to turn an arbitrary key (i.e., a string or object) into an index for a list. The hash function and list can later be used to determine where any particular piece of data is right away, without a search. By turning the data's key into something that can be used like a list index, we can get the same performance as with a list. In addition, instead of having to refer to data by a numerical index, which itself implies some ordering to the data, we can refer to it by this arbitrary key.

Inserting and Retrieving

To create a hash table from scratch, we start with some allocated memory, similar to what we started with for arrays. For an array, if we want to insert data, we simply find the smallest unused bucket and insert our data there (and resize if necessary). For hash tables, we must first figure out the placement of the data in this contiguous chunk of memory.

The placement of the new data is contingent on two properties of the data we are inserting: the hashed value of the key and how the value compares to other objects. This is because when we insert data, the key is first hashed and masked so that it turns into an effective index in an array.² The mask makes sure that the hash value, which can take the value of any integer, fits within the allocated number of buckets. So if we have allocated 8 blocks of memory and our hash value is 28975, we consider the bucket at index $28975 \ \& \ 0b111 = 7$. If, however, our dictionary has grown to require 512 blocks of memory, the mask becomes $0b11111111$ (and in this case, we would consider the bucket at index $28975 \ \& \ 0b11111111$).

Now we must check if this bucket is already in use. If it is empty, we can insert the key and the value into this block of memory. We store the key so that we can make sure we are retrieving the correct value on lookups. If it is in use and the value of the bucket is equal to the value we wish to insert (a comparison done with the `cmp` builtin), then the key/value pair is already in the hash table and we can return. However, if the values don't match, we must find a new place to put the data.

² A *mask* is a binary number that truncates the value of a number. So $0b11111101 \ \& \ 0b111 = 0b101 = 5$ represents the operation of $0b111$ masking the number $0b11111101$. This operation can also be thought of as taking a certain number of the least-significant digits of a number.

As an extra optimization, Python first appends the key/value data into a standard array and then stores only the *index* into this array in the hash table. This allows us to reduce the amount of memory used by 30–95%.³ In addition, this gives us the interesting property that we keep a record of the order which new items were added into the dictionary (which, since Python 3.7, is a guarantee that all dictionaries give).

To find the new index, we compute it using a simple linear function, a method called *probing*. Python’s probing mechanism adds a contribution from the higher-order bits of the original hash (recall that for a table of length 8 we considered only the last three bits of the hash for the initial index, through the use of a mask value of `mask = 0b111 = bin(8 - 1)`). Using these higher-order bits gives each hash a different sequence of next possible hashes, which helps to avoid future collisions.

There is a lot of freedom when picking the algorithm to generate a new index; however, it is quite important that the scheme visits every possible index in order to evenly distribute the data in the table. How well distributed the data is throughout the hash table is called the *load factor* and is related to the *entropy* of the hash function. The pseudocode in [Example 4-4](#) illustrates the calculation of hash indices used in CPython 3.7. This also shows an interesting fact about hash tables: most of the storage space they have is empty!

Example 4-4. Dictionary lookup sequence

```
def index_sequence(key, mask=0b111, PERTURB_SHIFT=5):
    perturb = hash(key) ❶
    i = perturb & mask
    yield i
    while True:
        perturb >>= PERTURB_SHIFT
        i = (i * 5 + perturb + 1) & mask
        yield i
```

- ❶ hash returns an integer, while the actual C code in CPython uses an unsigned integer. Because of that, this pseudocode doesn’t replicate exactly the behavior in CPython; however, it is a good approximation.

This probing is a modification of the naive method of *linear probing*. In linear probing, we simply yield the values `i = (i * 5 + perturb + 1) & mask`, where `i` is initialized to the hash value of the key.⁴ An important thing to note is that linear probing deals only with the last several bits of the hash and disregards the rest (i.e.,

3 The discussion that led to this improvement can be found at <https://oreil.ly/Pq7Lm>.

4 The value of 5 comes from the properties of a linear congruential generator (LCG), which is used in generating random numbers.

for a dictionary with eight elements, we look only at the last three bits since at that point the mask is 0×111). This means that if hashing two items gives the same last three binary digits, we will not only have a collision, but also the sequence of probed indices will be the same. The perturbed scheme that Python uses will start taking into consideration more bits from the items' hashes to resolve this problem.

A similar procedure is done when we are performing lookups on a specific key: the given key is transformed into an index, and that index is examined. If the key in that index matches (recall that we also store the original key when doing insert operations), then we can return that value. If it doesn't, we keep creating new indices using the same scheme, until we either find the data or hit an empty bucket. If we hit an empty bucket, we can conclude that the data does not exist in the table.

Figure 4-1 illustrates the process of adding data into a hash table. Here, we chose to create a hash function that simply uses the first letter of the input. We accomplish this by using Python's `ord` function on the first letter of the input to get the integer representation of that letter (recall that hash functions must return integers). As we'll see in [“Hash Functions and Entropy” on page 88](#), Python provides hashing functions for most of its types, so typically you won't have to provide one yourself except in extreme situations.

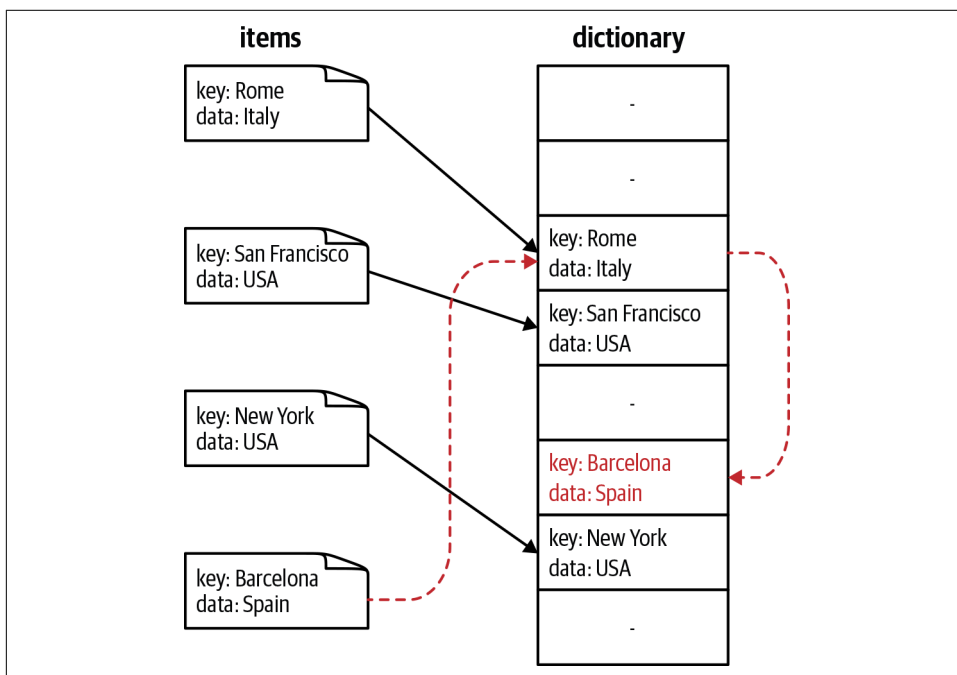


Figure 4-1. The resulting hash table from inserting with collisions

Insertion of the key Barcelona causes a collision, and a new index is calculated using the scheme in [Example 4-4](#). This dictionary can also be created in Python using the code in [Example 4-5](#).

Example 4-5. Custom hashing function

```
class City(str):
    def __hash__(self):
        return ord(self[0])

# We create a dictionary where we assign arbitrary values to cities
data = {
    City("Rome"): 'Italy',
    City("San Francisco"): 'USA',
    City("New York"): 'USA',
    City("Barcelona"): 'Spain',
}
```

In this case, Barcelona and Rome cause the hash collision ([Figure 4-1](#) shows the outcome of this insertion). We see this because, for a dictionary with four elements, we have a mask value of 0b111. As a result, Barcelona and Rome will try to use the same index:

```
hash("Barcelona") = ord("B") & 0b111
                  = 66 & 0b111
                  = 0b1000010 & 0b111
                  = 0b010 = 2

hash("Rome") = ord("R") & 0b111
             = 82 & 0b111
             = 0b1010010 & 0b111
             = 0b010 = 2
```

Exercise

Work through the following problems. A discussion of hash collisions follows:

1. *Finding an element*—Using the dictionary created in [Example 4-5](#), what would a lookup on the key Johannesburg look like? What indices would be checked?
2. *Deleting an element*—Using the dictionary created in [Example 4-5](#), how would you handle the deletion of the key Rome? How would subsequent lookups for the keys Rome and Barcelona be handled?
3. *Hash collisions*—Considering the dictionary created in [Example 4-5](#), how many hash collisions could you expect if 500 cities, with names all starting with an uppercase letter, were added into a hash table? How about 1,000 cities? Can you think of a way of lowering the number of collisions?

For 500 cities, there would be approximately 474 dictionary elements that collided with a previous value ($500 - 26$), with each hash having $500 / 26 = 19.2$ cities associated with it. For 1,000 cities, 974 elements would collide, and each hash would have $1,000 / 26 = 38.4$ cities associated with it. This is because the hash is based simply on the numerical value of the first letter, which can take only a value from A–Z, allowing for only 26 independent hash values. This means that a lookup in this table could require as many as 38 subsequent lookups to find the correct value. To fix this, we must increase the number of possible hash values by considering other aspects of the city in the hash. The default hash function on a string considers every character in order to maximize the number of possible values. See “[Hash Functions and Entropy](#)” on page 88 for more explanation.

Deletion

When a value is deleted from a hash table, we cannot simply write a NULL to that bucket of memory. This is because we have used NULLs as a sentinel value while probing for hash collisions. As a result, we must write a special value that signifies that the bucket is empty, but there still may be values after it to consider when resolving a hash collision. So if “Rome” was deleted from the dictionary, subsequent lookups for “Barcelona” will first see this sentinel value where “Rome” used to be and instead of stopping, continue to check the next indices given by the `index_sequence`. These empty slots can be written to in the future and are removed when the hash table is resized.

Resizing

As more items are inserted into the hash table, the table itself must be resized to accommodate them. It can be shown that a table that is no more than two-thirds full will have optimal space savings while still having a good bound on the number of collisions to expect. Thus, when a table reaches this critical point, it is grown. To do this, a larger table is allocated (i.e., more buckets in memory are reserved), the mask is adjusted to fit the new table, and all elements of the old table are reinserted into the new one. This requires recomputing indices, since the changed mask will change the resulting index. As a result, resizing large hash tables can be quite expensive! However, since we do this resizing operation only when the table is too small, as opposed to doing it on every insert, the amortized cost of an insert is still $O(1)$.⁵

By default, the smallest size of a dictionary or set is 8 (that is, if you are storing only three values, Python will still allocate eight elements), and it will resize by $3\times$ if the

⁵ Amortized analysis looks at the average complexity of an algorithm. This means that some inserts will be much more expensive, but on average, inserts will be $O(1)$.

dictionary is more than two-thirds full. So once the sixth item is being inserted into the originally empty dictionary, it will be resized to hold 18 elements. At this point, once the 13th element is inserted into the object, it will be resized to 39, then 81, and so on, always increasing the size by $3\times$ (we will explain how to calculate a dictionary's size in “Hash Functions and Entropy” on page 88). This gives the following possible sizes:

8; 18; 39; 81; 165; 333; 669; 1,341; 2,685; 5,373; 10,749; 21,501; 43,005; ...

It is important to note that resizing can happen to make a hash table larger *or* smaller. That is, if sufficiently many elements of a hash table are deleted, the table can be scaled down in size. However, *resizing happens only during an insert*.

Hash Functions and Entropy

Objects in Python are generally hashable, since they already have built-in `__hash__` and `__cmp__` functions associated with them. For numerical types (`int` and `float`), the hash is based simply on the bit value of the number they represent. Tuples and strings have a hash value that is based on their contents. Lists, on the other hand, do not support hashing because their values can change. Since a list's values can change, so could the hash that represents the list, which would change the relative placement of that key in the hash table.⁶

User-defined classes also have default hash and comparison functions. The default `__hash__` function simply returns the object's placement in memory as given by the built-in `id` function. Similarly, the `__cmp__` operator compares the numerical value of the object's placement in memory.

This is generally acceptable, since two instances of a class are generally different and should not collide in a hash table. However, in some cases we would like to use `set` or `dict` objects to disambiguate between items. Take the following class definition:

```
class Point(object):
    def __init__(self, x, y):
        self.x, self.y = x, y
```

If we were to instantiate multiple `Point` objects with the same values for `x` and `y`, they would all be independent objects in memory and thus have different placements in memory, which would give them all different hash values. This means that putting them all into a `set` would result in all of them having individual entries:

```
>>> p1 = Point(1,1)
>>> p2 = Point(1,1)
>>> set([p1, p2])
```

⁶ More information about this can be found at <https://oreil.ly/g4I5->.

```

set([<__main__.Point at 0x1099bfc90>, <__main__.Point at 0x1099bfd0>])
>>> Point(1,1) in set([p1, p2])
False

```

We can remedy this by forming a custom hash function that is based on the actual contents of the object as opposed to the object's placement in memory. The hash function can be any function as long as it consistently gives the same result for the same object (there are also considerations regarding the entropy of the hashing function, which we will discuss later.) The following redefinition of the `Point` class will yield the results we expect:

```

class Point(object):
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __hash__(self):
        return hash((self.x, self.y))

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

```

This allows us to create entries in a set or dictionary indexed by the properties of the `Point` object rather than the memory address of the instantiated object:

```

>>> p1 = Point(1,1)
>>> p2 = Point(1,1)
>>> set([p1, p2])
set([<__main__.Point at 0x109b95910>])
>>> Point(1, 1) in set([p1, p2])
True

```

As alluded to when we discussed hash collisions, a custom-selected hash function should be careful to evenly distribute hash values in order to avoid collisions. Having many collisions will degrade the performance of a hash table: if most keys have collisions, we need to constantly “probe” the other values, effectively walking a potentially large portion of the dictionary to find the key in question. In the worst case, when all keys in a dictionary collide, the performance of lookups in the dictionary is $O(n)$ and thus the same as if we were searching through a list.

If we know that we are storing 5,000 values in a dictionary and we need to create a hashing function for the object we wish to use as a key, we must be aware that the dictionary will be stored in a hash table of size 16,384⁷ and thus only the last 14 bits of our hash are being used to create an index (for a hash table of this size, the mask is `bin(16_384 - 1) = 0b11111111111111`).

⁷ 5,000 values need a dictionary that has at least 8,333 buckets. The first available size that can fit this many elements is 16,384.

This idea of “how well distributed my hash function is” is called the *entropy* of the hash function. Entropy is defined as

$$S = - \sum_i p(i) \cdot \log(p(i))$$

where $p(i)$ is the probability that the hash function gives hash i . It is maximized when every hash value has equal probability of being chosen. A hash function that maximizes entropy is called an *ideal* hash function since it guarantees the minimal number of collisions.

For an infinitely large dictionary, the hash function used for integers is ideal. This is because the hash value for an integer is simply the integer itself! For an infinitely large dictionary, the mask value is infinite, and thus we consider all bits in the hash value. Therefore, given any two numbers, we can guarantee that their hash values will not be the same.

However, if we made this dictionary finite, we could no longer have this guarantee. For example, for a dictionary with four elements, the mask we use is `0b111`. Thus the hash value for the number 5 is `5 & 0b111 = 5`, and the hash value for 501 is `501 & 0b111 = 5`, and so their entries will collide.



To find the mask for a dictionary with an arbitrary number of elements, N , we first find the minimum number of buckets that dictionary must have to still be two-thirds full ($N * (2 / 3 + 1)$). Then we find the smallest dictionary size that will hold this number of elements (8; 32; 128; 512; 2,048; etc.) and find the number of bits necessary to hold this number. For example, if $N=1039$, then we must have at least 1,731 buckets, which means we need a dictionary with 2,048 buckets. Thus the mask is `bin(2048 - 1) = 0b1111111111`.

There is no single best hash function to use when using a finite dictionary. However, knowing up front what range of values will be used and how large the dictionary will be helps in making a good selection. For example, if we are storing all 676 combinations of two lowercase letters as keys in a dictionary (*aa*, *ab*, *ac*, etc.), a good hashing function would be the one shown in [Example 4-6](#).

Example 4-6. Optimal two-letter hashing function

```
def twoletter_hash(key):
    offset = ord('a')
    k1, k2 = key
    return (ord(k2) - offset) + 26 * (ord(k1) - offset)
```

This gives no hash collisions for any combination of two lowercase letters, considering a mask of 0b111111111 (a dictionary of 676 values will be held in a hash table of length 2,048, which has a mask of $\text{bin}(2048 - 1) = 0b111111111$).

Example 4-7 very explicitly shows the ramifications of having a bad hashing function for a user-defined class—here, the cost of a bad hash function (in fact, it is the worst possible hash function!) is a 41.8× slowdown of lookups.

Example 4-7. Timing differences between good and bad hashing functions

```
import string
import timeit

class BadHash(str):
    def __hash__(self):
        return 42

class GoodHash(str):
    def __hash__(self):
        """
        This is a slightly optimized version of twoletter_hash
        """
        return ord(self[1]) + 26 * ord(self[0]) - 2619

baddict = set()
gooddict = set()
for i in string.ascii_lowercase:
    for j in string.ascii_lowercase:
        key = i + j
        baddict.add(BadHash(key))
        gooddict.add(GoodHash(key))

badtime = timeit.repeat(
    "key in baddict",
    setup = "from __main__ import baddict, BadHash; key = BadHash('zz')",
    repeat = 3,
    number = 1_000_000,
)
goodtime = timeit.repeat(
    "key in gooddict",
    setup = "from __main__ import gooddict, GoodHash; key = GoodHash('zz')",
    repeat = 3,
    number = 1_000_000,
)

print(f"Min lookup time for baddict: {min(badtime)}")
print(f"Min lookup time for gooddict: {min(goodtime)}")

# Results:
#   Min lookup time for baddict: 17.719061855008476
#   Min lookup time for gooddict: 0.42408075400453527
```

Exercise

1. Show that for an infinite dictionary (and thus an infinite mask), using an integer's value as its hash gives no collisions.
2. Show that the hashing function given in [Example 4-6](#) is ideal for a hash table of size 1,024. Why is it not ideal for smaller hash tables?

Dictionaries and Namespaces

Doing a lookup on a dictionary is fast; however, doing it unnecessarily will slow down your code, just as any extraneous lines will. One area where this surfaces is in Python's namespace management, which heavily uses dictionaries to do its lookups.

Whenever a variable, function, or module is invoked in Python, there is a hierarchy that determines where it looks for these objects. First, Python looks inside the `locals()` array, which has entries for all local variables. Python works hard to make local variable lookups fast, and this is the only part of the chain that doesn't require a dictionary lookup. If it doesn't exist there, the `globals()` dictionary is searched. Finally, if the object isn't found there, the `__builtin__` object is searched. It is important to note that while `locals()` and `globals()` are explicitly dictionaries and `__builtin__` is technically a module object, when searching `__builtin__` for a given property, we are just doing a dictionary lookup inside *its* `locals()` map (this is the case for all module objects and class objects!).

To make this clearer, let's look at a simple example of calling functions that are defined in different scopes ([Example 4-8](#)). We can disassemble the functions with the `dis` module ([Example 4-9](#)) to get a better understanding of how these namespace lookups are happening (see “Using the `dis` Module to Examine CPython Bytecode” on page 55).

Example 4-8. Namespace lookups

```
import math
from math import sin

def test1(x):
    """
    >>> %timeit test1(123_456)
    162 µs ± 3.82 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
    """
    res = 1
    for _ in range(1000):
        res += math.sin(x)
```

```

    return res

def test2(x):
    """
    >>> %timeit test2(123_456)
    124 µs ± 6.77 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
    """
    res = 1
    for _ in range(1000):
        res += sin(x)
    return res

def test3(x, sin=math.sin):
    """
    >>> %timeit test3(123_456)
    105 µs ± 3.35 µs per loop (mean ± std. dev. of 7 runs, 10000 loops each)
    """
    res = 1
    for _ in range(1000):
        res += sin(x)
    return res

```

Example 4-9. Namespace lookups disassembled

```

>>> dis.dis(test1)
...cut..
      20 LOAD_GLOBAL              1 (math)
      22 LOAD_METHOD              2 (sin)
      24 LOAD_FAST                 0 (x)
      26 CALL_METHOD              1
...cut..

>>> dis.dis(test2)
...cut...
      20 LOAD_GLOBAL              1 (sin)
      22 LOAD_FAST                 0 (x)
      24 CALL_FUNCTION             1
...cut...

>>> dis.dis(test3)
...cut...
      20 LOAD_FAST                 1 (sin)
      22 LOAD_FAST                 0 (x)
      24 CALL_FUNCTION             1
...cut...

```

The first function, `test1`, makes the call to `sin` by explicitly looking at the `math` library. This is also evident in the bytecode that is produced: first a reference to the `math` module must be loaded, and then we do an attribute lookup on this module until we finally have a reference to the `sin` function. This is done through two dictionary lookups: one to find the `math` module and one to find the `sin` function within the module.

On the other hand, `test2` explicitly imports the `sin` function from the `math` module, and the function is then directly accessible within the global namespace. This means we can avoid the lookup of the `math` module and the subsequent attribute lookup. However, we still must find the `sin` function within the global namespace. This is yet another reason to be explicit about what functions you are importing from a module. This practice not only makes code more readable, because the reader knows exactly what functionality is required from external sources, but it also simplifies changing the implementation of specific functions and generally speeds up code!

Finally, `test3` defines the `sin` function as a keyword argument, with its default value being a reference to the `sin` function within the `math` module. While we still do need to find a reference to this function within the module, this is necessary only when the `test3` function is first defined. After this, the reference to the `sin` function is stored within the function definition as a local variable in the form of a default keyword argument. As mentioned previously, local variables do not need a dictionary lookup to be found; they are stored in a very slim array that has very fast lookup times. Because of this, finding the function is quite fast!

While these effects are an interesting result of the way namespaces in Python are managed, `test3` is definitely not “Pythonic.” Luckily, these extra dictionary lookups start to degrade performance only when they are called a lot (i.e., in the innermost block of a very fast loop, such as in the Julia set example). With this in mind, a more readable solution would be to set a local variable with the global reference before the loop is started. We’ll still have to do the global lookup once whenever the function is called, but all the calls to that function in the loop will be made faster. This speaks to the fact that even minute slowdowns in code can be amplified if that code is being run millions of times. Even though a dictionary lookup itself may take only several hundred nanoseconds, if we are looping millions of times over this lookup, those nanoseconds can quickly add up.



A message about microbenchmarks: it may seem confusing that in **Example 4-8** we add in extra work with the `for` loop and the modification to the `res` variable. Originally, each of these functions simply had the relevant `return sin(x)` line and nothing else. As a result, we were also getting nanosecond runtimes and results that did not make any sense!

When we added a bigger workload within each function, as done through the loop and the modification of the `res` variable, we started seeing the results we expected. With a bigger workload inside the function, we can be more sure that we are not measuring overhead from the benchmarking/timing process. In general, when you are running benchmarks and have a difference in timing in the nanoseconds, it's important to sit back for a second and think through whether the experiment you are running is valid or whether you are measuring noise or unrelated timings as a result of instrumentation.

Wrap-Up

Dictionaries and sets provide a fantastic way to store data that can be indexed by a key. The way this key is used, through the hashing function, can greatly affect the resulting performance of the data structure. Furthermore, understanding how dictionaries work gives you a better understanding not only of how to organize your data but also of how to organize your code, since dictionaries are an intrinsic part of Python's internal functionality.

In the next chapter we will explore generators, which allow us to provide data to code with more control over ordering and without having to store full datasets in memory beforehand. This lets us sidestep many of the possible hurdles that we might encounter when using any of Python's intrinsic data structures.

PNP Ch 19 | Dictionary [companion]

Chapter 19: Dictionary

Parameter	Details
key	The desired key to lookup
value	The value to set or return

Section 19.1: Introduction to Dictionary

A dictionary is an example of a *key value store* also known as *Mapping* in Python. It allows you to store and retrieve elements by referencing a key. As dictionaries are referenced by key, they have very fast lookups. As they are primarily used for referencing items by key, they are not sorted.

creating a dict

Dictionaries can be initiated in many ways:

literal syntax

```
d = {} # empty dict
d = {'key': 'value'} # dict with initial values
```

Python 3.x Version ≥ 3.5

```
# Also unpacking one or multiple dictionaries with the literal syntax is possible

# makes a shallow copy of otherdict
d = {**otherdict}
# also updates the shallow copy with the contents of the yetanotherdict.
d = {**otherdict, **yetanotherdict}
```

dict comprehension

```
d = {k:v for k,v in [('key', 'value'),]}
```

see also: Comprehensions

built-in class: dict()

```
d = dict() # empty dict
d = dict(key='value') # explicit keyword arguments
d = dict([('key', 'value')]) # passing in a list of key/value pairs
# make a shallow copy of another dict (only possible if keys are only strings!)
d = dict(**otherdict)
```

modifying a dict

To add items to a dictionary, simply create a new key with a value:

```
d['newkey'] = 42
```

It also possible to add `list` and dictionary as value:

```
d['new_list'] = [1, 2, 3]
d['new_dict'] = {'nested_dict': 1}
```

To delete an item, delete the key from the dictionary:

```
del d['newkey']
```

Section 19.2: Avoiding KeyError Exceptions

One common pitfall when using dictionaries is to access a non-existent key. This typically results in a `KeyError` exception

```
mydict = {}
mydict['not there']

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'not there'
```

One way to avoid key errors is to use the `dict.get` method, which allows you to specify a default value to return in the case of an absent key.

```
value = mydict.get(key, default_value)
```

Which returns `mydict[key]` if it exists, but otherwise returns `default_value`. Note that this doesn't add key to `mydict`. So if you want to retain that key value pair, you should use `mydict.setdefault(key, default_value)`, which *does* store the key value pair.

```
mydict = {}
print(mydict)
# {}
print(mydict.get("foo", "bar"))
# bar
print(mydict)
# {}
print(mydict.setdefault("foo", "bar"))
# bar
print(mydict)
# {'foo': 'bar'}
```

An alternative way to deal with the problem is catching the exception

```
try:
    value = mydict[key]
except KeyError:
    value = default_value
```

You could also check if the key is in the dictionary.

```
if key in mydict:
    value = mydict[key]
else:
    value = default_value
```

Do note, however, that in multi-threaded environments it is possible for the key to be removed from the dictionary after you check, creating a race condition where the exception can still be thrown.

Another option is to use a subclass of `dict`, `collections.defaultdict`, that has a `default_factory` to create new entries in the dict when given a new_key.

Section 19.3: Iterating Over a Dictionary

If you use a dictionary as an iterator (e.g. in a `for` statement), it traverses the **keys** of the dictionary. For example:

```
d = {'a': 1, 'b': 2, 'c':3}
for key in d:
    print(key, d[key])
# c 3
# b 2
# a 1
```

The same is true when used in a comprehension

```
print([key for key in d])
# ['c', 'b', 'a']
```

Python 3.x Version ≥ 3.0

The `items()` method can be used to loop over both the **key** and **value** simultaneously:

```
for key, value in d.items():
    print(key, value)
# c 3
# b 2
# a 1
```

While the `values()` method can be used to iterate over only the values, as would be expected:

```
for key, value in d.values():
    print(key, value)
# 3
# 2
# 1
```

Python 2.x Version ≥ 2.2

Here, the methods `keys()`, `values()` and `items()` return lists, and there are the three extra methods `iterkeys()`, `itervalues()` and `iteritems()` to return iterators.

Section 19.4: Dictionary with default values

Available in the standard library as [defaultdict](#)

```
from collections import defaultdict

d = defaultdict(int)
d['key']           # 0
d['key'] = 5
d['key']           # 5

d = defaultdict(lambda: 'empty')
d['key']           # 'empty'
d['key'] = 'full'
d['key']           # 'full'
```

[*] Alternatively, if you must use the built-in `dict` class, using `dict.setdefault()` will allow you to create a default whenever you access a key that did not exist before:

```
>>> d = {}
{}
>>> d.setdefault('Another_key', []).append("This worked!")
>>> d
```

```
{'Another_key': ['This worked!']}
```

Keep in mind that if you have many values to add, `dict.setdefault()` will create a new instance of the initial value (in this example a `[]`) every time it's called - which may create unnecessary workloads.

[*] *Python Cookbook, 3rd edition, by David Beazley and Brian K. Jones (O'Reilly). Copyright 2013 David Beazley and Brian Jones, 978-1-449-34037-7.*

Section 19.5: Merging dictionaries

Consider the following dictionaries:

```
>>> fish = {'name': "Nemo", 'hands': "fins", 'special': "gills"}
>>> dog = {'name': "Clifford", 'hands': "paws", 'color': "red"}
```

Python 3.5+

```
>>> fishdog = {**fish, **dog}
>>> fishdog
{'hands': 'paws', 'color': 'red', 'name': 'Clifford', 'special': 'gills'}
```

As this example demonstrates, duplicate keys map to their lattermost value (for example "Clifford" overrides "Nemo").

Python 3.3+

```
>>> from collections import ChainMap
>>> dict(ChainMap(fish, dog))
{'hands': 'fins', 'color': 'red', 'special': 'gills', 'name': 'Nemo'}
```

With this technique the foremost value takes precedence for a given key rather than the last ("Clifford" is thrown out in favor of "Nemo").

Python 2.x, 3.x

```
>>> from itertools import chain
>>> dict(chain(fish.items(), dog.items()))
{'hands': 'paws', 'color': 'red', 'name': 'Clifford', 'special': 'gills'}
```

This uses the lattermost value, as with the `**`-based technique for merging ("Clifford" overrides "Nemo").

```
>>> fish.update(dog)
>>> fish
{'color': 'red', 'hands': 'paws', 'name': 'Clifford', 'special': 'gills'}
```

`dict.update` uses the latter dict to overwrite the previous one.

Section 19.6: Accessing keys and values

When working with dictionaries, it's often necessary to access all the keys and values in the dictionary, either in a `for` loop, a list comprehension, or just as a plain list.

Given a dictionary like:

```
mydict = {
    'a': '1',
```

```
'b': '2'  
}
```

You can get a list of keys using the `keys()` method:

```
print(mydict.keys())  
# Python2: ['a', 'b']  
# Python3: dict_keys(['b', 'a'])
```

If instead you want a list of values, use the `values()` method:

```
print(mydict.values())  
# Python2: ['1', '2']  
# Python3: dict_values(['2', '1'])
```

If you want to work with both the key and its corresponding value, you can use the `items()` method:

```
print(mydict.items())  
# Python2: [('a', '1'), ('b', '2')]  
# Python3: dict_items([('b', '2'), ('a', '1')])
```

NOTE: Because a `dict` is unsorted, `keys()`, `values()`, and `items()` have no sort order. Use `sort()`, `sorted()`, or an `OrderedDict` if you care about the order that these methods return.

Python 2/3 Difference: In Python 3, these methods return special iterable objects, not lists, and are the equivalent of the Python 2 `iterkeys()`, `itervalues()`, and `iteritems()` methods. These objects can be used like lists for the most part, though there are some differences. See [PEP 3106](#) for more details.

Section 19.7: Accessing values of a dictionary

```
dictionary = {"Hello": 1234, "World": 5678}  
print(dictionary["Hello"])
```

The above code will print `1234`.

The string `"Hello"` in this example is called a *key*. It is used to lookup a value in the `dict` by placing the key in square brackets.

The number `1234` is seen after the respective colon in the `dict` definition. This is called the *value* that `"Hello"` maps to in this `dict`.

Looking up a value like this with a key that does not exist will raise a `KeyError` exception, halting execution if uncaught. If we want to access a value without risking a `KeyError`, we can use the `dictionary.get` method. By default if the key does not exist, the method will return `None`. We can pass it a second value to return instead of `None` in the event of a failed lookup.

```
w = dictionary.get("whatever")  
x = dictionary.get("whatever", "nuh-uh")
```

In this example `w` will get the value `None` and `x` will get the value `"nuh-uh"`.

Section 19.8: Creating a dictionary

Rules for creating a dictionary:

- Every key must be **unique** (otherwise it will be overridden)
- Every key must be **hashable** (can use the `hash` function to hash it; otherwise `TypeError` will be thrown)
- There is no particular order for the keys.

```
# Creating and populating it with values
stock = {'eggs': 5, 'milk': 2}

# Or creating an empty dictionary
dictionary = {}

# And populating it after
dictionary['eggs'] = 5
dictionary['milk'] = 2

# Values can also be lists
mydict = {'a': [1, 2, 3], 'b': ['one', 'two', 'three']}

# Use list.append() method to add new elements to the values list
mydict['a'].append(4) # => {'a': [1, 2, 3, 4], 'b': ['one', 'two', 'three']}
mydict['b'].append('four') # => {'a': [1, 2, 3, 4], 'b': ['one', 'two', 'three', 'four']}

# We can also create a dictionary using a list of two-items tuples
iterable = [('eggs', 5), ('milk', 2)]
dictionary = dict(iterables)

# Or using keyword argument:
dictionary = dict(eggs=5, milk=2)

# Another way will be to use the dict.fromkeys:
dictionary = dict.fromkeys((milk, eggs)) # => {'milk': None, 'eggs': None}
dictionary = dict.fromkeys((milk, eggs), (2, 5)) # => {'milk': 2, 'eggs': 5}
```

Section 19.9: Creating an ordered dictionary

You can create an ordered dictionary which will follow a determined order when iterating over the keys in the dictionary.

Use `OrderedDict` from the `collections` module. This will always return the dictionary elements in the original insertion order when iterated over.

```
from collections import OrderedDict

d = OrderedDict()
d['first'] = 1
d['second'] = 2
d['third'] = 3
d['last'] = 4

# Outputs "first 1", "second 2", "third 3", "last 4"
for key in d:
    print(key, d[key])
```

Section 19.10: Unpacking dictionaries using the `**` operator

You can use the `**` keyword argument unpacking operator to deliver the key-value pairs in a dictionary into a function's arguments. A simplified example from the [official documentation](#):

```
>>>
```



```
>>> def parrot(voltage, state, action):
...     print("This parrot wouldn't", action, end=' ')
...     print("if you put", voltage, "volts through it.", end=' ')
...     print("E's", state, "!")
...
>>> d = {"voltage": "four million", "state": "bleedin' demised", "action": "VOOM"}
>>> parrot(**d)

This parrot wouldn't VOOM if you put four million volts through it. E's bleedin' demised !
```

As of Python 3.5 you can also use this syntax to merge an arbitrary number of `dict` objects.

```
>>> fish = {'name': "Nemo", 'hands': "fins", 'special': "gills"}
>>> dog = {'name': "Clifford", 'hands': "paws", 'color': "red"}
>>> fishdog = {**fish, **dog}
>>> fishdog

{'hands': 'paws', 'color': 'red', 'name': 'Clifford', 'special': 'gills'}
```

As this example demonstrates, duplicate keys map to their lattermost value (for example "Clifford" overrides "Nemo").

Section 19.11: The trailing comma

Like lists and tuples, you can include a trailing comma in your dictionary.

```
role = {"By day": "A typical programmer",
        "By night": "Still a typical programmer", }
```

PEP 8 dictates that you should leave a space between the trailing comma and the closing brace.

Section 19.12: The dict() constructor

The `dict()` constructor can be used to create dictionaries from keyword arguments, or from a single iterable of key-value pairs, or from a single dictionary and keyword arguments.

```
dict(a=1, b=2, c=3)           # {'a': 1, 'b': 2, 'c': 3}
dict([('d', 4), ('e', 5), ('f', 6)]) # {'d': 4, 'e': 5, 'f': 6}
dict([('a', 1)], b=2, c=3)      # {'a': 1, 'b': 2, 'c': 3}
dict({'a': 1, 'b': 2}, c=3)     # {'a': 1, 'b': 2, 'c': 3}
```

Section 19.13: Dictionaries Example

Dictionaries map keys to values.

```
car = {}
car["wheels"] = 4
car["color"] = "Red"
car["model"] = "Corvette"
```

Dictionary values can be accessed by their keys.

```
print "Little " + car["color"] + " " + car["model"] + "!"
# This would print out "Little Red Corvette!"
```

Dictionaries can also be created in a JSON style:

```
car = {"wheels": 4, "color": "Red", "model": "Corvette"}
```

Dictionary values can be iterated over:

```
for key in car:
    print key + ": " + car[key]

# wheels: 4
# color: Red
# model: Corvette
```

Section 19.14: All combinations of dictionary values

```
options = {
    "x": ["a", "b"],
    "y": [10, 20, 30]
}
```

Given a dictionary such as the one shown above, where there is a list representing a set of values to explore for the corresponding key. Suppose you want to explore "x"="a" with "y"=10, then "x"="a" with "y"=20, and so on until you have explored all possible combinations.

You can create a list that returns all such combinations of values using the following code.

```
import itertools

options = {
    "x": ["a", "b"],
    "y": [10, 20, 30]}

keys = options.keys()
values = (options[key] for key in keys)
combinations = [dict(zip(keys, combination)) for combination in itertools.product(*values)]
print combinations
```

This gives us the following list stored in the variable combinations:

```
[{'x': 'a', 'y': 10},
 {'x': 'b', 'y': 10},
 {'x': 'a', 'y': 20},
 {'x': 'b', 'y': 20},
 {'x': 'a', 'y': 30},
 {'x': 'b', 'y': 30}]
```

VIDEO: Machine Learning, Data Science and Deep Learning with Python

Complete hands-on machine learning tutorial with data science, Tensorflow, artificial intelligence, and neural networks



- ✓ Build artificial neural networks with Tensorflow and Keras
- ✓ Classify images, data, and sentiments using deep learning
- ✓ Make predictions using linear regression, polynomial regression, and multivariate regression
- ✓ Data Visualization with Matplotlib and Seaborn
- ✓ Implement machine learning at massive scale with Apache Spark's MLlib
- ✓ Understand reinforcement learning - and how to build a Pac-Man bot
- ✓ Classify data using K-Means clustering, Support Vector Machines (SVM), KNN, Decision Trees, Naive Bayes, and PCA
- ✓ Use train/test and K-Fold cross validation to choose and tune your models
- ✓ Build a movie recommender system using item-based and user-based collaborative filtering

Watch Today →

PNP Ch 7 | Enum [companion]

Chapter 7: Enum

Section 7.1: Creating an enum (Python 2.4 through 3.3)

Enums have been backported from Python 3.4 to Python 2.4 through Python 3.3. You can get this the [enum34](#) backport from PyPI.

```
pip install enum34
```

Creation of an enum is identical to how it works in Python 3.4+

```
from enum import Enum

class Color(Enum):
    red = 1
    green = 2
    blue = 3

print(Color.red) # Color.red
print(Color(1)) # Color.red
print(Color['red']) # Color.red
```

Section 7.2: Iteration

Enums are iterable:

```
class Color(Enum):
    red = 1
    green = 2
    blue = 3

[c for c in Color] # [<Color.red: 1>, <Color.green: 2>, <Color.blue: 3>]
```

PNP Ch 47 | Collections Module [companion]

Chapter 47: Collections module

The built-in `collections` package provides several specialized, flexible collection types that are both high-performance and provide alternatives to the general collection types of `dict`, `list`, `tuple` and `set`. The module also defines abstract base classes describing different types of collection functionality (such as `MutableSet` and `ItemsView`).

Section 47.1: collections.Counter

`Counter` is a dict sub class that allows you to easily count objects. It has utility methods for working with the frequencies of the objects that you are counting.

```
import collections
counts = collections.Counter([1,2,3])
```

the above code creates an object, counts, which has the frequencies of all the elements passed to the constructor. This example has the value `Counter({1: 1, 2: 1, 3: 1})`

Constructor examples

Letter Counter

```
>>> collections.Counter('Happy Birthday')
Counter({'a': 2, 'p': 2, 'y': 2, 'i': 1, 'r': 1, 'B': 1, ' ': 1, 'H': 1, 'd': 1, 'h': 1, 't': 1})
```

Word Counter

```
>>> collections.Counter('I am Sam Sam I am That Sam-I-am That Sam-I-am! I do not like that Sam-I-am'.split())
Counter({'I': 3, 'Sam': 2, 'Sam-I-am': 2, 'That': 2, 'am': 2, 'do': 1, 'Sam-I-am!': 1, 'that': 1, 'not': 1, 'like': 1})
```

Recipes

```
>>> c = collections.Counter({'a': 4, 'b': 2, 'c': -2, 'd': 0})
```

Get count of individual element

```
>>> c['a']
4
```

Set count of individual element

```
>>> c['c'] = -3
>>> c
Counter({'a': 4, 'b': 2, 'd': 0, 'c': -3})
```

Get total number of elements in counter (4 + 2 + 0 - 3)

```
>>> sum(c.itervalues()) # negative numbers are counted!
3
```

Get elements (only those with positive counter are kept)

```
>>> list(c.elements())
['a', 'a', 'a', 'a', 'b', 'b']
```

Remove keys with 0 or negative value

```
>>> c = collections.Counter()
Counter({'a': 4, 'b': 2})
```

Remove everything

```
>>> c.clear()
>>> c
Counter()
```

Add remove individual elements

```
>>> c.update({'a': 3, 'b': 3})
>>> c.update({'a': 2, 'c': 2}) # adds to existing, sets if they don't exist
>>> c
Counter({'a': 5, 'b': 3, 'c': 2})
>>> c.subtract({'a': 3, 'b': 3, 'c': 3}) # subtracts (negative values are allowed)
>>> c
Counter({'a': 2, 'b': 0, 'c': -1})
```

Section 47.2: collections.OrderedDict

The order of keys in Python dictionaries is arbitrary: they are not governed by the order in which you add them.

For example:

```
>>> d = {'foo': 5, 'bar': 6}
>>> print(d)
{'foo': 5, 'bar': 6}
>>> d['baz'] = 7
>>> print(d)
{'baz': 7, 'foo': 5, 'bar': 6}
>>> d['foobar'] = 8
>>> print(d)
{'baz': 7, 'foo': 5, 'bar': 6, 'foobar': 8}
...

```

(The arbitrary ordering implied above means that you may get different results with the above code to that shown here.)

The order in which the keys appear is the order which they would be iterated over, e.g. using a **for** loop.

The `collections.OrderedDict` class provides dictionary objects that retain the order of keys. `OrderedDicts` can be created as shown below with a series of ordered items (here, a list of tuple key-value pairs):

```
>>> from collections import OrderedDict
>>> d = OrderedDict([('foo', 5), ('bar', 6)])
>>> print(d)
OrderedDict([('foo', 5), ('bar', 6)])
>>> d['baz'] = 7
>>> print(d)
OrderedDict([('foo', 5), ('bar', 6), ('baz', 7)])
>>> d['foobar'] = 8
```



```
>>> print(d)
OrderedDict([('foo', 5), ('bar', 6), ('baz', 7), ('foobar', 8)])
```

Or we can create an empty `OrderedDict` and then add items:

```
>>> o = OrderedDict()
>>> o['key1'] = "value1"
>>> o['key2'] = "value2"
>>> print(o)
OrderedDict([('key1', 'value1'), ('key2', 'value2')])
```

Iterating through an `OrderedDict` allows key access in the order they were added.

What happens if we assign a new value to an existing key?

```
>>> d['foo'] = 4
>>> print(d)
OrderedDict([('foo', 4), ('bar', 6), ('baz', 7), ('foobar', 8)])
```

The key retains its original place in the `OrderedDict`.

Section 47.3: collections.defaultdict

[`collections.defaultdict`](#)(`default_factory`) returns a subclass of `dict` that has a default value for missing keys. The argument should be a function that returns the default value when called with no arguments. If there is nothing passed, it defaults to `None`.

```
>>> state_capitals = collections.defaultdict(str)
>>> state_capitals
defaultdict(<class 'str'>, {})
```

returns a reference to a `defaultdict` that will create a string object with its `default_factory` method.

A typical usage of `defaultdict` is to use one of the builtin types such as `str`, `int`, `list` or `dict` as the `default_factory`, since these return empty types when called with no arguments:

```
>>> str()
''
>>> int()
0
>>> list
[]
```

Calling the `defaultdict` with a key that does not exist does not produce an error as it would in a normal dictionary.

```
>>> state_capitals['Alaska']
''
>>> state_capitals
defaultdict(<class 'str'>, {'Alaska': ''})
```

Another example with `int`:

```
>>> fruit_counts = defaultdict(int)
>>> fruit_counts['apple'] += 2 # No errors should occur
>>> fruit_counts
defaultdict(int, {'apple': 2})
>>> fruit_counts['banana'] # No errors should occur
```

```
0
>>> fruit_counts # A new key is created
defaultdict(int, {'apple': 2, 'banana': 0})
```

Normal dictionary methods work with the default dictionary

```
>>> state_capitals['Alabama'] = 'Montgomery'
>>> state_capitals
defaultdict(<class 'str'>, {'Alabama': 'Montgomery', 'Alaska': ''})
```

Using `list` as the `default_factory` will create a list for each new key.

```
>>> s = [('NC', 'Raleigh'), ('VA', 'Richmond'), ('WA', 'Seattle'), ('NC', 'Asheville')]
>>> dd = collections.defaultdict(list)
>>> for k, v in s:
...     dd[k].append(v)
>>> dd
defaultdict(<class 'list'>,
            {'VA': ['Richmond'],
             'NC': ['Raleigh', 'Asheville'],
             'WA': ['Seattle']})
```

Section 47.4: collections.namedtuple

Define a new type `Person` using [namedtuple](#) like this:

```
Person = namedtuple('Person', ['age', 'height', 'name'])
```

The second argument is the list of attributes that the tuple will have. You can list these attributes also as either space or comma separated string:

```
Person = namedtuple('Person', 'age, height, name')
```

or

```
Person = namedtuple('Person', 'age height name')
```

Once defined, a named tuple can be instantiated by calling the object with the necessary parameters, e.g.:

```
dave = Person(30, 178, 'Dave')
```

Named arguments can also be used:

```
jack = Person(age=30, height=178, name='Jack S.')
```

Now you can access the attributes of the `namedtuple`:

```
print(jack.age) # 30
print(jack.name) # 'Jack S.'
```

The first argument to the `namedtuple` constructor (in our example `'Person'`) is the typename. It is typical to use the same word for the constructor and the typename, but they can be different:

```
Human = namedtuple('Person', 'age, height, name')
dave = Human(30, 178, 'Dave')
```

```
print(dave) # yields: Person(age=30, height=178, name='Dave')
```

Section 47.5: collections.deque

Returns a new deque object initialized left-to-right (using `append()`) with data from iterable. If iterable is not specified, the new deque is empty.

Deques are a generalization of stacks and queues (the name is pronounced “deck” and is short for “double-ended queue”). Deques support thread-safe, memory efficient appends and pops from either side of the deque with approximately the same $O(1)$ performance in either direction.

Though list objects support similar operations, they are optimized for fast fixed-length operations and incur $O(n)$ memory movement costs for `pop(0)` and `insert(0, v)` operations which change both the size and position of the underlying data representation.

New in version 2.4.

If `maxlen` is not specified or is `None`, deques may grow to an arbitrary length. Otherwise, the deque is bounded to the specified maximum length. Once a bounded length deque is full, when new items are added, a corresponding number of items are discarded from the opposite end. Bounded length deques provide functionality similar to the tail filter in Unix. They are also useful for tracking transactions and other pools of data where only the most recent activity is of interest.

Changed in version 2.6: Added `maxlen` parameter.

```
>>> from collections import deque
>>> d = deque('ghi')           # make a new deque with three items
>>> for elem in d:             # iterate over the deque's elements
...     print elem.upper()
G
H
I

>>> d.append('j')              # add a new entry to the right side
>>> d.appendleft('f')          # add a new entry to the left side
>>> d                          # show the representation of the deque
deque(['f', 'g', 'h', 'i', 'j'])

>>> d.pop()                    # return and remove the rightmost item
'j'
>>> d.popleft()                # return and remove the leftmost item
'f'
>>> list(d)                    # list the contents of the deque
['g', 'h', 'i']
>>> d[0]                       # peek at leftmost item
'g'
>>> d[-1]                      # peek at rightmost item
'i'

>>> list(reversed(d))          # list the contents of a deque in reverse
['i', 'h', 'g']
>>> 'h' in d                   # search the deque
True
>>> d.extend('jkl')            # add multiple elements at once
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])
>>> d.rotate(1)                # right rotation
>>> d
```

```

deque(['l', 'g', 'h', 'i', 'j', 'k'])
>>> d.rotate(-1)                # left rotation
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])

>>> deque(reversed(d))           # make a new deque in reverse order
deque(['l', 'k', 'j', 'i', 'h', 'g'])
>>> d.clear()                    # empty the deque
>>> d.pop()                      # cannot pop from an empty deque
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    d.pop()
IndexError: pop from an empty deque

>>> d.extendleft('abc')          # extendleft() reverses the input order
>>> d
deque(['c', 'b', 'a'])

```

Source: <https://docs.python.org/2/library/collections.html>

Section 47.6: collections.ChainMap

ChainMap is new in **version 3.3**

Returns a new ChainMap object given a number of maps. This object groups multiple dicts or other mappings together to create a single, updateable view.

ChainMaps are useful managing nested contexts and overlays. An example in the python world is found in the implementation of the Context class in Django's template engine. It is useful for quickly linking a number of mappings so that the result can be treated as a single unit. It is often much faster than creating a new dictionary and running multiple update() calls.

Anytime one has a chain of lookup values there can be a case for ChainMap. An example includes having both user specified values and a dictionary of default values. Another example is the POST and GET parameter maps found in web use, e.g. Django or Flask. Through the use of ChainMap one returns a combined view of two distinct dictionaries.

The maps parameter list is ordered from first-searched to last-searched. Lookups search the underlying mappings successively until a key is found. In contrast, writes, updates, and deletions only operate on the first mapping.

```

import collections

# define two dictionaries with at least some keys overlapping.
dict1 = {'apple': 1, 'banana': 2}
dict2 = {'coconut': 1, 'date': 1, 'apple': 3}

# create two ChainMaps with different ordering of those dicts.
combined_dict = collections.ChainMap(dict1, dict2)
reverse_ordered_dict = collections.ChainMap(dict2, dict1)

```

Note the impact of order on which value is found first in the subsequent lookup

```

for k, v in combined_dict.items():
    print(k, v)

date 1
apple 1
banana 2

```

```
coconut 1

for k, v in reverse_ordered_dict.items():
    print(k, v)

date 1
apple 3
banana 2
coconut 1
```

PNP Ch 49 | JSON Module [companion]

Chapter 49: JSON Module

Section 49.1: Storing data in a file

The following snippet encodes the data stored in `d` into JSON and stores it in a file (replace `filename` with the actual name of the file).

```
import json

d = {
    'foo': 'bar',
    'alice': 1,
    'wonderland': [1, 2, 3]
}

with open(filename, 'w') as f:
    json.dump(d, f)
```

Section 49.2: Retrieving data from a file

The following snippet opens a JSON encoded file (replace `filename` with the actual name of the file) and returns the object that is stored in the file.

```
import json

with open(filename, 'r') as f:
    d = json.load(f)
```

Section 49.3: Formatting JSON output

Let's say we have the following data:

```
>>> data = {"cats": [{"name": "Tubbs", "color": "white"}, {"name": "Pepper", "color": "black"}]}
```

Just dumping this as JSON does not do anything special here:

```
>>> print(json.dumps(data))
{"cats": [{"name": "Tubbs", "color": "white"}, {"name": "Pepper", "color": "black"}]}
```

Setting indentation to get prettier output

If we want pretty printing, we can set an indent size:

```
>>> print(json.dumps(data, indent=2))
{
  "cats": [
    {
      "name": "Tubbs",
      "color": "white"
    },
    {
      "name": "Pepper",
      "color": "black"
    }
  ]
}
```

Sorting keys alphabetically to get consistent output

By default the order of keys in the output is undefined. We can get them in alphabetical order to make sure we always get the same output:

```
>>> print(json.dumps(data, sort_keys=True))
{"cats": [{"color": "white", "name": "Tubbs"}, {"color": "black", "name": "Pepper"}]}
```

Getting rid of whitespace to get compact output

We might want to get rid of the unnecessary spaces, which is done by setting separator strings different from the default `,` and `:` :

```
>>> print(json.dumps(data, separators=(',', ':')))
{"cats":[{"name":"Tubbs","color":"white"}, {"name":"Pepper","color":"black"}]}
```

Section 49.4: `load` vs `loads`, `dump` vs `dumps`

The `json` module contains functions for both reading and writing to and from unicode strings, and reading and writing to and from files. These are differentiated by a trailing `s` in the function name. In these examples we use a `StringIO` object, but the same functions would apply for any file-like object.

Here we use the string-based functions:

```
import json

data = {"foo": "bar", "baz": []}
json_string = json.dumps(data)
# u'{"foo": "bar", "baz": []}'
json.loads(json_string)
# {"foo": "bar", "baz": []}
```

And here we use the file-based functions:

```
import json

from io import StringIO

json_file = StringIO()
data = {"foo": "bar", "baz": []}
json.dump(data, json_file)
json_file.seek(0) # Seek back to the start of the file before reading
json_file_content = json_file.read()
# u'{"foo": "bar", "baz": []}'
json_file.seek(0) # Seek back to the start of the file before reading
json.load(json_file)
# {"foo": "bar", "baz": []}
```

As you can see the main difference is that when dumping json data you must pass the file handle to the function, as opposed to capturing the return value. Also worth noting is that you must seek to the start of the file before reading or writing, in order to avoid data corruption. When opening a file the cursor is placed at position 0, so the below would also work:

```
import json

json_file_path = './data.json'
data = {"foo": "bar", "baz": []}
```



```

with open(json_file_path, 'w') as json_file:
    json.dump(data, json_file)

with open(json_file_path) as json_file:
    json_file_content = json_file.read()
    # u'{"foo": "bar", "baz": []}'

with open(json_file_path) as json_file:
    json.load(json_file)
    # {"foo": "bar", "baz": []}

```

Having both ways of dealing with json data allows you to idiomatically and efficiently work with formats which build upon json, such as pyspark's json-per-line:

```

# loading from a file
data = [json.loads(line) for line in open(file_path).splitlines()]

# dumping to a file
with open(file_path, 'w') as json_file:
    for item in data:
        json.dump(item, json_file)
        json_file.write('\n')

```

Section 49.5: Calling `json.tool` from the command line to pretty-print JSON output

Given some JSON file "foo.json" like:

```

{"foo": {"bar": {"baz": 1}}}

```

we can call the module directly from the command line (passing the filename as an argument) to pretty-print it:

```

$ python -m json.tool foo.json
{
  "foo": {
    "bar": {
      "baz": 1
    }
  }
}

```

The module will also take input from STDOUT, so (in Bash) we equally could do:

```

$ cat foo.json | python -m json.tool

```

Section 49.6: JSON encoding custom objects

If we just try the following:

```

import json
from datetime import datetime
data = {'datetime': datetime(2016, 9, 26, 4, 44, 0)}
print(json.dumps(data))

```

we get an error saying `TypeError: datetime.datetime(2016, 9, 26, 4, 44) is not JSON serializable`.

To be able to serialize the datetime object properly, we need to write custom code for how to convert it:

```
class DatetimeJSONEncoder(json.JSONEncoder):
    def default(self, obj):
        try:
            return obj.isoformat()
        except AttributeError:
            # obj has no isoformat method; let the builtin JSON encoder handle it
            return super(DatetimeJSONEncoder, self).default(obj)
```

and then use this encoder class instead of `json.dumps`:

```
encoder = DatetimeJSONEncoder()
print(encoder.encode(data))
# prints {"datetime": "2016-09-26T04:44:00"}
```

Section 49.7: Creating JSON from Python dict

```
import json
d = {
    'foo': 'bar',
    'alice': 1,
    'wonderland': [1, 2, 3]
}
json.dumps(d)
```

The above snippet will return the following:

```
'{"wonderland": [1, 2, 3], "foo": "bar", "alice": 1}'
```

Section 49.8: Creating Python dict from JSON

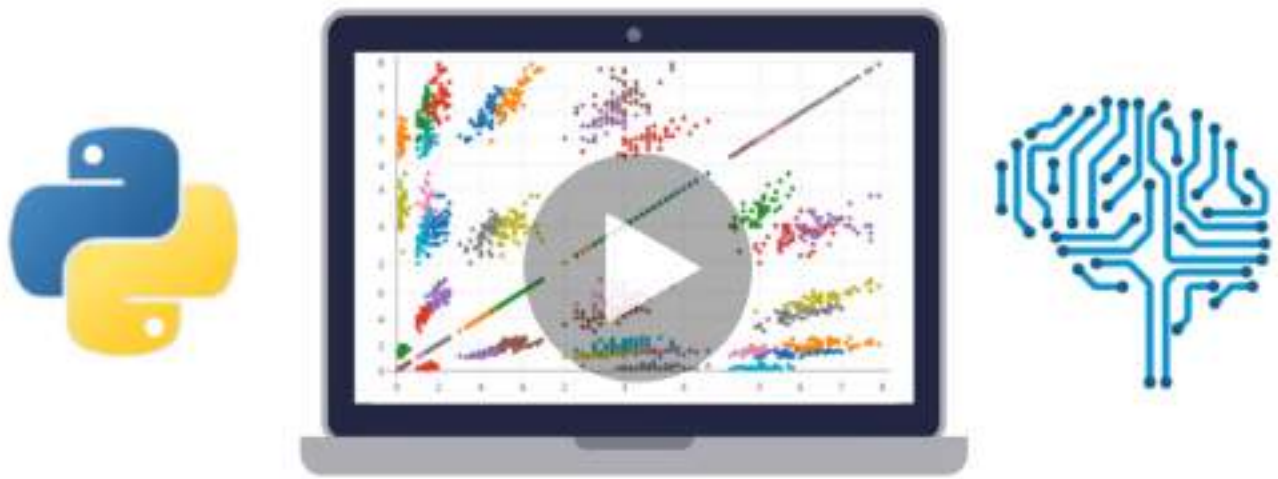
```
import json
s = '{"wonderland": [1, 2, 3], "foo": "bar", "alice": 1}'
json.loads(s)
```

The above snippet will return the following:

```
{u'alice': 1, u'foo': u'bar', u'wonderland': [1, 2, 3]}
```

VIDEO: Python for Data Science and Machine Learning Bootcamp

Learn how to use NumPy, Pandas, Seaborn, Matplotlib , Plotly, Scikit-Learn , Machine Learning, Tensorflow, and more!



- ✓ Use Python for Data Science and Machine Learning
- ✓ Use Spark for Big Data Analysis
- ✓ Implement Machine Learning Algorithms
- ✓ Learn to use NumPy for Numerical Data
- ✓ Learn to use Pandas for Data Analysis
- ✓ Learn to use Matplotlib for Python Plotting
- ✓ Learn to use Seaborn for statistical plots
- ✓ Use Plotly for interactive dynamic visualizations
- ✓ Use SciKit-Learn for Machine Learning Tasks
- ✓ K-Means Clustering
- ✓ Logistic Regression
- ✓ Linear Regression
- ✓ Random Forest and Decision Trees
- ✓ Neural Networks
- ✓ Support Vector Machines

Watch Today →

PHASE 03 · Functions, Types & Decorators

~2 weeks · 3 chapters

Understand the core pattern every ML framework is built on. Write type-safe code.

Chapters in this phase:

- FP Ch 5 | First-Class Functions [= plan Ch 7]
- FP Ch 7 | Function Decorators & Closures [= plan Ch 9]
- PNP Ch 37 | Decorators [companion]
- PNP Ch 43 | Importing Modules & importlib [companion]
- PNP Ch 44 | Modules vs Packages [companion]
- PNP Ch 56 | Functools Module [companion]
- PNP Ch 87 | Type Hints [companion · FP Ch 8 substitute]

FP Ch 5 | First-Class Functions [= plan Ch 7]

First-Class Functions

I have never considered Python to be heavily influenced by functional languages, no matter what people say or think. I was much more familiar with imperative languages such as C and Algol 68 and although I had made functions first-class objects, I didn't view Python as a functional programming language.¹

— Guido van Rossum
Python BDFL

Functions in Python are first-class objects. Programming language theorists define a “first-class object” as a program entity that can be:

- Created at runtime
- Assigned to a variable or element in a data structure
- Passed as an argument to a function
- Returned as the result of a function

Integers, strings, and dictionaries are other examples of first-class objects in Python—nothing fancy here. But if you came to Python from a language where functions are not first-class citizens, this chapter and the rest of Part III of the book focuses on the implications and practical applications of treating functions as objects.



The term “first-class functions” is widely used as shorthand for “functions as first-class objects.” It's not perfect because it seems to imply an “elite” among functions. In Python, all functions are first-class.

1. “Origins of Python's *Functional Features*”, from Guido's The History of Python blog.

Treating a Function Like an Object

The console session in [Example 5-1](#) shows that Python functions are objects. Here we create a function, call it, read its `__doc__` attribute, and check that the function object itself is an instance of the function class.

Example 5-1. Create and test a function, then read its `__doc__` and check its type

```
>>> def factorial(n): ❶
...     '''returns n!'''
...     return 1 if n < 2 else n * factorial(n-1)
...
>>> factorial(42)
1405006117752879898543142606244511569936384000000000
>>> factorial.__doc__ ❷
'returns n!'
>>> type(factorial) ❸
<class 'function'>
```

- ❶ This is a console session, so we're creating a function in “runtime.”
- ❷ `__doc__` is one of several attributes of function objects.
- ❸ `factorial` is an instance of the function class.

The `__doc__` attribute is used to generate the help text of an object. In the Python interactive console, the command `help(factorial)` will display a screen like that in [Figure 5-1](#).

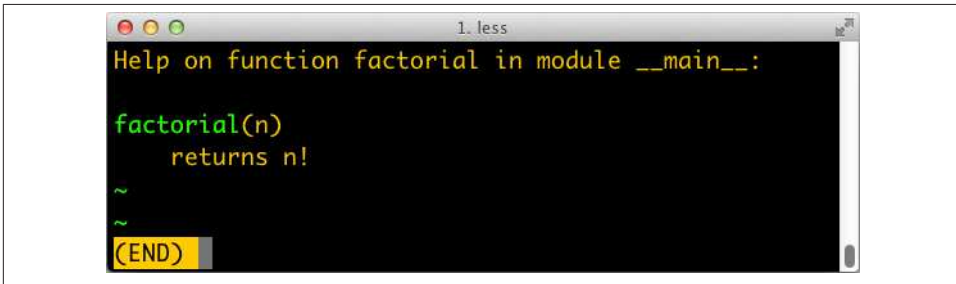


Figure 5-1. Help screen for the `factorial` function; the text is from the `__doc__` attribute of the function object

[Example 5-2](#) shows the “first class” nature of a function object. We can assign it a variable `fact` and call it through that name. We can also pass `factorial` as an argument to `map`. The `map` function returns an iterable where each item is the result of the application of the first argument (a function) to successive elements of the second argument (an iterable), `range(10)` in this example.

Example 5-2. Use function through a different name, and pass function as argument

```
>>> fact = factorial
>>> fact
<function factorial at 0x...>
>>> fact(5)
120
>>> map(factorial, range(11))
<map object at 0x...>
>>> list(map(fact, range(11)))
[1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800]
```

Having first-class functions enables programming in a functional style. One of the hallmarks of functional programming is the use of higher-order functions, our next topic.

Higher-Order Functions

A function that takes a function as argument or returns a function as the result is a *higher-order function*. One example is `map`, shown in [Example 5-2](#). Another is the built-in function `sorted`: an optional `key` argument lets you provide a function to be applied to each item for sorting, as seen in “[list.sort and the sorted Built-In Function](#)” on page 42.

For example, to sort a list of words by length, simply pass the `len` function as the key, as in [Example 5-3](#).

Example 5-3. Sorting a list of words by length

```
>>> fruits = ['strawberry', 'fig', 'apple', 'cherry', 'raspberry', 'banana']
>>> sorted(fruits, key=len)
['fig', 'apple', 'cherry', 'banana', 'raspberry', 'strawberry']
>>>
```

Any one-argument function can be used as the key. For example, to create a rhyme dictionary it might be useful to sort each word spelled backward. In [Example 5-4](#), note that the words in the list are not changed at all; only their reversed spelling is used as the sort criterion, so that the berries appear together.

Example 5-4. Sorting a list of words by their reversed spelling

```
>>> def reverse(word):
...     return word[::-1]
>>> reverse('testing')
'gnitset'
>>> sorted(fruits, key=reverse)
['banana', 'apple', 'fig', 'raspberry', 'strawberry', 'cherry']
>>>
```

In the functional programming paradigm, some of the best known higher-order functions are `map`, `filter`, `reduce`, and `apply`. The `apply` function was deprecated in Python 2.3 and removed in Python 3 because it's no longer necessary. If you need to call a

function with a dynamic set of arguments, you can just write `fn(*args, **key words)` instead of `apply(fn, args, kwargs)`.

The `map`, `filter`, and `reduce` higher-order functions are still around, but better alternatives are available for most of their use cases, as the next section shows.

Modern Replacements for `map`, `filter`, and `reduce`

Functional languages commonly offer the `map`, `filter`, and `reduce` higher-order functions (sometimes with different names). The `map` and `filter` functions are still built-ins in Python 3, but since the introduction of list comprehensions and generator expressions, they are not as important. A listcomp or a genexp does the job of `map` and `filter` combined, but is more readable. Consider [Example 5-5](#).

Example 5-5. Lists of factorials produced with `map` and `filter` compared to alternatives coded as list comprehensions

```
>>> list(map(fact, range(6))) ❶
[1, 1, 2, 6, 24, 120]
>>> [fact(n) for n in range(6)] ❷
[1, 1, 2, 6, 24, 120]
>>> list(map(factorial, filter(lambda n: n % 2, range(6)))) ❸
[1, 6, 120]
>>> [factorial(n) for n in range(6) if n % 2] ❹
[1, 6, 120]
>>>
```

- ❶ Build a list of factorials from 0! to 5!.
- ❷ Same operation, with a list comprehension.
- ❸ List of factorials of odd numbers up to 5!, using both `map` and `filter`.
- ❹ List comprehension does the same job, replacing `map` and `filter`, and making `lambda` unnecessary.

In Python 3, `map` and `filter` return generators—a form of iterator—so their direct substitute is now a generator expression (in Python 2, these functions returned lists, therefore their closest alternative is a listcomp).

The `reduce` function was demoted from a built-in in Python 2 to the `functools` module in Python 3. Its most common use case, summation, is better served by the `sum` built-in available since Python 2.3 was released in 2003. This is a big win in terms of readability and performance (see [Example 5-6](#)).

Example 5-6. Sum of integers up to 99 performed with `reduce` and `sum`

```
>>> from functools import reduce ❶
>>> from operator import add ❷
>>> reduce(add, range(100)) ❸
```

```
4950
>>> sum(range(100)) ❹
4950
>>>
```

- ❶ Starting with Python 3.0, `reduce` is not a built-in.
- ❷ Import `add` to avoid creating a function just to add two numbers.
- ❸ Sum integers up to 99.
- ❹ Same task using `sum`; import or adding function not needed.

The common idea of `sum` and `reduce` is to apply some operation to successive items in a sequence, accumulating previous results, thus reducing a sequence of values to a single value.

Other reducing built-ins are `all` and `any`:

`all(iterable)`

Returns `True` if every element of the `iterable` is `truthy`; `all([])` returns `True`.

`any(iterable)`

Returns `True` if any element of the `iterable` is `truthy`; `any([])` returns `False`.

I give a fuller explanation of `reduce` in “[Vector Take #4: Hashing and a Faster ==](#)” on [page 288](#) where an ongoing example provides a meaningful context for the use of this function. The reducing functions are summarized later in the book when iterables are in focus, in “[Iterable Reducing Functions](#)” on [page 434](#).

To use a higher-order function, sometimes it is convenient to create a small, one-off function. That is why anonymous functions exist. We’ll cover them next.

Anonymous Functions

The `lambda` keyword creates an anonymous function within a Python expression.

However, the simple syntax of Python limits the body of `lambda` functions to be pure expressions. In other words, the body of a `lambda` cannot make assignments or use any other Python statement such as `while`, `try`, etc.

The best use of anonymous functions is in the context of an argument list. For example, [Example 5-7](#) is the rhyme index example from [Example 5-4](#) rewritten with `lambda`, without defining a `reverse` function.

Example 5-7. Sorting a list of words by their reversed spelling using `lambda`

```
>>> fruits = ['strawberry', 'fig', 'apple', 'cherry', 'raspberry', 'banana']
>>> sorted(fruits, key=lambda word: word[::-1])
```

```
['banana', 'apple', 'fig', 'raspberry', 'strawberry', 'cherry']  
>>>
```

Outside the limited context of arguments to higher-order functions, anonymous functions are rarely useful in Python. The syntactic restrictions tend to make nontrivial lambdas either unreadable or unworkable.

Lundh's lambda Refactoring Recipe

If you find a piece of code hard to understand because of a `lambda`, Fredrik Lundh suggests this refactoring procedure:

1. Write a comment explaining what the heck that `lambda` does.
2. Study the comment for a while, and think of a name that captures the essence of the comment.
3. Convert the `lambda` to a `def` statement, using that name.
4. Remove the comment.

These steps are quoted from the [Functional Programming HOWTO](#), a must read.

The `lambda` syntax is just syntactic sugar: a `lambda` expression creates a function object just like the `def` statement. That is just one of several kinds of callable objects in Python. The following section reviews all of them.

The Seven Flavors of Callable Objects

The call operator (i.e., `()`) may be applied to other objects beyond user-defined functions. To determine whether an object is callable, use the `callable()` built-in function. The Python Data Model documentation lists seven callable types:

User-defined functions

Created with `def` statements or `lambda` expressions.

Built-in functions

A function implemented in C (for CPython), like `len` or `time.strftime`.

Built-in methods

Methods implemented in C, like `dict.get`.

Methods

Functions defined in the body of a class.

Classes

When invoked, a class runs its `__new__` method to create an instance, then `__init__` to initialize it, and finally the instance is returned to the caller. Because there is no new operator in Python, calling a class is like calling a function. (Usually calling a class creates an instance of the same class, but other behaviors are possible by overriding `__new__`. We'll see an example of this in “Flexible Object Creation with `__new__`” on page 592.)

Class instances

If a class defines a `__call__` method, then its instances may be invoked as functions. See “User-Defined Callable Types” on page 145.

Generator functions

Functions or methods that use the `yield` keyword. When called, generator functions return a generator object.

Generator functions are unlike other callables in many respects. Chapter 14 is devoted to them. They can also be used as coroutines, which are covered in Chapter 16.



Given the variety of existing callable types in Python, the safest way to determine whether an object is callable is to use the `callable()` built-in:

```
>>> abs, str, 13
(<built-in function abs>, <class 'str'>, 13)
>>> [callable(obj) for obj in (abs, str, 13)]
[True, True, False]
```

We now move on to building class instances that work as callable objects.

User-Defined Callable Types

Not only are Python functions real objects, but arbitrary Python objects may also be made to behave like functions. Implementing a `__call__` instance method is all it takes.

Example 5-8 implements a `BingoCage` class. An instance is built from any iterable, and stores an internal list of items, in random order. Calling the instance pops an item.

Example 5-8. *bingocall.py*: A `BingoCage` does one thing: picks items from a shuffled list

```
import random
```

```
class BingoCage:
```

```
    def __init__(self, items):
        self._items = list(items)  ❶
        random.shuffle(self._items)  ❷
```

```

def pick(self): ❸
    try:
        return self._items.pop()
    except IndexError:
        raise LookupError('pick from empty BingoCage') ❹

def __call__(self): ❺
    return self.pick()

```

- ❶ `__init__` accepts any iterable; building a local copy prevents unexpected side effects on any list passed as an argument.
- ❷ `shuffle` is guaranteed to work because `self._items` is a list.
- ❸ The main method.
- ❹ Raise exception with custom message if `self._items` is empty.
- ❺ Shortcut to `bingo.pick()`: `bingo()`.

Here is a simple demo of [Example 5-8](#). Note how a `bingo` instance can be invoked as a function, and the `callable(...)` built-in recognizes it as a callable object:

```

>>> bingo = BingoCage(range(3))
>>> bingo.pick()
1
>>> bingo()
0
>>> callable(bingo)
True

```

A class implementing `__call__` is an easy way to create function-like objects that have some internal state that must be kept across invocations, like the remaining items in the `BingoCage`. An example is a decorator. Decorators must be functions, but it is sometimes convenient to be able to “remember” something between calls of the decorator (e.g., for memoization—caching the results of expensive computations for later use).

A totally different approach to creating functions with internal state is to use closures. Closures, as well as decorators, are the subject of [Chapter 7](#).

We now move on to another aspect of handling functions as objects: runtime introspection.

Function Introspection

Function objects have many attributes beyond `__doc__`. See what the `dir` function reveals about our `factorial`:

```

>>> dir(factorial)
['__annotations__', '__call__', '__class__', '__closure__', '__code__',
 '__defaults__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',

```

```

['__format__', '__ge__', '__get__', '__getattr__', '__globals__',
 '__gt__', '__hash__', '__init__', '__kwdefaults__', '__le__', '__lt__',
 '__module__', '__name__', '__ne__', '__new__', '__qualname__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
 '__subclasshook__']
>>>

```

Most of these attributes are common to Python objects in general. In this section, we cover those that are especially relevant to treating functions as objects, starting with `__dict__`.

Like the instances of a plain user-defined class, a function uses the `__dict__` attribute to store user attributes assigned to it. This is useful as a primitive form of annotation. Assigning arbitrary attributes to functions is not a very common practice in general, but Django is one framework that uses it. See, for example, the `short_description`, `boolean`, and `allow_tags` attributes described in [The Django admin site](#) documentation. In the Django docs, this example shows attaching a `short_description` to a method, to determine the description that will appear in record listings in the Django admin when that method is used:

```

def upper_case_name(obj):
    return ("%s %s" % (obj.first_name, obj.last_name)).upper()
upper_case_name.short_description = 'Customer name'

```

Now let us focus on the attributes that are specific to functions and are not found in a generic Python user-defined object. Computing the difference of two sets quickly gives us a list of the function-specific attributes (see [Example 5-9](#)).

Example 5-9. Listing attributes of functions that don't exist in plain instances

```

>>> class C: pass # ❶
>>> obj = C() # ❷
>>> def func(): pass # ❸
>>> sorted(set(dir(func)) - set(dir(obj))) # ❹
['__annotations__', '__call__', '__closure__', '__code__', '__defaults__',
 '__get__', '__globals__', '__kwdefaults__', '__name__', '__qualname__']
>>>

```

- ❶ Create bare user-defined class.
- ❷ Make an instance of it.
- ❸ Create a bare function.
- ❹ Using set difference, generate a sorted list of the attributes that exist in a function but not in an instance of a bare class.

[Table 5-1](#) shows a summary of the attributes listed by [Example 5-9](#).

Table 5-1. Attributes of user-defined functions

Name	Type	Description
<code>__annotations__</code>	dict	Parameter and return annotations
<code>__call__</code>	method-wrapper	Implementation of the <code>()</code> operator; a.k.a. the callable object protocol
<code>__closure__</code>	tuple	The function closure, i.e., bindings for free variables (often is <code>None</code>)
<code>__code__</code>	code	Function metadata and function body compiled into bytecode
<code>__defaults__</code>	tuple	Default values for the formal parameters
<code>__get__</code>	method-wrapper	Implementation of the read-only descriptor protocol (see Chapter 20)
<code>__globals__</code>	dict	Global variables of the module where the function is defined
<code>__kwdefaults__</code>	dict	Default values for the keyword-only formal parameters
<code>__name__</code>	str	The function name
<code>__qualname__</code>	str	The qualified function name, e.g., <code>Random.choice</code> (see PEP-3155)

We will discuss the `__defaults__`, `__code__`, and `__annotations__` functions, used by IDEs and frameworks to extract information about function signatures, in later sections. But to fully appreciate these attributes, we will make a detour to explore the powerful syntax Python offers to declare function parameters and to pass arguments into them.

From Positional to Keyword-Only Parameters

One of the best features of Python functions is the extremely flexible parameter handling mechanism, enhanced with keyword-only arguments in Python 3. Closely related are the use of `*` and `**` to “explode” iterables and mappings into separate arguments when we call a function. To see these features in action, see the code for [Example 5-10](#) and tests showing its use in [Example 5-11](#).

Example 5-10. `tag` generates HTML; a keyword-only argument `cls` is used to pass “class” attributes as a workaround because `class` is a keyword in Python

```
def tag(name, *content, cls=None, **attrs):
    """Generate one or more HTML tags"""
    if cls is not None:
        attrs['class'] = cls
    if attrs:
        attr_str = ''.join(' %s="%s"' % (attr, value)
                           for attr, value
                           in sorted(attrs.items()))
    else:
        attr_str = ''
    if content:
        return '\n'.join('<%s%s>%s</%s>' %
                        (name, attr_str, c, name) for c in content)
    else:
        return '<%s%s />' % (name, attr_str)
```

The tag function can be invoked in many ways, as [Example 5-11](#) shows.

Example 5-11. Some of the many ways of calling the tag function from [Example 5-10](#)

```
>>> tag('br') ❶
'<br />'
>>> tag('p', 'hello') ❷
'<p>hello</p>'
>>> print(tag('p', 'hello', 'world'))
<p>hello</p>
<p>world</p>
>>> tag('p', 'hello', id=33) ❸
'<p id="33">hello</p>'
>>> print(tag('p', 'hello', 'world', cls='sidebar')) ❹
<p class="sidebar">hello</p>
<p class="sidebar">world</p>
>>> tag(content='testing', name='img') ❺
'<img content="testing" />'
>>> my_tag = {'name': 'img', 'title': 'Sunset Boulevard',
...           'src': 'sunset.jpg', 'cls': 'framed'}
>>> tag(**my_tag) ❻
''
```

- ❶ A single positional argument produces an empty tag with that name.
- ❷ Any number of arguments after the first are captured by `*content` as a tuple.
- ❸ Keyword arguments not explicitly named in the tag signature are captured by `**attrs` as a dict.
- ❹ The `cls` parameter can only be passed as a keyword argument.
- ❺ Even the first positional argument can be passed as a keyword when tag is called.
- ❻ Prefixing the `my_tag` dict with `**` passes all its items as separate arguments, which are then bound to the named parameters, with the remaining caught by `**attrs`.

Keyword-only arguments are a new feature in Python 3. In [Example 5-10](#), the `cls` parameter can only be given as a keyword argument—it will never capture unnamed positional arguments. To specify keyword-only arguments when defining a function, name them after the argument prefixed with `*`. If you don't want to support variable positional arguments but still want keyword-only arguments, put a `*` by itself in the signature, like this:

```
>>> def f(a, *, b):
...     return a, b
...
>>> f(1, b=2)
(1, 2)
```


Note that keyword-only arguments do not need to have a default value: they can be mandatory, like `b` in the preceding example.

We now move on to the introspection of function parameters, starting with a motivating example from a web framework, and on through introspection techniques.

Retrieving Information About Parameters

An interesting application of function introspection can be found in the Bobo HTTP micro-framework. To see that in action, consider a variation of the Bobo tutorial “Hello world” application in [Example 5-12](#).

Example 5-12. Bobo knows that `hello` requires a `person` argument, and retrieves it from the HTTP request

```
import bobo

@bobo.query('/')
def hello(person):
    return 'Hello %s!' % person
```

The `bobo.query` decorator integrates a plain function such as `hello` with the request handling machinery of the framework. We’ll cover decorators in [Chapter 7](#)—that’s not the point of this example here. The point is that Bobo introspects the `hello` function and finds out it needs one parameter named `person` to work, and it will retrieve a parameter with that name from the request and pass it to `hello`, so the programmer does not need to touch the request object at all.

If you install Bobo and point its development server to the script in [Example 5-12](#) (e.g., `bobo -f hello.py`), a hit on the URL `http://localhost:8080/` will produce the message “Missing form variable `person`” with a 403 HTTP code. This happens because Bobo understands that the `person` argument is required to call `hello`, but no such name was found in the request. [Example 5-13](#) is a shell session using `curl` to show this behavior.

Example 5-13. Bobo issues a 403 forbidden response if there are missing function arguments in the request; `curl -i` is used to dump the headers to standard output

```
$ curl -i http://localhost:8080/
HTTP/1.0 403 Forbidden
Date: Thu, 21 Aug 2014 21:39:44 GMT
Server: WSGIServer/0.2 CPython/3.4.1
Content-Type: text/html; charset=UTF-8
Content-Length: 103

<html>
<head><title>Missing parameter</title></head>
<body>Missing form variable person</body>
</html>
```

However, if you get `http://localhost:8080/?person=Jim`, the response will be the string `'Hello Jim!'`. See [Example 5-14](#).

Example 5-14. Passing the person parameter is required for an OK response

```
$ curl -i http://localhost:8080/?person=Jim
HTTP/1.0 200 OK
Date: Thu, 21 Aug 2014 21:42:32 GMT
Server: WSGIServer/0.2 CPython/3.4.1
Content-Type: text/html; charset=UTF-8
Content-Length: 10
```

Hello Jim!

How does Bobo know which parameter names are required by the function, and whether they have default values or not?

Within a function object, the `__defaults__` attribute holds a tuple with the default values of positional and keyword arguments. The defaults for keyword-only arguments appear in `__kwdefaults__`. The names of the arguments, however, are found within the `__code__` attribute, which is a reference to a code object with many attributes of its own.

To demonstrate the use of these attributes, we will inspect the function `clip` in a module `clip.py`, listed in [Example 5-15](#).

Example 5-15. Function to shorten a string by clipping at a space near the desired length

```
def clip(text, max_len=80):
    """Return text clipped at the last space before or after max_len
    """
    end = None
    if len(text) > max_len:
        space_before = text.rfind(' ', 0, max_len)
        if space_before >= 0:
            end = space_before
        else:
            space_after = text.rfind(' ', max_len)
            if space_after >= 0:
                end = space_after
    if end is None: # no spaces were found
        end = len(text)
    return text[:end].rstrip()
```

[Example 5-16](#) shows the values of `__defaults__`, `__code__.co_varnames`, and `__code__.co_argcount` for the `clip` function listed in [Example 5-15](#).

Example 5-16. Extracting information about the function arguments

```
>>> from clip import clip
>>> clip.__defaults__
```

```
(80,)
>>> clip.__code__ # doctest: +ELLIPSIS
<code object clip at 0x...>
>>> clip.__code__.co_varnames
('text', 'max_len', 'end', 'space_before', 'space_after')
>>> clip.__code__.co_argcount
2
```

As you can see, this is not the most convenient arrangement of information. The argument names appear in `__code__.co_varnames`, but that also includes the names of the local variables created in the body of the function. Therefore, the argument names are the first *N* strings, where *N* is given by `__code__.co_argcount` which—by the way—does not include any variable arguments prefixed with `*` or `**`. The default values are identified only by their position in the `__defaults__` tuple, so to link each with the respective argument, you have to scan from last to first. In the example, we have two arguments, `text` and `max_len`, and one default, `80`, so it must belong to the last argument, `max_len`. This is awkward.

Fortunately, there is a better way: the `inspect` module.

Take a look at [Example 5-17](#).

Example 5-17. Extracting the function signature

```
>>> from clip import clip
>>> from inspect import signature
>>> sig = signature(clip)
>>> sig # doctest: +ELLIPSIS
<inspect.Signature object at 0x...>
>>> str(sig)
'(text, max_len=80)'
>>> for name, param in sig.parameters.items():
...     print(param.kind, ': ', name, '=', param.default)
...
POSITIONAL_OR_KEYWORD : text = <class 'inspect._empty'>
POSITIONAL_OR_KEYWORD : max_len = 80
```

This is much better. `inspect.signature` returns an `inspect.Signature` object, which has a `parameters` attribute that lets you read an ordered mapping of names to `inspect.Parameter` objects. Each `Parameter` instance has attributes such as `name`, `default`, and `kind`. The special value `inspect._empty` denotes parameters with no default, which makes sense considering that `None` is a valid—and popular—default value.

The `kind` attribute holds one of five possible values from the `_ParameterKind` class:

`POSITIONAL_OR_KEYWORD`

A parameter that may be passed as a positional or as a keyword argument (most Python function parameters are of this kind).

VAR_POSITIONAL

A tuple of positional parameters.

VAR_KEYWORD

A dict of keyword parameters.

KEYWORD_ONLY

A keyword-only parameter (new in Python 3).

POSITIONAL_ONLY

A positional-only parameter; currently unsupported by Python function declaration syntax, but exemplified by existing functions implemented in C—like `divmod`—that do not accept parameters passed by keyword.

Besides `name`, `default`, and `kind`, `inspect.Parameter` objects have an `annotation` attribute that is usually `inspect._empty` but may contain function signature metadata provided via the new annotations syntax in Python 3 (annotations are covered in the next section).

An `inspect.Signature` object has a `bind` method that takes any number of arguments and binds them to the parameters in the signature, applying the usual rules for matching actual arguments to formal parameters. This can be used by a framework to validate arguments prior to the actual function invocation. [Example 5-18](#) shows how.

Example 5-18. Binding the function signature from the `tag` function in [Example 5-10](#) to a dict of arguments

```
>>> import inspect
>>> sig = inspect.signature(tag) ❶
>>> my_tag = {'name': 'img', 'title': 'Sunset Boulevard',
...          'src': 'sunset.jpg', 'cls': 'framed'}
>>> bound_args = sig.bind(**my_tag) ❷
>>> bound_args
<inspect.BoundArguments object at 0x...> ❸
>>> for name, value in bound_args.arguments.items(): ❹
...     print(name, '=', value)
...
name = img
cls = framed
attrs = {'title': 'Sunset Boulevard', 'src': 'sunset.jpg'}
>>> del my_tag['name'] ❺
>>> bound_args = sig.bind(**my_tag) ❻
Traceback (most recent call last):
...
TypeError: 'name' parameter lacking default value
```

❶ Get the signature from `tag` function in [Example 5-10](#).

❷ Pass a dict of arguments to `.bind()`.

- ③ An `inspect.BoundArguments` object is produced.
- ④ Iterate over the items in `bound_args.arguments`, which is an `OrderedDict`, to display the names and values of the arguments.
- ⑤ Remove the mandatory argument name from `my_tag`.
- ⑥ Calling `sig.bind(**my_tag)` raises a `TypeError` complaining of the missing name parameter.

This example shows how the Python data model, with the help of `inspect`, exposes the same machinery the interpreter uses to bind arguments to formal parameters in function calls.

Frameworks and tools like IDEs can use this information to validate code. Another feature of Python 3, function annotations, enhances the possible uses of this, as we will see next.

Function Annotations

Python 3 provides syntax to attach metadata to the parameters of a function declaration and its return value. [Example 5-19](#) is an annotated version of [Example 5-15](#). The only differences are in the first line.

Example 5-19. Annotated clip function

```
def clip(text:str, max_len:'int > 0'=80) -> str: ❶
    """Return text clipped at the last space before or after max_len
    """
    end = None
    if len(text) > max_len:
        space_before = text.rfind(' ', 0, max_len)
        if space_before >= 0:
            end = space_before
        else:
            space_after = text.rfind(' ', max_len)
            if space_after >= 0:
                end = space_after
    if end is None: # no spaces were found
        end = len(text)
    return text[:end].rstrip()
```

- ❶ The annotated function declaration.

Each argument in the function declaration may have an annotation expression preceded by `:`. If there is a default value, the annotation goes between the argument name and the `=` sign. To annotate the return value, add `->` and another expression between the `)` and the `:` at the tail of the function declaration. The expressions may be of any type. The

most common types used in annotations are classes, like `str` or `int`, or strings, like `'int > 0'`, as seen in the annotation for `max_len` in [Example 5-19](#).

No processing is done with the annotations. They are merely stored in the `__annotations__` attribute of the function, a dict:

```
>>> from clip_annot import clip
>>> clip.__annotations__
{'text': <class 'str'>, 'max_len': 'int > 0', 'return': <class 'str'>}
```

The item with key `'return'` holds the return value annotation marked with `->` in the function declaration in [Example 5-19](#).

The only thing Python does with annotations is to store them in the `__annotations__` attribute of the function. Nothing else: no checks, enforcement, validation, or any other action is performed. In other words, annotations have no meaning to the Python interpreter. They are just metadata that may be used by tools, such as IDEs, frameworks, and decorators. At this writing no tools that use this metadata exist in the standard library, except that `inspect.signature()` knows how to extract the annotations, as [Example 5-20](#) shows.

Example 5-20. Extracting annotations from the function signature

```
>>> from clip_annot import clip
>>> from inspect import signature
>>> sig = signature(clip)
>>> sig.return_annotation
<class 'str'>
>>> for param in sig.parameters.values():
...     note = repr(param.annotation).ljust(13)
...     print(note, ': ', param.name, '=', param.default)
<class 'str'> : text = <class 'inspect._empty'>
'int > 0'      : max_len = 80
```

The `signature` function returns a `Signature` object, which has a `return_annotation` attribute and a `parameters` dictionary mapping parameter names to `Parameter` objects. Each `Parameter` object has its own `annotation` attribute. That's how [Example 5-20](#) works.

In the future, frameworks such as Bobo could support annotations to further automate request processing. For example, an argument annotated as `price:float` may be automatically converted from a query string to the `float` expected by the function; a string annotation like `quantity:'int > 0'` might be parsed to perform conversion and validation of a parameter.

The biggest impact of function annotations will probably not be dynamic settings such as Bobo, but in providing optional type information for static type checking in tools like IDEs and linters.

After this deep dive into the anatomy of functions, the remainder of this chapter covers the most useful packages in the standard library that support functional programming.

Packages for Functional Programming

Although Guido makes it clear that Python does not aim to be a functional programming language, a functional coding style can be used to good extent, thanks to the support of packages like `operator` and `functools`, which we cover in the next two sections.

The operator Module

Often in functional programming it is convenient to use an arithmetic operator as a function. For example, suppose you want to multiply a sequence of numbers to calculate factorials without using recursion. To perform summation, you can use `sum`, but there is no equivalent function for multiplication. You could use `reduce`—as we saw in “Modern Replacements for `map`, `filter`, and `reduce`” on page 142—but this requires a function to multiply two items of the sequence. Example 5-21 shows how to solve this using `lambda`.

Example 5-21. Factorial implemented with `reduce` and an anonymous function

```
from functools import reduce

def fact(n):
    return reduce(lambda a, b: a*b, range(1, n+1))
```

To save you the trouble of writing trivial anonymous functions like `lambda a, b: a*b`, the `operator` module provides function equivalents for dozens of arithmetic operators. With it, we can rewrite Example 5-21 as Example 5-22.

Example 5-22. Factorial implemented with `reduce` and `operator.mul`

```
from functools import reduce
from operator import mul

def fact(n):
    return reduce(mul, range(1, n+1))
```

Another group of one-trick lambdas that `operator` replaces are functions to pick items from sequences or read attributes from objects: `itemgetter` and `attrgetter` actually build custom functions to do that.

Example 5-23 shows a common use of `itemgetter`: sorting a list of tuples by the value of one field. In the example, the cities are printed sorted by country code (field 1). Essentially, `itemgetter(1)` does the same as `lambda fields: fields[1]`: create a function that, given a collection, returns the item at index 1.

Example 5-23. Demo of `itemgetter` to sort a list of tuples (data from [Example 2-8](#))

```
>>> metro_data = [
...     ('Tokyo', 'JP', 36.933, (35.689722, 139.691667)),
...     ('Delhi NCR', 'IN', 21.935, (28.613889, 77.208889)),
...     ('Mexico City', 'MX', 20.142, (19.433333, -99.133333)),
...     ('New York-Newark', 'US', 20.104, (40.808611, -74.020386)),
...     ('Sao Paulo', 'BR', 19.649, (-23.547778, -46.635833)),
... ]
>>>
>>> from operator import itemgetter
>>> for city in sorted(metro_data, key=itemgetter(1)):
...     print(city)
...
('Sao Paulo', 'BR', 19.649, (-23.547778, -46.635833))
('Delhi NCR', 'IN', 21.935, (28.613889, 77.208889))
('Tokyo', 'JP', 36.933, (35.689722, 139.691667))
('Mexico City', 'MX', 20.142, (19.433333, -99.133333))
('New York-Newark', 'US', 20.104, (40.808611, -74.020386))
```

If you pass multiple index arguments to `itemgetter`, the function it builds will return tuples with the extracted values:

```
>>> cc_name = itemgetter(1, 0)
>>> for city in metro_data:
...     print(cc_name(city))
...
('JP', 'Tokyo')
('IN', 'Delhi NCR')
('MX', 'Mexico City')
('US', 'New York-Newark')
('BR', 'Sao Paulo')
>>>
```

Because `itemgetter` uses the `[]` operator, it supports not only sequences but also mappings and any class that implements `__getitem__`.

A sibling of `itemgetter` is `attrgetter`, which creates functions to extract object attributes by name. If you pass `attrgetter` several attribute names as arguments, it also returns a tuple of values. In addition, if any argument name contains a `.` (dot), `attrgetter` navigates through nested objects to retrieve the attribute. These behaviors are shown in [Example 5-24](#). This is not the shortest console session because we need to build a nested structure to showcase the handling of dotted attributes by `attrgetter`.

Example 5-24. Demo of `attrgetter` to process a previously defined list of `namedtuple` called `metro_data` (the same list that appears in [Example 5-23](#))

```
>>> from collections import namedtuple
>>> LatLong = namedtuple('LatLong', 'lat long') # ❶
>>> Metropolis = namedtuple('Metropolis', 'name cc pop coord') # ❷
```



```

>>> metro_areas = [Metropolis(name, cc, pop, LatLong(lat, long)) # ❸
...     for name, cc, pop, (lat, long) in metro_data]
>>> metro_areas[0]
Metropolis(name='Tokyo', cc='JP', pop=36.933, coord=LatLong(lat=35.689722,
long=139.691667))
>>> metro_areas[0].coord.lat # ❹
35.689722
>>> from operator import attrgetter
>>> name_lat = attrgetter('name', 'coord.lat') # ❺
>>>
>>> for city in sorted(metro_areas, key=attrgetter('coord.lat')): # ❻
...     print(name_lat(city)) # ❼
...
('Sao Paulo', -23.547778)
('Mexico City', 19.433333)
('Delhi NCR', 28.613889)
('Tokyo', 35.689722)
('New York-Newark', 40.808611)

```

- ❶ Use namedtuple to define LatLong.
- ❷ Also define Metropolis.
- ❸ Build metro_areas list with Metropolis instances; note the nested tuple unpacking to extract (lat, long) and use them to build the LatLong for the coord attribute of Metropolis.
- ❹ Reach into element metro_areas[0] to get its latitude.
- ❺ Define an attrgetter to retrieve the name and the coord.lat nested attribute.
- ❻ Use attrgetter again to sort list of cities by latitude.
- ❼ Use the attrgetter defined in ❺ to show only city name and latitude.

Here is a partial list of functions defined in operator (names starting with _ are omitted, because they are mostly implementation details):

```

>>> [name for name in dir(operator) if not name.startswith('_')]
['abs', 'add', 'and_', 'attrgetter', 'concat', 'contains',
'countOf', 'delitem', 'eq', 'floordiv', 'ge', 'getitem', 'gt',
'iadd', 'iand', 'iconcat', 'ifloordiv', 'ilshift', 'imod', 'imul',
'index', 'indexOf', 'inv', 'invert', 'ior', 'ipow', 'irshift',
'is_', 'is_not', 'isub', 'itemgetter', 'itruediv', 'ixor', 'le',
'length_hint', 'lshift', 'lt', 'methodcaller', 'mod', 'mul', 'ne',
'neg', 'not_', 'or_', 'pos', 'pow', 'rshift', 'setitem', 'sub',
'truediv', 'truth', 'xor']

```

Most of the 52 names listed are self-evident. The group of names prefixed with i and the name of another operator—e.g., iadd, iand, etc.—correspond to the augmented assignment operators—e.g., +=, &=, etc. These change their first argument in place, if it

is mutable; if not, the function works like the one without the `i` prefix: it simply returns the result of the operation.

Of the remaining operator functions, `methodcaller` is the last we will cover. It is somewhat similar to `attrgetter` and `itemgetter` in that it creates a function on the fly. The function it creates calls a method by name on the object given as argument, as shown in [Example 5-25](#).

Example 5-25. Demo of `methodcaller`: second test shows the binding of extra arguments

```
>>> from operator import methodcaller
>>> s = 'The time has come'
>>> upcase = methodcaller('upper')
>>> upcase(s)
'THE TIME HAS COME'
>>> hiphenate = methodcaller('replace', ' ', '-')
>>> hiphenate(s)
'The-time-has-come'
```

The first test in [Example 5-25](#) is there just to show `methodcaller` at work, but if you need to use the `str.upper` as a function, you can just call it on the `str` class and pass a string as argument, like this:

```
>>> str.upper(s)
'THE TIME HAS COME'
```

The second test in [Example 5-25](#) shows that `methodcaller` can also do a partial application to freeze some arguments, like the `functools.partial` function does. That is our next subject.

Freezing Arguments with `functools.partial`

The `functools` module brings together a handful of higher-order functions. The best known of them is probably `reduce`, which was covered in “[Modern Replacements for `map`, `filter`, and `reduce`](#)” on page 142. Of the remaining functions in `functools`, the most useful is `partial` and its variation, `partialmethod`.

`functools.partial` is a higher-order function that allows partial application of a function. Given a function, a partial application produces a new callable with some of the arguments of the original function fixed. This is useful to adapt a function that takes one or more arguments to an API that requires a callback with fewer arguments.

[Example 5-26](#) is a trivial demonstration.

Example 5-26. Using `partial` to use a two-argument function where a one-argument callable is required

```
>>> from operator import mul
>>> from functools import partial
>>> triple = partial(mul, 3) ❶
```

```
>>> triple(7) ❷
21
>>> list(map(triple, range(1, 10))) ❸
[3, 6, 9, 12, 15, 18, 21, 24, 27]
```

- ❶ Create new `triple` function from `mul`, binding first positional argument to 3.
- ❷ Test it.
- ❸ Use `triple` with `map`; `mul` would not work with `map` in this example.

A more useful example involves the `unicode.normalize` function that we saw in “[Normalizing Unicode for Safer Comparisons](#)” on page 117. If you work with text from many languages, you may want to apply `unicode.normalize('NFC', s)` to any string `s` before comparing or storing it. If you do that often, it’s handy to have an `nfc` function to do so, as in [Example 5-27](#).

Example 5-27. Building a convenient Unicode normalizing function with `partial`

```
>>> import unicodedata, functools
>>> nfc = functools.partial(unicodedata.normalize, 'NFC')
>>> s1 = 'café'
>>> s2 = 'cafe\u00301'
>>> s1, s2
('café', 'café')
>>> s1 == s2
False
>>> nfc(s1) == nfc(s2)
True
```

`partial` takes a callable as first argument, followed by an arbitrary number of positional and keyword arguments to bind.

[Example 5-28](#) shows the use of `partial` with the `tag` function from [Example 5-10](#), to freeze one positional argument and one keyword argument.

Example 5-28. Demo of `partial` applied to the function `tag` from [Example 5-10](#)

```
>>> from tagger import tag
>>> tag
<function tag at 0x10206d1e0> ❶
>>> from functools import partial
>>> picture = partial(tag, 'img', cls='pic-frame') ❷
>>> picture(src='wumpus.jpeg')
'' ❸
>>> picture
functools.partial(<function tag at 0x10206d1e0>, 'img', cls='pic-frame') ❹
>>> picture.func ❺
<function tag at 0x10206d1e0>
>>> picture.args
('img',)
```

```
>>> picture.keywords
{'cls': 'pic-frame'}
```

- ❶ Import tag from **Example 5-10** and show its ID.
- ❷ Create picture function from tag by fixing the first positional argument with 'img' and the cls keyword argument with 'pic-frame'.
- ❸ picture works as expected.
- ❹ partial() returns a functools.partial object.²
- ❺ A functools.partial object has attributes providing access to the original function and the fixed arguments.

The `functools.partialmethod` function (new in Python 3.4) does the same job as `partial`, but is designed to work with methods.

An impressive `functools` function is `lru_cache`, which does memoization—a form of automatic optimization that works by storing the results of function calls to avoid expensive recalculations. We will cover it in **Chapter 7**, where decorators are explained, along with other higher-order functions designed to be used as decorators: `singledispatch` and `wraps`.

Chapter Summary

The goal of this chapter was to explore the first-class nature of functions in Python. The main ideas are that you can assign functions to variables, pass them to other functions, store them in data structures, and access function attributes, allowing frameworks and tools to act on that information. Higher-order functions, a staple of functional programming, are common in Python—even if the use of `map`, `filter`, and `reduce` is not as frequent as it was—thanks to list comprehensions (and similar constructs like generator expressions) and the appearance of reducing built-ins like `sum`, `all`, and `any`. The `sorted`, `min`, `max` built-ins, and `functools.partial` are examples of commonly used higher-order functions in the language.

Callables come in seven different flavors in Python, from the simple functions created with `lambda` to instances of classes implementing `__call__`. They can all be detected by the `callable()` built-in. Every callable supports the same rich syntax for declaring formal parameters, including keyword-only parameters and annotations—both new features introduced with Python 3.

2. The **source code** for `functools.py` reveals that the `functools.partial` class is implemented in C and is used by default. If that is not available, a pure-Python implementation of `partial` is available since Python 3.4 in the `functools` module.

Python functions and their annotations have a rich set of attributes that can be read with the help of the `inspect` module, which includes the `Signature.bind` method to apply the flexible rules that Python uses to bind actual arguments to declared parameters.

Lastly, we covered some functions from the `operator` module and `functools.partial`, which facilitate functional programming by minimizing the need for the functionally challenged `lambda` syntax.

Further Reading

The next two chapters continue our exploration of programming with function objects. [Chapter 6](#) shows how first-class functions can simplify some classic object-oriented design patterns, while [Chapter 7](#) dives into function decorators—a special kind of higher-order function—and the closure mechanism that makes them work.

Chapter 7 of the *Python Cookbook, Third Edition* (O’Reilly), by David Beazley and Brian K. Jones, is an excellent complement to the current chapter as well as [Chapter 7](#) of this book, covering mostly the same concepts with a different approach.

In *The Python Language Reference*, “[3.2. The standard type hierarchy](#)” presents the seven callable types, along with all the other built-in types.

The Python-3-only features discussed in this chapter have their own PEPs: [PEP 3102 — Keyword-Only Arguments](#) and [PEP 3107 — Function Annotations](#).

For more about the current (as of mid-2014) use of annotations, two Stack Overflow questions are worth reading: “[What are good uses for Python3’s ‘Function Annotations’](#)” has a practical answer and insightful comments by Raymond Hettinger, and the answer for “[What good are Python function annotations?](#)” quotes extensively from Guido van Rossum.

[PEP 362 — Function Signature Object](#) is worth reading if you intend to use the `inspect` module that implements that feature.

A great introduction to functional programming in Python is A. M. Kuchling’s [Python Functional Programming HOWTO](#). The main focus of that text, however, is on the use of iterators and generators, which are the subject of [Chapter 14](#).

[fn.py](#) is a package to support functional programming in Python 2 and 3. According to its author, Alexey Kachayev, `fn.py` provides “implementation of missing features to enjoy FP” in Python. It includes a `@recur.tco` decorator that implements tail-call optimization for unlimited recursion in Python, among many other functions, data structures, and recipes.

The StackOverflow question “[Python: Why is functools.partial necessary?](#)” has a highly informative (and funny) reply by Alex Martelli, author of the classic *Python in a Nutshell*.

Jim Fulton’s Bobo was probably the first web framework that deserved to be called object-oriented. If you were intrigued by it and want to learn more about its modern rewrite, start at its [Introduction](#). A little of the early history of Bobo appears in a comment by Phillip J. Eby in a [discussion at Joel Spolsky’s blog](#).

Soapbox

About Bobo

I owe my Python career to Bobo. I used it in my first Python web project in 1998. I discovered Bobo while looking for an object-oriented way to code web applications, after trying Perl and Java alternatives.

In 1997, Bobo had pioneered the object publishing concept: direct mapping from URLs to a hierarchy of objects, with no need to configure routes. I was hooked when I saw the beauty of this. Bobo also featured automatic HTTP query handling based on analysis of the signatures of the methods or functions used to handle requests.

Bobo was created by Jim Fulton, known as “The Zope Pope” thanks to his leading role in the development of the Zope framework, the foundation of the Plone CMS, School-Tool, ERP5, and other large-scale Python projects. Jim is also the creator of ZODB—the Zope Object Database—a transactional object database that provides ACID (atomicity, consistency, isolation, and durability), designed for ease of use from Python.

Jim has since rewritten Bobo from scratch to support WSGI and modern Python (including Python 3). As of this writing, Bobo uses the `six` library to do the function introspection, in order to be compatible with Python 2 and Python 3 in spite of the changes in function objects and related APIs.

Is Python a Functional Language?

Around the year 2000, I was at a training in the United States when Guido van Rossum dropped by the classroom (he was not the instructor). In the Q&A that followed, somebody asked him which features of Python were borrowed from other languages. His answer: “Everything that is good in Python was stolen from other languages.”

Shriram Krishnamurthi, professor of Computer Science at Brown University, starts his “[Teaching Programming Languages in a Post-Linnaean Age](#)” paper with this:

Programming language “paradigms” are a moribund and tedious legacy of a bygone age. Modern language designers pay them no respect, so why do our courses slavishly adhere to them?

In that paper, Python is mentioned by name in this passage:

What else to make of a language like Python, Ruby, or Perl? Their designers have no patience for the niceties of these Linnaean hierarchies; they borrow features as they wish, creating melanges that utterly defy characterization.

Krishnamurthi submits that instead of trying to classify languages in some taxonomy, it's more useful to consider them as aggregations of features.

Even if it was not Guido's goal, endowing Python with first-class functions opened the door to functional programming. In his post "[Origins of Python's *Functional Features*](#)", he says that `map`, `filter`, and `reduce` were the motivation for adding `lambda` to Python in the first place. All of these features were contributed together by Amrit Prem for Python 1.0 in 1994 (according to [Misc/HISTORY](#) in the CPython source code).

`lambda`, `map`, `filter`, and `reduce` first appeared in Lisp, the original functional language. However, Lisp does not limit what can be done inside a `lambda`, because everything in Lisp is an expression. Python uses a statement-oriented syntax in which expressions cannot contain statements, and many language constructs are statements—including `try/catch`, which is what I miss most often when writing `lambdas`. This is the price to pay for Python's highly readable syntax.³ Lisp has many strengths, but readability is not one of them.

Ironically, stealing the list comprehension syntax from another functional language—Haskell—significantly diminished the need for `map` and `filter`, and also for `lambda`.

Besides the limited anonymous function syntax, the biggest obstacle to wider adoption of functional programming idioms in Python is the lack of tail-recursion elimination, an optimization that allows memory-efficient computation of a function that makes a recursive call at the "tail" of its body. In another blog post, "[Tail Recursion Elimination](#)", Guido gives several reasons why such optimization is not a good fit for Python. That post is a great read for the technical arguments, but even more so because the first three and most important reasons given are usability issues. It is no accident that Python is a pleasure to use, learn, and teach. Guido made it so.

So there you have it: Python is, by design, not a functional language—whatever that means. Python just borrows a few good ideas from functional languages.

The Problem with Anonymous Functions

Beyond the Python-specific syntax constraints, anonymous functions have a serious drawback in every language: they have no name.

I am only half joking here. Stack traces are easier to read when functions have names. Anonymous functions are a handy shortcut, people have fun coding with them, but sometimes they get carried away—especially if the language and environment encourage deep nesting of anonymous functions, like JavaScript on Node.js. Lots of nested anonymous functions make debugging and error handling hard. Asynchronous programming

3. There also the problem of lost indentation when pasting code to Web forums, but I digress.

in Python is more structured, perhaps because the limited `lambda` demands it. I promise to write more about asynchronous programming in the future, but this subject must be deferred to [Chapter 18](#). By the way, promises, futures, and deferreds are concepts used in modern asynchronous APIs. Along with coroutines, they provide an escape from the so-called “callback hell.” We’ll see how callback-free asynchronous programming works in [“From Callbacks to Futures and Coroutines”](#) on page 562.

FP Ch 7 | Function Decorators & Closures [= plan Ch 9]

Function Decorators and Closures

There's been a number of complaints about the choice of the name “decorator” for this feature. The major one is that the name is not consistent with its use in the GoF book.¹ The name *decorator* probably owes more to its use in the compiler area—a syntax tree is walked and annotated.

— PEP 318 — Decorators for Functions and Methods

Function decorators let us “mark” functions in the source code to enhance their behavior in some way. This is powerful stuff, but mastering it requires understanding closures.

One of the newest reserved keywords in Python is `nonlocal`, introduced in Python 3.0. You can have a profitable life as a Python programmer without ever using it if you adhere to a strict regimen of class-centered object orientation. However, if you want to implement your own function decorators, you must know closures inside out, and then the need for `nonlocal` becomes obvious.

Aside from their application in decorators, closures are also essential for effective asynchronous programming with callbacks, and for coding in a functional style whenever it makes sense.

The end goal of this chapter is to explain exactly how function decorators work, from the simplest registration decorators to the rather more complicated parameterized ones. However, before we reach that goal we need to cover:

- How Python evaluates decorator syntax
- How Python decides whether a variable is local
- Why closures exist and how they work

1. That's the 1995 *Design Patterns* book by the so-called Gang of Four.

- What problem is solved by `nonlocal`

With this grounding, we can tackle further decorator topics:

- Implementing a well-behaved decorator
- Interesting decorators in the standard library
- Implementing a parameterized decorator

We start with a very basic introduction to decorators, and then proceed with the rest of the items listed here.

Decorators 101

A decorator is a callable that takes another function as argument (the decorated function).² The decorator may perform some processing with the decorated function, and returns it or replaces it with another function or callable object.

In other words, assuming an existing decorator named `decorate`, this code:

```
@decorate
def target():
    print('running target()')
```

Has the same effect as writing this:

```
def target():
    print('running target()')

target = decorate(target)
```

The end result is the same: at the end of either of these snippets, the `target` name does not necessarily refer to the original `target` function, but to whatever function is returned by `decorate(target)`.

To confirm that the decorated function is replaced, see the console session in [Example 7-1](#).

Example 7-1. A decorator usually replaces a function with a different one

```
>>> def deco(func):
...     def inner():
...         print('running inner()')
...         return inner ❶
...
>>> @deco
... def target(): ❷
```

2. Python also supports class decorators. They are covered in [Chapter 21](#).

```

...     print('running target()')
...
>>> target() ❸
running inner()
>>> target ❹
<function deco.<locals>.inner at 0x10063b598>

```

- ❶ deco returns its inner function object.
- ❷ target is decorated by deco.
- ❸ Invoking the decorated target actually runs inner.
- ❹ Inspection reveals that target is now a reference to inner.

Strictly speaking, decorators are just syntactic sugar. As we just saw, you can always simply call a decorator like any regular callable, passing another function. Sometimes that is actually convenient, especially when doing *metaprogramming*—changing program behavior at runtime.

To summarize: the first crucial fact about decorators is that they have the power to replace the decorated function with a different one. The second crucial fact is that they are executed immediately when a module is loaded. This is explained next.

When Python Executes Decorators

A key feature of decorators is that they run right after the decorated function is defined. That is usually at *import time* (i.e., when a module is loaded by Python). Consider *registration.py* in [Example 7-2](#).

Example 7-2. The registration.py module

```

registry = [] ❶

def register(func): ❷
    print('running register(%s)' % func) ❸
    registry.append(func) ❹
    return func ❺

@register ❻
def f1():
    print('running f1()')

@register
def f2():
    print('running f2()')

def f3(): ❼
    print('running f3()')

def main(): ❽

```

```

print('running main()')
print('registry ->', registry)
f1()
f2()
f3()

if __name__ == '__main__':
    main() ⑨

```

- ❶ registry will hold references to functions decorated by @register.
- ❷ register takes a function as argument.
- ❸ Display what function is being decorated, for demonstration.
- ❹ Include func in registry.
- ❺ Return func: we must return a function; here we return the same received as argument.
- ❻ f1 and f2 are decorated by @register.
- ❼ f3 is not decorated.
- ❽ main displays the registry, then calls f1(), f2(), and f3().
- ❾ main() is only invoked if *registration.py* runs as a script.

The output of running *registration.py* as a script looks like this:

```

$ python3 registration.py
running register(<function f1 at 0x100631bf8>)
running register(<function f2 at 0x100631c80>)
running main()
registry -> [<function f1 at 0x100631bf8>, <function f2 at 0x100631c80>]
running f1()
running f2()
running f3()

```

Note that register runs (twice) before any other function in the module. When register is called, it receives as an argument the function object being decorated—for example, <function f1 at 0x100631bf8>.

After the module is loaded, the registry holds references to the two decorated functions: f1 and f2. These functions, as well as f3, are only executed when explicitly called by main.

If *registration.py* is imported (and not run as a script), the output is this:

```

>>> import registration
running register(<function f1 at 0x10063b1e0>)
running register(<function f2 at 0x10063b268>)

```

At this time, if you look at the registry, here is what you get:

```
>>> registration.registry
[<function f1 at 0x10063b1e0>, <function f2 at 0x10063b268>]
```

The main point of [Example 7-2](#) is to emphasize that function decorators are executed as soon as the module is imported, but the decorated functions only run when they are explicitly invoked. This highlights the difference between what Pythonistas call *import time* and *runtime*.

Considering how decorators are commonly employed in real code, [Example 7-2](#) is unusual in two ways:

- The decorator function is defined in the same module as the decorated functions. A real decorator is usually defined in one module and applied to functions in other modules.
- The `register` decorator returns the same function passed as argument. In practice, most decorators define an inner function and return it.

Even though the `register` decorator in [Example 7-2](#) returns the decorated function unchanged, that technique is not useless. Similar decorators are used in many Python web frameworks to add functions to some central registry—for example, a registry mapping URL patterns to functions that generate HTTP responses. Such registration decorators may or may not change the decorated function. The next section shows a practical example.

Decorator-Enhanced Strategy Pattern

A registration decorator is a good enhancement to the ecommerce promotional discount from “[Case Study: Refactoring Strategy](#)” on page 168.

Recall that our main issue with [Example 6-6](#) is the repetition of the function names in their definitions and then in the `promos` list used by the `best_promo` function to determine the highest discount applicable. The repetition is problematic because someone may add a new promotional strategy function and forget to manually add it to the `promos` list—in which case, `best_promo` will silently ignore the new strategy, introducing a subtle bug in the system. [Example 7-3](#) solves this problem with a registration decorator.

Example 7-3. The `promos` list is filled by the promotion decorator

```
promos = [] ❶

def promotion(promo_func): ❷
    promos.append(promo_func)
    return promo_func

@promotion ❸
def fidelity(order):
    """5% discount for customers with 1000 or more fidelity points"""
```

```

    return order.total() * .05 if order.customer.fidelity >= 1000 else 0

@promotion
def bulk_item(order):
    """10% discount for each LineItem with 20 or more units"""
    discount = 0
    for item in order.cart:
        if item.quantity >= 20:
            discount += item.total() * .1
    return discount

@promotion
def large_order(order):
    """7% discount for orders with 10 or more distinct items"""
    distinct_items = {item.product for item in order.cart}
    if len(distinct_items) >= 10:
        return order.total() * .07
    return 0

def best_promo(order): ❷
    """Select best discount available"""
    return max(promo(order) for promo in promos)

```

- ❶ The promos list starts empty.
- ❷ promotion decorator returns promo_func unchanged, after adding it to the promos list.
- ❸ Any function decorated by @promotion will be added to promos.
- ❹ No changes needed to best_promos, because it relies on the promos list.

This solution has several advantages over the others presented in “[Case Study: Refactoring Strategy](#)” on page 168:

- The promotion strategy functions don’t have to use special names (i.e., they don’t need to use the _promo suffix).
- The @promotion decorator highlights the purpose of the decorated function, and also makes it easy to temporarily disable a promotion: just comment out the decorator.
- Promotional discount strategies may be defined in other modules, anywhere in the system, as long as the @promotion decorator is applied to them.

Most decorators do change the decorated function. They usually do it by defining an inner function and returning it to replace the decorated function. Code that uses inner functions almost always depends on closures to operate correctly. To understand clo-

sure, we need to take a step back and have a close look at how variable scopes work in Python.

Variable Scope Rules

In [Example 7-4](#), we define and test a function that reads two variables: a local variable `a`, defined as function parameter, and variable `b` that is not defined anywhere in the function.

Example 7-4. Function reading a local and a global variable

```
>>> def f1(a):
...     print(a)
...     print(b)
...
>>> f1(3)
3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in f1
NameError: global name 'b' is not defined
```

The error we got is not surprising. Continuing from [Example 7-4](#), if we assign a value to a global `b` and then call `f1`, it works:

```
>>> b = 6
>>> f1(3)
3
6
```

Now, let's see an example that may surprise you.

Take a look at the `f2` function in [Example 7-5](#). Its first two lines are the same as `f1` in [Example 7-4](#), then it makes an assignment to `b`, and prints its value. But it fails at the second print, before the assignment is made.

Example 7-5. Variable `b` is local, because it is assigned a value in the body of the function

```
>>> b = 6
>>> def f2(a):
...     print(a)
...     print(b)
...     b = 9
...
>>> f2(3)
3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in f2
UnboundLocalError: local variable 'b' referenced before assignment
```

Note that the output starts with 3, which proves that the `print(a)` statement was executed. But the second one, `print(b)`, never runs. When I first saw this I was surprised, thinking that 6 should be printed, because there is a global variable `b` and the assignment to the local `b` is made after `print(b)`.

But the fact is, when Python compiles the body of the function, it decides that `b` is a local variable because it is assigned within the function. The generated bytecode reflects this decision and will try to fetch `b` from the local environment. Later, when the call `f2(3)` is made, the body of `f2` fetches and prints the value of the local variable `a`, but when trying to fetch the value of local variable `b` it discovers that `b` is unbound.

This is not a bug, but a design choice: Python does not require you to declare variables, but assumes that a variable assigned in the body of a function is local. This is much better than the behavior of JavaScript, which does not require variable declarations either, but if you do forget to declare that a variable is local (with `var`), you may clobber a global variable without knowing.

If we want the interpreter to treat `b` as a global variable in spite of the assignment within the function, we use the `global` declaration:

```
>>> def f3(a):
...     global b
...     print(a)
...     print(b)
...     b = 9
...
>>> f3(3)
3
6
>>> b
9

>>> f3(3)
a = 3
b = 8
b = 30
>>> b
30
>>>
```

After this closer look at how variable scopes work in Python, we can tackle closures in the next section, “[Closures](#)” on page 192. If you are curious about the bytecode differences between the functions in Examples 7-4 and 7-5, see the following sidebar.

Comparing Bytecodes

The `dis` module provides an easy way to disassemble the bytecode of Python functions. Read Examples 7-6 and 7-7 to see the bytecodes for `f1` and `f2` from Examples 7-4 and 7-5.

Example 7-6. Disassembly of the `f1` function from Example 7-4

```
>>> from dis import dis
>>> dis(f1)
 2          0 LOAD_GLOBAL           0 (print) ❶
          3 LOAD_FAST              0 (a) ❷
          6 CALL_FUNCTION         1 (1 positional, 0 keyword pair)
          9 POP_TOP

 3          10 LOAD_GLOBAL          0 (print)
          13 LOAD_GLOBAL          1 (b) ❸
          16 CALL_FUNCTION         1 (1 positional, 0 keyword pair)
          19 POP_TOP
          20 LOAD_CONST          0 (None)
          23 RETURN_VALUE
```

- ❶ Load global name `print`.
- ❷ Load local name `a`.
- ❸ Load global name `b`.

Contrast the bytecode for `f1` shown in Example 7-6 with the bytecode for `f2` in Example 7-7.

Example 7-7. Disassembly of the `f2` function from Example 7-5

```
>>> dis(f2)
 2          0 LOAD_GLOBAL           0 (print)
          3 LOAD_FAST              0 (a)
          6 CALL_FUNCTION         1 (1 positional, 0 keyword pair)
          9 POP_TOP

 3          10 LOAD_GLOBAL          0 (print)
          13 LOAD_FAST              1 (b) ❶
          16 CALL_FUNCTION         1 (1 positional, 0 keyword pair)
          19 POP_TOP

 4          20 LOAD_CONST          1 (9)
          23 STORE_FAST           1 (b)
          26 LOAD_CONST          0 (None)
          29 RETURN_VALUE
```

- ❶ Load *local* name `b`. This shows that the compiler considers `b` a local variable, even if the assignment to `b` occurs later, because the nature of the variable—whether it is local or not—cannot change the body of the function.

The CPython VM that runs the bytecode is a stack machine, so the operations `LOAD` and `POP` refer to the stack. It is beyond the scope of this book to further describe the Python opcodes, but they are documented along with the `dis` module in [dis — Disassembler for Python bytecode](#).

Closures

In the blogosphere, closures are sometimes confused with anonymous functions. The reason why many confuse them is historic: defining functions inside functions is not so common, until you start using anonymous functions. And closures only matter when you have nested functions. So a lot of people learn both concepts at the same time.

Actually, a closure is a function with an extended scope that encompasses nonglobal variables referenced in the body of the function but not defined there. It does not matter whether the function is anonymous or not; what matters is that it can access nonglobal variables that are defined outside of its body.

This is a challenging concept to grasp, and is better approached through an example.

Consider an `avg` function to compute the mean of an ever-increasing series of values; for example, the average closing price of a commodity over its entire history. Every day a new price is added, and the average is computed taking into account all prices so far.

Starting with a clean slate, this is how `avg` could be used:

```
>>> avg(10)
10.0
>>> avg(11)
10.5
>>> avg(12)
11.0
```

Where does `avg` come from, and where does it keep the history of previous values?

For starters, [Example 7-8](#) is a class-based implementation.

Example 7-8. `average_oo.py`: A class to calculate a running average

```
class Averager():

    def __init__(self):
        self.series = []

    def __call__(self, new_value):
```

```

self.series.append(new_value)
total = sum(self.series)
return total/len(self.series)

```

The `Averager` class creates instances that are callable:

```

>>> avg = Averager()
>>> avg(10)
10.0
>>> avg(11)
10.5
>>> avg(12)
11.0

```

Now, [Example 7-9](#) is a functional implementation, using the higher-order function `make_averager`.

Example 7-9. `average.py`: A higher-order function to calculate a running average

```

def make_averager():
    series = []

    def averager(new_value):
        series.append(new_value)
        total = sum(series)
        return total/len(series)

    return averager

```

When invoked, `make_averager` returns an `averager` function object. Each time an `averager` is called, it appends the passed argument to the series, and computes the current average, as shown in [Example 7-10](#).

Example 7-10. Testing [Example 7-9](#)

```

>>> avg = make_averager()
>>> avg(10)
10.0
>>> avg(11)
10.5
>>> avg(12)
11.0

```

Note the similarities of the examples: we call `Averager()` or `make_averager()` to get a callable object `avg` that will update the historical series and calculate the current mean. In [Example 7-8](#), `avg` is an instance of `Averager`, and in [Example 7-9](#) it is the inner function, `averager`. Either way, we just call `avg(n)` to include `n` in the series and get the updated mean.

It's obvious where the avg of the Averager class keeps the history: the `self.series` instance attribute. But where does the avg function in the second example find the series?

Note that `series` is a local variable of `make_averager` because the initialization `series = []` happens in the body of that function. But when `avg(10)` is called, `make_averager` has already returned, and its local scope is long gone.

Within `averager`, `series` is a *free variable*. This is a technical term meaning a variable that is not bound in the local scope. See [Figure 7-1](#).

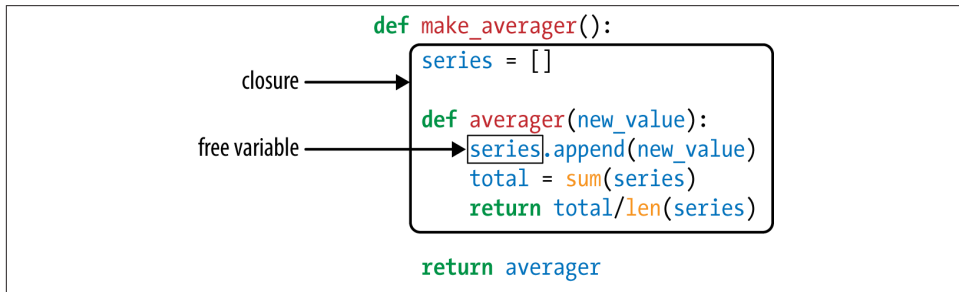


Figure 7-1. The closure for `averager` extends the scope of that function to include the binding for the free variable `series`.

Inspecting the returned `averager` object shows how Python keeps the names of local and free variables in the `__code__` attribute that represents the compiled body of the function. [Example 7-11](#) demonstrates.

Example 7-11. Inspecting the function created by `make_averager` in [Example 7-9](#)

```
>>> avg.__code__.co_varnames
('new_value', 'total')
>>> avg.__code__.co_freevars
('series',)
```

The binding for `series` is kept in the `__closure__` attribute of the returned function `avg`. Each item in `avg.__closure__` corresponds to a name in `avg.__code__.co_freevars`. These items are cells, and they have an attribute called `cell_contents` where the actual value can be found. [Example 7-12](#) shows these attributes.

Example 7-12. Continuing from [Example 7-10](#)

```
>>> avg.__code__.co_freevars
('series',)
>>> avg.__closure__
(<cell at 0x107a44f78: list object at 0x107a91a48>,)
>>> avg.__closure__[0].cell_contents
[10, 11, 12]
```

To summarize: a closure is a function that retains the bindings of the free variables that exist when the function is defined, so that they can be used later when the function is invoked and the defining scope is no longer available.

Note that the only situation in which a function may need to deal with external variables that are nonglobal is when it is nested in another function.

The nonlocal Declaration

Our previous implementation of `make_averager` was not efficient. In [Example 7-9](#), we stored all the values in the historical series and computed their sum every time `averager` was called. A better implementation would just store the total and the number of items so far, and compute the mean from these two numbers.

[Example 7-13](#) is a broken implementation, just to make a point. Can you see where it breaks?

Example 7-13. A broken higher-order function to calculate a running average without keeping all history

```
def make_averager():
    count = 0
    total = 0

    def averager(new_value):
        count += 1
        total += new_value
        return total / count

    return averager
```

If you try [Example 7-13](#), here is what you get:

```
>>> avg = make_averager()
>>> avg(10)
Traceback (most recent call last):
...
UnboundLocalError: local variable 'count' referenced before assignment
>>>
```

The problem is that the statement `count += 1` actually means the same as `count = count + 1`, when `count` is a number or any immutable type. So we are actually assigning to `count` in the body of `averager`, and that makes it a local variable. The same problem affects the `total` variable.

We did not have this problem in [Example 7-9](#) because we never assigned to the `series` name; we only called `series.append` and invoked `sum` and `len` on it. So we took advantage of the fact that lists are mutable.

But with immutable types like numbers, strings, tuples, etc., all you can do is read, but never update. If you try to rebind them, as in `count = count + 1`, then you are implicitly creating a local variable `count`. It is no longer a free variable, and therefore it is not saved in the closure.

To work around this, the `nonlocal` declaration was introduced in Python 3. It lets you flag a variable as a free variable even when it is assigned a new value within the function. If a new value is assigned to a `nonlocal` variable, the binding stored in the closure is changed. A correct implementation of our newest `make_averager` looks like [Example 7-14](#).

Example 7-14. Calculate a running average without keeping all history (fixed with the use of `nonlocal`)

```
def make_averager():
    count = 0
    total = 0

    def averager(new_value):
        nonlocal count, total
        count += 1
        total += new_value
        return total / count

    return averager
```



Getting by without `nonlocal` in Python 2

The lack of `nonlocal` in Python 2 requires workarounds, one of which is described in the third code snippet of [PEP 3104 — Access to Names in Outer Scopes](#), which introduced `nonlocal`. Essentially the idea is to store the variables the inner functions need to change (e.g., `count`, `total`) as items or attributes of some mutable object, like a `dict` or a simple instance, and bind that object to a free variable.

Now that we have Python closures covered, we can effectively implement decorators with nested functions.

Implementing a Simple Decorator

[Example 7-15](#) is a decorator that clocks every invocation of the decorated function and prints the elapsed time, the arguments passed, and the result of the call.

Example 7-15. A simple decorator to output the running time of functions

```
import time
```



```
def clock(func):
    def clocked(*args): # ❶
        t0 = time.perf_counter()
        result = func(*args) # ❷
        elapsed = time.perf_counter() - t0
        name = func.__name__
        arg_str = ', '.join(repr(arg) for arg in args)
        print('[%0.8fs] %s(%s) -> %r' % (elapsed, name, arg_str, result))
        return result
    return clocked # ❸
```

- ❶ Define inner function `clocked` to accept any number of positional arguments.
- ❷ This line only works because the closure for `clocked` encompasses the `func` free variable.
- ❸ Return the inner function to replace the decorated function.

Example 7-16 demonstrates the use of the `clock` decorator.

Example 7-16. Using the clock decorator

clockdeco_demo.py

```
import time
from clockdeco import clock

@clock
def snooze(seconds):
    time.sleep(seconds)

@clock
def factorial(n):
    return 1 if n < 2 else n*factorial(n-1)

if __name__ == '__main__':
    print('*' * 40, 'Calling snooze(.123)')
    snooze(.123)
    print('*' * 40, 'Calling factorial(6)')
    print('6! =', factorial(6))
```

The output of running **Example 7-16** looks like this:

```
$ python3 clockdeco_demo.py
***** Calling snooze(123)
[0.12405610s] snooze(.123) -> None
***** Calling factorial(6)
[0.00000191s] factorial(1) -> 1
[0.00004911s] factorial(2) -> 2
[0.00008488s] factorial(3) -> 6
[0.00013208s] factorial(4) -> 24
[0.00019193s] factorial(5) -> 120
```

```
[0.00026107s] factorial(6) -> 720  
6! = 720
```

How It Works

Remember that this code:

```
@clock  
def factorial(n):  
    return 1 if n < 2 else n*factorial(n-1)
```

Actually does this:

```
def factorial(n):  
    return 1 if n < 2 else n*factorial(n-1)  
  
factorial = clock(factorial)
```

So, in both examples, `clock` gets the `factorial` function as its `func` argument (see [Example 7-15](#)). It then creates and returns the `clocked` function, which the Python interpreter assigns to `factorial` behind the scenes. In fact, if you import the `clockdemo` module and check the `__name__` of `factorial`, this is what you get:

```
>>> import clockdemo  
>>> clockdemo.factorial.__name__  
'clocked'  
>>>
```

So `factorial` now actually holds a reference to the `clocked` function. From now on, each time `factorial(n)` is called, `clocked(n)` gets executed. In essence, `clocked` does the following:

1. Records the initial time `t0`.
2. Calls the original `factorial`, saving the result.
3. Computes the elapsed time.
4. Formats and prints the collected data.
5. Returns the result saved in step 2.

This is the typical behavior of a decorator: it replaces the decorated function with a new function that accepts the same arguments and (usually) returns whatever the decorated function was supposed to return, while also doing some extra processing.



In *Design Patterns* by Gamma et al., the short description of the Decorator pattern starts with: “Attach additional responsibilities to an object dynamically.” Function decorators fit that description. But at the implementation level, Python decorators bear little resemblance to the classic Decorator described in the original *Design Patterns* work. “Soapbox” on page 213 has more on this subject.

The clock decorator implemented in [Example 7-15](#) has a few shortcomings: it does not support keyword arguments, and it masks the `__name__` and `__doc__` of the decorated function. [Example 7-17](#) uses the `functools.wraps` decorator to copy the relevant attributes from `func` to `clocked`. Also, in this new version, keyword arguments are correctly handled.

Example 7-17. An improved clock decorator

```
# clockdeco2.py
```

```
import time
import functools
```

```
def clock(func):
    @functools.wraps(func)
    def clocked(*args, **kwargs):
        t0 = time.time()
        result = func(*args, **kwargs)
        elapsed = time.time() - t0
        name = func.__name__
        arg_lst = []
        if args:
            arg_lst.append(', '.join(repr(arg) for arg in args))
        if kwargs:
            pairs = ['%s=%r' % (k, w) for k, w in sorted(kwargs.items())]
            arg_lst.append(', '.join(pairs))
        arg_str = ', '.join(arg_lst)
        print('[%0.8fs] %s(%s) -> %r' % (elapsed, name, arg_str, result))
        return result
    return clocked
```

`functools.wraps` is just one of the ready-to-use decorators in the standard library. In the next section, we’ll meet two of the most impressive decorators that `functools` provides: `lru_cache` and `singledispatch`.

Decorators in the Standard Library

Python has three built-in functions that are designed to decorate methods: `property`, `classmethod`, and `staticmethod`. We will discuss `property` in [“Using a Property for](#)

Attribute Validation” on page 604 and the others in “classmethod Versus staticmethod” on page 252.

Another frequently seen decorator is `functools.wraps`, a helper for building well-behaved decorators. We used it in [Example 7-17](#). Two of the most interesting decorators in the standard library are `lru_cache` and the brand-new `singledispatch` (added in Python 3.4). Both are defined in the `functools` module. We’ll cover them next.

Memoization with `functools.lru_cache`

A very practical decorator is `functools.lru_cache`. It implements memoization: an optimization technique that works by saving the results of previous invocations of an expensive function, avoiding repeat computations on previously used arguments. The letters LRU stand for Least Recently Used, meaning that the growth of the cache is limited by discarding the entries that have not been read for a while.

A good demonstration is to apply `lru_cache` to the painfully slow recursive function to generate the n th number in the Fibonacci sequence, as shown in [Example 7-18](#).

Example 7-18. The very costly recursive way to compute the n th number in the Fibonacci series

```
from clockdeco import clock

@clock
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-2) + fibonacci(n-1)

if __name__ == '__main__':
    print(fibonacci(6))
```

Here is the result of running `fibonacci_demo.py`. Except for the last line, all output is generated by the `clock` decorator:

```
$ python3 fibo_demo.py
[0.00000095s] fibonacci(0) -> 0
[0.00000095s] fibonacci(1) -> 1
[0.00007892s] fibonacci(2) -> 1
[0.00000095s] fibonacci(1) -> 1
[0.00000095s] fibonacci(0) -> 0
[0.00000095s] fibonacci(1) -> 1
[0.00003815s] fibonacci(2) -> 1
[0.00007391s] fibonacci(3) -> 2
[0.00018883s] fibonacci(4) -> 3
[0.00000000s] fibonacci(1) -> 1
[0.00000095s] fibonacci(0) -> 0
[0.00000119s] fibonacci(1) -> 1
[0.00004911s] fibonacci(2) -> 1
```

```

[0.00009704s] fibonacci(3) -> 2
[0.00000000s] fibonacci(0) -> 0
[0.00000000s] fibonacci(1) -> 1
[0.00002694s] fibonacci(2) -> 1
[0.00000095s] fibonacci(1) -> 1
[0.00000095s] fibonacci(0) -> 0
[0.00000095s] fibonacci(1) -> 1
[0.00005102s] fibonacci(2) -> 1
[0.00008917s] fibonacci(3) -> 2
[0.00015593s] fibonacci(4) -> 3
[0.00029993s] fibonacci(5) -> 5
[0.00052810s] fibonacci(6) -> 8
8

```

The waste is obvious: `fibonacci(1)` is called eight times, `fibonacci(2)` five times, etc. But if we just add two lines to use `lru_cache`, performance is much improved. See [Example 7-19](#).

Example 7-19. Faster implementation using caching

```

import functools

from clockdeco import clock

@functools.lru_cache() # ❶
@clock # ❷
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-2) + fibonacci(n-1)

if __name__ == '__main__':
    print(fibonacci(6))

```

- ❶ Note that `lru_cache` must be invoked as a regular function—note the parentheses in the line: `@functools.lru_cache()`. The reason is that it accepts configuration parameters, as we’ll see shortly.
- ❷ This is an example of stacked decorators: `@lru_cache()` is applied on the function returned by `@clock`.

Execution time is halved, and the function is called only once for each value of `n`:

```

$ python3 fibo_demo_lru.py
[0.00000119s] fibonacci(0) -> 0
[0.00000119s] fibonacci(1) -> 1
[0.00010800s] fibonacci(2) -> 1
[0.00000787s] fibonacci(3) -> 2
[0.00016093s] fibonacci(4) -> 3
[0.00001216s] fibonacci(5) -> 5
[0.00025296s] fibonacci(6) -> 8

```

In another test, to compute `fibonacci(30)`, [Example 7-19](#) made the 31 calls needed in 0.0005s, while the uncached [Example 7-18](#) called `fibonacci` 2,692,537 times and took 17.7 seconds in an Intel Core i7 notebook.

Besides making silly recursive algorithms viable, `lru_cache` really shines in applications that need to fetch information from the Web.

It's important to note that `lru_cache` can be tuned by passing two optional arguments. Its full signature is:

```
functools.lru_cache(maxsize=128, typed=False)
```

The `maxsize` argument determines how many call results are stored. After the cache is full, older results are discarded to make room. For optimal performance, `maxsize` should be a power of 2. The `typed` argument, if set to `True`, stores results of different argument types separately, i.e., distinguishing between float and integer arguments that are normally considered equal, like 1 and 1.0. By the way, because `lru_cache` uses a `dict` to store the results, and the keys are made from the positional and keyword arguments used in the calls, all the arguments taken by the decorated function must be *hashable*.

Now let's consider the intriguing `functools.singledispatch` decorator.

Generic Functions with Single Dispatch

Imagine we are creating a tool to debug web applications. We want to be able to generate HTML displays for different types of Python objects.

We could start with a function like this:

```
import html

def htmlize(obj):
    content = html.escape(repr(obj))
    return '<pre>{}/pre>'.format(content)
```

That will work for any Python type, but now we want to extend it to generate custom displays for some types:

- `str`: replace embedded newline characters with '`<br>\n`' and use `<p>` tags instead of `<pre>`.
- `int`: show the number in decimal and hexadecimal.
- `list`: output an HTML list, formatting each item according to its type.

The behavior we want is shown in [Example 7-20](#).

Example 7-20. `htmlize` generates HTML tailored to different object types

```
>>> htmlize({1, 2, 3}) ❶
'<pre>{1, 2, 3}</pre>'
```

```
>>> htmlize(abs)
'<pre>&lt;built-in function abs&gt;</pre>'
>>> htmlize('Heimlich & Co.\n- a game') ❷
'<p>Heimlich & Co.<br>\n- a game</p>'
>>> htmlize(42) ❸
'<pre>42 (0x2a)</pre>'
>>> print(htmlize(['alpha', 66, {3, 2, 1}])) ❹
<ul>
<li><p>alpha</p></li>
<li><pre>66 (0x42)</pre></li>
<li><pre>{1, 2, 3}</pre></li>
</ul>
```

- ❶ By default, the HTML-escaped repr of an object is shown enclosed in `<pre></pre>`.
- ❷ `str` objects are also HTML-escaped but wrapped in `<p></p>` with `<br>` line breaks.
- ❸ An `int` is shown in decimal and hexadecimal, inside `<pre></pre>`.
- ❹ Each list item is formatted according to its type, and the whole sequence rendered as an HTML list.

Because we don't have method or function overloading in Python, we can't create variations of `htmlize` with different signatures for each data type we want to handle differently. A common solution in Python would be to turn `htmlize` into a dispatch function, with a chain of `if/elif/elif` calling specialized functions like `htmlize_str`, `htmlize_int`, etc. This is not extensible by users of our module, and is unwieldy: over time, the `htmlize` dispatcher would become too big, and the coupling between it and the specialized functions would be very tight.

The new `functools singledispatch` decorator in Python 3.4 allows each module to contribute to the overall solution, and lets you easily provide a specialized function even for classes that you can't edit. If you decorate a plain function with `@singledispatch`, it becomes a *generic function*: a group of functions to perform the same operation in different ways, depending on the type of the first argument.³ [Example 7-21](#) shows how.



`functools.singledispatch` was added in Python 3.4, but the [singledispatch package](#) available on PyPI is a backport compatible with Python 2.6 to 3.3.

3. This is what is meant by the term single-dispatch. If more arguments were used to select the specific functions, we'd have multiple-dispatch.

Example 7-21. singledispatch creates a custom htmlize.register to bundle several functions into a generic function

```
from functools import singledispatch
from collections import abc
import numbers
import html

@singledispatch ❶
def htmlize(obj):
    content = html.escape(repr(obj))
    return '<pre>{</pre>'.format(content)

@htmlize.register(str) ❷
def _(text): ❸
    content = html.escape(text).replace('\n', '<br>\n')
    return '<p>{</p>'.format(content)

@htmlize.register(numbers.Integral) ❹
def _(n):
    return '<pre>{<pre>'.format(n)

@htmlize.register(tuple) ❺
@htmlize.register(abc.MutableSequence)
def _(seq):
    inner = '</li>\n<li>'.join(htmlize(item) for item in seq)
    return '<ul>\n<li>' + inner + '</li>\n</ul>'
```

- ❶ @singledispatch marks the base function that handles the object type.
- ❷ Each specialized function is decorated with @«base_function».register(«type»).
- ❸ The name of the specialized functions is irrelevant; _ is a good choice to make this clear.
- ❹ For each additional type to receive special treatment, register a new function. numbers.Integral is a virtual superclass of int.
- ❺ You can stack several register decorators to support different types with the same function.

When possible, register the specialized functions to handle ABCs (abstract classes) such as numbers.Integral and abc.MutableSequence instead of concrete implementations like int and list. This allows your code to support a greater variety of compatible types. For example, a Python extension can provide alternatives to the int type with fixed bit lengths as subclasses of numbers.Integral.



Using ABCs for type checking allows your code to support existing or future classes that are either actual or virtual subclasses of those ABCs. The use of ABCs and the concept of a virtual subclass are subjects of [Chapter 11](#).

A notable quality of the `singledispatch` mechanism is that you can register specialized functions anywhere in the system, in any module. If you later add a module with a new user-defined type, you can easily provide a new custom function to handle that type. And you can write custom functions for classes that you did not write and can't change.

`singledispatch` is a well-thought-out addition to the standard library, and it offers more features than we can describe here. The best documentation for it is [PEP 443 — Single-dispatch generic functions](#).



`@singledispatch` is not designed to bring Java-style method overloading to Python. A single class with many overloaded variations of a method is better than a single function with a lengthy stretch of `if/elif/elif/elif` blocks. But both solutions are flawed because they concentrate too much responsibility in a single code unit—the class or the function. The advantage of `@singledispatch` is supporting modular extension: each module can register a specialized function for each type it supports.

Decorators are functions and therefore they may be composed (i.e., you can apply a decorator to a function that is already decorated, as shown in [Example 7-21](#)). The next section explains how that works.

Stacked Decorators

[Example 7-19](#) demonstrated the use of stacked decorators: `@lru_cache` is applied on the result of `@clock` over `fibonacci`. In [Example 7-21](#), the `@htmlize.register` decorator was applied twice to the last function in the module.

When two decorators `@d1` and `@d2` are applied to a function `f` in that order, the result is the same as `f = d1(d2(f))`.

In other words, this:

```
@d1
@d2
def f():
    print('f')
```

Is the same as:

```
def f():
    print('f')

f = d1(d2(f))
```

Besides stacked decorators, this chapter has shown some decorators that take arguments, for example, `@lru_cache()` and the `htmlize.register(«type»)` produced by `@singledispatch` in [Example 7-21](#). The next section shows how to build decorators that accept parameters.

Parameterized Decorators

When parsing a decorator in source code, Python takes the decorated function and passes it as the first argument to the decorator function. So how do you make a decorator accept other arguments? The answer is: make a decorator factory that takes those arguments and returns a decorator, which is then applied to the function to be decorated. Confusing? Sure. Let's start with an example based on the simplest decorator we've seen: `register` in [Example 7-22](#).

Example 7-22. Abridged `registration.py` module from [Example 7-2](#), repeated here for convenience

```
registry = []

def register(func):
    print('running register(%s)' % func)
    registry.append(func)
    return func

@register
def f1():
    print('running f1()')

print('running main()')
print('registry ->', registry)
f1()
```

A Parameterized Registration Decorator

In order to make it easy to enable or disable the function registration performed by `register`, we'll make it accept an optional `active` parameter which, if `False`, skips registering the decorated function. [Example 7-23](#) shows how. Conceptually, the new `register` function is not a decorator but a decorator factory. When called, it returns the actual decorator that will be applied to the target function.

Example 7-23. To accept parameters, the new register decorator must be called as a function

```
registry = set() ❶

def register(active=True): ❷
    def decorate(func): ❸
        print('running register(active=%s)->decorate(%s)'
              % (active, func))
        if active: ❹
            registry.add(func)
        else:
            registry.discard(func) ❺

        return func ❻
    return decorate ❼

@register(active=False) ❽
def f1():
    print('running f1()')

@register() ❾
def f2():
    print('running f2()')

def f3():
    print('running f3()')
```

- ❶ registry is now a set, so adding and removing functions is faster.
- ❷ register takes an optional keyword argument.
- ❸ The decorate inner function is the actual decorator; note how it takes a function as argument.
- ❹ Register func only if the active argument (retrieved from the closure) is True.
- ❺ If not active and func in registry, remove it.
- ❻ Because decorate is a decorator, it must return a function.
- ❼ register is our decorator factory, so it returns decorate.
- ❽ The @register factory must be invoked as a function, with the desired parameters.
- ❾ If no parameters are passed, register must still be called as a function—@register()—i.e., to return the actual decorator, decorate.

The main point is that register() returns decorate, which is then applied to the decorated function.

The code in [Example 7-23](#) is in a `registration_param.py` module. If we import it, this is what we get:

```
>>> import registration_param
running register(active=False)->decorate(<function f1 at 0x10063c1e0>)
running register(active=True)->decorate(<function f2 at 0x10063c268>)
>>> registration_param.registry
[<function f2 at 0x10063c268>]
```

Note how only the `f2` function appears in the registry; `f1` does not appear because `active=False` was passed to the `register` decorator factory, so the `decorate` that was applied to `f1` did not add it to the registry.

If, instead of using the `@` syntax, we used `register` as a regular function, the syntax needed to decorate a function `f` would be `register()(f)` to add `f` to the registry, or `register(active=False)(f)` to not add it (or remove it). See [Example 7-24](#) for a demo of adding and removing functions to the registry.

Example 7-24. Using the `registration_param` module listed in [Example 7-23](#)

```
>>> from registration_param import *
running register(active=False)->decorate(<function f1 at 0x10073c1e0>)
running register(active=True)->decorate(<function f2 at 0x10073c268>)
>>> registry # ❶
{<function f2 at 0x10073c268>}
>>> register()(f3) # ❷
running register(active=True)->decorate(<function f3 at 0x10073c158>)
<function f3 at 0x10073c158>
>>> registry # ❸
{<function f3 at 0x10073c158>, <function f2 at 0x10073c268>}
>>> register(active=False)(f2) # ❹
running register(active=False)->decorate(<function f2 at 0x10073c268>)
<function f2 at 0x10073c268>
>>> registry # ❺
{<function f3 at 0x10073c158>}
```

- ❶ When the module is imported, `f2` is in the registry.
- ❷ The `register()` expression returns `decorate`, which is then applied to `f3`.
- ❸ The previous line added `f3` to the registry.
- ❹ This call removes `f2` from the registry.
- ❺ Confirm that only `f3` remains in the registry.

The workings of parameterized decorators are fairly involved, and the one we've just discussed is simpler than most. Parameterized decorators usually replace the decorated function, and their construction requires yet another level of nesting. Touring such function pyramids is our next adventure.

The Parameterized Clock Decorator

In this section, we'll revisit the `clock` decorator, adding a feature: users may pass a format string to control the output of the decorated function. See [Example 7-25](#).



For simplicity, [Example 7-25](#) is based on the initial `clock` implementation from [Example 7-15](#), and not the improved one from [Example 7-17](#) that uses `@functools.wraps`, adding yet another function layer.

Example 7-25. Module `clockdeco_param.py`: the parameterized clock decorator

```
import time

DEFAULT_FMT = ' [{elapsed:0.8f}s] {name}({args}) -> {result}'

def clock(fmt=DEFAULT_FMT): ❶
    def decorate(func): ❷
        def clocked(*_args): ❸
            t0 = time.time()
            _result = func(*_args) ❹
            elapsed = time.time() - t0
            name = func.__name__
            args = ', '.join(repr(arg) for arg in _args) ❺
            result = repr(_result) ❻
            print(fmt.format(**locals())) ❼
            return _result ❽
        return clocked ❾
    return decorate ❿

if __name__ == '__main__':

    @clock() ⓫
    def snooze(seconds):
        time.sleep(seconds)

    for i in range(3):
        snooze(.123)
```

- ❶ `clock` is our parameterized decorator factory.
- ❷ `decorate` is the actual decorator.
- ❸ `clocked` wraps the decorated function.
- ❹ `_result` is the actual result of the decorated function.
- ❺ `_args` holds the actual arguments of `clocked`, while `args` is `str` used for display.
- ❻ `result` is the `str` representation of `_result`, for display.

- ⑦ Using `**locals()` here allows any local variable of `clocked` to be referenced in the `fmt`.
- ⑧ `clocked` will replace the decorated function, so it should return whatever that function returns.
- ⑨ `decorate` returns `clocked`.
- ⑩ `clock` returns `decorate`.
- ⑪ In this self test, `clock()` is called without arguments, so the decorator applied will use the default format `str`.

If you run **Example 7-25** from the shell, this is what you get:

```
$ python3 clockdeco_param.py
[0.12412500s] snooze(0.123) -> None
[0.12411904s] snooze(0.123) -> None
[0.12410498s] snooze(0.123) -> None
```

To exercise the new functionality, Examples 7-26 and 7-27 are two other modules using `clockdeco_param`, and the outputs they generate.

Example 7-26. `clockdeco_param_demo1.py`

```
import time
from clockdeco_param import clock

@clock('{name}: {elapsed}s')
def snooze(seconds):
    time.sleep(seconds)

for i in range(3):
    snooze(.123)
```

Output of **Example 7-26**:

```
$ python3 clockdeco_param_demo1.py
snooze: 0.12414693832397461s
snooze: 0.1241159439086914s
snooze: 0.12412118911743164s
```

Example 7-27. `clockdeco_param_demo2.py`

```
import time
from clockdeco_param import clock

@clock('{name}({args}) dt={elapsed:0.3f}s')
def snooze(seconds):
    time.sleep(seconds)

for i in range(3):
    snooze(.123)
```

Output of [Example 7-27](#):

```
$ python3 clockdeco_param_demo2.py
snooze(0.123) dt=0.124s
snooze(0.123) dt=0.124s
snooze(0.123) dt=0.124s
```

This ends our exploration of decorators as far as space permits within the scope of this book. See “[Further Reading](#)” on [page 212](#), in particular Graham Dumpleton’s blog and `wrapt` module for industrial-strength techniques when building decorators.



Graham Dumpleton and Lennart Regebro—one of this book’s technical reviewers—argue that decorators are best coded as classes implementing `__call__`, and not as functions like the examples in this chapter. I agree that approach is better for non-trivial decorators, but to explain the basic idea of this language feature, functions are easier to understand.

Chapter Summary

We covered a lot of ground in this chapter, but I tried to make the journey as smooth as possible even if the terrain is rugged. After all, we did enter the realm of metaprogramming.

We started with a simple `@register` decorator without an inner function, and finished with a parameterized `@clock()` involving two levels of nested functions.

Registration decorators, though simple in essence, have real applications in advanced Python frameworks. We applied the registration idea to an improvement of our Strategy design pattern refactoring from [Chapter 6](#).

Parameterized decorators almost always involve at least two nested functions, maybe more if you want to use `@functools.wraps` to produce a decorator that provides better support for more advanced techniques. One such technique is stacked decorators, which we briefly covered.

We also visited two awesome function decorators provided in the `functools` module of standard library: `@lru_cache()` and `@singledispatch`.

Understanding how decorators actually work required covering the difference between *import time* and *runtime*, then diving into variable scoping, closures, and the new `nonlocal` declaration. Mastering closures and `nonlocal` is valuable not only to build decorators, but also to code event-oriented programs for GUIs or asynchronous I/O with callbacks.

Further Reading

Chapter 9, “Metaprogramming,” of the *Python Cookbook, Third Edition* by David Beazley and Brian K. Jones (O’Reilly), has several recipes from elementary decorators to very sophisticated ones, including one that can be called as a regular decorator or as a decorator factory, e.g., `@clock` or `@clock()`. That’s “Recipe 9.6. Defining a Decorator That Takes an Optional Argument” in that cookbook.

Graham Dumpleton has a [series of in-depth blog posts](#) about techniques for implementing well-behaved decorators, starting with “[How You Implemented Your Python Decorator is Wrong](#)”. His deep expertise in this matter is also nicely packaged in the `wrapt` module he wrote to simplify the implementation of decorators and dynamic function wrappers, which support introspection and behave correctly when further decorated, when applied to methods and when used as descriptors. (Descriptors are the subject of chapter [Chapter 20](#).)

Michele Simionato authored a package aiming to “simplify the usage of decorators for the average programmer, and to popularize decorators by showing various non-trivial examples,” according to the docs. It’s available on PyPI as the [decorator package](#).

Created when decorators were still a new feature in Python, the [Python Decorator Library wiki page](#) has dozens of examples. Because that page started years ago, some of the techniques shown have been superseded, but the page is still an excellent source of inspiration.

[PEP 443](#) provides the rationale and a detailed description of the single-dispatch generic functions’ facility. An old (March 2005) blog post by Guido van Rossum, “[Five-Minute Multimethods in Python](#)”, walks through an implementation of generic functions (a.k.a. multimethods) using decorators. His code supports multiple-dispatch (i.e., dispatch based on more than one positional argument). Guido’s multimethods code is interesting, but it’s a didactic example. For a modern, production-ready implementation of multiple-dispatch generic functions, check out [Reg](#) by Martijn Faassen—author of the model-driven and REST-savvy [Morepath](#) web framework.

“[Closures in Python](#)” is a short blog post by Fredrik Lundh that explains the terminology of closures.

[PEP 3104 — Access to Names in Outer Scopes](#) describes the introduction of the `nonlocal` declaration to allow rebinding of names that are neither local nor global. It also includes an excellent overview of how this issue is resolved in other dynamic languages (Perl, Ruby, JavaScript, etc.) and the pros and cons of the design options available to Python.

On a more theoretical level, [PEP 227 — Statically Nested Scopes](#) documents the introduction of lexical scoping as an option in Python 2.1 and as a standard in Python 2.2,

explaining the rationale and design choices for the implementation of closures in Python.

Soapbox

The designer of any language with first-class functions faces this issue: being first-class objects, functions are defined in a certain scope but may be invoked in other scopes. The question is: how to evaluate the free variables? The first and simplest answer is “dynamic scope.” This means that free variables are evaluated by looking into the environment where the function is invoked.

If Python had dynamic scope and no closures, we could improvise `avg`—similar to [Example 7-9](#)—like this:

```
>>> ### this is not a real Python console session! ###
>>> avg = make_averager()
>>> series = [] # ❶
>>> avg(10)
10.0
>>> avg(11) # ❷
10.5
>>> avg(12)
11.0
>>> series = [1] # ❸
>>> avg(5)
3.0
```

- ❶ Before using `avg`, we have to define `series = []` ourselves, so we must know that `averager` (inside `make_averager`) refers to a list by that name.
- ❷ Behind the scenes, `series` is used to accumulate the values to be averaged.
- ❸ When `series = [1]` is executed, the previous list is lost. This could happen by accident, when handling two independent running averages at the same time.

Functions should be black boxes, with their implementation hidden from users. But with dynamic scope, if a function uses free variables, the programmer has to know its internals to set up an environment where it works correctly.

On the other hand, dynamic scope is easier to implement, which is probably why it was the path taken by John McCarthy when he created Lisp, the first language to have first-class functions. Paul Graham’s article “[The Roots of Lisp](#)” is an accessible explanation of John McCarthy’s original paper about the Lisp language: “[Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I](#)”. McCarthy’s paper is a masterpiece as great as Beethoven’s 9th Symphony. Paul Graham translated it for the rest of us, from mathematics to English and running code.

Paul Graham’s commentary also shows how tricky dynamic scoping is. Quoting from “The Roots of Lisp”:

It's an eloquent testimony to the dangers of dynamic scope that even the very first example of higher-order Lisp functions was broken because of it. It may be that McCarthy was not fully aware of the implications of dynamic scope in 1960. Dynamic scope remained in Lisp implementations for a surprisingly long time—until Sussman and Steele developed Scheme in 1975. Lexical scope does not complicate the definition of `eval` very much, but it may make compilers harder to write.

Today, lexical scope is the norm: free variables are evaluated considering the environment where the function is defined. Lexical scope complicates the implementation of languages with first-class functions, because it requires the support of closures. On the other hand, lexical scope makes source code easier to read. Most languages invented since Algol have lexical scope.

For many years, Python `lambdas` did not provide closures, contributing to the bad name of this feature among functional-programming geeks in the blogosphere. This was fixed in Python 2.2 (December 2001), but the blogosphere has a long memory. Since then, `lambda` is embarrassing only because of its limited syntax.

Python Decorators and the Decorator Design Pattern

Python function decorators fit the general description of Decorator given by Gamma et al. in *Design Patterns*: “Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.”

At the implementation level, Python decorators do not resemble the classic Decorator design pattern, but an analogy can be made.

In the design pattern, `Decorator` and `Component` are abstract classes. An instance of a concrete decorator wraps an instance of a concrete component in order to add behaviors to it. Quoting from *Design Patterns*:

The decorator conforms to the interface of the component it decorates so that its presence is transparent to the component's clients. The decorator forwards requests to the component and may perform additional actions (such as drawing a border) before or after forwarding. Transparency lets you nest decorators recursively, thereby allowing an unlimited number of added responsibilities.” (p. 175)

In Python, the decorator function plays the role of a concrete `Decorator` subclass, and the inner function it returns is a decorator instance. The returned function wraps the function to be decorated, which is analogous to the component in the design pattern. The returned function is transparent because it conforms to the interface of the component by accepting the same arguments. It forwards calls to the component and may perform additional actions either before or after it. Borrowing from the previous citation, we can adapt the last sentence to say that “Transparency lets you nest decorators recursively, thereby allowing an unlimited number of added behaviors.” That is what enable stacked decorators to work.

Note that I am not suggesting that function decorators should be used to implement the Decorator pattern in Python programs. Although this can be done in specific situations,

in general the Decorator pattern is best implemented with classes to represent the Decorator and the components it will wrap.

Object-Oriented Idioms

PNP Ch 37 | Decorators [companion]

Chapter 37: Decorators

Parameter	Details
f	The function to be decorated (wrapped)

Decorator functions are software design patterns. They dynamically alter the functionality of a function, method, or class without having to directly use subclasses or change the source code of the decorated function. When used correctly, decorators can become powerful tools in the development process. This topic covers implementation and applications of decorator functions in Python.

Section 37.1: Decorator function

Decorators augment the behavior of other functions or methods. Any function that takes a function as a parameter and returns an augmented function can be used as a **decorator**.

```
# This simplest decorator does nothing to the function being decorated. Such
# minimal decorators can occasionally be used as a kind of code markers.
def super_secret_function(f):
    return f

@super_secret_function
def my_function():
    print("This is my secret function.")
```

The @-notation is syntactic sugar that is equivalent to the following:

```
my_function = super_secret_function(my_function)
```

It is important to bear this in mind in order to understand how the decorators work. This "unsugared" syntax makes it clear why the decorator function takes a function as an argument, and why it should return another function. It also demonstrates what would happen if you *don't* return a function:

```
def disabled(f):
    """
    This function returns nothing, and hence removes the decorated function
    from the local scope.
    """
    pass

@disabled
def my_function():
    print("This function can no longer be called...")

my_function()
# TypeError: 'NoneType' object is not callable
```

Thus, we usually define a *new function* inside the decorator and return it. This new function would first do something that it needs to do, then call the original function, and finally process the return value. Consider this simple decorator function that prints the arguments that the original function receives, then calls it.

```
#This is the decorator
def print_args(func):
    def inner_func(*args, **kwargs):
        print(args)
        print(kwargs)
```



```

    return func(*args, **kwargs) #Call the original function with its arguments.
return inner_func

@print_args
def multiply(num_a, num_b):
    return num_a * num_b

print(multiply(3, 5))
#Output:
# (3,5) - This is actually the 'args' that the function receives.
# {} - This is the 'kwargs', empty because we didn't specify keyword arguments.
# 15 - The result of the function.

```

Section 37.2: Decorator class

As mentioned in the introduction, a decorator is a function that can be applied to another function to augment its behavior. The syntactic sugar is equivalent to the following: `my_func = decorator(my_func)`. But what if the decorator was instead a class? The syntax would still work, except that now `my_func` gets replaced with an instance of the decorator class. If this class implements the `__call__()` magic method, then it would still be possible to use `my_func` as if it was a function:

```

class Decorator(object):
    """Simple decorator class."""

    def __init__(self, func):
        self.func = func

    def __call__(self, *args, **kwargs):
        print('Before the function call.')
        res = self.func(*args, **kwargs)
        print('After the function call.')
        return res

@Decorator
def testfunc():
    print('Inside the function.')

testfunc()
# Before the function call.
# Inside the function.
# After the function call.

```

Note that a function decorated with a class decorator will no longer be considered a "function" from type-checking perspective:

```

import types
isinstance(testfunc, types.FunctionType)
# False
type(testfunc)
# <class '__main__.Decorator'>

```

Decorating Methods

For decorating methods you need to define an additional `__get__`-method:

```

from types import MethodType

class Decorator(object):
    def __init__(self, func):

```

```

    self.func = func

def __call__(self, *args, **kwargs):
    print('Inside the decorator.')
    return self.func(*args, **kwargs)

def __get__(self, instance, cls):
    # Return a Method if it is called on an instance
    return self if instance is None else MethodType(self, instance)

class Test(object):
    @Decorator
    def __init__(self):
        pass

a = Test()

```

Inside the decorator.

Warning!

Class Decorators only produce one instance for a specific function so decorating a method with a class decorator will share the same decorator between all instances of that class:

```

from types import MethodType

class CountCallsDecorator(object):
    def __init__(self, func):
        self.func = func
        self.ncalls = 0    # Number of calls of this method

    def __call__(self, *args, **kwargs):
        self.ncalls += 1    # Increment the calls counter
        return self.func(*args, **kwargs)

    def __get__(self, instance, cls):
        return self if instance is None else MethodType(self, instance)

class Test(object):
    def __init__(self):
        pass

    @CountCallsDecorator
    def do_something(self):
        return 'something was done'

a = Test()
a.do_something()
a.do_something.ncalls    # 1
b = Test()
b.do_something()
b.do_something.ncalls    # 2

```

Section 37.3: Decorator with arguments (decorator factory)

A decorator takes just one argument: the function to be decorated. There is no way to pass other arguments.

But additional arguments are often desired. The trick is then to make a function which takes arbitrary arguments

and returns a decorator.

Decorator functions

```
def decoratorfactory(message):
    def decorator(func):
        def wrapped_func(*args, **kwargs):
            print('The decorator wants to tell you: {}'.format(message))
            return func(*args, **kwargs)
        return wrapped_func
    return decorator

@decoratorfactory('Hello World')
def test():
    pass

test()
```

The decorator wants to tell you: Hello World

Important Note:

With such decorator factories you **must** call the decorator with a pair of parentheses:

```
@decoratorfactory # Without parentheses
def test():
    pass

test()
```

TypeError: decorator() missing 1 required positional argument: 'func'

Decorator classes

```
def decoratorfactory(*decorator_args, **decorator_kwargs):

    class Decorator(object):
        def __init__(self, func):
            self.func = func

        def __call__(self, *args, **kwargs):
            print('Inside the decorator with arguments {}'.format(decorator_args))
            return self.func(*args, **kwargs)

    return Decorator

@decoratorfactory(10)
def test():
    pass

test()
```

Inside the decorator with arguments (10,)

Section 37.4: Making a decorator look like the decorated function

Decorators normally strip function metadata as they aren't the same. This can cause problems when using meta-programming to dynamically access function metadata. Metadata also includes function's docstrings and its name. [functools.wraps](#) makes the decorated function look like the original function by copying several attributes to the wrapper function.

```
from functools import wraps
```

The two methods of wrapping a decorator are achieving the same thing in hiding that the original function has been decorated. There is no reason to prefer the function version to the class version unless you're already using one over the other.

As a function

```
def decorator(func):
    # Copies the docstring, name, annotations and module to the decorator
    @wraps(func)
    def wrapped_func(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapped_func

@decorator
def test():
    pass

test.__name__
```

```
'test'
```

As a class

```
class Decorator(object):
    def __init__(self, func):
        # Copies name, module, annotations and docstring to the instance.
        self._wrapped = wraps(func)(self)

    def __call__(self, *args, **kwargs):
        return self._wrapped(*args, **kwargs)

@Decorator
def test():
    """Docstring of test."""
    pass

test.__doc__
```

```
'Docstring of test.'
```

Section 37.5: Using a decorator to time a function

```
import time
def timer(func):
    def inner(*args, **kwargs):
```

```

    t1 = time.time()
    f = func(*args, **kwargs)
    t2 = time.time()
    print 'Runtime took {0} seconds'.format(t2-t1)
    return f
return inner

@timer
def example_function():
    #do stuff

example_function()

```

Section 37.6: Create singleton class with a decorator

A singleton is a pattern that restricts the instantiation of a class to one instance/object. Using a decorator, we can define a class as a singleton by forcing the class to either return an existing instance of the class or create a new instance (if it doesn't exist).

```

def singleton(cls):
    instance = [None]
    def wrapper(*args, **kwargs):
        if instance[0] is None:
            instance[0] = cls(*args, **kwargs)
        return instance[0]
    return wrapper

```

This decorator can be added to any class declaration and will make sure that at most one instance of the class is created. Any subsequent calls will return the already existing class instance.

```

@singleton
class SomeSingletonClass:
    x = 2
    def __init__(self):
        print("Created!")

instance = SomeSingletonClass() # prints: Created!
instance = SomeSingletonClass() # doesn't print anything
print(instance.x)               # 2

instance.x = 3
print(SomeSingletonClass().x)   # 3

```

So it doesn't matter whether you refer to the class instance via your local variable or whether you create another "instance", you always get the same object.

PNP Ch 43 | Importing Modules & importlib [companion]

Chapter 43: Importing modules

Section 43.1: Importing a module

Use the **import** statement:

```
>>> import random
>>> print(random.randint(1, 10))
4
```

import module will import a module and then allow you to reference its objects -- values, functions and classes, for example -- using the `module.name` syntax. In the above example, the **random** module is imported, which contains the `randint` function. So by importing **random** you can call `randint` with `random.randint`.

You can import a module and assign it to a different name:

```
>>> import random as rn
>>> print(rn.randint(1, 10))
4
```

If your python file `main.py` is in the same folder as `custom.py`. You can import it like this:

```
import custom
```

It is also possible to import a function from a module:

```
>>> from math import sin
>>> sin(1)
0.8414709848078965
```

To import specific functions deeper down into a module, the dot operator may be used **only** on the left side of the **import** keyword:

```
from urllib.request import urlopen
```

In python, we have two ways to call function from top level. One is **import** and another is **from**. We should use **import** when we have a possibility of name collision. Suppose we have `hello.py` file and `world.py` files having same function named `function`. Then **import** statement will work good.

```
from hello import function
from world import function

function() #world's function will be invoked. Not hello's
```

In general **import** will provide you a namespace.

```
import hello
import world

hello.function() # exclusively hello's function will be invoked
world.function() # exclusively world's function will be invoked
```

But if you are sure enough, in your whole project there is no way having same function name you should use **from** statement

Multiple imports can be made on the same line:

```
>>> # Multiple modules
>>> import time, sockets, random
>>> # Multiple functions
>>> from math import sin, cos, tan
>>> # Multiple constants
>>> from math import pi, e

>>> print(pi)
3.141592653589793
>>> print(cos(45))
0.5253219888177297
>>> print(time.time())
1482807222.7240417
```

The keywords and syntax shown above can also be used in combinations:

```
>>> from urllib.request import urlopen as geturl, pathname2url as path2url, getproxies
>>> from math import factorial as fact, gamma, atan as arctan
>>> import random.randint, time, sys

>>> print(time.time())
1482807222.7240417
>>> print(arctan(60))
1.554131203080956
>>> filepath = "/dogs/jumping poodle (december).png"
>>> print(path2url(filepath))
/dogs/jumping%20poodle%20%28december%29.png
```

Section 43.2: The `__all__` special variable

Modules can have a special variable named `__all__` to restrict what variables are imported when using `from mymodule import *`.

Given the following module:

```
# mymodule.py

__all__ = ['imported_by_star']

imported_by_star = 42
not_imported_by_star = 21
```

Only `imported_by_star` is imported when using `from mymodule import *`:

```
>>> from mymodule import *
>>> imported_by_star
42
>>> not_imported_by_star
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'not_imported_by_star' is not defined
```

However, `not_imported_by_star` can be imported explicitly:

```
>>> from mymodule import not_imported_by_star
>>> not_imported_by_star
```


Section 43.3: Import modules from an arbitrary filesystem location

If you want to import a module that doesn't already exist as a built-in module in the [Python Standard Library](#) nor as a side-package, you can do this by adding the path to the directory where your module is found to `sys.path`. This may be useful where multiple python environments exist on a host.

```
import sys
sys.path.append("/path/to/directory/containing/your/module")
import mymodule
```

It is important that you append the path to the *directory* in which `mymodule` is found, not the path to the module itself.

Section 43.4: Importing all names from a module

```
from module_name import *
```

for example:

```
from math import *
sqrt(2)      # instead of math.sqrt(2)
ceil(2.7)    # instead of math.ceil(2.7)
```

This will import all names defined in the `math` module into the global namespace, other than names that begin with an underscore (which indicates that the writer feels that it is for internal use only).

Warning: If a function with the same name was already defined or imported, it will be **overwritten**. Almost always importing only specific names `from math import sqrt, ceil` is the **recommended way**:

```
def sqrt(num):
    print("I don't know what's the square root of {}".format(num))

sqrt(4)
# Output: I don't know what's the square root of 4.

from math import *
sqrt(4)
# Output: 2.0
```

Starred imports are only allowed at the module level. Attempts to perform them in class or function definitions result in a `SyntaxError`.

```
def f():
    from math import *
```

and

```
class A:
    from math import *
```

both fail with:

SyntaxError: `import *` only allowed at module level

Section 43.5: Programmatic importing

Python 2.x Version ≥ 2.7

To import a module through a function call, use the `importlib` module (included in Python starting in version 2.7):

```
import importlib
random = importlib.import_module("random")
```

The `importlib.import_module()` function will also import the submodule of a package directly:

```
collections_abc = importlib.import_module("collections.abc")
```

For older versions of Python, use the `imp` module.

Python 2.x Version ≤ 2.7

Use the functions `imp.find_module` and `imp.load_module` to perform a programmatic import.

Taken from [standard library documentation](#)

```
import imp, sys
def import_module(name):
    fp, pathname, description = imp.find_module(name)
    try:
        return imp.load_module(name, fp, pathname, description)
    finally:
        if fp:
            fp.close()
```

Do **NOT** use `__import__()` to programmatically import modules! There are subtle details involving `sys.modules`, the `fromlist` argument, etc. that are easy to overlook which `importlib.import_module()` handles for you.

Section 43.6: PEP8 rules for Imports

Some recommended [PEP8](#) style guidelines for imports:

1. Imports should be on separate lines:

```
from math import sqrt, ceil    # Not recommended
from math import sqrt          # Recommended
from math import ceil
```

2. Order imports as follows at the top of the module:

- Standard library imports
- Related third party imports
- Local application/library specific imports

3. Wildcard imports should be avoided as it leads to confusion in names in the current namespace. If you do

`from module import *`, it can be unclear if a specific name in your code comes from `module` or not. This is

doubly true if you have multiple **from** module **import** *-type statements.

4. Avoid using relative imports; use explicit imports instead.

Section 43.7: Importing specific names from a module

Instead of importing the complete module you can import only specified names:

```
from random import randint # Syntax "from MODULENAME import NAME1[, NAME2[, ...]]"
print(randint(1, 10))      # Out: 5
```

from random is needed, because the python interpreter has to know from which resource it should import a function or class and **import randint** specifies the function or class itself.

Another example below (similar to the one above):

```
from math import pi
print(pi)                    # Out: 3.14159265359
```

The following example will raise an error, because we haven't imported a module:

```
random.randrange(1, 10)     # works only if "import random" has been run before
```

Outputs:

```
NameError: name 'random' is not defined
```

The python interpreter does not understand what you mean with **random**. It needs to be declared by adding **import random** to the example:

```
import random
random.randrange(1, 10)
```

Section 43.8: Importing submodules

```
from module.submodule import function
```

This imports function from module.submodule.

Section 43.9: Re-importing a module

When using the interactive interpreter, you might want to reload a module. This can be useful if you're editing a module and want to import the newest version, or if you've monkey-patched an element of an existing module and want to revert your changes.

Note that you **can't** just **import** the module again to revert:

```
import math
math.pi = 3
print(math.pi)    # 3
import math
print(math.pi)    # 3
```

This is because the interpreter registers every module you import. And when you try to reimport a module, the interpreter sees it in the register and does nothing. So the hard way to reimport is to use **import** after removing the corresponding item from the register:

```
print(math.pi)    # 3
import sys
if 'math' in sys.modules: # Is the `math` module in the register?
    del sys.modules['math'] # If so, remove it.
import math
print(math.pi)    # 3.141592653589793
```

But there is more a straightforward and simple way.

Python 2

Use the **reload** function:

Python 2.x Version $\geq$ 2.3

```
import math
math.pi = 3
print(math.pi)    # 3
reload(math)
print(math.pi)    # 3.141592653589793
```

Python 3

The **reload** function has moved to **importlib**:

Python 3.x Version $\geq$ 3.0

```
import math
math.pi = 3
print(math.pi)    # 3
from importlib import reload
reload(math)
print(math.pi)    # 3.141592653589793
```

Section 43.10: `__import__()` function

The `__import__()` function can be used to import modules where the name is only known at runtime

```
if user_input == "os":
    os = __import__("os")

# equivalent to import os
```

This function can also be used to specify the file path to a module

```
mod = __import__(r"C:/path/to/file/anywhere/on/computer/module.py")
```

PNP Ch 44 | Modules vs Packages [companion]

Chapter 4 4: Difference between Module and Package

Section 4 4.1: Modules

A module is a single Python file that can be imported. Using a module looks like this:

module.py

```
def hi():  
    print("Hello world!")
```

my_script.py

```
import module  
module.hi()
```

in an interpreter

```
>>> from module import hi  
>>> hi()  
# Hello world!
```

Section 4 4.2: Packages

A package is made up of multiple Python files (or modules), and can even include libraries written in C or C++. Instead of being a single file, it is an entire folder structure which might look like this:

Folder package

- `__init__.py`
- `dog.py`
- `hi.py`

`__init__.py`

```
from package.dog import woof  
from package.hi import hi
```

`dog.py`

```
def woof():  
    print("WOOF!!!")
```

`hi.py`

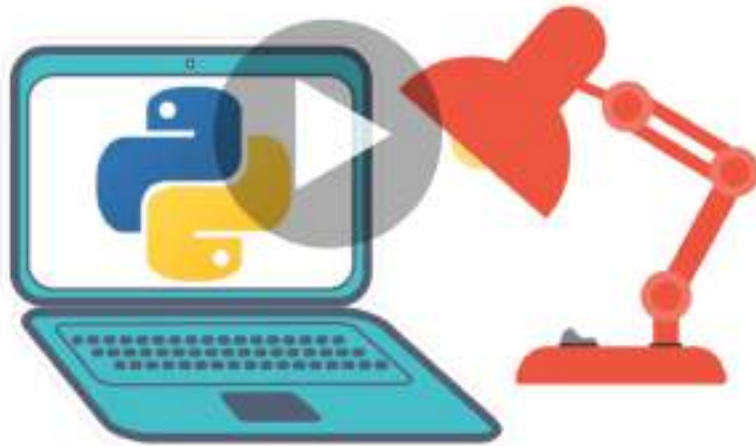
```
def hi():  
    print("Hello world!")
```

All Python packages must contain an `__init__.py` file. When you import a package in your script (`import package`), the `__init__.py` script will be run, giving you access to the all of the functions in the package. In this case, it allows you to use the `package.hi` and `package.woof` functions.

VIDEO: Complete Python Bootcamp: Go from zero to hero in Python 3

Learn Python like a Professional! Start from the basics and go all the way to creating your own applications and games!

COMPLETE PYTHON 3 BOOTCAMP



- ✓ Learn to use Python professionally, learning both Python 2 and Python 3!
- ✓ Create games with Python, like Tic Tac Toe and Blackjack!
- ✓ Learn advanced Python features, like the collections module and how to work with timestamps!
- ✓ Learn to use Object Oriented Programming with classes!
- ✓ Understand complex topics, like decorators.
- ✓ Understand how to use both the Jupyter Notebook and create .py files
- ✓ Get an understanding of how to create GUIs in the Jupyter Notebook system!
- ✓ Build a complete understanding of Python from the ground up!

Watch Today →

PNP Ch 56 | Functools Module [companion]

Chapter 56: Functools Module

Section 56.1: partial

The `partial` function creates partial function application from another function. It is used to *bind* values to some of the function's arguments (or keyword arguments) and produce a *callable* without the already defined arguments.

```
>>> from functools import partial
>>> unhex = partial(int, base=16)
>>> unhex.__doc__ = 'Convert base16 string to int'
>>> unhex('ca11ab1e')
3390155550
```

`partial()`, as the name suggests, allows a partial evaluation of a function. Let's look at following example:

```
In [2]: from functools import partial

In [3]: def f(a, b, c, x):
...:     return 1000*a + 100*b + 10*c + x
...:

In [4]: g = partial(f, 1, 1, 1)

In [5]: print g(2)
1112
```

When `g` is created, `f`, which takes four arguments(`a`, `b`, `c`, `x`), is also partially evaluated for the first three arguments, `a`, `b`, `c`,. Evaluation of `f` is completed when `g` is called, `g(2)`, which passes the fourth argument to `f`.

One way to think of `partial` is a shift register; pushing in one argument at the time into some function. `partial` comes handy for cases where data is coming in as stream and we cannot pass more than one argument.

Section 56.2: cmp_to_key

Python changed its sorting methods to accept a key function. Those functions take a value and return a key which is used to sort the arrays.

Old comparison functions used to take two values and return -1, 0 or +1 if the first argument is small, equal or greater than the second argument respectively. This is incompatible to the new key-function.

That's where `functools.cmp_to_key` comes in:

```
>>> import functools
>>> import locale
>>> sorted(["A", "S", "F", "D"], key=functools.cmp_to_key(locale.strcoll))
['A', 'D', 'F', 'S']
```

Example taken and adapted from the [Python Standard Library Documentation](#).

Section 56.3: lru_cache

The `@lru_cache` decorator can be used wrap an expensive, computationally-intensive function with a [Least Recently Used](#) cache. This allows function calls to be memoized, so that future calls with the same parameters can return instantly instead of having to be recomputed.

```
@lru_cache(maxsize=None) # Boundless cache
def fibonacci(n):
    if n < 2:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

>>> fibonacci(15)
```

In the example above, the value of `fibonacci(3)` is only calculated once, whereas if `fibonacci` didn't have an LRU cache, `fibonacci(3)` would have been computed upwards of 230 times. Hence, `@lru_cache` is especially great for recursive functions or dynamic programming, where an expensive function could be called multiple times with the same exact parameters.

`@lru_cache` has two arguments

- `maxsize`: Number of calls to save. When the number of unique calls exceeds `maxsize`, the LRU cache will remove the least recently used calls.
- `typed` (added in 3.3): Flag for determining if equivalent arguments of different types belong to different cache records (i.e. if `3.0` and `3` count as different arguments)

We can see cache stats too:

```
>>> fib.cache_info()
CacheInfo(hits=13, misses=16, maxsize=None, cursize=16)
```

NOTE: Since `@lru_cache` uses dictionaries to cache results, all parameters for the function must be hashable for the cache to work.

[Official Python docs for @lru_cache](#). `@lru_cache` was added in 3.2.

Section 56.4: total_ordering

When we want to create an orderable class, normally we need to define the methods `__eq__()`, `__lt__()`, `__le__()`, `__gt__()` and `__ge__()`.

The `total_ordering` decorator, applied to a class, permits the definition of `__eq__()` and only one between `__lt__()`, `__le__()`, `__gt__()` and `__ge__()`, and still allow all the ordering operations on the class.

```
@total_ordering
class Employee:
    ...

    def __eq__(self, other):
        return ((self.surname, self.name) == (other.surname, other.name))

    def __lt__(self, other):
        return ((self.surname, self.name) < (other.surname, other.name))
```

The decorator uses a composition of the provided methods and algebraic operations to derive the other comparison methods. For example if we defined `__lt__()` and `__eq__()` and we want to derive `__gt__()`, we can simply check `not __lt__()` and `not __eq__()`.

Note: The `total_ordering` function is only available since Python 2.7.

Section 56.5: reduce

In Python 3.x, the `reduce` function already explained here has been removed from the built-ins and must now be imported from `functools`.

```
from functools import reduce
def factorial(n):
    return reduce(lambda a, b: (a*b), range(1, n+1))
```

PNP Ch 87 | Type Hints [companion · FP Ch 8 substitute]

Chapter 87: Type Hints

Section 87.1: Adding types to a function

Let's take an example of a function which receives two arguments and returns a value indicating their sum:

```
def two_sum(a, b):  
    return a + b
```

By looking at this code, one can not safely and without doubt indicate the type of the arguments for function `two_sum`. It works both when supplied with `int` values:

```
print(two_sum(2, 1)) # result: 3
```

and with strings:

```
print(two_sum("a", "b")) # result: "ab"
```

and with other values, such as `lists`, `tuples` et cetera.

Due to this dynamic nature of python types, where many are applicable for a given operation, any type checker would not be able to reasonably assert whether a call for this function should be allowed or not.

To assist our type checker we can now provide type hints for it in the Function definition indicating the type that we allow.

To indicate that we only want to allow `int` types we can change our function definition to look like:

```
def two_sum(a: int, b: int):  
    return a + b
```

Annotations follow the argument name and are separated by a `:` character.

Similarly, to indicate only `str` types are allowed, we'd change our function to specify it:

```
def two_sum(a: str, b: str):  
    return a + b
```

Apart from specifying the type of the arguments, one could also indicate the return value of a function call. This is done by adding the `->` character followed by the type after the closing parenthesis in the argument list *but* before the `:` at the end of the function declaration:

```
def two_sum(a: int, b: int) -> int:  
    return a + b
```

Now we've indicated that the return value when calling `two_sum` should be of type `int`. Similarly we can define appropriate values for `str`, `float`, `list`, `set` and others.

Although type hints are mostly used by type checkers and IDEs, sometimes you may need to retrieve them. This can be done using the `__annotations__` special attribute:

```
two_sum.__annotations__  
# {'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
```

Section 87.2: NamedTuple

Creating a namedtuple with type hints is done using the function `NamedTuple` from the `typing` module:

```
import typing
Point = typing.NamedTuple('Point', [('x', int), ('y', int)])
```

Note that the name of the resulting type is the first argument to the function, but it should be assigned to a variable with the same name to ease the work of type checkers.

Section 87.3: Generic Types

The `typing.TypeVar` is a generic type factory. Its primary goal is to serve as a parameter/placeholder for generic function/class/method annotations:

```
import typing

T = typing.TypeVar("T")

def get_first_element(l: typing.Sequence[T]) -> T:
    """Gets the first element of a sequence."""
    return l[0]
```

Section 87.4: Variables and Attributes

Variables are annotated using comments:

```
x = 3 # type: int
x = negate(x)
x = 'a type-checker might catch this error'
```

Python 3.x Version ≥ 3.6

Starting from Python 3.6, there is also [new syntax for variable annotations](#). The code above might use the form

```
x: int = 3
```

Unlike with comments, it is also possible to just add a type hint to a variable that was not previously declared, without setting a value to it:

```
y: int
```

Additionally if these are used in the module or the class level, the type hints can be retrieved using `typing.get_type_hints(class_or_module)`:

```
class Foo:
    x: int
    y: str = 'abc'

print(typing.get_type_hints(Foo))
# ChainMap({'x': <class 'int'>, 'y': <class 'str'>}, {})
```

Alternatively, they can be accessed by using the `__annotations__` special variable or attribute:

```
x: int
print(__annotations__)
```

```
# {'x': <class 'int'>}

class C:
    s: str
print(C.__annotations__)
# {'s': <class 'str'>}
```

Section 87.5: Class Members and Methods

```
class A:
    x = None # type: float
    def __init__(self, x: float) -> None:
        """
        self should not be annotated
        init should be annotated to return None
        """
        self.x = x

    @classmethod
    def from_int(cls, x: int) -> 'A':
        """
        cls should not be annotated
        Use forward reference to refer to current class with string literal 'A'
        """
        return cls(float(x))
```

Forward reference of the current class is needed since annotations are evaluated when the function is defined. Forward references can also be used when referring to a class that would cause a circular import if imported.

Section 87.6: Type hints for keyword arguments

```
def hello_world(greeting: str = 'Hello'):
    print(greeting + ' world!')
```

Note the spaces around the equal sign as opposed to how keyword arguments are usually styled.

PHASE 04 · Iterators, Numerical Computing & Compilation

~4 weeks · 5 chapters

Build memory-efficient pipelines, master NumPy/Pandas, escape Python's speed ceiling.

Chapters in this phase:

- FP Ch 14 | Iterables, Iterators & Generators [= plan Ch 17]
- HPP Ch 5 | Iterators and Generators (performance)
- HPP Ch 6 | Matrix and Vector Computation
- HPP Ch 11 | Using Less RAM
- HPP Ch 7 | Compiling to C
- PNP Ch 53 | Itertools Module [companion]
- PNP Ch 67 | Generators [companion]

FP Ch 14 | Iterables, Iterators & Generators [= plan Ch 17]

Iterables, Iterators, and Generators

When I see patterns in my programs, I consider it a sign of trouble. The shape of a program should reflect only the problem it needs to solve. Any other regularity in the code is a sign, to me at least, that I'm using abstractions that aren't powerful enough—often that I'm generating by hand the expansions of some macro that I need to write.¹

— Paul Graham
Lisp hacker and venture capitalist

Iteration is fundamental to data processing. And when scanning datasets that don't fit in memory, we need a way to fetch the items *lazily*, that is, one at a time and on demand. This is what the Iterator pattern is about. This chapter shows how the Iterator pattern is built into the Python language so you never need to implement it by hand.

Python does not have macros like Lisp (Paul Graham's favorite language), so abstracting away the Iterator pattern required changing the language: the `yield` keyword was added in Python 2.2 (2001).² The `yield` keyword allows the construction of generators, which work as iterators.



Every generator is an iterator: generators fully implement the iterator interface. But an iterator—as defined in the GoF book—retrieves items from a collection, while a generator can produce items “out of thin air.” That’s why the Fibonacci sequence generator is a common example: an infinite series of numbers cannot be stored in a collection. However, be aware that the Python community treats *iterator* and *generator* as synonyms most of the time.

1. From “[Revenge of the Nerds](#)”, a blog post.
2. Python 2.2 users could use `yield` with the directive `from __future__ import generators`; `yield` became available by default in Python 2.3.

Python 3 uses generators in many places. Even the `range()` built-in now returns a generator-like object instead of full-blown lists like before. If you must build a list from range, you have to be explicit (e.g., `list(range(100))`).

Every collection in Python is *iterable*, and iterators are used internally to support:

- for loops
- Collection types construction and extension
- Looping over text files line by line
- List, dict, and set comprehensions
- Tuple unpacking
- Unpacking actual parameters with `*` in function calls

This chapter covers the following topics:

- How the `iter(...)` built-in function is used internally to handle iterable objects
- How to implement the classic Iterator pattern in Python
- How a generator function works in detail, with line-by-line descriptions
- How the classic Iterator can be replaced by a generator function or generator expression
- Leveraging the general-purpose generator functions in the standard library
- Using the new `yield from` statement to combine generators
- A case study: using generator functions in a database conversion utility designed to work with large datasets
- Why generators and coroutines look alike but are actually very different and should not be mixed

We'll get started studying how the `iter(...)` function makes sequences iterable.

Sentence Take #1: A Sequence of Words

We'll start our exploration of iterables by implementing a `Sentence` class: you give its constructor a string with some text, and then you can iterate word by word. The first version will implement the sequence protocol, and it's iterable because all sequences are iterable, as we've seen before, but now we'll see exactly why.

Example 14-1 shows a `Sentence` class that extracts words from a text by index.

Example 14-1. sentence.py: A Sentence as a sequence of words

```
import re
import reprlib

RE_WORD = re.compile('\w+')

class Sentence:

    def __init__(self, text):
        self.text = text
        self.words = RE_WORD.findall(text) ❶

    def __getitem__(self, index):
        return self.words[index] ❷

    def __len__(self): ❸
        return len(self.words)

    def __repr__(self):
        return 'Sentence(%s)' % reprlib.repr(self.text) ❹
```

- ❶ `re.findall` returns a list with all nonoverlapping matches of the regular expression, as a list of strings.
- ❷ `self.words` holds the result of `.findall`, so we simply return the word at the given index.
- ❸ To complete the sequence protocol, we implement `__len__`—but it is not needed to make an iterable object.
- ❹ `reprlib.repr` is a utility function to generate abbreviated string representations of data structures that can be very large.³

By default, `reprlib.repr` limits the generated string to 30 characters. See the console session in [Example 14-2](#) to see how `Sentence` is used.

Example 14-2. Testing iteration on a `Sentence` instance

```
>>> s = Sentence('"The time has come," the Walrus said,') # ❶
>>> s
Sentence('"The time ha... Walrus said,') # ❷
>>> for word in s: # ❸
...     print(word)
The
time
has
come
```

3. We first used `reprlib` in “Vector Take #1: Vector2d Compatible” on page 276.

```

the
Walrus
said
>>> list(s) # ④
['The', 'time', 'has', 'come', 'the', 'Walrus', 'said']

```

- ❶ A sentence is created from a string.
- ❷ Note the output of `__repr__` using `...` generated by `reprlib.repr`.
- ❸ Sentence instances are iterable; we'll see why in a moment.
- ❹ Being iterable, Sentence objects can be used as input to build lists and other iterable types.

In the following pages, we'll develop other Sentence classes that pass the tests in [Example 14-2](#). However, the implementation in [Example 14-1](#) is different from all the others because it's also a sequence, so you can get words by index:

```

>>> s[0]
'The'
>>> s[5]
'Walrus'
>>> s[-1]
'said'

```

Every Python programmer knows that sequences are iterable. Now we'll see precisely why.

Why Sequences Are Iterable: The `iter` Function

Whenever the interpreter needs to iterate over an object `x`, it automatically calls `iter(x)`.

The `iter` built-in function:

1. Checks whether the object implements `__iter__`, and calls that to obtain an iterator.
2. If `__iter__` is not implemented, but `__getitem__` is implemented, Python creates an iterator that attempts to fetch items in order, starting from index 0 (zero).
3. If that fails, Python raises `TypeError`, usually saying “C object is not iterable,” where C is the class of the target object.

That is why any Python sequence is iterable: they all implement `__getitem__`. In fact, the standard sequences also implement `__iter__`, and yours should too, because the special handling of `__getitem__` exists for backward compatibility reasons and may be gone in the future (although it is not deprecated as I write this).

As mentioned in “[Python Digs Sequences](#)” on page 310, this is an extreme form of duck typing: an object is considered iterable not only when it implements the special method

`__iter__`, but also when it implements `__getitem__`, as long as `__getitem__` accepts int keys starting from 0.

In the goose-typing approach, the definition for an iterable is simpler but not as flexible: an object is considered iterable if it implements the `__iter__` method. No subclassing or registration is required, because `abc.Iterable` implements the `__subclasshook__`, as seen in “[Geese Can Behave as Ducks](#)” on page 338. Here is a demonstration:

```
>>> class Foo:
...     def __iter__(self):
...         pass
...
>>> from collections import abc
>>> issubclass(Foo, abc.Iterable)
True
>>> f = Foo()
>>> isinstance(f, abc.Iterable)
True
```

However, note that our initial `Sentence` class does not pass the `issubclass(Sentence, abc.Iterable)` test, even though it is iterable in practice.



As of Python 3.4, the most accurate way to check whether an object `x` is iterable is to call `iter(x)` and handle a `TypeError` exception if it isn't. This is more accurate than using `isinstance(x, abc.Iterable)`, because `iter(x)` also considers the legacy `__getitem__` method, while the `Iterable` ABC does not.

Explicitly checking whether an object is iterable may not be worthwhile if right after the check you are going to iterate over the object. After all, when the iteration is attempted on a noniterable, the exception Python raises is clear enough: `TypeError: 'C' object is not iterable`. If you can do better than just raising `TypeError`, then do so in a `try/except` block instead of doing an explicit check. The explicit check may make sense if you are holding on to the object to iterate over it later; in this case, catching the error early may be useful.

The next section makes explicit the relationship between iterables and iterators.

Iterables Versus Iterators

From the explanation in “[Why Sequences Are Iterable: The `iter` Function](#)” on page 404 we can extrapolate a definition:

iterable

Any object from which the `iter` built-in function can obtain an iterator. Objects implementing an `__iter__` method returning an *iterator* are iterable. Sequences

are always iterable; as are objects implementing a `__getitem__` method that takes 0-based indexes.

It's important to be clear about the relationship between iterables and iterators: Python obtains iterators from iterables.

Here is a simple for loop iterating over a `str`. The `str` `'ABC'` is the iterable here. You don't see it, but there is an iterator behind the curtain:

```
>>> s = 'ABC'
>>> for char in s:
...     print(char)
...
A
B
C
```

If there was no `for` statement and we had to emulate the `for` machinery by hand with a `while` loop, this is what we'd have to write:

```
>>> s = 'ABC'
>>> it = iter(s) # ❶
>>> while True:
...     try:
...         print(next(it)) # ❷
...     except StopIteration: # ❸
...         del it # ❹
...         break # ❺
...
A
B
C
```

- ❶ Build an iterator `it` from the iterable.
- ❷ Repeatedly call `next` on the iterator to obtain the next item.
- ❸ The iterator raises `StopIteration` when there are no further items.
- ❹ Release reference to `it`—the iterator object is discarded.
- ❺ Exit the loop.

`StopIteration` signals that the iterator is exhausted. This exception is handled internally in `for` loops and other iteration contexts like list comprehensions, tuple unpacking, etc.

The standard interface for an iterator has two methods:

`__next__`

Returns the next available item, raising `StopIteration` when there are no more items.

`__iter__`

Returns `self`; this allows iterators to be used where an iterable is expected, for example, in a `for` loop.

This is formalized in the `collections.abc.Iterator` ABC, which defines the `__next__` abstract method, and subclasses `Iterable`—where the abstract `__iter__` method is defined. See [Figure 14-1](#).

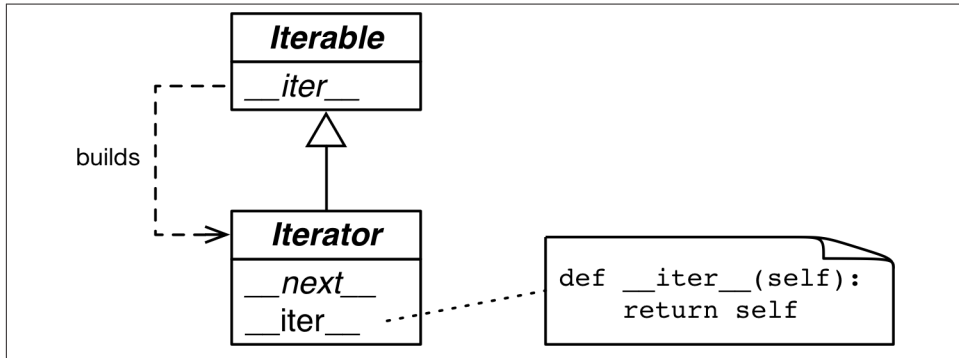


Figure 14-1. The `Iterable` and `Iterator` ABCs. Methods in italics are abstract. A concrete `Iterable.iter` should return a new `Iterator` instance. A concrete `Iterator` must implement `next`. The `Iterator.iter` method just returns the instance itself.

The `Iterator` ABC implements `__iter__` by doing `return self`. This allows an iterator to be used wherever an iterable is required. The source code for `abc.Iterator` is in [Example 14-3](#).

Example 14-3. `abc.Iterator` class; extracted from `Lib/_collections_abc.py`

```
class Iterator(Iterable):

    __slots__ = ()

    @abstractmethod
    def __next__(self):
        'Return the next item from the iterator. When exhausted, raise StopIteration'
        raise StopIteration

    def __iter__(self):
        return self

    @classmethod
    def __subclasshook__(cls, C):
        if cls is Iterator:
            if (any("__next__" in B.__dict__ for B in C.__mro__) and
                any("__iter__" in B.__dict__ for B in C.__mro__)):
                return True
            return False
```



```
return True
return NotImplemented
```



The Iterator ABC abstract method is `it.__next__()` in Python 3 and `it.next()` in Python 2. As usual, you should avoid calling special methods directly. Just use the `next(it)`; this built-in function does the right thing in Python 2 and 3.

The *Lib/types.py* module source code in Python 3.4 has a comment that says:

```
# Iterators in Python aren't a matter of type but of protocol. A large
# and changing number of builtin types implement *some* flavor of
# iterator. Don't check the type! Use hasattr to check for both
# "__iter__" and "__next__" attributes instead.
```

In fact, that's exactly what the `__subclasshook__` method of the `abc.Iterator` ABC does (see [Example 14-3](#)).



Taking into account the advice from *Lib/types.py* and the logic implemented in *Lib/collections_abc.py*, the best way to check if an object `x` is an iterator is to call `isinstance(x, abc.Iterator)`. Thanks to `Iterator.__subclasshook__`, this test works even if the class of `x` is not a real or virtual subclass of `Iterator`.

Back to our `Sentence` class from [Example 14-1](#), you can clearly see how the iterator is built by `iter(...)` and consumed by `next(...)` using the Python console:

```
>>> s3 = Sentence('Pig and Pepper') # ❶
>>> it = iter(s3) # ❷
>>> it # doctest: +ELLIPSIS
<iterator object at 0x...>
>>> next(it) # ❸
'Pig'
>>> next(it)
'and'
>>> next(it)
'Pepper'
>>> next(it) # ❹
Traceback (most recent call last):
...
StopIteration
>>> list(it) # ❺
[]
>>> list(iter(s3)) # ❻
['Pig', 'and', 'Pepper']
```

- ❶ Create a sentence `s3` with three words.

- ❷ Obtain an iterator from `s3`.
- ❸ `next(it)` fetches the next word.
- ❹ There are no more words, so the iterator raises a `StopIteration` exception.
- ❺ Once exhausted, an iterator becomes useless.
- ❻ To go over the sentence again, a new iterator must be built.

Because the only methods required of an iterator are `__next__` and `__iter__`, there is no way to check whether there are remaining items, other than to call `next()` and catch `StopIteration`. Also, it's not possible to “reset” an iterator. If you need to start over, you need to call `iter(...)` on the iterable that built the iterator in the first place. Calling `iter(...)` on the iterator itself won't help, because—as mentioned—`Iterator.__iter__` is implemented by returning `self`, so this will not reset a depleted iterator.

To wrap up this section, here is a definition for *iterator*:

iterator

Any object that implements the `__next__` no-argument method that returns the next item in a series or raises `StopIteration` when there are no more items. Python iterators also implement the `__iter__` method so they are *iterable* as well.

This first version of `Sentence` was iterable thanks to the special treatment the `iter(...)` built-in gives to sequences. Now we'll implement the standard iterable protocol.

Sentence Take #2: A Classic Iterator

The next `Sentence` class is built according to the classic Iterator design pattern following the blueprint in the GoF book. Note that this is not idiomatic Python, as the next refactorings will make very clear. But it serves to make explicit the relationship between the iterable collection and the iterator object.

Example 14-4 shows an implementation of a `Sentence` that is iterable because it implements the `__iter__` special method, which builds and returns a `SentenceIterator`. This is how the Iterator design pattern is described in the original *Design Patterns* book.

We are doing it this way here just to make clear the crucial distinction between an iterable and an iterator and how they are connected.

Example 14-4. sentence_iter.py: Sentence implemented using the Iterator pattern

```
import re
import replib

RE_WORD = re.compile('\w+')
```

```
class Sentence:
```

```
    def __init__(self, text):
        self.text = text
        self.words = RE_WORD.findall(text)

    def __repr__(self):
        return 'Sentence(%s)' % reprlib.repr(self.text)

    def __iter__(self): ❶
        return SentenceIterator(self.words) ❷
```

```
class SentenceIterator:
```

```
    def __init__(self, words):
        self.words = words ❸
        self.index = 0 ❹

    def __next__(self):
        try:
            word = self.words[self.index] ❺
        except IndexError:
            raise StopIteration() ❻
        self.index += 1 ❼
        return word ❽

    def __iter__(self): ❾
        return self
```

- ❶ The `__iter__` method is the only addition to the previous `Sentence` implementation. This version has no `__getitem__`, to make it clear that the class is iterable because it implements `__iter__`.
- ❷ `__iter__` fulfills the iterable protocol by instantiating and returning an iterator.
- ❸ `SentenceIterator` holds a reference to the list of words.
- ❹ `self.index` is used to determine the next word to fetch.
- ❺ Get the word at `self.index`.
- ❻ If there is no word at `self.index`, raise `StopIteration`.
- ❼ Increment `self.index`.
- ❽ Return the word.
- ❾ Implement `self.__iter__`.

The code in [Example 14-4](#) passes the tests in [Example 14-2](#).

Note that implementing `__iter__` in `SentenceIterator` is not actually needed for this example to work, but it's the right thing to do: iterators are supposed to implement both `__next__` and `__iter__`, and doing so makes our iterator pass the `issubclass(SentenceIterator, abc.Iterator)` test. If we had subclassed `SentenceIterator` from `abc.Iterator`, we'd inherit the concrete `abc.Iterator.__iter__` method.

That is a lot of work (for us lazy Python programmers, anyway). Note how most code in `SentenceIterator` deals with managing the internal state of the iterator. Soon we'll see how to make it shorter. But first, a brief detour to address an implementation shortcut that may be tempting, but is just wrong.

Making Sentence an Iterator: Bad Idea

A common cause of errors in building iterables and iterators is to confuse the two. To be clear: iterables have an `__iter__` method that instantiates a new iterator every time. Iterators implement a `__next__` method that returns individual items, and an `__iter__` method that returns `self`.

Therefore, iterators are also iterable, but iterables are not iterators.

It may be tempting to implement `__next__` in addition to `__iter__` in the `Sentence` class, making each `Sentence` instance at the same time an iterable and iterator over itself. But this is a terrible idea. It's also a common anti-pattern, according to Alex Martelli who has a lot of experience with Python code reviews.

The “Applicability” section⁴ of the Iterator design pattern in the *GoF book* says:

Use the Iterator pattern

- to access an aggregate object's contents without exposing its internal representation.
- to support multiple traversals of aggregate objects.
- to provide a uniform interface for traversing different aggregate structures (that is, to support polymorphic iteration).

To “support multiple traversals” it must be possible to obtain multiple independent iterators from the same iterable instance, and each iterator must keep its own internal state, so a proper implementation of the pattern requires each call to `iter(my_iterable)` to create a new, independent, iterator. That is why we need the `SentenceIterator` class in this example.

4. Gamma et. al., *Design Patterns: Elements of Reusable Object-Oriented Software*, p. 259.



An iterable should never act as an iterator over itself. In other words, iterables must implement `__iter__`, but not `__next__`.

On the other hand, for convenience, iterators should be iterable.

An iterator's `__iter__` should just return `self`.

Now that the classic Iterator pattern is properly demonstrated, we can get let it go. The next section presents a more idiomatic implementation of `Sentence`.

Sentence Take #3: A Generator Function

A Pythonic implementation of the same functionality uses a generator function to replace the `SequenceIterator` class. A proper explanation of the generator function comes right after [Example 14-5](#).

Example 14-5. `sentence_gen.py`: `Sentence` implemented using a generator function

```
import re
import reprlib

RE_WORD = re.compile('\w+')

class Sentence:

    def __init__(self, text):
        self.text = text
        self.words = RE_WORD.findall(text)

    def __repr__(self):
        return 'Sentence(%s)' % reprlib.repr(self.text)

    def __iter__(self):
        for word in self.words: ❶
            yield word ❷
        return ❸

# done! ❹
```

❶ Iterate over `self.words`.

❷ Yield the current word.

- ③ This `return` is not needed; the function can just “fall-through” and return automatically. Either way, a generator function doesn’t raise `StopIteration`: it simply exits when it’s done producing values.⁵
- ④ No need for a separate iterator class!

Here again we have a different implementation of `Sentence` that passes the tests in [Example 14-2](#).

Back in the `Sentence` code in [Example 14-4](#), `__iter__` called the `SentenceIterator` constructor to build an iterator and return it. Now the iterator in [Example 14-5](#) is in fact a generator object, built automatically when the `__iter__` method is called, because `__iter__` here is a generator function.

A full explanation of generator functions follows.

How a Generator Function Works

Any Python function that has the `yield` keyword in its body is a generator function: a function which, when called, returns a generator object. In other words, a generator function is a generator factory.



The only syntax distinguishing a plain function from a generator function is the fact that the latter has a `yield` keyword somewhere in its body. Some argued that a new keyword like `gen` should be used for generator functions instead of `def`, but Guido did not agree. His arguments are in [PEP 255 — Simple Generators](#).⁶

Here is the simplest function useful to demonstrate the behavior of a generator:⁷

```
>>> def gen_123(): # ①
...     yield 1 # ②
...     yield 2
...     yield 3
...
>>> gen_123 # doctest: +ELLIPSIS
<function gen_123 at 0x...> # ③
```

- 5. When reviewing this code, Alex Martelli suggested the body of this method could simply be `return iter(self.words)`. He is correct, of course: the result of calling `__iter__` would also be an iterator, as it should be. However, I used a `for` loop with `yield` here to introduce the syntax of a generator function, which will be covered in detail in the next section.
- 6. Sometimes I add a `gen` prefix or suffix when naming generator functions, but this is not a common practice. And you can’t do that if you’re implementing an iterable, of course: the necessary special method must be named `__iter__`.
- 7. Thanks to David Kwast for suggesting this example.

```

>>> gen_123() # doctest: +ELLIPSIS
<generator object gen_123 at 0x...> # ④
>>> for i in gen_123(): # ⑤
...     print(i)
1
2
3
>>> g = gen_123() # ⑥
>>> next(g) # ⑦
1
>>> next(g)
2
>>> next(g)
3
>>> next(g) # ⑧
Traceback (most recent call last):
...
StopIteration

```

- ① Any Python function that contains the `yield` keyword is a generator function.
- ② Usually the body of a generator function has loop, but not necessarily; here I just repeat `yield` three times.
- ③ Looking closely, we see `gen_123` is a function object.
- ④ But when invoked, `gen_123()` returns a generator object.
- ⑤ Generators are iterators that produce the values of the expressions passed to `yield`.
- ⑥ For closer inspection, we assign the generator object to `g`.
- ⑦ Because `g` is an iterator, calling `next(g)` fetches the next item produced by `yield`.
- ⑧ When the body of the function completes, the generator object raises a `StopIteration`.

A generator function builds a generator object that wraps the body of the function. When we invoke `next(...)` on the generator object, execution advances to the next `yield` in the function body, and the `next(...)` call evaluates to the value yielded when the function body is suspended. Finally, when the function body returns, the enclosing generator object raises `StopIteration`, in accordance with the `Iterator` protocol.



I find it helpful to be strict when talking about the results obtained from a generator: I say that a generator *yields* or *produces* values. But it's confusing to say a generator “returns” values. Functions return values. Calling a generator function returns a generator. A generator yields or produces values. A generator doesn't “return” values in the usual way: the return statement in the body of a generator function causes `StopIteration` to be raised by the generator object.⁸

Example 14-6 makes the interaction between a for loop and the body of the function more explicit.

Example 14-6. A generator function that prints messages when it runs

```
>>> def gen_AB(): # ❶
...     print('start')
...     yield 'A' # ❷
...     print('continue')
...     yield 'B' # ❸
...     print('end. ') # ❹
...
>>> for c in gen_AB(): # ❺
...     print('-->', c) # ❻
...
start # ❼
--> A # ❽
continue # ❾
--> B # ❿
end. # ⓫
>>> # ⓫
```

- ❶ The generator function is defined like any function, but uses `yield`.
- ❷ The first implicit call to `next()` in the for loop at ❺ will print 'start' and stop at the first `yield`, producing the value 'A'.
- ❸ The second implicit call to `next()` in the for loop will print 'continue' and stop at the second `yield`, producing the value 'B'.
- ❹ The third call to `next()` will print 'end. ' and fall through the end of the function body, causing the generator object to raise `StopIteration`.

8. Prior to Python 3.3, it was an error to provide a value with the `return` statement in a generator function. Now that is legal, but the `return` still causes a `StopIteration` exception to be raised. The caller can retrieve the return value from the exception object. However, this is only relevant when using a generator function as a coroutine, as we'll see in “Returning a Value from a Coroutine” on page 475.

- ⑤ To iterate, the for machinery does the equivalent of `g = iter(gen_AB())` to get a generator object, and then `next(g)` at each iteration.
- ⑥ The loop block prints `-->` and the value returned by `next(g)`. But this output will be seen only after the output of the `print` calls inside the generator function.
- ⑦ The string `'start'` appears as a result of `print('start')` in the generator function body.
- ⑧ `yield 'A'` in the generator function body produces the value `A` consumed by the for loop, which gets assigned to the `c` variable and results in the output `--> A`.
- ⑨ Iteration continues with a second call `next(g)`, advancing the generator function body from `yield 'A'` to `yield 'B'`. The text `continue` is output because of the second `print` in the generator function body.
- ⑩ `yield 'B'` produces the value `B` consumed by the for loop, which gets assigned to the `c` loop variable, so the loop prints `--> B`.
- ⑪ Iteration continues with a third call `next(it)`, advancing to the end of the body of the function. The text `end.` appears in the output because of the third `print` in the generator function body.
- ⑫ When the generator function body runs to the end, the generator object raises `StopIteration`. The for loop machinery catches that exception, and the loop terminates cleanly.

Now hopefully it's clear how `Sentence.__iter__` in [Example 14-5](#) works: `__iter__` is a generator function which, when called, builds a generator object that implements the iterator interface, so the `SentenceIterator` class is no longer needed.

This second version of `Sentence` is much shorter than the first, but it's not as lazy as it could be. Nowadays, laziness is considered a good trait, at least in programming languages and APIs. A lazy implementation postpones producing values to the last possible moment. This saves memory and may avoid useless processing as well.

We'll build a lazy `Sentence` class next.

Sentence Take #4: A Lazy Implementation

The `Iterator` interface is designed to be lazy: `next(my_iterator)` produces one item at a time. The opposite of lazy is eager: lazy evaluation and eager evaluation are actual technical terms in programming language theory.

Our `Sentence` implementations so far have not been lazy because the `__init__` eagerly builds a list of all words in the text, binding it to the `self.words` attribute. This will entail processing the entire text, and the list may use as much memory as the text itself

(probably more; it depends on how many nonword characters are in the text). Most of this work will be in vain if the user only iterates over the first couple words.

Whenever you are using Python 3 and start wondering “Is there a lazy way of doing this?”, often the answer is “Yes.”

The `re.finditer` function is a lazy version of `re.findall` which, instead of a list, returns a generator producing `re.MatchObject` instances on demand. If there are many matches, `re.finditer` saves a lot of memory. Using it, our third version of `Sentence` is now lazy: it only produces the next word when it is needed. The code is in [Example 14-7](#).

Example 14-7. `sentence_gen2.py`: `Sentence` implemented using a generator function calling the `re.finditer` generator function

```
import re
import reprlib

RE_WORD = re.compile('\w+')

class Sentence:

    def __init__(self, text):
        self.text = text ❶

    def __repr__(self):
        return 'Sentence(%s)' % reprlib.repr(self.text)

    def __iter__(self):
        for match in RE_WORD.finditer(self.text): ❷
            yield match.group() ❸
```

- ❶ No need to have a words list.
- ❷ `finditer` builds an iterator over the matches of `RE_WORD` on `self.text`, yielding `MatchObject` instances.
- ❸ `match.group()` extracts the actual matched text from the `MatchObject` instance.

Generator functions are an awesome shortcut, but the code can be made even shorter with a generator expression.

Sentence Take #5: A Generator Expression

Simple generator functions like the one in the previous `Sentence` class ([Example 14-7](#)) can be replaced by a generator expression.

A generator expression can be understood as a lazy version of a list comprehension: it does not eagerly build a list, but returns a generator that will lazily produce the items

on demand. In other words, if a list comprehension is a factory of lists, a generator expression is a factory of generators.

Example 14-8 is a quick demo of a generator expression, comparing it to a list comprehension.

Example 14-8. The `gen_AB` generator function is used by a list comprehension, then by a generator expression

```
>>> def gen_AB(): # ❶
...     print('start')
...     yield 'A'
...     print('continue')
...     yield 'B'
...     print('end.')
...
>>> res1 = [x*3 for x in gen_AB()] # ❷
start
continue
end.
>>> for i in res1: # ❸
...     print('-->', i)
...
--> AAA
--> BBB
>>> res2 = (x*3 for x in gen_AB()) # ❹
>>> res2 # ❺
<generator object <genexpr> at 0x10063c240>
>>> for i in res2: # ❻
...     print('-->', i)
...
start
--> AAA
continue
--> BBB
end.
```

- ❶ This is the same `gen_AB` function from **Example 14-6**.
- ❷ The list comprehension eagerly iterates over the items yielded by the generator object produced by calling `gen_AB()`: 'A' and 'B'. Note the output in the next lines: start, continue, end.
- ❸ This for loop is iterating over the `res1` list produced by the list comprehension.
- ❹ The generator expression returns `res2`. The call to `gen_AB()` is made, but that call returns a generator, which is not consumed here.
- ❺ `res2` is a generator object.

- ⑥ Only when the for loop iterates over `res2`, the body of `gen_AB` actually executes. Each iteration of the for loop implicitly calls `next(res2)`, advancing `gen_AB` to the next `yield`. Note the output of `gen_AB` with the output of the `print` in the for loop.

So, a generator expression produces a generator, and we can use it to further reduce the code in the `Sentence` class. See [Example 14-9](#).

Example 14-9. `sentence_genexp.py`: `Sentence` implemented using a generator expression

```
import re
import reprlib

RE_WORD = re.compile('\w+')

class Sentence:

    def __init__(self, text):
        self.text = text

    def __repr__(self):
        return 'Sentence(%s)' % reprlib.repr(self.text)

    def __iter__(self):
        return (match.group() for match in RE_WORD.finditer(self.text))
```

The only difference from [Example 14-7](#) is the `__iter__` method, which here is not a generator function (it has no `yield`) but uses a generator expression to build a generator and then returns it. The end result is the same: the caller of `__iter__` gets a generator object.

Generator expressions are syntactic sugar: they can always be replaced by generator functions, but sometimes are more convenient. The next section is about generator expression usage.

Generator Expressions: When to Use Them

I used several generator expressions when implementing the `Vector` class in [Example 10-16](#). Each of the methods `__eq__`, `__hash__`, `__abs__`, `angle`, `angles`, `format`, `__add__`, and `__mul__` has a generator expression. In all those methods, a list comprehension would also work, at the cost of using more memory to store the intermediate list values.

In [Example 14-9](#), we saw that a generator expression is a syntactic shortcut to create a generator without defining and calling a function. On the other hand, generator func-

tions are much more flexible: you can code complex logic with multiple statements, and can even use them as *coroutines* (see [Chapter 16](#)).

For the simpler cases, a generator expression will do, and it's easier to read at a glance, as the `Vector` example shows.

My rule of thumb in choosing the syntax to use is simple: if the generator expression spans more than a couple of lines, I prefer to code a generator function for the sake of readability. Also, because generator functions have a name, they can be reused. You can always name a generator expression and use it later by assigning it to a variable, of course, but that is stretching its intended usage as a one-off generator.



Syntax Tip

When a generator expression is passed as the single argument to a function or constructor, you don't need to write a set of parentheses for the function call and another to enclose the generator expression. A single pair will do, like in the `Vector` call from the `__mul__` method in [Example 10-16](#), reproduced here. However, if there are more function arguments after the generator expression, you need to enclose it in parentheses to avoid a `SyntaxError`:

```
def __mul__(self, scalar):
    if isinstance(scalar, numbers.Real):
        return Vector(n * scalar for n in self)
    else:
        return NotImplemented
```

The `Sentence` examples we've seen exemplify the use of generators playing the role of classic iterators: retrieving items from a collection. But generators can also be used to produce values independent of a data source. The next section shows an example of that.

Another Example: Arithmetic Progression Generator

The classic `Iterator` pattern is all about traversal: navigating some data structure. But a standard interface based on a method to fetch the next item in a series is also useful when the items are produced on the fly, instead of retrieved from a collection. For example, the `range` built-in generates a bounded arithmetic progression (AP) of integers, and the `itertools.count` function generates a boundless AP.

We'll cover `itertools.count` in the next section, but what if you need to generate a bounded AP of numbers of any type?

[Example 14-10](#) shows a few console tests of an `ArithmeticProgression` class we will see in a moment. The signature of the constructor in [Example 14-10](#) is `ArithmeticProgression(begin, step[, end])`. The `range()` function is similar to the `Arithme`

ticProgression here, but its full signature is `range(start, stop[, step])`. I chose to implement a different signature because for an arithmetic progression the step is mandatory but end is optional. I also changed the argument names from start/stop to begin/end to make it very clear that I opted for a different signature. In each test in [Example 14-10](#) I call `list()` on the result to inspect the generated values.

Example 14-10. Demonstration of an ArithmeticProgression class

```
>>> ap = ArithmeticProgression(0, 1, 3)
>>> list(ap)
[0, 1, 2]
>>> ap = ArithmeticProgression(1, .5, 3)
>>> list(ap)
[1.0, 1.5, 2.0, 2.5]
>>> ap = ArithmeticProgression(0, 1/3, 1)
>>> list(ap)
[0.0, 0.3333333333333333, 0.6666666666666666]
>>> from fractions import Fraction
>>> ap = ArithmeticProgression(0, Fraction(1, 3), 1)
>>> list(ap)
[Fraction(0, 1), Fraction(1, 3), Fraction(2, 3)]
>>> from decimal import Decimal
>>> ap = ArithmeticProgression(0, Decimal('.1'), .3)
>>> list(ap)
[Decimal('0.0'), Decimal('0.1'), Decimal('0.2')]
```

Note that type of the numbers in the resulting arithmetic progression follows the type of begin or step, according to the numeric coercion rules of Python arithmetic. In [Example 14-10](#), you see lists of int, float, Fraction, and Decimal numbers.

[Example 14-11](#) lists the implementation of the ArithmeticProgression class.

Example 14-11. The ArithmeticProgression class

```
class ArithmeticProgression:
```

```
    def __init__(self, begin, step, end=None): ❶
        self.begin = begin
        self.step = step
        self.end = end # None -> "infinite" series

    def __iter__(self):
        result = type(self.begin + self.step)(self.begin) ❷
        forever = self.end is None ❸
        index = 0
        while forever or result < self.end: ❹
            yield result ❺
            index += 1
            result = self.begin + self.step * index ❻
```

- ❶ `__init__` requires two arguments: `begin` and `step`. `end` is optional, if it's `None`, the series will be unbounded.
- ❷ This line produces a `result` value equal to `self.begin`, but coerced to the type of the subsequent additions.⁹
- ❸ For readability, the `forever` flag will be `True` if the `self.end` attribute is `None`, resulting in an unbounded series.
- ❹ This loop runs forever or until the result matches or exceeds `self.end`. When this loop exits, so does the function.
- ❺ The current `result` is produced.
- ❻ The next potential result is calculated. It may never be yielded, because the `while` loop may terminate.

In the last line of [Example 14-11](#), instead of simply incrementing the `result` with `self.step` iteratively, I opted to use an index variable and calculate each `result` by adding `self.begin` to `self.step` multiplied by `index` to reduce the cumulative effect of errors when working with floats.

The `ArithmeticProgression` class from [Example 14-11](#) works as intended, and is a clear example of the use of a generator function to implement the `__iter__` special method. However, if the whole point of a class is to build a generator by implementing `__iter__`, the class can be reduced to a generator function. A generator function is, after all, a generator factory.

[Example 14-12](#) shows a generator function called `aritprog_gen` that does the same job as `ArithmeticProgression` but with less code. The tests in [Example 14-10](#) all pass if you just call `aritprog_gen` instead of `ArithmeticProgression`.¹⁰

Example 14-12. The `aritprog_gen` generator function

```
def aritprog_gen(begin, step, end=None):
    result = type(begin + step)(begin)
    forever = end is None
    index = 0
    while forever or result < end:
        yield result
```

9. In Python 2, there was a `coerce()` built-in function but it's gone in Python 3, deemed unnecessary because the numeric coercion rules are implicit in the arithmetic operator methods. So the best way I could think of to coerce the initial value to be of the same type as the rest of the series was to perform the addition and use its type to convert the result. I asked about this in the Python-list and got an excellent [response from Steven D'Aprano](#).

10. The `14-it-generator/` directory in the [Fluent Python code repository](#) includes doctests and a script, `aritprog_runner.py`, which runs the tests against all variations of the `aritprog*.py` scripts.

```
index += 1
result = begin + step * index
```

Example 14-12 is pretty cool, but always remember: there are plenty of ready-to-use generators in the standard library, and the next section will show an even cooler implementation using the `itertools` module.

Arithmetic Progression with `itertools`

The `itertools` module in Python 3.4 has 19 generator functions that can be combined in a variety of interesting ways.

For example, the `itertools.count` function returns a generator that produces numbers. Without arguments, it produces a series of integers starting with 0. But you can provide optional start and step values to achieve a result very similar to our `aritprog_gen` functions:

```
>>> import itertools
>>> gen = itertools.count(1, .5)
>>> next(gen)
1
>>> next(gen)
1.5
>>> next(gen)
2.0
>>> next(gen)
2.5
```

However, `itertools.count` never stops, so if you call `list(count())`, Python will try to build a list larger than available memory and your machine will be very grumpy long before the call fails.

On the other hand, there is the `itertools.takewhile` function: it produces a generator that consumes another generator and stops when a given predicate evaluates to `False`. So we can combine the two and write this:

```
>>> gen = itertools.takewhile(lambda n: n < 3, itertools.count(1, .5))
>>> list(gen)
[1, 1.5, 2.0, 2.5]
```

Leveraging `takewhile` and `count`, **Example 14-13** is sweet and short.

Example 14-13. `aritprog_v3.py`: this works like the previous `aritprog_gen` functions

```
import itertools
```

```
def aritprog_gen(begin, step, end=None):
    first = type(begin + step)(begin)
    ap_gen = itertools.count(first, step)
    if end is not None:
```



```
ap_gen = itertools.takewhile(lambda n: n < end, ap_gen)
return ap_gen
```

Note that `aritprog_gen` is not a generator function in [Example 14-13](#): it has no `yield` in its body. But it returns a generator, so it operates as a generator factory, just as a generator function does.

The point of [Example 14-13](#) is: when implementing generators, know what is available in the standard library, otherwise there's a good chance you'll reinvent the wheel. That's why the next section covers several ready-to-use generator functions.

Generator Functions in the Standard Library

The standard library provides many generators, from plain-text file objects providing line-by-line iteration, to the awesome `os.walk` function, which yields filenames while traversing a directory tree, making recursive filesystem searches as simple as a `for` loop.

The `os.walk` generator function is impressive, but in this section I want to focus on general-purpose functions that take arbitrary iterables as arguments and return generators that produce selected, computed, or rearranged items. In the following tables, I summarize two dozen of them, from the built-in, `itertools`, and `functools` modules. For convenience, I grouped them by high-level functionality, regardless of where they are defined.



Perhaps you know all the functions mentioned in this section, but some of them are underused, so a quick overview may be good to recall what's already available.

The first group are filtering generator functions: they yield a subset of items produced by the input iterable, without changing the items themselves. We used `itertools.takewhile` previously in this chapter, in [“Arithmetic Progression with itertools” on page 423](#). Like `takewhile`, most functions listed in [Table 14-1](#) take a predicate, which is a one-argument Boolean function that will be applied to each item in the input to determine whether the item is included in the output.

Table 14-1. Filtering generator functions

Module	Function	Description
<code>itertools</code>	<code>compress(it, selector_it)</code>	Consumes two iterables in parallel; yields items from <code>it</code> whenever the corresponding item in <code>selector_it</code> is truthy
<code>itertools</code>	<code>dropwhile(predicate, it)</code>	Consumes <code>it</code> skipping items while predicate computes truthy, then yields every remaining item (no further checks are made)

Module	Function	Description
(built-in)	<code>filter(predicate, it)</code>	Applies predicate to each item of iterable, yielding the item if <code>predicate(item)</code> is truthy; if predicate is None, only truthy items are yielded
<code>iter tools</code>	<code>filterfalse(predicate, it)</code>	Same as <code>filter</code> , with the predicate logic negated: yields items whenever predicate computes falsy
<code>iter tools</code>	<code>islice(it, stop)</code> or <code>islice(it, start, stop, step=1)</code>	Yields items from a slice of it, similar to <code>s[:stop]</code> or <code>s[start:stop:step]</code> except it can be any iterable, and the operation is lazy
<code>iter tools</code>	<code>takewhile(predicate, it)</code>	Yields items while predicate computes truthy, then stops and no further checks are made

The console listing in [Example 14-14](#) shows the use of all functions in [Table 14-1](#).

Example 14-14. Filtering generator functions examples

```
>>> def vowel(c):
...     return c.lower() in 'aeiou'
...
>>> list(filter(vowel, 'Aardvark'))
['A', 'a', 'a']
>>> import itertools
>>> list(itertools.filterfalse(vowel, 'Aardvark'))
['r', 'd', 'v', 'r', 'k']
>>> list(itertools.dropwhile(vowel, 'Aardvark'))
['r', 'd', 'v', 'a', 'r', 'k']
>>> list(itertools.takewhile(vowel, 'Aardvark'))
['A', 'a']
>>> list(itertools.compress('Aardvark', (1,0,1,1,0,1)))
['A', 'r', 'd', 'a']
>>> list(itertools.islice('Aardvark', 4))
['A', 'a', 'r', 'd']
>>> list(itertools.islice('Aardvark', 4, 7))
['v', 'a', 'r']
>>> list(itertools.islice('Aardvark', 1, 7, 2))
['a', 'd', 'a']
```

The next group are the mapping generators: they yield items computed from each individual item in the input iterable—or iterables, in the case of `map` and `starmap`.¹¹ The generators in [Table 14-2](#) yield one result per item in the input iterables. If the input comes from more than one iterable, the output stops as soon as the first input iterable is exhausted.

11. Here the term “mapping” is unrelated to dictionaries, but has to do with the `map` built-in.

Table 14-2. Mapping generator functions

Module	Function	Description
itertools	accumulate(it, [func])	Yields accumulated sums; if func is provided, yields the result of applying it to the first pair of items, then to the first result and next item, etc.
(built-in)	enumerate(iterable, start=0)	Yields 2-tuples of the form (index, item), where index is counted from start, and item is taken from the iterable
(built-in)	map(func, it1, [it2, ..., itN])	Applies func to each item of it, yielding the result; if N iterables are given, func must take N arguments and the iterables will be consumed in parallel
itertools	starmap(func, it)	Applies func to each item of it, yielding the result; the input iterable should yield iterable items it, and func is applied as func(*it)

Example 14-15 demonstrates some uses of `itertools.accumulate`.

Example 14-15. `itertools.accumulate` generator function examples

```
>>> sample = [5, 4, 2, 8, 7, 6, 3, 0, 9, 1]
>>> import itertools
>>> list(itertools.accumulate(sample)) # ❶
[5, 9, 11, 19, 26, 32, 35, 35, 44, 45]
>>> list(itertools.accumulate(sample, min)) # ❷
[5, 4, 2, 2, 2, 2, 2, 0, 0, 0]
>>> list(itertools.accumulate(sample, max)) # ❸
[5, 5, 5, 8, 8, 8, 8, 8, 8, 9]
>>> import operator
>>> list(itertools.accumulate(sample, operator.mul)) # ❹
[5, 20, 40, 320, 2240, 13440, 40320, 0, 0, 0]
>>> list(itertools.accumulate(range(1, 11), operator.mul))
[1, 2, 6, 24, 120, 720, 5040, 40320, 362880, 3628800] # ❺
```

- ❶ Running sum.
- ❷ Running minimum.
- ❸ Running maximum.
- ❹ Running product.
- ❺ Factorials from 1! to 10!.

The remaining functions of Table 14-2 are shown in Example 14-16.

Example 14-16. Mapping generator function examples

```
>>> list(enumerate('albatroz', 1)) # ❶
[(1, 'a'), (2, 'l'), (3, 'b'), (4, 'a'), (5, 't'), (6, 'r'), (7, 'o'), (8, 'z')]
>>> import operator
>>> list(map(operator.mul, range(11), range(11))) # ❷
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
>>> list(map(operator.mul, range(11), [2, 4, 8])) # ❸
[0, 4, 16]
>>> list(map(lambda a, b: (a, b), range(11), [2, 4, 8])) # ❹
```

```

[(0, 2), (1, 4), (2, 8)]
>>> import itertools
>>> list(itertools.starmap(operator.mul, enumerate('albatroz', 1))) # ❸
['a', 'll', 'bbb', 'aaaa', 'ttttt', 'rrrrrr', 'ooooooo', 'zzzzzzzz']
>>> sample = [5, 4, 2, 8, 7, 6, 3, 0, 9, 1]
>>> list(itertools.starmap(lambda a, b: b/a,
...     enumerate(itertools.accumulate(sample), 1))) # ❹
[5.0, 4.5, 3.6666666666666665, 4.75, 5.2, 5.333333333333333,
5.0, 4.375, 4.888888888888889, 4.5]

```

- ❶ Number the letters in the word, starting from 1.
- ❷ Squares of integers from 0 to 10.
- ❸ Multiplying numbers from two iterables in parallel: results stop when the shortest iterable ends.
- ❹ This is what the zip built-in function does.
- ❺ Repeat each letter in the word according to its place in it, starting from 1.
- ❻ Running average.

Next, we have the group of merging generators—all of these yield items from multiple input iterables. `chain` and `chain.from_iterable` consume the input iterables sequentially (one after the other), while `product`, `zip`, and `zip_longest` consume the input iterables in parallel. See [Table 14-3](#).

Table 14-3. Generator functions that merge multiple input iterables

Module	Function	Description
itertools	<code>chain(it1, ..., itN)</code>	Yield all items from <code>it1</code> , then from <code>it2</code> etc., seamlessly
itertools	<code>chain.from_iterable(it)</code>	Yield all items from each iterable produced by <code>it</code> , one after the other, seamlessly; <code>it</code> should yield iterable items, for example, a list of iterables
itertools	<code>product(it1, ..., itN, repeat=1)</code>	Cartesian product: yields N-tuples made by combining items from each input iterable like nested for loops could produce; <code>repeat</code> allows the input iterables to be consumed more than once
(built-in)	<code>zip(it1, ..., itN)</code>	Yields N-tuples built from items taken from the iterables in parallel, silently stopping when the first iterable is exhausted
itertools	<code>zip_longest(it1, ..., itN, fillvalue=None)</code>	Yields N-tuples built from items taken from the iterables in parallel, stopping only when the last iterable is exhausted, filling the blanks with the <code>fillvalue</code>

Example 14-17 shows the use of the `itertools.chain` and `zip` generator functions and their siblings. Recall that the `zip` function is named after the zip fastener or zipper (no relation with compression). Both `zip` and `itertools.zip_longest` were introduced in “The Awesome zip” on page 293.

Example 14-17. Merging generator function examples

```
>>> list(itertools.chain('ABC', range(2))) # ❶
['A', 'B', 'C', 0, 1]
>>> list(itertools.chain(enumerate('ABC'))) # ❷
[(0, 'A'), (1, 'B'), (2, 'C')]
>>> list(itertools.chain.from_iterable(enumerate('ABC'))) # ❸
[0, 'A', 1, 'B', 2, 'C']
>>> list(zip('ABC', range(5))) # ❹
[('A', 0), ('B', 1), ('C', 2)]
>>> list(zip('ABC', range(5), [10, 20, 30, 40])) # ❺
[('A', 0, 10), ('B', 1, 20), ('C', 2, 30)]
>>> list(itertools.zip_longest('ABC', range(5))) # ❻
[('A', 0), ('B', 1), ('C', 2), (None, 3), (None, 4)]
>>> list(itertools.zip_longest('ABC', range(5), fillvalue='?')) # ❼
[('A', 0), ('B', 1), ('C', 2), ('?', 3), ('?', 4)]
```

- ❶ chain is usually called with two or more iterables.
- ❷ chain does nothing useful when called with a single iterable.
- ❸ But chain.from_iterable takes each item from the iterable, and chains them in sequence, as long as each item is itself iterable.
- ❹ zip is commonly used to merge two iterables into a series of two-tuples.
- ❺ Any number of iterables can be consumed by zip in parallel, but the generator stops as soon as the first iterable ends.
- ❻ itertools.zip_longest works like zip, except it consumes all input iterables to the end, padding output tuples with None as needed.
- ❼ The fillvalue keyword argument specifies a custom padding value.

The `itertools.product` generator is a lazy way of computing Cartesian products, which we built using list comprehensions with more than one for clause in “[Cartesian Products](#)” on page 23. Generator expressions with multiple for clauses can also be used to produce Cartesian products lazily. [Example 14-18](#) demonstrates `itertools.product`.

Example 14-18. `itertools.product` generator function examples

```
>>> list(itertools.product('ABC', range(2))) # ❶
[('A', 0), ('A', 1), ('B', 0), ('B', 1), ('C', 0), ('C', 1)]
>>> suits = 'spades hearts diamonds clubs'.split()
>>> list(itertools.product('AK', suits)) # ❷
[('A', 'spades'), ('A', 'hearts'), ('A', 'diamonds'), ('A', 'clubs'),
 ('K', 'spades'), ('K', 'hearts'), ('K', 'diamonds'), ('K', 'clubs')]
>>> list(itertools.product('ABC')) # ❸
[('A',), ('B',), ('C',)]
>>> list(itertools.product('ABC', repeat=2)) # ❹
[('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'B'),
 ('B', 'C'), ('C', 'A'), ('C', 'B'), ('C', 'C')]
>>> list(itertools.product(range(2), repeat=3))
```

```
[(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1), (1, 0, 0),
(1, 0, 1), (1, 1, 0), (1, 1, 1)]
>>> rows = itertools.product('AB', range(2), repeat=2)
>>> for row in rows: print(row)
...
('A', 0, 'A', 0)
('A', 0, 'A', 1)
('A', 0, 'B', 0)
('A', 0, 'B', 1)
('A', 1, 'A', 0)
('A', 1, 'A', 1)
('A', 1, 'B', 0)
('A', 1, 'B', 1)
('B', 0, 'A', 0)
('B', 0, 'A', 1)
('B', 0, 'B', 0)
('B', 0, 'B', 1)
('B', 1, 'A', 0)
('B', 1, 'A', 1)
('B', 1, 'B', 0)
('B', 1, 'B', 1)
```

- ❶ The Cartesian product of a str with three characters and a range with two integers yields six tuples (because $3 * 2$ is 6).
- ❷ The product of two card ranks ('AK'), and four suits is a series of eight tuples.
- ❸ Given a single iterable, product yields a series of one-tuples, not very useful.
- ❹ The repeat=N keyword argument tells product to consume each input iterable N times.

Some generator functions expand the input by yielding more than one value per input item. They are listed in [Table 14-4](#).

Table 14-4. Generator functions that expand each input item into multiple output items

Module	Function	Description
itertools	combinations(it, out_len)	Yield combinations of out_len items from the items yielded by it
itertools	combinations_with_replacement(it, out_len)	Yield combinations of out_len items from the items yielded by it, including combinations with repeated items
itertools	count(start=0, step=1)	Yields numbers starting at start, incremented by step, indefinitely
itertools	cycle(it)	Yields items from it storing a copy of each, then yields the entire sequence repeatedly, indefinitely
itertools	permutations(it, out_len=None)	Yield permutations of out_len items from the items yielded by it; by default, out_len is len(list(it))

Module	Function	Description
itertools	repeat(item, [times])	Yield the given item repeatedly, indefinitely unless a number of times is given

The count and repeat functions from `itertools` return generators that conjure items out of nothing: neither of them takes an iterable as input. We saw `itertools.count` in “Arithmetic Progression with `itertools`” on page 423. The cycle generator makes a backup of the input iterable and yields its items repeatedly. Example 14-19 illustrates the use of count, repeat, and cycle.

Example 14-19. count, cycle, and repeat

```
>>> ct = itertools.count() # ❶
>>> next(ct) # ❷
0
>>> next(ct), next(ct), next(ct) # ❸
(1, 2, 3)
>>> list(itertools.islice(itertools.count(1, .3), 3)) # ❹
[1, 1.3, 1.6]
>>> cy = itertools.cycle('ABC') # ❺
>>> next(cy)
'A'
>>> list(itertools.islice(cy, 7)) # ❻
['B', 'C', 'A', 'B', 'C', 'A', 'B']
>>> rp = itertools.repeat(7) # ❼
>>> next(rp), next(rp)
(7, 7)
>>> list(itertools.repeat(8, 4)) # ❽
[8, 8, 8, 8]
>>> list(map(operator.mul, range(11), itertools.repeat(5))) # ❾
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50]
```

- ❶ Build a count generator `ct`.
- ❷ Retrieve the first item from `ct`.
- ❸ I can’t build a list from `ct`, because `ct` never stops, so I fetch the next three items.
- ❹ I can build a list from a count generator if it is limited by `islice` or `takewhile`.
- ❺ Build a cycle generator from `'ABC'` and fetch its first item, `'A'`.
- ❻ A list can only be built if limited by `islice`; the next seven items are retrieved here.
- ❼ Build a repeat generator that will yield the number 7 forever.
- ❽ A repeat generator can be limited by passing the `times` argument: here the number 8 will be produced 4 times.

- ⑨ A common use of `repeat`: providing a fixed argument in `map`; here it provides the 5 multiplier.

The `combinations`, `combinations_with_replacement`, and `permutations` generator functions—together with `product`—are called the *combinatoric generators* in the [itertools documentation page](#). There is a close relationship between `itertools.product` and the remaining *combinatoric* functions as well, as [Example 14-20](#) shows.

Example 14-20. Combinatoric generator functions yield multiple values per input item

```
>>> list(itertools.combinations('ABC', 2)) # ❶
[('A', 'B'), ('A', 'C'), ('B', 'C')]
>>> list(itertools.combinations_with_replacement('ABC', 2)) # ❷
[('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'B'), ('B', 'C'), ('C', 'C')]
>>> list(itertools.permutations('ABC', 2)) # ❸
[('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
>>> list(itertools.product('ABC', repeat=2)) # ❹
[('A', 'A'), ('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'B'), ('B', 'C'),
 ('C', 'A'), ('C', 'B'), ('C', 'C')]
```

- ❶ All combinations of `len()==2` from the items in `'ABC'`; item ordering in the generated tuples is irrelevant (they could be sets).
- ❷ All combinations of `len()==2` from the items in `'ABC'`, including combinations with repeated items.
- ❸ All permutations of `len()==2` from the items in `'ABC'`; item ordering in the generated tuples is relevant.
- ❹ Cartesian product from `'ABC'` and `'ABC'` (that's the effect of `repeat=2`).

The last group of generator functions we'll cover in this section are designed to yield all items in the input iterables, but rearranged in some way. Here are two functions that return multiple generators: `itertools.groupby` and `itertools.tee`. The other generator function in this group, the reversed built-in, is the only one covered in this section that does not accept any iterable as input, but only sequences. This makes sense: because `reversed` will yield the items from last to first, it only works with a sequence with a known length. But it avoids the cost of making a reversed copy of the sequence by yielding each item as needed. I put the `itertools.product` function together with the *merging generators* in [Table 14-3](#) because they all consume more than one iterable, while the generators in [Table 14-5](#) all accept at most one input iterable.

Table 14-5. Rearranging generator functions

Module	Function	Description
<code>itertools</code>	<code>groupby(it, key=None)</code>	Yields 2-tuples of the form <code>(key, group)</code> , where <code>key</code> is the grouping criterion and <code>group</code> is a generator yielding the items in the group

Module	Function	Description
(built-in)	<code>reversed(seq)</code>	Yields items from <code>seq</code> in reverse order, from last to first; <code>seq</code> must be a sequence or implement the <code>__reversed__</code> special method
<code>itertools</code>	<code>tee(it, n=2)</code>	Yields a tuple of <i>n</i> generators, each yielding the items of the input iterable independently

Example 14-21 demonstrates the use of `itertools.groupby` and the `reversed` built-in. Note that `itertools.groupby` assumes that the input iterable is sorted by the grouping criterion, or at least that the items are clustered by that criterion—even if not sorted.

Example 14-21. `itertools.groupby`

```
>>> list(itertools.groupby('LLLLAAGGG')) # ❶
[('L', <itertools._grouper object at 0x102227cc0>),
 ('A', <itertools._grouper object at 0x102227b38>),
 ('G', <itertools._grouper object at 0x102227b70>)]
>>> for char, group in itertools.groupby('LLLLAAGG'): # ❷
...     print(char, '->', list(group))
...
L -> ['L', 'L', 'L', 'L']
A -> ['A', 'A',]
G -> ['G', 'G', 'G']
>>> animals = ['duck', 'eagle', 'rat', 'giraffe', 'bear',
...           'bat', 'dolphin', 'shark', 'lion']
>>> animals.sort(key=len) # ❸
>>> animals
['rat', 'bat', 'duck', 'bear', 'lion', 'eagle', 'shark',
 'giraffe', 'dolphin']
>>> for length, group in itertools.groupby(animals, len): # ❹
...     print(length, '->', list(group))
...
3 -> ['rat', 'bat']
4 -> ['duck', 'bear', 'lion']
5 -> ['eagle', 'shark']
7 -> ['giraffe', 'dolphin']
>>> for length, group in itertools.groupby(reversed(animals), len): # ❺
...     print(length, '->', list(group))
...
7 -> ['dolphin', 'giraffe']
5 -> ['shark', 'eagle']
4 -> ['lion', 'bear', 'duck']
3 -> ['bat', 'rat']
>>>
```

- ❶ `groupby` yields tuples of (key, `group_generator`).
- ❷ Handling `groupby` generators involves nested iteration: in this case, the outer for loop and the inner `list` constructor.
- ❸ To use `groupby`, the input should be sorted; here the words are sorted by length.

- ④ Again, loop over the key and group pair, to display the key and expand the group into a list.
- ⑤ Here the reverse generator is used to iterate over animals from right to left.

The last of the generator functions in this group is `iterator.tee`, which has a unique behavior: it yields multiple generators from a single input iterable, each yielding every item from the input. Those generators can be consumed independently, as shown in [Example 14-22](#).

Example 14-22. `itertools.tee` yields multiple generators, each yielding every item of the input generator

```
>>> list(itertools.tee('ABC'))
[<itertools._tee object at 0x10222abc8>, <itertools._tee object at 0x10222ac08>]
>>> g1, g2 = itertools.tee('ABC')
>>> next(g1)
'A'
>>> next(g2)
'A'
>>> next(g2)
'B'
>>> list(g1)
['B', 'C']
>>> list(g2)
['C']
>>> list(zip(*itertools.tee('ABC')))
[('A', 'A'), ('B', 'B'), ('C', 'C')]
```

Note that several examples in this section used combinations of generator functions. This is a great feature of these functions: because they all take generators as arguments and return generators, they can be combined in many different ways.

While on the subject of combining generators, the `yield from` statement, new in Python 3.3, is a tool for doing just that.

New Syntax in Python 3.3: `yield from`

Nested for loops are the traditional solution when a generator function needs to yield values produced from another generator.

For example, here is a homemade implementation of a chaining generator:¹²

```
>>> def chain(*iterables):
...     for it in iterables:
...         for i in it:
```

12. The `itertools.chain` from the standard library is written in C.

```

...         yield i
...
>>> s = 'ABC'
>>> t = tuple(range(3))
>>> list(chain(s, t))
['A', 'B', 'C', 0, 1, 2]

```

The chain generator function is delegating to each received iterable in turn. [PEP 380](#) — [Syntax for Delegating to a Subgenerator](#) introduced new syntax for doing that, shown in the next console listing:

```

>>> def chain(*iterables):
...     for i in iterables:
...         yield from i
...
>>> list(chain(s, t))
['A', 'B', 'C', 0, 1, 2]

```

As you can see, `yield from i` replaces the inner `for` loop completely. The use of `yield from` in this example is correct, and the code reads better, but it seems like mere syntactic sugar. Besides replacing a loop, `yield from` creates a channel connecting the inner generator directly to the client of the outer generator. This channel becomes really important when generators are used as coroutines and not only produce but also consume values from the client code. [Chapter 16](#) dives into coroutines, and has several pages explaining why `yield from` is much more than syntactic sugar.

After this first encounter with `yield from`, we'll go back to our review of iterable-savvy functions in the standard library.

Iterable Reducing Functions

The functions in [Table 14-6](#) all take an iterable and return a single result. They are known as “reducing,” “folding,” or “accumulating” functions. Actually, every one of the built-ins listed here can be implemented with `functools.reduce`, but they exist as built-ins because they address some common use cases more easily. Also, in the case of `all` and `any`, there is an important optimization that can't be done with `reduce`: these functions short-circuit (i.e., they stop consuming the iterator as soon as the result is determined). See the last test with `any` in [Example 14-23](#).

Table 14-6. Built-in functions that read iterables and return single values

Module	Function	Description
(built-in)	<code>all(it)</code>	Returns True if all items in <code>it</code> are truthy, otherwise False; <code>all([])</code> returns True
(built-in)	<code>any(it)</code>	Returns True if any item in <code>it</code> is truthy, otherwise False; <code>any([])</code> returns False

Module	Function	Description
(built-in)	<code>max(it, [key=,] [default=])</code>	Returns the maximum value of the items in <code>it</code> ; ^a <code>key</code> is an ordering function, as in <code>sorted</code> ; <code>default</code> is returned if the iterable is empty
(built-in)	<code>min(it, [key=,] [default=])</code>	Returns the minimum value of the items in <code>it</code> . ^b <code>key</code> is an ordering function, as in <code>sorted</code> ; <code>default</code> is returned if the iterable is empty
<code>functools</code>	<code>reduce(func, it, [initial])</code>	Returns the result of applying <code>func</code> to the first pair of items, then to that result and the third item and so on; if given, <code>initial</code> forms the initial pair with the first item
(built-in)	<code>sum(it, start=0)</code>	The sum of all items in <code>it</code> , with the optional <code>start</code> value added (use <code>math.fsum</code> for better precision when adding floats)

^a May also be called as `max(arg1, arg2, ..., [key=?])`, in which case the maximum among the arguments is returned.

^b May also be called as `min(arg1, arg2, ..., [key=?])`, in which case the minimum among the arguments is returned.

The operation of `all` and `any` is exemplified in [Example 14-23](#).

Example 14-23. Results of `all` and `any` for some sequences

```
>>> all([1, 2, 3])
True
>>> all([1, 0, 3])
False
>>> all([])
True
>>> any([1, 2, 3])
True
>>> any([1, 0, 3])
True
>>> any([0, 0.0])
False
>>> any([])
False
>>> g = (n for n in [0, 0.0, 7, 8])
>>> any(g)
True
>>> next(g)
8
```

A longer explanation about `functools.reduce` appeared in [“Vector Take #4: Hashing and a Faster ==”](#) on page 288.

Another built-in that takes an iterable and returns something else is `sorted`. Unlike `reversed`, which is a generator function, `sorted` builds and returns an actual list. After all, every single item of the input iterable must be read so they can be sorted, and the sorting happens in a list, therefore `sorted` just returns that list after it’s done. I mention `sorted` here because it does consume an arbitrary iterable.

Of course, `sorted` and the reducing functions only work with iterables that eventually stop. Otherwise, they will keep on collecting items and never return a result.

We'll now go back to the `iter()` built-in: it has a little-known feature that we haven't covered yet.

A Closer Look at the `iter` Function

As we've seen, Python calls `iter(x)` when it needs to iterate over an object `x`.

But `iter` has another trick: it can be called with two arguments to create an iterator from a regular function or any callable object. In this usage, the first argument must be a callable to be invoked repeatedly (with no arguments) to yield values, and the second argument is a sentinel: a marker value which, when returned by the callable, causes the iterator to raise `StopIteration` instead of yielding the sentinel.

The following example shows how to use `iter` to roll a six-sided die until a 1 is rolled:

```
>>> def d6():
...     return randint(1, 6)
...
>>> d6_iter = iter(d6, 1)
>>> d6_iter
<callable_iterator object at 0x00000000029BE6A0>
>>> for roll in d6_iter:
...     print(roll)
...
4
3
6
3
```

Note that the `iter` function here returns a `callable_iterator`. The `for` loop in the example may run for a very long time, but it will never display 1, because that is the sentinel value. As usual with iterators, the `d6_iter` object in the example becomes useless once exhausted. To start over, you must rebuild the iterator by invoking `iter(...)` again.

A useful example is found in the [iter built-in function documentation](#). This snippet reads lines from a file until a blank line is found or the end of file is reached:

```
with open('mydata.txt') as fp:
    for line in iter(fp.readline, ''):
        process_line(line)
```

To close this chapter, I present a practical example of using generators to handle a large volume of data efficiently.

Case Study: Generators in a Database Conversion Utility

A few years ago I worked at BIREME, a digital library run by PAHO/WHO (Pan-American Health Organization/World Health Organization) in São Paulo, Brazil. Among the bibliographic datasets created by BIREME are LILACS (Latin American and Caribbean Health Sciences index) and SciELO (Scientific Electronic Library Online), two comprehensive databases indexing the scientific and technical literature produced in the region.

Since the late 1980s, the database system used to manage LILACS is CDS/ISIS, a non-relational, document database created by UNESCO and eventually rewritten in C by BIREME to run on GNU/Linux servers. One of my jobs was to research alternatives for a possible migration of LILACS—and eventually the much larger SciELO—to a modern, open source, document database such as CouchDB or MongoDB.

As part of that research, I wrote a Python script, *isis2json.py*, that reads a CDS/ISIS file and writes a JSON file suitable for importing to CouchDB or MongoDB. Initially, the script read files in the ISO-2709 format exported by CDS/ISIS. The reading and writing had to be done incrementally because the full datasets were much bigger than main memory. That was easy enough: each iteration of the main for loop read one record from the *.iso* file, massaged it, and wrote it to the *.json* output.

However, for operational reasons, it was deemed necessary that *isis2json.py* supported another CDS/ISIS data format: the binary *.mst* files used in production at BIREME—to avoid the costly export to ISO-2709.

Now I had a problem: the libraries used to read ISO-2709 and *.mst* files had very different APIs. And the JSON writing loop was already complicated because the script accepted a variety of command-line options to restructure each output record. Reading data using two different APIs in the same for loop where the JSON was produced would be unwieldy.

The solution was to isolate the reading logic into a pair of generator functions: one for each supported input format. In the end, the *isis2json.py* script was split into four functions. You can see the main Python 2 script in [Example A-5](#), but the full source code with dependencies is in [fluentpython/isis2json](#) on GitHub.

Here is a high-level overview of how the script is structured:

`main`

The `main` function uses `argparse` to read command-line options that configure the structure of the output records. Based on the input filename extension, a suitable generator function is selected to read the data and yield the records, one by one.

`iter_iso_records`

This generator function reads *.iso* files (assumed to be in the ISO-2709 format). It takes two arguments: the filename and `isis_json_type`, one of the options related to the record structure. Each iteration of its for loop reads one record, creates an empty dict, populates it with field data, and yields the dict.

`iter_mst_records`

This other generator functions reads *.mst* files.¹³ If you look at the source code for *isis2json.py*, you'll see that it's not as simple as `iter_iso_records`, but its interface and overall structure is the same: it takes a filename and an `isis_json_type` argument and enters a for loop, which builds and yields one dict per iteration, representing a single record.

`write_json`

This function performs the actual writing of the JSON records, one at a time. It takes numerous arguments, but the first one—`input_gen`—is a reference to a generator function: either `iter_iso_records` or `iter_mst_records`. The main for loop in `write_json` iterates over the dictionaries yielded by the selected `input_gen` generator, massages it in several ways as determined by the command-line options, and appends the JSON record to the output file.

By leveraging generator functions, I was able to decouple the reading logic from the writing logic. Of course, the simplest way to decouple them would be to read all records to memory, then write them to disk. But that was not a viable option because of the size of the datasets. Using generators, the reading and writing is interleaved, so the script can process files of any size.

Now if *isis2json.py* needs to support an additional input format—say, MARCXML, a DTD used by the U.S. Library of Congress to represent ISO-2709 data—it will be easy to add a third generator function to implement the reading logic, without changing anything in the complicated `write_json` function.

This is not rocket science, but it's a real example where generators provided a flexible solution to processing databases as a stream of records, keeping memory usage low regardless of the amount of data. Anyone who manages large datasets finds many opportunities for using generators in practice.

The next section addresses an aspect of generators that we'll actually skip for now. Read on to understand why.

13. The library used to read the complex *.mst* binary is actually written in Java, so this functionality is only available when *isis2json.py* is executed with the Jython interpreter, version 2.5 or newer. For further details, see the *README.rst* file in the repository. The dependencies are imported inside the generator functions that need them, so the script can run even if only one of the external libraries is available.

Generators as Coroutines

About five years after generator functions with the `yield` keyword were introduced in Python 2.2, [PEP 342 — Coroutines via Enhanced Generators](#) was implemented in Python 2.5. This proposal added extra methods and functionality to generator objects, most notably the `.send()` method.

Like `.__next__()`, `.send()` causes the generator to advance to the next `yield`, but it also allows the client using the generator to send data into it: whatever argument is passed to `.send()` becomes the value of the corresponding `yield` expression inside the generator function body. In other words, `.send()` allows two-way data exchange between the client code and the generator—in contrast with `.__next__()`, which only lets the client receive data from the generator.

This is such a major “enhancement” that it actually changes the nature of generators: when used in this way, they become *coroutines*. David Beazley—probably the most prolific writer and speaker about coroutines in the Python community—warned in a famous [PyCon US 2009 tutorial](#):

- Generators produce data for iteration
- Coroutines are consumers of data
- To keep your brain from exploding, you don’t mix the two concepts together
- Coroutines are not related to iteration
- Note: There is a use of having `yield` produce a value in a coroutine, but it’s not tied to iteration.¹⁴

— David Beazley
“A Curious Course on Coroutines and Concurrency”

I will follow Dave’s advice and close this chapter—which is really about iteration techniques—without touching `send` and the other features that make generators usable as coroutines. Coroutines will be covered in [Chapter 16](#).

Chapter Summary

Iteration is so deeply embedded in the language that I like to say that Python groks iterators.¹⁵ The integration of the Iterator pattern in the semantics of Python is a prime example of how design patterns are not equally applicable in all programming lan-

14. Slide 33, “Keeping It Straight,” in [“A Curious Course on Coroutines and Concurrency”](#).

15. According to the [Jargon file](#), to *grok* is not merely to learn something, but to absorb it so “it becomes part of you, part of your identity.”

guages. In Python, a classic iterator implemented “by hand” as in [Example 14-4](#) has no practical use, except as a didactic example.

In this chapter, we built a few versions of a class to iterate over individual words in text files that may be very long. Thanks to the use of generators, the successive refactorings of the `Sentence` class become shorter and easier to read—when you know how they work.

We then coded a generator of arithmetic progressions and showed how to leverage the `itertools` module to make it simpler. An overview of 24 general-purpose generator functions in the standard library followed.

Following that, we looked at the `iter` built-in function: first, to see how it returns an iterator when called as `iter(o)`, and then to study how it builds an iterator from any function when called as `iter(func, sentinel)`.

For practical context, I described the implementation of a database conversion utility using generator functions to decouple the reading to the writing logic, enabling efficient handling of large datasets and making it easy to support more than one data input format.

Also mentioned in this chapter were the `yield from` syntax, new in Python 3.3, and coroutines. Both topics were just introduced here; they get more coverage later in the book.

Further Reading

A detailed technical explanation of generators appears in *The Python Language Reference* in [6.2.9. Yield expressions](#). The PEP where generator functions were defined is [PEP 255 — Simple Generators](#).

The [itertools module documentation](#) is excellent because of all the examples included. Although the functions in that module are implemented in C, the documentation shows how many of them would be written in Python, often by leveraging other functions in the module. The usage examples are also great: for instance, there is a snippet showing how to use the `accumulate` function to amortize a loan with interest, given a list of payments over time. There is also an [Itertools Recipes](#) section with additional high-performance functions that use the `itertools` functions as building blocks.

Chapter 4, “Iterators and Generators,” of *Python Cookbook, 3E* (O’Reilly), by David Beazley and Brian K. Jones, has 16 recipes covering this subject from many different angles, always focusing on practical applications.

The `yield from` syntax is explained with examples in *What’s New in Python 3.3* (see [PEP 380: Syntax for Delegating to a Subgenerator](#)). We’ll also cover it in detail in “Using `yield from`” on page 477 and “The Meaning of `yield from`” on page 483 in [Chapter 16](#).

If you are interested in document databases and would like to learn more about the context of “[Case Study: Generators in a Database Conversion Utility](#)” on page 437, the Code4Lib Journal—which covers the intersection between libraries and technology—published my paper “[From ISIS to CouchDB: Databases and Data Models for Bibliographic Records](#)”. One section of the paper describes the *isis2json.py* script. The rest of it explains why and how the semistructured data model implemented by document databases like CouchDB and MongoDB are more suitable for cooperative bibliographic data collection than the relational model.

Soapbox

Generator Function Syntax: More Sugar Would Be Nice

Designers need to ensure that controls and displays for different purposes are significantly different from one another.

— Donald Norman
The Design of Everyday Things

Source code plays the role of “controls and displays” in programming languages. I think Python is exceptionally well designed; its source code is often as readable as pseudocode. But nothing is perfect. Guido van Rossum should have followed Donald Norman’s advice (previously quoted) and introduced another keyword for defining generator expressions, instead of reusing `def`. The “BDFL Pronouncements” section of [PEP 255 — Simple Generators](#) actually argues:

A “`yield`” statement buried in the body is not enough warning that the semantics are so different.

But Guido hates introducing new keywords and he did not find that argument convincing, so we are stuck with `def`.

Reusing the function syntax for generators has other bad consequences. In the paper and experimental work “Python, the Full Monty: A Tested Semantics for the Python Programming Language,” Politz¹⁶ et al. show this trivial example of a generator function (section 4.1 of the paper):

```
def f(): x=0
    while True:
        x += 1
        yield x
```

The authors then make the point that we can’t abstract the process of yielding with a function call ([Example 14-24](#)).

16. Joe Gibbs Politz, Alejandro Martinez, Matthew Milano, Sumner Warren, Daniel Patterson, Junsong Li, Anand Chitipothu, and Shriram Krishnamurthi, “Python: The Full Monty,” SIGPLAN Not. 48, 10 (October 2013), 217-232.

Example 14-24. “[This] seems to perform a simple abstraction over the process of yielding” (Politz et al.)

```
def f():
    def do_yield(n):
        yield n
    x = 0
    while True:
        x += 1
        do_yield(x)
```

If we call `f()` in [Example 14-24](#), we get an infinite loop, and not a generator, because the `yield` keyword only makes the immediately enclosing function a generator function. Although generator functions look like functions, we cannot delegate another generator function with a simple function call. As a point of comparison, the Lua language does not impose this limitation. A Lua coroutine can call other functions and any of them can yield to the original caller.

The new `yield from` syntax was introduced to allow a Python generator or coroutine to delegate work to another, without requiring the workaround of an inner `for` loop. [Example 14-24](#) can be “fixed” by prefixing the function call with `yield from`, as in [Example 14-25](#).

Example 14-25. This actually performs a simple abstraction over the process of yielding

```
def f():
    def do_yield(n):
        yield n
    x = 0
    while True:
        x += 1
        yield from do_yield(x)
```

Reusing `def` for declaring generators was a usability mistake, and the problem was compounded in Python 2.5 with coroutines, which are also coded as functions with `yield`. In the case of coroutines, the `yield` just happens to appear—usually—on the righthand side of an assignment, because it receives the argument of the `.send()` call from the client. As David Beazley says:

Despite some similarities, generators and coroutines are basically two different concepts.¹⁷

I believe coroutines also deserved their own keyword. As we’ll see later, coroutines are often used with special decorators, which do set them apart from other functions. But generator functions are not decorated as frequently, so we have to scan their bodies for `yield` to realize they are not functions at all, but a completely different beast.

17. Slide 31, “[A Curious Course on Coroutines and Concurrency](#)”.

It can be argued that, because those features were made to work with little additional syntax, extra syntax would be merely “syntactic sugar.” I happen to like syntactic sugar when it makes features that are different look different. The lack of syntactic sugar is the main reason why Lisp code is hard to read: every language construct in Lisp looks like a function call.

Semantics of Generator Versus Iterator

There are at least three ways of thinking about the relationship between iterators and generators.

The first is the interface viewpoint. The Python iterator protocol defines two methods: `__next__` and `__iter__`. Generator objects implement both, so from this perspective, every generator is an iterator. By this definition, objects created by the `enumerate()` built-in are iterators:

```
>>> from collections import abc
>>> e = enumerate('ABC')
>>> isinstance(e, abc.Iterator)
True
```

The second is the implementation viewpoint. From this angle, a generator is a Python language construct that can be coded in two ways: as a function with the `yield` keyword or as a generator expression. The generator objects resulting from calling a generator function or evaluating a generator expression are instances of an internal **Generator Type**. From this perspective, every generator is also an iterator, because **Generator Type** instances implement the iterator interface. But you can write an iterator that is not a generator—by implementing the classic Iterator pattern, as we saw in [Example 14-4](#), or by coding an extension in C. The `enumerate` objects are not generators from this perspective:

```
>>> import types
>>> e = enumerate('ABC')
>>> isinstance(e, types.GeneratorType)
False
```

This happens because `types.GeneratorType` is defined as “The type of generator-iterator objects, produced by calling a generator function.”

The third is the conceptual viewpoint. In the classic Iterator design pattern—as defined in the GoF book—the iterator traverses a collection and yields items from it. The iterator may be quite complex; for example, it may navigate through a tree-like data structure. But, however much logic is in a classic iterator, it always reads values from an existing data source, and when you call `next(it)`, the iterator is not expected to change the item it gets from the source; it’s supposed to just yield it as is.

In contrast, a generator may produce values without necessarily traversing a collection, like `range` does. And even if attached to a collection, generators are not limited to yielding just the items in it, but may yield some other values derived from them. A clear example of this is the `enumerate` function. By the original definition of the design pat-

tern, the generator returned by `enumerate` is not an iterator because it creates the tuples it yields.

At this conceptual level, the implementation technique is irrelevant. You can write a generator without using a Python generator object. **Example 14-26** is a Fibonacci generator I wrote just to make this point.

Example 14-26. fibo_by_hand.py: Fibonacci generator without `GeneratorType` instances

```
class Fibonacci:

    def __iter__(self):
        return FibonacciGenerator()

class FibonacciGenerator:

    def __init__(self):
        self.a = 0
        self.b = 1

    def __next__(self):
        result = self.a
        self.a, self.b = self.b, self.a + self.b
        return result

    def __iter__(self):
        return self
```

Example 14-26 works but is just a silly example. Here is the Pythonic Fibonacci generator:

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
```

And of course, you can always use the generator language construct to perform the basic duties of an iterator: traversing a collection and yielding items from it.

In reality, Python programmers are not strict about this distinction: generators are also called iterators, even in the official docs. The canonical definition of an iterator in the **Python Glossary** is so general it encompasses both iterators and generators:

Iterator: An object representing a stream of data. [...]

The full definition of *iterator* in the Python Glossary is worth reading. On the other hand, the definition of *generator* there treats *iterator* and *generator* as synonyms, and uses the word “generator” to refer both to the generator function and the generator

object it builds. So, in the Python community lingo, iterator and generator are fairly close synonyms.

The Minimalistic Iterator Interface in Python

In the “Implementation” section of the Iterator pattern,¹⁸ the *Gang of Four* wrote:

The minimal interface to Iterator consists of the operations First, Next, IsDone, and CurrentItem.

However, that very sentence has a footnote which reads:

We can make this interface even smaller by merging Next, IsDone, and CurrentItem into a single operation that advances to the next object and returns it. If the traversal is finished, then this operation returns a special value (0, for instance) that marks the end of the iteration.

This is close to what we have in Python: the single method `__next__` does the job. But instead of using a sentinel, which could be overlooked by mistake, the `StopIteration` exception signals the end of the iteration. Simple and correct: that’s the Python way.

18. Gamma et. al., *Design Patterns: Elements of Reusable Object-Oriented Software*, p. 261.

HPP Ch 5 | Iterators and Generators (performance)

Iterators and Generators

Questions You'll Be Able to Answer After This Chapter

- How do generators save memory?
- When is the best time to use a generator?
- How can I use `itertools` to create complex generator workflows?
- When is lazy evaluation beneficial, and when is it not?

When many people with experience in another language start learning Python, they are taken aback by the difference in `for` loop notation. That is to say, instead of writing

```
# Other languages
for (i=0; i<N; i++) {
    do_work(i);
}
```

they are introduced to a new function called `range`:

```
# Python
for i in range(N):
    do_work(i)
```

It seems that in the Python code sample we are calling a function, `range`, which creates all of the data we need for the `for` loop to continue. Intuitively, this can be quite a time-consuming process—if we are trying to loop over the numbers 1 through 100,000,000, then we need to spend a lot of time creating that array! However, this is where *generators* come into play: they essentially allow us to lazily evaluate these sorts

of functions so we can have the code-readability of these special-purpose functions without the performance impacts.

To understand this concept, let's implement a function that calculates several Fibonacci numbers both by filling a list and by using a generator:

```
def fibonacci_list(num_items):
    numbers = []
    a, b = 0, 1
    while len(numbers) < num_items:
        numbers.append(a)
        a, b = b, a+b
    return numbers

def fibonacci_gen(num_items):
    a, b = 0, 1
    while num_items:
        yield a ❶
        a, b = b, a+b
        num_items -= 1
```

- ❶ This function will yield many values instead of returning one value. This turns this regular-looking function into a generator that can be polled repeatedly for the next available value.

The first thing to note is that the `fibonacci_list` implementation must create and store the list of all the relevant Fibonacci numbers. So if we want to have 10,000 numbers of the sequence, the function will do 10,000 appends to the `numbers` list (which, as we discussed in [Chapter 3](#), has overhead associated with it) and then return it.

On the other hand, the generator is able to “return” many values. Every time the code gets to the `yield`, the function emits its value, and when another value is requested, the function resumes running (maintaining its previous state) and emits the new value. When the function reaches its end, a `StopIteration` exception is thrown, indicating that the given generator has no more values. As a result, even though both functions must, in the end, do the same number of calculations, the `fibonacci_list` version of the preceding loop uses 10,000× more memory (or `num_items` times more memory).

With this code in mind, we can decompose the `for` loops that use our implementations of `fibonacci_list` and `fibonacci_gen`. In Python, `for` loops require that the object we are looping over supports iteration. This means that we must be able to create an iterator out of the object we want to loop over. To create an iterator from almost any object, we can use Python's built-in `iter` function. This function, for lists, tuples, dictionaries, and sets, returns an iterator over the items or keys in the object. For more complex objects, `iter` returns the result of the `__iter__` property of the object. Since `fibonacci_gen` already returns an iterator, calling `iter` on it is a trivial

operation, and it returns the original object (so `type(fibonacci_gen(10)) == type(iter(fibonacci_gen(10)))`). However, since `fibonacci_list` returns a list, we must create a new object, a list iterator, that will iterate over all values in the list. In general, once an iterator is created, we call the `next()` function with it, retrieving new values until a `StopIteration` exception is thrown. This gives us a good deconstructed view of for loops, as illustrated in [Example 5-1](#).

Example 5-1. Python for loop deconstructed

```
# The Python loop
for i in object:
    do_work(i)

# Is equivalent to
object_iterator = iter(object)
while True:
    try:
        i = next(object_iterator)
    except StopIteration:
        break
    else:
        do_work(i)
```

The for loop code shows that we are doing extra work calling `iter` when using `fibonacci_list` instead of `fibonacci_gen`. When using `fibonacci_gen`, we create a generator that is trivially transformed into an iterator (since it is already an iterator!); however, for `fibonacci_list` we need to allocate a new list and precompute its values, and then we still must create an iterator.

More importantly, precomputing the `fibonacci_list` list requires allocating enough space for the full dataset and setting each element to the correct value, even though we always require only one value at a time. This also makes the list allocation useless. In fact, it may even make the loop unrunnable, because it may be trying to allocate more memory than is available (`fibonacci_list(100_000_000)` would create a list 3.1 GB large!). By timing the results, we can see this very explicitly:

```
def test_fibonacci_list():
    """
    >>> %timeit test_fibonacci_list()
    332 ms ± 13.1 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

    >>> %memit test_fibonacci_list()
    peak memory: 492.82 MiB, increment: 441.75 MiB
    """
    for i in fibonacci_list(100_000):
        pass
```

```
def test_fibonacci_gen():
    """
    >>> %timeit test_fibonacci_gen()
    126 ms ± 905 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)

    >>> %memit test_fibonacci_gen()
    peak memory: 51.13 MiB, increment: 0.00 MiB
    """
    for i in fibonacci_gen(100_000):
        pass
```

As we can see, the generator version is over twice as fast and requires no measurable memory as compared to the `fibonacci_list`'s 441 MB. It may seem at this point that you should use generators everywhere in place of creating lists, but that would create many complications.

What if, for example, you needed to reference the list of Fibonacci numbers multiple times? In this case, `fibonacci_list` would provide a precomputed list of these digits, while `fibonacci_gen` would have to recompute them over and over again. In general, changing to using generators instead of precomputed arrays requires algorithmic changes that are sometimes not so easy to understand.¹



An important choice that must be made when architecting your code is whether you are going to optimize CPU speed or memory efficiency. In some cases, using extra memory so that you have values precalculated and ready for future reference will save in overall speed. Other times, memory may be so constrained that the only solution is to recalculate values as opposed to saving them in memory. Every problem has its own considerations for this CPU/memory trade-off.

One simple example of this that is often seen in source code is using a generator to create a sequence of numbers, only to use list comprehension to calculate the length of the result:

```
divisible_by_three = len([n for n in fibonacci_gen(100_000) if n % 3 == 0])
```

While we are still using `fibonacci_gen` to generate the Fibonacci sequence as a generator, we are then saving all values divisible by 3 into an array, only to take the length of that array and then throw away the data. In the process, we're consuming 86 MB of data for no reason.² In fact, if we were doing this for a long enough

¹ In general, algorithms that are *online* or *single pass* are a great fit for generators. However, when making the switch you have to ensure your algorithm can still function without being able to reference the data more than once.

² Calculated with `%memit len([n for n in fibonacci_gen(100_000) if n % 3 == 0])`.

Fibonacci sequence, the preceding code wouldn't be able to run because of memory issues, even though the calculation itself is quite simple!

Recall that we can create a list comprehension using a statement of the form [*<value>* for *<item>* in *<sequence>* if *<condition>*]. This will create a list of all the *<value>* items. Alternatively, we can use similar syntax to create a generator of the *<value>* items instead of a list with (*<value>* for *<item>* in *<sequence>* if *<condition>*).

Using this subtle difference between list comprehension and generator comprehension, we can optimize the preceding code for `divisible_by_three`. However, generators do not have a `length` property. As a result, we will have to be a bit clever:

```
divisible_by_three = sum(1 for n in fibonacci_gen(100_000) if n % 3 == 0)
```

Here, we have a generator that emits a value of 1 whenever it encounters a number divisible by 3, and nothing otherwise. By summing all elements in this generator, we are essentially doing the same as the list comprehension version and consuming no significant memory.



Many of Python's built-in functions that operate on sequences are generators themselves (albeit sometimes a special type of generator). For example, `range` returns a generator of values as opposed to the actual list of numbers within the specified range. Similarly, `map`, `zip`, `filter`, `reversed`, and `enumerate` all perform the calculation as needed and don't store the full result. This means that the operation `zip(range(100_000), range(100_000))` will always have only two numbers in memory in order to return its corresponding values, instead of precalculating the result for the entire range beforehand.

The performance of the two versions of this code is almost equivalent for these smaller sequence lengths, but the memory impact of the generator version is far less than that of the list comprehension. Furthermore, we transform the list version into a generator, because all that matters for each element of the list is its current value—either the number is divisible by 3 or it is not; it doesn't matter where its placement is in the list of numbers or what the previous/next values are. More complex functions can also be transformed into generators, but depending on their reliance on state, this can become a difficult thing to do.

Iterators for Infinite Series

Instead of calculating a known number of Fibonacci numbers, what if we instead attempted to calculate all of them?

```
def fibonacci():  
    i, j = 0, 1
```

```

while True:
    yield j
    i, j = j, i + j

```

In this code we are doing something that wouldn't be possible with the previous `fibonacci_list` code: we are encapsulating an infinite series of numbers into a function. This allows us to take as many values as we'd like from this stream and terminate when our code thinks it has had enough.

One reason generators aren't used as much as they could be is that a lot of the logic within them can be encapsulated in your logic code. Generators are really a way of organizing your code and having smarter loops. For example, we could answer the question "How many Fibonacci numbers below 5,000 are odd?" in multiple ways:

```

def fibonacci_naive():
    i, j = 0, 1
    count = 0
    while j <= 5000:
        if j % 2:
            count += 1
        i, j = j, i + j
    return count

def fibonacci_transform():
    count = 0
    for f in fibonacci():
        if f > 5000:
            break
        if f % 2:
            count += 1
    return count

from itertools import takewhile
def fibonacci_succinct():
    first_5000 = takewhile(lambda x: x <= 5000,
                           fibonacci())
    return sum(1 for x in first_5000
               if x % 2)

```

All of these methods have similar runtime properties (as measured by their memory footprint and runtime performance), but the `fibonacci_transform` function benefits from several things. First, it is much more verbose than `fibonacci_succinct`, which means it will be easy for another developer to debug and understand. The latter mainly stands as a warning for the next section, where we cover some common workflows using `itertools`—while the module greatly simplifies many simple actions with iterators, it can also quickly make Python code very un-Pythonic. Conversely, `fibonacci_naive` is doing multiple things at a time, which hides the actual calculation it is doing! While it is obvious in the generator function that we are iterating over the Fibonacci numbers, we are not overencumbered by the actual calculation.

Last, `fibonacci_transform` is more generalizable. This function could be renamed `num_odd_under_5000` and take in the generator by argument, and thus work over any series.

One additional benefit of the `fibonacci_transform` and `fibonacci_succinct` functions is that they support the notion that in computation there are two phases: generating data and transforming data. These functions are very clearly performing a transformation on data, while the `fibonacci` function generates it. This demarcation adds extra clarity and functionality: we can move a transformative function to work on a new set of data, or perform multiple transformations on existing data. This paradigm has always been important when creating complex programs; however, generators facilitate this clearly by making generators responsible for creating the data and normal functions responsible for acting on the generated data.

Lazy Generator Evaluation

As touched on previously, the way we get the memory benefits with a generator is by dealing only with the current values of interest. At any point in our calculation with a generator, we have only the current value and cannot reference any other items in the sequence (algorithms that perform this way are generally called *single pass* or *online*). This can sometimes make generators more difficult to use, but many modules and functions can help.

The main library of interest is `itertools`, in the standard library. It supplies many other useful functions, including these:

`islice`

Allows slicing a potentially infinite generator

`chain`

Chains together multiple generators

`takewhile`

Adds a condition that will end a generator

`cycle`

Makes a finite generator infinite by constantly repeating it

Let's build up an example of using generators to analyze a large dataset. Let's say we've had an analysis routine going over temporal data, one piece of data per second, for the last 20 years—that's 631,152,000 data points! The data is stored in a file, one second per line, and we cannot load the entire dataset into memory. As a result, if we wanted to do some simple anomaly detection, we'd have to use generators to save memory!

The problem will be: Given a datafile of the form “timestamp, value,” find days whose values differ from normal distribution. We start by writing the code that will read the file, line by line, and output each line’s value as a Python object. We will also create a `read_fake_data` generator to generate fake data that we can test our algorithms with. For this function we still take the argument `filename`, so as to have the same function signature as `read_data`; however, we will simply disregard it. These two functions, shown in [Example 5-2](#), are indeed lazily evaluated—we read the next line in the file, or generate new fake data, only when the `next()` function is called.

Example 5-2. Lazily reading data

```
from random import normalvariate, randint
from itertools import count
from datetime import datetime

def read_data(filename):
    with open(filename) as fd:
        for line in fd:
            data = line.strip().split(',')
            timestamp, value = map(int, data)
            yield datetime.fromtimestamp(timestamp), value

def read_fake_data(filename):
    for timestamp in count():
        # We insert an anomalous data point approximately once a week
        if randint(0, 7 * 60 * 60 * 24 - 1) == 1:
            value = normalvariate(0, 1)
        else:
            value = 100
        yield datetime.fromtimestamp(timestamp), value
```

Now we’d like to create a function that outputs groups of data that occur in the same day. For this, we can use the `groupby` function in `itertools` ([Example 5-3](#)). This function works by taking in a sequence of items and a key used to group these items. The output is a generator that produces tuples whose items are the key for the group and a generator for the items in the group. As our key function, we will output the calendar day that the data was recorded. This “key” function could be anything—we could group our data by hour, by year, or by some property in the actual value. The only limitation is that groups will be formed only for data that is sequential. So if we had the input `A A A B B A A` and had `groupby` group by the letter, we would get three groups: `(A, [A, A, A, A])`, `(B, [B, B])`, and `(A, [A, A])`.

Example 5-3. Grouping our data

```
from itertools import groupby

def groupby_day(iterable):
    key = lambda row: row[0].day
    for day, data_group in groupby(iterable, key):
        yield list(data_group)
```

Now to do the actual anomaly detection. We do this in [Example 5-4](#) by creating a function that, given one group of data, returns whether it follows the normal distribution (using `scipy.stats.normaltest`). We can use this check with `itertools.filterfalse` to filter down the full dataset only to inputs that *don't* pass the test. These inputs are what we consider to be anomalous.



In [Example 5-3](#), we cast `data_group` into a list, even though it is provided to us as an iterator. This is because the `normaltest` function requires an array-like object. We could, however, write our own `normaltest` function that is “one-pass” and could operate on a single view of the data. This could be done without too much trouble by using [Welford's online averaging algorithm](#) to calculate the skew and kurtosis of the numbers. This would save us even more memory by always storing only a single value of the dataset in memory at once instead of storing a full day at a time. However, performance time regressions and development time should be taken into consideration: is storing one day of data in memory at a time sufficient for this problem, or does it need to be further optimized?

Example 5-4. Generator-based anomaly detection

```
from scipy.stats import normaltest
from itertools import filterfalse

def is_normal(data, threshold=1e-3):
    _, values = zip(*data)
    k2, p_value = normaltest(values)
    if p_value < threshold:
        return False
    return True

def filter_anomalous_groups(data):
    yield from filterfalse(is_normal, data)
```

Finally, we can put together the chain of generators to get the days that had anomalous data ([Example 5-5](#)).

Example 5-5. Chaining together our generators

```
from itertools import islice

def filter_anomalous_data(data):
    data_group = groupby_day(data)
    yield from filter_anomalous_groups(data_group)

data = read_data(filename)
anomaly_generator = filter_anomalous_data(data)
first_five_anomalies = islice(anomaly_generator, 5)

for data_anomaly in first_five_anomalies:
    start_date = data_anomaly[0][0]
    end_date = data_anomaly[-1][0]
    print(f"Anomaly from {start_date} - {end_date}")

# Output of above code using "read_fake_data"
Anomaly from 1970-01-10 00:00:00 - 1970-01-10 23:59:59
Anomaly from 1970-01-17 00:00:00 - 1970-01-17 23:59:59
Anomaly from 1970-01-18 00:00:00 - 1970-01-18 23:59:59
Anomaly from 1970-01-23 00:00:00 - 1970-01-23 23:59:59
Anomaly from 1970-01-29 00:00:00 - 1970-01-29 23:59:59
```

This method allows us to get the list of days that are anomalous without having to load the entire dataset. Only enough data is read to generate the first five anomalies. Additionally, the `anomaly_generator` object can be read further to continue retrieving anomalous data. This is called *lazy evaluation*—only the calculations that are explicitly requested are performed, which can drastically reduce overall runtime if there is an early termination condition.

Another nicety about organizing analysis this way is it allows us to do more expansive calculations easily, without having to rework large parts of the code. For example, if we want to have a moving window of one day instead of chunking up by days, we can replace the `groupby_day` in [Example 5-3](#) with something like this:

```
from datetime import datetime

def groupby_window(data, window_size=3600):
    window = tuple(islice(data, window_size))
    for item in data:
        yield window
        window = window[1:] + (item,)
```

In this version, we also see very explicitly the memory guarantee of this and the previous method—it will store only the window’s worth of data as state (in both cases, one day, or 3,600 data points). Note that the first item retrieved by the `for` loop is the `window_size`-th value. This is because `data` is an iterator, and in the previous line we consumed the first `window_size` values.

A final note: in the `groupby_window` function, we are constantly creating new tuples, filling them with data, and yielding them to the caller. We can greatly optimize this by using the deque object in the `collections` module. This object gives us $O(1)$ appends and removals to and from the beginning or end of a list (while normal lists are $O(1)$ for appends or removals to/from the end of the list and $O(n)$ for the same operations at the beginning of the list). Using the deque object, we can append the new data to the right (or end) of the list and use `deque.popleft()` to delete data from the left (or beginning) of the list without having to allocate more space or perform long $O(n)$ operations. However, we would have to work on the deque object in-place and destroy previous views to the rolling window (see “[Memory Allocations and In-Place Operations](#)” on page 133 for more about in-place operations). The only way around this would be to copy the data into a tuple before yielding it back to the caller, which gets rid of any benefit of the change!

Wrap-Up

By formulating our anomaly-finding algorithm with iterators, we can process much more data than could fit into memory. What’s more, we can do it faster than if we had used lists, since we avoid all the costly append operations.

Since iterators are a primitive type in Python, this should always be a go-to method for trying to reduce the memory footprint of an application. The benefits are that results are lazily evaluated, so you process only the data you need, and memory is saved since we don’t store previous results unless explicitly required to. In [Chapter 11](#), we will talk about other methods that can be used for more specific problems and introduce some new ways of looking at problems when RAM is an issue.

Another benefit of solving problems using iterators is that it prepares your code to be used on multiple CPUs or multiple computers, as we will see in [Chapters 9 and 10](#). As we discussed in “[Iterators for Infinite Series](#)” on page 101, when working with iterators, you must always think about the various states that are necessary for your algorithm to work. Once you figure out how to package the state necessary for the algorithm to run, it doesn’t matter where it runs. We can see this sort of paradigm, for example, with the `multiprocessing` and `ipython` modules, both of which use a `map`-like function to launch parallel tasks.

HPP Ch 6 | Matrix and Vector Computation

Matrix and Vector Computation

Questions You'll Be Able to Answer After This Chapter

- What are the bottlenecks in vector calculations?
- What tools can I use to see how efficiently the CPU is doing my calculations?
- Why is `numpy` better at numerical calculations than pure Python?
- What are cache-misses and page-faults?
- How can I track the memory allocations in my code?

Regardless of what problem you are trying to solve on a computer, you will encounter vector computation at some point. Vector calculations are integral to how a computer works and how it tries to speed up runtimes of programs down at the silicon level—the only thing the computer knows how to do is operate on numbers, and knowing how to do several of those calculations at once will speed up your program.

In this chapter we try to unwrap some of the complexities of this problem by focusing on a relatively simple mathematical problem, solving the diffusion equation, and understanding what is happening at the CPU level. By understanding how different Python code affects the CPU and how to effectively probe these things, we can learn how to understand other problems as well.

We will start by introducing the problem and coming up with a quick solution using pure Python. After identifying some memory issues and trying to fix them using pure Python, we will introduce `numpy` and identify how and why it speeds up our code. Then we will start doing some algorithmic changes and specialize our code to solve the problem at hand. By removing some of the generality of the libraries we are using, we will yet again be able to gain more speed. Next, we introduce some extra modules

that will help facilitate this sort of process out in the field, and also explore a cautionary tale about optimizing before profiling.

Finally, we'll take a look at the Pandas library, which builds upon `numpy` by taking columns of homogeneous data and storing them in a table of heterogeneous types. Pandas has grown beyond using pure `numpy` types and now can mix its own missing-data-aware types with `numpy` datatypes. While Pandas is incredibly popular with scientific developers and data scientists, there's a lot of misinformation about ways to make it run quickly; we address some of these issues and give you tips for writing performant and supportable analysis code.

Introduction to the Problem

To explore matrix and vector computation in this chapter, we will repeatedly use the example of diffusion in fluids. Diffusion is one of the mechanisms that moves fluids and tries to make them uniformly mixed.



This section is meant to give a deeper understanding of the equations we will be solving throughout the chapter. It is not strictly necessary that you understand this section in order to approach the rest of the chapter. If you wish to skip this section, make sure to at least look at the algorithm in Examples 6-1 and 6-2 to understand the code we will be optimizing.

On the other hand, if you read this section and want even more explanation, read Chapter 17 of *Numerical Recipes*, 3rd Edition, by William Press et al. (Cambridge University Press).

In this section we will explore the mathematics behind the diffusion equation. This may seem complicated, but don't worry! We will quickly simplify this to make it more understandable. Also, it is important to note that while having a basic understanding of the final equation we are solving will be useful while reading this chapter, it is not completely necessary; the subsequent chapters will focus mainly on various formulations of the code, not the equation. However, having an understanding of the equations will help you gain intuition about ways of optimizing your code. This is true in general—understanding the motivation behind your code and the intricacies of the algorithm will give you deeper insight about possible methods of optimization.

One simple example of diffusion is dye in water: if you put several drops of dye into water at room temperature, the dye will slowly move out until it fully mixes with the water. Since we are not stirring the water, nor is it warm enough to create convection currents, diffusion will be the main process mixing the two liquids. When solving these equations numerically, we pick what we want the initial condition to look like

and are able to evolve the initial condition forward in time to see what it will look like at a later time (see [Figure 6-2](#)).

All this being said, the most important thing to know about diffusion for our purposes is its formulation. Stated as a partial differential equation in one dimension (1D), the diffusion equation is written as follows:

$$\frac{\partial}{\partial t} u(x, t) = D \cdot \frac{\partial^2}{\partial x^2} u(x, t)$$

In this formulation, u is the vector representing the quantities we are diffusing. For example, we could have a vector with values of 0 where there is only water, and of 1 where there is only dye (and values in between where there is mixing). In general, this will be a 2D or 3D matrix representing an actual area or volume of fluid. In this way, we could have u be a 3D matrix representing the fluid in a glass, and instead of simply doing the second derivative along the x direction, we'd have to take it over all axes. In addition, D is a physical value that represents properties of the fluid we are simulating. A large value of D represents a fluid that can diffuse very easily. For simplicity, we will set $D = 1$ for our code but still include it in the calculations.



The diffusion equation is also called the *heat equation*. In this case, u represents the temperature of a region, and D describes how well the material conducts heat. Solving the equation tells us how the heat is being transferred. So instead of solving for how a couple of drops of dye diffuse through water, we might be solving for how the heat generated by a CPU diffuses into a heat sink.

What we will do is take the diffusion equation, which is continuous in space and time, and approximate it using discrete volumes and discrete times. We will do so using Euler's method. *Euler's method* simply takes the derivative and writes it as a difference, such that

$$\frac{\partial}{\partial t} u(x, t) \approx \frac{u(x, t + dt) - u(x, t)}{dt}$$

where dt is now a fixed number. This fixed number represents the time step, or the resolution in time for which we wish to solve this equation. It can be thought of as the frame rate of the movie we are trying to make. As the frame rate goes up (or dt goes down), we get a clearer picture of what happens. In fact, as dt approaches zero, Euler's approximation becomes exact (note, however, that this exactness can be achieved only theoretically, since there is only finite precision on a computer and numerical errors will quickly dominate any results). We can thus rewrite this

equation to figure out what $u(x, t + dt)$ is, given $u(x, t)$. What this means for us is that we can start with some initial state ($u(x, 0)$, representing the glass of water just as we add a drop of dye into it) and churn through the mechanisms we've outlined to "evolve" that initial state and see what it will look like at future times ($u(x, dt)$). This type of problem is called an *initial value problem* or *Cauchy problem*. Doing a similar trick for the derivative in x using the finite differences approximation, we arrive at the final equation:

$$u(x, t + dt) = u(x, t) + dt * D * \frac{u(x + dx, t) + u(x - dx, t) - 2 * u(x, t)}{dx^2}$$

Here, similar to how dt represents the frame rate, dx represents the resolution of the images—the smaller dx is, the smaller a region every cell in our matrix represents. For simplicity, we will set $D = 1$ and $dx = 1$. These two values become very important when doing proper physical simulations; however, since we are solving the diffusion equation for illustrative purposes, they are not important to us.

Using this equation, we can solve almost any diffusion problem. However, there are some considerations regarding this equation. First, we said before that the spatial index in u (i.e., the x parameter) will be represented as the indices into a matrix. What happens when we try to find the value at $x - dx$ when x is at the beginning of the matrix? This problem is called the *boundary condition*. You can have fixed boundary conditions that say "any value out of the bounds of my matrix will be set to 0" (or any other value). Alternatively, you can have periodic boundary conditions that say that the values will wrap around. (That is, if one of the dimensions of your matrix has length N , the value in that dimension at index -1 is the same as at $N - 1$, and the value at N is the same as at index 0 . In other words, if you are trying to access the value at index i , you will get the value at index $(i \% N)$.)

Another consideration is how we are going to store the multiple time components of u . We could have one matrix for every time value we do our calculation for. At minimum, it seems that we will need two matrices: one for the current state of the fluid and one for the next state of the fluid. As we'll see, there are very drastic performance considerations for this particular question.

So what does it look like to solve this problem in practice? [Example 6-1](#) contains some pseudocode to illustrate the way we can use our equation to solve the problem.

Example 6-1. Pseudocode for 1D diffusion

```
# Create the initial conditions
u = vector of length N
for i in range(N):
    u = 0 if there is water, 1 if there is dye
```

```

# Evolve the initial conditions
D = 1
t = 0
dt = 0.0001
while True:
    print(f"Current time is: {t}")
    unew = vector of size N

    # Update step for every cell
    for i in range(N):
        unew[i] = u[i] + D * dt * (u[(i+1)%N] + u[(i-1)%N] - 2 * u[i])
    # Move the updated solution into u
    u = unew

    visualize(u)

```

This code will take an initial condition of the dye in water and tell us what the system looks like at every 0.0001-second interval in the future. The results can be seen in [Figure 6-1](#), where we evolve our very concentrated drop of dye (represented by the top-hat function) into the future. We can see how, far into the future, the dye becomes well mixed, to the point where everywhere has a similar concentration of the dye.

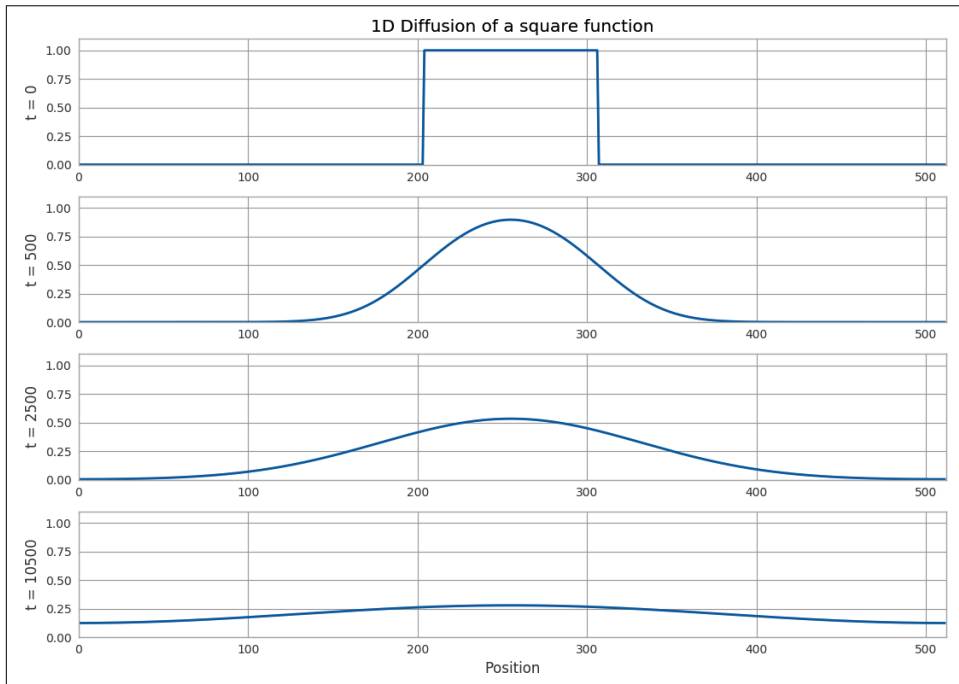


Figure 6-1. Example of 1D diffusion

For the purposes of this chapter, we will be solving the 2D version of the preceding equation. All this means is that instead of operating over a vector (or in other words, a matrix with one index), we will be operating over a 2D matrix. The only change to the equation (and thus to the subsequent code) is that we must now also take the second derivative in the y direction. This simply means that the original equation we were working with becomes the following:

$$\frac{\partial}{\partial t} u(x, y, t) = D \cdot \left(\frac{\partial^2}{\partial x^2} u(x, y, t) + \frac{\partial^2}{\partial y^2} u(x, y, t) \right)$$

This numerical diffusion equation in 2D translates to the pseudocode in [Example 6-2](#), using the same methods we used before.

Example 6-2. Algorithm for calculating 2D diffusion

```
for i in range(N):
    for j in range(M):
        unew[i][j] = u[i][j] + dt * (
            (u[(i + 1) % N][j] + u[(i - 1) % N][j] - 2 * u[i][j]) + # d^2 u / dx^2
            (u[i][(j + 1) % M] + u[i][(j - 1) % M] - 2 * u[i][j]) # d^2 u / dy^2
        )
```

We can now put all of this together and write the full Python 2D diffusion code that we will use as the basis for our benchmarks for the rest of this chapter. While the code looks more complicated, the results are similar to that of the 1D diffusion (as can be seen in [Figure 6-2](#)).

If you'd like to do some additional reading on the topics in this section, check out the [Wikipedia page on the diffusion equation](#) and [Chapter 7](#) of *Numerical Methods for Complex Systems* by S. V. Gurevich.

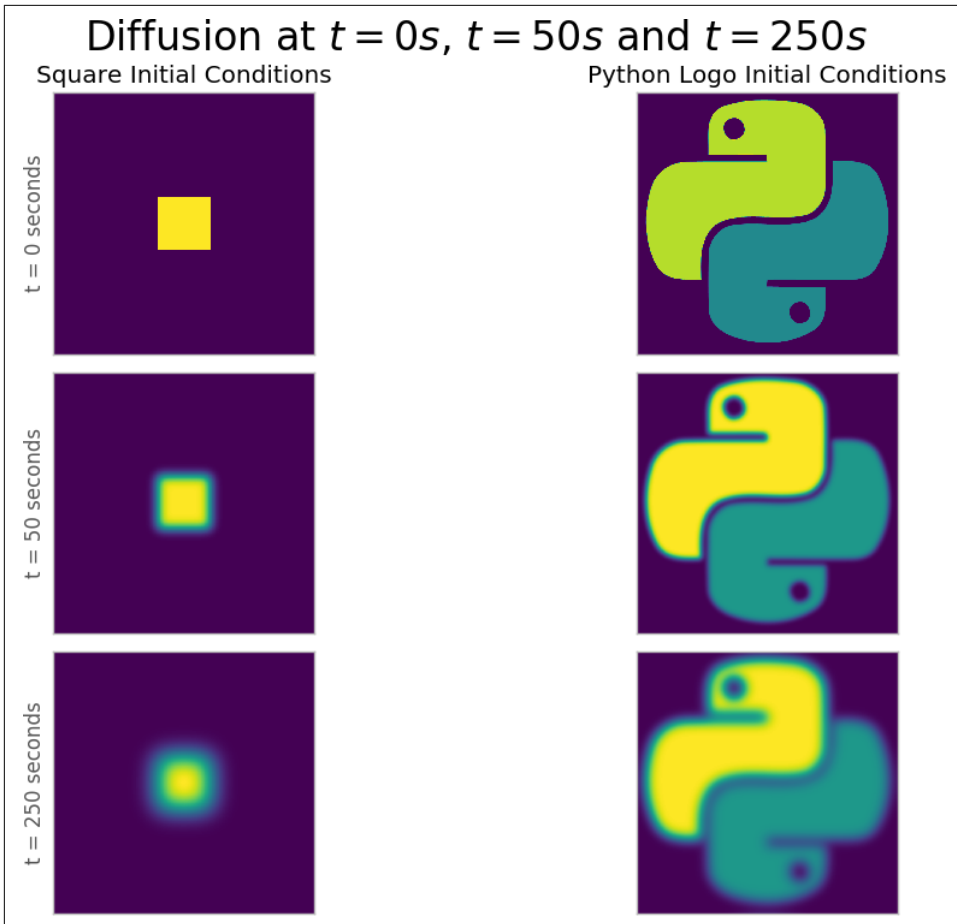


Figure 6-2. Example of 2D diffusion for two sets of initial conditions

Aren't Python Lists Good Enough?

Let's take our pseudocode from [Example 6-1](#) and formalize it so we can better analyze its runtime performance. The first step is to write out the evolution function that takes in the matrix and returns its evolved state. This is shown in [Example 6-3](#).

Example 6-3. Pure Python 2D diffusion

```
grid_shape = (640, 640)
```

```
def evolve(grid, dt, D=1.0):
    xmax, ymax = grid_shape
    new_grid = [[0.0] * ymax for x in range(xmax)]
```

```

for i in range(xmax):
    for j in range(ymax):
        grid_xx = (
            grid[(i + 1) % xmax][j] + grid[(i - 1) % xmax][j] - 2.0 * grid[i][j]
        )
        grid_yy = (
            grid[i][(j + 1) % ymax] + grid[i][(j - 1) % ymax] - 2.0 * grid[i][j]
        )
        new_grid[i][j] = grid[i][j] + D * (grid_xx + grid_yy) * dt
return new_grid

```



Instead of preallocating the `new_grid` list, we could have built it up in the for loop by using appends. While this would have been noticeably faster than what we have written, the conclusions we draw are still applicable. We chose this method because it is more illustrative.

The global variable `grid_shape` designates how big a region we will simulate; and, as explained in “[Introduction to the Problem](#)” on page 110, we are using periodic boundary conditions (which is why we use modulo for the indices). To actually use this code, we must initialize a grid and call `evolve` on it. The code in [Example 6-4](#) is a very generic initialization procedure that will be reused throughout the chapter (its performance characteristics will not be analyzed since it must run only once, as opposed to the `evolve` function, which is called repeatedly).

Example 6-4. Pure Python 2D diffusion initialization

```

def run_experiment(num_iterations):
    # Setting up initial conditions ❶
    xmax, ymax = grid_shape
    grid = [[0.0] * ymax for x in range(xmax)]

    # These initial conditions are simulating a drop of dye in the middle of our
    # simulated region
    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    for i in range(block_low, block_high):
        for j in range(block_low, block_high):
            grid[i][j] = 0.005

    # Evolve the initial conditions
    start = time.time()
    for i in range(num_iterations):
        grid = evolve(grid, 0.1)
    return time.time() - start

```

- ❶ The initial conditions used here are the same as in the square example in [Figure 6-2](#).

The values for `dt` and grid elements have been chosen to be sufficiently small that the algorithm is stable. See *Numerical Recipes* for a more in-depth treatment of this algorithm’s convergence characteristics.

Problems with Allocating Too Much

By using `line_profiler` on the pure Python evolution function, we can start to unravel what is contributing to a possibly slow runtime. Looking at the profiler output in [Example 6-5](#), we see that most of the time in the function is spent doing the derivative calculation and updating the grid.¹ This is what we want, since this is a purely CPU-bound problem—any time not spent on solving the CPU-bound problem is an obvious place for optimization.

Example 6-5. Pure Python 2D diffusion profiling

```
$ kernprof -lv diffusion_python.py
Wrote profile results to diffusion_python.py.lprof
Timer unit: 1e-06 s

Total time: 787.161 s
File: diffusion_python.py
Function: evolve at line 12
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
12					<code>@profile</code>
13					<code>def evolve(grid, dt, D=1.0):</code>
14	500	843.0	1.7	0.0	<code> xmax, ymax = grid_shape ①</code>
15	500	24764794.0	49529.6	3.1	<code> new_grid = [[0.0 for x in ...</code>
16	320500	208683.0	0.7	0.0	<code> for i in range(xmax): ②</code>
17	205120000	128928913.0	0.6	16.4	<code> for j in range(ymax):</code>
18	204800000	222422192.0	1.1	28.3	<code> grid_xx = ...</code>
19	204800000	228660607.0	1.1	29.0	<code> grid_yy = ...</code>
20	204800000	182174957.0	0.9	23.1	<code> new_grid[i][j] = ...</code>
21	500	331.0	0.7	0.0	<code> return new_grid ③</code>

- ① This statement takes such a long time per hit because `grid_shape` must be retrieved from the local namespace (see “[Dictionaries and Namespaces](#)” on page 92 for more information).

¹ This is the code from [Example 6-3](#), truncated to fit within the page margins. Recall that `kernprof` requires functions to be decorated with `@profile` in order to be profiled (see “[Using line_profiler for Line-by-Line Measurements](#)” on page 40).

- ❷ This line has 320,500 hits associated with it, because the grid we operated over had `xmax = 640` and the we ran the function 500 times. The calculation is $(640 + 1) * 500$, where the extra one evaluation is from the termination of the loop.
- ❸ This line has 500 hits associated with it, which informs us that the function was profiled over 500 runs.

It's interesting to see the big difference in the `Per Hit` and `% Time` fields for line 15, where we allocate the new grid. This difference occurs because, while the line itself is quite slow (the `Per Hit` field shows that each run takes 0.0495 seconds, much slower than all other lines), it isn't called as often as other lines inside the loop. If we were to reduce the size of the grid and do more iterations (i.e., reduce the number of iterations of the loop but increase the number of times we call the function), we would see the `% Time` of this line increase and quickly dominate the runtime.

This is a waste, because the properties of `new_grid` do not change—no matter what values we send to `evolve`, the `new_grid` list will always be the same shape and size and contain the same values. A simple optimization would be to allocate this list once and simply reuse it. This way, we need to run this code only once, no matter the size of the grid or the number of iterations. This sort of optimization is similar to moving repetitive code outside a fast loop:

```
from math import sin

def loop_slow(num_iterations):
    """
    >>> %timeit loop_slow(int(1e4))
    1.68 ms ± 61.3 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
    """
    result = 0
    for i in range(num_iterations):
        result += i * sin(num_iterations) ❶
    return result

def loop_fast(num_iterations):
    """
    >>> %timeit loop_fast(int(1e4))
    551 µs ± 23.5 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
    """
    result = 0
    factor = sin(num_iterations)
    for i in range(num_iterations):
        result += i
    return result * factor
```

- ❶ The value of `sin(num_iterations)` doesn't change throughout the loop, so there is no use recalculating it every time.

We can do a similar transformation to our diffusion code, as illustrated in [Example 6-6](#). In this case, we would want to instantiate `new_grid` in [Example 6-4](#) and send it in to our `evolve` function. That function will do the same as it did before: read the grid list and write to the `new_grid` list. Then we can simply swap `new_grid` with `grid` and continue again.

Example 6-6. Pure Python 2D diffusion after reducing memory allocations

```
def evolve(grid, dt, out, D=1.0):
    xmax, ymax = grid_shape
    for i in range(xmax):
        for j in range(ymax):
            grid_xx = (
                grid[(i + 1) % xmax][j] + grid[(i - 1) % xmax][j] - 2.0 * grid[i][j]
            )
            grid_yy = (
                grid[i][(j + 1) % ymax] + grid[i][(j - 1) % ymax] - 2.0 * grid[i][j]
            )
            out[i][j] = grid[i][j] + D * (grid_xx + grid_yy) * dt

def run_experiment(num_iterations):
    # Setting up initial conditions
    xmax, ymax = grid_shape
    next_grid = [[0.0] * ymax for x in range(xmax)]
    grid = [[0.0] * ymax for x in range(xmax)]

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    for i in range(block_low, block_high):
        for j in range(block_low, block_high):
            grid[i][j] = 0.005

    start = time.time()
    for i in range(num_iterations):
        # evolve modifies grid and next_grid in-place
        evolve(grid, 0.1, next_grid)
        grid, next_grid = next_grid, grid
    return time.time() - start
```

We can see from the line profile of the modified version of the code in [Example 6-7](#) that this small change has given us a 31.25% speedup.² This leads us to a conclusion similar to the one made during our discussion of append operations on lists (see “[Lists as Dynamic Arrays](#)” on page 72): memory allocations are not cheap. Every time

² The code profiled in [Example 6-7](#) is the code from [Example 6-6](#); it has been truncated to fit within the page margins.

we request memory to store a variable or a list, Python must take its time to talk to the operating system in order to allocate the new space, and then we must iterate over the newly allocated space to initialize it to some value.

Whenever possible, reusing space that has already been allocated will give performance speedups. However, be careful when implementing these changes. While the speedups can be substantial, as always you should profile to make sure you are achieving the results you want and are not simply polluting your code base.

Example 6-7. Line profiling Python diffusion after reducing allocations

```
$ kernprof -lv diffusion_python_memory.py
Wrote profile results to diffusion_python_memory.py.lprof
Timer unit: 1e-06 s

Total time: 541.138 s
File: diffusion_python_memory.py
Function: evolve at line 12
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
12					@profile
13					def evolve(grid, dt, out, D=1.0):
14	500	503.0	1.0	0.0	xmax, ymax = grid_shape
15	320500	131498.0	0.4	0.0	for i in range(xmax):
16	205120000	81105090.0	0.4	15.0	for j in range(ymax):
17	204800000	166271837.0	0.8	30.7	grid_xx = ...
18	204800000	169216352.0	0.8	31.3	grid_yy = ...
19	204800000	124412452.0	0.6	23.0	out[i][j] = ...

Memory Fragmentation

The Python code we wrote in [Example 6-6](#) still has a problem that is at the heart of using Python for these sorts of vectorized operations: Python doesn't natively support vectorization. There are two reasons for this: Python lists store pointers to the actual data, and Python bytecode is not optimized for vectorization, so for loops cannot predict when using vectorization would be beneficial.

The fact that Python lists store *pointers* means that, instead of actually holding the data we care about, lists store locations where that data can be found. For most uses, this is good because it allows us to store whatever type of data we like inside a list. However, when it comes to vector and matrix operations, this is a source of a lot of performance degradation.

The degradation occurs because every time we want to fetch an element from the grid matrix, we must do multiple lookups. For example, doing `grid[5][2]` requires us to first do a list lookup for index 5 on the list `grid`. This will return a pointer to

where the data at that location is stored. Then we need to do another list lookup on this returned object, for the element at index 2. Once we have this reference, we have the location where the actual data is stored.



Rather than creating a grid as a list-of-lists (`grid[x][y]`), how would you create a grid indexed by a tuple (`grid[(x, y)]`)? How would this affect the performance of the code?

The overhead for one such lookup is not big and can be, in most cases, disregarded. However, if the data we wanted was located in one contiguous block in memory, we could move *all* of the data in one operation instead of needing two operations for each element. This is one of the major points with data fragmentation: when your data is fragmented, you must move each piece over individually instead of moving the entire block over. This means you are invoking more memory transfer overhead, and you are forcing the CPU to wait while data is being transferred. We will see with `perf` just how important this is when looking at the `cache-misses`.

This problem of getting the right data to the CPU when it is needed is related to the *von Neumann bottleneck*. This refers to the limited bandwidth that exists between the memory and the CPU as a result of the tiered memory architecture that modern computers use. If we could move data infinitely fast, we would not need any cache, since the CPU could retrieve any data it needed instantly. This would be a state in which the bottleneck is nonexistent.

Since we can't move data infinitely fast, we must prefetch data from RAM and store it in smaller but faster CPU caches so that, hopefully, when the CPU needs a piece of data, it will be in a location that can be read from quickly. While this is a severely idealized way of looking at the architecture, we can still see some of the problems with it—how do we know what data will be needed in the future? The CPU does a good job with mechanisms called *branch prediction* and *pipelining*, which try to predict the next instruction and load the relevant portions of memory into the cache while still working on the current instruction. However, the best way to minimize the effects of the bottleneck is to be smart about how we allocate our memory and how we do our calculations over our data.

Probing how well memory is being moved to the CPU can be quite hard; however, in Linux the `perf` tool can be used to get amazing amounts of insight into how the CPU is dealing with the program being run.³ For example, we can run `perf` on the pure

³ On macOS you can get similar metrics by using Google's [gperftools](#) and the provided Instruments app. For Windows, we are told Visual Studio Profiler works well; however, we don't have experience with it.

Python code from [Example 6-6](#) and see just how efficiently the CPU is running our code.

The results are shown in [Example 6-8](#). Note that the output in this example and the following `perf` examples has been truncated to fit within the margins of the page. The removed data included variances for each measurement, indicating how much the values changed throughout multiple benchmarks. This is useful for seeing how much a measured value is dependent on the actual performance characteristics of the program versus other system properties, such as other running programs using system resources.

Example 6-8. Performance counters for pure Python 2D diffusion with reduced memory allocations (grid size: 640×640 , 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_python_memory.py
```

Performance counter stats for 'python diffusion_python_memory.py':

415,864,974,126	cycles	#	2.889 GHz
1,210,522,769,388	instructions	#	2.91 insn per cycle
656,345,027	cache-references	#	4.560 M/sec
349,562,390	cache-misses	#	53.259 % of all cache refs
251,537,944,600	branches	#	1747.583 M/sec
1,970,031,461	branch-misses	#	0.78% of all branches
143934.730837	task-clock (msec)	#	1.000 CPUs utilized
12,791	faults	#	0.089 K/sec
12,791	minor-faults	#	0.089 K/sec
117	cs	#	0.001 K/sec
6	migrations	#	0.000 K/sec

143.935522122 seconds time elapsed

Understanding perf

Let's take a second to understand the various performance metrics that `perf` is giving us and their connection to our code. The `task-clock` metric tells us how many clock cycles our task took. This is different from the total runtime, because if our program took one second to run but used two CPUs, then the `task-clock` would be 2000 (`task-clock` is generally in milliseconds). Conveniently, `perf` does the calculation for us and tells us, next to this metric, how many CPUs were utilized (where it says "XXXX CPUs utilized"). This number wouldn't be exactly 2 even when two CPUs are being used, though, because the process sometimes relied on other subsystems to do instructions for it (for example, when memory was allocated).

On the other hand, `instructions` tells us how many CPU instructions our code issued, and `cycles` tells us how many CPU cycles it took to run all of these instructions. The difference between these two numbers gives us an indication of how well our code is vectorizing and pipelining. With pipelining, the CPU is able to run the current operation while fetching and preparing the next one.

`cs` (representing “context switches”) and `CPU-migrations` tell us about how the program is halted in order to wait for a kernel operation to finish (such as I/O), to let other applications run, or to move execution to another CPU core. When a context-switch happens, the program’s execution is halted and another program is allowed to run instead. This is a *very* time-intensive task and is something we would like to minimize as much as possible, but we don’t have too much control over when this happens. The kernel delegates when programs are allowed to be switched out; however, we can do things to disincentivize the kernel from moving *our* program. In general, the kernel suspends a program when it is doing I/O (such as reading from memory, disk, or the network). As you’ll see in later chapters, we can use asynchronous routines to make sure that our program still uses the CPU even when waiting for I/O, which will let us keep running without being context-switched. In addition, we could set the `nice` value of our program to give our program priority and stop the kernel from context-switching it.⁴ Similarly, `CPU-migrations` happen when the program is halted and resumed on a different CPU than the one it was on before, in order to have all CPUs have the same level of utilization. This can be seen as an especially bad context switch, as not only is our program being temporarily halted, but we also lose whatever data we had in the L1 cache (recall that each CPU has its own L1 cache).

A `page-fault` (or just `fault`) is part of the modern Unix memory allocation scheme. When memory is allocated, the kernel doesn’t do much except give the program a reference to memory. Later, however, when the memory is first used, the operating system throws a minor page fault interrupt, which pauses the program that is being run and properly allocates the memory. This is called a *lazy allocation system*. While this method is an optimization over previous memory allocation systems, minor page faults are quite an expensive operation since most of the operations are done outside the scope of the program you are running. There is also a major page fault, which happens when the program requests data from a device (disk, network, etc.) that hasn’t been read yet. These are even more expensive operations: not only do they interrupt your program, but they also involve reading from whichever device the data lives on. This sort of page fault does not generally affect CPU-bound work; however, it will be a source of pain for any program that does disk or network reads/writes.⁵

⁴ This can be done by running the Python process through the `nice` utility (`nice -n -20 python program.py`). A `nice` value of `-20` will make sure it yields execution as little as possible.

⁵ A good survey of the various faults can be found at <https://oreil.ly/12Beq>.

Once we have data in memory and we reference it, the data makes its way through the various tiers of memory (L1/L2/L3 memory—see “[Communications Layers](#)” on [page 8](#) for a discussion of this). Whenever we reference data that is in our cache, the cache-references metric increases. If we do not already have this data in the cache and need to fetch it from RAM, this counts as a cache-miss. We won’t get a cache miss if we are reading data we have read recently (that data will still be in the cache) or data that is located *near* data we have recently read (data is sent from RAM into the cache in chunks). Cache misses can be a source of slowdowns when it comes to CPU-bound work, since we need to wait to fetch the data from RAM *and* we interrupt the flow of our execution pipeline (more on this in a second). As a result, reading through an array in order will give many cache-references but not many cache-misses since if we read element i , element $i + 1$ will already be in cache. If, however, we read randomly through an array or otherwise don’t lay out our data in memory well, every read will require access to data that couldn’t possibly already be in cache. Later in this chapter we will discuss how to reduce this effect by optimizing the layout of data in memory.

A branch is a time in the code where the execution flow changes. Think of an `if...then` statement—depending on the result of the conditional, we will be executing either one section of code or another. This is essentially a branch in the execution of the code—the next instruction in the program could be one of two things. To optimize this, especially with regard to the pipeline, the CPU tries to guess which direction the branch will take and preload the relevant instructions. When this prediction is incorrect, we will get a branch-miss. Branch misses can be quite confusing and can result in many strange effects (for example, some loops will run substantially faster on sorted lists than on unsorted lists simply because there will be fewer branch misses).⁶

There are a lot more metrics that `perf` can keep track of, many of which are very specific to the CPU you are running the code on. You can run `perf list` to get the list of currently supported metrics on your system. For example, in the previous edition of this book, we ran on a machine that also supported `stalled-cycles-frontend` and `stalled-cycles-backend`, which tell us how many cycles our program was waiting for the frontend or backend of the pipeline to be filled. This can happen because of a cache miss, a mispredicted branch, or a resource conflict. The frontend of the pipeline is responsible for fetching the next instruction from memory and decoding it into a valid operation, while the backend is responsible for actually running the operation. These sorts of metrics can help tune the performance of a code to the optimizations and architecture choices of a particular CPU; however, unless you are always running on the same chip-set, it may be excessive to worry too much about them.

⁶ This effect is beautifully explained in this [Stack Overflow response](#).



If you would like a more thorough explanation of what is going on at the CPU level with the various performance metrics, check out Gurpur M. Prabhu's fantastic "[Computer Architecture Tutorial](#)." It deals with the problems at a very low level, which will give you a good understanding of what is going on under the hood when you run your code.

Making Decisions with perf's Output

With all this in mind, the performance metrics in [Example 6-8](#) are telling us that while running our code, the CPU had to reference the L1/L2 cache 656,345,027 times. Of those references, 349,562,390 (or 53.3%) were requests for data that wasn't in memory at the time and had to be retrieved. In addition, we can see that in each CPU cycle we are able to perform an average of 2.91 instructions, which tells us the total speed boost from pipelining, out-of-order execution, and hyperthreading (or any other CPU feature that lets you run more than one instruction per clock cycle).

Fragmentation increases the number of memory transfers to the CPU. Additionally, since you don't have multiple pieces of data ready in the CPU cache when a calculation is requested, you cannot vectorize the calculations. As explained in "[Communications Layers](#)" on [page 8](#), vectorization of computations (or having the CPU do multiple computations at a time) can occur only if we can fill the CPU cache with all the relevant data. Since the bus can only move contiguous chunks of memory, this is possible only if the grid data is stored sequentially in RAM. Since a list stores pointers to data instead of the actual data, the actual values in the grid are scattered throughout memory and cannot be copied all at once.

We can alleviate this problem by using the `array` module instead of lists. These objects store data sequentially in memory, so that a slice of the array actually represents a continuous range in memory. However, this doesn't completely fix the problem—now we have data that is stored sequentially in memory, but Python still does not know how to vectorize our loops. What we would like is for any loop that does arithmetic on our array one element at a time to work on chunks of data, but as mentioned previously, there is no such bytecode optimization in Python (partly because of the extremely dynamic nature of the language).



Why doesn't having the data we want stored sequentially in memory automatically give us vectorization? If we look at the raw machine code that the CPU is running, vectorized operations (such as multiplying two arrays) use a different part of the CPU and different instructions than nonvectorized operations. For Python to use these special instructions, we must have a module that was created to use them. We will soon see how `numpy` gives us access to these specialized instructions.

Furthermore, because of implementation details, using the `array` type when creating lists of data that must be iterated on is actually *slower* than simply creating a `list`. This is because the `array` object stores a very low-level representation of the numbers it stores, and this must be converted into a Python-compatible version before being returned to the user. This extra overhead happens every time you index an `array` type. That implementation decision has made the `array` object less suitable for math and more suitable for storing fixed-type data more efficiently in memory.

Enter numpy

To deal with the fragmentation we found using `perf`, we must find a package that can efficiently vectorize operations. Luckily, `numpy` has all of the features we need—it stores data in contiguous chunks of memory and supports vectorized operations on its data. As a result, any arithmetic we do on `numpy` arrays happens in chunks without us having to explicitly loop over each element.⁷ Not only is it much easier to do matrix arithmetic this way, but it is also faster. Let's look at an example:

```
from array import array
import numpy

def norm_square_list(vector):
    """
    >>> vector = list(range(1_000_000))
    >>> %timeit norm_square_list(vector)
    85.5 ms ± 1.65 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
    """
    norm = 0
    for v in vector:
        norm += v * v
    return norm

def norm_square_list_comprehension(vector):
    """
    >>> vector = list(range(1_000_000))
    >>> %timeit norm_square_list_comprehension(vector)
    80.3 ms ± 1.37 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
    """
    return sum([v * v for v in vector])

def norm_square_array(vector):
    """
```

⁷ For an in-depth look at `numpy` over a variety of problems, check out *From Python to Numpy* by Nicolas P. Rougier.

```

>>> vector_array = array('l', range(1_000_000))
>>> %timeit norm_square_array(vector_array)
101 ms ± 4.69 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
"""

norm = 0
for v in vector:
    norm += v * v
return norm

def norm_square_numpy(vector):
    """
    >>> vector_np = numpy.arange(1_000_000)
    >>> %timeit norm_square_numpy(vector_np)
    3.22 ms ± 136 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
    """
    return numpy.sum(vector * vector) ❶

def norm_square_numpy_dot(vector):
    """
    >>> vector_np = numpy.arange(1_000_000)
    >>> %timeit norm_square_numpy_dot(vector_np)
    960 µs ± 41.1 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
    """
    return numpy.dot(vector, vector) ❷

```

- ❶ This creates two implied loops over `vector`, one to do the multiplication and one to do the sum. These loops are similar to the loops from `norm_square_list_comprehension`, but they are executed using `numpy`'s optimized numerical code.
- ❷ This is the preferred way of doing vector norms in `numpy` by using the vectorized `numpy.dot` operation. The less efficient `norm_square_numpy` code is provided for illustration.

The simpler `numpy` code runs 89× faster than `norm_square_list` and 83.65× faster than the “optimized” Python list comprehension. The difference in speed between the pure Python looping method and the list comprehension method shows the benefit of doing more calculation behind the scenes rather than explicitly in your Python code. By performing calculations using Python’s already-built machinery, we get the speed of the native C code that Python is built on. This is also partly the same reasoning behind why we have such a drastic speedup in the `numpy` code: instead of using the generalized list structure, we have a finely tuned and specially built object for dealing with arrays of numbers.

In addition to more lightweight and specialized machinery, the `numpy` object gives us memory locality and vectorized operations, which are incredibly important when dealing with numerical computations. The CPU is exceedingly fast, and most of the time simply getting it the data it needs faster is the best way to optimize code quickly. Running each function using the `perf` tool we looked at earlier shows that the `array` and pure Python functions take about 10^{12} instructions, while the `numpy` version takes about 10^9 instructions. In addition, the `array` and pure Python versions have around 53% cache misses, while `numpy` has around 20%.

In our `norm_square_numpy` code, when doing `vector * vector`, there is an *implied* loop that `numpy` will take care of. The implied loop is the same loop we have explicitly written out in the other examples: loop over all items in `vector`, multiplying each item by itself. However, since we tell `numpy` to do this instead of explicitly writing it out in Python code, `numpy` can take advantage of all the optimizations it wants. In the background, `numpy` has very optimized C code that has been made specifically to take advantage of any vectorization the CPU has enabled. In addition, `numpy` arrays are represented sequentially in memory as low-level numerical types, which gives them the same space requirements as `array` objects (from the `array` module).

As an added bonus, we can reformulate the problem as a dot product, which `numpy` supports. This gives us a single operation to calculate the value we want, as opposed to first taking the product of the two vectors and then summing them. As you can see in [Figure 6-3](#), this operation, `norm_numpy_dot`, outperforms all the others by quite a substantial margin—this is thanks to the specialization of the function, and because we don't need to store the intermediate value of `vector * vector` as we did in `norm_numpy`.

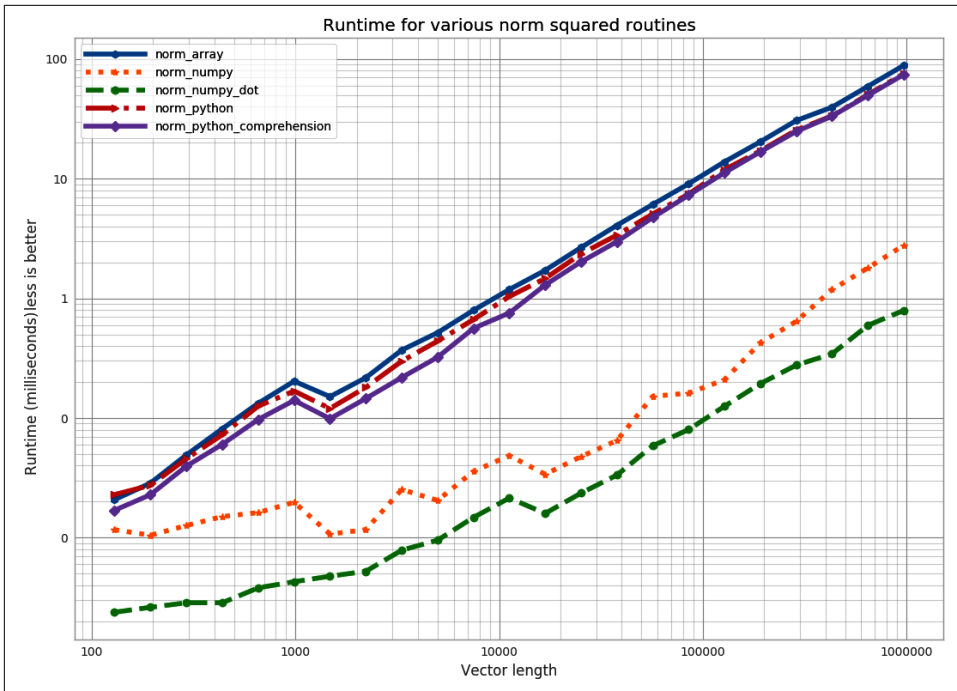


Figure 6-3. Runtimes for the various norm squared routines with vectors of different length

Applying numpy to the Diffusion Problem

Using what we've learned about numpy, we can easily adapt our pure Python code to be vectorized. The only new functionality we must introduce is numpy's roll function. This function does the same thing as our modulo-index trick, but it does so for an entire numpy array. In essence, it vectorizes this reindexing:

```
>>> import numpy as np
>>> np.roll([1,2,3,4], 1)
array([4, 1, 2, 3])

>>> np.roll([[1,2,3],[4,5,6]], 1, axis=1)
array([[3, 1, 2],
       [6, 4, 5]])
```

The roll function creates a new numpy array, which can be thought of as both good and bad. The downside is that we are taking time to allocate new space, which then needs to be filled with the appropriate data. On the other hand, once we have created this new rolled array, we will be able to vectorize operations on it quite quickly without suffering from cache misses from the CPU cache. This can substantially affect the speed of the actual calculation we must do on the grid. Later in this chapter

we will rewrite this so that we get the same benefit without having to constantly allocate more memory.

With this additional function we can rewrite the Python diffusion code from [Example 6-6](#) using simpler, and vectorized, numpy arrays. [Example 6-9](#) shows our initial numpy diffusion code.

Example 6-9. Initial numpy diffusion

```
from numpy import (zeros, roll)

grid_shape = (640, 640)

def laplacian(grid):
    return (
        roll(grid, +1, 0) +
        roll(grid, -1, 0) +
        roll(grid, +1, 1) +
        roll(grid, -1, 1) -
        4 * grid
    )

def evolve(grid, dt, D=1):
    return grid + dt * D * laplacian(grid)

def run_experiment(num_iterations):
    grid = zeros(grid_shape)

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    grid[block_low:block_high, block_low:block_high] = 0.005

    start = time.time()
    for i in range(num_iterations):
        grid = evolve(grid, 0.1)
    return time.time() - start
```

Immediately we see that this code is much shorter. This is sometimes a good indication of performance: we are doing a lot of the heavy lifting outside the Python interpreter, and hopefully inside a module specially built for performance and for solving a particular problem (however, this should always be tested!). One of the assumptions here is that numpy is using better memory management to give the CPU the data it needs more quickly. However, since whether this happens or not relies on the actual implementation of numpy, let's profile our code to see whether our hypothesis is correct. [Example 6-10](#) shows the results.

Example 6-10. Performance counters for numpy 2D diffusion (grid size: 640 × 640, 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy.py
```

Performance counter stats for 'python diffusion_numpy.py':

8,432,416,866	cycles	#	2.886 GHz
7,114,758,602	instructions	#	0.84 insn per cycle
1,040,831,469	cache-references	#	356.176 M/sec
216,490,683	cache-misses	#	20.800 % of all cache refs
1,252,928,847	branches	#	428.756 M/sec
8,174,531	branch-misses	#	0.65% of all branches
2922.239426	task-clock (msec)	#	1.285 CPUs utilized
403,282	faults	#	0.138 M/sec
403,282	minor-faults	#	0.138 M/sec
96	cs	#	0.033 K/sec
5	migrations	#	0.002 K/sec

2.274377105 seconds time elapsed

This shows that the simple change to numpy has given us a 63.3× speedup over the pure Python implementation with reduced memory allocations (Example 6-8). How was this achieved?

First of all, we can thank the vectorization that numpy gives. Although the numpy version seems to be running fewer instructions per cycle, each instruction does much more work. That is to say, one vectorized instruction can multiply four (or more) numbers in an array together instead of requiring four independent multiplication instructions. Overall, this results in fewer total instructions being necessary to solve the same problem.

Several other factors contribute to the numpy version requiring a lower absolute number of instructions to solve the diffusion problem. One of them has to do with the full Python API being available when running the pure Python version, but not necessarily for the numpy version—for example, the pure Python grids can be appended to in pure Python but not in numpy. Even though we aren’t explicitly using this (or other) functionality, there is overhead in providing a system where it *could* be available. Since numpy can make the assumption that the data being stored is always going to be numbers, everything regarding the arrays can be optimized for operations over numbers. We will continue on the track of removing necessary functionality in favor of performance when we talk about Cython (see “Cython” on page 167), where it is even possible to remove list bounds checking to speed up list lookups.

Normally, the number of instructions doesn't necessarily correlate with performance—the program with fewer instructions may not issue them efficiently, or they may be slow instructions. However, we see that in addition to reducing the number of instructions, the numpy version has also reduced a large inefficiency: cache misses (20.8% cache misses instead of 53.3%). As explained in “[Memory Fragmentation](#)” on [page 120](#), cache misses slow down computations because the CPU must wait for data to be retrieved from slower memory instead of having the data immediately available in its cache. In fact, memory fragmentation is such a dominant factor in performance that if we disable vectorization in numpy but keep everything else the same,⁸ we still see a sizable speed increase compared to the pure Python version ([Example 6-11](#)).

Example 6-11. Performance counters for numpy 2D diffusion without vectorization (grid size: 640 × 640, 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy.py
```

Performance counter stats for 'python diffusion_numpy.py':

50,086,999,350	cycles	#	2.888 GHz
53,611,608,977	instructions	#	1.07 insn per cycle
1,131,742,674	cache-references	#	65.266 M/sec
322,483,897	cache-misses	#	28.494 % of all cache refs
4,001,923,035	branches	#	230.785 M/sec
6,211,101	branch-misses	#	0.16% of all branches
17340.464580	task-clock (msec)	#	1.000 CPUs utilized
403,193	faults	#	0.023 M/sec
403,193	minor-faults	#	0.023 M/sec
74	cs	#	0.004 K/sec
6	migrations	#	0.000 K/sec

17.339656586 seconds time elapsed

This shows us that the dominant factor in our 63.3× speedup when introducing numpy is not the vectorized instruction set but rather the memory locality and reduced memory fragmentation. In fact, we can see from the preceding experiment that vectorization accounts for only about 13% of that 63.3× speedup.⁹

⁸ We do this by compiling numpy with the `-fno-tree-vectorize` flag. For this experiment, we built numpy 1.17.3 with the following command: `$ OPT='-fno-tree-vectorize' FOPT='-fno-tree-vectorize' BLAS=None LAPACK=None ATLAS=None python setup.py build`.

⁹ This is contingent on what CPU is being used.

This realization that memory issues are the dominant factor in slowing down our code doesn't come as too much of a shock. Computers are very well designed to do exactly the calculations we are requesting them to do with this problem—multiplying and adding numbers together. The bottleneck is in getting those numbers to the CPU fast enough to see it do the calculations as fast as it can.

Memory Allocations and In-Place Operations

To optimize the memory-dominated effects, let's try using the same method we used in [Example 6-6](#) to reduce the number of allocations we make in our `numpy` code. Allocations are quite a bit worse than the cache misses we discussed previously. Instead of simply having to find the right data in RAM when it is not found in the cache, an allocation also must make a request to the operating system for an available chunk of data and then reserve it. The request to the operating system generates quite a lot more overhead than simply filling a cache—while filling a cache miss is a hardware routine that is optimized on the motherboard, allocating memory requires talking to another process, the kernel, in order to complete.

To remove the allocations in [Example 6-9](#), we will preallocate some scratch space at the beginning of the code and then use only in-place operations. In-place operations (such as `+=`, `*=`, etc.) reuse one of the inputs as their output. This means that we don't need to allocate space to store the result of the calculation.

To show this explicitly, we will look at how the `id` of a `numpy` array changes as we perform operations on it ([Example 6-12](#)). The `id` is a good way of tracking this for `numpy` arrays, since the `id` indicates which section of memory is being referenced. If two `numpy` arrays have the same `id`, they are referencing the same section of memory.

10

Example 6-12. In-place operations reducing memory allocations

```
>>> import numpy as np
>>> array1 = np.random.random((10,10))
>>> array2 = np.random.random((10,10))
>>> id(array1)
140199765947424 ❶
>>> array1 += array2
>>> id(array1)
140199765947424 ❷
>>> array1 = array1 + array2
```

10 This is not strictly true, since two `numpy` arrays can reference the same section of memory but use different striding information to represent the same data in different ways. These two `numpy` arrays will have different `ids`. There are many subtleties to the `id` structure of `numpy` arrays that are outside the scope of this discussion.

```
>>> id(array1)
140199765969792 ❸
```

- ❶, ❷ These two `ids` are the same, since we are doing an in-place operation. This means that the memory address of `array1` does not change; we are simply changing the data contained within it.
- ❸ Here, the memory address has changed. When doing `array1 + array2`, a new memory address is allocated and filled with the result of the computation. This does have benefits, however, for when the original data needs to be preserved (i.e., `array3 = array1 + array2` allows you to keep using `array1` and `array2`, while in-place operations destroy some of the original data).

Furthermore, we can see an expected slowdown from the non-in-place operation. In [Example 6-13](#), we see that using in-place operations gives us a 27% speedup for an array of 100×100 elements. This margin will become larger as the arrays grow, since the memory allocations become more strenuous. However, it is important to note that this effect happens only when the array sizes are bigger than the CPU cache! When the arrays are smaller and the two inputs and the output can all fit into cache, the out-of-place operation is faster because it can benefit from vectorization.

Example 6-13. Runtime differences between in-place and out-of-place operations

```
>>> import numpy as np

>>> %%timeit array1, array2 = np.random.random((2, 100, 100)) ❶ ❸
... array1 = array1 + array2
6.45 µs ± 53.3 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

>>> %%timeit array1, array2 = np.random.random((2, 100, 100)) ❶
... array1 += array2
5.06 µs ± 78.7 ns per loop (mean ± std. dev. of 7 runs, 100000 loops each)

>>> %%timeit array1, array2 = np.random.random((2, 5, 5)) ❷
... array1 = array1 + array2
518 ns ± 4.88 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)

>>> %%timeit array1, array2 = np.random.random((2, 5, 5)) ❷
... array1 += array2
1.18 µs ± 6 ns per loop (mean ± std. dev. of 7 runs, 1000000 loops each)
```

- ❶ These arrays are too big to fit into the CPU cache, and the in-place operation is faster because of fewer allocations and cache misses.
- ❷ These arrays easily fit into cache, and we see the out-of-place operations as faster.

- ③ Note that we use `%timeit` instead of `%time`, which allows us to specify code to set up the experiment that doesn't get timed.

The downside is that while rewriting our code from [Example 6-9](#) to use in-place operations is not very complicated, it does make the resulting code a bit harder to read. In [Example 6-14](#), we can see the results of this refactoring. We instantiate `grid` and `next_grid` vectors, and we constantly swap them with each other. `grid` is the current information we know about the system, and after running `evolve`, `next_grid` contains the updated information.

Example 6-14. Making most numpy operations in-place

```
def laplacian(grid, out):
    np.copyto(out, grid)
    out *= -4
    out += np.roll(grid, +1, 0)
    out += np.roll(grid, -1, 0)
    out += np.roll(grid, +1, 1)
    out += np.roll(grid, -1, 1)

def evolve(grid, dt, out, D=1):
    laplacian(grid, out)
    out *= D * dt
    out += grid

def run_experiment(num_iterations):
    next_grid = np.zeros(grid_shape)
    grid = np.zeros(grid_shape)

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    grid[block_low:block_high, block_low:block_high] = 0.005

    start = time.time()
    for i in range(num_iterations):
        evolve(grid, 0.1, next_grid)
        grid, next_grid = next_grid, grid ①
    return time.time() - start
```

- ① Since the output of `evolve` gets stored in the output vector `next_grid`, we must swap these two variables so that, for the next iteration of the loop, `grid` has the most up-to-date information. This swap operation is quite cheap because only the references to the data are changed, not the data itself.



It is important to remember that since we want each operation to be in-place, whenever we do a vector operation, we must put it on its own line. This can make something as simple as $A = A * B + C$ become quite convoluted. Since Python has a heavy emphasis on readability, we should make sure that the changes we have made give us sufficient speedups to be justified.

Comparing the performance metrics from Examples 6-15 and 6-10, we see that removing the spurious allocations sped up our code by 30.9%. This speedup comes partly from a reduction in the number of cache misses but mostly from a reduction in minor faults. This comes from reducing the number of memory allocations necessary in the code by reusing already allocated space.

Minor faults are caused when a program accesses newly allocated space in memory. Since memory addresses are lazily allocated by the kernel, when you first access newly allocated data, the kernel pauses your execution while it makes sure that the required space exists and creates references to it for the program to use. This added machinery is quite expensive to run and can slow a program down substantially. On top of those additional operations that need to be run, we also lose any state that we had in cache and any possibility of doing instruction pipelining. In essence, we have to drop everything we're doing, including all associated optimizations, in order to go out and allocate some memory.

Example 6-15. Performance metrics for numpy with in-place memory operations (grid size: 640×640 , 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy_memory.py
```

Performance counter stats for 'python diffusion_numpy_memory.py':

6,880,906,446	cycles	#	2.886 GHz
5,848,134,537	instructions	#	0.85 insn per cycle
1,077,550,720	cache-references	#	452.000 M/sec
217,974,413	cache-misses	#	20.229 % of all cache refs
1,028,769,315	branches	#	431.538 M/sec
7,492,245	branch-misses	#	0.73% of all branches
2383.962679	task-clock (msec)	#	1.373 CPUs utilized
13,521	faults	#	0.006 M/sec
13,521	minor-faults	#	0.006 M/sec
100	cs	#	0.042 K/sec
8	migrations	#	0.003 K/sec

1.736322099 seconds time elapsed

Selective Optimizations: Finding What Needs to Be Fixed

Looking at the code from [Example 6-14](#), it seems like we have addressed most of the issues at hand: we have reduced the CPU burden by using numpy, and we have reduced the number of allocations necessary to solve the problem. However, there is always more investigation to be done. If we do a line profile on that code ([Example 6-16](#)), we see that the majority of our work is done within the `laplacian` function. In fact, 84% of the time that `evolve` takes to run is spent in `laplacian`.

Example 6-16. Line profiling shows that `laplacian` is taking too much time

```
Wrote profile results to diffusion_numpy_memory.py.lprof
Timer unit: 1e-06 s
```

```
Total time: 1.58502 s
File: diffusion_numpy_memory.py
Function: evolve at line 21
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
21					@profile
22					def evolve(grid, dt, out, D=1):
23	500	1327910.0	2655.8	83.8	laplacian(grid, out)
24	500	100733.0	201.5	6.4	out *= D * dt
25	500	156377.0	312.8	9.9	out += grid

There could be many reasons `laplacian` is so slow. However, there are two main high-level issues to consider. First, it looks like the calls to `np.roll` are allocating new vectors (we can verify this by looking at the documentation for the function). This means that even though we removed seven memory allocations in our previous refactoring, four outstanding allocations remain. Furthermore, `np.roll` is a very generalized function that has a lot of code to deal with special cases. Since we know exactly what we want to do (move just the first column of data to be the last in every dimension), we can rewrite this function to eliminate most of the spurious code. We could even merge the `np.roll` logic with the add operation that happens with the rolled data to make a very specialized `roll_add` function that does exactly what we want with the fewest number of allocations and the least extra logic.

[Example 6-17](#) shows what this refactoring would look like. All we need to do is create our new `roll_add` function and have `laplacian` use it. Since numpy supports fancy indexing, implementing such a function is just a matter of not jumbling up the indices. However, as stated earlier, while this code may be more performant, it is much less readable.



Notice the extra work that has gone into having an informative docstring for the function, in addition to full tests. When you are taking a route similar to this one, it is important to maintain the readability of the code, and these steps go a long way toward making sure that your code is always doing what it was intended to do and that future programmers can modify your code and know what things do and when things are not working.

Example 6-17. Creating our own roll function

```
import numpy as np
```

```
def roll_add(rollee, shift, axis, out):
```

```
    """
```

```
    Given a matrix, a rollee, and an output matrix, out, this function will
    perform the calculation:
```

```
        >>> out += np.roll(rollee, shift, axis=axis)
```

```
    This is done with the following assumptions:
```

```
        * rollee is 2D
```

```
        * shift will only ever be +1 or -1
```

```
        * axis will only ever be 0 or 1 (also implied by the first assumption)
```

```
    Using these assumptions, we are able to speed up this function by avoiding
    extra machinery that numpy uses to generalize the roll function and also
    by making this operation intrinsically in-place.
```

```
    """
```

```
    if shift == 1 and axis == 0:
```

```
        out[1:, :] += rollee[:-1, :]
```

```
        out[0, :] += rollee[-1, :]
```

```
    elif shift == -1 and axis == 0:
```

```
        out[:-1, :] += rollee[1:, :]
```

```
        out[-1, :] += rollee[0, :]
```

```
    elif shift == 1 and axis == 1:
```

```
        out[:, 1:] += rollee[:, :-1]
```

```
        out[:, 0] += rollee[:, -1]
```

```
    elif shift == -1 and axis == 1:
```

```
        out[:, :-1] += rollee[:, 1:]
```

```
        out[:, -1] += rollee[:, 0]
```

```
def test_roll_add():
```

```
    rollee = np.asarray([[1, 2], [3, 4]])
```

```
    for shift in (-1, +1):
```

```
        for axis in (0, 1):
```

```
            out = np.asarray([[6, 3], [9, 2]])
```

```
            expected_result = np.roll(rollee, shift, axis=axis) + out
```

```
            roll_add(rollee, shift, axis, out)
```

```
            assert np.all(expected_result == out)
```

```
def laplacian(grid, out):
    np.copyto(out, grid)
    out *= -4
    roll_add(grid, +1, 0, out)
    roll_add(grid, -1, 0, out)
    roll_add(grid, +1, 1, out)
    roll_add(grid, -1, 1, out)
```

If we look at the performance counters in [Example 6-18](#) for this rewrite, we see that while it is 22% faster than [Example 6-14](#), most of the counters are about the same. The major difference again is cache-misses, which is down 7×. This change also seems to have affected the throughput of instructions to the CPU, increasing the instructions per cycle from 0.85 to 0.99 (a 14% gain). Similarly, the faults went down 12.85%. This seems to be a result of first doing the rolls in place as well as reducing the amount of numpy machinery that needs to be in-place to do all of our desired computation. Instead of first rolling our array, doing other computation required by numpy in order to do bounds checking and error handling, and then adding the result, we are doing it all in one shot and not requiring the computer to refill the cache every time. This theme of trimming out unnecessary machinery in both numpy and Python in general will continue in “[Cython](#)” on page 167.

Example 6-18. Performance metrics for numpy with in-place memory operations and custom laplacian function (grid size: 640 × 640, 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy_memory2.py
```

Performance counter stats for 'python diffusion_numpy_memory2.py':

5,971,464,515	cycles	#	2.888 GHz
5,893,131,049	instructions	#	0.99 insn per cycle
1,001,582,133	cache-references	#	484.398 M/sec
30,840,612	cache-misses	#	3.079 % of all cache refs
1,038,649,694	branches	#	502.325 M/sec
7,562,009	branch-misses	#	0.73% of all branches
2067.685884	task-clock (msec)	#	1.456 CPUs utilized
11,981	faults	#	0.006 M/sec
11,981	minor-faults	#	0.006 M/sec
95	cs	#	0.046 K/sec
3	migrations	#	0.001 K/sec

1.419869071 seconds time elapsed

numexpr: Making In-Place Operations Faster and Easier

One downfall of numpy's optimization of vector operations is that it occurs on only one operation at a time. That is to say, when we are doing the operation $A * B + C$ with numpy vectors, first the entire $A * B$ operation completes, and the data is stored in a temporary vector; then this new vector is added with C . The in-place version of the diffusion code in [Example 6-14](#) shows this quite explicitly.

However, many modules can help with this. numexpr is a module that can take an entire vector expression and compile it into very efficient code that is optimized to minimize cache misses and temporary space used. In addition, the expressions can utilize multiple CPU cores (see [Chapter 9](#) for more information) and take advantage of the specialized instructions your CPU may support in order to get even greater speedups. It even supports OpenMP, which parallels out operations across multiple cores on your machine.

It is very easy to change code to use numexpr: all that's required is to rewrite the expressions as strings with references to local variables. The expressions are compiled behind the scenes (and cached so that calls to the same expression don't incur the same cost of compilation) and run using optimized code. [Example 6-19](#) shows the simplicity of changing the evolve function to use numexpr. In this case, we chose to use the out parameter of the evaluate function so that numexpr doesn't allocate a new vector to which to return the result of the calculation.

Example 6-19. Using numexpr to further optimize large matrix operations

```
from numexpr import evaluate

def evolve(grid, dt, next_grid, D=1):
    laplacian(grid, next_grid)
    evaluate("next_grid * D * dt + grid", out=next_grid)
```

An important feature of numexpr is its consideration of CPU caches. It specifically moves data around so that the various CPU caches have the correct data in order to minimize cache misses. When we run perf on the updated code ([Example 6-20](#)), we see a speedup. However, if we look at the performance on a smaller, 256×256 grid, we see a decrease in speed (see [Table 6-2](#)). Why is this?

Example 6-20. Performance metrics for numpy with in-place memory operations, custom laplacian function, and numexpr (grid size: 640×640 , 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_numpy_memory2_numexpr.py
```

Performance counter stats for 'python diffusion_numpy_memory2_numexpr.py':

8,856,947,179	cycles	#	2.872 GHz
9,354,357,453	instructions	#	1.06 insn per cycle
1,077,518,384	cache-references	#	349.423 M/sec
59,407,830	cache-misses	#	5.513 % of all cache refs
1,018,525,317	branches	#	330.292 M/sec
11,941,430	branch-misses	#	1.17% of all branches
3083.709890	task-clock (msec)	#	1.991 CPUs utilized
15,820	faults	#	0.005 M/sec
15,820	minor-faults	#	0.005 M/sec
8,671	cs	#	0.003 M/sec
2,096	migrations	#	0.680 K/sec

1.548924090 seconds time elapsed

Much of the extra machinery we are bringing into our program with numexpr deals with cache considerations. When our grid size is small and all the data we need for our calculations fits in the cache, this extra machinery simply adds more instructions that don't help performance. In addition, compiling the vector operation that we encoded as a string adds a large overhead. When the total runtime of the program is small, this overhead can be quite noticeable. However, as we increase the grid size, we should expect to see numexpr utilize our cache better than native numpy does. In addition, numexpr utilizes multiple cores to do its calculation and tries to saturate each of the cores' caches. When the size of the grid is small, the extra overhead of managing the multiple cores overwhelms any possible increase in speed.

The particular computer we are running the code on has a 8,192 KB cache (Intel Core i7-7820HQ). Since we are operating on two arrays, one for input and one for output, we can easily do the calculation for the size of the grid that will fill up our cache. The number of grid elements we can store in total is $8,192 \text{ KB} / 64 \text{ bit} = 1,024,000$. Since we have two grids, this number is split between two objects (so each one can be at most $1,024,000 / 2 = 512,000$ elements). Finally, taking the square root of this number gives us the size of the grid that uses that many grid elements. All in all, this means that approximately two 2D arrays of size 715×715 would fill up the cache ($\sqrt{8192 \text{ KB} / 64 \text{ bit} / 2} = 715.5$). In practice, however, we do not get to fill up the cache ourselves (other programs will fill up parts of the cache), so realistically we can probably fit two 640×640 arrays. Looking at Tables 6-1 and 6-2, we see that when the grid size jumps from 512×512 to $1,024 \times 1,024$, the numexpr code starts to outperform pure numpy.

A Cautionary Tale: Verify “Optimizations” (scipy)

An important thing to take away from this chapter is the approach we took to every optimization: profile the code to get a sense of what is going on, come up with a possible solution to fix slow parts, and then profile to make sure the fix actually worked. Although this sounds straightforward, things can get complicated quickly, as we saw in how the performance of `numexpr` depended greatly on the size of the grid we were considering.

Of course, our proposed solutions don’t always work as expected. While writing the code for this chapter, we saw that the `laplacian` function was the slowest routine and hypothesized that the `scipy` routine would be considerably faster. This thinking came from the fact that Laplacians are a common operation in image analysis and probably have a very optimized library to speed up the calls. `scipy` has an image submodule, so we must be in luck!

The implementation was quite simple (Example 6-21) and required little thought about the intricacies of implementing the periodic boundary conditions (or “wrap” condition, as `scipy` calls it).

Example 6-21. Using `scipy`’s `laplace` filter

```
from scipy.ndimage.filters import laplace

def laplacian(grid, out):
    laplace(grid, out, mode="wrap")
```

Ease of implementation is quite important and definitely won this method some points before we considered performance. However, once we benchmarked the `scipy` code (Example 6-22), we had a revelation: this method offers no substantial speedup compared to the code it is based on (Example 6-14). In fact, as we increase the grid size, this method starts performing worse (see Figure 6-4 at the end of the chapter).

Example 6-22. Performance metrics for diffusion with `scipy`’s `laplace` function (grid size: 640×640 , 500 iterations)

```
$ perf stat -e cycles,instructions,\
    cache-references,cache-misses,branches,branch-misses,task-clock,faults,\
    minor-faults,cs,migrations python diffusion_scipy.py
```

Performance counter stats for 'python diffusion_scipy.py':

10,051,801,725	cycles	#	2.886 GHz
16,536,981,020	instructions	#	1.65 insn per cycle
1,554,557,564	cache-references	#	446.405 M/sec

126,627,735	cache-misses	#	8.146 % of all cache refs
2,673,416,633	branches	#	767.696 M/sec
9,626,762	branch-misses	#	0.36% of all branches
3482.391211	task-clock (msec)	#	1.228 CPUs utilized
14,013	faults	#	0.004 M/sec
14,013	minor-faults	#	0.004 M/sec
95	cs	#	0.027 K/sec
5	migrations	#	0.001 K/sec

2.835263796 seconds time elapsed

Comparing the performance metrics of the `scipy` version of the code with those of our custom `laplacian` function ([Example 6-18](#)), we can start to get some indication as to why we aren't getting the speedup we were expecting from this rewrite.

The metric that stands out the most is `instructions`. This shows us that the `scipy` code is requesting that the CPU do more than double the amount of work as our custom `laplacian` code. Even though these instructions are more numerically optimized (as we can see with the higher `insn_per_cycle` count, which says how many instructions the CPU can do in one clock cycle), the extra optimization doesn't win out over the sheer number of added instructions.

This could be in part because the `scipy` code is written very generally so that it can process all sorts of inputs with different boundary conditions (which requires extra code and thus more instructions). We can see this, in fact, by the high number of branches that the `scipy` code requires. When code has many branches, it means that we run commands based on a condition (like having code inside an `if` statement). The problem is that we don't know whether we can evaluate an expression until we check the conditional, so vectorizing or pipelining isn't possible. The machinery of branch prediction helps with this, but it isn't perfect. This speaks more to the point of the speed of specialized code: if you don't need to constantly check what you need to do and instead know the specific problem at hand, you can solve it much more efficiently.

Lessons from Matrix Optimizations

Looking back on our optimizations, we seem to have taken two main routes: reducing the time taken to get data to the CPU and reducing the amount of work that the CPU had to do. Tables [6-1](#) and [6-2](#) compare of the results achieved by our various optimization efforts, for various dataset sizes, in relation to the original pure Python implementation.

[Figure 6-4](#) shows graphically how all these methods compared to one another. We can see three bands of performance that correspond to these two methods: the band along the bottom shows the small improvement made in relation to our pure Python

implementation by our first effort at reducing memory allocations; the middle band shows what happened when we used `numpy` and further reduced allocations; and the upper band illustrates the results achieved by reducing the work done by our process.

Table 6-1. Total runtime of all schemes for various grid sizes and 500 iterations of the `evolve` function

Method	256 x 256	512 x 512	1024 x 1024	2048 x 2048	4096 x 4096
Python	2.40s	10.43s	41.75s	168.82s	679.16s
Python + memory	2.26s	9.76s	38.85s	157.25s	632.76s
numpy	0.01s	0.09s	0.69s	3.77s	14.83s
numpy + memory	0.01s	0.07s	0.60s	3.80s	14.97s
numpy + memory + laplacian	0.01s	0.05s	0.48s	1.86s	7.50s
numpy + memory + laplacian + numexpr	0.02s	0.06s	0.41s	1.60s	6.45s
numpy + memory + scipy	0.05s	0.25s	1.15s	6.83s	91.43s

Table 6-2. Speedup compared to naive Python (Example 6-3) for all schemes and various grid sizes over 500 iterations of the `evolve` function

Method	256 x 256	512 x 512	1024 x 1024	2048 x 2048	4096 x 4096
Python	1.00x	1.00x	1.00x	1.00x	1.00x
Python + memory	1.06x	1.07x	1.07x	1.07x	1.07x
numpy	170.59x	116.16x	60.49x	44.80x	45.80x
numpy + memory	185.97x	140.10x	69.67x	44.43x	45.36x
numpy + memory + laplacian	203.66x	208.15x	86.41x	90.91x	90.53x
numpy + memory + laplacian + numexpr	97.41x	167.49x	102.38x	105.69x	105.25x
numpy + memory + scipy	52.27x	42.00x	36.44x	24.70x	7.43x

One important lesson to take away from this is that you should always take care of any administrative things the code must do during initialization. This may include allocating memory, or reading configuration from a file, or even precomputing values that will be needed throughout the lifetime of the program. This is important for two reasons. First, you are reducing the total number of times these tasks must be done by doing them once up front, and you know that you will be able to use those resources without too much penalty in the future. Second, you are not disrupting the flow of the program; this allows it to pipeline more efficiently and keep the caches filled with more pertinent data.

You also learned more about the importance of data locality and how important simply getting data to the CPU is. CPU caches can be quite complicated, and often it is best to allow the various mechanisms designed to optimize them take care of the issue. However, understanding what is happening and doing all that is possible to optimize the way memory is handled can make all the difference. For example, by

understanding how caches work, we are able to understand that the decrease in performance that leads to a saturated speedup no matter the grid size in Figure 6-4 can probably be attributed to the L3 cache being filled up by our grid. When this happens, we stop benefiting from the tiered memory approach to solving the von Neumann bottleneck.

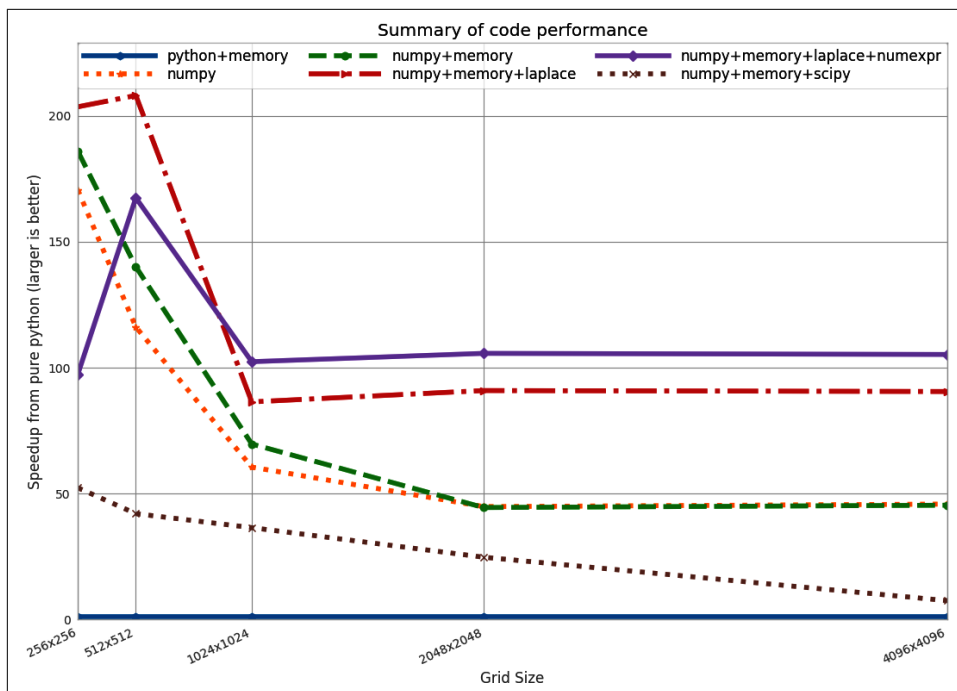


Figure 6-4. Summary of speedups from the methods attempted in this chapter

Another important lesson concerns the use of external libraries. Python is fantastic for its ease of use and readability, which allow you to write and debug code fast. However, tuning performance down to the external libraries is essential. These external libraries can be extremely fast, because they can be written in lower-level languages—but since they interface with Python, you can also still write code that uses them quickly.

Finally, we learned the importance of benchmarking everything and forming hypotheses about performance before running the experiment. By forming a hypothesis before running the benchmark, we are able to form conditions to tell us whether our optimization actually worked. Was this change able to speed up runtime? Did it reduce the number of allocations? Is the number of cache misses lower? Optimization can be an art at times because of the vast complexity of computer systems, and having a quantitative probe into what is actually happening can help enormously.

One last point about optimization is that a lot of care must be taken to make sure that the optimizations you make generalize to different computers (the assumptions you make and the results of benchmarks you do may be dependent on the architecture of the computer you are running on, how the modules you are using were compiled, etc.). In addition, when making these optimizations, it is incredibly important to consider other developers and how the changes will affect the readability of your code. For example, we realized that the solution we implemented in [Example 6-17](#) was potentially vague, so care was taken to make sure that the code was fully documented and tested to help not only us but also other people on the team.

Sometimes, however, your numerical algorithms also require quite a lot of data wrangling and manipulation that aren't just clear-cut mathematical operations. In these cases, Pandas is a very popular solution, and it has its own performance characteristics. We'll now do a deep dive into Pandas and understand how to better use it to write performant numerical code.

Pandas

Pandas is the de facto data manipulation tool in the scientific Python ecosystem for tabular data. It enables easy manipulation with Excel-like tables of heterogeneous datatypes, known as DataFrames, and has strong support for time-series operations. Both the public interface and the internal machinery have evolved a lot since 2008, and there's a lot of conflicting information in public forums on “fast ways to solve problems.” In this section, we'll fix some misconceptions about common use cases of Pandas.

We'll review the internal model for Pandas, find out how to apply a function efficiently across a DataFrame, see why concatenating to a DataFrame repeatedly is a poor way to build up a result, and look at faster ways of handling strings.

Pandas's Internal Model

Pandas uses an in-memory, 2D, table-like data structure—if you have in mind an Excel sheet, you have a good initial mental model. Originally, Pandas focused on NumPy's dtype objects such as signed and unsigned numbers for each column. As the library evolved, it expanded beyond NumPy types and can now handle both Python strings and extension types (including nullable Int64 objects—note the capital “I”—and IP addresses).

Operations on a DataFrame apply to all cells in a column (or all cells in a row if the `axis=1` parameter is used), all operations are executed eagerly, and there's no support for query planning. Operations on columns often generate temporary intermediate arrays, which consume RAM. The general advice is to expect a temporary memory usage envelope of up to three to five times your current usage when you're

manipulating your DataFrames. Typically, Pandas works well for datasets under 10 GB in size, assuming you have sufficient RAM for temporary results.

Operations can be single-threaded and may be limited by Python’s global interpreter lock (GIL). Increasingly, improved internal implementations are allowing the GIL to be disabled automatically, enabling parallelized operations. We’ll explore an approach to parallelization with Dask in [“Parallel Pandas with Dask” on page 322](#).

Behind the scenes, columns of the same dtype are grouped together by a `BlockManager`. This piece of hidden machinery works to make row-wise operations on columns of the same datatype faster. It is one of the many hidden technical details that make the Pandas code base complex but make the high-level user-facing operations faster.¹¹

Performing operations on a subset of data from a single common block typically generates a view, while taking a slice of rows that cross blocks of different dtypes can cause a copy, which may be slower. One consequence is that while numeric columns directly reference their NumPy data, string columns reference a list of Python strings, and these individual strings are scattered in memory—this can lead to unexpected speed differences for numeric and string operations.

Behind the scenes, Pandas uses a mix of NumPy datatypes and its own extension datatypes. Examples from NumPy include `int8` (1 byte), `int64` (8 bytes—and note the lowercase “i”), `float16` (2 bytes), `float64` (8 bytes), and `bool` (1 byte). Additional types provided by Pandas include `categorical` and `datetime64`. Externally, they appear to work similarly, but behind the scenes in the Pandas code base they cause a lot of type-specific Pandas code and duplication.



Whilst Pandas originally used only numpy datatypes, it has evolved its own set of additional Pandas datatypes that understand missing data (NaN) behavior with three-valued logic. You must distinguish the numpy `int64`, which is not NaN-aware, from the Pandas `Int64`, which uses two columns of data behind the scenes for the integers and for the NaN bit mask. Note that the numpy `float64` is naturally NaN-aware.

¹¹ See the DataQuest blog post [“Tutorial: Using Pandas with Large Data Sets in Python”](#) for more details.

One side effect of using NumPy's datatypes has been that, while a `float` has a NaN (missing value) state, the same is not true for `int` and `bool` objects. If you introduce a NaN value into an `int` or `bool` Series in Pandas, your Series will be promoted to a `float`. Promoting `int` types to a `float` may reduce the numeric accuracy that can be represented in the same bits, and the smallest `float` is `float16`, which takes twice as many bytes as a `bool`.

The nullable `Int64` (note the capitalized "I") was introduced in version 0.24 as an extension type in Pandas. Internally, it uses a NumPy `int64` and a second Boolean array as a NaN-mask. Equivalents exist for `Int32` and `Int8`. As of Pandas version 1.0, there is also an equivalent nullable Boolean (with dtype `boolean` as opposed to the numpy `bool`, which isn't NaN-aware). A `StringDType` has been introduced that may in the future offer higher performance and less memory usage than the standard Python `str`, which is stored in a column of object dtype.

Applying a Function to Many Rows of Data

It is very common to apply functions to rows of data in Pandas. There's a selection of approaches, and the idiomatic Python approaches using loops are generally the slowest. We'll work through an example based on a real-world challenge, showing different ways of solving this problem and ending with a reflection on the trade-off between speed and maintainability.

Ordinary Least Squares (OLS) is a bread-and-butter method in data science for fitting a line to data. It solves for the slope and intercept in the $m * x + c$ equation, given some data. This can be incredibly useful when trying to understand the trend of the data: is it generally increasing, or is it decreasing?

An example of its use from our work is a research project for a telecommunications company where we want to analyze a set of potential user-behavior signals (e.g., marketing campaigns, demographics, and geographic behavior). The company has the number of hours a person spends on their cell phone every day, and its question is: is this person increasing or decreasing their usage, and how does this change over time?

One way to approach this problem is to take the company's large dataset of millions of users over years of data and break it into smaller windows of data (each window, for example, representing 14 days out of the years of data). For each window, we model the users' use through OLS and record whether they are increasing or decreasing their usage.

In the end, we have a sequence for each user showing whether, for a given 14-day period, their use was generally increasing or decreasing. However, to get there, we have to run OLS a massive number of times!

For one million users and two years of data, we might have 730 windows,¹² and thus 730,000,000 calls to OLS! To solve this problem practically, our OLS implementation should be fairly well tuned.

In order to understand the performance of various OLS implementations, we will generate some smaller but representative synthetic data to give us good indications of what to expect on the larger dataset. We'll generate data for 100,000 rows, each representing a synthetic user, and each containing 14 columns representing “hours used per day” for 14 days, as a continuous variable.

We'll draw from a Poisson distribution (with `lambda==60` as minutes) and divide by 60 to give us simulated hours of usage as continuous values. The true nature of the random data doesn't matter for this experiment; it is convenient to use a distribution that has a minimum value of 0 as this represents the real-world minimum. You can see a sample in [Example 6-23](#).

Example 6-23. A snippet of our data

	0	1	2	...	12	13
0	1.016667	0.883333	1.033333	...	1.016667	0.833333
1	1.033333	1.016667	0.833333	...	1.133333	0.883333
2	0.966667	1.083333	1.183333	...	1.000000	0.950000

¹² If we're going to use a sliding window, it might be possible to apply rolling window optimized functions such as `RollingOLS` from `statsmodels`.

In [Figure 6-5](#), we see three rows of 14 days of synthetic data.

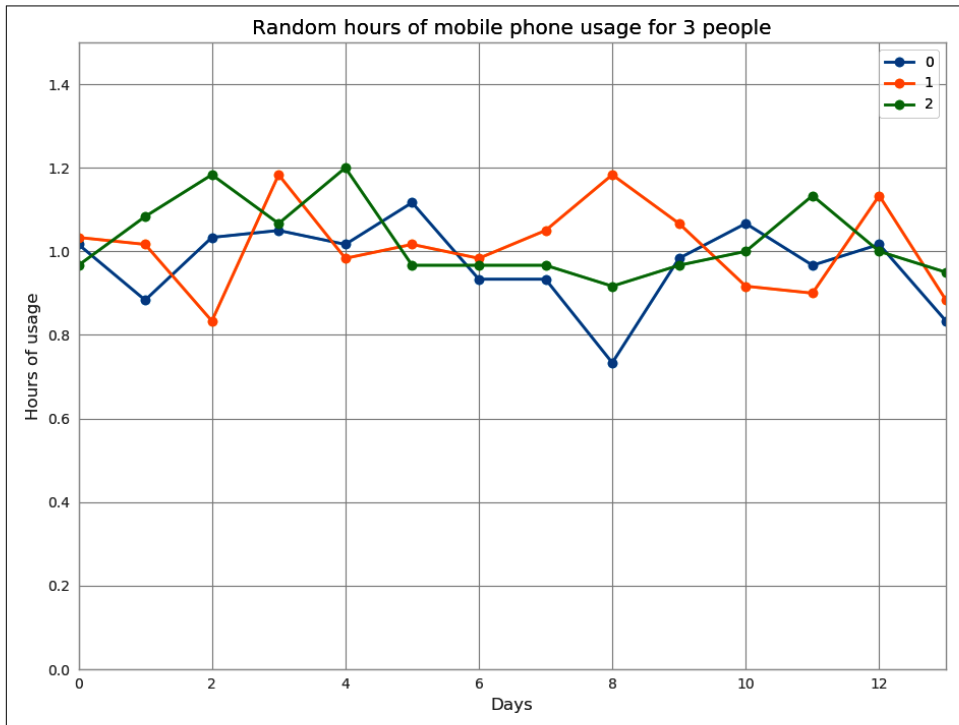


Figure 6-5. Synthetic data for the first three simulated users showing 14 days of cell phone usage

A bonus of generating 100,000 rows of data is that some rows will, by random variation alone, exhibit “increasing counts,” and some will exhibit “decreasing counts.” Note that there is no signal behind this in our synthetic data since the points are drawn independently; simply because we generate many rows of data, we’re going to see a variance in the ultimate slopes of the lines we calculate.

This is convenient, as we can identify the “most growing” and “most declining” lines and draw them as a validation that we’re identifying the sort of signal we hope to export on the real-world problem. [Figure 6-6](#) shows two of our random traces with maximal and minimal slopes (m).

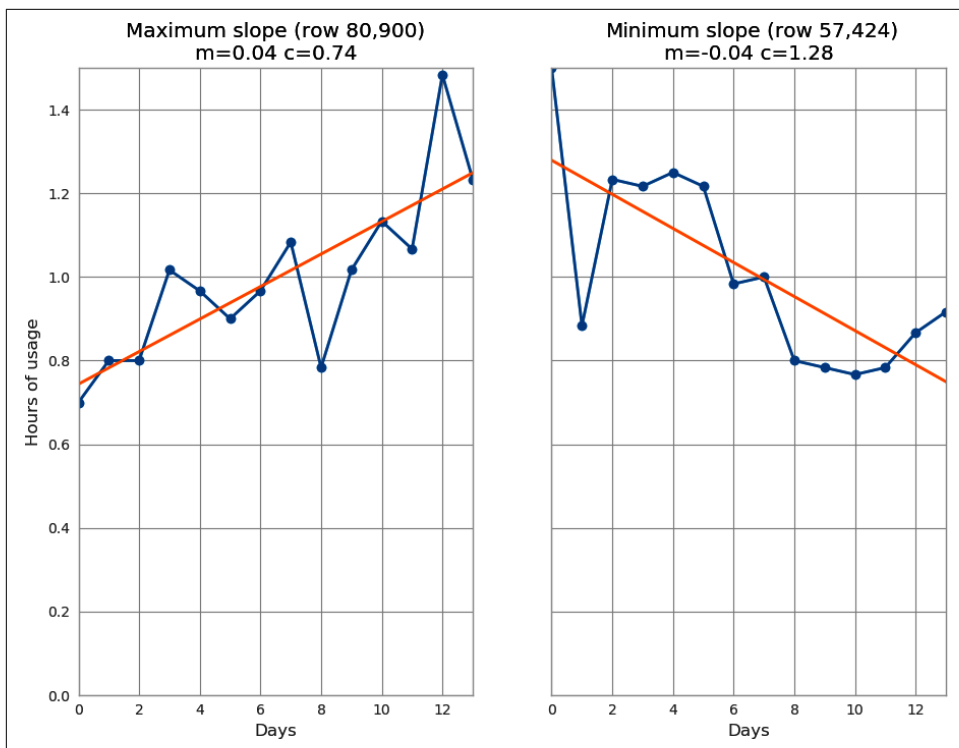


Figure 6-6. The “most increasing” and “most decreasing” usage in our randomly generated dataset

We’ll start with scikit-learn’s `LinearRegression` estimator to calculate each m . While this method is correct, we’ll see in the following section that it incurs a surprising overhead against another approach.

Which OLS implementation should we use?

Example 6-24 shows three implementations that we’d like to try. We’ll evaluate the scikit-learn implementation against the direct linear algebra implementation using NumPy. Both methods ultimately perform the same job and calculate the slopes (m) and intercept (c) of the target data from each Pandas row given an increasing x range (with values $[0, 1, \dots, 13]$).

scikit-learn will be a default choice for many machine learning practitioners, while a linear algebra solution may be preferred by those coming from other disciplines.

Example 6-24. Solving Ordinary Least Squares with NumPy and scikit-learn

```
def ols_sklearn(row):
    """Solve OLS using scikit-learn's LinearRegression"""
    est = LinearRegression()
    X = np.arange(row.shape[0]).reshape(-1, 1) # shape (14, 1)
    # note that the intercept is built inside LinearRegression
    est.fit(X, row.values)
    m = est.coef_[0] # note c is in est.intercept_
    return m

def ols_lstsq(row):
    """Solve OLS using numpy.linalg.lstsq"""
    # build X values for [0, 13]
    X = np.arange(row.shape[0]) # shape (14,)
    ones = np.ones(row.shape[0]) # constant used to build intercept
    A = np.vstack((X, ones)).T # shape(14, 2)
    # lstsq returns the coefficient and intercept as the first result
    # followed by the residuals and other items
    m, c = np.linalg.lstsq(A, row.values, rcond=-1)[0]
    return m

def ols_lstsq_raw(row):
    """Variant of `ols_lstsq` where row is a numpy array (not a Series)"""
    X = np.arange(row.shape[0])
    ones = np.ones(row.shape[0])
    A = np.vstack((X, ones)).T
    m, c = np.linalg.lstsq(A, row, rcond=-1)[0]
    return m
```

Surprisingly, if we call `ols_sklearn` 10,000 times with the `timeit` module, we find that it takes at least 0.483 microseconds to execute, while `ols_lstsq` on the same data takes 0.182 microseconds. The popular scikit-learn solution takes more than twice as long as the terse NumPy variant!

Building on the profiling from “Using `line_profiler` for Line-by-Line Measurements” on page 40, we can use the object interface (rather than the command line or Jupyter magic interfaces) to learn *why* the scikit-learn implementation is slower. In [Example 6-25](#), we tell `LineProfiler` to profile `est.fit` (that’s the scikit-learn fit method on our `LinearRegression` estimator) and then call `run` with arguments based on the `DataFrame` we used before.

We see a couple of surprises. The very last line of `fit` calls the same `linalg.lstsq` that we’ve called in `ols_lstsq`—so what else is going on to cause our slowdown? `LineProfiler` reveals that scikit-learn is calling two other expensive methods, namely `check_X_y` and `_preprocess_data`.

Both of these are designed to help us avoid making mistakes—indeed, your author Ian has been saved repeatedly from passing in inappropriate data such as a wrongly

shaped array or one containing NaNs to scikit-learn estimators. A consequence of this checking is that it takes more time—more safety makes things run slower! We’re trading developer time (and sanity) against execution time.

Example 6-25. Digging into scikit-learn’s `LinearRegression.fit` call

```
...
lp = LineProfiler(est.fit)
print("Run on a single row")
lp.run("est.fit(X, row.values)")
lp.print_stats()
```

Line #	% Time	Line Contents
=====		
438		def fit(self, X, y, sample_weight=None):
...		
462	0.3	X, y = check_X_y(X, y,
		accept_sparse=['csr', 'csc', 'coo'],
463	35.0	y_numeric=True, multi_output=True)
...		
468	0.3	X, y, X_offset, y_offset, X_scale = \
		self._preprocess_data(
469	0.3	X, y,
		fit_intercept=self.fit_intercept,
		normalize=self.normalize,
470	0.2	copy=self.copy_X,
		sample_weight=sample_weight,
471	29.3	return_mean=True)
...		
502		self.coef_, self._residues,
		self.rank_, self.singular_ = \
503	29.2	linalg.lstsq(X, y)

Behind the scenes, these two methods are performing various checks, including these:

- Checking for appropriate sparse NumPy arrays (even though we’re using dense arrays in this example)
- Offsetting the input array to a mean of 0 to improve numerical stability on wider data ranges than we’re using
- Checking that we’re providing a 2D X array
- Checking that we’re not providing NaN or Inf values
- Checking that we’re providing non-empty arrays of data

Generally, we prefer to have all of these checks enabled—they’re here to help us avoid painful debugging sessions, which kill developer productivity. If we know that our data is of the correct form for our chosen algorithm, these checks will add a penalty.

It is up to you to decide when the safety of these methods is hurting your overall productivity.

As a general rule—stay with the safer implementations (scikit-learn, in this case) *unless* you're confident that your data is in the right form and you're optimizing for performance. We're after increased performance, so we'll continue with the `ols_lstsq` approach.

Applying `lstsq` to our rows of data

We'll start with an approach that many Python developers who come from other programming languages may try. This is *not* idiomatic Python, nor is it common or even efficient for Pandas. It does have the advantage of being very easy to understand. In [Example 6-26](#), we'll iterate over the index of the DataFrame from row 0 to row 99,999; on each iteration we'll use `iloc` to retrieve a row, and then we'll calculate OLS on that row.

The calculation is common to each of the following methods—what's different is how we iterate over the rows. This method takes 18.6 seconds; it is by far the slowest approach (by a factor of 3) among the options we're evaluating.

Behind the scenes, each dereference is expensive—`iloc` does a lot of work to get to the row using a fresh `row_idx`, which is then converted into a new Series object, which is returned and assigned to `row`.

Example 6-26. Our worst implementation—counting and fetching rows one at a time with `iloc`

```
ms = []
for row_idx in range(df.shape[0]):
    row = df.iloc[row_idx]
    m = ols_lstsq(row)
    ms.append(m)
results = pd.Series(ms)
```

Next, we'll take a more idiomatic Python approach: in [Example 6-27](#), we iterate over the rows using `iterrows`, which looks similar to how we might iterate over a Python iterable (e.g., a list or set) with a for loop. This method looks sensible and is a little faster—it takes 12.4 seconds.

This is more efficient, as we don't have to do so many lookups—`iterrows` can walk along the rows without doing lots of sequential lookups. `row` is still created as a fresh Series on each iteration of the loop.

Example 6-27. iterrows for more efficient and “Python-like” row operations

```
ms = []
for row_idx, row in df.iterrows():
    m = ols_lstsq(row)
    ms.append(m)
results = pd.Series(ms)
```

Example 6-28 skips a lot of the Pandas machinery, so a lot of overhead is avoided. `apply` passes the function `ols_lstsq` a new row of data directly (again, a fresh Series is constructed behind the scenes for each row) without creating Python intermediate references. This costs 6.8 seconds—this is a significant improvement, and the code is more compact and readable!

Example 6-28. apply for idiomatic Pandas function application

```
ms = df.apply(ols_lstsq, axis=1)
results = pd.Series(ms)
```

Our final variant in **Example 6-29** uses the same `apply` call with an additional `raw=True` argument. Using `raw=True` stops the creation of an intermediate Series object. As we don’t have a Series object, we have to use our third OLS function, `ols_lstsq_raw`; this variant has direct access to the underlying NumPy array.

By avoiding the creation and dereferencing of an intermediate Series object, we shave our execution time a little more, down to 5.3 seconds.

Example 6-29. Avoiding intermediate Series creation using raw=True

```
ms = df.apply(ols_lstsq_raw, axis=1, raw=True)
results = pd.Series(ms)
```

The use of `raw=True` gives us the option to compile with Numba (“**Numba to Compile NumPy for Pandas**” on page 182) or with Cython as it removes the complication of compiling Pandas layers that currently aren’t supported.

We’ll summarize the execution times in **Table 6-3** for 100,000 rows of data on a single window of 14 columns of simulated data. New Pandas users often use `iloc` and `iterrows` (or the similar `itertuples`) when `apply` would be preferred.

By performing our analysis and considering our potential need to perform OLS on 1,000,000 rows by up to 730 windows of data, we can see that a first naive approach combining `iloc` with `ols_sklearn` might cost $10 \text{ (our larger dataset factor)} * 730 * 18 \text{ seconds} * 2 \text{ (our slowdown factor against } \text{ols_lstsq})} = 73 \text{ hours}$.

If we used `ols_lstsq_raw` and our fastest approach, the same calculations might take $10 * 730 * 5.3$ seconds == 10 hours. This is a significant saving for a task that represents what might be a suite of similar operations. We'll see even faster solutions if we compile and run on multiple cores.

Table 6-3. Cost for using `lstsq` with various Pandas row-wise approaches

Method	Time in seconds
<code>iloc</code>	18.6
<code>iterrows</code>	12.4
<code>apply</code>	6.8
<code>apply raw=True</code>	5.3

Earlier we discovered that the scikit-learn approach adds significant overhead to our execution time by covering our data with a safety net of checks. We can remove this safety net but with a potential cost on developer debugging time. Your authors strongly urge you to consider adding unit-tests to your code that would verify that a well-known and well-debugged method is used to test any optimized method you settle on. If you added a unit test to compare the scikit-learn `LinearRegression` approach against `ols_lstsq`, you'd be giving yourself and other colleagues a future hint about why you developed a less obvious solution to what appeared to be a standard problem.

Having experimented, you may also conclude that the heavily tested scikit-learn approach is more than fast enough for your application and that you're more comfortable using a library that is well known by other developers. This could be a very sane conclusion.

Later in “[Parallel Pandas with Dask](#)” on page 322, we'll look at running Pandas operations across multiple cores by dividing data into groups of rows using Dask and Swifter. In “[Numba to Compile NumPy for Pandas](#)” on page 182, we look at compiling the `raw=True` variant of `apply` to achieve an order of magnitude speedup. Compilation and parallelization can be combined for a really significant final speedup, dropping our expected runtime from around 10 hours to just 30 minutes.

Building DataFrames and Series from Partial Results Rather than Concatenating

You may have wondered in [Example 6-26](#) why we built up a list of partial results that we then turned into a Series, rather than incrementally building up the Series as we went. Our earlier approach required building up a list (which has a memory overhead) and then building a *second* structure for the Series, giving us two objects in memory. This brings us to another common mistake when using Pandas and NumPy.

As a general rule, you should avoid repeated calls to `concat` in Pandas (and to the equivalent `concatenate` in NumPy). In [Example 6-30](#), we see a similar solution to the preceding one but without the intermediate `ms` list. This solution takes 56 seconds, as opposed to the solution using a list at 18.6 seconds!

Example 6-30. Concatenating each result incurs a significant overhead—avoid this!

```
results = None
for row_idx in range(df.shape[0]):
    row = df.iloc[row_idx]
    m = ols_lstsq(row)
    if results is None:
        results = pd.Series([m])
    else:
        results = pd.concat((results, pd.Series([m])))
```

Each concatenation creates an entirely *new* Series object in a new section of memory that is one row longer than the previous item. In addition, we have to make a temporary Series object for each new `m` on each iteration. We strongly recommend building up lists of intermediate results and then constructing a Series or DataFrame from this list, rather than concatenating to an existing object.

There's More Than One (and Possibly a Faster) Way to Do a Job

Because of the evolution of Pandas, there are typically a couple of approaches to solving the same task, some of which incur more overhead than others. Let's take the OLS DataFrame and convert one column into a string; we'll then time some string operations. With string-based columns containing names, product identifiers, or codes, it is common to have to preprocess the data to turn it into something we can analyze.

Let's say that we need to find the location, if it exists, of the number 9 in the digits from one of the columns. While this operation serves no real purpose, it is very similar to checking for the presence of a code-bearing symbol in an identifier's sequence or checking for an honorific in a name. Typically for these operations, we'd use `strip` to remove extraneous whitespace, `lower` and `replace` to normalize the string, and `find` to locate something of interest.

In [Example 6-31](#), we first build a new Series named `0_as_str`, which is the zeroth Series of random numbers converted into a printable string form. We'll then run two variants of string manipulation code—both will remove the leading digit and decimal point and then use Python's `find` to locate the first 9 if it exists, returning `-1` otherwise.

Example 6-31. str Series operations versus apply for string processing

```
In [10]: df['0_as_str'] = df[0].apply(lambda v: str(v))
Out[10]:
```

```
      0      0_as_str
0  1.016667  1.0166666666666666
1  1.033333  1.0333333333333334
2  0.966667  0.9666666666666667
...
```

```
def find_9(s):
    """Return -1 if '9' not found else its location at position >= 0"""
    return s.split('.')[1].find('9')
```

```
In [11]: df['0_as_str'].str.split('.', expand=True)[1].str.find('9')
Out[11]:
```

```
0    -1
1    -1
2     0
```

```
In [12]: %timeit df['0_as_str'].str.split('.', expand=True)[1].str.find('9')
Out[12]: 183 ms ± 4.62 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

```
In [13]: %timeit df['0_as_str'].apply(find_9)
Out[13]: 51 ms ± 987 µs per loop (mean ± std. dev. of 7 runs, 10 loops each)
```

The one-line approach uses Pandas’s `str` operations to access Python’s string methods for a Series. For `split`, we expand the returned result into two columns (the first column contains the leading digit, and the second contains everything after the decimal place), and we select column 1. We then apply `find` to locate the digit 9. The second approach uses `apply` and the function `find_9`, which reads like a regular Python string-processing function.

We can use `%timeit` to check the runtime—this shows us that there’s a 3.5× speed difference between the two methods, even though they both produce the same result! In the former one-line case, Pandas has to make several new intermediate Series objects, which adds overhead; in the `find_9` case, all of the string-processing work occurs one line at a time without creating new intermediate Pandas objects.

Further benefits of the `apply` approach are that we could parallelize this operation (see “[Parallel Pandas with Dask](#)” on page 322 for an example with Dask and Swifter), and we can write a unit test that succinctly confirms the operations performed by `find_9`, which will aid in readability and maintenance.

Advice for Effective Pandas Development

Install the optional dependencies `numexpr` and `bottleneck` for additional performance improvements. These don't get installed by default, and you won't be told if they're missing. `bottleneck` is rarely used in the code base; `numexpr`, however, will give significant speedups in some situations when you use `exec`. You can test for the presence of both in your environment with `import bottleneck` and `import numexpr`.

Don't write your code too tersely; remember to make your code easy to read and debug to help your future self. While the “method chaining” style is supported, your authors would caution against chaining too many rows of Pandas operations in sequence. It typically becomes difficult to figure out which line has problems when debugging, and then you have to split up the lines—you're better off chaining only a couple of operations together at most to simplify your maintenance.

Avoid doing more work than necessary: it is preferable to filter your data before calculating on the remaining rows rather than filtering after calculating, if possible. For high performance in general, we want to ask the machine to do as little computation as possible; if you can filter out or mask away portions of your data, you're probably winning. If you're consuming from a SQL source and later joining or filtering in Pandas, you might want to try to filter first at the SQL level, to avoid pulling more data than necessary into Pandas. You may *not* want to do this at first if you're investigating data quality, as having a simplified view on the variety of datatypes you have might be more beneficial.

Check the schema of your DataFrames as they evolve; with a tool like `bulwark`, you guarantee at runtime that your schema is being met, and you can visually confirm when you're reviewing code that your expectations are being met. Keep renaming your columns as you generate new results so that your DataFrame's contents make sense to you; sometimes `groupby` and other operations give you silly default names, which can later be confusing. Drop columns that you no longer need with `.drop()` to reduce bloat and memory usage.

For large Series containing strings with low cardinality (“yes” and “no,” for example, or “type_a,” “type_b,” and “type_c”), try converting the Series to a Category dtype with `df['series_of_strings'].astype('category')`; you may find that operations like `value_counts` and `groupby` run faster, and the Series is likely to consume less RAM.

Similarly, you may want to convert 8-byte `float64` and `int64` columns to smaller datatypes—perhaps the 2-byte `float16` or 1-byte `int8` if you need a smaller range to further save RAM.

As you evolve DataFrames and generate new copies, remember that you can use the `del` keyword to delete earlier references and clear them from memory, if they're large and wasting space. You can also use the Pandas `drop` method to delete unused columns.

If you're manipulating large DataFrames while you prepare your data for processing, it may make sense to do these operations once in a function or a separate script and then persist the prepared version to disk by using `to_pickle`. You can subsequently work on the prepared DataFrame without having to process it each time.

Avoid the `inplace=True` operator—in-place operations are scheduled to be removed from the library over time.

Finally, always add unit tests to any processing code, as it will quickly become more complex and harder to debug. Developing your tests up front guarantees that your code meets your expectations and helps you to avoid silly mistakes creeping in later that cost developer time to debug.

Existing tools for making Pandas go faster include **Modin** and the GPU-focused **cuDF**. Modin and cuDF take different approaches to parallelizing common data operations on a Pandas DataFrame-like object.

We'd like to also give an honorable mention to the new **Vaex library**. Vaex is designed to work on very large datasets that exceed RAM by using lazy evaluation while retaining a similar interface to that of Pandas. In addition, Vaex offers a slew of built-in visualization functions. One design goal is to use as many CPUs as possible, offering parallelism for free where possible.

Vaex specializes in both larger datasets and string-heavy operations; the authors have rewritten many of the string operations to avoid the standard Python functions and instead use faster Vaex implementations in C++. Note that Vaex is not guaranteed to work in the same way as Pandas, so it is possible that you'll find edge cases with different behavior—as ever, back your code with unit tests to gain confidence if you're trying both Pandas and Vaex to process the same data.

Wrap-Up

In the next chapter, we will talk about how to create your own external modules that can be finely tuned to solve specific problems with much greater efficiencies. This allows us to follow the rapid prototyping method of making our programs—first solve the problem with slow code, then identify the elements that are slow, and finally, find ways to make those elements faster. By profiling often and trying to optimize only the sections of code we *know* are slow, we can save ourselves time while still making our programs run as fast as possible.

HPP Ch 11 | Using Less RAM

Using Less RAM

Questions You'll Be Able to Answer After This Chapter

- Why should I use less RAM?
- Why are `numpy` and `array` better for storing lots of numbers?
- How can lots of text be efficiently stored in RAM?
- How could I count (approximately!) to 10^{76} using just 1 byte?
- What are Bloom filters, and why might I need them?

We rarely think about how much RAM we're using until we run out of it. If you run out while scaling your code, it can become a sudden blocker. Fitting more into a machine's RAM means fewer machines to manage, and it gives you a route to planning capacity for larger projects. Knowing why RAM gets eaten up and considering more efficient ways to use this scarce resource will help you deal with scaling issues. We'll use the Memory Profiler and IPython Memory Usage tools to measure the actual RAM usage, along with some tools that introspect objects to try to guess how much RAM they're using.

Another route to saving RAM is to use containers that utilize features in your data for compression. In this chapter, we'll look at a trie (ordered tree data structures) and a directed acyclic word graph (DAWG) that can compress a 1.2 GB set of strings down to just 30 MB with little change in performance. A third approach is to trade storage for accuracy. For this we'll look at approximate counting and approximate set membership, which use dramatically less RAM than their exact counterparts.

A consideration with RAM usage is the notion that “data has mass.” The more there is of it, the slower it moves around. If you can be parsimonious in your use of RAM, your data will probably get consumed faster, as it’ll move around buses faster and more of it will fit into constrained caches. If you need to store it in offline storage (e.g., a hard drive or a remote data cluster), it’ll move far more slowly to your machine. Try to choose appropriate data structures so all your data can fit onto one machine. We’ll use NumExpr to efficiently calculate with NumPy and Pandas with fewer data movements than the more direct method, which will save us time and make certain larger calculations feasible in a fixed amount of RAM.

Counting the amount of RAM used by Python objects is surprisingly tricky. We don’t necessarily know how an object is represented behind the scenes, and if we ask the operating system for a count of bytes used, it will tell us about the total amount allocated to the process. In both cases, we can’t see exactly how each individual Python object adds to the total.

As some objects and libraries don’t report their full internal allocation of bytes (or they wrap external libraries that do not report their allocation at all), this has to be a case of best-guessing. The approaches explored in this chapter can help us to decide on the best way to represent our data so we use less RAM overall.

We’ll also look at several lossy methods for storing strings in scikit-learn and counts in data structures. This works a little like a JPEG compressed image—we lose some information (and we can’t undo the operation to recover it), and we gain a lot of compression as a result. By using hashes on strings, we compress the time and memory usage for a natural language processing task in scikit-learn, and we can count huge numbers of events with only a small amount of RAM.

Objects for Primitives Are Expensive

It’s common to work with containers like the `list`, storing hundreds or thousands of items. As soon as you store a large number, RAM usage becomes an issue.

A `list` with 100 million items consumes approximately 760 MB of RAM, *if the items are the same object*. If we store 100 million *different* items (e.g., unique integers), we can expect to use gigabytes of RAM! Each unique object has a memory cost.

In [Example 11-1](#), we store many `0` integers in a `list`. If you stored 100 million references to any object (regardless of how large one instance of that object was), you’d still expect to see a memory cost of roughly 760 MB, as the `list` is storing references to (not copies of) the object. Refer back to [“Using `memory\_profiler` to Diagnose Memory Usage” on page 46](#) for a reminder of how to use `memory_profiler`; here, we load it as a new magic function in IPython using `%load_ext memory_profiler`.

Example 11-1. Measuring memory usage of 100 million of the same integer in a list

```
In [1]: %load_ext memory_profiler # load the %memit magic function
In [2]: %memit [0] * int(1e8)
peak memory: 806.33 MiB, increment: 762.77 MiB
```

For our next example, we'll start with a fresh shell. As the results of the first call to `memit` in [Example 11-2](#) reveal, a fresh IPython shell consumes approximately 40 MB of RAM. Next, we can create a temporary list of 100 million *unique* numbers. In total, this consumes approximately 3.8 GB.



Memory can be cached in the running process, so it is always safer to exit and restart the Python shell when using `memit` for profiling.

After the `memit` command finishes, the temporary list is deallocated. The final call to `memit` shows that the memory usage drops to its previous level.

Example 11-2. Measuring memory usage of 100 million different integers in a list

```
# we use a new IPython shell so we have a clean memory
In [1]: %load_ext memory_profiler
In [2]: %memit # show how much RAM this process is consuming right now
peak memory: 43.39 MiB, increment: 0.11 MiB
In [3]: %memit [n for n in range(int(1e8))]
peak memory: 3850.29 MiB, increment: 3806.59 MiB
In [4]: %memit
peak memory: 44.79 MiB, increment: 0.00 MiB
```

A subsequent `memit` in [Example 11-3](#) to create a second 100-million-item list consumes approximately 3.8 GB again.

Example 11-3. Measuring memory usage again for 100 million different integers in a list

```
In [5]: %memit [n for n in range(int(1e8))]
peak memory: 3855.78 MiB, increment: 3810.96 MiB
```

Next, we'll see that we can use the `array` module to store 100 million integers far more cheaply.

The array Module Stores Many Primitive Objects Cheaply

The `array` module efficiently stores primitive types like integers, floats, and characters, but *not* complex numbers or classes. It creates a contiguous block of RAM to hold the underlying data.

In [Example 11-4](#), we allocate 100 million integers (8 bytes each) into a contiguous chunk of memory. In total, approximately 760 MB is consumed by the process. The difference between this approach and the previous list-of-unique-integers approach is $3100\text{MB} - 760\text{MB} = 2.3\text{GB}$. This is a huge savings in RAM.

Example 11-4. Building an array of 100 million integers with 760 MB of RAM

```
In [1]: %load_ext memory_profiler
In [2]: import array
In [3]: %memit array.array('l', range(int(1e8)))
peak memory: 837.88 MiB, increment: 761.39 MiB
In [4]: arr = array.array('l')
In [5]: arr.itemsize
Out[5]: 8
```

Note that the unique numbers in the array are *not* Python objects; they are bytes in the array. If we were to dereference any of them, a new Python `int` object would be constructed. If you're going to compute on them, no overall savings will occur, but if instead you're going to pass the array to an external process or use only some of the data, you should see a good savings in RAM compared to using a list of integers.



If you're working with a large array or matrix of numbers with Cython and you don't want an external dependency on `numpy`, be aware that you can store your data in an array and pass it into Cython for processing without any additional memory overhead.

The `array` module works with a limited set of datatypes with varying precisions (see [Example 11-5](#)). Choose the smallest precision that you need, so that you allocate just as much RAM as needed and no more. Be aware that the byte size is platform-dependent—the sizes here refer to a 32-bit platform (it states *minimum* size), whereas we're running the examples on a 64-bit laptop.

Example 11-5. The basic types provided by the array module

```
In [5]: array.array? # IPython magic, similar to help(array)
Init signature: array.array(self, /, *args, **kwargs)
Docstring:
array(typecode [, initializer]) -> array
```

Return a new array whose items are restricted by typecode, and initialized from the optional initializer value, which must be a list, string, or iterable over elements of the appropriate type.

Arrays represent basic values and behave very much like lists, except the type of objects stored in them is constrained. The type is specified at object creation time by using a type code, which is a single character. The following type codes are defined:

Type code	C Type	Minimum size in bytes
'b'	signed integer	1
'B'	unsigned integer	1
'u'	Unicode character	2 (see note)
'h'	signed integer	2
'H'	unsigned integer	2
'i'	signed integer	2
'I'	unsigned integer	2
'l'	signed integer	4
'L'	unsigned integer	4
'q'	signed integer	8 (see note)
'Q'	unsigned integer	8 (see note)
'f'	floating point	4
'd'	floating point	8

NumPy has arrays that can hold a wider range of datatypes—you have more control over the number of bytes per item, and you can use complex numbers and datetime objects. A `complex128` object takes 16 bytes per item: each item is a pair of 8-byte floating-point numbers. You can't store complex objects in a Python array, but they come for free with numpy. If you'd like a refresher on numpy, look back to [Chapter 6](#).

In [Example 11-6](#), you can see an additional feature of numpy arrays; you can query for the number of items, the size of each primitive, and the combined total storage of the underlying block of RAM. Note that this doesn't include the overhead of the Python object (typically, this is tiny in comparison to the data you store in the arrays).



Be wary of lazy allocation with zeros. In the following example the call to zeros costs “zero” RAM, whilst the call to ones costs 1.5 GB. Both calls will ultimately cost 1.5 GB, but the call to zeros will allocate the RAM only after it is used, so the cost is seen later.

Example 11-6. Storing more complex types in a numpy array

```
In [1]: %load_ext memory_profiler
In [2]: import numpy as np
# NOTE that zeros have lazy allocation so misreport the memory used!
In [3]: %memit arr=np.zeros(int(1e8), np.complex128)
peak memory: 58.37 MiB, increment: 0.00 MiB
In [4]: %memit arr=np.ones(int(1e8), np.complex128)
peak memory: 1584.41 MiB, increment: 1525.89 MiB
In [5]: f"{arr.size:,}"
Out[5]: '100,000,000'
In [6]: f"{arr.nbytes:,}"
Out[6]: '1,600,000,000'
In [7]: arr.nbytes/arr.size
Out[7]: 16.0
In [8]: arr.itemsize
Out[8]: 16
```

Using a regular list to store many numbers is much less efficient in RAM than using an array object. More memory allocations have to occur, which each take time; calculations also occur on larger objects, which will be less cache friendly, and more RAM is used overall, so less RAM is available to other programs.

However, if you do any work on the contents of the array in Python numex, the primitives are likely to be converted into temporary objects, negating their benefit. Using them as a data store when communicating with other processes is a great use case for the array.

numpy arrays are almost certainly a better choice if you are doing anything heavily numeric, as you get more datatype options and many specialized and fast functions. You might choose to avoid numpy if you want fewer dependencies for your project, though Cython works equally well with array and numpy arrays; Numba works with numpy arrays only.

Python provides a few other tools to understand memory usage, as we'll see in the following section.

Using Less RAM in NumPy with NumExpr

Large vectorized expressions in NumPy (which also happen behind the scenes in Pandas) can create intermediate large arrays during complex operations. These occur invisibly and may draw attention to themselves only when an out-of-memory error occurs. These calculations can also be slow, as large vectors will not be cache friendly—a cache can be megabytes or smaller, and large vectors of hundreds of megabytes or gigabytes of data will stop the cache from being used effectively. NumExpr is a tool that both speeds up and reduces the size of intermediate operations; we introduced it in “[numexpr: Making In-Place Operations Faster and Easier](#)” on page 140.

We’ve also introduced the Memory Profiler before, in “[Using memory_profiler to Diagnose Memory Usage](#)” on page 46. Here, we build on it with the [IPython Memory Usage tool](#), which reports line-by-line memory changes inside the IPython shell or in a Jupyter Notebook. Let’s look at how these can be used to check that NumExpr is generating a result more efficiently.



Remember to install the optional NumExpr when using Pandas. If NumExpr is installed in Pandas, calls to `eval` will run more quickly—but note that Pandas does not tell you if you *haven’t* installed NumExpr.

We’ll use the cross entropy formula to calculate the error for a machine learning classification challenge. *Cross entropy* (or *Log Loss*) is a common metric for classification challenges; it penalizes large errors significantly more than small errors. Each row in a machine learning problem needs to be scored during the training and prediction phases:

$$-\log P(yt \mid yp) = -(yt \log(yp) + (1 - yt) \log(1 - yp))$$

We’ll use random numbers in the range $[0, 1]$ here to simulate the results of a machine learning system from a package like scikit-learn or TensorFlow. [Figure 11-1](#) shows the natural logarithm for the range $[0, 1]$ on the right, and on the left it shows the result of calculating the cross entropy for any probability if the target is either 0 or 1.

If the target `yt` is 1, the first half of the formula is active and the second half goes to zero. If the target is 0, the second half of the formula is active and the first part goes to zero. This result is calculated for every row of data that needs to be scored and often for many iterations of the machine learning algorithm.

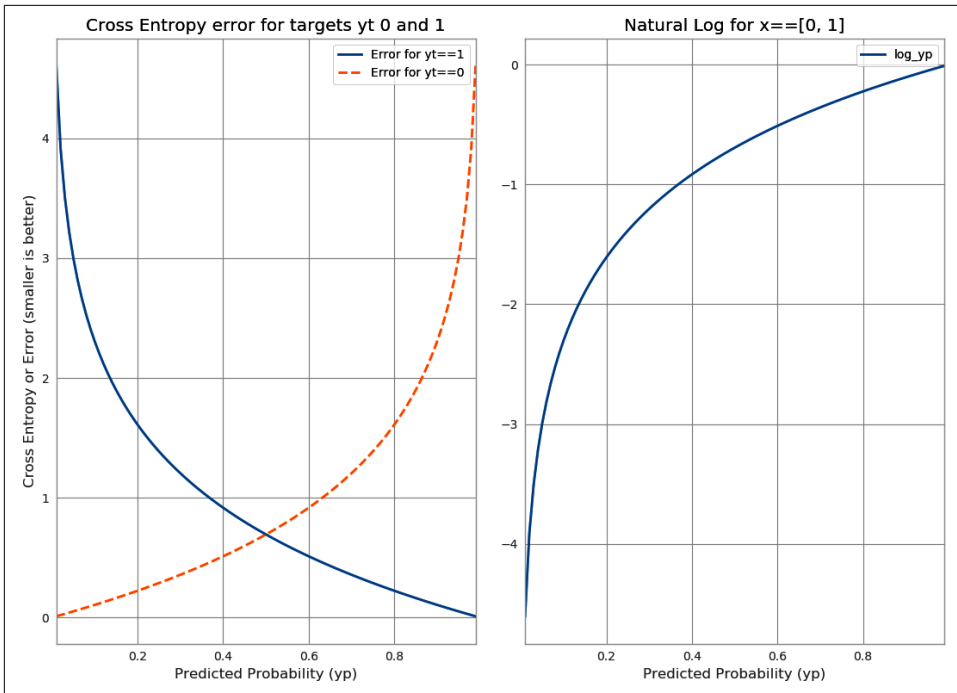


Figure 11-1. Cross entropy for y_t (the “truth”) with values 0 and 1

In [Example 11-7](#), we generate 200 million random numbers in the range [0, 1] as `yp`. `yt` is the desired truth—in this case, an array of 1s. In a real application, we’d see `yp` generated by a machine learning algorithm, and `yt` would be the ground truth mixing 0s and 1s for the target we’d be learning provided by the machine learning researcher.

Example 11-7. The hidden cost of temporaries with large NumPy arrays

```
In [1]: import ipython_memory_usage.ipython_memory_usage as imu; import numpy as np
In [2]: %ipython_memory_usage_start
Out[2]: 'memory profile enabled'
In [3]: nbr_items = 200_000_000
In [4]: yp = np.random.uniform(low=0.0000001, size=nbr_items)
In [4] used 1526.0508 MiB RAM in 2.18s, peaked 0.00 MiB above current,
       total RAM usage 1610.05 MiB
In [5]: yt = np.ones(shape=nbr_items)
In [5] used 1525.8516 MiB RAM in 0.44s, peaked 0.00 MiB above current,
       total RAM usage 3135.90 MiB

In [6]: answer = -(yt * np.log(yp) + ((1-yt) * (np.log(1-yp))))
In [6] used 1525.8594 MiB RAM in 18.63s, peaked 4565.70 MiB above current,
       total RAM usage 4661.76 MiB
```

```
In [7]: del answer
In [7] used -1525.8242 MiB RAM in 0.11s, peaked 0.00 MiB above current,
      total RAM usage 3135.93 MiB
```

Both `yp` and `yt` take 1.5 GB each, bringing the total RAM usage to just over 3.1 GB. The `answer` vector has the same dimension as the inputs and thus adds a further 1.5 GB. Note that the calculation peaks at 4.5 GB over the current RAM usage, so while we end with a 4.6 GB result, we had over 9 GB allocated during the calculation. The cross entropy calculation creates several temporaries (notably $1 - yt$, $\text{np.log}(1 - yp)$, and their multiplication). If you had an 8 GB machine, you'd have failed to calculate this result because of memory exhaustion.

In [Example 11-8](#), we see the same expression placed as a string inside `numexpr.evaluate`. It peaks at 0 GB above the current usage—it doesn't need any additional RAM in this case. Significantly, it also calculates much more quickly: the previous direct vector calculation in `In[6]` took 18 seconds, while here with `NumExpr` the same calculation takes 2.6 seconds.

`NumExpr` breaks the long vectors into shorter, cache-friendly chunks and processes each in series, so local chunks of results are calculated in a cache-friendly way. This explains both the requirement for no extra RAM and the increased speed.

Example 11-8. NumExpr breaks the vectorized calculations into cache-efficient chunks

```
In [8]: import numexpr
In [8] used 0.0430 MiB RAM in 0.12s, peaked 0.00 MiB above current,
      total RAM usage 3135.95 MiB
In [9]: answer = numexpr.evaluate("-(yt * log(yp) + ((1-yt) * (log(1-yp))))")
In [9] used 1525.8281 MiB RAM in 2.67s, peaked 0.00 MiB above current,
      total RAM usage 4661.78 MiB
```

We can see a similar benefit in `Pandas` in [Example 11-9](#). We construct a `DataFrame` with the same items as in the preceding example and invoke `NumExpr` by using `df.eval`. The `Pandas` machinery has to unpack the `DataFrame` for `NumExpr`, and more RAM is used overall; behind the scenes, `NumExpr` is still calculating the result in a cache-friendly manner. Note that here `NumExpr` was installed in addition to `Pandas`.

Example 11-9. Pandas eval uses NumExpr if it is available

```
In [2] df = pd.DataFrame({'yp': np.random.uniform(low=0.0000001, size=nbr_items),
      'yt': np.ones(nbr_items)})
In [3]: answer_eval = df.eval("-(yt * log(yp) + ((1-yt) * (log(1-yp))))")
In [3] used 3052.1953 MiB RAM in 5.26s, peaked 3045.77 MiB above current,
      total RAM usage 6185.45 MiB
```

Contrast the preceding example with [Example 11-10](#), where NumExpr has *not* been installed. The call to `df.eval` falls back on the Python interpreter—the same result is calculated but with a 34-second execution time (compared to 5.2 seconds before) and a much larger peak memory usage. You can test whether NumExpr is installed with `import numexpr`—if this fails, you’ll want to install it.

Example 11-10. Beware that Pandas without NumExpr makes a slow and costly call to eval!

```
In [2] df = pd.DataFrame({'yp': np.random.uniform(low=0.0000001, size=nbr_items),
                          'yt': np.ones(nbr_items)})
In [3]: answer_eval = df.eval("-(yt * log(yp) + ((1-yt) * (log(1-yp))))")
In [3] used 3052.5625 MiB RAM in 34.88s, peaked 7620.15 MiB above current,
      total RAM usage 6185.24 MiB
```

Complex vector operations on large arrays will run faster if you can use NumExpr. Pandas will not warn you that NumExpr hasn’t been installed, so we recommend that you add it as part of your setup if you use `eval`. The IPython Memory Usage tool will help you to diagnose where your RAM is being spent if you have large arrays to process; this can help you fit more into RAM on your current machine so you don’t have to start dividing your data and introducing greater engineering effort.

Understanding the RAM Used in a Collection

You may wonder if you can ask Python about the RAM that’s used by each object. Python’s `sys.getsizeof(obj)` call will tell us *something* about the memory used by an object (most but not all objects provide this). If you haven’t seen it before, be warned that it won’t give you the answer you’d expect for a container!

Let’s start by looking at some primitive types. An `int` in Python is a variable-sized object of arbitrary size, well above the range of an 8-byte C integer. The basic object costs 24 bytes in Python 3.7 when initialized with 0. More bytes are added as you count to larger numbers:

```
In [1]: sys.getsizeof(0)
Out[1]: 24
In [2]: sys.getsizeof(1)
Out[2]: 28
In [3]: sys.getsizeof((2**30)-1)
Out[3]: 28
In [4]: sys.getsizeof((2**30))
Out[4]: 32
```

Behind the scenes, sets of 4 bytes are added each time the size of the number you’re counting steps above the previous limit. This affects only the memory usage; you don’t see any difference externally.

We can do the same check for byte strings. An empty byte sequence costs 33 bytes, and each additional character adds 1 byte to the cost:

```
In [5]: sys.getsizeof(b'')
Out[5]: 33
In [6]: sys.getsizeof(b'a')
Out[6]: 34
In [7]: sys.getsizeof(b'ab')
Out[7]: 35
In [8]: sys.getsizeof(b'abc')
Out[8]: 36
```

When we use a list, we see different behavior. `getsizeof` isn't counting the cost of the contents of the list—just the cost of the list itself. An empty list costs 64 bytes, and each item in the list takes another 8 bytes on a 64-bit laptop:

```
# goes up in 8-byte steps rather than the 24+ we might expect!
In [9]: sys.getsizeof([])
Out[9]: 64
In [10]: sys.getsizeof([1])
Out[10]: 72
In [11]: sys.getsizeof([1, 2])
Out[11]: 80
```

This is more obvious if we use byte strings—we'd expect to see much larger costs than `getsizeof` is reporting:

```
In [12]: sys.getsizeof([b''])
Out[12]: 72
In [13]: sys.getsizeof([b'abcdefghijklm'])
Out[13]: 72
In [14]: sys.getsizeof([b'a', b'b'])
Out[14]: 80
```

`getsizeof` reports only some of the cost, and often just for the parent object. As noted previously, it also isn't always implemented, so it can have limited usefulness.

A better tool is `asizeof` in `pympler`. This will walk a container's hierarchy and make a best guess about the size of each object it finds, adding the sizes to a total. Be warned that it is quite slow.

In addition to relying on guesses and assumptions, `asizeof` also cannot count memory allocated behind the scenes (such as a module that wraps a C library may not report the bytes allocated in the C library). It is best to use this as a guide. We prefer to use `memit`, as it gives us an accurate count of memory usage on the machine in question.

We can check the estimate it makes for a large list—here we'll use 10 million integers:

```
In [1]: from pympler.asizeof import asizeof
In [2]: asizeof([x for x in range(int(1e7))])
Out[2]: 401528048
In [3]: %memit [x for x in range(int(1e7))]
peak memory: 401.91 MiB, increment: 326.77 MiB
```

We can validate this estimate by using `memit` to see how the process grew. Both reports are approximate—`memit` takes snapshots of the RAM usage reported by the operating system while the statement is executing, and `sizeof` asks the objects about their size (which may not be reported correctly). We can conclude that 10 million integers in a list cost between 320 and 400 MB of RAM.

Generally, the `sizeof` process is slower than using `memit`, but `sizeof` can be useful when you're analyzing small objects. `memit` is probably more useful for real-world applications, as the actual memory usage of the process is measured rather than inferred.

Bytes Versus Unicode

One of the (many!) advantages of Python 3.x over Python 2.x is the switch to Unicode-by-default. Previously, we had a mix of single-byte strings and multibyte Unicode objects, which could cause a headache during data import and export. In Python 3.x, all strings are Unicode by default, and if you want to deal in bytes, you'll explicitly create a byte sequence.

Unicode objects have more efficient RAM usage in Python 3.7 than in Python 2.x. In [Example 11-11](#), we can see a 100-million-character sequence being built as a collection of bytes and as a Unicode object. The Unicode variant for common characters (here we're assuming UTF 8 as the system's default encoding) costs the same—a single-byte implementation is used for these common characters.

Example 11-11. Unicode objects can be as cheap as bytes in Python 3.x

```
In [1]: %load_ext memory_profiler
In [2]: type(b"b")
Out[2]: bytes
In [3]: %memit b"a" * int(1e8)
peak memory: 121.55 MiB, increment: 78.17 MiB
In [4]: type("u")
Out[4]: str
In [5]: %memit "u" * int(1e8)
peak memory: 122.43 MiB, increment: 78.49 MiB
In [6]: %memit "Σ" * int(1e8)
peak memory: 316.40 MiB, increment: 176.17 MiB
```

The sigma character (Σ) is more expensive—it is represented in UTF 8 as 2 bytes. We gained the flexible Unicode representation from Python 3.3 thanks to [PEP 393](#). It works by observing the range of characters in the string and using a smaller number of bytes to represent the lower-order characters, if possible.

The UTF-8 encoding of a Unicode object uses 1 byte per ASCII character and more bytes for less frequently seen characters. If you're not sure about Unicode encodings versus Unicode objects, go and watch [Net Batchelder's "Pragmatic Unicode, or, How Do I Stop the Pain?"](#).

Efficiently Storing Lots of Text in RAM

A common problem with text is that it occupies a lot of RAM—but if we want to test if we have seen strings before or count their frequency, having them in RAM is more convenient than paging them to and from a disk. Storing the strings naively is expensive, but tries and directed acyclic word graphs (DAWGs) can be used to compress their representation and still allow fast operations.

These more advanced algorithms can save you a significant amount of RAM, which means that you might not need to expand to more servers. For production systems, the savings can be huge. In this section we'll look at compressing a set of strings costing 1.2 GB down to 30 MB using a trie, with only a small change in performance.

For this example, we'll use a text set built from a partial dump of Wikipedia. This set contains 11 million unique tokens from a portion of the English Wikipedia and takes up 120 MB on disk.

The tokens are split on whitespace from their original articles; they have variable length and contain Unicode characters and numbers. They look like this:

```
faddishness  
'melanesians'  
Kharálampos  
PizzaInACup™  
url="http://en.wikipedia.org/wiki?curid=363886"  
VIIIa),  
Superbagnères.
```

We'll use this text sample to test how quickly we can build a data structure holding one instance of each unique word, and then we'll see how quickly we can query for a known word (we'll use the uncommon “Zwiebel,” from the painter Alfred Zwiebel). This lets us ask, “Have we seen Zwiebel before?” Token lookup is a common problem, and being able to do it quickly is important.



When you try these containers on your own problems, be aware that you will probably see different behaviors. Each container builds its internal structures in different ways; passing in different types of token is likely to affect the build time of the structure, and different lengths of token will affect the query time. Always test in a methodical way.

Trying These Approaches on 11 Million Tokens

Figure 11-2 shows the 11-million-token text file (120 MB raw data) stored using a number of containers that we'll discuss in this section. The x-axis shows RAM usage for each container, the y-axis tracks the query time, and the size of each point relates to the time taken to build the structure (larger means it took longer).

As we can see in this diagram, the `set` and `list` examples use a lot of RAM; the `list` example is both large *and* slow! The Marisa trie example is the most RAM-efficient for this dataset, while the DAWG runs twice as fast for a relatively small increase in RAM usage.

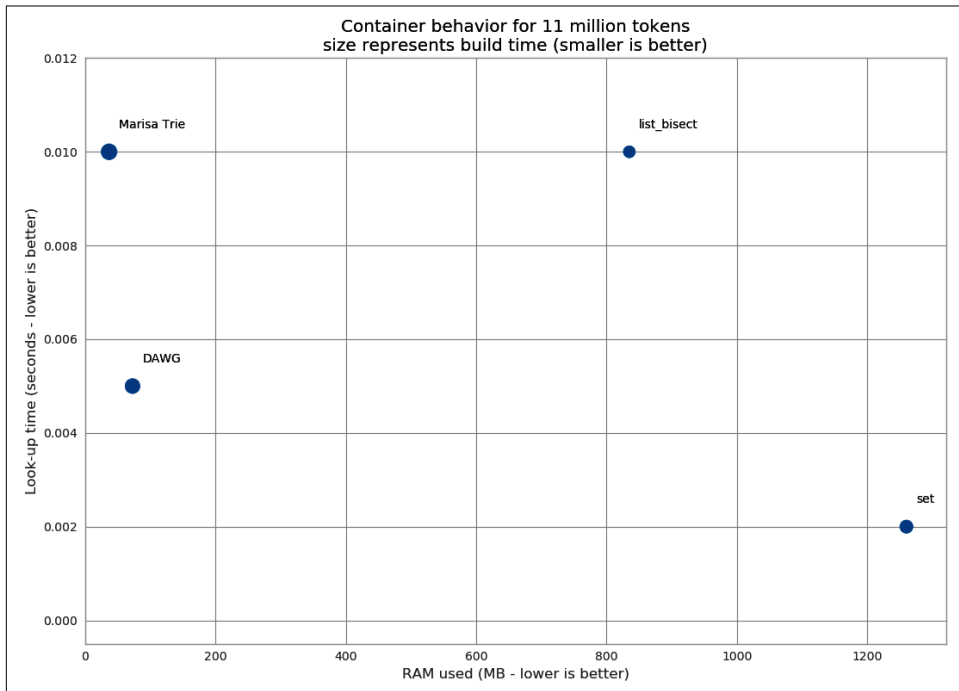


Figure 11-2. DAWG and tries versus built-in containers

The figure doesn't show the lookup time for the naive `list` without sort approach, which we'll introduce shortly, as it takes far too long. Do be aware that you must test

your problem with a variety of containers—each offers different trade-offs, such as construction time and API flexibility.

Next, we'll build up a process to test the behavior of each container.

list

Let's start with the simplest approach. We'll load our tokens into a `list` and then query it using an $O(n)$ linear search. You can't do this on the large example that we've already mentioned—the search takes far too long—so we'll demonstrate the technique with a much smaller (500,000 tokens) example.

In each of the following examples, we use a generator, `text_example.readers`, that extracts one Unicode token at a time from the input file. This means that the read process uses only a tiny amount of RAM:

```
print("RAM at start {:.1f}MiB".format(memory_profiler.memory_usage()[0]))
t1 = time.time()
words = [w for w in text_example.readers]
print("Loading {} words".format(len(words)))
t2 = time.time()
print("RAM after creating list {:.1f}MiB, took {:.1f}s" \
      .format(memory_profiler.memory_usage()[0], t2 - t1))
```

We're interested in how quickly we can query this list. Ideally, we want to find a container that will store our text and allow us to query it and modify it without penalty. To query it, we look for a known word a number of times by using `timeit`:

```
assert 'Zwiebel' in words
time_cost = sum(timeit.repeat(stmt="'Zwiebel' in words",
                              setup="from __main__ import words",
                              number=1,
                              repeat=10000))
print("Summed time to look up word {:.4f}s".format(time_cost))
```

Our test script reports that approximately 34 MB was used to store the original 5 MB file as a list, and that the aggregate lookup time was 53 seconds:

```
$ python text_example_list.py
RAM at start 36.6MiB
Loading 499056 words
RAM after creating list 70.9MiB, took 1.0s
Summed time to look up word 53.5657s
```

Storing text in an unsorted list is obviously a poor idea; the $O(n)$ lookup time is expensive, as is the memory usage. This is the worst of all worlds! If we tried this method on the following larger dataset, we'd expect an aggregate lookup time of 25 minutes rather than a fraction of a second for the methods we discuss.

We can improve the lookup time by sorting the list and using a binary search via the `bisect` module; this gives us a sensible lower bound for future queries. In

Example 11-12, we time how long it takes to sort the list. Here, we switch to the larger 11-million-token set.

Example 11-12. Timing the sort operation to prepare for using bisect

```
print("RAM at start {:.1f}MiB".format(memory_profiler.memory_usage()[0]))
t1 = time.time()
words = [w for w in text_example.readers]
print("Loading {} words".format(len(words)))
t2 = time.time()
print("RAM after creating list {:.1f}MiB, took {:.1f}s" \
      .format(memory_profiler.memory_usage()[0], t2 - t1))
print("The list contains {} words".format(len(words)))
words.sort()
t3 = time.time()
print("Sorting list took {:.1f}s".format(t3 - t2))
```

Next, we do the same lookup as before, but with the addition of the index method, which uses bisect:

```
import bisect
...
def index(a, x):
    'Locate the leftmost value exactly equal to x'
    i = bisect.bisect_left(a, x)
    if i != len(a) and a[i] == x:
        return i
    raise ValueError
...
time_cost = sum(timeit.repeat(stmt="index(words, 'Zwiebel')",
                              setup="from __main__ import words, index",
                              number=1,
                              repeat=10000))
```

In **Example 11-13**, we see that the RAM usage is much larger than before, as we're loading significantly more data. The sort takes a further 0.6 seconds, and the cumulative lookup time is 0.01 seconds.

Example 11-13. Timings for using bisect on a sorted list

```
$ python text_example_list_bisect.py
RAM at start 36.6MiB
Loading 11595290 words
RAM after creating list 871.9MiB, took 20.6s
The list contains 11595290 words
Sorting list took 0.6s
Summed time to look up word 0.0109s
```

We now have a sensible baseline for timing string lookups: RAM usage must get better than 871 MB, and the total lookup time should be better than 0.01 seconds.

set

Using the built-in `set` might seem to be the most obvious way to tackle our task. In [Example 11-14](#), the `set` stores each string in a hashed structure (see [Chapter 4](#) if you need a refresher). It is quick to check for membership, but each string must be stored separately, which is expensive on RAM.

Example 11-14. Using a `set` to store the data

```
words_set = set(text_example.readers)
```

As we can see in [Example 11-15](#), the `set` uses more RAM than the `list` by a further 250 MB; however, it gives us a very fast lookup time without requiring an additional index function or an intermediate sorting operation.

Example 11-15. Running the `set` example

```
$ python text_example_set.py
RAM at start 36.6MiB
RAM after creating set 1295.3MiB, took 24.0s
The set contains 11595290 words
Summed time to look up word 0.0023s
```

If RAM isn't at a premium, this might be the most sensible first approach.

We have now lost the *ordering* of the original data, though. If that's important to you, note that you could store the strings as keys in a dictionary, with each value being an index connected to the original read order. This way, you could ask the dictionary if the key is present and for its index.

More efficient tree structures

Let's introduce a set of algorithms that use RAM more efficiently to represent our strings.

[Figure 11-3](#) from [Wikimedia Commons](#) shows the difference in representation of four words—"tap", "taps", "top", and "tops"—between a trie and a DAWG.¹ With a `list` or a `set`, each of these words would be stored as a separate string. Both the DAWG and the trie share parts of the strings, so that less RAM is used.

¹ This example is taken from the Wikipedia article on the [deterministic acyclic finite state automaton](#) (DAFSA). DAFSA is another name for DAWG. The accompanying image is from Wikimedia Commons.

The main difference between these is that a trie shares just common prefixes, while a DAWG shares common prefixes and suffixes. In languages (like English) that have many common word prefixes and suffixes, this can save a lot of repetition.

Exact memory behavior will depend on your data's structure. Typically, a DAWG cannot assign a value to a key because of the multiple paths from the start to the end of the string, but the version shown here can accept a value mapping. Tries can also accept a value mapping. Some structures have to be constructed in a pass at the start, and others can be updated at any time.

A big strength of some of these structures is that they provide a *common prefix search*; that is, you can ask for all words that share the prefix you provide. With our list of four words, the result when searching for “ta” would be “tap” and “taps.” Furthermore, since these are discovered through the graph structure, the retrieval of these results is very fast. If you’re working with DNA, for example, compressing millions of short strings by using a trie can be an efficient way to reduce RAM usage.

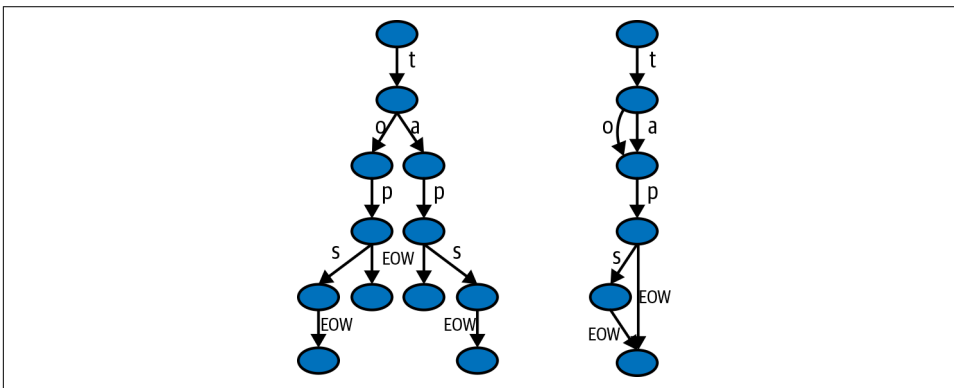


Figure 11-3. Trie and DAWG structures (image by [Chkno](#) [CC BY-SA 3.0])

In the following sections, we take a closer look at DAWGs, tries, and their usage.

Directed acyclic word graph

The **directed acyclic word graph** (MIT license) attempts to efficiently represent strings that share common prefixes and suffixes.

Note that at the time of writing, an [open Pull Request on GitHub](#) has to be applied to make this DAWG work with Python 3.7.

In **Example 11-16**, you see the very simple setup for a DAWG. For this implementation, the DAWG cannot be modified after construction; it reads an iterator to construct itself once. The lack of post-construction updates might be a deal breaker for your use case. If so, you might need to look into using a trie instead. The DAWG

does support rich queries, including prefix lookups; it also allows persistence and supports storing integer indices as values along with byte and record values.

Example 11-16. Using a DAWG to store the data

```
import dawg
...
words_dawg = dawg.DAWG(text_example.readers)
```

As you can see in [Example 11-17](#), for the same set of strings it uses significantly less RAM than the earlier set example during the *construction* phase. More similar input text will cause stronger compression.

Example 11-17. Running the DAWG example

```
$ python text_example_dawg.py
RAM at start 38.1MiB
RAM after creating dawg 200.8MiB, took 31.6s
Summed time to look up word 0.0044s
```

More importantly, if we persist the DAWG to disk, as shown in [Example 11-18](#), and then load it back into a fresh Python instance, we see a dramatic reduction in RAM usage—the disk file and the memory usage after loading are both 70 MB; this is a significant savings compared to the 1.2 GB set variant we built earlier!

Example 11-18. Loading the DAWG that was built and saved in an earlier session is more RAM efficient

```
$ python text_example_dawg_load_only.py
RAM at start 38.4MiB
RAM after load 109.0MiB
Summed time to look up word 0.0051s
```

Given that you'll typically create a DAWG once and then load it many times, you'll benefit from the construction costs repeatedly after you've persisted the structure to disk.

Marisa trie

The [Marisa trie](#) (dual-licensed LGPL and BSD) is a static [trie](#) using Cython bindings to an external library. As it is static, it cannot be modified after construction. Like the DAWG, it supports storing integer indices as values, as well as byte values and record values.

A key can be used to look up a value, and vice versa. All keys sharing the same prefix can be found efficiently. The trie's contents can be persisted. [Example 11-19](#) illustrates using a Marisa trie to store our sample data.

Example 11-19. Using a Marisa trie to store the data

```
import marisa_trie
...
words_trie = marisa_trie.Trie(text_example.readers)
```

In [Example 11-20](#), we can see that lookup times are slower than those offered by the DAWG.

Example 11-20. Running the Marisa trie example

```
$ python text_example_trie.py
RAM at start 38.3MiB
RAM after creating trie 419.9MiB, took 35.1s
The trie contains 11595290 words
Summed time to look up word 0.0148s
```

The trie offers a further saving in memory on this dataset. While the lookups are a little slower in [Example 11-21](#), the disk and RAM usage are approximately 30 MB in the following snippet, if we save the trie to disk and then load it back into a fresh process; this is twice as good as what the DAWG achieved.

Example 11-21. Loading the Trie that was built and saved in an earlier session is more RAM efficient

```
$ python text_example_trie_load_only.py
RAM at start 38.5MiB
RAM after loading trie from disk 76.7MiB, took 0.0s
The trie contains 11595290 words
Summed time to look up word 0.0092s
```

The trade-off between storage sizes after construction and lookup times will need to be investigated for your application. You may find that using one of these “works just well enough,” so you might avoid benchmarking other options and simply move on to your next challenge. We suggest that the Marisa trie is your first choice in this case; it has more stars than the DAWG on GitHub.

Using tries (and DAWGs) in production systems

The trie and DAWG data structures offer good benefits, but you must still benchmark them on your problem rather than blindly adopting them. If you have overlapping sequences in your strings, you'll likely see a RAM improvement.

Tries and DAWGs are less well known, but they can provide strong benefits in production systems. We have an impressive success story in “[Large-Scale Social Media Analysis at Smesh \(2014\)](#)” on page 422. Jamie Matthews at DabApps (a Python software house based in the United Kingdom) also has a story about the use of tries in client systems to enable more efficient and cheaper deployments for customers:

At DabApps, we often try to tackle complex technical architecture problems by dividing them into small, self-contained components, usually communicating over the network using HTTP. This approach (referred to as a *service-oriented* or *microservice* architecture) has all sorts of benefits, including the possibility of reusing or sharing the functionality of a single component between multiple projects.

One such task that is often a requirement in our consumer-facing client projects is postcode geocoding. This is the task of converting a full UK postcode (for example: BN1 1AG) into a latitude and longitude coordinate pair, to enable the application to perform geospatial calculations such as distance measurement.

At its most basic, a geocoding database is a simple mapping between strings and can conceptually be represented as a dictionary. The dictionary keys are the postcodes, stored in a normalized form (BN11AG), and the values are a representation of the coordinates (we used a geohash encoding, but for simplicity imagine a comma-separated pair such as 50.822921,-0.142871).

The UK has approximately 1.7 million postcodes. Naively loading the full dataset into a Python dictionary, as described previously, uses several hundred megabytes of memory. Persisting this data structure to disk using Python’s native Pickle format requires an unacceptably large amount of storage space. We knew we could do better.

We experimented with several different in-memory and on-disk storage and serialization formats, including storing the data externally in databases such as Redis and LevelDB, and compressing the key/value pairs. Eventually, we hit on the idea of using a trie. Tries are extremely efficient at representing large numbers of strings in memory, and the available open source libraries (we chose “marisa-trie”) make them very simple to use.

The resulting application, including a tiny web API built with the Flask framework, uses only 30 MB of memory to represent the entire UK postcode database, and can comfortably handle a high volume of postcode lookup requests. The code is simple; the service is very lightweight and painless to deploy and run on a free hosting platform such as Heroku, with no external requirements or dependencies on databases. Our implementation is open source, available at <https://github.com/j4mie/postcodeserver>.

—Jamie Matthews, *technical director of DabApps.com (UK)*

DAWG and tries are powerful data structures that can help you save RAM and time in exchange for a little additional effort in preparation. These data structures will be unfamiliar to many developers, so consider separating this code into a module that is reasonably isolated from the rest of your code to simplify maintenance.

Modeling More Text with Scikit-Learn’s FeatureHasher

Scikit-learn is Python’s best-known machine learning framework, and it has excellent support for text-based natural language processing (NLP) challenges. Here, we’ll look at classifying public posts from Usenet archives to one of 20 prespecified categories; this is similar to the two-category spam classification process that cleans up our email inboxes.

One difficulty with text processing is that the vocabulary under analysis quickly explodes. The English language uses many nouns (e.g., the names of people and places, medical labels, and religious terms) and verbs (the “doing words” that often end in “-ing,” like “running,” “taking,” “making,” and “talking”) and their conjugations (turning the verb “talk” into “talked,” “talking,” “talks”), along with all the other rich forms of language. Punctuation and capitalization add an extra nuance to the representation of words.

A powerful and simple technique for classifying text is to break the original text into *n*-grams, often unigrams, bigrams, and trigrams (also known as 1-grams, 2-grams, and 3-grams). A sentence like “there is a cat and a dog” can be turned into unigrams (“there,” “is,” “a,” and so on), bigrams (“there is,” “is a,” “a cat,” etc.), and trigrams (“there is a,” “is a cat,” “a cat and,” ...).

There are 7 unigrams, 6 bigrams, and 5 trigrams for this sentence; in total this sentence can be represented in this form by a vocabulary of 6 unique unigrams (since the term “a” is used twice), 6 unique bigrams, and 5 unique trigrams, making 17 descriptive items in total. As you can see, the *n*-gram vocabulary used to represent a sentence quickly grows; some terms are very common, and some are very rare.

There are techniques to control the explosion of a vocabulary, such as eliminating stop-words (removing the most common and often uninformative terms, like “a,” “the,” and “of”), lowercasing everything, and ignoring less frequent types of terms (such as punctuation, numbers, and brackets). If you practice natural language processing, you’ll quickly come across these approaches.

Introducing DictVectorizer and FeatureHasher

Before we look at the Usenet classification task, let’s look at two of scikit-learn’s feature processing tools that help with NLP challenges. The first is `DictVectorizer`, which takes a dictionary of terms and their frequencies and converts them into a variable-width sparse matrix (we will discuss sparse matrices in “[SciPy’s Sparse Matrices](#)” on page 366). The second is `FeatureHasher`, which converts the same dictionary of terms and frequencies into a fixed-width sparse matrix.

Example 11-22 shows two sentences—“there is a cat” and “there is a cat and a dog”—in which terms are shared between the sentences and the term “a” is used twice in

one of the sentences. `DictVectorizer` is given the sentences in the call to `fit`; in a first pass it builds a list of words into an internal `vocabulary_`, and in a second pass it builds up a sparse matrix containing a reference to each term and its count.

Doing two passes takes longer than the one pass of `FeatureHasher`, and storing a vocabulary costs additional RAM. Building a vocabulary is often a serial process; by avoiding this stage, the feature hashing can potentially operate in parallel for additional speed.

Example 11-22. Lossless text representation with `DictVectorizer`

```
In [2]: from sklearn.feature_extraction import DictVectorizer
...:
...: dv = DictVectorizer()
...: # frequency counts for ["there is a cat", "there is a cat and a dog"]
...: token_dict = [{'there': 1, 'is': 1, 'a': 1, 'cat': 1},
...:                {'there': 1, 'is': 1, 'a': 2, 'cat': 1, 'and': 1, 'dog': 1}]
```

```
In [3]: dv.fit(token_dict)
...:
...: print("Vocabulary:")
...: pprint(dv.vocabulary_)
```

```
Vocabulary:
{'a': 0, 'and': 1, 'cat': 2, 'dog': 3, 'is': 4, 'there': 5}
```

```
In [4]: X = dv.transform(token_dict)
```

To make the output a little clearer, see the Pandas DataFrame view of the matrix `X` in [Figure 11-4](#), where the columns are set to the vocabulary. Note that here we've made a *dense* representation of the matrix—we have 2 rows and 6 columns, and each of the 12 cells contains a number. In the sparse form, we store only the 10 counts that are present and we do not store anything for the 2 items that are missing. With a larger corpus, the larger storage required by a dense representation, containing mostly 0s, quickly becomes prohibitive. For NLP the sparse representation is standard.

	a	and	cat	dog	is	there
0	1	0	1	0	1	1
1	2	1	1	1	1	1

Figure 11-4. Transformed output of `DictVectorizer` shown in a Pandas DataFrame

One feature of `DictVectorizer` is that we can give it a matrix and reverse the process. In [Example 11-23](#), we use the vocabulary to recover the original frequency representation. Note that this does *not* recover the original sentence; there’s more than one way to interpret the ordering of words in the first example (both “there is a cat” and “a cat is there” are valid interpretations). If we used bigrams, we’d start to introduce a constraint on the ordering of words.

Example 11-23. Reversing the output of matrix `X` to the original dictionary representation

```
In [5]: print("Reversing the transform:")
...: pprint(dv.inverse_transform(X))

Reversing the transform:
[{'a': 1, 'cat': 1, 'is': 1, 'there': 1},
 {'a': 2, 'and': 1, 'cat': 1, 'dog': 1, 'is': 1, 'there': 1}]
```

`FeatureHasher` takes the same input and generates a similar output but with one key difference: it does not store a vocabulary and instead employs a hashing algorithm to assign token frequencies to columns.

We’ve already looked at hash functions in [“How Do Dictionaries and Sets Work?” on page 83](#). A hash converts a unique item (in this case a text token) into a number, where multiple unique items might map to the same hashed value, in which case we get a collision. Good hash functions cause few collisions. Collisions are inevitable if we’re hashing many unique items to a smaller representation. One feature of a hash function is that it can’t easily be reversed, so we can’t take a hashed value and convert it back to the original token.

In [Example 11-24](#), we ask for a fixed-width 10-column matrix—the default is a fixed-width matrix of 1 million elements, but we’ll use a tiny matrix here to show a collision. The default 1-million-element width is a sensible default for many applications.

The hashing process uses the fast `MurmurHash3` algorithm, which transforms each token into a number; this is then converted into the range we specify. Larger ranges have few collisions; a small range like our range of 10 will have many collisions. Since every token has to be mapped to one of only 10 columns, we’ll get many collisions if we add a lot of sentences.

The output `X` has 2 rows and 10 columns; each token maps to one column, and we don’t immediately know which column represents each word since hashing functions are one-way, so we can’t map the output back to the input. In this case we can deduce, using `extra_token_dict`, that the tokens `there` and `is` both map to column 8, so we get nine 0s and one count of 2 in column 8.

Example 11-24. Using a 10-column FeatureHasher to show a hash collision

```
In [6]: from sklearn.feature_extraction import FeatureHasher
...:
...: fh = FeatureHasher(n_features=10, alternate_sign=False)
...: fh.fit(token_dict)
...: X = fh.transform(token_dict)
...: pprint(X.toarray().astype(np.int_))
...:
array([[1, 0, 0, 0, 0, 0, 0, 2, 0, 1],
       [2, 0, 0, 1, 0, 1, 0, 2, 0, 1]])

In [7]: extra_token_dict = [{'there': 1, 'is': 1}, ]
...: X = fh.transform(extra_token_dict)
...: print(X.toarray().astype(np.int_))
...:
[[0 0 0 0 0 0 0 2 0 0]]
```

Despite the occurrence of collisions, more than enough signal is often retained in this representation (assuming the default number of columns is used) to enable similar quality machine learning results with FeatureHasher compared to DictVectorizer.

Comparing DictVectorizer and FeatureHasher on a Real Problem

If we take the full 20 Newsgroups dataset, we have 20 categories, with approximately 18,000 emails spread across the categories. While some categories such as “sci.med” are relatively unique, others like “comp.os.ms-windows.misc” and “comp.windows.x” will contain emails that share similar terms. The machine learning task is to correctly identify the correct newsgroup from the 20 options for each item in the test set. The test set has approximately 4,000 emails; the training set used to learn the mapping of terms to the matching category has approximately 14,000 emails.

Note that this example does *not* deal with some of the necessities of a realistic training challenge. We haven’t stripped newsgroup metadata, which can be used to overfit on this challenge; rather than generalize just from the text of the emails, some extraneous metadata artificially boosts the scores. We have randomly shuffled the emails. Here, we’re not trying to achieve a single excellent machine learning result; instead, we’re demonstrating that a lossy hashed representation can be equivalent to a non-lossy and more memory-hungry variant.

In [Example 11-25](#), we take 18,846 documents and build up a training and test set representation using both DictVectorizer and FeatureHasher with unigrams, bigrams, and trigrams. The DictVectorizer sparse array has shape (14,134, 4,335,793) for the training set, where our 14,134 emails are represented using 4 million tokens. Building the vocabulary and transforming the training data takes 42 seconds.

Contrast this with the `FeatureHasher`, which has a fixed 1-million-element-wide hashed representation, and where the transformation takes 21 seconds. Note that in both cases, roughly 9.8 million nonzero items are stored in the sparse matrices, so they're storing similar quantities of information. The hashed version stores approximately 10,000 fewer items because of collisions.

If we'd used a dense matrix, we'd have 14 thousand rows by 10 million columns, making 140,000,000,000 cells of 8 bytes each—significantly more RAM than is typically available in any current machine. Only a tiny fraction of this matrix would be nonzero. Sparse matrices avoid this RAM consumption.

Example 11-25. Comparing DictVectorizer and FeatureHasher on a real machine learning problem

```
Loading 20 newsgroups training data
18846 documents - 35.855MB
```

```
DictVectorizer on frequency dicts
DictVectorizer has shape (14134, 4335793) with 78,872,376 bytes
  and 9,859,047 non-zero items in 42.15 seconds
Vocabulary has 4,335,793 tokens
LogisticRegression score 0.89 in 1179.33 seconds
```

```
FeatureHasher on frequency dicts
FeatureHasher has shape (14134, 1048576) with 78,787,936 bytes
  and 9,848,492 non-zero items in 21.59 seconds
LogisticRegression score 0.89 in 903.35 seconds
```

Critically, the `LogisticRegression` classifier used on `DictVectorizer` takes 30% longer to train with 4 million columns compared to the 1 million columns used by the `FeatureHasher`. Both show a score of 0.89, so for this challenge the results are reasonably equivalent.

Using `FeatureHasher`, we've achieved the same score on the test set, built our training matrix faster, and avoided building and storing a vocabulary, and we've trained faster than using the more common `DictVectorizer` approach. In exchange, we've lost the ability to transform a hashed representation back into the original features for debugging and explanation, and since we often want the ability to diagnose *why* a decision was made, this might be too costly a trade for you to make.

SciPy's Sparse Matrices

In “[Introducing DictVectorizer and FeatureHasher](#)” on page 362, we created a large feature representation using `DictVectorizer`, which uses a sparse matrix in the background. These sparse matrices can be used for general computation as well and are extremely useful when working with sparse data.

A *sparse matrix* is a matrix in which most matrix elements are 0. For these sorts of matrices, there are many ways to encode the nonzero values and then simply say “all other values are zero.” In addition to these memory savings, many algorithms have special ways of dealing with sparse matrices that give additional computational benefits:

```
>>> from scipy import sparse
>>> A_sparse = sparse.random(2048, 2048, 0.05).tocsr()
>>> A_sparse
<2048x2048 sparse matrix of type '<class 'numpy.float64'>'
    with 209715 stored elements in Compressed Sparse Row format>
>>> %timeit A_sparse * A_sparse
150 ms ± 1.71 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)
>>> A_dense = A_sparse.todense()
>>> type(A_dense)
numpy.matrix
>>> %timeit A_dense * A_dense
571 ms ± 14.5 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

The simplest implementation of this is for COO matrices in SciPy, where for each non-zero element we store the value in addition to the location of the value. This means that for each nonzero value, we store three numbers in total. As long as our matrix has at least 66% zero entries, we are reducing the amount of memory needed to represent the data as a sparse matrix as opposed to a standard numpy array. However, COO matrices are generally used only to *construct* sparse matrices and not for doing actual computation (for that, *CSR/CSC* is preferred).

We can see in [Figure 11-5](#) that for low densities, sparse matrices are much faster than their dense counterparts. On top of this, they also use much less memory.

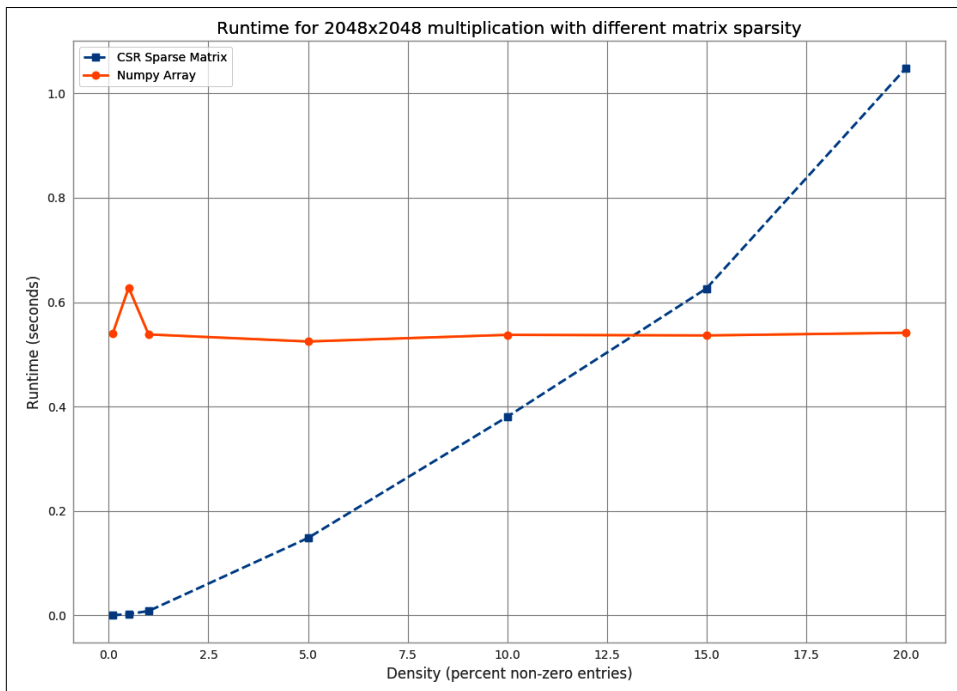


Figure 11-5. Sparse versus dense matrix multiplication

In [Figure 11-6](#), the dense matrix is always using 32.7 MB of memory ($2048 \times 2048 \times 64$ -bit). However, a sparse matrix of 20% density uses only 10 MB, representing a 70% savings! As the density of the sparse matrix goes up, numpy quickly dominates in terms of speed because of the benefits of vectorization and better cache performance.

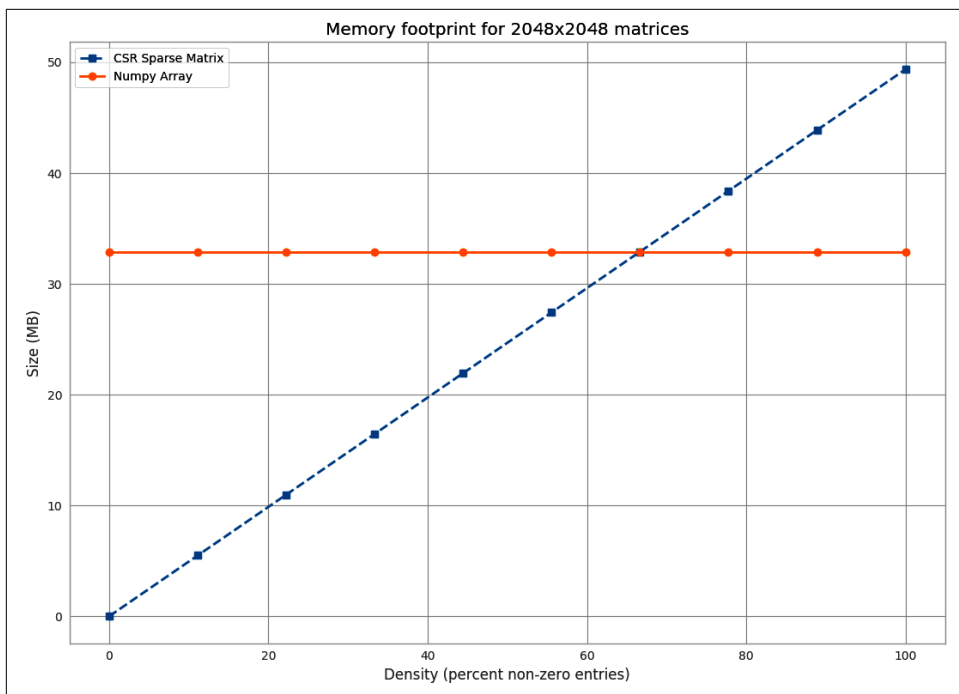


Figure 11-6. Sparse versus dense memory footprint

This extreme reduction in memory use is partly why the speeds are so much better. In addition to running only the multiplication operation for elements that are non-zero (thus reducing the number of operations needed), we also don't need to allocate such a large amount of space to save our result in. This is the push and pull of speed-ups with sparse arrays—it is a balance between losing the use of efficient caching and vectorization versus not having to do a lot of the calculations associated with the zero values of the matrix.

One operation that sparse matrices are particularly good at is cosine similarity. In fact, when creating a `DictVectorizer`, as we did in “[Introducing DictVectorizer and FeatureHasher](#)” on page 362, it's common to use cosine similarity to see how similar two pieces of text are. In general for these item-to-item comparisons (where the value of a particular matrix element is compared to another matrix element), sparse matrices do quite well. Since the calls to `numpy` are the same whether we are using a normal matrix or a sparse matrix, we can benchmark the benefits of using a sparse matrix without changing the code of the algorithm.

While this is impressive, there are severe limitations. The amount of support for sparse matrices is quite low, and unless you are running special sparse algorithms or doing only basic operations, you'll probably hit a wall in terms of support. In

addition, SciPy's `sparse` module offers multiple implementations of sparse matrices, all of which have different benefits and drawbacks. Understanding which is the best one to use and when to use it demands some expert knowledge and often leads to conflicting requirements. As a result, sparse matrices probably aren't something you'll be using often, but when they are the correct tool, they are invaluable.

Tips for Using Less RAM

Generally, if you can avoid putting it into RAM, do. Everything you load costs you RAM. You might be able to load just a part of your data, for example, using a [memory-mapped file](#); alternatively, you might be able to use generators to load only the part of the data that you need for partial computations rather than loading it all at once.

If you are working with numeric data, you'll almost certainly want to switch to using `numpy` arrays—the package offers many fast algorithms that work directly on the underlying primitive objects. The RAM savings compared to using lists of numbers can be huge, and the time savings can be similarly amazing. Furthermore, if you are dealing with very sparse arrays, using SciPy's sparse array functionality can save incredible amounts of memory, albeit with a reduced feature set as compared to normal `numpy` arrays.

If you're working with strings, stick to `str` rather than `bytes` unless you have strong reasons to work at the byte level. Dealing with a myriad set of text encodings is painful by hand, and UTF-8 (or other Unicode formats) tends to make these problems disappear. If you're storing many Unicode objects in a static structure, you probably want to investigate the DAWG and trie structures that we've just discussed.

If you're working with lots of bit strings, investigate `numpy` and the `bitarray` package; both have efficient representations of bits packed into bytes. You might also benefit from looking at Redis, which offers efficient storage of bit patterns.

The PyPy project is experimenting with more efficient representations of homogeneous data structures, so long lists of the same primitive type (e.g., integers) might cost much less in PyPy than the equivalent structures in CPython. The [MicroPython](#) project will be interesting to anyone working with embedded systems: this tiny-memory-footprint implementation of Python is aiming for Python 3 compatibility.

It goes (almost!) without saying that you know you have to benchmark when you're trying to optimize on RAM usage, and that it pays handsomely to have a unit test suite in place before you make algorithmic changes.

Having reviewed ways of compressing strings and storing numbers efficiently, we'll now look at trading accuracy for storage space.

Probabilistic Data Structures

Probabilistic data structures allow you to make trade-offs in accuracy for immense decreases in memory usage. In addition, the number of operations you can do on them is much more restricted than with a set or a trie. For example, with a single HyperLogLog++ structure using 2.56 KB, you can count the number of unique items up to approximately 7,900,000,000 items with 1.625% error.

This means that if we're trying to count the number of unique license plate numbers for cars, and our HyperLogLog++ counter said there were 654,192,028, we would be confident that the actual number is between 643,561,407 and 664,822,648. Furthermore, if this accuracy isn't sufficient, you can simply add more memory to the structure and it will perform better. Giving it 40.96 KB of resources will decrease the error from 1.625% to 0.4%. However, storing this data in a set would take 3.925 GB, even assuming no overhead!

On the other hand, the HyperLogLog++ structure would only be able to count a set of license plates and merge with another set. So, for example, we could have one structure for every state, find how many unique license plates are in each of those states, and then merge them all to get a count for the whole country. If we were given a license plate, we couldn't tell you with very good accuracy whether we've seen it before, and we couldn't give you a sample of license plates we have already seen.

Probabilistic data structures are fantastic when you have taken the time to understand the problem and need to put something into production that can answer a very small set of questions about a very large set of data. Each structure has different questions it can answer at different accuracies, so finding the right one is just a matter of understanding your requirements.



Much of this section goes into a deep dive into the mechanisms that power many of the popular probabilistic data structures. This is useful, because once you understand the mechanisms, you can use parts of them in algorithms you are designing. If you are just beginning with probabilistic data structures, it may be useful to first look at the real-world example ([“Real-World Example” on page 389](#)) before diving into the internals.

In almost all cases, probabilistic data structures work by finding an alternative representation for the data that is more compact and contains the relevant information for answering a certain set of questions. This can be thought of as a type of lossy compression, where we may lose some aspects of the data but we retain the necessary components. Since we are allowing the loss of data that isn't necessarily relevant for the particular set of questions we care about, this sort of lossy compression can be much more efficient than the lossless compression we looked at before with tries. It is

because of this that the choice of which probabilistic data structure you will use is quite important—you want to pick one that retains the right information for your use case!

Before we dive in, it should be made clear that all the “error rates” here are defined in terms of *standard deviations*. This term comes from describing Gaussian distributions and says how spread out the function is around a center value. When the standard deviation grows, so do the number of values further away from the center point. Error rates for probabilistic data structures are framed this way because all the analyses around them are probabilistic. So, for example, when we say that the HyperLogLog++ algorithm has an error of $err = \frac{1.04}{\sqrt{m}}$, we mean that 68% of the time the error will be smaller than err , 95% of the time it will be smaller than $2 \times err$, and 99.7% of the time it will be smaller than $3 \times err$.²

Very Approximate Counting with a 1-Byte Morris Counter

We’ll introduce the topic of probabilistic counting with one of the earliest probabilistic counters, the Morris counter (by Robert Morris of the NSA and Bell Labs). Applications include counting millions of objects in a restricted-RAM environment (e.g., on an embedded computer), understanding large data streams, and working on problems in AI like image and speech recognition.

The Morris counter keeps track of an exponent and models the counted state as 2^{exponent} (rather than a correct count)—it provides an *order of magnitude* estimate. This estimate is updated using a probabilistic rule.

We start with the exponent set to 0. If we ask for the *value* of the counter, we’ll be given $\text{pow}(2, \text{exponent})=1$ (the keen reader will note that this is off by one—we did say this was an *approximate* counter!). If we ask the counter to increment itself, it will generate a random number (using the uniform distribution), and it will test if $\text{random.uniform}(0, 1) \leq 1/\text{pow}(2, \text{exponent})$, which will always be true ($\text{pow}(2, 0) == 1$). The counter increments, and the exponent is set to 1.

The second time we ask the counter to increment itself, it will test if $\text{random.uniform}(0, 1) \leq 1/\text{pow}(2, 1)$. This will be true 50% of the time. If the test passes, the exponent is incremented. If not, the exponent is not incremented for this increment request.

Table 11-1 shows the likelihoods of an increment occurring for each of the first exponents.

² These numbers come from the 68-95-99.7 rule of Gaussian distributions. More information can be found in the [Wikipedia entry](#).

Table 11-1. Morris counter details

Exponent	$\text{pow}(2, \text{exponent})$	$P(\text{increment})$
0	1	1
1	2	0.5
2	4	0.25
3	8	0.125
4	16	0.0625
...	...	...
254	2.894802×10^{76}	3.454467×10^{-77}

The maximum we could approximately count where we use a single unsigned byte for the exponent is $\text{math.pow}(2, 255) \approx 5 \times 10^{76}$. The error relative to the actual count will be fairly large as the counts increase, but the RAM savings is tremendous, as we use only 1 byte rather than the 32 unsigned bytes we'd otherwise have to use.

Example 11-26 shows a simple implementation of the Morris counter.

Example 11-26. Simple Morris counter implementation

```

"""Approximate Morris counter supporting many counters"""
import math
import random
import array

SMALLEST_UNSIGNED_INTEGER = 'B' # unsigned char, typically 1 byte

class MorrisCounter(object):
    """Approximate counter, stores exponent and counts approximately 2^exponent
    https://en.wikipedia.org/wiki/Approximate_counting_algorithm"""
    def __init__(self, type_code=SMALLEST_UNSIGNED_INTEGER, nbr_counters=1):
        self.exponents = array.array(type_code, [0] * nbr_counters)

    def __len__(self):
        return len(self.exponents)

    def add_counter(self):
        """Add a new zeroed counter"""
        self.exponents.append(0)

    def get(self, counter=0):
        """Calculate approximate value represented by counter"""
        return math.pow(2, self.exponents[counter])

    def add(self, counter=0):
        """Probabilistically add 1 to counter"""
        value = self.get(counter)

```

```

        probability = 1.0 / value
        if random.uniform(0, 1) < probability:
            self.exponents[counter] += 1

if __name__ == "__main__":
    mc = MorrisCounter()
    print("MorrisCounter has {} counters".format(len(mc)))
    for n in range(10):
        print("Iteration %d, MorrisCounter has: %d" % (n, mc.get()))
        mc.add()

    for n in range(990):
        mc.add()
    print("Iteration 1000, MorrisCounter has: %d" % (mc.get()))

```

Using this implementation, we can see in [Example 11-27](#) that the first request to increment the counter succeeds, the second succeeds, and the third doesn't.³

Example 11-27. Morris counter library example

```

>>> mc = MorrisCounter()
>>> mc.get()
1.0

>>> mc.add()
>>> mc.get()
2.0

>>> mc.add()
>>> mc.get()
4.0

>>> mc.add()
>>> mc.get()
4.0

```

In [Figure 11-7](#), the thick black line shows a normal integer incrementing on each iteration. On a 64-bit computer, this is an 8-byte integer. The evolution of three 1-byte Morris counters is shown as dotted lines; the y-axis shows their values, which approximately represent the true count for each iteration. Three counters are shown to give you an idea about their different trajectories and the overall trend; the three counters are entirely independent of one another.

³ A more fully fleshed-out implementation that uses an array of bytes to make many counters is available at https://github.com/ianozsvald/morris_counter.

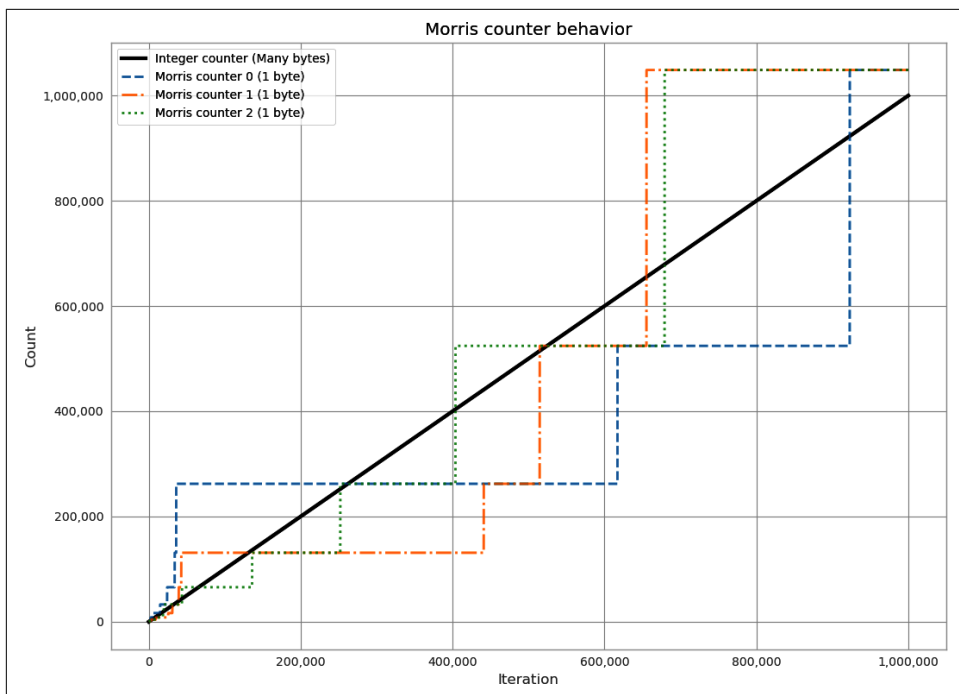


Figure 11-7. Three 1-byte Morris counters versus an 8-byte integer

This diagram gives you some idea about the error to expect when using a Morris counter. Further details about the error behavior are available [online](#).

K-Minimum Values

In the Morris counter, we lose any sort of information about the items we insert. That is to say, the counter's internal state is the same whether we do `.add("micha")` or `.add("ian")`. This extra information is useful and, if used properly, could help us have our counters count only unique items. In this way, calling `.add("micha")` thousands of times would increase the counter only once.

To implement this behavior, we will exploit properties of hashing functions (see [“Hash Functions and Entropy” on page 88](#) for a more in-depth discussion of hash functions). The main property we would like to take advantage of is the fact that the hash function takes input and *uniformly* distributes it. For example, let's assume we have a hash function that takes in a string and outputs a number between 0 and 1. For that function to be uniform means that when we feed it in a string, we are equally likely to get a value of 0.5 as a value of 0.2 or any other value. This also means that if we feed it in many string values, we would expect the values to be relatively evenly spaced. Remember, this is a probabilistic argument: the values won't always be evenly

spaced, but if we have many strings and try this experiment many times, they will tend to be evenly spaced.

Suppose we took 100 items and stored the hashes of those values (the hashes being numbers from 0 to 1). Knowing the spacing is even means that instead of saying, “We have 100 items,” we could say, “We have a distance of 0.01 between every item.” This is where the K-Minimum Values algorithm finally comes in⁴--if we keep the k smallest unique hash values we have seen, we can approximate the overall spacing between hash values and infer the total number of items. In [Figure 11-8](#), we can see the state of a K-Minimum Values structure (also called a KMV) as more and more items are added. At first, since we don’t have many hash values, the largest hash we have kept is quite large. As we add more and more, the largest of the k hash values we have kept gets smaller and smaller. Using this method, we can get error rates of $O\left(\sqrt{\frac{2}{\pi(k-2)}}\right)$.

The larger k is, the more we can account for the hashing function we are using not being completely uniform for our particular input and for unfortunate hash values. An example of unfortunate hash values would be hashing ['A', 'B', 'C'] and getting the values [0.01, 0.02, 0.03]. If we start hashing more and more values, it is less and less probable that they will clump up.

Furthermore, since we are keeping only the smallest *unique* hash values, the data structure considers only unique inputs. We can see this easily because if we are in a state where we store only the smallest three hashes and currently [0.1, 0.2, 0.3] are the smallest hash values, then if we add in something with the hash value of 0.4, our state also will not change. Similarly, if we add more items with a hash value of 0.3, our state will also not change. This is a property called *idempotence*; it means that if we do the same operation, with the same inputs, on this structure multiple times, the state will not be changed. This is in contrast to, for example, an `append` on a `list`, which will always change its value. This concept of idempotence carries on to all of the data structures in this section except for the Morris counter.

[Example 11-28](#) shows a very basic K-Minimum Values implementation. Of note is our use of a `sortedset`, which, like a set, can contain only unique items. This uniqueness gives our `KMinValues` structure idempotence for free. To see this, follow the code through: when the same item is added more than once, the data property does not change.

⁴ Kevin Beyer et al., “On Synopses for Distinct-Value Estimation under Multiset Operations,” in *Proceedings of the 2007 ACM SIGMOD International Conference on Management of Data* (New York: ACM, 2007), 199–210, <https://doi.org/10.1145/1247480.1247504>.

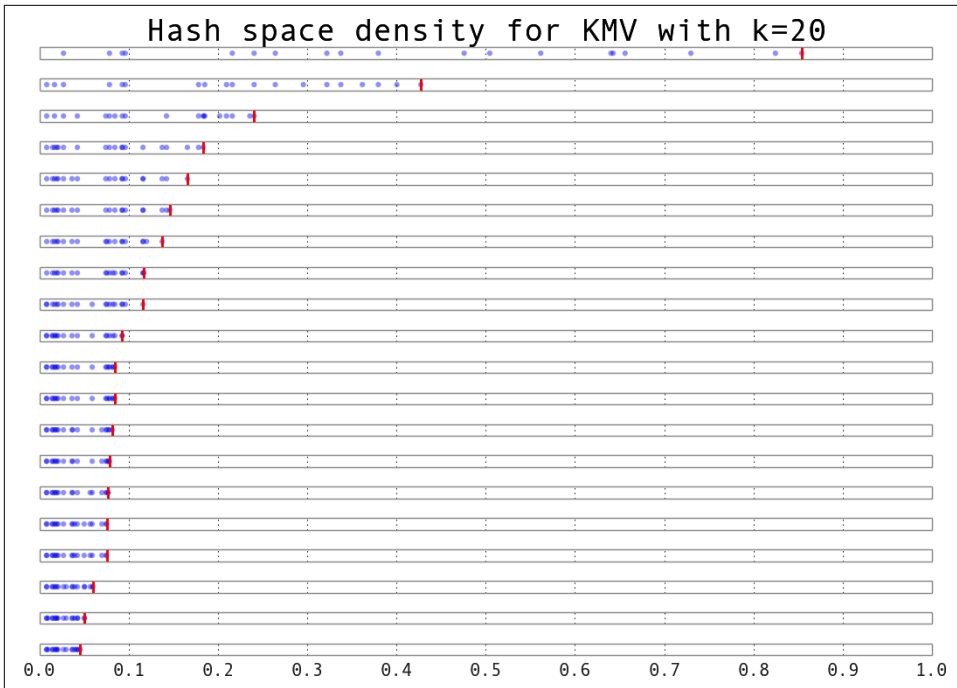


Figure 11-8. The value stores in a KMV structure as more elements are added

Example 11-28. Simple *KMinValues* implementation

```
import mmh3
from blist import sortedset

class KMinValues:
    def __init__(self, num_hashes):
        self.num_hashes = num_hashes
        self.data = sortedset()

    def add(self, item):
        item_hash = mmh3.hash(item)
        self.data.add(item_hash)
        if len(self.data) > self.num_hashes:
            self.data.pop()

    def __len__(self):
        if len(self.data) <= 2:
            return 0
        length = (self.num_hashes - 1) * (2 ** 32 - 1) /
            (self.data[-2] + 2 ** 31 - 1)
        return int(length)
```

Using the `KMinValues` implementation in the Python package `countmemaybe` (Example 11-29), we can begin to see the utility of this data structure. This implementation is very similar to the one in Example 11-28, but it fully implements the other set operations, such as union and intersection. Also note that “size” and “cardinality” are used interchangeably (the word “cardinality” is from set theory and is used more in the analysis of probabilistic data structures). Here, we can see that even with a reasonably small value for `k`, we can store 50,000 items and calculate the cardinality of many set operations with relatively low error.

Example 11-29. `countmemaybe` `KMinValues` implementation

```
>>> from countmemaybe import KMinValues

>>> kmv1 = KMinValues(k=1024)

>>> kmv2 = KMinValues(k=1024)

>>> for i in range(0,50000): ❶
    kmv1.add(str(i))
...:

>>> for i in range(25000, 75000): ❷
    kmv2.add(str(i))
...:

>>> print(len(kmv1))
50416

>>> print(len(kmv2))
52439

>>> print(kmv1.cardinality_intersection(kmv2))
25900.2862992

>>> print(kmv1.cardinality_union(kmv2))
75346.2874158
```

- ❶ We put 50,000 elements into `kmv1`.
- ❷ `kmv2` also gets 50,000 elements, 25,000 of which are also in `kmv1`.



With these sorts of algorithms, the choice of hash function can have a drastic effect on the quality of the estimates. Both of these implementations use `mmh3`, a Python implementation of murmur hash3 that has nice properties for hashing strings. However, different hash functions could be used if they are more convenient for your particular dataset.

Bloom Filters

Sometimes we need to be able to do other types of set operations, for which we need to introduce new types of probabilistic data structures. *Bloom filters* were created to answer the question of whether we've seen an item before.⁵

Bloom filters work by having multiple hash values in order to represent a value as multiple integers. If we later see something with the same set of integers, we can be reasonably confident that it is the same value.

To do this in a way that efficiently utilizes available resources, we implicitly encode the integers as the indices of a list. This could be thought of as a list of `bool` values that are initially set to `False`. If we are asked to add an object with hash values `[10, 4, 7]`, we set the tenth, fourth, and seventh indices of the list to `True`. In the future, if we are asked if we have seen a particular item before, we simply find its hash values and check if all the corresponding spots in the `bool` list are set to `True`.

This method gives us no false negatives and a controllable rate of false positives. If the Bloom filter says we have not seen an item before, we can be 100% sure that we haven't seen the item before. On the other hand, if the Bloom filter states that we *have* seen an item before, there is a probability that we actually have not and we are simply seeing an erroneous result. This erroneous result comes from the fact that we will have hash collisions, and sometimes the hash values for two objects will be the same even if the objects themselves are not the same. However, in practice Bloom filters are set to have error rates below 0.5%, so this error can be acceptable.



We can simulate having as many hash functions as we want simply by having two hash functions that are independent of each other. This method is called *double hashing*. If we have a hash function that gives us two independent hashes, we can do this:

```
def multi_hash(key, num_hashes):
    hash1, hash2 = hashfunction(key)
    for i in range(num_hashes):
        yield (hash1 + i * hash2) % (232 - 1)
```

The modulo ensures that the resulting hash values are 32-bit (we would modulo by $2^{64} - 1$ for 64-bit hash functions).

⁵ Burton H. Bloom, "Space/Time Trade-Offs in Hash Coding with Allowable Errors," *Communications of the ACM* 13, no. 7 (1970): 422–26, <http://doi.org/10.1145/362686.362692>.

The exact length of the `bool` list and the number of hash values per item we need will be fixed based on the capacity and the error rate we require. With some reasonably simple statistical arguments,⁶ we see that the ideal values are as follows:

$$\text{num_bits} = -\text{capacity} \times \frac{\log(\text{error})}{\log(2)^2}$$

$$\text{num_hashes} = \text{num_bits} \times \frac{\log(2)}{\text{capacity}}$$

If we wish to store 50,000 objects (no matter how big the objects themselves are) at a false positive rate of 0.05% (that is to say, 0.05% of the times we say we have seen an object before, we actually have not), it would require 791,015 bits of storage and 11 hash functions.

To further improve our efficiency in terms of memory use, we can use single bits to represent the `bool` values (a native `bool` actually takes 4 bits). We can do this easily by using the `bitarray` module. [Example 11-30](#) shows a simple Bloom filter implementation.

Example 11-30. Simple Bloom filter implementation

```
import math

import bitarray
import mmh3

class BloomFilter:
    def __init__(self, capacity, error=0.005):
        """
        Initialize a Bloom filter with given capacity and false positive rate
        """
        self.capacity = capacity
        self.error = error
        self.num_bits = int((-capacity * math.log(error)) // math.log(2) ** 2 + 1)
        self.num_hashes = int((self.num_bits * math.log(2)) // capacity + 1)
        self.data = bitarray.bitarray(self.num_bits)

    def _indexes(self, key):
        h1, h2 = mmh3.hash64(key)
        for i in range(self.num_hashes):
            yield (h1 + i * h2) % self.num_bits
```

⁶ The [Wikipedia page on Bloom filters](#) has a very simple proof for the properties of a Bloom filter.

```

def add(self, key):
    for index in self._indexes(key):
        self.data[index] = True

def __contains__(self, key):
    return all(self.data[index] for index in self._indexes(key))

def __len__(self):
    bit_off_num = self.data.count(True)
    bit_off_percent = 1.0 - bit_off_num / self.num_bits
    length = -1.0 * self.num_bits * math.log(bit_off_percent) / self.num_hashes
    return int(length)

@staticmethod
def union(bloom_a, bloom_b):
    assert bloom_a.capacity == bloom_b.capacity, "Capacities must be equal"
    assert bloom_a.error == bloom_b.error, "Error rates must be equal"

    bloom_union = BloomFilter(bloom_a.capacity, bloom_a.error)
    bloom_union.data = bloom_a.data | bloom_b.data
    return bloom_union

```

What happens if we insert more items than we specified for the capacity of the Bloom filter? At the extreme end, all the items in the bool list will be set to True, in which case we say that we have seen every item. This means that Bloom filters are very sensitive to what their initial capacity was set to, which can be quite aggravating if we are dealing with a set of data whose size is unknown (for example, a stream of data).

One way of dealing with this is to use a variant of Bloom filters called *scalable* Bloom filters.⁷ They work by chaining together multiple Bloom filters whose error rates vary in a specific way.⁸ By doing this, we can guarantee an overall error rate and add a new Bloom filter when we need more capacity. To check if we've seen an item before, we iterate over all of the sub-Blooms until either we find the object or we exhaust the list. A sample implementation of this structure can be seen in [Example 11-31](#), where we use the previous Bloom filter implementation for the underlying functionality and have a counter to simplify knowing when to add a new Bloom.

7 Paolo Sérgio Almeida et al., “Scalable Bloom Filters,” *Information Processing Letters* 101, no. 6 (2007) 255–61, <https://doi.org/10.1016/j.ipl.2006.10.007>.

8 The error values actually decrease like the geometric series. This way, when you take the product of all the error rates, it approaches the desired error rate.

Example 11-31. Simple scaling Bloom filter implementation

```
from bloomfilter import BloomFilter

class ScalingBloomFilter:
    def __init__(self, capacity, error=0.005, max_fill=0.8,
                  error_tightening_ratio=0.5):
        self.capacity = capacity
        self.base_error = error
        self.max_fill = max_fill
        self.items_until_scale = int(capacity * max_fill)
        self.error_tightening_ratio = error_tightening_ratio
        self.bloom_filters = []
        self.current_bloom = None
        self._add_bloom()

    def _add_bloom(self):
        new_error = self.base_error * self.error_tightening_ratio ** len(
            self.bloom_filters
        )
        new_bloom = BloomFilter(self.capacity, new_error)
        self.bloom_filters.append(new_bloom)
        self.current_bloom = new_bloom
        return new_bloom

    def add(self, key):
        if key in self:
            return True
        self.current_bloom.add(key)
        self.items_until_scale -= 1
        if self.items_until_scale == 0:
            bloom_size = len(self.current_bloom)
            bloom_max_capacity = int(self.current_bloom.capacity * self.max_fill)

            # We may have been adding many duplicate values into the Bloom, so
            # we need to check if we actually need to scale or if we still have
            # space
            if bloom_size >= bloom_max_capacity:
                self._add_bloom()
                self.items_until_scale = bloom_max_capacity
            else:
                self.items_until_scale = int(bloom_max_capacity - bloom_size)
        return False

    def __contains__(self, key):
        return any(key in bloom for bloom in self.bloom_filters)

    def __len__(self):
        return int(sum(len(bloom) for bloom in self.bloom_filters))
```

Another way of dealing with this is using a method called *timing Bloom filters*. This variant allows elements to be expired out of the data structure, thus freeing up space for more elements. This is especially nice for dealing with streams, since we can have elements expire after, say, an hour and have the capacity set large enough to deal with the amount of data we see per hour. Using a Bloom filter this way would give us a nice view into what has been happening in the last hour.

Using this data structure will feel much like using a `set` object. In the following interaction, we use the scalable Bloom filter to add several objects, test if we've seen them before, and then try to experimentally find the false positive rate:

```
>>> bloom = BloomFilter(100)

>>> for i in range(50):
....:     bloom.add(str(i))
....:

>>> "20" in bloom
True

>>> "25" in bloom
True

>>> "51" in bloom
False

>>> num_false_positives = 0

>>> num_true_negatives = 0

>>> # None of the following numbers should be in the Bloom.
>>> # If one is found in the Bloom, it is a false positive.
>>> for i in range(51,10000):
....:     if str(i) in bloom:
....:         num_false_positives += 1
....:     else:
....:         num_true_negatives += 1
....:

>>> num_false_positives
54

>>> num_true_negatives
9895

>>> false_positive_rate = num_false_positives / float(10000 - 51)

>>> false_positive_rate
0.005427681173987335
```

```
>>> bloom.error
0.005
```

We can also do unions with Bloom filters to join multiple sets of items:

```
>>> bloom_a = BloomFilter(200)

>>> bloom_b = BloomFilter(200)

>>> for i in range(50):
...:     bloom_a.add(str(i))
...:

>>> for i in range(25,75):
...:     bloom_b.add(str(i))
...:

>>> bloom = BloomFilter.union(bloom_a, bloom_b)

>>> "51" in bloom_a ❶
Out[9]: False

>>> "24" in bloom_b ❷
Out[10]: False

>>> "55" in bloom ❸
Out[11]: True

>>> "25" in bloom
Out[12]: True
```

- ❶ The value of 51 is not in bloom_a.
- ❷ Similarly, the value of 24 is not in bloom_b.
- ❸ However, the bloom object contains all the objects in both bloom_a and bloom_b!

One caveat is that you can take the union of only two Blooms with the same capacity and error rate. Furthermore, the final Bloom's used capacity can be as high as the sum of the used capacities of the two Blooms unioned to make it. This means that you could start with two Bloom filters that are a little more than half full and, when you union them together, get a new Bloom that is over capacity and not reliable!



A **Cuckoo filter** is a modern Bloom filter–like data structure that provides similar functionality to a Bloom filter with the addition of better object deletion. Furthermore, the Cuckoo filter in most cases has lower overhead, leading to better space efficiencies than the Bloom filter. When a fixed number of objects needs to be kept track of, it is often a better option. However, its performance degrades dramatically when its load limit is reached and there are no options for automatic scaling of the data structure (as we saw with the scaling Bloom filter).

The work of doing fast set inclusion in a memory-efficient way is a very important and active part of database research. Cuckoo filters, **Bloomier filters**, **Xor filters**, and more are being constantly released. However, for most applications, it is still best to stick with the well-known, well-supported Bloom filter.

LogLog Counter

LogLog-type counters are based on the realization that the individual bits of a hash function can also be considered random. That is to say, the probability of the first bit of a hash being 1 is 50%, the probability of the first two bits being 01 is 25%, and the probability of the first three bits being 001 is 12.5%. Knowing these probabilities, and keeping the hash with the most 0s at the beginning (i.e., the least probable hash value), we can come up with an estimate of how many items we’ve seen so far.

A good analogy for this method is flipping coins. Imagine we would like to flip a coin 32 times and get heads every time. The number 32 comes from the fact that we are using 32-bit hash functions. If we flip the coin once and it comes up tails, we will store the number 0, since our best attempt yielded 0 heads in a row. Since we know the probabilities behind this coin flip, we can also tell you that our longest series was 0 long, and you can estimate that we’ve tried this experiment $2^0 = 1$ time. If we keep flipping our coin and we’re able to get 10 heads before getting a tail, then we would store the number 10. Using the same logic, you could estimate that we’ve tried the experiment $2^{10} = 1024$ times. With this system, the highest we could count would be the maximum number of flips we consider (for 32 flips, this is $2^{32} = 4,294,967,296$).

To encode this logic with LogLog-type counters, we take the binary representation of the hash value of our input and see how many 0s there are before we see our first 1. The hash value can be thought of as a series of 32 coin flips, where 0 means a flip for heads and 1 means a flip for tails (i.e., 000010101101 means we flipped four heads before our first tails, and 010101101 means we flipped one head before flipping our first tail). This gives us an idea of how many tries happened before this hash value was reached. The mathematics behind this system is almost equivalent to that of the Morris counter, with one major exception: we acquire the “random” values by

looking at the actual input instead of using a random number generator. This means that if we keep adding the same value to a LogLog counter, its internal state will not change. [Example 11-32](#) shows a simple implementation of a LogLog counter.

Example 11-32. Simple implementation of LogLog register

```
import mmh3

def trailing_zeros(number):
    """
    Returns the index of the first bit set to 1 from the right side of a 32-bit
    integer
    >>> trailing_zeros(0)
    32
    >>> trailing_zeros(0b1000)
    3
    >>> trailing_zeros(0b100000000)
    7
    """
    if not number:
        return 32
    index = 0
    while (number >> index) & 1 == 0:
        index += 1
    return index

class LogLogRegister:
    counter = 0
    def add(self, item):
        item_hash = mmh3.hash(str(item))
        return self._add(item_hash)

    def _add(self, item_hash):
        bit_index = trailing_zeros(item_hash)
        if bit_index > self.counter:
            self.counter = bit_index

    def __len__(self):
        return 2**self.counter
```

The biggest drawback of this method is that we may get a hash value that increases the counter right at the beginning and skews our estimates. This would be similar to flipping 32 tails on the first try. To remedy this, we should have many people flipping coins at the same time and combine their results. The law of large numbers tells us that as we add more and more flippers, the total statistics become less affected by anomalous samples from individual flippers. The exact way that we combine the results is the root of the difference between LogLog-type methods (classic LogLog, SuperLogLog, HyperLogLog, HyperLogLog++, etc.).

We can accomplish this “multiple flipper” method by taking the first couple of bits of a hash value and using that to designate which of our flippers had that particular result. If we take the first 4 bits of the hash, this means we have $2^4 = 16$ flippers. Since we used the first 4 bits for this selection, we have only 28 bits left (corresponding to 28 individual coin flips per coin flipper), meaning each counter can count only up to $2^{28} = 268,435,456$. In addition, there is a constant (alpha) that depends on the number of flippers, which normalizes the estimation.⁹ All of this together gives us an algorithm with $1.05 / \sqrt{m}$ accuracy, where m is the number of registers (or flippers) used. **Example 11-33** shows a simple implementation of the LogLog algorithm.

Example 11-33. Simple implementation of LogLog

```
import mmh3

from llregister import LLRegister

class LL:
    def __init__(self, p):
        self.p = p
        self.num_registers = 2 ** p
        self.registers = [LLRegister() for i in range(int(2 ** p))]
        self.alpha = 0.7213 / (1.0 + 1.079 / self.num_registers)

    def add(self, item):
        item_hash = mmh3.hash(str(item))
        register_index = item_hash & (self.num_registers - 1)
        register_hash = item_hash >> self.p
        self.registers[register_index]._add(register_hash)

    def __len__(self):
        register_sum = sum(h.counter for h in self.registers)
        length = (self.num_registers * self.alpha *
                  2 ** (register_sum / self.num_registers))
        return int(length)
```

In addition to this algorithm deduplicating similar items by using the hash value as an indicator, it has a tunable parameter that can be used to dial whatever sort of accuracy versus storage compromise you are willing to make.

⁹ A full description of the basic LogLog and SuperLogLog algorithms can be found at http://bit.ly/algorithm_desc.

In the `__len__` method, we are averaging the estimates from all of the individual LogLog registers. This, however, is not the most efficient way to combine the data! This is because we may get some unfortunate hash values that make one particular register spike up while the others are still at low values. Because of this, we are able to achieve an error rate of only $O\left(\frac{1.30}{\sqrt{m}}\right)$, where m is the number of registers used.

SuperLogLog was devised as a fix to this problem.¹⁰ With this algorithm, only the lowest 70% of the registers were used for the size estimate, and their value was limited by a maximum value given by a restriction rule. This addition decreased the error rate to $O\left(\frac{1.05}{\sqrt{m}}\right)$. This is counterintuitive, since we got a better estimate by disregarding information!

Finally, HyperLogLog came out in 2007 and gave us further accuracy gains.¹¹ It did so by changing the method of averaging the individual registers: instead of just averaging, we use a spherical averaging scheme that also has special considerations for different edge cases the structure could be in. This brings us to the current best error rate of $O\left(\frac{1.04}{\sqrt{m}}\right)$. In addition, this formulation removes a sorting operation that is necessary with SuperLogLog. This can greatly speed up the performance of the data structure when you are trying to insert items at a high volume. [Example 11-34](#) shows a basic implementation of HyperLogLog.

Example 11-34. Simple implementation of HyperLogLog

```
import math

from ll import LL

class HyperLogLog(LL):
    def __len__(self):
        indicator = sum(2 ** -m.counter for m in self.registers)
        E = self.alpha * (self.num_registers ** 2) / indicator

        if E <= 5.0 / 2.0 * self.num_registers:
            V = sum(1 for m in self.registers if m.counter == 0)
            if V != 0:
                Estar = (self.num_registers *
                        math.log(self.num_registers / (1.0 * V), 2))
```

10 Marianne Durand and Philippe Flajolet, “LogLog Counting of Large Cardinalities,” in *Algorithms—ESA 2003*, ed. Giuseppe Di Battista and Uri Zwick, vol. 2832 (Berlin, Heidelberg: Springer, 2003), 605–17, https://doi.org/10.1007/978-3-540-39658-1_55.

11 Philippe Flajolet et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *AOFA ’07: Proceedings of the 2007 International Conference on Analysis of Algorithms*, (AOFA, 2007), 127–46.

```

        else:
            Estar = E
    else:
        if E <= 2 ** 32 / 30.0:
            Estar = E
        else:
            Estar = -2 ** 32 * math.log(1 - E / 2 ** 32, 2)
    return int(Estar)

if __name__ == "__main__":
    import mmh3

    hll = HyperLogLog(8)
    for i in range(100000):
        hll.add(mmh3.hash(str(i)))
    print(len(hll))

```

The only further increase in accuracy was given by the HyperLogLog++ algorithm, which increases the accuracy of the data structure while it is relatively empty. When more items are inserted, this scheme reverts to standard HyperLogLog. This is actually quite useful, since the statistics of the LogLog-type counters require a lot of data to be accurate—having a scheme for allowing better accuracy with fewer number items greatly improves the usability of this method. This extra accuracy is achieved by having a smaller but more accurate HyperLogLog structure that can later be converted into the larger structure that was originally requested. Also, some empirically derived constants are used in the size estimates that remove biases.

Real-World Example

To obtain a better understanding of the data structures, we first created a dataset with many unique keys, and then one with duplicate entries. Figures 11-9 and 11-10 show the results when we feed these keys into the data structures we’ve just looked at and periodically query, “How many unique entries have there been?” We can see that the data structures that contain more stateful variables (such as HyperLogLog and KMin Values) do better, since they more robustly handle bad statistics. On the other hand, the Morris counter and the single LogLog register can quickly have very high error rates if one unfortunate random number or hash value occurs. For most of the algorithms, however, we know that the number of stateful variables is directly correlated with the error guarantees, so this makes sense.

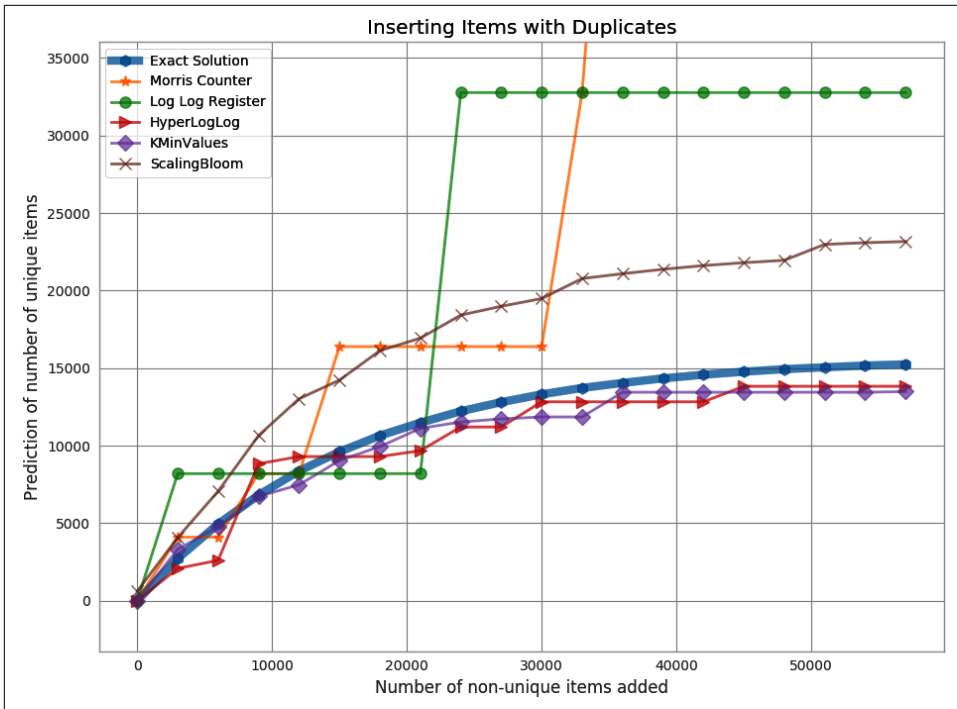


Figure 11-9. The approximate count of duplicate data using multiple probabilistic data structures. To do this, we generate 60,000 items with many duplicates and insert them into the various probabilistic data structures. Graphed is the structures prediction of the number of unique items during the process.

Looking just at the probabilistic data structures that have the best performance (which really are the ones you will probably use), we can summarize their utility and their approximate memory usage (see Table 11-2). We can see a huge change in memory usage depending on the questions we care to ask. This simply highlights the fact that when using a probabilistic data structure, you must first consider what questions you really need to answer about the dataset before proceeding. Also note that only the Bloom filter's size depends on the number of elements. The sizes of the HyperLogLog and KMinValues structures are sizes dependent *only* on the error rate.

As another, more realistic test, we chose to use a dataset derived from the text of Wikipedia. We ran a very simple script in order to extract all single-word tokens with five or more characters from all articles and store them in a newline-separated file. The question then was, "How many unique tokens are there?" The results can be seen in Table 11-3.

Table 11-2. Comparison of major probabilistic data structures and the set operations available on them

	Size	Union ^a	Intersection	Contains	Size ^b
HyperLogLog	Yes ($O\left(\frac{1.04}{\sqrt{m}}\right)$)	Yes	No ^c	No	2.704 MB
KMinValues	Yes ($O\left(\sqrt{\frac{2}{n(m-2)}}\right)$)	Yes	Yes	No	20.372 MB
Bloom filter	Yes ($O\left(\frac{0.78}{\sqrt{m}}\right)$)	Yes	No ^c	Yes	197.8 MB

^a Union operations occur without increasing the error rate.

^b Size of data structure with 0.05% error rate, 100 million unique elements, and using a 64-bit hashing function.

^c These operations *can* be done but at a considerable penalty in terms of accuracy.

The major takeaway from this experiment is that if you are able to specialize your code, you can get amazing speed and memory gains. This has been true throughout the entire book: when we specialized our code in “[Selective Optimizations: Finding What Needs to Be Fixed](#)” on page 137, we were similarly able to get speed increases.

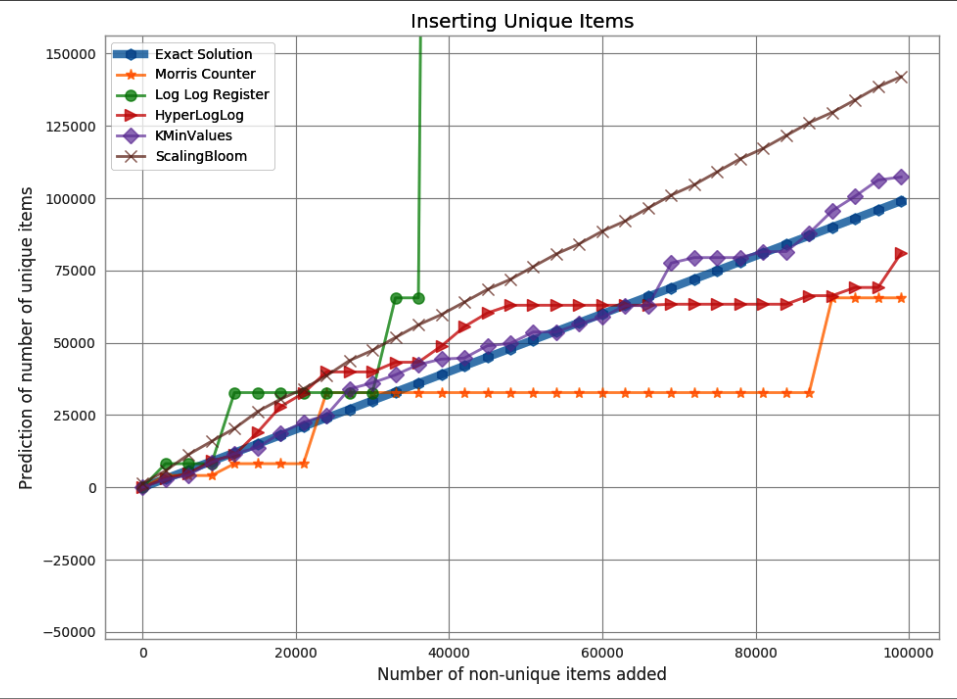


Figure 11-10. The approximate count of unique data using multiple probabilistic data structures. To do this, we insert the numbers 1 through 100,000 into the data structures. Graphed is the structures prediction of the number of unique items during the process.

Probabilistic data structures are an algorithmic way of specializing your code. We store only the data we need in order to answer specific questions with given error bounds. By having to deal with only a subset of the information given, we can not only make the memory footprint much smaller, but also perform most operations over the structure faster.

Table 11-3. Size estimates for the number of unique words in Wikipedia

	Elements	Relative error	Processing time ^a	Structure size ^b
Morris counter ^c	1,073,741,824	6.52%	751s	5 bits
LogLog register	1,048,576	78.84%	1,690s	5 bits
LogLog	4,522,232	8.76%	2,112s	5 bits
HyperLogLog	4,983,171	−0.54%	2,907s	40 KB
KMinValues	4,912,818	0.88%	3,503s	256 KB
Scaling Bloom	4,949,358	0.14%	10,392s	11,509 KB
True value	4,956,262	0.00%	-----	49,558 KB ^d

^a Processing time has been adjusted to remove the time to read the dataset from disk. We also use the simple implementations provided earlier for testing.

^b Structure size is theoretical given the amount of data since the implementations used were not optimized.

^c Since the Morris counter doesn't deduplicate input, the size and relative error are given with regard to the total number of values.

^d The dataset is 49,558 KB considering only unique tokens, or 8.742 GB with all tokens.

As a result, whether or not you use probabilistic data structures, you should always keep in mind what questions you are going to be asking of your data and how you can most effectively store that data in order to ask those specialized questions. This may come down to using one particular type of list over another, using one particular type of database index over another, or maybe even using a probabilistic data structure to throw out all but the relevant data!

HPP Ch 7 | Compiling to C

Compiling to C

Questions You'll Be Able to Answer After This Chapter

- How can I have my Python code run as lower-level code?
- What is the difference between a JIT compiler and an AOT compiler?
- What tasks can compiled Python code perform faster than native Python?
- Why do type annotations speed up compiled Python code?
- How can I write modules for Python using C or Fortran?
- How can I use libraries from C or Fortran in Python?

The easiest way to get your code to run faster is to make it do less work. Assuming you've already chosen good algorithms and you've reduced the amount of data you're processing, the easiest way to execute fewer instructions is to compile your code down to machine code.

Python offers a number of options for this, including pure C-based compiling approaches like Cython; LLVM-based compiling via Numba; and the replacement virtual machine PyPy, which includes a built-in just-in-time (JIT) compiler. You need to balance the requirements of code adaptability and team velocity when deciding which route to take.

Each of these tools adds a new dependency to your toolchain, and Cython requires you to write in a new language type (a hybrid of Python and C), which means you need a new skill. Cython's new language may hurt your team's velocity, as team members without knowledge of C may have trouble supporting this code; in practice,

though, this is probably a minor concern, as you'll use Cython only in well-chosen, small regions of your code.

It is worth noting that performing CPU and memory profiling on your code will probably start you thinking about higher-level algorithmic optimizations that you might apply. These algorithmic changes (such as additional logic to avoid computations or caching to avoid recalculation) could help you avoid doing unnecessary work in your code, and Python's expressivity helps you to spot these algorithmic opportunities. Radim Řehůřek discusses how a Python implementation can beat a pure C implementation in [“Making Deep Learning Fly with RadimRehurek.com \(2014\)” on page 415](#).

In this chapter we'll review the following:

- Cython, the most commonly used tool for compiling to C, covering both `numpy` and normal Python code (requires some knowledge of C)
- Numba, a new compiler specialized for `numpy` code
- PyPy, a stable just-in-time compiler for non-`numpy` code that is a replacement for the normal Python executable

Later in the chapter we'll look at foreign function interfaces, which allow C code to be compiled into extension modules for Python. Python's native API is used with `ctypes` or with `cffi` (from the authors of PyPy), along with the `f2py` Fortran-to-Python converter.

What Sort of Speed Gains Are Possible?

Gains of an order of magnitude or more are quite possible if your problem yields to a compiled approach. Here, we'll look at various ways to achieve speedups of one to two orders of magnitude on a single core, along with using multiple cores through OpenMP.

Python code that tends to run faster after compiling is mathematical, and it has lots of loops that repeat the same operations many times. Inside these loops, you're probably making lots of temporary objects.

Code that calls out to external libraries (such as regular expressions, string operations, and calls to database libraries) is unlikely to show any speedup after compiling. Programs that are I/O-bound are also unlikely to show significant speedups.

Similarly, if your Python code focuses on calling vectorized `numpy` routines, it may not run any faster after compilation—it'll run faster only if the code being compiled is mainly Python (and probably if it is mainly looping). We looked at `numpy`

operations in [Chapter 6](#); compiling doesn't really help because there aren't many intermediate objects.

Overall, it is very unlikely that your compiled code will run any faster than a hand-crafted C routine, but it is also unlikely to run much slower. It is quite possible that the generated C code from your Python will run as fast as a handwritten C routine, unless the C coder has particularly good knowledge of ways to tune the C code to the target machine's architecture.

For math-focused code, it is possible that a handcoded Fortran routine will beat an equivalent C routine, but again, this probably requires expert-level knowledge. Overall, a compiled result (probably using Cython) will be as close to a handcoded-in-C result as most programmers will need.

Keep the diagram in [Figure 7-1](#) in mind when you profile and work on your algorithm. A small amount of work understanding your code through profiling should enable you to make smarter choices at an algorithmic level. After this, some focused work with a compiler should buy you an additional speedup. It will probably be possible to keep tweaking your algorithm, but don't be surprised to see increasingly small improvements coming from increasingly large amounts of work on your part. Know when additional effort isn't useful.

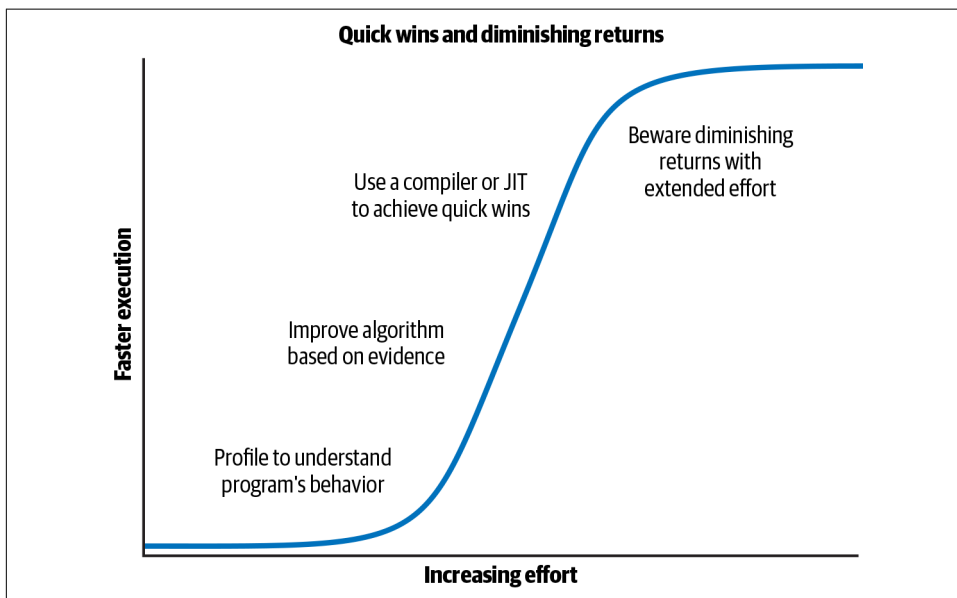


Figure 7-1. Some effort profiling and compiling brings a lot of reward, but continued effort tends to pay increasingly less

If you're dealing with Python code and batteries-included libraries without `numpy`, `Cython` and `PyPy` are your main choices. If you're working with `numpy`, `Cython` and `Numba` are the right choices. These tools all support Python 3.6+.

Some of the following examples require a little understanding of C compilers and C code. If you lack this knowledge, you should learn a little C and compile a working C program before diving in too deeply.

JIT Versus AOT Compilers

The tools we'll look at split roughly into two sets: tools for compiling ahead of time, or AOT (`Cython`), and tools for compiling "just in time," or JIT (`Numba`, `PyPy`).

By compiling AOT, you create a static library that's specialized to your machine. If you download `numpy`, `scipy`, or `scikit-learn`, it will compile parts of the library using `Cython` on your machine (or you'll use a prebuilt compiled library, if you're using a distribution like `Continuum's Anaconda`). By compiling ahead of use, you'll have a library that can instantly be used to work on solving your problem.

By compiling JIT, you don't have to do much (if any) work up front; you let the compiler step in to compile just the right parts of the code at the time of use. This means you have a "cold start" problem—if most of your program could be compiled and currently none of it is, when you start running your code, it'll run very slowly while it compiles. If this happens every time you run a script and you run the script many times, this cost can become significant. `PyPy` suffers from this problem, so you may not want to use it for short but frequently running scripts.

The current state of affairs shows us that compiling ahead of time buys us the best speedups, but often this requires the most manual effort. Just-in-time compiling offers some impressive speedups with very little manual intervention, but it can also run into the problem just described. You'll have to consider these trade-offs when choosing the right technology for your problem.

Why Does Type Information Help the Code Run Faster?

Python is dynamically typed—a variable can refer to an object of any type, and any line of code can change the type of the object that is referred to. This makes it difficult for the virtual machine to optimize how the code is executed at the machine code level, as it doesn't know which fundamental datatype will be used for future operations. Keeping the code generic makes it run more slowly.

In the following example, `v` is either a floating-point number or a pair of floating-point numbers that represent a complex number. Both conditions could occur in the same loop at different points in time, or in related serial sections of code:

```

v = -1.0
print(type(v), abs(v))

<class 'float'> 1.0

v = 1-1j
print(type(v), abs(v))

<class 'complex'> 1.4142135623730951

```

The `abs` function works differently depending on the underlying datatype. Using `abs` for an integer or a floating-point number turns a negative value into a positive value. Using `abs` for a complex number involves taking the square root of the sum of the squared components:

$$abs(c) = \sqrt{c.real^2 + c.imag^2}$$

The machine code for the complex example involves more instructions and will take longer to run. Before calling `abs` on a variable, Python first has to look up the type of the variable and then decide which version of a function to call—this overhead adds up when you make a lot of repeated calls.

Inside Python, every fundamental object, such as an integer, will be wrapped up in a higher-level Python object (e.g., an `int` for an integer). The higher-level object has extra functions like `__hash__` to assist with storage and `__str__` for printing.

Inside a section of code that is CPU-bound, it is often the case that the types of variables do not change. This gives us an opportunity for static compilation and faster code execution.

If all we want are a lot of intermediate mathematical operations, we don't need the higher-level functions, and we may not need the machinery for reference counting either. We can drop down to the machine code level and do our calculations quickly using machine code and bytes, rather than manipulating the higher-level Python objects, which involves greater overhead. To do this, we determine the types of our objects ahead of time so we can generate the correct C code.

Using a C Compiler

In the following examples, we'll use `gcc` and `g++` from the GNU C Compiler toolset. You could use an alternative compiler (e.g., Intel's `icc` or Microsoft's `cl`) if you configure your environment correctly. Cython uses `gcc`.

`gcc` is a very good choice for most platforms; it is well supported and quite advanced. It is often possible to squeeze out more performance by using a tuned compiler (e.g., Intel's `icc` may produce faster code than `gcc` on Intel devices), but the cost is that you

have to gain more domain knowledge and learn how to tune the flags on the alternative compiler.

C and C++ are often used for static compilation rather than other languages like Fortran because of their ubiquity and the wide range of supporting libraries. The compiler and the converter, such as Cython, can study the annotated code to determine whether static optimization steps (like inlining functions and unrolling loops) can be applied.

Aggressive analysis of the intermediate abstract syntax tree (performed by Numba and PyPy) provides opportunities to combine knowledge of Python's way of expressing things to inform the underlying compiler how best to take advantage of the patterns that have been seen.

Reviewing the Julia Set Example

Back in [Chapter 2](#) we profiled the Julia set generator. This code uses integers and complex numbers to produce an output image. The calculation of the image is CPU-bound.

The main cost in the code was the CPU-bound nature of the inner loop that calculates the output list. This list can be drawn as a square pixel array, where each value represents the cost to generate that pixel.

The code for the inner function is shown in [Example 7-1](#).

Example 7-1. Reviewing the Julia function's CPU-bound code

```
def calculate_z_serial_purepython(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and abs(z) < 2:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

On Ian's laptop, the original Julia set calculation on a 1,000 × 1,000 grid with `maxiter=300` takes approximately 8 seconds using a pure Python implementation running on CPython 3.7.

Cython

Cython is a compiler that converts type-annotated Python into a compiled extension module. The type annotations are C-like. This extension can be imported as a regular Python module using `import`. Getting started is simple, but a learning curve must be climbed with each additional level of complexity and optimization. For Ian, this is the tool of choice for turning calculation-bound functions into faster code, because of its wide usage, its maturity, and its OpenMP support.

With the OpenMP standard, it is possible to convert parallel problems into multiprocessing-aware modules that run on multiple CPUs on one machine. The threads are hidden from your Python code; they operate via the generated C code.

Cython (announced in 2007) is a fork of Pyrex (announced in 2002) that expands the capabilities beyond the original aims of Pyrex. Libraries that use Cython include SciPy, scikit-learn, lxml, and ZeroMQ.

Cython can be used via a *setup.py* script to compile a module. It can also be used interactively in IPython via a “magic” command. Typically, the types are annotated by the developer, although some automated annotation is possible.

Compiling a Pure Python Version Using Cython

The easy way to begin writing a compiled extension module involves three files. Using our Julia set as an example, they are as follows:

- The calling Python code (the bulk of our Julia code from earlier)
- The function to be compiled in a new *.pyx* file
- A *setup.py* that contains the instructions for calling Cython to make the extension module

Using this approach, the *setup.py* script is called to use Cython to compile the *.pyx* file into a compiled module. On Unix-like systems, the compiled module will probably be a *.so* file; on Windows it should be a *.pyd* (DLL-like Python library).

For the Julia example, we’ll use the following:

- *julia1.py* to build the input lists and call the calculation function
- *cythonfn.pyx*, which contains the CPU-bound function that we can annotate
- *setup.py*, which contains the build instructions

The result of running *setup.py* is a module that can be imported. In our *julia1.py* script in [Example 7-2](#), we need only to make some tiny changes to `import` the new module and call our function.

Example 7-2. Importing the newly compiled module into our main code

```
...
import cythonfn # as defined in setup.py
...
def calc_pure_python(desired_width, max_iterations):
    # ...
    start_time = time.time()
    output = cythonfn.calculate_z(max_iterations, zs, cs)
    end_time = time.time()
    secs = end_time - start_time
    print(f"Took {secs:0.2f} seconds")
...
```

In [Example 7-3](#), we will start with a pure Python version without type annotations.

Example 7-3. Unmodified pure Python code in `cythonfn.pyx` (renamed from `.py`) for Cython's `setup.py`

```
# cythonfn.pyx
def calculate_z(maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and abs(z) < 2:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

The `setup.py` script shown in [Example 7-4](#) is short; it defines how to convert `cythonfn.pyx` into `calculate.so`.

Example 7-4. `setup.py`, which converts `cythonfn.pyx` into C code for compilation by Cython

```
from distutils.core import setup
from Cython.Build import cythonize

setup(ext_modules=cythonize("cythonfn.pyx",
                             compiler_directives={"language_level": "3"}))
```

When we run the `setup.py` script in [Example 7-5](#) with the argument `build_ext`, Cython will look for `cythonfn.pyx` and build `cythonfn[...].so`. The `language_level` is hardcoded to 3 here to force Python 3.x support.



Remember that this is a manual step—if you update your *.pyx* or *setup.py* and forget to rerun the build command, you won't have an updated *.so* module to import. If you're unsure whether you compiled the code, check the timestamp for the *.so* file. If in doubt, delete the generated C files and the *.so* file and build them again.

Example 7-5. Running *setup.py* to build a new compiled module

```
$ python setup.py build_ext --inplace
Compiling cythonfn.pyx because it changed.
[1/1] Cythonizing cythonfn.pyx
running build_ext
building 'cythonfn' extension
gcc -pthread -B /home/ian/miniconda3/envs/high_performance_python_book_2e/...
gcc -pthread -shared -B /home/ian/miniconda3/envs/high_performance_python_...
```

The `--inplace` argument tells Cython to build the compiled module into the current directory rather than into a separate *build* directory. After the build has completed, we'll have the intermediate *cythonfn.c*, which is rather hard to read, along with *cythonfn[...].so*.

Now when the *julia1.py* code is run, the compiled module is imported, and the Julia set is calculated on Ian's laptop in 4.7 seconds, rather than the more usual 8.3 seconds. This is a useful improvement for very little effort.

pyximport

A simplified build system has been introduced via `pyximport`. If your code has a simple setup and doesn't require third-party modules, you may be able to do away with *setup.py* completely.

By importing `pyximport` as seen in [Example 7-6](#) and calling `install`, any subsequently imported *.pyx* file will be automatically compiled. This *.pyx* file can include annotations, or in this case, it can be the unannotated code. The result runs in 4.7 seconds, as before; the only difference is that we didn't have to write a *setup.py* file.

Example 7-6. Using *pyximport* to replace *setup.py*

```
import pyximport
pyximport.install(language_level=3)
import cythonfn
# followed by the usual code
```


Cython Annotations to Analyze a Block of Code

The preceding example shows that we can quickly build a compiled module. For tight loops and mathematical operations, this alone often leads to a speedup. Obviously, though, we should not optimize blindly—we need to know which lines of code take a lot of time so we can decide where to focus our efforts.

Cython has an annotation option that will output an HTML file we can view in a browser. We use the command `cython -a cythonfn.pyx`, and the output file `cythonfn.html` is generated. Viewed in a browser, it looks something like [Figure 7-2](#). A similar image is available in the [Cython documentation](#).

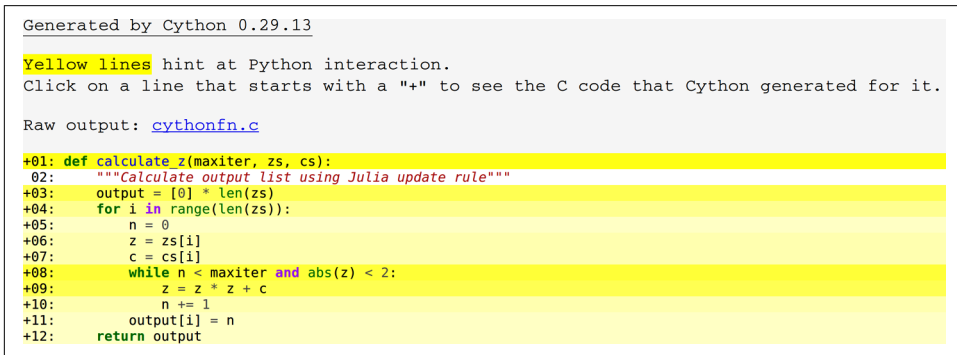


Figure 7-2. Colored Cython output of unannotated function

Each line can be expanded with a double-click to show the generated C code. More yellow means “more calls into the Python virtual machine,” while more white means “more non-Python C code.” The goal is to remove as many of the yellow lines as possible and end up with as much white as possible.

Although “more yellow lines” means more calls into the virtual machine, this won’t necessarily cause your code to run slower. Each call into the virtual machine has a cost, but the cost of those calls will be significant only if the calls occur inside large loops. Calls outside large loops (for example, the line used to create output at the start of the function) are not expensive relative to the cost of the inner calculation loop. Don’t waste your time on the lines that don’t cause a slowdown.

In our example, the lines with the most calls back into the Python virtual machine (the “most yellow”) are lines 4 and 8. From our previous profiling work, we know that line 8 is likely to be called over 30 million times, so that’s a great candidate to focus on.

Lines 9, 10, and 11 are almost as yellow, and we also know they’re inside the tight inner loop. In total they’ll be responsible for the bulk of the execution time of this function, so we need to focus on these first. Refer back to [“Using line_profiler for](#)

Line-by-Line Measurements” on page 40 if you need to remind yourself of how much time is spent in this section.

Lines 6 and 7 are less yellow, and since they’re called only 1 million times, they’ll have a much smaller effect on the final speed, so we can focus on them later. In fact, since they are `list` objects, there’s actually nothing we can do to speed up their access except, as you’ll see in **“Cython and numpy” on page 176**, to replace the `list` objects with `numpy` arrays, which will buy a small speed advantage.

To better understand the yellow regions, you can expand each line. In **Figure 7-3**, we can see that to create the output list, we iterate over the length of `zs`, building new Python objects that are reference-counted by the Python virtual machine. Even though these calls are expensive, they won’t really affect the execution time of this function.

To improve the execution time of our function, we need to start declaring the types of objects that are involved in the expensive inner loops. These loops can then make fewer of the relatively expensive calls back into the Python virtual machine, saving us time.

In general, the lines that probably cost the most CPU time are those:

- Inside tight inner loops
- Dereferencing `list`, `array`, or `np.array` items
- Performing mathematical operations

```
Generated by Cython 0.29.13

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code that Cython generated for it.

Raw output: cythonfn.c

+01: def calculate_z(maxiter, zs, cs):
02:     """Calculate output list using Julia update rule"""
+03:     output = [0] * len(zs)
    __pyx_t_1 = PyObject_Length(__pyx_v_zs); if (unlikely(!__pyx_t_1 == ((Py_ssize_t)-1))) __PYX_ERR(0, 3, __pyx_l1_error)
    __pyx_t_2 = PyList_New(1 * ((__pyx_t_1 < 0) ? 0 : __pyx_t_1)); if (unlikely(!__pyx_t_2)) __PYX_ERR(0, 3, __pyx_l1_error)
    Py_INCREF(__pyx_t_2);
    { Py_ssize_t __pyx_temp;
      for (__pyx_temp=0; __pyx_temp < __pyx_t_1; __pyx_temp++) {
        __Pyx_INCREF(__pyx_int_0);
        __Pyx_GIVEREF(__pyx_int_0);
        PyList_SET_ITEM(__pyx_t_2, __pyx_temp, __pyx_int_0);
      }
    }
    __pyx_v_output = ((PyObject*)__pyx_t_2);
    __pyx_t_2 = 0;
+04:     for i in range(len(zs)):
+05:         n = 0
+06:         z = zs[i]
+07:         c = cs[i]
+08:         while n < maxiter and abs(z) < 2:
+09:             z = z * z + c
+10:             n += 1
+11:         output[i] = n
+12:     return output
```

Figure 7-3. C code behind a line of Python code



If you don't know which lines are most frequently executed, using a profiling tool—`line_profiler`, discussed in “[Using line_profiler for Line-by-Line Measurements](#)” on page 40, would be the most appropriate. You'll learn which lines are executed most frequently and which lines cost the most inside the Python virtual machine, so you'll have clear evidence of which lines you need to focus on to get the best speed gain.

Adding Some Type Annotations

Figure 7-2 shows that almost every line of our function is calling back into the Python virtual machine. All of our numeric work is also calling back into Python as we are using the higher-level Python objects. We need to convert these into local C objects, and then, after doing our numerical coding, we need to convert the result back to a Python object.

In [Example 7-7](#), we see how to add some primitive types by using the `cdef` syntax.



It is important to note that these types will be understood only by Cython and *not* by Python. Cython uses these types to convert the Python code to C objects, which do not have to call back into the Python stack; this means the operations run at a faster speed, but they lose flexibility and development speed.

The types we add are as follows:

- `int` for a signed integer
- `unsigned int` for an integer that can only be positive
- `double complex` for double-precision complex numbers

The `cdef` keyword lets us declare variables inside the function body. These must be declared at the top of the function, as that's a requirement from the C language specification.

Example 7-7. Adding primitive C types to start making our compiled function run faster by doing more work in C and less via the Python virtual machine

```
def calculate_z(int maxiter, zs, cs):  
    """Calculate output list using Julia update rule"""  
    cdef unsigned int i, n  
    cdef double complex z, c  
    output = [0] * len(zs)  
    for i in range(len(zs)):  
        n = 0  
        z = zs[i]
```

```

c = cs[i]
while n < maxiter and abs(z) < 2:
    z = z * z + c
    n += 1
output[i] = n
return output

```



When adding Cython annotations, you're adding non-Python code to the `.pyx` file. This means you lose the interactive nature of developing Python in the interpreter. For those of you familiar with coding in C, we go back to the code-compile-run-debug cycle.

You might wonder if we could add a type annotation to the lists that we pass in. We can use the `list` keyword, but this has no practical effect for this example. The `list` objects still have to be interrogated at the Python level to pull out their contents, and this is very slow.

The act of giving types to some of the primitive objects is reflected in the annotated output in [Figure 7-4](#). Critically, lines 11 and 12—two of our most frequently called lines—have now turned from yellow to white, indicating that they no longer call back to the Python virtual machine. We can anticipate a great speedup compared to the previous version.

```

Generated by Cython 0.29.13

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code that Cython generated for it.

Raw output: cythonfn.c

+01: def calculate_z(int maxiter, zs, cs):
02:     """Calculate output list using Julia update rule"""
03:     cdef unsigned int i, n
04:     cdef double complex z, c
+05:     output = [0] * len(zs)
+06:     for i in range(len(zs)):
07:         n = 0
+08:         z = zs[i]
+09:         c = cs[i]
+10:         while n < maxiter and abs(z) < 2:
11:             z = z * z + c
12:             n += 1
+13:         output[i] = n
+14:     return output

```

Figure 7-4. Our first type annotations

After compiling, this version takes 0.49 seconds to complete. With only a few changes to the function, we are running at 15 times the speed of the original Python version.

It is important to note that the reason we are gaining speed is that more of the frequently performed operations are being pushed down to the C level—in this case, the updates to `z` and `n`. This means that the C compiler can optimize the way the lower-level functions are operating on the bytes that represent these variables, without calling into the relatively slow Python virtual machine.

As noted earlier in this chapter, `abs` for a complex number involves taking the square root of the sum of the squares of the real and imaginary components. In our test, we want to see if the square root of the result is less than 2. Rather than taking the square root, we can instead square the other side of the comparison, so we turn `< 2` into `< 4`. This avoids having to calculate the square root as the final part of the `abs` function.

In essence, we started with

$$\sqrt{c.real^2 + c.imag^2} < \sqrt{4}$$

and we have simplified the operation to

$$c.real^2 + c.imag^2 < 4$$

If we retained the `sqrt` operation in the following code, we would still see an improvement in execution speed. One of the secrets to optimizing code is to make it do as little work as possible. Removing a relatively expensive operation by considering the ultimate aim of a function means that the C compiler can focus on what it is good at, rather than trying to intuit the programmer's ultimate needs.

Writing equivalent but more specialized code to solve the same problem is known as *strength reduction*. You trade worse flexibility (and possibly worse readability) for faster execution.

This mathematical unwinding leads to [Example 7-8](#), in which we have replaced the relatively expensive `abs` function with a simplified line of expanded mathematics.

Example 7-8. Expanding the `abs` function by using Cython

```
def calculate_z(int maxiter, zs, cs):
    """Calculate output list using Julia update rule"""
    cdef unsigned int i, n
    cdef double complex z, c
    output = [0] * len(zs)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

By annotating the code, we see that the `while` on line 10 ([Figure 7-5](#)) has become a little more yellow—it looks as though it might be doing more work rather than less. It

isn't immediately obvious how much of a speed gain we'll get, but we know that this line is called over 30 million times, so we anticipate a good improvement.

This change has a dramatic effect—by reducing the number of Python calls in the innermost loop, we greatly reduce the calculation time of the function. This new version completes in just 0.19 seconds, an amazing 40× speedup over the original version. As ever, take a guide from what you see, but *measure* to test all of your changes!

```
Generated by Cython 0.29.13

Yellow lines hint at Python interaction.
Click on a line that starts with a "+" to see the C code that Cython generated for it.

Raw output: cythonfn.c

+01: def calculate_z(int maxiter, zs, cs):
02:     """Calculate output list using Julia update rule"""
03:     cdef unsigned int i, n
04:     cdef double complex z, c
+05:     output = [0] * len(zs)
+06:     for i in range(len(zs)):
+07:         n = 0
+08:         z = zs[i]
+09:         c = cs[i]
+10:         while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
+11:             z = z * z + c
+12:             n += 1
+13:         output[i] = n
+14:     return output
```

Figure 7-5. Expanded math to get a final win



Cython supports several methods of compiling to C, some easier than the full-type-annotation method described here. You should familiarize yourself with the **pure Python mode** if you'd like an easier start to using Cython, and look at `pyximport` to ease the introduction of Cython to colleagues.

For a final possible improvement on this piece of code, we can disable bounds checking for each dereference in the list. The goal of the bounds checking is to ensure that the program does not access data outside the allocated array—in C it is easy to accidentally access memory outside the bounds of an array, and this will give unexpected results (and probably a segmentation fault!).

By default, Cython protects the developer from accidentally addressing outside the list's limits. This protection costs a little bit of CPU time, but it occurs in the outer loop of our function, so in total it won't account for much time. Disabling bounds checking is usually safe unless you are performing your own calculations for array addressing, in which case you will have to be careful to stay within the bounds of the list.

Cython has a set of flags that can be expressed in various ways. The easiest is to add them as single-line comments at the start of the `.pyx` file. It is also possible to use a decorator or compile-time flag to change these settings. To disable bounds checking, we add a directive for Cython inside a comment at the start of the `.pyx` file:

```
#cython: boundscheck=False
def calculate_z(int maxiter, zs, cs):
```

As noted, disabling the bounds checking will save only a little bit of time as it occurs in the outer loop, not in the inner loop, which is more expensive. For this example, it doesn't save us any more time.



Try disabling bounds checking and wraparound checking if your CPU-bound code is in a loop that is dereferencing items frequently.

Cython and numpy

list objects (for background, see [Chapter 3](#)) have an overhead for each dereference, as the objects they reference can occur anywhere in memory. In contrast, array objects store primitive types in contiguous blocks of RAM, which enables faster addressing.

Python has the array module, which offers 1D storage for basic primitives (including integers, floating-point numbers, characters, and Unicode strings). NumPy's `numpy.array` module allows multidimensional storage and a wider range of primitive types, including complex numbers.

When iterating over an array object in a predictable fashion, the compiler can be instructed to avoid asking Python to calculate the appropriate address and instead to move to the next primitive item in the sequence by going directly to its memory address. Since the data is laid out in a contiguous block, it is trivial to calculate the address of the next item in C by using an offset, rather than asking CPython to calculate the same result, which would involve a slow call back into the virtual machine.

You should note that if you run the following numpy version *without* any Cython annotations (that is, if you just run it as a plain Python script), it'll take about 21 seconds to run—far in excess of the plain Python list version, which takes around 8 seconds. The slowdown is because of the overhead of dereferencing individual elements in the numpy lists—it was never designed to be used this way, even though to a beginner this might feel like the intuitive way of handling operations. By compiling the code, we remove this overhead.

Cython has two special syntax forms for this. Older versions of Cython had a special access type for numpy arrays, but more recently the generalized buffer interface protocol has been introduced through the `memoryview`—this allows the same low-level access to any object that implements the buffer interface, including numpy arrays and Python arrays.

An added bonus of the buffer interface is that blocks of memory can easily be shared with other C libraries, without any need to convert them from Python objects into another form.

The code block in [Example 7-9](#) looks a little like the original implementation, except that we have added `memoryview` annotations. The function’s second argument is `double complex[:] zs`, which means we have a double-precision complex object using the buffer protocol as specified using `[]`, which contains a one-dimensional data block specified by the single colon `:`.

Example 7-9. Annotated numpy version of the Julia calculation function

```
# cythonfn.pyx
import numpy as np
cimport numpy as np

def calculate_z(int maxiter, double complex[:] zs, double complex[:] cs):
    """Calculate output list using Julia update rule"""
    cdef unsigned int i, n
    cdef double complex z, c
    cdef int[:] output = np.empty(len(zs), dtype=np.int32)
    for i in range(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
            z = z * z + c
            n += 1
        output[i] = n
    return output
```

In addition to specifying the input arguments by using the buffer annotation syntax, we also annotate the output variable, assigning a 1D numpy array to it via `empty`. The call to `empty` will allocate a block of memory but will not initialize the memory with sane values, so it could contain anything. We will overwrite the contents of this array in the inner loop so we don’t need to reassign it with a default value. This is slightly faster than allocating and setting the contents of the array with a default value.

We also expanded the call to `abs` by using the faster, more explicit math version. This version runs in 0.18 seconds—a slightly faster result than the original Cythonized version of the pure Python Julia example in [Example 7-8](#). The pure Python version has an overhead every time it dereferences a Python `complex` object, but these dereferences occur in the outer loop and so don’t account for much of the execution time. After the outer loop, we make native versions of these variables, and they operate at “C speed.” The inner loop for both this numpy example and the former pure Python

example are doing the same work on the same data, so the time difference is accounted for by the outer loop dereferences and the creation of the output arrays.

For reference, if we use the preceding code but don't expand the abs math, then the Cythonized result takes 0.49 seconds. This result makes it identical to the earlier equivalent pure Python version's runtime.

Parallelizing the Solution with OpenMP on One Machine

As a final step in the evolution of this version of the code, let's look at the use of the OpenMP C++ extensions to parallelize our embarrassingly parallel problem. If your problem fits this pattern, you can quickly take advantage of multiple cores in your computer.

Open Multi-Processing (OpenMP is a well-defined cross-platform API that supports parallel execution and memory sharing for C, C++, and Fortran. It is built into most modern C compilers, and if the C code is written appropriately, the parallelization occurs at the compiler level, so it comes with relatively little effort to the developer through Cython.

With Cython, OpenMP can be added by using the `prange` (parallel range) operator and adding the `-fopenmp` compiler directive to `setup.py`. Work in a `prange` loop can be performed in parallel because we disable the (GIL). The GIL protects access to Python objects, preventing multiple threads or processes from accessing the same memory simultaneously, which might lead to corruption. By manually disabling the GIL, we're asserting that we won't corrupt our own memory. Be careful when you do this, and keep your code as simple as possible to avoid subtle bugs.

A modified version of the code with `prange` support is shown in [Example 7-10](#). with `nogil`: specifies the block, where the GIL is disabled; inside this block, we use `prange` to enable an OpenMP parallel for loop to independently calculate each `i`.



When disabling the GIL, we must *not* operate on regular Python objects (such as lists); we must operate only on primitive objects and objects that support the `memoryview` interface. If we operated on normal Python objects in parallel, we'd have to solve the associated memory-management problems that the GIL deliberately avoids. Cython doesn't prevent us from manipulating Python objects, and only pain and confusion can result if you do this!

Example 7-10. Adding `prange` to enable parallelization using OpenMP

```
# cythonfn.pyx
from cython.parallel import prange
import numpy as np
cimport numpy as np
```

```

def calculate_z(int maxiter, double complex[:] zs, double complex[:] cs):
    """Calculate output list using Julia update rule"""
    cdef unsigned int i, length
    cdef double complex z, c
    cdef int[:] output = np.empty(len(zs), dtype=np.int32)
    length = len(zs)
    with nogil:
        for i in prange(length, schedule="guided"):
            z = zs[i]
            c = cs[i]
            output[i] = 0
            while output[i] < maxiter and (z.real * z.real + z.imag * z.imag) < 4:
                z = z * z + c
                output[i] += 1
    return output

```

To compile *cythonfn.pyx*, we have to modify the *setup.py* script as shown in [Example 7-11](#). We tell it to inform the C compiler to use `-fopenmp` as an argument during compilation to enable OpenMP and to link with the OpenMP libraries.

Example 7-11. Adding the OpenMP compiler and linker flags to setup.py for Cython

```

#setup.py
from distutils.core import setup
from distutils.extension import Extension
import numpy as np

ext_modules = [Extension("cythonfn",
                        ["cythonfn.pyx"],
                        extra_compile_args=['-fopenmp'],
                        extra_link_args=['-fopenmp'])]

from Cython.Build import cythonize
setup(ext_modules=cythonize(ext_modules,
                           compiler_directives={"language_level": "3"}),
      include_dirs=[np.get_include()])

```

With Cython's `prange`, we can choose different scheduling approaches. With `static`, the workload is distributed evenly across the available CPUs. Some of our calculation regions are expensive in time, and some are cheap. If we ask Cython to schedule the work chunks equally using `static` across the CPUs, the results for some regions will complete faster than others, and those threads will then sit idle.

Both the `dynamic` and `guided` schedule options attempt to mitigate this problem by allocating work in smaller chunks dynamically at runtime, so that the CPUs are more evenly distributed when the workload's calculation time is variable. The correct choice will vary depending on the nature of your workload.

By introducing OpenMP and using `schedule="guided"`, we drop our execution time to approximately 0.05 seconds—the guided schedule will dynamically assign work, so fewer threads will wait for new work.

We also could have disabled the bounds checking for this example by using `#cython: boundscheck=False`, but it wouldn't improve our runtime.

Numba

Numba from Continuum Analytics is a just-in-time compiler that specializes in numpy code, which it compiles via the LLVM compiler (*not* via g++ or gcc++, as used by our earlier examples) at runtime. It doesn't require a precompilation pass, so when you run it against new code, it compiles each annotated function for your hardware. The beauty is that you provide a decorator telling it which functions to focus on and then you let Numba take over. It aims to run on all standard numpy code.

Numba has been rapidly evolving since the first edition of this book. It is now fairly stable, so if you use numpy arrays and have nonvectorized code that iterates over many items, Numba should give you a quick and very painless win. Numba does not bind to external C libraries (which Cython can do), but it can automatically generate code for GPUs (which Cython cannot).

One drawback when using Numba is the toolchain—it uses LLVM, and this has many dependencies. We recommend that you use Continuum's Anaconda distribution, as everything is provided; otherwise, getting Numba installed in a fresh environment can be a very time-consuming task.

Example 7-12 shows the addition of the `@jit` decorator to our core Julia function. This is all that's required; the fact that numba has been imported means that the LLVM machinery kicks in at execution time to compile this function behind the scenes.

Example 7-12. Applying the @jit decorator to a function

```
from numba import jit
...
@jit()
def calculate_z_serial_purepython(maxiter, zs, cs, output):
```

If the `@jit` decorator is removed, this is just the numpy version of the Julia demo running with Python 3.7, and it takes 21 seconds. Adding the `@jit` decorator drops the execution time to 0.75 seconds. This is very close to the result we achieved with Cython, but without all of the annotation effort.

If we run the same function a second time in the same Python session, it runs even faster at 0.47 seconds—there’s no need to compile the target function on the second pass if the argument types are the same, so the overall execution speed is faster. On the second run, the Numba result is equivalent to the Cython with numpy result we obtained before (so it came out as fast as Cython for very little work!). PyPy has the same warm-up requirement.

If you’d like to read another view on what Numba offers, see “Numba” on page 403, where core developer Valentin Haenel talks about the `@jit` decorator, viewing the original Python source, and going further with parallel options and the typed `List` and typed `Dict` for pure Python compiled interoperability.

Just as with Cython, we can add OpenMP parallelization support with `prange`. **Example 7-13** expands the decorator to require `nopython` and `parallel`. The `nopython` specifier means that if Numba cannot compile all of the code, it will fail. Without this, Numba can silently fall back on a Python mode that is slower; your code will run correctly, but you won’t see any speedups. Adding `parallel` enables support for `prange`. This version drops the general runtime from 0.47 seconds to 0.06 seconds. Currently Numba lacks support for OpenMP scheduling options (and with Cython, the guided scheduler runs slightly faster for this problem), but we expect support will be added in a future version.

Example 7-13. Using `prange` to add parallelization

```
@jit(nopython=False, parallel=True)
def calculate_z(maxiter, zs, cs, output):
    """Calculate output list using Julia update rule"""
    for i in prange(len(zs)):
        n = 0
        z = zs[i]
        c = cs[i]
        while n < maxiter and (z.real*z.real + z.imag*z.imag) < 4:
            z = z * z + c
            n += 1
        output[i] = n
```

When debugging with Numba, it is useful to note that you can ask Numba to show both the intermediate representation and the types for the function call. In **Example 7-14**, we can see that `calculate_z` takes an `int64` and three array types.

Example 7-14. Debugging inferred types

```
print(calculate_z.inspect_types())
# calculate_z (int64, array(complex128, 1d, C),
              array(complex128, 1d, C), array(int32, 1d, C))
```

Example 7-15 shows the continued output from the call to `inspect_types()`, where each line of compiled code is shown augmented by type information. This output is invaluable if you find that you can't get `nopython=True` to work; here, you'll be able to discover where your code isn't recognized by Numba.

Example 7-15. Viewing the intermediate representation from Numba

```
...
def calculate_z(maxiter, zs, cs, output):

    # --- LINE 14 ---

    """Calculate output list using Julia update rule"""

    # --- LINE 15 ---
    # maxiter = arg(0, name=maxiter) :: int64
    # zs = arg(1, name=zs) :: array(complex128, 1d, C)
    # cs = arg(2, name=cs) :: array(complex128, 1d, C)
    # output = arg(3, name=output) :: array(int32, 1d, C)
    # jump 2
    # label 2
    # $2.1 = global(range: <class 'range'>) :: Function(<class 'range'>)
    ...
```

Numba is a powerful JIT compiler that is now maturing. Do not expect magic on your first attempt—you may have to introspect the generated code to figure out how to make your code compile in `nopython` mode. Once you've solved this, you'll likely see good wins. Your best approach will be to break your current code into small (<10 line) and discrete functions and to tackle these one at a time. Do not try to throw a large function into Numba; you can debug the process far more quickly if you have only small, discrete chunks to review individually.

Numba to Compile NumPy for Pandas

In “**Pandas**” on page 146, we looked at solving the slope calculation task for 100,000 rows of data in a Pandas DataFrame using Ordinary Least Squares. We can make that approach an order of magnitude faster by using Numba.

We can take the `ols_lstsq_raw` function that we used before and, decorated with `numba.jit` as shown in **Example 7-16**, can generate a compiled variant. Note the `nopython=True` argument—this forces Numba to raise an exception if we pass in a datatype that it doesn't understand, where it would otherwise fall back to a pure Python mode silently. We don't want it to run correctly but slowly on the wrong datatype if we pass in a Pandas Series; here we want to be informed that we've passed in the wrong data. In this edition, Numba can compile only NumPy datatypes, not Pandas types like Series.

Example 7-16. Solving Ordinary Least Squares with *numpy* on a *Pandas* *DataFrame*

```
def ols_lstsq_raw(row):  
    """Variant of `ols_lstsq` where row is a numpy array (not a Series)"""  
    X = np.arange(row.shape[0])  
    ones = np.ones(row.shape[0])  
    A = np.vstack((X, ones)).T  
    m, c = np.linalg.lstsq(A, row, rcond=-1)[0]  
    return m
```

```
# generate a Numba compiled variant
```

```
ols_lstsq_raw_values_numba = jit(ols_lstsq_raw, nopython=True)
```

```
results = df.apply(ols_lstsq_raw_values_numba, axis=1, raw=True)
```

The first time we call this function in, we get the expected short delay while the function is compiled; processing 100,000 rows takes 2.3 seconds including compilation time. Subsequent calls to process 100,000 rows are very fast—the noncompiled `ols_lstsq_raw` takes 5.3 seconds per 100,000 rows, whereas after using Numba it takes 0.58 seconds. That’s nearly a tenfold speedup!

PyPy

PyPy is an alternative implementation of the Python language that includes a tracing just-in-time compiler; it is compatible with Python 3.5+. Typically, it lags behind the most recent version of Python; at the time of writing this second edition Python 3.7 is standard, and PyPy supports up to Python 3.6.

PyPy is a drop-in replacement for CPython and offers all the built-in modules. The project comprises the RPython Translation Toolchain, which is used to build PyPy (and could be used to build other interpreters). The JIT compiler in PyPy is very effective, and good speedups can be seen with little or no work on your part. See “**PyPy for Successful Web and Data Processing Systems (2014)**” on page 426 for a large PyPy deployment success story.

PyPy runs our pure Python Julia demo without any modifications. With CPython it takes 8 seconds, and with PyPy it takes 0.9 seconds. This means that PyPy achieves a result that’s very close to the Cython example in **Example 7-8**, without *any effort at all*—that’s pretty impressive! As we observed in our discussion of Numba, if the calculations are run again *in the same session*, then the second and subsequent runs are faster than the first one, as they are already compiled.

By expanding the math and removing the call to `abs`, the PyPy runtime drops to 0.2 seconds. This is equivalent to the Cython versions using pure Python and *numpy* without any work! Note that this result is true only if you’re not using *numpy* with PyPy.

The fact that PyPy supports all the built-in modules is interesting—this means that multiprocessing works as it does in CPython. If you have a problem that runs with the batteries-included modules and can run in parallel with multiprocessing, you can expect that all the speed gains you might hope to get will be available.

PyPy's speed has evolved over time. The older chart in [Figure 7-6](#) (from speed.pypy.org) will give you an idea about PyPy's maturity. These speed tests reflect a wide range of use cases, not just mathematical operations. It is clear that PyPy offers a faster experience than CPython.

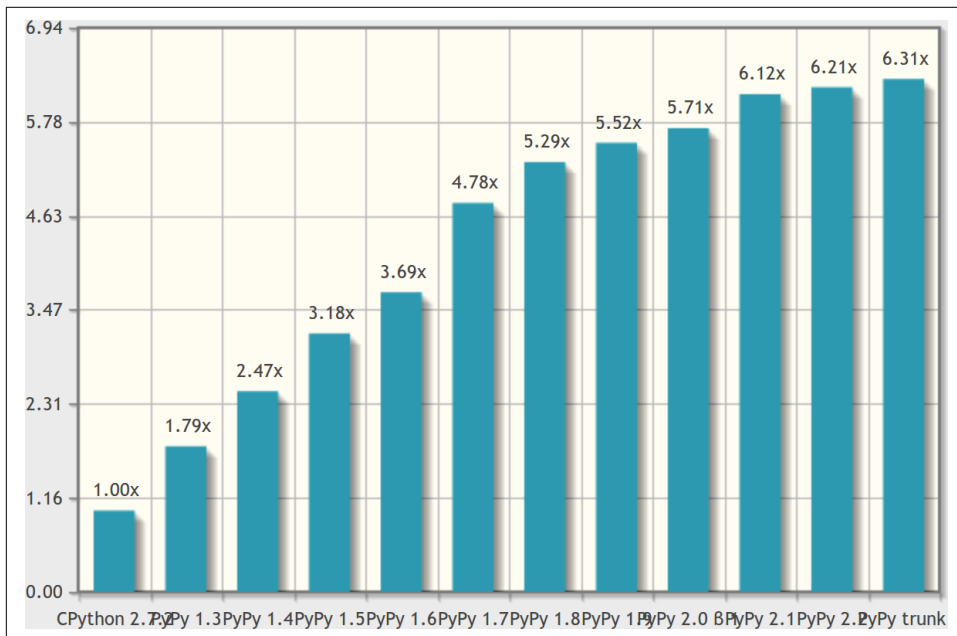


Figure 7-6. Each new version of PyPy offers speed improvements

Garbage Collection Differences

PyPy uses a different type of garbage collector than CPython, and this can cause some nonobvious behavior changes to your code. Whereas CPython uses reference counting, PyPy uses a modified mark-and-sweep approach that may clean up an unused object much later. Both are correct implementations of the Python specification; you just have to be aware that code modifications might be required when swapping.

Some coding approaches seen in CPython depend on the behavior of the reference counter—particularly the flushing of files, if you open and write to them without an explicit file close. With PyPy the same code will run, but the updates to the file might get flushed to disk later, when the garbage collector next runs. An alternative form that works in both PyPy and Python is to use a context manager using `with` to open

and automatically close files. The [Differences Between PyPy and CPython](#) page on the PyPy website lists the details.

Running PyPy and Installing Modules

If you've never run an alternative Python interpreter, you might benefit from a short example. Assuming you've downloaded and extracted PyPy, you'll now have a folder structure containing a *bin* directory. Run it as shown in [Example 7-17](#) to start PyPy.

Example 7-17. Running PyPy to see that it implements Python 3.6

```
...
$ pypy3
Python 3.6.1 (784b254d6699, Apr 14 2019, 10:22:42)
[PyPy 7.1.1-beta0 with GCC 6.2.0 20160901] on linux
Type "help", "copyright", "credits", or "license" for more information.
And now for something completely different
...
```

Note that PyPy 7.1 runs as Python 3.6. Now we need to set up `pip`, and we'll want to install IPython. The steps shown in [Example 7-18](#) are the same as you might have performed with CPython if you've installed `pip` without the help of an existing distribution or package manager. Note that when running IPython, we get the same build number as we see when running `pypy3` in the preceding example.

You can see that IPython runs with PyPy just the same as with CPython, and using the `%run` syntax, we execute the Julia script inside IPython to achieve 0.2-second runtimes.

Example 7-18. Installing `pip` for PyPy to install third-party modules like IPython

```
...
$ pypy3 -m ensurepip
Collecting setuptools
Collecting pip
Installing collected packages: setuptools, pip
Successfully installed pip-9.0.1 setuptools-28.8.0

$ pip3 install ipython
Collecting ipython

$ ipython
Python 3.6.1 (784b254d6699, Apr 14 2019, 10:22:42)
Type 'copyright', 'credits', or 'license' for more information
IPython 7.8.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: %run julia1_nopil_expanded_math.py
Length of x: 1000
```



```
Total elements: 1000000
calculate_z_serial_purepython took 0.2143106460571289 seconds
Length of x: 1000
Total elements: 1000000
calculate_z_serial_purepython took 0.1965022087097168 seconds
...
```

Note that PyPy supports projects like `numpy` that require C bindings through the CPython extension compatibility layer `cpyext`, but it has an overhead of 4–6×, which generally makes `numpy` too slow. If your code is mostly pure Python with only a few calls to `numpy`, you may still see significant overall gains. If your code, like the Julia example, makes many calls to `numpy`, then it'll run significantly slower. The Julia benchmark here with `numpy` arrays runs 6× slower than when it is run with CPython.

If you need other packages, they should install thanks to the `cpyext` compatibility module, which is essentially PyPy's version of `python.h`. It handles the different memory management requirements of PyPy and CPython; however, this management incurs a cost of 4–6× per managed call, so the speed advantages of `numpy` can be negated by this overhead. A new project named `HPy` (formerly `PyHandle`) aims to remove this overhead by providing a higher-level object handle—one not tied to CPython's implementation—which can be shared with other projects like `Cython`.

If you want to understand PyPy's performance characteristics, look at the `vmprof` lightweight sampling profiler. It is thread-safe and supports a web-based user interface.

Another downside of PyPy is that it can use a lot of RAM. Each release is better in this respect, but in practice it may use more RAM than CPython. RAM is fairly cheap, though, so it makes sense to try to trade it for enhanced performance. Some users have also reported *lower* RAM usage when using PyPy. As ever, perform an experiment using representative data if this is important to you.

A Summary of Speed Improvements

To summarize the previous results, in [Table 7-1](#) we see that PyPy on a pure Python math-based code sample is approximately 9× faster than CPython with no code changes, and it's even faster if the `abs` line is simplified. `Cython` runs faster than PyPy in both instances but requires annotated code, which increases development and support effort.

Table 7-1. Julia (no numpy) results

	Speed
CPython	8.00s
Cython	0.49s
Cython on expanded math	0.19s
PyPy	0.90s
PyPy on expanded math	0.20s

The Julia solver with `numpy` enables the investigation of OpenMP. In [Table 7-2](#), we see that both Cython and Numba run faster than the non-`numpy` versions with expanded math. When we add OpenMP, both Cython and Numba provide further speedups for very little additional coding.

Table 7-2. Julia (with numpy and expanded math) results

	Speed
CPython	21.00s
Cython	0.18s
Cython and OpenMP “guided”	0.05s
Numba (2nd & subsequent runs)	0.17s
Numba and OpenMP	0.06s

For pure Python code, PyPy is an obvious first choice. For `numpy` code, Numba is a great first choice.

When to Use Each Technology

If you’re working on a numeric project, then each of these technologies could be useful to you. [Table 7-3](#) summarizes the main options.

Table 7-3. Compiler options

	Cython	Numba	PyPy
Mature	Y	Y	Y
Widespread	Y	—	—
<code>numpy</code> support	Y	Y	Y
Nonbreaking code changes	—	Y	Y
Needs C knowledge	Y	—	—
Supports OpenMP	Y	Y	—

Numba may offer quick wins for little effort, but it too has limitations that might stop it working well on your code. It is also a relatively young project.

Cython probably offers the best results for the widest set of problems, but it does require more effort and has an additional “support tax” due to mixing Python with C annotations.

PyPy is a strong option if you’re not using `numpy` or other hard-to-port C extensions.

If you’re deploying a production tool, you probably want to stick with well-understood tools—Cython should be your main choice, and you may want to check out “[Making Deep Learning Fly with Radim Rehurek.com \(2014\)](#)” on page 415. PyPy is also being used in production settings (see “[PyPy for Successful Web and Data Processing Systems \(2014\)](#)” on page 426).

If you’re working with light numeric requirements, note that Cython’s buffer interface accepts `array.array` matrices—this is an easy way to pass a block of data to Cython for fast numeric processing without having to add `numpy` as a project dependency.

Overall, Numba is maturing and is a promising project, whereas Cython is mature. PyPy is regarded as being fairly mature now and should definitely be evaluated for long-running processes.

In a class run by Ian, a capable student implemented a C version of the Julia algorithm and was disappointed to see it execute more slowly than his Cython version. It transpired that he was using 32-bit floats on a 64-bit machine—these run more slowly than 64-bit doubles on a 64-bit machine. The student, despite being a good C programmer, didn’t know that this could involve a speed cost. He changed his code, and the C version, despite being significantly shorter than the autogenerated Cython version, ran at roughly the same speed. The act of writing the raw C version, comparing its speed, and figuring out how to fix it took longer than using Cython in the first place.

This is just an anecdote; we’re not suggesting that Cython will generate the best code, and competent C programmers can probably figure out how to make *their* code run faster than the version generated by Cython. It is worth noting, though, that the assumption that handwritten C will be faster than converted Python is not a safe assumption. You must always benchmark and make decisions using evidence. C compilers are pretty good at converting code into fairly efficient machine code, and Python is pretty good at letting you express your problem in an easy-to-understand language—combine these two powers sensibly.

Other Upcoming Projects

The [PyData compilers page](#) lists a set of high performance and compiler tools.

Pythran is an AOT compiler aimed at scientists who are using `numpy`. Using few annotations, it will compile Python numeric code to a faster binary—it produces speedups that are very similar to Cython but for much less work. Among other features, it always releases the GIL and can use both SIMD instructions and OpenMP. Like Numba, it doesn't support classes. If you have tight, locally bound loops in `NumPy`, Pythran is certainly worth evaluating. The associated FluidPython project aims to make Pythran even easier to write and provides JIT capability.

Transonic attempts to unify Cython, Pythran, and Numba, and potentially other compilers, behind one interface to enable quick evaluation of multiple compilers without having to rewrite code.

ShedSkin is an AOT compiler aimed at nonscientific, pure Python code. It has no `numpy` support, but if your code is pure Python, ShedSkin produces speedups similar to those seen by PyPy (without using `numpy`). It supports Python 2.7 with some Python 3.x support.

PyCUDA and **PyOpenCL** offer CUDA and OpenCL bindings into Python for direct access to GPUs. Both libraries are mature and support Python 3.4+.

Nuitka is a Python compiler that aims to be an alternative to the usual CPython interpreter, with the option of creating compiled executables. It supports all of Python 3.7, though in our testing it didn't produce any noticeable speed gains for our plain Python numerical tests.

Our community is rather blessed with a wide array of compilation options. While they all have trade-offs, they also offer a lot of power so that complex projects can take advantage of the full power of CPUs and multicore architectures.

Graphics Processing Units (GPUs)

Graphics Processing Units (GPUs) are becoming incredibly popular as a method to speed up arithmetic-heavy computational workloads. Originally designed to help handle the heavy linear algebra requirements of 3D graphics, GPUs are particularly well suited for solving easily parallelizable problems.

Interestingly, GPUs themselves are slower than most CPUs if we just look at clock speeds. This may seem counterintuitive, but as we discussed in “**Computing Units**” on page 2, clock speed is just one measurement of hardware's ability to compute. GPUs excel at massively parallelize tasks because of the staggering number of compute cores they have. CPUs generally have on the order of 12 cores, while modern-day GPUs have thousands. For example, on the machine used to run bench-

marks for this section, the AMD Ryzen 7 1700 CPU has 8 cores, each at 3.2 GHz, while the NVIDIA RTX 2080 TI GPU has 4,352 cores, each at 1.35 GHz.¹

This incredible amount of parallelism can speed up many numerical tasks by a staggering amount. However, programming on these devices can be quite tough. Because of the amount of parallelism, data locality needs to be considered and can be make-or-break in terms of getting speedups. There are many tools out there to write native GPU code (also called kernels) in Python, such as **CuPy**. However, the needs of modern deep learning algorithms have been pushing new interfaces into GPUs that are easy and intuitive to use.

The two front-runners in terms of easy-to-use GPU mathematics libraries are TensorFlow and PyTorch. We will focus on PyTorch for its ease of use and great speed.²

Dynamic Graphs: PyTorch

PyTorch is a static computational graph tensor library that is particularly user-friendly and has a very intuitive API for anyone familiar with **numpy**. In addition, since it is a tensor library, it has all the same functionality as **numpy**, with the added advantages of being able to create functions through its static computational graph and calculate derivatives of those functions by using a mechanism called **autograd**.



The **autograd** functionality of PyTorch is left out since it isn't relevant to our discussion. However, this module is quite amazing and can take the derivative of any arbitrary function made up of PyTorch operations. It can do it on the fly and at any value. For a long time, taking derivatives of complex functions could have been the makings of a PhD thesis; however, now we can do it incredibly simply and efficiently. While this may also be off-topic for your work, we recommend learning about **autograd** and automatic-differentiation in general, as it truly is an incredible advancement in numerical computation.

By *static computational graph*, we mean that performing operations on PyTorch objects creates a dynamic definition of a program that gets compiled to GPU code in the background when it is executed (exactly like a JIT from “**JIT Versus AOT Compilers**” on page 164). Since it is dynamic, changes to the Python code automatically

1 The RTX 2080 TI also includes 544 tensor cores specifically created to help with mathematical operations that are particularly useful for deep learning.

2 For comparisons of the performance of TensorFlow and PyTorch, see <https://oreil.ly/8NOJW> and <https://oreil.ly/4BKM5>.

get reflected in changes in the GPU code without an explicit compilation step needed. This hugely aids debugging and interactivity, as compared to static graph libraries like TensorFlow.

In a static graph, like TensorFlow, we first set up our computation, then compile it. From then on, our compute is fixed in stone, and we can change it only by recompiling the entire thing. With the dynamic graph of PyTorch, we can conditionally change our compute graph or build it up iteratively. This allows us to have conditional debugging in our code or lets us play with the GPU in an interactive session in IPython. The ability to flexibly control the GPU is a complete game changer when dealing with complex GPU-based workloads.

As an example of the library's ease of use as well as its speed, in [Example 7-19](#) we port the numpy code from [Example 6-9](#) to use the GPU using PyTorch.

Example 7-19. PyTorch 2D diffusion

```
import torch
from torch import (roll, zeros) ❶

grid_shape = (640, 640)

def laplacian(grid):
    return (
        roll(grid, +1, 0)
        + roll(grid, -1, 0)
        + roll(grid, +1, 1)
        + roll(grid, -1, 1)
        - 4 * grid
    )

def evolve(grid, dt, D=1):
    return grid + dt * D * laplacian(grid)

def run_experiment(num_iterations):
    grid = zeros(grid_shape)

    block_low = int(grid_shape[0] * 0.4)
    block_high = int(grid_shape[0] * 0.5)
    grid[block_low:block_high, block_low:block_high] = 0.005

    grid = grid.cuda() ❷
    for i in range(num_iterations):
        grid = evolve(grid, 0.1)
    return grid
```

❶, ❷ The only changes needed.

Most of the work is done in the modified import, where we changed `numpy` to `torch`. In fact, if we just wanted to run our optimized code on the CPU, we could stop here.³ To use the GPU, we simply need to move our data to the GPU, and then `torch` will automatically compile all computations we do on that data into GPU code.

As we can see in [Figure 7-7](#), this small code change has given us an incredible speedup.⁴ For a 512×512 grid, we have a $5.3\times$ speedup, and for a $4,096 \times 4,096$ grid we have a $102\times$ speedup! It is interesting that the GPU code doesn't seem to be as affected by increases to grid size as the `numpy` code is.

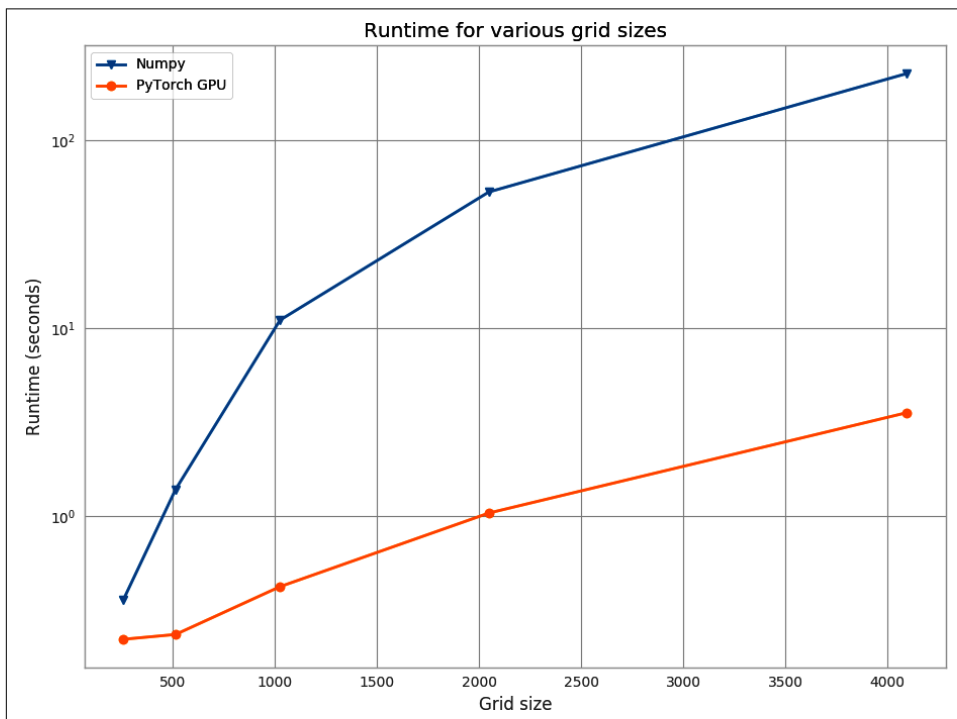


Figure 7-7. PyTorch versus `numpy` performance

This speedup is a result of how parallelizable the diffusion problem is. As we said before, the GPU we are using has 4,362 independent computation cores. It seems that

³ PyTorch CPU performance isn't terribly great unless you install from source. When installing from source, optimized linear algebra libraries are used to give speeds comparable to NumPy.

⁴ As with any JIT, the first time a function is called, it will have the overhead of having to compile the code. In [Example 7-19](#), we profile the functions multiple times and ignore the first time in order to measure only the runtime speeds.

once the diffusion problem is parallelized, none of these GPU cores are being fully utilized.



When timing the performance of GPU code, it is important to set the environmental flag `CUDA_LAUNCH_BLOCKING=1`. By default, GPU operations are run asynchronously to allow more operations to be pipelined together and thus to minimize the total utilization of the GPU and increase parallelization. When the asynchronous behavior is enabled, we can guarantee that the computations are done only when either the data is copied to another device or a `torch.cuda.synchronize()` command is issued. By enabling the preceding environmental variable, we can make sure that computations are completed when they are issued and that we are indeed measuring compute time.

Basic GPU Profiling

One way to verify exactly how much of the GPU we are utilizing is by using the `nvidia-smi` command to inspect the resource utilization of the GPU. The two values we are most interested in are the power usage and the GPU utilization:

```
$ nvidia-smi
+-----+
| NVIDIA-SMI 440.44                Driver Version: 440.44                CUDA Version: 10.2                |
+-----+-----+-----+-----+-----+-----+
| GPU   Name           Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
+-----+-----+-----+-----+-----+-----+
|    0  GeForce RTX 208...    Off | 00000000:06:00.0 Off |                  N/A |
| 30%   58C   P2     96W / 260W | 1200MiB / 11018MiB |    95%      Default  |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes:                                     GPU Memory |
|  GPU       PID    Type   Process name                               Usage      |
+-----+-----+-----+-----+-----+-----+
|    0       26329    C     .../.pyenv/versions/3.7.2/bin/python         1189MiB |
+-----+-----+-----+-----+-----+-----+
```

GPU utilization, here at 95%, is a slightly mislabeled field. It tells us what percentage of the last second has been spent running at least one kernel. So it isn't telling us what percentage of the GPU's total computational power we're using but rather how much time was spent *not* being idle. This is a very useful measurement to look at when debugging memory transfer issues and making sure that the CPU is providing the GPU with enough work.

Power usage, on the other hand, is a good proxy for judging how much of the GPU's compute power is being used. As a rule of thumb, the more power the GPU is

drawing, the more compute it is currently doing. If the GPU is waiting for data from the CPU or using only half of the available cores, power use will be reduced from the maximum.

Another useful tool is **gpustat**. This project provides a nice view into many of NVIDIA's stats using a much friendlier interface than `nvidia-smi`.

To help understand what specifically is causing slowdowns in your PyTorch code, the project provides a special profiling tool. Running your code with `python -m torch.utils.bottleneck` will show both CPU and GPU runtime stats to help you identify potential optimizations to either portion of your code.

Performance Considerations of GPUs

Since a GPU is a completely auxiliary piece of hardware on the computer, with its own architecture as compared with the CPU, there are many GPU-specific performance considerations to keep in mind.

The biggest speed consideration for GPUs is the transfer time of data from the system memory to the GPU memory. When we use `tensor.to(DEVICE)`, we are triggering a transfer of data that may take some time depending on the speed of the GPU's bus and the amount of data being transferred.

Other operations may trigger a transfer as well. In particular, `tensor.items()` and `tensor.tolist()` often cause problems when introduced for debugging purposes. In fact, running `tensor.numpy()` to convert a PyTorch tensor into a numpy array specifically requires an explicit copy out of the GPU, which ensures you understand the potential penalty.

As an example, let's add a `grid.cpu()` call inside the solver loop of our diffusion code:

```
grid = grid.to(device)
for i in range(num_iterations):
    grid = evolve(grid, 0.1)
    grid.cpu()
```

To ensure that we have a fair comparison, we will also add `torch.cuda.synchronize()` to the control code so that we are simply testing the time to copy the data from the CPU. In addition to this slowing down your code by triggering a data transfer from the GPU to the system memory, your code will slow down because the GPU will pause code execution that would have continued in the background until the transfer is complete.

This change to the code for a $2,048 \times 2,048$ grid slows down our code by 2.54×! Even though our GPU has an advertised bandwidth of 616.0 GB/s, this overhead can quickly add up. In addition, other overhead costs are associated with a memory copy.

First, we are creating a hard stop to any potential pipelining of our code execution. Then, because we are no longer pipelining, our data on the GPU must all be synchronized out of the memory of the individual CUDA cores. Finally, space on system memory needs to be prepared to receive the new data from the GPU.

While this seems like a ridiculous addition to make to our code, this sort of thing happens all the time. In fact, one of the biggest things slowing down PyTorch code when it comes to deep learning is copying training data from the host into the GPU. Often the training data is simply too big to fit on the GPU, and doing these constant data transfers is an unavoidable penalty.

There are ways to alleviate the overhead from this inevitable data transfer when the problem is going from CPU to GPU. First, the memory region can be marked as `pin` ned. This can be done by calling the `Tensor.pin_memory()` method, which returns a copy of the CPU tensor that is copied to a “page locked” region of memory. This page-locked region can be copied to the GPU much more quickly, and it can be copied asynchronously so as to not disturb any computations being done by the GPU. While training a deep learning model, data loading is generally done with the `DataLoader` class, which conveniently has a `pin_memory` parameter that can automatically do this for all your training data.⁵

The most important step is to profile your code by using the tools outlined in “**Basic GPU Profiling**” on page 193. When your code is spending most of its time doing data transfers, you will see a low power draw, a smaller GPU utilization (as reported by `nvidia-smi`), and most of your time being spent in the `to` function (as reported by `bottleneck`). Ideally, you will be using the maximum amount of power the GPU can support and have 100% utilization. This is possible even when large amounts of data transfer are required—even when training deep learning models with a large number of images!



GPUs are not particularly good at running multiple tasks at the same time. When starting up a task that requires heavy use of the GPU, ensure that no other tasks are utilizing it by running `nvidia-smi`. However, if you are running a graphical environment, you may have no choice but to have your desktop and GPU code use the GPU at the same time.

⁵ The `DataLoader` object also supports running with multiple workers. Having several workers is recommended if data is being loaded from disk in order to minimize I/O time.

When to Use GPUs

We've seen that GPUs can be incredibly quick; however, memory considerations can be quite devastating to this runtime. This seems to indicate that if your task requires mainly linear algebra and matrix manipulations (like multiplication, addition, and Fourier transforms), then GPUs are a fantastic tool. This is particularly true if the calculation can happen on the GPU uninterrupted for a period of time before being copied back into system memory.

As an example of a task that requires a lot of branching, we can imagine code where every step of the computation requires the previous result. If we compare running [Example 7-20](#) using PyTorch versus using NumPy, we see that NumPy is consistently faster (98% faster for the included example!). This makes sense given the architecture of the GPU. While the GPU can run many more tasks at once than the CPU can, each of those tasks runs more slowly on the GPU than on the CPU. This example task can run only one computation at a time, so having many compute cores doesn't help at all; it's better to simply have one very fast core.

Example 7-20. Highly branching task

```
import torch

def task(A, target):
    """
    Given an int array of length N with values from {0, N} and a target value,
    iterates through the array, using the current value to find the next array
    item to look at, until we have seen a total value of at least `target`.
    Returns how many iterations until the value was reached.
    """
    result = 0
    i = 0
    N = 0
    while result < target:
        r = A[i]
        result += r
        i = A[i]
        N += 1
    return N

if __name__ == "__main__":
    N = 1000

    A_py = (torch.rand(N) * N).type(torch.int).to('cuda:0')
    A_np = A_py.cpu().numpy()

    task(A_py, 500)
    task(A_np, 500)
```

In addition, because of the limited memory of the GPU, it is not a good tool for tasks that require exceedingly large amounts of data, many conditional manipulations of the data, or changing data. Most GPUs made for computational tasks have around 12 GB of memory, which puts a significant limitation on “large amounts of data.” However, as technology improves, the size of GPU memory increases, so hopefully this limitation becomes less drastic in the future.

The general recipe for evaluating whether to use the GPU consists of the following steps:

1. Ensure that the memory use of the problem will fit within the GPU (in [“Using memory_profiler to Diagnose Memory Usage” on page 46](#), we explore profiling memory use).
2. Evaluate whether the algorithm requires a lot of branching conditions versus vectorized operations. As a rule of thumb, numpy functions generally vectorize very well, so if your algorithm can be written in terms of numpy calls, your code probably will vectorize well! You can also check the branches result when running `perf` (as explained in [“Understanding perf” on page 122](#)).
3. Evaluate how much data needs to be moved between the GPU and the CPU. Some questions to ask here are “How much computation can I do before I need to plot/save results?” and “Are there times my code will have to copy the data to run in a library I know isn’t GPU-compatible?”
4. Make sure PyTorch supports the operations you’d like to do! PyTorch implements a large portion of the numpy API, so this shouldn’t be an issue. For the most part, the API is even the same, so you don’t need to change your code at all. However, in some cases either PyTorch doesn’t support an operation (such as dealing with complex numbers) or the API is slightly different (for example, with generating random numbers).

Considering these four points will help give you confidence that a GPU approach would be worthwhile. There are no hard rules for when the GPU will work better than the CPU, but these questions will help you gain some intuition. However, PyTorch also makes converting code to use the GPU painless, so the barrier to entry is quite low, even if you are just evaluating the GPU’s performance.

Foreign Function Interfaces

Sometimes the automatic solutions just don’t cut it, and you need to write custom C or Fortran code yourself. This could be because the compilation methods don’t find some potential optimizations, or because you want to take advantage of libraries or language features that aren’t available in Python. In all of these cases, you’ll need to

use foreign function interfaces, which give you access to code written and compiled in another language.

For the rest of this chapter, we will attempt to use an external library to solve the 2D diffusion equation in the same way we did in [Chapter 6](#).⁶ The code for this library, shown in [Example 7-21](#), could be representative of a library you’ve installed or code that you have written. The methods we’ll look at serve as great ways to take small parts of your code and move them to another language in order to do very targeted language-based optimizations.

Example 7-21. Sample C code for solving the 2D diffusion problem

```
void evolve(double in[][512], double out[][512], double D, double dt) {
    int i, j;
    double laplacian;
    for (i=1; i<511; i++) {
        for (j=1; j<511; j++) {
            laplacian = in[i+1][j] + in[i-1][j] + in[i][j+1] + in[i][j-1] \
                - 4 * in[i][j];
            out[i][j] = in[i][j] + D * dt * laplacian;
        }
    }
}
```



We fix the grid size to be 512×512 in order to simplify the example code. To accept an arbitrarily sized grid, you must pass in the `in` and `out` parameters as double pointers and include function arguments for the actual size of the grid.

To use this code, we must compile it into a shared module that creates a `.so` file. We can do this using `gcc` (or any other C compiler) by following these steps:

```
$ gcc -O3 -std=gnu11 -c diffusion.c
$ gcc -shared -o diffusion.so diffusion.o
```

We can place this final shared library file anywhere that is accessible to our Python code, but standard *nix organization stores shared libraries in `/usr/lib` and `/usr/local/lib`.

⁶ For simplicity, we will not implement the boundary conditions.

ctypes

The most basic foreign function interface in CPython is through the ctypes module.⁷ The bare-bones nature of this module can be quite inhibitive at times—you are in charge of doing everything, and it can take quite a while to make sure that you have everything in order. This extra level of complexity is evident in our ctypes diffusion code, shown in [Example 7-22](#).

Example 7-22. ctypes 2D diffusion code

```
import ctypes

grid_shape = (512, 512)
_diffusion = ctypes.CDLL("diffusion.so") ❶

# Create references to the C types that we will need to simplify future code
TYPE_INT = ctypes.c_int
TYPE_DOUBLE = ctypes.c_double
TYPE_DOUBLE_SS = ctypes.POINTER(ctypes.POINTER(ctypes.c_double))

# Initialize the signature of the evolve function to:
# void evolve(int, int, double**, double**, double, double)
_diffusion.evolve.argtypes = [TYPE_DOUBLE_SS, TYPE_DOUBLE_SS, TYPE_DOUBLE,
                              TYPE_DOUBLE]
_diffusion.evolve.restype = None

def evolve(grid, out, dt, D=1.0):
    # First we convert the Python types into the relevant C types
    assert grid.shape == (512, 512)
    cdt = TYPE_DOUBLE(dt)
    cD = TYPE_DOUBLE(D)
    pointer_grid = grid.ctypes.data_as(TYPE_DOUBLE_SS) ❷
    pointer_out = out.ctypes.data_as(TYPE_DOUBLE_SS)

    # Now we can call the function
    _diffusion.evolve(pointer_grid, pointer_out, cD, cdt) ❸
```

- ❶ This is similar to importing the `diffusion.so` library. Either this file is in one of the standard system paths for library files or we can enter an absolute path.
- ❷ `grid` and `out` are both numpy arrays.
- ❸ We finally have all the setup necessary and can call the C function directly.

⁷ This is CPython-dependent. Other versions of Python may have their own versions of ctypes, which may work differently.

This first thing we do is “import” our shared library. This is done with the `ctypes.CDLL` call. In this line, we can specify any shared library that Python can access (for example, the `ctypes-opencv` module loads the `libcv.so` library). From this, we get a `_diffusion` object that contains all the members that the shared library contains. In this example, `diffusion.so` contains only one function, `evolve`, which is now made available to us as a property of the `_diffusion` object. If `diffusion.so` had many functions and properties, we could access them all through the `_diffusion` object.

However, even though the `_diffusion` object has the `evolve` function available within it, Python doesn’t know how to use it. C is statically typed, and the function has a very specific signature. To properly work with the `evolve` function, we must explicitly set the input argument types and the return type. This can become quite tedious when developing libraries in tandem with the Python interface, or when dealing with a quickly changing library. Furthermore, since `ctypes` can’t check if you have given it the correct types, your code may silently fail or segfault if you make a mistake!

Furthermore, in addition to setting the arguments and return type of the function object, we also need to convert any data we care to use with it (this is called *casting*). Every argument we send to the function must be carefully casted into a native C type. Sometimes this can get quite tricky, since Python is very relaxed about its variable types. For example, if we had `num1 = 1e5`, we would have to know that this is a Python float, and thus we should use a `ctype.c_float`. On the other hand, for `num2 = 1e300`, we would have to use `ctype.c_double`, because it would overflow a standard C float.

That being said, `numpy` provides a `.ctypes` property to its arrays that makes it easily compatible with `ctypes`. If `numpy` didn’t provide this functionality, we would have had to initialize a `ctypes` array of the correct type and then find the location of our original data and have our new `ctypes` object point there.



Unless the object you are turning into a `ctype` object implements a buffer (as do the `array` module, `numpy` arrays, `io.StringIO`, etc.), your data will be copied into the new object. In the case of casting an `int` to a `float`, this doesn’t mean much for the performance of your code. However, if you are casting a very long Python list, this can incur quite a penalty! In these cases, using the `array` module or a `numpy` array, or even building up your own buffered object using the `struct` module, would help. This does, however, hurt the readability of your code, since these objects are generally less flexible than their native Python counterparts.

This memory management can get even more complicated if you have to send the library a complicated data structure. For example, if your library expects a C struct representing a point in space with the properties `x` and `y`, you would have to define the following:

```
from ctypes import Structure

class cPoint(Structure):
    _fields_ = ("x", c_int), ("y", c_int)
```

At this point you could start creating C-compatible objects by initializing a `cPoint` object (i.e., `point = cPoint(10, 5)`). This isn't a terrible amount of work, but it can become tedious and results in some fragile code. What happens if a new version of the library is released that slightly changes the structure? This will make your code very hard to maintain and generally results in stagnant code, where the developers simply decide never to upgrade the underlying libraries that are being used.

For these reasons, using the `ctypes` module is great if you already have a good understanding of C and want to be able to tune every aspect of the interface. It has great portability since it is part of the standard library, and if your task is simple, it provides simple solutions. Just be careful because the complexity of `ctypes` solutions (and similar low-level solutions) quickly becomes unmanageable.

cffi

Realizing that `ctypes` can be quite cumbersome to use at times, `cffi` attempts to simplify many of the standard operations that programmers use. It does this by having an internal C parser that can understand function and structure definitions.

As a result, we can simply write the C code that defines the structure of the library we wish to use, and then `cffi` will do all the heavy work for us: it imports the module and makes sure we specify the correct types to the resulting functions. In fact, this work can be almost trivial if the source for the library is available, since the header files (the files ending in `.h`) will include all the relevant definitions we need.⁸

Example 7-23 shows the `cffi` version of the 2D diffusion code.

Example 7-23. cffi 2D diffusion code

```
from cffi import FFI, verifier

grid_shape = (512, 512)

ffi = FFI()
```

⁸ In Unix systems, header files for system libraries can be found in `/usr/include`.


```

ffi.cdef(
    "void evolve(double **in, double **out, double D, double dt);" ❶
)
lib = ffi.dlopen("../diffusion.so")

def evolve(grid, dt, out, D=1.0):
    pointer_grid = ffi.cast("double**", grid.ctypes.data) ❷
    pointer_out = ffi.cast("double**", out.ctypes.data)
    lib.evolve(pointer_grid, pointer_out, D, dt)

```

- ❶ The contents of this definition can normally be acquired from the manual of the library that you are using or by looking at the library's header files.
- ❷ While we still need to cast nonnative Python objects for use with our C module, the syntax is very familiar to those with experience in C.

In the preceding code, we can think of the `cffi` initialization as being two-stepped. First, we create an FFI object and give it all the global C declarations we need. This can include datatypes in addition to function signatures. These signatures don't necessarily contain any code; they simply need to define what the code will look like. Then we can import a shared library containing the actual implementation of the functions by using `dlopen`. This means we could have told FFI about the function signature for the `evolve` function and then loaded up two different implementations and stored them in different objects (which is fantastic for debugging and profiling!).

In addition to easily importing a shared C library, `cffi` allows you to write C code and have it be dynamically compiled using the `verify` function. This has many immediate benefits—for example, you can easily rewrite small portions of your code to be in C without invoking the large machinery of a separate C library. Alternatively, if there is a library you wish to use, but some glue code in C is required to have the interface work perfectly, you can inline it into your `cffi` code, as shown in [Example 7-24](#), to have everything be in a centralized location. In addition, since the code is being dynamically compiled, you can specify compile instructions to every chunk of code you need to compile. Note, however, that this compilation has a one-time penalty every time the `verify` function is run to actually perform the compilation.

Example 7-24. `cffi` with inline 2D diffusion code

```

ffi = FFI()
ffi.cdef(
    "void evolve(double **in, double **out, double D, double dt);"
)
lib = ffi.verify(
    r"""

```

```

void evolve(double in[][512], double out[][512], double D, double dt) {
    int i, j;
    double laplacian;
    for (i=1; i<511; i++) {
        for (j=1; j<511; j++) {
            laplacian = in[i+1][j] + in[i-1][j] + in[i][j+1] + in[i][j-1] \
                - 4 * in[i][j];
            out[i][j] = in[i][j] + D * dt * laplacian;
        }
    }
}
"""
extra_compile_args=["-O3"], ❶
)

```

- ❶ Since we are just-in-time compiling this code, we can also provide relevant compilation flags.

Another benefit of the `verify` functionality is that it plays nicely with complicated `cdef` statements. For example, if we were using a library with a complicated structure but wanted to use only a part of it, we could use the partial struct definition. To do this, we add a `...` in the struct definition in `ffi.cdef` and `#include` the relevant header file in a later `verify`.

For example, suppose we were working with a library with header `complicated.h` that included a structure that looked like this:

```

struct Point {
    double x;
    double y;
    bool isActive;
    char *id;
    int num_times_visited;
}

```

If we cared only about the `x` and `y` properties, we could write some simple `cffi` code that cares only about those values:

```

from cffi import FFI

ffi = FFI()
ffi.cdef(r"""
    struct Point {
        double x;
        double y;
        ...;
    };
    struct Point do_calculation();
""")
lib = ffi.verify(r"""

```

```
#include <complicated.h>
""" )
```

We could then run the `do_calculation` function from the `complicated.h` library and have returned to us a `Point` object with its `x` and `y` properties accessible. This is amazing for portability, since this code will run just fine on systems with a different implementation of `Point` or when new versions of `complicated.h` come out, as long as they all have the `x` and `y` properties.

All of these niceties really make `cffi` an amazing tool to have when you're working with C code in Python. It is much simpler than `ctypes`, while still giving you the same amount of fine-grained control you may want when working directly with a foreign function interface.

f2py

For many scientific applications, Fortran is still the gold standard. While its days of being a general-purpose language are over, it still has many niceties that make vector operations easy to write and quite quick. In addition, many performance math libraries are written in Fortran ([LAPACK](#), [BLAS](#), etc.), all of which are fundamental in libraries such as SciPy, and being able to use them in your performance Python code may be critical.

For such situations, [f2py](#) provides a dead-simple way of importing Fortran code into Python. This module is able to be so simple because of the explicitness of types in Fortran. Since the types can be easily parsed and understood, `f2py` can easily make a CPython module that uses the native foreign function support within C to use the Fortran code. This means that when you are using `f2py`, you are actually autogenerating a C module that knows how to use Fortran code! As a result, a lot of the confusion inherent in the `ctypes` and `cffi` solutions simply doesn't exist.

In [Example 7-25](#), we can see some simple `f2py`-compatible code for solving the diffusion equation. In fact, all native Fortran code is `f2py`-compatible; however, the annotations to the function arguments (the statements prefaced by `!f2py`) simplify the resulting Python module and make for an easier-to-use interface. The annotations implicitly tell `f2py` whether we intend for an argument to be only an output or only an input, or to be something we want to modify in place or hidden completely. The hidden type is particularly useful for the sizes of vectors: while Fortran may need those numbers explicitly, our Python code already has this information on hand. When we set the type as “hidden,” `f2py` can automatically fill those values for us, essentially keeping them hidden from us in the final Python interface.

Example 7-25. Fortran 2D diffusion code with f2py annotations

```
SUBROUTINE evolve(grid, next_grid, D, dt, N, M)
  !f2py threadsafe
  !f2py intent(in) grid
  !f2py intent(inplace) next_grid
  !f2py intent(in) D
  !f2py intent(in) dt
  !f2py intent(hide) N
  !f2py intent(hide) M
  INTEGER :: N, M
  DOUBLE PRECISION, DIMENSION(N,M) :: grid, next_grid
  DOUBLE PRECISION, DIMENSION(N-2, M-2) :: laplacian
  DOUBLE PRECISION :: D, dt

  laplacian = grid(3:N, 2:M-1) + grid(1:N-2, 2:M-1) + &
    grid(2:N-1, 3:M) + grid(2:N-1, 1:M-2) - 4 * grid(2:N-1, 2:M-1)
  next_grid(2:N-1, 2:M-1) = grid(2:N-1, 2:M-1) + D * dt * laplacian
END SUBROUTINE evolve
```

To build the code into a Python module, we run the following command:

```
$ f2py -c -m diffusion --fcompiler=gfortran --opt='-O3' diffusion.f90
```



We specifically use `gfortran` in the preceding call to `f2py`. Make sure it is installed on your system or that you change the corresponding argument to use the Fortran compiler you have installed.

This will create a library file pinned to your Python version and operating system (*diffusion.cpython-37m-x86_64-linux-gnu.so*, in our case) that can be imported directly into Python.

If we play around with the resulting module interactively, we can see the niceties that `f2py` has given us, thanks to our annotations and its ability to parse the Fortran code:

```
>>> import diffusion

>>> diffusion?
Type:          module
String form:   <module 'diffusion' from '[...]cpython-37m-x86_64-linux-gnu.so'>
File:          [...].cut.../diffusion.cpython-37m-x86_64-linux-gnu.so
Docstring:
This module 'diffusion' is auto-generated with f2py (version:2).
Functions:
  evolve(grid,scratch,d,dt)
  .

>>> diffusion.evolve?
Call signature: diffusion.evolve(*args, **kwargs)
```

```

Type:          fortran
String form:   <fortran object>
Docstring:
evolve(grid,scratch,d,dt)

Wrapper for ``evolve``.

Parameters
grid : input rank-2 array('d') with bounds (n,m)
scratch : rank-2 array('d') with bounds (n,m)
d : input float
dt : input float

```

This code shows that the result from the f2py generation is automatically documented, and the interface is quite simplified. For example, instead of us having to extract the sizes of the vectors, f2py has figured out how to automatically find this information and simply hides it in the resulting interface. In fact, the resulting `evolve` function looks exactly the same in its signature as the pure Python version we wrote in [Example 6-14](#).

The only thing we must be careful of is the ordering of the numpy arrays in memory. Since most of what we do with numpy and Python focuses on code derived from C, we always use the C convention for ordering data in memory (called *row-major ordering*). Fortran uses a different convention (*column-major ordering*) that we must make sure our vectors abide by. These orderings simply state whether, for a 2D array, columns or rows are contiguous in memory.⁹ Luckily, this simply means we specify the `order='F'` parameter to numpy when declaring our vectors.



The difference in ordering basically changes which is the outer loop when iterating over a multidimensional array. In Python and C, if you define an array as `array[X][Y]`, your outer loop will be over `X` and your inner loop will be over `Y`. In Fortran, your outer loop will be over `Y` and your inner loop will be over `X`. If you use the wrong loop ordering, you will at best suffer a major performance penalty because of an increase in cache-misses (see “[Memory Fragmentation](#)” on page 120) and at worst access the wrong data!

⁹ For more information, see [the Wikipedia page](#) on row-major and column-major ordering.

This results in the following code to use our Fortran subroutine. This code looks exactly the same as what we used in [Example 6-14](#), except for the import from the f2py-derived library and the explicit Fortran ordering of our data:

```
from diffusion import evolve

def run_experiment(num_iterations):
    scratch = np.zeros(grid_shape, dtype=np.double, order="F") ❶
    grid = np.zeros(grid_shape, dtype=np.double, order="F")

    initialize_grid(grid)

    for i in range(num_iterations):
        evolve(grid, scratch, 1.0, 0.1)
        grid, scratch = scratch, grid
```

- ❶ Fortran orders numbers differently in memory, so we must remember to set our numpy arrays to use that standard.

CPython Module

Finally, we can always go right down to the CPython API level and write a CPython module. This requires us to write code in the same way that CPython is developed and take care of all of the interactions between our code and the implementation of CPython.

This has the advantage that it is incredibly portable, depending on the Python version. We don't require any external modules or libraries, just a C compiler and Python! However, this doesn't necessarily scale well to new versions of Python. For example, CPython modules written for Python 2.7 won't work with Python 3, and vice versa.



In fact, much of the slowdown in the Python 3 rollout was rooted in the difficulty in making this change. When creating a CPython module, you are coupled very closely to the actual Python implementation, and large changes in the language (such as the change from 2.7 to 3) require large modifications to your module.

That portability comes at a big cost, though—you are responsible for every aspect of the interface between your Python code and the module. This can make even the simplest tasks take dozens of lines of code. For example, to interface with the diffusion library from [Example 7-21](#), we must write 28 lines of code simply to read the arguments to a function and parse them ([Example 7-26](#)). Of course, this does mean that you have incredibly fine-grained control over what is happening. This goes all the way down to being able to manually change the reference counts for Python's garbage collection (which can be the cause of a lot of pain when creating CPython

modules that deal with native Python types). Because of this, the resulting code tends to be minutely faster than other interface methods.



All in all, this method should be left as a last resort. While it is quite informative to write a CPython module, the resulting code is not as reusable or maintainable as other potential methods. Making subtle changes in the module can often require completely reworking it. In fact, we include the module code and the required *setup.py* to compile it (Example 7-27) as a cautionary tale.

Example 7-26. CPython module to interface to the 2D diffusion library

```
// python_interface.c
// - cpython module interface for diffusion.c

#define NPY_NO_DEPRECATED_API  NPY_1_7_API_VERSION

#include <Python.h>
#include <numpy/arrayobject.h>
#include "diffusion.h"

/* Docstrings */
static char module_docstring[] =
    "Provides optimized method to solve the diffusion equation";
static char cdiffusion_evolve_docstring[] =
    "Evolve a 2D grid using the diffusion equation";

PyArrayObject *py_evolve(PyObject *, PyObject *);

/* Module specification */
static PyMethodDef module_methods[] =
{
    /* { method name , C function , argument types , docstring } */
    { "evolve", (PyCFunction)py_evolve, METH_VARARGS, cdiffusion_evolve_docstring },
    { NULL, NULL, 0, NULL }
};

static struct PyModuleDef cdiffusionmodule =
{
    PyModuleDef_HEAD_INIT,
    "cdiffusion", /* name of module */
    module_docstring, /* module documentation, may be NULL */
    -1, /* size of per-interpreter state of the module,
        * or -1 if the module keeps state in global variables. */
    module_methods
};

PyArrayObject *py_evolve(PyObject *self, PyObject *args)
{
    PyArrayObject *data;
```

```

PyArrayObject *next_grid;
double         dt, D = 1.0;

/* The "evolve" function will have the signature:
 *     evolve(data, next_grid, dt, D=1)
 */
if (!PyArg_ParseTuple(args, "Ood|d", &data, &next_grid, &dt, &D))
{
    PyErr_SetString(PyExc_RuntimeError, "Invalid arguments");
    return(NULL);
}

/* Make sure that the numpy arrays are contiguous in memory */
if (!PyArray_Check(data) || !PyArray_ISCONTIGUOUS(data))
{
    PyErr_SetString(PyExc_RuntimeError, "data is not a contiguous array.");
    return(NULL);
}
if (!PyArray_Check(next_grid) || !PyArray_ISCONTIGUOUS(next_grid))
{
    PyErr_SetString(PyExc_RuntimeError, "next_grid is not a contiguous array.");
    return(NULL);
}

/* Make sure that grid and next_grid are of the same type and have the same
 * dimensions
 */
if (PyArray_TYPE(data) != PyArray_TYPE(next_grid))
{
    PyErr_SetString(PyExc_RuntimeError,
                    "next_grid and data should have same type.");
    return(NULL);
}
if (PyArray_NDIM(data) != 2)
{
    PyErr_SetString(PyExc_RuntimeError, "data should be two dimensional");
    return(NULL);
}
if (PyArray_NDIM(next_grid) != 2)
{
    PyErr_SetString(PyExc_RuntimeError, "next_grid should be two dimensional");
    return(NULL);
}
if ((PyArray_DIM(data, 0) != PyArray_DIM(next_grid, 0)) ||
    (PyArray_DIM(data, 1) != PyArray_DIM(next_grid, 1)))
{
    PyErr_SetString(PyExc_RuntimeError,
                    "data and next_grid must have the same dimensions");
    return(NULL);
}

evolve(

```



```

        PyArray_DATA(data),
        PyArray_DATA(next_grid),
        D,
        dt
    );

    Py_XINCREF(next_grid);
    return(next_grid);
}

/* Initialize the module */
PyMODINIT_FUNC
PyInit_cdifffusion(void)
{
    PyObject *m;

    m = PyModule_Create(&cdifffusionmodule);
    if (m == NULL)
    {
        return(NULL);
    }

    /* Load `numpy` functionality. */
    import_array();

    return(m);
}

```

To build this code, we need to create a *setup.py* script that uses the `distutils` module to figure out how to build the code such that it is Python-compatible ([Example 7-27](#)). In addition to the standard `distutils` module, `numpy` provides its own module to help with adding `numpy` integration in your CPython modules.

Example 7-27. Setup file for the CPython module diffusion interface

```

"""
setup.py for cpython diffusion module. The extension can be built by running

    $ python setup.py build_ext --inplace

which will create the __cdifffusion.so__ file, which can be directly imported into
Python.
"""

from distutils.core import setup, Extension
import numpy.distutils.misc_util

__version__ = "0.1"

cdifffusion = Extension(
    'cdifffusion',

```

```

sources = ['cdiffusion/cdiffusion.c', 'cdiffusion/python_interface.c'],
extra_compile_args = ["-O3", "-std=c11", "-Wall", "-p", "-pg", ],
extra_link_args = ["-lc"],
)

setup (
    name = 'diffusion',
    version = __version__,
    ext_modules = [cdiffusion,],
    packages = ["diffusion", ],
    include_dirs = numpy.distutils.misc_util.get_numpy_include_dirs(),
)

```

The result from this is a *cdiffusion.so* file that can be imported directly from Python and used quite easily. Since we had complete control over the signature of the resulting function and exactly how our C code interacted with the library, we were able to (with some hard work) create a module that is easy to use:

```

from cdiffusion import evolve

def run_experiment(num_iterations):
    next_grid = np.zeros(grid_shape, dtype=np.double)
    grid = np.zeros(grid_shape, dtype=np.double)

    # ... standard initialization ...

    for i in range(num_iterations):
        evolve(grid, next_grid, 1.0, 0.1)
        grid, next_grid = next_grid, grid

```

Wrap-Up

The various strategies introduced in this chapter allow you to specialize your code to different degrees in order to reduce the number of instructions the CPU must execute and to increase the efficiency of your programs. Sometimes this can be done algorithmically, although often it must be done manually (see “[JIT Versus AOT Compilers](#)” on page 164). Furthermore, sometimes these methods must be employed simply to use libraries that have already been written in other languages. Regardless of the motivation, Python allows us to benefit from the speedups that other languages can offer on some problems, while still maintaining verbosity and flexibility when needed.

We also looked into how to use the GPU to use purpose-specific hardware to solve problems faster than a CPU could alone. These devices are very specialized and can have very different performance considerations than classical high performance programming. However, we’ve seen that new libraries like PyTorch make evaluating the GPU much simpler than it ever has been.

It is important to note, though, that these optimizations are done to optimize the efficiency of compute instructions only. If you have I/O-bound processes coupled to a compute-bound problem, simply compiling your code may not provide any reasonable speedups. For these problems, we must rethink our solutions and potentially use parallelism to run different tasks at the same time.

PNP Ch 53 | Itertools Module [companion]

Chapter 53: Itertools Module

Section 53.1: Combinations method in Itertools Module

`itertools.combinations` will return a generator of the k -combination sequence of a list.

In other words: It will return a generator of tuples of all the possible k -wise combinations of the input list.

For Example:

If you have a list:

```
a = [1,2,3,4,5]
b = list(itertools.combinations(a, 2))
print b
```

Output:

```
[(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (4, 5)]
```

The above output is a generator converted to a list of tuples of all the possible *pair*-wise combinations of the input list `a`

You can also find all the 3-combinations:

```
a = [1,2,3,4,5]
b = list(itertools.combinations(a, 3))
print b
```

Output:

```
[(1, 2, 3), (1, 2, 4), (1, 2, 5), (1, 3, 4),
 (1, 3, 5), (1, 4, 5), (2, 3, 4), (2, 3, 5),
 (2, 4, 5), (3, 4, 5)]
```

Section 53.2: `itertools.dropwhile`

`itertools.dropwhile` enables you to take items from a sequence after a condition first becomes `False`.

```
def is_even(x):
    return x % 2 == 0

lst = [0, 2, 4, 12, 18, 13, 14, 22, 23, 44]
result = list(itertools.dropwhile(is_even, lst))

print(result)
```

This outputs `[13, 14, 22, 23, 44]`.

(This example is same as the example for `takewhile` but using `dropwhile`.)

Note that, the first number that violates the predicate (i.e.: the function returning a Boolean value) `is_even` is, 13. All the elements before that, are discarded.

The **output produced** by `dropwhile` is similar to the output generated from the code below.

```
def dropwhile(predicate, iterable):
    iterable = iter(iterable)
    for x in iterable:
        if not predicate(x):
            yield x
            break
    for x in iterable:
        yield x
```

The concatenation of results produced by `takewhile` and `dropwhile` produces the original iterable.

```
result = list(itertools.takewhile(is_even, lst)) + list(itertools.dropwhile(is_even, lst))
```

Section 53.3: Zipping two iterators until they are both exhausted

Similar to the built-in function `zip()`, `itertools.zip_longest` will continue iterating beyond the end of the shorter of two iterables.

```
from itertools import zip_longest
a = [i for i in range(5)] # Length is 5
b = ['a', 'b', 'c', 'd', 'e', 'f', 'g'] # Length is 7
for i in zip_longest(a, b):
    x, y = i # Note that zip longest returns the values as a tuple
    print(x, y)
```

An optional `fillvalue` argument can be passed (defaults to `' '`) like so:

```
for i in zip_longest(a, b, fillvalue='Hogwash!'):
    x, y = i # Note that zip longest returns the values as a tuple
    print(x, y)
```

In Python 2.6 and 2.7, this function is called `itertools.izip_longest`.

Section 53.4: Take a slice of a generator

`Itertools "islice"` allows you to slice a generator:

```
results = fetch_paged_results() # returns a generator
limit = 20 # Only want the first 20 results
for data in itertools.islice(results, limit):
    print(data)
```

Normally you cannot slice a generator:

```
def gen():
    n = 0
    while n < 20:
        n += 1
        yield n

for part in gen()[ :3]:
    print(part)
```

Will give

```
Traceback (most recent call last):
  File "gen.py", line 6, in <module>
    for part in gen()[3]:
TypeError: 'generator' object is not subscriptable
```

However, this works:

```
import itertools

def gen():
    n = 0
    while n < 20:
        n += 1
        yield n

for part in itertools.islice(gen(), 3):
    print(part)
```

Note that like a regular slice, you can also use start, stop and step arguments:

```
itertools.islice(iterable, 1, 30, 3)
```

Section 53.5: Grouping items from an iterable object using a function

Start with an iterable which needs to be grouped

```
lst = [("a", 5, 6), ("b", 2, 4), ("a", 2, 5), ("c", 2, 6)]
```

Generate the grouped generator, grouping by the second element in each tuple:

```
def testGroupBy(lst):
    groups = itertools.groupby(lst, key=lambda x: x[1])
    for key, group in groups:
        print(key, list(group))

testGroupBy(lst)

# 5 [('a', 5, 6)]
# 2 [('b', 2, 4), ('a', 2, 5), ('c', 2, 6)]
```

Only groups of consecutive elements are grouped. You may need to sort by the same key before calling groupby
For E.g, (Last element is changed)

```
lst = [("a", 5, 6), ("b", 2, 4), ("a", 2, 5), ("c", 5, 6)]
testGroupBy(lst)

# 5 [('a', 5, 6)]
# 2 [('b', 2, 4), ('a', 2, 5)]
# 5 [('c', 5, 6)]
```

The group returned by groupby is an iterator that will be invalid before next iteration. E.g the following will not work if you want the groups to be sorted by key. Group 5 is empty below because when group 2 is fetched it invalidates 5

```
lst = [("a", 5, 6), ("b", 2, 4), ("a", 2, 5), ("c", 2, 6)]
groups = itertools.groupby(lst, key=lambda x: x[1])
for key, group in sorted(groups):
```

```
print(key, list(group))

# 2 [('c', 2, 6)]
# 5 []
```

To correctly do sorting, create a list from the iterator before sorting

```
groups = itertools.groupby(lst, key=lambda x: x[1])
for key, group in sorted((key, list(group)) for key, group in groups):
    print(key, list(group))

# 2 [('b', 2, 4), ('a', 2, 5), ('c', 2, 6)]
# 5 [('a', 5, 6)]
```

Section 53.6: itertools.takewhile

`itertools.takewhile` enables you to take items from a sequence until a condition first becomes `False`.

```
def is_even(x):
    return x % 2 == 0

lst = [0, 2, 4, 12, 18, 13, 14, 22, 23, 44]
result = list(itertools.takewhile(is_even, lst))

print(result)
```

This outputs `[0, 2, 4, 12, 18]`.

Note that, the first number that violates the predicate (i.e.: the function returning a Boolean value) `is_even` is, 13. Once `takewhile` encounters a value that produces `False` for the given predicate, it breaks out.

The **output produced** by `takewhile` is similar to the output generated from the code below.

```
def takewhile(predicate, iterable):
    for x in iterable:
        if predicate(x):
            yield x
        else:
            break
```

Note: The concatenation of results produced by `takewhile` and `dropwhile` produces the original iterable.

```
result = list(itertools.takewhile(is_even, lst)) + list(itertools.dropwhile(is_even, lst))
```

Section 53.7: itertools.permutations

`itertools.permutations` returns a generator with successive `r`-length permutations of elements in the iterable.

```
a = [1, 2, 3]
list(itertools.permutations(a))
# [(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]

list(itertools.permutations(a, 2))
[(1, 2), (1, 3), (2, 1), (2, 3), (3, 1), (3, 2)]
```


if the list `a` has duplicate elements, the resulting permutations will have duplicate elements, you can use `set` to get unique permutations:

```
a = [1,2,1]
list(itertools.permutations(a))
# [(1, 2, 1), (1, 1, 2), (2, 1, 1), (2, 1, 1), (1, 1, 2), (1, 2, 1)]

set(itertools.permutations(a))
# {(1, 1, 2), (1, 2, 1), (2, 1, 1)}
```

Section 53.8: itertools.repeat

Repeat something `n` times:

```
>>> import itertools
>>> for i in itertools.repeat('over-and-over', 3):
...     print(i)
over-and-over
over-and-over
over-and-over
```

Section 53.9: Get an accumulated sum of numbers in an iterable

Python 3.x Version ≥ 3.2

`accumulate` yields a cumulative sum (or product) of numbers.

```
>>> import itertools as it
>>> import operator

>>> list(it.accumulate([1,2,3,4,5]))
[1, 3, 6, 10, 15]

>>> list(it.accumulate([1,2,3,4,5], func=operator.mul))
[1, 2, 6, 24, 120]
```

Section 53.10: Cycle through elements in an iterator

`cycle` is an infinite iterator.

```
>>> import itertools as it
>>> it.cycle('ABCD')
A B C D A B C D A B C D ...
```

Therefore, take care to give boundaries when using this to avoid an infinite loop. Example:

```
>>> # Iterate over each element in cycle for a fixed range
>>> cycle_iterator = it.cycle('abc123')
>>> [next(cycle_iterator) for i in range(0, 10)]
['a', 'b', 'c', '1', '2', '3', 'a', 'b', 'c', '1']
```

Section 53.11: itertools.product

This function lets you iterate over the Cartesian product of a list of iterables.

For example,

```
for x, y in itertools.product(xrange(10), xrange(10)):
    print x, y
```

is equivalent to

```
for x in xrange(10):
    for y in xrange(10):
        print x, y
```

Like all python functions that accept a variable number of arguments, we can pass a list to `itertools.product` for unpacking, with the `*` operator.

Thus,

```
its = [xrange(10)] * 2
for x,y in itertools.product(*its):
    print x, y
```

produces the same results as both of the previous examples.

```
>>> from itertools import product
>>> a=[1,2,3,4]
>>> b=['a','b','c']
>>> product(a,b)
<itertools.product object at 0x000000002712F78>
>>> for i in product(a,b):
...     print i
...
(1, 'a')
(1, 'b')
(1, 'c')
(2, 'a')
(2, 'b')
(2, 'c')
(3, 'a')
(3, 'b')
(3, 'c')
(4, 'a')
(4, 'b')
(4, 'c')
```

Section 53.12: itertools.count

Introduction:

This simple function generates infinite series of numbers. For example...

```
for number in itertools.count():
    if number > 20:
        break
    print(number)
```

Note that we must break or it prints forever!

Output:

```
0
1
2
3
4
5
6
7
8
9
10
```

Arguments:

`count()` takes two arguments, start and step:

```
for number in itertools.count(start=10, step=4):
    print(number)
    if number > 20:
        break
```

Output:

```
10
14
18
22
```

Section 53.13: Chaining multiple iterators together

Use `itertools.chain` to create a single generator which will yield the values from several generators in sequence.

```
from itertools import chain
a = (x for x in ['1', '2', '3', '4'])
b = (x for x in ['x', 'y', 'z'])
' '.join(chain(a, b))
```

Results in:

```
'1 2 3 4 x y z'
```

As an alternate constructor, you can use the classmethod `chain.from_iterable` which takes as its single parameter an iterable of iterables. To get the same result as above:

```
' '.join(chain.from_iterable([a, b]))
```

While `chain` can take an arbitrary number of arguments, `chain.from_iterable` is the only way to chain an *infinite* number of iterables.

PNP Ch 67 | Generators [companion]

Chapter 67: Generators

Generators are lazy iterators created by generator functions (using **yield**) or generator expressions (using `(an_expression for x in an_iterator)`).

Section 67.1: Introduction

Generator expressions are similar to list, dictionary and set comprehensions, but are enclosed with parentheses. The parentheses do not have to be present when they are used as the sole argument for a function call.

```
expression = (x**2 for x in range(10))
```

This example generates the 10 first perfect squares, including 0 (in which $x = 0$).

Generator functions are similar to regular functions, except that they have one or more **yield** statements in their body. Such functions cannot **return** any values (however empty **returns** are allowed if you want to stop the generator early).

```
def function():
    for x in range(10):
        yield x**2
```

This generator function is equivalent to the previous generator expression, it outputs the same.

Note: all generator expressions have their own *equivalent* functions, but not vice versa.

A generator expression can be used without parentheses if both parentheses would be repeated otherwise:

```
sum(i for i in range(10) if i % 2 == 0)    #Output: 20
any(x = 0 for x in foo)                   #Output: True or False depending on foo
type(a > b for a in foo if a % 2 == 1)    #Output: <class 'generator'>
```

Instead of:

```
sum((i for i in range(10) if i % 2 == 0))
any((x = 0 for x in foo))
type((a > b for a in foo if a % 2 == 1))
```

But not:

```
fooFunction(i for i in range(10) if i % 2 == 0, foo, bar)
return x = 0 for x in foo
barFunction(baz, a > b for a in foo if a % 2 == 1)
```

Calling a generator function produces a **generator object**, which can later be iterated over. Unlike other types of iterators, generator objects may only be traversed once.

```
g1 = function()
print(g1) # Out: <generator object function at 0x1012e1888>
```

Notice that a generator's body is **not** immediately executed: when you call `function()` in the example above, it immediately returns a generator object, without executing even the first print statement. This allows generators to consume less memory than functions that return a list, and it allows creating generators that produce infinitely long sequences.

For this reason, generators are often used in data science, and other contexts involving large amounts of data. Another advantage is that other code can immediately use the values yielded by a generator, without waiting for the complete sequence to be produced.

However, if you need to use the values produced by a generator more than once, and if generating them costs more than storing, it may be better to store the yielded values as a **list** than to re-generate the sequence. See 'Resetting a generator' below for more details.

Typically a generator object is used in a loop, or in any function that requires an iterable:

```
for x in g1:
    print("Received", x)

# Output:
# Received 0
# Received 1
# Received 4
# Received 9
# Received 16
# Received 25
# Received 36
# Received 49
# Received 64
# Received 81

arr1 = list(g1)
# arr1 = [], because the loop above already consumed all the values.
g2 = function()
arr2 = list(g2) # arr2 = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Since generator objects are iterators, one can iterate over them manually using the [next\(\)](#) function. Doing so will return the yielded values one by one on each subsequent invocation.

Under the hood, each time you call `next()` on a generator, Python executes statements in the body of the generator function until it hits the next **yield** statement. At this point it returns the argument of the **yield** command, and remembers the point where that happened. Calling `next()` once again will resume execution from that point and continue until the next **yield** statement.

If Python reaches the end of the generator function without encountering any more **yields**, a **StopIteration** exception is raised (this is normal, all iterators behave in the same way).

```
g3 = function()
a = next(g3) # a becomes 0
b = next(g3) # b becomes 1
c = next(g3) # c becomes 2
...
j = next(g3) # Raises StopIteration, j remains undefined
```

Note that in Python 2 generator objects had `.next()` methods that could be used to iterate through the yielded values manually. In Python 3 this method was replaced with the `__next__()` standard for all iterators.

Resetting a generator

Remember that you can only iterate through the objects generated by a generator *once*. If you have already iterated through the objects in a script, any further attempt do so will yield **None**.

If you need to use the objects generated by a generator more than once, you can either define the generator

function again and use it a second time, or, alternatively, you can store the output of the generator function in a list on first use. Re-defining the generator function will be a good option if you are dealing with large volumes of data, and storing a list of all data items would take up a lot of disc space. Conversely, if it is costly to generate the items initially, you may prefer to store the generated items in a list so that you can re-use them.

Section 67.2: Infinite sequences

Generators can be used to represent infinite sequences:

```
def integers_starting_from(n):
    while True:
        yield n
        n += 1

natural_numbers = integers_starting_from(1)
```

Infinite sequence of numbers as above can also be generated with the help of [itertools.count](#). The above code could be written as below

```
natural_numbers = itertools.count(1)
```

You can use generator comprehensions on infinite generators to produce new generators:

```
multiples_of_two = (x * 2 for x in natural_numbers)
multiples_of_three = (x for x in natural_numbers if x % 3 == 0)
```

Be aware that an infinite generator does not have an end, so passing it to any function that will attempt to consume the generator entirely will have **dire consequences**:

```
list(multiples_of_two) # will never terminate, or raise an OS-specific error
```

Instead, use list/set comprehensions with [range](#) (or [xrange](#) for python < 3.0):

```
first_five_multiples_of_three = [next(multiples_of_three) for _ in range(5)]
# [3, 6, 9, 12, 15]
```

or use [itertools.islice\(\)](#) to slice the iterator to a subset:

```
from itertools import islice
multiples_of_four = (x * 4 for x in integers_starting_from(1))
first_five_multiples_of_four = list(islice(multiples_of_four, 5))
# [4, 8, 12, 16, 20]
```

Note that the original generator is updated too, just like all other generators coming from the same "root":

```
next(natural_numbers) # yields 16
next(multiples_of_two) # yields 34
next(multiples_of_four) # yields 24
```

An infinite sequence can also be iterated with a **for**-loop. Make sure to include a conditional **break** statement so that the loop would terminate eventually:

```
for idx, number in enumerate(multiples_of_two):
    print(number)
    if idx == 9:
```

```
break # stop after taking the first 10 multiplies of two
```

Classic example - Fibonacci numbers

```
import itertools

def fibonacci():
    a, b = 1, 1
    while True:
        yield a
        a, b = b, a + b

first_ten_fibs = list(itertools.islice(fibonacci(), 10))
# [1, 1, 2, 3, 5, 8, 13, 21, 34, 55]

def nth_fib(n):
    return next(itertools.islice(fibonacci(), n - 1, n))

ninety_nineth_fib = nth_fib(99) # 354224848179261915075
```

Section 67.3: Sending objects to a generator

In addition to receiving values from a generator, it is possible to *send* an object to a generator using the `send()` method.

```
def accumulator():
    total = 0
    value = None
    while True:
        # receive sent value
        value = yield total
        if value is None: break
        # aggregate values
        total += value

generator = accumulator()

# advance until the first "yield"
next(generator) # 0

# from this point on, the generator aggregates values
generator.send(1) # 1
generator.send(10) # 11
generator.send(100) # 111
# ...

# Calling next(generator) is equivalent to calling generator.send(None)
next(generator) # StopIteration
```

What happens here is the following:

- When you first call `next(generator)`, the program advances to the first **yield** statement, and returns the value of `total` at that point, which is 0. The execution of the generator suspends at this point.
- When you then call `generator.send(x)`, the interpreter takes the argument `x` and makes it the return value of the last **yield** statement, which gets assigned to `value`. The generator then proceeds as usual, until it yields the next value.
- When you finally call `next(generator)`, the program treats this as if you're sending `None` to the generator. There is nothing special about `None`, however, this example uses `None` as a special value to ask the generator to stop.

Section 67.4: Yielding all values from another iterable

Python 3.x Version $\geq$ 3.3

Use **yield from** if you want to yield all values from another iterable:

```
def foob(x):
    yield from range(x * 2)
    yield from range(2)

list(foob(5)) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1]
```

This works with generators as well.

```
def fibto(n):
    a, b = 1, 1
    while True:
        if a >= n: break
        yield a
        a, b = b, a + b

def usefib():
    yield from fibto(10)
    yield from fibto(20)

list(usefib()) # [1, 1, 2, 3, 5, 8, 1, 1, 2, 3, 5, 8, 13]
```

Section 67.5: Iteration

A generator object supports the *iterator protocol*. That is, it provides a `next()` method (`__next__()` in Python 3.x), which is used to step through its execution, and its `__iter__` method returns itself. This means that a generator can be used in any language construct which supports generic iterable objects.

```
# naive partial implementation of the Python 2.x xrange()
def xrange(n):
    i = 0
    while i < n:
        yield i
        i += 1

# looping
for i in xrange(10):
    print(i) # prints the values 0, 1, ..., 9

# unpacking
a, b, c = xrange(3) # 0, 1, 2

# building a list
l = list(xrange(10)) # [0, 1, ..., 9]
```

Section 67.6: The next() function

The `next()` built-in is a convenient wrapper which can be used to receive a value from any iterator (including a generator iterator) and to provide a default value in case the iterator is exhausted.

```
def nums():
    yield 1
```

```

    yield 2
    yield 3
generator = nums()

next(generator, None) # 1
next(generator, None) # 2
next(generator, None) # 3
next(generator, None) # None
next(generator, None) # None
# ...

```

The syntax is `next(iterator[, default])`. If iterator ends and a default value was passed, it is returned. If no default was provided, `StopIteration` is raised.

Section 67.7: Coroutines

Generators can be used to implement coroutines:

```

# create and advance generator to the first yield
def coroutine(func):
    def start(*args,**kwargs):
        cr = func(*args,**kwargs)
        next(cr)
        return cr
    return start

# example coroutine
@coroutine
def adder(sum = 0):
    while True:
        x = yield sum
        sum += x

# example use
s = adder()
s.send(1) # 1
s.send(2) # 3

```

Coroutines are commonly used to implement state machines, as they are primarily useful for creating single-method procedures that require a state to function properly. They operate on an existing state and return the value obtained on completion of the operation.

Section 67.8: Refactoring list-building code

Suppose you have complex code that creates and returns a list by starting with a blank list and repeatedly appending to it:

```

def create():
    result = []
    # logic here...
    result.append(value) # possibly in several places
    # more logic...
    return result # possibly in several places

values = create()

```

When it's not practical to replace the inner logic with a list comprehension, you can turn the entire function into a generator in-place, and then collect the results:

```
def create_gen():
    # logic...
    yield value
    # more logic
    return # not needed if at the end of the function, of course

values = list(create_gen())
```

If the logic is recursive, use **yield from** to include all the values from the recursive call in a "flattened" result:

```
def preorder_traversal(node):
    yield node.value
    for child in node.children:
        yield from preorder_traversal(child)
```

Section 67.9: Yield with recursion: recursively listing all files in a directory

First, import the libraries that work with files:

```
from os import listdir
from os.path import isfile, join, exists
```

A helper function to read only files from a directory:

```
def get_files(path):
    for file in listdir(path):
        full_path = join(path, file)
        if isfile(full_path):
            if exists(full_path):
                yield full_path
```

Another helper function to get only the subdirectories:

```
def get_directories(path):
    for directory in listdir(path):
        full_path = join(path, directory)
        if not isfile(full_path):
            if exists(full_path):
                yield full_path
```

Now use these functions to recursively get all files within a directory and all its subdirectories (using generators):

```
def get_files_recursive(directory):
    for file in get_files(directory):
        yield file
    for subdirectory in get_directories(directory):
        for file in get_files_recursive(subdirectory): # here the recursive call
            yield file
```

This function can be simplified using **yield from**:

```
def get_files_recursive(directory):
    yield from get_files(directory)
    for subdirectory in get_directories(directory):
        yield from get_files_recursive(subdirectory)
```

Section 67.10: Generator expressions

It's possible to create generator iterators using a comprehension-like syntax.

```
generator = (i * 2 for i in range(3))

next(generator) # 0
next(generator) # 2
next(generator) # 4
next(generator) # raises StopIteration
```

If a function doesn't necessarily need to be passed a list, you can save on characters (and improve readability) by placing a generator expression inside a function call. The parenthesis from the function call implicitly make your expression a generator expression.

```
sum(i ** 2 for i in range(4)) # 0^2 + 1^2 + 2^2 + 3^2 = 0 + 1 + 4 + 9 = 14
```

Additionally, you will save on memory because instead of loading the entire list you are iterating over ([0, 1, 2, 3] in the above example), the generator allows Python to use values as needed.

Section 67.11: Using a generator to find Fibonacci Numbers

A practical use case of a generator is to iterate through values of an infinite series. Here's an example of finding the first ten terms of the [Fibonacci Sequence](#).

```
def fib(a=0, b=1):
    """Generator that yields Fibonacci numbers. `a` and `b` are the seed values"""
    while True:
        yield a
        a, b = b, a + b

f = fib()
print(', '.join(str(next(f)) for _ in range(10)))
```

0, 1, 1, 2, 3, 5, 8, 13, 21, 34

Section 67.12: Searching

The next function is useful even without iterating. Passing a generator expression to next is a quick way to search for the first occurrence of an element matching some predicate. Procedural code like

```
def find_and_transform(sequence, predicate, func):
    for element in sequence:
        if predicate(element):
            return func(element)
    raise ValueError

item = find_and_transform(my_sequence, my_predicate, my_func)
```

can be replaced with:

```
item = next(my_func(x) for x in my_sequence if my_predicate(x))
# StopIteration will be raised if there are no matches; this exception can
# be caught and transformed, if desired.
```

For this purpose, it may be desirable to create an alias, such as `first = next`, or a wrapper function to convert the exception:

```
def first(generator):  
    try:  
        return next(generator)  
    except StopIteration:  
        raise ValueError
```

Section 67.13: Iterating over generators in parallel

To iterate over several generators in parallel, use the `zip` builtin:

```
for x, y in zip(a,b):  
    print(x,y)
```

Results in:

```
1 x  
2 y  
3 z
```

In python 2 you should use `itertools.izip` instead. Here we can also see that the all the `zip` functions yield tuples.

Note that `zip` will stop iterating as soon as one of the iterables runs out of items. If you'd like to iterate for as long as the longest iterable, use `itertools.zip_longest()`.

PHASE 05 · Classes & Protocols

~3 weeks · 5 chapters

Build proper ML components: models, datasets, trainers, evaluation loops.

Chapters in this phase:

- FP Ch 9 | A Pythonic Object [= plan Ch 11]
- FP Ch 10 | Sequence Hacking, Hashing & Slicing [= plan Ch 12]
- FP Ch 11 | Interfaces: From Protocols to ABCs [= plan Ch 13]
- FP Ch 15 | Context Managers & else Blocks [= plan Ch 18]
- PNP Ch 38 | Classes [companion]
- PNP Ch 77 | Context Managers [companion]
- PNP Ch 87 | Type Hints [companion · FP Ch 15 substitute]

FP Ch 9 | A Pythonic Object [= plan Ch 11]

CHAPTER 9

A Pythonic Object

Never, ever use two leading underscores. This is annoyingly private.¹

— Ian Bicking

Creator of pip, virtualenv, Paste and many other projects

Thanks to the Python data model, your user-defined types can behave as naturally as the built-in types. And this can be accomplished without inheritance, in the spirit of *duck typing*: you just implement the methods needed for your objects to behave as expected.

In previous chapters, we presented the structure and behavior of many built-in objects. We will now build user-defined classes that behave as real Python objects.

This chapter starts where [Chapter 1](#) ended, by showing how to implement several special methods that are commonly seen in Python objects of many different types.

In this chapter, we will see how to:

- Support the built-in functions that produce alternative object representations (e.g., `repr()`, `bytes()`, etc).
- Implement an alternative constructor as a class method.
- Extend the format mini-language used by the `format()` built-in and the `str.format()` method.
- Provide read-only access to attributes.
- Make an object hashable for use in sets and as dict keys.
- Save memory with the use of `__slots__`.

1. From the [Paste Style Guide](#).

We'll do all that as we develop a simple two-dimensional Euclidean vector type.

The evolution of the example will be paused to discuss two conceptual topics:

- How and when to use the `@classmethod` and `@staticmethod` decorators.
- Private and protected attributes in Python: usage, conventions, and limitations.

Let's get started with the object representation methods.

Object Representations

Every object-oriented language has at least one standard way of getting a string representation from any object. Python has two:

`repr()`

Return a string representing the object as the developer wants to see it.

`str()`

Return a string representing the object as the user wants to see it.

As you know, we implement the special methods `__repr__` and `__str__` to support `repr()` and `str()`.

There are two additional special methods to support alternative representations of objects: `__bytes__` and `__format__`. The `__bytes__` method is analogous to `__str__`: it's called by `bytes()` to get the object represented as a byte sequence. Regarding `__format__`, both the built-in function `format()` and the `str.format()` method call it to get string displays of objects using special formatting codes. We'll cover `__bytes__` in the next example, and `__format__` after that.



If you're coming from Python 2, remember that in Python 3 `__repr__`, `__str__`, and `__format__` must always return Unicode strings (type `str`). Only `__bytes__` is supposed to return a byte sequence (type `bytes`).

Vector Class Redux

In order to demonstrate the many methods used to generate object representations, we'll use a `Vector2d` class similar to the one we saw in [Chapter 1](#). We will build on it in this and future sections. [Example 9-1](#) illustrates the basic behavior we expect from a `Vector2d` instance.

Example 9-1. Vector2d instances have several representations

```
>>> v1 = Vector2d(3, 4)
>>> print(v1.x, v1.y) ❶
```

```

3.0 4.0
>>> x, y = v1 ❷
>>> x, y
(3.0, 4.0)
>>> v1 ❸
Vector2d(3.0, 4.0)
>>> v1_clone = eval(repr(v1)) ❹
>>> v1 == v1_clone ❺
True
>>> print(v1) ❻
(3.0, 4.0)
>>> octets = bytes(v1) ❼
>>> octets
b'd\\x00\\x00\\x00\\x00\\x00\\x00\\x08@\\x00\\x00\\x00\\x00\\x00\\x00\\x10@'
>>> abs(v1) ❽
5.0
>>> bool(v1), bool(Vector2d(0, 0)) ❾
(True, False)

```

- ❶ The components of a `Vector2d` can be accessed directly as attributes (no getter method calls).
- ❷ A `Vector2d` can be unpacked to a tuple of variables.
- ❸ The `repr` of a `Vector2d` emulates the source code for constructing the instance.
- ❹ Using `eval` here shows that the `repr` of a `Vector2d` is a faithful representation of its constructor call.²
- ❺ `Vector2d` supports comparison with `==`; this is useful for testing.
- ❻ `print` calls `str`, which for `Vector2d` produces an ordered pair display.
- ❼ `bytes` uses the `__bytes__` method to produce a binary representation.
- ❽ `abs` uses the `__abs__` method to return the magnitude of the `Vector2d`.
- ❾ `bool` uses the `__bool__` method to return `False` for a `Vector2d` of zero magnitude or `True` otherwise.

`Vector2d` from [Example 9-1](#) is implemented in `vector2d_v0.py` ([Example 9-2](#)). The code is based on [Example 1-2](#), but the infix operators will be implemented in [Chapter 13](#)—except for `==` (which is useful for testing). At this point, `Vector2d` uses several special methods to provide operations that a Pythonista expects in a well-designed object.

Example 9-2. `vector2d_v0.py`: methods so far are all special methods

```

from array import array
import math

```

2. I used `eval` to clone the object here just to make a point about `repr`; to clone an instance, the `copy.copy` function is safer and faster.

```

class Vector2d:
    typecode = 'd' ❶

    def __init__(self, x, y):
        self.x = float(x) ❷
        self.y = float(y)

    def __iter__(self):
        return (i for i in (self.x, self.y)) ❸

    def __repr__(self):
        class_name = type(self).__name__
        return '{}{!r}, {!r}'.format(class_name, *self) ❹

    def __str__(self):
        return str(tuple(self)) ❺

    def __bytes__(self):
        return (bytes([ord(self.typecode)]) + ❻
                bytes(array(self.typecode, self))) ❼

    def __eq__(self, other):
        return tuple(self) == tuple(other) ❽

    def __abs__(self):
        return math.hypot(self.x, self.y) ❾

    def __bool__(self):
        return bool(abs(self)) ❿

```

- ❶ typecode is a class attribute we'll use when converting Vector2d instances to/from bytes.
- ❷ Converting x and y to float in `__init__` catches errors early, which is helpful in case Vector2d is called with unsuitable arguments.
- ❸ `__iter__` makes a Vector2d iterable; this is what makes unpacking work (e.g, `x, y = my_vector`). We implement it simply by using a generator expression to yield the components one after the other.³
- ❹ `__repr__` builds a string by interpolating the components with `{!r}` to get their repr; because Vector2d is iterable, `*self` feeds the x and y components to format.
- ❺ From an iterable Vector2d, it's easy to build a tuple for display as an ordered pair.

3. This line could also be written as `yield self.x; yield self.y`. I have a lot more to say about the `__iter__` special method, generator expressions, and the `yield` keyword in [Chapter 14](#).

- ⑥ To generate bytes, we convert the typecode to bytes and concatenate...
- ⑦ ...bytes converted from an array built by iterating over the instance.
- ⑧ To quickly compare all components, build tuples out of the operands. This works for operands that are instances of `Vector2d`, but has issues. See the following warning.
- ⑨ The magnitude is the length of the hypotenuse of the triangle formed by the x and y components.
- ⑩ `__bool__` uses `abs(self)` to compute the magnitude, then converts it to `bool`, so `0.0` becomes `False`, `nonzero` is `True`.



Method `__eq__` in [Example 9-2](#) works for `Vector2d` operands but also returns `True` when comparing `Vector2d` instances to other iterables holding the same numeric values (e.g., `Vector(3, 4) == [3, 4]`). This may be considered a feature or a bug. Further discussion needs to wait until [Chapter 13](#), when we cover operator overloading.

We have a fairly complete set of basic methods, but one obvious operation is missing: rebuilding a `Vector2d` from the binary representation produced by `bytes()`.

An Alternative Constructor

Because we can export a `Vector2d` as bytes, naturally we need a method that imports a `Vector2d` from a binary sequence. Looking at the standard library for inspiration, we find that `array.array` has a class method named `.frombytes` that suits our purpose—we saw it in [“Arrays” on page 48](#). We adopt its name and use its functionality in a class method for `Vector2d` in `vector2d_v1.py` ([Example 9-3](#)).

Example 9-3. Part of `vector2d_v1.py`: this snippet shows only the `frombytes` class method, added to the `Vector2d` definition in `vector2d_v0.py` ([Example 9-2](#))

```
@classmethod ①
def frombytes(cls, octets): ②
    typecode = chr(octets[0]) ③
    memv = memoryview(octets[1:]).cast(typecode) ④
    return cls(*memv) ⑤
```

- ① Class method is modified by the `classmethod` decorator.
- ② No `self` argument; instead, the class itself is passed as `cls`.
- ③ Read the typecode from the first byte.

- ④ Create a `memoryview` from the octets binary sequence and use the `typecode` to cast it.⁴
- ⑤ Unpack the `memoryview` resulting from the cast into the pair of arguments needed for the constructor.

Because we just used a `classmethod` decorator, and it is very Python-specific, let's have a word about it.

classmethod Versus staticmethod

The `classmethod` decorator is not mentioned in the Python tutorial, and neither is `staticmethod`. Anyone who has learned OO in Java may wonder why Python has both of these decorators and not just one of them.

Let's start with `classmethod`. [Example 9-3](#) shows its use: to define a method that operates on the class and not on instances. `classmethod` changes the way the method is called, so it receives the class itself as the first argument, instead of an instance. Its most common use is for alternative constructors, like `frombytes` in [Example 9-3](#). Note how the last line of `frombytes` actually uses the `cls` argument by invoking it to build a new instance: `cls(*memv)`. By convention, the first parameter of a class method should be named `cls` (but Python doesn't care how it's named).

In contrast, the `staticmethod` decorator changes a method so that it receives no special first argument. In essence, a static method is just like a plain function that happens to live in a class body, instead of being defined at the module level. [Example 9-4](#) contrasts the operation of `classmethod` and `staticmethod`.

Example 9-4. Comparing behaviors of `classmethod` and `staticmethod`

```
>>> class Demo:
...     @classmethod
...     def klassmeth(*args):
...         return args # ①
...     @staticmethod
...     def statmeth(*args):
...         return args # ②
...
>>> Demo.klassmeth() # ③
(<class '__main__.Demo'>,)
>>> Demo.klassmeth('spam')
(<class '__main__.Demo'>, 'spam')
>>> Demo.statmeth() # ④
()
```

4. We had a brief introduction to `memoryview`, explaining its `.cast` method in “Memory Views” on page 51.

```
>>> Demo.statmeth('spam')
('spam',)
```

- ❶ `klassmeth` just returns all positional arguments.
- ❷ `statmeth` does the same.
- ❸ No matter how you invoke it, `Demo.klassmeth` receives the `Demo` class as the first argument.
- ❹ `Demo.statmeth` behaves just like a plain old function.



The `classmethod` decorator is clearly useful, but I’ve never seen a compelling use case for `staticmethod`. If you want to define a function that does not interact with the class, just define it in the module. Maybe the function is closely related even if it never touches the class, so you want to them nearby in the code. Even so, defining the function right before or after the class in the same module is close enough for all practical purposes.⁵

Now that we’ve seen what `classmethod` is good for (and that `staticmethod` is not very useful), let’s go back to the issue of object representation and see how to support formatted output.

Formatted Displays

The `format()` built-in function and the `str.format()` method delegate the actual formatting to each type by calling their `__format__(format_spec)` method. The `format_spec` is a formatting specifier, which is either:

- The second argument in `format(my_obj, format_spec)`, or
- Whatever appears after the colon in a replacement field delimited with `{}` inside a format string used with `str.format()`

For example:

```
>>> brl = 1/2.43 # BRL to USD currency conversion rate
>>> brl
0.4115226337448559
>>> format(brl, '0.4f') # ❶
'0.4115'
```

5. Leonardo Rochaël, one of the technical reviewers of this book disagrees with my low opinion of `staticmethod`, and recommends the blog post “[The Definitive Guide on How to Use Static, Class or Abstract Methods in Python](#)” by Julien Danjou as a counter-argument. Danjou’s post is very good; I do recommend it. But it wasn’t enough to change my mind about `staticmethod`. You’ll have to decide for yourself.

```
>>> '1 BRL = {rate:0.2f} USD'.format(rate=brl) # ❷
'1 BRL = 0.41 USD'
```

- ❶ Formatting specifier is '0.4f'.
- ❷ Formatting specifier is '0.2f'. The 'rate' substring in the replacement field is called the field name. It's unrelated to the formatting specifier, but determines which argument of `.format()` goes into that replacement field.

The second callout makes an important point: a format string such as '{0.mass:5.3e}' actually uses two separate notations. The '0.mass' to the left of the colon is the `field_name` part of the replacement field syntax; the '5.3e' after the colon is the formatting specifier. The notation used in the formatting specifier is called the **Format Specification Mini-Language**.



If `format()` and `str.format()` are new to you, classroom experience has shown that it's best to study the `format()` function first, which uses just the **Format Specification Mini-Language**. After you get the gist of that, read **Format String Syntax** to learn about the `{:}` replacement field notation, used in the `str.format()` method (including the `!s`, `!r`, and `!a` conversion flags).

A few built-in types have their own presentation codes in the Format Specification Mini-Language. For example—among several other codes—the `int` type supports `b` and `x` for base 2 and base 16 output, respectively, while `float` implements `f` for a fixed-point display and `%` for a percentage display:

```
>>> format(42, 'b')
'101010'
>>> format(2/3, '.1%')
'66.7%'
```

The Format Specification Mini-Language is extensible because each class gets to interpret the `format_spec` argument as it likes. For instance, the classes in the `datetime` module use the same format codes in the `strftime()` functions and in their `__format__` methods. Here are a couple examples using the `format()` built-in and the `str.format()` method:

```
>>> from datetime import datetime
>>> now = datetime.now()
>>> format(now, '%H:%M:%S')
'18:49:05'
>>> "It's now {:I:%M %p}.".format(now)
"It's now 06:49 PM"
```

If a class has no `__format__`, the method inherited from `object` returns `str(my_object)`. Because `Vector2d` has a `__str__`, this works:

```
>>> v1 = Vector2d(3, 4)
>>> format(v1)
'(3.0, 4.0)'
```

However, if you pass a format specifier, `object.__format__` raises `TypeError`:

```
>>> format(v1, '.3f')
Traceback (most recent call last):
...
TypeError: non-empty format string passed to object.__format__
```

We will fix that by implementing our own format mini-language. The first step will be to assume the format specifier provided by the user is intended to format each float component of the vector. This is the result we want:

```
>>> v1 = Vector2d(3, 4)
>>> format(v1)
'(3.0, 4.0)'
```

```
>>> format(v1, '.2f')
'(3.00, 4.00)'
```

```
>>> format(v1, '.3e')
'(3.000e+00, 4.000e+00)'
```

Example 9-5 implements `__format__` to produce the displays just shown.

Example 9-5. `Vector2d.format` method, take #1

```
# inside the Vector2d class

def __format__(self, fmt_spec=''):
    components = (format(c, fmt_spec) for c in self) # ❶
    return '({}, {})'.format(*components) # ❷
```

- ❶ Use the format built-in to apply the `fmt_spec` to each vector component, building an iterable of formatted strings.
- ❷ Plug the formatted strings in the formula `'(x, y)'`.

Now let's add a custom formatting code to our mini-language: if the format specifier ends with a `'p'`, we'll display the vector in polar coordinates: `<r, θ>`, where `r` is the magnitude and `θ` (theta) is the angle in radians. The rest of the format specifier (whatever comes before the `'p'`) will be used as before.



When choosing the letter for the custom format code I avoided overlapping with codes used by other types. In **Format Specification Mini-Language** we see that integers use the codes `'bcdxXn'`, floats use `'eEfFgGn%'`, and strings use `'s'`. So I picked `'p'` for polar coordinates. Because each class interprets these codes independently, reusing a code letter in a custom format for a new type is not an error, but may be confusing to users.

To generate polar coordinates we already have the `__abs__` method for the magnitude, and we'll code a simple angle method using the `math.atan2()` function to get the angle. This is the code:

```
# inside the Vector2d class

def angle(self):
    return math.atan2(self.y, self.x)
```

With that, we can enhance our `__format__` to produce polar coordinates. See [Example 9-6](#).

Example 9-6. Vector2d.format method, take #2, now with polar coordinates

```
def __format__(self, fmt_spec=''):
    if fmt_spec.endswith('p'): ❶
        fmt_spec = fmt_spec[:-1] ❷
        coords = (abs(self), self.angle()) ❸
        outer_fmt = '<{}, {}>' ❹
    else:
        coords = self ❺
        outer_fmt = '({}, {})' ❻
    components = (format(c, fmt_spec) for c in coords) ❼
    return outer_fmt.format(*components) ❽
```

- ❶ Format ends with 'p': use polar coordinates.
- ❷ Remove 'p' suffix from `fmt_spec`.
- ❸ Build tuple of polar coordinates: (magnitude, angle).
- ❹ Configure outer format with angle brackets.
- ❺ Otherwise, use `x`, `y` components of `self` for rectangular coordinates.
- ❻ Configure outer format with parentheses.
- ❼ Generate iterable with components as formatted strings.
- ❽ Plug formatted strings into outer format.

With [Example 9-6](#), we get results similar to these:

```
>>> format(Vector2d(1, 1), 'p')
'<1.4142135623730951, 0.7853981633974483>'
>>> format(Vector2d(1, 1), '.3ep')
'<1.414e+00, 7.854e-01>'
>>> format(Vector2d(1, 1), '0.5fp')
'<1.41421, 0.78540>'
```

As this section shows, it's not hard to extend the format specification mini-language to support user-defined types.

Now let's move to a subject that's not just about appearances: we will make our `Vector2d` hashable, so we can build sets of vectors, or use them as `dict` keys. But before we can do that, we must make vectors immutable. We'll do what it takes next.

A Hashable Vector2d

As defined, so far our `Vector2d` instances are unhashable, so we can't put them in a set:

```
>>> v1 = Vector2d(3, 4)
>>> hash(v1)
Traceback (most recent call last):
...
TypeError: unhashable type: 'Vector2d'
>>> set([v1])
Traceback (most recent call last):
...
TypeError: unhashable type: 'Vector2d'
```

To make a `Vector2d` hashable, we must implement `__hash__` (`__eq__` is also required, and we already have it). We also need to make vector instances immutable, as we've seen in “What Is Hashable?” on page 65.

Right now, anyone can do `v1.x = 7` and there is nothing in the code to suggest that changing a `Vector2d` is forbidden. This is the behavior we want:

```
>>> v1.x, v1.y
(3.0, 4.0)
>>> v1.x = 7
Traceback (most recent call last):
...
AttributeError: can't set attribute
```

We'll do that by making the `x` and `y` components read-only properties in [Example 9-7](#).

Example 9-7. `vector2d_v3.py`: only the changes needed to make `Vector2d` immutable are shown here; see full listing in [Example 9-9](#)

```
class Vector2d:
    typecode = 'd'

    def __init__(self, x, y):
        self.__x = float(x) ❶
        self.__y = float(y)

    @property ❷
    def x(self): ❸
        return self.__x ❹

    @property ❺
    def y(self):
        return self.__y
```

```
def __iter__(self):
    return (i for i in (self.x, self.y)) ❹

# remaining methods follow (omitted in book listing)
```

- ❶ Use exactly two leading underscores (with zero or one trailing underscore) to make an attribute private.⁶
- ❷ The `@property` decorator marks the getter method of a property.
- ❸ The getter method is named after the public property it exposes: `x`.
- ❹ Just return `self.__x`.
- ❺ Repeat same formula for `y` property.
- ❻ Every method that just reads the `x`, `y` components can stay as they were, reading the public properties via `self.x` and `self.y` instead of the private attribute, so this listing omits the rest of the code for the class.



`Vector.x` and `Vector.y` are examples of read-only properties. Read/write properties will be covered in [Chapter 19](#), where we dive deeper into the `@property`.

Now that our vectors are reasonably immutable, we can implement the `__hash__` method. It should return an `int` and ideally take into account the hashes of the object attributes that are also used in the `__eq__` method, because objects that compare equal should have the same hash. The `__hash__` special method [documentation](#) suggests using the bitwise XOR operator (`^`) to mix the hashes of the components, so that's what we do. The code for our `Vector2d.__hash__` method is really simple, as shown in [Example 9-8](#).

Example 9-8. `vector2d_v3.py`: implementation of hash

```
# inside class Vector2d:

def __hash__(self):
    return hash(self.x) ^ hash(self.y)
```

With the addition of the `__hash__` method, we now have hashable vectors:

```
>>> v1 = Vector2d(3, 4)
>>> v2 = Vector2d(3.1, 4.2)
>>> hash(v1), hash(v2)
```

6. This is not how Ian Bicking would do it; recall the quote at the start of the chapter. The pros and cons of private attributes are the subject of the upcoming “[Private and “Protected” Attributes in Python](#)” on page 262.

```
(7, 384307168202284039)
>>> set([v1, v2])
{Vector2d(3.1, 4.2), Vector2d(3.0, 4.0)}
```



It's not strictly necessary to implement properties or otherwise protect the instance attributes to create a hashable type. Implementing `__hash__` and `__eq__` correctly is all it takes. But the hash value of an instance is never supposed to change, so this provides an excellent opportunity to talk about read-only properties.

If you are creating a type that has a sensible scalar numeric value, you may also implement the `__int__` and `__float__` methods, invoked by the `int()` and `float()` constructors—which are used for type coercion in some contexts. There's also a `__complex__` method to support the `complex()` built-in constructor. Perhaps `Vector2d` should provide `__complex__`, but I'll leave that as an exercise for you.

We have been working on `Vector2d` for a while, showing just snippets, so [Example 9-9](#) is a consolidated, full listing of `vector2d_v3.py`, including all the doctests I used when developing it.

Example 9-9. `vector2d_v3.py`: the full monty

```
"""
A two-dimensional vector class

>>> v1 = Vector2d(3, 4)
>>> print(v1.x, v1.y)
3.0 4.0
>>> x, y = v1
>>> x, y
(3.0, 4.0)
>>> v1
Vector2d(3.0, 4.0)
>>> v1_clone = eval(repr(v1))
>>> v1 == v1_clone
True
>>> print(v1)
(3.0, 4.0)
>>> octets = bytes(v1)
>>> octets
b'd|\x00|\x00|\x00|\x00|\x00|\x00|\x08@\x00|\x00|\x00|\x00|\x00|\x00|\x10@'
>>> abs(v1)
5.0
>>> bool(v1), bool(Vector2d(0, 0))
(True, False)
```

Test of ``.frombytes()`` class method:

```
>>> v1_clone = Vector2d.frombytes(bytes(v1))
>>> v1_clone
Vector2d(3.0, 4.0)
>>> v1 == v1_clone
True
```

Tests of ``format()`` with Cartesian coordinates:

```
>>> format(v1)
'(3.0, 4.0)'
>>> format(v1, '.2f')
'(3.00, 4.00)'
>>> format(v1, '.3e')
'(3.000e+00, 4.000e+00)'
```

Tests of the ``angle`` method::

```
>>> Vector2d(0, 0).angle()
0.0
>>> Vector2d(1, 0).angle()
0.0
>>> epsilon = 10**-8
>>> abs(Vector2d(0, 1).angle() - math.pi/2) < epsilon
True
>>> abs(Vector2d(1, 1).angle() - math.pi/4) < epsilon
True
```

Tests of ``format()`` with polar coordinates:

```
>>> format(Vector2d(1, 1), 'p') # doctest:+ELLIPSIS
'<1.414213..., 0.785398...>'
>>> format(Vector2d(1, 1), '.3ep')
'<1.414e+00, 7.854e-01>'
>>> format(Vector2d(1, 1), '0.5fp')
'<1.41421, 0.78540>'
```

Tests of ``x`` and ``y`` read-only properties:

```
>>> v1.x, v1.y
(3.0, 4.0)
>>> v1.x = 123
Traceback (most recent call last):
...
AttributeError: can't set attribute
```

Tests of hashing:

```
>>> v1 = Vector2d(3, 4)
>>> v2 = Vector2d(3.1, 4.2)
>>> hash(v1), hash(v2)
(7, 384307168202284039)
>>> len(set([v1, v2]))
2
```

```
"""
```

```
from array import array
import math
```

```
class Vector2d:
    typecode = 'd'

    def __init__(self, x, y):
        self.__x = float(x)
        self.__y = float(y)

    @property
    def x(self):
        return self.__x

    @property
    def y(self):
        return self.__y

    def __iter__(self):
        return (i for i in (self.x, self.y))

    def __repr__(self):
        class_name = type(self).__name__
        return '{}({!r}, {!r})'.format(class_name, *self)

    def __str__(self):
        return str(tuple(self))

    def __bytes__(self):
        return (bytes([ord(self.typecode)]) +
                bytes(array(self.typecode, self)))

    def __eq__(self, other):
        return tuple(self) == tuple(other)

    def __hash__(self):
        return hash(self.x) ^ hash(self.y)

    def __abs__(self):
        return math.hypot(self.x, self.y)

    def __bool__(self):
        return bool(abs(self))
```

```

def angle(self):
    return math.atan2(self.y, self.x)

def __format__(self, fmt_spec=''):
    if fmt_spec.endswith('p'):
        fmt_spec = fmt_spec[:-1]
        coords = (abs(self), self.angle())
        outer_fmt = '<{}, {}>'
    else:
        coords = self
        outer_fmt = '({}, {})'
    components = (format(c, fmt_spec) for c in coords)
    return outer_fmt.format(*components)

@classmethod
def frombytes(cls, octets):
    typecode = chr(octets[0])
    memv = memoryview(octets[1:]).cast(typecode)
    return cls(*memv)

```

To recap, in this and the previous sections, we saw some essential special methods that you may want to implement to have a full-fledged object. Of course, it is a bad idea to implement all of these methods if your application has no real use for them. Customers don't care if your objects are “Pythonic” or not.

As coded in [Example 9-9](#), `Vector2d` is a didactic example with a laundry list of special methods related to object representation, not a template for every user-defined class.

In the next section, we'll take a break from `Vector2d` to discuss the design and drawbacks of the private attribute mechanism in Python—the double-underscore prefix in `self.__x`.

Private and “Protected” Attributes in Python

In Python, there is no way to create private variables like there is with the `private` modifier in Java. What we have in Python is a simple mechanism to prevent accidental overwriting of a “private” attribute in a subclass.

Consider this scenario: someone wrote a class named `Dog` that uses a `mood` instance attribute internally, without exposing it. You need to subclass `Dog` as `Beagle`. If you create your own `mood` instance attribute without being aware of the name clash, you will clobber the `mood` attribute used by the methods inherited from `Dog`. This would be a pain to debug.

To prevent this, if you name an instance attribute in the form `__mood` (two leading underscores and zero or at most one trailing underscore), Python stores the name in the instance `__dict__` prefixed with a leading underscore and the class name, so in the

Dog class, `__mood` becomes `_Dog__mood`, and in Beagle it's `_Beagle__mood`. This language feature goes by the lovely name of *name mangling*.

Example 9-10 shows the result in the `Vector2d` class from **Example 9-7**.

Example 9-10. Private attribute names are “mangled” by prefixing the `_` and the class name

```
>>> v1 = Vector2d(3, 4)
>>> v1.__dict__
{'_Vector2d__y': 4.0, '_Vector2d__x': 3.0}
>>> v1._Vector2d__x
3.0
```

Name mangling is about safety, not security: it's designed to prevent accidental access and not intentional wrongdoing (**Figure 9-1** illustrates another safety device).

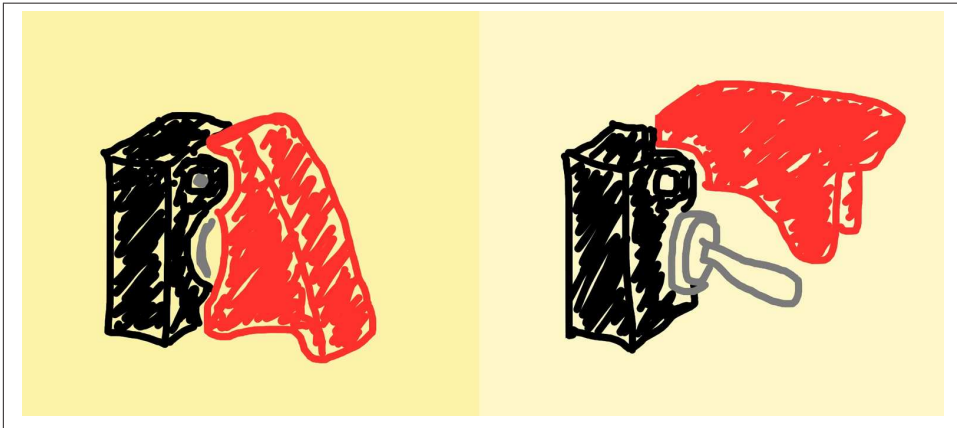


Figure 9-1. A cover on a switch is a safety device, not a security one: it prevents accidental activation, not malicious use

Anyone who knows how private names are mangled can read the private attribute directly, as the last line of **Example 9-10** shows—that's actually useful for debugging and serialization. They can also directly assign a value to a private component of a `Vector2d` by simply writing `v1._Vector__x = 7`. But if you are doing that in production code, you can't complain if something blows up.

The name mangling functionality is not loved by all Pythonistas, and neither is the skewed look of names written as `self.__x`. Some prefer to avoid this syntax and use just one underscore prefix to “protect” attributes by convention (e.g., `self._x`). Critics of the automatic double-underscore mangling suggest that concerns about accidental attribute clobbering should be addressed by naming conventions. This is the full quote from the prolific Ian Bicking, cited at the beginning of this chapter:

Never, ever use two leading underscores. This is annoyingly private. If name clashes are a concern, use explicit name mangling instead (e.g., `_MyThing_blahblah`). This is essentially the same thing as double-underscore, only it's transparent where double underscore obscures.⁷

The single underscore prefix has no special meaning to the Python interpreter when used in attribute names, but it's a very strong convention among Python programmers that you should not access such attributes from outside the class.⁸ It's easy to respect the privacy of an object that marks its attributes with a single `_`, just as it's easy respect the convention that variables in ALL_CAPS should be treated as constants.

Attributes with a single `_` prefix are called “protected” in some corners of the Python documentation.⁹ The practice of “protecting” attributes by convention with the form `self._x` is widespread, but calling that a “protected” attribute is not so common. Some even call that a “private” attribute.

To conclude: the `Vector2d` components are “private” and our `Vector2d` instances are “immutable”—with scare quotes—because there is no way to make them really private and immutable.¹⁰

We'll now come back to our `Vector2d` class. In this final section, we cover a special attribute (not a method) that affects the internal storage of an object, with potentially huge impact on the use of memory but little effect on its public interface: `__slots__`.

Saving Space with the `__slots__` Class Attribute

By default, Python stores instance attributes in a per-instance dict named `__dict__`. As we saw in “[Practical Consequences of How dict Works](#)” on page 90, dictionaries have a significant memory overhead because of the underlying hash table used to provide fast access. If you are dealing with millions of instances with few attributes, the `__slots__` class attribute can save a lot of memory, by letting the interpreter store the instance attributes in a tuple instead of a dict.

7. From the [Paste Style Guide](#).

8. In modules, a single `_` in front of a top-level name does have an effect: if you write `from mymod import *` the names with a `_` prefix are not imported from `mymod`. However, you can still write `from mymod import _privatefunc`. This is explained in the [Python Tutorial, section 6.1. More on Modules](#).

9. One example is in the [gettext module docs](#).

10. If this state of affairs depresses you, and makes you wish Python was more like Java in this regard, don't read my discussion of the relative strength of the Java `private` modifier in “[Soapbox](#)” on page 272.



A `__slots__` attribute inherited from a superclass has no effect. Python only takes into account `__slots__` attributes defined in each class individually.

To define `__slots__`, you create a class attribute with that name and assign it an iterable of `str` with identifiers for the instance attributes. I like to use a tuple for that, because it conveys the message that the `__slots__` definition cannot change. See [Example 9-11](#).

Example 9-11. `vector2d_v3_slots.py`: the `slots` attribute is the only addition to `Vector2d`

```
class Vector2d:
    __slots__ = ('_x', '_y')

    typecode = 'd'

    # methods follow (omitted in book listing)
```

By defining `__slots__` in the class, you are telling the interpreter: “These are all the instance attributes in this class.” Python then stores them in a tuple-like structure in each instance, avoiding the memory overhead of the per-instance `__dict__`. This can make a huge difference in memory usage if you have millions of instances active at the same time.



If you are handling millions of objects with numeric data, you should really be using NumPy arrays (see “[NumPy and SciPy](#)” on [page 52](#)), which are not only memory-efficient but have highly optimized functions for numeric processing, many of which operate on the entire array at once. I designed the `Vector2d` class just to provide context when discussing special methods, because I try to avoid vague foo and bar examples when I can.

[Example 9-12](#) shows two runs of a script that simply builds a list, using a list comprehension, with 10,000,000 instances of `Vector2d`. The `mem_test.py` script takes the name of a module with a `Vector2d` class variant as command-line argument. In the first run, I am using `vector2d_v3.Vector2d` (from [Example 9-7](#)); in the second run, the `__slots__` version of `vector2d_v3_slots.Vector2d` is used.

Example 9-12. `mem_test.py` creates 10 million `Vector2d` instances using the class defined in the named module (e.g., `vector2d_v3.py`)

```
$ time python3 mem_test.py vector2d_v3.py
Selected Vector2d type: vector2d_v3.Vector2d
Creating 10,000,000 Vector2d instances
Initial RAM usage:      5,623,808
```

```

Final RAM usage: 1,558,482,944

real 0m16.721s
user 0m15.568s
sys 0m1.149s
$ time python3 mem_test.py vector2d_v3_slots.py
Selected Vector2d type: vector2d_v3_slots.Vector2d
Creating 10,000,000 Vector2d instances
Initial RAM usage: 5,718,016
Final RAM usage: 655,466,496

real 0m13.605s
user 0m13.163s
sys 0m0.434s

```

As [Example 9-12](#) reveals, the RAM footprint of the script grows to 1.5 GB when instance `__dict__` is used in each of the 10 million `Vector2d` instances, but that is reduced to 655 MB when `Vector2d` has a `__slots__` attribute. The `__slots__` version is also faster. The *mem_test.py* script in this test basically deals with loading a module, checking memory usage, and formatting results. The code is not really relevant here so it's in [Appendix A, Example A-4](#).



When `__slots__` is specified in a class, its instances will not be allowed to have any other attributes apart from those named in `__slots__`. This is really a side effect, and not the reason why `__slots__` exists. It's considered bad form to use `__slots__` just to prevent users of your class from creating new attributes in the instances if they want to. `__slots__` should be used for optimization, not for programmer restraint.

It may be possible, however, to “save memory and eat it too”: if you add the `'__dict__'` name to the `__slots__` list, your instances will keep attributes named in `__slots__` in the per-instance tuple, but will also support dynamically created attributes, which will be stored in the usual `__dict__`. Of course, having `'__dict__'` in `__slots__` may entirely defeat its purpose, depending on the number of static and dynamic attributes in each instance and how they are used. Careless optimization is even worse than premature optimization.

There is another special per-instance attribute that you may want to keep: the `__weakref__` attribute is necessary for an object to support weak references (covered in [“Weak References” on page 236](#)). That attribute is present by default in instances of user-defined classes. However, if the class defines `__slots__`, and you need the instances to be targets of weak references, then you need to include `'__weakref__'` among the attributes named in `__slots__`.

To summarize, `__slots__` has some caveats and should not be abused just for the sake of limiting what attributes can be assigned by users. It is mostly useful when working with tabular data such as database records where the schema is fixed by definition and the datasets may be very large. However, if you do this kind of work often, you must check out not only [NumPy](#), but also [the pandas data analysis library](#), which can handle nonnumeric data and import/export to many different tabular data formats.

The Problems with `__slots__`

To summarize, `__slots__` may provide significant memory savings if properly used, but there are a few caveats:

- You must remember to redeclare `__slots__` in each subclass, because the inherited attribute is ignored by the interpreter.
- Instances will only be able to have the attributes listed in `__slots__`, unless you include `'__dict__'` in `__slots__` (but doing so may negate the memory savings).
- Instances cannot be targets of weak references unless you remember to include `'__weakref__'` in `__slots__`.

If your program is not handling millions of instances, it's probably not worth the trouble of creating a somewhat unusual and tricky class whose instances may not accept dynamic attributes or may not support weak references. Like any optimization, `__slots__` should be used only if justified by a present need and when its benefit is proven by careful profiling.

The last topic in this chapter has to do with overriding a class attribute in instances and subclasses.

Overriding Class Attributes

A distinctive feature of Python is how class attributes can be used as default values for instance attributes. In `Vector2d` there is the `typecode` class attribute. It's used twice in the `__bytes__` method, but we read it as `self.typecode` by design. Because `Vector2d` instances are created without a `typecode` attribute of their own, `self.typecode` will get the `Vector2d.typecode` class attribute by default.

But if you write to an instance attribute that does not exist, you create a new instance attribute—e.g., a `typecode` instance attribute—and the class attribute by the same name is untouched. However, from then on, whenever the code handling that instance reads `self.typecode`, the instance `typecode` will be retrieved, effectively shadowing the class attribute by the same name. This opens the possibility of customizing an individual instance with a different `typecode`.

The default `Vector2d.typecode` is `'d'`, meaning each vector component will be represented as an 8-byte double precision float when exporting to bytes. If we set the type code of a `Vector2d` instance to `'f'` prior to exporting, each component will be exported as a 4-byte single precision float. [Example 9-13](#) demonstrates.



We are discussing adding a custom instance attribute, therefore **Example 9-13** uses the `Vector2d` implementation without `__slots__` as listed in **Example 9-9**.

Example 9-13. Customizing an instance by setting the typecode attribute that was formerly inherited from the class

```
>>> from vector2d_v3 import Vector2d
>>> v1 = Vector2d(1.1, 2.2)
>>> dumpd = bytes(v1)
>>> dumpd
b'\xd\x9a\x99\x99\x99\x99\xf1?\x9a\x99\x99\x99\x99\x01@'
>>> len(dumpd) # ①
17
>>> v1.typecode = 'f' # ②
>>> dumpf = bytes(v1)
>>> dumpf
b'f\xcd\xcc\x8c?\xcd\xcc\x0c@'
>>> len(dumpf) # ③
9
>>> Vector2d.typecode # ④
'd'
```

- 1 Default bytes representation is 17 bytes long.
- 2 Set typecode to 'f' in the v1 instance.
- 3 Now the bytes dump is 9 bytes long.
- 4 `Vector2d.typecode` is unchanged; only the `v1` instance uses typecode 'f'.

Now it should be clear why the bytes export of a Vector2d is prefixed by the type code: we wanted to support different export formats.

If you want to change a class attribute you must set it on the class directly, not through an instance. You could change the default `typecode` for all instances (that don't have their own `typecode`) by doing this:

```
>>> Vector2d.typecode = 'f'
```

However, there is an idiomatic Python way of achieving a more permanent effect, and being more explicit about the change. Because class attributes are public, they are inherited by subclasses, so it's common practice to subclass just to customize a class data

attribute. The Django class-based views use this technique extensively. [Example 9-14](#) shows how.

Example 9-14. The `ShortVector2d` is a subclass of `Vector2d`, which only overwrites the default `typecode`

```
>>> from vector2d_v3 import Vector2d
>>> class ShortVector2d(Vector2d): # ❶
...     typecode = 'f'
...
>>> sv = ShortVector2d(1/11, 1/27) # ❷
>>> sv
ShortVector2d(0.09090909090909091, 0.037037037037037035) # ❸
>>> len(bytes(sv)) # ❹
9
```

- ❶ Create `ShortVector2d` as a `Vector2d` subclass just to overwrite the `typecode` class attribute.
- ❷ Build `ShortVector2d` instance `sv` for demonstration.
- ❸ Inspect the repr of `sv`.
- ❹ Check that the length of the exported bytes is 9, not 17 as before.

This example also explains why I did not hardcode the `class_name` in `Vector2d.__repr__`, but instead got it from `type(self).__name__`, like this:

```
# inside class Vector2d:

def __repr__(self):
    class_name = type(self).__name__
    return '{}{!r}, {!r}'.format(class_name, *self)
```

If I had hardcoded the `class_name`, subclasses of `Vector2d` like `ShortVector2d` would have to overwrite `__repr__` just to change the `class_name`. By reading the name from the type of the instance, I made `__repr__` safer to inherit.

This ends our coverage of implementing a simple class that leverages the data model to play well with the rest of Python—offering different object representations, implementing a custom formatting code, exposing read-only attributes, and supporting `hash()` to integrate with sets and mappings.

Chapter Summary

The aim of this chapter was to demonstrate the use of special methods and conventions in the construction of a well-behaved Pythonic class.

Is `vector2d_v3.py` ([Example 9-9](#)) more Pythonic than `vector2d_v0.py` ([Example 9-2](#))? The `Vector2d` class in `vector2d_v3.py` certainly exhibits more Python features. But

whether the first or the last `Vector2d` implementation is more idiomatic depends on the context where it would be used. Tim Peter's Zen of Python says:

Simple is better than complex.

A Pythonic object should be as simple as the requirements allow—and not a parade of language features.

But my goal in expanding the `Vector2d` code was to provide context for discussing Python special methods and coding conventions. If you look back at [Table 1-1](#), the several listings in this chapter demonstrated:

- All string/bytes representation methods: `__repr__`, `__str__`, `__format__`, and `__bytes__`.
- Several methods for converting an object to a number: `__abs__`, `__bool__`, `__hash__`.
- The `__eq__` operator, to test bytes conversion and to enable hashing (along with `__hash__`).

While supporting conversion to bytes we also implemented an alternative constructor, `Vector2d.frombytes()`, which provided the context for discussing the decorators `@classmethod` (very handy) and `@staticmethod` (not so useful, module-level functions are simpler). The `frombytes` method was inspired by its namesake in the `array.array` class.

We saw that the [Format Specification Mini-Language](#) is extensible by implementing a `__format__` method that does some minimal parsing of `format_spec` provided to the `format(obj, format_spec)` built-in or within replacement fields `'{:«format_spec»}'` in strings used with the `str.format` method.

In preparation to make `Vector2d` instances hashable, we made an effort to make them immutable, at least preventing accidental changes by coding the `x` and `y` attributes as private, and exposing them as read-only properties. We then implemented `__hash__` using the recommended technique of xor-ing the hashes of the instance attributes.

We then discussed the memory savings and the caveats of declaring a `__slots__` attribute in `Vector2d`. Because using `__slots__` is somewhat tricky, it really makes sense only when handling a very large number of instances—think millions of instances, not just thousands.

The last topic we covered was the overriding of a class attribute accessed via the instances (e.g., `self.typecode`). We did that first by creating an instance attribute, and then by subclassing and overwriting at the class level.

Throughout the chapter, I mentioned how design choices in the examples were informed by studying the API of standard Python objects. If this chapter can be summarized in one sentence, this is it:

To build Pythonic objects, observe how real Python objects behave.

— Ancient Chinese proverb

Further Reading

This chapter covered several special methods of the data model, so naturally the primary references are the same as the ones provided in [Chapter 1](#), which gave a high-level view of the same topic. For convenience, I'll repeat those four earlier recommendations here, and add a few other ones:

“Data Model” chapter of The Python Language Reference

Most of the methods we used in this chapter are documented in [“3.3.1. Basic customization”](#).

Python in a Nutshell, 2nd Edition, by Alex Martelli

Excellent coverage of the data model, even if only Python 2.5 is covered (in the second edition). The fundamental concepts are all the same and most of the Data Model APIs haven't changed at all since Python 2.2, when built-in types and user-defined classes became more compatible.

Python Cookbook, 3rd Edition, by David Beazley and Brian K. Jones

Very modern coding practices demonstrated through recipes. Chapter 8, “Classes and Objects” in particular has several solutions related to discussions in this chapter.

Python Essential Reference, 4th Edition, by David Beazley

Covers the data model in detail in the context of Python 2.6 and Python 3.

In this chapter, we covered every special method related to object representation, except `__index__`. It's used to coerce an object to an integer index in the specific context of sequence slicing, and was created to solve a need in NumPy. In practice, you and I are not likely to need to implement `__index__` unless we decide to write a new numeric data type, and we want it to be usable as arguments to `__getitem__`. If you are curious about it, A.M. Kuchling's [What's New in Python 2.5](#) has a short explanation, and [PEP 357 — Allowing Any Object to be Used for Slicing](#) details the need for `__index__`, from the perspective of an implementor of a C-extension, Travis Oliphant, the lead author of NumPy.

An early realization of the need for distinct string representations for objects appeared in Smalltalk. The 1996 article [“How to Display an Object as a String: `printString` and `displayString`”](#) by Bobby Woolf discusses the implementation of the `printString` and `displayString` methods in that language. From that article, I borrowed the pithy de-

scriptions “the way the developer wants to see it” and “the way the user wants to see it” when defining `repr()` and `str()` in “Object Representations” on page 248.

Soapbox

Properties Help Reduce Upfront Costs

In the initial versions of `Vector2d`, the `x` and `y` attributes were public, as are all Python instance and class attributes by default. Naturally, users of vectors need to be able to access its components. Although our vectors are iterable and can be unpacked into a pair of variables, it’s also desirable to be able to write `my_vector.x` and `my_vector.y` to get each component.

When we felt the need to avoid accidental updates to the `x` and `y` attributes, we implemented properties, but nothing changed elsewhere in the code and in the public interface of `Vector2d`, as verified by the doctests. We are still able to access `my_vector.x` and `my_vector.y`.

This shows that we can always start our classes in the simplest possible way, with public attributes, because when (or if) we later need to impose more control with getters and setters, these can be implemented through properties without changing any of the code that already interacts with our objects through the names (e.g., `x` and `y`) that were initially simple public attributes.

This approach is the opposite of that encouraged by the Java language: a Java programmer cannot start with simple public attributes and only later, if needed, implement properties, because they don’t exist in the language. Therefore, writing getters and setters is the norm in Java—even when those methods do nothing useful—because the API cannot evolve from simple public attributes to getters and setters without breaking all code that uses those attributes.

In addition, as our technical reviewer Alex Martelli points out, typing getter/setter calls everywhere is goofy. You have to write stuff like:

```
---
>>> my_object.set_foo(my_object.get_foo() + 1)
---
```

Just to do this:

```
---
>>> my_object.foo += 1
---
```

Ward Cunningham, inventor of the wiki and an Extreme Programming pioneer, recommends asking “What’s the simplest thing that could possibly work?” The idea is to

focus on the goal.¹¹ Implementing setters and getters up front is a distraction from the goal. In Python, we can simply use public attributes knowing we can change them to properties later, if the need arises.

Safety Versus Security in Private Attributes

Perl doesn't have an infatuation with enforced privacy. It would prefer that you stayed out of its living room because you weren't invited, not because it has a shotgun.

— Larry Wall
Creator of Perl

Python and Perl are polar opposites in many regards, but Larry and Guido seem to agree on object privacy.

Having taught Python to many Java programmers over the years, I've found a lot of them put too much faith in the privacy guarantees that Java offers. As it turns out, the Java `private` and `protected` modifiers normally provide protection against accidents only (i.e., safety). They can only guarantee security against malicious intent if the application is deployed with a security manager, and that seldom happens in practice, even in corporate settings.

To prove my point, I like to show this Java class ([Example 9-15](#)).

Example 9-15. Confidential.java: a Java class with a private field named secret

```
public class Confidential {  
  
    private String secret = "";  
  
    public Confidential(String text) {  
        secret = text.toUpperCase();  
    }  
}
```

In [Example 9-15](#), I store the text in the `secret` field after converting it to uppercase, just to make it obvious that whatever is in that field will be in all caps.

The actual demonstration consists of running *expose.py* with Jython. That script uses introspection (“reflection” in Java parlance) to get the value of a private field. The code is in [Example 9-16](#).

Example 9-16. expose.py: Jython code to read the content of a private field in another class

```
import Confidential  
  
message = Confidential('top secret text')  
secret_field = Confidential.getDeclaredField('secret')
```

11. See “Simplest Thing that Could Possibly Work: A Conversation with Ward Cunningham, Part V”.

```
secret_field.setAccessible(True) # break the lock!
print 'message.secret =', secret_field.get(message)
```

If you run **Example 9-16**, this is what you get:

```
$ jython expose.py
message.secret = TOP SECRET TEXT
```

The string 'TOP SECRET TEXT' was read from the secret private field of the Confidential class.

There is no black magic here: *expose.py* uses the Java reflection API to get a reference to the private field named 'secret', and then calls 'secret_field.setAccessible(True)' to make it readable. The same thing can be done with Java code, of course (but it takes more than three times as many lines to do it; see the file *Expose.java* in the *Fluent Python code repository*).

The crucial call `.setAccessible(True)` will fail only if the Jython script or the Java main program (e.g., `Expose.class`) is running under the supervision of a **SecurityManager**. But in the real world, Java applications are rarely deployed with a `SecurityManager`—except for Java applets (remember those?).

My point is: in Java too, access control modifiers are mostly about safety and not security, at least in practice. So relax and enjoy the power Python gives you. Use it responsibly.

FP Ch 10 | Sequence Hacking, Hashing & Slicing [= plan Ch 12]

Sequence Hacking, Hashing, and Slicing

Don't check whether it *is*-a duck: check whether it *quacks*-like-a duck, *walks*-like-a duck, etc, etc, depending on exactly what subset of duck-like behavior you need to play your language-games with. (`comp.lang.python`, Jul. 26, 2000)

— Alex Martelli

In this chapter, we will create a class to represent a multidimensional `Vector` class—a significant step up from the two-dimensional `Vector2d` of [Chapter 9](#). `Vector` will behave like a standard Python immutable flat sequence. Its elements will be floats, and it will support the following by the end of this chapter:

- Basic sequence protocol: `__len__` and `__getitem__`.
- Safe representation of instances with many items.
- Proper slicing support, producing new `Vector` instances.
- Aggregate hashing taking into account every contained element value.
- Custom formatting language extension.

We'll also implement dynamic attribute access with `__getattr__` as a way of replacing the read-only properties we used in `Vector2d`—although this is not typical of sequence types.

The code-intensive presentation will be interrupted by a conceptual discussion about the idea of protocols as an informal interface. We'll talk about how protocols and *duck typing* are related, and its practical implications when you create your own types.

Let's get started.

Vector Applications Beyond Three Dimensions

Who needs a vector with 1,000 dimensions? Hint: not 3D artists! However, n -dimensional vectors (with large values of n) are widely used in information retrieval, where documents and text queries are represented as vectors, with one dimension per word. This is called the **Vector space model**. In this model, a key relevance metric is the cosine similarity (i.e., the cosine of the angle between a query vector and a document vector). As the angle decreases, the cosine approaches the maximum value of 1, and so does the relevance of the document to the query.

Having said that, the `Vector` class in this chapter is a didactic example and we'll not do much math here. Our goal is just to demonstrate some Python special methods in the context of a sequence type.

NumPy and SciPy are the tools you need for real-world vector math. The PyPI package **gemsim**, by Radim Rehurek, implements vector space modeling for natural language processing and information retrieval, using NumPy and SciPy.

Vector: A User-Defined Sequence Type

Our strategy to implement `Vector` will be to use composition, not inheritance. We'll store the components in an array of floats, and will implement the methods needed for our `Vector` to behave like an immutable flat sequence.

But before we implement the sequence methods, let's make sure we have a baseline implementation of `Vector` that is compatible with our earlier `Vector2d` class—except where such compatibility would not make sense.

Vector Take #1: Vector2d Compatible

The first version of `Vector` should be as compatible as possible with our earlier `Vector2d` class.

However, by design, the `Vector` constructor is not compatible with the `Vector2d` constructor. We could make `Vector(3, 4)` and `Vector(3, 4, 5)` work, by taking arbitrary arguments with `*args` in `__init__`, but the best practice for a sequence constructor is to take the data as an iterable argument in the constructor, like all built-in sequence types do. **Example 10-1** shows some ways of instantiating our new `Vector` objects.

Example 10-1. Tests of `Vector.__init__` and `Vector.__repr__`

```
>>> Vector([3.1, 4.2])
Vector([3.1, 4.2])
>>> Vector((3, 4, 5))
Vector([3.0, 4.0, 5.0])
```

```
>>> Vector(range(10))
Vector([0.0, 1.0, 2.0, 3.0, 4.0, ...])
```

Apart from new constructor signature, I made sure every test I did with `Vector2d` (e.g., `Vector2d(3, 4)`) passed and produced the same result with a two-component `Vector([3, 4])`.



When a `Vector` has more than six components, the string produced by `repr()` is abbreviated with `...` as seen in the last line of [Example 10-1](#). This is crucial in any collection type that may contain a large number of items, because `repr` is used for debugging (and you don't want a single large object to span thousands of lines in your console or log). Use the `reprlib` module to produce limited-length representations, as in [Example 10-2](#).

The `reprlib` module is called `repr` in Python 2. The 2to3 tool rewrites imports from `repr` automatically.

[Example 10-2](#) lists the implementation of our first version of `Vector` (this example builds on the code shown in [Examples 9-2](#) and [9-3](#)).

Example 10-2. `vector_v1.py`: derived from `vector2d_v1.py`

```
from array import array
import reprlib
import math

class Vector:
    typecode = 'd'

    def __init__(self, components):
        self._components = array(self.typecode, components) ❶

    def __iter__(self):
        return iter(self._components) ❷

    def __repr__(self):
        components = reprlib.repr(self._components) ❸
        components = components[components.find('['):-1] ❹
        return 'Vector({})'.format(components)

    def __str__(self):
        return str(tuple(self))

    def __bytes__(self):
        return (bytes([ord(self.typecode)]) +
                bytes(self._components)) ❺
```

```

def __eq__(self, other):
    return tuple(self) == tuple(other)

def __abs__(self):
    return math.sqrt(sum(x * x for x in self)) ❹

def __bool__(self):
    return bool(abs(self))

@classmethod
def frombytes(cls, octets):
    typecode = chr(octets[0])
    memv = memoryview(octets[1:]).cast(typecode)
    return cls(memv) ❺

```

- ❶ The `self._components` instance “protected” attribute will hold an array with the Vector components.
- ❷ To allow iteration, we return an iterator over `self._components`.¹
- ❸ Use `reprlib.repr()` to get a limited-length representation of `self._components` (e.g., `array('d', [0.0, 1.0, 2.0, 3.0, 4.0, ...])`).
- ❹ Remove the `array('d',` prefix and the trailing `)` before plugging the string into a Vector constructor call.
- ❺ Build a bytes object directly from `self._components`.
- ❻ We can’t use `hypot` anymore, so we sum the squares of the components and compute the `sqrt` of that.
- ❼ The only change needed from the earlier `frombytes` is in the last line: we pass the `memoryview` directly to the constructor, without unpacking with `*` as we did before.

The way I used `reprlib.repr` deserves some elaboration. That function produces safe representations of large or recursive structures by limiting the length of the output string and marking the cut with `'...'`. I wanted the repr of a Vector to look like `Vector([3.0, 4.0, 5.0])` and not `Vector(array('d', [3.0, 4.0, 5.0]))`, because the fact that there is an array inside a Vector is an implementation detail. Because these constructor calls build identical Vector objects, I prefer the simpler syntax using a list argument.

When coding `__repr__`, I could have produced the simplified components display with this expression: `reprlib.repr(list(self._components))`. However, this would be wasteful, as I’d be copying every item from `self._components` to a list just to use the list repr. Instead, I decided to apply `reprlib.repr` to the `self._components` array

1. The `iter()` function is covered in [Chapter 14](#), along with the `__iter__` method.

directly, and then chop off the characters outside of the `[]`. That's what the second line of `__repr__` does in [Example 10-2](#).



Because of its role in debugging, calling `repr()` on an object should never raise an exception. If something goes wrong inside your implementation of `__repr__`, you must deal with the issue and do your best to produce some serviceable output that gives the user a chance of identifying the target object.

Note that the `__str__`, `__eq__`, and `__bool__` methods are unchanged from `Vector2d`, and only one character was changed in `frombytes` (`a *` was removed in the last line). This is one of the benefits of making the original `Vector2d` iterable.

By the way, we could have subclassed `Vector` from `Vector2d`, but I chose not to do it for two reasons. First, the incompatible constructors really make subclassing not advisable. I could work around that with some clever parameter handling in `__init__`, but the second reason is more important: I want `Vector` to be a standalone example of a class implementing the sequence protocol. That's what we'll do next, after a discussion of the term *protocol*.

Protocols and Duck Typing

As early as [Chapter 1](#), we saw that you don't need to inherit from any special class to create a fully functional sequence type in Python; you just need to implement the methods that fulfill the sequence protocol. But what kind of protocol are we talking about?

In the context of object-oriented programming, a protocol is an informal interface, defined only in documentation and not in code. For example, the sequence protocol in Python entails just the `__len__` and `__getitem__` methods. Any class `Spam` that implements those methods with the standard signature and semantics can be used anywhere a sequence is expected. Whether `Spam` is a subclass of this or that is irrelevant; all that matters is that it provides the necessary methods. We saw that in [Example 1-1](#), reproduced here in [Example 10-3](#).

Example 10-3. Code from [Example 1-1](#), reproduced here for convenience

```
import collections

Card = collections.namedtuple('Card', ['rank', 'suit'])

class FrenchDeck:
    ranks = [str(n) for n in range(2, 11)] + list('JQKA')
    suits = 'spades diamonds clubs hearts'.split()

    def __init__(self):
```

```

self._cards = [Card(rank, suit) for suit in self.suits
               for rank in self.ranks]

def __len__(self):
    return len(self._cards)

def __getitem__(self, position):
    return self._cards[position]

```

The FrenchDeck class in [Example 10-3](#) takes advantage of many Python facilities because it implements the sequence protocol, even if that is not declared anywhere in the code. Any experienced Python coder will look at it and understand that it *is* a sequence, even if it subclasses object. We say it *is* a sequence because it *behaves* like one, and that is what matters.

This became known as *duck typing*, after Alex Martelli’s post quoted at the beginning of this chapter.

Because protocols are informal and unenforced, you can often get away with implementing just part of a protocol, if you know the specific context where a class will be used. For example, to support iteration, only `__getitem__` is required; there is no need to provide `__len__`.

We’ll now implement the sequence protocol in `Vector`, initially without proper support for slicing, but later adding that.

Vector Take #2: A Sliceable Sequence

As we saw with the FrenchDeck example, supporting the sequence protocol is really easy if you can delegate to a sequence attribute in your object, like our `self._components` array. These `__len__` and `__getitem__` one-liners are a good start:

```

class Vector:
    # many lines omitted
    # ...

    def __len__(self):
        return len(self._components)

    def __getitem__(self, index):
        return self._components[index]

```

With these additions, all of these operations now work:

```

>>> v1 = Vector([3, 4, 5])
>>> len(v1)
3
>>> v1[0], v1[-1]
(3.0, 5.0)
>>> v7 = Vector(range(7))

```

```
>>> v7[1:4]
array('d', [1.0, 2.0, 3.0])
```

As you can see, even slicing is supported—but not very well. It would be better if a slice of a `Vector` was also a `Vector` instance and not a `array`. The old `FrenchDeck` class has a similar problem: when you slice it, you get a `list`. In the case of `Vector`, a lot of functionality is lost when slicing produces plain arrays.

Consider the built-in sequence types: every one of them, when sliced, produces a new instance of its own type, and not of some other type.

To make `Vector` produce slices as `Vector` instances, we can't just delegate the slicing to `array`. We need to analyze the arguments we get in `__getitem__` and do the right thing.

Now, let's see how Python turns the syntax `my_seq[1:3]` into arguments for `my_seq.__getitem__(...)`.

How Slicing Works

A demo is worth a thousand words, so take a look at [Example 10-4](#).

Example 10-4. Checking out the behavior of `__getitem__` and slices

```
>>> class MySeq:
...     def __getitem__(self, index):
...         return index # ❶
...
>>> s = MySeq()
>>> s[1] # ❷
1
>>> s[1:4] # ❸
slice(1, 4, None)
>>> s[1:4:2] # ❹
slice(1, 4, 2)
>>> s[1:4:2, 9] # ❺
(slice(1, 4, 2), 9)
>>> s[1:4:2, 7:9] # ❻
(slice(1, 4, 2), slice(7, 9, None))
```

- ❶ For this demonstration, `__getitem__` merely returns whatever is passed to it.
- ❷ A single index, nothing new.
- ❸ The notation `1:4` becomes `slice(1, 4, None)`.
- ❹ `slice(1, 4, 2)` means start at 1, stop at 4, step by 2.
- ❺ Surprise: the presence of commas inside the `[]` means `__getitem__` receives a tuple.
- ❻ The tuple may even hold several slice objects.

Now let's take a closer look at `slice` itself in [Example 10-5](#).

Example 10-5. Inspecting the attributes of the slice class

```
>>> slice # ❶
<class 'slice'>
>>> dir(slice) # ❷
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattr__', '__gt__',
 '__hash__', '__init__', '__le__', '__lt__', '__ne__',
 '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__',
 'indices', 'start', 'step', 'stop']
```

- ❶ `slice` is a built-in type (we saw it first in [“Slice Objects” on page 34](#)).
- ❷ Inspecting a `slice` we find the data attributes `start`, `stop`, and `step`, and an `indices` method.

In [Example 10-5](#), calling `dir(slice)` reveals an `indices` attribute, which turns out to be a very interesting but little-known method. Here is what `help(slice.indices)` reveals:

```
S.indices(len) -> (start, stop, stride)
```

Assuming a sequence of length `len`, calculate the `start` and `stop` indices, and the `stride` length of the extended slice described by `S`. Out of bounds indices are clipped in a manner consistent with the handling of normal slices.

In other words, `indices` exposes the tricky logic that's implemented in the built-in sequences to gracefully handle missing or negative indices and slices that are longer than the target sequence. This method produces “normalized” tuples of nonnegative `start`, `stop`, and `stride` integers adjusted to fit within the bounds of a sequence of the given length.

Here are a couple of examples, considering a sequence of `len == 5`, e.g., `'ABCDE'`:

```
>>> slice(None, 10, 2).indices(5) # ❶
(0, 5, 2)
>>> slice(-3, None, None).indices(5) # ❷
(2, 5, 1)
```

- ❶ `'ABCDE'[:10:2]` is the same as `'ABCDE'[0:5:2]`
- ❷ `'ABCDE'[-3:]` is the same as `'ABCDE'[2:5:1]`



As I write this, the `slice.indices` method is apparently not documented in the online Python Library Reference. The Python Python/C API Reference Manual documents a similar C-level function, `PySlice_GetIndicesEx`. I discovered `slice.indices` while exploring slice objects in the Python console, using `dir()` and `help()`. Yet another evidence of the value of the interactive console as a discovery tool.

In our `Vector` code, we'll not need the `slice.indices()` method because when we get a slice argument we'll delegate its handling to the `_components` array. But if you can't count on the services of an underlying sequence, this method can be a huge time saver.

Now that we know how to handle slices, let's take a look at the improved `Vector.__getitem__` implementation.

A Slice-Aware `__getitem__`

Example 10-6 lists the two methods needed to make `Vector` behave as a sequence: `__len__` and `__getitem__` (the latter now implemented to handle slicing correctly).

*Example 10-6. Part of `vector_v2.py`: `__len__` and `__getitem__` methods added to `Vector` class from `vector_v1.py` (see **Example 10-2**)*

```
def __len__(self):
    return len(self._components)

def __getitem__(self, index):
    cls = type(self)  ❶
    if isinstance(index, slice):  ❷
        return cls(self._components[index])  ❸
    elif isinstance(index, numbers.Integral):  ❹
        return self._components[index]  ❺
    else:
        msg = '{cls.__name__} indices must be integers'
        raise TypeError(msg.format(cls=cls))  ❻
```

- ❶ Get the class of the instance (i.e., `Vector`) for later use.
- ❷ If the index argument is a slice...
- ❸ ...invoke the class to build another `Vector` instance from a slice of the `_components` array.
- ❹ If the index is an `int` or some other kind of integer...
- ❺ ...just return the specific item from `_components`.
- ❻ Otherwise, raise an exception.



Excessive use of `isinstance` may be a sign of bad OO design, but handling slices in `__getitem__` is a justified use case. Note in [Example 10-6](#) the test against numbers. `Integral`—an Abstract Base Class. Using ABCs in `isinstance` tests makes an API more flexible and future-proof. [Chapter 11](#) explains why. Unfortunately, there is no ABC for `slice` in the Python 3.4 standard library.

To discover which exception to raise in the `else` clause of `__getitem__`, I used the interactive console to check the result of `'ABC'[1, 2]`. I then learned that Python raises a `TypeError`, and I also copied the wording from the error message: “indices must be integers.” To create Pythonic objects, mimic Python’s own objects.

Once the code in [Example 10-6](#) is added to the `Vector` class, we have proper slicing behavior, as [Example 10-7](#) demonstrates.

Example 10-7. Tests of enhanced `Vector.getitem` from [Example 10-6](#)

```
>>> v7 = Vector(range(7))
>>> v7[-1] ❶
6.0
>>> v7[1:4] ❷
Vector([1.0, 2.0, 3.0])
>>> v7[-1:] ❸
Vector([6.0])
>>> v7[1,2] ❹
Traceback (most recent call last):
...
TypeError: Vector indices must be integers
```

- ❶ An integer index retrieves just one component value as a float.
- ❷ A slice index creates a new `Vector`.
- ❸ A slice of `len == 1` also creates a `Vector`.
- ❹ `Vector` does not support multidimensional indexing, so a tuple of indices or slices raises an error.

Vector Take #3: Dynamic Attribute Access

In the evolution from `Vector2d` to `Vector`, we lost the ability to access vector components by name (e.g., `v.x`, `v.y`). We are now dealing with vectors that may have a large number of components. Still, it may be convenient to access the first few components with shortcut letters such as `x`, `y`, `z` instead of `v[0]`, `v[1]` and `v[2]`.

Here is the alternative syntax we want to provide for reading the first four components of a vector:

```
>>> v = Vector(range(10))
>>> v.x
0.0
>>> v.y, v.z, v.t
(1.0, 2.0, 3.0)
```

In `Vector2d`, we provided read-only access to `x` and `y` using the `@property` decorator (Example 9-7). We could write four properties in `Vector`, but it would be tedious. The `__getattr__` special method provides a better way.

“The `__getattr__` method is invoked by the interpreter when attribute lookup fails. In simple terms, given the expression `my_obj.x`, Python checks if the `my_obj` instance has an attribute named `x`; if not, the search goes to the class (`my_obj.__class__`), and then up the inheritance graph.² If the `x` attribute is not found, then the `__getattr__` method defined in the class of `my_obj` is called with `self` and the name of the attribute as a string (e.g., `'x'`).

Example 10-8 lists our `__getattr__` method. Essentially it checks whether the attribute being sought is one of the letters `xyzt` and if so, returns the corresponding vector component.

Example 10-8. Part of `vector_v3.py`: `__getattr__` method added to `Vector` class from `vector_v2.py`

```
shortcut_names = 'xyzt'

def __getattr__(self, name):
    cls = type(self)  ❶
    if len(name) == 1:  ❷
        pos = cls.shortcut_names.find(name)  ❸
        if 0 <= pos < len(self._components):  ❹
            return self._components[pos]
    msg = '{!r} object has no attribute {!r}'  ❺
    raise AttributeError(msg.format(cls, name))
```

- ❶ Get the `Vector` class for later use.
- ❷ If the name is one character, it may be one of the `shortcut_names`.
- ❸ Find position of 1-letter name; `str.find` would also locate `'yz'` and we don't want that, this is the reason for the test above.
- ❹ If the position is within range, return the array element.
- ❺ If either test failed, raise `AttributeError` with a standard message text.

2. Attribute lookup is more complicated than this; we'll see the gory details in Part VI. For now, this simplified explanation will do.

It's not hard to implement `__getattr__`, but in this case it's not enough. Consider the bizarre interaction in [Example 10-9](#).

Example 10-9. Inappropriate behavior: assigning to `v.x` raises no error, but introduces an inconsistency

```
>>> v = Vector(range(5))
>>> v
Vector([0.0, 1.0, 2.0, 3.0, 4.0])
>>> v.x # ❶
0.0
>>> v.x = 10 # ❷
>>> v.x # ❸
10
>>> v
Vector([0.0, 1.0, 2.0, 3.0, 4.0]) # ❹
```

- ❶ Access element `v[0]` as `v.x`.
- ❷ Assign new value to `v.x`. This should raise an exception.
- ❸ Reading `v.x` shows the new value, 10.
- ❹ However, the vector components did not change.

Can you explain what is happening? In particular, why the second time `v.x` returns 10 if that value is not in the vector components array? If you don't know right off the bat, study the explanation of `__getattr__` given right before [Example 10-8](#). It's a bit subtle, but a very important foundation to understand a lot of what comes later in the book.

The inconsistency in [Example 10-9](#) was introduced because of the way `__getattr__` works: Python only calls that method as a fall back, when the object does not have the named attribute. However, after we assign `v.x = 10`, the `v` object now has an `x` attribute, so `__getattr__` will no longer be called to retrieve `v.x`: the interpreter will just return the value 10 that is bound to `v.x`. On the other hand, our implementation of `__getattr__` pays no attention to instance attributes other than `self._components`, from where it retrieves the values of the “virtual attributes” listed in `shortcut_names`.

We need to customize the logic for setting attributes in our `Vector` class in order to avoid this inconsistency.

Recall that in the latest `Vector2d` examples from [Chapter 9](#), trying to assign to the `.x` or `.y` instance attributes raised `AttributeError`. In `Vector` we want the same exception with any attempt at assigning to all single-letter lowercase attribute names, just to avoid confusion. To do that, we'll implement `__setattr__` as listed in [Example 10-10](#).

Example 10-10. Part of `vector_v3.py`: `__setattr__` method in `Vector` class

```
def __setattr__(self, name, value):
    cls = type(self)
```



```

if len(name) == 1: ❶
    if name in cls.shortcut_names: ❷
        error = 'readonly attribute {attr_name!r}'
    elif name.islower(): ❸
        error = "can't set attributes 'a' to 'z' in {cls_name!r}"
    else:
        error = '' ❹
    if error: ❺
        msg = error.format(cls_name=cls.__name__, attr_name=name)
        raise AttributeError(msg)
super().__setattr__(name, value) ❻

```

- ❶ Special handling for single-character attribute names.
- ❷ If name is one of xyzzt, set specific error message.
- ❸ If name is lowercase, set error message about all single-letter names.
- ❹ Otherwise, set blank error message.
- ❺ If there is a nonblank error message, raise `AttributeError`.
- ❻ Default case: call `__setattr__` on superclass for standard behavior.



The `super()` function provides a way to access methods of super-classes dynamically, a necessity in a dynamic language supporting multiple inheritance like Python. It's used to delegate some task from a method in a subclass to a suitable method in a superclass, as seen in [Example 10-10](#). There is more about `super` in “[Multiple Inheritance and Method Resolution Order](#)” on page 351.

While choosing the error message to display with `AttributeError`, my first check was the behavior of the built-in `complex` type, because they are immutable and have a pair of data attributes `real` and `imag`. Trying to change either of those in a `complex` instance raises `AttributeError` with the message “can't set attribute”. On the other hand, trying to set a read-only attribute protected by a property as we did in “[A Hashable Vector2d](#)” on page 257 produces the message “readonly attribute”. I drew inspiration from both wordings to set the error string in `__setitem__`, but was more explicit about the forbidden attributes.

Note that we are not disallowing setting all attributes, only single-letter, lowercase ones, to avoid confusion with the supported read-only attributes `x`, `y`, `z`, and `t`.



Knowing that declaring `__slots__` at the class level prevents setting new instance attributes, it's tempting to use that feature instead of implementing `__setattr__` as we did. However, because of all the caveats discussed in “The Problems with `__slots__`” on page 267, using `__slots__` just to prevent instance attribute creation is not recommended. `__slots__` should be used only to save memory, and only if that is a real issue.

Even without supporting writing to the `Vector` components, here is an important take-away from this example: very often when you implement `__getattr__` you need to code `__setattr__` as well, to avoid inconsistent behavior in your objects.

If we wanted to allow changing components, we could implement `__setitem__` to enable `v[0] = 1.1` and/or `__setattr__` to make `v.x = 1.1` work. But `Vector` will remain immutable because we want to make it hashable in the coming section.

Vector Take #4: Hashing and a Faster `==`

Once more we get to implement a `__hash__` method. Together with the existing `__eq__`, this will make `Vector` instances hashable.

The `__hash__` in Example 9-8 simply computed `hash(self.x) ^ hash(self.y)`. We now would like to apply the `^` (xor) operator to the hashes of every component, in succession, like this: `v[0] ^ v[1] ^ v[2]....` That is what the `functools.reduce` function is for. Previously I said that `reduce` is not as popular as before,³ but computing the hash of all vector components is a perfect job for it. Figure 10-1 depicts the general idea of the `reduce` function.

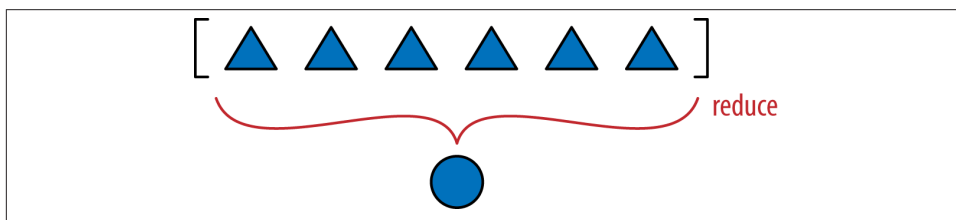


Figure 10-1. Reducing functions—*reduce*, *sum*, *any*, *all*—produce a single aggregate result from a sequence or from any finite iterable object.

3. The `sum`, `any`, and `all` cover the most common uses of `reduce`. See the discussion in “Modern Replacements for `map`, `filter`, and `reduce`” on page 142.

So far we've seen that `functools.reduce()` can be replaced by `sum()`, but now let's properly explain how it works. The key idea is to reduce a series of values to a single value. The first argument to `reduce()` is a two-argument function, and the second argument is an iterable. Let's say we have a two-argument function `fn` and a list `lst`. When you call `reduce(fn, lst)`, `fn` will be applied to the first pair of elements—`fn(lst[0], lst[1])`—producing a first result, `r1`. Then `fn` is applied to `r1` and the next element—`fn(r1, lst[2])`—producing a second result, `r2`. Now `fn(r2, lst[3])` is called to produce `r3` ... and so on until the last element, when a single result, `rN`, is returned.

Here is how you could use `reduce` to compute $5!$ (the factorial of 5):

```
>>> 2 * 3 * 4 * 5 # the result we want: 5! == 120
120
>>> import functools
>>> functools.reduce(lambda a,b: a*b, range(1, 6))
120
```

Back to our hashing problem, [Example 10-11](#) shows the idea of computing the aggregate xor by doing it in three ways: with a for loop and two `reduce` calls.

Example 10-11. Three ways of calculating the accumulated xor of integers from 0 to 5

```
>>> n = 0
>>> for i in range(1, 6): # ❶
...     n ^= i
...
>>> n
1
>>> import functools
>>> functools.reduce(lambda a, b: a^b, range(6)) # ❷
1
>>> import operator
>>> functools.reduce(operator.xor, range(6)) # ❸
1
```

- ❶ Aggregate xor with a for loop and an accumulator variable.
- ❷ `functools.reduce` using an anonymous function.
- ❸ `functools.reduce` replacing custom lambda with `operator.xor`.

From the alternatives in [Example 10-11](#), the last one is my favorite, and the for loop comes second. What is your preference?

As seen in “[The operator Module](#)” on page 156, `operator` provides the functionality of all Python infix operators in function form, lessening the need for lambda.

To code `Vector.__hash__` in my preferred style, we need to import the `functools` and `operator` modules. [Example 10-12](#) shows the relevant changes.

Example 10-12. Part of `vector_v4.py`: two imports and `__hash__` method added to `Vector` class from `vector_v3.py`

```
from array import array
import replib
import math
import functools # ❶
import operator # ❷

class Vector:
    typecode = 'd'

    # many lines omitted in book listing...

    def __eq__(self, other): # ❸
        return tuple(self) == tuple(other)

    def __hash__(self):
        hashes = (hash(x) for x in self._components) # ❹
        return functools.reduce(operator.xor, hashes, 0) # ❺

    # more lines omitted...
```

- ❶ Import `functools` to use `reduce`.
- ❷ Import `operator` to use `xor`.
- ❸ No change to `__eq__`; I listed it here because it's good practice to keep `__eq__` and `__hash__` close in source code, because they need to work together.
- ❹ Create a generator expression to lazily compute the hash of each component.
- ❺ Feed hashes to `reduce` with the `xor` function to compute the aggregate hash value; the third argument, `0`, is the initializer (see next warning).



When using `reduce`, it's good practice to provide the third argument, `reduce(function, iterable, initializer)`, to prevent this exception: `TypeError: reduce() of empty sequence with no initial value` (excellent message: explains the problem and how to fix it). The `initializer` is the value returned if the sequence is empty and is used as the first argument in the reducing loop, so it should be the identity value of the operation. As examples, for `+`, `|`, `^` the `initializer` should be `0`, but for `*`, `&` it should be `1`.

As implemented, the `__hash__` method in [Example 10-8](#) is a perfect example of a map-reduce computation ([Figure 10-2](#)).

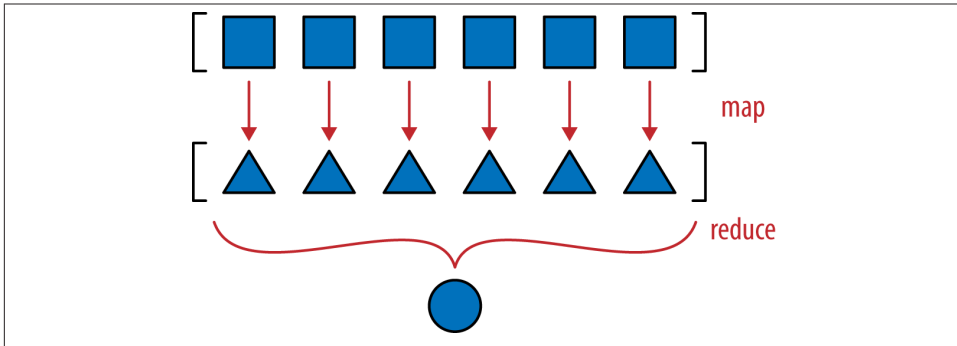


Figure 10-2. Map-reduce: apply function to each item to generate a new series (map), then compute aggregate (reduce)

The mapping step produces one hash for each component, and the reduce step aggregates all hashes with the xor operator. Using map instead of a *genexp* makes the mapping step even more visible:

```
def __hash__(self):
    hashes = map(hash, self._components)
    return functools.reduce(operator.xor, hashes)
```



The solution with map would be less efficient in Python 2, where the map function builds a new list with the results. But in Python 3, map is lazy: it creates a generator that yields the results on demand, thus saving memory—just like the generator expression we used in the `__hash__` method of [Example 10-8](#).

While we are on the topic of reducing functions, we can replace our quick implementation of `__eq__` with another one that will be cheaper in terms of processing and memory, at least for large vectors. As introduced in [Example 9-2](#), we have this very concise implementation of `__eq__`:

```
def __eq__(self, other):
    return tuple(self) == tuple(other)
```

This works for `Vector2d` and for `Vector`—it even considers `Vector([1, 2])` equal to `(1, 2)`, which may be a problem, but we’ll overlook that for now.⁴ But for `Vector` instances that may have thousands of components, it’s very inefficient. It builds two tuples copying the entire contents of the operands just to use the `__eq__` of the tuple

4. We’ll seriously consider the matter of `Vector([1, 2]) == (1, 2)` in “Operator Overloading 101” on page 372.

type. For `Vector2d` (with only two components), it's a good shortcut, but not for the large multidimensional vectors. A better way of comparing one `Vector` to another `Vector` or iterable would be [Example 10-13](#).

Example 10-13. `Vector.eq` using `zip` in a `for` loop for more efficient comparison

```
def __eq__(self, other):
    if len(self) != len(other): # ❶
        return False
    for a, b in zip(self, other): # ❷
        if a != b: # ❸
            return False
    return True # ❹
```

- ❶ If the `len` of the objects are different, they are not equal.
- ❷ `zip` produces a generator of tuples made from the items in each iterable argument. See “[The Awesome `zip`](#)” on [page 293](#) if `zip` is new to you. The `len` comparison above is needed because `zip` stops producing values without warning as soon as one of the inputs is exhausted.
- ❸ As soon as two components are different, exit returning `False`.
- ❹ Otherwise, the objects are equal.

[Example 10-13](#) is efficient, but the `all` function can produce the same aggregate computation of the `for` loop in one line: if all comparisons between corresponding components in the operands are `True`, the result is `True`. As soon as one comparison is `False`, `all` returns `False`. [Example 10-14](#) shows how `__eq__` looks using `all`.

Example 10-14. `Vector.eq` using `zip` and `all`: same logic as [Example 10-13](#)

```
def __eq__(self, other):
    return len(self) == len(other) and all(a == b for a, b in zip(self, other))
```

Note that we first check that the operands have equal length, because `zip` will stop at the shortest operand.

[Example 10-14](#) is the implementation we choose for `__eq__` in `vector_v4.py`.

We wrap up this chapter by bringing back the `__format__` method from `Vector2d` to `Vector`.

The Awesome zip

Having a for loop that iterates over items without fiddling with index variables is great and prevents lots of bugs, but demands some special utility functions. One of them is the zip built-in, which makes it easy to iterate in parallel over two or more iterables by returning tuples that you can unpack into variables, one for each item in the parallel inputs. See [Example 10-15](#).



The zip function is named after the zipper fastener because the physical device works by interlocking pairs of teeth taken from both zipper sides, a good visual analogy for what zip(left, right) does. No relation with compressed files.

Example 10-15. The zip built-in at work

```
>>> zip(range(3), 'ABC') # ❶
<zip object at 0x10063ae48>
>>> list(zip(range(3), 'ABC')) # ❷
[(0, 'A'), (1, 'B'), (2, 'C')]
>>> list(zip(range(3), 'ABC', [0.0, 1.1, 2.2, 3.3])) # ❸
[(0, 'A', 0.0), (1, 'B', 1.1), (2, 'C', 2.2)]
>>> from itertools import zip_longest # ❹
>>> list(zip_longest(range(3), 'ABC', [0.0, 1.1, 2.2, 3.3], fillvalue=-1))
[(0, 'A', 0.0), (1, 'B', 1.1), (2, 'C', 2.2), (-1, -1, 3.3)]
```

- ❶ zip returns a generator that produces tuples on demand.
- ❷ Here we build a list from it just for display; usually we iterate over the generator.
- ❸ zip has a surprising trait: it stops without warning when one of the iterables is exhausted.⁵
- ❹ The itertools.zip_longest function behaves differently: it uses an optional fillvalue (None by default) to complete missing values so it can generate tuples until the last iterable is exhausted.

The enumerate built-in is another generator function often used in for loops to avoid manual handling of index variables. If you are not familiar with enumerate, you should definitely check it out in the [“Built-in functions” documentation](#). The zip and enumer

5. That’s surprising (to me, at least). I think zip should raise ValueError if the sequences are not all of the same length, which is what happens when unpacking an iterable to a tuple of variables of different length.

ate built-ins, along with several other generator functions in the standard library, are covered in “[Generator Functions in the Standard Library](#)” on page 424.

Vector Take #5: Formatting

The `__format__` method of `Vector` will resemble that of `Vector2d`, but instead of providing a custom display in polar coordinates, `Vector` will use spherical coordinates—also known as “hyperspherical” coordinates, because now we support n dimensions, and spheres are “hyperspheres” in 4D and beyond.⁶ Accordingly, we’ll change the custom format suffix from `'p'` to `'h'`.



As we saw in “[Formatted Displays](#)” on page 253, when extending the [Format Specification Mini-Language](#) it’s best to avoid reusing format codes supported by built-in types. In particular, our extended mini-language also uses the float formatting codes `'eEfFgGn%'` in their original meaning, so we definitely must avoid these. Integers use `'bcdxXn'` and strings use `'s'`. I picked `'p'` for `Vector2d` polar coordinates. Code `'h'` for hyperspherical coordinates is a good choice.

For example, given a `Vector` object in 4D space (`len(v) == 4`), the `'h'` code will produce a display like `< r,  $\Phi_1$ ,  $\Phi_2$ ,  $\Phi_3$ >` where r is the magnitude (`abs(v)`) and the remaining numbers are the angular coordinates Φ_1 , Φ_2 , Φ_3 .

Here are some samples of the spherical coordinate format in 4D, taken from the doctests of `vector_v5.py` (see [Example 10-16](#)):

```
>>> format(Vector([-1, -1, -1, -1]), 'h')
'<2.0, 2.0943951023931957, 2.186276035465284, 3.9269908169872414>'
>>> format(Vector([2, 2, 2, 2]), '.3eh')
'<4.000e+00, 1.047e+00, 9.553e-01, 7.854e-01>'
>>> format(Vector([0, 1, 0, 0]), '0.5fh')
'<1.00000, 1.57080, 0.00000, 0.00000>'
```

Before we can implement the minor changes required in `__format__`, we need to code a pair of support methods: `angle(n)` to compute one of the angular coordinates (e.g., Φ_1), and `angles()` to return an iterable of all angular coordinates. I’ll not describe the math here; if you’re curious, Wikipedia’s “[n-sphere](#)” entry has the formulas I used to calculate the spherical coordinates from the Cartesian coordinates in the `Vector` components array.

6. The Wolfram Mathworld site has an article on [Hypersphere](#); on Wikipedia, “hypersphere” redirects to the “[n-sphere](#)” entry.

Example 10-16 is a full listing of `vector_v5.py` consolidating all we've implemented since "Vector Take #1: Vector2d Compatible" on page 276 and introducing custom formatting.

Example 10-16. `vector_v5.py`: doctests and all code for final `Vector` class; callouts highlight additions needed to support `__format__`

```
"""
A multidimensional ``Vector`` class, take 5

A ``Vector`` is built from an iterable of numbers::

    >>> Vector([3.1, 4.2])
    Vector([3.1, 4.2])
    >>> Vector((3, 4, 5))
    Vector([3.0, 4.0, 5.0])
    >>> Vector(range(10))
    Vector([0.0, 1.0, 2.0, 3.0, 4.0, ...])

Tests with two dimensions (same results as ``vector2d_v1.py``)::

    >>> v1 = Vector([3, 4])
    >>> x, y = v1
    >>> x, y
    (3.0, 4.0)
    >>> v1
    Vector([3.0, 4.0])
    >>> v1_clone = eval(repr(v1))
    >>> v1 == v1_clone
    True
    >>> print(v1)
    (3.0, 4.0)
    >>> octets = bytes(v1)
    >>> octets
    b'd|\x00|\x00|\x00|\x00|\x00|\x08|\x00|\x00|\x00|\x00|\x00|\x10@'
    >>> abs(v1)
    5.0
    >>> bool(v1), bool(Vector([0, 0]))
    (True, False)
```

Test of `Vector.frombytes()` class method:

```
>>> v1_clone = Vector.frombytes(bytes(v1))
>>> v1_clone
Vector([3.0, 4.0])
>>> v1 == v1_clone
True
```

Tests with three dimensions::

```

>>> v1 = Vector([3, 4, 5])
>>> x, y, z = v1
>>> x, y, z
(3.0, 4.0, 5.0)
>>> v1
Vector([3.0, 4.0, 5.0])
>>> v1_clone = eval(repr(v1))
>>> v1 == v1_clone
True
>>> print(v1)
(3.0, 4.0, 5.0)
>>> abs(v1) # doctest:+ELLIPSIS
7.071067811...
>>> bool(v1), bool(Vector([0, 0, 0]))
(True, False)

```

Tests with many dimensions::

```

>>> v7 = Vector(range(7))
>>> v7
Vector([0.0, 1.0, 2.0, 3.0, 4.0, ...])
>>> abs(v7) # doctest:+ELLIPSIS
9.53939201...

```

Test of ``\_\_bytes\_\_`` and ``.frombytes()`` methods::

```

>>> v1 = Vector([3, 4, 5])
>>> v1_clone = Vector.frombytes(bytes(v1))
>>> v1_clone
Vector([3.0, 4.0, 5.0])
>>> v1 == v1_clone
True

```

Tests of sequence behavior::

```

>>> v1 = Vector([3, 4, 5])
>>> len(v1)
3
>>> v1[0], v1[len(v1)-1], v1[-1]
(3.0, 5.0, 5.0)

```

Test of slicing::

```

>>> v7 = Vector(range(7))
>>> v7[-1]
6.0
>>> v7[1:4]
Vector([1.0, 2.0, 3.0])

```

```
>>> v7[-1:]
Vector([6.0])
>>> v7[1,2]
Traceback (most recent call last):
...
TypeError: Vector indices must be integers
```

Tests of dynamic attribute access::

```
>>> v7 = Vector(range(10))
>>> v7.x
0.0
>>> v7.y, v7.z, v7.t
(1.0, 2.0, 3.0)
```

Dynamic attribute lookup failures::

```
>>> v7.k
Traceback (most recent call last):
...
AttributeError: 'Vector' object has no attribute 'k'
>>> v3 = Vector(range(3))
>>> v3.t
Traceback (most recent call last):
...
AttributeError: 'Vector' object has no attribute 't'
>>> v3.spam
Traceback (most recent call last):
...
AttributeError: 'Vector' object has no attribute 'spam'
```

Tests of hashing::

```
>>> v1 = Vector([3, 4])
>>> v2 = Vector([3.1, 4.2])
>>> v3 = Vector([3, 4, 5])
>>> v6 = Vector(range(6))
>>> hash(v1), hash(v3), hash(v6)
(7, 2, 1)
```

Most hash values of non-integers vary from a 32-bit to 64-bit CPython build::

```
>>> import sys
>>> hash(v2) == (384307168202284039 if sys.maxsize > 2**32 else 357915986)
True
```

Tests of ``format()`` with Cartesian coordinates in 2D::

```
>>> v1 = Vector([3, 4])
>>> format(v1)
'(3.0, 4.0)'
>>> format(v1, '.2f')
'(3.00, 4.00)'
>>> format(v1, '.3e')
'(3.000e+00, 4.000e+00)'
```

Tests of ``format()`` with Cartesian coordinates in 3D and 7D::

```
>>> v3 = Vector([3, 4, 5])
>>> format(v3)
'(3.0, 4.0, 5.0)'
>>> format(Vector(range(7)))
'(0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 6.0)'
```

Tests of ``format()`` with spherical coordinates in 2D, 3D and 4D::

```
>>> format(Vector([1, 1]), 'h') # doctest:+ELLIPSIS
'<1.414213..., 0.785398...>'
>>> format(Vector([1, 1]), '.3eh')
'<1.414e+00, 7.854e-01>'
>>> format(Vector([1, 1]), '0.5fh')
'<1.41421, 0.78540>'
>>> format(Vector([1, 1, 1]), 'h') # doctest:+ELLIPSIS
'<1.73205..., 0.95531..., 0.78539...>'
>>> format(Vector([2, 2, 2]), '.3eh')
'<3.464e+00, 9.553e-01, 7.854e-01>'
>>> format(Vector([0, 0, 0]), '0.5fh')
'<0.00000, 0.00000, 0.00000>'
>>> format(Vector([-1, -1, -1, -1]), 'h') # doctest:+ELLIPSIS
'<2.0, 2.09439..., 2.18627..., 3.92699...>'
>>> format(Vector([2, 2, 2, 2]), '.3eh')
'<4.000e+00, 1.047e+00, 9.553e-01, 7.854e-01>'
>>> format(Vector([0, 1, 0, 0]), '0.5fh')
'<1.00000, 1.57080, 0.00000, 0.00000>'
"""
```

```
from array import array
import reprlib
import math
import numbers
import functools
import operator
import itertools ❶
```

```
class Vector:
    typecode = 'd'
```

```

def __init__(self, components):
    self._components = array(self.typecode, components)

def __iter__(self):
    return iter(self._components)

def __repr__(self):
    components = reprlib.repr(self._components)
    components = components[components.find('['):-1]
    return 'Vector({})'.format(components)

def __str__(self):
    return str(tuple(self))

def __bytes__(self):
    return (bytes([ord(self.typecode)]) +
            bytes(self._components))

def __eq__(self, other):
    return (len(self) == len(other) and
            all(a == b for a, b in zip(self, other)))

def __hash__(self):
    hashes = (hash(x) for x in self)
    return functools.reduce(operator.xor, hashes, 0)

def __abs__(self):
    return math.sqrt(sum(x * x for x in self))

def __bool__(self):
    return bool(abs(self))

def __len__(self):
    return len(self._components)

def __getitem__(self, index):
    cls = type(self)
    if isinstance(index, slice):
        return cls(self._components[index])
    elif isinstance(index, numbers.Integral):
        return self._components[index]
    else:
        msg = '{.__name__} indices must be integers'
        raise TypeError(msg.format(cls))

shortcut_names = 'xyzt'

def __getattr__(self, name):
    cls = type(self)
    if len(name) == 1:
        pos = cls.shortcut_names.find(name)
        if 0 <= pos < len(self._components):

```

```

        return self._components[pos]
    msg = '{.__name__!r} object has no attribute {!r}'
    raise AttributeError(msg.format(cls, name))

def angle(self, n): ❷
    r = math.sqrt(sum(x * x for x in self[n:]))
    a = math.atan2(r, self[n-1])
    if (n == len(self) - 1) and (self[-1] < 0):
        return math.pi * 2 - a
    else:
        return a

def angles(self): ❸
    return (self.angle(n) for n in range(1, len(self)))

def __format__(self, fmt_spec=''):
    if fmt_spec.endswith('h'): # hyperspherical coordinates
        fmt_spec = fmt_spec[:-1]
        coords = itertools.chain([abs(self)],
                                self.angles()) ❹

        outer_fmt = '<{}>' ❺
    else:
        coords = self
        outer_fmt = '({})' ❻
    components = (format(c, fmt_spec) for c in coords) ❼
    return outer_fmt.format(', '.join(components)) ❽

@classmethod
def frombytes(cls, octets):
    typecode = chr(octets[0])
    memv = memoryview(octets[1:]).cast(typecode)
    return cls(memv)

```

- ❶ Import `itertools` to use `chain` function in `__format__`.
- ❷ Compute one of the angular coordinates, using formulas adapted from the *[n-sphere article](#)*.
- ❸ Create generator expression to compute all angular coordinates on demand.
- ❹ Use `itertools.chain` to produce *genexp* to iterate seamlessly over the magnitude and the angular coordinates.
- ❺ Configure spherical coordinate display with angular brackets.
- ❻ Configure Cartesian coordinate display with parentheses.
- ❼ Create generator expression to format each coordinate item on demand.
- ❽ Plug formatted components separated by commas inside brackets or parentheses.



We are making heavy use of generator expressions in `__format__`, `angle`, and `angles` but our focus here is in providing `__format__` to bring `Vector` to the same implementation level as `Vector2d`. When we cover generators in [Chapter 14](#) we'll use some of the code in `Vector` as examples, and then the generator tricks will be explained in detail.

This concludes our mission for this chapter. The `Vector` class will be enhanced with infix operators in [Chapter 13](#), but our goal here was to explore techniques for coding special methods that are useful in a wide variety of collection classes.

Chapter Summary

The `Vector` example in this chapter was designed to be compatible with `Vector2d`, except for the use of a different constructor signature accepting a single iterable argument, just like the built-in sequence types do. The fact that `Vector` behaves as a sequence just by implementing `__getitem__` and `__len__` prompted a discussion of protocols, the informal interfaces used in duck-typed languages.

We then looked at how the `my_seq[a:b:c]` syntax works behind the scenes, by creating a `slice(a, b, c)` object and handing it to `__getitem__`. Armed with this knowledge, we made `Vector` respond correctly to slicing, by returning new `Vector` instances, just like a Pythonic sequence is expected to do.

The next step was to provide read-only access to the first few `Vector` components using notation such as `my_vec.x`. We did it by implementing `__getattr__`. Doing that opened the possibility of tempting the user to assign to those special components by writing `my_vec.x = 7`, revealing a potential bug. We fixed it by implementing `__setattr__` as well, to forbid assigning values to single-letter attributes. Very often, when you code a `__getattr__` you need to add `__setattr__` too, in order to avoid inconsistent behavior.

Implementing the `__hash__` function provided the perfect context for using `functools.reduce`, because we needed to apply the xor operator `^` in succession to the hashes of all `Vector` components to produce an aggregate hash value for the whole `Vector`. After applying `reduce` in `__hash__`, we used the `all` reducing built-in to create a more efficient `__eq__` method.

The last enhancement to `Vector` was to reimplement the `__format__` method from `Vector2d` by supporting spherical coordinates as an alternative to the default Cartesian coordinates. We used quite a bit of math and several generators to code `__format__` and its auxiliary functions, but these are implementation details—and we'll come back to the generators in [Chapter 14](#). The goal of that last section was to support a custom

format, thus fulfilling the promise of a `Vector` that could do everything a `Vector2d` did, and more.

As we did in [Chapter 9](#), here we often looked at how standard Python objects behave, to emulate them and provide a “Pythonic” look-and-feel to `Vector`.

In [Chapter 13](#), we will implement several infix operators on `Vector`. The math will be much simpler than that in the `angle()` method here, but exploring how infix operators work in Python is a great lesson in OO design. But before we get to operator overloading, we’ll step back from working on one class and look at organizing multiple classes with interfaces and inheritance, the subjects of [Chapters 11](#) and [11](#).

Further Reading

Most special methods covered in the `Vector` example also appear in the `Vector2d` example from [Chapter 9](#), so the references in “[Further Reading](#)” on page 271 are all relevant here.

The powerful `reduce` higher-order function is also known as `fold`, `accumulate`, `aggregate`, `compress`, and `inject`. For more information, see Wikipedia’s “[Fold \(higher-order function\)](#)” [article](#), which presents applications of that higher-order function with emphasis on functional programming with recursive data structures. The article also includes a table listing fold-like functions in dozens of programming languages.

Soapbox

Protocols as Informal Interfaces

Protocols are not an invention of Python. The Smalltalk team, who also coined the expression “object oriented,” used “protocol” as a synonym for what we now call interfaces. Some Smalltalk programming environments allowed programmers to tag a group of methods as a protocol, but that was merely a documentation and navigation aid, and not enforced by the language. That’s why I believe “informal interface” is a reasonable short explanation for “protocol” when I speak to an audience that is more familiar with formal (and compiler enforced) interfaces.

Established protocols naturally evolve in any language that uses dynamic typing, that is, when type-checking done at runtime because there is no static type information in method signatures and variables. Ruby is another important OO language that has dynamic typing and uses protocols.

In the Python documentation, you can often tell when a protocol is being discussed when you see language like “a file-like object.” This is a quick way of saying “something that behaves sufficiently like a file, by implementing the parts of the file interface that are relevant in the context.”

You may think that implementing only part of a protocol is sloppy, but it has the advantage of keeping things simple. [Section 3.3](#) of the “Data Model” chapter suggests:

When implementing a class that emulates any built-in type, it is important that the emulation only be implemented to the degree that it makes sense for the object being modeled. For example, some sequences may work well with retrieval of individual elements, but extracting a slice may not make sense.

— “Data Model” chapter of *The Python Language Reference*

When we don’t need to code nonsense methods just to fulfill some over-designed interface contract and keep the compiler happy, it becomes easier to follow the [KISS principle](#).

I’ll have more to say about protocols and interfaces in [Chapter 11](#), where that is actually the main focus.

Origins of Duck Typing

I believe the Ruby community, more than any other, helped popularize the term “duck typing,” as they preached to the Java masses. But the expression has been used in Python discussions before either Ruby or Python were “popular.” According to Wikipedia, an early example of the duck analogy in object-oriented programming is a message to the Python-list by Alex Martelli from July 26, 2000: [polymorphism \(was Re: Type checking in python?\)](#). That’s where the quote at the beginning of this chapter came from. If you are curious about the literary origins of the “duck typing” term, and the applications of this OO concept in many languages, check out Wikipedia’s [“Duck typing” entry](#).

A safe *format*, with Enhanced Usability

While implementing `__format__`, we did not take any precautions regarding `Vector` instances with a very large number of components, as we did in `__repr__` using `reprlib`. The reasoning is that `repr()` is for debugging and logging, so it must always generate some serviceable output, while `__format__` is used to display output to end users who presumably want to see the entire `Vector`. If you think this is dangerous, then it would be cool to implement a further extension to the format specifier mini-language.

Here is how I’d do it: by default, any formatted `Vector` would display a reasonable but limited number of components, say 30. If there are more elements than that, the default behavior would be similar to what the `reprlib` does: chop the excess and put `...` in its place. However, if the format specifier ended with the special `*` code, meaning “all,” then the size limitation would be disabled. So a user who’s unaware of the problem of very long displays will not be bitten by it by accident. But if the default limitation becomes a nuisance, then the presence of the `...` should prompt the user to research the documentation and discover the `*` formatting code.

Send a pull request to the [Fluent Python repository on GitHub](#) if you implement this!

The Search for a Pythonic Sum

There's no single answer to "What is Pythonic?" just as there's no single answer to "What is beautiful?" Saying, as I often do, that it means using "idiomatic Python" is not 100% satisfactory, because what may be "idiomatic" for you may not be for me. One thing I know: "idiomatic" does not mean using the most obscure language features.

In the [Python-list](#), there's a thread from April 2003 titled "[Pythonic Way to Sum n-th List Element?](#)". It's relevant to our discussion of `reduce` in this chapter.

The original poster, Guy Middleton, asked for an improvement on this solution, stating he did not like to use `lambda`:⁷

```
>>> my_list = [[1, 2, 3], [40, 50, 60], [9, 8, 7]]
>>> import functools
>>> functools.reduce(lambda a, b: a+b, [sub[1] for sub in my_list])
60
```

That code uses lots of idioms: `lambda`, `reduce`, and a list comprehension. It would probably come last in a popularity contest, because it offends people who hate `lambda` and those who despise list comprehensions—pretty much both sides of a divide.

If you're going to use `lambda`, there's probably no reason to use a list comprehension—except for filtering, which is not the case here.

Here is a solution of my own that will please the `lambda` lovers:

```
>>> functools.reduce(lambda a, b: a + b[1], my_list, 0)
60
```

I did not take part in the original thread, and I wouldn't use that in real code, because I don't like `lambda` too much myself, but I wanted to show an example without a list comprehension.

The first answer came from Fernando Perez, creator of IPython, highlighting that NumPy supports *n*-dimensional arrays and *n*-dimensional slicing:

```
>>> import numpy as np
>>> my_array = np.array(my_list)
>>> np.sum(my_array[:, 1])
60
```

I think Perez's solution is cool, but Guy Middleton praised this next solution, by Paul Rubin and Skip Montanaro:

```
>>> import operator
>>> functools.reduce(operator.add, [sub[1] for sub in my_list], 0)
60
```

Then Evan Simpson asked, "What's wrong with this?":

7. I adapted the code for this presentation: in 2003, `reduce` was a built-in, but in Python 3 we need to import it; also, I replaced the names `x` and `y` with `my_list` and `sub`, for sub-list.

```
>>> t = 0
>>> for sub in my_list:
...     total += sub[1]
>>> t
60
```

Lots of people agreed that was quite Pythonic. Alex Martelli went as far as saying that's probably how Guido would code it.

I like Evan Simpson's code but I also like David Eppstein's comment on it:

If you want the sum of a list of items, you should write it in a way that looks like “the sum of a list of items”, not in a way that looks like “loop over these items, maintain another variable t, perform a sequence of additions”. Why do we have high level languages if not to express our intentions at a higher level and let the language worry about what low-level operations are needed to implement it?

Then Alex Martelli comes back to suggest:

“The sum” is so frequently needed that I wouldn't mind at all if Python singled it out as a built-in. But “reduce(operator.add, ...)” just isn't a great way to express it, in my opinion (and yet as an old APLer, and FP-liker, I *should* like it—but I don't).

Alex goes on to suggest a `sum()` function, which he contributed. It became a built-in in Python 2.3, released only three months after that conversation took place. So Alex's preferred syntax became the norm:

```
>>> sum([sub[1] for sub in my_list])
60
```

By the end of the next year (November 2004), Python 2.4 was launched with generator expressions, providing what is now in my opinion the most Pythonic answer to Guy Middleton's original question:

```
>>> sum(sub[1] for sub in my_list)
60
```

This is not only more readable than `reduce` but also avoids the trap of the empty sequence: `sum([])` is 0, simple as that.

In the same conversation, Alex Martelli suggests the `reduce` built-in in Python 2 was more trouble than it was worth, because it encouraged coding idioms that were hard to explain. He was most convincing: the function was demoted to the `functools` module in Python 3.

Still, `functools.reduce` has its place. It solved the problem of our `Vector.__hash__` in a way that I would call Pythonic.

FP Ch 11 | Interfaces: From Protocols to ABCs [= plan Ch 13]

Interfaces: From Protocols to ABCs

An abstract class represents an interface.¹

— Bjarne Stroustrup
Creator of C++

Interfaces are the subject of this chapter: from the dynamic protocols that are the hallmark of *duck typing* to abstract base classes (ABCs) that make interfaces explicit and verify implementations for conformance.

If you have a Java, C#, or similar background, the novelty here is in the informal protocols of duck typing. But for the long-time Pythonista or Rubyist, that is the “normal” way of thinking about interfaces, and the news is the formality and type-checking of ABCs. The language was 15 years old when ABCs were introduced in Python 2.6.

We’ll start the chapter by reviewing how the Python community traditionally understood interfaces as somewhat loose—in the sense that a partially implemented interface is often acceptable. We’ll make that clear through a couple examples that highlight the dynamic nature of duck typing.

Then, a guest essay by Alex Martelli will introduce ABCs and give name to a new trend in Python programming. The rest of the chapter will be devoted to ABCs, starting with their common use as superclasses when you need to implement an interface. We’ll then see when an ABC checks concrete subclasses for conformance to the interface it defines, and how a registration mechanism lets developers declare that a class implements an interface without subclassing. Finally, we’ll see how an ABC can be programmed to automatically “recognize” arbitrary classes that conform to its interface—without subclassing or explicit registration.

1. Bjarne Stroustrup, *The Design and Evolution of C++* (Addison-Wesley, 1994), p. 278.

We will implement a new ABC to see how that works, but Alex Martelli and I don't want to encourage you to start writing your own ABCs left and right. The risk of over-engineering with ABCs is very high.



ABCs, like descriptors and metaclasses, are tools for building frameworks. Therefore, only a very small minority of Python developers can create ABCs without imposing unreasonable limitations and needless work on fellow programmers.

Let's get started with the Pythonic view of interfaces.

Interfaces and Protocols in Python Culture

Python was already highly successful before ABCs were introduced, and most existing code does not use them at all. Since [Chapter 1](#), we've been talking about *duck typing* and protocols. In [“Protocols and Duck Typing” on page 279](#), protocols are defined as the informal interfaces that make polymorphism work in languages with dynamic typing like Python.

How do interfaces work in a dynamic-typed language? First, the basics: even without an `interface` keyword in the language, and regardless of ABCs, every class has an interface: the set public attributes (methods or data attributes) implemented or inherited by the class. This includes special methods, like `__getitem__` or `__add__`.

By definition, protected and private attributes are not part of an interface, even if “protected” is merely a naming convention (the single leading underscore) and private attributes are easily accessed (recall [“Private and Protected” Attributes in Python” on page 262](#)). It is bad form to violate these conventions.

On the other hand, it's not a sin to have public data attributes as part of the interface of an object, because—if necessary—a data attribute can always be turned into a property implementing getter/setter logic without breaking client code that uses the plain `obj.attr` syntax. We did that in the `Vector2d` class: in [Example 11-1](#), we see the first implementation with public `x` and `y` attributes.

Example 11-1. `vector2d_v0.py`: `x` and `y` are public data attributes (same code as [Example 9-2](#))

```
class Vector2d:
    typecode = 'd'

    def __init__(self, x, y):
        self.x = float(x)
        self.y = float(y)

    def __iter__(self):
```

```

    return (i for i in (self.x, self.y))

# more methods follow (omitted in this listing)

```

In [Example 9-7](#), we turned `x` and `y` into read-only properties ([Example 11-2](#)). This is a significant refactoring, but an essential part of the interface of `Vector2d` is unchanged: users can still read `my_vector.x` and `my_vector.y`.

Example 11-2. `vector2d_v3.py`: `x` and `y` reimplemented as properties (see full listing in [Example 9-9](#))

```

class Vector2d:
    typecode = 'd'

    def __init__(self, x, y):
        self.__x = float(x)
        self.__y = float(y)

    @property
    def x(self):
        return self.__x

    @property
    def y(self):
        return self.__y

    def __iter__(self):
        return (i for i in (self.x, self.y))

# more methods follow (omitted in this listing)

```

A useful complementary definition of interface is: the subset of an object’s public methods that enable it to play a specific role in the system. That’s what is implied when the Python documentation mentions “a file-like object” or “an iterable,” without specifying a class. An interface seen as a set of methods to fulfill a role is what Smalltalkers called a *protocol*, and the term spread to other dynamic language communities. Protocols are independent of inheritance. A class may implement several protocols, enabling its instances to fulfill several roles.

Protocols are interfaces, but because they are informal—defined only by documentation and conventions—protocols cannot be enforced like formal interfaces can (we’ll see how ABCs enforce interface conformance later in this chapter). A protocol may be partially implemented in a particular class, and that’s OK. Sometimes all a specific API requires from “a file-like object” is that it has a `.read()` method that returns bytes. The remaining file methods may or may not be relevant in the context.

As I write this, the [Python 3 documentation of `memoryview`](#) says that it works with objects that “support the buffer protocol, which is only documented at the C API level. The [`bytearray` constructor](#) accepts an “an object conforming to the buffer interface.” Now

there is a move to adopt “bytes-like object” as a friendlier term.² I point this out to emphasize that “X-like object,” “X protocol,” and “X interface” are synonyms in the minds of Pythonistas.

One of the most fundamental interfaces in Python is the sequence protocol. The interpreter goes out of its way to handle objects that provide even a minimal implementation of that protocol, as the next section demonstrates.

Python Digs Sequences

The philosophy of the Python data model is to cooperate with essential protocols as much as possible. When it comes to sequences, Python tries hard to work with even the simplest implementations.

Figure 11-1 shows how the formal Sequence interface is defined as an ABC.

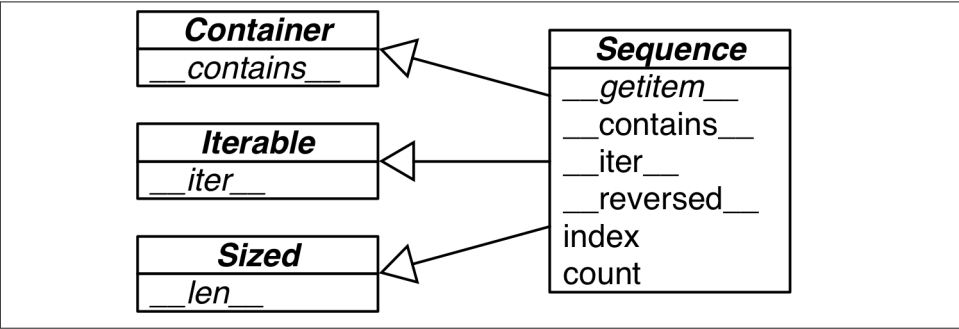


Figure 11-1. UML class diagram for the Sequence ABC and related abstract classes from `collections.abc`. Inheritance arrows point from subclass to its superclasses. Names in italic are abstract methods.

Now, take a look at the `Foo` class in Example 11-3. It does not inherit from `abc.Sequence`, and it only implements one method of the sequence protocol: `__getitem__` (`__len__` is missing).

Example 11-3. Partial sequence protocol implementation with `__getitem__`: enough for item access, iteration, and the `in` operator

```
>>> class Foo:
...     def __getitem__(self, pos):
...         return range(0, 30, 10)[pos]
```

2. Issue16518: “add buffer protocol to glossary” was actually resolved by replacing many mentions of “object that supports the buffer protocol/interface/API” with “bytes-like object”; a follow-up issue is “Other mentions of the buffer protocol”.

```

...
>>> f[1]
10
>>> f = Foo()
>>> for i in f: print(i)
...
0
10
20
>>> 20 in f
True
>>> 15 in f
False

```

There is no method `__iter__` yet `Foo` instances are iterable because—as a fallback—when Python sees a `__getitem__` method, it tries to iterate over the object by calling that method with integer indexes starting with 0. Because Python is smart enough to iterate over `Foo` instances, it can also make the `in` operator work even if `Foo` has no `__contains__` method: it does a full scan to check if an item is present.

In summary, given the importance of the sequence protocol, in the absence `__iter__` and `__contains__` Python still manages to make iteration and the `in` operator work by invoking `__getitem__`.

Our original `FrenchDeck` from [Chapter 1](#) does not subclass from `abc.Sequence` either, but it does implement both methods of the sequence protocol: `__getitem__` and `__len__`. See [Example 11-4](#).

Example 11-4. A deck as a sequence of cards (same as [Example 1-1](#))

```

import collections

Card = collections.namedtuple('Card', ['rank', 'suit'])

class FrenchDeck:
    ranks = [str(n) for n in range(2, 11)] + list('JQKA')
    suits = 'spades diamonds clubs hearts'.split()

    def __init__(self):
        self._cards = [Card(rank, suit) for suit in self.suits
                        for rank in self.ranks]

    def __len__(self):
        return len(self._cards)

    def __getitem__(self, position):
        return self._cards[position]

```

A good part of the demos in [Chapter 1](#) work because of the special treatment Python gives to anything vaguely resembling a sequence. Iteration in Python represents an

extreme form of duck typing: the interpreter tries two different methods to iterate over objects.

Now let's study another example emphasizing the dynamic nature of protocols.

Monkey-Patching to Implement a Protocol at Runtime

The FrenchDeck class from [Example 11-4](#) has a major flaw: it cannot be shuffled. Years ago when I first wrote the FrenchDeck example I did implement a `shuffle` method. Later I had a Pythonic insight: if a FrenchDeck acts like a sequence, then it doesn't need its own `shuffle` method because there is already `random.shuffle`, [documented](#) as “Shuffle the sequence *x* in place.”



When you follow established protocols, you improve your chances of leveraging existing standard library and third-party code, thanks to duck typing.

The standard `random.shuffle` function is used like this:

```
>>> from random import shuffle
>>> l = list(range(10))
>>> shuffle(l)
>>> l
[5, 2, 9, 7, 8, 3, 1, 4, 0, 6]
```

However, if we try to shuffle a FrenchDeck instance, we get an exception, as in [Example 11-5](#).

Example 11-5. random.shuffle cannot handle FrenchDeck

```
>>> from random import shuffle
>>> from frenchdeck import FrenchDeck
>>> deck = FrenchDeck()
>>> shuffle(deck)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File ".../python3.3/random.py", line 265, in shuffle
    x[i], x[j] = x[j], x[i]
TypeError: 'FrenchDeck' object does not support item assignment
```

The error message is quite clear: “'FrenchDeck' object does not support item assignment.” The problem is that `shuffle` operates by swapping items inside the collection, and FrenchDeck only implements the *immutable* sequence protocol. Mutable sequences must also provide a `__setitem__` method.

Because Python is dynamic, we can fix this at runtime, even at the interactive console. **Example 11-6** shows how to do it.

*Example 11-6. Monkey patching FrenchDeck to make it mutable and compatible with random.shuffle (continuing from **Example 11-5**)*

```
>>> def set_card(deck, position, card): ❶
...     deck._cards[position] = card
...
>>> FrenchDeck.__setitem__ = set_card ❷
>>> shuffle(deck) ❸
>>> deck[:5]
[Card(rank='3', suit='hearts'), Card(rank='4', suit='diamonds'), Card(rank='4',
suit='clubs'), Card(rank='7', suit='hearts'), Card(rank='9', suit='spades')]
```

- ❶ Create a function that takes deck, position, and card as arguments.
- ❷ Assign that function to an attribute named `__setitem__` in the FrenchDeck class.
- ❸ deck can now be sorted because FrenchDeck now implements the necessary method of the mutable sequence protocol.

The signature of the `__setitem__` special method is defined in *The Python Language Reference* in “3.3.6. Emulating container types”. Here we named the arguments `deck`, `position`, `card`—and not `self`, `key`, `value` as in the language reference—to show that every Python method starts life as a plain function, and naming the first argument `self` is merely a convention. This is OK in a console session, but in a Python source file it’s much better to use `self`, `key`, and `value` as documented.

The trick is that `set_card` knows that the deck object has an attribute named `_cards`, and `_cards` must be a mutable sequence. The `set_card` function is then attached to the FrenchDeck class as the `__setitem__` special method. This is an example of *monkey patching*: changing a class or module at runtime, without touching the source code. Monkey patching is powerful, but the code that does the actual patching is very tightly coupled with the program to be patched, often handling private and undocumented parts.

Besides being an example of monkey patching, **Example 11-6** highlights that protocols are dynamic: `random.shuffle` doesn’t care what type of argument it gets, it only needs the object to implement part of the mutable sequence protocol. It doesn’t even matter if the object was “born” with the necessary methods or if they were somehow acquired later.

The theme of this chapter so far has been “duck typing”: operating with objects regardless of their types, as long as they implement certain protocols.

When we did present diagrams with ABCs, the intent was to show how the protocols are related to the explicit interfaces documented in the abstract classes, but we did not actually inherit from any ABC so far.

In the following sections, we will leverage ABCs directly, and not just as documentation.

Alex Martelli's Waterfowl

After reviewing the usual protocol-style interfaces of Python, we move to ABCs. But before diving into examples and details, Alex Martelli explains in a guest essay why ABCs were a great addition to Python.



I am very grateful to Alex Martelli. He was already the most cited person in this book before he became one of the technical editors. His insights have been invaluable, and then he offered to write this essay. We are incredibly lucky to have him. Take it away, Alex!

Waterfowl and ABCs

By Alex Martelli

I've been [credited on Wikipedia](#) for helping spread the helpful meme and sound-bite “*duck typing*” (i.e, ignoring an object's actual type, focusing instead on ensuring that the object implements the method names, signatures, and semantics required for its intended use).

In Python, this mostly boils down to avoiding the use of `isinstance` to check the object's type (not to mention the even worse approach of checking, for example, whether `type(foo) is bar`—which is rightly anathema as it inhibits even the simplest forms of inheritance!).

The overall *duck typing* approach remains quite useful in many contexts—and yet, in many others, an often preferable one has evolved over time. And herein lies a tale...

In recent generations, the taxonomy of genus and species (including but not limited to the family of waterfowl known as Anatidae) has mostly been driven by *phenetics*—an approach focused on similarities of morphology and behavior... chiefly, *observable* traits. The analogy to “duck typing” was strong.

However, parallel evolution can often produce similar traits, both morphological and behavioral ones, among species that are actually unrelated, but just happened to evolve in similar, though separate, ecological niches. Similar “accidental similarities” happen in programming, too—for example, consider the classic OOP example:

```

class Artist:
    def draw(self): ...

class Gunslinger:
    def draw(self): ...

class Lottery:
    def draw(self): ...

```

Clearly, the mere existence of a method called `draw`, callable without arguments, is far from sufficient to assure us that two objects `x` and `y` such that `x.draw()` and `y.draw()` can be called are in any way exchangeable or abstractly equivalent—nothing about the similarity of the semantics resulting from such calls can be inferred. Rather, we need a knowledgeable programmer to somehow positively *assert* that such an equivalence holds at some level!

In biology (and other disciplines) this issue has led to the emergence (and, on many facets, the dominance) of an approach that's an alternative to phenetics, known as *cladistics*—focusing taxonomical choices on characteristics that are inherited from common ancestors, rather than ones that are independently evolved. (Cheap and rapid DNA sequencing can make cladistics highly practical in many more cases, in recent years.)

For example, sheldgeese (once classified as being closer to other geese) and shelducks (once classified as being closer to other ducks) are now grouped together within the subfamily Tadornidae (implying they're closer to each other than to any other Anatidae, as they share a closer common ancestor). Furthermore, DNA analysis has shown, in particular, that the white-winged wood duck is not as close to the Muscovy duck (the latter being a shelduck) as similarity in looks and behavior had long suggested—so the wood duck was reclassified into its own genus, and entirely out of the subfamily!

Does this matter? It depends on the context! For such purposes as deciding how best to cook a waterfowl once you've bagged it, for example, specific observable traits (not all of them—plumage, for example, is *de minimis* in such a context), mostly texture and flavor (old-fashioned phenetics!), may be far more relevant than cladistics. But for other issues, such as susceptibility to different pathogens (whether you're trying to raise waterfowl in captivity, or preserve them in the wild), DNA closeness can matter much more...

So, by very loose analogy with these taxonomic revolutions in the world of waterfowls, I'm recommending supplementing (not entirely replacing—in certain contexts it shall still serve) good old *duck typing* with... *goose typing*!

What *goose typing* means is: `isinstance(obj, cls)` is now just fine... as long as `cls` is an abstract base class—in other words, `cls's metaclass is abc.ABCMeta`.

You can find many useful existing abstract classes in `collections.abc` (and additional ones in the `numbers` module of *The Python Standard Library*).³

Among the many conceptual advantages of ABCs over concrete classes (e.g., Scott Meyer’s “all non-leaf classes should be abstract”—see [Item 33](#) in his book, *More Effective C++*), Python’s ABCs add one major practical advantage: the `register` class method, which lets end-user code “declare” that a certain class becomes a “virtual” subclass of an ABC (for this purpose the registered class must meet the ABC’s method name and signature requirements, and more importantly the underlying semantic contract—but it need not have been developed with any awareness of the ABC, and in particular need not inherit from it!). This goes a long way toward breaking the rigidity and strong coupling that make inheritance something to use with much more caution than typically practiced by most OOP programmers...

Sometimes you don’t even need to register a class for an ABC to recognize it as a subclass!

That’s the case for the ABCs whose essence boils down to a few special methods. For example:

```
>>> class Struggle:
...     def __len__(self): return 23
...
>>> from collections import abc
>>> isinstance(Struggle(), abc.Sized)
True
```

As you see, `abc.Sized` recognizes `Struggle` as “a subclass,” with no need for registration, as implementing the special method named `__len__` is all it takes (it’s supposed to be implemented with the proper syntax—callable without arguments—and semantics—returning a nonnegative integer denoting an object’s “length”; any code that implements a specially named method, such as `__len__`, with arbitrary, non-compliant syntax and semantics has much worse problems anyway).

So, here’s my valediction: whenever you’re implementing a class embodying any of the concepts represented in the ABCs in `numbers`, `collections.abc`, or other framework you may be using, be sure (if needed) to subclass it from, or register it into, the corresponding ABC. At the start of your programs using some library or framework defining classes which have omitted to do that, perform the registrations yourself; then, when you must check for (most typically) an argument being, e.g., “a sequence,” check whether:

```
isinstance(the_arg, collections.abc.Sequence)
```

3. You can also, of course, define your own ABCs—but I would discourage all but the most advanced Pythonistas from going that route, just as I would discourage them from defining their own custom metaclasses... and even for said “most advanced Pythonistas,” those of us sporting deep mastery of every fold and crease in the language, these are not tools for frequent use: such “deep metaprogramming,” if ever appropriate, is intended for authors of broad frameworks meant to be independently extended by vast numbers of separate development teams... less than 1% of “most advanced Pythonistas” may ever need that! — *A.M.*

And, *don't* define custom ABCs (or metaclasses) in production code... if you feel the urge to do so, I'd bet it's likely to be a case of “all problems look like a nail”-syndrome for somebody who just got a shiny new hammer—you (and future maintainers of your code) will be much happier sticking with straightforward and simple code, eschewing such depths. *Valē!*

Besides coining the “goose typing,” Alex makes the point that inheriting from an ABC is more than implementing the required methods: it's also a clear declaration of intent by the developer. That intent can also be made explicit through registering a virtual subclass.

In addition, the use of `isinstance` and `issubclass` becomes more acceptable to test against ABCs. In the past, these functions worked against duck typing, but with ABCs they become more flexible. After all, if a component does not implement an ABC by subclassing, it can always be registered after the fact so it passes those explicit type checks.

However, even with ABCs, you should beware that excessive use of `isinstance` checks may be a *code smell*—a symptom of bad OO design. It's usually *not* OK to have a chain of `if/elif/elif` with `isinstance` checks performing different actions depending on the type of an object: you should be using polymorphism for that—i.e., designing your classes so that the interpreter dispatches calls to the proper methods, instead of you hardcoding the dispatch logic in `if/elif/elif` blocks.



There is a common, practical exception to the preceding recommendation: some Python APIs accept a single `str` or a sequence of `str` items; if it's just a single `str`, you want to wrap it in a `list`, to ease processing. Because `str` is a sequence type, the simplest way to distinguish it from any other immutable sequence is to do an explicit `isinstance(x, str)` check.⁴

On the other hand, it's usually OK to perform an `isinstance` check against an ABC if you must enforce an API contract: “Dude, you have to implement this if you want to call me,” as technical reviewer Lennart Regebro put it. That's particularly useful in systems that have a plug-in architecture. Outside of frameworks, duck typing is often simpler and more flexible than type checks.

4. Unfortunately, in Python 3.4, there is no ABC that helps distinguish a `str` from `tuple` or other immutable sequences, so we must test against `str`. In Python 2, the `basestr` type exists to help with tests like these. It's not an ABC, but it's a superclass of both `str` and `unicode`; however, in Python 3, `basestr` is gone. Curiously, there is in Python 3 a `collections.abc.ByteString` type, but it only helps detecting bytes and `bytearray`.

For example, in several classes in this book, when I needed to take a sequence of items and process them as a list, instead of requiring a list argument by type checking, I simply took the argument and immediately built a list from it: that way I can accept any iterable, and if the argument is not iterable, the call will fail soon enough with a very clear message. One example of this code pattern is in the `__init__` method in [Example 11-13](#), later in this chapter. Of course, this approach wouldn't work if the sequence argument shouldn't be copied, either because it's too large or because my code needs to change it in place. Then an `instance(x, abc.MutableSequence)` would be better. If any iterable is acceptable, then calling `iter(x)` to obtain an iterator would be the way to go, as we'll see in [“Why Sequences Are Iterable: The iter Function” on page 404](#).

Another example is how you might imitate the handling of the `field_names` argument in `collections.namedtuple`: `field_names` accepts a single string with identifiers separated by spaces or commas, or a sequence of identifiers. It might be tempting to use `isinstance`, but [Example 11-7](#) shows how I'd do it using duck typing.⁵

Example 11-7. Duck typing to handle a string or an iterable of strings

```
try: ❶
    field_names = field_names.replace(',', ' ').split() ❷
except AttributeError: ❸
    pass ❹
field_names = tuple(field_names) ❺
```

- ❶ Assume it's a string (EAFP = it's easier to ask forgiveness than permission).
- ❷ Convert commas to spaces and split the result into a list of names.
- ❸ Sorry, `field_names` doesn't quack like a `str`... there's either no `.replace`, or it returns something we can't `.split`.
- ❹ Now we assume it's already an iterable of names.
- ❺ To make sure it's an iterable and to keep our own copy, create a tuple out of what we have.

Finally, in his essay, Alex reinforces more than once the need for restraint in the creation of ABCs. An ABC epidemic would be disastrous, imposing excessive ceremony in a language that became popular because it's practical and pragmatic. During the *Fluent Python* review process, Alex wrote:

ABCs are meant to encapsulate very general concepts, abstractions, introduced by a framework—things like “a sequence” and “an exact number.” [Readers] most likely don't need to write any new ABCs, just use existing ones correctly, to get 99.9% of the benefits without serious risk of misdesign.

5. This snippet was extracted from [Example 21-2](#).

Now let's see goose typing in practice.

Subclassing an ABC

Following Martelli's advice, we'll leverage an existing ABC, `collections.MutableSequence`, before daring to invent our own. In [Example 11-8](#), `FrenchDeck2` is explicitly declared a subclass of `collections.MutableSequence`.

Example 11-8. frenchdeck2.py: FrenchDeck2, a subclass of collections.MutableSequence

```
import collections

Card = collections.namedtuple('Card', ['rank', 'suit'])

class FrenchDeck2(collections.MutableSequence):
    ranks = [str(n) for n in range(2, 11)] + list('JQKA')
    suits = 'spades diamonds clubs hearts'.split()

    def __init__(self):
        self._cards = [Card(rank, suit) for suit in self.suits
                        for rank in self.ranks]

    def __len__(self):
        return len(self._cards)

    def __getitem__(self, position):
        return self._cards[position]

    def __setitem__(self, position, value): # ❶
        self._cards[position] = value

    def __delitem__(self, position): # ❷
        del self._cards[position]

    def insert(self, position, value): # ❸
        self._cards.insert(position, value)
```

- ❶ `__setitem__` is all we need to enable shuffling...
- ❷ But subclassing `MutableSequence` forces us to implement `__delitem__`, an abstract method of that ABC.
- ❸ We are also required to implement `insert`, the third abstract method of `MutableSequence`.

Python does not check for the implementation of the abstract methods at import time (when the *frenchdeck2.py* module is loaded and compiled), but only at runtime when we actually try to instantiate `FrenchDeck2`. Then, if we fail to implement any abstract method, we get a `TypeError` exception with a message such as "Can't instantiate

abstract class `FrenchDeck2` with abstract methods `__delitem__`, `insert`". That's why we must implement `__delitem__` and `insert`, even if our `FrenchDeck2` examples do not need those behaviors: the `MutableSequence` ABC demands them.

As [Figure 11-2](#) shows, not all methods of the `Sequence` and `MutableSequence` ABCs are abstract.

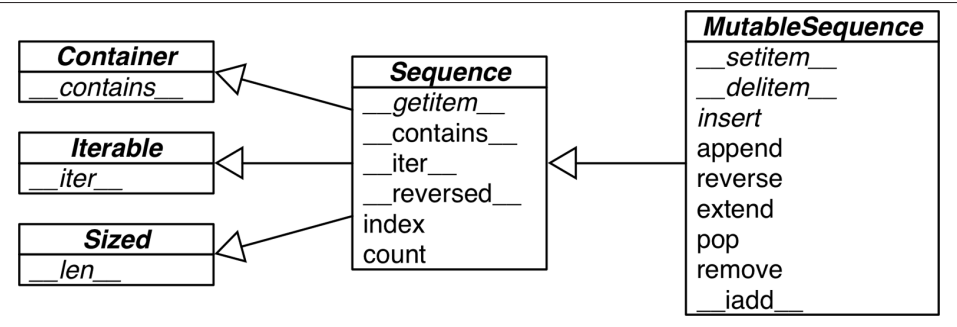


Figure 11-2. UML class diagram for the `MutableSequence` ABC and its superclasses from `collections.abc` (inheritance arrows point from subclasses to ancestors; names in italic are abstract classes and abstract methods)

From `Sequence`, `FrenchDeck2` inherits the following ready-to-use concrete methods: `__contains__`, `__iter__`, `__reversed__`, `index`, and `count`. From `MutableSequence`, it gets `append`, `reverse`, `extend`, `pop`, `remove`, and `__iadd__`.

The concrete methods in each `collections.abc` ABC are implemented in terms of the public interface of the class, so they work without any knowledge of the internal structure of instances.



As the coder of a concrete subclass, you may be able to override methods inherited from ABCs with more efficient implementations. For example, `__contains__` works by doing a full scan of the sequence, but if your concrete sequence keeps its items sorted, you can write a faster `__contains__` that does a binary search using `bisect` function (see [“Managing Ordered Sequences with `bisect`” on page 44](#)).

To use ABCs well, you need to know what's available. We'll review the collections ABCs next.

ABCs in the Standard Library

Since Python 2.6, ABCs are available in the standard library. Most are defined in the `collections.abc` module, but there are others. You can find ABCs in the `numbers` and `io` packages, for example. But the most widely used is `collections.abc`. Let's see what is available there.

ABCs in `collections.abc`



There are two modules named `abc` in the standard library. Here we are talking about `collections.abc`. To reduce loading time, in Python 3.4, it's implemented outside of the `collections` package, in *Lib/_collections_abc.py*, so it's imported separately from `collections`. The other `abc` module is just `abc` (i.e., *Lib/abc.py*) where the `abc.ABC` class is defined. Every ABC depends on it, but we don't need to import it ourselves except to create a new ABC.

Figure 11-3 is a summary UML class diagram (without attribute names) of all 16 ABCs defined in `collections.abc` as of Python 3.4. The official documentation of `collections.abc` has a *nice table* summarizing the ABCs, their relationships, and their abstract and concrete methods (called “mixin methods”). There is plenty of multiple inheritance going on in **Figure 11-3**. We'll devote most of **Chapter 12** to multiple inheritance, but for now it's enough to say that it is usually not a problem when ABCs are concerned.⁶

6. Multiple inheritance was *considered harmful* and excluded from Java, except for interfaces: Java interfaces can extend multiple interfaces, and Java classes can implement multiple interfaces.

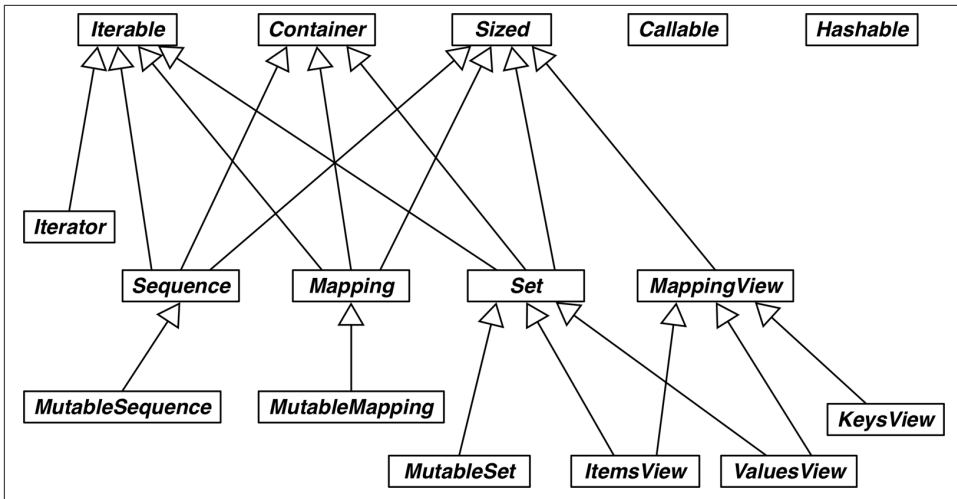


Figure 11-3. UML class diagram for ABCs in `collections.abc`

Let's review the clusters in [Figure 11-3](#):

Iterable, Container, and Sized

Every collection should either inherit from these ABCs or at least implement compatible protocols. `Iterable` supports iteration with `__iter__`, `Container` supports the in operator with `__contains__`, and `Sized` supports `len()` with `__len__`.

Sequence, Mapping, and Set

These are the main immutable collection types, and each has a mutable subclass. A detailed diagram for `MutableSequence` is in [Figure 11-2](#); for `MutableMapping` and `MutableSet`, there are diagrams in [Chapter 3](#) (Figures 3-1 and 3-2).

MappingView

In Python 3, the objects returned from the mapping methods `.items()`, `.keys()`, and `.values()` inherit from `ItemsView`, `KeysView`, and `ValuesView`, respectively. The first two also inherit the rich interface of `Set`, with all the operators we saw in [“Set Operations”](#) on page 82.

Callable and Hashable

These ABCs are not so closely related to collections, but `collections.abc` was the first package to define ABCs in the standard library, and these two were deemed important enough to be included. I've never seen subclasses of either `Callable` or

Hashable. Their main use is to support the `isinstance` built-in as a safe way of determining whether an object is callable or hashable.⁷

Iterator

Note that iterator subclasses `Iterable`. We discuss this further in [Chapter 14](#).

After the `collections.abc` package, the most useful package of ABCs in the standard library is `numbers`, covered next.

The Numbers Tower of ABCs

The `numbers` package defines the so-called “numerical tower” (i.e., this linear hierarchy of ABCs), where `Number` is the topmost superclass, `Complex` is its immediate subclass, and so on, down to `Integral`:

- `Number`
- `Complex`
- `Real`
- `Rational`
- `Integral`

So if you need to check for an integer, use `isinstance(x, numbers.Integral)` to accept `int`, `bool` (which subclasses `int`) or other integer types that may be provided by external libraries that register their types with the `numbers` ABCs. And to satisfy your check, you or the users of your API may always register any compatible type as a virtual subclass of `numbers.Integral`.

If, on the other hand, a value can be a floating-point type, you write `isinstance(x, numbers.Real)`, and your code will happily take `bool`, `int`, `float`, `fractions.Fraction`, or any other noncomplex numerical type provided by an external library, such as NumPy, which is suitably registered.



Somewhat surprisingly, `decimal.Decimal` is not registered as a virtual subclass of `numbers.Real`. The reason is that, if you need the precision of `Decimal` in your program, then you want to be protected from accidental mixing of decimals with other less precise numeric types, particularly floats.

7. For callable detection, there is the `callable()` built-in function—but there is no equivalent `hashable()` function, so `isinstance(my_obj, Hashable)` is the preferred way to test for a hashable object.

After looking at some existing ABCs, let's practice goose typing by implementing an ABC from scratch and putting it to use. The goal here is not to encourage everyone to start coding ABCs left and right, but to learn how to read the source code of the ABCs you'll find in the standard library and other packages.

Defining and Using an ABC

To justify creating an ABC, we need to come up with a context for using it as an extension point in a framework. So here is our context: imagine you need to display advertisements on a website or a mobile app in random order, but without repeating an ad before the full inventory of ads is shown. Now let's assume we are building an ad management framework called ADAM. One of its requirements is to support user-provided nonrepeating random-picking classes.⁸ To make it clear to ADAM users what is expected of a “nonrepeating random-picking” component, we'll define an ABC.

Taking a clue from “stack” and “queue” (which describe abstract interfaces in terms of physical arrangements of objects), I will use a real-world metaphor to name our ABC: bingo cages and lottery blowers are machines designed to pick items at random from a finite set, without repeating, until the set is exhausted.

The ABC will be named `Tombola`, after the Italian name of bingo and the tumbling container that mixes the numbers.⁹

The `Tombola` ABC has four methods. The two abstract methods are:

- `.load(...)`: put items into the container.
- `.pick()`: remove one item at random from the container, returning it.

The concrete methods are:

- `.loaded()`: return `True` if there is at least one item in the container.
- `.inspect()`: return a sorted `tuple` built from the items currently in the container, without changing its contents (its internal ordering is not preserved).

Figure 11-4 shows the `Tombola` ABC and three concrete implementations.

8. Perhaps the client needs to audit the randomizer; or the agency wants to provide a rigged one. You never know...

9. The Oxford English Dictionary defines *tombola* as “A kind of lottery resembling lotto.”

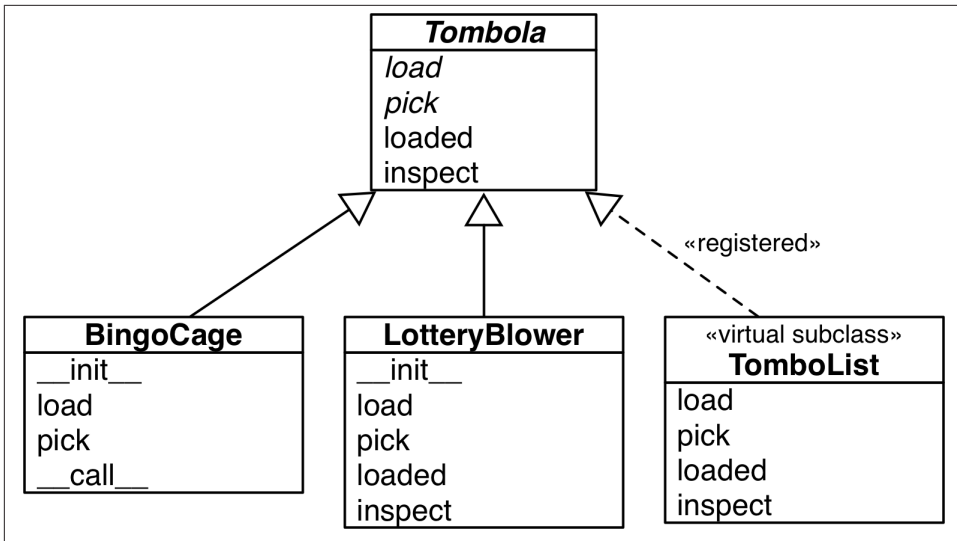


Figure 11-4. UML diagram for an ABC and three subclasses. The name of the *Tombola* ABC and its abstract methods are written in italics, per UML conventions. The dashed arrow is used for interface implementation, here we are using it to show that *TomboList* is a virtual subclass of *Tombola* because it is registered, as we will see later in this chapter.¹⁰

Example 11-9 shows the definition of the *TomboLa* ABC.

Example 11-9. *tombola.py*: *Tombola* is an ABC with two abstract methods and two concrete methods

```

import abc

class Tombola(abc.ABC): ❶

    @abc.abstractmethod
    def load(self, iterable): ❷
        """Add items from an iterable."""

    @abc.abstractmethod
    def pick(self): ❸
        """Remove item at random, returning it.

        This method should raise `LookupError` when the instance is empty.
        """
  
```

10. «registered» and «virtual subclass» are not standard UML words. We are using them to represent a class relationship that is specific to Python.


```

def loaded(self): ❹
    """Return `True` if there's at least 1 item, `False` otherwise."""
    return bool(self.inspect()) ❺

def inspect(self):
    """Return a sorted tuple with the items currently inside."""
    items = []
    while True: ❻
        try:
            items.append(self.pick())
        except LookupError:
            break
    self.load(items) ❼
    return tuple(sorted(items))

```

- ❶ To define an ABC, subclass `abc.ABC`.
- ❷ An abstract method is marked with the `@abstractmethod` decorator, and often its body is empty except for a docstring.¹¹
- ❸ The docstring instructs implementers to raise `LookupError` if there are no items to pick.
- ❹ An ABC may include concrete methods.
- ❺ Concrete methods in an ABC must rely only on the interface defined by the ABC (i.e., other concrete or abstract methods or properties of the ABC).
- ❻ We can't know how concrete subclasses will store the items, but we can build the `inspect` result by emptying the `TomboLa` with successive calls to `.pick()`...
- ❼ ...then use `.load(...)` to put everything back.



An abstract method can actually have an implementation. Even if it does, subclasses will still be forced to override it, but they will be able to invoke the abstract method with `super()`, adding functionality to it instead of implementing from scratch. See the [abc module documentation](#) for details on `@abstractmethod` usage.

The `.inspect()` method in [Example 11-9](#) is perhaps a silly example, but it shows that, given `.pick()` and `.load(...)` we can inspect what's inside the `TomboLa` by picking all items and loading them back. The point of this example is to highlight that it's OK to provide concrete methods in ABCs, as long as they only depend on other methods in the interface. Being aware of their internal data structures, concrete subclasses of `Tom`

11. Before ABCs existed, abstract methods would use the statement `raise NotImplementedError` to signal that subclasses were responsible for their implementation.

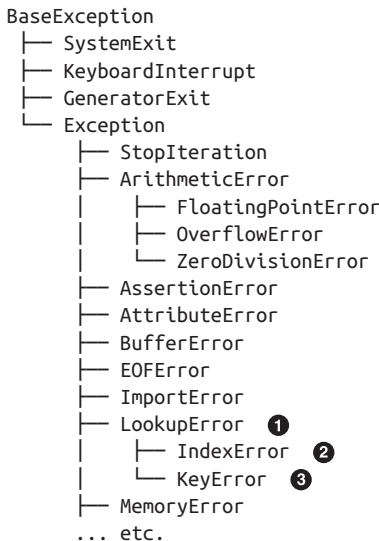
`bola` may always override `.inspect()` with a smarter implementation, but they don't have to.

The `.loaded()` method in [Example 11-9](#) may not be as silly, but it's expensive: it calls `.inspect()` to build the sorted tuple just to apply `bool()` on it. This works, but a concrete subclass can do much better, as we'll see.

Note that our roundabout implementation of `.inspect()` requires that we catch a `LookupError` thrown by `self.pick()`. The fact that `self.pick()` may raise `LookupError` is also part of its interface, but there is no way to declare this in Python, except in the documentation (see the docstring for the abstract `pick` method in [Example 11-9](#).)

I chose the `LookupError` exception because of its place in the Python hierarchy of exceptions in relation to `IndexError` and `KeyError`, the most likely exceptions to be raised by the data structures used to implement a concrete `Tombola`. Therefore, implementations can raise `LookupError`, `IndexError`, or `KeyError` to comply. See [Example 11-10](#) (for a complete tree, see “5.4. Exception hierarchy” of *The Python Standard Library*).

Example 11-10. Part of the Exception class hierarchy



- ❶ `LookupError` is the exception we handle in `Tombola.inspect`.
- ❷ `IndexError` is the `LookupError` subclass raised when we try to get an item from a sequence with an index beyond the last position.
- ❸ `KeyError` is raised when we use a nonexistent key to get an item from a mapping.

We now have our very own Tombo!a ABC. To witness the interface checking performed by an ABC, let's try to fool Tombo!a with a defective implementation in [Example 11-11](#).

Example 11-11. A fake Tombo!a doesn't go undetected

```
>>> from tombo!a import Tombo!a
>>> class Fake(Tombo!a): # ❶
...     def pick(self):
...         return 13
...
>>> Fake # ❷
<class '__main__.Fake'>
<class 'abc.ABC'>, <class 'object'>
>>> f = Fake() # ❸
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class Fake with abstract methods load
```

- ❶ Declare Fake as a subclass of Tombo!a.
- ❷ The class was created, no errors so far.
- ❸ TypeError is raised when we try to instantiate Fake. The message is very clear: Fake is considered abstract because it failed to implement load, one of the abstract methods declared in the Tombo!a ABC.

So we have our first ABC defined, and we put it to work validating a class. We'll soon subclass the Tombo!a ABC, but first we must cover some ABC coding rules.

ABC Syntax Details

The best way to declare an ABC is to subclass `abc.ABC` or any other ABC.

However, the `abc.ABC` class is new in Python 3.4, so if you are using an earlier version of Python—and it does not make sense to subclass another existing ABC—then you must use the `metaclass=` keyword in the class statement, pointing to `abc.ABCMeta` (not `abc.ABC`). In [Example 11-9](#), we would write:

```
class Tombo!a(metaclass=abc.ABCMeta):
    # ...
```

The `metaclass=` keyword argument was introduced in Python 3. In Python 2, you must use the `__metaclass__` class attribute:

```
class Tombo!a(object): # this is Python 2!!!
    __metaclass__ = abc.ABCMeta
    # ...
```

We'll explain metaclasses in [Chapter 21](#). For now, let's accept that a metaclass is a special kind of class, and agree that an ABC is a special kind of class; for example, "regular" classes don't check subclasses, so this is a special behavior of ABCs.

Besides the `@abstractmethod`, the `abc` module defines the `@abstractclassmethod`, `@abstractstaticmethod`, and `@abstractproperty` decorators. However, these last three are deprecated since Python 3.3, when it became possible to stack decorators on top of `@abstractmethod`, making the others redundant. For example, the preferred way to declare an abstract class method is:

```
class MyABC(abc.ABC):
    @classmethod
    @abc.abstractmethod
    def an_abstract_classmethod(cls, ...):
        pass
```



The order of stacked function decorators usually matters, and in the case of `@abstractmethod`, the documentation is explicit:

When `abstractmethod()` is applied in combination with other method descriptors, it should be applied as the innermost decorator, ...¹²

In other words, no other decorator may appear between `@abstractmethod` and the `def` statement.

Now that we got these ABC syntax issues covered, let's put `Tombola` to use by implementing some full-fledged concrete descendants of it.

Subclassing the `Tombola` ABC

Given the `Tombola` ABC, we'll now develop two concrete subclasses that satisfy its interface. These classes were pictured in [Figure 11-4](#), along with the virtual subclass to be discussed in the next section.

The `BingoCage` class in [Example 11-12](#) is a variation of [Example 5-8](#) using a better randomizer. This `BingoCage` implements the required abstract methods `load` and `pick`, inherits `loaded` from `Tombola`, overrides `inspect`, and adds `__call__`.

Example 11-12. `bingo.py`: `BingoCage` is a concrete subclass of `Tombola`

```
import random

from tombola import Tombola
```

12. `@abc.abstractmethod` entry in the `abc` module documentation.

```

class BingoCage(Tombola): ❶

    def __init__(self, items):
        self._randomizer = random.SystemRandom() ❷
        self._items = []
        self.load(items) ❸

    def load(self, items):
        self._items.extend(items)
        self._randomizer.shuffle(self._items) ❹

    def pick(self): ❺
        try:
            return self._items.pop()
        except IndexError:
            raise LookupError('pick from empty BingoCage')

    def __call__(self): ❻
        self.pick()

```

- ❶ This `BingoCage` class explicitly extends `Tombola`.
- ❷ Pretend we'll use this for online gaming. `random.SystemRandom` implements the random API on top of the `os.urandom(...)` function, which provides random bytes “suitable for cryptographic use” according to the [os module docs](#).
- ❸ Delegate initial loading to the `.load(...)` method.
- ❹ Instead of the plain `random.shuffle()` function, we use the `.shuffle()` method of our `SystemRandom` instance.
- ❺ `pick` is implemented as in [Example 5-8](#).
- ❻ `__call__` is also from [Example 5-8](#). It's not needed to satisfy the `Tombola` interface, but there's no harm in adding extra methods.

`BingoCage` inherits the expensive `loaded` and the silly `inspect` methods from `Tombola`. Both could be overridden with much faster one-liners, as in [Example 11-13](#). The point is: we can be lazy and just inherit the suboptimal concrete methods from an ABC. The methods inherited from `Tombola` are not as fast as they could be for `BingoCage`, but they do provide correct results for any `Tombola` subclass that correctly implements `pick` and `load`.

[Example 11-13](#) shows a very different but equally valid implementation of the `Tombola` interface. Instead of shuffling the “balls” and popping the last, `LotteryBlower` pops from a random position.

Example 11-13. lotto.py: LotteryBlower is a concrete subclass that overrides the inspect and loaded methods from Tombola

```
import random

from tombola import Tombola


class LotteryBlower(Tombola):

    def __init__(self, iterable):
        self._balls = list(iterable) ❶

    def load(self, iterable):
        self._balls.extend(iterable)

    def pick(self):
        try:
            position = random.randrange(len(self._balls)) ❷
        except ValueError:
            raise LookupError('pick from empty BingoCage')
        return self._balls.pop(position) ❸

    def loaded(self): ❹
        return bool(self._balls)

    def inspect(self): ❺
        return tuple(sorted(self._balls))
```

- ❶ The initializer accepts any iterable: the argument is used to build a list.
- ❷ The `random.randrange(...)` function raises `ValueError` if the range is empty, so we catch that and throw `LookupError` instead, to be compatible with `Tombola`.
- ❸ Otherwise the randomly selected item is popped from `self._balls`.
- ❹ Override `loaded` to avoid calling `inspect` (as `Tombola.loaded` does in [Example 11-9](#)). We can make it faster by working with `self._balls` directly—no need to build a whole sorted tuple.
- ❺ Override `inspect` with one-liner.

Example 11-13 illustrates an idiom worth mentioning: in `__init__`, `self._balls` stores `list(iterable)` and not just a reference to `iterable` (i.e., we did not merely assign `iterable` to `self._balls`). As mentioned before,¹³ this makes our `LotteryBlower` flexible because the `iterable` argument may be any iterable type. At the same time, we make sure to store its items in a `list` so we can `pop` items. And even if we always get lists as the `iterable` argument, `list(iterable)` produces a copy of the argument,

13. I gave this as an example of duck typing after Martelli's “[Waterfowl and ABCs](#)” on page 314.

which is a good practice considering we will be removing items from it and the client may not be expecting the `list` of items she provided to be changed.¹⁴

We now come to the crucial dynamic feature of goose typing: declaring virtual subclasses with the `register` method.

A Virtual Subclass of `Tombola`

An essential characteristic of goose typing—and the reason why it deserves a waterfowl name—is the ability to register a class as a *virtual subclass* of an ABC, even if it does not inherit from it. When doing so, we promise that the class faithfully implements the interface defined in the ABC—and Python will believe us without checking. If we lie, we’ll be caught by the usual runtime exceptions.

This is done by calling a `register` method on the ABC. The registered class then becomes a virtual subclass of the ABC, and will be recognized as such by functions like `issubclass` and `isinstance`, but it will not inherit any methods or attributes from the ABC.



Virtual subclasses do not inherit from their registered ABCs, and are not checked for conformance to the ABC interface at any time, not even when they are instantiated. It’s up to the subclass to actually implement all the methods needed to avoid runtime errors.

The `register` method is usually invoked as a plain function (see “[Usage of `register` in Practice](#)” on page 338), but it can also be used as a decorator. In [Example 11-14](#), we use the decorator syntax and implement `TombolList`, a virtual subclass of `Tombola` depicted in [Figure 11-5](#).

`TombolList` works as advertised, and the doctests that prove it are described in “[How the `Tombola` Subclasses Were Tested](#)” on page 335.

14. “[Defensive Programming with Mutable Parameters](#)” on page 232 in [Chapter 8](#) was devoted to the aliasing issue we just avoided here.

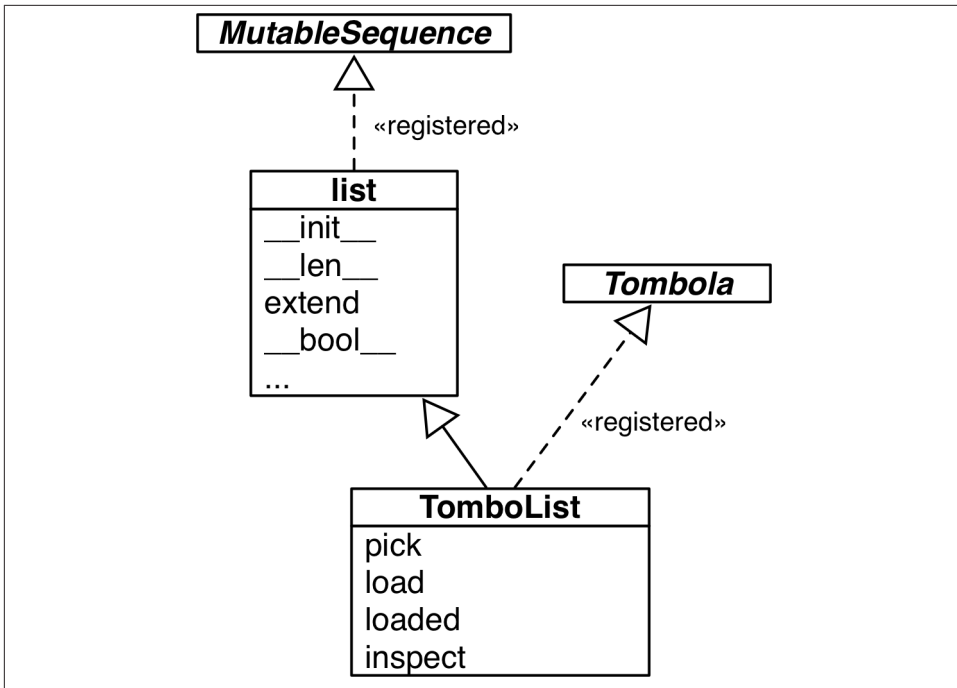


Figure 11-5. UML class diagram for the *TomboList*, a real subclass of *list* and a virtual subclass of *Tombola*

Example 11-14. *tombolist.py*: class *TomboList* is a virtual subclass of *Tombola*

```
from random import randrange
```

```
from tombola import Tombola
```

```
@Tombola.register # ❶
```

```
class TomboList(list): # ❷
```

```
    def pick(self):
```

```
        if self: # ❸
```

```
            position = randrange(len(self))
```

```
            return self.pop(position) # ❹
```

```
        else:
```

```
            raise LookupError('pop from empty TomboList')
```

```
    load = list.extend # ❺
```

```
    def loaded(self):
```

```
        return bool(self) # ❻
```

```
    def inspect(self):
```

```
        return tuple(sorted(self))
```



```
# Tombola.register(TombolList) # 7
```

- ❶ TombolList is registered as a virtual subclass of Tombola.
- ❷ TombolList extends list.
- ❸ TombolList inherits `__bool__` from list, and that returns True if the list is not empty.
- ❹ Our pick calls `self.pop`, inherited from list, passing a random item index.
- ❺ `TombolList.load` is the same as `list.extend`.
- ❻ `loaded` delegates to `bool`.¹⁵
- ❼ If you're using Python 3.3 or earlier, you can't use `.register` as a class decorator. You must use standard call syntax.

Note that because of the registration, the functions `issubclass` and `isinstance` act as if `TombolList` is a subclass of `Tombola`:

```
>>> from tombola import Tombola
>>> from tombolist import TombolList
>>> issubclass(TombolList, Tombola)
True
>>> t = TombolList(range(100))
>>> isinstance(t, Tombola)
True
```

However, inheritance is guided by a special class attribute named `__mro__`—the Method Resolution Order. It basically lists the class and its superclasses in the order Python uses to search for methods.¹⁶ If you inspect the `__mro__` of `TombolList`, you'll see that it lists only the “real” superclasses—`list` and `object`:

```
>>> TombolList.__mro__
(<class 'tombolist.TombolList'>, <class 'list'>, <class 'object'>)
```

`Tombola` is not in `TombolList.__mro__`, so `TombolList` does not inherit any methods from `Tombola`.

15. The same trick I used with `load` doesn't work with `loaded`, because the `list` type does not implement `__bool__`, the method I'd have to bind to `loaded`. On the other hand, the `bool` built-in function doesn't need `__bool__` to work because it can also use `__len__`. See “4.1. Truth Value Testing” in the “Built-in Types” chapter.

16. There is a whole section explaining the `__mro__` class attribute in “Multiple Inheritance and Method Resolution Order” on page 351. Right now, this quick explanation will do.

As I coded different classes to implement the same interface, I wanted a way to submit them all to the same suite of doctests. The next section shows how I leveraged the API of regular classes and ABCs to do it.

How the Tombola Subclasses Were Tested

The script I used to test the Tombola examples uses two class attributes that allow introspection of a class hierarchy:

`__subclasses__()`

Method that returns a list of the immediate subclasses of the class. The list does not include virtual subclasses.

`_abc_registry`

Data attribute—available only in ABCs—that is bound to a `WeakSet` with weak references to registered virtual subclasses of the abstract class.

To test all Tombola subclasses, I wrote a script to iterate over a list built from `Tombola.__subclasses__()` and `Tombola._abc_registry`, and bind each class to the name `ConcreteTombola` used in the doctests.

A successful run of the test script looks like this:

```
$ python3 tombola_runner.py
BingoCage      23 tests, 0 failed - OK
LotteryBlower  23 tests, 0 failed - OK
TumblingDrum   23 tests, 0 failed - OK
TomboList      23 tests, 0 failed - OK
```

The test script is [Example 11-15](#) and the doctests are in [Example 11-16](#).

Example 11-15. `tombola_runner.py`: test runner for Tombola subclasses

```
import doctest

from tombola import Tombola

# modules to test
import bingo, lotto, tombolist, drum ❶

TEST_FILE = 'tombola_tests.rst'
TEST_MSG = '{0:16} {1.attempted:2} tests, {1.failed:2} failed - {2}'

def main(argv):
    verbose = '-v' in argv
    real_subclasses = Tombola.__subclasses__() ❷
    virtual_subclasses = list(Tombola._abc_registry) ❸

    for cls in real_subclasses + virtual_subclasses: ❹
        test(cls, verbose)
```

```
def test(cls, verbose=False):

    res = doctest.testfile(
        TEST_FILE,
        globs={'ConcreteTombola': cls},    ❸
        verbose=verbose,
        optionflags=doctest.REPORT_ONLY_FIRST_FAILURE)
    tag = 'FAIL' if res.failed else 'OK'
    print(TEST_MSG.format(cls.__name__, res, tag))    ❹

if __name__ == '__main__':
    import sys
    main(sys.argv)
```

- ❶ Import modules containing real or virtual subclasses of `Tombola` for testing.
- ❷ `__subclasses__()` lists the direct descendants that are alive in memory. That's why we imported the modules to test, even if there is no further mention of them in the source code: to load the classes into memory.
- ❸ Build a list from `_abc_registry` (which is a `WeakSet`) so we can concatenate it with the result of `__subclasses__()`.
- ❹ Iterate over the subclasses found, passing each to the test function.
- ❺ The `cls` argument—the class to be tested—is bound to the name `ConcreteTombola` in the global namespace provided to run the doctest.
- ❻ The test result is printed with the name of the class, the number of tests attempted, tests failed, and an 'OK' or 'FAIL' label.

The doctest file is [Example 11-16](#).

Example 11-16. tombola_tests.rst: doctests for Tombola subclasses

```
=====
Tombola tests
=====
```

Every concrete subclass of `Tombola` should pass these tests.

Create and load instance from `iterable::`

```
>>> balls = list(range(3))
>>> globe = ConcreteTombola(balls)
>>> globe.loaded()
True
>>> globe.inspect()
(0, 1, 2)
```

Pick and collect balls::

```
>>> picks = []
>>> picks.append(globe.pick())
>>> picks.append(globe.pick())
>>> picks.append(globe.pick())
```

Check state and results::

```
>>> globe.loaded()
False
>>> sorted(picks) == balls
True
```

Reload::

```
>>> globe.load(balls)
>>> globe.loaded()
True
>>> picks = [globe.pick() for i in balls]
>>> globe.loaded()
False
```

Check that `LookupError` (or a subclass) is the exception thrown when the device is empty::

```
>>> globe = ConcreteTombola([])
>>> try:
...     globe.pick()
... except LookupError as exc:
...     print('OK')
OK
```

Load and pick 100 balls to verify that they all come out::

```
>>> balls = list(range(100))
>>> globe = ConcreteTombola(balls)
>>> picks = []
>>> while globe.inspect():
...     picks.append(globe.pick())
>>> len(picks) == len(balls)
True
>>> set(picks) == set(balls)
True
```

Check that the order has changed and is not simply reversed::

```
>>> picks != balls
True
>>> picks[::-1] != balls
True
```

Note: the previous 2 tests have a *very* small chance of failing even if the implementation is OK. The probability of the 100 balls coming out, by chance, in the order they were inspect is 1/100!, or approximately 1.07e-158. It's much easier to win the Lotto or to become a billionaire working as a programmer.

THE END

This concludes our Tombola ABC case study. In the next section, we'll address how the register ABC function is used in the wild.

Usage of register in Practice

In [Example 11-14](#), we used `Tombola.register` as a class decorator. Prior to Python 3.3, `register` could not be used like that—it had to be called as a plain function after the class definition, as suggested by the comment at the end of [Example 11-14](#).

However, even if `register` can now be used as a decorator, it's more widely deployed as a function to register classes defined elsewhere. For example, in the [source code](#) for the `collections.abc` module, the built-in types `tuple`, `str`, `range`, and `memoryview` are registered as virtual subclasses of `Sequence` like this:

```
Sequence.register(tuple)
Sequence.register(str)
Sequence.register(range)
Sequence.register(memoryview)
```

Several other built-in types are registered to ABCs in `_collections_abc.py`. Those registrations happen only when that module is imported, which is OK because you'll have to import it anyway to get the ABCs: you need access to `MutableMapping` to be able to write `isinstance(my_dict, MutableMapping)`.

We'll wrap up this chapter by explaining a bit of ABC magic that Alex Martelli performed in [“Waterfowl and ABCs” on page 314](#).

Geese Can Behave as Ducks

In his *Waterfowl and ABCs* essay, Alex shows that a class can be recognized as a virtual subclass of an ABC even without registration. Here is his example again, with an added test using `issubclass`:

```
>>> class Struggle:
...     def __len__(self): return 23
...
>>> from collections import abc
>>> isinstance(Struggle(), abc.Sized)
True
>>> issubclass(Struggle, abc.Sized)
True
```

Class `Struggle` is considered a subclass of `abc.Sized` by the `issubclass` function (and, consequently, by `isinstance` as well) because `abc.Sized` implements a special class method named `__subclasshook__`. See [Example 11-17](#).

Example 11-17. Sized definition from the source code of `Lib/_collections_abc.py` (Python 3.4)

```
class Sized(metaclass=ABCMeta):

    __slots__ = ()

    @abstractmethod
    def __len__(self):
        return 0

    @classmethod
    def __subclasshook__(cls, C):
        if cls is Sized:
            if any("__len__" in B.__dict__ for B in C.__mro__): # ❶
                return True # ❷
            return NotImplemented # ❸
```

- ❶ If there is an attribute named `__len__` in the `__dict__` of any class listed in `C.__mro__` (i.e., `C` and its superclasses)...
- ❷ ...return `True`, signaling that `C` is a virtual subclass of `Sized`.
- ❸ Otherwise return `NotImplemented` to let the subclass check proceed.

If you are interested in the details of the subclass check, see the source code for the `ABCMeta.__subclasscheck__` method in `Lib/abc.py`. Beware: it has lots of ifs and two recursive calls.

The `__subclasshook__` adds some duck typing DNA to the whole goose typing proposition. You can have formal interface definitions with ABCs, you can make `isinstance` checks everywhere, and still have a completely unrelated class play along just because it implements a certain method (or because it does whatever it takes to convince a `__subclasshook__` to vouch for it). Of course, this only works for ABCs that do provide a `__subclasshook__`.

Is it a good idea to implement `__subclasshook__` in our own ABCs? Probably not. All the implementations of `__subclasshook__` I've seen in the Python source code are in ABCs like `Sized` that declare just one special method, and they simply check for that special method name. Given their “special” status, you can be pretty sure that any method named `__len__` does what you expect. But even in the realm of special methods and fundamental ABCs, it can be risky to make such assumptions. For example, mappings implement `__len__`, `__getitem__`, and `__iter__` but they are rightly not considered a subtype of `Sequence`, because you can't retrieve items using an integer offset and they make no guarantees about the ordering of items—except of course for `OrderedDict`, which preserves the insertion order, but does support item retrieval by offset either.

For ABCs that you and I may write, a `__subclasshook__` would be even less dependable. I am not ready to believe that any class named `Spam` that implements or inherits `load`, `pick`, `inspect`, and `loaded` is guaranteed to behave as a `Tombola`. It's better to let the programmer affirm it by subclassing `Spam` from `Tombola`, or at least registering: `Tombola.register(Spam)`. Of course, your `__subclasshook__` could also check method signatures and other features, but I just don't think it's worthwhile.

Chapter Summary

The goal of this chapter was to travel from the highly dynamic nature of informal interfaces—called protocols—visit the static interface declarations of ABCs, and conclude with the dynamic side of ABCs: virtual subclasses and dynamic subclass detection with `__subclasshook__`.

We started the journey by reviewing the traditional understanding of interfaces in the Python community. For most of the history of Python, we've been mindful of interfaces, but they were informal like the protocols from Smalltalk, and the official docs used language such as “foo protocol,” “foo interface,” and “foo-like object” interchangeably. Protocol-style interfaces have nothing to do with inheritance; each class stands alone when implementing a protocol. That's what interfaces look like when you embrace duck typing.

With [Example 11-3](#), we observed how deeply Python supports the sequence protocol. If a class implements `__getitem__` and nothing else, Python manages to iterate over it, and the `in` operator just works. We then went back to the old `FrenchDeck` example of [Chapter 1](#) to support shuffling by dynamically adding a method. This illustrated monkey patching and emphasized the dynamic nature of protocols. Again we saw how a partially implemented protocol can be useful: just adding `__setitem__` from the mutable sequence protocol allowed us to leverage a ready-to-use function from the standard library: `random.shuffle`. Being aware of existing protocols lets us make the most of the rich Python standard library.

Alex Martelli then introduced the term “goose typing”¹⁷ to describe a new style of Python programming. With “goose typing,” ABCs are used to make interfaces explicit and classes may claim to implement an interface by subclassing an ABC or by registering with it—without requiring the strong and static link of an inheritance relationship.

The FrenchDeck2 example made clear the main drawbacks and advantages of explicit ABCs. Inheriting from `abc.MutableSequence` forced us to implement two methods we did not really need: `insert` and `__delitem__`. On the other hand, even a Python newbie can look at FrenchDeck2 and see that it’s a mutable sequence. And, as bonus, we inherited 11 ready-to-use methods from `abc.MutableSequence` (five indirectly from `abc.Sequence`).

After a panoramic view of existing ABCs from `collections.abc` in Figure 11-3, we wrote an ABC from scratch. Doug Hellmann, creator of the cool PyMOTW.com (Python Module of the Week) explains the motivation:

By defining an abstract base class, a common API can be established for a set of subclasses. This capability is especially useful in situations where someone less familiar with the source for an application is going to provide plug-in extensions...¹⁸

Putting the TomboLa ABC to work, we created three concrete subclasses: two inheriting from TomboLa, the other a virtual subclass registered with it, all passing the same suite of tests.

In concluding the chapter, we mentioned how several built-in types are registered to ABCs in the `collections.abc` module so you can ask `isinstance(memoryview, abc.Sequence)` and get `True`, even if `memoryview` does not inherit from `abc.Sequence`. And finally we went over the `__subclasshook__` magic, which lets an ABC recognize any unregistered class as a subclass, as long as it passes a test that can be as simple or as complex as you like—the examples in the standard library merely check for method names.

To sum up, I’d like to restate Alex Martelli’s admonition that we should refrain from creating our own ABCs, except when we are building user-extensible frameworks—which most of the time we are not. On a daily basis, our contact with ABCs should be subclassing or registering classes with existing ABCs. Less often than subclassing or registering, we might use ABCs for `isinstance` checks. And even more rarely—if ever—we find occasion to write a new ABC from scratch.

After 15 years of Python, the first abstract class I ever wrote that is not a didactic example was the `Board` class of the [Pingo](https://pingsong.org/) project. The drivers that support different single board computers and controllers are subclasses of `Board`, thus sharing the same interface. In

17. Alex coined the expression “goose typing” and this is the first time ever it appears in a book!

18. PyMOTW, `abc` module page, section “Why use Abstract Base Classes?”

reality, although conceived and implemented as an abstract class, the `pingo.Board` class does not subclass `abc.ABC` as I write this.¹⁹ I intend to make `Board` an explicit `ABC` eventually—but there are more important things to do in the project.

Here is a fitting quote to end this chapter:

Although ABCs facilitate type checking, it's not something that you should overuse in a program. At its heart, Python is a dynamic language that gives you great flexibility. Trying to enforce type constraints everywhere tends to result in code that is more complicated than it needs to be. You should embrace Python's flexibility.²⁰

— David Beazley and Brian Jones
Python Cookbook

Or, as technical reviewer Leonardo Rocha wrote: “If you feel tempted to create a custom ABC, please first try to solve your problem through regular duck-typing.”

Further Reading

Beazley and Jones's *Python Cookbook, 3rd Edition* (O'Reilly) has a section about defining an ABC (Recipe 8.12). The book was written before Python 3.4, so they don't use the now preferred syntax when declaring ABCs by subclassing from `abc.ABC` instead of using the `metaclass` keyword. Apart from this small detail, the recipe covers the major ABC features very well, and ends with the valuable advice quoted at the end of the previous section.

The Python Standard Library by Example by Doug Hellmann (Addison-Wesley), has a chapter about the `abc` module. It's also available on the Web in Doug's excellent **Py-MOTW — Python Module of the Week**. Both the book and the site focus on Python 2; therefore, adjustments must be made if you are using Python 3. And for Python 3.4, remember that the only recommended ABC method decorator is `@abstractmethod`—the others were deprecated. The other quote about ABCs in the chapter summary is from Doug's site and book.

When using ABCs, multiple inheritance is not only common but practically inevitable, because each of the fundamental collection ABCs—`Sequence`, `Mapping`, and `Set`—extends multiple ABCs (see **Figure 11-3**). Therefore, **Chapter 12** is an important follow-up to this one.

PEP 3119 — Introducing Abstract Base Classes gives the rationale for ABCs, and **PEP 3141 - A Type Hierarchy for Numbers** presents the ABCs of the `numbers` module.

19. You'll find that in the Python standard library too: classes that are in fact abstract but nobody ever made them explicitly so.

20. *Python Cookbook, 3rd Edition* (O'Reilly), “Recipe 8.12. Defining an Interface or Abstract Base Class”, p. 276.

For a discussion of the pros and cons of dynamic typing, see Guido van Rossum's interview to Bill Venners in [“Contracts in Python: A Conversation with Guido van Rossum, Part IV”](#).

The `zope.interface` package provides a way of declaring interfaces, checking whether objects implement them, registering providers, and querying for providers of a given interface. The package started as a core piece of Zope 3, but it can and has been used outside of Zope. It is the basis of the flexible component architecture of large-scale Python projects like Twisted, Pyramid, and Plone. Lennart Regebro has a great introduction to `zope.interface` in [“A Python Component Architecture”](#). Baiju M wrote an entire book about it: [A Comprehensive Guide to Zope Component Architecture](#).

Soapbox

Type Hints

Probably the biggest news in the Python world in 2014 was that Guido van Rossum gave a green light to the implementation of optional static type checking using function annotations, similar to what the `Mypy` checker does. This happened in the Python-ideas mailing-list on August 15. The message is [Optional static typing — the crossroads](#). The next month, [PEP 484 - Type Hints](#) was published as a draft, authored by Guido.

The idea is to let programmers optionally use annotations to declare parameter and return types in function definitions. The key word here is *optionally*. You'd only add such annotations if you want the benefits and constraints that come with them, and you could put them in some functions but not in others.

On the surface, this may sound like what Microsoft did with TypeScript, its JavaScript superset, except that TypeScript goes much further: it adds new language constructs (e.g., modules, classes, explicit interfaces, etc.), allows typed variable declarations, and actually compiles down to plain JavaScript. As of this writing, the goals of optional static typing in Python are much less ambitious.

To understand the reach of this proposal, there is a key point that Guido makes in the historic August 15, 2014, email:

I am going to make one additional assumption: the main use cases will be linting, IDEs, and doc generation. These all have one thing in common: it should be possible to run a program even though it fails to type check. Also, adding types to a program should not hinder its performance (nor will it help :-).

So, it seems this is not such a radical move as it seems at first. [PEP 482 - Literature Overview for Type Hints](#) is referenced by [PEP 484 - Type Hints](#), and briefly documents type hints in third-party Python tools and in other languages.

Radical or not, type hints are upon us: support for PEP 484 in the form of a typing module is likely to land in Python 3.5 already. The way the proposal is worded and

implemented makes it clear that no existing code will stop running because of the lack of type hints—or their addition, for that matter.

Finally, PEP 484 clearly states:

It should also be emphasized that Python will remain a dynamically typed language, and the authors have no desire to ever make type hints mandatory, even by convention.

Is Python Weakly Typed?

Discussions about language typing disciplines are sometimes confused due to lack of a uniform terminology. Some writers (like Bill Venners in the interview with Guido mentioned in “[Further Reading](#)” on page 342), say that Python has weak typing, which puts it into the same category of JavaScript and PHP. A better way of talking about typing discipline is to consider two different axes:

Strong versus weak typing

If the language rarely performs implicit conversion of types, it’s considered strongly typed; if it often does it, it’s weakly typed. Java, C++, and Python are strongly typed. PHP, JavaScript, and Perl are weakly typed.

Static versus dynamic typing

If type-checking is performed at compile time, the language is statically typed; if it happens at runtime, it’s dynamically typed. Static typing requires type declarations (some modern languages use type inference to avoid some of that). Fortran and Lisp are the two oldest programming languages still alive and they use, respectively, static and dynamic typing.

Strong typing helps catch bugs early.

Here are some examples of why weak typing is bad:²¹

```
// this is JavaScript (tested with Node.js v0.10.33)
'' == '0'    // false
0 == ''     // true
0 == '0'    // true
'' < 0      // false
'' < '0'    // true
```

Python does not perform automatic coercion between strings and numbers, so the `==` expressions all result `False`—preserving the transitivity of `==`—and the `<` comparisons raise `TypeError` in Python 3.

Static typing makes it easier for tools (compilers, IDEs) to analyze code to detect errors and provide other services (optimization, refactoring, etc.). Dynamic typing increases opportunities for reuse, reducing line count, and allows interfaces to emerge naturally as protocols, instead of being imposed early on.

21. Adapted from Douglas Crockford’s *JavaScript: The Good Parts* (O’Reilly), Appendix B, p. 109.

To summarize, Python uses dynamic and strong typing. [PEP 484 - Type Hints](#) will not change that, but will allow API authors to add optional type annotations so that tools can perform some static type checking.

Monkey Patching

Monkey patching has a bad reputation. If abused, it can lead to systems that are hard to understand and maintain. The patch is usually tightly coupled with its target, making it brittle. Another problem is that two libraries that apply monkey-patches may step on each other's toes, with the second library to run destroying patches of the first.

But monkey patching can also be useful, for example, to make a class implement a protocol at runtime. The adapter design pattern solves the same problem by implementing a whole new class.

It's easy to monkey-patch Python code, but there are limitations. Unlike Ruby and JavaScript, Python does not let you monkey-patch the built-in types. I actually consider this an advantage, because you can be certain that a `str` object will always have those same methods. This limitation reduces the chance that external libraries try to apply conflicting patches.

Interfaces in Java, Go, and Ruby

Since C++ 2.0 (1989), abstract classes have been used to specify interfaces in that language. The designers of Java opted not to have multiple inheritance of classes, which precluded the use of abstract classes as interface specifications—because often a class needs to implement more than one interface. But they added the `interface` as a language construct, and a class can implement more than one interface—a form of multiple inheritance. Making interface definitions more explicit than ever was a great contribution of Java. With Java 8, an interface can provide method implementations, called [Default Methods](#). With this, Java interfaces became closer to abstract classes in C++ and Python.

The Go language has a completely different approach. First of all, there is no inheritance in Go. You can define interfaces, but you don't need (and you actually can't) explicitly say that a certain type implements an interface. The compiler determines that automatically. So what they have in Go could be called “static duck typing,” in the sense that interfaces are checked at compile time but what matters is what types actually implement.

Compared to Python, it's as if, in Go, every ABC implemented the `__subclasshook__` checking function names and signatures, and you never subclassed or registered an ABC. If we wanted Python to look more like Go, we would have to perform type checks on all function arguments. Some of the infrastructure is available (recall [“Function Annotations” on page 154](#)). Guido has already said he thinks it's OK to use those annotations for type checking—at least in support tools. See [“Soapbox” on page 163 in Chapter 5](#) for more about this.

Rubyists are firm believers in duck typing, and Ruby has no formal way to declare an interface or an abstract class, except to do the same we did in Python prior to 2.6: raise

`NotImplementedError` in the body of methods to make them abstract by forcing the user to subclass and implement them.

Meanwhile, I read that Yukihiro “Matz” Matsumoto, creator of Ruby, said in a keynote in September 2014 that static typing may be in the future of the language. That was at Ruby Kaigi in Japan, one of the most important Ruby conferences every year. As I write this, I haven’t seen a transcript, but Godfrey Chan posted about it on his blog: “[Ruby Kaigi 2014: Day 2](#)”. From Chan’s report, it seems Matz focused on function annotations. There is even mention of Python function annotations.

I wonder if function annotations would be really good without ABCs to add structure to the type system without losing flexibility. So maybe formal interfaces are also in the future of Ruby.

I believe Python ABCs, with the `register` function and `__subclasshook__`, brought formal interfaces to the language without throwing away the advantages of dynamic typing.

Perhaps the geese are poised to overtake the ducks.

Metaphors and Idioms in Interfaces

A metaphor fosters understanding by making constraints clear. That’s the value of the words “stack” and “queue” in describing those fundamental data structures: they make clear how items can be added or removed. On the other hand, Alan Cooper writes in *About Face, 4E* (Wiley):

Strict adherence to metaphors ties interfaces unnecessarily tightly to the workings of the physical world.

He’s referring to user interfaces, but the admonition applies to APIs as well. But Cooper does grant that when a “truly appropriate” metaphor “falls on our lap,” we can use it (he writes “falls on our lap” because it’s so hard to find fitting metaphors that you should not spend time actively looking for them). I believe the bingo machine imagery I used in this chapter is appropriate and I stand by it.

About Face is by far the best book about UI design I’ve read—and I’ve read a few. Letting go of metaphors as a design paradigm, and replacing it with “idiomatic interfaces” was the most valuable thing I learned from Cooper’s work. As mentioned, Cooper does not deal with APIs, but the more I think about his ideas, the more I see how they apply to Python. The fundamental protocols of the language are what Cooper calls “idioms.” Once we learn what a “sequence” is we can apply that knowledge in different contexts. This is a main theme of *Fluent Python*: highlighting the fundamental idioms of the language, so your code is concise, effective, and readable—for a fluent Pythonista.

FP Ch 15 | Context Managers & else Blocks [= plan Ch 18]

Context Managers and else Blocks

Context managers may end up being almost as important as the subroutine itself. We've only scratched the surface with them. [...] Basic has a `with` statement, there are `with` statements in lots of languages. But they don't do the same thing, they all do something very shallow, they save you from repeated dotted [attribute] lookups, they don't do setup and tear down. Just because it's the same name don't think it's the same thing. The `with` statement is a very big deal.¹

— Raymond Hettinger
Eloquent Python evangelist

In this chapter, we will discuss control flow features that are not so common in other languages, and for this reason tend to be overlooked or underused in Python. They are:

- The `with` statement and context managers
- The `else` clause in `for`, `while`, and `try` statements

The `with` statement sets up a temporary context and reliably tears it down, under the control of a context manager object. This prevents errors and reduces boilerplate code, making APIs at the same time safer and easier to use. Python programmers are finding lots of uses for `with` blocks beyond automatic file closing.

The `else` clause is completely unrelated to `with`. But this is [Part V](#), and I couldn't find another place for covering `else`, and I wouldn't have a one-page chapter about it, so here it is.

Let's review the smaller topic to get to the real substance of this chapter.

1. PyCon US 2013 keynote: “[What Makes Python Awesome](#)”; the part about `with` starts at 23:00 and ends at 26:15.

Do This, Then That: else Blocks Beyond if

This is no secret, but it is an underappreciated language feature: the `else` clause can be used not only in `if` statements but also in `for`, `while`, and `try` statements.

The semantics of `for/else`, `while/else`, and `try/else` are closely related, but very different from `if/else`. Initially the word `else` actually hindered my understanding of these features, but eventually I got used to it.

Here are the rules:

`for`

The `else` block will run only if and when the `for` loop runs to completion (i.e., not if the `for` is aborted with a `break`).

`while`

The `else` block will run only if and when the `while` loop exits because the condition became *falsy* (i.e., not when the `while` is aborted with a `break`).

`try`

The `else` block will only run if no exception is raised in the `try` block. The **official docs** also state: “Exceptions in the `else` clause are not handled by the preceding `except` clauses.”

In all cases, the `else` clause is also skipped if an exception or a `return`, `break`, or `continue` statement causes control to jump out of the main block of the compound statement.



I think `else` is a very poor choice for the keyword in all cases except `if`. It implies an excluding alternative, like “Run this loop, otherwise do that,” but the semantics for `else` in loops is the opposite: “Run this loop, then do that.” This suggests then as a better keyword—which would also make sense in the `try` context: “Try this, then do that.” However, adding a new keyword is a breaking change to the language, and Guido avoids it like the plague.

Using `else` with these statements often makes the code easier to read and saves the trouble of setting up control flags or adding extra `if` statements.

The use of `else` in loops generally follows the pattern of this snippet:

```
for item in my_list:
    if item.flavor == 'banana':
        break
else:
    raise ValueError('No banana flavor found!')
```


In the case of try/except blocks, else may seem redundant at first. After all, the `after_call()` in the following snippet will run only if the `dangerous_call()` does not raise an exception, correct?

```
try:
    dangerous_call()
    after_call()
except OSError:
    log('OSError...')
```

However, doing so puts the `after_call()` inside the try block for no good reason. For clarity and correctness, the body of a try block should only have the statements that may generate the expected exceptions. This is much better:

```
try:
    dangerous_call()
except OSError:
    log('OSError...')
else:
    after_call()
```

Now it's clear that the try block is guarding against possible errors in `dangerous_call()` and not in `after_call()`. It's also more obvious that `after_call()` will only execute if no exceptions are raised in the try block.

In Python, try/except is commonly used for control flow, and not just for error handling. There's even an acronym/slogan for that documented in the [official Python glossary](#):

EAFP

Easier to ask for forgiveness than permission. This common Python coding style assumes the existence of valid keys or attributes and catches exceptions if the assumption proves false. This clean and fast style is characterized by the presence of many try and except statements. The technique contrasts with the *LBYL* style common to many other languages such as C.

The glossary then defines *LBYL*:

LBYL

Look before you leap. This coding style explicitly tests for pre-conditions before making calls or lookups. This style contrasts with the *EAFP* approach and is characterized by the presence of many if statements. In a multi-threaded environment, the *LBYL* approach can risk introducing a race condition between “the looking” and “the leaping”. For example, the code, if key in mapping: return mapping[key] can fail if another thread removes key from mapping after the test, but before the lookup. This issue can be solved with locks or by using the *EAFP* approach.

Given the *EAFP* style, it makes even more sense to know and use well `else` blocks in try/except statements.

Now let's address the main topic of this chapter: the powerful `with` statement.

Context Managers and with Blocks

Context manager objects exist to control a `with` statement, just like iterators exist to control a `for` statement.

The `with` statement was designed to simplify the `try/finally` pattern, which guarantees that some operation is performed after a block of code, even if the block is aborted because of an exception, a `return` or `sys.exit()` call. The code in the `finally` clause usually releases a critical resource or restores some previous state that was temporarily changed.

The context manager protocol consists of the `__enter__` and `__exit__` methods. At the start of the `with`, `__enter__` is invoked on the context manager object. The role of the `finally` clause is played by a call to `__exit__` on the context manager object at the end of the `with` block.

The most common example is making sure a file object is closed. See [Example 15-1](#) for a detailed demonstration of using `with` to close a file.

Example 15-1. Demonstration of a file object as a context manager

```
>>> with open('mirror.py') as fp: # ❶
...     src = fp.read(60) # ❷
...
>>> len(src)
60
>>> fp # ❸
<_io.TextIOWrapper name='mirror.py' mode='r' encoding='UTF-8'>
>>> fp.closed, fp.encoding # ❹
(True, 'UTF-8')
>>> fp.read(60) # ❺
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: I/O operation on closed file.
```

- ❶ `fp` is bound to the opened file because the file's `__enter__` method returns `self`.
- ❷ Read some data from `fp`.
- ❸ The `fp` variable is still available.²
- ❹ You can read the attributes of the `fp` object.
- ❺ But you can't perform I/O with `fp` because at the end of the `with` block, the `TextIOWrapper.__exit__` method is called and closes the file.

2. `with` blocks don't define a new scope, as functions and modules do.

The first callout in **Example 15-1** makes a subtle but crucial point: the context manager object is the result of evaluating the expression after `with`, but the value bound to the target variable (in the `as` clause) is the result of calling `__enter__` on the context manager object.

It just happens that in **Example 15-1**, the `open()` function returns an instance of `TextIOWrapper`, and its `__enter__` method returns `self`. But the `__enter__` method may also return some other object instead of the context manager.

When control flow exits the `with` block in any way, the `__exit__` method is invoked on the context manager object, not on whatever is returned by `__enter__`.

The `as` clause of the `with` statement is optional. In the case of `open`, you'll always need it to get a reference to the file, but some context managers return `None` because they have no useful object to give back to the user.

Example 15-2 shows the operation of a perfectly frivolous context manager designed to highlight the distinction between the context manager and the object returned by its `__enter__` method.

Example 15-2. Test driving the LookingGlass context manager class

```
>>> from mirror import LookingGlass
>>> with LookingGlass() as what: ❶
...     print('Alice, Kitty and Snowdrop') ❷
...     print(what)
...
pordwonS dna yttiK ,ecila ❸
YKCOWREBBAJ
>>> what ❹
'JABBERWOCKY'
>>> print('Back to normal.') ❺
Back to normal.
```

- ❶ The context manager is an instance of `LookingGlass`; Python calls `__enter__` on the context manager and the result is bound to `what`.
- ❷ Print a `str`, then the value of the target variable `what`.
- ❸ The output of each `print` comes out backward.
- ❹ Now the `with` block is over. We can see that the value returned by `__enter__`, held in `what`, is the string `'JABBERWOCKY'`.
- ❺ Program output is no longer backward.

Example 15-3 shows the implementation of `LookingGlass`.

Example 15-3. *mirror.py*: code for the *LookingGlass* context manager class

```
class LookingGlass:
```

```
    def __enter__(self): ❶
        import sys
        self.original_write = sys.stdout.write ❷
        sys.stdout.write = self.reverse_write ❸
        return 'JABBERWOCKY' ❹

    def reverse_write(self, text): ❺
        self.original_write(text[::-1])

    def __exit__(self, exc_type, exc_value, traceback): ❻
        import sys ❼
        sys.stdout.write = self.original_write ❽
        if exc_type is ZeroDivisionError: ❾
            print('Please DO NOT divide by zero!')
            return True ❿
        11
```

- ❶ Python invokes `__enter__` with no arguments besides `self`.
- ❷ Hold the original `sys.stdout.write` method in an instance attribute for later use.
- ❸ Monkey-patch `sys.stdout.write`, replacing it with our own method.
- ❹ Return the 'JABBERWOCKY' string just so we have something to put in the target variable `what`.
- ❺ Our replacement to `sys.stdout.write` reverses the `text` argument and calls the original implementation.
- ❻ Python calls `__exit__` with `None`, `None`, `None` if all went well; if an exception is raised, the three arguments get the exception data, as described next.
- ❼ It's cheap to import modules again because Python caches them.
- ❽ Restore the original method to `sys.stdout.write`.
- ❾ If the exception is not `None` and its type is `ZeroDivisionError`, print a message...
- ❿ ...and return `True` to tell the interpreter that the exception was handled.
- 11 If `__exit__` returns `None` or anything but `True`, any exception raised in the `with` block will be propagated.



When real applications take over standard output, they often want to replace `sys.stdout` with another file-like object for a while, then switch back to the original. The `contextlib.redirect_stdout` context manager does exactly that: just pass it the file-like object that will stand in for `sys.stdout`.

The interpreter calls the `__enter__` method with no arguments—beyond the implicit `self`. The three arguments passed to `__exit__` are:

`exc_type`

The exception class (e.g., `ZeroDivisionError`).

`exc_value`

The exception instance. Sometimes, parameters passed to the exception constructor—such as the error message—can be found in `exc_value.args`.

`traceback`

A traceback object.³

For a detailed look at how a context manager works, see [Example 15-4](#), where `LookingGlass` is used outside of a `with` block, so we can manually call its `__enter__` and `__exit__` methods.

Example 15-4. Exercising `LookingGlass` without a `with` block

```
>>> from mirror import LookingGlass
>>> manager = LookingGlass() ❶
>>> manager
<mirror.LookingGlass object at 0x2a578ac>
>>> monster = manager.__enter__() ❷
>>> monster == 'JABBERWOCKY' ❸
eurT
>>> monster
'YKcOWREBBaJ'
>>> manager
>ca875a2x0 ta tcejbo ssalGgnikool.rorrim<
>>> manager.__exit__(None, None, None) ❹
>>> monster
'JABBERWOCKY'
```

- ❶ Instantiate and inspect the `manager` instance.
- ❷ Call the context manager `__enter__()` method and store result in `monster`.
- ❸ `monster` is the string `'JABBERWOCKY'`. The `True` identifier appears reversed because all output via `stdout` goes through the `write` method we patched in `__enter__`.
- ❹ Call `manager.__exit__` to restore previous `stdout.write`.

3. The three arguments received by `self` are exactly what you get if you call `sys.exc_info()` in the `finally` block of a `try/finally` statement. This makes sense, considering that the `with` statement is meant to replace most uses of `try/finally`, and calling `sys.exc_info()` was often necessary to determine what clean-up action would be required.

Context managers are a fairly novel feature and slowly but surely the Python community is finding new, creative uses for them. Some examples from the standard library are:

- Managing transactions in the `sqlite3` module; see “12.6.7.3. Using the connection as a context manager”.
- Holding locks, conditions, and semaphores in threading code; see “17.1.10. Using locks, conditions, and semaphores in the with statement”.
- Setting up environments for arithmetic operations with `Decimal` objects; see the `decimal.localcontext` documentation.
- Applying temporary patches to objects for testing; see the `unittest.mock.patch` function.

The standard library also includes the `contextlib` utilities, covered next.

The contextlib Utilities

Before rolling your own context manager classes, take a look at “29.6 `contextlib` — Utilities for with-statement contexts” in *The Python Standard Library*. Besides the already mentioned `redirect_stdout`, the `contextlib` module includes classes and other functions that are more widely applicable:

`closing`

A function to build context managers out of objects that provide a `close()` method but don’t implement the `__enter__`/`__exit__` protocol.

`suppress`

A context manager to temporarily ignore specified exceptions.

`@contextmanager`

A decorator that lets you build a context manager from a simple generator function, instead of creating a class and implementing the protocol.

`ContextDecorator`

A base class for defining class-based context managers that can also be used as function decorators, running the entire function within a managed context.

`ExitStack`

A context manager that lets you enter a variable number of context managers. When the `with` block ends, `ExitStack` calls the stacked context managers’ `__exit__` methods in LIFO order (last entered, first exited). Use this class when you don’t know beforehand how many context managers you need to enter in your `with` block; for example, when opening all files from an arbitrary list of files at the same time.

The most widely used of these utilities is surely the `@contextmanager` decorator, so it deserves more attention. That decorator is also intriguing because it shows a use for the `yield` statement unrelated to iteration. This paves the way to the concept of a coroutine, the theme of the next chapter.

Using `@contextmanager`

The `@contextmanager` decorator reduces the boilerplate of creating a context manager: instead of writing a whole class with `__enter__`/`__exit__` methods, you just implement a generator with a single `yield` that should produce whatever you want the `__enter__` method to return.

In a generator decorated with `@contextmanager`, `yield` is used to split the body of the function in two parts: everything before the `yield` will be executed at the beginning of the `while` block when the interpreter calls `__enter__`; the code after `yield` will run when `__exit__` is called at the end of the block.

Here is an example. **Example 15-5** replaces the `LookingGlass` class from **Example 15-3** with a generator function.

Example 15-5. `mirror_gen.py`: a context manager implemented with a generator

```
import contextlib
```

```
@contextlib.contextmanager ❶
def looking_glass():
    import sys
    original_write = sys.stdout.write ❷

    def reverse_write(text): ❸
        original_write(text[::-1])

    sys.stdout.write = reverse_write ❹
    yield 'JABBERWOCKY' ❺
    sys.stdout.write = original_write ❻
```

- ❶ Apply the `contextmanager` decorator.
- ❷ Preserve original `sys.stdout.write` method.
- ❸ Define custom `reverse_write` function; `original_write` will be available in the closure.
- ❹ Replace `sys.stdout.write` with `reverse_write`.
- ❺ Yield the value that will be bound to the target variable in the `as` clause of the `with` statement. This function pauses at this point while the body of the `with` executes.

- ⑥ When control exits the with block in any way, execution continues after the yield; here the original `sys.stdout.write` is restored.

Example 15-6 shows the `looking_glass` function in operation.

Example 15-6. Test driving the `looking_glass` context manager function

```
>>> from mirror_gen import looking_glass
>>> with looking_glass() as what: ①
...     print('Alice, Kitty and Snowdrop')
...     print(what)
...
pordwonS dna yttik ,ecila
YKCOWREBBAJ
>>> what
'JABBERWOCKY'
```

- ① The only difference from Example 15-2 is the name of the context manager: `looking_glass` instead of `LookingGlass`.

Essentially the `contextlib.contextmanager` decorator wraps the function in a class that implements the `__enter__` and `__exit__` methods.⁴

The `__enter__` method of that class:

1. Invokes the generator function and holds on to the generator object—let's call it `gen`.
2. Calls `next(gen)` to make it run to the `yield` keyword.
3. Returns the value yielded by `next(gen)`, so it can be bound to a target variable in the `with/as` form.

When the `with` block terminates, the `__exit__` method:

1. Checks an exception was passed as `exc_type`; if so, `gen.throw(exception)` is invoked, causing the exception to be raised in the `yield` line inside the generator function body.
2. Otherwise, `next(gen)` is called, resuming the execution of the generator function body after the `yield`.

Example 15-5 has a serious flaw: if an exception is raised in the body of the `with` block, the Python interpreter will catch it and raise it again in the `yield` expression inside `looking_glass`. But there is no error handling there, so the `looking_glass` function

4. The actual class is named `_GeneratorContextManager`. If you want to see exactly how it works, read its [source code](#) in `Lib/contextlib.py` in the Python 3.4 distribution.

will abort without ever restoring the original `sys.stdout.write` method, leaving the system in an invalid state.

Example 15-7 adds special handling of the `ZeroDivisionError` exception, making it functionally equivalent to the class-based **Example 15-3**.

*Example 15-7. `mirror_gen_exc.py`: generator-based context manager implementing exception handling—same external behavior as **Example 15-3***

```
import contextlib

@contextlib.contextmanager
def looking_glass():
    import sys
    original_write = sys.stdout.write

    def reverse_write(text):
        original_write(text[::-1])

    sys.stdout.write = reverse_write
    msg = '' ❶
    try:
        yield 'JABBERWOCKY'
    except ZeroDivisionError: ❷
        msg = 'Please DO NOT divide by zero!'
    finally:
        sys.stdout.write = original_write ❸
        if msg:
            print(msg) ❹
```

- ❶ Create a variable for a possible error message; this is the first change in relation to **Example 15-5**.
- ❷ Handle `ZeroDivisionError` by setting an error message.
- ❸ Undo monkey-patching of `sys.stdout.write`.
- ❹ Display error message, if it was set.

Recall that the `__exit__` method tells the interpreter that it has handled the exception by returning `True`; in that case, the interpreter suppresses the exception. On the other hand, if `__exit__` does not explicitly return a value, the interpreter gets the usual `None`, and propagates the exception. With `@contextmanager`, the default behavior is inverted: the `__exit__` method provided by the decorator assumes any exception sent into the generator is handled and should be suppressed.⁵ You must explicitly re-raise an

5. The exception is sent into the generator using the `throw` method, covered in “[Coroutine Termination and Exception Handling](#)” on page 471.

exception in the decorated function if you don't want `@contextmanager` to suppress it.
6



Having a `try/finally` (or a `with` block) around the `yield` is an unavoidable price of using `@contextmanager`, because you never know what the users of your context manager are going to do inside their `with` block.⁷

An interesting real-life example of `@contextmanager` outside of the standard library is Martijn Pieters' [in-place file rewriting context manager](#). [Example 15-8](#) shows how it's used.

Example 15-8. A context manager for rewriting files in place

```
import csv

with inplace(csvfilename, 'r', newline='') as (infh, outf):
    reader = csv.reader(infh)
    writer = csv.writer(outfh)

    for row in reader:
        row += ['new', 'columns']
        writer.writerow(row)
```

The `inplace` function is a context manager that gives you two handles—`infh` and `outfh` in the example—to the same file, allowing your code to read and write to it at the same time. It's easier to use than the standard library's `fileinput.input` function (which also provides a context manager, by the way).

If you want to study Martijn's `inplace` source code (listed in [the post](#)), find the `yield` keyword: everything before it deals with setting up the context, which entails creating a backup file, then opening and yielding references to the readable and writable file handles that will be returned by the `__enter__` call. The `__exit__` processing after the `yield` closes the file handles and restores the file from the backup if something went wrong.

Note that the use of `yield` in a generator used with the `@contextmanager` decorator has nothing to do with iteration. In the examples shown in this section, the generator function is operating more like a coroutine: a procedure that runs up to a point, then sus-

6. This convention was adopted because when context managers were created, generators could not return values, only `yield`. They now can, as explained in “[Returning a Value from a Coroutine](#)” on page 475. As you'll see, returning a value from a generator does involve an exception.

7. This tip is quoted literally from a comment by Leonardo Rochaël, one of the tech reviewers for this book. Nicely said, Leo!

pend to let the client code run until the client wants the coroutine to proceed with its job. [Chapter 16](#) is all about coroutines.

Chapter Summary

This chapter started easily enough with discussion of `else` blocks in `for`, `while`, and `try` statements. Once you get used to the peculiar meaning of the `else` clause in these statements, I believe `else` can clarify your intentions.

We then covered context managers and the meaning of the `with` statement, quickly moving beyond its common use to automatically close opened files. We implemented a custom context manager: the `LookingGlass` class with the `__enter__`/`__exit__` methods, and saw how to handle exceptions in the `__exit__` method. A key point that Raymond Hettinger made in his PyCon US 2013 keynote is that `with` is not just for resource management, but it's a tool for factoring out common setup and teardown code, or any pair of operations that need to be done before and after another procedure ([slide 21, What Makes Python Awesome?](#)).

Finally, we reviewed functions in the `contextlib` standard library module. One of them, the `@contextmanager` decorator, makes it possible to implement a context manager using a simple generator with one `yield`—a leaner solution than coding a class with at least two methods. We reimplemented the `LookingGlass` as a `looking_glass` generator function, and discussed how to do exception handling when using `@contextmanager`.

The `@contextmanager` decorator is an elegant and practical tool that brings together three distinctive Python features: a function decorator, a generator, and the `with` statement.

Further Reading

[Chapter 8, “Compound Statements,”](#) in *The Python Language Reference* says pretty much everything there is to say about `else` clauses in `if`, `for`, `while`, and `try` statements. Regarding Pythonic usage of `try/except`, with or without `else`, Raymond Hettinger has a brilliant answer to the question “[Is it a good practice to use try-except-else in Python?](#)” in StackOverflow. Alex Martelli’s *Python in a Nutshell, 2E* (O’Reilly), has a chapter about exceptions with an excellent discussion of the EAFP style, crediting computing pioneer Grace Hopper for coining the phrase “It’s easier to ask forgiveness than permission.”

The *Python Standard Library*, Chapter 4, “Built-in Types,” has a section devoted to [Context Manager Types](#). The `__enter__`/`__exit__` special methods are also documented in *The Python Language Reference* in “[3.3.8. With Statement Context Managers](#)”. Context managers were introduced in [PEP 343 — The “with” Statement](#). This PEP

is not easy reading because it spends a lot of time covering corner cases and arguing against alternative proposals. That's the nature of PEPs.

Raymond Hettinger highlighted the `with` statement as a “winning language feature” in his [PyCon US 2013 keynote](#). He also showed some interesting applications of context managers in his talk “[Transforming Code into Beautiful, Idiomatic Python](#)” at the same conference.

Jeff Presling's blog post “[The Python with Statement by Example](#)” is interesting for the examples using context managers with the `pycairo` graphics library.

Beazley and Jones devised context managers for very different purposes in their *Python Cookbook*, 3E (O'Reilly). “Recipe 8.3. Making Objects Support the Context-Management Protocol” implements a `LazyConnection` class whose instances are context managers that open and close network connections automatically in `with` blocks. “Recipe 9.22. Defining Context Managers the Easy Way” introduces a context manager for timing code, and another for making transactional changes to a `list` object: within the `with` block, a working copy of the `list` instance is made, and all changes are applied to that working copy. Only when the `with` block completes without an exception, the working copy replaces the original list. Simple and ingenious.

Soapbox

Factoring Out the Bread

In his PyCon US 2013 keynote, “[What Makes Python Awesome](#),” Raymond Hettinger says when he first saw the `with` statement proposal he thought it was “a little bit arcane.” Initially, I had a similar reaction. PEPs are often hard to read, and PEP 343 is typical in that regard.

Then—Hettinger told us—he had an insight: subroutines are the most important invention in the history of computer languages. If you have sequences of operations like `A;B;C` and `P;B;Q`, you can factor out `B` in a subroutine. It's like factoring out the filling in a sandwich: using tuna with different breads. But what if you want to factor out the bread, to make sandwiches with wheat bread, using a different filling each time? That's what the `with` statement offers. It's the complement of the subroutine. Hettinger went on to say:

The `with` statement is a very big deal. I encourage you to go out and take this tip of the iceberg and drill deeper. You can probably do profound things with the `with` statement. The best uses of it have not been discovered yet. I expect that if you make good use of it, it will be copied into other languages and all future languages will have it. You can be part of discovering something almost as profound as the invention of the subroutine itself.

Hettinger admits he is overselling the `with` statement. Nevertheless, it is a very useful feature. When he used the sandwich analogy to explain how `with` is the complement to the subroutine, many possibilities opened up in my mind.

If you need to convince anyone that Python is awesome, you should watch Hettinger's keynote. The bit about context managers is from 23:00 to 26:15. But the entire keynote is excellent.

PNP Ch 38 | Classes [companion]

Chapter 38: Classes

Python offers itself not only as a popular scripting language, but also supports the object-oriented programming paradigm. Classes describe data and provide methods to manipulate that data, all encompassed under a single object. Furthermore, classes allow for abstraction by separating concrete implementation details from abstract representations of data.

Code utilizing classes is generally easier to read, understand, and maintain.

Section 38.1: Introduction to classes

A class, functions as a template that defines the basic characteristics of a particular object. Here's an example:

```
class Person(object):
    """A simple class."""           # docstring
    species = "Homo Sapiens"        # class attribute

    def __init__(self, name):       # special method
        """This is the initializer. It's a special
        method (see below).
        """
        self.name = name           # instance attribute

    def __str__(self):              # special method
        """This method is run when Python tries
        to cast the object to a string. Return
        this string when using print(), etc.
        """
        return self.name

    def rename(self, renamed):      # regular method
        """Reassign and print the name attribute."""
        self.name = renamed
        print("Now my name is {}".format(self.name))
```

There are a few things to note when looking at the above example.

1. The class is made up of *attributes* (data) and *methods* (functions).
2. Attributes and methods are simply defined as normal variables and functions.
3. As noted in the corresponding docstring, the `__init__()` method is called the *initializer*. It's equivalent to the constructor in other object oriented languages, and is the method that is first run when you create a new object, or new instance of the class.
4. Attributes that apply to the whole class are defined first, and are called *class attributes*.
5. Attributes that apply to a specific instance of a class (an object) are called *instance attributes*. They are generally defined inside `__init__()`; this is not necessary, but it is recommended (since attributes defined outside of `__init__()` run the risk of being accessed before they are defined).
6. Every method, included in the class definition passes the object in question as its first parameter. The word `self` is used for this parameter (usage of `self` is actually by convention, as the word `self` has no inherent meaning in Python, but this is one of Python's most respected conventions, and you should always follow it).
7. Those used to object-oriented programming in other languages may be surprised by a few things. One is that Python has no real concept of private elements, so everything, by default, imitates the behavior of the C++/Java `public` keyword. For more information, see the "Private Class Members" example on this page.
8. Some of the class's methods have the following form: `__functionname__(self, other_stuff)`. All such methods are called "magic methods" and are an important part of classes in Python. For instance, operator overloading in Python is implemented with magic methods. For more information, see the relevant

documentation.

Now let's make a few instances of our Person class!

```
>>> # Instances
>>> kelly = Person("Kelly")
>>> joseph = Person("Joseph")
>>> john_doe = Person("John Doe")
```

We currently have three Person objects, kelly, joseph, and john_doe.

We can access the attributes of the class from each instance using the dot operator `.`. Note again the difference between class and instance attributes:

```
>>> # Attributes
>>> kelly.species
'Homo Sapiens'
>>> john_doe.species
'Homo Sapiens'
>>> joseph.species
'Homo Sapiens'
>>> kelly.name
'Kelly'
>>> joseph.name
'Joseph'
```

We can execute the methods of the class using the same dot operator `.`:

```
>>> # Methods
>>> john_doe.__str__()
'John Doe'
>>> print(john_doe)
'John Doe'
>>> john_doe.rename("John")
'Now my name is John'
```

Section 38.2: Bound, unbound, and static methods

The idea of bound and unbound methods was [removed in Python 3](#). In Python 3 when you declare a method within a class, you are using a `def` keyword, thus creating a function object. This is a regular function, and the surrounding class works as its namespace. In the following example we declare method `f` within class `A`, and it becomes a function `A.f`:

Python 3.x Version ≥ 3.0

```
class A(object):
    def f(self, x):
        return 2 * x

A.f
# <function A.f at ...> (in Python 3.x)
```

In Python 2 the behavior was different: function objects within the class were implicitly replaced with objects of type `instancemethod`, which were called *unbound methods* because they were not bound to any particular class instance. It was possible to access the underlying function using `.__func__` property.

Python 2.x Version ≥ 2.3

```
A.f
```

```
# <unbound method A.f> (in Python 2.x)
A.f.__class__
# <type 'instancemethod'>
A.f.__func__
# <function f at ...>
```

The latter behaviors are confirmed by inspection - methods are recognized as functions in Python 3, while the distinction is upheld in Python 2.

Python 3.x Version ≥ 3.0

```
import inspect

inspect.isfunction(A.f)
# True
inspect.ismethod(A.f)
# False
```

Python 2.x Version ≥ 2.3

```
import inspect

inspect.isfunction(A.f)
# False
inspect.ismethod(A.f)
# True
```

In both versions of Python function/method `A.f` can be called directly, provided that you pass an instance of class `A` as the first argument.

```
A.f(1, 7)
# Python 2: TypeError: unbound method f() must be called with
#           A instance as first argument (got int instance instead)
# Python 3: 14
a = A()
A.f(a, 20)
# Python 2 & 3: 40
```

Now suppose `a` is an instance of class `A`, what is `a.f` then? Well, intuitively this should be the same method `f` of class `A`, only it should somehow "know" that it was applied to the object `a` – in Python this is called method *bound* to `a`.

The nitty-gritty details are as follows: writing `a.f` invokes the magic `__getattr__` method of `a`, which first checks whether `a` has an attribute named `f` (it doesn't), then checks the class `A` whether it contains a method with such a name (it does), and creates a new object `m` of type `method` which has the reference to the original `A.f` in `m.__func__`, and a reference to the object `a` in `m.__self__`. When this object is called as a function, it simply does the following: `m(...) => m.__func__(m.__self__, ...)`. Thus this object is called a **bound method** because when invoked it knows to supply the object it was bound to as the first argument. (These things work same way in Python 2 and 3).

```
a = A()
a.f
# <bound method A.f of <__main__.A object at ...>>
a.f(2)
# 4

# Note: the bound method object a.f is recreated *every time* you call it:
a.f is a.f # False
# As a performance optimization you can store the bound method in the object's
# __dict__, in which case the method object will remain fixed:
a.f = a.f
```

```
a.f is a.f # True
```

Finally, Python has **class methods** and **static methods** – special kinds of methods. Class methods work the same way as regular methods, except that when invoked on an object they bind to the *class* of the object instead of to the object. Thus `m.__self__ = type(a)`. When you call such bound method, it passes the class of `a` as the first argument. Static methods are even simpler: they don't bind anything at all, and simply return the underlying function without any transformations.

```
class D(object):
    multiplier = 2

    @classmethod
    def f(cls, x):
        return cls.multiplier * x

    @staticmethod
    def g(name):
        print("Hello, %s" % name)

D.f
# <bound method type.f of <class '__main__.D'>>
D.f(12)
# 24
D.g
# <function D.g at ...>
D.g("world")
# Hello, world
```

Note that class methods are bound to the class even when accessed on the instance:

```
d = D()
d.multiplier = 1337
(D.multiplier, d.multiplier)
# (2, 1337)
d.f
# <bound method D.f of <class '__main__.D'>>
d.f(10)
# 20
```

It is worth noting that at the lowest level, functions, methods, staticmethods, etc. are actually descriptors that invoke `__get__`, `__set__` and optionally `__del__` special methods. For more details on classmethods and staticmethods:

- [What is the difference between @staticmethod and @classmethod in Python?](#)
- [Meaning of @classmethod and @staticmethod for beginner?](#)

Section 38.3: Basic inheritance

Inheritance in Python is based on similar ideas used in other object oriented languages like Java, C++ etc. A new class can be derived from an existing class as follows.

```
class BaseClass(object):
    pass

class DerivedClass(BaseClass):
    pass
```

The BaseClass is the already existing (*parent*) class, and the DerivedClass is the new (*child*) class that inherits (or *subclasses*) attributes from BaseClass. **Note:** As of Python 2.2, all [classes implicitly inherit from the object class](#), which is the base class for all built-in types.

We define a parent Rectangle class in the example below, which implicitly inherits from `object`:

```
class Rectangle():
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h

    def perimeter(self):
        return 2 * (self.w + self.h)
```

The Rectangle class can be used as a base class for defining a Square class, as a square is a special case of rectangle.

```
class Square(Rectangle):
    def __init__(self, s):
        # call parent constructor, w and h are both s
        super(Square, self).__init__(s, s)
        self.s = s
```

The Square class will automatically inherit all attributes of the Rectangle class as well as the object class. `super()` is used to call the `__init__()` method of Rectangle class, essentially calling any overridden method of the base class.

Note: in Python 3, `super()` does not require arguments.

Derived class objects can access and modify the attributes of its base classes:

```
r.area()
# Output: 12
r.perimeter()
# Output: 14

s.area()
# Output: 4
s.perimeter()
# Output: 8
```

Built-in functions that work with inheritance

`issubclass(DerivedClass, BaseClass)`: returns `True` if DerivedClass is a subclass of the BaseClass

`isinstance(s, Class)`: returns `True` if s is an instance of Class or any of the derived classes of Class

```
# subclass check
issubclass(Square, Rectangle)
# Output: True

# instantiate
r = Rectangle(3, 4)
s = Square(2)

isinstance(r, Rectangle)
# Output: True
isinstance(r, Square)
```

```
# Output: False
# A rectangle is not a square

isinstance(s, Rectangle)
# Output: True
# A square is a rectangle
isinstance(s, Square)
# Output: True
```

Section 38.4: Monkey Patching

In this case, "monkey patching" means adding a new variable or method to a class after it's been defined. For instance, say we defined class A as

```
class A(object):
    def __init__(self, num):
        self.num = num

    def __add__(self, other):
        return A(self.num + other.num)
```

But now we want to add another function later in the code. Suppose this function is as follows.

```
def get_num(self):
    return self.num
```

But how do we add this as a method in A? That's simple we just essentially place that function into A with an assignment statement.

```
A.get_num = get_num
```

Why does this work? Because functions are objects just like any other object, and methods are functions that belong to the class.

The function `get_num` shall be available to all existing (already created) as well to the new instances of A

These additions are available on all instances of that class (or its subclasses) automatically. For example:

```
foo = A(42)

A.get_num = get_num

bar = A(6);

foo.get_num() # 42

bar.get_num() # 6
```

Note that, unlike some other languages, this technique does not work for certain built-in types, and it is not considered good style.

Section 38.5: New-style vs. old-style classes

Python 2.x Version $\geq 2.2.0$

New-style classes were introduced in Python 2.2 to unify *classes* and *types*. They inherit from the top-level `object`

type. A new-style class is a user-defined type, and is very similar to built-in types.

```
# new-style class
class New(object):
    pass

# new-style instance
new = New()

new.__class__
# <class '__main__.New'>
type(new)
# <class '__main__.New'>
issubclass(New, object)
# True
```

Old-style classes do **not** inherit from `object`. Old-style instances are always implemented with a built-in instance type.

```
# old-style class
class Old:
    pass

# old-style instance
old = Old()

old.__class__
# <class '__main__.Old at ...'>
type(old)
# <type 'instance'>
issubclass(Old, object)
# False
```

Python 3.x Version $\geq$ 3.0.0

In Python 3, old-style classes were removed.

New-style classes in Python 3 implicitly inherit from `object`, so there is no need to specify `MyClass(object)` anymore.

```
class MyClass:
    pass

my_inst = MyClass()

type(my_inst)
# <class '__main__.MyClass'>
my_inst.__class__
# <class '__main__.MyClass'>
issubclass(MyClass, object)
# True
```

Section 38.6: Class methods: alternate initializers

Class methods present alternate ways to build instances of classes. To illustrate, let's look at an example.

Let's suppose we have a relatively simple Person class:

```
class Person(object):
```

```
def __init__(self, first_name, last_name, age):
    self.first_name = first_name
    self.last_name = last_name
    self.age = age
    self.full_name = first_name + " " + last_name

def greet(self):
    print("Hello, my name is " + self.full_name + ".")
```

It might be handy to have a way to build instances of this class specifying a full name instead of first and last name separately. One way to do this would be to have `last_name` be an optional parameter, and assuming that if it isn't given, we passed the full name in:

```
class Person(object):

    def __init__(self, first_name, age, last_name=None):
        if last_name is None:
            self.first_name, self.last_name = first_name.split(" ", 2)
        else:
            self.first_name = first_name
            self.last_name = last_name

        self.full_name = self.first_name + " " + self.last_name
        self.age = age

    def greet(self):
        print("Hello, my name is " + self.full_name + ".")
```

However, there are two main problems with this bit of code:

1. The parameters `first_name` and `last_name` are now misleading, since you can enter a full name for `first_name`. Also, if there are more cases and/or more parameters that have this kind of flexibility, the `if/elif/else` branching can get annoying fast.
2. Not quite as important, but still worth pointing out: what if `last_name` is `None`, but `first_name` doesn't split into two or more things via spaces? We have yet another layer of input validation and/or exception handling...

Enter class methods. Rather than having a single initializer, we will create a separate initializer, called `from_full_name`, and decorate it with the (built-in) `classmethod` decorator.

```
class Person(object):

    def __init__(self, first_name, last_name, age):
        self.first_name = first_name
        self.last_name = last_name
        self.age = age
        self.full_name = first_name + " " + last_name

    @classmethod
    def from_full_name(cls, name, age):
        if " " not in name:
            raise ValueError
        first_name, last_name = name.split(" ", 2)
        return cls(first_name, last_name, age)

    def greet(self):
        print("Hello, my name is " + self.full_name + ".")
```

Notice `cls` instead of `self` as the first argument to `from_full_name`. Class methods are applied to the overall class, *not* an instance of a given class (which is what `self` usually denotes). So, if `cls` is our `Person` class, then the returned value from the `from_full_name` class method is `Person(first_name, last_name, age)`, which uses `Person's __init__` to create an instance of the `Person` class. In particular, if we were to make a subclass `Employee` of `Person`, then `from_full_name` would work in the `Employee` class as well.

To show that this works as expected, let's create instances of `Person` in more than one way without the branching in `__init__`:

```
In [2]: bob = Person("Bob", "Bobberson", 42)

In [3]: alice = Person.from_full_name("Alice Henderson", 31)

In [4]: bob.greet()
Hello, my name is Bob Bobberson.

In [5]: alice.greet()
Hello, my name is Alice Henderson.
```

Other references:

- [Python @classmethod and @staticmethod for beginner?](#)
- <https://docs.python.org/2/library/functions.html#classmethod>
- <https://docs.python.org/3.5/library/functions.html#classmethod>

Section 38.7: Multiple Inheritance

Python uses the [C3 linearization](#) algorithm to determine the order in which to resolve class attributes, including methods. This is known as the Method Resolution Order (MRO).

Here's a simple example:

```
class Foo(object):
    foo = 'attr foo of Foo'

class Bar(object):
    foo = 'attr foo of Bar' # we won't see this.
    bar = 'attr bar of Bar'

class FooBar(Foo, Bar):
    foobar = 'attr foobar of FooBar'
```

Now if we instantiate `FooBar`, if we look up the `foo` attribute, we see that `Foo's` attribute is found first

```
fb = FooBar()
```

and

```
>>> fb.foo
'attr foo of Foo'
```

Here's the MRO of `FooBar`:


```
>>> FooBar.mro()
[<class '__main__.FooBar'>, <class '__main__.Foo'>, <class '__main__.Bar'>, <type 'object'>]
```

It can be simply stated that Python's MRO algorithm is

1. Depth first (e.g. FooBar then Foo) unless
2. a shared parent (`object`) is blocked by a child (Bar) and
3. no circular relationships allowed.

That is, for example, Bar cannot inherit from FooBar while FooBar inherits from Bar.

For a comprehensive example in Python, see the [wikipedia entry](#).

Another powerful feature in inheritance is `super`. `super` can fetch parent classes features.

```
class Foo(object):
    def foo_method(self):
        print "foo Method"

class Bar(object):
    def bar_method(self):
        print "bar Method"

class FooBar(Foo, Bar):
    def foo_method(self):
        super(FooBar, self).foo_method()
```

Multiple inheritance with init method of class, when every class has own init method then we try for multiple inheritance then only init method get called of class which is inherit first.

for below example only Foo class **init** method getting called **Bar** class init not getting called

```
class Foo(object):
    def __init__(self):
        print "foo init"

class Bar(object):
    def __init__(self):
        print "bar init"

class FooBar(Foo, Bar):
    def __init__(self):
        print "foobar init"
        super(FooBar, self).__init__()

a = FooBar()
```

Output:

```
foobar init
foo init
```

But it doesn't mean that **Bar** class is not inherit. Instance of final **FooBar** class is also instance of **Bar** class and **Foo** class.

```
print isinstance(a, FooBar)
print isinstance(a, Foo)
```

```
print isinstance(a, Bar)
```

Output:

```
True
True
True
```

Section 38.8: Properties

Python classes support **properties**, which look like regular object variables, but with the possibility of attaching custom behavior and documentation.

```
class MyClass(object):

    def __init__(self):
        self._my_string = ""

    @property
    def string(self):
        """A profoundly important string."""
        return self._my_string

    @string.setter
    def string(self, new_value):
        assert isinstance(new_value, str), \
            "Give me a string, not a %r!" % type(new_value)
        self._my_string = new_value

    @string.deleter
    def x(self):
        self._my_string = None
```

The object's of class `MyClass` will *appear* to have a property `.string`, however it's behavior is now tightly controlled:

```
mc = MyClass()
mc.string = "String!"
print(mc.string)
del mc.string
```

As well as the useful syntax as above, the property syntax allows for validation, or other augmentations to be added to those attributes. This could be especially useful with public APIs - where a level of help should be given to the user.

Another common use of properties is to enable the class to present 'virtual attributes' - attributes which aren't actually stored but are computed only when requested.

```
class Character(object):
    def __init__(name, max_hp):
        self._name = name
        self._hp = max_hp
        self._max_hp = max_hp

    # Make hp read only by not providing a set method
    @property
    def hp(self):
        return self._hp
```

```

# Make name read only by not providing a set method
@property
def name(self):
    return self.name

def take_damage(self, damage):
    self.hp -= damage
    self.hp = 0 if self.hp < 0 else self.hp

@property
def is_alive(self):
    return self.hp != 0

@property
def is_wounded(self):
    return self.hp < self.max_hp if self.hp > 0 else False

@property
def is_dead(self):
    return not self.is_alive

bilbo = Character('Bilbo Baggins', 100)
bilbo.hp
# out : 100
bilbo.hp = 200
# out : AttributeError: can't set attribute
# hp attribute is read only.

bilbo.is_alive
# out : True
bilbo.is_wounded
# out : False
bilbo.is_dead
# out : False

bilbo.take_damage( 50 )

bilbo.hp
# out : 50

bilbo.is_alive
# out : True
bilbo.is_wounded
# out : True
bilbo.is_dead
# out : False

bilbo.take_damage( 50 )
bilbo.hp
# out : 0

bilbo.is_alive
# out : False
bilbo.is_wounded
# out : False
bilbo.is_dead
# out : True

```

Section 38.9: Default values for instance variables

If the variable contains a value of an immutable type (e.g. a string) then it is okay to assign a default value like this

```

class Rectangle(object):
    def __init__(self, width, height, color='blue'):
        self.width = width
        self.height = height
        self.color = color

    def area(self):
        return self.width * self.height

# Create some instances of the class
default_rectangle = Rectangle(2, 3)
print(default_rectangle.color) # blue

red_rectangle = Rectangle(2, 3, 'red')
print(red_rectangle.color) # red

```

One needs to be careful when initializing mutable objects such as lists in the constructor. Consider the following example:

```

class Rectangle2D(object):
    def __init__(self, width, height, pos=[0,0], color='blue'):
        self.width = width
        self.height = height
        self.pos = pos
        self.color = color

r1 = Rectangle2D(5,3)
r2 = Rectangle2D(7,8)
r1.pos[0] = 4
r1.pos # [4, 0]
r2.pos # [4, 0] r2's pos has changed as well

```

This behavior is caused by the fact that in Python default parameters are bound at function execution and not at function declaration. To get a default instance variable that's not shared among instances, one should use a construct like this:

```

class Rectangle2D(object):
    def __init__(self, width, height, pos=None, color='blue'):
        self.width = width
        self.height = height
        self.pos = pos or [0, 0] # default value is [0, 0]
        self.color = color

r1 = Rectangle2D(5,3)
r2 = Rectangle2D(7,8)
r1.pos[0] = 4
r1.pos # [4, 0]
r2.pos # [0, 0] r2's pos hasn't changed

```

See also [Mutable Default Arguments](#) and [“Least Astonishment” and the Mutable Default Argument](#).

Section 38.10: Class and instance variables

Instance variables are unique for each instance, while class variables are shared by all instances.

```

class C:
    x = 2 # class variable

```

```

def __init__(self, y):
    self.y = y # instance variable

C.x
# 2
C.y
# AttributeError: type object 'C' has no attribute 'y'

c1 = C(3)
c1.x
# 2
c1.y
# 3

c2 = C(4)
c2.x
# 2
c2.y
# 4

```

Class variables can be accessed on instances of this class, but assigning to the class attribute will create an instance variable which shadows the class variable

```

c2.x = 4
c2.x
# 4
C.x
# 2

```

Note that *mutating* class variables from instances can lead to some unexpected consequences.

```

class D:
    x = []
    def __init__(self, item):
        self.x.append(item) # note that this is not an assignment!

d1 = D(1)
d2 = D(2)

d1.x
# [1, 2]
d2.x
# [1, 2]
D.x
# [1, 2]

```

Section 38.11: Class composition

Class composition allows explicit relations between objects. In this example, people live in cities that belong to countries. Composition allows people to access the number of all people living in their country:

```

class Country(object):
    def __init__(self):
        self.cities=[]

    def addCity(self,city):
        self.cities.append(city)

```

```

class City(object):
    def __init__(self, numPeople):
        self.people = []
        self.numPeople = numPeople

    def addPerson(self, person):
        self.people.append(person)

    def join_country(self, country):
        self.country = country
        country.addCity(self)

        for i in range(self.numPeople):
            person(i).join_city(self)

class Person(object):
    def __init__(self, ID):
        self.ID=ID

    def join_city(self, city):
        self.city = city
        city.addPerson(self)

    def people_in_my_country(self):
        x= sum([len(c.people) for c in self.city.country.cities])
        return x

US=Country()
NYC=City(10).join_country(US)
SF=City(5).join_country(US)

print(US.cities[0].people[0].people_in_my_country())

```

15

Section 38.12: Listing All Class Members

The `dir()` function can be used to get a list of the members of a class:

```
dir(Class)
```

For example:

```

>>> dir(list)
['__add__', '__class__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',
 '__eq__', '__format__', '__ge__', '__getattribute__', '__getitem__', '__gt__', '__hash__',
 '__iadd__', '__imul__', '__init__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__',
 '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__',
 '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count',
 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']

```

It is common to look only for "non-magic" members. This can be done using a simple comprehension that lists members with names not starting with `__`:

```

>>> [m for m in dir(list) if not m.startswith('__')]
['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse',
 'sort']

```

Caveats:

Classes can define a `__dir__()` method. If that method exists calling `dir()` will call `__dir__()`, otherwise Python will try to create a list of members of the class. This means that the `dir` function can have unexpected results. Two quotes of importance from [the official python documentation](#):

If the object does not provide `dir()`, the function tries its best to gather information from the object's `dict` attribute, if defined, and from its type object. The resulting list is not necessarily complete, and may be inaccurate when the object has a custom `getattr()`.

Note: Because `dir()` is supplied primarily as a convenience for use at an interactive prompt, it tries to supply an interesting set of names more than it tries to supply a rigorously or consistently defined set of names, and its detailed behavior may change across releases. For example, metaclass attributes are not in the result list when the argument is a class.

Section 38.13: Singleton class

A singleton is a pattern that restricts the instantiation of a class to one instance/object. For more info on python singleton design patterns, see [here](#).

```
class Singleton:
    def __new__(cls):
        try:
            it = cls.__it__
        except AttributeError:
            it = cls.__it__ = object.__new__(cls)
        return it

    def __repr__(self):
        return '<{}>'.format(self.__class__.__name__.upper())

    def __eq__(self, other):
        return other is self
```

Another method is to decorate your class. Following the example from this [answer](#) create a Singleton class:

```
class Singleton:
    """
    A non-thread-safe helper class to ease implementing singletons.
    This should be used as a decorator -- not a metaclass -- to the
    class that should be a singleton.

    The decorated class can define one `__init__` function that
    takes only the `self` argument. Other than that, there are
    no restrictions that apply to the decorated class.

    To get the singleton instance, use the `Instance` method. Trying
    to use `__call__` will result in a `TypeError` being raised.

    Limitations: The decorated class cannot be inherited from.

    """

    def __init__(self, decorated):
```

```

self._decorated = decorated

def Instance(self):
    """
    Returns the singleton instance. Upon its first call, it creates a
    new instance of the decorated class and calls its `__init__` method.
    On all subsequent calls, the already created instance is returned.

    """
    try:
        return self._instance
    except AttributeError:
        self._instance = self._decorated()
        return self._instance

def __call__(self):
    raise TypeError('Singletons must be accessed through `Instance()`.')

def __instancecheck__(self, inst):
    return isinstance(inst, self._decorated)

```

To use you can use the Instance method

```

@Singleton
class Single:
    def __init__(self):
        self.name=None
        self.val=0
    def getName(self):
        print(self.name)

x=Single.Instance()
y=Single.Instance()
x.name='I\'m single'
x.getName() # outputs I'm single
y.getName() # outputs I'm single

```

Section 38.14: Descriptors and Dotted Lookups

Descriptors are objects that are (usually) attributes of classes and that have any of `__get__`, `__set__`, or `__delete__` special methods.

Data Descriptors have any of `__set__`, or `__delete__`

These can control the dotted lookup on an instance, and are used to implement functions, `staticmethod`, `classmethod`, and `property`. A dotted lookup (e.g. instance foo of class Foo looking up attribute bar - i.e. foo.bar) uses the following algorithm:

1. bar is looked up in the class, Foo. If it is there and it is a **Data Descriptor**, then the data descriptor is used. That's how `property` is able to control access to data in an instance, and instances cannot override this. If a **Data Descriptor** is not there, then
2. bar is looked up in the instance `__dict__`. This is why we can override or block methods being called from an instance with a dotted lookup. If bar exists in the instance, it is used. If not, we then
3. look in the class Foo for bar. If it is a **Descriptor**, then the descriptor protocol is used. This is how functions (in this context, unbound methods), `classmethod`, and `staticmethod` are implemented. Else it simply returns the object there, or there is an `AttributeError`

PNP Ch 77 | Context Managers [companion]

Chapter 77: Context Managers (“with” Statement)

While Python's context managers are widely used, few understand the purpose behind their use. These statements, commonly used with reading and writing files, assist the application in conserving system memory and improve resource management by ensuring specific resources are only in use for certain processes. This topic explains and demonstrates the use of Python's context managers.

Section 77.1: Introduction to context managers and the with statement

A context manager is an object that is notified when a context (a block of code) *starts* and *ends*. You commonly use one with the **with** statement. It takes care of the notifying.

For example, file objects are context managers. When a context ends, the file object is closed automatically:

```
open_file = open(filename)
with open_file:
    file_contents = open_file.read()

# the open_file object has automatically been closed.
```

The above example is usually simplified by using the **as** keyword:

```
with open(filename) as open_file:
    file_contents = open_file.read()

# the open_file object has automatically been closed.
```

Anything that ends execution of the block causes the context manager's `exit` method to be called. This includes exceptions, and can be useful when an error causes you to prematurely exit from an open file or connection. Exiting a script without properly closing files/connections is a bad idea, that may cause data loss or other problems. By using a context manager you can ensure that precautions are always taken to prevent damage or loss in this way. This feature was added in Python 2.5.

Section 77.2: Writing your own context manager

A context manager is any object that implements two magic methods `__enter__()` and `__exit__()` (although it can implement other methods as well):

```
class AContextManager():

    def __enter__(self):
        print("Entered")
        # optionally return an object
        return "A-instance"

    def __exit__(self, exc_type, exc_value, traceback):
        print("Exited" + (" (with an exception)" if exc_type else ""))
        # return True if you want to suppress the exception
```

If the context exits with an exception, the information about that exception will be passed as a triple `exc_type`, `exc_value`, `traceback` (these are the same variables as returned by the `sys.exc_info()` function). If the context

exits normally, all three of these arguments will be `None`.

If an exception occurs and is passed to the `__exit__` method, the method can return `True` in order to suppress the exception, or the exception will be re-raised at the end of the `__exit__` function.

```
with AContextManager() as a:
    print("a is %r" % a)
# Entered
# a is 'A-instance'
# Exited

with AContextManager() as a:
    print("a is %d" % a)
# Entered
# Exited (with an exception)
# Traceback (most recent call last):
#   File "<stdin>", line 2, in <module>
# TypeError: %d format: a number is required, not str
```

Note that in the second example even though an exception occurs in the middle of the body of the with-statement, the `__exit__` handler still gets executed, before the exception propagates to the outer scope.

If you only need an `__exit__` method, you can return the instance of the context manager:

```
class MyContextManager:
    def __enter__(self):
        return self

    def __exit__(self):
        print('something')
```

Section 77.3: Writing your own contextmanager using generator syntax

It is also possible to write a context manager using generator syntax thanks to the [contextlib.contextmanager](#) decorator:

```
import contextlib

@contextlib.contextmanager
def context_manager(num):
    print('Enter')
    yield num + 1
    print('Exit')

with context_manager(2) as cm:
    # the following instructions are run when the 'yield' point of the context
    # manager is reached.
    # 'cm' will have the value that was yielded
    print('Right in the middle with cm = {}'.format(cm))
```

produces:

```
Enter
Right in the middle with cm = 3
Exit
```

The decorator simplifies the task of writing a context manager by converting a generator into one. Everything before the yield expression becomes the `__enter__` method, the value yielded becomes the value returned by the generator (which can be bound to a variable in the with statement), and everything after the yield expression becomes the `__exit__` method.

If an exception needs to be handled by the context manager, a `try...except...finally`-block can be written in the generator and any exception raised in the `with`-block will be handled by this exception block.

```
@contextlib.contextmanager
def error_handling_context_manager(num):
    print("Enter")
    try:
        yield num + 1
    except ZeroDivisionError:
        print("Caught error")
    finally:
        print("Cleaning up")
    print("Exit")

with error_handling_context_manager(-1) as cm:
    print("Dividing by cm = {}".format(cm))
    print(2 / cm)
```

This produces:

```
Enter
Dividing by cm = 0
Caught error
Cleaning up
Exit
```

Section 77.4: Multiple context managers

You can open several content managers at the same time:

```
with open(input_path) as input_file, open(output_path, 'w') as output_file:

    # do something with both files.

    # e.g. copy the contents of input_file into output_file
    for line in input_file:
        output_file.write(line + '\n')
```

It has the same effect as nesting context managers:

```
with open(input_path) as input_file:
    with open(output_path, 'w') as output_file:
        for line in input_file:
            output_file.write(line + '\n')
```

Section 77.5: Assigning to a target

Many context managers return an object when entered. You can assign that object to a new name in the `with` statement.

For example, using a database connection in a `with` statement could give you a cursor object:

```
with database_connection as cursor:
    cursor.execute(sql_query)
```

File objects return themselves, this makes it possible to both open the file object and use it as a context manager in one expression:

```
with open(filename) as open_file:
    file_contents = open_file.read()
```

Section 77.6: Manage Resources

```
class File():
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode

    def __enter__(self):
        self.open_file = open(self.filename, self.mode)
        return self.open_file

    def __exit__(self, *args):
        self.open_file.close()
```

`__init__()` method sets up the object, in this case setting up the file name and mode to open file. `__enter__()` opens and returns the file and `__exit__()` just closes it.

Using these magic methods (`__enter__`, `__exit__`) allows you to implement objects which can be used easily **with** the with statement.

Use File class:

```
for _ in range(10000):
    with File('foo.txt', 'w') as f:
        f.write('foo')
```

PNP Ch 87 | Type Hints [companion · FP Ch 15 substitute]

Chapter 87: Type Hints

Section 87.1: Adding types to a function

Let's take an example of a function which receives two arguments and returns a value indicating their sum:

```
def two_sum(a, b):  
    return a + b
```

By looking at this code, one can not safely and without doubt indicate the type of the arguments for function `two_sum`. It works both when supplied with `int` values:

```
print(two_sum(2, 1)) # result: 3
```

and with strings:

```
print(two_sum("a", "b")) # result: "ab"
```

and with other values, such as `lists`, `tuples` et cetera.

Due to this dynamic nature of python types, where many are applicable for a given operation, any type checker would not be able to reasonably assert whether a call for this function should be allowed or not.

To assist our type checker we can now provide type hints for it in the Function definition indicating the type that we allow.

To indicate that we only want to allow `int` types we can change our function definition to look like:

```
def two_sum(a: int, b: int):  
    return a + b
```

Annotations follow the argument name and are separated by a `:` character.

Similarly, to indicate only `str` types are allowed, we'd change our function to specify it:

```
def two_sum(a: str, b: str):  
    return a + b
```

Apart from specifying the type of the arguments, one could also indicate the return value of a function call. This is done by adding the `->` character followed by the type after the closing parenthesis in the argument list *but* before the `:` at the end of the function declaration:

```
def two_sum(a: int, b: int) -> int:  
    return a + b
```

Now we've indicated that the return value when calling `two_sum` should be of type `int`. Similarly we can define appropriate values for `str`, `float`, `list`, `set` and others.

Although type hints are mostly used by type checkers and IDEs, sometimes you may need to retrieve them. This can be done using the `__annotations__` special attribute:

```
two_sum.__annotations__  
# {'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}
```

Section 87.2: NamedTuple

Creating a namedtuple with type hints is done using the function `NamedTuple` from the `typing` module:

```
import typing
Point = typing.NamedTuple('Point', [('x', int), ('y', int)])
```

Note that the name of the resulting type is the first argument to the function, but it should be assigned to a variable with the same name to ease the work of type checkers.

Section 87.3: Generic Types

The `typing.TypeVar` is a generic type factory. Its primary goal is to serve as a parameter/placeholder for generic function/class/method annotations:

```
import typing

T = typing.TypeVar("T")

def get_first_element(l: typing.Sequence[T]) -> T:
    """Gets the first element of a sequence."""
    return l[0]
```

Section 87.4: Variables and Attributes

Variables are annotated using comments:

```
x = 3 # type: int
x = negate(x)
x = 'a type-checker might catch this error'
```

Python 3.x Version ≥ 3.6

Starting from Python 3.6, there is also [new syntax for variable annotations](#). The code above might use the form

```
x: int = 3
```

Unlike with comments, it is also possible to just add a type hint to a variable that was not previously declared, without setting a value to it:

```
y: int
```

Additionally if these are used in the module or the class level, the type hints can be retrieved using `typing.get_type_hints(class_or_module)`:

```
class Foo:
    x: int
    y: str = 'abc'

print(typing.get_type_hints(Foo))
# ChainMap({'x': <class 'int'>, 'y': <class 'str'>}, {})
```

Alternatively, they can be accessed by using the `__annotations__` special variable or attribute:

```
x: int
print(__annotations__)
```



```
# {'x': <class 'int'>}

class C:
    s: str
print(C.__annotations__)
# {'s': <class 'str'>}
```

Section 87.5: Class Members and Methods

```
class A:
    x = None # type: float
    def __init__(self, x: float) -> None:
        """
        self should not be annotated
        init should be annotated to return None
        """
        self.x = x

    @classmethod
    def from_int(cls, x: int) -> 'A':
        """
        cls should not be annotated
        Use forward reference to refer to current class with string literal 'A'
        """
        return cls(float(x))
```

Forward reference of the current class is needed since annotations are evaluated when the function is defined. Forward references can also be used when referring to a class that would cause a circular import if imported.

Section 87.6: Type hints for keyword arguments

```
def hello_world(greeting: str = 'Hello'):
    print(greeting + ' world!')
```

Note the spaces around the equal sign as opposed to how keyword arguments are usually styled.

PHASE T · Testing & Debugging Interlude

~1 week · off-book module

The biggest gap between 'read Fluent Python' and 'ship ML systems'. Do not skip.

Chapters in this phase:

- PNP Ch 137 | Logging Module [companion]
- PNP Ch 88 | Exceptions [companion]

PNP Ch 137 | Logging Module [companion]

Chapter 137: Logging

Section 137.1: Introduction to Python Logging

This module defines functions and classes which implement a flexible event logging system for applications and libraries.

The key benefit of having the logging API provided by a standard library module is that all Python modules can participate in logging, so your application log can include your own messages integrated with messages from third-party modules.

So, let's start:

Example Configuration Directly in Code

```
import logging

logger = logging.getLogger()
handler = logging.StreamHandler()
formatter = logging.Formatter(
    '%(asctime)s %(name)-12s %(levelname)-8s %(message)s')
handler.setFormatter(formatter)
logger.addHandler(handler)
logger.setLevel(logging.DEBUG)

logger.debug('this is a %s test', 'debug')
```

Output example:

```
2016-07-26 18:53:55,332 root DEBUG this is a debug test
```

Example Configuration via an INI File

Assuming the file is named `logging_config.ini`. More details for the file format are in the [logging configuration](#) section of the [logging tutorial](#).

```
[loggers]
keys=root

[handlers]
keys=stream_handler

[formatters]
keys=formatter

[logger_root]
level=DEBUG
handlers=stream_handler

[handler_stream_handler]
class=StreamHandler
level=DEBUG
formatter=formatter
args=(sys.stderr,)

[formatter_formatter]
```

```
format=%(asctime)s %(name)-12s %(levelname)-8s %(message)s
```

Then use `logging.config.fileConfig()` in the code:

```
import logging
from logging.config import fileConfig

fileConfig('logging_config.ini')
logger = logging.getLogger()
logger.debug('often makes a very good meal of %s', 'visiting tourists')
```

Example Configuration via a Dictionary

As of Python 2.7, you can use a dictionary with configuration details. [PEP 391](#) contains a list of the mandatory and optional elements in the configuration dictionary.

```
import logging
from logging.config import dictConfig

logging_config = dict(
    version = 1,
    formatters = {
        'f': {'format':
              '%(asctime)s %(name)-12s %(levelname)-8s %(message)s'}
    },
    handlers = {
        'h': {'class': 'logging.StreamHandler',
              'formatter': 'f',
              'level': logging.DEBUG}
    },
    root = {
        'handlers': ['h'],
        'level': logging.DEBUG,
    },
)

dictConfig(logging_config)

logger = logging.getLogger()
logger.debug('often makes a very good meal of %s', 'visiting tourists')
```

Section 137.2: Logging exceptions

If you want to log exceptions you can and should make use of the `logging.exception(msg)` method:

```
>>> import logging
>>> logging.basicConfig()
>>> try:
...     raise Exception('foo')
... except:
...     logging.exception('bar')
...
ERROR:root:bar
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
Exception: foo
```

Do not pass the exception as argument:

As `logging.exception(msg)` expects a `msg` arg, it is a common pitfall to pass the exception into the logging call like this:

```
>>> try:
...     raise Exception('foo')
... except Exception as e:
...     logging.exception(e)
...
ERROR:root:foo
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
Exception: foo
```

While it might look as if this is the right thing to do at first, it is actually problematic due to the reason how exceptions and various encoding work together in the logging module:

```
>>> try:
...     raise Exception(u'föö')
... except Exception as e:
...     logging.exception(e)
...
Traceback (most recent call last):
  File ".../python2.7/logging/__init__.py", line 861, in emit
    msg = self.format(record)
  File ".../python2.7/logging/__init__.py", line 734, in format
    return fmt.format(record)
  File ".../python2.7/logging/__init__.py", line 469, in format
    s = self._fmt % record.__dict__
UnicodeEncodeError: 'ascii' codec can't encode characters in position 1-2: ordinal not in
range(128)
Logged from file <stdin>, line 4
```

Trying to log an exception that contains unicode chars, this way will [fail miserably](#). It will hide the stacktrace of the original exception by overriding it with a new one that is raised during formatting of your `logging.exception(e)` call.

Obviously, in your own code, you might be aware of the encoding in exceptions. However, 3rd party libs might handle this in a different way.

Correct Usage:

If instead of the exception you just pass a message and let python do its magic, it will work:

```
>>> try:
...     raise Exception(u'föö')
... except Exception as e:
...     logging.exception('bar')
...
ERROR:root:bar
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
Exception: f\xfa\xfb
```

As you can see we don't actually use `e` in that case, the call to `logging.exception(...)` magically formats the most recent exception.

Logging exceptions with non ERROR log levels

If you want to log an exception with another log level than ERROR, you can use the `exc_info` argument of the default loggers:

```
logging.debug('exception occurred', exc_info=1)
logging.info('exception occurred', exc_info=1)
logging.warning('exception occurred', exc_info=1)
```

Accessing the exception's message

Be aware that libraries out there might throw exceptions with messages as any of unicode or (utf-8 if you're lucky) byte-strings. If you really need to access an exception's text, the only reliable way, that will always work, is to use `repr(e)` or the `%r` string formatting:

```
>>> try:
...     raise Exception(u'föö')
... except Exception as e:
...     logging.exception('received this exception: %r' % e)
...
ERROR:root:received this exception: Exception(u'f\x66\x6f\x6f',)
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
Exception: f\x66\x6f\x6f
```

PNP Ch 88 | Exceptions [companion]

Chapter 88: Exceptions

Errors detected during execution are called exceptions and are not unconditionally fatal. Most exceptions are not handled by programs; it is possible to write programs that handle selected exceptions. There are specific features in Python to deal with exceptions and exception logic. Furthermore, exceptions have a rich type hierarchy, all inheriting from the `BaseException` type.

Section 88.1: Catching Exceptions

Use `try...except`: to catch exceptions. You should specify as precise an exception as you can:

```
try:
    x = 5 / 0
except ZeroDivisionError as e:
    # `e` is the exception object
    print("Got a divide by zero! The exception was:", e)
    # handle exceptional case
    x = 0
finally:
    print "The END"
    # it runs no matter what execute.
```

The exception class that is specified - in this case, `ZeroDivisionError` - catches any exception that is of that class or of any subclass of that exception.

For example, `ZeroDivisionError` is a subclass of `ArithmeticError`:

```
>>> ZeroDivisionError.__bases__
(<class 'ArithmeticError'>,)
```

And so, the following will still catch the `ZeroDivisionError`:

```
try:
    5 / 0
except ArithmeticError:
    print("Got arithmetic error")
```

Section 88.2: Do not catch everything!

While it's often tempting to catch every `Exception`:

```
try:
    very_difficult_function()
except Exception:
    # log / try to reconnect / exit graciously
finally:
    print "The END"
    # it runs no matter what execute.
```

Or even everything (that includes `BaseException` and all its children including `Exception`):

```
try:
    even_more_difficult_function()
except:
    pass # do whatever needed
```

In most cases it's bad practice. It might catch more than intended, such as `SystemExit`, `KeyboardInterrupt` and `MemoryError` - each of which should generally be handled differently than usual system or logic errors. It also means there's no clear understanding for what the internal code may do wrong and how to recover properly from that condition. If you're catching every error, you won't know what error occurred or how to fix it.

This is more commonly referred to as 'bug masking' and should be avoided. Let your program crash instead of silently failing or even worse, failing at deeper level of execution. (Imagine it's a transactional system)

Usually these constructs are used at the very outer level of the program, and will log the details of the error so that the bug can be fixed, or the error can be handled more specifically.

Section 88.3: Re-raising exceptions

Sometimes you want to catch an exception just to inspect it, e.g. for logging purposes. After the inspection, you want the exception to continue propagating as it did before.

In this case, simply use the `raise` statement with no parameters.

```
try:
    5 / 0
except ZeroDivisionError:
    print("Got an error")
    raise
```

Keep in mind, though, that someone further up in the caller stack can still catch the exception and handle it somehow. The done output could be a nuisance in this case because it will happen in any case (caught or not caught). So it might be a better idea to raise a different exception, containing your comment about the situation as well as the original exception:

```
try:
    5 / 0
except ZeroDivisionError as e:
    raise ZeroDivisionError("Got an error", e)
```

But this has the drawback of reducing the exception trace to exactly this `raise` while the `raise` without argument retains the original exception trace.

In Python 3 you can keep the original stack by using the `raise-from` syntax:

```
raise ZeroDivisionError("Got an error") from e
```

Section 88.4: Catching multiple exceptions

There are a few ways to [catch multiple exceptions](#).

The first is by creating a tuple of the exception types you wish to catch and handle in the same manner. This example will cause the code to ignore `KeyError` and `AttributeError` exceptions.

```
try:
    d = {}
    a = d[1]
    b = d.non_existing_field
except (KeyError, AttributeError) as e:
    print("A KeyError or an AttributeError exception has been caught.")
```

If you wish to handle different exceptions in different ways, you can provide a separate exception block for each type. In this example, we still catch the `KeyError` and `AttributeError`, but handle the exceptions in different manners.

```
try:
    d = {}
    a = d[1]
    b = d.non_existing_field
except KeyError as e:
    print("A KeyError has occurred. Exception message:", e)
except AttributeError as e:
    print("An AttributeError has occurred. Exception message:", e)
```

Section 88.5: Exception Hierarchy

Exception handling occurs based on an exception hierarchy, determined by the inheritance structure of the exception classes.

For example, `IOError` and `OSError` are both subclasses of `EnvironmentError`. Code that catches an `IOError` will not catch an `OSError`. However, code that catches an `EnvironmentError` will catch both `IOErrors` and `OSErrors`.

The hierarchy of built-in exceptions:

Python 2.x Version ≥ 2.3

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
+-- StopIteration
+-- StandardError
| +-- BufferError
| +-- ArithmeticError
| | +-- FloatingPointError
| | +-- OverflowError
| | +-- ZeroDivisionError
| +-- AssertionError
| +-- AttributeError
| +-- EnvironmentError
| | +-- IOError
| | +-- OSError
| | +-- WindowsError (Windows)
| | +-- VMSError (VMS)
| +-- EOFError
| +-- ImportError
| +-- LookupError
| | +-- IndexError
| | +-- KeyError
| +-- MemoryError
| +-- NameError
| | +-- UnboundLocalError
| +-- ReferenceError
| +-- RuntimeError
| | +-- NotImplementedError
| +-- SyntaxError
| | +-- IndentationError
| | +-- TabError
| +-- SystemError
| +-- TypeError
| +-- ValueError
```

```
| +-- UnicodeError
| +-- UnicodeDecodeError
| +-- UnicodeEncodeError
| +-- UnicodeTranslateError
+-- Warning
+-- DeprecationWarning
+-- PendingDeprecationWarning
+-- RuntimeWarning
+-- SyntaxWarning
+-- UserWarning
+-- FutureWarning
+-- ImportWarning
+-- UnicodeWarning
+-- BytesWarning
```

Python 3.x Version ≥ 3.0

```
BaseException
+-- SystemExit
+-- KeyboardInterrupt
+-- GeneratorExit
+-- Exception
+-- StopIteration
+-- StopAsyncIteration
+-- ArithmeticError
| +-- FloatingPointError
| +-- OverflowError
| +-- ZeroDivisionError
+-- AssertionError
+-- AttributeError
+-- BufferError
+-- EOFError
+-- ImportError
+-- LookupError
| +-- IndexError
| +-- KeyError
+-- MemoryError
+-- NameError
| +-- UnboundLocalError
+-- OSError
| +-- BlockingIOError
| +-- ChildProcessError
| +-- ConnectionError
| | +-- BrokenPipeError
| | +-- ConnectionAbortedError
| | +-- ConnectionRefusedError
| | +-- ConnectionResetError
| +-- FileExistsError
| +-- FileNotFoundError
| +-- InterruptedError
| +-- IsADirectoryError
| +-- NotADirectoryError
| +-- PermissionError
| +-- ProcessLookupError
| +-- TimeoutError
+-- ReferenceError
+-- RuntimeError
| +-- NotImplementedError
| +-- RecursionError
+-- SyntaxError
| +-- IndentationError
| +-- TabError
+-- SystemError
+-- TypeError
```

```
+-- ValueError
| +-- UnicodeError
| +-- UnicodeDecodeError
| +-- UnicodeEncodeError
| +-- UnicodeTranslateError
+-- Warning
+-- DeprecationWarning
+-- PendingDeprecationWarning
+-- RuntimeWarning
+-- SyntaxWarning
+-- UserWarning
+-- FutureWarning
+-- ImportWarning
+-- UnicodeWarning
+-- BytesWarning
+-- ResourceWarning
```

Section 88.6: Else

Code in an `else` block will only be run if no exceptions were raised by the code in the `try` block. This is useful if you have some code you don't want to run if an exception is thrown, but you don't want exceptions thrown by that code to be caught.

For example:

```
try:
    data = {1: 'one', 2: 'two'}
    print(data[1])
except KeyError as e:
    print('key not found')
else:
    raise ValueError()
# Output: one
# Output: ValueError
```

Note that this kind of `else:` cannot be combined with an `if` starting the `else`-clause to an `elif`. If you have a following `if` it needs to stay indented below that `else::`

```
try:
    ...
except ...:
    ...
else:
    if ...:
        ...
    elif ...:
        ...
    else:
        ...
```

Section 88.7: Raising Exceptions

If your code encounters a condition it doesn't know how to handle, such as an incorrect parameter, it should raise the appropriate exception.

```
def even_the_odds(odds):
    if odds % 2 != 1:
        raise ValueError("Did not get an odd number")
```

```
return odds + 1
```

Section 88.8: Creating custom exception types

Create a class inheriting from `Exception`:

```
class FooException(Exception):
    pass
try:
    raise FooException("insert description here")
except FooException:
    print("A FooException was raised.")
```

or another exception type:

```
class NegativeError(ValueError):
    pass

def foo(x):
    # function that only accepts positive values of x
    if x < 0:
        raise NegativeError("Cannot process negative numbers")
    ... # rest of function body
try:
    result = foo(int(input("Enter a positive integer: "))) # raw_input in Python 2.x
except NegativeError:
    print("You entered a negative number!")
else:
    print("The result was " + str(result))
```

Section 88.9: Practical examples of exception handling

User input

Imagine you want a user to enter a number via `input`. You want to ensure that the input is a number. You can use `try/except` for this:

Python 3.x Version ≥ 3.0

```
while True:
    try:
        nb = int(input('Enter a number: '))
        break
    except ValueError:
        print('This is not a number, try again.')
```

Note: Python 2.x would use `raw_input` instead; the function `input` exists in Python 2.x but has different semantics. In the above example, `input` would also accept expressions such as `2 + 2` which evaluate to a number.

If the input could not be converted to an integer, a `ValueError` is raised. You can catch it with `except`. If no exception is raised, `break` jumps out of the loop. After the loop, `nb` contains an integer.

Dictionaries

Imagine you are iterating over a list of consecutive integers, like `range(n)`, and you have a list of dictionaries `d` that contains information about things to do when you encounter some particular integers, say *skip the `d[i]` next ones*.

```
d = [{7: 3}, {25: 9}, {38: 5}]

for i in range(len(d)):
    do_stuff(i)
    try:
        dic = d[i]
        i += dic[i]
    except KeyError:
        i += 1
```

A `KeyError` will be raised when you try to get a value from a dictionary for a key that doesn't exist.

Section 88.10: Exceptions are Objects too

Exceptions are just regular Python objects that inherit from the built-in `BaseException`. A Python script can use the `raise` statement to interrupt execution, causing Python to print a stack trace of the call stack at that point and a representation of the exception instance. For example:

```
>>> def failing_function():
...     raise ValueError('Example error!')
>>> failing_function()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in failing_function
ValueError: Example error!
```

which says that a `ValueError` with the message `'Example error!'` was raised by our `failing_function()`, which was executed in the interpreter.

Calling code can choose to handle any and all types of exception that a call can raise:

```
>>> try:
...     failing_function()
... except ValueError:
...     print('Handled the error')
Handled the error
```

You can get hold of the exception objects by assigning them in the `except ...` part of the exception handling code:

```
>>> try:
...     failing_function()
... except ValueError as e:
...     print('Caught exception', repr(e))
Caught exception ValueError('Example error!',)
```

A complete list of built-in Python exceptions along with their descriptions can be found in the Python Documentation: <https://docs.python.org/3.5/library/exceptions.html>. And here is the full list arranged hierarchically: Exception Hierarchy.

Section 88.11: Running clean-up code with finally

Sometimes, you may want something to occur regardless of whatever exception happened, for example, if you have to clean up some resources.

The `finally` block of a `try` clause will happen regardless of whether any exceptions were raised.

```

resource = allocate_some_expensive_resource()
try:
    do_stuff(resource)
except SomeException as e:
    log_error(e)
    raise # re-raise the error
finally:
    free_expensive_resource(resource)

```

This pattern is often better handled with context managers (using the `with` statement).

Section 88.12: Chain exceptions with `raise from`

In the process of handling an exception, you may want to raise another exception. For example, if you get an `IOError` while reading from a file, you may want to raise an application-specific error to present to the users of your library, instead.

Python 3.x Version ≥ 3.0

You can chain exceptions to show how the handling of exceptions proceeded:

```

>>> try:
    5 / 0
except ZeroDivisionError as e:
    raise ValueError("Division failed") from e

```

```

Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
ZeroDivisionError: division by zero

```

The above exception was the direct cause of the following exception:

```

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
ValueError: Division failed

```


PHASE 06 · Concurrency & Async

~3 weeks · 5 chapters

Parallelize data loading, training pipelines, and ML model serving APIs.

Chapters in this phase:

- FP Ch 17 | Concurrency with Futures [= plan Ch 19+20]
- FP Ch 18 | Concurrency with asyncio [= plan Ch 21]
- HPP Ch 9 | The multiprocessing Module
- HPP Ch 8 | Asynchronous I/O
- PNP Ch 54 | Asyncio Module [companion]
- PNP Ch 59 | Queue Module [companion]

FP Ch 17 | Concurrency with Futures [= plan Ch 19+20]

Concurrency with Futures

The people bashing threads are typically system programmers which have in mind use cases that the typical application programmer will never encounter in her life. [...] In 99% of the use cases an application programmer is likely to run into, the simple pattern of spawning a bunch of independent threads and collecting the results in a queue is everything one needs to know.¹

— Michele Simionato
Python deep thinker

This chapter focuses on the `concurrent.futures` library introduced in Python 3.2, but also available for Python 2.5 and newer as the **futures package** on PyPI. This library encapsulates the pattern described by Michele Simionato in the preceding quote, making it almost trivial to use.

Here I also introduce the concept of “futures”—objects representing the asynchronous execution of an operation. This powerful idea is the foundation not only of `concurrent.futures` but also of the `asyncio` package, which we’ll cover in **Chapter 18**.

We’ll start with a motivating example.

Example: Web Downloads in Three Styles

To handle network I/O efficiently, you need concurrency, as it involves high latency—so instead of wasting CPU cycles waiting, it’s better to do something else until a response comes back from the network.

To make this last point with code, I wrote three simple programs to download images of 20 country flags from the Web. The first one, *flags.py*, runs sequentially: it only re-

1. From Michele Simionato’s post **Threads, processes and concurrency in Python: some thoughts**, subtitled “Removing the hype around the multicore (non) revolution and some (hopefully) sensible comment about threads and other forms of concurrency.”

requests the next image when the previous one is downloaded and saved to disk. The other two scripts make concurrent downloads: they request all images practically at the same time, and save the files as they arrive. The *flags_threadpool.py* script uses the `concurrent.futures` package, while *flags_asyncio.py* uses `asyncio`.

Example 17-1 shows the result of running the three scripts, three times each. I also posted a [73s video on YouTube](#) so you can watch them running while an OS X Finder window displays the flags as they are saved. The scripts are downloading images from *flupy.org*, which is behind a CDN, so you may see slower results in the first runs. The results in **Example 17-1** were obtained after several runs, so the CDN cache was warm.

*Example 17-1. Three typical runs of the scripts *flags.py*, *flags_threadpool.py*, and *flags_asyncio.py**

```
$ python3 flags.py
BD BR CD CN DE EG ET FR ID IN IR JP MX NG PH PK RU TR US VN ❶
20 flags downloaded in 7.26s ❷
$ python3 flags.py
BD BR CD CN DE EG ET FR ID IN IR JP MX NG PH PK RU TR US VN
20 flags downloaded in 7.20s
$ python3 flags.py
BD BR CD CN DE EG ET FR ID IN IR JP MX NG PH PK RU TR US VN
20 flags downloaded in 7.09s
$ python3 flags_threadpool.py
DE BD CN JP ID EG NG BR RU CD IR MX US PH FR PK VN IN ET TR
20 flags downloaded in 1.37s ❸
$ python3 flags_threadpool.py
EG BR FR IN BD JP DE RU PK PH CD MX ID US NG TR CN VN ET IR
20 flags downloaded in 1.60s
$ python3 flags_threadpool.py
BD DE EG CN ID RU IN VN ET MX FR CD NG US JP TR PK BR IR PH
20 flags downloaded in 1.22s
$ python3 flags_asyncio.py ❹
BD BR IN ID TR DE CN US IR PK PH FR RU NG VN ET MX EG JP CD
20 flags downloaded in 1.36s
$ python3 flags_asyncio.py
RU CN BR IN FR BD TR EG VN IR PH CD ET ID NG DE JP PK MX US
20 flags downloaded in 1.27s
$ python3 flags_asyncio.py
RU IN ID DE BR VN PK MX US IR ET EG NG BD FR CN JP PH CD TR ❺
20 flags downloaded in 1.42s
```

- ❶ The output for each run starts with the country codes of the flags as they are downloaded, and ends with a message stating the elapsed time.
- ❷ It took *flags.py* an average 7.18s to download 20 images.
- ❸ The average for *flags_threadpool.py* was 1.40s.
- ❹ For *flags_asyncio.py*, 1.35 was the average time.

- ⑤ Note the order of the country codes: the downloads happened in a different order every time with the concurrent scripts.

The difference in performance between the concurrent scripts is not significant, but they are both more than five times faster than the sequential script—and this is just for a fairly small task. If you scale the task to hundreds of downloads, the concurrent scripts can outpace the sequential one by a factor of 20 or more.



While testing concurrent HTTP clients on the public Web you may inadvertently launch a denial-of-service (DoS) attack, or be suspected of doing so. In the case of [Example 17-1](#), it's OK to do it because those scripts are hardcoded to make only 20 requests. For testing nontrivial HTTP clients, you should set up your own test server. The *17-futures/countries/README.rst* file in the *Fluent Python code* GitHub repository has instructions for setting a local Nginx server.

Now let's study the implementations of two of the scripts tested in [Example 17-1](#): *flags.py* and *flags_threadpool.py*. I will leave the third script, *flags_asyncio.py*, for [Chapter 18](#), but I wanted to demonstrate all three together to make a point: regardless of the concurrency strategy you use—threads or `asyncio`—you'll see vastly improved throughput over sequential code in I/O-bound applications, if you code it properly.

On to the code.

A Sequential Download Script

[Example 17-2](#) is not very interesting, but we'll reuse most of its code and settings to implement the concurrent scripts, so it deserves some attention.



For clarity, there is no error handling in [Example 17-2](#). We will deal with exceptions later, but here we want to focus on the basic structure of the code, to make it easier to contrast this script with the concurrent ones.

Example 17-2. flags.py: sequential download script; some functions will be reused by the other scripts

```
import os
import time
import sys

import requests ①
```

```

POP20_CC = ('CN IN US ID BR PK NG BD RU JP '
            'MX PH VN ET EG DE IR TR CD FR').split() ❷

BASE_URL = 'http://flupy.org/data/flags' ❸

DEST_DIR = 'downloads/' ❹

def save_flag(img, filename): ❺
    path = os.path.join(DEST_DIR, filename)
    with open(path, 'wb') as fp:
        fp.write(img)

def get_flag(cc): ❻
    url = '{}/{cc}/{cc}.gif'.format(BASE_URL, cc=cc.lower())
    resp = requests.get(url)
    return resp.content

def show(text): ❼
    print(text, end=' ')
    sys.stdout.flush()

def download_many(cc_list): ❽
    for cc in sorted(cc_list): ❾
        image = get_flag(cc)
        show(cc)
        save_flag(image, cc.lower() + '.gif')

    return len(cc_list)

def main(download_many): ❿
    t0 = time.time()
    count = download_many(POP20_CC)
    elapsed = time.time() - t0
    msg = '\n{} flags downloaded in {:.2f}s'
    print(msg.format(count, elapsed))

if __name__ == '__main__':
    main(download_many) ⓫

```

- ❶ Import the requests library; it's not part of the standard library, so by convention we import it after the standard library modules os, time, and sys, and separate it from them with a blank line.
- ❷ List of the ISO 3166 country codes for the 20 most populous countries in order of decreasing population.

- ③ The website with the flag images.²
- ④ Local directory where the images are saved.
- ⑤ Simply save the `img` (a byte sequence) to `filename` in the `DEST_DIR`.
- ⑥ Given a country code, build the URL and download the image, returning the binary contents of the response.
- ⑦ Display a string and flush `sys.stdout` so we can see progress in a one-line display; this is needed because Python normally waits for a line break to flush the `stdout` buffer.
- ⑧ `download_many` is the key function to compare with the concurrent implementations.
- ⑨ Loop over the list of country codes in alphabetical order, to make it clear that the ordering is preserved in the output; return the number of country codes downloaded.
- ⑩ `main` records and reports the elapsed time after running `download_many`.
- ⑪ `main` must be called with the function that will make the downloads; we pass the `download_many` function as an argument so that `main` can be used as a library function with other implementations of `download_many` in the next examples.



The `requests` library by Kenneth Reitz is [available on PyPI](#) and is more powerful and easier to use than the `urllib.request` module from the Python 3 standard library. In fact, `requests` is considered a model Pythonic API. It is also compatible with Python 2.6 and up, while the `urllib2` from Python 2 was moved and renamed in Python 3, so it's more convenient to use `requests` regardless of the Python version you're targeting.

There's really nothing new to *flags.py*. It serves as a baseline for comparing the other scripts and I used it as a library to avoid redundant code when implementing them. Now let's see a reimplementation using `concurrent.futures`.

Downloading with `concurrent.futures`

The main features of the `concurrent.futures` package are the `ThreadPoolExecutor` and `ProcessPoolExecutor` classes, which implement an interface that allows you to submit callables for execution in different threads or processes, respectively. The classes manage an internal pool of worker threads or processes, and a queue of tasks to be

2. The images are originally from the [CIA World Factbook](#), a public-domain, U.S. government publication. I copied them to my site to avoid the risk of launching a DOS attack on CIA.gov.

executed. But the interface is very high level and we don't need to know about any of those details for a simple use case like our flag downloads.

Example 17-3 shows the easiest way to implement the downloads concurrently, using the `ThreadPoolExecutor.map` method.

Example 17-3. flags_threadpool.py: threaded download script using futures.ThreadPoolExecutor

```
from concurrent import futures

from flags import save_flag, get_flag, show, main ❶

MAX_WORKERS = 20 ❷

def download_one(cc): ❸
    image = get_flag(cc)
    show(cc)
    save_flag(image, cc.lower() + '.gif')
    return cc

def download_many(cc_list):
    workers = min(MAX_WORKERS, len(cc_list)) ❹
    with futures.ThreadPoolExecutor(workers) as executor: ❺
        res = executor.map(download_one, sorted(cc_list)) ❻

    return len(list(res)) ❼

if __name__ == '__main__':
    main(download_many) ❽
```

- ❶ Reuse some functions from the `flags` module ([Example 17-2](#)).
- ❷ Maximum number of threads to be used in the `ThreadPoolExecutor`.
- ❸ Function to download a single image; this is what each thread will execute.
- ❹ Set the number of worker threads: use the smaller number between the maximum we want to allow (`MAX_WORKERS`) and the actual items to be processed, so no unnecessary threads are created.
- ❺ Instantiate the `ThreadPoolExecutor` with that number of worker threads; the `executor.__exit__` method will call `executor.shutdown(wait=True)`, which will block until all threads are done.
- ❻ The `map` method is similar to the `map` built-in, except that the `download_one` function will be called concurrently from multiple threads; it returns a generator that can be iterated over to retrieve the value returned by each function.

- ⑦ Return the number of results obtained; if any of the threaded calls raised an exception, that exception would be raised here as the implicit `next()` call tried to retrieve the corresponding return value from the iterator.
- ⑧ Call the `main` function from the `flags` module, passing the enhanced version of `download_many`.

Note that the `download_one` function from [Example 17-3](#) is essentially the body of the `for` loop in the `download_many` function from [Example 17-2](#). This is a common refactoring when writing concurrent code: turning the body of a sequential `for` loop into a function to be called concurrently.

The library is called `concurrency.futures` yet there are no futures to be seen in [Example 17-3](#), so you may be wondering where they are. The next section explains.

Where Are the Futures?

Futures are essential components in the internals of `concurrent.futures` and of `asyncio`, but as users of these libraries we sometimes don't see them. [Example 17-3](#) leverages futures behind the scenes, but the code I wrote does not touch them directly. This section is an overview of futures, with an example that shows them in action.

As of Python 3.4, there are two classes named `Future` in the standard library: `concurrent.futures.Future` and `asyncio.Future`. They serve the same purpose: an instance of either `Future` class represents a deferred computation that may or may not have completed. This is similar to the `Deferred` class in Twisted, the `Future` class in Tornado, and `Promise` objects in various JavaScript libraries.

Futures encapsulate pending operations so that they can be put in queues, their state of completion can be queried, and their results (or exceptions) can be retrieved when available.

An important thing to know about futures in general is that you and I should not create them: they are meant to be instantiated exclusively by the concurrency framework, be it `concurrent.futures` or `asyncio`. It's easy to understand why: a `Future` represents something that will eventually happen, and the only way to be sure that something will happen is to schedule its execution. Therefore, `concurrent.futures.Future` instances are created only as the result of scheduling something for execution with a `concurrent.futures.Executor` subclass. For example, the `Executor.submit()` method takes a callable, schedules it to run, and returns a future.

Client code is not supposed to change the state of a future: the concurrency framework changes the state of a future when the computation it represents is done, and we can't control when that happens.

Both types of `Future` have a `.done()` method that is nonblocking and returns a Boolean that tells you whether the callable linked to that future has executed or not. Instead of asking whether a future is done, client code usually asks to be notified. That's why both `Future` classes have an `.add_done_callback()` method: you give it a callable, and the callable will be invoked with the future as the single argument when the future is done.

There is also a `.result()` method, which works the same in both classes when the future is done: it returns the result of the callable, or re-raises whatever exception might have been thrown when the callable was executed. However, when the future is not done, the behavior of the `result` method is very different between the two flavors of `Future`. In a `concurrency.futures.Future` instance, invoking `f.result()` will block the caller's thread until the result is ready. An optional `timeout` argument can be passed, and if the future is not done in the specified time, a `TimeoutError` exception is raised. In “[asyncio.Future: Nonblocking by Design](#)” on page 545, we'll see that the `asyncio.Future.result` method does not support timeout, and the preferred way to get the result of futures in that library is to use `yield from`—which doesn't work with `concurrency.futures.Future` instances.

Several functions in both libraries return futures; others use them in their implementation in a way that is transparent to the user. An example of the latter is the `Executor.map` we saw in [Example 17-3](#): it returns an iterator in which `__next__` calls the `result` method of each future, so what we get are the results of the futures, and not the futures themselves.

To get a practical look at futures, we can rewrite [Example 17-3](#) to use the `concurrent.futures.as_completed` function, which takes an iterable of futures and returns an iterator that yields futures as they are done.

Using `futures.as_completed` requires changes to the `download_many` function only. The higher-level `executor.map` call is replaced by two `for` loops: one to create and schedule the futures, the other to retrieve their results. While we are at it, we'll add a few `print` calls to display each future before and after it's done. [Example 17-4](#) shows the code for a new `download_many` function. The code for `download_many` grew from 5 to 17 lines, but now we get to inspect the mysterious futures. The remaining functions are the same as in [Example 17-3](#).

Example 17-4. `flags_threadpool_ac.py`: replacing `executor.map` with `executor.submit` and `futures.as_completed` in the `download_many` function

```
def download_many(cc_list):
    cc_list = cc_list[:5]  ❶
    with futures.ThreadPoolExecutor(max_workers=3) as executor:  ❷
        to_do = []
        for cc in sorted(cc_list):  ❸
            future = executor.submit(download_one, cc)  ❹
            to_do.append(future)  ❺
```

```

msg = 'Scheduled for {}: {}'
print(msg.format(cc, future)) ❹

results = []
for future in futures.as_completed(to_do): ❺
    res = future.result() ❻
    msg = '{} result: {!r}'
    print(msg.format(future, res)) ❼
    results.append(res)

return len(results)

```

- ❶ For this demonstration, use only the top five most populous countries.
- ❷ Hardcode `max_workers` to 3 so we can observe pending futures in the output.
- ❸ Iterate over country codes alphabetically, to make it clear that results arrive out of order.
- ❹ `executor.submit` schedules the callable to be executed, and returns a future representing this pending operation.
- ❺ Store each future so we can later retrieve them with `as_completed`.
- ❻ Display a message with the country code and the respective future.
- ❼ `as_completed` yields futures as they are completed.
- ❸ Get the result of this future.
- ❼ Display the future and its result.

Note that the `future.result()` call will never block in this example because the future is coming out of `as_completed`. [Example 17-5](#) shows the output of one run of [Example 17-4](#).

Example 17-5. Output of `flags_threadpool_ac.py`

```

$ python3 flags_threadpool_ac.py
Scheduled for BR: <Future at 0x100791518 state=running> ❶
Scheduled for CN: <Future at 0x100791710 state=running>
Scheduled for ID: <Future at 0x100791a90 state=running>
Scheduled for IN: <Future at 0x101807080 state=pending> ❷
Scheduled for US: <Future at 0x101807128 state=pending>
CN <Future at 0x100791710 state=finished returned str> result: 'CN' ❸
BR ID <Future at 0x100791518 state=finished returned str> result: 'BR' ❹
<Future at 0x100791a90 state=finished returned str> result: 'ID'
IN <Future at 0x101807080 state=finished returned str> result: 'IN'
US <Future at 0x101807128 state=finished returned str> result: 'US'

5 flags downloaded in 0.70s

```

- ❶ The futures are scheduled in alphabetical order; the `repr()` of a future shows its state: the first three are running, because there are three worker threads.
- ❷ The last two futures are pending, waiting for worker threads.
- ❸ The first CN here is the output of `download_one` in a worker thread; the rest of the line is the output of `download_many`.
- ❹ Here two threads output codes before `download_many` in the main thread can display the result of the first thread.



If you run `flags_threadpool_ac.py` several times, you'll see the order of the results varying. Increasing the `max_workers` argument to 5 will increase the variation in the order of the results. Decreasing it to 1 will make this code run sequentially, and the order of the results will always be the order of the `submit` calls.

We saw two variants of the download script using `concurrent.futures`: [Example 17-3](#) with `ThreadPoolExecutor.map` and [Example 17-4](#) with `futures.as_completed`. If you are curious about the code for `flags_asyncio.py`, you may peek at [Example 18-5](#) in [Chapter 18](#).

Strictly speaking, none of the concurrent scripts we tested so far can perform downloads in parallel. The `concurrent.futures` examples are limited by the GIL, and the `flags_asyncio.py` is single-threaded.

At this point, you may have questions about the informal benchmarks we just did:

- How can `flags_threadpool.py` perform 5× faster than `flags.py` if Python threads are limited by a Global Interpreter Lock (GIL) that only lets one thread run at any time?
- How can `flags_asyncio.py` perform 5× faster than `flags.py` when both are single threaded?

I will answer the second question in “[Running Circling Around Blocking Calls](#)” on page 552.

Read on to understand why the GIL is nearly harmless with I/O-bound processing.

Blocking I/O and the GIL

The CPython interpreter is not thread-safe internally, so it has a Global Interpreter Lock (GIL), which allows only one thread at a time to execute Python bytecodes. That's why a single Python process usually cannot use multiple CPU cores at the same time.³

When we write Python code, we have no control over the GIL, but a built-in function or an extension written in C can release the GIL while running time-consuming tasks. In fact, a Python library coded in C can manage the GIL, launch its own OS threads, and take advantage of all available CPU cores. This complicates the code of the library considerably, and most library authors don't do it.

However, all standard library functions that perform blocking I/O release the GIL when waiting for a result from the OS. This means Python programs that are I/O bound can benefit from using threads at the Python level: while one Python thread is waiting for a response from the network, the blocked I/O function releases the GIL so another thread can run.

That's why David Beazley says: "Python threads are great at doing nothing."⁴



Every blocking I/O function in the Python standard library releases the GIL, allowing other threads to run. The `time.sleep()` function also releases the GIL. Therefore, Python threads are perfectly usable in I/O-bound applications, despite the GIL.

Now let's take a brief look at a simple way to work around the GIL for CPU-bound jobs using `concurrent.futures`.

Launching Processes with `concurrent.futures`

The [concurrent.futures documentation page](#) is subtitled "Launching parallel tasks". The package does enable truly parallel computations because it supports distributing work among multiple Python processes using the `ProcessPoolExecutor` class—thus bypassing the GIL and leveraging all available CPU cores, if you need to do CPU-bound processing.

Both `ProcessPoolExecutor` and `ThreadPoolExecutor` implement the generic `Executor` interface, so it's very easy to switch from a thread-based to a process-based solution using `concurrent.futures`.

3. This is a limitation of the CPython interpreter, not of the Python language itself. Jython and IronPython are not limited in this way; but PyPy, the fastest Python interpreter available, also has a GIL.

4. Slide 106 of "Generators: The Final Frontier".

There is no advantage in using a `ProcessPoolExecutor` for the flags download example or any I/O-bound job. It's easy to verify this; just change these lines in [Example 17-3](#):

```
def download_many(cc_list):  
    workers = min(MAX_WORKERS, len(cc_list))  
    with futures.ThreadPoolExecutor(workers) as executor:
```

To this:

```
def download_many(cc_list):  
    with futures.ProcessPoolExecutor() as executor:
```

For simple uses, the only notable difference between the two concrete executor classes is that `ThreadPoolExecutor.__init__` requires a `max_workers` argument setting the number of threads in the pool. That is an optional argument in `ProcessPoolExecutor`, and most of the time we don't use it—the default is the number of CPUs returned by `os.cpu_count()`. This makes sense: for CPU-bound processing, it makes no sense to ask for more workers than CPUs. On the other hand, for I/O-bound processing, you may use 10, 100, or 1,000 threads in a `ThreadPoolExecutor`; the best number depends on what you're doing and the available memory, and finding the optimal number will require careful testing.

A few tests revealed that the average time to download the 20 flags increased to 1.8s with a `ProcessPoolExecutor`—compared to 1.4s in the original `ThreadPoolExecutor` version. The main reason for this is likely to be the limit of four concurrent downloads on my four-core machine, against 20 workers in the thread pool version.

The value of `ProcessPoolExecutor` is in CPU-intensive jobs. I did some performance tests with a couple of CPU-bound scripts:

arcfour_futures.py

Encrypt and decrypt a dozen byte arrays with sizes from 149 KB to 384 KB using a pure-Python implementation of the RC4 algorithm (listing: [Example A-7](#)).

sha_futures.py

Compute the SHA-256 hash of a dozen 1 MB byte arrays with the standard library `hashlib` package, which uses the OpenSSL library (listing: [Example A-9](#)).

Neither of these scripts do I/O except to display summary results. They build and process all their data in memory, so I/O does not interfere with their execution time.

[Table 17-1](#) shows the average timings I got after 64 runs of the RC4 example and 48 runs of the SHA example. The timings include the time to actually spawn the worker processes.

Table 17-1. Time and speedup factor for the RC4 and SHA examples with one to four workers on an Intel Core i7 2.7 GHz quad-core machine, using Python 3.4

Workers	RC4 time	RC4 factor	SHA time	SHA factor
1	11.48s	1.00x	22.66s	1.00x
2	8.65s	1.33x	14.90s	1.52x
3	6.04s	1.90x	11.91s	1.90x
4	5.58s	2.06x	10.89s	2.08x

In summary, for cryptographic algorithms, you can expect to double the performance by spawning four worker processes with a `ProcessPoolExecutor`, if you have four CPU cores.

For the pure-Python RC4 example, you can get results 3.8 times faster if you use PyPy and four workers, compared with CPython and four workers. That's a speedup of 7.8 times in relation to the baseline of one worker with CPython in [Table 17-1](#).



If you are doing CPU-intensive work in Python, you should try **PyPy**. The `arcfour_futures.py` example ran from 3.8 to 5.1 times faster using PyPy, depending on the number of workers used. I tested with PyPy 2.4.0, which is compatible with Python 3.2.5, so it has `concurrent.futures` in the standard library.

Now let's investigate the behavior of a thread pool with a demonstration program that launches a pool with three workers, running five callables that output timestamped messages.

Experimenting with `Executor.map`

The simplest way to run several callables concurrently is with the `Executor.map` function we first saw in [Example 17-3](#). [Example 17-6](#) is a script to demonstrate how `Executor.map` works in some detail. Its output appears in [Example 17-7](#).

Example 17-6. `demo_executor_map.py`: Simple demonstration of the `map` method of `ThreadPoolExecutor`

```
from time import sleep, strftime
from concurrent import futures

def display(*args): ❶
    print(strftime('%H:%M:%S'), end=' ')
    print(*args)
```

```

def loiter(n): ❷
    msg = '{}loiter({}): doing nothing for {}s...'
    display(msg.format('\t'*n, n, n))
    sleep(n)
    msg = '{}loiter({}): done.'
    display(msg.format('\t'*n, n))
    return n * 10 ❸

def main():
    display('Script starting.')
    executor = futures.ThreadPoolExecutor(max_workers=3) ❹
    results = executor.map(loiter, range(5)) ❺
    display('results:', results) ❻.
    display('Waiting for individual results:')
    for i, result in enumerate(results): ❼
        display('result {}: {}'.format(i, result))

main()

```

- ❶ This function simply prints whatever arguments it gets, preceded by a timestamp in the format [HH:MM:SS].
- ❷ loiter does nothing except display a message when it starts, sleep for *n* seconds, then display a message when it ends; tabs are used to indent the messages according to the value of *n*.
- ❸ loiter returns *n* * 10 so we can see how to collect results.
- ❹ Create a ThreadPoolExecutor with three threads.
- ❺ Submit five tasks to the executor (because there are only three threads, only three of those tasks will start immediately: the calls loiter(0), loiter(1), and loiter(2)); this is a nonblocking call.
- ❻ Immediately display the results of invoking executor.map: it's a generator, as the output in [Example 17-7](#) shows.
- ❼ The enumerate call in the for loop will implicitly invoke next(results), which in turn will invoke _f.result() on the (internal) _f future representing the first call, loiter(0). The result method will block until the future is done, therefore each iteration in this loop will have to wait for the next result to be ready.

I encourage you to run [Example 17-6](#) and see the display being updated incrementally. While you're at it, play with the max_workers argument for the ThreadPoolExecutor and with the range function that produces the arguments for the executor.map call—or replace it with lists of handpicked values to create different delays.

[Example 17-7](#) shows a sample run of [Example 17-6](#).

Example 17-7. Sample run of demo_executor_map.py from Example 17-6

```
$ python3 demo_executor_map.py
[15:56:50] Script starting. ❶
[15:56:50] loiter(0): doing nothing for 0s... ❷
[15:56:50] loiter(0): done.
[15:56:50] loiter(1): doing nothing for 1s... ❸
[15:56:50] loiter(2): doing nothing for 2s...
[15:56:50] results: <generator object result_iterator at 0x106517168> ❹
[15:56:50] loiter(3): doing nothing for 3s... ❺
[15:56:50] Waiting for individual results:
[15:56:50] result 0: 0 ❻
[15:56:51] loiter(1): done. ❼
[15:56:51] loiter(4): doing nothing for 4s...
[15:56:51] result 1: 10 ❽
[15:56:52] loiter(2): done. ❾
[15:56:52] result 2: 20
[15:56:53] loiter(3): done.
[15:56:53] result 3: 30
[15:56:55] loiter(4): done. ❿
[15:56:55] result 4: 40
```

- ❶ This run started at 15:56:50.
- ❷ The first thread executes `loiter(0)`, so it will sleep for 0s and return even before the second thread has a chance to start, but YMMV.⁵
- ❸ `loiter(1)` and `loiter(2)` start immediately (because the thread pool has three workers, it can run three functions concurrently).
- ❹ This shows that the results returned by `executor.map` is a generator; nothing so far would block, regardless of the number of tasks and the `max_workers` setting.
- ❺ Because `loiter(0)` is done, the first worker is now available to start the fourth thread for `loiter(3)`.
- ❻ This is where execution may block, depending on the parameters given to the `loiter` calls: the `__next__` method of the results generator must wait until the first future is complete. In this case, it won't block because the call to `loiter(0)` finished before this loop started. Note that everything up to this point happened within the same second: 15:56:50.
- ❼ `loiter(1)` is done one second later, at 15:56:51. The thread is freed to start `loiter(4)`.

5. Your mileage may vary: with threads, you never know the exact sequencing of events that should happen practically at the same time; it's possible that, in another machine, you see `loiter(1)` starting before `loiter(0)` finishes, particularly because `sleep` always releases the GIL so Python may switch to another thread even if you sleep for 0s.

- 8 The result of `loiter(1)` is shown: 10. Now the `for` loop will block waiting for the result of `loiter(2)`.
- 9 The pattern repeats: `loiter(2)` is done, its result is shown; same with `loiter(3)`.
- 10 There is a 2s delay until `loiter(4)` is done, because it started at 15:56:51 and did nothing for 4s.

The `Executor.map` function is easy to use but it has a feature that may or may not be helpful, depending on your needs: it returns the results exactly in the same order as the calls are started: if the first call takes 10s to produce a result, and the others take 1s each, your code will block for 10s as it tries to retrieve the first result of the generator returned by `map`. After that, you'll get the remaining results without blocking because they will be done. That's OK when you must have all the results before proceeding, but often it's preferable to get the results as they are ready, regardless of the order they were submitted. To do that, you need a combination of the `Executor.submit` method and the `futures.as_completed` function, as we saw in [Example 17-4](#). We'll come back to this technique in ["Using `futures.as\_completed`" on page 527](#).



The combination of `executor.submit` and `futures.as_completed` is more flexible than `executor.map` because you can submit different callables and arguments, while `executor.map` is designed to run the same callable on the different arguments. In addition, the set of futures you pass to `futures.as_completed` may come from more than one executor—perhaps some were created by a `ThreadPoolExecutor` instance while others are from a `ProcessPoolExecutor`.

In the next section, we will resume the `flag` download examples with new requirements that will force us to iterate over the results of `futures.as_completed` instead of using `executor.map`.

Downloads with Progress Display and Error Handling

As mentioned, the scripts in ["Example: Web Downloads in Three Styles" on page 505](#) have no error handling to make them easier to read and to contrast the structure of the three approaches: sequential, threaded, and asynchronous.

In order to test the handling of a variety of error conditions, I created the `flags2` examples:

flags2_common.py

This module contains common functions and settings used by all `flags2` examples, including a `main` function, which takes care of command-line parsing, timing, and

reporting results. This is really support code, not directly relevant to the subject of this chapter, so the source code is in [Appendix A, Example A-10](#).

flags2_sequential.py

A sequential HTTP client with proper error handling and progress bar display. Its `download_one` function is also used by `flags2_threadpool.py`.

flags2_threadpool.py

Concurrent HTTP client based on `futures.ThreadPoolExecutor` to demonstrate error handling and integration of the progress bar.

flags2_asyncio.py

Same functionality as previous example but implemented with `asyncio` and `aiohttp`. This will be covered in “Enhancing the `asyncio` downloader Script” on page 554, in [Chapter 18](#).



Be Careful When Testing Concurrent Clients

When testing concurrent HTTP clients on public HTTP servers, you may generate many requests per second, and that’s how denial-of-service (DoS) attacks are made. We don’t want to attack anyone, just learn how to build high-performance clients. Carefully throttle your clients when hitting public servers. For high-concurrency experiments, set up a local HTTP server for testing. Instructions for doing it are in the *README.rst* file in the `17-futures/countries/` directory of the [Fluent Python code repository](#).

The most visible feature of the `flags2` examples is that they have an animated, text-mode progress bar implemented with the [TQDM package](#). I posted a [108s video on YouTube](#) to show the progress bar and contrast the speed of the three `flags2` scripts. In the video, I start with the sequential download, but I interrupt it after 32s because it was going to take more than 5 minutes to hit on 676 URLs and get 194 flags; I then run the threaded and `asyncio` scripts three times each, and every time they complete the job in 6s or less (i.e., more than 60 times faster). [Figure 17-1](#) shows two screenshots: during and after running `flags2_threadpool.py`.

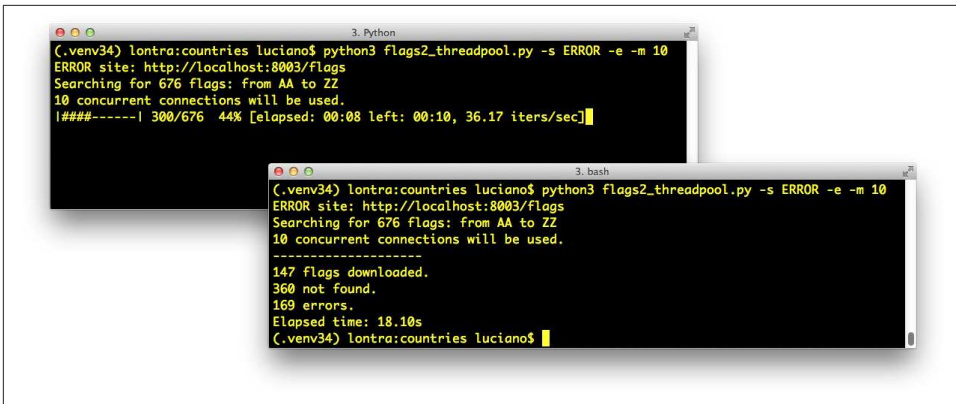


Figure 17-1. Top-left: `flags2_threadpool.py` running with live progress bar generated by `tqdm`; bottom-right: same terminal window after the script is finished.

TQDM is very easy to use, the simplest example appears in an animated `.gif` in the project's [README.md](#). If you type the following code in the Python console after installing the `tqdm` package, you'll see an animated progress bar where the comment is:

```
>>> import time
>>> from tqdm import tqdm
>>> for i in tqdm(range(1000)):
...     time.sleep(.01)
...
>>> # -> progress bar will appear here <-
```

Besides the neat effect, the `tqdm` function is also interesting conceptually: it consumes any iterable and produces an iterator which, while it's consumed, displays the progress bar and estimates the remaining time to complete all iterations. To compute that estimate, `tqdm` needs to get an iterable that has a `len`, or receive as a second argument the expected number of items. Integrating TQDM with our `flags2` examples provide an opportunity to look deeper into how the concurrent scripts actually work, by forcing us to use the `futures.as_completed` and the `asyncio.as_completed` functions so that `tqdm` can display progress as each future is completed.

The other feature of the `flags2` example is a command-line interface. All three scripts accept the same options, and you can see them by running any of the scripts with the `-h` option. [Example 17-8](#) shows the help text.

Example 17-8. Help screen for the scripts in the `flags2` series

```
$ python3 flags2_threadpool.py -h
usage: flags2_threadpool.py [-h] [-a] [-e] [-l N] [-m CONCURRENT] [-s LABEL]
                           [-v]
                           [CC [CC ...]]
```

Download flags for country codes. Default: top 20 countries by population.

positional arguments:

CC country code or 1st letter (eg. B for BA...BZ)

optional arguments:

-h, --help show this help message and exit
-a, --all get all available flags (AD to ZW)
-e, --every get flags for every possible code (AA...ZZ)
-l N, --limit N limit to N first codes
-m CONCURRENT, --max_req CONCURRENT
maximum concurrent requests (default=30)
-s LABEL, --server LABEL
Server to hit; one of DELAY, ERROR, LOCAL, REMOTE
(default=LOCAL)
-v, --verbose output detailed progress info

All arguments are optional. The most important arguments are discussed next.

One option you can't ignore is `-s/--server`: it lets you choose which HTTP server and base URL will be used in the test. You can pass one of four strings to determine where the script will look for the flags (the strings are case insensitive):

LOCAL

Use `http://localhost:8001/flags`; this is the default. You should configure a local HTTP server to answer at port 8001. I used Nginx for my tests. The *README.rst* file for this chapter's example code explains how to install and configure it.

REMOTE

Use `http://flupy.org/data/flags`; that is a public website owned by me, hosted on a shared server. Please do not pound it with too many concurrent requests. The `flupy.org` domain is handled by a free account on the **Cloudflare CDN** so you may notice that the first downloads are slower, but they get faster when the CDN cache warms up.⁶

DELAY

Use `http://localhost:8002/flags`; a proxy delaying HTTP responses should be listening at port 8002. I used a Mozilla Vaurien in front of my local Nginx to introduce delays. The previously mentioned *README.rst* file has instructions for running a Vaurien proxy.

6. Before configuring Cloudflare, I got HTTP 503 errors—Service Temporarily Unavailable—when testing the scripts with a few dozen concurrent requests on my inexpensive shared host account. Now those errors are gone.

ERROR

Use `http://localhost:8003/flags`; a proxy introducing HTTP errors and delaying responses should be installed at port 8003. I used a different Vaurien configuration for this.



The LOCAL option only works if you configure and start a local HTTP server on port 8001. The DELAY and ERROR options require proxies listening on ports 8002 and 8003. Configuring Nginx and Mozilla Vaurien to enable these options is explained in the *17-futures/countries/README.rst* file in the *Fluent Python code repository* on GitHub.

By default, each `flags2` script will fetch the flags of the 20 most populous countries from the LOCAL server (`http://localhost:8001/flags`) using a default number of concurrent connections, which varies from script to script. **Example 17-9** shows a sample run of the `flags2_sequential.py` script using all defaults.

Example 17-9. Running `flags2_sequential.py` with all defaults: LOCAL site, top-20 flags, 1 concurrent connection

```
$ python3 flags2_sequential.py
LOCAL site: http://localhost:8001/flags
Searching for 20 flags: from BD to VN
1 concurrent connection will be used.
-----
20 flags downloaded.
Elapsed time: 0.10s
```

You can select which flags will be downloaded in several ways. **Example 17-10** shows how to download all flags with country codes starting with the letters A, B, or C.

Example 17-10. Run `flags2_threadpool.py` to fetch all flags with country codes prefixes A, B, or C from DELAY server

```
$ python3 flags2_threadpool.py -s DELAY a b c
DELAY site: http://localhost:8002/flags
Searching for 78 flags: from AA to CZ
30 concurrent connections will be used.
-----
43 flags downloaded.
35 not found.
Elapsed time: 1.72s
```

Regardless of how the country codes are selected, the number of flags to fetch can be limited with the `-l/--limit` option. **Example 17-11** demonstrates how to run exactly 100 requests, combining the `-a` option to get all flags with `-l 100`.

Example 17-11. Run `flags2_asyncio.py` to get 100 flags (-al 100) from the ERROR server, using 100 concurrent requests (-m 100)

```
$ python3 flags2_asyncio.py -s ERROR -al 100 -m 100
ERROR site: http://localhost:8003/flags
Searching for 100 flags: from AD to LK
100 concurrent connections will be used.
-----
73 flags downloaded.
27 errors.
Elapsed time: 0.64s
```

That's the user interface of the flags2 examples. Let's see how they are implemented.

Error Handling in the flags2 Examples

The common strategy adopted in all three examples to deal with HTTP errors is that 404 errors (Not Found) are handled by the function in charge of downloading a single file (`download_one`). Any other exception propagates to be handled by the `download_many` function.

Again, we'll start by studying the sequential code, which is easier to follow—and mostly reused by the thread pool script. **Example 17-12** shows the functions that perform the actual downloads in the `flags2_sequential.py` and `flags2_threadpool.py` scripts.

Example 17-12. `flags2_sequential.py`: basic functions in charge of downloading; both are reused in `flags2_threadpool.py`

```
def get_flag(base_url, cc):
    url = '{}/{cc}/{cc}.gif'.format(base_url, cc=cc.lower())
    resp = requests.get(url)
    if resp.status_code != 200: ❶
        resp.raise_for_status()
    return resp.content

def download_one(cc, base_url, verbose=False):
    try:
        image = get_flag(base_url, cc)
    except requests.exceptions.HTTPError as exc: ❷
        res = exc.response
        if res.status_code == 404:
            status = HTTPStatus.not_found ❸
            msg = 'not found'
        else: ❹
            raise
    else:
        save_flag(image, cc.lower() + '.gif')
        status = HTTPStatus.ok
        msg = 'OK'
```

```

if verbose: ❸
    print(cc, msg)

return Result(status, cc) ❹

```

- ❶ `get_flag` does no error handling, it uses `requests.Response.raise_for_status` to raise an exception for any HTTP code other than 200.
- ❷ `download_one` catches `requests.exceptions.HTTPError` to handle HTTP code 404 specifically...
- ❸ ...by setting its local `status` to `HTTPStatus.not_found`; `HTTPStatus` is an Enum imported from `flags2_common` (Example A-10).
- ❹ Any other `HTTPError` exception is re-raised; other exceptions will just propagate to the caller.
- ❺ If the `-v/--verbose` command-line option is set, the country code and status message will be displayed; this how you'll see progress in the verbose mode.
- ❻ The `Result` namedtuple returned by `download_one` will have a `status` field with a value of `HTTPStatus.not_found` or `HTTPStatus.ok`.

Example 17-13 lists the sequential version of the `download_many` function. This code is straightforward, but its worth studying to contrast with the concurrent versions coming up. Focus on how it reports progress, handles errors, and tallies downloads.

Example 17-13. `flags2_sequential.py`: the sequential implementation of `download_many`

```

def download_many(cc_list, base_url, verbose, max_req):
    counter = collections.Counter() ❶
    cc_iter = sorted(cc_list) ❷
    if not verbose:
        cc_iter = tqdm.tqdm(cc_iter) ❸
    for cc in cc_iter: ❹
        try:
            res = download_one(cc, base_url, verbose) ❺
        except requests.exceptions.HTTPError as exc: ❻
            error_msg = 'HTTP error {res.status_code} - {res.reason}'
            error_msg = error_msg.format(res=exc.response)
        except requests.exceptions.ConnectionError as exc: ❼
            error_msg = 'Connection error'
        else: ❽
            error_msg = ''
            status = res.status

        if error_msg:
            status = HTTPStatus.error ❾
        counter[status] += 1 ❿
    if verbose and error_msg: ⓫

```



```

        print('*** Error for {}: {}'.format(cc, error_msg))

    return counter 12

```

- ❶ This Counter will tally the different download outcomes: HTTPStatus.ok, HTTPStatus.not_found, or HTTPStatus.error.
- ❷ cc_iter holds the list of the country codes received as arguments, ordered alphabetically.
- ❸ If not running in verbose mode, cc_iter is passed to the tqdm function, which will return an iterator that yields the items in cc_iter while also displaying the animated progress bar.
- ❹ This for loop iterates over cc_iter and...
- ❺ ...performs the download by successive calls to download_one.
- ❻ HTTP-related exceptions raised by get_flag and not handled by download_one are handled here.
- ❼ Other network-related exceptions are handled here. Any other exception will abort the script, because the flags2_common.main function that calls download_many has no try/except.
- ❽ If no exception escaped download_one, then the status is retrieved from the HTTPStatus namedtuple returned by download_one.
- ❾ If there was an error, set the local status accordingly.
- ❿ Increment the counter by using the value of the HTTPStatus Enum as key.
- ⓫ If running in verbose mode, display the error message for the current country code, if any.
- ⓬ Return the counter so that the main function can display the numbers in its final report.

We'll now study the refactored thread pool example, *flags2_threadpool.py*.

Using futures.as_completed

In order to integrate the TQDM progress bar and handle errors on each request, the *flags2_threadpool.py* script uses `futures.ThreadPoolExecutor` with the `futures.as_completed` function we've already seen. [Example 17-14](#) is the full listing of *flags2_threadpool.py*. Only the `download_many` function is implemented; the other functions are reused from the `flags2_common` and `flags2_sequential` modules.

Example 17-14. flags2_threadpool.py: full listing

```

import collections
from concurrent import futures

```

```

import requests
import tqdm ❶

from flags2_common import main, HTTPStatus ❷
from flags2_sequential import download_one ❸

DEFAULT_CONCUR_REQ = 30 ❹
MAX_CONCUR_REQ = 1000 ❺


def download_many(cc_list, base_url, verbose, concur_req):
    counter = collections.Counter()
    with futures.ThreadPoolExecutor(max_workers=concur_req) as executor: ❻
        to_do_map = {} ❼
        for cc in sorted(cc_list): ❽
            future = executor.submit(download_one,
                                     cc, base_url, verbose) ❾
            to_do_map[future] = cc ❿
        done_iter = futures.as_completed(to_do_map) ⓫
        if not verbose:
            done_iter = tqdm.tqdm(done_iter, total=len(cc_list)) ⓬
        for future in done_iter: ⓭
            try:
                res = future.result() ⓮
            except requests.exceptions.HTTPError as exc: ⓯
                error_msg = 'HTTP {res.status_code} - {res.reason}'
                error_msg = error_msg.format(res=exc.response)
            except requests.exceptions.ConnectionError as exc:
                error_msg = 'Connection error'
            else:
                error_msg = ''
                status = res.status

            if error_msg:
                status = HTTPStatus.error
            counter[status] += 1
            if verbose and error_msg:
                cc = to_do_map[future] ⓰
                print('*** Error for {}: {}'.format(cc, error_msg))

    return counter


if __name__ == '__main__':
    main(download_many, DEFAULT_CONCUR_REQ, MAX_CONCUR_REQ)

```

- ❶ Import the progress-bar display library.
- ❷ Import one function and one Enum from the flags2_common module.
- ❸ Reuse the download_one from flags2_sequential (Example 17-12).

- ④ If the `-m/--max_req` command-line option is not given, this will be the maximum number of concurrent requests, implemented as the size of the thread pool; the actual number may be smaller, if the number of flags to download is smaller.
- ⑤ `MAX_CONCUR_REQ` caps the maximum number of concurrent requests regardless of the number of flags to download or the `-m/--max_req` command-line option; it's a safety precaution.
- ⑥ Create the executor with `max_workers` set to `concur_req`, computed by the `main` function as the smaller of: `MAX_CONCUR_REQ`, the length of `cc_list`, and the value of the `-m/--max_req` command-line option. This avoids creating more threads than necessary.
- ⑦ This `dict` will map each `Future` instance—representing one download—with the respective country code for error reporting.
- ⑧ Iterate over the list of country codes in alphabetical order. The order of the results will depend on the timing of the HTTP responses more than anything, but if the size of the thread pool (given by `concur_req`) is much smaller than `len(cc_list)`, you may notice the downloads batched alphabetically.
- ⑨ Each call to `executor.submit` schedules the execution of one callable and returns a `Future` instance. The first argument is the callable, the rest are the arguments it will receive.
- ⑩ Store the future and the country code in the `dict`.
- ⑪ `futures.as_completed` returns an iterator that yields futures as they are done.
- ⑫ If not in verbose mode, wrap the result of `as_completed` with the `tqdm` function to display the progress bar; because `done_iter` has no `len`, we must tell `tqdm` what is the expected number of items as the `total=` argument, so `tqdm` can estimate the work remaining.
- ⑬ Iterate over the futures as they are completed.
- ⑭ Calling the `result` method on a future either returns the value returned by the callable, or raises whatever exception was caught when the callable was executed. This method may block waiting for a resolution, but not in this example because `as_completed` only returns futures that are done.
- ⑮ Handle the potential exceptions; the rest of this function is identical to the sequential version of `download_many` (Example 17-13), except for the next callout.
- ⑯ To provide context for the error message, retrieve the country code from the `to_do_map` using the current future as key. This was not necessary in the sequential version because we were iterating over the list of country codes, so we had the current `cc`; here we are iterating over the futures.

Example 17-14 uses an idiom that's very useful with `futures.as_completed`: building a dict to map each future to other data that may be useful when the future is completed. Here the `to_do_map` maps each future to the country code assigned to it. This makes it easy to do follow-up processing with the result of the futures, despite the fact that they are produced out of order.

Python threads are well suited for I/O-intensive applications, and the `concurrent.futures` package makes them trivially simple to use for certain use cases. This concludes our basic introduction to `concurrent.futures`. Let's now discuss alternatives for when `ThreadPoolExecutor` or `ProcessPoolExecutor` are not suitable.

Threading and Multiprocessing Alternatives

Python has supported threads since its release 0.9.8 (1993); `concurrent.futures` is just the latest way of using them. In Python 3, the original `thread` module was deprecated in favor of the higher-level **threading module**.⁷ If `futures.ThreadPoolExecutor` is not flexible enough for a certain job, you may need to build your own solution out of basic threading components such as `Thread`, `Lock`, `Semaphore`, etc.—possibly using the thread-safe queues of the **queue module** for passing data between threads. Those moving parts are encapsulated by `futures.ThreadPoolExecutor`.

For CPU-bound work, you need to sidestep the GIL by launching multiple processes. The `futures.ProcessPoolExecutor` is the easiest way to do it. But again, if your use case is complex, you'll need more advanced tools. The **multiprocessing package** emulates the threading API but delegates jobs to multiple processes. For simple programs, `multiprocessing` can replace `threading` with few changes. But `multiprocessing` also offers facilities to solve the biggest challenge faced by collaborating processes: how to pass around data.

Chapter Summary

We started the chapter by comparing two concurrent HTTP clients with a sequential one, demonstrating significant performance gains over the sequential script.

After studying the first example based on `concurrent.futures`, we took a closer look at future objects, either instances of `concurrent.futures.Future`, or `asyncio.Future`, emphasizing what these classes have in common (their differences will be emphasized in **Chapter 18**). We saw how to create futures by calling `Executor.submit`

7. The `threading` module has been available since Python 1.5.1 (1998), yet some insist on using the old `thread` module. In Python 3, it was renamed to `_thread` to highlight the fact that it's just a low-level implementation detail, and shouldn't be used in application code.

`mit(...)`, and iterate over completed futures with `concurrent.futures.as_completed(...)`.

Next, we saw why Python threads are well suited for I/O-bound applications, despite the GIL: every standard library I/O function written in C releases the GIL, so while a given thread is waiting for I/O, the Python scheduler can switch to another thread. We then discussed the use of multiple processes with the `concurrent.futures.ProcessPoolExecutor` class, to go around the GIL and use multiple CPU cores to run cryptographic algorithms, achieving speedups of more than 100% when using four workers.

In the following section, we took a close look at how the `concurrent.futures.ThreadPoolExecutor` works, with a didactic example launching tasks that did nothing for a few seconds, except displaying their status with a timestamp.

Next we went back to the flag downloading examples. Enhancing them with a progress bar and proper error handling prompted further exploration of the `future.as_completed` generator function showing a common pattern: storing futures in a `dict` to link further information to them when submitting, so that we can use that information when the future comes out of the `as_completed` iterator.

We concluded the coverage of concurrency with threads and processes with a brief reminder of the lower-level, but more flexible `threading` and `multiprocessing` modules, which represent the traditional way of leveraging threads and processes in Python.

Further Reading

The `concurrent.futures` package was contributed by Brian Quinlan, who presented it in a great talk titled “[The Future Is Soon!](#)” at PyCon Australia 2010. Quinlan’s talk has no slides; he shows what the library does by typing code directly in the Python console. As a motivating example, the presentation features a short video with XKCD cartoonist/programmer Randall Munroe making an unintended DOS attack on Google Maps to build a colored map of driving times around his city. The formal introduction to the library is [PEP 3148 - futures - execute computations asynchronously](#). In the PEP, Quinlan wrote that the `concurrent.futures` library was “heavily influenced by the `java.util.concurrent` package.”

Parallel Programming with Python (Packt), by Jan Palach, covers several tools for concurrent programming, including the `concurrent.futures`, `threading`, and `multiprocessing` modules. It goes beyond the standard library to discuss [Celery](#), a task queue used to distribute work across threads and processes, even on different machines. In the Django community, Celery is probably the most widely used system to offload heavy tasks such as PDF generation to other processes, thus avoiding delays in producing an HTTP response.

In the Beazley and Jones *Python Cookbook, 3E* (O'Reilly) there are recipes using `current.futures` starting with “Recipe 11.12. Understanding Event-Driven I/O.” “Recipe 12.7. Creating a Thread Pool” shows a simple TCP echo server, and “Recipe 12.8. Performing Simple Parallel Programming” offers a very practical example: analyzing a whole directory of gzip compressed Apache logfiles with the help of a `ProcessPoolExecutor`. For more about threads, the entire Chapter 12 of Beazley and Jones is great, with special mention to “Recipe 12.10. Defining an Actor Task,” which demonstrates the Actor model: a proven way of coordinating threads through message passing.

Brett Slatkin's *Effective Python* (Addison-Wesley) has a multitopic chapter about concurrency, including coverage of coroutines, `concurrent.futures` with threads and processes, and the use of locks and queues for thread programming without the `ThreadPoolExecutor`.

High Performance Python (O'Reilly) by Micha Gorelick and Ian Ozsvald and *The Python Standard Library by Example* (Addison-Wesley), by Doug Hellmann, also cover threads and processes.

For a modern take on concurrency without threads or callbacks, *Seven Concurrency Models in Seven Weeks*, by Paul Butcher (Pragmatic Bookshelf) is an excellent read. I love its subtitle: “When Threads Unravel.” In that book, threads and locks are covered in Chapter 1, and the remaining six chapters are devoted to modern alternatives to concurrent programming, as supported by different languages. Python, Ruby, and JavaScript are not among them.

If you are intrigued about the GIL, start with the *Python Library and Extension FAQ* (“Can't we get rid of the Global Interpreter Lock?”). Also worth reading are posts by Guido van Rossum and Jesse Noller (contributor of the `multiprocessing` package): “It isn't Easy to Remove the GIL” and “Python Threads and the Global Interpreter Lock.” Finally, David Beazley has a detailed exploration on the inner workings of the GIL: “Understanding the Python GIL.”⁸ In slide #54 of the [presentation](#), Beazley reports some alarming results, including a 20× increase in processing time for a particular benchmark with the new GIL algorithm introduced in Python 3.2. However, Beazley apparently used an empty `while True: pass` to simulate CPU-bound work, and that is not realistic. The issue is not significant with real workloads, according to [a comment](#) by Antoine Pitrou—who implemented the new GIL algorithm—in the bug report submitted by Beazley.

While the GIL is real problem and is not likely to go away soon, Jesse Noller and Richard Oudkerk contributed a library to make it easier to work around it in CPU-bound applications: the `multiprocessing` package, which emulates the `threading` API across processes, along with supporting infrastructure of locks, queues, pipes, shared memory,

8. Thanks to Lucas Bruniati for sending me a link to this talk.

etc. The package was introduced in [PEP 371 — Addition of the multiprocessing package to the standard library](#). The [official documentation for the package](#) is a 93 KB *.rst* file—that’s about 63 pages—making it one of the longest chapters in the Python standard library. Multiprocessing is the basis for the `concurrent.futures.ProcessPoolExecutor`.

For CPU- and data-intensive parallel processing, a new option with a lot of momentum in the big data community is the [Apache Spark](#) distributed computing engine, offering a friendly Python API and support for Python objects as data, as shown in their [examples page](#).

Two elegant and super easy libraries for parallelizing tasks over processes are [lelo](#) by João S. O. Bueno and [python-parallelize](#) by Nat Pryce. The `lelo` package defines a `@parallel` decorator that you can apply to any function to magically make it unblocking: when you call the decorated function, its execution is started in another process. Nat Pryce’s `python-parallelize` package provides a `parallelize` generator that you can use to distribute the execution of a `for` loop over multiple CPUs. Both packages use the `multiprocessing` module under the covers.

Soapbox

Thread Avoidance

Concurrency: one of the most difficult topics in computer science (usually best avoided).⁹

— David Beazley

Python coach and mad scientist

I agree with the apparently contradictory quotes by David Beazley, above, and Michele Simionato at the start of this chapter. After attending a concurrency course at the university—in which “concurrent programming” was equated to managing threads and locks—I came to the conclusion that I don’t want to manage threads and locks myself, any more than I want to manage memory allocation and deallocation. Those jobs are best carried out by the systems programmers who have the know-how, the inclination, and the time to get them right—hopefully.

That’s why I think the `concurrent.futures` package is exciting: it treats threads, processes, and queues as infrastructure at your service, not something you have to deal with directly. Of course, it’s designed with simple jobs in mind, the so-called “[embarrassingly parallel](#)” problems. But that’s a large slice of the concurrency problems we face when writing applications—as opposed to operating systems or database servers, as Simionato points out in that quote.

9. Slide #9 from “[A Curious Course on Coroutines and Concurrency](#),” tutorial presented at PyCon 2009.

For “nonembarrassing” concurrency problems, threads and locks are not the answer either. Threads will never disappear at the OS level, but every programming language I’ve found exciting in the last several years provides better, higher-level, concurrency abstractions, as the *Seven Concurrency Models* book demonstrates. Go, Elixir, and Clojure are among them. Erlang—the implementation language of Elixir—is a prime example of a language designed from the ground up with concurrency in mind. It doesn’t excite me for a simple reason: I find its syntax ugly. Python spoiled me that way.

José Valim, well-known as a Ruby on Rails core contributor, designed Elixir with a pleasant, modern syntax. Like Lisp and Clojure, Elixir implements syntactic macros. That’s a double-edged sword. Syntactic macros enable powerful DSLs, but the proliferation of sublanguages can lead to incompatible codebases and community fragmentation. Lisp drowned in a flood of macros, with each Lisp shop using its own arcane dialect. Standardizing around Common Lisp resulted in a bloated language. I hope José Valim can inspire the Elixir community to avoid a similar outcome.

Like Elixir, Go is a modern language with fresh ideas. But, in some regards, it’s a conservative language, compared to Elixir. Go doesn’t have macros, and its syntax is simpler than Python’s. Go doesn’t support inheritance or operator overloading, and it offers fewer opportunities for metaprogramming than Python. These limitations are considered features. They lead to more predictable behavior and performance. That’s a big plus in the highly concurrent, mission-critical settings where Go aims to replace C++, Java, and Python.

While Elixir and Go are direct competitors in the high-concurrency space, their design philosophies appeal to different crowds. Both are likely to thrive. But in the history of programming languages, the conservative ones tend to attract more coders. I’d like to become fluent in Go and Elixir.

About the GIL

The GIL simplifies the implementation of the CPython interpreter and of extensions written in C, so we can thank the GIL for the vast number of extensions in C available for Python—and that is certainly one of the key reasons why Python is so popular today.

For many years, I was under the impression that the GIL made Python threads nearly useless beyond toy applications. It was not until I discovered that *every* blocking I/O call in the standard library releases the GIL that I realized Python threads are excellent for I/O-bound systems—the kind of applications customers usually pay me to develop, given my professional experience.

Concurrency in the Competition

MRI—the reference implementation of Ruby—also has a GIL, so its threads are under the same limitations as Python’s. Meanwhile, JavaScript interpreters don’t support user-level threads at all; asynchronous programming with callbacks is their only path to concurrency. I mention this because Ruby and JavaScript are the closest direct competitors to Python as general-purpose, dynamic programming languages.

Looking at the concurrency-savvy new crop of languages, Go and Elixir are probably the ones best positioned to eat Python’s lunch. But now we have `asyncio`. If hordes of people believe Node.js with raw callbacks is a viable platform for concurrent programming, how hard can it be to win them over to Python when the `asyncio` ecosystem matures? But that’s a topic for the next “Soapbox” on page 580.

FP Ch 18 | Concurrency with asyncio [= plan Ch 21]

Concurrency with `asyncio`

Concurrency is about dealing with lots of things at once.

Parallelism is about doing lots of things at once.

Not the same, but related.

One is about structure, one is about execution.

Concurrency provides a way to structure a solution to solve a problem that may (but not necessarily) be parallelizable.¹

— Rob Pike

Co-inventor of the Go language

Professor Imre Simon² liked to say there are two major sins in science: using different words to mean the same thing and using one word to mean different things. If you do any research on concurrent or parallel programming you will find different definitions for “concurrency” and “parallelism.” I will adopt the informal definitions by Rob Pike, quoted above.

For real parallelism, you must have multiple cores. A modern laptop has four CPU cores but is routinely running more than 100 processes at any given time under normal, casual use. So, in practice, most processing happens concurrently and not in parallel. The computer is constantly dealing with 100+ processes, making sure each has an opportunity to make progress, even if the CPU itself can’t do more than four things at once. Ten years ago we used machines that were also able to handle 100 processes concurrently, but on a single core. That’s why Rob Pike titled that talk “Concurrency Is Not Parallelism (It’s Better).”

1. Slide 5 of the talk “[Concurrency Is Not Parallelism \(It’s Better\)](#)”.
2. Imre Simon (1943–2009) was a pioneer of computer science in Brazil who made seminal contributions to Automata Theory and started the field of Tropical Mathematics. He was also an advocate of free software and free culture. I was fortunate to study, work, and hang out with him.

This chapter introduces `asyncio`, a package that implements concurrency with coroutines driven by an event loop. It's one of the largest and most ambitious libraries ever added to Python. Guido van Rossum developed `asyncio` outside of the Python repository and gave the project a code name of “Tulip”—so you'll see references to that flower when researching this topic online. For example, the main discussion group is still called [python-tulip](#).

Tulip was renamed to `asyncio` when it was added to the standard library in Python 3.4. It's also compatible with Python 3.3—you can find it on PyPI under the new [official name](#). Because it uses `yield from` expressions extensively, `asyncio` is incompatible with older versions of Python.



The [Trollius project](#)—also named after a flower—is a backport of `asyncio` to Python 2.6 and newer, replacing `yield from` with `yield` and clever callables named `From` and `Return`. A `yield from ...` expression becomes `yield From(...)`; and when a coroutine needs to return a result, you write `raise Return(result)` instead of `return result`. Trollius is led by Victor Stinner, who is also an `asyncio` core developer, and who kindly agreed to review this chapter as this book was going into production.

In this chapter we'll see:

- A comparison between a simple threaded program and the `asyncio` equivalent, showing the relationship between threads and asynchronous tasks
- How the `asyncio.Future` class differs from `concurrent.futures.Future`
- Asynchronous versions of the flag download examples from [Chapter 17](#)
- How asynchronous programming manages high concurrency in network applications, without using threads or processes
- How coroutines are a major improvement over callbacks for asynchronous programming
- How to avoid blocking the event loop by offloading blocking operations to a thread pool
- Writing `asyncio` servers, and how to rethink web applications for high concurrency
- Why `asyncio` is poised to have a big impact in the Python ecosystem

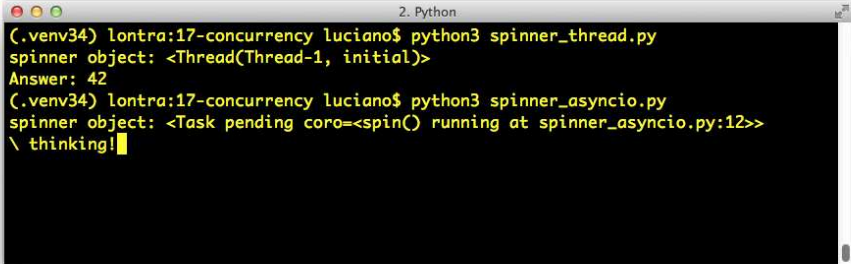
Let's get started with the simple example contrasting threading and `asyncio`.

Thread Versus Coroutine: A Comparison

During a discussion about threads and the GIL, Michele Simionato [posted a simple but fun example](#) using multiprocessing to display an animated spinner made with the ASCII characters "|/-\" on the console while some long computation is running.

I adapted Simionato's example to use a thread with the Threading module and then a coroutine with `asyncio`, so you can see the two examples side by side and understand how to code concurrent behavior without threads.

The output shown in Examples 18-1 and 18-2 is animated, so you really should run the scripts to see what happens. If you're in the subway (or somewhere else without a WiFi connection), take a look at [Figure 18-1](#) and imagine the \ bar before the word "thinking" is spinning.



```
2. Python
(.venv34) lontra:17-concurrency luciano$ python3 spinner_thread.py
spinner object: <Thread(Thread-1, initial)>
Answer: 42
(.venv34) lontra:17-concurrency luciano$ python3 spinner_asyncio.py
spinner object: <Task pending coro=<spin() running at spinner_asyncio.py:12>
\ thinking!
```

Figure 18-1. The scripts `spinner_thread.py` and `spinner_asyncio.py` produce similar output: the repr of a spinner object and the text `Answer: 42`. In the screenshot, `spinner_asyncio.py` is still running, and the spinner message `\ thinking!` is shown; when the script ends, that line will be replaced by the `Answer: 42`.

Let's review the `spinner_thread.py` script first ([Example 18-1](#)).

Example 18-1. `spinner_thread.py`: animating a text spinner with a thread

```
import threading
import itertools
import time
import sys
```

```
class Signal: ❶
    go = True
```

```
def spin(msg, signal): ❷
```

```

write, flush = sys.stdout.write, sys.stdout.flush
for char in itertools.cycle('|/-\\'): ❸
    status = char + ' ' + msg
    write(status)
    flush()
    write('\x08' * len(status)) ❹
    time.sleep(.1)
    if not signal.go: ❺
        break
write(' ' * len(status) + '\x08' * len(status)) ❻

def slow_function(): ❼
    # pretend waiting a long time for I/O
    time.sleep(3) ❽
    return 42

def supervisor(): ❾
    signal = Signal()
    spinner = threading.Thread(target=spin,
                               args=('thinking!', signal))
    print('spinner object:', spinner) ❿
    spinner.start() ⓫
    result = slow_function() ⓬
    signal.go = False ⓭
    spinner.join() ⓮
    return result

def main():
    result = supervisor() ⓯
    print('Answer:', result)

if __name__ == '__main__':
    main()

```

- ❶ This class defines a simple mutable object with a `go` attribute we'll use to control the thread from outside.
- ❷ This function will run in a separate thread. The `signal` argument is an instance of the `Signal` class just defined.
- ❸ This is actually an infinite loop because `itertools.cycle` produces items cycling from the given sequence forever.
- ❹ The trick to do text-mode animation: move the cursor back with backspace characters (`\x08`).
- ❺ If the `go` attribute is no longer `True`, exit the loop.

- ⑥ Clear the status line by overwriting with spaces and moving the cursor back to the beginning.
- ⑦ Imagine this is some costly computation.
- ⑧ Calling `sleep` will block the main thread, but crucially, the GIL will be released so the secondary thread will proceed.
- ⑨ This function sets up the secondary thread, displays the thread object, runs the slow computation, and kills the thread.
- ⑩ Display the secondary thread object. The output looks like `<Thread(Thread-1, initial)>`.
- ⑪ Start the secondary thread.
- ⑫ Run `slow_function`; this blocks the main thread. Meanwhile, the spinner is animated by the secondary thread.
- ⑬ Change the state of the `signal`; this will terminate the `for` loop inside the `spin` function.
- ⑭ Wait until the spinner thread finishes.
- ⑮ Run the supervisor function.

Note that, by design, there is no API for terminating a thread in Python. You must send it a message to shut down. Here I used the `signal.go` attribute: when the main thread sets it to false, the spinner thread will eventually notice and exit cleanly.

Now let's see how the same behavior can be achieved with an `@asyncio.coroutine` instead of a thread.



As noted in the “[Chapter Summary](#)” on page 498 ([Chapter 16](#)), `asyncio` uses a stricter definition of “coroutine.” A coroutine suitable for use with the `asyncio` API must use `yield from` and not `yield` in its body. Also, an `asyncio` coroutine should be driven by a caller invoking it through `yield from` or by passing the coroutine to one of the `asyncio` functions such as `asyncio.async(...)` and others covered in this chapter. Finally, the `@asyncio.coroutine` decorator should be applied to coroutines, as shown in the examples.

Take a look at [Example 18-2](#).

Example 18-2. `spinner_asyncio.py`: animating a text spinner with a coroutine

```
import asyncio
import itertools
import sys
```



```

@asyncio.coroutine ❶
def spin(msg): ❷
    write, flush = sys.stdout.write, sys.stdout.flush
    for char in itertools.cycle('|/-\\'):
        status = char + ' ' + msg
        write(status)
        flush()
        write('\x08' * len(status))
        try:
            yield from asyncio.sleep(.1) ❸
        except asyncio.CancelledError: ❹
            break
    write(' ' * len(status) + '\x08' * len(status))

@asyncio.coroutine
def slow_function(): ❺
    # pretend waiting a long time for I/O
    yield from asyncio.sleep(3) ❻
    return 42

@asyncio.coroutine
def supervisor(): ❼
    spinner = asyncio.async(spin('thinking!')) ❽
    print('spinner object:', spinner) ❾
    result = yield from slow_function() ❿
    spinner.cancel() ⓫
    return result

def main():
    loop = asyncio.get_event_loop() ⓫
    result = loop.run_until_complete(supervisor()) ⓬
    loop.close()
    print('Answer:', result)

if __name__ == '__main__':
    main()

```

- ❶ Coroutines intended for use with asyncio should be decorated with `@asyncio.coroutine`. This not mandatory, but is highly advisable. See explanation following this listing.
- ❷ Here we don't need the signal argument that was used to shut down the thread in the spin function of [Example 18-1](#).

- ❸ Use `yield from asyncio.sleep(.1)` instead of just `time.sleep(.1)`, to sleep without blocking the event loop.
- ❹ If `asyncio.CancelledError` is raised after `spin` wakes up, it's because cancellation was requested, so exit the loop.
- ❺ `slow_function` is now a coroutine, and uses `yield from` to let the event loop proceed while this coroutine pretends to do I/O by sleeping.
- ❻ The `yield from asyncio.sleep(3)` expression handles the control flow to the main loop, which will resume this coroutine after the sleep delay.
- ❼ `supervisor` is now a coroutine as well, so it can drive `slow_function` with `yield from`.
- ❽ `asyncio.async(...)` schedules the `spin` coroutine to run, wrapping it in a `Task` object, which is returned immediately.
- ❾ Display the `Task` object. The output looks like `<Task pending coro=<spin() running at spinner_asyncio.py:12>>`.
- ❿ Drive the `slow_function()`. When that is done, get the returned value. Meanwhile, the event loop will continue running because `slow_function` ultimately uses `yield from asyncio.sleep(3)` to hand control back to the main loop.
- ⓫ A `Task` object can be cancelled; this raises `asyncio.CancelledError` at the `yield` line where the coroutine is currently suspended. The coroutine may catch the exception and delay or even refuse to cancel.
- ⓬ Get a reference to the event loop.
- ⓭ Drive the `supervisor` coroutine to completion; the return value of the coroutine is the return value of this call.



Never use `time.sleep(...)` in `asyncio` coroutines unless you want to block the main thread, therefore freezing the event loop and probably the whole application as well. If a coroutine needs to spend some time doing nothing, it should `yield from asyncio.sleep(DELAY)`.

The use of the `@asyncio.coroutine` decorator is not mandatory, but highly recommended: it makes the coroutines stand out among regular functions, and helps with debugging by issuing a warning when a coroutine is garbage collected without being yielded from—which means some operation was left unfinished and is likely a bug. This is not a *priming decorator*.

Note that the line count of *spinner_thread.py* and *spinner_asyncio.py* is nearly the same. The supervisor functions are the heart of these examples. Let's compare them in detail. **Example 18-3** lists only the supervisor from the Threading example.

Example 18-3. spinner_thread.py: the threaded supervisor function

```
def supervisor():
    signal = Signal()
    spinner = threading.Thread(target=spin,
                               args=('thinking!', signal))
    print('spinner object:', spinner)
    spinner.start()
    result = slow_function()
    signal.go = False
    spinner.join()
    return result
```

For comparison, **Example 18-4** shows the supervisor coroutine.

Example 18-4. spinner_asyncio.py: the asynchronous supervisor coroutine

```
@asyncio.coroutine
def supervisor():
    spinner = asyncio.async(spin('thinking!'))
    print('spinner object:', spinner)
    result = yield from slow_function()
    spinner.cancel()
    return result
```

Here is a summary of the main differences to note between the two supervisor implementations:

- An `asyncio.Task` is roughly the equivalent of a `threading.Thread`. Victor Stinner, special technical reviewer for this chapter, points out that “a Task is like a green thread in libraries that implement cooperative multitasking, such as `gevent`.”
- A Task drives a coroutine, and a Thread invokes a callable.
- You don't instantiate Task objects yourself, you get them by passing a coroutine to `asyncio.async(...)` or `loop.create_task(...)`.
- When you get a Task object, it is already scheduled to run (e.g., by `asyncio.async`); a Thread instance must be explicitly told to run by calling its `start` method.
- In the threaded supervisor, the `slow_function` is a plain function and is directly invoked by the thread. In the `asyncio` supervisor, `slow_function` is a coroutine driven by `yield from`.
- There's no API to terminate a thread from the outside, because a thread could be interrupted at any point, leaving the system in an invalid state. For tasks, there is

the `Task.cancel()` instance method, which raises `CancelledError` inside the coroutine. The coroutine can deal with this by catching the exception in the `yield` where it's suspended.

- The supervisor coroutine must be executed with `loop.run_until_complete` in the `main` function.

This comparison should help you understand how concurrent jobs are orchestrated with `asyncio`, in contrast to how it's done with the more familiar `Threading` module.

One final point related to threads versus coroutines: if you've done any nontrivial programming with threads, you know how challenging it is to reason about the program because the scheduler can interrupt a thread at any time. You must remember to hold locks to protect the critical sections of your program, to avoid getting interrupted in the middle of a multistep operation—which could leave data in an invalid state.

With coroutines, everything is protected against interruption by default. You must explicitly `yield` to let the rest of the program run. Instead of holding locks to synchronize the operations of multiple threads, you have coroutines that are “synchronized” by definition: only one of them is running at any time. And when you want to give up control, you use `yield` or `yield from` to give control back to the scheduler. That's why it is possible to safely cancel a coroutine: by definition, a coroutine can only be cancelled when it's suspended at a `yield` point, so you can perform cleanup by handling the `CancelledError` exception.

We'll now see how the `asyncio.Future` class differs from the `concurrent.futures.Future` class we saw in [Chapter 17](#).

asyncio.Future: Nonblocking by Design

The `asyncio.Future` and the `concurrent.futures.Future` classes have mostly the same interface, but are implemented differently and are not interchangeable. [PEP-3156 — Asynchronous IO Support Rebooted: the “asyncio” Module](#) has this to say about this unfortunate situation:

In the future (pun intended) we may unify `asyncio.Future` and `concurrent.futures.Future` (e.g., by adding an `__iter__` method to the latter that works with `yield from`).

As mentioned in [“Where Are the Futures?” on page 511](#), futures are created only as the result of scheduling something for execution. In `asyncio`, `BaseEventLoop.create_task(...)` takes a coroutine, schedules it to run, and returns an `asyncio.Task` instance—which is also an instance of `asyncio.Future` because `Task` is a subclass of `Future` designed to wrap a coroutine. This is analogous to how we create `concurrent.futures.Future` instances by invoking `Executor.submit(...)`.

Like its `concurrent.futures.Future` counterpart, the `asyncio.Future` class provides `.done()`, `.add_done_callback(...)`, and `.results()` methods, among others. The first two methods work as described in “Where Are the Futures?” on page 511, but `.result()` is very different.

In `asyncio.Future`, the `.result()` method takes no arguments, so you can’t specify a timeout. Also, if you call `.result()` and the future is not done, it does not block waiting for the result. Instead, an `asyncio.InvalidStateError` is raised.

However, the usual way to get the result of an `asyncio.Future` is to `yield from` it, as we’ll see in [Example 18-8](#).

Using `yield from` with a future automatically takes care of waiting for it to finish, without blocking the event loop—because in `asyncio`, `yield from` is used to give control back to the event loop.

Note that using `yield from` with a future is the coroutine equivalent of the functionality offered by `add_done_callback`: instead of triggering a callback, when the delayed operation is done, the event loop sets the result of the future, and the `yield from` expression produces a return value inside our suspended coroutine, allowing it to resume.

In summary, because `asyncio.Future` is designed to work with `yield from`, these methods are often not needed:

- You don’t need `my_future.add_done_callback(...)` because you can simply put whatever processing you would do after the future is done in the lines that follow `yield from my_future` in your coroutine. That’s the big advantage of having coroutines: functions that can be suspended and resumed.
- You don’t need `my_future.result()` because the value of a `yield from` expression on a future is the result (e.g., `result = yield from my_future`).

Of course, there are situations in which `.done()`, `.add_done_callback(...)`, and `.results()` are useful. But in normal usage, `asyncio` futures are driven by `yield from`, not by calling those methods.

We’ll now consider how `yield from` and the `asyncio` API brings together futures, tasks, and coroutines.

Yielding from Futures, Tasks, and Coroutines

In `asyncio`, there is a close relationship between futures and coroutines because you can get the result of an `asyncio.Future` by yielding from it. This means that `res = yield from foo()` works if `foo` is a coroutine function (therefore it returns a coroutine object when called) or if `foo` is a plain function that returns a `Future` or `Task` instance.

This is one of the reasons why coroutines and futures are interchangeable in many parts of the `asyncio` API.

In order to execute, a coroutine must be scheduled, and then it's wrapped in an `asyncio.Task`. Given a coroutine, there are two main ways of obtaining a `Task`:

```
asyncio.async(coro_or_future, *, loop=None)
```

This function unifies coroutines and futures; the first argument can be either one. If it's a `Future` or `Task`, it's returned unchanged. If it's a coroutine, `async` calls `loop.create_task(...)` on it to create a `Task`. An optional event loop may be passed as the `loop=` keyword argument; if omitted, `async` gets the `loop` object by calling `asyncio.get_event_loop()`.

```
BaseEventLoop.create_task(coro)
```

This method schedules the coroutine for execution and returns an `asyncio.Task` object. If called on a custom subclass of `BaseEventLoop`, the object returned may be an instance of some other `Task`-compatible class provided by an external library (e.g., `Tornado`).



`BaseEventLoop.create_task(...)` is only available in Python 3.4.2 or later. If you're using an older version of Python 3.3 or 3.4, you need to use `asyncio.async(...)`, or install a more recent version of `asyncio` from [PyPI](#).

Several `asyncio` functions accept coroutines and wrap them in `asyncio.Task` objects automatically, using `asyncio.async` internally. One example is `BaseEventLoop.run_until_complete(...)`.

If you want to experiment with futures and coroutines on the Python console or in small tests, you can use the following snippet:³

```
>>> import asyncio
>>> def run_sync(coro_or_future):
...     loop = asyncio.get_event_loop()
...     return loop.run_until_complete(coro_or_future)
...
>>> a = run_sync(some_coroutine())
```

The relationship between coroutines, futures, and tasks is documented in section [18.5.3. Tasks and coroutines](#) of the `asyncio` documentation, where you'll find this note:

In this documentation, some methods are documented as coroutines, even if they are plain Python functions returning a `Future`. This is intentional to have a freedom of tweaking the implementation of these functions in the future.

3. Suggested by Petr Viktorin in a [September 11, 2014, message](#) to the Python-ideas list.

Having covered these fundamentals, we'll now study the code for the asynchronous flag download script *flags_asyncio.py* demonstrated along with the sequential and thread pool scripts in [Example 17-1](#) ([Chapter 17](#)).

Downloading with asyncio and aiohttp

As of Python 3.4, `asyncio` only supports TCP and UDP directly. For HTTP or any other protocol, we need third-party packages; `aiohttp` is the one everyone seems to be using for `asyncio` HTTP clients and servers at this time.

[Example 18-5](#) is the full listing for the flag downloading script *flags_asyncio.py*. Here is a high-level view of how it works:

1. We start the process in `download_many` by feeding the event loop with several coroutine objects produced by calling `download_one`.
2. The `asyncio` event loop activates each coroutine in turn.
3. When a client coroutine such as `get_flag` uses `yield from` to delegate to a library coroutine—such as `aiohttp.request`—control goes back to the event loop, which can execute another previously scheduled coroutine.
4. The event loop uses low-level APIs based on callbacks to get notified when a blocking operation is completed.
5. When that happens, the main loop sends a result to the suspended coroutine.
6. The coroutine then advances to the next `yield from`, for example, `yield from resp.read()` in `get_flag`. The event loop takes charge again. Steps 4, 5, and 6 repeat until the event loop is terminated.

This is similar to the example we looked at in “[The Taxi Fleet Simulation](#)” on [page 490](#), where a main loop started several taxi processes in turn. As each taxi process yielded, the main loop scheduled the next event for that taxi (to happen in the future), and proceeded to activate the next taxi in the queue. The taxi simulation is much simpler, and you can easily understand its main loop. But the general flow is the same as in `asyncio`: a single-threaded program where a main loop activates queued coroutines one by one. Each coroutine advances a few steps, then yields control back to the main loop, which then activates the next coroutine in the queue.

Now let's review [Example 18-5](#) play by play.

Example 18-5. flags_asyncio.py: asynchronous download script with asyncio and aiohttp

```
import asyncio
```

```
import aiohttp ❶
```

```

from flags import BASE_URL, save_flag, show, main ❷

@asyncio.coroutine ❸
def get_flag(cc):
    url = '{}/{cc}/{cc}.gif'.format(BASE_URL, cc=cc.lower())
    resp = yield from aiohttp.request('GET', url) ❹
    image = yield from resp.read() ❺
    return image

@asyncio.coroutine
def download_one(cc): ❻
    image = yield from get_flag(cc) ❼
    show(cc)
    save_flag(image, cc.lower() + '.gif')
    return cc

def download_many(cc_list):
    loop = asyncio.get_event_loop() ❽
    to_do = [download_one(cc) for cc in sorted(cc_list)] ❾
    wait_coro = asyncio.wait(to_do) ❿
    res, _ = loop.run_until_complete(wait_coro) ⓫
    loop.close() ⓫

    return len(res)

if __name__ == '__main__':
    main(download_many)

```

- ❶ aiohttp must be installed—it's not in the standard library.
- ❷ Reuse some functions from the flags module (Example 17-2).
- ❸ Coroutines should be decorated with `@asyncio.coroutine`.
- ❹ Blocking operations are implemented as coroutines, and your code delegates to them via `yield from` so they run asynchronously.
- ❺ Reading the response contents is a separate asynchronous operation.
- ❻ `download_one` must also be a coroutine, because it uses `yield from`.
- ❼ The only difference from the sequential implementation of `download_one` are the words `yield from` in this line; the rest of the function body is exactly as before.
- ❽ Get a reference to the underlying event-loop implementation.
- ❾ Build a list of generator objects by calling the `download_one` function once for each flag to be retrieved.

- ❶❩ Despite its name, `wait` is not a blocking function. It's a coroutine that completes when all the coroutines passed to it are done (that's the default behavior of `wait`; see explanation after this example).
- ❶❪ Execute the event loop until `wait_coro` is done; this is where the script will block while the event loop runs. We ignore the second item returned by `run_until_complete`. The reason is explained next.
- ❶❫ Shut down the event loop.



It would be nice if event loop instances were context managers, so we could use a `with` block to make sure the loop is closed. However, the situation is complicated by the fact that client code never creates the event loop directly, but gets a reference to it by calling `asyncio.get_event_loop()`. Sometimes our code does not “own” the event loop, so it would be wrong to close it. For example, when using an external GUI event loop with a package like [Quamash](#), the Qt library is responsible for shutting down the loop when the application quits.

The `asyncio.wait(...)` coroutine accepts an iterable of futures or coroutines; `wait` wraps each coroutine in a `Task`. The end result is that all objects managed by `wait` become instances of `Future`, one way or another. Because it is a coroutine function, calling `wait(...)` returns a coroutine/generator object; this is what the `wait_coro` variable holds. To drive the coroutine, we pass it to `loop.run_until_complete(...)`.

The `loop.run_until_complete` function accepts a future or a coroutine. If it gets a coroutine, `run_until_complete` wraps it into a `Task`, similar to what `wait` does. Coroutines, futures, and tasks can all be driven by `yield from`, and this is what `run_until_complete` does with the `wait_coro` object returned by the `wait` call. When `wait_coro` runs to completion, it returns a 2-tuple where the first item is the set of completed futures, and the second is the set of those not completed. In [Example 18-5](#), the second set will always be empty—that's why we explicitly ignore it by assigning to `_`. But `wait` accepts two keyword-only arguments that may cause it to return even if some of the futures are not complete: `timeout` and `return_when`. See the [asyncio.wait documentation](#) for details.

Note that in [Example 18-5](#) I could not reuse the `get_flag` function from *flags.py* ([Example 17-2](#)) because that uses the `requests` library, which performs blocking I/O. To leverage `asyncio`, we must replace every function that hits the network with an asynchronous version that is invoked with `yield from`, so that control is given back to the event loop. Using `yield from` in `get_flag` means that it must be driven as a coroutine.

That's why I could not reuse the `download_one` function from *flags_threadpool.py* (Example 17-3) either. The code in Example 18-5 drives `get_flag` with `yield from`, so `download_one` is itself also a coroutine. For each request, a `download_one` coroutine object is created in `download_many`, and they are all driven by the `loop.run_until_complete` function, after being wrapped by the `asyncio.wait` coroutine.

There are a lot of new concepts to grasp in `asyncio` but the overall logic of Example 18-5 is easy to follow if you employ a trick suggested by Guido van Rossum himself: squint and pretend the `yield from` keywords are not there. If you do that, you'll notice that the code is as easy to read as plain old sequential code.

For example, imagine that the body of this coroutine...

```
@asyncio.coroutine
def get_flag(cc):
    url = '{}/{cc}/{cc}.gif'.format(BASE_URL, cc=cc.lower())
    resp = yield from aiohttp.request('GET', url)
    image = yield from resp.read()
    return image
```

...works like the following function, except that it never blocks:

```
def get_flag(cc):
    url = '{}/{cc}/{cc}.gif'.format(BASE_URL, cc=cc.lower())
    resp = aiohttp.request('GET', url)
    image = resp.read()
    return image
```

Using the `yield from` foo syntax avoids blocking because the current coroutine is suspended (i.e., the delegating generator where the `yield from` code is), but the control flow goes back to the event loop, which can drive other coroutines. When the foo future or coroutine is done, it returns a result to the suspended coroutine, resuming it.

At the end of the section “Using `yield from`” on page 477, I stated two facts about every usage of `yield from`. Here they are, summarized:

- Every arrangement of coroutines chained with `yield from` must be ultimately driven by a caller that is not a coroutine, which invokes `next(...)` or `.send(...)` on the outermost delegating generator, explicitly or implicitly (e.g., in a `for` loop).
- The innermost subgenerator in the chain must be a simple generator that uses just `yield`—or an iterable object.

When using `yield from` with the `asyncio` API, both facts remain true, with the following specifics:

- The coroutine chains we write are always driven by passing our outermost delegating generator to an `asyncio` API call, such as `loop.run_until_complete(...)`.

In other words, when using `asyncio` our code doesn't drive a coroutine chain by calling `next(...)` or `.send(...)` on it—the `asyncio` event loop does that.

- The coroutine chains we write always end by delegating with `yield from` to some `asyncio` coroutine function or coroutine method (e.g., `yield from asyncio.sleep(...)` in [Example 18-2](#)) or coroutines from libraries that implement higher-level protocols (e.g., `resp = yield from aiohttp.request('GET', url)` in the `get_flag` coroutine of [Example 18-5](#)).

In other words, the innermost subgenerator will be a library function that does the actual I/O, not something we write.

To summarize: as we use `asyncio`, our asynchronous code consists of coroutines that are delegating generators driven by `asyncio` itself and that ultimately delegate to `asyncio` library coroutines—possibly by way of some third-party library such as `aiohttp`. This arrangement creates pipelines where the `asyncio` event loop drives—through our coroutines—the library functions that perform the low-level asynchronous I/O.

We are now ready to answer one question raised in [Chapter 17](#):

- How can `flags_asyncio.py` perform 5× faster than `flags.py` when both are single threaded?

Running Circling Around Blocking Calls

Ryan Dahl, the inventor of Node.js, introduces the philosophy of his project by saying “We’re doing I/O completely wrong.”⁴ He defines a *blocking function* as one that does disk or network I/O, and argues that we can’t treat them as we treat nonblocking functions. To explain why, he presents the numbers in the first two columns of [Table 18-1](#).

Table 18-1. Modern computer latency for reading data from different devices; third column shows proportional times in a scale easier to understand for us slow humans

Device	CPU cycles	Proportional “human” scale
L1 cache	3	3 seconds
L2 cache	14	14 seconds
RAM	250	250 seconds
disk	41,000,000	1.3 years
network	240,000,000	7.6 years

4. Video: [Introduction to Node.js](#) at 4:55.

To make sense of [Table 18-1](#), bear in mind that modern CPUs with GHz clocks run billions of cycles per second. Let's say that a CPU runs exactly 1 billion cycles per second. That CPU can make 333,333,333 L1 cache reads in one second, or 4 (four!) network reads in the same time. The third column of [Table 18-1](#) puts those numbers in perspective by multiplying the second column by a constant factor. So, in an alternate universe, if one read from L1 cache took 3 seconds, then a network read would take 7.6 years!

There are two ways to prevent blocking calls to halt the progress of the entire application:

- Run each blocking operation in a separate thread.
- Turn every blocking operation into a nonblocking asynchronous call.

Threads work fine, but the memory overhead for each OS thread—the kind that Python uses—is on the order of megabytes, depending on the OS. We can't afford one thread per connection if we are handling thousands of connections.

Callbacks are the traditional way to implement asynchronous calls with low memory overhead. They are a low-level concept, similar to the oldest and most primitive concurrency mechanism of all: hardware interrupts. Instead of waiting for a response, we register a function to be called when something happens. In this way, every call we make can be nonblocking. Ryan Dahl advocates callbacks for their simplicity and low overhead.

Of course, we can only make callbacks work because the event loop underlying our asynchronous applications can rely on infrastructure that uses interrupts, threads, polling, background processes, etc. to ensure that multiple concurrent requests make progress and they eventually get done.⁵ When the event loop gets a response, it calls back our code. But the single main thread shared by the event loop and our application code is never blocked—if we don't make mistakes.

When used as coroutines, generators provide an alternative way to do asynchronous programming. From the perspective of the event loop, invoking a callback or calling `.send()` on a suspended coroutine is pretty much the same. There is a memory overhead for each suspended coroutine, but it's orders of magnitude smaller than the overhead for each thread. And they avoid the dreaded “callback hell,” which we'll discuss in [“From Callbacks to Futures and Coroutines” on page 562](#).

Now the five-fold performance advantage of *flags_asyncio.py* over *flags.py* should make sense: *flags.py* spends billions of CPU cycles waiting for each download, one after the other. The CPU is actually doing a lot meanwhile, just not running your program. In contrast, when `loop_until_complete` is called in the `download_many` function of

5. In fact, although Node.js does not support user-level threads written in JavaScript, behind the scenes it implements a thread pool in C with the `libeio` library, to provide its callback-based file APIs—because as of 2014 there are no stable and portable asynchronous file handling APIs for most OSes.

flags_asyncio.py, the event loop drives each `download_one` coroutine to the first `yield` from, and this in turn drives each `get_flag` coroutine to the first `yield` from, calling `aihttp.request(...)`. None of these calls are blocking, so all requests are started in a fraction of a second.

As the `asyncio` infrastructure gets the first response back, the event loop sends it to the waiting `get_flag` coroutine. As `get_flag` gets a response, it advances to the next `yield` from, which calls `resp.read()` and yields control back to the main loop. Other responses arrive in close succession (because they were made almost at the same time). As each `get_flag` returns, the delegating generator `download_flag` resumes and saves the image file.



For maximum performance, the `save_flag` operation should be asynchronous, but `asyncio` does not provide an asynchronous filesystem API at this time—as Node does. If that becomes a bottleneck in your application, you can use the `loop.run_in_executor` function to run `save_flag` in a thread pool. [Example 18-9](#) will show how.

Because the asynchronous operations are interleaved, the total time needed to download many images concurrently is much less than doing it sequentially. When making 600 HTTP requests with `asyncio` I got all results back more than 70 times faster than with a sequential script.

Now let's go back to the HTTP client example to see how we can display an animated progress bar and perform proper error handling.

Enhancing the asyncio downloader Script

Recall from “[Downloads with Progress Display and Error Handling](#)” on page 520 that the `flags2` set of examples share the same command-line interface. This includes the *flags2_asyncio.py* we will analyze in this section. For instance, [Example 18-6](#) shows how to get 100 flags (`-al 100`) from the ERROR server, using 100 concurrent requests (`-m 100`).

Example 18-6. Running flags2_asyncio.py

```
$ python3 flags2_asyncio.py -s ERROR -al 100 -m 100
ERROR site: http://localhost:8003/flags
Searching for 100 flags: from AD to LK
100 concurrent connections will be used.
-----
73 flags downloaded.
27 errors.
Elapsed time: 0.64s
```



Act Responsibly When Testing Concurrent Clients

Even if the overall download time is not different between the threaded and `asyncio` HTTP clients, `asyncio` can send requests faster, so it's even more likely that the server will suspect a DOS attack. To really exercise these concurrent clients at full speed, set up a local HTTP server for testing, as explained in the *RE-ADME.rst* inside the *17-futures/countries/* directory of the *Fluent Python* code repository.

Now let's see how *flags2_asyncio.py* is implemented.

Using `asyncio.as_completed`

In *Example 18-5*, I passed a list of coroutines to `asyncio.wait`, which—when driven by `loop.run_until_complete`—would return the results of the downloads when all were done. But to update a progress bar we need to get results as they are done. Fortunately, there is an `asyncio` equivalent of the `as_completed` generator function we used in the thread pool example with the progress bar (*Example 17-14*).

Writing a *flags2* example to leverage `asyncio` entails rewriting several functions that the `concurrent.futures` version could reuse. That's because there's only one main thread in an `asyncio` program and we can't afford to have blocking calls in that thread, as it's the same thread that runs the event loop. So I had to rewrite `get_flag` to use `yield from` for all network access. Now `get_flag` is a coroutine, so `download_one` must drive it with `yield from`, therefore `download_one` itself becomes a coroutine. Previously, in *Example 18-5*, `download_one` was driven by `download_many`: the calls to `download_one` were wrapped in an `asyncio.wait` call and passed to `loop.run_until_complete`. Now we need finer control for progress reporting and error handling, so I moved most of the logic from `download_many` into a new `downloader_coro` coroutine, and use `download_many` just to set up the event loop and schedule `downloader_coro`.

Example 18-7 shows the top of the *flags2_asyncio.py* script where the `get_flag` and `download_one` coroutines are defined. *Example 18-8* lists the rest of the source, with `downloader_coro` and `download_many`.

Example 18-7. flags2_asyncio.py: Top portion of the script; remaining code is in Example 18-8

```
import asyncio
import collections

import aiohttp
from aiohttp import web
import tqdm
```

```
from flags2_common import main, HTTPStatus, Result, save_flag
```

```
# default set low to avoid errors from remote site, such as  
# 503 - Service Temporarily Unavailable
```

```
DEFAULT_CONCUR_REQ = 5
```

```
MAX_CONCUR_REQ = 1000
```

```
class FetchError(Exception): ❶  
    def __init__(self, country_code):  
        self.country_code = country_code
```

```
@asyncio.coroutine
```

```
def get_flag(base_url, cc): ❷  
    url = '{}/{cc}/{cc}.gif'.format(base_url, cc=cc.lower())  
    resp = yield from aiohttp.request('GET', url)  
    if resp.status == 200:  
        image = yield from resp.read()  
        return image  
    elif resp.status == 404:  
        raise web.HTTPNotFound()  
    else:  
        raise aiohttp.HttpProcessingError(  
            code=resp.status, message=resp.reason,  
            headers=resp.headers)
```

```
@asyncio.coroutine
```

```
def download_one(cc, base_url, semaphore, verbose): ❸  
    try:  
        with (yield from semaphore): ❹  
            image = yield from get_flag(base_url, cc) ❺  
    except web.HTTPNotFound: ❻  
        status = HTTPStatus.not_found  
        msg = 'not found'  
    except Exception as exc: ❼  
        raise FetchError(cc) from exc  
    else:  
        save_flag(image, cc.lower() + '.gif') ❽  
        status = HTTPStatus.ok  
        msg = 'OK'  
  
    if verbose and msg:  
        print(cc, msg)  
  
    return Result(status, cc)
```

- ❶ This custom exception will be used to wrap other HTTP or network exceptions and carry the `country_code` for error reporting.

- ❷ `get_flag` will either return the bytes of the image downloaded, raise `web.HTTPNotFound` if the HTTP response status is 404, or raise an `aiohttp.HttpProcessingError` for other HTTP status codes.
- ❸ The `semaphore` argument is an instance of `asyncio.Semaphore`, a synchronization device that limits the number of concurrent requests.
- ❹ A semaphore is used as a context manager in a `yield from` expression so that the system as whole is not blocked: only this coroutine is blocked while the semaphore counter is at the maximum allowed number.
- ❺ When this `with` statement exits, the semaphore counter is decremented, unblocking some other coroutine instance that may be waiting for the same semaphore object.
- ❻ If the flag was not found, just set the status for the `Result` accordingly.
- ❼ Any other exception will be reported as a `FetchError` with the country code and the original exception chained using the `raise X from Y` syntax introduced in [PEP 3134 — Exception Chaining and Embedded Tracebacks](#).
- ❽ This function call actually saves the flag image to disk.

In [Example 18-7](#), you can see that the code for `get_flag` and `download_one` changed significantly from the sequential version because these functions are now coroutines using `yield from` to make asynchronous calls.

Network client code of the sort we are studying should always use some throttling mechanism to avoid pounding the server with too many concurrent requests—the overall performance of the system may degrade if the server is overloaded. In *flags2_threadpool.py* ([Example 17-14](#)), the throttling was done by instantiating the `ThreadPoolExecutor` with the required `max_workers` argument set to `concur_req` in the `download_many` function, so only `concur_req` threads are started in the pool. In *flags2_asyncio.py*, I used an `asyncio.Semaphore`, which is created by the `download_er_coro` function (shown next, in [Example 18-8](#)) and is passed as the `semaphore` argument to `download_one` in [Example 18-7](#).⁶

A `Semaphore` is an object that holds an internal counter that is decremented whenever we call the `.acquire()` coroutine method on it, and incremented when we call the `.release()` coroutine method. The initial value of the counter is set when the `Semaphore` is instantiated, as in this line of `downloader_coro`:

```
semaphore = asyncio.Semaphore(concur_req)
```

6. Thanks to Guto Maia who noted that `Semaphore` was not explained in the book draft.

Calling `.acquire()` does not block when the counter is greater than zero, but if the counter is zero, `.acquire()` will block the calling coroutine until some other coroutine calls `.release()` on the same Semaphore, thus incrementing the counter. In [Example 18-7](#), I don't call `.acquire()` or `.release()`, but use the semaphore as a context manager in this block of code inside `download_one`:

```
with (yield from semaphore):
    image = yield from get_flag(base_url, cc)
```

That snippet guarantees that no more than `concur_req` instances of `get_flags` coroutines will be started at any time.

Now let's take a look at the rest of the script in [Example 18-8](#). Note that most functionality of the old `download_many` function is now in a coroutine, `downloader_coro`. This was necessary because we must use `yield from` to retrieve the results of the futures yielded by `asyncio.as_completed`, therefore `as_completed` must be invoked in a coroutine. However, I couldn't simply turn `download_many` into a coroutine, because I must pass it to the `main` function from `flags2_common` in the last line of the script, and that `main` function is not expecting a coroutine, just a plain function. Therefore I created `downloader_coro` to run the `as_completed` loop, and now `download_many` simply sets up the event loop and schedules `downloader_coro` by passing it to `loop.run_until_complete`.

Example 18-8. `flags2_asyncio.py`: Script continued from [Example 18-7](#)

```
@asyncio.coroutine
def downloader_coro(cc_list, base_url, verbose, concur_req): ❶
    counter = collections.Counter()
    semaphore = asyncio.Semaphore(concur_req) ❷
    to_do = [download_one(cc, base_url, semaphore, verbose)
             for cc in sorted(cc_list)] ❸

    to_do_iter = asyncio.as_completed(to_do) ❹
    if not verbose:
        to_do_iter = tqdm.tqdm(to_do_iter, total=len(cc_list)) ❺
    for future in to_do_iter: ❻
        try:
            res = yield from future ❼
        except FetchError as exc: ❽
            country_code = exc.country_code ❾
            try:
                error_msg = exc.__cause__.args[0] ❿
            except IndexError:
                error_msg = exc.__cause__.__class__.__name__ ⓫
            if verbose and error_msg:
                msg = '*** Error for {}: {}'
                print(msg.format(country_code, error_msg))
            status = HTTPStatus.error
    else:
```

```

        status = res.status

        counter[status] += 1 ❷

    return counter ❸

def download_many(cc_list, base_url, verbose, concur_req):
    loop = asyncio.get_event_loop()
    coro = downloader_coro(cc_list, base_url, verbose, concur_req)
    counts = loop.run_until_complete(coro) ❹
    loop.close() ❺

    return counts

if __name__ == '__main__':
    main(download_many, DEFAULT_CONCUR_REQ, MAX_CONCUR_REQ)

```

- ❶ The coroutine receives the same arguments as `download_many`, but it cannot be invoked directly from `main` precisely because it's a coroutine function and not a plain function like `download_many`.
- ❷ Create an `asyncio.Semaphore` that will allow up to `concur_req` active coroutines among those using this semaphore.
- ❸ Create a list of coroutine objects, one per call to the `download_one` coroutine.
- ❹ Get an iterator that will return futures as they are done.
- ❺ Wrap the iterator in the `tqdm` function to display progress.
- ❻ Iterate over the completed futures; this loop is very similar to the one in `download_many` in [Example 17-14](#); most changes have to do with exception handling because of differences in the HTTP libraries (`requests` versus `aiohttp`).
- ❼ The easiest way to retrieve the result of an `asyncio.Future` is using `yield from` instead of calling `future.result()`.
- ❽ Every exception in `download_one` is wrapped in a `FetchError` with the original exception chained.
- ❾ Get the country code where the error occurred from the `FetchError` exception.
- ❿ Try to retrieve the error message from the original exception (`__cause__`).
- ⓫ If the error message cannot be found in the original exception, use the name of the chained exception class as the error message.
- ⓬ Tally outcomes.
- ⓭ Return the counter, as done in the other scripts.

- 14 `download_many` simply instantiates the coroutine and passes it to the event loop with `run_until_complete`.
- 15 When all work is done, shut down the event loop and return counts.

In [Example 18-8](#), we could not use the mapping of futures to country codes we saw in [Example 17-14](#) because the futures returned by `asyncio.as_completed` are not necessarily the same futures we pass into the `as_completed` call. Internally, the `asyncio` machinery replaces the future objects we provide with others that will, in the end, produce the same results.⁷

Because I could not use the futures as keys to retrieve the country code from a dict in case of failure, I implemented the custom `FetchError` exception (shown in [Example 18-7](#)). `FetchError` wraps a network exception and holds the country code associated with it, so the country code can be reported with the error in verbose mode. If there is no error, the country code is available as the result of the `yield` from future expression at the top of the `for` loop.

This wraps up the discussion of an `asyncio` example functionally equivalent to the *flags2_threadpool.py* we saw earlier. Next, we'll implement enhancements to *flags2_asyncio.py* that will let us explore `asyncio` further.

While discussing [Example 18-7](#), I noted that `save_flag` performs disk I/O and should be executed asynchronously. The following section shows how.

Using an Executor to Avoid Blocking the Event Loop

In the Python community, we tend to overlook the fact that local filesystem access is blocking, rationalizing that it doesn't suffer from the higher latency of network access (which is also dangerously unpredictable). In contrast, Node.js programmers are constantly reminded that all filesystem functions are blocking because their signatures require a callback. Recall from [Table 18-1](#) that blocking for disk I/O wastes millions of CPU cycles, and this may have a significant impact on the performance of the application.

In [Example 18-7](#), the blocking function is `save_flag`. In the threaded version of the script ([Example 17-14](#)), `save_flag` blocks the thread that's running the `download_one` function, but that's only one of several worker threads. Behind the scenes, the blocking I/O call releases the GIL, so another thread can proceed. But in *flags2_asyncio.py*, `save_flag` blocks the single thread our code shares with the `asyncio` event loop, there-

7. A detailed discussion about this can be found in a thread I started in the python-tulip group, titled “Which other futures my come out of `asyncio.as_completed`?”. Guido responds, and gives insight on the implementation of `as_completed` as well as the close relationship between futures and coroutines in `asyncio`.

fore the whole application freezes while the file is being saved. The solution to this problem is the `run_in_executor` method of the event loop object.

Behind the scenes, the `asyncio` event loop has a thread pool executor, and you can send callables to be executed by it with `run_in_executor`. To use this feature in our example, only a few lines need to change in the `download_one` coroutine, as shown in [Example 18-9](#).

Example 18-9. `flags2_asyncio_executor.py`: Using the default thread pool executor to run `save_flag`

```
@asyncio.coroutine
def download_one(cc, base_url, semaphore, verbose):
    try:
        with (yield from semaphore):
            image = yield from get_flag(base_url, cc)
    except web.HTTPNotFound:
        status = HTTPStatus.not_found
        msg = 'not found'
    except Exception as exc:
        raise FetchError(cc) from exc
    else:
        loop = asyncio.get_event_loop() ❶
        loop.run_in_executor(None, ❷
                             save_flag, image, cc.lower() + '.gif') ❸
        status = HTTPStatus.ok
        msg = 'OK'

    if verbose and msg:
        print(cc, msg)

    return Result(status, cc)
```

- ❶ Get a reference to the event loop object.
- ❷ The first argument to `run_in_executor` is an executor instance; if `None`, the default thread pool executor of the event loop is used.
- ❸ The remaining arguments are the callable and its positional arguments.



When I tested [Example 18-9](#), there was no noticeable change in performance for using `run_in_executor` to save the image files because they are not large (13 KB each, on average). But you'll see an effect if you edit the `save_flag` function in `flags2_com-mon.py` to save 10 times as many bytes on each file—just by coding `fp.write(img*10)` instead of `fp.write(img)`. With an average download size of 130 KB, the advantage of using `run_in_executor` becomes clear. If you're downloading megapixel images, the speedup will be significant.

The advantage of coroutines over callbacks becomes evident when we need to coordinate asynchronous requests, and not just make completely independent requests. The next section explains the problem and the solution.

From Callbacks to Futures and Coroutines

Event-oriented programming with coroutines requires some effort to master, so it's good to be clear on how it improves on the classic callback style. This is the theme of this section.

Anyone with some experience in callback-style event-oriented programming knows the term “callback hell”: the nesting of callbacks when one operation depends on the result of the previous operation. If you have three asynchronous calls that must happen in succession, you need to code callbacks nested three levels deep. [Example 18-10](#) is an example in JavaScript.

*Example 18-10. Callback hell in JavaScript: nested anonymous functions, a.k.a. **Pyramid of Doom***

```
api_call1(request1, function (response1) {  
    // stage 1  
    var request2 = step1(response1);  
  
    api_call2(request2, function (response2) {  
        // stage 2  
        var request3 = step2(response2);  
  
        api_call3(request3, function (response3) {  
            // stage 3  
            step3(response3);  
        });  
    });  
});
```

In [Example 18-10](#), `api_call1`, `api_call2`, and `api_call3` are library functions your code uses to retrieve results asynchronously—perhaps `api_call1` goes to a database and `api_call2` gets data from a web service, for example. Each of these take a callback function, which in JavaScript are often anonymous functions (they are named `stage1`, `stage2`, and `stage3` in the following Python example). The `step1`, `step2`, and `step3` here represent regular functions of your application that process the responses received by the callbacks.

[Example 18-11](#) shows what callback hell looks like in Python.

Example 18-11. Callback hell in Python: chained callbacks

```
def stage1(response1):  
    request2 = step1(response1)  
    api_call2(request2, stage2)
```

```
def stage2(response2):
    request3 = step2(response2)
    api_call3(request3, stage3)
```

```
def stage3(response3):
    step3(response3)
```

```
api_call1(request1, stage1)
```

Although the code in [Example 18-11](#) is arranged very differently from [Example 18-10](#), they do exactly the same thing, and the JavaScript example could be written using the same arrangement (but the Python code can't be written in the JavaScript style because of the syntactic limitations of `lambda`).

Code organized as [Example 18-10](#) or [Example 18-11](#) is hard to read, but it's even harder to write: each function does part of the job, sets up the next callback, and returns, to let the event loop proceed. At this point, all local context is lost. When the next callback (e.g., `stage2`) is executed, you don't have the value of `request2` any more. If you need it, you must rely on closures or external data structures to store it between the different stages of the processing.

That's where coroutines really help. Within a coroutine, to perform three asynchronous actions in succession, you `yield` three times to let the event loop continue running. When a result is ready, the coroutine is activated with a `.send()` call. From the perspective of the event loop, that's similar to invoking a callback. But for the users of a coroutine-style asynchronous API, the situation is vastly improved: the entire sequence of three operations is in one function body, like plain old sequential code with local variables to retain the context of the overall task under way. See [Example 18-12](#).

Example 18-12. Coroutines and `yield` from enable asynchronous programming without callbacks

```
@asyncio.coroutine
def three_stages(request1):
    response1 = yield from api_call1(request1)
    # stage 1
    request2 = step1(response1)
    response2 = yield from api_call2(request2)
    # stage 2
    request3 = step2(response2)
    response3 = yield from api_call3(request3)
    # stage 3
    step3(response3)
```

```
loop.create_task(three_stages(request1)) # must explicitly schedule execution
```

Example 18-12 is much easier to follow the previous JavaScript and Python examples: the three stages of the operation appear one after the other inside the same function. This makes it trivial to use previous results in follow-up processing. It also provides a context for error reporting through exceptions.

Suppose in **Example 18-11** the processing of the call `api_call2(request2, stage2)` raises an I/O exception (that's the last line of the `stage1` function). The exception cannot be caught in `stage1` because `api_call2` is an asynchronous call: it returns immediately, before any I/O is performed. In callback-based APIs, this is solved by registering two callbacks for each asynchronous call: one for handling the result of successful operations, another for handling errors. Work conditions in callback hell quickly deteriorate when error handling is involved.

In contrast, in **Example 18-12**, all the asynchronous calls for this three-stage operation are inside the same function, `three_stages`, and if the asynchronous calls `api_call1`, `api_call2`, and `api_call3` raise exceptions we can handle them by putting the respective `yield from` from lines inside `try/except` blocks.

This is a much better place than callback hell, but I wouldn't call it coroutine heaven because there is a price to pay. Instead of regular functions, you must use coroutines and get used to `yield from`, so that's the first obstacle. Once you write `yield from` in a function, it's now a coroutine and you can't simply call it, like we called `api_call1(request1, stage1)` in **Example 18-11** to start the callback chain. You must explicitly schedule the execution of the coroutine with the event loop, or activate it using `yield from` in another coroutine that is scheduled for execution. Without the call `loop.create_task(three_stages(request1))` in the last line, nothing would happen in **Example 18-12**.

The next example puts this theory into practice.

Doing Multiple Requests for Each Download

Suppose you want to save each country flag with the name of the country and the country code, instead of just the country code. Now you need to make two HTTP requests per flag: one to get the flag image itself, the other to get the `metadata.json` file in the same directory as the image: that's where the name of the country is recorded.

Articulating multiple requests in the same task is easy in the threaded script: just make one request then the other, blocking the thread twice, and keeping both pieces of data (country code and name) in local variables, ready to use when saving the files. If you need to do the same in an asynchronous script with callbacks, you start to smell the sulfur of callback hell: the country code and name will need to be passed around in a

closure or held somewhere until you can save the file because each callback runs in a different local context. Coroutines and `yield from` provide relief from that. The solution is not as simple as with threads, but more manageable than chained or nested callbacks.

Example 18-13 shows code from the third variation of the `asyncio` flag downloading script, using the country name to save each flag. The `download_many` and `download_coro` are unchanged from `flags2_asyncio.py` (Examples 18-7 and 18-8). The changes are:

`download_one`

This coroutine now uses `yield from` to delegate to `get_flag` and the new `get_country` coroutine.

`get_flag`

Most code from this coroutine was moved to a new `http_get` coroutine so it can also be used by `get_country`.

`get_country`

This coroutine fetches the `metadata.json` file for the country code, and gets the name of the country from it.

`http_get`

Common code for getting a file from the Web.

Example 18-13. `flags3_asyncio.py`: more coroutine delegation to perform two requests per flag

```
@asyncio.coroutine
def http_get(url):
    res = yield from aiohttp.request('GET', url)
    if res.status == 200:
        ctype = res.headers.get('Content-type', '').lower()
        if 'json' in ctype or url.endswith('.json'):
            data = yield from res.json() ❶
        else:
            data = yield from res.read() ❷
        return data

    elif res.status == 404:
        raise web.HTTPNotFound()
    else:
        raise aiohttp.errors.HttpProcessingError(
            code=res.status, message=res.reason,
            headers=res.headers)

@asyncio.coroutine
def get_country(base_url, cc):
    url = '{}/{cc}/metadata.json'.format(base_url, cc=cc.lower())
```



```

metadata = yield from http_get(url) ❸
return metadata['country']

```

```

@asyncio.coroutine
def get_flag(base_url, cc):
    url = '{}/{cc}/{cc}.gif'.format(base_url, cc=cc.lower())
    return (yield from http_get(url)) ❹

```

```

@asyncio.coroutine
def download_one(cc, base_url, semaphore, verbose):
    try:
        with (yield from semaphore): ❺
            image = yield from get_flag(base_url, cc)
        with (yield from semaphore):
            country = yield from get_country(base_url, cc)
    except web.HTTPNotFound:
        status = HTTPStatus.not_found
        msg = 'not found'
    except Exception as exc:
        raise FetchError(cc) from exc
    else:
        country = country.replace(' ', '_')
        filename = '{}-{}.gif'.format(country, cc)
        loop = asyncio.get_event_loop()
        loop.run_in_executor(None, save_flag, image, filename)
        status = HTTPStatus.ok
        msg = 'OK'

    if verbose and msg:
        print(cc, msg)

    return Result(status, cc)

```

- ❶ If the content type has 'json' in it or the url ends with .json, use the response .json() method to parse it and return a Python data structure—in this case, a dict.
- ❷ Otherwise, use .read() to fetch the bytes as they are.
- ❸ metadata will receive a Python dict built from the JSON contents.
- ❹ The outer parentheses here are required because the Python parser gets confused and produces a syntax error when it sees the keywords return yield from lined up like that.
- ❺ I put the calls to get_flag and get_country in separate with blocks controlled by the semaphore because I want to keep it acquired for the shortest possible time.

The `yield from` syntax appears nine times in [Example 18-13](#). By now you should be getting the hang of how this construct is used to delegate from one coroutine to another without blocking the event loop.

The challenge is to know when you have to use `yield from` and when you can't use it. The answer in principle is easy, you `yield from` coroutines and `asyncio.Future` instances—including tasks. But some APIs are tricky, mixing coroutines and plain functions in seemingly arbitrary ways, like the `StreamWriter` class we'll use in one of the servers in the next section.

[Example 18-13](#) wraps up the `flags2` set of examples. I encourage you to play with them to develop an intuition of how concurrent HTTP clients perform. Use the `-a`, `-e`, and `-l` command-line options to control the number of downloads, and the `-m` option to set the number of concurrent downloads. Run tests against the `LOCAL`, `REMOTE`, `DELAY`, and `ERROR` servers. Discover the optimum number of concurrent downloads to maximize throughput against each server. Tweak the settings of the *[vaurien_error_delay.sh](#)* script to add or remove errors and delays.

We'll now go from client scripts to writing servers with `asyncio`.

Writing `asyncio` Servers

The classic toy example of a TCP server is an echo server. We'll build slightly more interesting toys: Unicode character finders, first using plain TCP, then using HTTP. These servers will allow clients to query for Unicode characters based on words in their canonical names, using the `unicodedata` module we discussed in [“The Unicode Database” on page 127](#). A Telnet session with the TCP character finder server, searching for chess pieces and characters with the word “sun” is shown in [Figure 18-2](#).

```
lontra:charfinder luciano$ telnet localhost 2323
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
?> chess black
U+265A ♠      BLACK CHESS KING
U+265B ♚      BLACK CHESS QUEEN
U+265C ♖      BLACK CHESS ROOK
U+265D ♗      BLACK CHESS BISHOP
U+265E ♘      BLACK CHESS KNIGHT
U+265F ♙      BLACK CHESS PAWN
6 matches for 'chess black'
?> sun
U+2600 ☀      BLACK SUN WITH RAYS
U+2609 ☼      SUN
U+263C ☼      WHITE SUN WITH RAYS
U+26C5 ☁      SUN BEHIND CLOUD
U+2E9C ☼      CJK RADICAL SUN
U+2F47 ☼      KANGXI RADICAL SUN
U+3230 ☼      PARENTHEZIZED IDEOGRAPH SUN
U+3290 ☼      CIRCLED IDEOGRAPH SUN
U+C21C ☼      HANGUL SYLLABLE SUN
U+1F31E ☺     SUN WITH FACE
10 matches for 'sun'
?> ^C
Connection closed by foreign host.
lontra:charfinder luciano$
```

Figure 18-2. A Telnet session with the `tcp_charfinder.py` server: querying for “chess black” and “sun”.

Now, on to the implementations.

An asyncio TCP Server

Most of the logic in these examples is in the `charfinder.py` module, which has nothing concurrent about it. You can use `charfinder.py` as a command-line character finder, but more importantly, it was designed to provide content for our `asyncio` servers. The code for `charfinder.py` is in the [Fluent Python code repository](#).

The `charfinder` module indexes each word that appears in character names in the Unicode database bundled with Python, and creates an inverted index stored in a dict. For example, the inverted index entry for the key 'SUN' contains a set with the 10 Unicode characters that have that word in their names. The inverted index is saved in a local `charfinder_index.pickle` file. If multiple words appear in the query, `charfinder` computes the intersection of the sets retrieved from the index.

We'll now focus on the `tcp_charfinder.py` script that is answering the queries in [Figure 18-2](#). Because I have a lot to say about this code, I've split it into two parts: [Example 18-14](#) and [Example 18-15](#).

Example 18-14. `tcp_charfinder.py`: a simple TCP server using `asyncio.start_server`; code for this module continues in [Example 18-15](#)

```
import sys
import asyncio

from charfinder import UnicodeNameIndex ❶

CRLF = b'\r\n'
PROMPT = b'?> '

index = UnicodeNameIndex() ❷

@asyncio.coroutine
def handle_queries(reader, writer): ❸
    while True: ❹
        writer.write(PROMPT) # can't yield from! ❺
        yield from writer.drain() # must yield from! ❻
        data = yield from reader.readline() ❼
        try:
            query = data.decode().strip()
        except UnicodeDecodeError: ❽
            query = '\x00'
        client = writer.get_extra_info('peername') ❾
        print('Received from {}: {}'.format(client, query)) ❿
        if query:
            if ord(query[:1]) < 32: ⓫
                break
            lines = list(index.find_description_strs(query)) ⓬
            if lines:
                writer.writelines(line.encode() + CRLF for line in lines) ⓭
                writer.write(index.status(query, len(lines)).encode() + CRLF) ⓮

                yield from writer.drain() ⓯
                print('Sent {} results'.format(len(lines))) ⓰

        print('Close the client socket') ⓱
        writer.close() ⓲
```

- ❶ UnicodeNameIndex is the class that builds the index of names and provides querying methods.

- ❷ When instantiated, `UnicodeNameIndex` uses `charfinder_index.pickle`, if available, or builds it, so the first run may take a few seconds longer to start.⁸
- ❸ This is the coroutine we need to pass to `asyncio.startserver`; the arguments received are an `asyncio.StreamReader` and an `asyncio.StreamWriter`.
- ❹ This loop handles a session that lasts until any control character is received from the client.
- ❺ The `StreamWriter.write` method is not a coroutine, just a plain function; this line sends the `?>` prompt.
- ❻ `StreamWriter.drain` flushes the writer buffer; it is a coroutine, so it must be called with `yield from`.
- ❼ `StreamWriter.readline` is a coroutine; it returns bytes.
- ❽ A `UnicodeDecodeError` may happen when the Telnet client sends control characters; if that happens, we pretend a null character was sent, for simplicity.
- ❾ This returns the remote address to which the socket is connected.
- ❿ Log the query to the server console.
- ⓫ Exit the loop if a control or null character was received.
- ⓬ This returns a generator that yields strings with the Unicode codepoint, the actual character and its name (e.g., `U+0039\t9\tDIGIT NINE`); for simplicity, I build a list from it.
- ⓭ Send the lines converted to bytes using the default UTF-8 encoding, appending a carriage return and a line feed to each; note that the argument is a generator expression.
- ⓮ Write a status line such as `627 matches for 'digit'`.
- ⓯ Flush the output buffer.
- ⓰ Log the response to the server console.
- ⓱ Log the end of the session to the server console.
- ⓲ Close the `StreamWriter`.

The `handle_queries` coroutine has a plural name because it starts an interactive session and handles multiple queries from each client.

8. Leonardo Rochaël pointed out that building the `UnicodeNameIndex` could be delegated to another thread using `loop.run_with_executor()` in the main function of [Example 18-15](#), so the server would be ready to take requests immediately while the index is built. That is true, but querying the index is the only thing this app does, so that would not be a big win. It's an interesting exercise to do as Leo suggests, though. Go ahead and do it, if you like.

Note that all I/O in [Example 18-14](#) is in bytes. We need to decode the strings received from the network, and encode strings sent out. In Python 3, the default encoding is UTF-8, and that's what we are using implicitly.

One caveat is that some of the I/O methods are coroutines and must be driven with `yield from`, while others are simple functions. For example, `StreamWriter.write` is a plain function, on the assumption that most of the time it does not block because it writes to a buffer. On the other hand, `StreamWriter.drain`, which flushes the buffer and performs the actual I/O is a coroutine, as is `StreamReader.readline`. While I was writing this book, a major improvement to the `asyncio` API docs was the clear labeling of coroutines as such.

[Example 18-15](#) lists the main function for the module started in [Example 18-14](#).

Example 18-15. `tcp_charfinder.py` (continued from [Example 18-14](#)): the main function sets up and tears down the event loop and the socket server

```
def main(address='127.0.0.1', port=2323): ❶
    port = int(port)
    loop = asyncio.get_event_loop()
    server_coro = asyncio.start_server(handle_queries, address, port,
                                      loop=loop) ❷
    server = loop.run_until_complete(server_coro) ❸

    host = server.sockets[0].getsockname() ❹
    print('Serving on {}'. Hit CTRL-C to stop.'.format(host)) ❺
    try:
        loop.run_forever() ❻
    except KeyboardInterrupt: # CTRL+C pressed
        pass

    print('Server shutting down.')
    server.close() ❼
    loop.run_until_complete(server.wait_closed()) ❽
    loop.close() ❾

if __name__ == '__main__':
    main(*sys.argv[1:]) ❿
```

- ❶ The main function can be called with no arguments.
- ❷ When completed, the coroutine object returned by `asyncio.start_server` returns an instance of `asyncio.Server`, a TCP socket server.
- ❸ Drive `server_coro` to bring up the server.
- ❹ Get address and port of the first socket of the server and...
- ❺ ...display it on the server console. This is the first output generated by this script on the server console.

- ❹ Run the event loop; this is where `main` will block until killed when CTRL-C is pressed on the server console.
- ❺ Close the server.
- ❻ `server.wait_closed()` returns a future; use `loop.run_until_complete` to let the future do its job.
- ❼ Terminate the event loop.
- ❽ This is a shortcut for handling optional command-line arguments: `explode sys.argv[1:]` and pass it to a `main` function with suitable default arguments.

Note how `run_until_complete` accepts either a coroutine (the result of `start_server`) or a Future (the result of `server.wait_closed`). If `run_until_complete` gets a coroutine as argument, it wraps the coroutine in a Task.

You may find it easier to understand how control flows in `tcp_charfinder.py` if you take a close look at the output it generates on the server console, listed in [Example 18-16](#).

Example 18-16. `tcp_charfinder.py`: this is the server side of the session depicted in [Figure 18-2](#)

```
$ python3 tcp_charfinder.py
Serving on ('127.0.0.1', 2323). Hit CTRL-C to stop. ❶
Received from ('127.0.0.1', 62910): 'chess black' ❷
Sent 6 results
Received from ('127.0.0.1', 62910): 'sun' ❸
Sent 10 results
Received from ('127.0.0.1', 62910): '\x00' ❹
Close the client socket ❺
```

- ❶ This is output by `main`.
- ❷ First iteration of the while loop in `handle_queries`.
- ❸ Second iteration of the while loop.
- ❹ The user hit CTRL-C; the server receives a control character and closes the session.
- ❺ The client socket is closed but the server is still running, ready to service another client.

Note how `main` almost immediately displays the `Serving on...` message and blocks in the `loop.run_forever()` call. At that point, control flows into the event loop and stays there, occasionally coming back to the `handle_queries` coroutine, which yields control back to the event loop whenever it needs to wait for the network as it sends or receives data. While the event loop is alive, a new instance of the `handle_queries` coroutine will be started for each client that connects to the server. In this way, multiple clients can be

handled concurrently by this simple server. This continues until a `KeyboardInterrupt` occurs or the process is killed by the OS.

The `tcp_charfinder.py` code leverages the high-level `asyncio Streams API` that provides a ready-to-use server so you only need to implement a handler function, which can be a plain callback or a coroutine. There is also a lower-level `Transports and Protocols API`, inspired by the transport and protocols abstractions in the Twisted framework. Refer to the `asyncio Transports and Protocols documentation` for more information, including a TCP echo server implemented with that lower-level API.

The next section presents an HTTP character finder server.

An aiohttp Web Server

The `aiohttp` library we used for the `asyncio` flags examples also supports server-side HTTP, so that's what I used to implement the `http_charfinder.py` script. **Figure 18-3** shows the simple web interface of the server, displaying the result of a search for a “cat face” emoji.

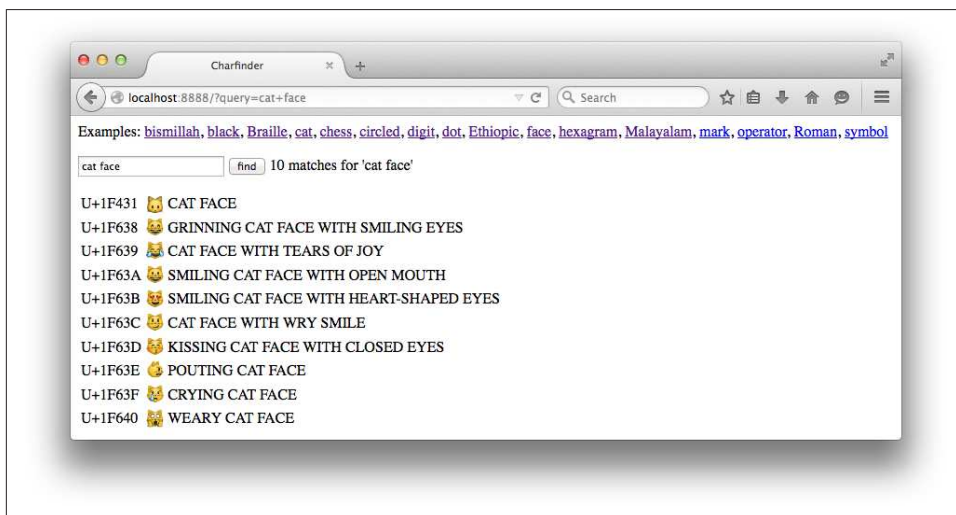


Figure 18-3. Browser window displaying search results for “cat face” on the `http_charfinder.py` server



Some browsers are better than others at displaying Unicode. The screenshot in [Figure 18-3](#) was captured with Firefox on OS X, and I got the same result with Safari. But up-to-date Chrome and Opera browsers on the same machine did not display emoji characters like the cat faces. Other search results (e.g., “chess”) looked fine, so it’s likely a font issue on Chrome and Opera on OSX.

We’ll start by analyzing the most interesting part of *http_charfinder.py*: the bottom half where the event loop and the HTTP server is set up and torn down. See [Example 18-17](#).

Example 18-17. http_charfinder.py: the main and init functions

```
@asyncio.coroutine
def init(loop, address, port): ❶
    app = web.Application(loop=loop) ❷
    app.router.add_route('GET', '/', home) ❸
    handler = app.make_handler() ❹
    server = yield from loop.create_server(handler,
                                           address, port) ❺
    return server.sockets[0].getsockname() ❻

def main(address="127.0.0.1", port=8888):
    port = int(port)
    loop = asyncio.get_event_loop()
    host = loop.run_until_complete(init(loop, address, port)) ❼
    print('Serving on {}'.format(host))
    try:
        loop.run_forever() ❽
    except KeyboardInterrupt: # CTRL+C pressed
        pass
    print('Server shutting down.')
    loop.close() ❾

if __name__ == '__main__':
    main(*sys.argv[1:])
```

- ❶ The `init` coroutine yields a server for the event loop to drive.
- ❷ The `aiohttp.web.Application` class represents a web application...
- ❸ ...with routes mapping URL patterns to handler functions; here `GET /` is routed to the `home` function (see [Example 18-18](#)).
- ❹ The `app.make_handler` method returns an `aiohttp.web.RequestHandler` instance to handle HTTP requests according to the routes set up in the app object.
- ❺ `create_server` brings up the server, using `handler` as the protocol handler and binding it to `address` and `port`.

- ⑥ Return the address and port of the first server socket.
- ⑦ Run `init` to start the server and get its address and port.
- ⑧ Run the event loop; `main` will block here while the event loop is in control.
- ⑨ Close the event loop.

As you get acquainted with the `asyncio` API, it's interesting to contrast how the servers are set up in [Example 18-17](#) and in the TCP example ([Example 18-15](#)) shown earlier.

In the earlier TCP example, the server was created and scheduled to run in the `main` function with these two lines:

```
server_coro = asyncio.start_server(handle_queries, address, port,  
                                   loop=loop)  
server = loop.run_until_complete(server_coro)
```

In the HTTP example, the `init` function creates the server like this:

```
server = yield from loop.create_server(handler,  
                                       address, port)
```

But `init` itself is a coroutine, and what makes it run is the `main` function, with this line:

```
host = loop.run_until_complete(init(loop, address, port))
```

Both `asyncio.start_server` and `loop.create_server` are coroutines that return `asyncio.Server` objects. In order to start up a server and return a reference to it, each of these coroutines must be driven to completion. In the TCP example, that was done by calling `loop.run_until_complete(server_coro)`, where `server_coro` was the result of `asyncio.start_server`. In the HTTP example, `create_server` is invoked on a `yield from` expression inside the `init` coroutine, which is in turn driven by the `main` function when it calls `loop.run_until_complete(init(...))`.

I mention this to emphasize this essential fact we've discussed before: a coroutine only does anything when driven, and to drive an `asyncio.coroutine` you either use `yield from` or pass it to one of several `asyncio` functions that take coroutine or future arguments, such as `run_until_complete`.

[Example 18-18](#) shows the `home` function, which is configured to handle the `/` (root) URL in our HTTP server.

Example 18-18. `http_charfinder.py`: the `home` function

```
def home(request): ①  
    query = request.GET.get('query', '').strip() ②  
    print('Query: {!r}'.format(query)) ③  
    if query: ④  
        descriptions = list(index.find_descriptions(query))  
        res = '\n'.join(ROW_TPL.format(**vars(descr))  
                        for descr in descriptions)
```

```

    msg = index.status(query, len(descriptions))
else:
    descriptions = []
    res = ''
    msg = 'Enter words describing characters.'

html = template.format(query=query, result=res, ❸
                        message=msg)
print('Sending {} results'.format(len(descriptions))) ❹
return web.Response(content_type=CONTENT_TYPE, text=html) ❺

```

- ❶ A route handler receives an `aihttp.web.Request` instance.
- ❷ Get the query string stripped of leading and trailing blanks.
- ❸ Log query to server console.
- ❹ If there was a query, bind `res` to HTML table rows rendered from result of the query to the `index`, and `msg` to a status message.
- ❺ Render the HTML page.
- ❻ Log response to server console.
- ❼ Build Response and return it.

Note that `home` is not a coroutine, and does not need to be if there are no `yield` from expressions in it. The `aihttp` documentation for the `add_route` method states that the handler “is converted to coroutine internally when it is a regular function.”

There is a downside to the simplicity of the `home` function in [Example 18-18](#). The fact that it’s a plain function and not a coroutine is a symptom of a larger issue: the need to rethink how we code web applications to achieve high concurrency. Let’s consider this matter.

Smarter Clients for Better Concurrency

The `home` function in [Example 18-18](#) looks very much like a view function in Django or Flask. There is nothing asynchronous about its implementation: it gets a request, fetches data from a database, and builds a response by rendering a full HTML page. In this example, the “database” is the `UnicodeNameIndex` object, which is in memory. But accessing a real database should be done asynchronously, otherwise you’re blocking the event loop while waiting for database results. For example, the `aiopg` package provides an asynchronous PostgreSQL driver compatible with `asyncio`; it lets you use `yield from` to send queries and fetch results, so your view function can behave as a proper coroutine.

Besides avoiding blocking calls, highly concurrent systems must split large chunks of work into smaller pieces to stay responsive. The `http_charfinder.py` server illustrates this

point: if you search for “cjkl” you’ll get back 75,821 Chinese, Japanese, and Korean ideographs.⁹ In this case, the `home` function will return a 5.3 MB HTML document, featuring a table with 75,821 rows.

On my machine, it takes 2s to fetch the response to the “cjkl” query, using the `curl` command-line HTTP client from a local `http_charfinder.py` server. A browser takes even longer to actually layout the page with such a huge table. Of course, most queries return much smaller responses: a query for “braille” returns 256 rows in a 19 KB page and takes 0.017s on my machine. But if the server spends 2s serving a single “cjkl” query, all the other clients will be waiting for at least 2s, and that is not acceptable.

The way to avoid the long response problem is to implement pagination: return results with at most, say, 200 rows, and have the user click or scroll the page to fetch more. If you look up the `charfinder.py` module in the *Fluent Python code repository*, you’ll see that the `UnicodeNameIndex.find_descriptions` method takes optional `start` and `stop` arguments: they are offsets to support pagination. So you could return the first 200 results, then use AJAX or even WebSockets to send the next batch when—and if—the user wants to see it.

Most of the necessary coding for sending results in batches would be on the browser. This explains why Google and all large-scale Internet properties rely on lots of client-side coding to build their services: smart asynchronous clients make better use of server resources.

Although smart clients can help even old-style Django applications, to really serve them well we need frameworks that support asynchronous programming all the way: from the handling of HTTP requests and responses, to the database access. This is especially true if you want to implement real-time services such as games and media streaming with WebSockets.¹⁰

Enhancing `http_charfinder.py` to support progressive download is left as an exercise to the reader. Bonus points if you implement “infinite scroll,” like Twitter does. With this challenge, I wrap up our coverage of concurrent programming with `asyncio`.

Chapter Summary

This chapter introduced a whole new way of coding concurrency in Python, leveraging `yield from`, coroutines, futures, and the `asyncio` event loop. The first simple examples, the spinner scripts, were designed to demonstrate a side-by-side comparison of the `threading` and the `asyncio` approaches to concurrency.

9. That’s what CJK stands for: the ever-expanding set of Chinese, Japanese, and Korean characters. Future versions of Python may support more CJK ideographs than Python 3.4 does.

10. I have more to say about this trend in “Soapbox” on page 580.

We then discussed the specifics of `asyncio.Future`, focusing on its support for `yield from`, and its relationship with coroutines and `asyncio.Task`. Next, we analyzed the `asyncio`-based flag download script.

We then reflected on Ryan Dahl's numbers for I/O latency and the effect of blocking calls. To keep a program alive despite the inevitable blocking functions, there are two solutions: using threads or asynchronous calls—the latter being implemented as callbacks or coroutines.

In practice, asynchronous libraries depend on lower-level threads to work—down to kernel-level threads—but the user of the library doesn't create threads and doesn't need to be aware of their use in the infrastructure. At the application level, we just make sure none of our code is blocking, and the event loop takes care of the concurrency under the hood. Avoiding the overhead of user-level threads is the main reason why asynchronous systems can manage more concurrent connections than multithreaded systems.

Resuming the flag downloading examples, adding a progress bar and proper error handling required significant refactoring, particularly with the switch from `asyncio.wait` to `asyncio.as_completed`, which forced us to move most of the functionality of `download_many` to a new `downloader_coro` coroutine, so we could use `yield from` to get the results from the futures produced by `asyncio.as_completed`, one by one.

We then saw how to delegate blocking jobs—such as saving a file—to a thread pool using the `loop.run_in_executor` method.

This was followed by a discussion of how coroutines solve the main problems of callbacks: loss of context when carrying out multistep asynchronous tasks, and lack of a proper context for error handling.

The next example—fetching the country names along with the flag images—demonstrated how the combination of coroutines and `yield from` avoids the so-called callback hell. A multistep procedure making asynchronous calls with `yield from` looks like simple sequential code, if you pay no attention to the `yield from` keywords.

The final examples in the chapter were `asyncio` TCP and HTTP servers that allow searching for Unicode characters by name. Analysis of the HTTP server ended with a discussion on the importance of client-side JavaScript to support higher concurrency on the server side, by enabling the client to make smaller requests on demand, instead of downloading large HTML pages.

Further Reading

Nick Coghlan, a Python core developer, made the following comment on the draft of [PEP-3156 — Asynchronous IO Support Rebooted: the “asyncio” Module](#) in January 2013:

Somewhere early in the PEP, there may need to be a concise description of the two APIs for waiting for an asynchronous `Future`:

1. `f.add_done_callback(...)`
2. `yield from f` in a coroutine (resumes the coroutine when the future completes, with either the result or exception as appropriate)

At the moment, these are buried in amongst much larger APIs, yet they’re key to understanding the way everything above the core event loop layer interacts.¹¹

Guido van Rossum, the author of [PEP-3156](#), did not heed Coghlan’s advice. Starting with PEP-3156, the `asyncio` documentation is very detailed but not user friendly. The nine `.rst` files that make up the [asyncio package docs](#) total 128 KB—that’s roughly 71 pages. In the standard library, only the [“Built-in Types” chapter](#) is bigger, and it covers the API for the numeric types, sequence types, generators, mappings, sets, `bool`, context managers, etc.

Most pages in the `asyncio` manual focus on concepts and the API. There are useful diagrams and examples scattered all over it, but one section that is very practical is [“18.5.11. Develop with asyncio,”](#) which presents essential usage patterns. The `asyncio` docs need more content explaining how `asyncio` should be used.

Because it’s very new, `asyncio` lacks coverage in print. Jan Palach’s *Parallel Programming with Python* (Packt, 2014) is the only book I found that has a chapter about `asyncio`, but it’s a short chapter.

There are, however, excellent presentations about `asyncio`. The best I found is Brett Slatkin’s [“Fan-In and Fan-Out: The Crucial Components of Concurrency,”](#) subtitled “Why do we need Tulip? (a.k.a., PEP 3156—asyncio),” which he presented at PyCon 2014 in Montréal ([video](#)). In 30 minutes, Slatkin shows a simple web crawler example, highlighting how `asyncio` is intended to be used. Guido van Rossum is in the audience and mentions that he also wrote a web crawler as a motivating example for `asyncio`; [Guido’s code](#) does not depend on `aiohttp`—it uses only the standard library. Slatkin also wrote the insightful post [“Python’s asyncio Is for Composition, Not Raw Performance.”](#)

11. Comment on PEP-3156 in a [Jan. 20, 2013 message](#) to the python-ideas list.

Other must-see `asyncio` talks are by Guido van Rossum himself: the [PyCon US 2013 keynote](#), and talks he gave at [LinkedIn](#) and [Twitter University](#). Also recommended are Saúl Ibarra Corretgé’s “A Deep Dive into PEP-3156 and the New `asyncio` Module” ([slides](#), [video](#)).

Dino Viehland showed how `asyncio` can be integrated with the Tkinter event loop in his “[Using futures for async GUI programming in Python 3.3](#)” talk at PyCon US 2013. Viehland shows how easy it is to implement the essential parts of the `asyncio.AbstractEventLoop` interface on top of another event loop. His code was written with Tulip, prior to the addition of `asyncio` to the standard library; I adapted it to work with the Python 3.4 release of `asyncio`. My updated refactoring is on [GitHub](#).

Victor Stinner—an `asyncio` core contributor and author of the [Trollius](#) backport—regularly updates a list of relevant links: [The new Python asyncio module aka “tulip”](#). Other collections of `asyncio` resources are [Asyncio.org](#) and [aio-lib](#)s on Github, where you’ll find asynchronous drivers for PostgreSQL, MySQL, and several NoSQL databases. I haven’t tested these drivers, but the projects seem very active as I write this.

Web services are going to be an important use case for `asyncio`. Your code will likely depend on the [aiohttp](#) library led by Andrew Svetlov. You’ll also want to set up an environment to test your error handling code, and the [Vaurien](#) “chaos TCP proxy” designed by Alexis Métaireau and Tarek Ziadé is invaluable for that. Vaurien was created for the [Mozilla Services](#) project and lets you introduce delays and random errors into the TCP traffic between your program and backend servers such as databases and web services providers.

Soapbox

The One Loop

For a long time, asynchronous programming has been the approach favored by most Pythonistas for network applications, but there was always the dilemma of picking one of the mutually incompatible libraries. Ryan Dahl cites Twisted as a source of inspiration for Node.js, and Tornado championed the use of coroutines for event-oriented programming in Python.

In the JavaScript world, there is some debate between advocates of simple callbacks and proponents of various competing higher-level abstractions. Early versions the Node.js API used Promises—similar to our Futures—but Ryan Dahl decided to standardize on callbacks only. James Corgan argues this was [Node’s biggest missed opportunity](#).

In Python, the debate is over: the addition of `asyncio` to the standard library establishes coroutines and futures as the Pythonic way of writing asynchronous code. Furthermore, the `asyncio` package defines standard interfaces for asynchronous futures and the event loop, providing reference implementations for them.

The *Zen of Python* applies perfectly:

There should be one—and preferably only one—obvious way to do it.

Although that way may not be obvious at first unless you're Dutch.

Maybe it takes a Dutch passport to find `yield from` obvious. It was not obvious at first for this Brazilian, but after a while I got the hang of it.

More importantly, `asyncio` was designed so that its event loop can be replaced by an external package. That's why the `asyncio.get_event_loop` and `set_event_loop` functions exist; they are part of an abstract **Event Loop Policy** API.

Tornado already has an **AsyncIOMainLoop** class that implements the `asyncio.AbstractEventLoop` interface, so you can run asynchronous code using both libraries on the same event loop. There is also the intriguing **Quamash** project that integrates `asyncio` to the Qt event loop for developing GUI applications with PyQt or PySide. These are just two of a growing number of interoperable event-oriented packages made possible by `asyncio`.

Smarter HTTP clients such as single-page web applications (like Gmail) or smartphone apps demand quick, lightweight responses and push updates. These needs are better served by asynchronous frameworks instead of traditional web frameworks like Django, which are designed to serve fully rendered HTML pages and lack support for asynchronous database access.

The WebSockets protocol was designed to enable real-time updates for clients that are always connected, from games to streaming applications. This requires highly concurrent asynchronous servers able to keep ongoing interactions with hundreds or thousands of clients. WebSockets is very well supported by the `asyncio` architecture and at least two libraries already implement it on top of `asyncio`: **Autobahn|Python** and **WebSockets**.

This overall trend—dubbed “the real-time Web”—is a key factor in the demand for Node.js, and the reason why rallying around `asyncio` is so important for the Python ecosystem. There's still a lot of work to do. For starters, we need an asynchronous HTTP server and client API in the standard library, an asynchronous **DBAPI** 3.0, and new database drivers built on `asyncio`.

The biggest advantage Python 3.4 with `asyncio` has over Node.js is Python itself: a better designed language, with coroutines and `yield from` to make asynchronous code more maintainable than the primitive callbacks of JavaScript. Our biggest disadvantage is the libraries: Python comes with “batteries included,” but our batteries are not designed for asynchronous programming. The rich ecosystem of libraries for Node.js is entirely built around `async` calls. But Python and Node.js both have a problem that Go and Erlang have solved from the start: we have no transparent way to write code that leverages all available CPU cores.

Standardizing the event loop interface and an asynchronous library was a major coup, and only our BDFL could have pulled it off, given that there were well-entrenched, high-quality alternatives available. He did it in consultation with the authors of the major Python asynchronous frameworks. The influence of Glyph Lefkowitz, the leader of Twisted, is most evident. Guido’s “[Deconstructing Deferred](#)” [post](#) to the Python-tulip group is a must-read if you want to understand why `asyncio.Future` is not like the Twisted `Deferred` class. Making clear his respect for the oldest and largest Python asynchronous framework, Guido also started the meme WWTD—What Would Twisted Do?—when discussing design options in the python-twisted group.¹²

Fortunately, Guido van Rossum led the charge so Python is better positioned to face the concurrency challenges of the present. Mastering `asyncio` takes effort. But if you plan to write concurrent network applications in Python, seek the One Loop:

*One Loop to rule them all, One Loop to find them,
One Loop to bring them all and in liveness bind them.*

12. See Guido’s [January 29, 2015, message](#), immediately followed by an answer from Glyph.

PART VI

Metaprogramming

HPP Ch 9 | The multiprocessing Module

The multiprocessing Module

Questions You'll Be Able to Answer After This Chapter

- What does the `multiprocessing` module offer?
- What's the difference between processes and threads?
- How do I choose the right size for a process pool?
- How do I use nonpersistent queues for work processing?
- What are the costs and benefits of interprocess communication?
- How can I process `numpy` data with many CPUs?
- How would I use `Joblib` to simplify parallelized and cached scientific work?
- Why do I need locking to avoid data loss?

CPython doesn't use multiple CPUs by default. This is partly because Python was designed back in a single-core era, and partly because parallelizing can actually be quite difficult to do efficiently. Python gives us the tools to do it but leaves us to make our own choices. It is painful to see your multicore machine using just one CPU on a long-running process, though, so in this chapter we'll review ways of using all the machine's cores at once.



We just mentioned *CPython*—the common implementation that we all use. Nothing in the Python language stops it from using multicore systems. CPython's implementation cannot efficiently use multiple cores, but future implementations may not be bound by this restriction.

We live in a multicore world—4 cores are common in laptops, and 32-core desktop configurations are available. If your job can be split to run on multiple CPUs *without* too much engineering effort, this is a wise direction to consider.

When Python is used to parallelize a problem over a set of CPUs, you can expect *up to* an n -times ($n\times$) speedup with n cores. If you have a quad-core machine and you can use all four cores for your task, it might run in a quarter of the original runtime. You are unlikely to see a greater than $4\times$ speedup; in practice, you'll probably see gains of 3–4 $\times$.

Each additional process will increase the communication overhead and decrease the available RAM, so you rarely get a full n -times speedup. Depending on which problem you are solving, the communication overhead can even get so large that you can see very significant slowdowns. These sorts of problems are often where the complexity lies for any sort of parallel programming and normally require a change in algorithm. This is why parallel programming is often considered an art.

If you're not familiar with [Amdahl's law](#), it is worth doing some background reading. The law shows that if only a small part of your code can be parallelized, it doesn't matter how many CPUs you throw at it; it still won't run much faster overall. Even if a large fraction of your runtime could be parallelized, there's a finite number of CPUs that can be used efficiently to make the overall process run faster before you get to a point of diminishing returns.

The `multiprocessing` module lets you use process- and thread-based parallel processing, share work over queues, and share data among processes. It is mostly focused on single-machine multicore parallelism (there are better options for multimachine parallelism). A very common use is to parallelize a task over a set of processes for a CPU-bound problem. You might also use OpenMP to parallelize an I/O-bound problem, but as we saw in [Chapter 8](#), there are better tools for this (e.g., the new `asyncio` module in Python 3 and `tornado`).



OpenMP is a low-level interface to multiple cores—you might wonder whether to focus on it rather than `multiprocessing`. We introduced it with Cython back in [Chapter 7](#), but we don't cover it in this chapter. `multiprocessing` works at a higher level, sharing Python data structures, while OpenMP works with C primitive objects (e.g., integers and floats) once you've compiled to C. Using it makes sense only if you're compiling your code; if you're not compiling (e.g., if you're using efficient `numpy` code and you want to run on many cores), then sticking with `multiprocessing` is probably the right approach.

To parallelize your task, you have to think a little differently from the normal way of writing a serial process. You must also accept that debugging a parallelized task is

harder—often, it can be very frustrating. We’d recommend keeping the parallelism as simple as possible (even if you’re not squeezing every last drop of power from your machine) so that your development velocity is kept high.

One particularly difficult topic is the sharing of state in a parallel system—it feels like it should be easy, but it incurs lots of overhead and can be hard to get right. There are many use cases, each with different trade-offs, so there’s definitely no one solution for everyone. In “[Verifying Primes Using Interprocess Communication](#)” on page 278, we’ll go through state sharing with an eye on the synchronization costs. Avoiding shared state will make your life far easier.

In fact, an algorithm can be analyzed to see how well it’ll perform in a parallel environment almost entirely by how much state must be shared. For example, if we can have multiple Python processes all solving the same problem without communicating with one another (a situation known as *embarrassingly parallel*), not much of a penalty will be incurred as we add more and more Python processes.

On the other hand, if each process needs to communicate with every other Python process, the communication overhead will slowly overwhelm the processing and slow things down. This means that as we add more and more Python processes, we can actually slow down our overall performance.

As a result, sometimes some counterintuitive algorithmic changes must be made to efficiently solve a problem in parallel. For example, when solving the diffusion equation ([Chapter 6](#)) in parallel, each process actually does some redundant work that another process also does. This redundancy reduces the amount of communication required and speeds up the overall calculation!

Here are some typical jobs for the `multiprocessing` module:

- Parallelize a CPU-bound task with `Process` or `Pool` objects
- Parallelize an I/O-bound task in a `Pool` with threads using the (oddly named) `dummy` module
- Share pickled work via a `Queue`
- Share state between parallelized workers, including bytes, primitive datatypes, dictionaries, and lists

If you come from a language where threads are used for CPU-bound tasks (e.g., C++ or Java), you should know that while threads in Python are OS-native (they’re not simulated—they are actual operating system threads), they are bound by the GIL, so only one thread may interact with Python objects at a time.

By using processes, we run a number of Python interpreters in parallel, each with a private memory space with its own GIL, and each runs in series (so there’s no competition for each GIL). This is the easiest way to speed up a CPU-bound task in Python.

If we need to share state, we need to add some communication overhead; we'll explore that in “[Verifying Primes Using Interprocess Communication](#)” on page 278.

If you work with numpy arrays, you might wonder if you can create a larger array (e.g., a large 2D matrix) and ask processes to work on segments of the array in parallel. You can, but it is hard to discover how by trial and error, so in “[Sharing numpy Data with multiprocessing](#)” on page 295 we'll work through sharing a 25 GB numpy array across four CPUs. Rather than sending partial copies of the data (which would at least double the working size required in RAM and create a massive communication overhead), we share the underlying bytes of the array among the processes. This is an ideal approach to sharing a large array among local workers on one machine.

In this chapter we also introduce the [Joblib](#) library—this builds on the multiprocessing library and offers improved cross-platform compatibility, a simple API for parallelization, and convenient persistence of cached results. Joblib is designed for scientific use, and we urge you to check it out.



Here, we discuss multiprocessing on *nix-based machines (this chapter is written using Ubuntu; the code should run unchanged on a Mac). Since Python 3.4, the quirks that appeared on Windows have been dealt with. Joblib has stronger cross-platform support than multiprocessing, and we recommend you review it ahead of multiprocessing.

In this chapter we'll hardcode the number of processes (`NUM_PROCESSES=4`) to match the four physical cores on Ian's laptop. By default, multiprocessing will use as many cores as it can see (the machine presents eight—four CPUs and four hyperthreads). Normally you'd avoid hardcoding the number of processes to create unless you were specifically managing your resources.

An Overview of the multiprocessing Module

The multiprocessing module provides a low-level interface to process- and thread-based parallelism. Its main components are as follows:

Process

A forked copy of the current process; this creates a new process identifier, and the task runs as an independent child process in the operating system. You can start and query the state of the Process and provide it with a target method to run.

Pool

Wraps the Process or threading.Thread API into a convenient pool of workers that share a chunk of work and return an aggregated result.

Queue

A FIFO queue allowing multiple producers and consumers.

Pipe

A uni- or bidirectional communication channel between two processes.

Manager

A high-level managed interface to share Python objects between processes.

ctypes

Allows sharing of primitive datatypes (e.g., integers, floats, and bytes) between processes after they have forked.

Synchronization primitives

Locks and semaphores to synchronize control flow between processes.



In Python 3.2, the `concurrent.futures` module was introduced (via [PEP 3148](#)); this provides the core behavior of multiprocessing, with a simpler interface based on Java's `java.util.concurrent`. It is available as a [backport to earlier versions of Python](#). We expect multiprocessing to continue to be preferred for CPU-intensive work and won't be surprised if `concurrent.futures` becomes more popular for I/O-bound tasks.

In the rest of the chapter, we'll introduce a set of examples to demonstrate common ways of using the `multiprocessing` module.

We'll estimate pi using a Monte Carlo approach with a `Pool` of processes or threads, using normal Python and `numpy`. This is a simple problem with well-understood complexity, so it parallelizes easily; we can also see an unexpected result from using threads with `numpy`. Next, we'll search for primes using the same `Pool` approach; we'll investigate the nonpredictable complexity of searching for primes and look at how we can efficiently (and inefficiently!) split the workload to best use our computing resources. We'll finish the primes search by switching to queues, where we introduce `Process` objects in place of a `Pool` and use a list of work and poison pills to control the lifetime of workers.

Next, we'll tackle interprocess communication (IPC) to validate a small set of possible primes. By splitting each number's workload across multiple CPUs, we use IPC to end the search early if a factor is found so that we can significantly beat the speed of a single-CPU search process. We'll cover shared Python objects, OS primitives, and a Redis server to investigate the complexity and capability trade-offs of each approach.

We can share a 25 GB `numpy` array across four CPUs to split a large workload *without* copying data. If you have large arrays with parallelizable operations, this technique

should buy you a great speedup, since you have to allocate less space in RAM and copy less data. Finally, we'll look at synchronizing access to a file and a variable (as a Value) between processes without corrupting data to illustrate how to correctly lock shared state.



PyPy (discussed in [Chapter 7](#)) has full support for the multiprocessing library, and the following CPython examples (though not the numpy examples, at the time of this writing) all run far quicker using PyPy. If you're using only CPython code (no C extensions or more complex libraries) for parallel processing, PyPy might be a quick win for you.

Estimating Pi Using the Monte Carlo Method

We can estimate pi by throwing thousands of imaginary darts into a “dartboard” represented by a unit circle. The relationship between the number of darts falling inside the circle's edge and the number falling outside it will allow us to approximate pi.

This is an ideal first problem, as we can split the total workload evenly across a number of processes, each one running on a separate CPU. Each process will end at the same time since the workload for each is equal, so we can investigate the speedups available as we add new CPUs and hyperthreads to the problem.

In [Figure 9-1](#), we throw 10,000 darts into the unit square, and a percentage of them fall into the quarter of the unit circle that's drawn. This estimate is rather bad—10,000 dart throws does not reliably give us a three-decimal-place result. If you ran your own code, you'd see this estimate vary between 3.0 and 3.2 on each run.

To be confident of the first three decimal places, we need to generate 10,000,000 random dart throws.¹ This is massively inefficient (and better methods for pi's estimation exist), but it is rather convenient to demonstrate the benefits of parallelization using multiprocessing.

With the Monte Carlo method, we use the [Pythagorean theorem](#) to test if a dart has landed inside our circle:

$$x^2 + y^2 \leq 1^2 = 1$$

¹ See [Brett Foster's PowerPoint presentation](#) on using the Monte Carlo method to estimate pi.

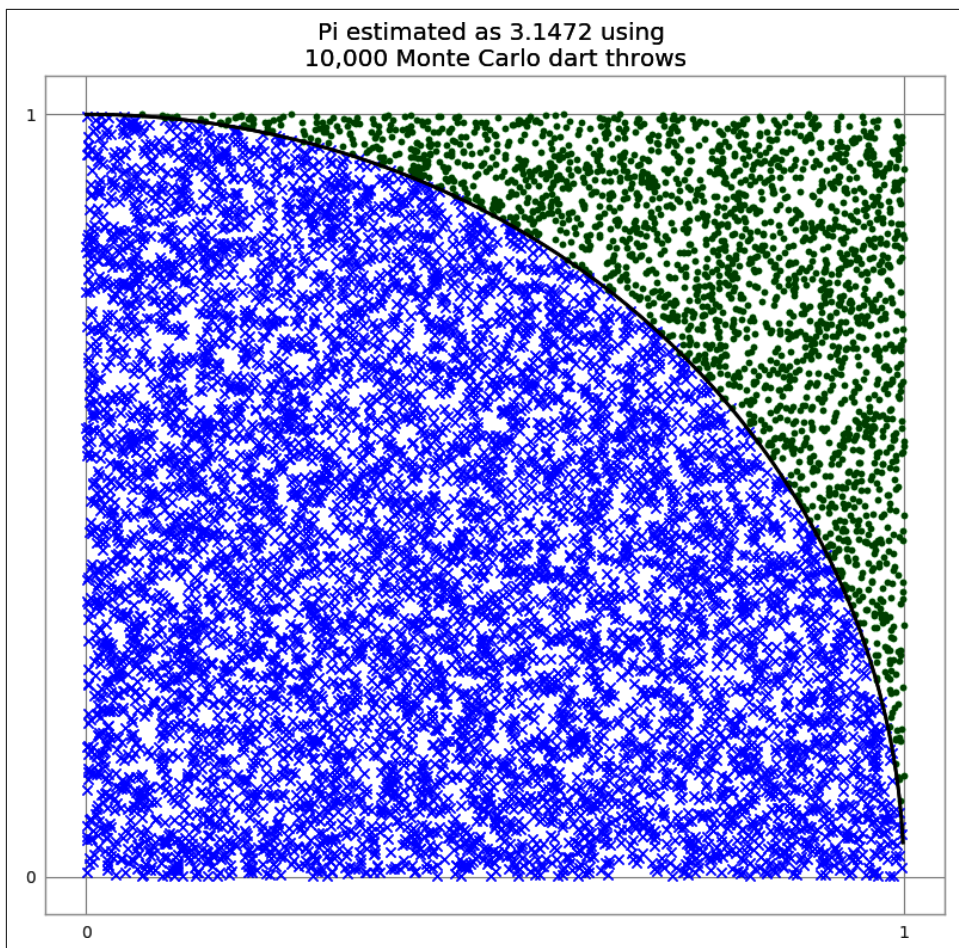


Figure 9-1. Estimating π using the Monte Carlo method

We'll look at a loop version of this in [Example 9-1](#). We'll implement both a normal Python version and, later, a numpy version, and we'll use both threads and processes to parallelize the problem.

Estimating Pi Using Processes and Threads

It is easier to understand a normal Python implementation, so we'll start with that in this section, using float objects in a loop. We'll parallelize this using processes to use all of our available CPUs, and we'll visualize the state of the machine as we use more CPUs.

Using Python Objects

The Python implementation is easy to follow, but it carries an overhead, as each Python float object has to be managed, referenced, and synchronized in turn. This overhead slows down our runtime, but it has bought us thinking time, as the implementation was quick to put together. By parallelizing this version, we get additional speedups for very little extra work.

Figure 9-2 shows three implementations of the Python example:

- No use of multiprocessing (named “Serial”)—one for loop in the main process
- Using threads
- Using processes

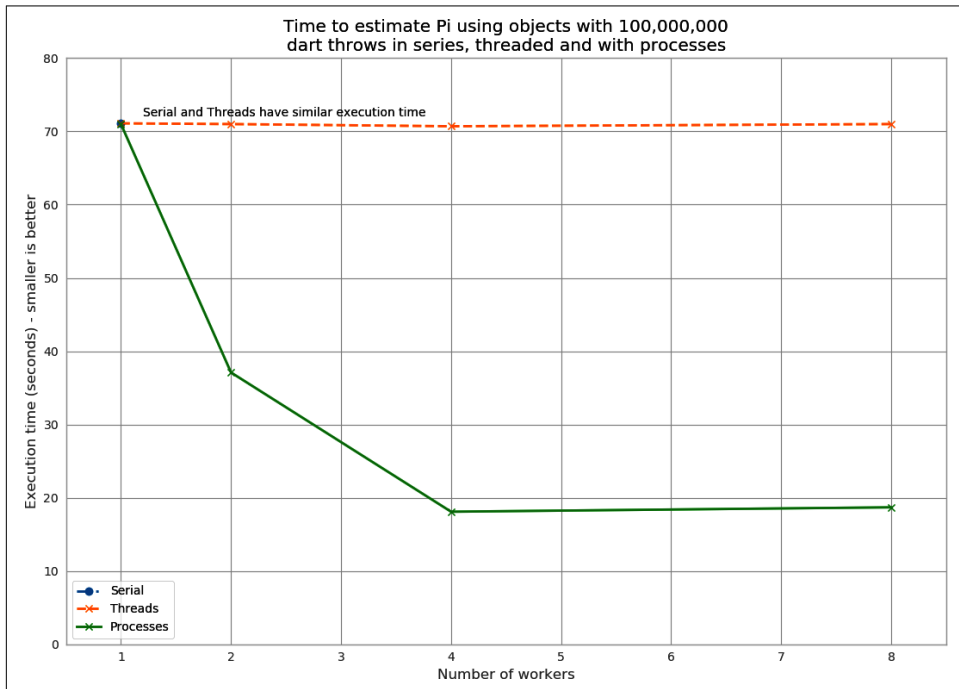


Figure 9-2. Working in series, with threads, and with processes

When we use more than one thread or process, we’re asking Python to calculate the same total number of dart throws and to divide the work evenly between workers. If we want 100,000,000 dart throws in total using our Python implementation and we use two workers, we’ll be asking both threads or both processes to generate 50,000,000 dart throws per worker.

Using one thread takes approximately 71 seconds, with no speedup when using more threads. By using two or more processes, we make the runtime *shorter*. The cost of using no processes or threads (the series implementation) is the same as running with one process.

By using processes, we get a linear speedup when using two or four cores on Ian's laptop. For the eight-worker case, we're using Intel's Hyper-Threading Technology—the laptop has only four physical cores, so we get barely any change in speedup by running eight processes.

Example 9-1 shows the Python version of our pi estimator. If we're using threads, each instruction is bound by the GIL, so although each thread could run on a separate CPU, it will execute only when no other threads are running. The process version is not bound by this restriction, as each forked process has a private Python interpreter running as a single thread—there's no GIL contention, as no objects are shared. We use Python's built-in random number generator, but see “**Random Numbers in Parallel Systems**” on page 263 for some notes about the dangers of parallelized random number sequences.

Example 9-1. Estimating pi using a loop in Python

```
def estimate_nbr_points_in_quarter_circle(nbr_estimates):
    """Monte Carlo estimate of the number of points in a
    quarter circle using pure Python"""
    print(f"Executing estimate_nbr_points_in_quarter_circle \
          with {nbr_estimates:,} on pid {os.getpid()}")
    nbr_trials_in_quarter_unit_circle = 0
    for step in range(int(nbr_estimates)):
        x = random.uniform(0, 1)
        y = random.uniform(0, 1)
        is_in_unit_circle = x * x + y * y <= 1.0
        nbr_trials_in_quarter_unit_circle += is_in_unit_circle

    return nbr_trials_in_quarter_unit_circle
```

Example 9-2 shows the `__main__` block. Note that we build the `Pool` before we start the timer. Spawning threads is relatively instant; spawning processes involves a fork, and this takes a measurable fraction of a second. We ignore this overhead in **Figure 9-2**, as this cost will be a tiny fraction of the overall execution time.

Example 9-2. main for estimating pi using a loop

```
from multiprocessing import Pool
...

if __name__ == "__main__":
    nbr_samples_in_total = 1e8
```

```

nbr_parallel_blocks = 4
pool = Pool(processes=nbr_parallel_blocks)
nbr_samples_per_worker = nbr_samples_in_total / nbr_parallel_blocks
print("Making {:,} samples per {} worker".format(nbr_samples_per_worker,
                                                  nbr_parallel_blocks))
nbr_trials_per_process = [nbr_samples_per_worker] * nbr_parallel_blocks
t1 = time.time()
nbr_in_quarter_unit_circles = pool.map(estimate_nbr_points_in_quarter_circle,
                                       nbr_trials_per_process)
pi_estimate = sum(nbr_in_quarter_unit_circles) * 4 / float(nbr_samples_in_total)
print("Estimated pi", pi_estimate)
print("Delta:", time.time() - t1)

```

We create a list containing `nbr_estimates` divided by the number of workers. This new argument will be sent to each worker. After execution, we'll receive the same number of results back; we'll sum these to estimate the number of darts in the unit circle.

We import the process-based `Pool` from `multiprocessing`. We also could have used `from multiprocessing.dummy import Pool` to get a threaded version. The “dummy” name is rather misleading (we confess to not understanding why it is named this way); it is simply a light wrapper around the `threading` module to present the same interface as the process-based `Pool`.



Each process we create consumes some RAM from the system. You can expect a forked process using the standard libraries to take on the order of 10–20 MB of RAM; if you're using many libraries and lots of data, you might expect each forked copy to take hundreds of megabytes. On a system with a RAM constraint, this might be a significant issue—if you run out of RAM and the system reverts to using the disk's swap space, any parallelization advantage will be massively lost to the slow paging of RAM back and forth to disk!

The following figures plot the average CPU utilization of Ian's laptop's four physical cores and their four associated hyperthreads (each hyperthread runs on unutilized silicon in a physical core). The data gathered for these figures *includes* the startup time of the first Python process and the cost of starting subprocesses. The CPU sampler records the entire state of the laptop, not just the CPU time used by this task.

Note that the following diagrams are created using a different timing method with a slower sampling rate than [Figure 9-2](#), so the overall runtime is a little longer.

The execution behavior in [Figure 9-3](#) with one process in the Pool (along with the parent process) shows some overhead in the first seconds as the Pool is created, and then a consistent close-to-100% CPU utilization throughout the run. With one process, we're efficiently using one core.

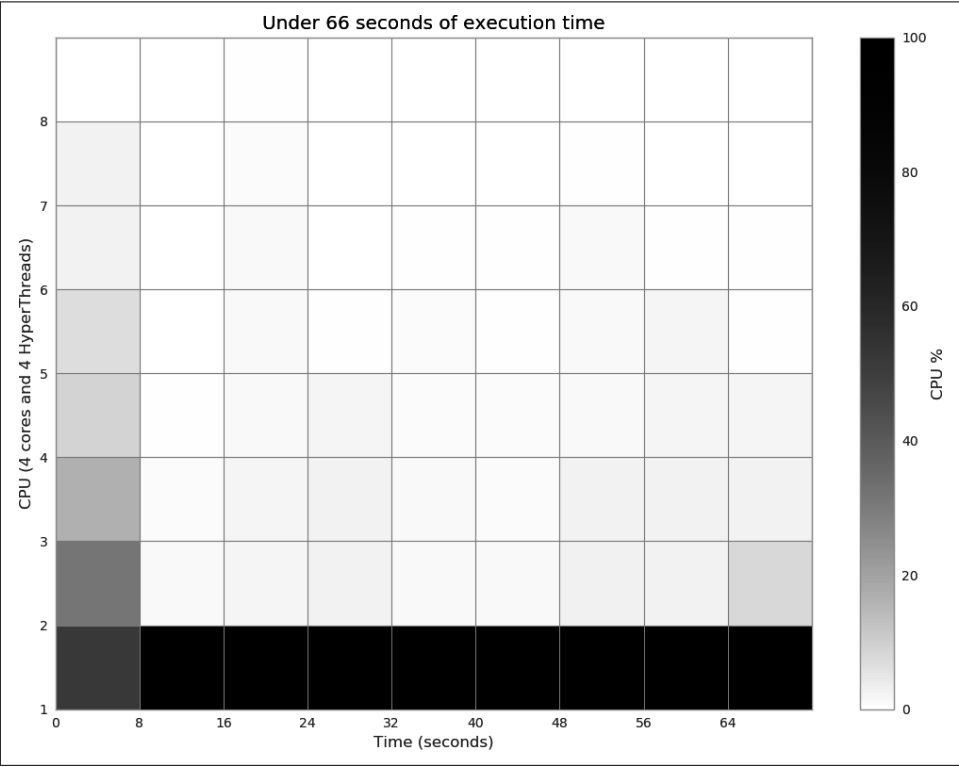


Figure 9-3. Estimating pi using Python objects and one process

Next we'll add a second process, effectively saying `Pool(processes=2)`. As you can see in [Figure 9-4](#), adding a second process roughly halves the execution time to 37 seconds, and two CPUs are fully occupied. This is the best result we can expect—we've efficiently used all the new computing resources, and we're not losing any speed to other overheads like communication, paging to disk, or contention with competing processes that want to use the same CPUs.

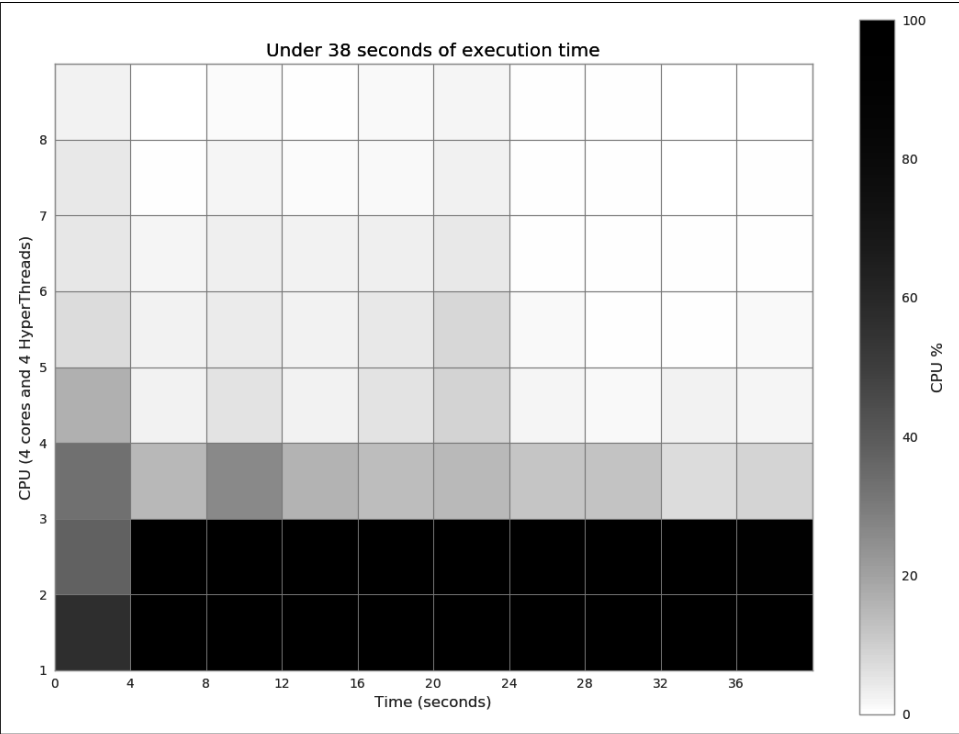


Figure 9-4. Estimating pi using Python objects and two processes

Figure 9-5 shows the results when using all four physical CPUs—now we are using all of the raw power of this laptop. Execution time is roughly a quarter that of the single-process version, at 19 seconds.

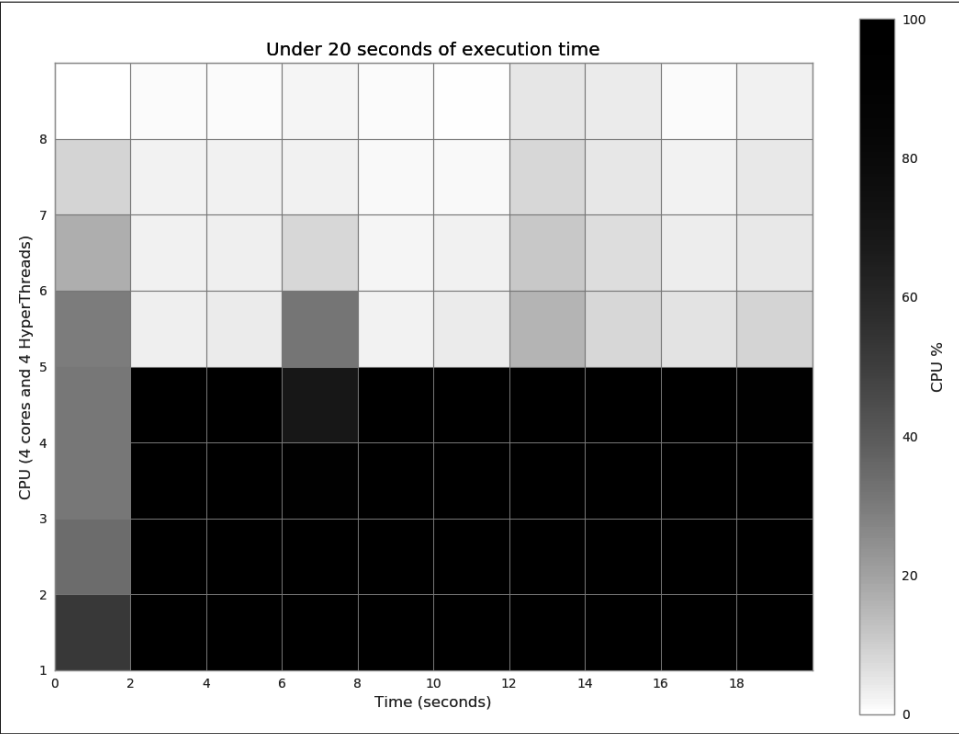


Figure 9-5. Estimating pi using Python objects and four processes

By switching to eight processes, as seen in [Figure 9-6](#), we cannot achieve more than a tiny speedup compared to the four-process version. That is because the four hyperthreads are able to squeeze only a little extra processing power out of the spare silicon on the CPUs, and the four CPUs are already maximally utilized.

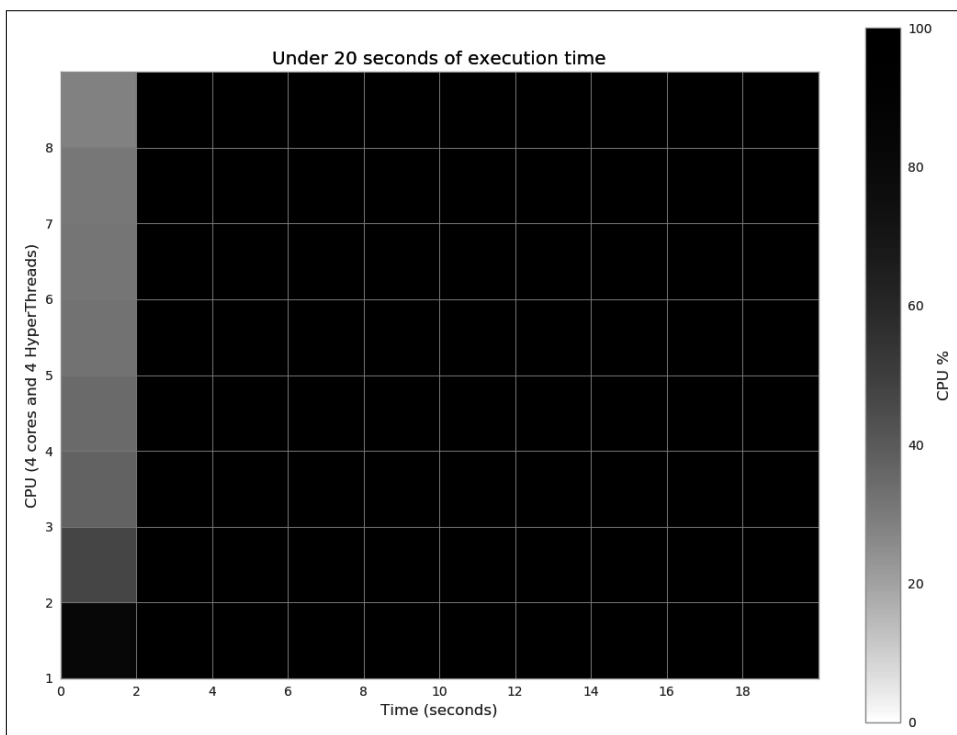


Figure 9-6. Estimating pi using Python objects and eight processes, with little additional gain

These diagrams show that we’re efficiently using more of the available CPU resources at each step, and that the hyperthread resources are a poor addition. The biggest problem when using hyperthreads is that CPython is using a lot of RAM—hyperthreading is not cache friendly, so the spare resources on each chip are very poorly utilized. As we’ll see in the next section, numpy makes better use of these resources.



In our experience, hyperthreading can give up to a 30% performance gain *if* there are enough spare computing resources. This works if, for example, you have a mix of floating-point and integer arithmetic rather than just the floating-point operations we have here. By mixing the resource requirements, the hyperthreads can schedule more of the CPU's silicon to be working concurrently. Generally, we see hyperthreads as an added bonus and not a resource to be optimized against, as adding more CPUs is probably more economical than tuning your code (which adds a support overhead).

Now we'll switch to using threads in one process, rather than multiple processes.

Figure 9-7 shows the results of running the same code that we used in **Figure 9-5**, but with threads in place of processes. Although a number of CPUs are being used, they each share the workload lightly. If each thread was running without the GIL, then we'd see 100% CPU utilization on the four CPUs. Instead, each CPU is partially utilized (because of the GIL).

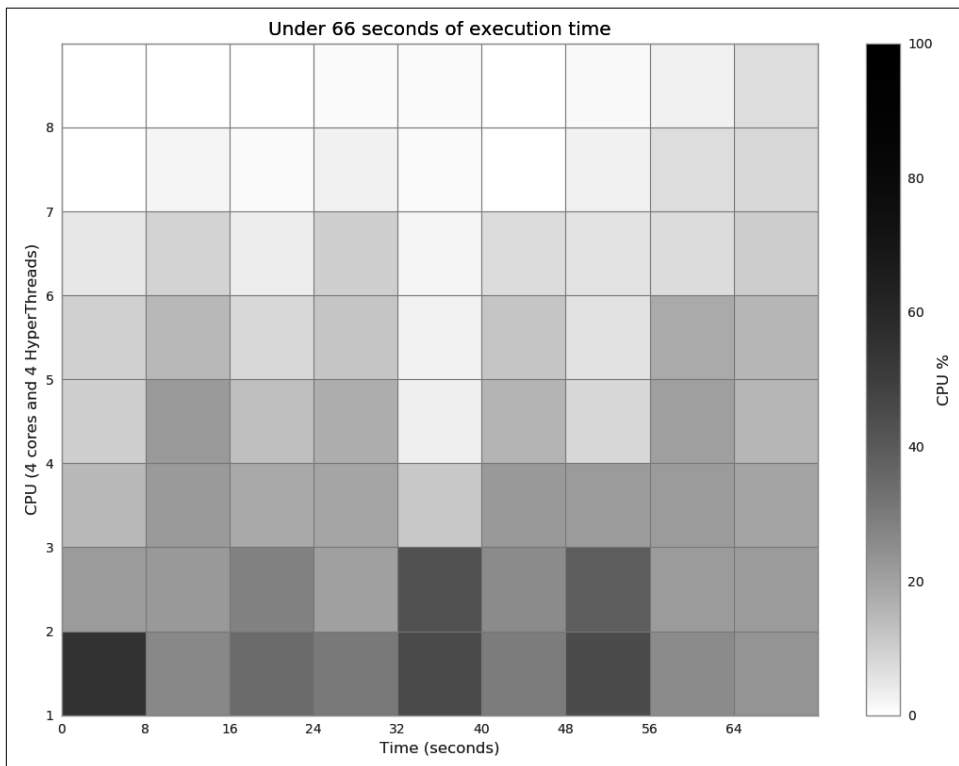


Figure 9-7. Estimating pi using Python objects and four threads

Replacing multiprocessing with Joblib

Joblib is an improvement on multiprocessing that enables lightweight pipelining with a focus on easy parallel computing and transparent disk-based caching of results. It focuses on NumPy arrays for scientific computing. It may offer a quick win for you if you're

- Using pure Python, with or without NumPy, to process a loop that could be embarrassingly parallel
- Calling expensive functions that have no side effects, where the output could be cached to disk between sessions
- Able to share NumPy data between processes but don't know how (and you haven't yet read ["Sharing numpy Data with multiprocessing" on page 295](#))

Joblib builds on the Loky library (itself an improvement over Python's `concurrent.futures`) and uses `cloudpickle` to enable the pickling of functions defined in the interactive scope. This solves a couple of common issues that are encountered with the built-in multiprocessing library.

For parallel computing, we need the `Parallel` class and the `delayed` decorator. The `Parallel` class sets up a process pool, similar to the multiprocessing pool we used in the previous section. The `delayed` decorator wraps our target function so it can be applied to the instantiated `Parallel` object via an iterator.

The syntax is a little confusing to read—take a look at [Example 9-3](#). The call is written on one line; this includes our target function `estimate_nbr_points_in_quarter_circle` and the iterator (`delayed(...)(nbr_samples_per_worker)` for `sample_idx` in `range(nbr_parallel_blocks)`). Let's break this down.

Example 9-3. Using Joblib to parallelize pi estimation

```
...
from joblib import Parallel, delayed

if __name__ == "__main__":
    ...
    nbr_in_quarter_unit_circles = Parallel(n_jobs=nbr_parallel_blocks, verbose=1) \
        (delayed(estimate_nbr_points_in_quarter_circle)(nbr_samples_per_worker) \
         for sample_idx in range(nbr_parallel_blocks))
    ...
```

`Parallel` is a class; we can set parameters such as `n_jobs` to dictate how many processes will run, along with optional arguments like `verbose` for debugging information. Other arguments can set time-outs, change between threads or processes,

change the backends (which can help speed up certain edge cases), and configure memory mapping.

`Parallel` has a `__call__` callable method that takes an iterable. We supply the iterable in the following round brackets (... for `sample_idx` in `range(...)`). The callable iterates over each `delayed(estimate_nbr_points_in_quarter_circle)` function, batching the execution of these functions to their arguments (in this case, `nbr_samples_per_worker`). Ian has found it helpful to build up a parallelized call one step at a time, starting from a function with no arguments and building up arguments as needed. This makes diagnosing missteps much easier.

`nbr_in_quarter_unit_circles` will be a list containing the count of positive cases for each call as before. **Example 9-4** shows the console output for eight parallel blocks; each process ID (PID) is freshly created, and a summary is printed in a progress bar at the end of the output. In total this takes 19 seconds, the same amount of time as when we created our own `Pool` in the previous section.



Avoid passing large structures; passing large pickled objects to each process may be expensive. Ian had a case with a prebuilt cache of Pandas DataFrames in a dictionary object; the cost of serializing these via the `Pickle` module negated the gains from parallelization, and the serial version actually worked faster overall. The solution in this case was to build the DataFrame cache using Python's built-in **shelve module**, storing the dictionary to a file. A single DataFrame was loaded with `shelve` on each call to the target function; hardly anything had to be passed to the functions, and then the parallelized benefit of `Joblib` was clear.

Example 9-4. Output of `Joblib` calls

```
Making 12,500,000 samples per 8 worker
[Parallel(n_jobs=8)]: Using backend LokyBackend with 8 concurrent workers.
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10313
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10315
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10311
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10316
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10312
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10314
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10317
Executing estimate_nbr_points_in_quarter_circle with 12,500,000 on pid 10318
[Parallel(n_jobs=8)]: Done   2 out of   8 | elapsed:   18.9s remaining:   56.6s
[Parallel(n_jobs=8)]: Done   8 out of   8 | elapsed:   19.3s finished
Estimated pi 3.14157744
Delta: 19.32842755317688
```



To simplify debugging, we can set `n_jobs=1`, and the parallelized code is dropped. You don't have to modify your code any further, and you can drop a call to `breakpoint()` in your function to ease your debugging.

Intelligent caching of function call results

A useful feature in Joblib is the Memory cache; this is a decorator that caches function results based on the input arguments to a disk cache. This cache persists between Python sessions, so if you turn off your machine and then run the same code the next day, the cached results will be used.

For our pi estimation, this presents a small problem. We don't pass in unique arguments to `estimate_nbr_points_in_quarter_circle`; for each call we pass in `nbr_estimates`, so the call signature is the same, but we're after different results.

In this situation, once the first call has completed (taking around 19 seconds), any subsequent call with the same argument will get the cached result. This means that if we rerun our code a second time, it completes instantly, but it uses only one of the eight sample results as the result for each call—this obviously breaks our Monte Carlo sampling! If the last process to complete resulted in 9815738 points in the quarter circle, the cache for the function call would always answer this. Repeating the call eight times would generate [9815738, 9815738, 9815738, 9815738, 9815738, 9815738, 9815738, 9815738] rather than eight unique estimates.

The solution in [Example 9-5](#) is to pass in a second argument, `idx`, which takes on a value between 0 and `nbr_parallel_blocks-1`. This unique combination of arguments will let the cache store each positive count, so that on the second run we get the same result as on the first run, but without the wait.

This is configured using Memory, which takes a folder for persisting the function results. This persistence is kept between Python sessions; it is refreshed if you change the function that is being called, or if you empty the files in the cache folder.

Note that this refresh applies only to a change to the function that's been decorated (in this case, `estimate_nbr_points_in_quarter_circle_with_idx`), not to any sub-functions that are called from inside that function.

Example 9-5. Caching results with Joblib

```
...
from joblib import Memory

memory = Memory("./joblib_cache", verbose=0)

@memory.cache
```

```
def estimate_nbr_points_in_quarter_circle_with_idx(nbr_estimates, idx):
    print(f"Executing estimate_nbr_points_in_quarter_circle with \
          {nbr_estimates} on sample {idx} on pid {os.getpid()}")
    ...

if __name__ == "__main__":
    ...
    nbr_in_quarter_unit_circles = Parallel(n_jobs=nbr_parallel_blocks) \
        (delayed(estimate_nbr_points_in_quarter_circle_with_idx) \
         (nbr_samples_per_worker, idx) for idx in range(nbr_parallel_blocks))
    ...
```

In [Example 9-6](#), we can see that while the first call costs 19 seconds, the second call takes only a fraction of a second and has the same estimated pi. In this run, the estimates were [9817605, 9821064, 9818420, 9817571, 9817688, 9819788, 9816377, 9816478].

Example 9-6. The zero-cost second call to the code thanks to cached results

```
$ python pi_lists_parallel_joblib_cache.py
Making 12,500,000 samples per 8 worker
Executing estimate_nbr_points_in_... with 12500000 on sample 0 on pid 10672
Executing estimate_nbr_points_in_... with 12500000 on sample 1 on pid 10676
Executing estimate_nbr_points_in_... with 12500000 on sample 2 on pid 10677
Executing estimate_nbr_points_in_... with 12500000 on sample 3 on pid 10678
Executing estimate_nbr_points_in_... with 12500000 on sample 4 on pid 10679
Executing estimate_nbr_points_in_... with 12500000 on sample 5 on pid 10674
Executing estimate_nbr_points_in_... with 12500000 on sample 6 on pid 10673
Executing estimate_nbr_points_in_... with 12500000 on sample 7 on pid 10675
Estimated pi 3.14179964
Delta: 19.28862953186035

$ python %run pi_lists_parallel_joblib_cache.py
Making 12,500,000 samples per 8 worker
Estimated pi 3.14179964
Delta: 0.02478170394897461
```

Joblib wraps up a lot of multiprocessing functionality with a simple (if slightly hard to read) interface. Ian has moved to using Joblib in favor of multiprocessing; he recommends you try it too.

Random Numbers in Parallel Systems

Generating good random number sequences is a hard problem, and it is easy to get it wrong if you try to do it yourself. Getting a good sequence quickly in parallel is even harder—suddenly you have to worry about whether you’ll get repeating or correlated sequences in the parallel processes.

We used Python’s built-in random number generator in [Example 9-1](#), and we’ll use the numpy random number generator in [Example 9-7](#) in the next section. In both cases, the random number generators are seeded in their forked process. For the Python random example, the seeding is handled internally by multiprocessing—if during a fork it sees that random is in the namespace, it will force a call to seed the generators in each of the new processes.



Set the numpy seed when parallelizing your function calls. In the forthcoming numpy example, we have to explicitly set the random number seed. If you forget to seed the random number sequence with numpy, each of your forked processes will generate an identical sequence of random numbers—it’ll appear to be working as you wanted it to, but behind the scenes each parallel process will evolve with identical results!

If you care about the quality of the random numbers used in the parallel processes, we urge you to research this topic. *Probably* the numpy and Python random number generators are good enough, but if significant outcomes depend on the quality of the random sequences (e.g., for medical or financial systems), then you must read up on this area.

In Python 3, the [Mersenne Twister algorithm](#) is used—it has a long period, so the sequence won’t repeat for a long time. It is heavily tested, as it is used in other languages, and it is thread-safe. It is probably not suitable for cryptographic purposes.

Using numpy

In this section, we switch to using numpy. Our dart-throwing problem is ideal for numpy vectorized operations—we generate the same estimate 25 times faster than the previous Python examples.

The main reason that numpy is faster than pure Python when solving the same problem is that numpy is creating and manipulating the same object types at a very low level in contiguous blocks of RAM, rather than creating many higher-level Python objects that each require individual management and addressing.

As numpy is far more cache friendly, we’ll also get a small speed boost when using the four hyperthreads. We didn’t get this in the pure Python version, as caches aren’t used efficiently by larger Python objects.

In [Figure 9-8](#), we see three scenarios:

- No use of multiprocessing (named “Serial”)
- Using threads

- Using processes

The serial and single-worker versions execute at the same speed—there’s no overhead to using threads with numpy (and with only one worker, there’s also no gain).

When using multiple processes, we see a classic 100% utilization of each additional CPU. The result mirrors the plots shown in Figures 9-3, 9-4, 9-5, and 9-6, but the code is much faster using numpy.

Interestingly, the threaded version runs *faster* with more threads. As discussed on the [SciPy wiki](#), by working outside the GIL, numpy can achieve some level of additional speedup around threads.

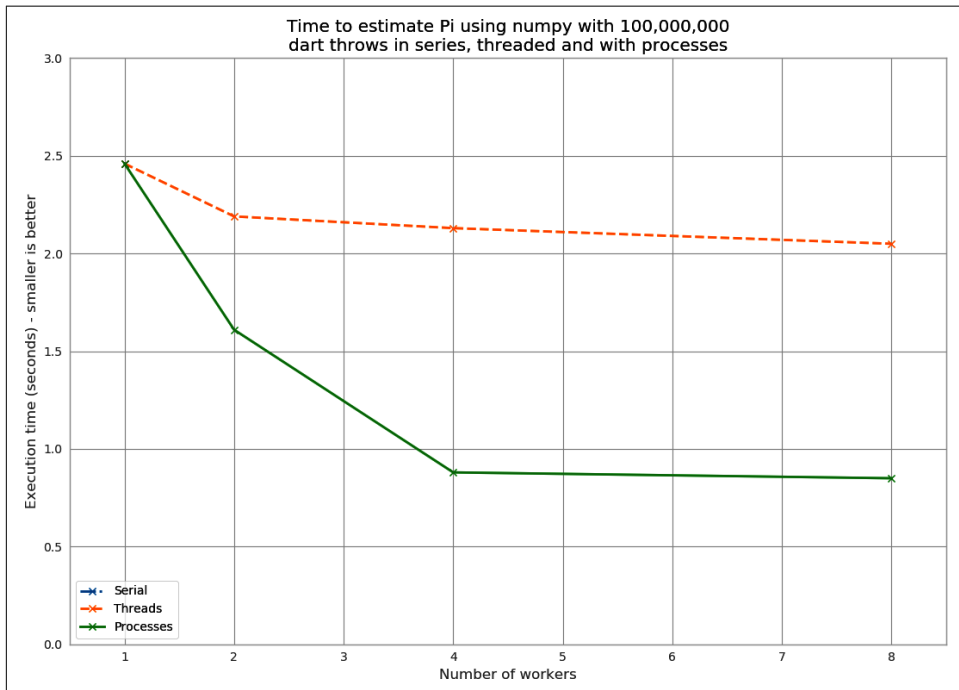


Figure 9-8. Working in series, with threads, and with processes using numpy

Using processes gives us a predictable speedup, just as it did in the pure Python example. A second CPU doubles the speed, and using four CPUs quadruples the speed.

Example 9-7 shows the vectorized form of our code. Note that the random number generator is seeded when this function is called. For the threaded version, this isn’t necessary, as each thread shares the same random number generator and they access it in series. For the process version, as each new process is a fork, all the forked

versions will share the *same state*. This means the random number calls in each will return the same sequence!



Remember to call `seed()` per process with `numpy` to ensure that each of the forked processes generates a unique sequence of random numbers, as a random source is used to set the seed for each call. Look back at “Random Numbers in Parallel Systems” on page 263 for some notes about the dangers of parallelized random number sequences.

Example 9-7. Estimating pi using numpy

```
def estimate_nbr_points_in_quarter_circle(nbr_samples):
    """Estimate pi using vectorized numpy arrays"""
    np.random.seed() # remember to set the seed per process
    xs = np.random.uniform(0, 1, nbr_samples)
    ys = np.random.uniform(0, 1, nbr_samples)
    estimate_inside_quarter_unit_circle = (xs * xs + ys * ys) <= 1
    nbr_trials_in_quarter_unit_circle = np.sum(estimate_inside_quarter_unit_circle)
    return nbr_trials_in_quarter_unit_circle
```

A short code analysis shows that the calls to `random` run a little slower on this machine when executed with multiple threads, and the call to `(xs * xs + ys * ys) <= 1` parallelizes well. Calls to the random number generator are GIL-bound, as the internal state variable is a Python object.

The process to understand this was basic but reliable:

1. Comment out all of the `numpy` lines, and run with *no* threads using the serial version. Run several times and record the execution times using `time.time()` in `__main__`.
2. Add a line back (we added `xs = np.random.uniform(...)` first) and run several times, again recording completion times.
3. Add the next line back (now adding `ys = ...`), run again, and record completion time.
4. Repeat, including the `nbr_trials_in_quarter_unit_circle = np.sum(...)` line.
5. Repeat this process again, but this time with four threads. Repeat line by line.
6. Compare the difference in runtime at each step for no threads and four threads.

Because we’re running code in parallel, it becomes harder to use tools like `line_profiler` or `cProfile`. Recording the raw runtimes and observing the differences in behavior with different configurations takes patience but gives solid evidence from which to draw conclusions.



If you want to understand the serial behavior of the `uniform` call, take a look at the `mtrand` code in the numpy source and follow the call to `def uniform` in `mtrand.pyx`. This is a useful exercise if you haven't looked at the numpy source code before.

The libraries used when building numpy are important for some of the parallelization opportunities. Depending on the underlying libraries used when building numpy (e.g., whether Intel's Math Kernel Library or OpenBLAS were included or not), you'll see different speedup behavior.

You can check your numpy configuration using `numpy.show_config()`. Stack Overflow has some [example timings](#) if you're curious about the possibilities. Only some numpy calls will benefit from parallelization by external libraries.

Finding Prime Numbers

Next, we'll look at testing for prime numbers over a large number range. This is a different problem from estimating pi, as the workload varies depending on your location in the number range, and each individual number's check has an unpredictable complexity. We can create a serial routine that checks for primality and then pass sets of possible factors to each process for checking. This problem is embarrassingly parallel, which means there is no state that needs to be shared.

The multiprocessing module makes it easy to control the workload, so we shall investigate how we can tune the work queue to use (and misuse!) our computing resources, and we will explore an easy way to use our resources slightly more efficiently. This means we'll be looking at *load balancing* to try to efficiently distribute our varying-complexity tasks to our fixed set of resources.

We'll use an algorithm that is slightly removed from the one earlier in the book (see [“Idealized Computing Versus the Python Virtual Machine” on page 10](#)); it exits early if we have an even number—see [Example 9-8](#).

Example 9-8. Finding prime numbers using Python

```
def check_prime(n):
    if n % 2 == 0:
        return False
    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True
```

How much variety in the workload do we see when testing for a prime with this approach? **Figure 9-9** shows the increasing time cost to check for primality as the possibly prime n increases from 10,000 to 1,000,000.

Most numbers are nonprime; they're drawn with a dot. Some can be cheap to check for, while others require the checking of many factors. Primes are drawn with an x and form the thick darker band; they're the most expensive to check for. The time cost of checking a number increases as n increases, as the range of possible factors to check increases with the square root of n . The sequence of primes is not predictable, so we can't determine the expected cost of a range of numbers (we could estimate it, but we can't be sure of its complexity).

For the figure, we test each n two hundred times and take the fastest result to remove jitter from the results. If we took only one result, we'd see wide variance in timing that would be caused by system load from other processes; by taking many readings and keeping the fastest, we get to see the expected best-case timing.

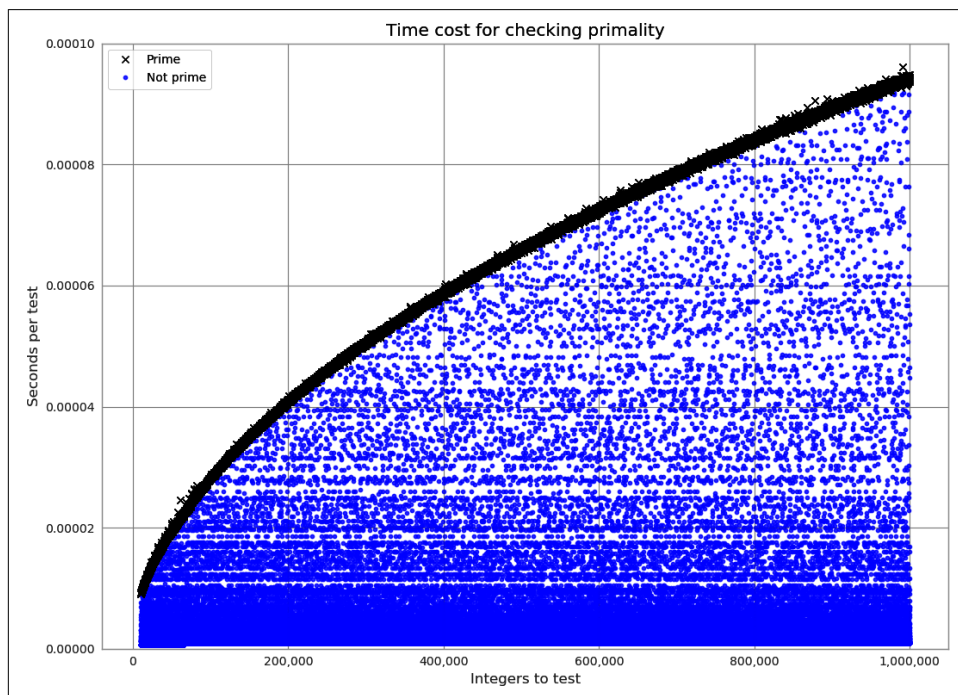


Figure 9-9. Time required to check primality as n increases

When we distribute work to a Pool of processes, we can specify how much work is passed to each worker. We could divide all of the work evenly and aim for one pass, or we could make many chunks of work and pass them out whenever a CPU is free. This is controlled using the `chunksiz` parameter. Larger chunks of work mean less

communication overhead, while smaller chunks of work mean more control over how resources are allocated.

For our prime finder, a single piece of work is a number n that is checked by `check_prime`. A chunksize of 10 would mean that each process handles a list of 10 integers, one list at a time.

In [Figure 9-10](#), we can see the effect of varying the chunksize from 1 (every job is a single piece of work) to 64 (every job is a list of 64 numbers). Although having many tiny jobs gives us the greatest flexibility, it also imposes the greatest communication overhead. All four CPUs will be utilized efficiently, but the communication pipe will become a bottleneck as each job and result is passed through this single channel. If we double the chunksize to 2, our task gets solved twice as quickly, as we have less contention on the communication pipe. We might naively assume that by increasing the chunksize, we will continue to improve the execution time. However, as you can see in the figure, we will again come to a point of diminishing returns.

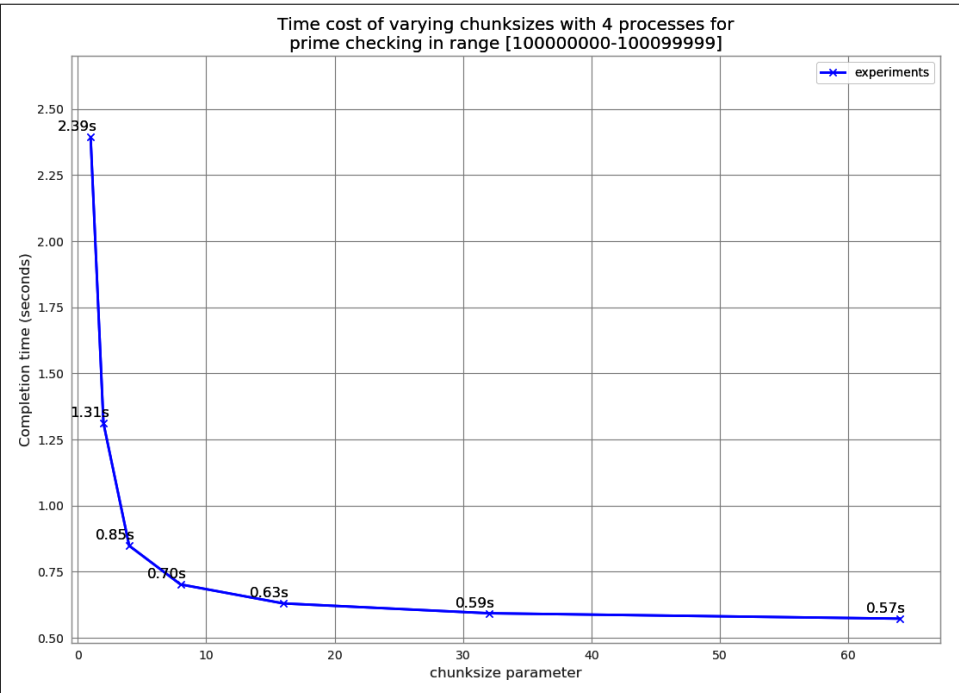


Figure 9-10. Choosing a sensible chunksize value

We can continue to increase the chunksize until we start to see a worsening of behavior. In [Figure 9-11](#), we expand the range of chunksizes, making them not just tiny but also huge. At the larger end of the scale, the worst result shown is 1.08 seconds, where we've asked for chunksize to be 50000—this means our 100,000 items

are divided into two work chunks, leaving two CPUs idle for that entire pass. With a chunksize of 10000 items, we are creating ten chunks of work; this means that four chunks of work will run twice in parallel, followed by the two remaining chunks. This leaves two CPUs idle in the third round of work, which is an inefficient usage of resources.

An optimal solution in this case is to divide the total number of jobs by the number of CPUs. This is the default behavior in multiprocessing, shown as the “default” blue dot in the figure.

As a general rule, the default behavior is sensible; tune it only if you expect to see a real gain, and definitely confirm your hypothesis against the default behavior.

Unlike the Monte Carlo pi problem, our prime testing calculation has varying complexity—sometimes a job exits quickly (an even number is detected the fastest), and sometimes the number is large and a prime (this takes a much longer time to check).

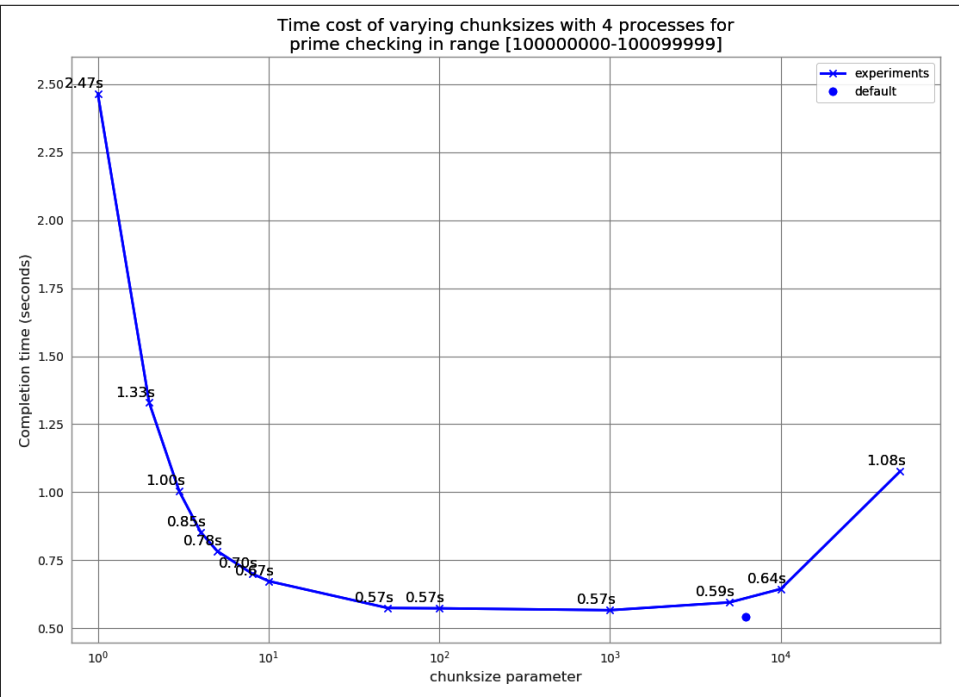


Figure 9-11. Choosing a sensible chunksize value (continued)

What happens if we randomize our job sequence? For this problem, we squeeze out a 2% performance gain, as you can see in Figure 9-12. By randomizing, we reduce the likelihood of the final job in the sequence taking longer than the others, leaving all but one CPU active.

As our earlier example using a chunksize of 10000 demonstrated, misaligning the workload with the number of available resources leads to inefficiency. In that case, we created three rounds of work: the first two rounds used 100% of the resources, and the last round used only 50%.

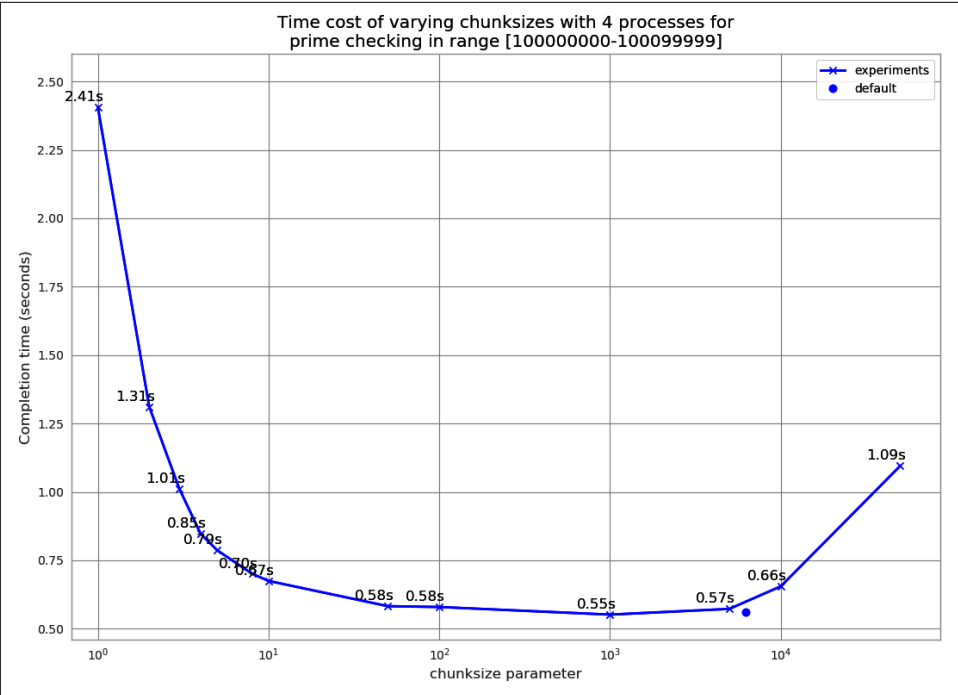


Figure 9-12. Randomizing the job sequence

Figure 9-13 shows the odd effect that occurs when we misalign the number of chunks of work against the number of processors. Mismatches will underutilize the available resources. The slowest overall runtime occurs when only one chunk of work is created: this leaves three unutilized. Two work chunks leave two CPUs unutilized, and so on; only when we have four work chunks are we using all of our resources. But if we add a fifth work chunk, we're underutilizing our resources again—four CPUs will work on their chunks, and then one CPU will run to calculate the fifth chunk.

As we increase the number of chunks of work, we see that the inefficiencies decrease—the difference in runtime between 29 and 32 work chunks is approximately 0.03 seconds. The general rule is to make lots of small jobs for efficient resource utilization if your jobs have varying runtimes.

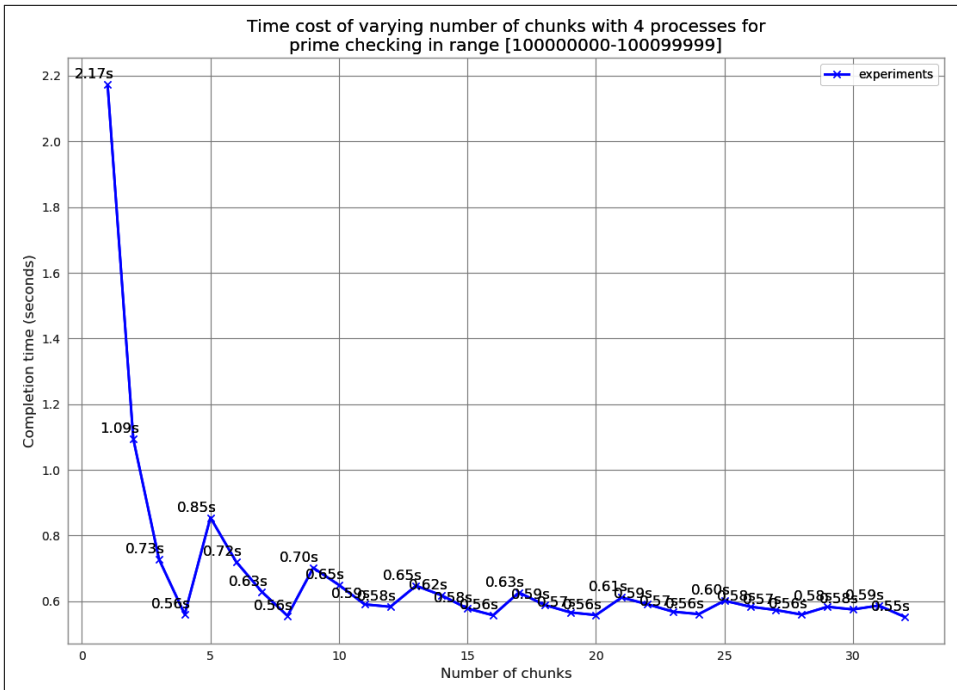


Figure 9-13. The danger of choosing an inappropriate number of chunks

Here are some strategies for efficiently using multiprocessing for embarrassingly parallel problems:

- Split your jobs into independent units of work.
- If your workers take varying amounts of time, consider randomizing the sequence of work (another example would be for processing variable-sized files).
- Sorting your work queue so that the slowest jobs go first may be an equally useful strategy.
- Use the default chunksize unless you have verified reasons for adjusting it.
- Align the number of jobs with the number of physical CPUs. (Again, the default chunksize takes care of this for you, although it will use any hyperthreads by default, which may not offer any additional gain.)

Note that by default `multiprocessing` will see hyperthreads as additional CPUs. This means that on Ian's laptop, it will allocate eight processes when only four will really be running at 100% speed. The additional four processes could be taking up valuable RAM while barely offering any additional speed gain.

With a Pool, we can split up a chunk of predefined work up front among the available CPUs. This is less helpful if we have dynamic workloads, though, and particularly if we have workloads that arrive over time. For this sort of workload, we might want to use a Queue, introduced in the next section.

Queues of Work

`multiprocessing.Queue` objects give us nonpersistent queues that can send any pickleable Python objects between processes. They carry an overhead, as each object must be pickled to be sent and then unpickled in the consumer (along with some locking operations). In the following example, we'll see that this cost is not negligible. However, if your workers are processing larger jobs, the communication overhead is probably acceptable.

Working with the queues is fairly easy. In this example, we'll check for primes by consuming a list of candidate numbers and posting confirmed primes back to a `definite_primes_queue`. We'll run this with one, two, four, and eight processes and confirm that the latter three approaches all take longer than just running a single process that checks the same range.

A Queue gives us the ability to perform lots of interprocess communication using native Python objects. This can be useful if you're passing around objects with lots of state. Since the Queue lacks persistence, though, you probably don't want to use queues for jobs that might require robustness in the face of failure (e.g., if you lose power or a hard drive gets corrupted).

Example 9-9 shows the `check_prime` function. We're already familiar with the basic primality test. We run in an infinite loop, blocking (waiting until work is available) on `possible_primes_queue.get()` to consume an item from the queue. Only one process can get an item at a time, as the Queue object takes care of synchronizing the accesses. If there's no work in the queue, the `.get()` blocks until a task is available. When primes are found, they are put back on the `definite_primes_queue` for consumption by the parent process.

Example 9-9. Using two queues for interprocess communication (IPC)

```
FLAG_ALL_DONE = b"WORK_FINISHED"
FLAG_WORKER_FINISHED_PROCESSING = b"WORKER_FINISHED_PROCESSING"

def check_prime(possible_primes_queue, definite_primes_queue):
    while True:
        n = possible_primes_queue.get()
        if n == FLAG_ALL_DONE:
            # flag that our results have all been pushed to the results queue
            definite_primes_queue.put(FLAG_WORKER_FINISHED_PROCESSING)
```

```

        break
    else:
        if n % 2 == 0:
            continue
        for i in range(3, int(math.sqrt(n)) + 1, 2):
            if n % i == 0:
                break
        else:
            definite_primes_queue.put(n)

```

We define two flags: one is fed by the parent process as a poison pill to indicate that no more work is available, while the second is fed by the worker to confirm that it has seen the poison pill and has closed itself down. The first poison pill is also known as a *sentinel*, as it guarantees the termination of the processing loop.

When dealing with queues of work and remote workers, it can be helpful to use flags like these to record that the poison pills were sent and to check that responses were sent from the children in a sensible time window, indicating that they are shutting down. We don't handle that process here, but adding some timekeeping is a fairly simple addition to the code. The receipt of these flags can be logged or printed during debugging.

The Queue objects are created out of a Manager in [Example 9-10](#). We'll use the familiar process of building a list of Process objects that each contain a forked process. The two queues are sent as arguments, and multiprocessing handles their synchronization. Having started the new processes, we hand a list of jobs to the possible_primes_queue and end with one poison pill per process. The jobs will be consumed in FIFO order, leaving the poison pills for last. In check_prime we use a blocking .get(), as the new processes will have to wait for work to appear in the queue. Since we use flags, we could add some work, deal with the results, and then iterate by adding more work, and signal the end of life of the workers by adding the poison pills later.

Example 9-10. Building two queues for IPC

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Project description")
    parser.add_argument(
        "nbr_workers", type=int, help="Number of workers e.g. 1, 2, 4, 8"
    )
    args = parser.parse_args()
    primes = []

    manager = multiprocessing.Manager()
    possible_primes_queue = manager.Queue()
    definite_primes_queue = manager.Queue()

```

```

pool = Pool(processes=args.nbr_workers)
processes = []
for _ in range(args.nbr_workers):
    p = multiprocessing.Process(
        target=check_prime, args=(possible_primes_queue,
                                   definite_primes_queue)
    )
    processes.append(p)
    p.start()

t1 = time.time()
number_range = range(100_000_000, 101_000_000)

# add jobs to the inbound work queue
for possible_prime in number_range:
    possible_primes_queue.put(possible_prime)

# add poison pills to stop the remote workers
for n in range(args.nbr_workers):
    possible_primes_queue.put(FLAGS_ALL_DONE)

```

To consume the results, we start another infinite loop in [Example 9-11](#), using a blocking `.get()` on the `definite_primes_queue`. If the finished-processing flag is found, we take a count of the number of processes that have signaled their exit. If not, we have a new prime, and we add this to the primes list. We exit the infinite loop when all of our processes have signaled their exit.

Example 9-11. Using two queues for IPC

```

processors Indicating they have finished = 0
while True:
    new_result = definite_primes_queue.get() # block while waiting for results
    if new_result == FLAG_WORKER_FINISHED_PROCESSING:
        processors Indicating they have finished += 1
        if processors Indicating they have finished == args.nbr_workers:
            break
    else:
        primes.append(new_result)
assert processors Indicating they have finished == args.nbr_workers

print("Took:", time.time() - t1)
print(len(primes), primes[:10], primes[-10:])

```

There is quite an overhead to using a Queue, due to the pickling and synchronization. As you can see in [Figure 9-14](#), using a Queue-less single-process solution is significantly faster than using two or more processes. The reason in this case is because our workload is very light—the communication cost dominates the overall time for this task. With Queues, two processes complete this example a little faster than one process, while four and eight processes are both slower.

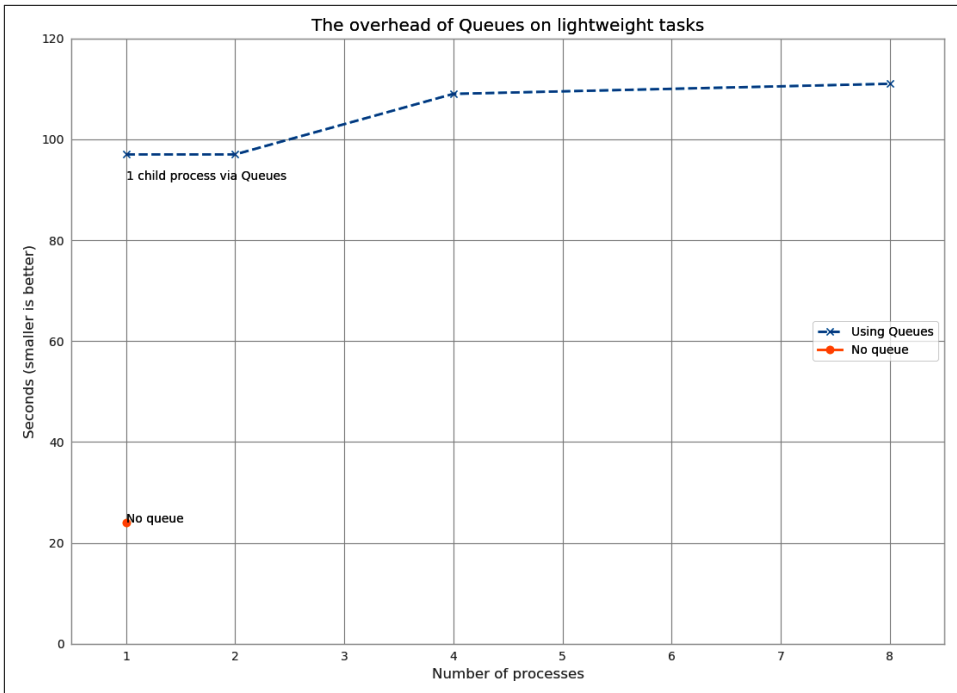


Figure 9-14. Cost of using Queue objects

If your task has a long completion time (at least a sizable fraction of a second) with a small amount of communication, a Queue approach might be the right answer. You will have to verify whether the communication cost makes this approach useful enough.

You might wonder what happens if we remove the redundant half of the job queue (all the even numbers—these are rejected very quickly in `check_prime`). Halving the size of the input queue halves our execution time in each case, but it still doesn't beat the single-process non-Queue example! This helps to illustrate that the communication cost is the dominating factor in this problem.

Asynchronously adding jobs to the Queue

By adding a Thread into the main process, we can feed jobs asynchronously into the `possible_primes_queue`. In [Example 9-12](#), we define a `feed_new_jobs` function: it performs the same job as the job setup routine that we had in `__main__` before, but it does it in a separate thread.

Example 9-12. Asynchronous job-feeding function

```
def feed_new_jobs(number_range, possible_primes_queue, nbr_poison_pills):
    for possible_prime in number_range:
        possible_primes_queue.put(possible_prime)
    # add poison pills to stop the remote workers
    for n in range(nbr_poison_pills):
        possible_primes_queue.put(FLAG_ALL_DONE)
```

Now, in [Example 9-13](#), our `__main__` will set up the Thread using the `possible_primes_queue` and then move on to the result-collection phase *before* any work has been issued. The asynchronous job feeder could consume work from external sources (e.g., from a database or I/O-bound communication) while the `__main__` thread handles each processed result. This means that the input sequence and output sequence do not need to be created in advance; they can both be handled on the fly.

Example 9-13. Using a thread to set up an asynchronous job feeder

```
if __name__ == "__main__":
    primes = []
    manager = multiprocessing.Manager()
    possible_primes_queue = manager.Queue()

    ...

    import threading
    thrd = threading.Thread(target=feed_new_jobs,
                           args=(number_range,
                                possible_primes_queue,
                                NBR_PROCESSES))

    thrd.start()

    # deal with the results
```

If you want robust asynchronous systems, you should almost certainly look to using `asyncio` or an external library such as `tornado`. For a full discussion of these approaches, check out [Chapter 8](#). The examples we’ve looked at here will get you started, but pragmatically they are more useful for very simple systems and education than for production systems.

Be *very aware* that asynchronous systems require a special level of patience—you will end up tearing out your hair while you are debugging. We suggest the following:

- Applying the “Keep It Simple, Stupid” principle
- Avoiding asynchronous self-contained systems (like our example) if possible, as they will grow in complexity and quickly become hard to maintain

- Using mature libraries like `gevent` (described in the previous chapter) that give you tried-and-tested approaches to dealing with certain problem sets

Furthermore, we strongly suggest using an external queue system that gives you external visibility on the state of the queues (e.g., NSQ, discussed in “[NSQ for Robust Production Clustering](#)” on page 326; ZeroMQ; or Celery). This requires more thought but is likely to save you time because of increased debug efficiency and better system visibility for production systems.



Consider using a task graph for resilience. Data science tasks requiring long-running queues are frequently served well by specifying pipelines of work in acyclic graphs. Two strong libraries are [Airflow](#) and [Luigi](#). These are very frequently used in industrial settings and enable arbitrary task chaining, online monitoring, and flexible scaling.

Verifying Primes Using Interprocess Communication

Prime numbers are numbers that have no factor other than themselves and 1. It stands to reason that the most common factor is 2 (every even number cannot be a prime). After that, the low prime numbers (e.g., 3, 5, 7) become common factors of larger nonprimes (e.g., 9, 15, and 21, respectively).

Let’s say that we are given a large number and are asked to verify if it is prime. We will probably have a large space of factors to search. [Figure 9-15](#) shows the frequency of each factor for nonprimes up to 10,000,000. Low factors are far more likely to occur than high factors, but there’s no predictable pattern.

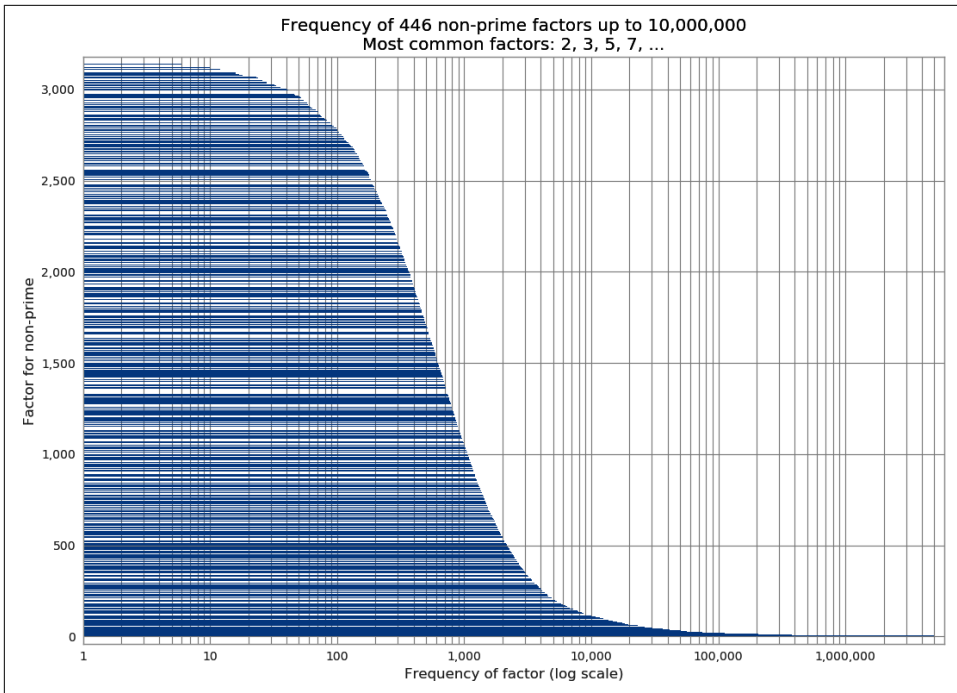


Figure 9-15. The frequency of factors of nonprimes

Let's define a new problem—suppose we have a *small* set of numbers, and our task is to efficiently use our CPU resources to figure out if each number is a prime, one number at a time. Possibly we'll have just one large number to test. It no longer makes sense to use one CPU to do the check; we want to coordinate the work across many CPUs.

For this section we'll look at some larger numbers, one with 15 digits and four with 18 digits:

- Small nonprime: 112,272,535,095,295
- Large nonprime 1: 100,109,100,129,100,369
- Large nonprime 2: 100,109,100,129,101,027
- Prime 1: 100,109,100,129,100,151
- Prime 2: 100,109,100,129,162,907

By using a smaller nonprime and some larger nonprimes, we get to verify that our chosen process not only is faster at checking for primes but also is not getting slower at checking nonprimes. We'll assume that we don't know the size or type of numbers that we're being given, so we want the fastest possible result for all our use cases.



If you own the previous edition of the book, you might be surprised to see that these runtimes with CPython 3.7 are *slightly slower* than the CPython 2.7 runtimes in the last edition, which ran on a slower laptop. The code here is one edge case where Python 3.x is currently slower than CPython 2.7. This code depends on integer operations; CPython 2.7 had system integers mixed with “long” integers (which can store arbitrarily sized numbers but at a cost in speed). CPython 3.x uses only “long” integers for all operations. This implementation is optimized but is still slower in some cases compared to the old (and more complicated) implementation.

We never have to worry about which “kind” of integer is being used, and in CPython 3.7 we take a small speed hit as a consequence. This is a microbenchmark that is incredibly unlikely to affect your own code, as CPython 3.x is faster than CPython 2.x in so many other ways. Our recommendation is not to worry about this, unless you depend on integer operations for most of your execution time—and in that case, we’d strongly suggest you look at PyPy, which doesn’t suffer from this slowdown.

Cooperation comes at a cost—the cost of synchronizing data and checking the shared data can be quite high. We’ll work through several approaches here that can be used in different ways for task coordination.

Note that we’re *not* covering the somewhat specialized message passing interface (MPI) here; we’re looking at batteries-included modules and Redis (which is very common). If you want to use MPI, we assume you already know what you’re doing. The [MPI4PY project](#) would be a good place to start. It is an ideal technology if you want to control latency when lots of processes are collaborating, whether you have one or many machines.

For the following runs, each test is performed 20 times, and the minimum time is taken to show the fastest speed that is possible for that method. In these examples we’re using various techniques to share a flag (often as 1 byte). We could use a basic object like a Lock, but then we’d be able to share only 1 bit of state. We’re choosing to show you how to share a primitive type so that more expressive state sharing is possible (even though we don’t need a more expressive state for this example).

We must emphasize that sharing state tends to make things *complicated*—you can easily end up in another hair-pulling state. Be careful and try to keep things as simple as they can be. It might be the case that less efficient resource usage is trumped by developer time spent on other challenges.

First we’ll discuss the results and then we’ll work through the code.

Figure 9-16 shows the first approaches to trying to use interprocess communication to test for primality faster. The benchmark is the serial version, which does not use any interprocess communication; each attempt to speed up our code must at least be faster than this.

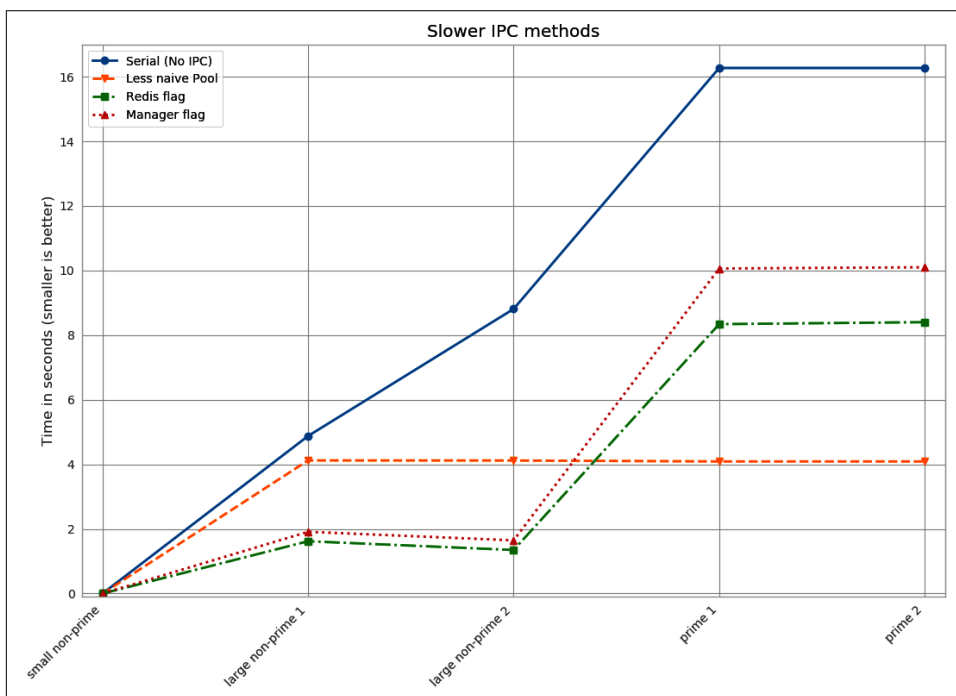


Figure 9-16. The slower ways to use IPC to validate primality

The Less Naive Pool version has a predictable (and good) speed. It is good enough to be rather hard to beat. Don't overlook the obvious in your search for high-speed solutions—sometimes a dumb and good-enough solution is all you need.

The approach for the Less Naive Pool solution is to take our number under test, divide its possible-factor range evenly among the available CPUs, and then push the work out to each CPU. If any CPU finds a factor, it will exit early, but it won't communicate this fact; the other CPUs will continue to work through their part of the range. This means for an 18-digit number (our four larger examples), the search time is the same whether it is prime or nonprime.

The Redis and Manager solutions are slower when it comes to testing a larger number of factors for primality because of the communication overhead. They use a shared flag to indicate that a factor has been found and the search should be called off.

Redis lets you share state not just with other Python processes but also with other tools and other machines, and even to expose that state over a web-browser interface (which might be useful for remote monitoring). The `Manager` is a part of `multiprocessing`; it provides a high-level synchronized set of Python objects (including primitives, the `list`, and the `dict`).

For the larger nonprime cases, although there is a cost to checking the shared flag, this is dwarfed by the savings in search time gained by signaling early that a factor has been found.

For the prime cases, though, there is no way to exit early, as no factor will be found, so the cost of checking the shared flag will become the dominating cost.



A little bit of thought is often enough. Here we explore various IPC-based solutions to making the prime-validation task faster. In terms of “minutes of typing” versus “gains made,” the first step—introducing naive parallel processing—gave us the largest win for the smallest effort. Subsequent gains took a lot of extra experimentation. Always think about the ultimate run-time, especially for ad hoc tasks. Sometimes it is just easier to let a loop run all weekend for a one-off task than to optimize the code so it runs quicker.

Figure 9-17 shows that we can get a considerably faster result with a bit of effort. The Less Naive Pool result is still our benchmark, but the `RawValue` and `MMap` (memory map) results are much faster than the previous `Redis` and `Manager` results. The real magic comes from taking the fastest solution and performing some less-obvious code manipulations to make a near-optimal `MMap` solution—this final version is faster than the Less Naive Pool solution for nonprimes and almost as fast for primes.

In the following sections, we’ll work through various ways of using IPC in Python to solve our cooperative search problem. We hope you’ll see that IPC is fairly easy but generally comes with a cost.

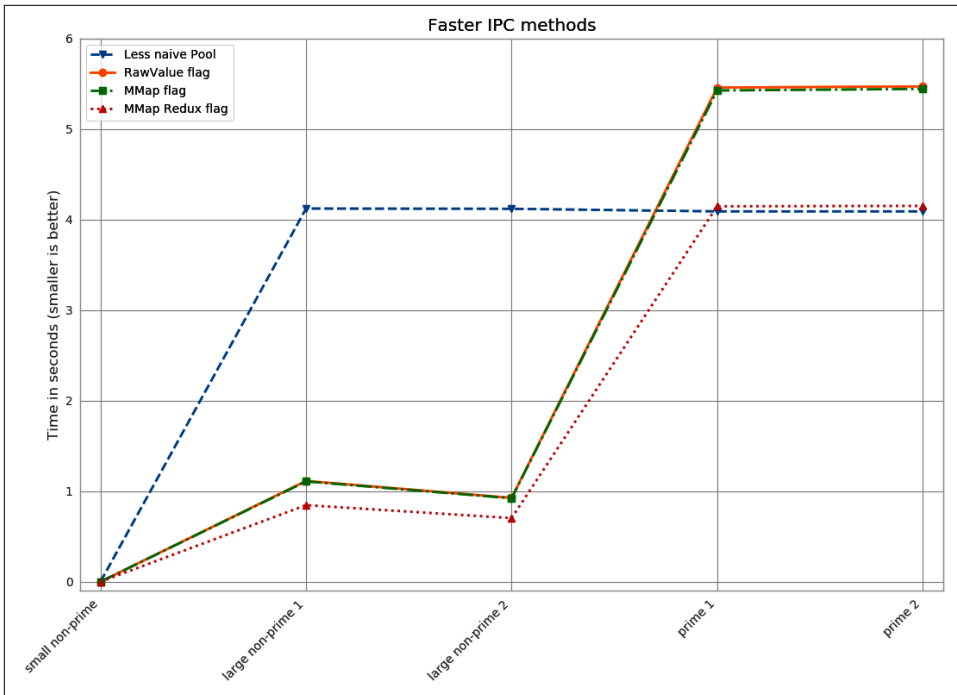


Figure 9-17. The faster ways to use IPC to validate primality

Serial Solution

We'll start with the same serial factor-checking code that we used before, shown again in [Example 9-14](#). As noted earlier, for any nonprime with a large factor, we could more efficiently search the space of factors in parallel. Still, a serial sweep will give us a sensible baseline to work from.

Example 9-14. Serial verification

```
def check_prime(n):
    if n % 2 == 0:
        return False
    from_i = 3
    to_i = math.sqrt(n) + 1
    for i in range(from_i, int(to_i), 2):
        if n % i == 0:
            return False
    return True
```

Naive Pool Solution

The Naive Pool solution works with a `multiprocessing.Pool`, similar to what we saw in “Finding Prime Numbers” on page 267 and “Estimating Pi Using Processes and Threads” on page 251 with four forked processes. We have a number to test for primality, and we divide the range of possible factors into four tuples of subranges and send these into the `Pool`.

In [Example 9-15](#), we use a new method, `create_range.create` (which we won’t show—it’s quite boring), that splits the work space into equal-sized regions. Each item in `ranges_to_check` is a pair of lower and upper bounds to search between. For the first 18-digit nonprime (100,109,100,129,100,369), with four processes we’ll have the factor ranges `ranges_to_check == [(3, 79_100_057), (79_100_057, 158_200_111), (158_200_111, 237_300_165), (237_300_165, 316_400_222)]` (where 316,400,222 is the square root of 100,109,100,129,100,369 plus 1). In `__main__` we first establish a `Pool`; `check_prime` then splits the `ranges_to_check` for each possibly prime number `n` via a `map`. If the result is `False`, we have found a factor and we do not have a prime.

Example 9-15. Naive Pool solution

```
def check_prime(n, pool, nbr_processes):
    from_i = 3
    to_i = int(math.sqrt(n)) + 1
    ranges_to_check = create_range.create(from_i, to_i, nbr_processes)
    ranges_to_check = zip(len(ranges_to_check) * [n], ranges_to_check)
    assert len(ranges_to_check) == nbr_processes
    results = pool.map(check_prime_in_range, ranges_to_check)
    if False in results:
        return False
    return True

if __name__ == "__main__":
    NBR_PROCESSES = 4
    pool = Pool(processes=NBR_PROCESSES)
    ...
```

We modify the previous `check_prime` in [Example 9-16](#) to take a lower and upper bound for the range to check. There’s no value in passing a complete list of possible factors to check, so we save time and memory by passing just two numbers that define our range.

Example 9-16. check_prime_in_range

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
```

```

if n % 2 == 0:
    return False
assert from_i % 2 != 0
for i in range(from_i, int(to_i), 2):
    if n % i == 0:
        return False
return True

```

For the “small nonprime” case, the verification time via the Pool is 0.1 seconds, a significantly longer time than the original 0.000002 seconds in the Serial solution. Despite this one worse result, the overall result is a speedup across the board. We could perhaps accept that one slower result isn’t a problem—but what if we might get lots of smaller nonprimes to check? It turns out we can avoid this slowdown; we’ll see that next with the Less Naive Pool solution.

A Less Naive Pool Solution

The previous solution was inefficient at validating the smaller nonprime. For any smaller (fewer than 18 digits) nonprime, it is likely to be slower than the serial method, because of the overhead of sending out partitioned work and not knowing if a very small factor (which is a more likely factor) will be found. If a small factor is found, the process will still have to wait for the other larger factor searches to complete.

We could start to signal between the processes that a small factor has been found, but since this happens so frequently, it will add a lot of communication overhead. The solution presented in [Example 9-17](#) is a more pragmatic approach—a serial check is performed quickly for likely small factors, and if none are found, then a parallel search is started. Combining a serial precheck before launching a relatively more expensive parallel operation is a common approach to avoiding some of the costs of parallel computing.

Example 9-17. Improving the Naive Pool solution for the small-nonprime case

```

def check_prime(n, pool, nbr_processes):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = 21
    if not check_prime_in_range((n, (from_i, to_i))):
        return False

    # continue to check for larger factors in parallel
    from_i = to_i
    to_i = int(math.sqrt(n)) + 1
    ranges_to_check = create_range.create(from_i, to_i, nbr_processes)
    ranges_to_check = zip(len(ranges_to_check) * [n], ranges_to_check)
    assert len(ranges_to_check) == nbr_processes

```

```

results = pool.map(check_prime_in_range, ranges_to_check)
if False in results:
    return False
return True

```

The speed of this solution is equal to or better than that of the original serial search for each of our test numbers. This is our new benchmark.

Importantly, this Pool approach gives us an optimal case for the prime-checking situation. If we have a prime, there's no way to exit early; we have to manually check all possible factors before we can exit.

There's no faster way to check though these factors: any approach that adds complexity will have more instructions, so the check-all-factors case will cause the most instructions to be executed. See the various mmap solutions covered in “Using mmap as a Flag” on page 291 for a discussion on how to get as close to this current result for primes as possible.

Using Manager.Value as a Flag

The multiprocessing.Manager() lets us share higher-level Python objects between processes as managed shared objects; the lower-level objects are wrapped in proxy objects. The wrapping and safety have a speed cost but also offer great flexibility. You can share both lower-level objects (e.g., integers and floats) and lists and dictionaries.

In [Example 9-18](#), we create a Manager and then create a 1-byte (character) manager.Value(b"0", FLAG_CLEAR) flag. You could create any of the ctypes primitives (which are the same as the array.array primitives) if you wanted to share strings or numbers.

Note that FLAG_CLEAR and FLAG_SET are assigned a byte (b'0' and b'1', respectively). We chose to use the leading b to be very explicit (it might default to a Unicode or string object if left as an implicit string, depending on your environment and Python version).

Now we can flag across all of our processes that a factor has been found, so the search can be called off early. The difficulty is balancing the cost of reading the flag against the speed savings that is possible. Because the flag is synchronized, we don't want to check it too frequently—this adds more overhead.

Example 9-18. Passing a Manager.Value object as a flag

```

SERIAL_CHECK_CUTOFF = 21
CHECK_EVERY = 1000
FLAG_CLEAR = b'0'
FLAG_SET = b'1'
print("CHECK_EVERY", CHECK_EVERY)

```

```

if __name__ == "__main__":
    NBR_PROCESSES = 4
    manager = multiprocessing.Manager()
    value = manager.Value(b'c', FLAG_CLEAR) # 1-byte character
    ...

```

check_prime_in_range will now be aware of the shared flag, and the routine will be checking to see if a prime has been spotted by another process. Even though we've yet to begin the parallel search, we must clear the flag as shown in [Example 9-19](#) before we start the serial check. Having completed the serial check, if we haven't found a factor, we know that the flag must still be false.

Example 9-19. Clearing the flag with a Manager.Value

```

def check_prime(n, pool, nbr_processes, value):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = SERIAL_CHECK_CUTOFF
    value.value = FLAG_CLEAR
    if not check_prime_in_range((n, (from_i, to_i), value)):
        return False

    from_i = to_i
    ...

```

How frequently should we check the shared flag? Each check has a cost, both because we're adding more instructions to our tight inner loop and because checking requires a lock to be made on the shared variable, which adds more cost. The solution we've chosen is to check the flag every one thousand iterations. Every time we check, we look to see if value.value has been set to FLAG_SET, and if so, we exit the search. If in the search the process finds a factor, then it sets value.value = FLAG_SET and exits (see [Example 9-20](#)).

Example 9-20. Passing a Manager.Value object as a flag

```

def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i), value) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_every = CHECK_EVERY
    for i in range(from_i, int(to_i), 2):
        check_every -= 1
        if not check_every:
            if value.value == FLAG_SET:
                return False
            check_every = CHECK_EVERY

```

```

if n % i == 0:
    value.value = FLAG_SET
    return False
return True

```

The thousand-iteration check in this code is performed using a `check_every` local counter. It turns out that this approach, although readable, is suboptimal for speed. By the end of this section, we'll replace it with a less readable but significantly faster approach.

You might be curious about the total number of times we check for the shared flag. In the case of the two large primes, with four processes we check for the flag 316,405 times (we check it this many times in all of the following examples). Since each check has an overhead due to locking, this cost really adds up.

Using Redis as a Flag

Redis is a key/value in-memory storage engine. It provides its own locking and each operation is atomic, so we don't have to worry about using locks from inside Python (or from any other interfacing language).

By using Redis, we make the data storage language-agnostic—any language or tool with an interface to Redis can share data in a compatible way. You could share data between Python, Ruby, C++, and PHP equally easily. You can share data on the local machine or over a network; to share to other machines, all you need to do is change the Redis default of sharing only on `localhost`.

Redis lets you store the following:

- Lists of strings
- Sets of strings
- Sorted sets of strings
- Hashes of strings

Redis stores everything in RAM and snapshots to disk (optionally using journaling) and supports master/slave replication to a cluster of instances. One possibility with Redis is to use it to share a workload across a cluster, where other machines read and write state and Redis acts as a fast centralized data repository.

We can read and write a flag as a text string (all values in Redis are strings) in just the same way as we have been using Python flags previously. We create a `StrictRedis` interface as a global object, which talks to the external Redis server. We could create a new connection inside `check_prime_in_range`, but this is slower and can exhaust the limited number of Redis handles that are available.

We talk to the Redis server using a dictionary-like access. We can set a value using `rds[SOME_KEY] = SOME_VALUE` and read the string back using `rds[SOME_KEY]`.

Example 9-21 is very similar to the previous Manager example—we’re using Redis as a substitute for the local Manager. It comes with a similar access cost. You should note that Redis supports other (more complex) data structures; it is a powerful storage engine that we’re using just to share a flag for this example. We encourage you to familiarize yourself with its features.

Example 9-21. Using an external Redis server for our flag

```
FLAG_NAME = b'redis_primes_flag'
FLAG_CLEAR = b'0'
FLAG_SET = b'1'

rds = redis.StrictRedis()

def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_every = CHECK_EVERY
    for i in range(from_i, int(to_i), 2):
        check_every -= 1
        if not check_every:
            flag = rds[FLAG_NAME]
            if flag == FLAG_SET:
                return False
            check_every = CHECK_EVERY

    if n % i == 0:
        rds[FLAG_NAME] = FLAG_SET
        return False
    return True

def check_prime(n, pool, nbr_processes):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = SERIAL_CHECK_CUTOFF
    rds[FLAG_NAME] = FLAG_CLEAR
    if not check_prime_in_range((n, (from_i, to_i))):
        return False

    ...
    if False in results:
        return False
    return True
```

To confirm that the data is stored outside these Python instances, we can invoke `redis-cli` at the command line, as in [Example 9-22](#), and get the value stored in the key `redis_primes_flag`. You'll note that the returned item is a string (not an integer). All values returned from Redis are strings, so if you want to manipulate them in Python, you'll have to convert them to an appropriate datatype first.

Example 9-22. `redis-cli`

```
$ redis-cli
redis 127.0.0.1:6379> GET "redis_primes_flag"
"0"
```

One powerful argument in favor of the use of Redis for data sharing is that it lives outside the Python world—non-Python developers on your team will understand it, and many tools exist for it. They'll be able to look at its state while reading (but not necessarily running and debugging) your code and follow what's happening. From a team-velocity perspective, this might be a big win for you, despite the communication overhead of using Redis. While Redis is an additional dependency on your project, you should note that it is a very commonly deployed tool, and one that is well debugged and well understood. Consider it a powerful tool to add to your armory.

Redis has many configuration options. By default it uses a TCP interface (that's what we're using), although the benchmark documentation notes that sockets might be much faster. It also states that while TCP/IP lets you share data over a network between different types of OS, other configuration options are likely to be faster (but also are likely to limit your communication options):

When the server and client benchmark programs run on the same box, both the TCP/IP loopback and unix domain sockets can be used. It depends on the platform, but unix domain sockets can achieve around 50% more throughput than the TCP/IP loopback (on Linux for instance). The default behavior of `redis-benchmark` is to use the TCP/IP loopback. The performance benefit of unix domain sockets compared to TCP/IP loopback tends to decrease when pipelining is heavily used (i.e., long pipelines).

—[Redis documentation](#)

Redis is widely used in industry and is mature and well trusted. If you're not familiar with the tool, we strongly suggest you take a look at it; it has a place in your high performance toolkit.

Using `RawValue` as a Flag

`multiprocessing.RawValue` is a thin wrapper around a `ctypes` block of bytes. It lacks synchronization primitives, so there's little to get in our way in our search for

the fastest way to set a flag between processes. It will be almost as fast as the following `mmap` example (it is slower only because a few more instructions get in the way).

Again, we could use any `ctypes` primitive; there's also a `RawArray` option for sharing an array of primitive objects (which will behave similarly to `array.array`). `RawValue` avoids any locking—it is faster to use, but you don't get atomic operations.

Generally, if you avoid the synchronization that Python provides during IPC, you'll come unstuck (once again, back to that pulling-your-hair-out situation). *However*, in this problem it doesn't matter if one or more processes set the flag at the same time—the flag gets switched in only one direction, and every other time it is read, it is just to learn if the search can be called off.

Because we never reset the state of the flag during the parallel search, we don't need synchronization. Be aware that this may not apply to your problem. If you avoid synchronization, please make sure you are doing it for the right reasons.

If you want to do things like update a shared counter, look at the documentation for the `Value` and use a context manager with `value.get_lock()`, as the implicit locking on a `Value` doesn't allow for atomic operations.

This example looks very similar to the previous `Manager` example. The only difference is that in [Example 9-23](#) we create the `RawValue` as a one-character (byte) flag.

Example 9-23. Creating and passing a `RawValue`

```
if __name__ == "__main__":
    NBR_PROCESSES = 4
    value = multiprocessing.RawValue('b', FLAG_CLEAR) # 1-byte character
    pool = Pool(processes=NBR_PROCESSES)
    ...
```

The flexibility to use managed and raw values is a benefit of the clean design for data sharing in `multiprocessing`.

Using `mmap` as a Flag

Finally, we get to the fastest way of sharing bytes. [Example 9-24](#) shows a memory-mapped (shared memory) solution using the `mmap` module. The bytes in a shared memory block are not synchronized, and they come with very little overhead. They act like a file—in this case, they are a block of memory with a file-like interface. We have to seek to a location and read or write sequentially. Typically, `mmap` is used to give a short (memory-mapped) view into a larger file, but in our case, rather than specifying a file number as the first argument, we instead pass `-1` to indicate that we want an anonymous block of memory. We could also specify whether we want read-only or write-only access (we want both, which is the default).

Example 9-24. Using a shared memory flag via mmap

```
sh_mem = mmap.mmap(-1, 1) # memory map 1 byte as a flag
```

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_every = CHECK_EVERY
    for i in range(from_i, int(to_i), 2):
        check_every -= 1
        if not check_every:
            sh_mem.seek(0)
            flag = sh_mem.read_byte()
            if flag == FLAG_SET:
                return False
            check_every = CHECK_EVERY

    if n % i == 0:
        sh_mem.seek(0)
        sh_mem.write_byte(FLAG_SET)
        return False
    return True

def check_prime(n, pool, nbr_processes):
    # cheaply check high-probability set of possible factors
    from_i = 3
    to_i = SERIAL_CHECK_CUTOFF
    sh_mem.seek(0)
    sh_mem.write_byte(FLAG_CLEAR)
    if not check_prime_in_range((n, (from_i, to_i))):
        return False

    ...
    if False in results:
        return False
    return True
```

mmap supports a number of methods that can be used to move around in the file that it represents (including find, readline, and write). We are using it in the most basic way—we seek to the start of the memory block before each read or write, and since we’re sharing just 1 byte, we use read_byte and write_byte to be explicit.

There is no Python overhead for locking and no interpretation of the data; we’re dealing with bytes directly with the operating system, so this is our fastest communication method.

Using mmap as a Flag Redux

While the previous `mmap` result was the best overall, we couldn't help but think that we should be able to get back to the Naive Pool result for the most expensive case of having primes. The goal is to accept that there is no early exit from the inner loop and to minimize the cost of anything extraneous.

This section presents a slightly more complex solution. The same changes can be made to the other flag-based approaches we've seen, although this `mmap` result will still be fastest.

In our previous examples, we've used `CHECK EVERY`. This means we have the `check_next` local variable to track, decrement, and use in Boolean tests—and each operation adds a bit of extra time to every iteration. In the case of validating a large prime, this extra management overhead occurs over 300,000 times.

The first optimization, shown in [Example 9-25](#), is to realize that we can replace the decremented counter with a look-ahead value, and then we only have to do a Boolean comparison on the inner loop. This removes a decrement, which, because of Python's interpreted style, is quite slow. This optimization works in this test in CPython 3.7, but it is unlikely to offer any benefit in a smarter compiler (e.g., PyPy or Cython). This step saved 0.1 seconds when checking one of our large primes.

Example 9-25. Starting to optimize away our expensive logic

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    check_next = from_i + CHECK EVERY
    for i in range(from_i, int(to_i), 2):
        if check_next == i:
            sh_mem.seek(0)
            flag = sh_mem.read_byte()
            if flag == FLAG_SET:
                return False
            check_next += CHECK EVERY

        if n % i == 0:
            sh_mem.seek(0)
            sh_mem.write_byte(FLAG_SET)
            return False
    return True
```

We can also entirely replace the logic that the counter represents, as shown in [Example 9-26](#), by unrolling our loop into a two-stage process. First, the outer loop covers the expected range, but in steps, on `CHECK EVERY`. Second, a new inner loop

replaces the `check_every` logic—it checks the local range of factors and then finishes. This is equivalent to the `if not check_every:` test. We follow this with the previous `sh_mem` logic to check the early-exit flag.

Example 9-26. Optimizing away our expensive logic

```
def check_prime_in_range(n_from_i_to_i):
    (n, (from_i, to_i)) = n_from_i_to_i
    if n % 2 == 0:
        return False
    assert from_i % 2 != 0
    for outer_counter in range(from_i, int(to_i), CHECK_EVERY):
        upper_bound = min(int(to_i), outer_counter + CHECK_EVERY)
        for i in range(outer_counter, upper_bound, 2):
            if n % i == 0:
                sh_mem.seek(0)
                sh_mem.write_byte(FLAG_SET)
                return False
        sh_mem.seek(0)
        flag = sh_mem.read_byte()
        if flag == FLAG_SET:
            return False
    return True
```

The speed impact is dramatic. Our nonprime case improves even further, but more importantly, our prime-checking case is nearly as fast as the Less Naive Pool version (it is now just 0.1 seconds slower). Given that we’re doing a lot of extra work with interprocess communication, this is an interesting result. Do note, though, that it is specific to CPython and unlikely to offer any gains when run through a compiler.

In the last edition of the book we went even further with a final example that used loop unrolling and local references to global objects and eked out a further performance gain at the expense of readability. This example in Python 3 yields a minor slowdown, so we’ve removed it. We’re happy about this—fewer hoops needed jumping through to get the most performant example, and the preceding code is more likely to be supported correctly in a team than one that makes implementation-specific code changes.



These examples work just fine with PyPy, where they run around seven times faster than in CPython. Sometimes the better solution will be to investigate other runtimes rather than to go down rabbit holes with CPython.

Sharing numpy Data with multiprocessing

When working with large numpy arrays, you're bound to wonder if you can share the data for read and write access, without a copy, between processes. It is possible, though a little fiddly. We'd like to acknowledge Stack Overflow user *pv* for the inspiration for this demo.²



Do not use this method to re-create the behaviors of BLAS, MKL, Accelerate, and ATLAS. These libraries all have multithreading support in their primitives, and they likely are better-debugged than any new routine that you create. They can require some configuration to enable multithreading support, but it would be wise to see if these libraries can give you free speedups before you invest time (and lose time to debugging!) writing your own.

Sharing a large matrix between processes has several benefits:

- Only one copy means no wasted RAM.
- No time is wasted copying large blocks of RAM.
- You gain the possibility of sharing partial results between the processes.

Thinking back to the pi estimation demo using numpy in “Using numpy” on page 264, we had the problem that the random number generation was a serial process. Here, we can imagine forking processes that share one large array, each one using a differently seeded random number generator to fill in a section of the array with random numbers, and therefore completing the generation of a large random block faster than is possible with a single process.

To verify this, we modified the forthcoming demo to create a large random matrix ($10,000 \times 320,000$ elements) as a serial process and by splitting the matrix into four segments where `random` is called in parallel (in both cases, one row at a time). The serial process took 53 seconds, and the parallel version took 29 seconds. Refer back to “Random Numbers in Parallel Systems” on page 263 to understand some of the dangers of parallelized random number generation.

For the rest of this section, we'll use a simplified demo that illustrates the point while remaining easy to verify.

In Figure 9-18, you can see the output from `htop` on Ian's laptop. It shows four child processes of the parent (with PID 27628), where all five processes are sharing a single 10,000-by-320,000-element numpy array of doubles. One copy of this array costs

² See the [Stack Overflow topic](#).

25.6 GB, and the laptop has only 32 GB—you can see in `htop` by the process meters that the Mem reading shows a maximum of 31.1 GB RAM.

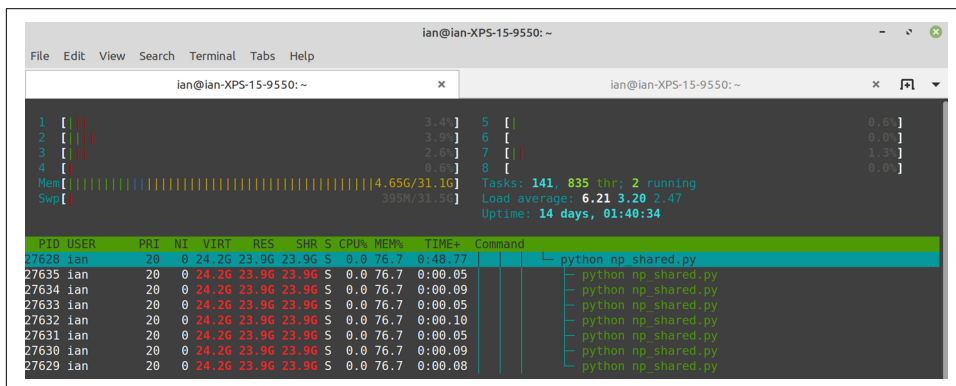


Figure 9-18. `htop` showing RAM and swap usage

To understand this demo, we'll first walk through the console output, and then we'll look at the code. In [Example 9-27](#), we start the parent process: it allocates a 25.6 GB double array of dimensions $10,000 \times 320,000$, filled with the value zero. The 10,000 rows will be passed out as indices to the worker function, and the worker will operate on each column of 320,000 items in turn. Having allocated the array, we fill it with the answer to life, the universe, and everything (42!). We can test in the worker function that we're receiving this modified array and not a filled-with-0s version to confirm that this code is behaving as expected.

Example 9-27. Setting up the shared array

```
$ python np_shared.py
Created shared array with 25,600,000,000 nbytes
Shared array id is 139636238840896 in PID 27628
Starting with an array of 0 values:
[[ 0.  0.  0. ...,  0.  0.  0.]
 ...,
 [ 0.  0.  0. ...,  0.  0.  0.]]

Original array filled with value 42:
[[ 42.  42.  42. ...,  42.  42.  42.]
 ...,
 [ 42.  42.  42. ...,  42.  42.  42.]]
Press a key to start workers using multiprocessing...
```

In [Example 9-28](#), we've started four processes working on this shared array. No copy of the array was made; each process is looking at the same large block of memory, and each process has a different set of indices to work from. Every few thousand lines, the worker outputs the current index and its PID, so we can observe its behav-

ior. The worker's job is trivial—it will check that the current element is still set to the default (so we know that no other process has modified it already), and then it will overwrite this value with the current PID. Once the workers have completed, we return to the parent process and print the array again. This time, we see that it is filled with PIDs rather than 42.

Example 9-28. Running worker_fn on the shared array

```
worker_fn: with idx 0
  id of local_nparray_in_process is 139636238840896 in PID 27751
worker_fn: with idx 2000
  id of local_nparray_in_process is 139636238840896 in PID 27754
worker_fn: with idx 1000
  id of local_nparray_in_process is 139636238840896 in PID 27752
worker_fn: with idx 4000
  id of local_nparray_in_process is 139636238840896 in PID 27753
...
worker_fn: with idx 8000
  id of local_nparray_in_process is 139636238840896 in PID 27752
```

The default value has been overwritten with worker_fn's result:

```
[[27751. 27751. 27751. ... 27751. 27751. 27751.]
...
[27751. 27751. 27751. ... 27751. 27751. 27751.]]
```

Finally, in [Example 9-29](#) we use a Counter to confirm the frequency of each PID in the array. As the work was evenly divided, we expect to see each of the four PIDs represented an equal number of times. In our 3,200,000,000-element array, we see four sets of 800,000,000 PIDs. The table output is presented using [PrettyTable](#).

Example 9-29. Verifying the result on the shared array

Verification - extracting unique values from 3,200,000,000 items
in the numpy array (this might be slow)...

Unique values in main_nparray:

```
+-----+-----+
|  PID  |  Count  |
+-----+-----+
| 27751.0 | 800000000 |
| 27752.0 | 800000000 |
| 27753.0 | 800000000 |
| 27754.0 | 800000000 |
+-----+-----+
```

Press a key to exit...

Having completed, the program now exits, and the array is deleted.

We can take a peek inside each process under Linux by using `ps` and `pmap`. [Example 9-30](#) shows the result of calling `ps`. Breaking apart this command line:

- `ps` tells us about the process.
- `-A` lists all processes.
- `-o pid,size,vsize,cmd` outputs the PID, size information, and the command name.
- `grep` is used to filter all other results and leave only the lines for our demo.

The parent process (PID 27628) and its four forked children are shown in the output. The result is similar to what we saw in `htop`. We can use `pmap` to look at the memory map of each process, requesting extended output with `-x`. We `grep` for the pattern `s-` to list blocks of memory that are marked as being shared. In the parent process and the child processes, we see a 25,000,000 KB (25.6 GB) block that is shared between them.

Example 9-30. Using `pmap` and `ps` to investigate the operating system’s view of the processes

```
$ ps -A -o pid,size,vsize,cmd | grep np_shared
27628 279676 25539428 python np_shared.py
27751 279148 25342688 python np_shared.py
27752 279148 25342688 python np_shared.py
27753 279148 25342688 python np_shared.py
27754 279148 25342688 python np_shared.py

ian@ian-Latitude-E6420 $ pmap -x 27628 | grep s-
Address      Kbytes      RSS    Dirty Mode  Mapping
00007ef9a2853000 25000000 25000000 2584636 rw-s- pym-27628-npfjsxl6 (deleted)
...
ian@ian-Latitude-E6420 $ pmap -x 27751 | grep s-
Address      Kbytes      RSS    Dirty Mode  Mapping
00007ef9a2853000 25000000 6250104 1562508 rw-s- pym-27628-npfjsxl6 (deleted)
...
```

We’ll use a `multiprocessing.Array` to allocate a shared block of memory as a 1D array and then instantiate a `numpy` array from this object and reshape it to a 2D array. Now we have a `numpy`-wrapped block of memory that can be shared between processes and addressed as though it were a normal `numpy` array. `numpy` is not managing the RAM; `multiprocessing.Array` is managing it.

In [Example 9-31](#), you can see that each forked process has access to a global `main_nparray`. While the forked process has a copy of the `numpy` object, the underlying bytes that the object accesses are stored as shared memory. Our `worker_fn` will overwrite a chosen row (via `idx`) with the current process identifier.

Example 9-31. worker_fn for sharing numpy arrays using multiprocessing

```
import os
import multiprocessing
from collections import Counter
import ctypes
import numpy as np
from prettytable import PrettyTable

SIZE_A, SIZE_B = 10_000, 320_000 # 24GB

def worker_fn(idx):
    """Do some work on the shared np array on row idx"""
    # confirm that no other process has modified this value already
    assert main_narray[idx, 0] == DEFAULT_VALUE
    # inside the subprocess print the PID and ID of the array
    # to check we don't have a copy
    if idx % 1000 == 0:
        print("{}: with idx {} \n id of local_narray_in_process is {} in PID {}".format(
            worker_fn.__name__, idx, id(main_narray), os.getpid()))
    # we can do any work on the array; here we set every item in this row to
    # have the value of the process ID for this process
    main_narray[idx, :] = os.getpid()
```

In our `__main__` in [Example 9-32](#), we'll work through three major stages:

1. Build a shared `multiprocessing.Array` and convert it into a numpy array.
2. Set a default value into the array, and spawn four processes to work on the array in parallel.
3. Verify the array's contents after the processes return.

Typically, you'd set up a numpy array and work on it in a single process, probably doing something like `arr = np.array((100, 5), dtype=np.float_)`. This is fine in a single process, but you can't share this data across processes for both reading and writing.

The trick is to make a shared block of bytes. One way is to create a `multiprocessing.Array`. By default the `Array` is wrapped in a lock to prevent concurrent edits, but we don't need this lock as we'll be careful about our access patterns. To communicate this clearly to other team members, it is worth being explicit and setting `lock=False`.

If you don't set `lock=False`, you'll have an object rather than a reference to the bytes, and you'll need to call `.get_obj()` to get to the bytes. By calling `.get_obj()`, you bypass the lock, so there's no value in not being explicit about this in the first place.

Next, we take this block of shareable bytes and wrap a numpy array around them using `frombuffer`. The `dtype` is optional, but since we're passing bytes around, it is always

sensible to be explicit. We reshape so we can address the bytes as a 2D array. By default the array values are set to 0. [Example 9-32](#) shows our `__main__` in full.

Example 9-32. `__main__` to set up numpy arrays for sharing

```
if __name__ == '__main__':
    DEFAULT_VALUE = 42
    NBR_OF_PROCESSES = 4

    # create a block of bytes, reshape into a local numpy array
    NBR_ITEMS_IN_ARRAY = SIZE_A * SIZE_B
    shared_array_base = multiprocessing.Array(ctypes.c_double,
                                              NBR_ITEMS_IN_ARRAY, lock=False)
    main_npparray = np.frombuffer(shared_array_base, dtype=ctypes.c_double)
    main_npparray = main_npparray.reshape(SIZE_A, SIZE_B)
    # assert no copy was made
    assert main_npparray.base.base is shared_array_base
    print("Created shared array with {:,} nbytes".format(main_npparray.nbytes))
    print("Shared array id is {} in PID {}".format(id(main_npparray), os.getpid()))
    print("Starting with an array of 0 values:")
    print(main_npparray)
    print()
```

To confirm that our processes are operating on the same block of data that we started with, we set each item to a new `DEFAULT_VALUE` (we again use 42, the answer to life, the universe, and everything)—you’ll see that at the top of [Example 9-33](#). Next, we build a `Pool` of processes (four in this case) and then send batches of row indices via the call to `map`.

Example 9-33. `__main__` for sharing numpy arrays using `multiprocessing`

```
# Modify the data via our local numpy array
main_npparray.fill(DEFAULT_VALUE)
print("Original array filled with value {}".format(DEFAULT_VALUE))
print(main_npparray)

input("Press a key to start workers using multiprocessing...")
print()

# create a pool of processes that will share the memory block
# of the global numpy array, share the reference to the underlying
# block of data so we can build a numpy array wrapper in the new processes
pool = multiprocessing.Pool(processes=NBR_OF_PROCESSES)
# perform a map where each row index is passed as a parameter to the
# worker_fn
pool.map(worker_fn, range(SIZE_A))
```

Once we've completed the parallel processing, we return to the parent process to verify the result (Example 9-34). The verification step runs through a flattened view on the array (note that the view does *not* make a copy; it just creates a 1D iterable view on the 2D array), counting the frequency of each PID. Finally, we perform some `assert` checks to make sure we have the expected counts.

Example 9-34. `__main__` to verify the shared result

```
print("Verification - extracting unique values from {:,} items\n in the numpy \
      array (this might be slow)...".format(NBR_ITEMS_IN_ARRAY))
# main_nparray.flat iterates over the contents of the array, it doesn't
# make a copy
counter = Counter(main_nparray.flat)
print("Unique values in main_nparray:")
tbl = PrettyTable(["PID", "Count"])
for pid, count in list(counter.items()):
    tbl.add_row([pid, count])
print(tbl)

total_items_set_in_array = sum(counter.values())

# check that we have set every item in the array away from DEFAULT_VALUE
assert DEFAULT_VALUE not in list(counter.keys())
# check that we have accounted for every item in the array
assert total_items_set_in_array == NBR_ITEMS_IN_ARRAY
# check that we have NBR_OF_PROCESSES of unique keys to confirm that every
# process did some of the work
assert len(counter) == NBR_OF_PROCESSES

input("Press a key to exit...")
```

We've just created a 1D array of bytes, converted it into a 2D array, shared the array among four processes, and allowed them to process concurrently on the same block of memory. This recipe will help you parallelize over many cores. Be careful with concurrent access to the *same* data points, though—you'll have to use the locks in multiprocessing if you want to avoid synchronization problems, and this will slow down your code.

Synchronizing File and Variable Access

In the following examples, we'll look at multiple processes sharing and manipulating a state—in this case, four processes incrementing a shared counter a set number of times. Without a synchronization process, the counting is incorrect. If you're sharing data in a coherent way you'll always need a method to synchronize the reading and writing of data, or you'll end up with errors.

Typically, the synchronization methods are specific to the OS you're using, and they're often specific to the language you use. Here, we look at file-based synchronization using a Python library and sharing an integer object between Python processes.

File Locking

Reading and writing to a file will be the slowest example of data sharing in this section.

You can see our first work function in [Example 9-35](#). The function iterates over a local counter. In each iteration it opens a file and reads the existing value, increments it by one, and then writes the new value over the old one. On the first iteration the file will be empty or won't exist, so it will catch an exception and assume the value should be zero.



The examples given here are simplified—in practice it is safer to use a context manager to open a file using `with open(filename, "r") as f:`. If an exception is raised inside the context, the file `f` will correctly be closed.

Example 9-35. work function without a lock

```
def work(filename, max_count):
    for n in range(max_count):
        f = open(filename, "r")
        try:
            nbr = int(f.read())
        except ValueError as err:
            print("File is empty, starting to count from 0, error: " + str(err))
            nbr = 0
        f = open(filename, "w")
        f.write(str(nbr + 1) + '\n')
        f.close()
```

Let's run this example with one process. You can see the output in [Example 9-36](#). `work` is called one thousand times, and as expected it counts correctly without losing any data. On the first read, it sees an empty file. This raises the `invalid literal for int() error for int()` (as `int()` is called on an empty string). This error occurs only once; afterward, we always have a valid value to read and convert into an integer.

Example 9-36. Timing of file-based counting without a lock and with one process

```
$ python ex1_nolock1.py
Starting 1 process(es) to count to 1000
File is empty, starting to count from 0,
```

```
error: invalid literal for int() with base 10: ''
Expecting to see a count of 1000
count.txt contains:
1000
```

Now we'll run the same work function with four concurrent processes. We don't have any locking code, so we'll expect some odd results.



Before you look at the following code, what *two* types of error can you expect to see when two processes simultaneously read from or write to the same file? Think about the two main states of the code (the start of execution for each process and the normal running state of each process).

Take a look at [Example 9-37](#) to see the problems. First, when each process starts, the file is empty, so each tries to start counting from zero. Second, as one process writes, the other can read a partially written result that can't be parsed. This causes an exception, and a zero will be written back. This, in turn, causes our counter to keep getting reset! Can you see how `\n` and two values have been written by two concurrent processes to the same open file, causing an invalid entry to be read by a third process?

Example 9-37. Timing of file-based counting without a lock and with four processes

```
$ python ex1_nolock4.py
Starting 4 process(es) to count to 4000
File is empty, starting to count from 0,
error: invalid literal for int() with base 10: ''
# many errors like these
File is empty, starting to count from 0,
error: invalid literal for int() with base 10: ''
Expecting to see a count of 4000
count.txt contains:
112

$ python -m timeit -s "import ex1_nolock4" "ex1_nolock4.run_workers()"
2 loops, best of 5: 158 msec per loop
```

[Example 9-38](#) shows the multiprocessing code that calls work with four processes. Note that rather than using a `map`, we're building a list of `Process` objects. Although we don't use the functionality here, the `Process` object gives us the power to introspect the state of each `Process`. We encourage you to [read the documentation](#) to learn about why you might want to use a `Process`.

Example 9-38. run_workers setting up four processes

```
import multiprocessing
import os

...
MAX_COUNT_PER_PROCESS = 1000
FILENAME = "count.txt"
...

def run_workers():
    NBR_PROCESSES = 4
    total_expected_count = NBR_PROCESSES * MAX_COUNT_PER_PROCESS
    print("Starting {} process(es) to count to {}".format(NBR_PROCESSES,
                                                            total_expected_count))

    # reset counter
    f = open(FILENAME, "w")
    f.close()

    processes = []
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work, args=(FILENAME,
                                                       MAX_COUNT_PER_PROCESS))

        p.start()
        processes.append(p)

    for p in processes:
        p.join()

    print("Expecting to see a count of {}".format(total_expected_count))
    print("{} contains:".format(FILENAME))
    os.system('more ' + FILENAME)

if __name__ == "__main__":
    run_workers()
```

Using the **fasteners** module, we can introduce a synchronization method so only one process gets to write at a time and the others each await their turn. The overall process therefore runs more slowly, but it doesn't make mistakes. You can see the correct output in [Example 9-39](#). Be aware that the locking mechanism is specific to Python, so other processes that are looking at this file will *not* care about the "locked" nature of this file.

Example 9-39. Timing of file-based counting with a lock and four processes

```
$ python ex1_lock.py
Starting 4 process(es) to count to 4000
File is empty, starting to count from 0,
error: invalid literal for int() with base 10: ''
Expecting to see a count of 4000
```



```
count.txt contains:
4000
$ python -m timeit -s "import ex1_lock" "ex1_lock.run_workers()"
10 loops, best of 3: 401 msec per loop
```

Using `fasteners` adds a single line of code in [Example 9-40](#) with the `@fasteners.interprocess_locked` decorator; the filename can be anything, but using a similar name as the file you want to lock probably makes debugging from the command line easier. Note that we haven't had to change the inner function; the decorator gets the lock on each call, and it will wait until it can get the lock before the call into `work` proceeds.

Example 9-40. `work` function with a lock

```
@fasteners.interprocess_locked('/tmp/tmp_lock')
def work(filename, max_count):
    for n in range(max_count):
        f = open(filename, "r")
        try:
            nbr = int(f.read())
        except ValueError as err:
            print("File is empty, starting to count from 0, error: " + str(err))
            nbr = 0
        f = open(filename, "w")
        f.write(str(nbr + 1) + '\n')
        f.close()
```

Locking a Value

The `multiprocessing` module offers several options for sharing Python objects between processes. We can share primitive objects with a low communication overhead, and we can also share higher-level Python objects (e.g., dictionaries and lists) using a `Manager` (but note that the synchronization cost will significantly slow down the data sharing).

Here, we'll use a `multiprocessing.Value` object to share an integer between processes. While a `Value` has a lock, the lock doesn't do quite what you might expect—it prevents simultaneous reads or writes but does *not* provide an atomic increment. [Example 9-41](#) illustrates this. You can see that we end up with an incorrect count; this is similar to the file-based unsynchronized example we looked at earlier.

Example 9-41. No locking leads to an incorrect count

```
$ python ex2_nolock.py
Expecting to see a count of 4000
We have counted to 2340
```

```
$ python -m timeit -s "import ex2_nolock" "ex2_nolock.run_workers()"
20 loops, best of 5: 9.97 msec per loop
```

No corruption occurs to the data, but we do miss some of the updates. This approach might be suitable if you're writing to a `Value` from one process and consuming (but not modifying) that `Value` in other processes.

The code to share the `Value` is shown in [Example 9-42](#). We have to specify a datatype and an initialization value—using `Value("i", 0)`, we request a signed integer with a default value of 0. This is passed as a regular argument to our `Process` object, which takes care of sharing the same block of bytes between processes behind the scenes. To access the primitive object held by our `Value`, we use `.value`. Note that we're asking for an in-place addition—we'd expect this to be an atomic operation, but that's not supported by `Value`, so our final count is lower than expected.

Example 9-42. The counting code without a `Lock`

```
import multiprocessing

def work(value, max_count):
    for n in range(max_count):
        value.value += 1

def run_workers():
    ...
    value = multiprocessing.Value('i', 0)
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work, args=(value, MAX_COUNT_PER_PROCESS))
        p.start()
        processes.append(p)
    ...
```

You can see the correctly synchronized count in [Example 9-43](#) using a `multiprocessing.Lock`.

Example 9-43. Using a `Lock` to synchronize writes to a `Value`

```
# lock on the update, but this isn't atomic
$ python ex2_lock.py
Expecting to see a count of 4000
We have counted to 4000
$ python -m timeit -s "import ex2_lock" "ex2_lock.run_workers()"
20 loops, best of 5: 19.3 msec per loop
```

In [Example 9-44](#), we've used a context manager (with `Lock`) to acquire the lock.

Example 9-44. Acquiring a Lock using a context manager

```
import multiprocessing

def work(value, max_count, lock):
    for n in range(max_count):
        with lock:
            value.value += 1

def run_workers():
    ...
    processes = []
    lock = multiprocessing.Lock()
    value = multiprocessing.Value('i', 0)
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work,
                                   args=(value, MAX_COUNT_PER_PROCESS, lock))
        p.start()
        processes.append(p)
    ...
```

If we avoid the context manager and directly wrap our increment with acquire and release, we can go a little faster, but the code is less readable compared to using the context manager. We suggest sticking to the context manager to improve readability. The snippet in [Example 9-45](#) shows how to acquire and release the Lock object.

Example 9-45. Inline locking rather than using a context manager

```
lock.acquire()
value.value += 1
lock.release()
```

Since a Lock doesn't give us the level of granularity that we're after, the basic locking that it provides wastes a bit of time unnecessarily. We can replace the Value with a [RawValue](#), as in [Example 9-46](#), and achieve an incremental speedup. If you're interested in seeing the bytecode behind this change, read [Eli Bendersky's blog post](#) on the subject.

Example 9-46. Console output showing the faster RawValue and Lock approach

```
# RawValue has no lock on it
$ python ex2_lock_rawvalue.py
Expecting to see a count of 4000
We have counted to 4000
$ python -m timeit -s "import ex2_lock_rawvalue" "ex2_lock_rawvalue.run_workers()"
50 loops, best of 5: 9.49 msec per loop
```

To use a `RawValue`, just swap it for a `Value`, as shown in [Example 9-47](#).

Example 9-47. Example of using a `RawValue` integer

```
...
def run_workers():
...
    lock = multiprocessing.Lock()
    value = multiprocessing.RawValue('i', 0)
    for process_nbr in range(NBR_PROCESSES):
        p = multiprocessing.Process(target=work,
                                   args=(value, MAX_COUNT_PER_PROCESS, lock))

        p.start()
        processes.append(p)
```

We could also use a `RawArray` in place of a `multiprocessing.Array` if we were sharing an array of primitive objects.

We’ve looked at various ways of dividing up work on a single machine between multiple processes, along with sharing a flag and synchronizing data sharing between these processes. Remember, though, that sharing data can lead to headaches—try to avoid it if possible. Making a machine deal with all the edge cases of state sharing is hard; the first time you have to debug the interactions of multiple processes, you’ll realize why the accepted wisdom is to avoid this situation if possible.

Do consider writing code that runs a bit slower but is more likely to be understood by your team. Using an external tool like Redis to share state leads to a system that can be inspected at runtime by people *other* than the developers—this is a powerful way to enable your team to keep on top of what’s happening in your parallel systems.

Definitely bear in mind that tweaked performant Python code is less likely to be understood by more junior members of your team—they’ll either be scared of it or break it. Avoid this problem (and accept a sacrifice in speed) to keep team velocity high.

Wrap-Up

We’ve covered a lot in this chapter. First, we looked at two embarrassingly parallel problems, one with predictable complexity and the other with nonpredictable complexity. We’ll use these examples again on multiple machines when we discuss clustering in [Chapter 10](#).

Next, we looked at Queue support in `multiprocessing` and its overheads. In general, we recommend using an external queue library so that the state of the queue is more transparent. Preferably, you should use an easy-to-read job format so that it is easy to debug, rather than pickled data.

The IPC discussion should have impressed upon you how difficult it is to use IPC efficiently, and that it can make sense just to use a naive parallel solution (without IPC). Buying a faster computer with more cores might be a far more pragmatic solution than trying to use IPC to exploit an existing machine.

Sharing numpy matrices in parallel without making copies is important for only a small set of problems, but when it counts, it'll really count. It takes a few extra lines of code and requires some sanity checking to make sure that you're really not copying the data between processes.

Finally, we looked at using file and memory locks to avoid corrupting data—this is a source of subtle and hard-to-track errors, and this section showed you some robust and lightweight solutions.

In the next chapter we'll look at clustering using Python. With a cluster, we can move beyond single-machine parallelism and utilize the CPUs on a group of machines. This introduces a new world of debugging pain—not only can your code have errors, but the other machines can also have errors (either from bad configuration or from failing hardware). We'll show how to parallelize the pi estimation demo using the Parallel Python module and how to run research code inside IPython using an IPython cluster.

HPP Ch 8 | Asynchronous I/O

Asynchronous I/O

Questions You'll Be Able to Answer After This Chapter

- What is concurrency, and how is it helpful?
- What is the difference between concurrency and parallelism?
- Which tasks can be done concurrently, and which can't?
- What are the various paradigms for concurrency?
- When is the right time to take advantage of concurrency?
- How can concurrency speed up my programs?

So far we have focused on speeding up code by increasing the number of compute cycles that a program can complete in a given time. However, in the days of big data, getting the relevant data to your code can be the bottleneck, as opposed to the actual code itself. When this is the case, your program is called *I/O bound*; in other words, the speed is bounded by the efficiency of the input/output.

I/O can be quite burdensome to the flow of a program. Every time your code reads from a file or writes to a network socket, it must pause to contact the kernel, request that the actual read happens, and then wait for it to complete. This is because it is not your program but the kernel that does the actual read operation, since the kernel is responsible for managing any interaction with hardware. The additional layer may not seem like the end of the world, especially once you realize that a similar operation happens every time memory is allocated; however, if we look back at [Figure 1-3](#), we see that most of the I/O operations we perform are on devices that are orders of magnitude slower than the CPU. So even if the communication with the kernel is fast,

we'll be waiting quite some time for the kernel to get the result from the device and return it to us.

For example, in the time it takes to write to a network socket, an operation that typically takes about 1 millisecond, we could have completed 2,400,000 instructions on a 2.4 GHz computer. Worst of all, our program is halted for much of this 1 millisecond of time—our execution is paused, and then we wait for a signal that the write operation has completed. This time spent in a paused state is called *I/O wait*.

Asynchronous I/O helps us utilize this wasted time by allowing us to perform other operations while we are in the I/O wait state. For example, in [Figure 8-1](#) we see a depiction of a program that must run three tasks, all of which have periods of I/O wait within them. If we run them serially, we suffer the I/O wait penalty three times. However, if we run these tasks concurrently, we can essentially hide the wait time by running another task in the meantime. It is important to note that this is all still happening on a single thread and still uses only one CPU at a time!

This is possible because while a program is in I/O wait, the kernel is simply waiting for whatever device we've requested to read from (hard drive, network adapter, GPU, etc.) to send a signal that the requested data is ready. Instead of waiting, we can create a mechanism (the event loop) so that we can dispatch requests for data, continue performing compute operations, and be notified when the data is ready to be read. This is in stark contrast to the multiprocessing/multithreading ([Chapter 9](#)) paradigm, where a new process is launched that does experience I/O wait but uses the multitasking nature of modern CPUs to allow the main process to continue. However, the two mechanisms are often used in tandem, where we launch multiple processes, each of which is efficient at asynchronous I/O, in order to fully take advantage of our computer's resources.



Since concurrent programs run on a single thread, they are generally easier to write and manage than standard multithreaded programs. All concurrent functions share the same memory space, so sharing data between them works in the normal ways you would expect. However, you still need to be careful about race conditions since you can't be sure which lines of code get run when.

By modeling a program in this event-driven way, we are able to take advantage of I/O wait to perform more operations on a single thread than would otherwise be possible.

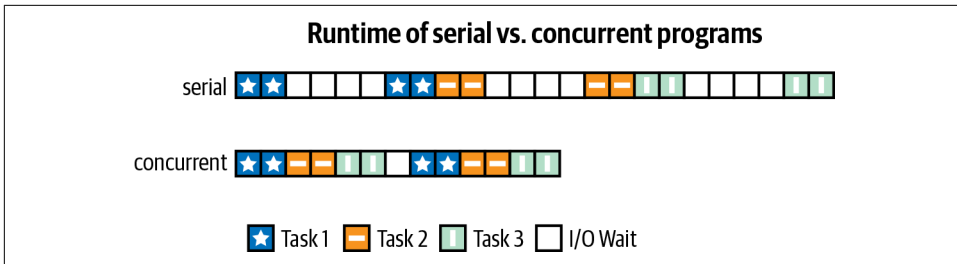


Figure 8-1. Comparison of serial and concurrent programs

Introduction to Asynchronous Programming

Typically, when a program enters I/O wait, the execution is paused so that the kernel can perform the low-level operations associated with the I/O request (this is called a *context switch*), and it is not resumed until the I/O operation is completed. Context switching is quite a heavy operation. It requires us to save the state of our program (losing any sort of caching we had at the CPU level) and give up the use of the CPU. Later, when we are allowed to run again, we must spend time reinitializing our program on the motherboard and getting ready to resume (of course, all this happens behind the scenes).

With concurrency, on the other hand, we typically have an *event loop* running that manages what gets to run in our program, and when. In essence, an event loop is simply a list of functions that need to be run. The function at the top of the list gets run, then the next, etc. [Example 8-1](#) shows a simple example of an event loop.

Example 8-1. Toy event loop

```
from queue import Queue
from functools import partial

eventloop = None

class EventLoop(Queue):
    def start(self):
        while True:
            function = self.get()
            function()

def do_hello():
    global eventloop
    print("Hello")
    eventloop.put(do_world)

def do_world():
    global eventloop
```

```

print("world")
eventloop.put(do_hello)

if __name__ == "__main__":
    eventloop = EventLoop()
    eventloop.put(do_hello)
    eventloop.start()

```

This may not seem like a big change; however, we can couple event loops with asynchronous (async) I/O operations for massive gains when performing I/O tasks. In this example, the call `eventloop.put(do_world)` approximates an asynchronous call to the `do_world` function. This operation is called *nonblocking*, meaning it will return immediately but guarantee that `do_world` is called at some point later. Similarly, if this were a network write with an async function, it will return right away even though the write has not happened yet. When the write has completed, an event fires so our program knows about it.

Putting these two concepts together, we can have a program that, when an I/O operation is requested, runs other functions while waiting for the original I/O operation to complete. This essentially allows us to still do meaningful calculations when we otherwise would have been in I/O wait.



Switching from function to function does have a cost. The kernel must take the time to set up the function to be called in memory, and the state of our caches won't be as predictable. It is because of this that concurrency gives the best results when your program has a lot of I/O wait—the cost associated with switching can be much less than what is gained by making use of I/O wait time.

Generally, programming using event loops can take two forms: callbacks or futures. In the callback paradigm, functions are called with an argument that is generally called the *callback*. Instead of the function returning its value, it calls the callback function with the value instead. This sets up long chains of functions that are called, with each function getting the result of the previous function in the chain (these chains are sometimes referred to as “callback hell”). **Example 8-2** is a simple example of the callback paradigm.

Example 8-2. Example with callbacks

```

from functools import partial
from some_database_library import save_results_to_db

def save_value(value, callback):
    print(f"Saving {value} to database")
    save_result_to_db(result, callback) ❶

```

```
def print_response(db_response):
    print("Response from database: {db_response}")

if __name__ == "__main__":
    eventloop.put(partial(save_value, "Hello World", print_response))
```

- ❶ `save_result_to_db` is an asynchronous function; it will return immediately, and the function will end and allow other code to run. However, once the data is ready, `print_response` will be called.

Before Python 3.4, the callback paradigm was quite popular. However, the `asyncio` standard library module and PEP 492 made the future’s mechanism native to Python. This was done by creating a standard API for dealing with asynchronous I/O and the new `await` and `async` keywords, which define an asynchronous function and a way to wait for a result.

In this paradigm, an asynchronous function returns a `Future` object, which is a promise of a future result. Because of this, if we want the result at some point we must wait for the future that is returned by this sort of asynchronous function to complete and be filled with the value we desire (either by doing an `await` on it or by running a function that explicitly waits for a value to be ready). However, this also means that the result can be available in the callers context, while in the callback paradigm the result is available only in the callback function. While waiting for the `Future` object to be filled with the data we requested, we can do other calculations. If we couple this with the concept of generators—functions that can be paused and whose execution can later be resumed—we can write asynchronous code that looks very close to serial code in form:

```
from some_async_database_library import save_results_to_db

async def save_value(value):
    print(f"Saving {value} to database")
    db_response = await save_result_to_db(result) ❶
    print("Response from database: {db_response}")

if __name__ == "__main__":
    eventloop.put(
        partial(save_value, "Hello World", print)
    )
```

- ❶ In this case, `save_result_to_db` returns a `Future` type. By awaiting it, we ensure that `save_value` gets paused until the value is ready and then resumes and completes its operations.

It’s important to realize that the `Future` object returned by `save_result_to_db` holds the *promise* of a `Future` result and doesn’t hold the result itself or even call any of the `save_result_to_db` code. In fact, if we simply did `db_response_future =`

`save_result_to_db(result)`, the statement would complete immediately and we could do other things with the `Future` object. For example, we could collect a list of futures and wait for all of them at the same time.

How Does `async/await` Work?

An `async` function (defined with `async def`) is called a *coroutine*. In Python, coroutines are implemented with the same philosophies as generators. This is convenient because generators already have the machinery to pause their execution and resume later. Using this paradigm, an `await` statement is similar in function to a `yield` statement; the execution of the current function gets paused while other code is run. Once the `await` or `yield` resolves with data, the function is resumed. So in the preceding example, our `save_result_to_db` will return a `Future` object, and the `await` statement pauses the function until that `Future` contains a result. The event loop is responsible for scheduling the resumption of `save_value` after the `Future` is ready to return a result.

For Python 2.7 implementations of future-based concurrency, things can get a bit strange when we're trying to use coroutines as actual functions. Remember that generators cannot return values, so libraries deal with this issue in various ways. The in Python 3.4, new machinery has been introduced in order to easily create coroutines and have them still return values. However, many asynchronous libraries that have been around since Python 2.7 have legacy code meant to deal with this awkward transition (in particular, `tornado's gen` module).

It is critical to realize our reliance on an event loop when running concurrent code. In general, this leads to most fully concurrent code's main code entry point consisting mainly of setting up and starting the event loop. However, this assumes that your entire program is concurrent. In other cases, a set of futures is created within the program, and then a temporary event loop is started simply to manage the existing futures, before the event loop exits and the code can resume normally. This is generally done with either the `loop.run_until_complete(coro)` or `loop.run_forever()` method from the `asyncio.loop` module. However, `asyncio` also provides a convenience function (`asyncio.run(coro)`) to simplify this process.

In this chapter we will analyze a web crawler that fetches data from an HTTP server that has latency built into it. This represents the general response-time latency that will occur whenever dealing with I/O. We will first create a serial crawler that looks at the naive Python solution to this problem. Then we will build up to a full `aiohttp` solution by iterating through `gevent` and then `tornado`. Finally, we will look at combining `async` I/O tasks with CPU tasks in order to effectively hide any time spent doing I/O.



The web server we implemented can support multiple connections at a time. This will be true for most services that you will be performing I/O with—most databases can support multiple requests at a time, and most web servers support 10,000+ simultaneous connections. However, when interacting with a service that cannot handle multiple connections at a time, we will always have the same performance as the serial case.

Serial Crawler

For the control in our experiment with concurrency, we will write a serial web scraper that takes a list of URLs, fetches them, and sums the total length of the content from the pages. We will use a custom HTTP server that takes two parameters, `name` and `delay`. The `delay` field will tell the server how long, in milliseconds, to pause before responding. The `name` field is for logging purposes.

By controlling the `delay` parameter, we can simulate the time it takes a server to respond to our query. In the real world, this could correspond to a slow web server, a strenuous database call, or any I/O call that takes a long time to perform. For the serial case, this leads to more time that our program is stuck in I/O wait, but in the concurrent examples later on, it will provide an opportunity to spend the I/O wait time doing other tasks.

In [Example 8-3](#), we chose to use the `requests` module to perform the HTTP call. We made this choice because of the simplicity of the module. We use HTTP in general for this section because it is a simple example of I/O and can be performed easily. In general, any call to a HTTP library can be replaced with any I/O.

Example 8-3. Serial HTTP scraper

```
import random
import string

import requests

def generate_urls(base_url, num_urls):
    """
    We add random characters to the end of the URL to break any caching
    mechanisms in the requests library or the server
    """
    for i in range(num_urls):
        yield base_url + "".join(random.sample(string.ascii_lowercase, 10))

def run_experiment(base_url, num_iter=1000):
    response_size = 0
    for url in generate_urls(base_url, num_iter):
        response = requests.get(url)
```

```

        response_size += len(response.text)
    return response_size

if __name__ == "__main__":
    import time

    delay = 100
    num_iter = 1000
    base_url = f"http://127.0.0.1:8080/add?name=serial&delay={delay}&"

    start = time.time()
    result = run_experiment(base_url, num_iter)
    end = time.time()
    print(f"Result: {result}, Time: {end - start}")

```

When running this code, an interesting metric to look at is the start and stop time of each request as seen by the HTTP server. This tells us how efficient our code was during I/O wait—since our task is to launch HTTP requests and then sum the number of characters that were returned, we should be able to launch more HTTP requests, and process any responses, while waiting for other requests to complete.

We can see in [Figure 8-2](#) that, as expected, there is no interleaving of our requests. We do one request at a time and wait for the previous request to happen before we move to the next request. In fact, the total runtime of the serial process makes perfect sense knowing this. Since each request takes 0.1 seconds (because of our `delay` parameter) and we are doing 500 requests, we expect this runtime to be about 50 seconds.

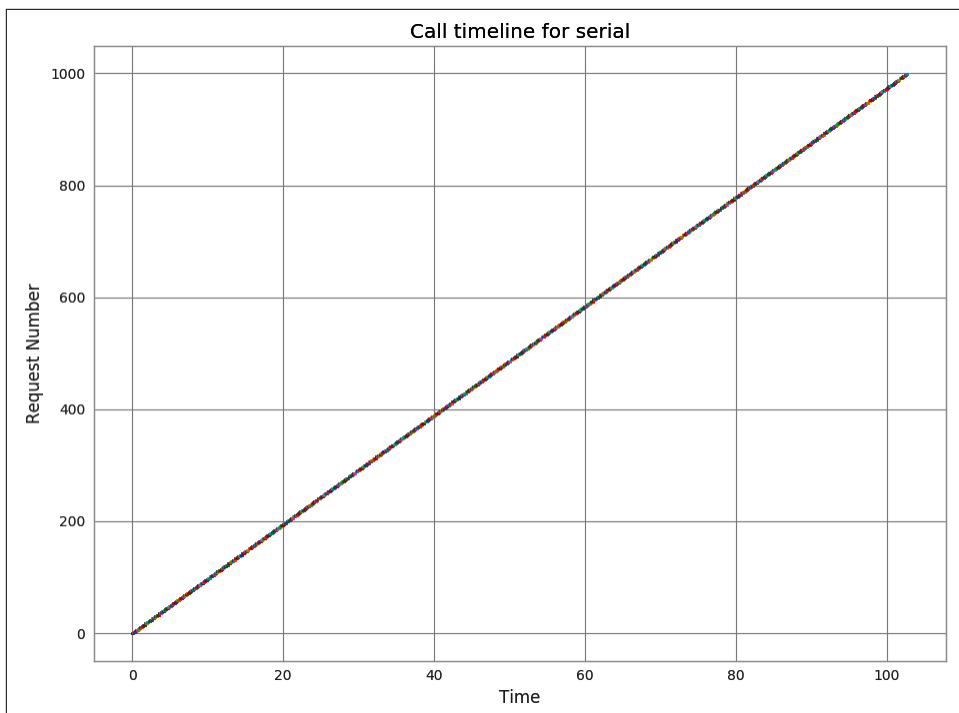


Figure 8-2. Request time for *Example 8-3*

Gevent

One of the simplest asynchronous libraries is `gevent`. It follows the paradigm of having asynchronous functions return futures, which means most of the logic in your code can stay the same. In addition, `gevent` monkey patches the standard I/O functions to be asynchronous, so most of the time you can simply use the standard I/O packages and benefit from asynchronous behavior.

`Gevent` provides two mechanisms to enable asynchronous programming—as mentioned before, it patches the standard library with asynchronous I/O functions, and it has a `Greenlets` object that can be used for concurrent execution. A *greenlet* is a type of coroutine and can be thought of as a thread (see [Chapter 9](#) for a discussion of threads); however, all greenlets run on the same physical thread. Instead of using multiple CPUs to run all the greenlets, we have an event loop on a single CPU that is able to switch between them during I/O wait. For the most part, `gevent` tries to make the handling of the event loop as transparent as possible through the use of `wait` functions. The `wait` function will start an event loop and run it as long as is needed for all greenlets to finish. Because of this, most of your `gevent` code will run serially; then at some point you will set up many greenlets to do a concurrent task and start

the event loop with the `wait` function. While the `wait` function is executing, all of the concurrent tasks you have queued up will run until completion (or some stopping condition), and then your code will go back to being serial again.

The futures are created with `gevent.spawn`, which takes a function and the arguments to that function and launches a greenlet that is responsible for running that function. The greenlet can be thought of as a future since, once the function we specified completes, its value will be contained within the greenlet's `value` field.

This patching of Python standard modules can make it harder to control the subtleties of what is going on. For example, one thing we want to ensure when doing async I/O is that we don't open too many files or connections at one time. If we do, we can overload the remote server or slow down our process by having to context-switch between too many operations.

To limit the number of open files manually, we use a semaphore to only do HTTP requests from one hundred greenlets at a time. A semaphore works by making sure that only a certain number of coroutines can enter the context block at a time. As a result, we launch all the greenlets that we need to fetch the URLs right away; however, only one hundred of them can make HTTP calls at a time. Semaphores are one type of locking mechanism used a lot in various parallel code flows. By restricting the progression of your code based on various rules, locks can help you make sure that the various components of your program don't interfere with one another.

Now that we have all the futures set up and have put in a locking mechanism to control the flow of the greenlets, we can wait until we start having results by using the `gevent.iwait` function, which will take a sequence of futures and iterate over the ready items. Conversely, we could have used `gevent.wait`, which would block execution of our program until all requests are done.

We go through the trouble of grouping our requests with the semaphore instead of sending them all at once because overloading the event loop can cause performance decreases (and this is true for all asynchronous programming). In addition, the server we are communicating with will have a limit to the number of concurrent requests it can respond to at the same time.

From experimentation (shown in [Figure 8-3](#)), we generally see that one hundred or so open connections at a time is optimal for requests with a reply time of about 50 milliseconds. If we were to use fewer connections, we would still have wasted time during I/O wait. With more, we are switching contexts too often in the event loop and adding unnecessary overhead to our program. We can see this effect come into play with four hundred concurrent requests for 50-millisecond requests. That being said, this value of one hundred depends on many things—the computer the code is being run on, the implementation of the event loop, the properties of the remote

host, the expected time to respond of the remote server, and so on. We recommend doing some experimentation before settling on a choice.

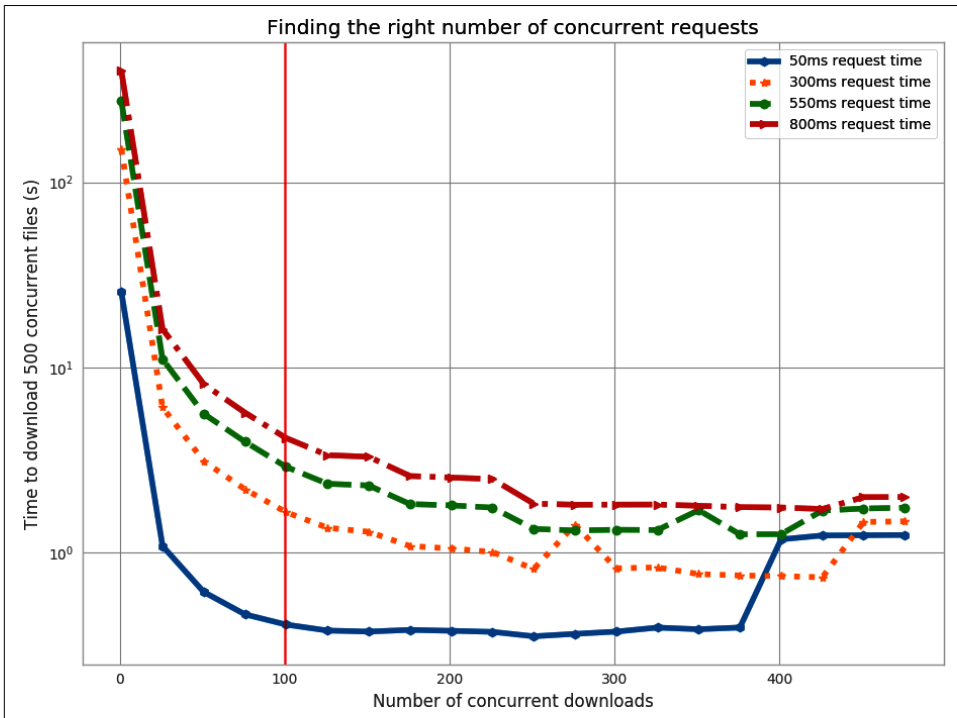


Figure 8-3. Experimenting with different numbers of concurrent requests for various request times

In [Example 8-4](#), we implement the `gevent` scraper by using a semaphore to ensure only 100 requests at a time.

Example 8-4. `gevent` HTTP scraper

```
import random
import string
import urllib.error
import urllib.parse
import urllib.request
from contextlib import closing

import gevent
from gevent import monkey
from gevent.lock import Semaphore

monkey.patch_socket()
```

```

def generate_urls(base_url, num_urls):
    for i in range(num_urls):
        yield base_url + ".".join(random.sample(string.ascii_lowercase, 10))

def download(url, semaphore):
    with semaphore: ❷
        with closing(urllib.request.urlopen(url)) as data:
            return data.read()

def chunked_requests(urls, chunk_size=100):
    """
    Given an iterable of urls, this function will yield back the contents of the
    URLs. The requests will be batched up in "chunk_size" batches using a
    semaphore
    """
    semaphore = Semaphore(chunk_size) ❶
    requests = [gevent.spawn(download, u, semaphore) for u in urls] ❸
    for response in gevent.iwait(requests):
        yield response

def run_experiment(base_url, num_iter=1000):
    urls = generate_urls(base_url, num_iter)
    response_futures = chunked_requests(urls, 100) ❹
    response_size = sum(len(r.value) for r in response_futures)
    return response_size

if __name__ == "__main__":
    import time

    delay = 100
    num_iter = 1000
    base_url = f"http://127.0.0.1:8080/add?name=gevent&delay={delay}&"

    start = time.time()
    result = run_experiment(base_url, num_iter)
    end = time.time()
    print(f"Result: {result}, Time: {end - start}")

```

- ❶ Here we generate a semaphore that lets `chunk_size` downloads happen.
- ❷ By using the semaphore as a context manager, we ensure that only `chunk_size` greenlets can run the body of the context at one time.
- ❸ We can queue up as many greenlets as we need, knowing that none of them will run until we start an event loop with `wait` or `iwait`.
- ❹ `response_futures` now holds a generator over completed futures, all of which have our desired data in the `.value` property.

An important thing to note is that we have used `gevent` to make our I/O requests asynchronous, but we are not doing any non-I/O computations while in I/O wait. However, in [Figure 8-4](#) we can see the massive speedup we get (see [Table 8-1](#)). By launching more requests while waiting for previous requests to finish, we are able to achieve a 90× speedup! We can explicitly see that requests are being sent out before previous requests finish through the stacked horizontal lines representing the requests. This is in sharp contrast to the case of the serial crawler ([Figure 8-2](#)), where a line starts only when the previous line finishes.

Furthermore, we can see more interesting effects reflected in the shape of the `gevent` request timeline in [Figure 8-4](#). For example, at around the 100th request, we see a pause where new requests are not launched. This is because it is the first time that our semaphore is hit, and we are able to lock the semaphore before any previous requests finish. After this, the semaphore goes into an equilibrium: it locks just as another request finishes and unlocks it.

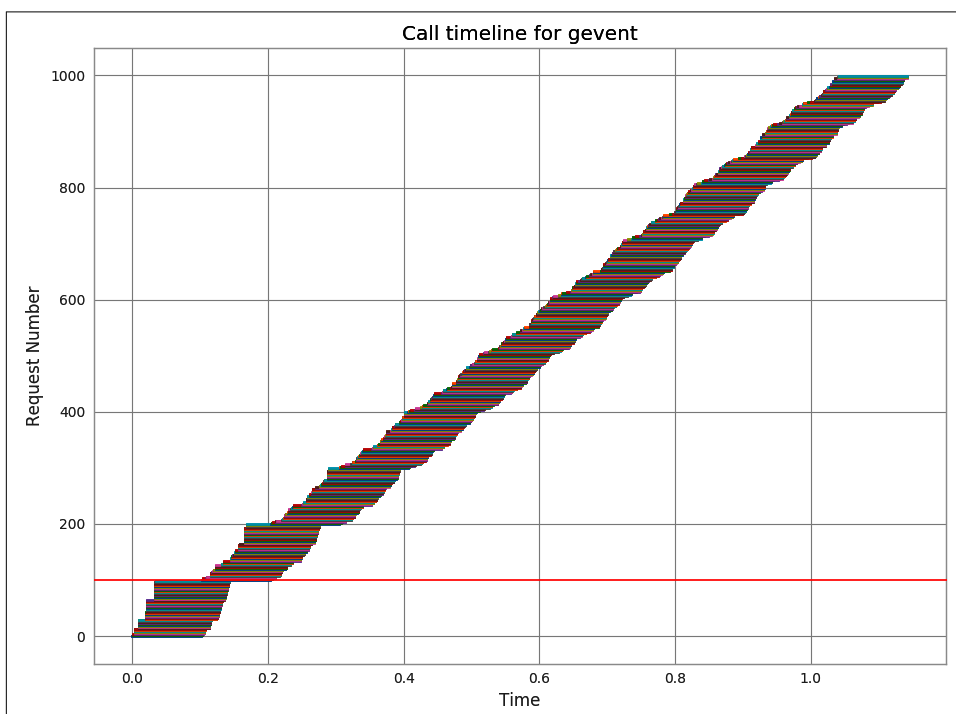


Figure 8-4. Request times for `gevent` scraper—the red line highlights the 100th request, where we can see a pause before subsequent requests are issued

tornado

Another frequently used package for asynchronous I/O in Python is **tornado**, originally developed by Facebook primarily for HTTP clients and servers. This framework has been around since Python 3.5, when `async/await` was introduced, and originally used a system of callbacks to organize asynchronous calls. Recently, however, the maintainers of the project have chosen to embrace the use of coroutines and in general have been critical in the architecture of the `asyncio` module.

Currently, **tornado** can be used either by using `async/await` syntax, as is standard in Python, or by using Python's `tornado.gen` module. This module was provided as a precursor to the native coroutines in Python. It did so by providing a decorator to turn a method into a coroutine (i.e., a way to get the same result as defining a function with `async def`) and various utilities to manage the runtime of coroutines. Currently this decorator approach is necessary only if you intend on providing support to Python versions older than 3.5.¹



When using **tornado**, make sure to have `pycurl` installed. It is an optional backend for **tornado** but performs better, especially with DNS requests, than the default backend.

In **Example 8-5**, we implement the same web crawler as we did for **gevent**, but we use the **tornado** I/O loop (its version of an event loop) and HTTP client. This saves us the trouble of having to batch our requests and deal with other, more low-level aspects of our code.

Example 8-5. tornado HTTP scraper

```
import asyncio
import random
import string
from functools import partial

from tornado.httpclient import AsyncHTTPClient

AsyncHTTPClient.configure(
    "tornado.curl_httpclient.CurlAsyncHTTPClient",
    max_clients=100 ❶
)

def generate_urls(base_url, num_urls):
```

¹ Which we're sure you're not doing!

```

    for i in range(num_urls):
        yield base_url + "".join(random.sample(string.ascii_lowercase, 10))

async def run_experiment(base_url, num_iter=1000):
    http_client = AsyncHTTPClient()
    urls = generate_urls(base_url, num_iter)
    response_sum = 0
    tasks = [http_client.fetch(url) for url in urls] ❷
    for task in asyncio.as_completed(tasks): ❸
        response = await task ❹
        response_sum += len(response.body)
    return response_sum

if __name__ == "__main__":
    import time

    delay = 100
    num_iter = 1000
    run_func = partial(
        run_experiment,
        f"http://127.0.0.1:8080/add?name=tornado&delay={delay}&",
        num_iter,
    )

    start = time.time()
    result = asyncio.run(run_func) ❺
    end = time.time()
    print(f"Result: {result}, Time: {end - start}")

```

- ❶ We can configure our HTTP client and pick what backend library we wish to use and how many requests we would like to batch together. Tornado defaults to a max of 10 concurrent requests.
- ❷ We generate many Future objects to queue the task of fetching the URL contents.
- ❸ This will run all of the coroutines that are queued in the tasks list and yield them as they complete.
- ❹ Since the coroutine is already completed, the await statement here returns immediately with the result of the earliest completed task.
- ❺ `ioloop.run_sync` will start the `IOLoop` just for the duration of the runtime of the specified function. `ioloop.start()`, on the other hand, starts an `IOLoop` that must be terminated manually.

An important difference between the tornado code in [Example 8-5](#) and the gevent code in [Example 8-4](#) is when the event loop runs. For gevent, the event loop is

running only while the `wait` function is running. On the other hand, in `tornado` the event loop is running the entire time and controls the complete execution flow of the program, not just the asynchronous I/O parts.

This makes `tornado` ideal for any application that is mostly I/O-bound and where most, if not all, of the application should be asynchronous. This is where `tornado` makes its biggest claim to fame, as a performant web server. In fact, Micha has on many occasions written `tornado`-backed databases and data structures that require a lot of I/O.²

On the other hand, since `gevent` makes no requirements of your program as a whole, it is an ideal solution for mainly CPU-based problems that sometimes involve heavy I/O—for example, a program that does a lot of computations over a dataset and then must send the results back to the database for storage. This becomes even simpler with the fact that most databases have simple HTTP APIs, which means you can even use `grequests`.

Another interesting difference between `gevent` and `tornado` is the way the internals change the request call graphs. Compare [Figure 8-5](#) with [Figure 8-4](#). For the `gevent` call graph, we see a very uniform call graph in which new requests are issued the second a slot in the semaphore opens up. On the other hand, the call graph for `tornado` is very stop-and-go. This means that the internal mechanism limiting the number of open connects is not reacting fast enough to request finishing. These areas of the call graph in which the line seems to be thinner/thicker than usual represent times when the event loop isn't doing its job optimally—times when we are either underutilizing or overutilizing our resources.



For all libraries that use `asyncio` to run the event loop, we can actually change the backend library that is being used. For example, the `uvloop` project supplies a drop-in replacement to `asyncio`'s event loop that claims massive speedups. These speedups are mainly seen server-side; in the client-side examples outlined in this chapter, they provide only a small performance boost. However, since it takes only two extra lines of code to use this event loop, there aren't many reasons not to use it!

We can start to understand this slowdown in light of the lesson we've been learning over and over again: that generalized code is useful because it solves all problems well but no individual problem perfectly. The mechanism to limit one hundred ongoing connections is fantastic when working with a large web app or a code base that may

² For example, `fuggetaboutit` is a special type of probabilistic data structure (see “[Probabilistic Data Structures](#)” on [page 371](#)) that uses the `tornado` `IOLoop` to schedule time-based tasks.

make HTTP requests in many different places. One simple configuration guarantees that overall we won't have more than the defined connections opened. However, in our situation we can benefit from being very specific as to how this is handled (as we did in the `gevent` example).

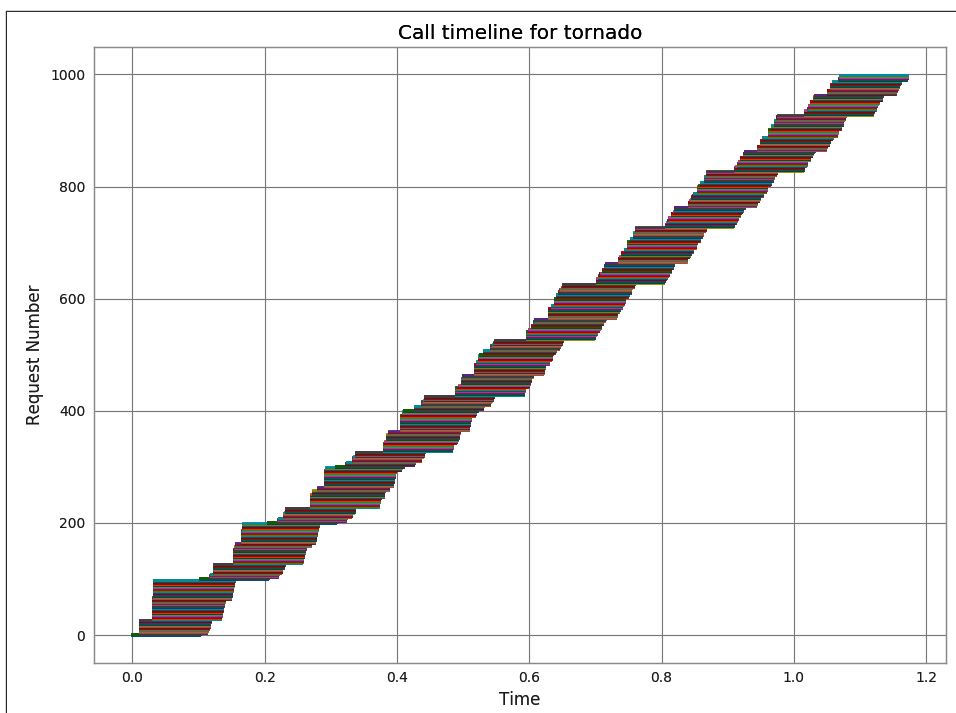


Figure 8-5. Chronology of HTTP requests for *Example 8-5*

aiohttp

In response to the popularity of using `async` functionality to deal with heavy I/O systems, Python 3.4+ introduced a revamping of the old `asyncio` standard library module. At the time, however, this module was quite low level, providing all of the low-level mechanisms for third-party libraries to create easy-to-use asynchronous libraries. `aiohttp` arose as the first popular library built entirely on the new `asyncio` library. It provides both HTTP client and server functionality and uses a similar API to those familiar with `tornado`. The entire project, `aio-lib`s, provides native asynchronous libraries for a wide variety of uses. In *Example 8-6*, we show how to implement the `asyncio` scraper using `aiohttp`.

Example 8-6. *asyncio HTTP scraper*

```
import asyncio
import random
import string

import aiohttp

def generate_urls(base_url, num_urls):
    for i in range(num_urls):
        yield base_url + "".join(random.sample(string.ascii_lowercase, 10))

def chunked_http_client(num_chunks):
    """
    Returns a function that can fetch from a URL, ensuring that only
    "num_chunks" of simultaneous connects are made.
    """
    semaphore = asyncio.Semaphore(num_chunks) ❶

    async def http_get(url, client_session): ❷
        nonlocal semaphore
        async with semaphore:
            async with client_session.request("GET", url) as response:
                return await response.content.read()

    return http_get

async def run_experiment(base_url, num_iter=1000):
    urls = generate_urls(base_url, num_iter)
    http_client = chunked_http_client(100)
    responses_sum = 0
    async with aiohttp.ClientSession() as client_session:
        tasks = [http_client(url, client_session) for url in urls] ❸
        for future in asyncio.as_completed(tasks): ❹
            data = await future
            responses_sum += len(data)
    return responses_sum

if __name__ == "__main__":
    import time

    loop = asyncio.get_event_loop()
    delay = 100
    num_iter = 1000

    start = time.time()
    result = loop.run_until_complete(
        run_experiment(
            f"http://127.0.0.1:8080/add?name=asyncio&delay={delay}&", num_iter
        )
    )
```

```
end = time.time()
print(f"Result: {result}, Time: {end - start}")
```

- ❶ As in the `gevent` example, we must use a semaphore to limit the number of requests.
- ❷ We return a new coroutine that will asynchronously download files and respect the locking of the semaphore.
- ❸ The `http_client` function returns futures. To keep track of progress, we save the futures into a list.
- ❹ As with `gevent`, we can wait for futures to become ready and iterate over them.

One immediate reaction to this code is the number of `async with`, `async def`, and `await` calls. In the definition for `http_get`, we use the `async` context manager to get access to shared resources in a concurrent-friendly way. That is to say, by using `async with`, we allow other coroutines to run while waiting to acquire the resources we are requesting. As a result, sharing things such as open semaphore slots or already opened connections to our host can be done more efficiently than we experienced with `tornado`.

In fact, the call graph in [Figure 8-6](#) shows a smooth transition similar to that of `gevent` in [Figure 8-4](#). Furthermore, the `asyncio` code runs slightly faster than the `gevent` code overall (1.10 seconds versus 1.14 seconds—see [Table 8-1](#)), even though the time for each request is slightly longer. This can be explained only by a faster resumption of coroutines paused by the semaphore or waiting for the HTTP client.

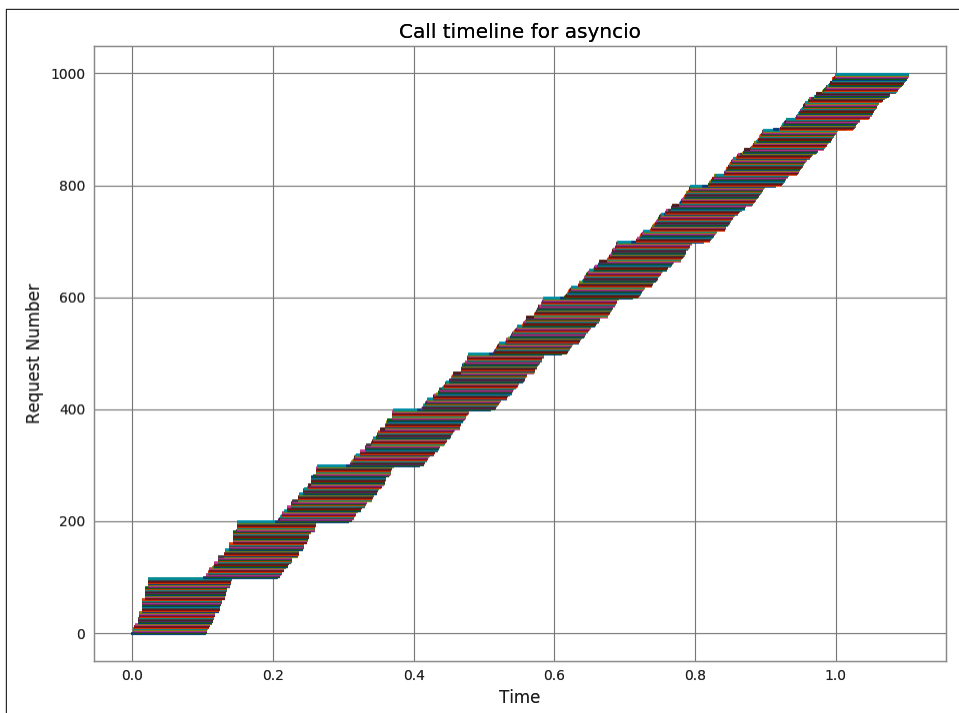


Figure 8-6. Chronology of HTTP requests for *Example 8-6*

The code sample also shows a big difference between using `aiohttp` and using `tornado` in that with `aiohttp`, we are very much in control of the event loop and the various subtleties of the request we are making. For example, we manually acquire the client session, which is responsible for caching open connections, and we manually read from the connection. If we wanted, we could change what time of connection caching is happening or decide to only write to the server and never read its response.

While this control may be a bit of overkill for such a simple example, in real-world applications we can use this to really tune the performance of our applications. Tasks can easily be added to the event loop without waiting for their response, and we can easily add time-outs to tasks so that their runtime is limited; we can even add functions that automatically get triggered when a task is completed. This allows us to create complicated runtime patterns that optimally utilize the time we gain by being able to run code during I/O wait. In particular, when we are running a web service (such as an API that may need to perform computational tasks for each request), this control can allow us to write “defensive” code that knows how to concede runtime to other tasks if a new request comes in. We will discuss this aspect more in “[Full Async](#)” on page 238.

Table 8-1. Comparison of total runtime for crawlers

	serial	gevent	tornado	aiohttp
Runtime (s)	102.684	1.142	1.171	1.101

Shared CPU–I/O Workload

To make the preceding examples more concrete, we will create another toy problem in which we have a CPU-bound problem that needs to communicate frequently with a database to save results. The CPU workload can be anything; in this case, we are taking the bcrypt hash of a random string with larger and larger workload factors to increase the amount of CPU-bound work (see Table 8-2 to understand how the “difficulty” parameter affects runtime). This problem is representative of any sort of problem in which your program has heavy calculations to do, and the results of those calculations must be stored into a database, potentially incurring a heavy I/O penalty. The only restrictions we are putting on our database are as follows:

- It has an HTTP API so we can use code like that in the earlier examples.³
- Response times are on the order of 100 milliseconds.
- The database can satisfy many requests at a time.⁴

The response time of this “database” was chosen to be higher than usual in order to exaggerate the turning point in the problem, where the time to do one of the CPU tasks is longer than one of the I/O tasks. For a database that is being used only to store simple values, a response time greater than 10 milliseconds should be considered slow!

Table 8-2. Time to calculate a single hash

Difficulty parameter	8	10	11	12
Seconds per iteration	0.0156	0.0623	0.1244	0.2487

Serial

We start with some simple code that calculates the bcrypt hash of a string and makes a request to the database’s HTTP API every time a result is calculated:

```
import random
import string
```

³ This is not necessary; it just serves to simplify our code.

⁴ This is true for all distributed databases and other popular databases, such as Postgres, MongoDB, and so on.

```

import bcrypt
import requests

def do_task(difficulty):
    """
    Hash a random 10 character string using bcrypt with a specified difficulty
    rating.
    """
    passwd = ("".join(random.sample(string.ascii_lowercase, 10))) ❶
    .encode("utf8"))
    salt = bcrypt.gensalt(difficulty) ❷
    result = bcrypt.hashpw(passwd, salt)
    return result.decode("utf8")

def save_result_serial(result):
    url = f"http://127.0.0.1:8080/add"
    response = requests.post(url, data=result)
    return response.json()

def calculate_task_serial(num_iter, task_difficulty):
    for i in range(num_iter):
        result = do_task(task_difficulty)
        save_number_serial(result)

```

- ❶ We generate a random 10-character byte array.
- ❷ The difficulty parameter sets how hard it is to generate the password by increasing the CPU and memory requirements of the hashing algorithm.

Just as in our serial example ([Example 8-3](#)), the request times for each database save (100 milliseconds) do not stack, and we must pay this penalty for each result. As a result, iterating six hundred times with a task difficulty of 8 takes 71 seconds. We know, however, that because of the way our serial requests work, we are spending 40 seconds at minimum doing I/O! 56% of our program’s runtime is being spent doing I/O and, moreover, just sitting around in “I/O wait,” when it could be doing something else!

Of course, as the CPU problem takes more and more time, the relative slowdown of doing this serial I/O decreases. This is simply because the cost of having a 100-millisecond pause after each task pales in comparison to the long amount of time needed to do this computation (as we can see in [Figure 8-7](#)). This fact highlights how important it is to understand your workload before considering which optimizations to make. If you have a CPU task that takes hours and an I/O task that takes only seconds, work done to speed up the I/O task will not bring the huge speedups you may be looking for!

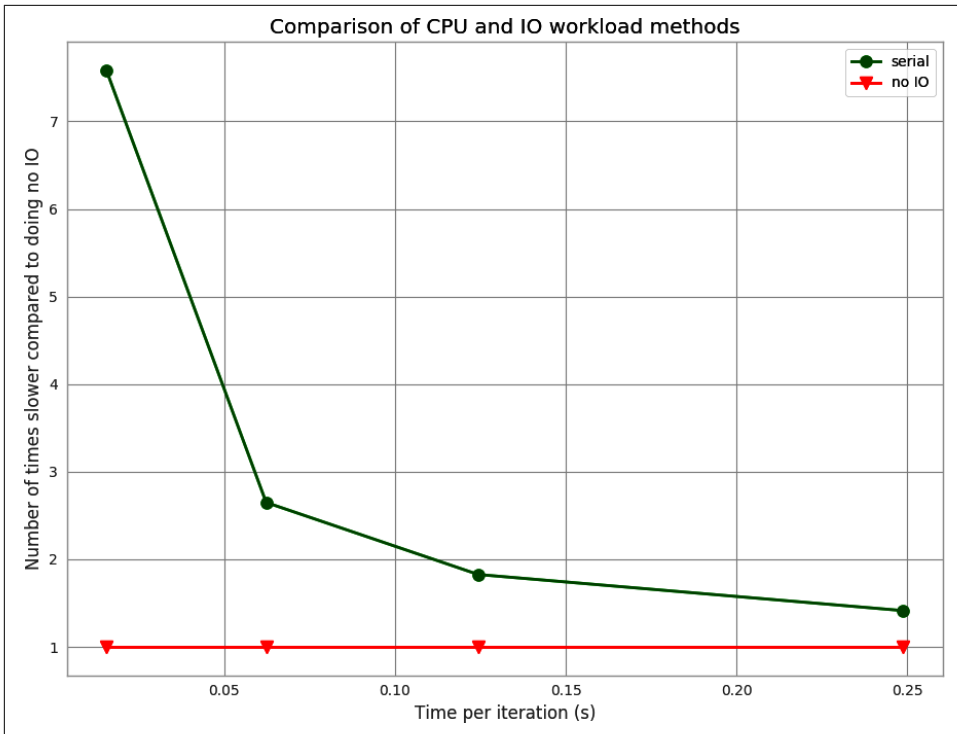


Figure 8-7. Comparison of the serial code to the CPU task with no I/O

Batched Results

Instead of immediately going to a full asynchronous solution, let's try an intermediate solution. If we don't need to know the results in our database right away, we can batch up the results and send them to the database asynchronously. To do this, we create an object, `AsyncBatcher`, that will take care of queuing results to be sent to the database in small asynchronous bursts. This will still pause the program and put it into I/O wait with no CPU tasks; however, during this time we can issue many concurrent requests instead of issuing them one at a time:

```
import asyncio
import aiohttp

class AsyncBatcher(object):
    def __init__(self, batch_size):
        self.batch_size = batch_size
        self.batch = []
        self.client_session = None
        self.url = f"http://127.0.0.1:8080/add"

    def __enter__(self):
```

```

    return self

def __exit__(self, *args, **kwargs):
    self.flush()

def save(self, result):
    self.batch.append(result)
    if len(self.batch) == self.batch_size:
        self.flush()

def flush(self):
    """
    Synchronous flush function which starts an IOloop for the purposes of
    running our async flushing function
    """
    loop = asyncio.get_event_loop()
    loop.run_until_complete(self.__aflush()) ❶

async def __aflush(self): ❷
    async with aiohttp.ClientSession() as session:
        tasks = [self.fetch(result, session) for result in self.batch]
        for task in asyncio.as_completed(tasks):
            await task
        self.batch.clear()

async def fetch(self, result, session):
    async with session.post(self.url, data=result) as response:
        return await response.json()

```

- ❶ We are able to start up an event loop just to run a single asynchronous function. The event loop will run until the asynchronous function is complete, and then the code will resume as normal.
- ❷ This function is nearly identical to that of [Example 8-6](#).

Now we can proceed almost in the same way as we did before. The main difference is that we add our results to our `AsyncBatcher` and let it take care of when to send the requests. Note that we chose to make this object into a context manager so that once we are done batching, the final `flush()` gets called. If we didn't do this, there would be a chance that we still have some results queued that didn't trigger a flush:

```

def calculate_task_batch(num_iter, task_difficulty):
    with AsyncBatcher(100) as batcher: ❶
        for i in range(num_iter):
            result = do_task(i, task_difficulty)
            batcher.save(result)

```

- ❶ We choose to batch at 100 requests, for reasons similar to those illustrated in [Figure 8-3](#).

With this change, we are able to bring our runtime for a difficulty of 8 down to 10.21 seconds. This represents a $6.95\times$ speedup without our having to do much work. In a constrained environment such as a real-time data pipeline, this extra speed could mean the difference between a system being able to keep up with demand and that system falling behind (in which case a queue will be required; you'll learn about these in [Chapter 10](#)).

To understand what is happening in this timing, let's consider the variables that could affect the timings of this batched method. If our database had infinite throughput (i.e., if we could send an infinite number of requests at the same time without penalty), we could take advantage of the fact that we get only the 100-millisecond penalty when our `AsyncBatcher` is full and does a flush. In this case, we'd get the best performance by just saving all of our requests to the database and doing them all at once when the calculation was finished.

However, in the real world, our databases have a maximum throughput that limits the number of concurrent requests they can process. In this case, our server is limited at 100 requests a second, which means we must flush our batcher every one hundred results and take the 100-millisecond penalty then. This is because the batcher still pauses the execution of the program, just as the serial code did; however, in that paused time it performs many requests instead of just one.

If we tried to save all our results to the end and then issued them all at once, the server would only *process* one hundred at a time, and we'd have an extra penalty in terms of the overhead to making all those requests at the same time in addition to overloading our database, which can cause all sorts of unpredictable slowdowns.

On the other hand, if our server had terrible throughput and could deal with only one request at a time, we may as well run our code in serial! Even if we kept our batching at one hundred results per batch, when we actually go to make the requests, only one would get responded to at a time, effectively invalidating any batching we made.

This mechanism of batching results, also known as *pipelining*, can help tremendously when trying to lower the burden of an I/O task (as seen in [Figure 8-8](#)). It offers a good compromise between the speeds of asynchronous I/O and the ease of writing serial programs. However, a determination of how much to pipeline at a time is very case-dependent and requires some profiling and tuning to get the best performance.

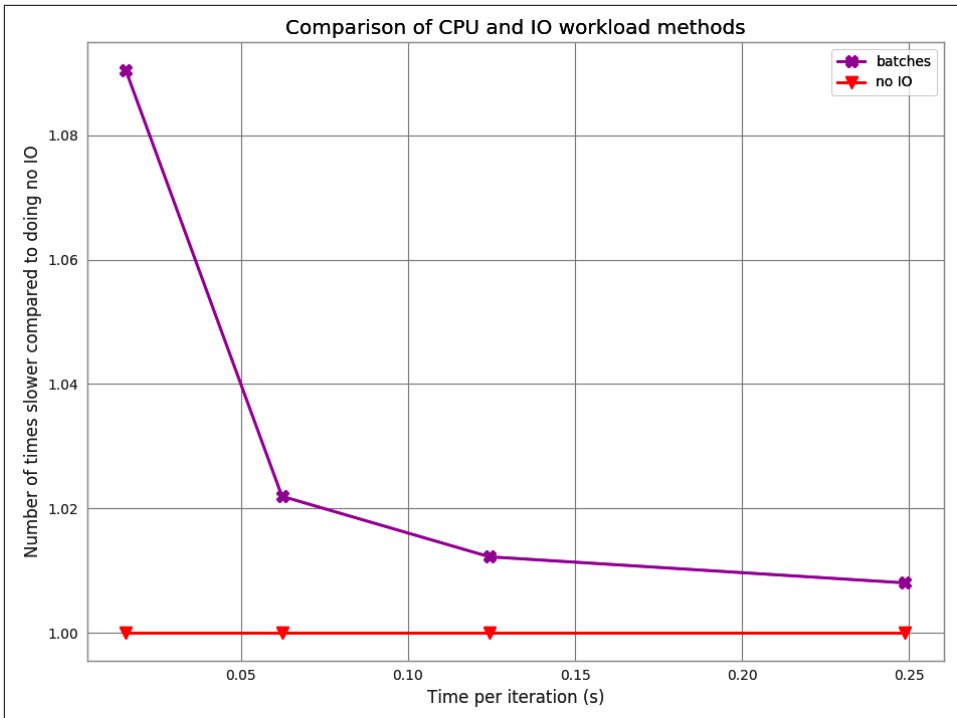


Figure 8-8. Comparison of batching requests versus not doing any I/O

Full Async

In some cases, we may need to implement a full asynchronous solution. This may happen if the CPU task is part of a larger I/O-bound program, such as an HTTP server. Imagine that you have an API service that, in response to some of its endpoints, has to perform heavy computational tasks. We still want the API to be able to handle concurrent requests and be performant in its tasks, but we also want the CPU task to run quickly.

The implementation of this solution in [Example 8-7](#) uses code very similar to that of [Example 8-6](#).

Example 8-7. Async CPU workload

```
def save_result_aiohttp(client_session):
    sem = asyncio.Semaphore(100)

    async def saver(result):
        nonlocal sem, client_session
        url = f"http://127.0.0.1:8080/add"
        async with sem:
```

```

        async with client_session.post(url, data=result) as response:
            return await response.json()

    return saver

async def calculate_task_aiohttp(num_iter, task_difficulty):
    tasks = []
    async with aiohttp.ClientSession() as client_session:
        saver = save_result_aiohttp(client_session)
        for i in range(num_iter):
            result = do_task(i, task_difficulty)
            task = asyncio.create_task(saver(result)) ❶
            tasks.append(task)
            await asyncio.sleep(0) ❷
        await asyncio.wait(tasks) ❸

```

- ❶ Instead of awaiting our database save immediately, we queue it into the event loop using `asyncio.create_task` and keep track of it so we can ensure that the task has completed before the end of the function.
- ❷ This is arguably the most important line in the function. Here, we pause the main function to allow the event loop to take care of any pending tasks. Without this, none of our queued tasks would run until the end of the function.
- ❸ Here we wait for any tasks that haven't completed yet. If we had not done the `asyncio.sleep` in the for loop, all the saves would happen here!

Before we go into the performance characteristics of this code, we should first talk about the importance of the `asyncio.sleep(0)` statement. It may seem strange to be sleeping for zero seconds, but this statement is a way to force the function to defer execution to the event loop and allow other tasks to run. In general in asynchronous code, this deferring happens every time an `await` statement is run. Since we generally don't `await` in CPU-bound code, it's important to force this deferment, or else no other task will run until the CPU-bound code is complete. In this case, if we didn't have the sleep statement, all the HTTP requests would be paused until the `asyncio.wait` statement, and then all the requests would be issued at once, which is definitely not what we want!

One nice thing about having this control is that we can choose the best times to defer back to the event loop. There are many considerations when doing this. Since the run state of the program changes when we defer, we don't want to do it in the middle of a calculation and potentially change our CPU cache. In addition, deferring to the event loop has an overhead cost, so we don't want to do it too frequently. However, while we are bound up doing the CPU task, we cannot do any I/O tasks. So if our full application is an API, no requests can be handled during the CPU time!

Our general rule of thumb is to try to issue an `asyncio.sleep(0)` at any loop that we expect to iterate every 50 to 100 milliseconds or so. Some applications use `time.perf_counter` and allow the CPU task to have a specific amount of runtime before forcing a sleep. For a situation such as this, though, since we have control of the number of CPU and I/O tasks, we just need to make sure that the time between sleeps coincides with the time needed for pending I/O tasks to complete.

One major performance benefit to the full asynchronous solution is that we can perform all of our I/O *while* we are doing our CPU work, effectively hiding it from our total runtime (as we can see from the overlapping lines in [Figure 8-9](#)). While it will never be completely hidden because of the overhead costs of the event loop, we can get very close. In fact, for a difficulty of 8 with 600 iterations, our code runs 7.3× faster than the serial code and performs its total I/O workload 2× faster than the batched code (and this benefit over the batched code would only get better as we do more iterations, since the batched code loses time versus the asynchronous code every time it has to pause the CPU task to flush a batch).

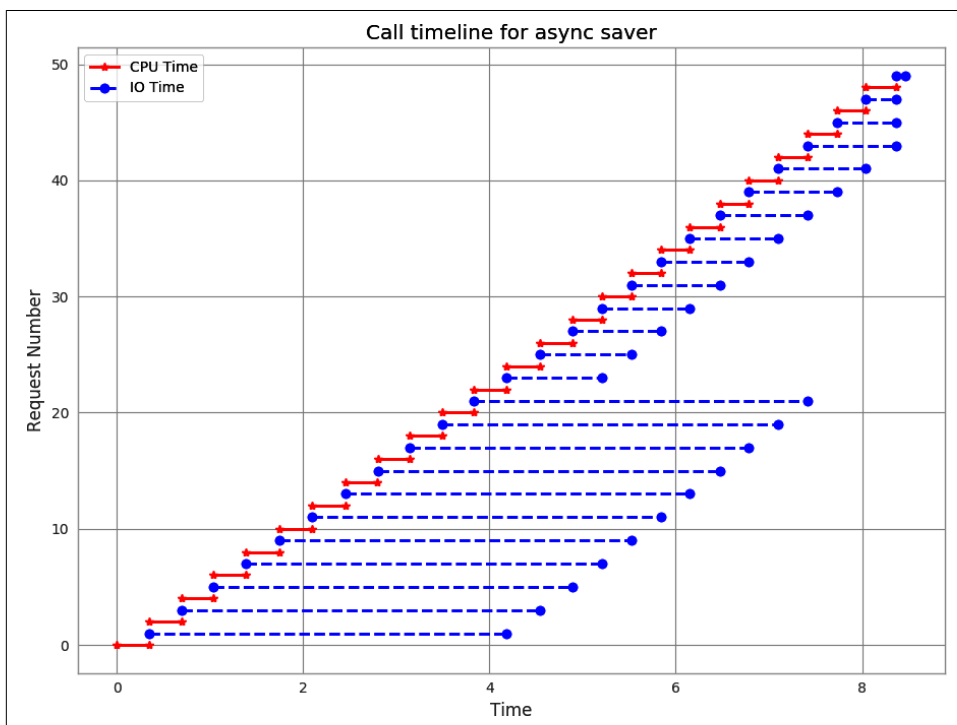


Figure 8-9. Call graph for 25 difficulty-8 CPU tasks using the `aiortt` solution—the red lines represent time working on a CPU task, while blue lines represent time sending a result to the server

In the call timeline, we can really see what is going on. What we've done is to mark the beginning and end of each CPU and I/O task for a short run of 25 CPU tasks with difficulty 8. The first several I/O tasks are the slowest, taking a while to make the initial connection to our server. Because of our use of `aiohttp`'s `ClientSession`, these connections are cached, and all subsequent connections to the same server are much faster.

After this, if we just focus on the blue lines, they seem to happen very regularly without much of a pause between CPU tasks. Indeed, we don't see the 100-millisecond delay from the HTTP request between tasks. Instead, we see the HTTP request being issued quickly at the end of each CPU task and later being marked as completed at the end of another CPU task.

We do see, though, that each individual I/O task takes longer than the 100-millisecond response time from the server. This longer wait time is given by the frequency of our `asyncio.sleep(0)` statements (since each CPU task has one `await`, while each I/O task has three) and the way the event loop decides which tasks come next. For the I/O task, this extra wait time is OK because it doesn't interrupt the CPU task at hand. In fact, at the end of the run we can see the I/O runtimes shorten until the final I/O task is run. This final blue line is triggered by the `asyncio.wait` statement and runs incredibly quickly since it is the only remaining task and never needs to switch to other tasks.

In Figures 8-10 and 8-11, we can see a summary of how these changes affect the runtime of our code for different workloads. The speedup in the async code over the serial code is significant, although we are still a ways away from the speeds achieved in the raw CPU problem. For this to be completely remedied, we would need to use modules like `multiprocessing` to have a completely separate process that can deal with the I/O burden of our program without slowing down the CPU portion of the problem.

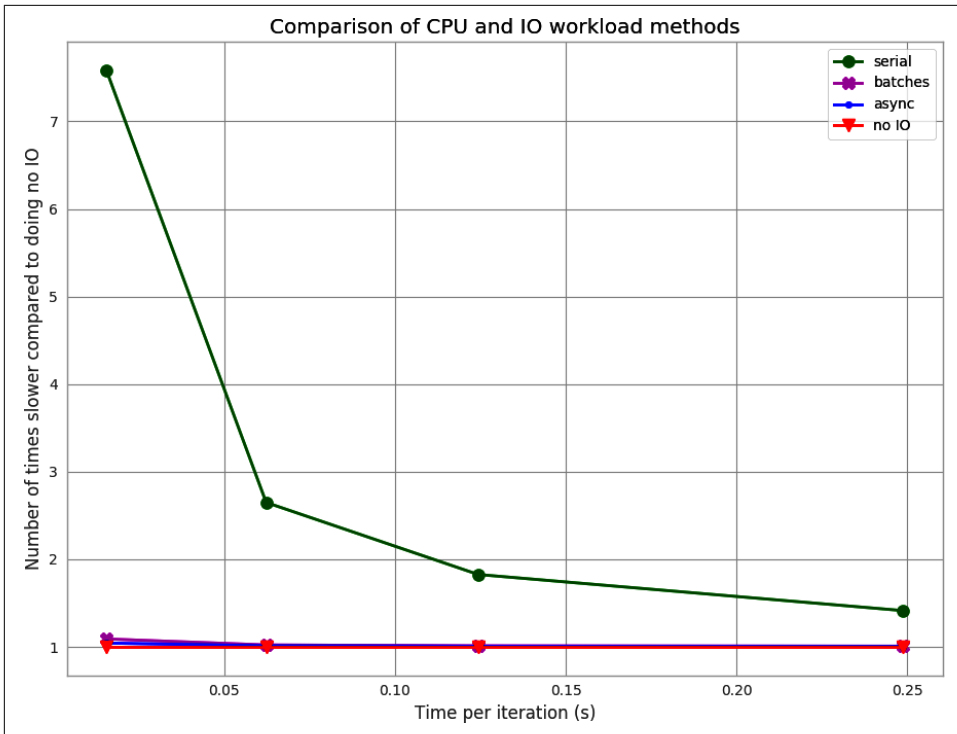


Figure 8-10. Processing time difference between serial I/O, batched async I/O, full async I/O, and a control case where I/O is completely disabled

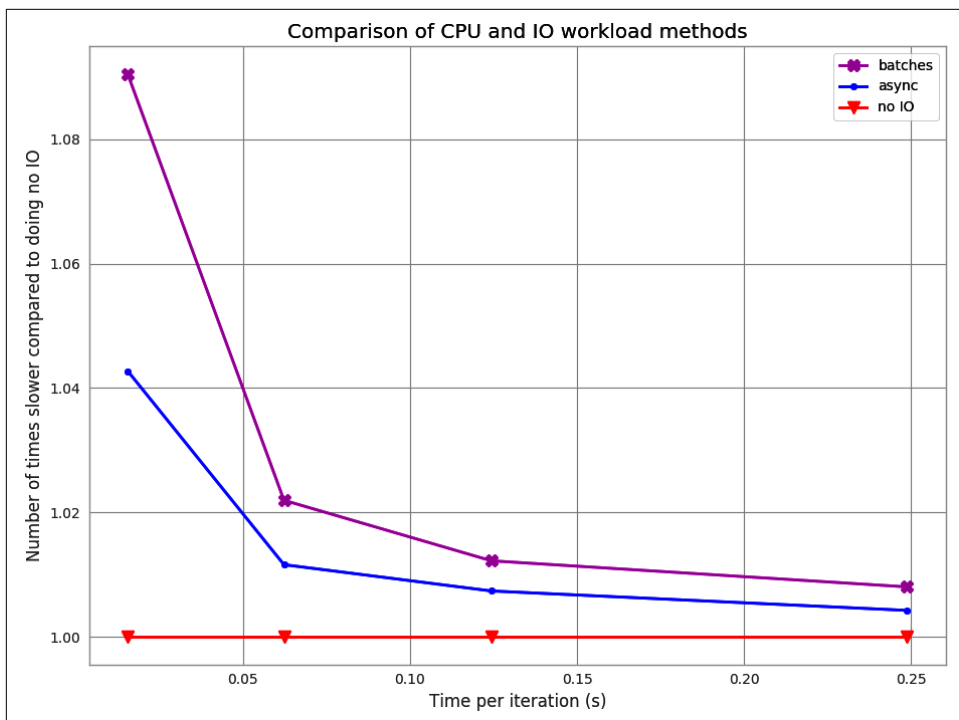


Figure 8-11. Processing time difference between batched async, full async I/O, and I/O disabled

Wrap-Up

When solving problems in real-world and production systems, it is often necessary to communicate with an outside source. This outside source could be a database running on another server, another worker computer, or a data service that is providing the raw data that must be processed. Whenever this is the case, your problem can quickly become I/O-bound, meaning that most of the runtime is dominated by dealing with input/output.

Concurrency helps with I/O-bound problems by allowing you to interleave computation with potentially multiple I/O operations. This allows you to exploit the fundamental difference between I/O and CPU operations in order to speed up overall runtime.

As we saw, `gevent` provides the highest-level interface for asynchronous I/O. On the other hand, `tornado` and `aiohttp` allow full control of an asynchronous I/O stack. In addition to the various levels of abstraction, every library uses a different paradigm for its syntax. However, `asyncio` is the binding glue for the asynchronous solutions and provides the fundamental mechanisms to control them all.

We also saw how to mesh CPU and I/O tasks together and how to consider the various performance characteristics of each to come up with a good solution to the problem. While it may be appealing to go to a full asynchronous code immediately, sometimes intermediate solutions work almost as well without having quite the engineering burden.

In the next chapter, we will take this concept of interleaving computation from I/O-bound problems and apply it to CPU-bound problems. With this new ability, we will be able to perform not only multiple I/O operations at once but also many computational operations. This capability will allow us to start to make fully scalable programs where we can achieve more speed by simply adding more computer resources that can each handle a chunk of the problem.

PNP Ch 54 | Asyncio Module [companion]

Chapter 54: Asyncio Module

Section 54.1: Coroutine and Delegation Syntax

Before Python 3.5+ was released, the `asyncio` module used generators to mimic asynchronous calls and thus had a different syntax than the current Python 3.5 release.

Python 3.x `Version ≥ 3.5`

Python 3.5 introduced the `async` and `await` keywords. Note the lack of parentheses around the `await func()` call.

```
import asyncio

async def main():
    print(await func())

async def func():
    # Do time intensive stuff...
    return "Hello, world!"

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main())
```

Python 3.x `Version ≥ 3.3 Version < 3.5`

Before Python 3.5, the `@asyncio.coroutine` decorator was used to define a coroutine. The `yield from` expression was used for generator delegation. Note the parentheses around the `yield from func()`.

```
import asyncio

@asyncio.coroutine
def main():
    print((yield from func()))

@asyncio.coroutine
def func():
    # Do time intensive stuff..
    return "Hello, world!"

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main())
```

Python 3.x `Version ≥ 3.5`

Here is an example that shows how two functions can be run asynchronously:

```
import asyncio

async def cor1():
    print("cor1 start")
    for i in range(10):
        await asyncio.sleep(1.5)
        print("cor1", i)

async def cor2():
    print("cor2 start")
    for i in range(15):
        await asyncio.sleep(1)
```

```
print("cor2", i)
```

```
loop = asyncio.get_event_loop()
cors = asyncio.wait([cor1(), cor2()])
loop.run_until_complete(cors)
```

Section 54.2: Asynchronous Executors

Note: Uses the Python 3.5+ `async/await` syntax

`asyncio` supports the use of `Executor` objects found in `concurrent.futures` for scheduling tasks asynchronously. Event loops have the function `run_in_executor()` which takes an `Executor` object, a `Callable`, and the `Callable`'s parameters.

Scheduling a task for an `Executor`

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

def func(a, b):
    # Do time intensive stuff...
    return a + b

async def main(loop):
    executor = ThreadPoolExecutor()
    result = await loop.run_in_executor(executor, func, "Hello,", " world!")
    print(result)

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main(loop))
```

Each event loop also has a "default" `Executor` slot that can be assigned to an `Executor`. To assign an `Executor` and schedule tasks from the loop you use the `set_default_executor()` method.

```
import asyncio
from concurrent.futures import ThreadPoolExecutor

def func(a, b):
    # Do time intensive stuff...
    return a + b

async def main(loop):
    # NOTE: Using `None` as the first parameter designates the `default` Executor.
    result = await loop.run_in_executor(None, func, "Hello,", " world!")
    print(result)

if __name__ == "__main__":
    loop = asyncio.get_event_loop()
    loop.set_default_executor(ThreadPoolExecutor())
    loop.run_until_complete(main(loop))
```

There are two main types of `Executor` in `concurrent.futures`, the `ThreadPoolExecutor` and the `ProcessPoolExecutor`. The `ThreadPoolExecutor` contains a pool of threads which can either be manually set to a specific number of threads through the constructor or defaults to the number of cores on the machine times 5. The `ThreadPoolExecutor` uses the pool of threads to execute tasks assigned to it and is generally better at CPU-bound operations rather than I/O bound operations. Contrast that to the `ProcessPoolExecutor` which spawns a new process for each task assigned to it. The `ProcessPoolExecutor` can only take tasks and parameters that are

pickable. The most common non-picklable tasks are the methods of objects. If you must schedule an object's method as a task in an Executor you must use a `ThreadPoolExecutor`.

Section 54.3: Using UVLoop

`uvloop` is an implementation for the `asyncio.AbstractEventLoop` based on `libuv` (Used by `nodejs`). It is compliant with 99% of `asyncio` features and is much faster than the traditional `asyncio.EventLoop`. `uvloop` is currently not available on Windows, install it with `pip install uvloop`.

```
import asyncio
import uvloop

if __name__ == "__main__":
    asyncio.set_event_loop(uvloop.new_event_loop())
    # Do your stuff here ...
```

One can also change the event loop factory by setting the `EventLoopPolicy` to the one in `uvloop`.

```
import asyncio
import uvloop

if __name__ == "__main__":
    asyncio.set_event_loop_policy(uvloop.EventLoopPolicy())
    loop = asyncio.new_event_loop()
```

Section 54.4: Synchronization Primitive: Event

Concept

Use an Event to **synchronize the scheduling of multiple coroutines**.

Put simply, an event is like the gun shot at a running race: it lets the runners off the starting blocks.

Example

```
import asyncio

# event trigger function
def trigger(event):
    print('EVENT SET')
    event.set() # wake up coroutines waiting

# event consumers
async def consumer_a(event):
    consumer_name = 'Consumer A'
    print('{} waiting'.format(consumer_name))
    await event.wait()
    print('{} triggered'.format(consumer_name))

async def consumer_b(event):
    consumer_name = 'Consumer B'
    print('{} waiting'.format(consumer_name))
    await event.wait()
    print('{} triggered'.format(consumer_name))

# event
event = asyncio.Event()

# wrap coroutines in one future
```

```

main_future = asyncio.wait([consumer_a(event),
                             consumer_b(event)])

# event loop
event_loop = asyncio.get_event_loop()
event_loop.call_later(0.1, functools.partial(trigger, event)) # trigger event in 0.1 sec

# complete main_future
done, pending = event_loop.run_until_complete(main_future)

```

Output:

```

Consumer B waiting
Consumer A waiting
EVENT SET
Consumer B triggered
Consumer A triggered

```

Section 54.5: A Simple Websocket

Here we make a simple echo websocket using `asyncio`. We define coroutines for connecting to a server and sending/receiving messages. The communications of the websocket are run in a `main` coroutine, which is run by an event loop. This example is modified from a [prior post](#).

```

import asyncio
import aiohttp

session = aiohttp.ClientSession() # handles the context manager
class EchoWebsocket:

    async def connect(self):
        self.websocket = await session.ws_connect("wss://echo.websocket.org")

    async def send(self, message):
        self.websocket.send_str(message)

    async def receive(self):
        result = (await self.websocket.receive())
        return result.data

async def main():
    echo = EchoWebsocket()
    await echo.connect()
    await echo.send("Hello World!")
    print(await echo.receive()) # "Hello World!"

if __name__ == '__main__':
    # The main loop
    loop = asyncio.get_event_loop()
    loop.run_until_complete(main())

```

Section 54.6: Common Misconception about asyncio

probably *the* most common misconception about `asyncio` is that it lets you run any task in parallel - sidestepping the GIL (global interpreter lock) and therefore execute blocking jobs in parallel (on separate threads). it does **not**!

asyncio (and libraries that are built to collaborate with asyncio) build on coroutines: functions that (collaboratively) yield the control flow back to the calling function. note `asyncio.sleep` in the examples above. this is an example of a non-blocking coroutine that waits 'in the background' and gives the control flow back to the calling function (when called with `await`). `time.sleep` is an example of a blocking function. the execution flow of the program will just stop there and only return after `time.sleep` has finished.

a real-live example is the [requests](#) library which consists (for the time being) on blocking functions only. there is no concurrency if you call any of its functions within asyncio. [aiohttp](#) on the other hand was built with asyncio in mind. its coroutines will run concurrently.

- if you have long-running CPU-bound tasks you would like to run in parallel asyncio is **not** for you. for that you need [threads](#) or [multiprocessing](#).
- if you have IO-bound jobs running, you *may* run them concurrently using asyncio.

VIDEO: Machine Learning A-Z: Hands-On Python In Data Science

Learn to create Machine Learning Algorithms in Python from two Data Science experts. Code templates included.



- ✓ Master Machine Learning on Python
- ✓ Have a great intuition of many Machine Learning models
- ✓ Make accurate predictions
- ✓ Make powerful analysis
- ✓ Make robust Machine Learning models
- ✓ Create strong added value to your business
- ✓ Use Machine Learning for personal purpose
- ✓ Handle specific topics like Reinforcement Learning, NLP and Deep Learning
- ✓ Handle advanced techniques like Dimensionality Reduction
- ✓ Know which Machine Learning model to choose for each type of problem
- ✓ Build an army of powerful Machine Learning models and know how to combine them to solve any problem

Watch Today →

PNP Ch 59 | Queue Module [companion]

Chapter 59: Queue Module

The Queue module implements multi-producer, multi-consumer queues. It is especially useful in threaded programming when information must be exchanged safely between multiple threads. There are three types of queues provided by the queue module, which are as follows: 1. Queue 2. LifoQueue 3. PriorityQueue. Exceptions which could occur are: 1. Full (queue overflow) 2. Empty (queue underflow)

Section 59.1: Simple example

```
from Queue import Queue

question_queue = Queue()

for x in range(1,10):
    temp_dict = ('key', x)
    question_queue.put(temp_dict)

while(not question_queue.empty()):
    item = question_queue.get()
    print(str(item))
```

Output:

```
('key', 1)
('key', 2)
('key', 3)
('key', 4)
('key', 5)
('key', 6)
('key', 7)
('key', 8)
('key', 9)
```


VIDEO: Machine Learning, Data Science and Deep Learning with Python

Complete hands-on machine learning tutorial with data science, Tensorflow, artificial intelligence, and neural networks



- ✓ Build artificial neural networks with Tensorflow and Keras
- ✓ Classify images, data, and sentiments using deep learning
- ✓ Make predictions using linear regression, polynomial regression, and multivariate regression
- ✓ Data Visualization with Matplotlib and Seaborn
- ✓ Implement machine learning at massive scale with Apache Spark's MLlib
- ✓ Understand reinforcement learning - and how to build a Pac-Man bot
- ✓ Classify data using K-Means clustering, Support Vector Machines (SVM), KNN, Decision Trees, Naive Bayes, and PCA
- ✓ Use train/test and K-Fold cross validation to choose and tune your models
- ✓ Build a movie recommender system using item-based and user-based collaborative filtering

Watch Today →